

UXE User Guide

Product Version 12.1
February 2013

© 2010-2013 Cadence Design Systems, Inc. All rights reserved.
Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Product UXE contains technology licensed from, and copyrighted by: Lucent Technologies, Inc, and is © 1991-1998, Lucent Technologies, Inc, All rights reserved; Carnegie Mellon University, and is © 1988, 1991 by Carnegie Mellon University, All rights reserved; loi Kim Lam, and is © 1993-1996 by loi Kim Lam, All rights reserved; Qlogic Fibre Channel Driver, © 1989, 1991 Free Software Foundation, All rights reserved; Emulex Fibre Channel Driver, © 1989, 1991 Free Software Foundation, Inc, All rights reserved; Prentice Hall, © 1995 by Prentice Hall, All rights reserved; © Regents of the University of California © Sun Microsystems, Inc. © Scriptics Corporation, Novus Copyright (C) 1996 - 2007 by Novas Software, Inc., Source III (C) Copyright 2006 Source III, Inc., U.C Riverside COPYRIGHT (C) 1995, 1996 by Nexsyn Design Technology Inc. All rights reserved.

Lucent Technologies disclaims all warranties with regard to this software, including all implied warranties of merchantability and fitness.

Associated third party license terms may be found at <ROOTDIR>/install/Copyrights

Open SystemC, Open SystemC Initiative, OSCI, SystemC, and SystemC Initiative are trademarks or registered trademarks of Open SystemC Initiative, Inc. in the United States and other countries and are used with permission.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522. All other trademarks are the property of their respective holders.

Restricted Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, this statement grants Cadence customers permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used only in accordance with a written agreement between Cadence and its customer.
2. The publication may not be modified in any way.
3. Any authorized copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
4. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor

Contents

1

<u>Overview</u>	31
<u>Compiling the Designs for ICE Mode</u>	33
<u>Compiling the Designs for SA Mode</u>	35
<u>Compiling the Designs for SA Mode Using <i>irun</i></u>	36
<u>Debugging the Designs</u>	36
<u>Debug Vision Modes</u>	37
<u>Debug Sessions</u>	38
<u>Debug Operations</u>	38
<u>Waveform Database</u>	42
<u>Setting Up the Environment for Palladium XP</u>	43
<u>Environment Variables for the Palladium XP System (Mandatory)</u>	43
<u>Environment Variables for IXCOM (Mandatory)</u>	44
<u>Environment Variables for <i>irun</i> (IES)</u>	45
<u>Environment Variables Required in Specific Cases</u>	46
<u>Viewing Global Error Log</u>	50
<u>Using the readerr Utility</u>	50

2

<u>Working with the Palladium XP Emulators</u>	53
<u>Configuring the Emulator or Emulation Host</u>	53
<u>Working with Emulators at Compile Time</u>	54
<u>Identifying Design Host and Domains</u>	54
<u>Compiling for Multiple Configurations (Symmetric Configuration)</u>	56
<u>Relocating Designs into Different Domains</u>	58
<u>Specifying the Connection to Target System</u>	61
<u>Duplicating Wires for Transmitting a Signal between Chips or Boards</u>	78
<u>Finding Faults Using ModelChecker</u>	80
<u>Working with Emulators at Run Time</u>	85

<u>Specifying the Emulation Host</u>	85
<u>Identifying Best Location for Downloading Design</u>	86
<u>Reserving Emulation Hosts</u>	87
<u>Reducing Emulator Initialization Time Using KeepHostAlive</u>	92
<u>Handling Failures at Run Time with Wire Duplication</u>	95
<u>Using Suspend and Resume to Manage Emulator Resources</u>	96

3

Compiling Designs for In-Circuit Emulation with xeCompile 99

<u>Preparing for Design Import</u>	101
<u>Creating the Directory Structure</u>	101
<u>Setting the Absolute Path for Libraries</u>	102
<u>Specifying RTL Design Files for Import</u>	104
<u>Specifying Verilog Designs for Import</u>	105
<u>Specifying VHDL Designs for Import</u>	107
<u>Specifying Mixed-Language Libraries for Import</u>	109
<u>Synthesizing the RTL Design for Emulation</u>	110
<u>Synthesizing Verilog or VHDL Designs</u>	111
<u>Synthesizing Mixed-Language Designs</u>	112
<u>Incremental Synthesis</u>	114
<u>Preserving the RTL Names</u>	114
<u>Maintaining Log Files</u>	116
<u>Using Synthesis Directives for Emulation</u>	116
<u>Synthesizing Designs with Memories</u>	119
<u>Using Memory Synthesis</u>	120
<u>Modeling Verilog Memories</u>	121
<u>Modeling VHDL Memories</u>	123
<u>Synthesizing Designs with Acceleratable Testbench (ATB) Constructs</u>	127
<u>Use Model for ATB</u>	127
<u>Using hdlConfig Command at Run Time</u>	128
<u>Supported Testbench Constructs</u>	129
<u>Synthesizing Initial Blocks with Synthesizable Constructs</u>	137
<u>Initializing Registers and Memories</u>	140
<u>Executing Distributed Synthesis during RTL Synthesis</u>	141
<u>Distributing the Synthesis Process</u>	142

UXE User Guide

<u>Scheduling the Tasks</u>	143
<u>Limitations of Distributed Synthesis</u>	144
<u>Generating a Debussy Database during Synthesis</u>	144
<u>Using Conformal to Verify HDL Compiler-Generated Netlist</u>	145
<u>Generating Script for Conformal</u>	146
<u>Starting Conformal</u>	147
<u>Analyzing Results Using Conformal</u>	147
<u>Using UNIX Commands for RTL Import and Synthesis</u>	149
<u>Library Mapping File</u>	150
<u>vavhdl</u>	152
<u>vavlog</u>	154
<u>vaelab</u>	158
<u>Importing the Design</u>	163
<u>Importing a Netlist</u>	164
<u>Compiling the Design</u>	168
<u>Opening the Design Library</u>	169
<u>Selecting the Compilation Mode</u>	169
<u>Determining the Vision Mode</u>	169
<u>Selecting a Vision Mode Based on Preferred Debug Strategy</u>	170
<u>Connecting Nets Using iScopedNets Program</u>	172
<u>Creating Nets File for iScopedNets</u>	173
<u>Providing Vector (Bus) Support</u>	174
<u>Breaking the Loops</u>	175
<u>Synthesizing Flip-Flops and Latches</u>	177
<u>Inserting the Instrumentation Logic</u>	177
<u>Running the Precompile Process</u>	178
<u>Preparing the Design Files for Compilation</u>	179
<u>Setting Up the Design for Precompile</u>	179
<u>Incorporating Physical IP</u>	180
<u>Running ICE Prepare</u>	181
<u>Support for Encryption in HDL Compiler</u>	182
<u>Debug Capability of Encrypted Source Code</u>	182
<u>Compiling the Design Files</u>	184
<u>Viewing Compile Results</u>	184
<u>Using Compiler Options for Better Compile-Time Performance</u>	185
<u>Using Multithreading Feature for Compilation</u>	193

<u>Synthesizing and Compiling ALUs</u>	195
<u>Controlling ALU Utilization during Synthesis</u>	196
<u>Controlling ALU-to-Gates Transformations at Compile Time</u>	197

4

Compiling Designs for Simulation Acceleration Using IXCOM

199

<u>Overview of the IXCOM Compilation Process</u>	201
<u>Compiling for Hardware Using IXCOM</u>	202
<u>Directing Design Compilation for Hardware</u>	203
<u>Modifying the IXCOM Compilation Flow</u>	203
<u>Analyzing Mixed-Language Designs</u>	205
<u>Instantiating Verilog in VHDL</u>	205
<u>Instantiating VHDL in Verilog</u>	205
<u>Associating Logical Libraries to Physical Directories Using libmaprc File</u>	210
<u>libmaprc File Syntax</u>	210
<u>Compiling Mixed-Language Designs</u>	214
<u>Partitioning Designs into RTL (Hardware) and Behavioral (Software)</u>	216
<u>Compilation Log Files</u>	217
<u>Compiling SystemC Designs</u>	219
<u>Supported Types</u>	219
<u>Developing and Simulating SystemC Designs</u>	220
<u>Instantiating SystemC under Verilog/VHDL</u>	222
<u>Instantiating Verilog/VHDL under SystemC</u>	223
<u>Support for Multiple SystemC Shared Libraries</u>	224
<u>Using Specman Elite with IXCOM</u>	225
<u>Compiling and Running a Verilog Design in the Specman Environment</u>	225
<u>Compiling and Running a Mixed-Language Design in the Specman Environment</u> ..	226
<u>Compiling Testbenches Separately</u>	229
<u>Linking DUT with Multiple Testbenches</u>	229
<u>Compiling Top-Level Testbenches Using +xcDesignTop</u>	230
<u>Compiling Embedded Testbenches Using +xcEmbeddedTb</u>	232
<u>Determining File Type by Source Filename Extension</u>	234
<u>Options for Verilog Design Compilation Mode</u>	234
<u>Options for File-Extension-Based Compilation Mode</u>	235

UXE User Guide

<u>Compiling Behavioral Constructs</u>	238
<u>Handling of Behavioral Constructs during Behavioral Compilation</u>	239
<u>Using Compiler Directives to Guide Behavioral Compilation</u>	248
<u>Process-Level Locking for Dependent and Conflicting Behavioral Processes</u>	252
<u>Compiling Behavioral Interface Constructs</u>	254
<u>Enabling Interface Compilation in IXCOT</u>	254
<u>Fine-Tuning Behavioral Performance</u>	263
<u>Reporting Behavioral Evaluation</u>	263
<u>Identifying the Source of Behavioral Evaluations</u>	264
<u>System Task Mapping in IXCOT flow</u>	265
<u>Mapping of \$display-Family Tasks</u>	266
<u>Using System Tasks and Procedures at Run time</u>	267
<u>Supported Verilog System Tasks and VHDL Procedures</u>	267
<u>Executing Non-Synthesizable Code in the Emulator</u>	268
<u>Partitioning Monitor Code</u>	268
<u>Activating Monitor Code Only in Software Simulation</u>	270
<u>Monitoring Signals and Displaying Messages</u>	272
<u>Compiling DUTs with Non-Synthesizable Code by Using Testbench Callback Technology</u>	273
<u>Writing to DUT Signals Using Testbench Callback Tasks</u>	282
<u>Controlling Palladium XP from within Testbench Callback Tasks</u>	282
<u>Compiling Designs in Stages using IXCOT Compiler</u>	284
<u>Generating Pulses</u>	287
<u>Pragma Handling in IXCOT</u>	289
<u>Using translate off and translate on Pragma</u>	290
<u>Ignoring Pragmas in VHDL Designs</u>	291
<u>Using //ixc Pragmas</u>	291
<u>Enabling irun to Handle Sparse Memories</u>	291
<u>Reading and Writing Signals in DUT Modules</u>	293
<u>Using Plus Arguments</u>	301
<u>Support for Encryption in SA Mode</u>	302
<u>Reading Encrypted Files</u>	303
<u>Encryption with Three Levels of Debug Capability</u>	303
<u>Application of Protection Rights by the IXCOT Compiler</u>	304
<u>Support for Strengths in SA Mode</u>	306
<u>Handling of UDPs by IXCOT</u>	306

UXE User Guide

<u>Support for Verilog Configuration Blocks</u>	307
<u>Examples</u>	308
<u>Compiling Designs with Verilog Configuration Blocks</u>	308
<u>Support for specify Blocks in SA Mode</u>	308
<u>Deviations from Verilog 2005 LRM for specify Blocks</u>	310
<u>Direct Access of DUT Memory Array and Register</u>	312
<u>Using the marg Library</u>	312
<u>Using marg Interface in Pure Simulation</u>	312
<u>C and Verilog Interface Functions</u>	313
<u>Tasks for Memory Array Buffer</u>	313
<u>Tasks for DUT Register Access</u>	318
<u>Examples: Passing Memory Data from Testbench to DUT</u>	320
<u>SystemVerilog DPI Functions</u>	321
<u>Specman Functions</u>	324
<u>Enabling and Disabling Features at Compilation Time</u>	326
<u>Syntax of the <feature file> File</u>	327
<u>Instance-Based Configuration</u>	330
<u>Design-Unit-Based Configuration</u>	331
<u>File-Based Configuration</u>	332
<u>Precedence Rules</u>	333
<u>Examples of <feature spec lines></u>	334
<u>Using Conformal to Verify the IXCOM-Compiler-Generated Output</u>	336
<u>Files Generated by the LEC Tool</u>	338
<u>Customizing the LEC Tool Invocation</u>	339
<u>Importing External Netlists into IXCOM Compiler</u>	341
<u>Importing Netlist Modules and Exporting RTL Modules</u>	343
<u>Using Hot Swap with Imported Netlists</u>	353
<u>Accessing Netlist Objects from RTL Hierarchy</u>	354
<u>Generating Clocks and Binding Clock signals in the Netlist Hierarchy</u>	355
<u>Import Options for Design Units Defined in Both Netlist and RTL Domains</u>	357

5

<u>Compiling and Running Simulation Acceleration Designs with <i>irun</i></u>	359
<u>Overview</u>	359

UXE User Guide

<u>Setting Up the Environment to Use <i>irun</i> for Simulation Acceleration</u>	360
<u>Compiling and Simulating Designs</u>	360
<u>SA-Specific Options of the <i>irun</i> Command</u>	361
<u>Compilation Options to Enable SA Mode</u>	362
<u>Run-Time Options for Using <i>xeDebug</i></u>	363
<u>Accessing Help Information for IXCOM Tools from <i>irun</i></u>	364
<u>Compiling Designs</u>	364
<u>Verilog-Only Designs</u>	365
<u>Mixed-Language (Verilog and VHDL) Designs</u>	366
<u>Designs with PLI/VPI and VHPi</u>	367
<u>Designs with SystemVerilog DPI</u>	367
<u>Designs with SystemC</u>	368
<u>Designs Using Specman <i>e</i> Testbenches</u>	369
<u>Parsing IXCOM and <i>irun</i> Command Options with the <i>irun</i> Tool</u>	370
<u>IES-Compatible Support for Testbench Compilation</u>	371
<u>User-Partitioned Testbench and DUT Compilation</u>	372
<u>Designs Using TBA and UVCs</u>	373
<u>Logical Library Support</u>	373
<u>Simulating Designs</u>	376
<u>Simulating Using <i>ncsim</i></u>	376
<u>Simulating Using <i>xeDebug</i></u>	377
<u>Compilation and Run-Time Logs</u>	377
<u>Limitations</u>	378

6

Compiling Designs with SystemVerilog Constructs in SA Mode

<u>SystemVerilog Design Constructs</u>	379
<u>Compiling a Pure SystemVerilog Design</u>	379
<u>Compiling a Mixed-Language Design</u>	380
<u>SystemVerilog Testbench Constructs (SVTB)</u>	380
<u>Compiling SVTB Constructs Supported by IXCOM</u>	381
<u>Support for Covergroups and Mailbox Constructs</u>	381
<u>Support for SystemVerilog Interface Constructs</u>	382
<u>Support for SystemVerilog Packages in SA Mode</u>	383

<u>List of Supported Constructs</u>	385
<u>Support for SVA and PSL Assertions with SystemVerilog Packages</u>	386
<u>Support for Coverage Constructs with SystemVerilog Packages</u>	387
<u>Support for DPI Declarations with SystemVerilog Packages</u>	387
<u>Support for Packages Referenced Across Multiple Logical Libraries</u>	387
<u>Partitioning of SystemVerilog Packages by the IXCOT Compiler</u>	390
<u>Support for Compilation Unit Scopes in SA Mode</u>	391

7

<u>Coverage Support</u>	395
<u>INCISIV Versions for Coverage Features</u>	396
<u>Code Coverage</u>	397
<u>Toggle Coverage Support with Palladium XP</u>	398
<u>Block Coverage Support with Palladium XP</u>	401
<u>Differences in Block Coverage Support on Palladium XP with Respect to IES</u>	402
<u>Functional Coverage Support with Palladium XP</u>	403
<u>Covergroup Support in IXCOT</u>	404
<u>Support for SystemVerilog Functional Coverage Constructs</u>	416
<u>Additional Limitations of Coverage Support</u>	428
<u>Reporting for Functional Coverage Constructs</u>	429
<u>Error Handling for Functional Coverage Constructs</u>	429
<u>General Flow for Using Coverage</u>	431
<u>Generating a Coverage Database at Run Time</u>	441
<u>Using Incisive Metrics Center for Analyzing Coverage Data</u>	445
<u>Performance Considerations for Coverage Support</u>	449
<u>Performance Considerations for Code Coverage</u>	449
<u>Managing Coverage Data</u>	451
<u>Considerations Related to Coverage Verification on Palladium XP Emulator</u>	451
<u>Deprecated Options</u>	453
<u>Deprecated Compile-Time Options</u>	453
<u>Deprecated Run-Time Options</u>	455
<u>Deprecated Use Models</u>	456
<u>Enabling the Block and Toggle Coverage Features for Instance Hierarchies</u>	456

8

<u>Compiling Designs with SystemVerilog DPI</u>	463
<u>Overview of SystemVerilog DPI</u>	463
<u>SystemVerilog Layer</u>	464
<u>Foreign Language (C/C++) Layer</u>	464
<u>Supported/Unsupported C/C++ Functions</u>	465
<u>import Functions and Tasks</u>	468
<u>Definition of Import Declaration</u>	468
<u>Example: SystemVerilog Layer Calling C/C++ Function</u>	470
<u>Accessing Two-Value Bit Values in C/C++</u>	473
<u>Accessing Four-Value Logic Values in C/C++</u>	473
<u>export Functions and Tasks</u>	474
<u>Example: C/C++ Layer Calling SystemVerilog Function</u>	475
<u>Using SystemVerilog DPI</u>	476
<u>SystemVerilog DPI Keywords in Verilog</u>	477
<u>Using SystemVerilog DPI with Emulator</u>	477
<u>Improving Performance in DPI Environment Using SDPI Mode</u>	478
<u>Support for Open Arrays in DPI</u>	479
<u>Example: Using Open Arrays in DPI</u>	479
<u>Querying and Accessing Open Arrays from the C Code</u>	480
<u>Limitations in Using Open Arrays in DPI</u>	482
<u>Supported Set of DPI Functions</u>	483

9

<u>PLI/VPI and VHPI Support</u>	487
<u>PLI/VPI Support in IXCOR</u>	487
<u>Creating a PLI Table File</u>	488
<u>Using the ncpli2tab Utility</u>	490
<u>Restrictions on Using PLI/VPI Applications</u>	493
<u>Using a PLI/VPI Application in a Mixed-HDL Design</u>	494
<u>Using a Verilog PLI Application in a Testbench</u>	494
<u>Using a PLI/VPI Application in a DUT Module</u>	495
<u>Statically Linking PLI/VPI Applications</u>	495
<u>Naming PLI/VPI Applications</u>	496

UXE User Guide

<u>Example: Using PLI and VPI Applications</u>	496
<u>VHPI Support in IXCOM</u>	500
<u>Restrictions on Using VHPI Applications</u>	500

10

Compiling and Running Designs with Assertions..... 501

<u>Enabling the Use of SVA</u>	502
<u>Enabling the Use of PSL Assertions</u>	503
<u>Disabling PSL Assertions in Selected Modules</u>	503
<u>Labeling PSL Assertions</u>	504
<u>Command Options for Compiling Designs with Assertions</u>	505
<u>Enabling and Disabling Assertions at Compile Time</u>	506
<u>Functional Coverage Support</u>	507
<u>Concurrent Cover Statement</u>	507
<u>Integration with Incisive Comprehensive Coverage Tool</u>	508
<u>SystemVerilog Bind Support</u>	508
<u>Using Assertion Control Tasks (ACTs) in HDL</u>	509
<u>Specifying Varying Widths for Assertion Counters</u>	509
<u>Running and Debugging Designs with Assertions</u>	510
<u>Debugging Assertion Failures</u>	513
<u>IES Simulator Required for Exercising Assertions</u>	514
<u>Support for Assertions on Emulator</u>	515
<u>Using Assertions with InfiniTrace</u>	516
<u>SDL Support for Assertions</u>	516
<u>Static Assertions</u>	517
<u>End-of-Sim Assertion Checks</u>	517
<u>Coverage Support</u>	523
<u>Assertion Reporting</u>	523
<u>Assertion Summary Report</u>	524
<u>Debugging Assertions Using Waveforms</u>	524
<u>Assertion States</u>	526
<u>Assertions-Related Compile-Time Command Options</u>	526
<u>Assertion Run-Time Commands</u>	527
<u>Assertion Tcl Variables</u>	528
<u>Examples of SVA Usage</u>	529

UXE User Guide

<u>Example 1: Using Concurrent Assert Statements</u>	529
<u>Example 2: Using Concurrent Cover Statements</u>	530
<u>Example 3: Using SV Bind</u>	531
<u>Example 4: Functional Coverage Information</u>	533
<u>Examples of PSL Assertion Usage</u>	538
<u>Example 1: Using Embedded PSL Assertions (VHDL Only)</u>	538
<u>Example 2: Using PSL Assertions in vunits (VHDL Only)</u>	542
<u>Example 3: Using Embedded PSL Assertions (Verilog Only)</u>	546
<u>Example 4: Using PSL Assertions in vunits (Verilog Only)</u>	550
<u>Debugging Designs with PSL Assertions</u>	554
<u>Using Assertions in Dynamic Target Mode</u>	561
<u>Running Assertions in Dynamic Target Mode</u>	561
<u>Support for Assertions Commands when Using Dynamic Targets</u>	562
<u>Support for Specific SVA Constructs</u>	565
<u>Support for PSL Constructs</u>	576
<u>Default Clock Implementation</u>	576
<u>Extended Support for Built-in Methods (VHDL Only)</u>	578
<u>Differences between IES and IXCOM for PSL Assertions</u>	578
<u>Support for Specific PSL Constructs in a Mixed-HDL Design</u>	579
<u>Support for Specific PSL Constructs in a Verilog Design</u>	595
<u>Support for Incisive Assertion Library Components</u>	606
<u>Supported Basic Types</u>	606
<u>Supported Interface Types</u>	607
<u>Supported Control Types</u>	607
<u>Supported Datapath Types</u>	607
<u>Unsupported Incisive Assertion Library Components</u>	608
<u>Support for Open Verification Library Components</u>	609
<u>Supported OVL Components</u>	609

11

<u>Using the Dynamic Target Mode</u>	611
<u>Overview of Dynamic Target Mode</u>	611
<u>Running Designs in Dynamic Target Mode</u>	612
<u>Stopping a Design</u>	612
<u>Switching between the Different Modes</u>	613

<u>Assertions Support in Dynamic Target Mode</u>	614
<u>Limitations of Dynamic Target Mode</u>	614

12

<u>Implementing SCE-MI Pipes</u>	617
<u>Overview of Transaction Pipes</u>	618
<u>New Features in SCE-MI 2.1 Pipes Interface</u>	618
<u>Deferred and Immediate Visibility Modes</u>	618
<u>Notification Threshold</u>	619
<u>Callback Threshold</u>	619
<u>New and Enhanced Functions of SCE-MI 2.1</u>	620
<u>Software Components of SCE-MI Pipes Interface</u>	621
<u>Encrypted HDL and Pre-Compiled Shared Libraries</u>	621
<u>Testbench-Side Thread-Neutral C Interfaces</u>	623
<u>DUT-Side HDL Interface</u>	625
<u>Executing the Simulator Thread</u>	626
<u>Reset of SCE-MI Pipes</u>	627
<u>SCE-MI Protocol-Reset Functionality</u>	628
<u>Using Language-Based Adaptors</u>	629
<u>SCE-MI Pipes Blocking C Interfaces</u>	630
<u>SCE-MI Pipes C++ Interface on Simulator Thread</u>	630
<u>SCE-MI Pipes SystemC Interface on SystemC Thread</u>	631
<u>SCE-MI Pipes SVTB Interface on Simulator Thread</u>	632
<u>SCE-MI Pipes Specman Interface on Specman Thread</u>	633
<u>Generating DUT Clocks with SCE-MI Pipes</u>	635
<u>Compiling and Running Designs with SCE-MI Pipes</u>	635
<u>Compiling and Running Designs in 32-bit and 64-bit Modes</u>	636
<u>Sample Makefiles</u>	636
<u>Compiling Designs</u>	637
<u>Examples: Implementing SCE-MI Pipes</u>	643
<u>SCE-MI 2.0 Examples</u>	643
<u>SCE-MI 2.1 Examples</u>	648

13

<u>IXCOM Hardware Model Interface</u>	651
<u>Linux Process Level</u>	651
<u>User Application Level</u>	652
<u>Selecting HMPI Tasks and Functions for IXCOM Compilation</u>	652
<u>Building the HMPI Adaptor for CVP</u>	653
<u>OSCI SystemC Adaptor</u>	653
<u>Linux pthread Adaptor</u>	653
<u>Third-Party SystemVerilog Adaptor</u>	654
<u>Synchronization between CVP and HMS</u>	656
<u>Strict Synchronization Execution Model</u>	656

14

<u>Design Profiling for Palladium XP Acceleration</u>	669
<u>Design Profiling on Software Using IES Profiler</u>	669
<u>Commands for Design Profiling Using the IES Profiler</u>	670
<u>Analyzing and Interpreting the IES Profiler Output in ncprof.out</u>	675
<u>Improving Profiling Accuracy and Caveats</u>	682
<u>Profiling Design Run-Time Performance on Hardware with XC Profiler</u>	684
<u>Enabling the XC Profiler</u>	684
<u>Generating Hardware-Run Profiling Information Using +xcprof Option</u>	684
<u>Generating Hardware-Run Profiling Information Using xc profon Command</u>	693

15

<u>Analyzing Power Consumption of the Designs</u>	697
<u>Considerations when Using DPA</u>	698
<u>Identifying Activity-Based Power Analysis Components</u>	698
<u>Using DPA Methods in Design Flow</u>	700
<u>Performing DPA in Offline Mode</u>	701
<u>Analyzing Power Consumption of RTL Designs</u>	702
<u>RTL Toggle Count Analysis</u>	702
<u>RTL Power Analysis</u>	706
<u>Analyzing Power Consumption of Gate-Level Netlists</u>	719
<u>Normal Toggle Count (NTC) Analysis</u>	720

UXE User Guide

<u>Weighted Toggle Count (WTC) Analysis</u>	720
<u>Gate-Level Power Analysis</u>	729
<u>Analyzing Power by Invoking RTL Compiler from xeDebug</u>	733
<u>Generating Backward SAIF Files for Power Analysis by Third-Party Tools</u>	739
<u>Loading the Library Forward SAIF File into Palladium XP</u>	740
<u>Specifying the Signals That Need to be Inserted in the Backward SAIF File</u>	740
<u>Defining the Scope of Design Hierarchy Search for Instances</u>	741
<u>Specifying the Time Windows for Backward SAIF File Generation</u>	741
<u>Generating the Backward SAIF File</u>	742
<u>Parallel Computing for DPA Using Multiple CPUs</u>	742
<u>Running DPA with Multiple Hosts</u>	743
<u>Requirement on Remote Hosts</u>	743
<u>Specifying Multiple Hosts Using Commands</u>	743
<u>Specifying Multiple Hosts with rsh</u>	744
<u>Specifying Multiple Hosts with LSF</u>	744
<u>Analyzing Power Consumption Using xeDebug GUI</u>	745
<u>Setup Tab</u>	746
<u>TCA Preparation Tab</u>	748
<u>TCA Tab</u>	749
<u>DPA Probe Tab</u>	760
<u>TCF Tab</u>	768
<u>SAIF Tab</u>	770
<u>PowerProfile Tab</u>	778
<u>Analyzing Power Consumption with CPF</u>	789
<u>Working with Power Domain in DPA GUI</u>	790
<u>Creating *.ppfdata File for Power Domain</u>	791
<u>Generating TCF and Power Profile Using Power Domain and Power Mode</u>	792
<u>Using Sparse Sampling with DPA</u>	792
<u>Enabling and Disabling Sparse Sampling</u>	793
<u>Estimating Power Using TCF/SAIF Files with Sparse Sampling</u>	794
<u>Improving Accuracy of Power Estimation</u>	794
<u>Generating High Quality TCF Files</u>	794
<u>Estimating Accurate Power Consumption of RTL Designs</u>	795
<u>Estimating Accurate Power Consumption of Gate-Level Netlists</u>	795
<u>DPA Licenses</u>	796
<u>DPA Tutorial Examples</u>	797

16

<u>Specifying Low-Power Intent on Palladium XP</u>	799
<u>Support for CPF Commands on Palladium XP</u>	800
<u>CPF 2.0 Features Supported on Palladium XP (Beta-Level Feature)</u>	803
<u>Design and Instance Semantic Differences in CPF 1.1 and CPF 2.0</u>	804
<u>Processing CPF Information on Palladium XP</u>	805
<u>Creating CPF Files</u>	806
<u>Adding CPF Information to a Design</u>	807
<u>Compiling a Design with Low-Power Intent</u>	808
<u>Running and Debugging a Compiled Design with Low-Power Intent</u>	808
<u>Design Considerations for Low-Power Verification</u>	827
<u>Creating Power Domains</u>	828
<u>Using Verilog Registers as Power-Domain Instances</u>	829
<u>Defining Power-Domain States</u>	830
<u>Resolving Inconsistencies between the Designs</u>	833
<u>Handling Power Shut-Off Designs</u>	834
<u>Handling Designs with Multiple Supply Voltages (MSV)</u>	836
<u>Handling Dynamic Voltage Frequency Scaling (DVFS)</u>	836
<u>Verifying Hard-Custom IPs Using Macro CPF Modules</u>	837
<u>Logging of CPF Events (Beta-Level Feature)</u>	840
<u>Types of CPF-Event Logs</u>	840
<u>Format of CPF-Events Log Messages</u>	843
<u>Show CPF-Events Log Messages from a Log File</u>	843
<u>Using Dynamic Power Analysis with Low-Power-Aware Designs</u>	844
<u>Selecting Low-Power-Aware Verification Method Based on Design Type</u>	846
<u>Emulating RTL Low-Power-Aware Design</u>	846
<u>Emulating Gate-Level Low-Power-Aware Design</u>	847

17

<u>Improving Emulation Speed</u>	849
<u>Using Design Retiming</u>	849
<u>Enabling or Disabling Design Retiming</u>	849
<u>Improving Oversampling</u>	850
<u>Upgrading from 4X Mode to 2X Mode</u>	851

UXE User Guide

<u>Upgrading from 2X Mode to 1X Mode</u>	851
<u>Upgrading from 1X Mode to 0.5 Mode</u>	851
<u>Improving Step Count</u>	852
<u>Factors Contributing to Step Count</u>	852
<u>General Techniques to Improve Step Count</u>	853
<u>Finding the Reason for a Poor Step Count</u>	853
<u>What to Look for in the Critical Path</u>	856
<u>Viewing the Critical Path in the Schematic Viewer</u>	859
<u>Effects of Memory on Step Count</u>	860
<u>Using breakNet and breakPin</u>	862
<u>A Possible Shortcoming of the Post-Schedule Path Report</u>	863
<u>Effect of Vision Mode</u>	864
<u>Use of compilerEffort, optimizationEffort and placementEffort</u>	864
<u>Files to Send to Customer Support</u>	864

18

<u>Debugging Designs on Palladium XP</u>	865
<u>Basics of Starting Debug Session</u>	867
<u>Invoking the Debugger Tool(s)</u>	867
<u>Command Sequence for ICE Mode</u>	867
<u>Debugging Process for SA Mode with xeDebug</u>	870
<u>Debugging Process for SA Mode with irun</u>	871
<u>Tracing Signals Using Probes</u>	872
<u>Setting Probes</u>	873
<u>Uploading Probes</u>	874
<u>Probing Modes</u>	875
<u>Changing the Vision Mode at Run time</u>	883
<u>Tracing Signals Using InfiniTrace</u>	884
<u>InfiniTrace Sessions</u>	885
<u>Using InifiniTrace in ICE Mode</u>	886
<u>Using infinTrace Command Options in SWIT and HWIT Modes</u>	888
<u>Comparing SWIT and HWIT Modes</u>	889
<u>Using InfiniTrace in SA Mode Using xeDebug</u>	890
<u>Using InfiniTrace in SA Mode Using irun</u>	891
<u>Comparing the InfiniTrace Feature in SA and ICE Modes</u>	892

UXE User Guide

<u>Bookmarking InfiniTrace Session</u>	893
<u>Tracing and Uploading Waveforms</u>	893
<u>Tracing and Uploading Waveforms in ICE Mode Using xeDebug Tool</u>	894
<u>Tracing and Uploading Waveforms in SA Mode Using xeDebug Tool</u>	896
<u>Tracing and Uploading Waveforms in SA Mode Using irun</u>	906
<u>Dynamic Swap between Hardware and Software in SA Mode</u>	906
<u>Using Hot Swap Options</u>	907
<u>Using Hot Swap Run Modes</u>	908
<u>Software Simulation Mode</u>	909
<u>Run Mode</u>	909
<u>TBrun Mode</u>	909
<u>NBrun Mode</u>	911
<u>Autorun Mode</u>	917
<u>Match Mode</u>	918
<u>Accessing Signal Values</u>	919
<u>Forcing Values</u>	919
<u>Releasing Values</u>	921
<u>Depositing Values</u>	921
<u>Using the Palladium Xref Browser of SimVision GUI in SA Mode</u>	923
<u>Using the Source Browsers of the Palladium Xref Browser</u>	924
<u>Debugging in Offline Session</u>	926
<u>Generating Trace Data and Preparing for Offline Session</u>	926
<u>Starting an Offline Session</u>	927
<u>Debugging in Offline Session to Save Emulator Time</u>	928
<u>Using Offline Debugging</u>	928
<u>Additional Debugging Features in Offline Session</u>	928
<u>Examples of Using Offline Session</u>	930
<u>Files Generated during Debugging</u>	932
<u>xeDebug.log</u>	932
<u>xe.msg</u>	932
<u>xeDebug.key</u>	933
<u>Run-Time Command Support in ICE and SA Modes</u>	934
<u>Running Designs with Uncontrolled Clocks Using xeDebug in SA Mode</u>	935
<u>Capture and Display Control</u>	936
<u>Stopping the Design Clocks and Data Capture</u>	937
<u>Using Uncontrolled Clocks with Other Debugging Operations</u>	940

19

Controlling the Run with SDL	943
SDL Concepts	943
Multiple SDL Instances	944
Enabling and Disabling Individual SDL Instances	945
Enabling and Disabling Individual Trigger, Display, or Exec Statements	946
Conditional Acquisition	946
State Machine versus Global trigger	947
SDL Counters	947
Statement Usage and Execution Semantics of an SDL State Machine	948
SDL Clocking	949
File Inclusion	949
Aliases	949
Use of Tcl Variables	949
Assertions	950
Auto Probing	950
Fast versus Accurate	950
Debugging SDL Operation with the SDL Trace Dump Facility	951
Saving and Restoring the Internal State of SDL	952
Detection of Events without Triggering	953
Limitations when Running with 1x Sampling Ratio	953
Limitations when Running with 0.5x Sampling Ratio	953
Interaction between SDL and XEL	953
Printing Messages from SDL with the DISPLAY Facility	955
Emulation Resources Consumed by SDL	957
Summary of Basic SDL features	958
SDL Preparation, Compilation and Execution	958
Setup during Design Compilation	958
Preparing the SDL Program	959
SDL Enabling, Activation, and Execution	959
Formatting and General Syntax Rules	960
Format	960
Comments	961
White Space	961
Case Sensitivity	961

UXE User Guide

<u>File Inclusion</u>	961
<u>Using Values of Tcl Variables Inside SDL</u>	962
<u>SDL Language Constructs</u>	963
<u>Expressions</u>	968
<u>Operators</u>	974
<u>Global or Instance Definition Sections</u>	981
<u>INSTANCE Statement</u>	988
<u>STATE Statement</u>	989
<u>IF - ELSE statement</u>	989
<u>Actions</u>	991
<u>Examples: SDL Program</u>	1001
<u>Example 1: A State-Less SDL Program</u>	1001
<u>Example 2: A Single SDL Instance</u>	1001
<u>Example 3: SDL Program with Two Instances</u>	1002
<u>Importing Breakpoints from the NC-Sim Simulator into SDL</u>	1003
<u>Theory of Operation</u>	1003
<u>Expressions in Breakpoints</u>	1004
<u>Error Conditions</u>	1004
<u>Limitations and Differences in Behavior</u>	1005

20

<u>Using Novas Tools</u>	1007
<u>Integrating Novas Tools with xeDebug</u>	1007
<u>Setting the Environment Variables for Running Novas Tools</u>	1007
<u>Functions Supported by Novas Tools</u>	1009
<u>Generating the Debussy OH Database</u>	1016
<u>Example of Generating Debussy OH Database Using HDL Compiler</u>	1018

A

<u>Loop-Breaking</u>	1019
<u>Definition of Loop-Breaking</u>	1019
<u>Problems Caused by Loop-Breaking</u>	1019
<u>Proactive Steps to Avoid Loop-Breaking Problems</u>	1020
<u>Steps to Break Loops and Long Paths</u>	1020
<u>Miscellaneous Steps for Clock Delays</u>	1021

UXE User Guide

<u>What to Do if a Loop-Breaking Problem is Suspected</u>	1021
<u>Delays in Palladium XP</u>	1022
<u>Avoiding the Need for Oversampling</u>	1023
<u>Preventing a Particular Break</u>	1024
<u>Identifying (clk) Nets with Precompile</u>	1025
<u>Avoiding Clock Delay with Precompile</u>	1026
<u>Solving Clock-Delay Problems Caused by Loop-Breaking</u>	1026
<u>Designs Containing Latches</u>	1027
<u>Clock Gating Cells</u>	1029
<u>Classification of Loops</u>	1029
<u>Using the Precompile -breakHalfCycleAll Command</u>	1030
<u>Using the Precompile -dontMinimizeDataBreaks (or -dataLoops101) Command</u>	1031
<u>The *.brk File</u>	1032
<u>The *.loops File</u>	1033
<u>Displaying a Loop in the Schematic Viewer</u>	1035
<u>Loops through Memory</u>	1035
<u>Loops Discovered after Precompile</u>	1036
<u>Detecting Clock Delay</u>	1036
<u>Rarely-Used Options</u>	1039

B

<u>Modeling Guide for Verilog Designs for ICE Mode</u>	1041
<u>Creating Synthesizable Verilog Designs</u>	1041
<u>Using Verilog HDL</u>	1042
<u>Identifying Simulation Directives</u>	1044
<u>Identifying System Tasks</u>	1045
<u>Identifying Keywords</u>	1045
<u>Identifying Unsupported Statements</u>	1046
<u>Considering Synthesis Assumptions</u>	1047
<u>Modeling Sequential Logic with always Blocks</u>	1049
<u>Register Inference</u>	1050
<u>Latch Logic Inference</u>	1052
<u>Synthesizing Tristate Logic</u>	1054
<u>Examples of Verilog Tristate Logic</u>	1054

UXE User Guide

<u>Enabling or Disabling Memory Synthesis</u>	1055
<u>Out of Module Reference</u>	1055
<u>Verilog -2001 Configuration</u>	1056
<u>SystemVerilog</u>	1057
<u>Supported SystemVerilog Features</u>	1057

C

<u>Modeling Guide for VHDL Designs for ICE Mode</u>	1071
<u>Entities and Architectures</u>	1072
<u>Objects</u>	1073
<u>Data Types</u>	1075
<u>Operators</u>	1080
<u>Operands</u>	1083
<u>Aggregates</u>	1085
<u>Subprograms</u>	1099
<u>Packages</u>	1102
<u>VHDL Synthesis Directives</u>	1106
<u>Enabling or Disabling Memory Synthesis</u>	1110
<u>Syntax and Semantic Restrictions</u>	1111
<u>Modeling Styles</u>	1112
<u>Configuration Declaration</u>	1125

D

<u>Guidelines for Writing Testbenches</u>	1129
<u>Following a Synthesizable Coding Style</u>	1130
<u>Using Initial Blocks with(Verilog Only)</u>	1130
<u>Initial Block Statements That Delay Swapping to Hardware</u>	1131
<u>Initial Block Statements That Prevent Swapping to Simulation Acceleration Mode</u>	1131
<u>Specifying Clocks</u>	1132
<u>Applying Stimulus to Your Design</u>	1134
<u>Synchronous Testbenches</u>	1134
<u>Applying Test Vectors Using Memory (\$random)</u>	1135
<u>Avoiding Testbench Race Conditions (Verilog Only)</u>	1137
<u>Using Hierarchical References in Testbench (Verilog Only)</u>	1137
<u>Accessing Memories Using Hierarchical References</u>	1139

UXE User Guide

<u>Using Verilog PLI in the Testbench</u>	1141
<u>Task/Function Routines Instead of Access Routines</u>	1141
<u>Reading and Writing Signals in DUT Modules</u>	1141
<u>Dealing with Testbench Monitoring Code</u>	1142
<u>Partitioning Monitor Code</u>	1143
<u>Activating Monitor Code Only in Software Simulation</u>	1145
<u>Monitoring Signals and Displaying Messages</u>	1146

E

<u>Support for SystemVerilog Constructs</u>	1149
<u>Support for SVTB Constructs</u>	1149
<u>Support for SystemVerilog Design Constructs</u>	1157
<u>Limitations when Using SystemVerilog Constructs</u>	1164
<u>Handling of Concatenation Operators</u>	1166

F

<u>Using the xeDebug GUI</u>	1167
<u>Invoking xeDebug GUI</u>	1167
<u>Using xeDebug Menu Bar and Toolbar Options</u>	1170
<u>Menu Bar</u>	1170
<u>Toolbar</u>	1182
<u>Starting a Debug Session</u>	1194
<u>Using Setup Tab in SA Mode</u>	1194
<u>Using Setup Tab in ICE Mode</u>	1201
<u>Running the Design Using the xeDebug Tabs</u>	1207
<u>Control Tab</u>	1209
<u>SDL Tab</u>	1222
<u>Probe Tab</u>	1226
<u>Memory Tab</u>	1228
<u>InfiniTrace Tab</u>	1233
<u>Force Tab</u>	1236
<u>RTL/Gate Debug Tab</u>	1238
<u>Deposit Tab</u>	1241
<u>Break Tab</u>	1243
<u>Clock Tab</u>	1246

UXE User Guide

<u>Saving/Restoring Debug Settings</u>	1249
<u>Using Expressions in the Filter Field</u>	1249
<u>Saving Data Entered on xeDebug GUI Tabs</u>	1249
<u>Customizing the xeDebug GUI</u>	1252
<u>Using xegui XEL Command</u>	1252
<u>Loading libtk.so Tk Library (for Advanced Users)</u>	1259

G

<u>Modeling Clocks</u>	1261
<u>Modeling Clocks in ICE Mode</u>	1261
<u>Supported Clock Technologies</u>	1261
<u>Using the 1X Mode</u>	1262
<u>Using the CAKE 0.5X Mode</u>	1285
<u>Inserting Delays between Cross-Domain Synchronizers</u>	1287
<u>Modeling Clocks in SA Mode</u>	1293
<u>Manually Modeling Clocks in SA Mode</u>	1293
<u>Using userData to Create a Clock-Generation Module for SA Mode</u>	1305

H

<u>irun - IXCOM Option Mapping</u>	1323
<u>Mapping irun Options to IXCOM Options with the Same Syntax and Behavior</u>	1324
<u>Mapping irun Options with IXCOM Options with Different Syntax but the Same Behavior</u> ..	1326

I

<u>Compiling and Running Designs with Memories</u>	1327
<u>Supported Memory File Formats</u>	1328
<u>Transforming Memories</u>	1328
<u>Transforming Shallow Memory Instances into Gate Arrays</u>	1329
<u>Merging Memory Instances</u>	1330
<u>Splitting Memory Read Ports</u>	1331
<u>Squeezing Memory Instances with Constant Address Pins</u>	1332
<u>Compacting Memories</u>	1332
<u>Handling Sparse Memories</u>	1334

UXE User Guide

<u>Modeling Sparsely-Used Memories at Compile Time in ICE Mode</u>	1335
<u>Accessing Sparsely-Used Memories at Run Time in ICE Mode</u>	1337
<u>Support for Sparse Memories in SA Mode</u>	1339
<u>Using Memory Primitives</u>	1346
<u>Probing Restrictions</u>	1346
<u>Building Memory Primitives</u>	1346
<u>Enforcing Design Rules for Memory Primitives</u>	1347
<u>Running Designs with Memories</u>	1350
<u>Global Memory Options</u>	1350
<u>The Memory Address Register</u>	1351
<u>Loading Memories</u>	1351
<u>Loading/Dumping Memory</u>	1352
<u>Dumping Memories</u>	1355
<u>Using Memory Streaming</u>	1356

J

<u>Support for axis smem Memory Model</u>	1361
<u>Memory C API Functions</u>	1365

K

<u>Files and File Formats</u>	1367
<u>Configuration Files</u>	1367
<u>Configuration File Information</u>	1368
<u>Basic Language Elements</u>	1368
<u>External Timing Files</u>	1369
<u>Design Files</u>	1370
<u>Memory File Format</u>	1370
<u>Binary Memory File Formats</u>	1371
<u>Raw Format</u>	1371
<u>memTran Format</u>	1372
<u>Cycle-By-Cycle (CBC) Format</u>	1373
<u>Clock Domain Crossing (CDC) Format</u>	1375

L

<u>Using Utilities</u>	1377
<u>cds2libmap</u>	1377
<u>Purpose</u>	1377
<u>Syntax</u>	1377
<u>Parameters</u>	1377
<u>vhdel</u>	1378
<u>Purpose</u>	1378
<u>Syntax</u>	1378
<u>vhls</u>	1379
<u>Purpose</u>	1379
<u>Syntax</u>	1379
<u>vhmake</u>	1380
<u>Purpose</u>	1380
<u>Syntax</u>	1380
<u>Description</u>	1381
<u>vhshell</u>	1382
<u>Purpose</u>	1382
<u>Syntax</u>	1382

M

<u>Using \$ixc_ctrl Pragma</u>	1387
<u>IXC_CTRL Pragmas for Elaboration and Mapping</u>	1387
<u>IXC_CTRL Pragmas for Interface Compilation</u>	1388
<u>IXC_CTRL Pragmas for Behavioral Compilation</u>	1389
<u>IXC_CTRL Pragmas for Signal-Driven HMPI Mode</u>	1389

N

<u>Methodology Guidelines for Compiling Assertions with the IXCOM Compiler</u>	1391
<u>Inspecting IXCOM Log File for Assertion-Related Information</u>	1391
<u>Number of Assertions Synthesized by the IXCOM Compiler</u>	1392
<u>Errors Generated during Compilation of a Design with Assertions</u>	1393

UXE User Guide

<u>Assertions Using X or Z Values</u>	1395
<u>Advanced Support for SVA and PSL Assertions with Behavioral Compilation</u>	1396
<u>Assertions with Multi-Clocked Consequent</u>	1397
<u>Local Variables and Sequence Match Items</u>	1398
<u>Handling of Large Delays</u>	1398
<u>Handling expect Statements</u>	1399
<u>Differences in Handling of Assertions</u>	1399
<u>Between IES Simulator and Emulator (Compiled Using the IXCOM Compiler)</u>	1400
<u>Between the IXCOM Compiler and LRM</u>	1400
<u>Best Practices for Assertions</u>	1402
<u>Using ifdef, Bind, or Vunits to Write Assertions</u>	1402
<u>Using Immediate Assertions when the Sequence Expression is a Constant Expression</u>	1402
<u>Using Variables Instead of Constants in Sequence Expressions</u>	1404
<u>Labeling Assertions</u>	1404

Q

<u>Invoking Palladium XP Compilers and Debugging Tools</u>	1407
<u>Invoking xeCompile</u>	1407
<u>Invoking xeDebug Tool</u>	1408
<u>Invoking <i>irun</i> Tool (for SA Mode)</u>	1409
<u>Using the XEL Shell</u>	1410
<u>Initializing the XEL Shell for Design Compilation</u>	1410
<u>Initializing the XEL Shell for Design Debug</u>	1410
<u>Quitting the XEL Shell</u>	1411
<u>Using Help for XEL Command Syntax</u>	1411
<u>Using Special Characters</u>	1412
<u>Using XEL Scripts</u>	1412

P

<u>Importing Netlists as Blackboxes in SA Mode</u>	1413
<u>Use Model for Importing a Netlist as Blackbox Using IXCOM</u>	1414
<u>Testbench Access to Netlist Objects</u>	1416

Q

<u>Third Party License Terms</u>	1423
--	------

Overview

Cadence® Palladium® XP is a verification computing platform that integrates the Incisive® Enterprise Simulator (IES) tool to work together with Palladium® XP emulator for accelerating your simulation session without changing the design methodology. The software tools that enable you to use the Palladium XP emulator have been collectively termed as Unified Xccelerator Emulator (UXE). The UXE design verification system unifies Simulation Acceleration (SA) and Emulation (ICE) capabilities in a single environment that consists of the Palladium XP emulator series and UXE software release.

This user guide describes the processes for compiling and running designs on the Palladium XP emulator. It assumes that the Palladium XP emulator has been installed and the required SA and host workstation setup is configured as per user requirements. For details about the installation of the Palladium XP emulator and workstation setup, refer to the *UXE Planning and Installation Guide* provided as a part of the UXE documentation set. A brief description to configure and work with Palladium XP emulator hosts at compile and run times is provided in [Chapter 2, “Working with the Palladium XP Emulators.”](#)

A design is compiled for different modes, namely In-Circuit Emulation (ICE) and Simulation Acceleration (SA) according to the requirements. After compilation, the compile result is imported into the run-time interface to run and debug the design on the selected Palladium XP emulator system.

Here is a brief description of the two modes:

■ In-Circuit Emulation (ICE) mode

In this mode, the design is completely synthesizable and runs entirely on the Palladium XP emulator. For ICE mode compilation, xeCompile tool is used, and the run-time and debug functionality on the emulator is handled by the xeDebug tool.

■ Simulation Acceleration (SA) mode

In this mode, only the synthesizable part of the design runs on the Palladium XP emulator. The rest of the design, that is, the non-synthesizable or behavioral part of the design, runs on the software simulator. For SA mode compilation, the SA compiler is primarily Incisive Xaccelerator **COM**piler, that is, IXCOM. IXCOM compiles RTL, gate-level netlists, and some behavioral HDL constructs into the

UXE User Guide

Overview

hardware emulator by internally invoking xeCompile, and keeps other behavioral elements and testbenches to run with the IES software simulator, *irun*. At run time, you can select the UXE debug tool as xeDebug and/or *irun* depending on the requirements. Even though xeDebug and *irun* are two separate tools, both offer run-time and debug capabilities for SA mode, in the UXE design verification environment, these work in a fairly integrated fashion to ensure that you can access and manipulate the design objects in both software and hardware portions at run time.

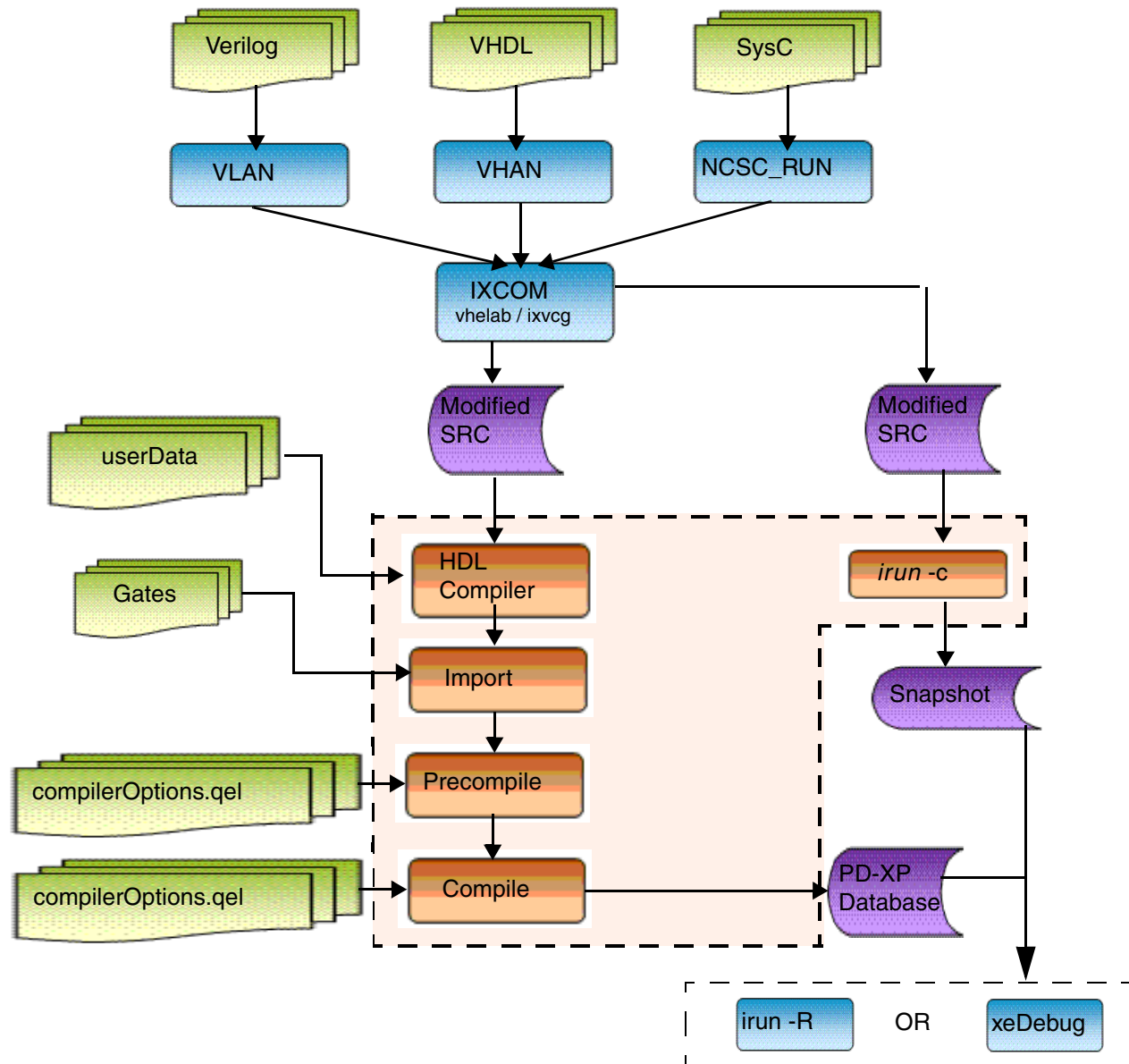
When compiling the designs using UXE software tools, determine the appropriate compilation mode, ICE or SA, according to the design requirements. The UXE compile tool is xeCompile for ICE mode and IXCOM for SA mode. For SA mode, you can also choose to use *irun*, which is the INCISIV software simulator tool to compile designs for Simulation Acceleration on Palladium XP emulator. *irun*, then invokes IXCOM, internally to compile the design for emulation.

Consequently, following are the possible compile and run use flows offered on Palladium XP emulator:

- Compile the design for ICE mode using xeCompile and run using xeDebug
Refer to [Chapter 3, “Compiling Designs for In-Circuit Emulation with xeCompile”](#) for details on the xeCompile tool and [Chapter 18, “Debugging Designs on Palladium XP”](#) for details on the xeDebug tool.
- Compile the design for SA mode using IXCOM and run using xeDebug
Refer to [Chapter 4, “Compiling Designs for Simulation Acceleration Using IXCOM”](#) for details on the IXCOM tool and [Chapter 18, “Debugging Designs on Palladium XP”](#) for details on the xeDebug tool.
- Compile the design for SA mode using IXCOM and run using *irun* (Software Simulation)
- Compile the design for SA mode using *irun* and run using *irun* (Simulation Acceleration)
Note: This use flow helps you to easily migrate to SA mode, while continuing to use *irun*, and accelerate the designs on the Palladium XP emulator. In this mode, too many changes in the compilation and simulation command scripts are not required. This mode is documented in [Chapter 5, “Compiling and Running Simulation Acceleration Designs with *irun*.”](#)

The following diagram illustrates the basic use flows available on Palladium XP emulator.

Figure 1-1 Palladium XP Use Flows



Compiling the Designs for ICE Mode

In the ICE mode, the xeCompile tool, which is the UXE Compiler, enables you to import the register-transfer-level (RTL) designs or gate-level netlist, prepare the design for compilation, and compile the design for emulation.

UXE User Guide

Overview

When you import RTL designs written in Verilog and/or VHDL, xeCompile works as an HDL compiler and enables conversion of the RTL designs to a gate-level netlist or a database that is used to process the compilation result.

The Palladium XP emulator can be connected to a target system. A target system (or just a target) is an external board-level electronic system that you provide. It communicates with the emulated design through special connectors on an external panel of the Palladium system. A static target is a target system with clocks that can be stopped and restarted without corrupting its state. A dynamic target is a target with clocks that cannot be stopped and restarted.

Refer to [Chapter 3, “Compiling Designs for In-Circuit Emulation with xeCompile”](#) for a detailed description of using xeCompile.

In the ICE mode, UXE supports Synthesizable TestBench (STB) and Logic Analyzer (LA) operation modes, as well as Vector-based Acceleration, also known as Vector Debug (VD).

Synthesizable Testbench (STB)

STB mode is used for emulating the DUT and the testbench, which is completely synthesizable, with a static target (that allows stopping and restarting the clock), or without a target system connection (targetless mode). You can start and stop the design clocks. All the design clocks must be generated inside the emulator. This is true whether a target system is connected or not. If a target system is connected to the emulator, stopping the emulator clocks will also stop the target system clocks.

STB is not suited for dynamic targets (whose clock cannot be stopped). If you try to use STB with a dynamic target, the emulator's clocks can get suspended by some features, and this can cause the target to malfunction.

Logic Analyzer (LA)

Logic Analyzer (LA) mode uses a dynamic target. Although it can also work successfully with a static target or without a target at all, such configurations are not very useful because Logic Analyzer mode has only a subset of the debugging features of STB mode. With Logic Analyzer mode, a design can either have the Palladium XP system drive the target clock, or have the target system drive the design clock in the Palladium XP system. The design clock is normally always running. In this mode, data is continuously collected into a circular trace buffer inside the Palladium XP emulator. The probed data at that point can be uploaded to the workstation's disk for viewing. Unlike STB mode, a trigger in LA mode only stops data collection, but not the design clocks. (It is possible to stop the design clock, but not to restart it.)

LA mode supports FV on Palladium XP. However, be aware that you cannot append multiple triggers to one waveform. The previous waveform is cleared unless a different waveform file is specified before the next trigger and upload occurs.

Vector Debug (VD) Mode

Vector Debug (VD) applies test stimulus to the design from a vector file and obtains response vectors. The vector file contains a list of vectors (one per cycle) where each vector defines the stimulus and expected results for that particular cycle. This mode is most suited for regression tests that were created from simulation results or ATPG (Automatic Test Pattern Generation) software.

You can either call PTRAN as an independent executable to translate from a variety of stimulus file formats into Cadence's standard file, which Vector Debug uses directly, or you can use the Graphical Debug Tools to select the stimulus file format and the Vector Debug tool that will call PTRAN to automatically perform this translation.

Vector Debug mode cannot use a target connection.

Compiling the Designs for SA Mode

SA mode integrates workstation simulation with the hardware emulator for increased speed. SA mode allows you to enhance performance in a hardware-assisted environment. It allows you to verify and debug designs in which part of the design is accelerated by Palladium, while another part (typically the testbench) is run using *irun*, while using the debugging capabilities of SimVision.

In the SA mode, the Palladium SA mode compiler, IXCOM, tightly integrates the IES simulation kernel with the emulator, thereby giving the performance of an accelerator or emulator.

IXCOM provides an easy-to-use, simulator-style interface and a single-step compile command for both modes: software simulation mode and simulation acceleration mode with the emulator. The source files and command-line options provided with the `ixcom` command are passed to the respective back-end compilation tools by the IXCOM tool. For the back-end compilation, IXCOM internally invokes xeCompile, the ICE compilation tool, passing on the options and settings that you specify to the IXCOM tool as compiler directives. IXCOM handles Verilog and VHDL designs and provides support for SystemVerilog Assertions (SVA) and Accellera Property Specification Language (PSL) assertions, SystemC components, and PLI integration using *irun*. IXCOM compiles the design and generates the simulation snapshot that can be run by using both the xeDebug and *irun* tools.

Refer to [Chapter 4, “Compiling Designs for Simulation Acceleration Using IXCOM”](#) for a detailed description of using IXCOM.

Compiling the Designs for SA Mode Using *irun*

This mode enables you to use *irun* for compiling and running the designs in SA mode on the Palladium XP emulator. The original *irun* commands can be modified by adding just a couple of command options to compile the designs for hardware. The *irun* use model for simulation has been enhanced to invoke IXCOM tools to compile a design for simulation acceleration and simulate the hardware-acceleratable snapshot. Thus, you can transition from the pure IES-based software simulation mode to simulation acceleration mode using Palladium XP. Details of this mode are described in [Chapter 5, “Compiling and Running Simulation Acceleration Designs with *irun*.”](#) Details about the *irun* tool are provided in the *irun* User Guide as a part of the INCISIV documentation set.

Debugging the Designs

After compilation, the compiled database is downloaded to the Palladium XP emulator, and can be run in the ICE and SA modes, as per user requirements:

- For debugging designs in ICE mode, use the xeDebug tool to run and debug the design on hardware. Refer to [Chapter 18, “Debugging Designs on Palladium XP”](#) for details on using the xeDebug tool.
- For debugging designs in SA mode, you can choose the debugging tool according to the portion of the design you need to access. When running in SA mode, the two portions of the design are distinguished as follows:

- ☐ Device Under Test (DUT) is the portion of the design that runs on the emulator hardware.

To access this portion, use the xeDebug tool.

- ☐ TestBench (TB) refers to the portion of the design that runs on the software simulator.

To access this portion, use the *irun* tool, the front-end tool for the IES software simulator, which is integrated with Palladium XP to provide the complete Simulation Acceleration solution. In UXE, *irun* can be run in:

- Software Simulation Mode - In Software Simulation mode, the entire design runs in software on the host workstation; the emulator is not used at all. Running simulation in this mode is very similar, but not identical, to software simulation for designs that were not compiled for emulator modes.

- Simulation Acceleration mode - In the Simulation Acceleration mode, those parts of your design (the Device Under Test or DUT) that meet certain requirements are downloaded to the emulator and accelerated. The remaining parts not meeting these requirements are run in software simulation. In a typical scenario, the synthesizable ASICs would run in the emulator, while the behavioral testbench would run in software simulation.

irun usage is analogous to that of IES with some exceptions. For detailed information on using *irun*, refer to [Chapter 5, “Compiling and Running Simulation Acceleration Designs with *irun*.”](#) More details on the *irun* tool are available in the *irun* User Guide of the INCISIV documentation set.

Debug Vision Modes

The options and design information available at the time of running and debugging the designs are determined by the Vision Mode selected at the compile time.

At compile time, you must chose from one of the following Vision modes:

- [Dynamic Probes](#) (Vision mode: DYNP)
- [FullVision](#) (Vision mode: FV - the default mode)

Dynamic Probes

In the Dynamic Probes (DYNP) Vision mode, you can specify the signals before the design run, during debug, depending on available probe channels. For DYNP, the maximum trace depth is 43 M cycles. Conditional acquisition is supported with DYNP.

Advantages of using DYNP:

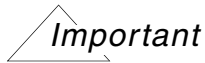
- Fastest waveform upload
- Avoids emulator capacity and emulation speed costs that occur in some designs when FullVision Vision mode is used

Disadvantage of using DYNP:

Before running a test, you must specify the signals to be probed in that run. The number of signals that can be probed in a specified run is limited.

FullVision

In the FullVision (FV) mode, all internal signals are visible at debug or run time. FV can be used when the emulator is connected to a dynamic target. The maximum trace depth in FV is the same as DYNP - 43 M cycles.



When using FV mode, you need to use 64-bit mode, which can be set using the `xeDesignBit 64` command.

In STB mode, FV supports incremental upload. You can mix run and trigger operations, add more probes, and upload probes. In SA mode, FV mode requires to capture every fclk cycle in which DUT status changes. During simulation acceleration, multiple DUT evaluations might be needed for one simulation time. This means that one simulation time might need to capture the DUT status multiple times. The waveform displays only the last capture for each simulation time. Since samples were captured at multiple times (which are to be discarded later), this leads to a smaller trace depth. Conditional acquisition is supported with FV.

Advantages of using FV:

- ☐ You do not need to choose the signals to be probed before running.
- ☐ An unlimited number of signals can be probed.

Disadvantage of using FV:

For most designs, FullVision has a capacity cost, compared to using DYNP. The capacity cost is usually 0-30% for a trace depth of 16K FCLKs.

Debug Sessions

To run and debug a design, you can start either an online debugging session or an offline debugging session. If you want to switch from an online debugging session to an offline debugging session (or vice versa), you must first release the current session.

Note: From this point onwards in this guide, the online debugging session will be referred as online session and the offline debugging session will be referred as offline session.

Debug Operations

When debugging the designs, you can perform signal trace operations, like specifying the signals to be traced, uploading the trace results to the trace file, and for power-aware designs,

adding power-domain probe-related functionality, and collecting toggle count data during run time for dynamic power analysis.

Following are the common debug operations supported by the UXE debug tools:

■ Tracing Signals

You can trace signals and nets by adding probes and viewing them in waveforms.

❑ Setting Probes:

Probes are the means by which you can get values of signals from the design being emulated. Refer to [Tracing Signals Using Probes](#) on page 872 for more details on setting probes.

❑ Tracing Signals using InfiniTrace:

The InfiniTrace feature helps save the state of the design during the run along a segment of simulation time, so that, this state can be replayed from any time points later to observe the emulation and simulation acceleration status. During the replay, you can observe the simulation status, dump out memory content, get the value of any of the nets in the design and add probes to upload the waveform of any nets. This feature offers infinite trace depth similar to simulation, and it overcomes the disadvantages of limited trace memory. You can run the design in offline session with InfiniTrace by using the Software InfiniTrace mode (SWIT). Refer to [Tracing Signals Using InfiniTrace](#) on page 884 for details about using InfiniTrace.

■ Accessing Signal Values

Both the xeDebug and *irun* tools enable you to access signals at run time and set the signal values as required by forcing and depositing the values. When required, you can also release the values of selected signals. Refer to [Accessing Signal Values](#) on page 919.

■ Accessing Design Memories

Memories can be read out (dumped) or written to (loaded) by using the `memory` command.

Also, individual memory locations can be viewed with the `value` command, and written into by the `deposit` command.

In emulation, the following memory file formats are supported:

- ❑ **For ASCII:** PD ASCII (`bin`, `oct`, `dec`, `hex`), `readmemb` or `readmemh`, `NC_SIM`, `%dn` (Denali memory format)
- ❑ **For binary:** `memtran`, `raw`, or `raw2`

Note: Loading and dumping memories to the *irun* format on Palladium XP is not identical to the behaviors on standard *irun*.

Refer to [Running Designs with Memories](#) section in [Appendix I, “Compiling and Running Designs with Memories”](#) for details about accessing design memories on Palladium XP emulator at run time.

■ Controlling the Run with SDL and Trigger Conditions

SDL is a run-time debugging feature that allows, based on design signal values, control over several aspects of the run, such as when to stop, when to capture probed data (tracing), and how to insert command execution into the emulation flow.

SDL is the trace control and break mechanism, which can control data capture In-Circuit Emulation (ICE) modes (both STB and LA modes) and Simulation Acceleration (SA) mode. It can stop the running design in Vector Debug (VD), STB and SA modes, and it can schedule execution of XEL commands during a run in STB and SA modes. SDL is specified as a program in human-readable text.

For detailed information on SDL and its command options, refer to the [Chapter 19, “Controlling the Run with SDL,”](#) and for the description of `sdl` command, refer to the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

■ Controlling Assertions

Assertions can be used to verify that specific components in the design behave as expected, and can help in debugging by isolating problem areas. You can use XEL commands for assertion tracing.

You can select the assertions compiled into the design, view the assertion state and status information, and perform various operations assertions like:

- ☐ Enabling assertions
- ☐ Disabling assertions
- ☐ Adding or removing probes to selected assertions

For more information on controlling assertions, refer to [Compiling and Running Designs with Assertions](#) on page 501.

■ Saving and Restoring Debug Environment

At times, you might choose to re-create a debugging session. For example, you might want to save a set of probes and breakpoints on a design under development, so that it can be re-created at any time. You might want to save the preferences, re-create a complex mnemonic map, or open the same set of browser windows every time you

simulate a design. The debugging environment is made up of the preference settings and the windows, cursors, Forcing and Releasing Testbench and DUT objects, and more.

In addition to saving and restoring debugging sessions, the entire state of the design (including both DUT and TB) can be saved on the disk and restored later.

■ Dynamically Swapping between Hardware and Software (Hot-Swap)

You can choose to run the designs either only on the software simulator or on both the software simulator and emulator. You can dynamically change between the software simulation and SA modes at run time; that is, the current simulation state of a design is instantaneously swapped from one mode to the other. This is called **hot-swapping** and provides major benefits, especially for debugging purposes. For example, you can swap the design to Software Simulation mode to access its extensive debugging capabilities, then swap to the Simulation Acceleration mode for higher performance. The Command-Line Interface (CLI) commands and special system tasks in your HDL source give you control over this.

At the time of hot swap (swap-out) the simulator takes full control and the emulator is released. Later, the design can be swapped back to the emulator (as available) and the emulator takes control (swap-in).

For more information on hot-swap feature, refer to [Dynamic Swap between Hardware and Software in SA Mode](#) on page 906.

■ Debugging in Offline Mode

Offline debugging allows you to run the design and save the traced results for later debugging without needing to run an emulator session. This frees the emulator for the use of others. Refer to [Debugging in Offline Session](#) on page 926 for details about debugging designs in offline session using xeDebug tool.

■ Verifying Low-Power Designs Using Common Power Format

Common Power Format (CPF) allows verification of low-power designs. The appropriate files are set up at compile time (as described in [Chapter 16, “Specifying Low-Power Intent on Palladium XP”](#)) and are read into either the FSDB or SST2 databases.

At run time, the design along with the power information is downloaded into the emulator and operated upon as needed. The information viewed in the waveform viewer displays the power information for the specified time or time interval.

■ Dynamic Power Analysis

Using the Dynamic Power Analysis (DPA) feature, you can estimate the total power consumed by a design to identify the power peaks that can break the functionality of the design and segregate the highly active modules. Details of the DPA feature are documented in [Chapter 15, “Analyzing Power Consumption of the Designs.”](#)

Waveform Database

To perform debug operations, you need to set the format of the waveform database at the time of invoking the debugger tool.

The waveform database can be generated in the following formats:

- SHM (also known as SST2)
- FSDB
- Value Change Dump (VCD)

The waveform database includes trace files that contain the data captured during the design debug process. The traced data can be viewed as waveforms using an appropriate waveform viewer.

SHM

SHM (also known as SST2) is the default waveform database format. The waveform output of the SHM format is stored in the output directory labeled as `<database_name>.shm`, where `<database_name>` is the same as specified with the `database -open XEL` command. This output directory contains two files, `<database_name>.dsn` and `<database_name>.trn`.

The waveform database files of SHM format can be imported and viewed in the SimVision waveform viewer.

FSDB

You can also choose to create a waveform database of FSDB format. At run time, the following three output files of FSDB format are created, but the `<database_name>_sdl.fsdb` file and `<database_name>_sdl.rc` file are created only when requesting to upload SDL activity, when SDL is enabled:

- A file named `<database_name>.fsdb` that holds the FSDB database
- A file named `<database_name>_sdl.fsdb` that holds the SDL COUNTERs and STATE
- A file named `<database_name>_sdl.rc` that has definitions for improved viewing, the most important one being a mapping of SDL state values to state names

The waveform database files of FSDB format can be imported into the Novas waveform viewers. The `.rc` file can be read by the Novas waveform viewer using its `restore signals` option.

Note: Before using the Novas tools to view waveforms of FSDB format, you need to set the required environment variables. For details, refer to [Setting the Environment Variables for Running Novas Tools](#) section in [Chapter 20, “Using Novas Tools.”](#)

Value Change Dump (VCD)

The VCD database records the data signal changes that occur during the design run. It records changes only on the signals that you have specified using the `probe -add <signal> XEL` command. The data capture starts after the design is run and trace results are uploaded to the trace file using the `database -upload XEL` command.

The VCD format significantly increases the debugging productivity by providing access to node history values from any point during run time, without requiring a rerun. The VCD format file is an ASCII file that comprises of header information, variable definitions, and the value changes for a variable in scalar or vector format, based on the variable type.

The SimVision (*irun*) and xeDebug waveform viewers provide support for viewing VCD files.

Setting Up the Environment for Palladium XP

To compile and run the designs, the environment variables discussed in the following sections must be set:

- ☐ [Environment Variables for the Palladium XP System \(Mandatory\)](#)
- ☐ [Environment Variables for IXCOM \(Mandatory\)](#)
- ☐ [Environment Variables for irun \(IES\)](#)
 - ☐ [Additional irun Setup Options in Working Directory \(Optional\)](#)
- ☐ [Environment Variables Required in Specific Cases](#)

Environment Variables for the Palladium XP System (Mandatory)

- ☐ PATH

Set the path variable so that it includes the path to the tools that are in the `<UXE_install_dir>/bin` directory.

A C shell example is:

```
set path = (<UXE_install_dir>/bin $path)
```

❑ LD_LIBRARY_PATH

This variable needs to be set only for SA mode, or if SimVision tool needs to be used at the time of running and debugging designs in ICE mode. Set the library path variable, LD_LIBRARY_PATH, to include the libraries for the appropriate architecture:

○ x86-lx2-32

○ x86-lx2-64

For using simulation with *irun*, set the LD_LIBRARY_PATH variable as follows:

Note: The order of the path settings must be the same as shown here, with the IUS libraries placed before Palladium libraries:

For 64-bits:

```
setenv LD_LIBRARY_PATH \
    ${IES_LIB_PATHS}:<UXE_install_dir>/tools.<platform>/lib/64bit
```

where *<platform>* is a platform-dependent string.

For example:

```
lnx86
```

Following is a C shell example for Linux 64-bit mode:

```
setenv LD_LIBRARY_PATH \
    ${IES_LIB_PATHS}:<UXE_install_dir>/tools.lnx86/lib/
64bit:${LD_LIBRARY_PATH}
```

Environment Variables for IXCOM (Mandatory)

❑ AXIS_HOME

Set this variable to point to the installation directory for UXE.

For example:

```
setenv AXIS_HOME <UXE_install_dir>/tools.<platform>
```

where *<platform>* is a platform-dependent string.

For example:

```
lnx86
```


Environment Variables for *irun* (IES)

IXCOM requires the use of the INCISIV tools and utilities in addition to the UXE software that is run on the Palladium XP system. For using UXE 12.1, the following are the compatible and supported versions:

- INCISIV12.10.001 or higher, except INCISIV12.10.005
- INCISIV11.10.009 or higher

At run time, IXCOM checks if the INCISIV version is compatible with the UXE software. For example, if the INCISIV version is found to be INCISIV 09.20.s009, the following error message will be issued:

```
[sim@cva-opt8 dl]$ irun -R
irun: 09.20-s009: (c) Copyright 1995-2009 Cadence Design Systems, Inc.
Loading snapshot xc_ncwork.test:module ..... Done
Error: Runtime library (libxcrt.so) is compatible with IES 09.20-s014 but you are
using IES 09.20-s009. Please use the IES 09.20-s014 or the later minor version of
IES 09.20 release. To disable this check please pass +ignoreNCVerCheck to irun.
```

You can bypass the compatibility check by using the `+ignoreNCVerCheck` option with the `ixcom` command, if you know that the UXE and INCISIV versions are compatible with each other. Bypassing the check for an incompatible INCISIV version will result in an error or a tool crash.

Following are the variables required for INCISIV simulation tools and utilities:

Variable	Description
PATH	Defines the appropriate bin directory for INCISIV tools.
LD_LIBRARY_PATH	Defines the library path for running INCISIV tools.

Note: Refer to the *Incisive Installation and Configuration Guide* for details about setting up the INCISIV environment.

Additional *irun* Setup Options in Working Directory (Optional)

IXCOM lets you use the *irun* options to compile a design for the emulator. IXCOM automatically generates a file named `<working_dir>/xc_work/hdl.var`, which enables the use of some commonly used *irun* options with the `ixcom` command. You can specify additional *irun* options to be used with IXCOM by adding or creating the following files in the working directory.

```
uxc_cds.lib
```

UXE User Guide

Overview

uxc_hdl.var

Here is an example of the uxc_hdl.var file:

```
DEFINE NCELABOPTS -loadpli1 myplifile.so:boot
DEFINE NCSIMOPTS -i myncsim.tcl
```

Environment Variables Required in Specific Cases

In specific cases, to perform a certain action, you need to set certain environment variables in addition to the mandatory variables discussed in the sections above. The following table lists such extra environment variables and their requirement:

Environment Variable	Required For
Applicable to All Modes	
CDN_MT_HOST	Specifying multiple workstations to improve the performance of uploading data and speed up the collection of the toggle count data. For details, see Specifying Multiple Hosts Using Commands in Chapter 15, “Analyzing Power Consumption of the Designs.”
CDN_FV_MAX_THREAD	Setting the maximum number of CPUs on a host that can be used for DPA. For details, see Parallel Computing for DPA Using Multiple CPUs in Chapter 15, “Analyzing Power Consumption of the Designs.”
■ RC_HOME ■ CDN_SYNTH_ROOT	Using the DPA_RC license for executing the dpa_rc command. For details, see Using DPA_RC Program to Read and Elaborate the RTL Design in Chapter 15, “Analyzing Power Consumption of the Designs.”
RC_LIC	Using the RC license for executing the dpa_rc command. For details, see Using DPA_RC Program to Read and Elaborate the RTL Design in Chapter 15, “Analyzing Power Consumption of the Designs.”

UXE User Guide

Overview

Environment Variable	Required For
SMDB_SAVE_MC	Viewing readable net names that are similar to the original net names in the <code>criticalPathReport.msg</code> file. For details, see Finding the Reason for a Poor Step Count in Chapter 17, “Improving Emulation Speed.”
XE_SECURE_CONNECTION	Establishing a secure connection with the remote network host. Must be set before the software startup.
Specific to ICE Mode	
CDS_LSF_OPTIONS	Specifying complete path to the file that contains the Load Sharing Facility (LSF) options. For details about the content of the LSF options file, see Using LSF to Distribute Compiles with Different Seeds among Different Computers in Chapter 3, “Compiling Designs for In-Circuit Emulation with xeCompile.”
IMPORT_MAX_PARALLEL	Changing, up or down, the maximum number of parallel invocations, either on same-host threads or multi-host LSF jobs. For details, see Using Parallel Import in Chapter 3, “Compiling Designs for In-Circuit Emulation with xeCompile.”
IMPORT_LSF_QUEUE	Changing the default LSF queue established by the LSF administrator. For details, see Using Parallel Import in Chapter 3, “Compiling Designs for In-Circuit Emulation with xeCompile.”
KEEP_ALIVE_XEDEBUG_DONT_EXIT	Retaining the debug session even after DBEngine ends. Note: By default, xeDebug exits within 120 seconds if DBEngine ends unexpectedly. Specifying the <code>KEEP_ALIVE_XEDEBUG_DONT_EXIT</code> environment variable avoids the unexpected shut down of xeDebug.

UXE User Guide

Overview

Environment Variable	Required For
■ LM_LICENSE_FILE ■ NOVAS_EXE ■ NOVAS_HOME ■ NOVAS_LIBPATHS ■ TURBO_ETC_DIR ■ TURBO_LIBPATHS ■ TURBO_LIBS	Invoking the Novas tools. For details, refer to the Setting the Environment Variables for Running Novas Tools section of Chapter 20, “Using Novas Tools.”
XE_RESERVE_KEY	Providing a resource reservation key when downloading the design. For details, see Reserving Emulation Hosts in Chapter 2, “Working with the Palladium XP Emulators.”
XE_DB_SVR_DISP_MSG	Printing detailed information when the design is running. Note: The XE_DB_SVR_DISP_MSG environment variable should be set in the DBServer command shell. For details, see Setting the DBServer in Chapter 2, “Working with the Palladium XP Emulators.”
XE_DB_SVR_RPC_NUM	Specifying a number that is unique for each user. Note: This environment variable must be set before starting the DBServer executable. For details, see Setting the DBServer in Chapter 2, “Working with the Palladium XP Emulators.”
XE_KEEP_HOST_ALIVE_TIMEOUT	Controlling the time-out period value for the KeepHostAlive feature. For details, see Enabling the KeepHostAlive Feature in Chapter 2, “Working with the Palladium XP Emulators.”
Specific to SA Mode	

UXE User Guide

Overview

Environment Variable	Required For
UXE_BOARDS	Specifying the boards and domains for configuring emulator for compilation. For details, see Identifying Design Host and Domains in Chapter 2, “Working with the Palladium XP Emulators.”
UXE_DYNP	Specifying the DYNP compilation mode when configuring emulator for compilation. For details, see Identifying Design Host and Domains in Chapter 2, “Working with the Palladium XP Emulators.”
UXE_HOST	Specifying the emulator to compile for. For details, see Identifying Design Host and Domains in Chapter 2, “Working with the Palladium XP Emulators.”
XC_WORK_DIR	Specifying the design database to the HMPI adapter. For details, see CVP Run-Time Control of HMS in Chapter 13, “IXCOM Hardware Model Interface.”
XC_HM_PORT	Specifying the port number to connect the IPC channel to the HMPI adapter. For details, see CVP Run-Time Control of HMS in Chapter 13, “IXCOM Hardware Model Interface.”
DBE_HOST	Specifying the emulator host. When the DBE_HOST environment variable is set before running a design in SA mode with <i>irun</i>, the Simulation Acceleration software automatically issues the following command: <pre>host \$DBE_HOST</pre> If the DBE_HOST environment variable is not set, it defaults to the <code>host .</code> command that selects the local workstation as the emulator host.

Viewing Global Error Log

UXE provides a feature to write a short summary of any error reported while using most of the UXE tools. The error log is created at a central location for subsequent analysis of all errors accumulated since the error logging feature was turned on. To activate the logging of errors, you must create a soft link from `<UXE_installation_directory>/share/uxe/install/allerrors.msg` to a convenient location to access it.

The error log contains information similar to that recorded in other log files like `xe.msg`, `xeCompile.log`, `xeDebug.log`, and so on. The error log has the following columns:

- date+time
- host name
- user name
- tool name
- error kind
- error ID
- error text (first 80 characters)

As the link originates from the UXE installation directory, every UXE version can have its own error log or all versions can share the same log.

Cadence recommends that an AE or an R&D manager, not an individual engineer, reviews the error log.

Using the readerr Utility

Use the `readerr` utility to view the error log in different formats, such as an error count or report errors grouped by the requested fields. This utility can accept a key to group the results recorded in the error log and criteria to select results.

The following is the syntax of this utility:

```
readerr [-file=<file>] [-key=<key>] [-sort=<0|1>] [-host=<host>] [-user=<user>] /  
        [-program=<program>] [-id=<id>]
```

Here,

- `key` is a list of columns to group error count: `host`, `user`, `program`, or `id`. The default key is `id`.

UXE User Guide

Overview

- ❑ `-host=<host>` gets the count for only the given host name.
- ❑ `-user=<user>` gets the count for only the given user name.
- ❑ `-sort=<0 | 1>` sorts the errors by *count* if set to 1 or by *key* if set to 0. The default value for sorting is set to 1.
- ❑ An asterisk (*) after the number of errors means that the error text differs in the database.

Examples:

```
readerr -key=host,id -user=joe
readerr -host=planet -sort=0
```

To use error analysis, you need to create a link from
`<UXE_installation_directory>/share/uxe/install/allerrors.msg`.

Usage

Calling the `readerr` utility without any options shows a sorted list of errors with the number of incidents for each error. The list is sorted by the number of incidents. For example:

```
% readerr
43984  4268* 20 DATABASE | 267: sim_configuration_management() failed: An SA
connection to the specified host co-
25444  3842* The program failed to initialize the hardware on host 'xxxxxxx' at
'03/12/2012 14:56:42'.
...
```

The first column lists the unique error ID, the second lists the number of incidents, and the last displays the text of the error message. If the text begins with *, it means that the text in the log is different for different incidents.

If you want to select errors by a specific criterion, you can specify the required value for each field. For example:

```
% readerr -user=abcUser1
50173          511* OpenSession: simulator exited.
43287          316* Error in keep alive thread: xeDebug died, exiting...
...
```

The list of acceptable fields is: `host`, `user`, `program`, and `ID`. You can view this list from the man help page for this utility that is invoked by giving the `readerr -help` command.

If you want to group errors by a key instead of error ID, you can specify it with the `-key` option. For example:

UXE User Guide

Overview

```
% readerr -key=host
xxxxxx    2635* 20 ET5 API* | 692: The physical configuration file did not load
properly (rc='0C000004Tx)
yyyyyy    2317* Severe: File xcva_top.etconfgs is empty or contains no valid
configuration fil-
...
```

In this case, you do not see the error IDs, but you see information such as a particular host had the displayed number of errors. A key can have several fields separated by commas. For example, `-key=host,id`

You can combine a key and criteria as illustrated in the following example:

```
% readerr -key=host -user=abcUser1 -program=xeDebug.bin
xxxxxx    355* The program failed to initialize the hardware on host 'xxxxxx' at '04/
09/2012 14:14:35'.
yyyyyy    151* current directory is already locked:    abcUser1 running on yyyyyy
(PID=17049)
...
```

The script is written in Perl and can be viewed to answer some questions not covered in this document. The fields in the log are separated by tabs (that is, `\t`) to simplify parsing, and a third-party analysis tool can be used for reports if needed.

Working with the Palladium XP Emulators

This chapter includes the following sections:

- Working with Emulators at Compile Time
- Working with Emulators at Run Time

Configuring the Emulator or Emulation Host

Following are the mandatory requirements for an emulation host:

- The Palladium XP software and supported operating system should be installed on the specified emulation host.
- The design directory should be accessible to the specified emulation host by using the same path as the workstation on which you run xeDebug or xeCompile.
- The specified emulation host should be able to accept either `rsh` or `ssh` remote execution command.
- The directory `/et3mach` on the emulation host needs to be readable by the workstation on which xeCompile and xeDebug are run.

Working with Emulators at Compile Time

At compile time, you can work with the emulator hosts in the following ways:

- [Identifying Design Host and Domains](#)
- [Compiling for Multiple Configurations \(Symmetric Configuration\)](#)
- [Relocating Designs into Different Domains](#)
- [Specifying the Connection to Target System](#)
- [Duplicating Wires for Transmitting a Signal between Chips or Boards](#)
- [Finding Faults Using ModelChecker](#)

Identifying Design Host and Domains

After opening the design, specify the emulation host and domains that must be used for compilation, as well as the compile mode with which the design is to be compiled. The supported compile modes are Logic Analyzer (LA), Synthesized Testbench (STB), and Vector Debug (VD).

To set up the emulator for compiling the design, use the `emulatorConfiguration XEL` command. The `emulatorConfiguration XEL` command generates a configuration file named `<top_cell>.et3config` in the `./dbFiles` directory, where the `<top_cell>` is the top cell name specified in the `design XEL` command. The `<top_cell>.et3config` configuration file is used during the back-end compilation and run-time and debug sessions.

The syntax of the `emulatorConfiguration XEL` command is as follows:

```
emulatorConfiguration -add {host <hostname>} [{boards <sub_board>}]
```

There is no restriction on the specification of a `<hostname>`. The `<hostname>` can be any string. If you execute the `emulatorConfiguration -add {host ../myhost}` command, the string `../myhost` is used as a host name.

Note: When compiling a design in the SA mode, you can alternatively use the environment variables—`UXE_HOST`, `UXE_BOARDS`, and `UXE_DYNP` to set the emulator configuration. To specify the emulator to compile for, set the `UXE_HOST` environment variable. To specify the boards and domains for compilation, set the `UXE_BOARDS` environment variable. To specify the DYNP compilation mode, set the `UXE_DYNP` environment variable.

- ☐ If `UXE_BOARDS` is specified, but not `UXE_HOST`, the current host is used.
- ☐ If `UXE_HOST` is specified, but not `UXE_BOARDS`, all boards are used.

- ❑ If neither `UXE_HOST` or `UXE_BOARDS` is specified, a generic single domain configuration is used.

Conserving Board Resources

Ideally, a design uses a full board, which allows the greatest flexibility. The following `emulatorConfiguration` command specifies the use of the entire emulation host:

```
emulatorConfiguration -add {host <hostname>}
```

However, to allow multiple users and/or conserving board resources, smaller designs can be fitted into subsets based on board domains. A board domain is a subset of the emulation modules on a board.

If the design does not require the complete set of host resources, use the following command to specify the subset of the emulation host to be used:

```
emulatorConfiguration -add {host <hostname>} {boards <sub_board>}
```

When the UXE system is first installed, diagnostics are run and all hardware resources are detected and verified. When diagnostics have run successfully, a `<hostname>.et3config` file is created in the `/et3mach` directory of the host workstation. This file is used to specify the UXE resources for compiling and downloading. For example, if the host name is `ninja`, the configuration file will be named `ninja.et3config`.

When compiling, you can specify a configuration file rather than a host name using the following command:

```
emulatorConfiguration -add {file <filename>} {boards <sub_board>}
```

This command is especially useful if the design needs to be compiled for an emulator that has not yet been installed.

On the same logic board, multiple designs can be compiled by using different domains on that board. During run time, compiled designs can be forced to use only specific domains by using a `<design>.bp` file, where the `<design>` is same as the `<top_cell>` name mentioned earlier with the `.et3config` filename. The `*.bp` file can also be used to distribute a design into domains of different boards based on the available space. For details, refer to the [Forcing Relocation Using a .bp File](#) section.

Use the `hwInfo XEL` command to display the detailed configuration information for a specific host emulator machine, configuration file, or design. This information is useful in studying the hardware system or subsystem and its connectivity.

For more information on the syntax and usage of the `emulatorConfiguration` and `hwInfo` commands, refer to the *User Data Commands* chapter of the *UXE Command Reference Manual*.

Compiling for Multiple Configurations (Symmetric Configuration)

If you have several domains (or boards, or emulators) with different unusable resources, you can use a `symmetricConfiguration` command to generate a compile database that can be downloaded to any of the domains, boards, or emulators.

Note: By default, jobs compiled with the `symmetricConfiguration` command run with the `allow bad wires` feature at run time. Jobs compiled without this command will run with *dont allow bad wires* feature as enabled by default. You can always override the defaults at run time using the `setenv QT_ET3_API_ARGS -<option>` command. Here, the *<option>* argument can have the value `-allow_bad_wire` or `-dont_allow_bad_wire`.

To compile for multiple Palladium XP configurations, create a list of hardware configurations using the `symmetricConfiguration` command. You can execute multiple `symmetricConfiguration` commands to add either hosts and boards of a specific emulator, or configuration files to the list of configurations targeted for symmetric compile. For example, the following commands specify a two-domain compile:

```
emulatorConfiguration -add {host emulA}{boards 0.0+0.1}
symmetricConfiguration -add {host emulA}{boards 0.0+0.1+0.2+0.3}
symmetricConfiguration -add {host emulB}{boards 1.2+1.3}
```

The resulting database can be downloaded to an emulator in either of the following ways:

- ❑ domains (0.0+0.1) or (0.2+0.3) of emulator `emulA`
- ❑ domains (1.2+1.3) of emulator `emulB`

Note:

- ❑ When multiple `symmetricConfiguration` commands are executed to add **different** boards for the **same** host, the configuration specified with the last `symmetricConfiguration` command overrides the previous ones.

For example, if following set of `symmetricConfiguration` commands are executed, the database is downloaded to domain 3.3 of emulator `emulA`:

```
symmetricConfiguration -add {host emulA} {boards 1.2}
symmetricConfiguration -add {host emulA} {boards 2.1}
symmetricConfiguration -add {host emulA} {boards 3.3}
```

To download the database to domains 1.2, 2.1, and 3.3 of emulator `emulA`, execute the following `symmetricConfiguration` command:

```
symmetricConfiguration -add {host emulA} {boards 1.2+2.1+3.3}
```

- ❑ When specifying configuration files for symmetric compile, make sure that the configuration files `<top_cell>.et3config` and `<host>.et3config` are not the

same; otherwise, the regular compile might be affected. If the two files are the same, remove the `<top_cell>.et3config` from the design directory.

Rules for Using `symmetricConfiguration` Command

When using `symmetricConfiguration` commands with Palladium XP, observe the following rules:

- All the emulators in `symmetricConfiguration` must be of the same type (that is, Palladium XP XL or GXL).
- The `emulatorConfiguration` command must be specified to identify the base configuration subset for symmetric compile.
- The base configuration subset for symmetric compile must also be included in one of the `symmetricConfiguration` commands.
- The number of domains and emulator type (that is, Palladium XP XL or GXL) specified using the `emulatorConfiguration` command must agree with the number and type specified using the `symmetricConfiguration` command.

Considering these rules, if all domains specified using the `symmetricConfiguration` commands for `emulA` and `emulB` have the same resources available, then in the two-domain compile example given above, the design will be compiled for any two consecutive domains, that is, (0.0+0.1), (0.1+0.2), (0.2+0.3), or (1.2+1.3). When the memory card resources are plugged on just the odd domains in `emulA`, the design will compile for consecutive domains that match the memory card requirements of the corresponding `emulatorConfiguration` domains, that is, (0.0+0.1), (0.2+0.3), or (1.2+1.3).

You can use the `symmetricPreserveCables` and/or `symmetricPreserveMemCards` options of the `compilerOption` command to instruct the compiler to process only subsets that have all the same cables and/or memory card resources available in the corresponding offsets of the base configuration subset. By default, both these options are set to `ON`, which means the subsets that do not match the cables or memory cards of the base configuration subset are not eligible and relocation might not be possible to these. To make **all** subsets eligible for processing, turn these options off by using the following commands:

```
compilerOption -add {symmetricPreserveCables OFF}
compilerOption -add {symmetricPreserveMemCards OFF}
```

However, if there is a mismatch in a required memory card or cable, in order to allow relocation, the mismatched resource is taken offline for the compile. This gives the most relocations, but might limit the compiler and increase the step count.

Relocating Designs into Different Domains

You can compile a design for a certain set of domains, but run it on a different set. Running a design on a different set of domains than it was compiled for is called *relocating* the design.

If your design does not use HDDC cables, the run-time software automatically relocates it to a set of domains that is available. The run-time software attempts to avoid using domains that contain resources (such as memory cards and HDDC cables) not needed by the design. If your design uses HDDC cables, the design will only be relocated if you force relocation using a `.bp` file, or using the xeDebug GUI field that specifies a domain or board to download to.

A Palladium XP GXL board has 16 domains, in four rows as shown below:

```
0  1  2  3
4  5  6  7
8  9 10 11
12 13 14 15
```

An XL board has only one row of four domains.

The following types of relocation are allowed:

1. Movement from one board, to the same set of domains on another board of the same type (XL or GXL). For example, move domains 0, 1, 5 on board 0 to 0, 1, 5 on board 1.
2. Horizontal movement on a board. For example, move domains 0, 1, 5 to domains 1, 2, 6 or 2, 3, 7.
3. Vertical movement on a board. For example, move domains 0, 1, 5 to domains 4, 5, 9 or 8, 9, 13. In horizontal and vertical movement, the entire design must be moved by the same amount, without going beyond the edge of a board.
4. A combination of 1, 2, and 3.

The domains you relocate to must have all the resources (memory cards, target cables, board-to-board cables) that are used by the design. These resources must occupy the same relative locations in the new set of domains that they occupy in the original set of domains.

If the domains to be relocated to have unusable (*minused out*) resources, compile using the `symmetricConfiguration User Data` command and specify the domains you wish to relocate to.

For example:

A design is compiled for domains 0, 1, 2 of a GXL board having memory cards on domains 1, 3, 5, 7, 9, 11, 13, 15.

UXE User Guide

Working with the Palladium XP Emulators

```
x  x  x  3
4  5  6  7
8  9 10 11
12 13 14 15
```

If the design does not use the memory card on domain 1, then you can relocate it to any of the following sets of domains: 1, 2, 3 or 4, 5, 6 or 5, 6, 7 or 8, 9, 10 or 9, 10, 11 or 12, 13, 14 or 13, 14, 15.

If the design uses the memory card on domain 1, then you cannot move it to domains 1, 2, 3 because there is no memory card on domain 2.

You can relocate it to any of the following sets of domains: 4, 5, 6 or 8, 9, 10 or 12, 13, 14.

Forcing Relocation Using a .bp File

When you compile a design, the compiled database includes a logical configuration starting with board 0. The logical configuration must correspond to a subset of the physical configuration of the emulator to which you download. Since this is a logical configuration, the program analyzes it and attempts to make a best fit onto the physical configuration by mapping logical boards to different physical boards, as necessary.

However, this best fit method can be overridden by utilizing a board preference (`.bp`) file, located in the user session directory or the current working directory. This `.bp` file will force the run-time software to place the design on designated physical logic boards specified in the `.bp` file.

When downloading a design, the software checks for the presence of a `.bp` file in the current user session directory (if user session is used) or the current design directory. If one exists, the design is downloaded to the configuration specified in the `.bp` file. Otherwise, it is downloaded to any available configuration.

The `.bp` file can be named as `<design>.bp` or `<host>.<design>.bp`, where `<design>` is the name of the top cell in the design. The `<host>` refers to the emulation host, as set by executing the `host` command in batch mode or by the *Emulator* field in xeDebug GUI. If both `.bp` files are present, the xeDebug software gives priority to the `<host>.<design>.bp` file. A warning message for this situation is printed in the log file.

The entries in the `.bp` file represent physical boards or board domains, where the first number represents the actual board and the second number represents the domain in that board. The file can contain entries for whole boards as illustrated below:

```
0 1
```

Examples of board domains can be:

1.2 1.3

The first entry, 1.2, corresponds to domain number 2 on board number 1. The entry 1.3 corresponds to domain 3 on board number 1.

When `.bp` file is specified, only the first domain or board number in the list are used as the starting location. So, specifying `.bp` file in decreasing order does not work as expected.

The entry in `.bp` file can only specify the starting domain or board number on the physical system that the design occupies. For example, specifying 1.2 for a design that requires two domains will actually map the design onto domains 1.2 and 1.3.

Use a board restriction (`.br`) file instead of a `.bp` file to restrict domains into which a design is downloaded. It can be named as `<host>.<design>.br` or `<design>.br`. Unlike the `.bp` file, which specifies domains into which to download the design, the `.br` file specifies the domains not to be used. When a `.br` file is present, the design will download onto any domains except those specified in the `.br` file. Restrictions can be for a domain, range of domains, or multiple domains. For example, the specified domains could be for domains with the attached target-I/O cables that are needed for testing other designs requiring those resources. By using a `.br` file, you specify that it does not matter where the configmgr auto-locates the download of the design, as long as it is NOT on the domain(s) given in the `.br` file. Thus, when both `.bp` and `.br` files are present, the `.bp` file is ignored and only the `.br` file is considered.

The `.br` file is manually created using a text editor. Its format consists of a list of emulator domains specified on a single line, separated by white space. Entire emulation boards can also be specified, as well as a combination of emulator boards and emulator domains. The following examples show the three possible scenarios:

Example 1: Forcing Relocation

Domains to exclude:

0.3 0.9 1.5 3.7

In example 1, domains 0.3, 0.9, 1.5, and 3.7 will not be used when the design is downloaded to the emulator.

Example 2: Forcing Relocation

Full boards to exclude:

0 3

In example 2, board-0 and board-3 will not be used when the design is downloaded.

Example 3: Forcing Relocation

Mixed scenario of excluded boards and domains:

0.3 0.7 2 3.5

In example 3, domain 0.3, 0.7 and 3.5, and all of board-2 will not be used when the design is downloaded to the emulator.

Specifying the Connection to Target System

The design-preparation step includes specification of the cable connection parameters that need to be used with a target system. These parameters include cable types, terminal assignment, and terminal timing commands.

Specifying the ICE Target Connection in ICE Mode

In the ICE mode, at compile time, you need to provide the following information to connect a target system to DUT (mapped onto the emulator):

- Specify the RTL or netlist for the design.
- Specify the design signals that connect to a target interface.
- Specify the cable connections between the emulator and the target system (using the `cableConnection` userdata command).
- Specify the interface pins used for the target connection (using the `terminalAssign` userdata command).
- Specify the timing constraint for the terminal that is connected to the target system (using the `terminalTiming` userdata command).

In SA mode, the steps that are required to connect the ICE target to the design mapped to the emulator are the same as listed above. However, the signal routed to the target interface cannot be shared with the connection to the host workstation (testbench). The IXCOS compiler generates an error under such condition. This means that the signal that is routed to the target interface cannot be read by the testbench directly. Refer to the section titled Specifying ICE Target Connection in SA Mode on page 69 for details about ICE target specification in the SA mode.

The following sub-sections provide details on the use model for connecting the ICE target system to the design mapped onto the emulator by using different approaches (or methods).

Assigning Target System Terminal

Palladium XP supports an HDDC cable carrying 192 logical signals (input, output, or bi-directional). Additionally, various adapters, such as TIB-192 and LTA, can be included. All I/Os from the top-level design unit (or cell) are available for the target connections.

Use the `terminalAssign` XEL command for associating specific pins with a terminal and cable name. The terminal timing constraints can then be applied using the `terminalTiming` XEL command. The information passed with the `terminalAssign` commands is used to determine the top-level I/O connection for establishing the target connections. Without additional User Data, the internal signals of the design are not available for connection to a target interface.

For TIB connections, the `<pin>` parameter is the combination of the TIB connector name and a valid pin number on the TIB connector. Valid TIB connector names are P2, P3, P4, P5, P6, P7, and P8.

Establishing Target Cable Connection

When the HDDC is first connected and powered on, both LEDs will glow amber. Once the FPGA is successfully programmed, the two LEDs function as their label describes to indicate power status.

To program the cable connections, use the `cableConnection` command that has the following syntax:

```
cableConnection -add {< cable_label > < conn_loc > {I/O_technology}}
```

The `< cable_label >` is composed of a user-specified interface name and interface type in the format `< interface_name > . < interface_type >`. The type corresponds to an HDDC cable type. Thus, an HDDC cable would have the type HDDC, and the `< cable_label >` would be `< mycable > . HDDC`.

The `< conn_loc >` specifies the location of the emulator connector to which the cable connects. The format is `< BCA_id > _ < J_id >`. The `< BCA_id >` is an integer between 0 and 8 corresponding to the BCA index to which the cable connects. The `< J_id >` has the format `Jx` where `x` corresponds to an integer between 1 and 24, corresponding to the J connector index number on the BCA to which the cable connects.

The `< I/O_technology >` is one of the supported Palladium XP technology names. The I/O technology also defines the drive strength of the cable. The default is `LVCMOS33_8`.

Examples

The following example shows how to assign cables using the `cableConnection` command.

UXE User Guide

Working with the Palladium XP Emulators

```
cableConnection -add {< cable_label> <LTA_A | LTA_B>  
                    <board>_< cable_connector> LVCMOS33_16}
```

This adds a cable connection supporting two TIB-88 adaptor connections, one on the A connector of the Legacy Target Adapter (LTA) and one on the B LTA connector.

Assigning Terminal for the TIB-192 Interface

In the `terminalAssign` command, the `<pin>` parameter is the combination of the TIB connector name and a valid pin number on the TIB connector. The TIB-192 connector's P2, P3, and P4, connectors correspond to the MTIB-88's P2, P3, and P4 connectors. The P6, P7, and P8 connectors correspond to the MTIB-88's P2, P3, and P4 connectors. The TIB-192's P5 connector has 16 pins, and this signal group is composed of signals from the other six connectors that are designed to mimic two MTIB-88 interfaces.

For example, a TIB-192 terminal assignment can be:

```
terminalAssign -add {myCable1 P2 17 DATA[0]}
```

If a TIB-192 interface is used, it must be explicitly specified in the `<cable_label>` field of the `cableConnection` command, as shown in the example below:

```
cableConnection -add {myCable3.TIB-192 2_J4, LVCMOS15_series}
```

Providing LTA Support

Use an LTA to adapt an MTIB or MTCC cable to the HDDC cable. In addition, an LTA can be used to turn a HDDC cable into one or two Palladium XP cables of type MTIB-88 and/or TCC (interface type name: DATA_CABLE).

Terminal assignment for LTA support requires the following command:

```
terminalAssign -add {<cable_name> MTIB-88 | DATA_CABLE <pin> TERM | NET  
                    <terminal_name>}
```

For LTAs, the `<cable_label>` in the `cableConnection` command should use the filename extension of `.MTIB-88` or `.DATA_CABLE`, respectively, followed by either `LTA_A` or `LTA_B`. These parameters indicate whether the A connector or B connector of the LTA is attached to the interface. The `LTA_A` or `LTA_B` parameter is required. For example:

```
cableConnection -add {pdCable1.MTIB-88 LTA_A 2_J9 LVCMOS15_SERIES}  
cableConnection -add {pdCable2.data_cable LTA_B 2_J9}
```

The hardware can be an extension cable, an adapter, or a connector.

UXE User Guide

Working with the Palladium XP Emulators

Each of the two LTA connectors can be individually programmed using the `cableConnection` command. The example given below specifies the connections for two different LTAs on the same cable:

```
cableConnection -add { pdCable1.MTIB-88    LTA_A    2_J9    LVCMOS15_series }
cableConnection -add { pdCable2.DATA_CABLE  LTA_B    2_J9 }
```

Note: Even though the two connectors might share the same HDDC, different cable names must be used.

For the complete syntax of the `cableConnection` command, refer to the *User Data Commands* chapter of the *UXE Command Reference Manual*.

Additionally, there are also two sets of 16 pins on either end of the LTA that can be directly connected to a target system using a user-supplied cable. Of these 16 pins, eight are always ground. Collectively, these pins are referred to as the extended cable pins (EXC) and are also programmed using the `cableConnection` command, where they are referred to as EXC LTA_C and EXC LTA_D.

Scripts from the Palladium XP configuration can be used by using Palladium XP notation for cable connections. The following guidelines show how connections are mapped.

- Connectors CB1 J1 and J2 are mapped to LTA_A 2_J1 and LTA_B 2_J1 respectively.
Connectors CB1 J3 and J4 are mapped to LTA_A 2_J2 and LTA_B 2_J2 respectively.
Connectors CB2 J1 and J2 are mapped to LTA_A 2_J3 and LTA_B 2_J3 respectively.

And so on, with the following order of Palladium XP connectors corresponding to increasing CB_i , where $i=1, 2 \dots 8$; and J_k indexes (according to the pattern above):
2_J1, 2_J2, 2_J3, 2_J4, 2_J5, 2_J6, 2_J7, 2_J8, 2_J9, 2_J10, 3_J1, 3_J2, 3_J3, 3_J4, 3_J5, 3_J6

- I/O connector CB16 J1 and J2 are mapped to LTA_A 3_J7 and LTA_B 3_J7 respectively.

I/O connector CB16 J3 and J4 are mapped to LTA_A 3_J8 and LTA_B 3_J8 respectively.

The order of Palladium XP connectors corresponds to increasing CB_i , where $i=16, 17, \dots 20521$ and J_k indexes (according to the pattern above): 3_J7, 3_J8, 3_J9, 3_J10, 5_J1, 5_J2, 5_J3, 5_J4, 5_J5, 5_J6, 5_J7, 5_J8.

Defining EXC Connection

On the LTA, there are 16 pins available as individual target interface pins. These 16 individual pins are the result of 192 pins minus twice 88 pins allocated to the LTA A and B connectors.

They are separated into two groups, with eight pins per group. As a group also corresponds to a connector, the connector location on the LTA is identified by either `LTA_C` or `LTA_D`. The interface type for these pins is called `EXC`.

Use the terminal assignment command as illustrated in the following example:

```
terminalAssign -add {myEXCcable EXC 7 Data[5]}
```

A cable connection statement for this special interface would be defined as illustrated in the following example:

```
cableConnection add {myIndividualPins.EXC LTA_C 2_J9}
```

Multiple Assignments to the Same Terminal

Use the `terminalAssign` command for making multiple assignments to the same terminal. This is useful for handling multiplexed terms. For example, to assign more than one net (`netA`, `netB`, and `netC`) to the same target cable `pin 1` of connector `P2` that has the `interface_type` `TIB-192` and `interface_name` `myCable6`, use:

```
terminalAssign -add {myCable6 P2 1 netA}  
terminalAssign -add {myCable6 P2 1 netB}  
terminalAssign -add {myCable6 P2 1 netC}
```

Note: To make the above command work, it is necessary to include the `multiplexCable -add myCable6` directive.

Synchronizing Output to a Target System Connected by ADC Cables

Groups of primary output signals to a target system connected via ADC cables can be synchronized for more precise timing. This can be useful for signals such as buses, where the same timing constraint can be applied to all signals.

Use the `terminalOutputSync XEL` command to specify one or more groups of primary output signals to be synchronized.

```
terminalOutputSync -add [<terminal> <group-number>]
```

The `<terminal>` signal is the primary output signal to be synchronized with the other primary output signals indicated by `<group-number>`. The `<terminal>` signal must be a valid terminal assignment. Only one statement can be used per command line; therefore, a statement must be issued for each signal for that group number. The actual group numbers are used only to identify the group to which they belong. They do not specify the exact step where the group is to be scheduled; they specify only that the signals having this number be scheduled at the same step. These signals will appear at the target at the same time.

Note: In a group, a signal can be a multi-sampled output only if all other signals of the group are also multi-sampled output.

For example, if the following statements are executed, the signals for `myterm_1` and `myterm_2` will be synchronized as a group, and `myterm_3` and `myterm_4` will be synchronized as another group:

```
terminalOutputSync -add {myterm_1 1}  
terminalOutputSync -add {myterm_2 1}  
terminalOutputSync -add {myterm_3 2}  
terminalOutputSync -add {myterm_4 2}
```

The MakeCTL program will check for syntax correctness, and whether the signals are all multi-sampled or all non-multi-sampled. For each synchronization group, it creates a new section in the `<design>.ctl` file containing the lines of primary output signals to be synchronized. For example:

```
*ADC_OUTPUT_SYNC_GROUP  
<signal_1>  
<signal_2>  
<signal_3>  
...
```

For multi-sampled signals, a separate shadow synchronization group, consisting of all corresponding shadow signals, is also created. The `bitGen` function will create a `<design>.ttime` file, containing the final target TIF steps of all signals.

Every new `ADC_OUTPUT_SYNC_GROUP` will have a different ID number.

Identifying Multisampled Input or Output (I/O)

Multisampled I/O allows you to use multiple asynchronous clock domains to identify portions of a design to implement so that they can be evaluated at a faster rate than other portions of the design.

Some designs have more than one clock domain. Usually, this is handled by operating the fast clock at the least common multiple of the various user clocks, or by establishing a (possibly irregular) clock pattern that accommodates all clock domains. However, in some designs, the combination of maximum path length and clock speed in different clock domains can result in the evaluation of all clock domains with respect to a common fast clock executing more slowly than the slowest clock domain.

By using multiple asynchronous clock domains you can modify clock domains to achieve better performance. These domains are restricted to those that interface to top-level input or output. These clock domains are identified by their top-level connections, and are mapped

into the emulator in a modified form that allows them to be evaluated up to twice as fast as other clock domains.

You must recognize the design characteristics that indicate use of the multi-sampled feature. Usually, this is indicated by a combination of unsatisfactory evaluation speed and the design containing a top-level I/O-connected clock domain that is clocked faster than other clock domains. Keep in mind that the increase in the speed depends on the design.

This feature is intended for expert users. It requires deep knowledge of your design, and how that design is compiled and evaluated on the emulator.

The mechanism that effects a speed-up where this feature is applied is not an increase in the frequency of evaluation. Instead, portions of the design are modified such that up to twice as much evaluation is compressed into one cycle. This modification can take as much as two times the capacity the unmodified circuitry would take.

As these portions of the design are effectively evaluated at up to twice the frequency of the rest of the design, the usual sampling mechanism of once per cycle is more complicated. In the sped-up portion of the design, signals are effectively sampled twice per cycle. However, you specify the sampling offset for normal sampling; the additional samples are derived from the normal sampling offset.

Note: This additional sampling complexity can result in anomalous-appearing waveforms. Exercise great care in setting the sampling offset, and understand the resultant data with respect to the specified offset.

Triggering is accomplished relative to normal (non-spaced-up) portions of the design. Because of the compressed evaluation of the modified portion of the design, triggering conditions derived from signals in that portion might not conform to expectation.

The `compilerOption -add {enableMultiSampledIO 0|1|2}` and `multiSampledTerminal -add|-rm <terminal_name>` XEL commands turn this feature on and off, and specify the top-level I/O connections to which this feature is to be applied. Multisampling can also be performed on inputs or both inputs and outputs. Refer to the *UXE Command Reference Manual* for more information on the command usage.

Handling Terminal Timing Constraints

When emulating with a target system, terminal timing constraints are handled using the `delayBox` and `terminalTiming` XEL commands.

For a top-level I/O terminal which is connected to a target system, terminal-timing constraints control the relative timing of clock and non-clock signals communicated between the emulator and the target, or specifies the behavior of the terminal as a pulse or multi-pulse output.

A delay box notifies the Palladium XP compiler to schedule the specified delay steps (TIFs or ns) on a particular output signal. If a delay box already exists, the command modifies the delay, and will add inputs from the specified inputs list to the existing list of inputs (that is, it will not replace the existing inputs list).

You can enable the use of internal negative delays for improving terminal timings. Use the `compilerOption -add {enhancedTerminalTiming OFF|ON}` command. By default, the use of internal negative delays is disabled. Although most of these enhancements for improving terminal timings are internal, you might experience performance benefits in terms of final step counts even when using your earlier scripts.

For more information on the command syntax and usage, refer to the *User Data Commands* chapter of the *UXE Command Reference Manual*.

Shaping the Pulse

The Palladium XP hardware can generate signals on external connector pins at a frequency higher than that of the emulation fast clock (FCLK). This capability is referred to as pulse shaping. Pulse shaping is useful when the clock of the external target system is controlled by the emulator and has divide-down circuitry as part of its clocking logic.

Any design to be emulated on Palladium XP must have one or more special signals you designate as clock signals. These signals are referred to as “design clocks” or “user’s clocks”; these terms are synonymous.

A cycle of the design clock is carried out by one or more fast clock cycles of the emulator. For some designs, a full cycle of the design clock can be emulated in just one cycle of the fast clock. Other designs require two or more cycles of the fast clock to emulate one cycle of your design clock.

The cycle of a fast clock is subdivided into emulation steps. The emulation steps are controlled in the hardware by the step clock. The step clock has a cycle time of two nanoseconds.

xeCompile determines the number of steps required to emulate one full cycle of the fast clock. For most designs, the compiler can schedule the full cycle of a fast clock in fewer than 256 steps.

The following two options to the `terminalTiming` command generate pulses on external connectors:

- `-pulse`: generates a single pulse per user clock cycle
- `-edgeInfo`: generates multiple pulses during a single fast-clock cycle

For more information, refer to the `terminalTiming` command in the *UXE Command Reference Manual*.

Specifying Delay for the Palladium XP Emulator

Palladium XP emulators have enhanced timing accuracy. To reduce step counts on the Palladium XP emulator use the `compilerOption -ioDelayUnit pd2tif | uatif | step | ns` command. This command allows you to specify whether the delay should be in step, tif, or nanoseconds format.

Specifying ICE Target Connection in SA Mode

In the SA mode, the task of establishing the ICE target connections is handled by the IXCOC compiler, and you need to do the following:

- Compile both the testbench and design files using the IXCOC compiler. Specify all the options to IXCOC compiler, such as `-ua`, `+dut`, and `+dutexcl`, which are required for identifying the design mapping to the emulator.
- Provide information about the ICE target to the design-interface connection.

You need to specify the design unit for ICE target connection by using the `+targetTop` option of the `ixcom` command.

In the ICE mode, the HDL Compiler determines the top-level I/O connections to the target system based on the `cableConnection`, `terminalAssign`, and `terminalTiming` User Data commands. Also, you can connect only the top-level design unit to the ICE target interface.

In the SA mode, on the other hand, you need to specify the `+targetTop+<top_level_designunit>` option of the `ixcom` command in addition to the `cableConnection`, `terminalAssign`, and `terminalTiming` User Data commands. Also, you need to specify the full hierarchical name (including the testbench hierarchy) for the `<terminal_name>` option with the `terminalAssign` and `terminalTiming` User Data commands.

You can specify multiple design units with the `+targetTop+` option. For each design unit that is specified as the target top, the compiler makes all the ports of the module available for target connection. The use of the `+targetTop` option is the same for both multi-step and single-step IXCOC compilation use models.

The interface type for the target connection is determined as follows for each design unit that is specified as the target top:

Design Unit is Empty	Design Unit has at least One Port Connected	Target Interface
No	No	All I/Os are made available for target connection.
No	Yes	Error: all ports of a target top design unit must be unconnected for use by target interface.
Yes	No	Error: target top design unit specified has is empty (or has no logic).
Yes	Yes	Design unit is made available for target connection as component adapter (caCell).

Example: ICE Target Connection in SA Mode

The following example illustrates the specification of ICE target interface connection as described in later sections.

```
//-----
// File : tb.sv
// Description:
//           Testbench module for IXCOM/ICE target connection unit testing
//-----
module Tb;
    // All ports must be unconnected for target top design units
    Clk cg(.clk());
    Dut dut(, , );
    // ...
endmodule // Tb
//-----
// File : dut.sv
// Description:
//           DUT design units for IXCOM/ICE target connection unit testing
//-----
module Dut(clk, din, dout);
    input clk;
    input [1:0] din;
    output [1:0] dout;
```

UXE User Guide

Working with the Palladium XP Emulators

```
Flop flop(clk, din, dout);
Cell cell(clk,, );
endmodule // Dut
module Clk(output reg clk);
    initial clk = 0;
    always
        #1 clk = ~clk;
endmodule // Clk
module Flop(clk, din, dout);
    input clk;
    input [1:0] din;
    output reg [1:0] dout;
    // non-empty module with logic
    always @(posedge clk)
        dout = din;
endmodule // Flop
module Cell(clk, din, dout);
    input clk;
    input [1:0] din;
    output reg [1:0] dout;
    // cell module must be empty (no logic)
endmodule // Cell
//-----
```

Connecting Design Unit I/Os to the ICE Target in SA Mode

This section illustrates the various ways or commands for specifying the connection of the top-level I/Os to the ICE target when using the IXCOM compiler.

While specifying the design units, you can specify a single `+targetTop` option with multiple values or multiple `+targetTop` options with one design unit. The use of the `+targetTop` option works in the same manner for both single-step and multi-step IXCOM compilation modes.

■ Connecting Top-level Design Unit I/Os to the ICE Target

- ❑ To connect the top-level design unit, use the following command:

```
ixcom -ua +dut+Dut +targetTop+Dut dut.sv
```

- ❑ To connect multiple top-level design units, use the following command:

```
ixcom -ua +dut+Dut +targetTop+Dut+Clk dut.sv
```

■ Connecting Instantiated (Non-top-level) Design Unit I/Os to the ICE Target

- ❑ To connect non-top-level design units, use the following command:

```
ixcom -ua +dut+Dut+Clk +targetTop+Dut tb.sv dut.sv
```

Note: In the above compilation, the testbench hierarchy is included with the DUT instantiated inside the testbench.

- ❑ To connect non-top-level design units from different hierarchies, use the following command:

```
ixcom -ua +dut+Dut+Clk +targetTop+Clk +targetTop+Flop tb.sv dut.sv
```

Note: In the above compilation, the testbench hierarchy is included with the DUT instantiated inside the testbench.

Note: In addition, refer to the points listed in the section [Points of Consideration for Specifying ICE Target Connections in SA Mode](#) on page 78.

For both top-level I/O and instantiated-design-unit connections to the target interface, the design unit specified as the target top must meet the following requirements:

- All non-top design units must have all ports unconnected.
- Design units must be non-empty (must have logic in it).

To connect the top-level design unit to the target interface, use the following steps:

1. Specify the target interface connection details in the `compilerOptions.gel` file (in the current design directory) with commands like the following:

```
# ...
cableConnection -add { <cable_label> <conn_loc> { <I/O_technology> } }
terminalAssign -add { <interface_name> [<interface_type>] <pin> Dut.clk } \
                { <interface_name> [<interface_type>] <pin> Dut.din[1] } \
                { <interface_name> [<interface_type>] <pin> Dut.din[0] } \
                { <interface_name> [<interface_type>] <pin> Dut.dout[1] } \
                { <interface_name> [<interface_type>] <pin> Dut.dout[0] }
terminalTiming -add { Dut.clk <constraint> }
# ...
```

Note: The `compilerOptions.gel` file can include other commands in the same order as required in the ICE mode.

2. Compile the design files using the `ixcom` command:

```
ixcom +targetTop+Dut dut.sv -ua +dut+Dut
```

Using Component Adapter (CACELL) for ICE Target Connection in SA Mode

In addition to connecting design units to the ICE target interface, you can mark some cell(s) as component adapter(s) for the target connection. Using the specified information, the precompiler ensures that the nets connected to these cell(s) are available for target interface connection. All internal signals of the design that need to be routed to the target connection are connected as actual to the instance of the component adapter.

In the ICE mode, the component adapters are marked by using the `caCell` IP command. In the SA mode, you use the `+targetTop` option of the `ixcom` command to specify the component adapter(s) for the target connection, in place of the `caCell` command that is used in the ICE mode.

In the SA mode, you need to create an empty (shell) module for the component adapter (referred in the `caCell` command) module. The IXCOC compiler compiles the design files along with the empty module as a part of the design hierarchy. The hierarchy under the design is partitioned to the hardware or software based on the content of the module. The external connections to the simulator for hardware or software interface signals are automatically created by the IXCOC compiler.

For all component adapter ICE target connections, the design unit, which is specified as the target top must meet the following requirements:

- The design unit must have at least one port connected.
- The design unit must be empty (shell module).
- The port direction (reversed) requirements remain the same as the ICE use model.

This section specifies the various ways or `ixcom` commands that you can use to connect a design unit to ICE target as a component adapter (`caCell`). The following commands are based on the [Example: ICE Target Connection in SA Mode](#).

To connect a top-level DUT module as a component adapter in the SA mode, use the following command:

```
ixcom -ua +dut+Dut +targetTop+Dut dut.sv
```

Note: In the above command, the `Dut` content is assumed to be empty like `Cell` module in the previous example ([Example: ICE Target Connection in SA Mode](#)).

To connect a non-top level DUT module as a component adapter in the SA mode, use the following command:

```
ixcom -ua +dut+Dut +targetTop+Cell tb.sv dut.sv
```

Note: For multi-step IXCOC compilation, the `+targetTop` option is used in the same manner as specified above.

To connect the design unit (or cell) identified as component adapter to the target interface, use the following compilation steps:

1. Specify the target interface connection details in the `compilerOptions.gel` file with details such as:

```
# ...
cableConnection -add { <cable_label> <conn_loc> { <I/O_technology> } }
terminalAssign -add \
    { <interface_name> [<interface_type>] <pin> Tb.dut.cell.clk[1] } \
    { <interface_name> [<interface_type>] <pin> Tb.dut.cell.din[0] } \
    { <interface_name> [<interface_type>] <pin> Tb.dut.cell.dout[1] } \
    { <interface_name> [<interface_type>] <pin> Tb.dut.cell.dout[0] }
terminalTiming -add { Tb.dut.cell.din[1] <constraint> }
# ...
```

2. Compile the design files using the `ixcom` command as following:

```
ixcom tb.sv dut.sv -ua +dut+Top +targetTop+Cell
```

When using the component adapter connection to target system, in ICE mode, the `<terminal_name>` option in the `terminalAssignment` and `terminalTiming` User Data commands include hierarchical path starting from the first component instance within the top-level cell. However, in the SA mode, you need to specify the full hierarchical name (including testbench hierarchy) for the `<terminal_name>` option referred in the `terminalAssign` and `terminalTiming` User Data commands.

Note: In addition, refer to the points listed in the section [Points of Consideration for Specifying ICE Target Connections in SA Mode](#) on page 78.

Connecting an Arbitrary Internal Signal to the ICE Target Interface

In the ICE mode, you can connect an arbitrary internal signal in the DUT to a target interface by using the `globalNet` XEL command or the `iScopedNets` program. For non-top design units, you must include the full hierarchical path to the signal.

The description of the `globalNet` command is available in the *User Data Commands* chapter of the *UXE Command Reference Manual*. Refer to the [Connecting Nets Using iScopedNets Program](#) section for a description of the `iScopedNets` command.

This section provides details on these methods.

Example of Connecting Using Arbitrary Internal Signals to ICE Target Interface

The following basic HDL example is used to explain the connection mechanism:

UXE User Guide

Working with the Palladium XP Emulators

```
// File: Dut.sv
module Dut(input clk, din/*, output doutT */)
  input clk;
  input din;
  // top-level dout terminal is created using various methods listed below

  Sub sub(clk, din, dout);
endmodule // Dut

// File Sub.sv
module Sub(input clk, din, output dout);
  always @(posedge clk)
    dout = din;
endmodule // Sub
```

All use models described in this section to connect an internal signal using *global net* require an explicit precompile step in the order listed below:

1. Make internal signal(s) available as global net.
2. Assign the top-level or component adapter terminal to a target system using the `terminalAssign User Data` command.
3. Invoke the precompile step to make *global net* available for target connection.
4. Assign the global net terminal to target system using the `terminalAssign User Data` command.
5. Specify the terminal timing details.

These steps are explained below for three different ways to create a *global net* to connect to target interface.

Connecting the Arbitrary Signal as Global Net to the Target Interface

In this method, an internal signal is connected to the top-level terminal as global net. The User Data specification syntax is:

```
globalNet -add { <net> <term> }
```

In the above User Data command, a global net is created for the hierarchical net `<net>` as top-level terminal `<term>`. The `<term>` must be unique, and must not be an existing top-level terminal. To connect top-level design unit and global net signals to the target interface, use the following compilation steps:

1. Specify the User Data commands to create global net and connect signals to target interface in the `compilerOptions.gel` file.

UXE User Guide

Working with the Palladium XP Emulators

```
# ...
# netlist import
# designImport
# design hdlldb xcva_top
# Create global net for Dut.sub.dout
# NOTE: Scope for this global net is set to top-level cell specified by design
command earlier
globalNet -add { Dut.sub.dout doutT }

# set up design for precompile
# compilerOption -add ...
# userData -load ...
cableConnection -add { <cable_label> <conn_loc> { I/O_technology } }

# Terminal assigns for top-level I/O (see earlier sections)

precompile

# set up design for compile

# Terminal assigns for new top-level terminals created using globalNet
# NOTE: Terminal name referred here belongs to top-level cell
terminalAssign -add { <interface_type> [<interface_name>] <pin> TERM doutT }

# Terminal timing (see earlier sections)
# ...
```

2. Compile the design files using the `ixcom` command as following:

```
ixcom dut.sv sub.sv -ua +dut+Dut +targetTop+Dut
```

Note:

- ☐ An explicit precompile step is required prior to setting up design for compile.
- ☐ In addition, refer to the points listed in the section Points of Consideration for Specifying ICE Target Connections in SA Mode on page 78.

Connecting User Global Net Using Pinprop Attribute

In this method, an internal signal is connected to top-level terminal as user global net using the specification in the Attribute File Format. User global net information is provided in the attribute file as follows:

1. Library cell to which the internal terminal signal or net belongs

2. Internal terminal signal name

3. Top-level terminal name

For the sample HDL example, the following attribute specification is used to create the global net:

```
<code>
LIBCELL Sub 0 0 0
PINPROP dout userGlobalNet doutT
```

To connect top-level design unit to the target interface, use the following compilation steps:

1. Specify global net using attribute specification in the `externalNetlists.qel` file.

```
# ...
# import attribute specification
netlistFile { attribute <attr_file> }
```

2. Specify the target connection details for the top-level signals and global net created.

```
# designImport
# design hdl db xcva_top

# set up design for precompile
# compilerOption -add ...
# userData -load ...
# cableConnection -add ...

# Terminal assigns for top-level I/O (see earlier sections)

precompile

# set up design for compile

# Terminal assigns for new top-level terminals created using userGlobalNet
# NOTE: Terminal name referred has implied top-level hierarchy
terminalAssign -add { <interface_type> <interface_name> <pin> TERM doutT }

# Terminal timing (see earlier sections)
# ...
```

3. Compile the design files using IXCOM tools as following:

```
ixcom dut.sv sub.sv -ua +dut+Dut +targetTop+Dut
```

Note:

- ❑ An explicit precompile step is required prior to setting up design for compile.
- ❑ The attribute file is imported just like any other netlist file in ICE mode. For details about this process, refer to the section titled Importing a Netlist on page 164.
- ❑ In addition, refer to the points listed in the section Points of Consideration for Specifying ICE Target Connections in SA Mode on page 78.

Points of Consideration for Specifying ICE Target Connections in SA Mode

Keep in mind the following points when specifying the ICE target connection in SA mode using design units (Connecting Design Unit I/Os to the ICE Target in SA Mode on page 71), component adapters (Using Component Adapter (CACELL) for ICE Target Connection in SA Mode on page 73), and/or arbitrary signals (Connecting an Arbitrary Internal Signal to the ICE Target Interface on page 74):

- ❑ Ensure that the syntax and semantics for the target-connection-specific commands included in the `compilerOptions.qel` file is the same as the ICE mode. You can include other commands in addition to the target-interface connection commands in this file, provided the order of these commands matches the ICE mode. The `compilerOptions.qel` file must be present in the IXCOM compilation directory.
- ❑ You can specify the testbench hierarchy when using the `+targetTop` option of the `ixcom` command.

Duplicating Wires for Transmitting a Signal between Chips or Boards

You can instruct the compiler to schedule two transmissions through different wires using the following command:

```
compilerOption -add {wireDuplication <value>}
```

For syntax description, refer to the `compilerOption` command in the *User Data Commands* chapter of the *UXE Command Reference Manual*.

Note: By default, jobs compiled with the `wireDuplication` option as enabled, run with the `allow bad wires` feature at run time. Jobs compiled without this option will run with *dont allow bad wires* feature as enabled by default. You can always override the defaults at run time using the `setenv QT_ET3_API_ARGS -<option>` command. Here, the *<option>* argument can have the value `-allow_bad_wire` or `-dont_allow_bad_wire`.

This compile-time setting enables you to:

- Avoid a need to recompile a design when a new wire fault is found after the compilation

- Allow a design to run on an emulator with a set of defective wires different from the one the design was compiled with
- Increase the chance of the existing compiled designs with large configurations to run even when new wire faults occur

When the `wireDuplication` compiler option is set, for a signal that requires a wire transmission between chips or boards, the compiler schedules two transmissions through different wires, instead of one. At the destination chip, the compiler takes the signal from one of the two used wires. If at run time the wire is found to be defective, the run-time software modifies the load module to switch the signal source to the input pin of the chip connected to the alternative wire.

All the duplicated wires are divided by pairs, this means that each wire has its assigned pair wire. When wire duplication is turned on, whenever we transmit a signal through wire W , the transmission is duplicated through the pair wire W' . The design will fail to run only if both wires in a pair fail.

The `<value>` argument of the `wireDuplication` compiler option allows you to decide the type of wires you want to duplicate, such as:

- inter-board cable wires
- board wires between modules (domains)
- wires within a module (domain) between chips
- any combination of the above; for instance, duplicate all wires, or cable and board wires only.

Handling of Known Defects

If there is a wire W known to be defected at the compilation time and its pair W' is also defective, the compiler will not use the entire pair. However, if W is defective and W' is not, there are the following two choices:

- Declare the entire pair defective and do not use it, or
- Ignore the defect and still use the pair

The first option provides the utmost reliability but might be too detrimental to the capacity and emulation speed. The situation gets worse if you compile the design targeting multiple sets of domains, boards, and/or emulators. Then, the list of defected wires has to be the disjunction of all the defect lists; if a wire is defected on any of the possible emulator configurations, the entire pair has to be taken out.

The second option on the other hand makes the defect lists significantly shorter without a tangible increase in the download failure probability. If wire w is known to be defected, it is still possible to run the design unless a post-compilation wire failure happens exactly to the pair wire w' . With tens of thousand wires per board and with the realistic failure rate, the probability of such event is comfortably low, even when compiling for several alternative sets of emulation hardware.

For information about handling the know defects at run time, refer to the [Handling Failures at Run Time with Wire Duplication](#) section.

Finding Faults Using ModelChecker

The ModelChecker is a static model checker that compares the netlist properties and attributes with the actual instructions running on the emulation processors while adhering to the current emulator configuration. The ModelChecker program finds faults when the expected functions of a logic block found in the netlist do not match the actual computed functions found in the processor's instructions.

The ModelChecker program cannot prove that the model is correct, but it can ensure that a good set of tests are covered. A problem discovered using the ModelChecker program contains meaningful signal information to cross-reference it to any problem found during run time.

Note: If a design was compiled with the `wireDuplication` compiler option as enabled, the ModelChecker program performs an additional pass to check the integrity of the compiled model after wire switching. For details about wire duplication, refer to the [Duplicating Wires for Transmitting a Signal between Chips or Boards](#) section.

Invoking the ModelChecker

The ModelChecker program can be invoked in two different ways – either as a standalone executable that can be invoked on the final design database, or automatically from the compiler. Both methods have the same semantics (arguments).

To invoke ModelChecker as a standalone executable, type the following syntax of the program on the UNIX prompt:

```
ModelChecker <design_name> [options]
```

Here, the `<design_name>` argument refers to the top cell name that is used for naming the .proto file.

The `[options]` argument can have values `-write_pgm`, `-write_fnets`, or `-omit [<check_type>]`.

Alternatively, you can invoke the ModelChecker program at compile time using the following equivalent set of commands

```
compilerOption -add {ModelChecker OFF | ON}
compilerOption -add {write_pgm OFF | ON}
compilerOption -add {write_fnets OFF | ON}
compilerOption -add {ModelCheckerOmit "<check_type> <check_type> ..."}
```

For command description and syntactical details, refer to the *UXE Command Reference Manual, Programs and Utilities* chapter (for ModelChecker) and *User Data Commands* chapter (for compilerOption).

For detailed information about *check_types*, refer to the Types of Checks Performed by ModelChecker section.

Important

ModelChecker program invoked at compile time using the compilerOption commands can double the running time for smaller designs. Therefore, invoking ModelChecker as a standalone application is recommended to avoid increase of compilation time.

Cadence recommends to always invoke ModelChecker on a design that might have issues during run time and save the debug time. As a best case, ModelChecker can be invoked always after compilation as a background task to know the results.

Types of Checks Performed by ModelChecker

This section outlines the following basic ModelChecker check types that are performed:

- LOGIC
- LATCH
- XBIO
- DBIO
- MEM
- CONNECTIVITY

Most of the check types are based on the BOXes found in the final *<design_name>.proto* file. By default, all check types are executed. However, you can turn off the check types that are not required by using either of the following commands:

- `ModelChecker <design_name> -omit [<check_type>]`
- `compilerOption -add {ModelCheckerOmit "<check_type>
<check_type> ..."} }`

For all check types, when ModelChecker finds a problem, a message of the following format is displayed to stderr and recorded in the `./tmp/ModelChecker.log` file:

```
WARNING (ModelChecker): <check_type> failed for BOX <box number>: <User  
Information>, INTERNAL: <Engineering Information>
```

Here, *<User Information>* includes a net name found in the post-synthesized netlist, such as signal name or memory instance name (depending on the check type).

Most checks compare the expected operands and logic functions to the actual operands and logic functions. The expected operands and logic functions are obtained by traversing the BOX netlist (agnostic of the actual emulation hardware or instructions). The actual operands and logic functions are obtained by traversing the emulation processor instructions.

LOGIC

It compares the actual and expected operands and logic functions. Then, makes sure that the operands are properly sequenced to be computed in the same cycle (not the previous cycle).

LATCH

It is similar to the LOGIC checks. However, the *d*-pin data is scheduled in the previous major cycle.

Note: When the LATCH check type is specified, all types of latches including regular latches, combo-cycle (CC) updating latches, and scattered probes chain latches are checked.

XBIO

It performs the checks on XBIB and XBOB box types.

The XBIO check type refers to the primary inputs and outputs of the design that drive or load target interfaces.

The expected operand of the XBOB box is the location of the data input. When traversing the instructions to compute the expected operands, make sure that the operand is computed in the current cycle.

The XBIB box checks ensure that primary inputs are properly stabilized and routed to their corresponding destination (load).

DBIO

It performs the checks on DBIB and DBOB box types for DAS (SA_CARD) IO. These checks are identical to XBIO checks.

MEM

It performs the checks on XMPR and XMPW box types.

XMPW checks make sure that all address and data operands arrive at the same time as the memory is enabled in the same major cycle. Proper slicing and constant propagation (for smaller memories) is also checked on the EDRAM and XDRAM interfaces.

XMPR checks are identical to XMPW checks with respect to setting up the address bus. However, a latency must be considered on the data read bus.

Both XMPW and XMPR boxes cover all RAM type checks that include EDRAM, XDRAM (UMC1), UMC2, UMC3, and UMC4. For EDRAM and UMC1 checks, assembly mode is also checked. For all types of RAM checks, proper spacing is also checked to make sure that operations to the same bank are not violating pending or previous requests.

CONNECTIVITY

This type of checks examine the basic overall configuration while looking for certain processor-to-processor connection violations. Some of these checks include delays and pipeline spacing of primary I/O between processors that cross chip-to-chip boundaries as well as board-to-board boundaries, through basic connectors or MBS (multi-board switches).

All connectivity checks do not necessarily start with the BOX types. However, majority of the BOX types can be covered during normal expected operand traversal of LOGIC and FLOP boxes. The remainder of the checks might not be related to BOXes and design integrity, and can be considered as *softer* checks.

Output Logs of ModelChecker

ModelChecker warnings are displayed to the standard error as well as logged in the `./tmp/ModelChecker.log` file. The warnings for the `wireDuplication` pass are logged in the `./tmp/ModelChecker_wireDup.log` file.

UXE User Guide

Working with the Palladium XP Emulators

Note: These log files are maintained separately because each pass through the ModelChecker program operates on a completely different compiled `<design_name>.proto` file. Separate log files help to find the results quickly and prevent overwriting of information when the wire duplication is disabled.

When the option to write `fnets` is specified, the `et3fnets` names are displayed in the standard output console and written to the `./tmp/et3compile.msg` file.

When the option to write `pgm` is specified, the `et5wrpgm` processor program is saved to the `./dbFiles/<design_name>.et5pgm` file.

Also, the `et3fnet` and `et5wrpgm` header information is displayed in the standard output console. This header information contains the version and launch notification of the applications.

When the `compilerOption userdata` command for invoking the ModelChecker program is given during an iterative compile process, such as while finding the best step count, the program is invoked only once on the final compiled database that is generated on completion of the compile process.

Working with Emulators at Run Time

You can work with the emulator hosts at run time in the following ways:

- [Specifying the Emulation Host](#)
- [Identifying Best Location for Downloading Design](#)
- [Reserving Emulation Hosts](#)
- [Reducing Emulator Initialization Time Using KeepHostAlive](#)
- [Handling Failures at Run Time with Wire Duplication](#)
- [Using Suspend and Resume to Manage Emulator Resources](#)

Specifying the Emulation Host

The emulation host can be specified at the following time points:

- During Design Compilation for ICE flow

`emulatorConfiguration` command in the compilation script

- During Run

By specifying the implicit or explicit `host .` command (‘.’ selects current host) or explicit `host <host_name>` command

During run time, use either the xeDebug GUI interface or the `host XEL` command to specify the emulation host to which the emulator is connected. For more information on the syntax and usage of the `host` command, refer to the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

Important

Emulation host is different from SA host. The emulation host is the workstation on which the DBEngine process is running. In SA mode, xeDebug must run on the SA host, while the ICE mode does not have this requirement. Note that, the SA host and emulation host can be two different workstations or the same workstation, depending on how the Palladium XP emulator was set up.

Specifying the Design Directory

By default, the `host` command starts the DBEngine in the same directory in which `xeDebug` is running. However, the directory name is translated to a portable format, like `/net/<machine_name>/...`

If the automatic path translation generates any performance issues or access problems, you can use the following `host` command to specify the design directory to be used:

```
host <hostname> [-path <directory_name>]
```

`<directory_name>` must be the path to the debug directory as seen from the workstation `<hostname>`. The directory path must use an absolute path. In most cases, you can use the following format:

```
host <hostname> -path [pwd]
```

The above command will suppress the automatic path translation without the need to use a hard-coded directory name.

Identifying Best Location for Downloading Design

You can use the `-location` option with the `test_server` command as following to find the best location for downloading your design:

```
test_server <top_cell_name> -location
```

Note: This command can be executed only if you have a `<top_cell_name>.proto` file in the `dbFiles` directory.

The command returns a list of domains on which your design can be downloaded. The domain at the top of the list is considered to be the location where the job will be placed by the `configmgr` if locking was enabled.

When a job is submitted for download, the `configmgr` matches the logical resource requirements of the design to the existing, available, hardware resources. It is likely that there can be more than one set of boards and domains where the resources for the job are available. The `configmgr` decides which of these possible locations to assign to the job. The `configmgr` does this by assigning a weighted score to each of the possible job locations, indicating how well the location 'fits' the job resource requirements. The better the score, the better the fit. The `configmgr` places the job on the location with the best fit.

The algorithm used by the `configmgr` to determine the score is based on the following encourage or discourage methodology:

- If a candidate job location includes resources, such as B2B cables, that are not part of the design, the `configmgr` adjusts the score to discourage placement of this job there.

The reason is that if possible, the B2B cable should be left free for some other job that requires it.

- If a candidate job location includes resources, such as -XBO or -XBI, that are not a part of the design, the configmgr adjusts the score to encourage placement of this job there. The reason is that another job that uses the -XBO or -XBI could not run at that location, so the current job should try to use it.

If you use a .bp file, usually, the job placement is limited to a single location. However, there are exceptions. For example, if a .bp file does not contain entries for every board of a multiple board job, the configmgr is free to locate the boards not specified according to best fit.

The configmgr checks whether the wire duplication or symmetric configuration feature was used at compile time. If neither of these compile options were used, then at run time, configmgr will not allow a job to re-locate to a subset with a bad wire that was used by the compile based on the emulatorConfiguration subset.

Reserving Emulation Hosts

Large designs can take a considerable amount of time to download on the emulator, and during this time, it is possible that the hardware resources might be taken by another user's design. To avoid this situation and guarantee the complete design to be downloaded, it is a good idea to reserve the emulator resources for some time before beginning the design download. The emulator resources can be reserved by using the test_server program with the compiled design or with user-generated <top_cell_name>.bp file.

Before reserving the emulator resources, you can execute the following options of the test_server program to identify the best hardware configuration for your design:

- `test_server <top_cell_name> -location`

Helps to identify the domain that is most suitable for downloading the specified design. For details, refer to the [Identifying Best Location for Downloading Design](#) section.

- `test_server -sainfo`

Helps to get detailed information about the Orion hosts connected to the emulator.

For more information about the above listed options, refer to the description of test_server program in the *Programs and Utilities* chapter of the *UXE Command Reference Manual*.

With respect to reserving hardware resources, the test_server program provides the options for:

- [Reserving the Emulation Resources](#) (if required, for a specific duration of time)

- Using the Reserved Resources for Downloading the Design
- Releasing the Reserved Resources

Reserving the Emulation Resources

Following is the syntax of the `test_server` program to reserve emulation resources:

```
test_server <top_cell_name> \  
    -reserve [-key <key_value>] \  
    [-timeout <t> [<unit>] | -holdtime <t> [<unit>]] \  
    [-retry [<num>]] \  
    [-bp | -proto]
```

For details on the command syntax for running the `test_server` program, refer to the *Programs and Utilities* chapter of the *UXE Commands Reference Manual*.

Note:

- ❑ In the ICE mode, the emulator resources required by the design are automatically reserved for maximum 10 minutes (for one time use only) on the execution of the `host <hostname>` command. In the SA mode, similar type of automatic emulator resource reservation is enabled when you specify the `host <hostname> -reserveFirst` command. For more information about the `host` command and its syntax, refer to the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.
- ❑ In ICE mode, to turn off the automatic emulator resource reservation done before design download, set the `DISABLE_RESERVE_FIRST` environment variable to 1 before invoking the `xeDebug` tool.

Reserve the emulator resources for a specific time period in either of the following modes:

- **Single-session mode** (`-timeout <t> <unit>` option) – In this mode, the emulator resources are reserved for the specified time limit `<t>` to enable a single download of the design. After the design has been downloaded, the reserved resources are immediately released, even if the time limit has not expired.
- **Multi-session mode** (`-holdtime <t> <unit>` option) – In this mode, the emulator resources are reserved to enable multiple downloads of the same or different designs until the time limit `<t>` expires. The reserved resources are released only after the time limit has expired.

When specifying the mode as single-session or multi-session, define the time limit `<t>` in terms of either seconds, minutes, or hours time `<unit>`. If the time limit is not specified, by default, the resources are reserved in the single-session mode for five minutes. Reservation time limit cannot exceed 24 hours (that is, 86400 seconds or 1440 minutes).

UXE User Guide

Working with the Palladium XP Emulators

The resource reservation takes place only after the `test_server` program has confirmed that the resources are available. Otherwise, an error message displays that the resources cannot be reserved. In such situations, the `-retry` option can be used to check the availability of the resources at regular intervals until resource reservation succeeds or the process is manually aborted.

If more than one set of resources are available for reservation, by default, only one is reserved. For example, a user has a one-domain design and a Palladium XP system with four domains available. When the following resource reservation command is executed, one best fit domain out of the available four domains will be reserved:

```
test_server <top_cell_name> -reserve
```

When resource reservation is successful, the `test_server` program displays the following confirmation:

```
Emulator domains reserved with key 999303234 for 300 seconds.
Reserved resources (boards/domains) are: 1.10
Exit code: 0
```

To determine that the resources are reserved, specify the `test_server` command without any arguments. The information about the reserved domains is displayed as shown in the following sample `test_server` output when two domains are reserved:

```
Configmgr Host: neon36          Hardware: Palladium XP GXL    System Status: ONLINE
Board 0 has 5 modules          Board: ONLINE              NCC: ONLINE
HSMA Status: Operational       NCC Version: RELEASE 5.9.3.8
SA Hosts:                      neon36(1), neon44(3)

Domain  Owner      PID      BCA      J-con    MemCard      Design  ElapTime
  0      user1      neon36:19800  --      --      0 1 2 3      top    00:34:23
  1      user2      0         --      --      0 1 2 3      RESERVED* 00:03:57*
  2      user3      0         --      --      0 1 2 3      RESERVED* 00:03:57*
  3      NONE      0         --      --      0 1 2 3      NONE     --
  4      NONE      0         --      --      0 1 2 3      NONE     --

Exit code: 0
```

In the `test_server` output, the *Owner* column displays the `<username>` adjacent to the reserved domains, the *Design* column displays `RESERVED*`, and the *ElapTime* column shows the reserved time left in `HH:MM:SS*` format. The `*` indicates the reservation expiration time left within which the design must be downloaded.

After design download, the `test_server` output displays the reserved domain resource as a regular domain with the actual `<design_name>` displayed under the *Design* column (note the design name `top` for domain 0 in the sample output given above), and the *ElapTime* column displaying the elapsed time since the design is downloaded.

Mapping Reservation to a Key

Each reservation of the emulator resources is mapped to a specific key value. If required, you can specify a value of your choice for the key by using the `-key <key_value>` option. The key value is unique for each user, and is stored in ASCII format in the `ts.<username>.resv.key` file in the `./dbFiles` directory. But the same user can make multiple reservations for different or same design. The key file is not protected, and can be corrupted. Therefore, Cadence recommends that multiple reservations for the same design at the same time should not be made from different terminals.

Later, the xeDebug tool reads the key value from the file in automatic key-use mode. This means that the key value need not be specified manually using the `XE_RESERVE_KEY` environment variable or the `xeset reserveKey` run-time parameter. Refer to `xeset` command description in the *Run-Time Commands* chapter of the *UXE Command Reference Manual* for syntax usage details.

Cadence recommends that both `test_server` program and xeDebug tool are run from the same design directory. This enables xeDebug to automatically locate the key file during design download. If the `test_server` program and xeDebug tool cannot be run from the same design directory, before initiating design download, the key must be manually specified to xeDebug. Refer to the [Using the Reserved Resources for Downloading the Design](#) section.

Using a .bp File for Resource Reservation

At run time, when a compiled design is being downloaded, the `<top_cell_name>.bp` file enables distribution of the design into boards or domains specified in the `.bp` file. To use only the `.bp` file for resource reservation without the compiled design database information, specify the `-bp` option with the `test_server <top_cell_name> -reserve` command.

When using the `-bp` option, ensure that the `<top_cell_name>.bp` file contains the complete boards or domains information. For example, if a design requires four domains to download, the `.bp` file should have the information about all the four domains.

When the `-bp` option is specified, the `test_server` program need not be run from the compiled design directory, which is the requirement for the default `test_server <top_cell_name> -reserve` command. However, ensure that the `<top_cell_name>.bp` file exists in the current directory from where the `test_server` program is being run; else the reservation of emulator resources fails.

When the `-bp` option is not specified, but a `<top_cell_name>.bp` file exists in the current compiled design directory, the `test_server` program will still use it. Without the `-bp` option, the starting domain or board specified in the `.bp` file together with the emulator resource requirement extracted from the compiled database will be used to reserve the resource. This

means that if the `.bp` file specifies more than one domain or board, only the starting domain or board is used, and the remaining items in the `.bp` file are ignored.

Note: The resource reservation process does not honor the `.br` file that contains the domains on which design download is restricted.

Using the Reserved Resources for Downloading the Design

Before starting to download the design, to override the key value recorded in the `./dbFiles/ts.<username>.resv.key` file, or to manually specify the key value if it was not saved to the key file by `test_server` earlier, use either of the following methods:

- Specify the following run-time parameter after invoking `xeDebug`:

```
xeset reserveKey <key_value>
```

- Set the following environment variable before invoking `xeDebug`:

```
setenv XE_RESERVE_KEY <key_value>
```

Specifying the key value manually is useful when the `test_server` program and the `xeDebug` tool are being run from different work directories. For example, when the `test_server` program needs to be run outside of the design directory.

When key value is not specified manually, it is assumed that the `test_server` program was run from the design directory. In this case, `xeDebug` automatically reads the key value from the key file, provided it exists and has at least one key value entry.

If the key value is not specified manually, and the key file either contains no values or does not exist, the design download proceeds as if no resources were reserved.

To use the reserved resources, perform the following actions before the specified time limit expires, or else the reserved resources will be released for other users:

- Download the design by using the:
 - ☐ *Download* button in the `xeDebug` GUI
 - ☐ `download` command
- Swap the design into hardware by using the `xc on -<hot_swap_mode>` command if design is running in SA mode.

If a user reserves two domains using the same key, and then uses that key to download two single domain designs, the two single domain designs will be downloaded to the two reserved domains, respectively.

If a user reserves two domains using different keys, and then uses one of the keys to download a design requiring both domains, the download will fail.

Releasing the Reserved Resources

The reserved emulator resources are automatically released when either of the following situations occur:

- The time limit for resource reservation expires
- The design download process finishes in the single-session mode

However, to manually release the reserved emulator resources, execute the following command:

```
test_server -rmkey [-key <key_value>]
```

Note: If the key value is known, any user can remove the resource reservation made by another user.

The emulator resources corresponding to the specified key value are released. If the `./dbFiles/ts.<username>.resv.key` file is used, the key value is deleted from the file. If the key file has more than one key values, the resources corresponding to the first key value are released, and the first key value is deleted from the file.

If the `dbFiles/ts.<username>.resv.key` file does not exist, the `test_server` program exits with an error and displays a message that prompts you to input the key value.

If the specified key value is not associated with any emulator resource, nothing happens, and no warning is generated.

Reducing Emulator Initialization Time Using KeepHostAlive

When you need to run the same design multiple times (for example, while running regression tests), the KeepHostAlive feature can be used to speed up the initial xeDebug connection and design download time. By using the KeepHostAlive feature, you can retain the connection between the host and the emulator even after the design run is finished.

Enabling the KeepHostAlive Feature

Enable the KeepHostAlive feature by running the following steps:

1. Before starting xeDebug, manually run the DBServer executable on the host machine connected to the emulator. For details, refer to the [Setting the DBServer](#) section.

2. Start xeDebug and execute the `debug` command.
3. Before downloading the design, use either of the following commands on the xeDebug command prompt:

a. `xeset keepHostAlive <1 | 2 | 3>`

b. `host -keepAlive <hostname>`

Note: Alternatively, before invoking xeDebug, set the `XE_KEEP_HOST_ALIVE` environment variable to either 1, 2, or 3. For example, in the C shell, set the environment variable as following:

```
setenv XE_KEEP_HOST_ALIVE 1
```

The level of restrictions is the highest when 1 is specified and lowest when 3 is specified. For details about the restrictions associated with each value, refer to the `keepHostAlive` run-time parameter in the `xeset` command description, *Run-Time Commands* chapter, *UXE Command Reference Manual*.

To use the KeepHostAlive feature in IXCOM-compiled designs, set the `xeset keepHostAlive` run-time parameter to either 2 or 3. For designs compiled in ICE mode, it can be set to either 1, 2, or 3.

When the `host -keepAlive <hostname>` command is used to enable this feature, the value of `xeset keepHostAlive` run-time parameter is set to 2 by default.

When KeepHostAlive feature is enabled, even when xeDebug exits, the hardware resources on the emulator, for the particular design, remain unavailable, for a certain amount of time-out period (default is 180 seconds). Start the new xeDebug process before this time-out period ends to reuse the existing hardware resources from the previous xeDebug session. This saves the initial connection and design download time. The design is reset to its initial state in the emulator hardware for each new xeDebug process.

The time-out period value can be controlled by using either of the following methods:

- Set the `keepHostAliveTimeout` run-time parameter at the xeDebug command prompt as,

```
xeset keepHostAliveTimeout <value>
```

- Set the `XE_KEEP_HOST_ALIVE_TIMEOUT` environment variable before starting xeDebug. For example, in the C shell, set the environment variable as,

```
setenv XE_KEEP_HOST_ALIVE_TIMEOUT <value>
```

The time-out value set using the `xeset keepHostAliveTimeout` run-time parameter is given priority if both the commands mentioned above are used with conflicting values. Also, the value of the run-time parameter can be updated anytime before exiting the xeDebug session.

 Important

When the KeepHostAlive feature is enabled, executing the `test_server` program to check the host and domain availability displays the used domains as occupied by the design being run.

Setting the DBServer

DBServer is a daemon process that you can optionally run on the emulator's host workstation to get direct control on spawning the DBEngine process. The DBEngine process is the process that communicates directly with the emulator and the xeDebug process (which can run on a different workstation).

By default, if you do not start DBServer manually, xeDebug starts DBEngine on the host workstation using a remote shell. In some cases, when this operation fails, for example due to file system mount point naming issues, you can start DBServer manually on the host workstation. Then, DBServer becomes responsible for spawning the DBEngine processes, as needed.

To manually start the DBServer executable, use the following process:

1. Set the `XE_DB_SVR_RPC_NUM` environment variable before starting the DBServer executable. Use the following command in the xeDebug and DBServer command shells to set this environment variable:

```
setenv XE_DB_SVR_RPC_NUM <unique_number>
```

The value of the `XE_DB_SVR_RPC_NUM` environment variable must be unique for each user. The DBServer can handle multiple xeDebug processes if the design directory path is different. Multiple users can also access the same design database, using a separate DBServer process for each user.

Note: In the DBServer command shell, user can also set the following environment variable to print detailed information when the design is running:

```
setenv XE_DB_SVR_DISP_MSG 1
```

2. Start DBServer using the following command at the UNIX or Linux command prompt:

```
DBServer &
```

Disabling the KeepHostAlive Feature

To disable the KeepHostAlive feature, either set the `keepHostAlive` run-time parameter to 0 or execute the `host -kill` command. Consequently, the emulator resources become available to be taken by another user's design at the termination of the current job.

Terminating KeepHostAlive Reservation Preemptively

Multiple DBEngine processes running on a host emulator using the KeepHostAlive feature can prevent a new xeDebug session from starting for a different design. In such situations, the DBEngine processes can be terminated preemptively to release the emulator resources for optimized use.

When the KeepHostAlive feature is enabled, DBServer records each DBEngine process it spawned. Also, DBServer tracks each DBEngine status and the domains it downloaded.

When a new xeDebug session connects to DBServer, it checks all DBEngine processes that are kept alive, and executes the following steps:

1. DBServer checks whether there is a DBEngine process that can be reused according to the keepHostAlive run-time parameter value. If such DBEngine process exists, reuse it; else execute steps (2) and (3) discussed below.
2. If none of the existing DBEngine processes can be reused, DBServer identifies the DBEngine processes that occupy the domains requested by the new xeDebug session. If such processes are identified, DBServer kills them and releases the corresponding domains.
3. If the number of domains released is less than the number of domains requested by the design in the new xeDebug session, DBServer kills other DBEngine processes too to release the sufficient number of domains.

This preemptive termination of KeepHostAlive reservation improves the efficiency of using the emulator hardware in multi-design debug environment.

Handling Failures at Run Time with Wire Duplication

Wire duplication provides redundancy to help deal with certain types of hardware failures. When you use wire duplication in compile, if a duplicated wire fails during run time (usually indicated by a parity error in `DBEngine.msg`), use the following process:

- Run wire diagnostics.
- If the wire is failing solidly, the failure is detected and the configuration file is updated when the `xeDiags` command execution.
- If the failure is intermittent, contact your FAE to have the configuration file updated. Your FAE will need the error messages from your failing run.

You can then download your design without recompiling. Save or restart is not supported across wire failures; so, you must start your test from time zero.

Note: The compile options for wire duplication allow you to specify which types of wires to duplicate. See the [Duplicating Wires for Transmitting a Signal between Chips or Boards](#) section for more information.

Using Suspend and Resume to Manage Emulator Resources

In the STB mode, you can suspend and resume an emulation run without stopping the clocks. The suspend operation releases the design from the emulator, while resume restores the design state to the emulator. The following sections discuss the process in detail.

Suspending the Design from the Emulator

A design running on an emulator can be suspended using either of the following methods:

- Executing the `suspend` command on the `xeDebug` command line in interactive mode.
- Invoking the `xeSuspend` script when running the design in batch mode. This script can also be used in interactive mode; however, its use in batch mode is strongly recommended. For details, refer to the [Using xeSuspend Script](#) section.

Note: The `xeSuspend` script is used to suspend and resume the design remotely.

- Using a `./dbFiles/suspendSession.qt<PID>` file to interrupt the emulation in batch mode during the same `xeDebug` session.

Note: To use this file, the `suspendEnable` run-time parameter must be set to 1.

- Clicking the *Suspend* button on the *Setup* tab if using the `xeDebug` GUI to run and debug the design.

Suspending a design run releases the emulator. The suspend operation saves the design snapshot and stores it in the `./dbFiles/suspendSession.qt<PID>` file. You can set the design prototype, memory content, and all the set or force signals in the file. The suspend operation also saves the SDL control signals. To prevent altering the hardware state, the hardware-related commands and `xeDebug` GUI tabs are disabled when a run is suspended. If a disabled command is executed, an error message is generated.

Resuming the Design State to the Emulator

Use the `resume` command to resume the design state to the emulator. Alternatively, if using the `xeDebug` GUI, click the *Resume* button on the *Setup* tab.

On resume, the emulation run continues from the same cycle, as the one before suspend. Resuming a session re-downloads the design and restores the last snapshot taken by the

`suspend` command. Memory content, all SDL control signals, and all set and forced signals are restored.

Important

The SDL state and all the triggers of the design are maintained during suspend or resume of emulation. This means, if `run` command is executed and the design gets suspended before the trigger, the trigger takes place when the design is resumed and the trigger condition holds true. In batch (or script) mode, once the design gets resumed, the remaining XEL commands following the `resume` command get executed normally.

Infinitrace session is also supported during suspend and resume, irrespective of emulation clock being run or stopped during a call to suspend.

On exiting a xeDebug session, all suspend or resume operations are cleared and the `suspendSession.<PID>` directory is deleted.

For more information on the `suspend` and `resume` commands, refer to *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

Using xeSuspend Script

The `xeSuspend` script can be used in both interactive mode and batch mode. In the batch mode, it primarily automates the use of the `suspend` and `resume` commands.

Depending on the configuration and platform, the `xeSuspend` script might need modification. The format of the script is:

```
xeSuspend <emulator> <PID> -suspend | -resume
```

For the `xeSuspend` script to successfully suspend the design, the following conditions must be true, otherwise the suspend request is ignored with an error message:

- Emulation clock must be running in either blocking or non-blocking mode.
- You must have permission to create files in the design directory.
- The design session to be suspended must have the `suspendEnable` run-time parameter set as 1.

Note: The default value of the `suspendEnable` run-time parameter is 0, which prevents emulation from being suspended remotely. In this manner, the existing user can prevent the xeDebug session from being suspended by any other user using the `xeSuspend` script.

UXE User Guide

Working with the Palladium XP Emulators

Examples of Suspending and Resuming the Design

The following example shows the use of the `suspend` and `resume` commands:

```
XE> debug .
XE> host emulator1
XE> download
XE> configPM -stb
XE> run -nowait 500000000          <<starts non-blocking run for 50M cycles
XE> suspend                      <<suspend emulation and release emulator
XE> getCycleNum                  <<suppose 20M cycles done when suspended
XE> resume <<reload design from last stopping point, and continue remaining cycles
XE> getCycleNum                  <<when previous run complete, getCycleNum would show 500000000
```

The following example shows the use of `xeSuspend` in interactive mode:

```
XE> design top top
XE> xeset suspendEnable 1          <<enables suspend or resume mode
XE> debug .
XE> host emulator1
XE> download
XE> configPM -stb
XE> run 1000000                   <<blocking run for 1 million cycles
                                <<xeSuspend emulator1 <PID> suspend issued from other xterm
XE> INFO(legacy-xxx): Design is suspended from hardware
                                <<xeSuspend emulator1 <PID> resume issued from other xterm
XE> INFO(legacy-xxx): Design is resumed from hardware
```

Compiling Designs for In-Circuit Emulation with xeCompile

The ICE compilation flow involves a series of steps to convert the design from the format created by the design tools into the compilation database that can be downloaded to the emulator. The structure of the compilation database depends on the specifications given at each step of the compile process. This compilation flow can be used when the design components are known to be synthesizable and downloadable after compilation to the emulator. This compilation flow is also automatically invoked in Simulation Acceleration flow by IXCOM to compile the synthesizable portions for the emulator while partitioning the rest of the design portions to the software simulator. The Simulation Acceleration flow with IXCOM is described in [Chapter 4, “Compiling Designs for Simulation Acceleration Using IXCOM.”](#)



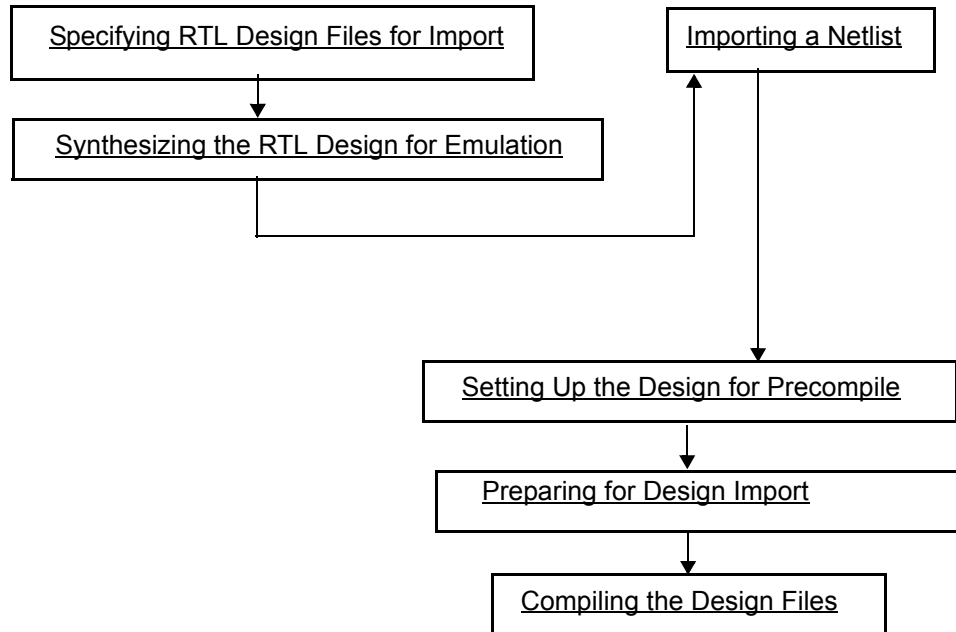
Important

The content of this chapter pertains only to the ICE mode.

The compile process uses XEL commands, UNIX commands, and input and output files created to import, check, and compile a design.

The flow chart shown below depicts a typical ICE compilation flow.

Figure 3-1 ICE Design Compilation Process



Preparing for Design Import

Before importing the design into the emulator, perform the following tasks:

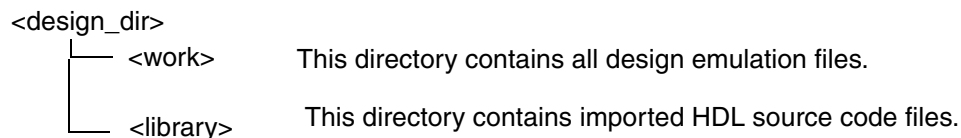
- Familiarize yourself with the directory structure.
- Create a design directory to store all emulation design files.
- Store the imported HDL source code files in a working library directory.

The following sections discuss the required preparations for design import.

Creating the Directory Structure

The directories that contain files used in xeCompile design import process are illustrated in [Figure 3-2](#).

Figure 3-2 Directory Structure



Design Directory

The directory, *<design_dir>*, is the design directory from which xeCompile is invoked. This directory will contain all of the RTL emulation design files, and can be anywhere on your network. It can contain the HDL source code that you want to import into xeCompile.

Working Library

A work library is a directory in which you store some or all of the imported HDL source code files. This library is not necessarily a subdirectory of the *<design_dir>*; it can be located anywhere on your network.

Note: For some designs, however, you might find it more convenient to store all of the design files, including all of the HDL working libraries, in the *<design_dir>* for the specific chip design project.

A default work library named `work` is automatically created during the import process. The default location of this library is a subdirectory of the `<design_dir>`. If you use this default, all of the compiled HDL source code for the design is stored in this single subdirectory.

Create Additional Working Libraries

Create additional working libraries in any directory, and store portions of the compiled RTL source code in separate work libraries. Ideally, these additional work libraries should be within the same overall directory tree as the `<design_dir>`.

Relative path can be used to specify working libraries that are in the same directory tree as the `<design_dir>`. However, if a working library is in a different directory tree, specify the full path to the working library.

Setting the Absolute Path for Libraries

For changing the current working directory at any time during the import or synthesis process, use the absolute path to define logic libraries when importing the design. A logic library is a symbolic name for a directory, usually a subdirectory of the `<design_dir>`.

Note: xeCompile supports four built-in logic libraries: IEEE, STD, QTREF, and Cadence. Therefore, **do not** create an additional working library with any of these names, or redefine any of these libraries or their content.

The symbolic names for the additional working libraries can represent any directory on the network. However, you should normally create the additional working libraries as subdirectories of the `<design_dir>`, and name the subdirectories the same as the working library name. This way, all files used in the design are in one directory, and the subdirectory names are easy to remember.

To set the absolute path for the libraries, execute the following XEL command before importing the design into xeCompile:

```
hdlDefineLib -add <library_name> <absolute_library_path>
```

For example:

```
hdlDefineLib -add IBMCU11VDK /nas/caenac/<working_directory>/log/siliconoptix/  
sr32854150/TestCase_infra/work
```

Example: Creating a VHDL Work Library

Consider the following VHDL source code that contains references to VHDL libraries other than `work`:

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

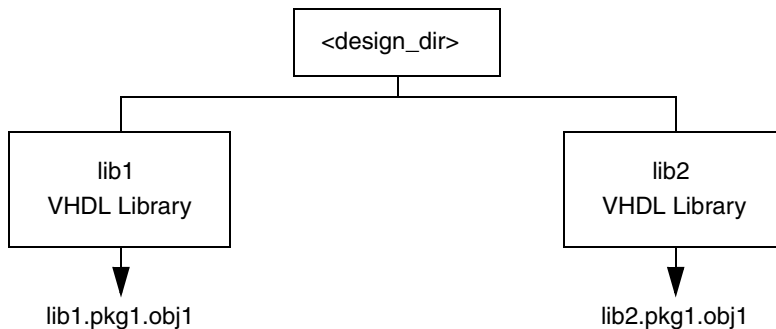
```
# Reference first library, package, and object.
LIBRARY lib1;
USE lib1.pkg1.obj1;

# Reference second library, package, and object.
LIBRARY lib2;
USE lib2.pkg1.obj1;
```

Note: Although the library names in this example are different (`lib1` and `lib2`), the package and object names are both `pkg1.obj1`, relative to each library.

In this example, before importing the design into the VHDL database, create two VHDL libraries, named `lib1` and `lib2` (refer to [Figure 3-3](#)).

Figure 3-3 Example of VHDL Libraries



While creating a VHDL logic library, assign the specified VHDL logic library `<lib>` to the selected directory `<dir>`. xeCompile then adds the new VHDL working library to the list of existing VHDL logic library names. In this example, the symbolic name `lib1` should represent the `./lib1` path, and the symbolic name `lib2` should represent the `./lib2` path.

After creating these two VHDL directories, first import the two `pkg1.obj1` source code files.

- When the first source code file named `pkg1.obj1` is imported into the `lib1` VHDL library, importing compiles the file into xeCompile database format, and stores it in the `<design_dir>/lib1` directory.
- When the second source code file named `pkg1.obj1` is imported into the `lib2` VHDL library, importing compiles the file into the xeCompile database format, and stores it in the `<design_dir>/lib2` directory.

Now import the remaining VHDL files into the default VHDL library, named `work`. After compiling these source code files into the xeCompile database, the import process stores the compiled files in the `<design_dir>/work` directory.

In this example, if the VHDL libraries named `lib1` and `lib2` are not created, or the two `pkg1.obj1` files are not imported into these VHDL libraries, xeCompile import cannot find all of the information that it needs. xeCompile cannot import the RTL design into the work library, and reports an error.

The VHDL work library that is being created is a symbolic name, or alias, for the physical directory path. Therefore, you should normally specify a relative path, such as `lib1`. This ensures that all VHDL libraries used in one design are subdirectories of the same `<design_dir>`.

Also, the relative path should usually match the symbolic VHDL working library name. For example, it could be confusing if the symbolic name `lib1` represents the directory name `directory1`.

Specifying RTL Design Files for Import

Register Transfer Level (RTL) designs are those that use hardware description language, either Verilog or VHDL. When an RTL code is imported, xeCompile takes the code and maps it to functionally equivalent gate-level code.

By default, source files are imported incrementally. If the source was previously imported into the working library, or if the time stamp for the file being imported is earlier than the last import, source is not re-imported. This makes incremental import faster than a full import, as files need not be re-read and compiled unless it is necessary.

When importing a design for the first time, or to make sure the design database does not contain previously imported design files (for instance, if some parts of the design were deleted), select to fully import the design.

Full import performs the following operations:

- Deletes the specified Verilog HDL database or VHDL working library, including all design files that were previously imported into that database or library.
- Creates a new, empty Verilog HDL database directory or VHDL working library with the specified name.
- Imports all the specified files into the Verilog HDL database or VHDL working library.

The import procedure for Verilog and VHDL designs is discussed in detail in the [Specifying Verilog Designs for Import](#) and [Specifying VHDL Designs for Import](#) sections.

Specifying Verilog Designs for Import

To import a Verilog RTL design into the emulation design database, use the following procedure.

Note: The import and synthesis process of the HDL Compile tool supports a subset of Verilog IEEE STD 1364-2001.

1. Change to the emulation design directory, if not already in it, by executing the following command on the UNIX command-line prompt:

```
% cd <design_dir>
```

2. Invoke xeCompile, if it is not already running.

```
% xeCompile
```

3. (Optional) Specify the path and library name of the directory containing the Verilog HDL database by executing the following command on the XEC> prompt:

```
hdlWorkPath <dir>
```

A Verilog HDL database is a symbolic name, or alias, for a directory that stores compiled versions of the Verilog RTL design files that were imported.

If the `hdlWorkPath` XEL command is not used, then the design files are imported into a default working library named `work`, representing a subdirectory named `work` in the emulation design directory.

4. Specify the names of the Verilog HDL source code files to import using the `hdlInputFile <file>` XEL command.

The `<file>` option can include the path to the specific Verilog HDL source code file.

Multiple `hdlInputFile` commands can be executed to add or delete from the list of design files to import. Refer to the *HDL Compiler Commands* chapter of the *UXE Command Reference Manual* for more details on the command syntax.

5. (Optional) Select an ASIC library to import using the `hdlInputFile -l <file>` XEL command.

This step is required only if the Verilog RTL source code files refer to Verilog modules that are not defined within that source code file—that is, if a mixed-level design is being imported.

When a gate-level netlist is synthesized from the imported design, the synthesis process first looks for cell definitions in the individual design files that were imported.

For cells that are not defined in the imported files, synthesis uses cell definitions from the ASIC library that were selected, or treats them as “blackboxes” (empty cell definitions) if

synthesis does not find cell definitions. Different parts of the design can use cell definitions from different ASIC libraries. The Import process resolves the blackboxes.

The resulting gate-level netlist completely defines all cells in the design, regardless of where each cell definition originated.

6. Execute the `hdlImport -2001` command to turn on the Verilog 2001 support and start the import process. By default, the `hdlImport` command updates or creates a library directory, as specified using the `hdlWorkPath` command in step 3.

- a. To import all Verilog RTL design files into the Verilog HDL database, use the following command:

```
hdlImport -full
```

Full import first removes all previously imported design files from the Verilog working library. It then imports all the Verilog RTL design files, even if none of these files were created or updated since the last Verilog design import.

- b. Using the `hdlImport` command without any parameter imports only new or updated Verilog RTL design files into the Verilog HDL database.

By default, xeCompile imports the Verilog RTL design files incrementally. Thus, only Verilog RTL design files that were created or updated since the last design import are imported into the Verilog HDL database.

- c. For referencing a library use the `hdlImport -l <lib>` command.

For example:

```
hdlImport -l lib1 -l lib2
```

If `<lib>` is an ASIC library, then the xeCompile looks for a cell definition in the ASIC library.

Importing creates an HDL database directory under the `<design_dir>`. xeCompile checks the individual Verilog modules in the imported design files to verify that internal logic can be synthesized for each module. It also performs some interface checking.

If a Verilog design file cannot be successfully imported, an error message is generated. If errors are encountered, fix those files for which errors are reported, and re-run with the Incremental Import function.

For more details about the supported sections of the IEEE Standard Verilog Hardware Description Language, IEEE Std 1364-2001, refer to the `hdlImport -2001` command in the *HDL Compiler Commands* chapter of the *UXE Command Reference Manual*.

Note: Alternatively, from the UNIX command-line interface, execute the `vavlog` utility that is comparable to the `hdlImport -2001` XEL command. It enables you to import Verilog source files. For syntax and usage details, refer to [vavlog](#).

Specifying VHDL Designs for Import

To import a VHDL design into the emulation design database, use the following procedure. The UXE Compiler import and synthesis for emulation supports a subset of VHDL IEEE STD 1076-1993.

1. Change to the emulation design directory, if not already in it, by executing the following command on the UNIX command-line prompt:

```
% cd <design_dir>
```

2. Invoke xeCompile, if it is not already running.

```
% xeCompile
```

3. (Optional) Specify the path and library name of the directory containing the VHDL design database using the following command:

```
hdlWorkPath <dir>
```

A VHDL working library is a symbolic name, or alias, for a directory that stores the compiled versions of the imported VHDL RTL design files.

If the `hdlWorkPath` command is not used, then the design files are imported into a default working library named `work`, representing a subdirectory named `work` in the emulation design directory.

4. Specify the HDL language of the design files that need to be imported. This can be done using the `hdlLang vhd1` command.

Note: If this command is not used, xeCompile considers the default language as Verilog.

5. Specify the names of the VHDL source code files to import using the `hdlInputFile <file> XEL` command.

The `<file>` option can include the path to the specific VHDL source code file.

Use multiple `hdlInputFile` commands to add or delete from the list of design files to be imported. Refer to the *HDL Compiler Commands* chapter of the *UXE Command Reference Manual* for more details on the command syntax.

6. Execute the `hdlImport` command to start the import process.

- a. To import all VHDL RTL design files into the VHDL working library, use the `-full` option with the `hdlImport` command as shown below:

```
hdlImport -full
```

Full import first removes all previously imported design files from the VHDL working library. It then imports all the VHDL RTL design files, even if none of these files were created or updated since the last VHDL design import.

- b.** Using the `hdlImport` command without any parameters imports only new or updated VHDL RTL design files into the VHDL working library.

By default, xeCompile imports the VHDL RTL design files incrementally. This option imports only VHDL RTL design files that were created or updated since the design was last imported into the VHDL working library.

Any design file not selected for import is not imported into the VHDL working library, even if it was created or updated since the design was last imported.

Any design file selected for import is not imported into the VHDL working library, if it was not created or updated since the design was last imported.

If this is the first time that the VHDL RTL design is being imported into the VHDL working library, incremental mode imports all selected design files, because they are all new files in the VHDL working library.

- c.** For referencing a library use either the `hdlImport -l <lib>` command or the use clauses in VHDL code.

For example:

```
hdlImport -l lib1 -l lib2  
library lib1, lib2; use lib1.pkg1.all; use lib2.all;
```

If `<lib>` is an ASIC library, then the xeCompile looks for a cell definition in the ASIC library.

Importing creates a VHDL working library under the `<design_dir>`, compiles the VHDL design into a format that xeCompile can synthesize into a gate-level netlist, and stores the compiled design in the VHDL working library.

xeCompile checks individual VHDL design units in the design files to ensure that the Synthesize HDL Design window can synthesize internal logic each VHDL design unit. It also does some interface checking.

If the Import HDL Design function cannot successfully import a VHDL design file, then it generates an error message. However, the importing continues until all source code files that can be successfully imported are in the VHDL work library. Fix the source code files for which errors are reported, and rerun the Import HDL Design process using the Incremental Import options. Only the changed files are imported.

Note: Alternatively, from the UNIX command-line interface, execute the `vavhdl` utility that is comparable to the `hdlImport` XEL command. It enables you to import VHDL source files. For syntax and usage details, refer to [vavhdl](#).

Specifying Mixed-Language Libraries for Import

Verilog modules and VHDL entities can be imported into the same library using separate `hdlImport` sessions as illustrated in the following example:

```
hdlLang vhd1
hdlInputFile -add sub.vhd
hdlImport
hdlLang verilog
hdlInputFile -add top.v
hdlImport
```

In the above example, the `hdlLang vhd1` command specifies that the HDL design file that needs to be imported into the working library is in VHDL language. The `hdlInputFile -add` command adds the `sub.vhd` file to the list of HDL design files available for import using the `hdlImport` command. After the VHDL file has been imported into the working library, executing the `hdlLang verilog` command clears the list of files from `hdlInputFile`, and resets the type of HDL design file as Verilog. This enables the import of `top.v` Verilog file into the working library.

Now, consider the following example.

```
hdlLang vhd1
hdlInputFile -add sub.vhd
hdlLang verilog
hdlInputFile -add top.v
hdlImport
```

Here, only the `top.v` Verilog file is imported because the `hdlImport` command was not executed to import the `sub.vhd` VHDL file. Consequently, the `hdlLang verilog` command cleared the list of files specified using the first `hdlInputFile` command and reset the HDL language setting to import Verilog files only.

Note: You will get an information message when `hdlLang` is used while files are in the `hdlInputFile` list.

`hdlLang` also resets the current working directory (`hdlWorkPath`) to the default `work`. So, `hdlWorkPath` should always be used after `hdlLang`.

The following XEL commands will import the VHDL entities and Verilog modules into the `./libtest` directory:

```
hdlLang vhd1
hdlWorkPath ./libtest
hdlInputFile -add sub.vhd
hdlImport
```

```
hdlLang vlog
hdlWorkPath ./libtest
hdlInputFile -add top.v
hdlImport
```

Binding the Components

When a Verilog entity is bound to a mixed-language library, all Verilog modules in the library are searched first to find a case-sensitive match; then all VHDL entities in the library are searched to find a case-insensitive match.

Component binding in VHDL is case-insensitive, but component binding in Verilog is case-sensitive. Two Verilog modules of the same name of different case, such as ADDER and Adder, are allowed in a Verilog library and are considered different modules. However, they are considered the same entity if they are in VHDL. The latest imported VHDL entity is used.

You cannot use a Verilog module and a VHDL entity with the same name, and this will cause xeCompile to fail with an error message.

Synthesizing the RTL Design for Emulation

Designs of RTL format need to be synthesized before starting the compilation. The synthesis process converts the imported RTL design written in Verilog or VHDL language into a functionally equivalent gate-level netlist or a database.

Synthesis operates on the design files that are imported into the Verilog HDL database or VHDL working library, but stores the gate-level netlist and other synthesis results in your *<design_dir>* (usually one directory higher than the HDL database or working library).

Optionally, the synthesized (gate-level) netlist can also be saved as a file for use with design and verification tools other than Palladium XP.

Cadence supports a synthesizable subset of the Verilog and VHDL languages, and you can add the required emulation directives into the source code to constrain synthesis. For information about unsupported Verilog and VHDL functions, and about Cadence emulation directives, refer to [Appendix B, “Modeling Guide for Verilog Designs for ICE Mode”](#) and [Appendix C, “Modeling Guide for VHDL Designs for ICE Mode.”](#)

Cadence recommends using Palladium XP HDL Compiler to synthesize the RTL designs. For usage details of Palladium XP HDL Compiler, refer to the *HDL Compiler Commands* chapter in the *UXE Command Reference Manual*.

The Palladium XP HDL compiler runs much faster (approximately 30 - 60 times faster) than silicon synthesis tools. It generates gate counts that are within $\pm 10\%$ of silicon-synthesis gate counts. This minor difference in gate count occurs because UXE compiler uses emulation-specific optimization, instead of the general-purpose technology mapping used by silicon synthesis tools.

The topics covered in this section are:

- ☐ [Synthesizing Verilog or VHDL Designs](#)
- ☐ [Synthesizing Mixed-Language Designs](#)
- ☐ [Synthesizing Mixed-Language Designs](#)
- [Synthesizing Designs with Memories](#)
- [Synthesizing Designs with Acceleratable Testbench \(ATB\) Constructs](#)
- [Executing Distributed Synthesis during RTL Synthesis](#)
- [Using Conformal to Verify HDL Compiler-Generated Netlist](#)
- [Using UNIX Commands for RTL Import and Synthesis](#)

Synthesizing Verilog or VHDL Designs

After importing the RTL design, the UXE compiler takes the RTL code and synthesizes it. Synthesis operates on the design files imported into the HDL database, and stores the gate-level netlist and other synthesis results in the `<design_dir>` for mapping into the PDB database for compile. The RTL design can be in Verilog or VHDL. A subset of VHDL IEEE STD 1076-1993 can be imported and synthesized for emulation.

Following steps describe the procedure for synthesizing a Verilog or VHDL RTL design:

1. Specify the file in which the output of HDL synthesis should be written by using the `hdlOutputFile` command.
2. (Optional) If certain modules or VHDL design units should not be synthesized, declare those using the `hdlSkipModule` command.
3. Use the `hdlSynthesize` command to perform the synthesis. Unless the `hdlImport -full` command was used during the design import, this command performs content-based incremental synthesis. This means that a module is synthesized again only if its contents are different from the previously synthesized design.

Synthesizing All or Specific Modules:

All modules and design units in the work library can also be synthesized by using the `-all` option with the `hdlSynthesize` command. However, this option should be used only when compiling the cells of a library that does not have a top module or design unit.

Note: Avoid using the `hdlSynthesize -all` command with the `hdlOutputFile -f qel` command, or the output file will not be functional.

You need to specify the name of the top-level module as the `<top_module>` or `<top_unit>` (for VHDL designs) option of the `hdlSynthesize` command.

The top module is the name of the Verilog design file describing the top level of the design hierarchy to be synthesized. The top design unit is the name of the VHDL design file that contains the entity and architecture description for the top-most design unit that is to be synthesized.

The specified top-level module (or top design unit) and all modules (and design units) in the design hierarchy below it are synthesized. If you do not want to synthesize the entire design, specify the name of the Verilog design module (or design unit) from which synthesis should start. The selected module (or design unit) is regarded as the top-level module (or design unit) of the design, and design modules (and units) at or above the same hierarchical level as the specified top-level module (or design unit) are not synthesized.

Synthesizing Memories:

To synthesize Verilog or VHDL memories use the `hdlSynthesize -memory` command. This enables to synthesize memories into constructs that preserve the functions defined in the RTL source code. For detailed information of the memory synthesis process, refer to the [Synthesizing Designs with Memories](#) section.

Using the `-keepVhdlCase` option with the `hdlSynthesize` command retains the case of the names within the VHDL design file as much as possible. If this command option is not specified, the VHDL names are set to all-uppercase in the output.

The length of time required to synthesize a design depends on the size and complexity of the design. While synthesis is running, messages describing the progress of the process are displayed on the screen.

Synthesizing Mixed-Language Designs

Mixed-language designs can also be synthesized using the XEL commands. A mixed-language design consists of modules in libraries with different languages.

Use the following commands:

- `hdlDefineLib`: Defines the directory path of either a Verilog or VHDL library.

- `hdlSynthesize`: Synthesizes an RTL design to get a gate-level netlist in Verilog, in VHDL, or in both Verilog and VHDL.
- `hdlImport`: Imports VHDL and Verilog files into different libraries.
- `hdlSynthesize -keepVhdlCase`: Retains the case of all the names in the RTL design in the Verilog and VHDL netlists.
- `hdlSynthesize -vhdlToLowercase`: Sets the case of all names in the RTL design to lower case in the Verilog and VHDL netlists.
- `hdlImport -l <lib>`: References libraries to search for modules undefined in library work.

Important

If a VHDL module is instantiated by a Verilog module, import the VHDL part of the RTL code first, then import the Verilog part of the RTL code.

Using VHDL Port Types for Verilog

In order to successfully use a mixed-language design that contains both Verilog and VHDL code, use the correct type of VHDL ports to interface with Verilog. These ports must be `std_logic` or `std_logic_vector`, `std_ulogic`, `std_ulogic_vector`, `bit`, or `bit_vector`.

To use either `std_logic` or `std_logic_vector` port type in VHDL, include the following statements:

```
library ieee;  
use ieee.std_logic_1164.all;
```

Example: Mixed-Language Synthesis

To synthesize a mixed-language design, run a script similar to the following example:

```
hdlLang vhdl  
hdlWorkPath lib1  
hdlInputFile a1.vhd a2.vhd  
hdlImport  
hdlLang verilog  
hdlWorkPath lib2  
hdlInputFile b1.v b2.v  
hdlImport -l lib1  
hdlOutputFile -f verilog g.ver
```

```
hdlSynthesize -keepVhdlCase verilog_top
```

Incremental Synthesis

The Palladium XP compiler supports the following types of incremental synthesis:

- Time-based or file-based incremental import – Only files with new time stamp will be re-imported.
- Content-based or module-based incremental synthesis – Only modified modules will be re-synthesized.

By default, incremental synthesis is enabled. If you are working with a previously synthesized gate-level netlist of a Verilog RTL design, only those Verilog design modules that were imported again are synthesized again. If the entire Verilog RTL design was fully imported, the entire design is synthesized again.

To ensure that all the design components are re-synthesized completely, use the following command at the time of HDL design import:

```
hdlImport -full
```

Content-based incremental synthesis enables you to quickly implement the Engineering Change Orders (ECOs). Additionally, Palladium XP compiler can perform a high-capacity synthesis of RTL designs to generate the gate-level netlist. It can process large, multi-million gate designs, using the available disk swap space of a workstation's memory. You do not need to manually partition the original RTL design into blocks for synthesis.

The names of all RTL signals and cell instances are preserved during the netlist generation; so it is easier to debug the design.

The memory information (such as number of ports, and width of the address and data bus) is extracted from the RTL Verilog or VHDL designs, and the memory compilation information is written directly into the design database. Optionally, you can also generate VHDL or Verilog memory files when synthesizing memories. You do not need to synthesize memories as blackboxes, unless you want to change the emulator memory configuration from the default.

Preserving the RTL Names

RTL name preservation allows for easy debugging. To use name preservation, follow the instructions below:

- Do not remove RTL net during optimization.

```
assign z1 = (!a & b) | (a&!b); Q_XOR2 u1 (z1, a, b);  
Q_INV u1 (n1,a);
```

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

```
assign z1 = !a & b;          Q_INV u2 (n2,b);
assign z2 = a & !b;          Q_AN02 u3 (z1, n1, b);
assign z = z1 | z2;          Q_AN02 u4 (z2, n2, a);
                             Q_OR02 u5 (z, z1, z2);
```

- Insert buffers for equivalent RTL nets:

```
input [1:0] a;
wire [1:0] b;                Q_BUF
                             u1 (.Z(b[0]), .Z (a[1]));

if (a==2'b10) b=2'b11;
else if (a==2'b11) b=2'b01;
else b=2'b10;
```

- Signals declared at the module level are 100% preserved. Variables declared locally inside a block, process, function, or task might not be fully preserved.

Using Command Options for Name Preservation

- Turn off name preservation to save gate counts:

```
hdlSynthesize -noNamePreservation
```

- Turn off name preservation on nets without fanout to primary output:

```
hdlSynthesize -fanoutNetOnly
```

- Avoid merging equivalent flip-flops and optimizing out flip-flops with constant inputs (useful for easy formal verification) and “force value” debugging.

Below is an example of `hdlSynthesize` without the `-keepAllFlipFlop` option:

```
hdlSynthesize
always @(posedge clk)      Q_FDP0 u1 (.CK(clk), .D(c), .Q(a));
begin                      Q_BUF u2 (.Z(b), .A(a));
a = c;
b = c;
end
```

Below is an example of `hdlSynthesize` with the `-keepAllFlipFlop` option:

```
hdlSynthesize -keepAllFlipFlop
always @(posedge clk)
begin
a = c;                      Q_FDP0 u1 (.CK(clk), .D(c), .Q(a));
b = c;                      Q_FDP0 u2 (.CK(clk), .D(c), .Q(b));
end
```

- For VHDL, a record-type signal name “a.b”, use “a.b” instead of “a_b” in the netlist by using:

```
hdlSynthesize -keepRtlSymbol
```

Handling Case Sensitivity

- To import Verilog in a case-insensitive way:

```
hdlImport -u
```

- By default, RTL VHDL symbols are converted into all uppercase in gate-level netlists.

- To preserve the VHDL case declared at RTL use:

```
hdlSynthesize -keepVhdlCase
```

- To convert RTL VHDL symbols at all lowercase use:

```
hdlSynthesize -vhdlToLowerCase
```

Maintaining Log Files

By default, the HDL synthesis log is maintained in the `hdlIce.log` file. To specify a user-specific log file, use the following command:

```
hdlLogfile <mylogfile>
```

Like other XEL commands executed on the xeCompile CLI, the commands executed for synthesizing the RTL designs are also recorded in the `xe.msg` file.

Using Synthesis Directives for Emulation

A set of directives are added to constrain the RTL synthesis process, and to help you to synthesize a design for emulation. Use these directives to debug designs and facilitate emulation in HDL designs. The HDL-specific synthesis directives are discussed below.

The following directives are supported in VHDL designs:

```
built_in SYN_INTEGER_TO_UNSIGNED  
built_in SYN_UNSIGNED_TO_INTEGER  
built_in SYN_INTEGER_TO_SIGNED  
built_in SYN_SIGNED_TO_INTEGER  
built_in SYN_ZERO_EXTEND  
built_in SYN_SIGN_EXTEND  
resolution_method three_state  
resolution_method wired_and  
resolution_method wired_or  
translate_off  
translate_on  
synthesis_off  
synthesis_on
```


UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

```
preserve
keep_net
```

The following directives are supported in Verilog designs:

```
translate_off
translate_on
parallel_case    // all items are mutually exclusive
full_case        // full coverage
preserve
keep_net
```

The default prefixes can be `pragma`, `synopsys`, and `quickturn`.

Use the command-line option `-synth_prefix` to add prefixes, and `-rm_synth_prefix` to remove `pragma` or `synopsys` default prefix. For example:

```
hdlInputFile -synth_prefix <pragma_name>
```

Here, `<pragma_name>` is the string used in the design.

Note: If the `-rm_synth_prefix` option is used, the `quickturn` pragma cannot be removed.

Examples

The following example shows the support of `built_in SYN_UNSIGNED_TO_INTEGER`:

```
function typeconv ( s : std_ulogic_vector ) return integer is:
-- pragma built_in SYN_UNSIGNED_TO_INTEGER
begin
--  synopsys translate_off
    <Actual description for this function>
--  synopsys translate_on
return int_result;
end typeconv;
```

In this example, `<Actual description for this function>` is ignored and function `typeconv` is replaced by built-in function `SYN_UNSIGNED_TO_INTEGER` in `xeCompile`.

Usage is the same for the following:

```
built_in SYN_INTEGER_TO_UNSIGNED
built_in SYN_UNSIGNED_TO_INTEGER
built_in SYN_INTEGER_TO_SIGNED
built_in SYN_SIGNED_TO_INTEGER
built_in SYN_ZERO_EXTEND
```

The following are used to define the resolution methods which are not synthesizable.

```
resolution_method three_state  
resolution_method wired_and  
resolution_method wired_or
```

Use the following directive definitions:

```
translate_off / synthesis_off
```

Stops checking the input file from this point to the end of the file, or until finding a `translate_on` directive.

```
translate_on / synthesis_on
```

Continues checking the input file from this point to the end of the file, or until finding a `translate_off` directive.

Preserving Expected Behavior when Nets are Forced

The synthesis process might not preserve the expected behavior of some RTL signals when other signals are forced. For example, suppose the design contains the following line:

```
wire B = A;
```

Then, the synthesis process considers A and B equivalent.

If the design contains another signal depending on B, for example:

```
wire D = B & C;
```

Then, because A and B are considered equivalent, the synthesis process might use A instead:

```
wire D = A & C;
```

Here, A is more likely to be used instead of B if A is the output of a flip-flop.

With this implementation, if you force signal B, then, contrary to expectation, signal D is not affected.

To inform the synthesis process that a certain signal might be forced at run time, use the `keep_net` directive. The directive must appear after the declaration of the affected signal.

Here is a Verilog example:

```
wire A;  
// quickturn keep_net A
```

Here is a VHDL example:

```
signal A : bit;
```

```
-- quickturn keep_net A
```

The `keep_net` directive is effective for Verilog wires, and for VHDL signals and Verilog registers that are assigned only by non-blocking assignments. If a signal or register is assigned by multiple blocking assignments in an `always` block or process, multiple names might be used and the `keep_net` directive might not have the desired effect.

Preserving Signal Names

The synthesis process preserves module-level signal names, but it does not always preserve the signals declared in a local name scope such as a block or process. To preserve such signals, use the `preserve` directive. The directive must appear after the declaration of the affected signal.

Here is a Verilog example:

```
wire A;  
// quickturn preserve A
```

Here is a VHDL example:

```
signal A : bit;  
-- quickturn preserve A
```

Synthesizing Designs with Memories

All the memories not modeled as gates are modeled with UXE memory primitives. These primitives are MPR (Memory Port Read) and MPW (Memory Port Write). These primitives can be combined to create a memory with any number of read and write ports. You can also create read-before-write and write-through ports. Cadence recommends that you obtain assistance from the respective Field Application Engineer before using these primitives.

Note: These primitives cannot be simulated, only accelerated. So, a congruent simulation is not possible.

Memory synthesis allows you to synthesize a wide variety of memory constructs, including variable arrays, signal arrays, multi-level nested arrays, sliced indexes, and both level-sensitive and edge-sensitive write enables. To preserve the original RTL functionality, memory synthesis does the following:

- Automatically coordinates multiple sequential memory access timing.
- Places address offsets in the memory.

- Wraps extra “glue logic” around the synthesized memory definition, as needed. This extra logic ensures that the emulation memory more closely resembles the original memories, as defined in the RTL source code.
- Calculates appropriate memory size.
- Models level-sensitive and edge-sensitive write enables within the memory.
- Performs port sharing and memory merging, to reduce the number of ports and gates in the memory.

Using Memory Synthesis

By default, memory synthesis is turned off, and all the arrays at RTL are mapped to gates and flip-flops in the output netlist. For write-only memory, in the netlist, the HDL Compiler keeps only the memories that are declared at module level.

Note: Using xeCompile without memory synthesis for large arrays can cause gate count explosion.

Turn on memory synthesis by using the following XEL command:

```
hdlSynthesize -memory
```

Note: Alternatively, from the UNIX command-line interface, execute the `vaelab` utility that is comparable to the `hdlSynthesize -memory` XEL command. This utility enables you to synthesize the design from top module. For syntax and usage details of the `vaelab` utility, refer to [vaeLab](#).

Using Memory Synthesis Selectively

Use the `hdlSynthesize -memory<n>` command to notify xeCompile that all memories smaller than `<n>` bits (width x depth) should be mapped to gates and flip-flops. For example:

```
hdlSynthesize -memory256 TOP
```

Note: Notice that there is no space between `-memory` and `256`. The `-memory256` option informs xeCompile that any memory smaller than 256 bits (width x depth) should be mapped to gates and flip-flops. In certain cases, larger memories might also be mapped to gates and flip-flops, but definitely all memories smaller than 256 bits will be. If the `-memory` option is specified with no number, the default is 32.

In some designs, using `-memory<n>` with `<n>` larger or smaller than the default improves the emulation speed or capacity.

You can use `no_hardmacro` directive, `quickturn no_hardmacro <array_name>` to tell xeCompile to map a specific memory to gates and flip-flops. For example, in VHDL:

```
type mem_type is array(127 downto 0) of std_logic_vector(0 downto 0);
signal MemArray : mem_type;
-- quickturn no_hardmacro MemArray
```

or in Verilog:

```
reg [DATA_WIDTH-1:0] MemArray [((1<<ADR_WIDTH)-1):0];
// quickturn no_hardmacro MemArray
```

The opposite of `no_hardmacro` is `use_hardmacro`. The `use_hardmacro` directive tells xeCompile to map a specific memory to UXE memory primitives regardless of how small the memory is.

Modeling Verilog Memories

This section gives examples of memory RTL descriptions which xeCompile can synthesize to UXE memories. In these examples, each module contains only one memory. However, xeCompile can also synthesize multiple memories within a single module.

Asynchronous Verilog Memory

The following example illustrates an asynchronous memory with one read-write port:

```
module mem(ADDR, DI, WE, DO);
parameter ADR_WIDTH = 4;
parameter DATA_WIDTH = 32;
input [ADR_WIDTH-1:0] ADDR; // Address input
input [DATA_WIDTH-1:0] DI; // Data input
input WE; // Write enable
output [DATA_WIDTH-1:0] DO; // Data output

reg [DATA_WIDTH-1:0] MemArray [((1<<ADR_WIDTH)-1):0];

always @ (WE or ADDR or DI)
begin
    if(WE)
        MemArray[ADDR] <= DI;
    end
assign DO = MemArray[ADDR];
endmodule
```

Synchronous Verilog Memory

The following example illustrates a synchronous memory with two read ports and two write ports, all controlled by a single clock. In this example, the assignments to `MemArray[WAC]` and `MemArray[WAD]` are done in a single `always` statement to avoid ambiguity when ports

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

C and D write to the same address on the same rising edge of CLK. To avoid differences between emulation and simulation, avoid writing Memory RTL, whose behavior is ambiguous.

```
module mem(CLK, RAA, DOA, RAB, DOB, WAC, DIC, WEC, WAD, DID, WED);
parameter ADR_WIDTH = 4;
parameter DATA_WIDTH = 32;
input      CLK;      // Clock input
input  [ADR_WIDTH-1:0] RAA; // Address input, port A
output [DATA_WIDTH-1:0] DOA; // Data output, port A
input  [ADR_WIDTH-1:0] RAB; // Address input, port B
output [DATA_WIDTH-1:0] DOB; // Data output, port B
input  [ADR_WIDTH-1:0] WAC; // Address input, port C
input  [DATA_WIDTH-1:0] DIC; // Data input, port C
input      WEC; // Write enable, port C
input  [ADR_WIDTH-1:0] WAD; // Address input, port D
input  [DATA_WIDTH-1:0] DID; // Data input, port D
input      WED; // Write enable, port D

reg [DATA_WIDTH-1:0] MemArray [(1<<ADR_WIDTH)-1:0];
reg [ADR_WIDTH-1:0]  RegRAA;
reg [ADR_WIDTH-1:0]  RegRAB;

always @ (posedge CLK)
begin
    if(WEC)
        MemArray[WAC] <= DIC;
    if(WED)
        MemArray[WAD] <= DID;
end

always @ (posedge CLK)
begin
    RegRAA <= RAA;
end
assign DOA = MemArray[RegRAA];
always @ (posedge CLK)
begin
    RegRAB <= RAB;
end
assign DOB = MemArray[RegRAB];
endmodule
```

Synchronous Verilog Memory with Byte Write

The following example illustrates a synchronous memory with one read-write port. It has a byte-write input allowing individual 8-bit-wide bytes to be written. A similar approach can be used to model a bit-write input, or byte-write with any width byte.

```
module mem(CLK, ADDR, DI, WE, BW, DO);
input      CLK;      // Clock input
input  [3:0]  ADDR;   // Address input
input  [31:0] DI;     // Data input
input      WE;       // Write enable
input  [3:0]  BW;     // Byte write
output [31:0] DO;     // Data output

reg [31:0] MemArray [15:0];
wire [31:0] TmpData;
```

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

```
wire [31:0] DataIn;
reg  [3:0]  RegADDR;

assign TempData = MemArray[ADDR];
assign DataIn[31:24] = (BW[3]) ? DI[31:24] : TempData[31:24];
assign DataIn[23:16] = (BW[2]) ? DI[23:16] : TempData[23:16];
assign DataIn[15:8]  = (BW[1]) ? DI[15:8]  : TempData[15:8];
assign DataIn[7:0]   = (BW[0]) ? DI[7:0]   : TempData[7:0];

always @ (posedge CLK)
begin
    if(WE)
        MemArray[ADDR] <= DataIn;
    end

always @ (posedge CLK)
begin
    RegADDR <= ADDR;
    end
assign DO = MemArray[RegADDR];
endmodule
```

Modeling VHDL Memories

This section gives examples of memory RTL descriptions which xeCompile can synthesize to UXE memories. In these examples, each module contains only one memory. However, xeCompile can also synthesize multiple memories within a single module.

Asynchronous VHDL Memory

The following example illustrates an asynchronous memory with one read-write port.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;

entity mem is
    port (
        ADDR : in std_logic_vector(31 downto 0);
        DI   : in std_logic_vector(31 downto 0);
        WE   : in std_logic;
        DO   : out std_logic_vector(31 downto 0)
    );
end mem;

architecture mem_arch of mem is

    type mem_type is array(15 downto 0) of std_logic_vector(31 downto 0);
    signal MemArray : mem_type;

begin
    process( WE, ADDR, DI )
    begin
        if( WE = '1' ) then
```

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

```
        MemArray(conv_integer(ADDR)) <= DI;
    end if;
end process;
DO <= MemArray(conv_integer(ADDR));

end mem_arch;
```

Synchronous VHDL Memory

The following example illustrates a synchronous memory with two read ports and two write ports, all controlled by a single clock. In this example, the assignments to MemArray[WAC] and MemArray[WAD] are done in a single process, to avoid ambiguity when ports C and D write to the same address on the same rising edge of CLK. To avoid differences between emulation and simulation, avoid writing Memory RTL whose behavior is ambiguous.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;

entity mem is
    port (
        CLK : in std_logic;
        RAA : in std_logic_vector(3 downto 0);
        DOA : out std_logic_vector(31 downto 0);
        RAB : in std_logic_vector(3 downto 0);
        DOB : out std_logic_vector(31 downto 0);
        WAC : in std_logic_vector(3 downto 0);
        DIC : in std_logic_vector(31 downto 0);
        WEC : in std_logic;
        WAD : in std_logic_vector(3 downto 0);
        DID : in std_logic_vector(31 downto 0);
        WED : in std_logic
    );
end mem;

architecture mem_arch of mem is

    type mem_type is array(15 downto 0) of std_logic_vector(31 downto 0) ;
    signal MemArray : mem_type;
    signal RegRAA : std_logic_vector(3 downto 0) ;
    signal RegRAB : std_logic_vector(3 downto 0) ;

begin

    process( CLK )
    begin
        if( CLK'event and CLK = '1' ) then
            if( WEC = '1' ) then
                MemArray(conv_integer(WAC)) <= DIC;
            end if;
            if( WED = '1' ) then
                MemArray(conv_integer(WAD)) <= DID;
            end if;
        end if;
    end process;
```



```
process( CLK )
begin
    if( CLK'event and CLK = '1' ) then
        RegRAA <= RAA;
    end if;
end process;
DOA <= MemArray(conv_integer(RegRAA));

process( CLK )
begin
    if( CLK'event and CLK = '1' ) then
        RegRAB <= RAB;
    end if;
end process;
DOB <= MemArray(conv_integer(RegRAB));
end mem_arch;
```

Synchronous VHDL Memory with Byte Write

When a memory write operation does not write every bit in the data bus, the xeCompile memory synthesis uses a read-modify-write approach to implement it. An extra read port with the same address is created, if it does not already exist. Then, the multiplexers are synthesized for the individual bit-enable signal. This synthesis process helps in identifying the memory data input, which can be either the data input or the data output read from the read port.

When the width of a segment selected by each write-enable condition is not in multiples of 8, using the read-modify-write approach derives better results. Forcing the use of byte-select signals in the hardware can increase the data width of the memory. Therefore, when the width of each segment is in multiples of 8, xeCompile tries to utilize the new byte-select pins of the memories to save the capacity for an extra read port and additional multiplexers.

The following example illustrates a synchronous memory with one read-write port. It has a byte-write input allowing individual 8-bit-wide bytes to be written. A similar approach can be used to model a bit-write input, or byte-write with any width byte.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;

entity mem is
    port (
        CLK : in std_logic;
        ADDR : in std_logic_vector(31 downto 0);
        DI : in std_logic_vector(31 downto 0);
        WE : in std_logic;
        BW : in std_logic_vector(31 downto 0);
        DO : out std_logic_vector(31 downto 0)
    );
end mem;

architecture mem_arch of mem is
```

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

```
type mem_type is array(15 downto 0) of std_logic_vector(31 downto 0) ;
signal TmpData : std_logic_vector(31 downto 0) ;
signal DataIn : std_logic_vector(31 downto 0) ;
signal MemArray : mem_type;
signal RegADDR : std_logic_vector(3 downto 0) ;

begin

    TmpData <= MemArray(conv_integer(ADDR));
    DataIn(31 downto 24) <=
        DI(31 downto 24) when BW(3) = '1' else TmpData(31 downto 24);
    DataIn(23 downto 16) <=
        DI(23 downto 16) when BW(2) = '1' else TmpData(23 downto 16);
    DataIn(15 downto 8) <=
        DI(15 downto 8) when BW(1) = '1' else TmpData(15 downto 8);
    DataIn(7 downto 0) <=
        DI(7 downto 0) when BW(0) = '1' else TmpData(7 downto 0);

    process( CLK )
    begin
        if( CLK'event and CLK = '1' ) then
            if( WE = '1' ) then
                MemArray(conv_integer(ADDR)) <= DataIn;
            end if;
        end if;
    end process;

    process( CLK )
    begin
        if( CLK'event and CLK = '1' ) then
            RegADDR <= ADDR;
        end if;
    end process;
    DO <= MemArray(conv_integer(RegADDR));
end mem_arch;
```

Dual Port Memories with Two Clocks

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
entity rad_edge_sensitive is
    generic( DataW : integer := 16 ; -- word-width
             AddrW : integer := 16 );
    port( RD0 : out std_logic_vector(DataW - 1 downto 0);
          WD0 : in std_logic_vector(DataW - 1 downto 0);
          RA0 : in std_logic_vector(AddrW - 1 downto 0);
          WA0 : in std_logic_vector(AddrW - 1 downto 0);
          WE0 : in std_logic;
          RD1 : out std_logic_vector(DataW - 1 downto 0);
          WD1 : in std_logic_vector(DataW - 1 downto 0);
          RA1 : in std_logic_vector(AddrW - 1 downto 0);
          WA1 : in std_logic_vector(AddrW - 1 downto 0);
          WE1 : in std_logic );
END rad_edge_sensitive;
architecture ram of rad_edge_sensitive is
    type mem_type is array (AddrW-1 downto 0) of std_logic_vector(DataW-1 downto 0);
```

```
signal MemArray : mem_type;
begin
  process(WA0, WE0, WD0)
  begin
    if (WE0'event and WE0 = '1') then
      MemArray(conv_integer(WA0)) <= WD0;
    end if;
  end process;
  process(WA1, WE1, WD1)
  begin
    if (WE1'event and WE1 = '1') then
      MemArray(conv_integer(WA1)) <= WD1;
    end if;
  end process;
  RD0 <= MemArray(conv_integer(RA0));
  RD1 <= MemArray(conv_integer(RA1));
end ram;
```

Synthesizing Designs with Acceleratable Testbench (ATB) Constructs

The Palladium HDL Compiler synthesizes some testbench constructs that are not normally synthesizable. These are then placed on the emulator for faster verification.

Use Model for ATB

To synthesize a module with ATB constructs, import the module using `hdlImport` with the `-atb` option. The HDL Compiler will pass ATB-related data to the emulator run-time system.

An `hdlOutputFile -f qel <file>` command should be added before `hdlSynthesize` in the XEL compiler script. Alternatively, the `-outputQel <file>` option can be used with the `vaelab` command for the same purpose. This command instructs the HDL Compiler to output a file containing run-time XEL commands.

Compile the design for STB or static ICE. After downloading the design, issue the `hdlConfig` run-time XEL command immediately after the `configPM` command. This initializes registers and memories.

Example:

The following script illustrates how to compile some modules with ATB mode:

```
hdlInputFile modA.v
hdlImport -atb
hdlOutputFile -f verilog hdl_gate.v
hdlOutputFile -f qel hdl_gate.qel
```

```
hdlSynthesize top
```

The following script illustrates a typical compile and run-time script:

```
//Compile Script
=====
delimiterRule Verilog
compilerOption -add {visionMode FV}
compilerOption -add {mode ice}
refLib qtref
netlistFile {verilog hdl_gate.v}
designImport
design design_top top
emulatorConfiguration -add {host emu_host}
precompile
compile

//Run-time Script
=====
debug .
host emu_host
download
configPM -stb
hdlConfig hdl_gate.qel
run
```

Using hdlConfig Command at Run Time

The `hdlConfig` command is used during run time, right after the command `configPM -stb`. There are two possible syntax variants for this command. Choose only one syntax for the `hdlConfig` command; the two syntax styles cannot be combined.

First Syntax for hdlConfig

This syntax, with *<file>*, must be invoked before the first `run` command at emulation time, and it can be invoked only once in an entire run session.

```
hdlConfig <file> [-cycleTime=<time> ] [-top=<instance>] \
               [-displayDebug][+plusargs]
```

For detailed information on the above syntax, refer to the *hdlConfig for Initializing the System Tasks* section of the *hdlConfig* command description in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

For details related to configuring `$test$plusargs` and `$value$plusargs` using the `+plusargs` option mentioned above, refer to [\\$test\\$plusargs and \\$value\\$plusargs](#) on page 133.

Second Syntax for hdlConfig

This syntax, without `<file>`, must be invoked after executing the `hdlConfig` command using the first syntax. It can be invoked at any time during a run session, and can be invoked more than once if necessary.

```
hdlConfig [-disable {<instPath> | -all | -top }]
          [-enable {<instPath> | -all | -top }]
          [-disableBuffer ] [ -enableBuffer]
          [-stopCallBack <tclProc>] [-finishCallBack <tclProc>]
          [-timeOut <cycleNum> ]
```

For detailed information on the above syntax, refer to the *hdlConfig for Enabling or Disabling the Compile Options* section of the *hdlConfig* command description in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

For details related to configuring `$stopCallBack` and `$finishCallBack` using the `-stopCallBack` and `-finishCallBack` options mentioned above, refer to [\\$finish and \\$stop system tasks](#) on page 136.

Supported Testbench Constructs

The following constructs are supported in a testbench:

- Initial blocks
- Initial values
- [\\$readmemb and \\$readmemh](#)
- [\\$writememb and \\$writememh](#)
- [\\$display and \\$write](#)
- [\\$time System Functions](#) used as an argument of `$display` or `$write`
- [\\$test\\$plusargs and \\$value\\$plusargs](#)
- [force and release Statements](#)
- [repeat Statements](#)
- [report Statements](#)

■ \$finish and \$stop system tasks

\$readmemb and \$readmemh

Both `$readmemb` and `$readmemh` system tasks are supported to download memory content file into a memory. The HDL Compiler supports all the syntaxes in the standard:

```
$readmemb("<filename>", <memname> [ , <start_addr> [ , <finish_addr> ] ] );  
$readmemh("<filename>", <memname> [ , <start_addr> [ , <finish_addr> ] ] );
```

The third and fourth arguments are optional and can be either constants or any expressions. The unknown value (x) and the high impedance value (z) in the memory content file will be converted into 0. Both system tasks can appear in either initial blocks or always blocks. However, it is not allowed to read or write the same memory after a `$readmemb` or `$readmemh` within the same clock cycle in an initial or always block. There can be multiple `$readmemb` or `$readmemh` system tasks for the same memory.

\$writememb and \$writememh

`$writememb` and `$writememh` dump memory contents to a file. The output files are readable by `$readmemb` and `$readmemh`.

`$writememb` and `$writememh` do not support enumeration types in SystemVerilog, or arrays with more than two dimensions.

\$display and \$write

Note: For useful information on performance enhancements, refer to [Memory Buffer for \\$display and \\$write](#) on page 131.

The following format specifications are supported for `$display` and `$write`:

- %h
- %m
- %d
- %o
- %b
- %c
- %s

■ %t

■ %x

Note: %x is the functional equivalent of %h. It is not covered in IEEE standards, but is supported by popular Verilog simulators.

%m with \$display

%m is supported in \$display and \$write. %m shows the hierarchical path of the instance where the \$display or \$write is located. The IES simulator shows a hierarchical path starting with the top module, followed by the instances in the path. The HDL Compiler follows this format.

Example

\$display in instance u1.u2 under top module test:

```
$display("ERROR: assertion failed at %m.");
```

will show the following message at run time:

```
ERROR: assertion failed at test.u1.u2.
```

Memory Buffer for \$display and \$write

The execution of the hardware emulator is suspended during the process of displaying a message. When \$display occurs frequently, the accumulated time of small interruptions slows down emulator performance.

A memory buffer to store messages is applied to improve \$display and \$write performance. A message is not immediately displayed, but is stored in memory. The accumulated messages are then displayed together at a later time.

The memory buffer is not enabled by default. To enable the memory buffer to improve performance when there are frequent occurrences of \$display, add the following option to the hdlSynthesize command:

```
-displayBuffer[+depth=<num>][+timeOut=<num>][+port=<num>]
```

where:

[depth=<num>]:

The depth of the memory for \$display buffer. The default value is 8192 (8K). Increase the depth to improve the performance if \$display still occurs frequently. Decrease the

depth if there is a capacity limit. The value should be at most 2^{30} , otherwise an error message will be shown during `hdlSynthesize`.

`timeOut=<num>:`

The maximum number of emulation fast clock cycles per `$display` which can stay in the buffer. The default value is 2^{28} , or approximately 256 million cycles. If the fast clock is 500KHz, that means a `$display` message can be delayed up to 128 seconds. Without such a limit, when a message does not occur, you will not know whether it is buffered, or whether something is wrong to prevent it from occurring. A smaller value also helps to reduce the width of the memory buffer. Increase the value to improve performance: Decrease the value to reduce memory width. The range of the values is from 2^{20} to 2^{30} ; an error message will be shown during `hdlSynthesize` if an out-of-range value is given.

`port=<num>:`

Specifies the number of ports of the memory for `$display` buffer. The default value is 1. If the number of simultaneous `$display` exceeds the number of ports, the messages are displayed immediately without buffering. Increase the ports to reduce performance overhead caused by simultaneous `$display`. Increasing the ports can increase emulation capacity or step count. Turn on the `-displayDebug` option for the `hdlConfig` command to determine how many memory ports are optimal.

Typically, you do not need to change the default value. You can type:

```
hdlSynthesize -displayBuffer ...
```

The action of dumping the contents of the memory buffer to displayed messages is referred to as a flush. A flush will happen when:

- The memory buffer is almost full; that is, the number of messages stored is close to the depth of the memory.
- The time out limit is reached, as calculated by the number of fast clock cycles from the time the first message is stored in the memory.
- The `run <cycle>` command finishes with the specified number of cycles.
- A user-specified trigger stops the emulator.
- An emulator stops because of `$finish` or `$stop`.
- A software error is detected.
- The number of instances of `$display` occurring at the same time exceeds the number of memory buffer ports.

\$time System Function

By default, the HDL Compiler ignores the `$time` system function in a way such that timing constraints can be satisfied. For instance:

```
if (delay < $time)
```

is always evaluated to TRUE while

```
if (delay > $time)
```

is always evaluated to FALSE.

When the `-atb` option is specified, `$time` will still be ignored as usual, unless it appears in an argument of `$display` or `$write`. The value of `$time` will be shown by `$display` or `$write`. The precision of `$time` is at most the cycle time of the fast clock cycle, so it is possible that there might be a minor difference between emulator `$time` and IES `$time`. `$time` cannot be combined in an expression, as for example, `$time + 2`, as an argument in `$display` or `$write`. Otherwise, an error message will be shown during `hdlImport`.

\$test\$plusargs and \$value\$plusargs

Note: When using this argument, you must specify `-keepSourceless` when doing precompile, otherwise the ATB designs with `$value$plusargs` will potentially generate an error during `hdlConfig`.

`$test$plusargs` existed in the Verilog-1995 standard, and `$value$plusargs` was added into the Verilog-2001 standard. Both system functions provide capabilities similar to `define` and `ifdef` to control the conditional execution of Verilog code. However, to change the values for `define` and `ifdef`, you formerly needed to recompile your design, whereas for `$test$plusargs` and `$value$plusargs`, you are able to change the values to control the conditions during run time without recompiling the design.

When the `-atb` option is executed with the `hdlImport` command, both `$test$plusargs` and `$value$plusargs` are supported together with other system tasks. `$test$plusargs` and `$value$plusargs` can appear in any synthesizable expressions as well as any initial or always block. Typically, these two system functions appear in `if` expressions like:

```
if ($test$plusargs("debug_on"))
    debug_mode = 1;
else
    debug_mode = 0;
if ($value$plusargs("maxCycle=%d", maxc))
    else maxc = 500;
```

To change the argument value during run time, you can add `+<arg>` to the `hdlConfig` command in the same manner as you would specify such arguments for NC-Sim. The syntax of the `hdlConfig` command becomes:

```
hdlConfig <file> [ -cycleTime=%d][ -top=%s] [ +plusargs ]...
```

For example:

```
hdlConfig tb_init.qel +debug_on + maxCycle=500
```

The implementation of `$test$plusargs` and `$value$plusargs` introduces no performance overhead at run time.

force and release Statements

Both `force` and `release` statements are invoked when the `-atb` option is used with the `hdlImport` command. Both statements can appear in either initial blocks or always blocks.

When the value being forced is not a constant, the behavior of Verilog `force` is illustrated by this example:

```
force a = b;
```

When Verilog `force` is executed, the value of signal `a` will change whenever the value of signal `b` changes. In this case, the HDL Compiler follows the Verilog standard in implementing Verilog `force` statements.

When two `force` statements on the same signal are activated at the same simulation time sequentially, that is, in the same initial or always block, the first `force` statement is disabled. Similarly, when a `force` statement and a `release` statement are activated sequentially at the same time, the first statement is disabled.

When a `force` statement and a `release` statement are activated at the same time concurrently, that is, in different initial or always blocks, the `force` statement takes precedence, and the `release` statement is ignored.

When two `force` statements on the same signal are activated at the same time concurrently, the order cannot be predicted.

The implementation of `force` and `release` causes no performance overhead during run time.

repeat Statements

Verilog `repeat` statements are not considered to be synthesizable by industry standard. However, the HDL Compiler synthesizes `repeat` statements when the repeat count can be

calculated into a constant during compile time. This capability is enabled by default, whether or not the `-atb` option is used. An example using a `repeat` statement is:

```
repeat (10) @(posedge clk);
```

In this case, the HDL Compiler waits for 10 clock cycles before the next `repeat` statement starts.

There is no performance overhead for `repeat` statements.

report Statements

By default, the HDL Compiler ignores a `report` statement. To enable support for such statements, you need to specify the `-atb` option with either `vavhdl` or `hdlImport` command.

Note: A `report` statement used with an `assert` statement is ignored.

A VHDL `report` statement requires an expression of string type. You can use `image` VHDL predefined attribute to convert a variable value into a string and concatenate it to other strings using the `&` operator as illustrated in the following example:

```
report "counter = " & integer'image(counter);
```

You can also append a severity (`NOTE`, `WARNING`, `ERROR`, `FAILURE`) to a `report` statement as shown below:

```
report "counter = " & integer'image(counter) severity WARNING ;
```

A report message starts with the word `REPORT`, followed by severity, the fast cycle number of the Palladium XP run (like `$time` system function does when specified in a Verilog `$display` statement), the instance path (like `%m` returns when specified in a Verilog `$display` statement), and the value of the string expression in the VHDL `report` statement.

For example:

```
REPORT/NOTE (cycle: 100) from test.u1.u2: counter = 32
```

In the report message illustrated above, `counter = 32` is the string expression from the example `report` statement shown before it, that is
`"counter = " & integer'image(counter).`

The report messages are buffered like Verilog `$display` statements when the `-displayBuffer` option is specified with the `hdlConfig` command to maximize Palladium XP performance.

\$finish and \$stop system tasks

Both `$finish` and `$stop` system tasks are supported. They can be used in either initial blocks or always blocks. According to the Verilog standard, `$finish` and `$stop` take three possible values: 0, 1, or 2; if no argument is given, the default value is 1. The HDL Compiler treats all non-zero parameter values as 1 for `$finish` and `$stop`. The HDL Compiler follows the format of Verilog-XL for `$finish`.

Callback Functions for \$finish and \$stop

`$finish`

`$finish` is useful for pausing an emulation run to conduct offline debugging. Using `$finish` prevents the next test result from over-writing the current emulation snapshot. For example, when an error occurs, you might want to upload the waveform, or continue to run 1000 cycles to collect additional debug information. You can institute a callback function to execute these actions automatically.

`$stop`

When `$stop` causes simulation interruptions, you have the option of executing debugging commands or continuing the simulation run. A callback function automatically executed after `$stop` will allow you to execute some XEL commands whose functionality cannot be supported through Verilog code.

At run time, the callback functions for `$finish` and `$stop` can be specified using the following options of the `hdlConfig` command:

```
-finishCallBack <tclProc>
-stopCallBack <tclProc>
```

`<tclProc>` is a user-defined TCL procedure. If the TCL procedure accepts at least one argument, the instance path where `$stop` or `$finish` occurs will pass as the first argument to the callback function. If the TCL procedure accepts at least two arguments, the RTL module name where `$stop` or `$finish` occurs will pass as the second argument. You can use this information to identify where the `$stop` or `$finish` occurs in the callback function.

An example callback function is shown below:

```
hdlConfig tb_init.qel
proc myCallback { inst cell } {
    puts "Finish at instance $inst of cell $cell."
}
hdlConfig -finishCallBack myCallback
run 200000
```

For the usage details and syntax of the `hdlConfig` command, refer to the *hdlConfig* command description in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

axis_smem memory model support

The IXCOM compilation flow provides a non-standard method of modeling memories, called `axis_smem`. For details, refer to [Appendix J, “Support for axis_smem Memory Model.”](#)

Note: There is no need to turn on the `-atb` switch to support `axis_smem`. However, you are required to turn on memory synthesis in order to map `axis_smem` into MPW or MPR primitives. An error message will be displayed for `axis_smem` during synthesis if the memory synthesis option is not enabled using the `hdlSynthesize -memory` command.

Synthesizing Initial Blocks with Synthesizable Constructs

By default, the HDL Compiler ignores initial blocks and their contents, and provides warning messages. Using the `-atb` option, the HDL Compiler synthesizes initial blocks with synthesizable constructs. If the statements in an `initial` block are synthesizable when they appear in an `always` block, the `initial` block is synthesizable; otherwise the HDL Compiler reports error messages for those non-synthesizable constructs in the `initial` blocks.

The HDL Compiler utilizes its capability for Verilog-2001 constant functions to evaluate initial blocks and statically calculate initial values during compile time. Consider the following example:

```
module test (clk, d, p);
    input clk;
    input [0:98] d;
    output [2:100] p;
    reg [2:100] p;
    integer i;
    function isPrime;
        input [6:0] n;
        integer i;
        begin
            isPrime = 1;
            begin: block
                for (i = 2; i < n; i = i + 1) begin
                    if (i * i > n)
                        disable block;
                    if (n % i == 0) begin
```

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

```
        isPrime = 0;
        disable block;
    end
end
end
end
end
endfunction
initial
for (i = 2; i <= 100; i = i + 1)
    p[i] = isPrime(i);
always @(posedge clk)
    p <= d;
endmodule
```

Here, all the function calls, for-loops, if-statements, and intermediate variables used only in the initial block will disappear in the synthesized netlist, and either 0 or 1 will be assigned an initial value to each of p[2] to p[100]. When the initial block is ignored, signal “p” will be synthesized into instances of Q_FDP0I1 cell. With initial block synthesis, a Q_FDP0I1 cell will drive a net with initial value 1 and a Q_FDP0I0 will drive a signal with initial value 0. For the above example, the following instances can be found in the synthesized netlist.

```
Q_FDP0I1 U89 ( .CK(clk), .D(d[9]), .Q(p[11]), .QN( ));
Q_FDP0I0 U91 ( .CK(clk), .D(d[7]), .Q(p[9]), .QN( ));
```

Each net driven by cell Q_FDP0I1 will be appended to a list in the output of the `hdlOutputFile -f XEL` command, and all nets in the list will be set to 1 by the `signal -set` command when `hdlConfig` is invoked during run time. Because the initial values of flip-flops are 0's, there is no action required during the run time for Q_FDP0I0 cell. The HDL Compiler also supports Verilog-2001 initial values, such as:

```
reg q = 1;
```

Initial value support relies on the `signal -set` command to set the value of a net which is forcible. Consider the following example:

```
module test (clk, d1, en, a, z);
input clk, d1, d2, en;
input [2:0] a;
output [2:0] z;
reg [2:0] z;
initial z = 3'b10z;
always @(posedge clk) begin
    if (en)
        z = a;
    else
```

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

```
z = 3'bzzz;  
endmodule
```

In the above example, signal *z* will be synthesized into tristate buffers. A tristate buffer output is not directly forcible, but the HDL Compiler can initialize its value to 1 by setting both the enable pin and the data pin of the tristate buffer to 1, or initialize it to Z by setting its enable pin to 0. The following VHDL example demonstrates the same technique to initialize a signal *q* which is a wired OR of 2 flip-flops.

```
package pack_res_func1 is function wired_or ( input : bit_vector ) return bit  
; end ;  
package body pack_res_func1 is  
function wired_or ( input : bit_vector ) return bit is  
-- pragma resolution_method wired_or  
begin  
for i in input'range loop  
if ( input (i) ='1' ) then return '1' ; end if;  
end loop ;  
return '0' ;  
end ;  
end ;  
use work.pack_res_func1.all;  
entity test is port ( clk1, clk2, d1, d2 : bit; res_sig : out bit); end ;  
architecture rtl of test is  
signal q : wired_or bit := '1';  
begin  
process begin  
wait until clk1 = '1';  
q <= d1;  
end process;  
process begin  
wait until clk2 = '1';  
q <= d2;  
end process;  
res_sig <= q;  
end;
```

By default, the HDL Compiler optimizes out flip-flops whose inputs are constant, and preserves all the flip-flops when the `-keepAllFlipFlop` option is used. With the `-atb` option, a flip-flop with a constant input will be optimized only when there is no initial value for it or the initial value is the same as the constant input value. Consider the following example:

```
module test (clk, q1, q2);  
input clk;
```

```
output q1, q2;
reg q1 = 1;
reg q2 = 0;
always @(posedge clk) begin
    q1 = 0;
    q2 = 0;
end
endmodule
```

For the above example, neither the `-atb` nor the `-keepAllFlipFlop` option, the synthesized netlist will have both q1 and q2 driven by ground. With only the `-atb` option used, q1 will be driven by a Q_FDP0I1, while q2 will be driven by ground. Using only the `-keepAllFlipFlop` option, both q1 and q2 will be driven by Q_FDP0 cells. Using both the `-atb` option and the `-keepAllFlipFlop` option, q1 will be driven by a Q_FDP0I1 and q2 will be driven by a Q_FDP0I0. With the `-atb` option, the HDL Compiler can selectively disable flip-flop optimization and reduce the overhead of `-keepAllFlipFlop`.

When there are multiple `@(posedge clk)` in an initial block, the HDL Compiler will synthesize it as an implicit finite-state machine. For an implicit finite-state machine in an `always` block, the last state will go back to the first state. For an implicit finite-state machine in an initial block, the last state will go back to itself.

Initializing Registers and Memories

The following constructs are supported:

- Assignment in Verilog initial blocks.

```
reg q;
initial q = 1;
```

- Declaration assignment in Verilog -2001.

```
reg q = 1;
reg [0:4] q = 'b1101;
```

- Declaration assignment in VHDL

```
signal q: std_logic_vector(3 downto 0) := "1001";
```

- VHDL initial process: a process with no sensitivity list and with a `WAIT` statement as the last statement.

```
signal q : std_logic_vector(3 downto 0);
process begin
    q <= "1001";
    WAIT;
```


end

- All HDL synthesizable constructs, including for-loop, while-loop, if-then-else, functional calls, and all expressions are allowed in a Verilog initial block or a VHDL initial process when applicable.

Example

The following Verilog example demonstrates how `p[i]` is initialized to 1 when `i` is a prime number.

```
reg [2:100] p;
integer i;
function isPrime;
input [6:0] n;
integer i;

begin
    isPrime = 1;
    begin: block
        for (i = 2; i < n; i = i + 1) begin
            if (i * i > n)
                disable block;
            if (n % i == 0) begin
                isPrime = 0;
                disable block;
            end
        end
    end
end
endfunction
initial
    for (i = 2; i <= 100; i = i + 1)
        p[i] = isPrime(i);
```

Executing Distributed Synthesis during RTL Synthesis

The distributed synthesis feature of the HDL Compiler is useful for synthesizing VHDL or Verilog RTL designs containing tens of million gates. The feature enables distribution of the synthesis process into tasks at module level. The tasks are dynamically allocated during run time to balance the load across multiple workstations or processes. This feature helps to increase the run-time speed during the RTL design synthesis.

Distributing the Synthesis Process

Distributed synthesis can be performed using either of the following three ways:

- Using parallel.host File
- Using Multiple CPUs of the Same Workstation
- Using Load Sharing Facility (LSF) Queue

Using parallel.host File

Distributed synthesis can be performed by using the workstations listed in the `parallel.host` file, which is located in the design directory. Each line of this file contains the name of a workstation. The workstations can be of mixed platforms, for example Red Hat EL 4.0 running on an AMD Opteron workstation or SuSE Linux Enterprise Server 11 running on an Intel Xeon-EM64T workstation.

Execute either of the following commands to run distributed synthesis using the `parallel.host` file:

```
vaelab -parallelHost <top_module>
hdlSynthesize -parallel <top_module>
```



Tip

The `vaelab` command can be executed either from the UNIX or xeCompile command prompt. However, the `hdlSynthesize` command can be executed only from the xeCompile command prompt.

The tasks are distributed across all workstations on the network. Each workstation checks out the licenses. For example, if you have four licenses installed and ten workstations are listed in `parallel.host`, the distributed synthesis feature of the HDL Compiler will select only the first four workstations specified in the `parallel.host` file to perform synthesis.

Using Multiple CPUs of the Same Workstation

If a particular workstation has multiple CPUs, several tasks can be distributed to run in parallel on those CPUs.

To distribute a definite number of tasks on the current workstation, use either of the following two commands:

```
vaelab -parallel <number> <top_module>
hdlSynthesize -parallel<number> <top_module>
```

Note: In the `hdlSynthesize` command syntax given above, the `-parallel` and `<number>` arguments should be specified together without any space in between.

Using Load Sharing Facility (LSF) Queue

Distributed synthesis processes can also be submitted to an LSF queue using either of the following two commands:

```
vaelab -parallelLSF <number> <top_module>
hdlSynthesize -parallelLSF<number> <top_module>
```

Note: In the `hdlSynthesize` command syntax given above, the `-parallelLSF` and `<number>` arguments should be specified together without any space in between. The `<number>` is a required argument that should be greater than zero. It is used to specify the number of jobs to submit to the LSF queue.

To use the LSF queue for distributed synthesis, ensure that the LSF environment has already been set up such that the `bsub` executable is available in the `PATH` UNIX environment variable. If `bsub` is not available, the HDL Compiler will exit the process with an error message.

Using LSF_options File

Options to the `bsub` executable can be specified in a file called `LSF_options`. See the [LSF_options File](#) section for information on the contents of the `LSF_options` file.

If a file named `LSF_options` exists under the current working directory, the string will be appended to the `bsub` command when a job is submitted to an LSF queue. All the options must be defined in one line. The HDL Compiler does not check for syntax of these options.

The `LSF_options` file may exist in a location other than the current working directory. Define the full path to the `LSF_options` file using the `CDS_LSF_OPTIONS` environment variable. When you execute the `hdlSynthesize` or `vaelab` command, the current working directory is searched for the `LSF_options` file. If the file is not found there, the full path specified with the `CDS_LSF_OPTIONS` environment variable is opened as an `LSF_options` file.

Scheduling the Tasks

The tasks are scheduled based on the dependency among the modules. For example, a parameterized Verilog module, or a VHDL entity with generic values, will not be synthesized until the upper-level modules are synthesized.

A single netlist is generated by automatically collecting results from all workstations. The final synthesis result is the same as that synthesized by a single workstation. The HDL Compiler occasionally can produce slightly different gate counts on different machines due to differing optimization ordering.

Limitations of Distributed Synthesis

The distributed synthesis feature of the HDL Compiler does not support the following:

- Use of the `-atb` option with the `-displayBuffer` option of either the `hdlSynthesize` or `vaelab` command.
- Call the `hdlSynthesize` or `vaelab` command with the `ixcom` command. For example, the command `ixcom -vaelabargs "-parallel" file1.v ...` is not supported
- Signals or variables in packages
- SystemVerilog compilation unit scope

Generating a Debussy Database during Synthesis

You can generate the Debussy database for the Verilog or VHDL designs by one of the following methods:

- Using the `vaelab` executable
- Using the `hdlOutputFile XEL` command

When the `hdlOutputFile XEL` command is used, you need to specify the `-f debussy` option. This option automatically generates the Debussy `vericom` or `vhdlcom` script, as required.

Here is the syntax of the `hdlOutputFile` command to generate the script for Debussy tools:

```
hdlOutputFile -add -f debussy <file>
```

This command creates the following two text files and a shell script file:

- `<file>.libDebussy`: This text file contains all the logical library and physical library mapping information.
- `<file>`: This text file lists the design files according to their dependency order, and indicates the library to which these files should be imported. It also indicates whether you need to invoke the VHDL compiler or the Verilog compiler.

- `debussyRTLImport.sh`: This shell script file receives the list of design files generated above as the `<file>` text file. Use the following command on the UNIX prompt to generate the Debussy database:

```
% debussyRTLImport.sh <file> &
```

The `debussyRTLImport.sh` command compiles the Debussy database compilation using the required VHDL or Verilog compiler, and generates the `<file_name>.turbo.f` file. It physically maps the Debussy libraries to the same location as the library for the Palladium compilation database.

For example, consider that you executed the following `hdlDefineLib` command:

```
hdlDefineLib -add abc ./work/abc
```

Then, the Debussy library map is specified as `abc = ./work/abc` and the physical location is `./work/abc.lib++`.

Note: The library mapping for Debussy is identical to that of xeCompile. The Debussy libraries have a `.lib++` extension. The top module is imported in the current directory with the name `<filename>.lib++`.

The Debussy schematic can be opened using either of the following two methods:

- Using the RTL/Gate tab of the xeDebug tool:

Specify `<file_name>.turbo.f` in the Command Options File text box.

- Running the Debussy executable at the UNIX command prompt, by specifying the following command:

```
% verdi -f <file_name>.turbo.f &
```

Note: To invoke the Novas tools from either the xeDebug tool or the UNIX command prompt, user needs to set the `NOVAS_HOME`, `LM_LICENSE_FILE`, and `NOVAS_EXE` environment variables. For details about using Novas Tools with Palladium XP emulation databases, refer to [Chapter 20, “Using Novas Tools.”](#)

Using Conformal to Verify HDL Compiler-Generated Netlist

After synthesizing an RTL design, the HDL Compiler generates a netlist that can be used as an input for the emulation process. Before using this netlist, you can use Conformal to check the correctness of synthesis. This helps in detecting synthesis problems at an early stage of the verification process with a minimum of effort.

Note:

- ❑ To use Conformal, you must purchase its licenses from Cadence. Refer to the *Conformal Equivalence Checking User Guide* for installation and platform support of Conformal software.
- ❑ Palladium XP execution should be the primary methodology to validate the correctness of the RTL mapping process. Occasionally, a netlist is functionally correct on Palladium XP with respect to the RTL specification, but Conformal does not conclude its equivalence to the RTL design because of capacity limitations, different interpretation between ASIC implementation and/or verification models. Conformal can be a valuable assistant in diagnosing potential issues and problems only when Palladium XP fails to perform the expected RTL functionality.

To improve the accuracy of an equivalency check, if there are plans to use Conformal, ensure that the product uses the same optimizations and interpretations of the RTL as Palladium XP. Cadence suggests that you consult the Conformal User Guide before attempting to verify the netlist.

The process requires you to perform the steps discussed in the following sections:

- Generating Script for Conformal
- Starting Conformal
- Analyzing Results Using Conformal

Generating Script for Conformal

While synthesizing the design using the HDL Compiler, generate a script for Conformal by giving the following `hdlOutputFile` XEL command:

```
hdlOutputFile -add -f lec <lec_script_file_name>  
hdlOutputFile -add -f verilog <verilog_netlist_name>
```

These commands should be used before the `hdlSynthesize` XEL command.

The `-f lec` option specifies the creation of a single script file which contains the Conformal commands necessary to perform a hierarchical comparison between RTL and HDL Compiler output Verilog netlists. A hierarchical comparison is useful because:

- ❑ It reduces the complexity of equivalent checking and speeds up the verification process.
- ❑ When Conformal reports non-equivalent items, it is easier to identify the module that is non-equivalent.

Note: The script collects information from all source files used by all synthesized modules. This process is supported for both sequential and distributed synthesis.

Example: Conformal Script Generated by hdlOutputFile

```
set log file lec.do.log -replace -nobackup
set dofile abort exit
add ignore rtlcheck -all
set compare effort auto
set compare option -allgenlatch
set undefined cell black_box -both -noascend
set naming rule "_" -parameter -golden
set naming rule "." "" -field_delimiter -golden
set mapping method -nets -nobbox_name_match
set flatten model -latch_transparent
set datapath option -auto
read library -verilog -both $XEHOME/etc/hdlice_ref.v
read design -vhdl -golden -nosensitive tri.vhd -noelaborate
read design -verilog -golden -nosensitive test.v -root test
write design lec.do.bbox -golden -bbox_only -replace
read design -revised -verilog -nosensitive -root test lec.do.bbox test.vs
write hier_compare dofile lec.do.hier -replace -threshold 0
dofile lec.do.hier
exit -force
```

Starting Conformal

Conformal can be started from the design directory with the following command:

```
lec [-XL ][-nogui ]-dofile <lec_script_file_name>
```

Analyzing Results Using Conformal

The following sections discuss the considerations for analyzing HDL Compiler results using Conformal:

Improving Comparison Accuracy

The HDL Compiler optimizes out certain registers during synthesis. Therefore, Conformal might not be able to map all the key points and it might subsequently report false non-equivalence. To avoid this, Cadence recommends that the `-keepAllFlipFlop` option be

specified with the `hdlSynthesize` command. This option is automatically turned on and a warning message is issued if this option is not specified with the `-f lec` option. This option will slightly increase the gate count of the HDL output netlist.

To improve the Conformal mapping process and the accuracy of comparison, Cadence recommends turning on the `-keepRtlSymbol` option. This instructs the HDL Compiler to use Conformal-friendly names when it writes output Verilog netlists. This option is automatically turned on if the Conformal script generation option is present and a warning message displays.

Comparing Output with Memory Instances

When there are memories inferred from the RTL arrays, xeCompile uses UXE memory primitives (MPR/MPW) which cannot be understood by Conformal. Even with the simulation model that xeCompile created in the netlist, Conformal either takes a long time for large memories to finish or reports that modules are non-equivalent in many instances.

The generated script includes the Conformal commands to exclude all the modules inferring UXE memory primitives from the comparison:

```
add nottranslate modules -design -golden <RTL_memory_module_name>
add nottranslate modules -design -revised <Netlist_memory_module_name>
```

Comparing Output with Operators (Adder/Subtractor/Multiplier/Divider)

When synthesizing a design with above operators, Conformal performs better (in terms of accuracy and speed) when you use the following command option to invoke Conformal, if there is a Conformal Ultra license:

```
lec -XL
```

Without a Conformal Ultra license, the above command has no effect except to issue a warning. Cadence recommends that you acquire the license if the design uses these operators.

Omitting Skipping Modules from Equivalence Comparison

To skip certain modules for equivalence comparison (in order to avoid long verification times, and so on), specify the module names in a text file named `<lec_script_file_name>.moduleSkipEC`. Each module name should be on a separate line. The Conformal script created by HDL Compiler reads this file, and applies it to the modules to be skipped.

Customizing LEC Script File Using a Configuration File

You can customize the default LEC script that is generated by HDL Compiler by including a set of custom commands in a configuration file. The configuration file should be a text file with the name `<lec_script_file_name>.config`, and it should be located in the same directory as the original LEC script file.

In this file, a set of LEC commands can be included, with each command beginning on a new line. The original LEC script generated by HDL Compiler sources this configuration file, if it exists in the same directory containing the original LEC script.

For example this configuration file can be used to set the multi-threading feature in LEC, which is not set by default in the LEC script.

```
set parallel option -threads 4
```

The above command in the configuration file sets the maximum number of threads that you want to use to 4.

Conformal Log Files

Conformal generates the following files in the directory in which it is executed:

- ❑ Log file (`<lec_script_file_name>.log`)
- ❑ Intermediate do file
- ❑ File of blackbox definitions

These files are not removed after Conformal finishes. So, you can check the results and investigate further, if required.

Using UNIX Commands for RTL Import and Synthesis

The UNIX command-line interface supports a [Library Mapping File](#) called `va.lib` and three executable, `vavhdl`, `vavlog`, and `vaelab`. These provide a way to import and synthesize an RTL design without using XEL commands.

These executables are designed to provide UNIX command-line interface in syntax similar to IES. For syntax and usage details of the three executables, refer to:

- [vavhdl](#)
- [vavlog](#)

■ vaelab

Library Mapping File

A library mapping file is used to provide a mapping between the logical library names and the physical directory paths where those libraries are stored. This file should be saved with the name `va.lib`. Add the details to the `va.lib` file in the following format:

```
DEFINE <logical_lib_name> <physical_lib_path>
DEFINE WORK [<logical_lib_name> | <physical_lib_path>]
```

For example:

```
DEFINE siliconLib /nas/caenac/group/testcase/lib1
DEFINE rtlLib $TESTHOME/rtlLib
DEFINE cpulib ../rtlLibs/cpulib
DEFINE WORK cpulib
```

In the `va.lib` file, multiple logical library names can be mapped to the same physical location as illustrated in the following example:

```
DEFINE libA ./cpu_lib
DEFINE libB ./cpu_lib
DEFINE libC ./cpu_lib
DEFINE WORK ./cpulib
```

Note:

- ❑ The logical library names and the `DEFINE` command are case-insensitive.
- ❑ If the `hdlImport` and `hdlSynthesize` XEL commands are used, the library mapping is specified using the `hdlDefineLib` XEL command, which is similar to a `DEFINE` command in the `va.lib` file.

Setting Variables in va.lib Files

The `set` command can be used to set the local variables. Use of `$var` or `${var}` indicates variable substitution. For example:

```
set libpath nas/caenac/group/testcase/allLibs
define cpuLib $libpath/cpuLib
```

There are three pre-defined variables: `vavlogopts`, `vavhdlopts`, and `vaelabopts`. The value of `vavlogopts` is appended to all vavlog commands. Similarly, the values of `vavhdlopts` and `vaelabopts` are appended to vavhdl and vaelab commands, respectively. For example:

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

```
set reflibs -reflib "libA libB libC"
set worklib -work work2
set vavlogopts $reflib $WORKLIB
set vaelabopts $WORKLIB
```

Note: If more than one pre-defined variables are set (as shown in the above example), only the last one takes effect.

Environment variables can also be used to specify the physical library paths, such as `$TESTHOME/rtlLib`. The following search sequence is followed:

- First, the variables already defined in the `va.lib` file are searched.
- Then, the environment variables are searched.

Note: Variable names defined in the `va.lib` file are case-insensitive, but environment variables are case-sensitive.

Additional Statements Supported in va.lib Files

In the `va.lib` file, the following statements can also be given:

- `UNDEFINE <logical_library_name>`

Undefines the logical library name. No error is issued if the logical library is not previously defined.

For example:

```
UNDEFINE cpulib
```

- `INCLUDE <path>`

Include the content of the file specified. Issue an error if the file is not found.

For example:

```
INCLUDE /global/group/va.lib
```

- `SOFTINCLUDE <path>`

Include the content of the file specified. No error is issued if the file is not found.

For example:

```
SOFTINCLUDE /global/group/va.lib
```

Other Syntax Rules Supported in va.lib Files

The following syntax rules are also supported in a `va.lib` file:

- Use a backslash (\) to extend a statement to the next line.
- Use a pound sign (#) or double hyphens (--) to begin a comment.

For example:

```
# This is a comment.  
Define cpuLib ./myCPU    -- This is a comment.
```

Specifying va.lib File to Palladium XP

By default, the `vavhdl`, `vavlog`, or `vaelab` executable looks for the `va.lib` file under the current directory. The `va.lib` file can also be specified using the `-valib` option on the UNIX command line.

For example:

```
vavlog -valib /project/va.lib.test file.v
```

If the `-valib` option is not specified and there is no `va.lib` file under the current directory, `$HOME/va.lib` is used if the `HOME` environment variable is defined and the `va.lib` file exists under the specified directory.

If the `va.lib` file is not found anywhere, the following is assumed:

```
DEFINE WORK ./work
```

This means that the default library is called `work` and is placed in the `./work` directory.

vavhdl

Imports the VHDL source files. Though giving one `vavhdl` command per file is supported, Cadence recommends importing of all VHDL source files together using a single `vavhdl` command, if possible, as illustrated below:

```
vavhdl -work worklib file1.vhd file2.vhd file3.vhd file4.vhd
```

Invoking the `vavhdl` command several times to run `vavhdl` once on each VHDL file slows down the import time and reduces the reliability of some features, such as hierarchical net reference support.

The syntax for this command is:

```
vavhdl [options] <source_file>
```

The `[options]` parameter in the above syntax can have the following values:

-append_log

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

-atb
-ErrorMax <arg>
-file <arg>
-full
-help
-Ignore Pragma <arg>
-lock
-logFile <log_file>
-refLib <arg>
-rm Synth Prefix <arg>
-synth Prefix <arg>
-vaLib <arg>
-V1987
-work <arg>

Example

```
vavhdl -work lib2 file3.vhd file4.vhd
```

The above command imports VHDL files `file3.vhd` and `file4.vhd` into library `lib2`.

The following table describes the options of the `vavhdl` command:

vavhdl Command Option	Option Description
<code>-append_log</code>	Appends log to an existing log file. When <code>-logFile</code> is not present, it appends to <code>vavhdl.log</code> . When <code>-logFile <log_file></code> is present, it appends to the specified <code><log_file></code> .
<code>-atb</code>	For a list of supported constructs with the <code>-atb</code> option refer to the Synthesizing Designs with Acceleratable Testbench (ATB) Constructs section in the Chapter 3, "Compiling Designs for In-Circuit Emulation with xeCompile."
<code>-ErrorMax <arg></code>	Specifies the maximum number of errors processed.
<code>-file <arg></code>	Specifies a file for additional command-line options. Use of <code>//</code> comments out the rest of the line in the file.

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

vavhdl Command Option	Option Description
<code>-full</code>	Imports all files even if not edited since last <code>vavhdl</code> command.
<code>-help</code>	Prints the usage information.
<code>-Ignore_Pragma <arg></code>	Ignores the pragmas that exist in the specified modules. Multiple modules can be specified with + (plus sign) used between each module name for concatenation.
<code>-lock</code>	Turns on library locking for concurrent import sessions.
<code>-logFile <log_file></code>	Specifies a log file name. If not specified, <code>vavhdl.log</code> is used. This command is comparable to the <code>hdlLogFile <log_file></code> XEL command. This overwrites the log file if exists; otherwise, creates it.
<code>-refLib <arg></code>	Specifies the library in which to search for undefined cells. This is comparable to XEL command <code>hdlImport -l</code> .
<code>-rm_Synth_Prefix <arg></code>	Removes the specified pragma for the synthesis directives from the default pragma list. The default pragmas are <code>pragma</code> , <code>synopsys</code> and <code>quickturn</code> .
<code>-synth_Prefix <arg></code>	Appends additional pragmas for the synthesis directives to the default pragma list.
<code>-vaLib <arg></code>	Specifies the <code>va.lib</code> file to be used. If not specified, the program searches under the current working directory; then, searches under the home directory.
<code>-V1987</code>	Turns off VHDL-1993 keywords and warnings.
<code>-work <arg></code>	Specifies the current working library, <code>WORK</code> . If not specified, use <code>WORK</code> as defined in <code>va.lib</code> . If <code>WORK</code> is not defined in <code>va.lib</code> either, use <code>work</code> .

vavlog

Imports the Verilog source files. This utility is comparable to the `hdlImport -2001` XEL command. Though giving one `vavlog` command per file is supported, Cadence recommends all of the Verilog source files should be imported together using a single `vavlog` command, if possible, as illustrated below:

```
vavlog -work worklib top.v
```

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

Invoking the `vavlog` command several times to run `vavlog` once on each Verilog file slows down the import time and reduces the reliability of some features, such as hierarchical net reference support.

The syntax for this command is:

```
vavlog [options] <source_file>
```

The [options] parameter in the above syntax can have the following values:

- append Log
- atb
- define <arg>
- ErrorMax <arg>
- file <arg>
- full
- help
- incDir <path>
- libDir <directory>
- libExt <ext>
- libFile <filename>
- logFile <log_file>
- refLib <arg>
- rm Synth Prefix <arg>
- sv
- synth Prefix <arg>
- upCase
- v1995
- vaLib <arg>
- work <arg>

Example

```
vavlog -work lib1 file1.v file2.v
```

The above command imports Verilog files `file1.v` and `file2.v` into library `lib1`.

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

The following table describes the options of the `vavlog` command.

vavlog Command Option	Option Description
<code>-append_Log</code>	Appends log to an existing log file. When <code>-logFile</code> is not present, it appends to <code>vavlog.log</code> . When <code>-logFile <log_file></code> is present, it appends to <code><log_file></code> .
<code>-atb</code>	For a list of supported constructs with the <code>-atb</code> option refer to the Synthesizing Designs with Acceleratable Testbench (ATB) Constructs section.
<code>-define <arg></code>	Defines a macro. This is comparable to the <code>hdlInputFile +define+<var1></code> XEL command.
<code>-ErrorMax <arg></code>	Specifies the maximum number of errors processed.
<code>-file <arg></code>	Specifies a file for additional command-line options. Use of <code>//</code> comments out the rest of the line in the file.
<code>-full</code>	Imports all files even if not edited since last <code>vavlog</code> command.
<code>-help</code>	Prints the usage information.
<code>-incDir <path></code>	Specifies path to an included directory. This is comparable to the <code>hdlInputFile +incdir+<path></code> XEL command.
<code>-libDir <directory></code>	Specifies a library directory. This is comparable to the <code>hdlInputFile -y <directory></code> XEL command.
<code>-libExt <ext></code>	Specifies the file extensions to be used when searching library directories. This is comparable to the <code>hdlInputFile +libext+<ext></code> XEL command.
<code>-libFile <filename></code>	Specifies a library file. This is comparable to the <code>hdlInputFile -v <filename></code> XEL command.
<code>-logFile <log_file></code>	Specifies a log file name. If not specified, <code>vavlog.log</code> is used. This command is comparable to <code>hdlLogFile <log_file></code> XEL command. This overwrites the log file, if it exists; otherwise, creates it.
<code>-refLib <arg></code>	Specifies the library in which to search for undefined cells. This is comparable to the <code>hdlImport -l</code> XEL command.
<code>-rm_Synth_Prefix <arg></code>	Removes the specified pragma for the synthesis directives from the default pragma list. The default pragmas are <code>pragma</code> , <code>synopsys</code> , and <code>quickturn</code> .

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

vavlog Command Option	Option Description
<code>-sv</code>	Turns on the SystemVerilog support.
<code>-synth_Prefix <arg></code>	Appends additional pragmas for the synthesis directives to the default pragma list.
<code>-upCase</code>	Changes all verilog names to uppercase. This is comparable to <code>hdlInputFile -u XEL</code> command.
<code>-v1995</code>	Turns off Verilog -2001 support. By default, Verilog -2001 is supported.
<code>-vaLib <arg></code>	Specifies the <code>va.lib</code> file to be used.
<code>-work <arg></code>	Specifies the current working library, <code>WORK</code> . If not specified, use <code>WORK</code> as defined in <code>va.lib</code> . If <code>WORK</code> is not defined in <code>va.lib</code> either, use <code>work</code> .

Verilog-XL

The following Verilog-XL options are also supported:

Table 3-1 Verilog-XL Commands

Command	Description
<code>+define+<var1>+...</code>	Define macros.
<code>-f <commandFile></code>	Specify a file for command arguments.
<code>+incdir+<path1>+...</code>	Specify paths for searching include files.
<code>+libext+<ext1>+...</code>	Specify library directory file extensions.
<code>-log -l <logFile></code>	Specify a file for log information.
<code>+librescan +liborder</code>	Specify library scan precedence.
<code>-u</code>	The analyzer will be case insensitive.
<code>-v <filename></code>	Specify a library file.
<code>-y <directory></code>	Specify a library directory.

vaelab

Synthesizes the design from top module. This utility is comparable to the following XEL command:

```
hdlSynthesize -memory -keepVhdlCase -keepRtlSymbol
```

Note: The `hdlSynthesize -d` command option for overriding parameter values does not support sized Verilog values like `1'b0`. This is important because the width of the parameter can be significant in sized arithmetic and concatenation expression. Similarly, the `-generic <arg>` option does not support sized Verilog parameters.

The syntax for this command is:

```
vaelab [ <topModule> [ -rename <arg>] | -all | -only <arg> ] [options]
```

The [options] parameter in the above syntax can have the following values:

[-alu | -MinALUSize <arg>]

-append Log

-append Output

-conformal <arg>

-debugHardmacro

-debussy <filename>

-displayBuffer

-displayDepth <arg>

-displayTimeOut <arg>

-displayPort <arg>

-dontStopOnError

-fanoutNetOnly

-file <arg>

-generic <arg>

-help

-keepAllFlipFlop

-legacyName

-logFile <log_file>

-minMemSize <arg>

-moduleSuffix <mysuffix>

-noMemory

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

-noNamePreservation
-outputOel <arg>
-outputVlog <arg>
[-parallel <number> | -parallelHost | -parallelLSF <number>]
-portNaming <arg>
-refLib <arg>
-skipModule <arg>
-vaLib <arg>
-vhdl2LCase
-vhdl2UCase
-work <arg>

Examples

- To synthesize the design from `top` module, use the following command:

```
vaelab testbench -outputvlog testbench.ver
```
- To overwrite the default VHDL generic or Verilog parameter feedback of the specified instance, use the following set of commands. The hierarchical path to the generic can be specified in either VHDL style with a colon (:) or Verilog style with a dot (.).

```
vaelab top -generic :sub0:feedback=30 -outputvlog top.ver  
vaelab top -generic top.sub0.feedback=30 -outputvlog top.ver  
vaelab top -generic ":qrbs_glue_1:genQRBSV(4):QRBS:feedback=555" \  
-outputvlog top.ver
```

The following table describes the options of the `vaelab` command.

vaelab Command Option	Option Description
-all	Compiles all modules in <code>WORK</code> library. Used when compiling all cells in a library.
-alu	Switches on ALU mapping with the default ALU threshold of 32.
-MinALUSize <arg>	Switches on ALU mapping with the ALU threshold as represented by <arg>.

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

vaelab Command Option	Option Description
<code>-append_Log</code>	Appends log to an existing logfile. When <code>-logFile</code> is not present, it appends to <code>vaelab.log</code> . When <code>-logFile <log_file></code> is present, it appends to <code><log_file></code> .
<code>-append_Output</code>	Appends to the HDL output file, rather than overwriting it. This is comparable to the <code>hdlSynthesize -append XEL</code> command.
<code>-conformal <arg></code>	Specifies the Conformal script file to be generated.
<code>-debugHardmacro</code>	User-friendly net names for hardmacro connections for easier debugging.
<code>-debussy <filename></code>	Creates the following files: ■ <code>debussyRTLImport.sh <filename></code> ■ <code><filename>.libDebussy</code>
<code>-displayBuffer</code>	Buffers <code>\$display</code> and <code>\$write</code> messages to improve emulator performance at run time.
<code>-displayDepth <arg></code>	Specifies the depth of the memory for the <code>\$display</code> buffer.
<code>-displayTimeOut <arg></code>	Specifies the maximum number of Palladium fast clock cycles a <code>\$display</code> message can stay in the buffer.
<code>-displayPort <arg></code>	Specifies the depth of the memory for the <code>\$display</code> buffer
<code>-dontStopOnError</code>	Specifies that synthesis must not be stopped even when the first module with an error is found.
<code>-fanoutNetOnly</code>	Only preserves names of nets connected to output ports to reduce the gate count.
<code>-file <arg></code>	Specifies a file for additional command-line options. Use <code>//</code> to comment out the rest of the line in the file.
<code>-generic <arg></code>	Assigns or replaces the default value of VHDL generics or Verilog parameters of the top module or at any level of the hierarchy. This is comparable to the <code>hdlSynthesize -D XEL</code> command. The <code>-Generic <arg></code> option does not support sized Verilog parameters.
<code>-help</code>	Prints the usage information.
<code>-keepAllFlipFlop</code>	Preserves all flip-flops.
<code>-legacyName</code>	Use the default naming rule in IXE5.1 or earlier.

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

vaelab Command Option	Option Description
<code>-logFile <log_file></code>	Specifies a log file name. If not specified, <code>vaelab.log</code> is used. This command is comparable to the <code>hdlLogFile <log_file></code> XEL command. This overwrites the log file if exists; otherwise, creates it.
<code>-minMemSize <arg></code>	Specifies the minimum size (number of bits) of the 2D array to be synthesized into MPR or MPW.
<code>-moduleSuffix <mysuffix></code>	This suffix appends to the end of the module name and may contain letters and digits. This is comparable to the <code>hdlSynthesize -moduleSuffix <mysuffix></code> XEL command.
<code>-noMemory</code>	Memories are implemented using flip-flops, gates and latches, instead of hard macros. This is comparable to the <code>hdlSynthesize</code> XEL command without the <code>-memory</code> option.
<code>-noNamePreservation</code>	Do not preserve names to reduce the gate count.
<code>-only <arg></code>	Synthesizes only the specified module.
<code>-outputQel <arg></code>	Specifies the filename of XEL commands for ATB mode, to be loaded at run time with the <code>hdlConfig</code> command.
<code>-outputVlog <arg></code>	Specifies a filename for the gate-level Verilog HDL output.
<code>-parallel <number></code>	Distributes the specified number of processes on the current machine.
<code>-parallelHost</code>	Distributes processes over hosts listed in the <code>parallel.host</code> file.
<code>-parallelLSF <number></code>	Distributes the specified number of processes using LSF.
<code>-portNaming <arg></code>	Uses the given pattern to name vectored ports. The default pattern is <code>%s[%d]</code> .
<code>-refLib <arg></code>	Specifies the library in which to search for undefined cells.
<code>-rename <arg></code>	Replaces the top module name in the HDL output files with the specified <code><arg></code> . This command structure is similar to the structure of the <code>hdlSynthesize -rename <arg></code> XEL command. Refer to the <i>HDL Compiler Commands</i> chapter of the <i>UXE Command Reference Manual</i> for more details.

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

vaelab Command Option	Option Description
<code>-skipModule <arg></code>	Specifies the module which is not to be synthesized. This is comparable to <code>hdlSkipModule <arg></code> XEL command. To specify a module within a library other than working library, prepend the library name to module name. For example, <code>lib.mod</code> .
<code>-vaLib <arg></code>	Specifies the <code>va.lib</code> file to be used. If not specified, the program searches under the current working directory; then searches under the home directory.
<code>-vhdl2LCase</code>	Converts all VHDL names to lowercase. This command option is comparable to the <code>hdlSynthesize -vhdlToLowerCase</code> XEL command.
<code>-vhdl2UCase</code>	Converts all VHDL names to uppercase. This command option is comparable to the <code>hdlSynthesize</code> XEL command without <code>-vhdlKeepVhdlCase</code> or <code>-vhdlToLowerCase</code> options.
<code>-work <arg></code>	Specifies the current working library, <code>WORK</code> . If not specified, use <code>WORK</code> as defined in <code>va.lib</code> . If <code>WORK</code> is not defined in <code>va.lib</code> either, use <code>work</code> .

Importing the Design

The ICE compilation process is initiated by importing the designs. The design methodology identifies the form of design import that must be used, that is, either netlist or RTL format. The following forms of design import are supported:

■ Register Transfer Level (RTL) Import

Import an HDL design in RTL Verilog or VHDL format, then synthesize it into a gate-level netlist for the design to be passed for compilation. The following sections of this chapter cover the information related to specifying RTL design for import:

- ❑ [“Specifying RTL Design Files for Import”](#) on page 104
 - [“Specifying Verilog Designs for Import”](#) on page 105
 - [“Specifying VHDL Designs for Import”](#) on page 107

Then, the specified RTL design must be synthesized before it can be used in xeCompile. Refer to [Synthesizing the RTL Design for Emulation](#) on page 110 for more details. Cadence recommends using XEL commands to specify and synthesize the RTL designs. For command syntax and usage details, refer to the *HDL Compiler Commands* chapter in the *UXE Command Reference Manual*.

■ Netlist Import

Designs described in structural Verilog netlist formats can be imported directly without synthesis. Designs described in a combination of HDL and gate-level source can also be imported directly. For detailed information on the netlist import process, refer to [Importing a Netlist](#).

- ❑ Import an existing gate-level netlist in Structural Verilog Hardware Description Language format.
- ❑ Pattern—Lists cell name patterns to create as empty cells for later definition, such as for memories or physical IP logic circuits.

Cadence recommends using XEL commands to import a netlist. For command syntax and usage details, refer to the *Import Commands* chapter in the *UXE Command Reference Manual*.

You need to keep a few considerations in mind when writing a Verilog or VHDL design to be compiled for emulation. These considerations and guidelines for modeling the Verilog and VHDL designs are described in [Appendix B, “Modeling Guide for Verilog Designs for ICE Mode”](#) and [Appendix C, “Modeling Guide for VHDL Designs for ICE Mode.”](#)

Importing a Netlist

A netlist file defines the internal, gate-level logic within each logic block of the system that needs to be emulated. The information includes definitions of cells, instances, terminals, nets, and connections.

The gate-level netlist defines all cells in the design, regardless of from where each cell definition originated. The netlist must also define the input and output ports for all memory cells in the netlist, either by using explicit cell definitions or by references to matching patterns in pattern files. The netlist import process uses these port definitions to generate empty cells for each memory instance.

Perform the following steps to import a netlist:

1. (Optional) Specify all input options for importing the design netlists using the `importOption` command. For command syntax details, refer to the *Import Commands* chapter of the *UXE Command Reference Manual*.

- a. Specify whether to incrementally or fully import the netlists by using the following option of the `importOption` command:

```
importOption {mode incr | full}
```

By default, netlist files are imported incrementally. As the netlist importer reads each cell definition in the netlist, it searches for a previous version of the netlist, and compares the new and old versions. It modifies only the cells that are different than the previous version, and adds new cells to the design directory. This makes incremental import faster than a full import because files need not be re-read and compiled unless it is necessary.

When importing a netlist for the first time, or to make sure the design database does not contain previously imported netlist files (for instance, if some parts of the netlist were deleted), specify the value of the `mode` option as `full` to fully import the netlist. The full import process overwrites previously-imported cells with the same names when the netlist is imported, even if the cell definitions have not changed.

- b. Specify the name of the target design library in which the netlist must be imported by using the following option of the `importOption` command:

```
importOption {library <lib>}
```

The netlist import process converts the netlists into a binary format, and places them in the defined *design library*. Several netlists can be imported into the same design library, or import each netlist into a different design library.

If neither you nor the netlist specifies a design library name, the netlist importer uses the default design library name, that is, `QT_design_top`.

- c. Execute the `importOption {compatibility sl|ns|ci|cs|cd|ir}` command for setting non-standard import options, such as whether or not to overwrite existing cell definitions or override the default case sensitivity or insensitivity setting for the language.
2. (Optional) Specify one or more ASIC or design library names by executing the `refLib` command. The syntax for this command is:

```
reflib <library_name>
```

Note: All ASIC libraries must be present in, or linked into, the `<design>/QTDB` directory.

The netlist importer searches these libraries for the cell definitions to correctly interpret the cells in the imported netlists. In addition, by default, the netlist importer searches the `qtref` library for definitions of the cell primitives that none of the netlists define. If one or more cells are not defined in any of the libraries, then the netlist importer generates an error.

3. Specify the name of the gate-level netlist file that needs to be imported into the QTDB database by executing the `netlistFile` command. The syntax of the command is as following:

```
netlistFile {<file_type> <netlist_filename>}
```

Netlist files can be either pattern files, netlists of a specific format, or attribute files.

At least one `netlistFile` command must be executed before initiating the netlist import process. You can execute multiple `netlistFile` commands, but those must be followed by a `designImport` command.

Note: If the netlist importer should explicitly exclude any modules during import, create a file containing a list of such modules and specify the name of that file using the `netlistFile {exclude <filename>}` command. Consequently, the Verilog parser excludes the modules listed in the specified `exclude` file, and neither the module's definition nor its instances are imported into the database.

4. Execute the `designImport` command to import the pattern and netlist files.

After the design netlist import has finished successfully, the memory cell patterns supplied by the ASIC libraries are used to generate memory cell definitions and map the design terminals to them. For most designs, memory cells are automatically configured based on the memory instance names defined in the netlist. However, if desired, memory cells can also be manually configured using a user-defined memory or generic pattern.

Using Parallel Import

By default, the netlist import distributes itself in multiple threads on the same host, if it can. Optionally, it can distribute itself on multiple hosts through LSF.

Normally, the `designImport` XEL command invokes the import process in *parallel import* mode. To disable this, use the `designImport -noparallel` command.

Also, the `designImport` XEL command invokes the import process without LSF. To enable multi-host parallel import with LSF, use the `designImport -useLSF` command. However, to use this feature, LSF must be present and appropriately configured.

To enable parallel import for a collection of netlists, make sure that the collection can be partitioned into groups that do not depend on each other. This means, netlists that contribute to the creation of the same library must be a part of the same group. In addition, netlists that consume that library (through `refLib`) must also be a part of that group, and any libraries that they help create must extend the group. Import automatically separates groups, if they are separable, and imports them in parallel.

For example, consider an XEL script containing the following commands:

```
refLib top1 qtref generic
importOption {library top2}
netlistFile {verilog n2.v}
```

```
refLib qtref generic
importOption {library top0}
netlistFile {verilog n0a.v}
```

```
importOption {library top0}
netlistFile {verilog n0b.v}
```

```
importOption {library top1}
netlistFile {verilog n1.v}
```

```
refLib top2 qtref generic
importOption {library top3}
netlistFile {verilog n3.v}
```

```
designImport
```

The above XEL script file allows import to use two parallel processes: one for netlists `n0a.v` and `n0b.v`, and another for netlists `n1.v`, `n2.v`, and `n3.v`.

The first group (`n0a.v` and `n0b.v`) results from both netlists being imported into the same library (`top0`).

The second group (`n1.v`, `n2.v`, and `n3.v`) results from netlists using libraries created within the group, such that:

- `n2.v` relies on `top1` library, which is created by `n1.v` netlist
- `n3.v` relies on `top2` library, which is created by `n2.v` netlist

Generally, most designs cannot be easily enabled for parallel import. Therefore, care must be taken to yield a design that can support parallel import.

By default, import supports a maximum of 16 parallel invocations, either on same-host threads or multi-host LSF jobs. This can be altered, up or down, by setting the following environment variable:

```
setenv IMPORT_MAX_PARALLEL <value>
```

If more than the maximum number of parallel invocations are needed, they are executed serially in sets of up to the specified maximum number.

By default, import uses the default LSF queue, established by the LSF administrator. This can be altered by setting the following environment variable:

```
setenv IMPORT_LSF_QUEUE <value>
```

By default, import has a time-out of 18000 seconds (that is, five hours) for an import job assigned by LSF. This can be altered by setting the following environment variable:

```
setenv IMPORT_LSF_TIMEOUT <value>
```

Compiling the Design

After importing and synthesizing the design, the next step in the ICE compilation process involves setting up of the design files to prepare for the compile process. This involves automatic processes that check design rules, generate logic by processes such as loop breaking and synthesis of flip-flops and latches, generating side files, and adding instrumentation logic. This affects behaviors in how the signals are handled relative to the standard fast clock (FCLK).

The basic steps performed are:

- The nets and primitives are initialized.
- The netlist is flattened.
- Design rule check and data qualification is performed.
- Logic transformation is performed on the design.
- Asynchronous loops are broken.
- Flops and latches are synthesized.
- FullVision probes are set.
- Side files are generated.

Files generated at the end of the precompile process include:

- `PDB/<lib>.<cellname>.loops` - Reports complete asynchronous loops in the design.
- `PDB/<lib>.<cellname>.brk` - Reports where asynchronous loops are broken.
- `PDB/<lib>.<cellname>.MultiClock` - Reports the gate count after precompile has run.
- `PDB/<lib>.<cellname>.rpt` - Reports error and warning messages from the design rule-check phase.

This section covers the following topics that detail the intermediary steps of the design setup process:

- [Opening the Design Library](#)
- [Selecting the Compilation Mode](#)
- [Determining the Vision Mode](#)

- [Connecting Nets Using iScopedNets Program](#)
- [Breaking the Loops](#)
- [Synthesizing Flip-Flops and Latches](#)
- [Inserting the Instrumentation Logic](#)
- [Running the Precompile Process](#)

Opening the Design Library

Open the design library and the top-level cell by executing the `design` XEL command on the xeCompile command line. This command has the following syntax:

```
design <lib> <cell>
```

The `<lib>` option defines the design library, which is the database containing the design. The `<cell>` option identifies the top-level design cell to open.

Selecting the Compilation Mode

In the ICE mode, UXE supports Synthesizeable TestBench (STB) and Logic Analyzer (LA) operation modes, as well as Vector-based Acceleration, also known as Vector Debug (VD).

The options of the `compilerOption` command enable you to define the preferred compilation mode. By default, xeCompile uses the ICE mode for compiling designs. Specifically, the option is `compilerOption -add {mode ice}`.

Determining the Vision Mode

After having identified the compilation mode, you must accordingly determine the vision mode, that is, whether FullVision or Dynamic Probes.

FullVision is the default vision mode during compilation. There are two FullVision models, depending on what debug strategy will be used. The dynamic vs. static in-circuit mode, (or Vector Debug) strategy must be stated before precompile (in the design Setup stage), so that the correct version will be compiled in for later debugging.

Use the following command to specify the vision mode:

```
compilerOption -add {visionMode FV | DYNP}
```

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

If `compilerOption -add {visionMode 0}` is specified, FullVision is turned off and thus the subsequent precompile generates database for DYNP mode.

If `compilerOption -add {visionMode FV}` is used, FullVision is turned on.

The default FullVision depth is 256K FCLK cycles. This can be increased to 512K FCLK cycles by using the `precompile -dblTrace` option, but at the expense of doubling the amount of trace memory needed. It can also potentially reduce the step count.

If dynamic probes are desired, execute the `compilerOption -add {visionMode DYNP}` command on the xeCompile command line.

You can change the number of dynamic probes that can be added at run time using the `compilerOption -add {numCapturedNetsPerDomain <value>}` command. For example, to reserve 2048 probes per DCC, compile the design with the following command:

```
compilerOption -add {numCapturedNetsPerDomain 2048}
```

Selecting a Vision Mode Based on Preferred Debug Strategy

Creating a compiled program suitable for different debug strategies requires setting up the design using different options such as emulation mode, selecting a vision mode (if any), and compiling the design.

The following table identifies the appropriate the `compilerOption` command options that must be executed to define the compilation and vision modes based on different debug strategies:

Table 3-2 Debug Strategy by Option

Debug Strategy	Compilation and Vision Modes	Required <code>compilerOption</code> Command Options
Vector Debug	Specify the compilation mode as <i>Vector Debug</i> . Vision mode specification is not required.	<code>-add {mode vd}</code>
Dynamic Probes (STB or Logic Analyzer)	Specify the compilation mode as <i>ICE</i> and vision mode as <i>Dynamic Probe</i> .	<code>-add {mode ICE}</code> <code>-add {visionMode DYNP}</code>

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

Debug Strategy	Compilation and Vision Modes	Required compilerOption Command Options
Synthesizable TestBench (STB)	Specify the compilation mode as <i>ICE</i> and vision mode as <i>FullVision</i> .	-add {mode ICE} -add {visionMode FV}
Logic Analyzer (LA)	Specify the compilation mode as <i>ICE</i> and vision mode as <i>FullVision</i> .	-add {mode ICE} -add {visionMode FV}

Using infiniTrace

ICE modes (FV and DYNP) automatically use infiniTrace. InfiniTrace is *on* by default. Leaving it set to the default is recommended, but in cases where there are a significant enough number of primary inputs as to possibly cause compilation to fail, it can be turned off. Use the following command to enable or disable InfiniTrace:

```
compilerOption -add {infiniTrace on | off}
```

Compiling for STB Mode

Select ICE through XEL using `compilerOption` command when setting up design compilation.

For FullVision:

```
compilerOption -add {mode ice}  
compilerOption -add {visionMode FV}
```

The maximum trace depth can be specified by using the command:

```
compilerOption-add {maxTraceDepth <value>}
```

where possible values are 16K, 32K, 64K, 128K, 256K, 512K, and auto. Auto is the default and means the compiler picks up as large a trace depth as possible.

For Dynamic Probes:

```
compilerOption -add {mode ice}  
compilerOption -add {visionMode DYNP}
```

Compiling for LA Mode

To set the ICE LA mode, through XEL using the `compilerOption` command when setting up design compilation, use the following commands:

For FullVision:

```
compilerOption -add {mode ice}  
compilerOption -add {visionMode FV}
```

For Dynamic Probes:

```
compilerOption -add {mode ice}  
compilerOption -add {visionMode DYNP}
```

LA mode supports FV on Palladium XP. However, be aware that you cannot append multiple triggers to one waveform. The previous waveform is cleared unless a different waveform file is specified before the next trigger and upload occurs. InfiniTrace cannot be used in LA mode.

Compiling for VD Mode

To use Vector Debug mode, use the following command:

```
compilerOption -add {mode vd}
```

Connecting Nets Using iScopedNets Program

The `iScopedNets` program allows to connect nets upward through the design hierarchy to a desired level. It can also be used to make a net the top-level input or output of the design.

The `iScopedNets` program modifies cell definitions in the `QTDB/<library>.llib` files. These files are created by the `designImport` command or by `xeCompile` synthesis.

The `iScopedNets` program can be run from either the UNIX or XEL command line using the following command syntax:

```
iScopedNets -f <nets_file>
```



This program should be run after RTL design synthesis or netlist import, and before the `design` command.

The `iScopedNets` program has additional capabilities not described here. For a complete description, execute the `iScopedNets` command without any arguments.

Creating Nets File for iScopedNets

To use iScopedNets, first create a file (*<nets_file>*) that describes the modifications for iScopedNets to perform. This file uses the format shown below, where each line after the VERSION line describes a net to be brought upward through the design hierarchy.

```
VERSION <version_number>
  <library> <cell> <path> <topName> [<separator>]
  <library> <cell> <path> <topName> [<separator>]
  ...
  <library> <cell> <path> <topName> [<separator>]
```

The components of these entries are as follows:

<version_number> is the version of the file. It should be at least version 3, except in legacy applications. (See [Providing Vector \(Bus\) Support](#), below.)

<library> specifies a library file (*<library>.llib*) in the QTDB directory.

<cell> specifies a cell within this library

<path> is the instance path to a new within *<cell>*

<topName> is the name of a terminal on the top cell that is to be created and connected to the net specified by the *<path>*. To connect the specified net upward through the hierarchy, provided the version is 3 or above, a new terminal named *<topName>* is created at each level and all levels but the lowest level containing *<path>*. If no net named *<topName>* exists at this level, a new net named *<topName>* will be created. If a net named *<topName>* already exists at the specified level, it is merged with the net being connected upward. If the version number is 2 or below, and the net specified by *<path>* already connects upward in the hierarchy, the existing upward connections are used to connect as far upward as possible. If an existing connection is used at a specified level, the net being connected upward is not merged with any existing net named *<topName>* at that level.

<separator> is an optional string delimiting the hierarchy. It defaults to the standard hierarchy delimiter of “.”.

Consider the following example:

```
MYLIB MYCELL u1__u2__net3 my_top_level_net3
```

The above line entry has iScopedNets locate the net named *u1__u2__net3* within the hierarchy of cell *MYCELL* in library *MYLIB*, and bring it upward through the hierarchy using the name *my_top_level_net3*. In addition, a new terminal named *my_top_level_net3* will be created. This terminal will be an output if the net connected to it has a driver anywhere inside *MYCELL*; otherwise it will be an input.



Caution

Running iScopedNets modifies cells in the specified libraries as well as in hierarchically-referenced libraries. This affects subsequent incremental netlist import. If netlists are to be re-imported, run FULL import of ALL netlists, followed by re-running iScopedNets.

Providing Vector (Bus) Support

If the input file (`<nets_file>`) version number (`<version_num>`) is 1, then `<path>` and `<topName>` are treated as scalars, that is, a single net connecting to a single terminal.

If the version number is 2 or higher, `<path>` and `<topName>` can be treated as vectors, referring to groups of nets and terminals.

Following two forms of vector notation are supported:

- Implicit Range Notation
- Explicit Range Notation

The `<path>` and `<topName>` arguments can have different notation forms; `<path>` can be explicit and `<topName>` implicit, or any other combination.

If more elements are specified by `<path>` than by `<topName>`, iScopedNets will fail with errors.

Implicit Range Notation

Implicit range notation consists of the basename, without any index information: for example “data_bus”.

If implicit range notation is specified, it is first tried as a scalar. If a scalar net matches the path, it is connected. Thus, if `a__b__data` and `a__b__data[...]` exist, and the path is specified via implicit range notation (`a__b__data`), only the scalar net is handled.

A path specified with implicit range notation is then compared with nets of the form `<path>[index]`, with the index starting at zero.

For example, if there exist nets `a__data[4]`, `a__data[5]`, `a__data[6]`, and `a__data[8]`, then the `<path>` of `a__data` is equivalent to the `<path>` of `a__data[6:4]`. `a__data[8]` is NOT included; ranges must be contiguous. The range is in the form `[larger:smaller]`.

If *<topName>* is expressed in implicit range notation, the width of the top port will be the width of the *<path>* vector. (If there are no connection errors, this guarantees that the number of elements in *<path>* and *<topName>* are equal.)

If *<topName>* is expressed in implicit range notation, the names of the top created terminals will bear the indexes of the connected nets.

Explicit Range Notation

Explicit range notation consists of the basename and an index range, for example:
`data_bus [7:0].`

When explicit range notation is used with *<topName>*, the bit order of the range is preserved. For example, a *<path>* of `a__d[0:3]` and a *<topName>* of `bit[3:0]` connects net `a__d[0]` to terminal `bit[3]`, and so on.

Breaking the Loops

When xeCompile runs through the design precompile cycle, it automatically breaks asynchronous feedback loops.

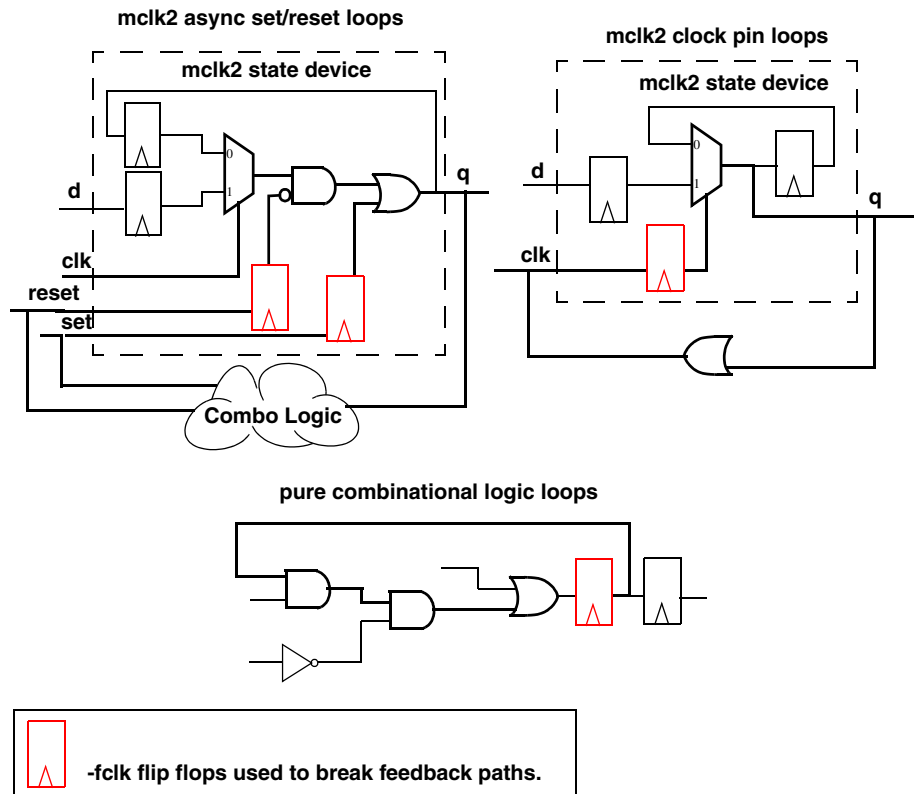
This process generates two log files to aid in debugging. These files are:

- `PDB/<lib>.<cellName>.brk`
- `PDB/<lib>.<cellName>.loop`

xeCompile automatically breaks loops without user intervention. However, by allowing you to manually break loops, performance can be enhanced. The In-Circuit Emulation performance of xeCompile is directly related to the step count, which is correlated to the length of combinatorial paths. By inserting delay elements to break false paths or multi-cycles paths, emulation performance can be improved, possibly dramatically, if the eliminated paths are the critical paths of the design.

Design modeling can also require the insertion of a delay to break combinatorial loops. These loops are not necessarily real combinatorial loops in the design. These can be static loops resulting from the modeling strategy used by xeCompile. To obtain a successful compile, it is necessary to break the loops. User intervention can aid in design modeling by allowing you to manually specify where loop breaks should occur.

Figure 3-4 Loop Breaking



There are three ways to manually break loops.

- By inserting a FCLK flip-flop (Q_FDP0B) in the netlist.
- By using the `breakNet` XEL command to insert an FCLK flip-flop at the driver of a particular net.
- By using the `breakPin` XEL command to insert a FCLK flip-flop at a particular pin of an instance.

Each of these methods can be used to break a feedback loop or long combinational path. They insert an FCLK flip-flop which gives a delay of one FCLK cycle. The `breakNet` and `breakPin` commands provide a way of breaking loops or paths without editing design sources. These commands should be run after design import and before the precompile step.

Note: Loop breaking should be executed with extreme caution. The compiler software follows user request and broken design functionality can result from improper use.

The `et3compile.msg` file and the `/dbfiles/<topcell>/memTransformList` indicates if compaction was performed and if so, gives a list of compacted memory instances in `<topcell>.memTransform`.

Detailed information on loop breaking can be found in [Appendix A, “Loop-Breaking.”](#)

Synthesizing Flip-Flops and Latches

The precompile step uses a default behavior that affects flip-flops and latches as described below.

For flip-flops, when reset goes active, the output goes low in the same FCLK cycle. When reset goes inactive, if the clock goes high in the same FCLK cycle, the output stays low even if data input D was high.

For latches, when reset goes active, the output goes low in the same FCLK cycle.

When the reset goes inactive, one of the following conditions can occur:

- if the enable goes active in the same FCLK cycle, the output gets the value that data input D has in that cycle.
- if the enable goes inactive in the same FCLK cycle, the output stays low even if data input D was high.

Inserting the Instrumentation Logic

This step can include the following:

■ Resolve the tristate drivers

Use the `precompileOption -add disconnectTristateDrivers` command to automatically disconnect the tristate drivers (with a warning) from any net that also has regular drivers. By default, if a net has both tristate and regular drivers, `precompile` fails.

■ Trim the parallel drivers

Use the following arguments of the `precompileOption -add` command to trim the parallel drivers:

- ☐ `tieSourceless`: Ties sourceless user nets to a "safe" constant
- ☐ `tieSourcelessHigh`: Ties sourceless user nets to power

❑ `tieSourcelessLow`: Ties sourceless user nets to ground

■ Remove the sourceless logic

Use the `precompileOption -add removeSourceless` command to disconnect the sourceless cones in the design considering them as "weak" logic that does not affect anything.

■ Remove the loadless logic

Use the `precompileOption -add removeLoadless` command to direct precompile to remove loadless logic including gates, flip-flops, latches, and memories. The goal of the removal is to reduce the emulator resources needed by a design.

■ Merge the buffer-inverter chains

Use the `precompileOption -add keepMem` command to cause precompile to automatically treat as `keepNets`, those bits of `HDL_MEMORY` arrays that are not driven by buffers, inverters, or combinational gates.

The *benefit* of treating the `HDL_MEMORY` array bits as `keepNets` is that the bits treated as `keepNets` can be loaded from a file (for example, using a `$readmemh` operation or `memory -load XEL` command). However, if the bits that are driven by combinational gates (including buffers or inverters) are loaded, the loaded values will remain only transiently before they are overwritten by the driving gates.

The *disadvantage* of treating `HDL_MEMORY` array bits as `keepNets` is a possible severe degradation in step count. The degradation happens when multiple array bits are driven by the same (or equivalent) combinational logic.

Running the Precompile Process

After specifying the required precompile options and User Data, you can explicitly execute the `precompile` command. However, Cadence recommends that you instead do the following:

1. Execute the `design` command to specify the design library and cell name.
2. Use the `precompileOption` command to specify options for precompile.
3. Specify the required User Data.
4. Specify User Data for the target interface (if any).
5. Run the `compile XEL` command.

On running the last step, xeCompile automatically invokes the precompile process before the core compilation. The progress of the precompile process, including warnings and errors, is

displayed in the xeCompile console. The process runs tasks such as initializing the nets and primitives; setting the clock, data paths, and exclusivity points; breaking the loops; and inserting the instrumentation logic.

Preparing the Design Files for Compilation

After having specified the precompile options and other required User Data, prepare the design files for core compilation and then run the compile.

This section covers the following topics:

- ❑ [Setting Up the Design for Precompile](#)
- ❑ [Incorporating Physical IP](#)
- ❑ [Running ICE Prepare](#)

Setting Up the Design for Precompile

This step involves setting up of the design files to prepare for the compile process through automatic processes that check design rules, generate logic by processes, such as loop breaking and synthesis of flip-flops and latches.

This step also includes specification of the following:

- Emulation host and domains that must be used for compilation
- Compilation and vision mode for which the design is to be compiled
- Parameters for activating clocks, signal properties, and instrumentation logic

Preparation for compilation includes specification of cable connection parameters, such as cable types, terminal assignment, and timing constraints, that need to be used with a target system. In addition, xeCompile runs data qualification and generates the control files for the design database. The design preparation step includes specification of the cable connection parameters that need to be used with a target system. These parameters include cable types, terminal assignment, and terminal timing commands. The commands for setting these parameters are described in the [Working with Emulators at Compile Time](#) on page 54 section.

Incorporating Physical IP

The Physical IP (Intellectual Property) refers to user hardware that substitutes for certain instances of the design. As part of design setup, you must specify the mapping between IPs and design cells.

In xeCompile, when Emulator Configuration is run, it records information on the IPs present in the IP chassis, as well as their cable connections, into an emulator configuration file.

Physical IP mapping between the IP terminals and pins on the IP chassis is performed at the ICE prepare step in emulation mode.

IP protection can be implemented by using the Cadence `ncprotect` utility. For more information, refer to the `ncprotect` documentation relevant to NC-VHDL and NC-Verilog, available from Cadence Online Support website.

IP Mapping

Input to the `icePrepare` mapping process is in the form of a `<IP-ID>.fem` file and an optional `<IP-ID>.ttc` file, both provided by the IP Builder utility and the Emulator Configuration file generated by `autoconfig` and user design. The `*.fem` (that is, *front end mapping*) file provides IP terminal connectivity information. The `*.ttc` file provides terminal timing constraints for the IP. If this `.fem` file is present in the design files, it will be automatically found by the `icePrepare` process.

During the `icePrepare` process, design instances are assigned to IPs, and IP terminals are mapped to their corresponding ICTI pins. This mapping is recorded in the output control (`*.ctl`) file. This process generates two reports, which are placed in the current design directory: `pichInstAssign.report` and `pichSignalMapping.report`. The `pichInstAssign.report` file reports the assignments of design instances to the IPs. The `pichSignalMapping.report` file reports the mapping of the IP terminals to the ICTI card pins on the IP chassis.

The IP Builder constructs the `*.fem` file from a user-supplied input file. The input file must have the structure shown in the example.

The first line must always be `IPID <IP-ID>`.

Use the `IPMAPFILE` entry to name the back end mapping file.

Following is an example of an IP Builder input file:

```
IPID 123
IPName test1
```


UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

```
IPDescription test ipBuilder
IPMAPFILE ipbmf_proto.map
Date xx/xx/xx 12:00
Revsion x
Comments
This is an example of ipBuilder input
.
CELL C1 C1
TERM in1 1-22
TERM in2 2-15
TERM out1 3-3
TERM out2 4-8
```

The `*.ttc` file is an XEL script containing the terminal timing constraint commands for a user's design. The sample contents can be:

```
#IPID 123
#IPNametest1
delayBox -add {db1 2 C1:in1}
terminalTiming -add {C1:out1 db1}
```

IP Builder

Palladium XP provides a special tool for creating a virtual representation of user Physical IP that can be used by xeCompile. IP Builder is a menu tool that performs IP mapping and creates a front-end mapping (`.fem`) or transposer pin (`.tp`) file for each IP Block instance. These front-end mapping files are read by the compiler, allowing the compiler to route the required signals to and from the Physical IP block.

Running ICE Prepare

The ICE Prepare step is a mandatory step for the ICE flow. During this step, target-specific information is generated for the design database. This includes the ICTI pin assignment, mapping between IP terminals and ICTIs on the IP chassis, and other configuration assignment processes are performed.

The ICE prepare step aims at processing the target-related User Data and generating files that will be used during compile. This process can be invoked in two ways:

- Explicitly execute the `icePrepare` XEL command on the command line
- Or

- Execute the `compile` command after the User Data definitions. This command will automatically call the ICE prepare function before the core compilation.

Support for Encryption in HDL Compiler

HDL Compiler supports RTL source files encrypted using the following encryption methods:

- Verilog-2001 standard encryption by `ncprotect`
- Verilog-XL encryption
- Verilog-2005 standard encryption by `ncprotect`

Irrespective of the encryption style used in the RTL, the gate-level netlist, generated by the HDL Compiler, will be encrypted using the `ncprotect` Verilog-2001 standard. You can import all files with any encryption style to the HDL Compiler together without using any additional option.

Debug Capability of Encrypted Source Code

You can use the `ncprotect` utility to encrypt a file with three levels of debug capability. For more information, refer to [Encryption with Three Levels of Debug Capability in Chapter 4, “Compiling Designs for Simulation Acceleration Using IXCOM.”](#)

xeDebug versus verigen

You can debug the following at xeDebug runtime through probe and waveform display:

- `debugall`: Probe the signals, nets, and content of the instances.
- `portview`: Probe ports of the IP module.
- `none`: Default option. Cannot probe an IP module.

You can use the `verigen` program to dump the following from a design database:

- `debugall`: Stop at IP module, but you can see the module name and port name as the full path name of waveform (cleartext). Cannot see instance contents and connectivity.
- `portview`: Stop at IP module, but you can see the module name and port name as the full path name of waveform (cleartext). Cannot see instance contents and connectivity.
- `none`: Stop at IP module and show the encrypted name of IP module (not cleartext).

Protection Right in HDL Compiler

A protection right is applied to a module based on the following conditions:

- When a module is enclosed within one protected region, the HDL Compiler gate-level output encrypts the entire module with the same protection right.
- When a module has both cleartext and multiple protected regions, the HDL Compiler splits the declared objects into cleartext objects and protected objects. In the gate-level netlist, all cleartext objects are declared in cleartext and the rest of the netlist is encrypted.
- When a module has multiple protected regions with inconsistent protection rights, the HDL Compiler encrypts the protected part of the netlist by using the strongest right. For example, if a region has `debugall` and another region has `none` in RTL, the protected regions in gate-level netlist will use `none`. If a region has `debugall` and another region has `portview` in RTL, the protected regions in gate-level netlist will use `portview`.

Importing Files by Using `designImport`

The import process, which is invoked with the `designImport` command, encrypts netlist entity names within the compiled database to respond to the protection rights assigned to a netlist source by a command-line option or an embedded pragma. Entity names derived from a protected source or a protected region within a source are, by default, encrypted.

If the rights to the protected source, or some region of the protected source, are specified as `debugall`, then the entity names derived from the specified source or region are stored as cleartext.

If the rights to the protected source, or some region of the protected source, are specified as `portview`, then the entity names of module, ports, and port-nets (nets connected to ports) within the specified source or region are cleartext, while the names of other entities are encrypted.

Compiling the Design Files

Compiling the design creates a compiled database that is used for the next step in verification, that is, running and debugging the design.

The compilation process places and routes the Palladium XP logic for emulation. Compile performance is dependant on the step count and seed used for compiling the design. Generally, there is a trade-off between the speed and performance of the compile. For the best results, the first priority is fastest emulation speed, and the second priority is least number of dropped probes.

The compilation process starts on executing the `compile` XEL command. While the design is being compiled, messages about the status of each phase of design compilation are displayed, such as partitioning the design, and whether each phase was completed successfully or failed.

Note: The `compile` XEL command starts the process of core compilation only after successful execution of the `precompile` process.

For syntax details and more information on the `compile` XEL command, refer to the *Compiler Commands* chapter of the *UXE Command Reference Manual*.

Note: A design must be run on the same platform for which it was compiled. For example, a design compiled on the Linux platform must be debugged on the Linux platform.

This section includes the following topics:

- ❑ [Viewing Compile Results](#)
- ❑ [Using Compiler Options for Better Compile-Time Performance](#)
- ❑ [Synthesizing and Compiling ALUs](#)

Viewing Compile Results

xeCompile compiles a design, and creates the download files to be used by xeDebug. These files configure the Palladium XP system for the emulation modes (Vector Debug or ICE/STB).

Also, when a compile session is started, xeCompile makes a main log named `xeCompile.log` in the design directory. This log file contains an exact copy of the terminal or console window. This file is incremental, that is, multiple compile sessions get sequentially recorded within the `xeCompile.log` file.

xeCompile also generates a log file named `xe.msg` that supports recursive logging, that is, nested command calls are also logged in the file. The `xe.msg` file contains a complete log of

all previous compile and run-time sessions. This file includes the output displayed on the screen (same as `xeCompile.log`), output from the internal calls that are not displayed on the screen, and logging from the XEL commands that otherwise are not recorded in the console log file.

The `xe.msg` file is a superset of `xeCompile.log`. It is also incremental like the `xeCompile.log` file. Therefore, many commands are logged twice; first as an echo to the screen and second from inside the commands. Echoes are shown with a prompt in upper case, like on the screen, and logging from inside commands are shown with the prompt in lower case.

xeCompile generates another log file named `xeCompile.key` that contains all the commands typed by user. This file contains only the commands. It is a record of the compile session, and if sourced, produces exactly the same results as the original session. However, the `xeCompile.key` log file cannot record correctly some inherent limitations of the technology, such as effects of timing.

Using Compiler Options for Better Compile-Time Performance

Consider the following possible user-actions to ensure a better compile-time performance:

- User can use the `compilerOption -add {compilerEffort LOW|HIGH}` command to fine-tune the compile. The default option is `HIGH`, which means that compiler produces the best quality results (lower step count). If `compilerEffort` is set to `LOW`, the compilation process is faster; however, the emulation might run slower or might require a larger hardware configuration.

For a very few unusual designs, the `compilerEffort HIGH` setting could actually cause the design to require more domains than the `compilerEffort LOW` setting. There is no easy way to characterize designs that experience this problem. Compiling a design several times, using both the `compilerEffort LOW` and `compilerEffort HIGH` settings for `compilerOption` can be used to determine if a specific design is one of these exceptional cases.

- Use the `compileFind XEL` command with different options (such as `-first`, `-best`, `-opt`, `-all`, and `-lsf`) to specify the conditions for finding the most appropriate compile run. For detailed information about each option, refer to the description of `compileFind` command in *Compile-time Commands in ICE Mode* chapter of the *UXE Command Reference Manual*.
 - The `-first` option tries to find the first passing compile, using the specified parameters, such as `-max_trials <n>` and `-min_steps <n>`.

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

- ❑ The `-best` option tries to find the best compile, using the specified parameters. Log messages for critical path information are placed in the file `./tmp/et3compile.msg`
- ❑ The `-opt` option allows to explore the effects of changing the optimization parameters on the compile success rate, capacity, and step count. This option runs a compile several times as a trial for each pair of values given for `optimizationEffort` and `placementEffort`.
- ❑ The `-all` command runs 30 compiles with sequential seeds and log the results. By using extra parameters, you can vary the number of compiles and seed to use, define the number of nets to use for probing, set step limits, and so on. The critical path information for the i^{th} (passed) trial for the `-all` option is located in the `./tmp/et3pass2.<i>.msg` file. For this option, the information is provided for each passed trial.
- ❑ When using LSF,
 - The `-lsf` option is a fast parallel alternative to the `-first`, `-best`, and `-all` options performing trials sequentially. The `-lsf` option performs the required number of trials with different seeds on different machines in parallel using LSF and chooses the best trial (the one with the minimum step count).
 - When both `-opt` and `-lsf` options are present, it is a fast parallel alternative to the `-opt` command that performs the trials sequentially. It performs the required number of trials with different seeds and different combinations of optimization effort and placement effort on different machines in parallel using LSF and chooses the best trial (the one with the minimum step count).

For detailed description on use of LSF in UXE, refer to the [Using LSF to Distribute Compiles with Different Seeds among Different Computers](#) section.

- ❑ As manual fine-tuning to find the best value for memory utilization can be cumbersome, use the `-mem` option of the `compileFind` command. This enables you to explore the effects of changing the memory utilization on results such as the compile success rate, capacity, and step count.

The `-mem` option executes a number of compiles for each memory utilization value, using user-defined decrements, starting from a user-defined maximum value (100, by default), down to a minimum value (50, by default). Each trial is recorded in the logfile. For each memory utilization value, the record will contain the value, number of the trial, seed value, compilation result (pass or fail), and step count. It will also report the success rate, average step count, and gate count, for each memory utilization value. It will also log a summary report similar to that given by running the `-all` option of the `compileFind` command. For each trial `<n>`, this summary

report will report each memory utilization value in a file named
`tmp/et3pass2.util<U>.<n>.msg`.

Note: Running the `-mem` option can be very time-consuming. Therefore, be aware of this trade-off.

- When a compile fails, user can repeat the command using different seed values until a successful compile is achieved.

Seed value is a number between 0 and $2^{32} - 1$ used as input to the hardware compile. The seed value influences logic partitioning and affects the success rate of compilation. Several seed options are available. The default is to use a random seed, but user can also specify to the compiler to use a specific number as the seed value. This is generally useful if you have already run a compile, and want to tune the results.

- For all compiles subsequent to a successful compile, user can use the same seed value of the last passing run.

Using LSF to Distribute Compiles with Different Seeds among Different Computers

As design size increases, the time taken to execute the `compileFind` command with options such as `-first`, `-best`, `-all`, and `-opt` might become prohibitively long. To mitigate this problem, use the `-lsf` option of the `compileFind` command. This command option distributes the compiles with different seeds among different computers using LSF, a task scheduler.

Important

To use the `-lsf` option of the `compileFind` command, you should have LSF software installed. Before running these commands, run the setup script required for LSF. For details, you must consult the LSF administrator.

When the `-lsf` option is executed, first, if necessary, the precompile process is performed in the design directory. Then, the first part of the back-end compile that is independent of the seed value is performed in the design directory. After that, the second part of the back-end compile that is dependent on the seed value is performed in parallel on different machines with different seed values.

Parallel trials are performed in subdirectories labeled as `ITER_1`, `ITER_2`,... of the working directory. The corresponding LSF outputs are placed in files named such as `report_from_trial_1`, `report_from_trial_2`,... in the working directory. The previous trial subdirectories, `ITER_<n>`, and LSF output files, `report_from_trial_<n>`, are moved to the `PREV_RESULTS` subdirectory.

By default, N random seeds $S_1, \dots, S_n, \dots, S_N$ are generated. The default value of N is 30.

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

For each n (that is, from 1 to N) the following command is synthesized and submitted to LSF:

```
bsub [<LSF_options>] -o report_from_trial_<n> remote_compile.sh <n> <design>  
<parallel_compile_options> -msg et3pass2.<n>
```

where,

- `bsub` is a LSF command.
- `<LSF_options>` is a string retrieved from the `LSF_options` file that contains the options to the `bsub` command. The `LSF_options` file is located in the design directory. The contents of this file are described in the [LSF_options File](#) section.
Note: The `LSF_options` file can also be saved in a location other than the design directory. To do so, before starting the compile process, define the full path to the `LSF_options` file using the `CDS_LSF_OPTIONS` environment variable. When you execute the `compileFind -lsf` command, the design directory is searched for the `LSF_options` file. If the file is not found in the design directory, the full path specified with the `CDS_LSF_OPTIONS` environment variable is searched for an `LSF_options` file.
- `-o report_from_trial_<n>` specifies to LSF to redirect the standard output of the job (together with error output) to the `report_from_trial_<n>` file.
- `remote_compile.sh` is a shell script (provided by Cadence).
- `<n>` is a trial number.
- `<design> ::= <top_lib_name> <top_cell_name>.`
- `<parallel_compile_options>` are described in the [Parallel Compile Options](#) section.
- `-msg et3pass2.<n>` specifies to the Compiler to write messages to the `ITER_1/tmp/-msg et3pass2.<n>` file.

LSF will send the jobs to the appropriate machines.

When all trials have finished and at least one trial has succeeded, the best trial (the one with the minimum step count) is chosen. Consider the best trial to be n . Then, the following is done:

1. Files created under from `ITER_<n>/dbFiles` are moved to `dbFiles`, like this:

```
mv ITER_<n>/dbFiles/<file> dbFiles
```

2. Message files (`tmp/*msg`) are from each i^{th} subdirectory to the design directory:

```
mv ITER_<n>/tmp/et3pass2.<n>.msg tmp  
mv ITER_<n>/tmp/qtet3pass2.<n>.msg tmp
```


3. The `xe.msg` file from each i^{th} subdirectory is attached to the `xe.msg` file in the design directory:

```
cat ITER_1/xe.msg >> xe.msg
```

4. In the design directory, `dbgPrepare` and `bitGen` are executed.

5. All the temporary subdirectories and files are removed:

```
rm -rf ITER_1 ITER_2...
rm -rf ITER_1.bak ITER_2.bak...
rm report_from_trial_1 report_from_trial_2...
rm report_from_trial_1.bak report_from_trial_2.bak...
```

LSF_options File

You can put options to the `bsub` LSF command in the `LSF_options` file. All the options must be defined in one line.

Note: The specified options are passed to LSF without any checking. This file can also be used for distributed synthesis. For details, refer to the [Executing Distributed Synthesis during RTL Synthesis](#) section.

To use an LSF queue for distributed synthesis, you can provide an `LSF_options` file of your choice using a command like `-q my_queue`.

Following are few usage examples:

- The following options instruct LSF to submit jobs only to machines with Linux operating system version 6.0 or higher and with memory at least 16000Mb:

```
-R "OSREL==6 && OSNAME==lnxOS && mem > 16000"
```

- Specifying the following command line in the `LSF_options` file indicates that jobs should be submitted only to the machines (defined by the LSF administrator) having more than 1000MB of available swap memory (`swp > 1000`), CPU utilization of less than 50% (`ut < 50`), and memory more than 16000MB (`mem > 16000`):

```
-R "swp > 1000 && ut < 50 && mem > 16000"
```

- The following options instruct LSF to submit jobs only to machines defined (by the LSF administrator) as having type `DEFAULT` and with memory at least 8000Mb:

```
-R "type == DEFAULT && mem > 8000"
```

Parallel Compile Options

The parallel compile options available with the `-lsf` option of the `compileFind` command are:

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

```
compileFind -lsf [-max_trials <number>] [-seed <number>] [-max_steps <number>] \
    [-min_steps <number>] [-crit_paths <crit_paths>] [-etOnly] \
    [-first] [-lpass] [-keep_all] [-rerun_best] [-compactDCC]
    [-no_export] [-names_crit] [-link <filename>] [-verify] \
    [-file <file>]
```

The following parallel compile options are available when the `-lsf` option is specified with the `-opt` option:

```
compileFind -opt -lsf [-max_trials_each <number>] [-seed <number>] \
    [-oE {<values>}] [-pE {<values>}] \
    [-max_steps <number>] [-min_steps <number>] \
    [-crit_paths <crit_paths>] [-etOnly] [-first] \
    [-keep_all] [-names_crit] [-rerun_best] [-no_export] \
    [-logfile <logfile>] [-verify] [-compactDCC] \
    [-link <filename>] [-file <file>]
```

For detailed description of the syntax of both the commands, refer to the *Compiler Commands* chapter of the *UXE Command Reference Manual*.

Parallel Scheduling and Partitioning for Enhancing Compile-Time Performance

Compiling a large multi-board design can take significant amount of time, up to several hours. The scheduler and partitioner processes are complex tasks that take up a significant portion of the design compilation time. To enhance the performance of the scheduler and partitioner, you can run them in sets of parallel processes. This helps in improving the compile-time performance by distributing the work of the scheduler and partitioner among several parallel processes running on different cores and computers.

LSF assigns the parallel processes to hosts and processors. Each process is responsible for scheduling a separate unit of the emulator, a board. The parallel processes communicate with each other over the network during compilation. If one of the parallel processes aborts, for example, when a host machine reboots, the entire compilation is stopped with an error message.

Using Parallel Scheduling and Partitioning

Perform the following two steps to enable parallel scheduling and partitioning:

1. Use the `parallelCompile` option of the `compilerOption` command.
2. Specify the required options for the LSF's `bsub` command.

 Important

The parallel scheduling feature requires LSF to be installed and appropriately configured on the emulator host. For details, consult your LSF administrator.

The parallel partitioning feature does not necessarily require LSF. If LSF is not installed and the parallel partitioning feature is enabled, all partitioner clients are run as local clients on the same machine as the server. This can lead to a potentially large number of client processes running at the same time on the same machine, and might cause excessive disk-swapping activities. Therefore, the parallel partitioning feature limits to running at the most four local clients at any single time. On the other hand, when LSF is installed, local clients have many communication advantages over network clients. Therefore, by default the first two clients are assigned to the local emulator host and subsequent clients to other network LSF hosts.

Setting the `parallelCompile` Option for the `compilerOption` Command

To enable parallel scheduling and partitioning, set the `parallelCompile` option with the `compilerOption` command, as shown below:

```
compilerOption -add {parallelCompile ON}
```

When you specify `parallelCompile ON`, the compiler considers factors, such as design size that must be at least two boards, to decide about enabling both parallel scheduling and partitioning. If these are enabled, the scheduler and the partitioner run sets of parallel processes managed by the LSF utility.

To disable both parallel scheduling and partitioning, specify the `parallelCompile OFF` option. When disabled, these processes run in the traditional single-process mode, which is the default mode.

You can also specify to enable only parallel scheduling (`parallelCompile SCHED`) or parallel partitioning (`parallelCompile PART`).

In situations when the system fails to start the parallel processes, for example, when LSF is not installed, a warning is displayed and the scheduler is run in a traditional, single-process mode.

For more information on the syntax and usage of the `compilerOption` command, refer to the *User Data Commands* chapter of the *UXE Command Reference Manual*.

Specifying Options for the LSF's bsub Command

Parallel scheduling and partitioning use LSF command `bsub` to launch the parallel processes. You can put the options for the `bsub` command in the `LSF_options` file located in the design directory, the same way as the LSF queue is specified using the `-lsf` option of the `compileFind` command. All the options must be specified on the same command line. For example, the following command instructs LSF to submit jobs only to machines (defined by the LSF administrator) having more than 1000MB of available swap memory (`swp > 1000`), CPU utilization less than 50% (`ut < 50`), and more than 16000MB of memory (`mem > 16000`).

```
-R "swp > 1000 && ut < 50 && mem > 16000"
```

If the `LSF_options` file does not exist in the design directory, LSF uses the default options set by the administrator. If the `LSF_options` file is not readable, an error is reported by the scheduler.

For more information on how LSF works, the syntax and usage of the `bsub` command, and the `LSF_options` file, refer to the section [Using LSF to Distribute Compiles with Different Seeds among Different Computers](#) on page 187.

Note: Design compilation performance can vary between the parallel and single-process modes. Due to network delays and the overhead, parallel scheduling and partitioning might not necessarily lead to performance benefits. For example, if there are not enough hosts with sufficient memory, the compilation can be slow or might even fail.

Also, some optimization techniques are disabled in the parallel scheduling and partitioning compilation mode. The single-process mode might deliver a slightly better design performance in certain situations.

Using the `parallelCompile` option of the `compilerOption` command together with the `compileFind -lsf` command can potentially use a large number of hosts. For example, if `compileFind -lsf` starts 10 trials and each one of them starts parallel partitioner with 4 boards, up to 40 processes can be run on various hosts. This might lead to competing resources and reducing speed, or at worst cause the host to run out of memory.

Using Multithreading Feature for Compilation

The following table lists the different multithreaded programs available in different phases of compilation:

Program	Default maximum number of threads	Local	Network	Ways to Control
hdlSynthesize command	1	Yes	Yes	For network-level multithreading, specify the <code>hdlSynthesize -parallel</code> command to use the <code>parallel.host</code> file. For details, refer to the Using parallel.host File section. For local-level multithreading, specify the <code>hdlSynthesize -parallel<number></code> command. For details, refer to the Using Multiple CPUs of the Same Workstation section.

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

Program	Default maximum number of threads	Local	Network	Ways to Control
Design import	16 Note: The default number can be changed using the <code>IMPORT_MAX_PARALLEL</code> environment variable that has an absolute maximum limit of 512 threads. If a larger number is specified, it is reduced to 512.	Yes	Yes	Use the <code>importOption {parallel on}</code> command. For details, refer to the Using Parallel Import section. Note: In addition, there is the following special-purpose option that can be used to disable the automatic creation of BCUs: <pre>importOption {parallel [on off]}</pre> This is a holdover from Incisive XE, for backward compatibility. If you want to script a set of parallel import operations acting on the same database (QTDB), and more than one such parallel invocation tries to create the same library (as can happen with automatic BCU creation), things do not work correctly. This option is obsolete in all except such legacy application.

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

Program	Default maximum number of threads	Local	Network	Ways to Control
Scheduler	2	Yes	Yes	Use the <code>parallelCompile</code> option of the <code>compileOption</code> command. For details, refer to the Parallel Scheduling and Partitioning for Enhancing Compile-Time Performance section.
<code>compileFind -lsf</code> command	No restrictions on the number of the processes	No	Yes	For details, refer to the Using LSF to Distribute Compiles with Different Seeds among Different Computers section.
<code>compileFind -opt -lsf</code> command	No restrictions on the number of the processes	No	Yes	For details, refer to the Using LSF to Distribute Compiles with Different Seeds among Different Computers section.

Synthesizing and Compiling ALUs

The Palladium XP hardware contains Arithmetic Logic Units (ALUs) that the HDL Compiler can use to synthesize computation-intensive parts of the design such as, the following 32-bit operations:

- Arithmetic operations, such as addition, subtraction, negation, increment, decrement, multiplication, and division with both signed and unsigned numbers
- Comparison operations, such as $A > B$, $A < B$, and $A == B$

Synthesizing a certain part of the netlist on to ALUs can potentially reduce the step count and also improve the emulator capacity. For example, when a 32-bit adder is mapped to an ALU, only one primitive is required, as compared to requiring about 64 primitives when implemented as boxes or look-up tables.

To switch on ALU synthesis, use the `-alu` option:

■ **vaelab syntax:**

```
vaelab -alu
```

■ **hdlSynthesize syntax:**

```
hdlSynthesize -alu
```

In the gate-level netlist, each ALU instruction appears as an instance of a `qtref` cell `Q_ALU_I`. A sequence of ALU instructions using the same ALU, passing intermediate values through registers, are linked together using `SYNC_IN`/`SYNC_OUT` ports. This is called an ALU chain.

For each ALU chain, the HDL Compiler also produces a Verilog simulation model, along with instances of `Q_ALU_I`, so that the behavior of the netlists can be verified in a simulator, such as IES, or in a formal verification tool, such as Conformal.

Controlling ALU Utilization during Synthesis

There are only 24 physical ALUs on a Palladium XP board. Each ALU has a limited number of registers and routing resources. Thus, having a large number of ALU primitives in a netlist can become a performance bottleneck and slow down the entire emulation cycle.

Use the synthesis option `-MinALUSize <arg>` with the `vaelab` command or `-alu<arg>` option with the `hdlSynthesize` command to control ALU utilization for a netlist.

■ **vaelab Syntax:**

```
vaelab -MinALUSize <arg>
```

■ **hdlsynthesize Syntax:**

```
hdlSynthesize -alu<arg>
```

The `<arg>` values specify the threshold values for the ALU mapping. For example, `-MinALUSize 40` option instructs the HDL Compiler to synthesize only those additions to the ALUs that have all the 40 bits non-constant for both of the operands; an addition smaller than 40 bits is flattened into gates without using ALUs. Meanwhile, `-MinALUSize 10` encourages the HDL compiler to map more additions, including small adders like a 10-bit adder, to the ALUs.

Note the following points about the value of `<arg>`:

- The value of `<arg>` should be between 2 to 1000.
- Fewer operations are mapped to ALU as `<arg>` becomes bigger.

- The value of `<arg>` represents the minimum threshold of the average number of non-constant inputs when an operation is mapped to ALU. Default threshold value is 32.
- Different operations can interpret this threshold in different ways.

By default, ALU mapping is switched off. Using either of the `-alu` or `-MinALUSize` command options enables ALU mapping. Thus, when the `-MinALUSize <arg>` option is present with the `vaelab` command, `-alu` option is optional for `vaelab`.

The compiler maps the ALU operations to physical ALUs. If there are not enough physical ALUs, the compiler can convert some ALU operations to gates. For information on the compilation support for ALUs, refer to [Synthesizing and Compiling ALUs](#) on page 195.

Controlling ALU-to-Gates Transformations at Compile Time

During synthesis, the chains of ALU instructions that are represented by `ALU_I` primitives are synthesized. During compilation, the compiler schedules the ALU chains to physical ALUs. When feasible, multiple ALU chains can be scheduled to the same physical ALU. Most of the ALU transformations are performed automatically, without any user intervention.

One of the possible ALU transformations is conversion of some ALU chains to gates. It can be necessary because HDL synthesis is performed on the hierarchical design representation when the design capacity and hardware configuration are not known.

There are only 24 physical ALUs on a Palladium XP module. As a result, too many ALU primitives in a netlist can cause a performance bottleneck, and slow down the entire schedule or even exceed the capacity of the available Palladium XP modules. The compiler can map some ALU instructions to gates in order to reduce the contention on ALU resources.

By default, based on factors, such as hardware configuration and design capacity, the compiler automatically decides how many and which ALU chains should be transformed to gates. You can modify this ALU-to-gates transformation by using the `aluTransform` User Data command.

Note: ALU-to-gates transformations are performed before the netlist-optimization step. Therefore, 100% utilization before netlist optimization might become 80% utilization (or even lesser) after netlist optimization.

Following are the various options for the `aluTransform` command that help you control ALU-to-gates transformations:

Command	Description
---------	-------------

UXE User Guide

Compiling Designs for In-Circuit Emulation with xeCompile

<code>-add{* alu2gates OFF}</code>	No ALU-to-gates transformations are performed.
<code>-add{* alu2gates ON}</code>	Restores the default behavior for ALU-to-gates transformations.
<code>-rm{* alu2gates}</code>	Restores the default behavior for ALU-to-gates transformations.
<code>-add{* aluMaxUtilization <number>}</code> where <number> is a percentage	Modifies the default maximum utilization percentage (100). Minimum value for <number> is 0 (no transformation is performed), and maximum value is 100.
<code>-rm {* aluMaxUtilization}</code>	Restores the default value for aluMaxUtilization.

Note: The asterisk * in these commands is a placeholder. This command structure is similar to the structure of the `memoryTransform` command. Refer to the *User Data Commands* chapter of the *UXE Command Reference Manual* for more details.



User Data from the above commands is stored and read from the dbFiles/<top>.aluRules file. Do not edit this file.

Information on the available and used physical ALU can be accessed from the back-end compilation log file (for example, `tmp/et3compile.msg`) and in the file `dbFiles/<topcell>.memplacement`.

Compiling Designs for Simulation Acceleration Using IXCOM

In the Incisive **X**accelerator **COM**piler (IXCOM) flow, the `vlan`, `vhan`, `ixcom` tools are used during the design analysis and compilation phases. These tools generate the intermediate database that is provided to the `irun` tool for compilation. The resultant database is referred as the IES or `irun` snapshot. This snapshot can be run in Software Simulation mode and Simulation Acceleration mode.

In the **Software Simulation mode**, the complete design is run on the `irun` simulator.

To compile a **VHDL or mixed-language** design for Software Simulation mode, first analyze the Verilog and VHDL source files using the `vlan` and `vhan` tools, respectively. Then, run the `ixcom` command, specifying the analyzed source files and relevant command-line options. IXCOM passes all input to the `vhelab` tool for design elaboration.

To compile a **Verilog-only** design for Software Simulation mode, provide the source files and command-line options with the `ixcom` command. IXCOM invokes the required tools for:

- ❑ Source analysis and design elaboration
- ❑ Source transformation and code generation
- ❑ Writing the relevant design information to the required internal databases

Finally, IXCOM runs `make` to generate the `irun` or NC-Sim snapshot.

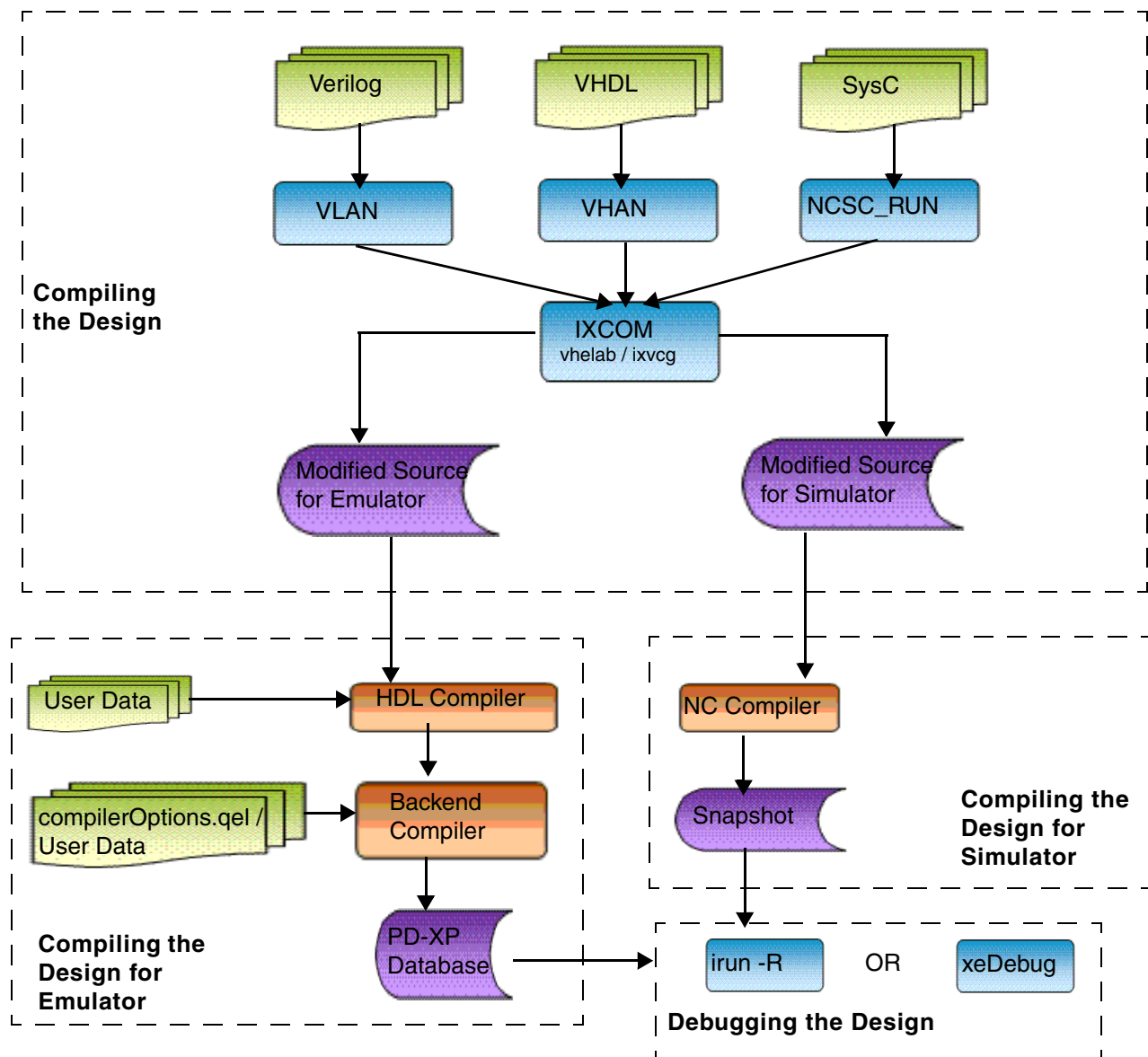
In the **Simulation Acceleration mode**, one part of the design is run on the `irun` simulator and the other part is run on the emulator. In addition to all the processing that the IXCOM compiler does for the Software Simulation mode, in the Simulation Acceleration mode, IXCOM also invokes the back-end compilers to generate the emulator configuration data. The back-end compilation process is described in [Chapter 3, “Compiling Designs for In-Circuit Emulation with xeCompile.”](#)

Important

The content of this chapter pertains only to the SA mode.

The following diagram illustrates the flows for compiling and running Mixed-HDL designs using IXCOM flow in both Software Simulation and Simulation Acceleration modes. The steps specific to the Simulation Acceleration mode are marked in blue colored dotted lines.

Figure 4-1 Compiling and Running Mixed-Language Designs



Overview of the IXCOM Compilation Process

The compile methodology of IXCOM is similar to that of other compiled-code simulators. It consists of multiple steps (multi-step compilation) for a mixed-HDL or VHDL design and a single step (single-step compilation) for a Verilog-only design:

For a mixed-HDL or VHDL Design, use the multi-step compilation process:

1. Analyze the VHDL and Verilog parts of the design separately. (Use `vlan` and `vhan` commands.) See [Analyzing Mixed-Language Designs](#) on page 205.
2. Compile the entire design to generate the simulation snapshot. (Use `ixcom` command.) See [Compiling Mixed-Language Designs](#) on page 214.

For a Verilog-only Design, use the single-step compilation process:

- ➔ Compile the entire design to generate the simulation snapshot. (Use `ixcom` command only; separate `vlan` and `vhan` steps are not required.)

The `ixcom` command supports various `vlan` command options to enable single-step compilation. When using multi-step compilation, do not use the options categorized as Analyzer/Compiler options with the `ixcom` command. Specify these options with the `vlan` command instead.

After the design has been compiled, use the resulting snapshot to run the design and debug it for errors and improving performance.

Important

The snapshot is platform-specific. This means that if the compilation was done on Linux, the snapshot would be Linux-specific, and at run time, the *irun* and xeDebug tools accessing this snapshot should also be run on Linux.

This section contains the following topics on using IXCOM to analyze and compile your design:

- [Analyzing Mixed-Language Designs](#)
- [Compiling Mixed-Language Designs](#)
- [Compiling SystemC Designs](#)
- [Using Specman Elite with IXCOM](#)
- [Compiling Testbenches Separately](#)
- [Determining File Type by Source Filename Extension](#)

- [Compiling Behavioral Constructs](#)
- [Compiling Behavioral Interface Constructs](#)
- [Fine-Tuning Behavioral Performance](#)
- [System Task Mapping in IXCOM flow](#)
- [Executing Non-Synthesizable Code in the Emulator](#)
- [Generating Pulses](#)
- [Pragma Handling in IXCOM](#)
- [Reading and Writing Signals in DUT Modules](#)
- [Using Plus Arguments](#)
- [Support for Encryption in SA Mode](#)
- [Support for Strengths in SA Mode](#)
- [Handling of UDPs by IXCOM](#)
- [Support for Verilog Configuration Blocks](#)
- [Support for specify Blocks in SA Mode](#)
- [Direct Access of DUT Memory Array and Register](#)
- [Using the marg Library](#)
- [C and Verilog Interface Functions](#)
- [SystemVerilog DPI Functions](#)
- [Specman Functions](#)
- [Importing External Netlists into IXCOM Compiler](#)

Compiling for Hardware Using IXCOM

By default, designs are compiled to run in software simulation only. To be able to run in Simulation Acceleration modes, add the `-ua` and `+dut+<entity_module>` options to the `ixcom` command line.

For mixed-HDL or VHDL Designs:

```
ixcom -top <top_level_Verilog_configuration_name> -ua \  
      +dut+entity|module other_options
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

For Verilog-only Designs:

```
ixcom -ua +dut+module other_options <design_files>
```

The `+dut+` option indicates the parts of the design that you want to be compiled for hardware. These parts will then execute in the emulator when running in Simulation Acceleration mode.

Directing Design Compilation for Hardware

For compiling the design for running on the hardware, certain compilation directives can be given to control the run time, such as defining the configuration of the emulator on which the design will be run after compilation. These directives are given by using the compiler and User Data commands. These commands are listed in the *Compiler Commands* and *User Data Commands* chapters of the *UXE Command Reference Manual*. To specify these commands, include these commands in a file called `compilerOptions.gel` and place this file in the directory in which the design is being compiled and run. IXCOM sources these commands from the `compilerOptions.gel` automatically while compiling the design.

Note: In the `compilerOptions.gel` file, if you need to specify a User Data command (for example, the `moduleFV` command) that requires naming the library, use `hdlldb` for IXCOM flow, because IXCOM uses a fixed library, `hdlldb`.

Passing Options to HDL Compilation Utilities from the IXCOM Command Line

To pass options to the HDL compilation utilities, such as `vavlog`, `vaelab`, and `vavhdl`, use the `-vaelabargs`, `-vavlogargs`, and `-vavhdlargs` options, respectively, with the `ixcom` command. For details about the usage of these utilities, refer to the section titled [Using UNIX Commands for RTL Import and Synthesis](#) on page 149.

For example:

```
ixcom -ua +dut+<top_level_module> <>.v -vaelabargs <vaelab_options>
```

Modifying the IXCOM Compilation Flow

The `compile.gel` file can be used to modify the IXCOM compilation flow. The commands that are not normally executed by IXCOM can be included in the file.

To modify the IXCOM compilation flow, use the following method:

1. Create the `compile.gel` file in the IXCOM compilation directory. You can use the `ixcom.gel` or `xeCompile.key` files as a template to create the `compile.gel` file.

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

For example:

```
cd <my_compile-dir>
cp $UXEHOME/share/uxe/etc/qel/ixcom.qel compile.qel
```

If the `ixcom.qel` file is used as a template, the following section should be removed from the `compile.qel` file:

```
if {[file exists compile.qel]} {
    source compile.qel
    exit
}
```

Note: The `compilerOptions.qel` file overrides the User Data in the `compile.qel` file if both files are present in the IXCOM compilation directory.

2. Modify the `compile.qel` file. You can include commands like `terminalAssign`, `icePrepare`, and `compileFind` in the file.

For example:

```
importOption {mode full}
importOption {library hdlldb}
importOption {V1995 ON}
netlistFile {verilog xc_work/xc_va.ver}
designImport
design hdlldb xcva_top
memoryTransform -add {* SHALLOW off }
precompile
terminalAssign -add {myCable1 P2 17 data[0]}
icePrepare
compileFind -best -maxtrials 10 -maxstep 400
```

3. Compile the design files using the `ixcom` command.

Analyzing Mixed-Language Designs

To analyze a mixed-HDL design, perform the following steps:

1. Set up a `libmaprc` file to associate the desired physical libraries to logical library names, for example:

```
VERILOG => /asic/verilog/verilog_lib
```

See [Associating Logical Libraries to Physical Directories Using libmaprc File](#) on page 210 for more detail.

2. Analyze the Verilog sources to one or more library using `vlan`. (The default library is `WORK`.)
3. Analyze the VHDL sources to one or more library using `vhan`. (The default library is `WORK`.)

You can mix VHDL and Verilog throughout a design, with a few caveats. These caveats are described as guidelines in the following sections:

- [Instantiating Verilog in VHDL](#)
- [Instantiating VHDL in Verilog](#)

Instantiating Verilog in VHDL

Instantiating a Verilog module in a VHDL architecture is like instantiating any other VHDL component: write a component declaration for the Verilog module, then instantiate the component. You can use the `vhshell` utility to automatically generate the component declaration for the Verilog. (Refer to [vhshell](#) on page 1382 for more information on this utility.)

Instantiating VHDL in Verilog

IXCOM supports the *defacto_standard* escaped identifier mechanism to explicitly bind a VHDL component instantiated in a Verilog module:

```
\library_name.configuration instance-name (port_list);  
\library_name.entity(architecture) instance-name (port_list);
```

The VHDL component is specified either with a *configuration* or an *entity(architecture)* pair. If the *(architecture)* parameter is omitted, the last compiled architecture is used. The `<library_name>` specifies the logical library containing the VHDL design unit. By default, it is `WORK`. To change the `<library_name>`, specify it with the `-work` option.

For more detail, see the following sections:

- [Searching Specific Libraries Using the -L library Search Switch](#)
- [Searching Specific Libraries Using the `uselib Compiler Directive for Verilog](#)
- [Design Binding and Search Rules in IXCOM](#)
- [Cadence NCSim-Compatible Search for IXCOM](#)
- [Instantiating VHDL Design Units in Verilog](#)
- [Mapping VHDL to Verilog Values](#)
- [Mapping Verilog to VHDL Values](#)

Searching Specific Libraries Using the -L library Search Switch

As an alternative to the previous syntax, you can specify just the VHDL entity name in a standard Verilog instantiation. Then, pass one or more `-L library` options to `ixcom` to search a set of analyzed libraries for the desired component. This option can be used to find any component (VHDL or Verilog) that is not explicitly bound to a particular design unit. The compiler will first search the library of the design unit that instantiated the component, then search the list of `-L` libraries.

For example, if you have two components instantiated as follows in Verilog:

```
\ramlib1.sram(behav) s1 (Clk, Addr, Data, WE, Q1);  
sram s2 (Clk, Addr, Data, WE, Q2);  
ixcom -top top_level_design_unit other_compiler_options -L ramlib2
```

A library (`ramlib1`) is specified in the instantiation of component `s1`, so only that library is searched. There is no library specified in the instantiation of component `s2`, so the library, `ramlib2` (specified with the `-L ramlib2` option), is searched.

Searching Specific Libraries Using the `uselib Compiler Directive for Verilog

The ``uselib` compiler directive specifies the libraries that should be searched for the definitions of modules and User Defined Primitives (UDPs) that are instantiated in a design description that does not include their definitions. Each ``uselib` directive explicitly defines the library search that resolves the instances that follow it until the compiler encounters another ``uselib` directive, which completely redefines the search. An empty ``uselib` directive makes the preceding ``uselib` directives ineffective. The functionality associated with these ``uselib` extensions is tied to the `cds.lib` and `hdl.var` files that are used by `irun`.

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

For details about the ``uselib` compiler directive, refer to the “*The `uselib Compiler Directive*” section in the *Verilog Simulation User Guide* available in the INCISIV documentation set.

IXCOM does not support the following IUS extensions of the ``uselib` syntax:

```
`uselib lib = <logical_lib_name>
`uselib view = <view_name>
```

You need to manually modify these *irun*-supported ``uselib` extensions and convert these to the corresponding Verilog-XL style syntax that IXCOM supports, as follows:

```
`uselib dir = <dir_name>
`uselib file = <file_name>
`uselib libext = <ext_name>
```

Following is the mapping of ``uselib lib` to the corresponding ``uselib dir` extension:

The `lib` version takes the logical library name as an argument.

The `dir` version takes the name of the physical directory (corresponding to that logical directory name, as specified in the `cds.lib` file) as the argument.

The ``uselib view` extension maps to the ``uselib libext` extension based on the contents of the `cds.lib` file.

Design Binding and Search Rules in IXCOM

IXCOM uses the following rules for module binding:

- For the top-level module, IXCOM searches for the modules in the following order:
 - a. Verilog module
 - b. VHDL module
 - c. SystemC module
- For binding VHDL modules, IXCOM searches for the specified VHDL module in the following order:
 - a. In the parent library
 - b. In the current library
 - c. In the libraries mentioned with the `use` clause
 - d. In all the libraries specified in the `libmaprc` file, if `-relax` option is passed

- e. Searches for a Verilog module if no VHDL module is found, in the following order:
 - In the parent library
 - In all the libraries, starting with the first entry in the `libmaprc` file, if no Verilog module is found in the parent library
- f. Searches for a SystemC module if no Verilog module is found, in the shared libraries specified with the `-loadsc` option.

Note: The *current* library refers to the library specified with the `-w <library>` option, and the *parent* library refers to the library that contains the parent module.

- For binding Verilog modules, IXCOM searches for the specified Verilog module in the following order:
 - a. In the parent library
 - b. In all the libraries specified in the `libmaprc` file, if `-relax` option is passed
 - c. Searches for a VHDL module if no Verilog module is found, in the following order:
 - In the parent library
 - In all the libraries specified in the `libmaprc` file
 - d. Searches for a SystemC module if no VHDL module is found in the parent library.
- For binding SystemC modules, IXCOM:
 - Searches for the specified SystemC module in the parent library.
 - Implements the same bind routine that is implemented for a Verilog module.

Note: In case of a SystemC module, the parent and the current libraries are same.

- When `-L <library>` option is passed, then IXCOM searches for:
 - The VHDL module in the library(s) specified with the `-L` option.
 - A Verilog module if no VHDL module is found, in the library(s) specified with the `-L` option.
 - The Verilog module in the library(s) specified with the `-L` option.
 - A VHDL module if no Verilog module is found, in the library(s) specified with the `-L` option.

Note: You can specify multiple libraries with the `-L` option.

- When the `-relax` option is passed to IXCOM, the VHDL modules are searched in all the libraries, starting with the first entry in the `libmaprc` file.

For more information on design-unit binding and search rules, refer to *NC-VHDL Simulator Help* user guide of the INCISIV documentation set.

Cadence NCSim-Compatible Search for IXCOM

The `-ncsim` option with `ixcom` can be used to enable Cadence NCSim-compatible searching of entities and modules not found in `-L` libraries. Libraries specified in the `libmaprc` file are searched for VHDL or Verilog components not found in libraries specified with `-L` options. Libraries are searched in the order specified in the `libmaprc` file.

Instantiating VHDL Design Units in Verilog

VHDL design units can be instantiated in Verilog only if the VHDL ports that connect to Verilog signals are of supported types and vectors of these types. Other data types cause errors at compile time.

Note: The VHDL signals cannot be forced from Verilog modules.

Mapping VHDL to Verilog Values

When a VHDL signal drives a Verilog port (`input` or `inout` to Verilog), the VHDL values are mapped to the Verilog values. The mapping of these values is described in detail in the *Mixed Verilog/VHDL Simulation* chapter of the *Verilog Simulation User Guide* and the *VHDL Simulation User Guide*. Both of these user guides are provided in the INCISIV documentation set.

Mapping Verilog to VHDL Values

When a Verilog signal drives a VHDL port (`input` or `inout` to VHDL), the Verilog values are mapped to the VHDL values. The mapping of these values is described in detail in the *Mixed Verilog/VHDL Simulation* chapter of the *Verilog Simulation User Guide* and the *VHDL Simulation User Guide*. Both of these user guides are provided in the INCISIV documentation set.

Associating Logical Libraries to Physical Directories Using libmaprc File

The *logical library* concept, an essential aspect of the mixed-HDL design flow, gives you a way to organize VHDL and Verilog design units into directory structures that can be easily referenced in VHDL statements and read by downstream tools.

The `libmaprc` file associates a logical library name with a physical directory path containing the actual analyzed design units. You can use one or more `libmaprc` files—the files are all read to generate a list of logical library definitions referenced during analysis and compilation. `libmaprc` files are required by the `vhan` analysis tool, the `ixcom` compiler, and other utilities like `vhls`.

libmaprc File Syntax

Here is the basic syntax for the `libmaprc` file:

```
logical-library-name => physical-directory-path
```

logical-library-name is not case sensitive, but *physical-directory-path* is case sensitive. For example:

```
RAMLIB => ./axis_compile_libs/ramlib
```

The logical library name RAMLIB (not case sensitive) is referenced in VHDL source code or in `-work` options to various mixed-HDL tools.

Note: The last directory in each directory path does not need to exist—it will be created automatically.

Comments in the libmaprc File

The `-` and `#` characters are used to delineate a comment in the `libmaprc` file. For example:

```
RAMLIB => ./axis_compile_libs/ramlib -- A comment  
# Another comment
```

include Keyword in the libmaprc File

Use the `include` keyword in your `libmaprc` file to include the contents of another `libmaprc` file at a specified point in the file. For example:

```
RAMLIB => ./axis_compile_libs/ramlib  
-- Following statement includes contents of another libmaprc file  
include /net/designs/ASIC/romlibs/libmaprc
```

Using libmaprc Files

The easiest way to use a libmaprc file is to copy the `$AXIS_HOME/packages/libmaprc` file to your local project directory; then add your own libraries to the file. The libmaprc file in the local directory will be found first by mixed-HDL tools.

Alternatively, you can specify a particular libmaprc file using the `-libmaprc` option to vhan, vlan, ixcom, or other mixed-HDL tool:

```
-libmaprc libmaprc_file
```

You can specify multiple `-libmaprc` options. The *libmaprc_file(s)* are read in the order specified to generate a list of logical libraries.

Note: If the same logical library is listed more than once in the libmaprc files, the first entry will take precedence.

Searching for libmaprc Files

If no `-libmaprc` options are specified, specific directory locations are searched for libmaprc files, in the following order:

1. Current directory
2. `$HOME` environment variable directory
3. `$AXIS_HOME` environment variable directory

Note: All libmaprc files found in these locations are read, in order to generate a list of logical libraries. If the same logical library is listed more than once in the libmaprc file, the first entry will take precedence.

Default libmaprc File

The default libmaprc file is located in the `$AXIS_HOME/packages` directory. Here are the contents of this file:

```
STD => $AXIS_HOME/packages/lib/STD
IEEE => $AXIS_HOME/packages/lib/IEEE
SYNOPSYS => $AXIS_HOME/packages/lib/SYNOPSYS
AXIS => $AXIS_HOME/packages/lib/AXIS
NCUTILS => $AXIS_HOME/packages/lib/NCUTILS
```

This file contains logic library associations for VHDL system libraries.

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

You can copy this file to your project directory and add your own design libraries as follows, for example:

```
WORK => ./axis_compile_libs/work
```

Alternatively, you can use the `include` keyword to include the `$AXIS_HOME/packages/libmaprc` file.

For example:

```
include $AXIS_HOME/packages/libmaprc  
<your_libraries_here>
```

System Libraries Referenced in the libmaprc File

The software installation includes the following VHDL system libraries, which are referenced in the default `libmaprc` file:

- STD
- IEEE
- SYNOPSYS
- AXIS
- NCUTILS

The default compiled system libraries are located in:

```
$AXIS_HOME/packages/lib
```

The source code for the system libraries is located in:

```
$AXIS_HOME/packages/src
```

Associating Multiple Logical Libraries to Same Physical Directory

You can associate more than one logical library to the same physical directory.

For example:

```
WORK => ./axis_compile_libs/work  
RAMLIB => ./axis_compile_libs/work
```

Note: A design unit (VHDL or Verilog) analyzed to a physical directory will override any design units that already exist in that directory, even if a different logical library was specified.

Working with Multiple libmaprc Files Using vhlib

The `vhlib` utility is useful when you are working with multiple `libmaprc` files. It provides a listing of all logical-library-to-physical-directory associations, as well as prints the `libmaprc` file from which each association was obtained. Here is the syntax for the `vhlib` utility:

```
vhlib [<logical_library_name>] [-libmaprc <libmaprc_file>] [-n]
```

If specified with no options, `vhlib` prints the logical-library-to-physical-directory associations of all `libmaprc` files specified with options or all `libmaprc` files found in the search order described in [Searching for libmaprc Files](#) on page 211. It also prints the full path to the `libmaprc` file in which each association was obtained.

If `<logical_library_name>` is specified, only the information for that particular library is specified.

If `-libmaprc libmaprc_file` is specified, `vhlib` prints the logical-library-to-physical-directory associations in the specified `libmaprc` file only, rather than all `libmaprc` files found.

Specifying `-n` causes only the logical library association to be printed and not the `libmaprc` file information.

For example:

```
vhlib RAMLIB
..
-- file:  /net/designs/ASIC1/libmaprc
RAMLIB => /net/designs/ASIC1/ramlib
..
```

Converting cds.lib Files to libmaprc Format

The `cds.lib` files are used with INCISIV utilities, and have the following syntax for mapping logical libraries to physical libraries:

```
DEFINE <logical_library> <physical_library_path>
```

For example:

```
DEFINE lib1 ./lib
DEFINE lib2 ./pci_lib
```

The above syntax is a little different from that of the `libmaprc` file. ([libmaprc File Syntax](#)) Therefore, the `cds2libmap` utility is provided to help convert existing `cds.lib` files to equivalent `libmaprc` files.

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

To use the `cds2libmap` utility, use the following command:

```
cds2libmap [-h] <cdslib_file> [-o <output_file>]
```

The `<output_file>` is the equivalent `libmaprc` file.

See [cds2libmap](#) on page 1377 for more information.

Note: `cds2libmap` utility can also be used to convert IXE specific `va.lib` files to equivalent `libmaprc` files.

Keep the following points in mind when using the `cds2libmap` utility:

- The `cds2libmap` utility needs to be run individually on all the relevant `cds.lib` files.
- For a particular `cds.lib` file, the `cds2libmap` utility generates a single `libmaprc` file that contains all the `DEFINES` resolved from the `cds.lib` file and all the included files.
- All the relative paths in the `cds.lib` file (and included `cds.lib` files) are resolved and dumped as absolute paths. Thus, if the design or some part of the design is moved to a different location, the conversion utility must be run again on the `cds.lib` files.
- The `cds2libmap` utility resolves the standard locations of the predefined libraries, such as IEEE and SYNOPSIS in the `cds.lib` file to the corresponding location in the IXCOM hierarchy.
- Multiple definitions for the same logical library are resolved to a single definition according to the `cds.lib` resolution semantics. Thus, the last definition overrides the older definitions.

Compiling Mixed-Language Designs

During compilation, IXCOM elaborates the design using the analyzed VHDL/Verilog design units. Then, the design is automatically partitioned into synthesizable and non-synthesizable parts. The synthesizable parts can run on the emulator; non-synthesizable parts can run only in the software simulator. See [Compiling for Hardware Using IXCOM](#) on page 202.

Following are the commands used for analyzing and compiling a mixed-language design.

```
vlan <file1.v> <file2.v>
vhan <file1.vhd> <file2.vhd>
ixcom -top <top_tb> -ua +dut+<top_dut>
```

Here is the basic syntax of the `ixcom` command:

```
ixcom -top top_level_VHDL_Verilog_unit_name \
      [-ua +dut+<entity_or_module> other_options]
```

The `ixcom` command performs the following tasks:

- Compiles the modules in the source files specified with the `.v` extension for a Verilog-only design. (IXCOM detects the file types according to the filename extensions. See [Determining File Type by Source Filename Extension](#) on page 234.)
- Elaborates the design.
- Generates the corresponding binary source code of the design.
- Compiles the design into a simulation snapshot that can be run on the emulator if the `-ua` option is given with the `ixcom` command.

The `ixcom` command supports all standard Verilog and/or VHDL options. In addition, it supports command-line options for integrating PLIs and the options for compiling. The `ixcom` command options are of following types:

- Dash options - beginning with a dash (-) symbol
- Plus arguments - beginning with a plus (+) symbol (See [Using Plus Arguments](#) on page 301)

Handling Time Unit and Precision Settings

The Verilog language allows a way to specify the time unit/precision per module. In VHDL, the time unit/precision needs to be set according to the tool being used.

- ❑ For VHDL: Pass the `-vhdl_time_precision <time_precision>` to IXCOM. For example, `-vhdl_time_precision 1ns`. IXCOM sets this time precision for all VHDL modules.
- ❑ For Verilog: If a module does not have the time unit/precision, you can set the time unit/precision by using the `-timescale <unit/precision>` option of the `ixcom` command.

The global minimum time unit/precision is set after considering both the VHDL and Verilog modules in the design.

Note: The same time unit and time precision settings should be specified for IXCOM and *irun*. For example, the `uxc_hdl.var` file, which is used to pass options to *irun*, should not include a separate setting if you have already passed these options to IXCOM.

Partitioning Designs into RTL (Hardware) and Behavioral (Software)

IXCOM parses the entire design, running RTL checks, to determine the synthesizable and non-synthesizable portions. It automatically partitions the non-synthesizable portions to be run in the software partition and the synthesizable portions to be run on the hardware emulator.

The running of the RTL checks is controlled by the `-rtlchk` option of the `ixcom` command. The `-rtlchk` option has three values: `warn`, `off`, and `error`. Of these, `-rtlchk warn` is the default setting. This means that all modules in the DUT module hierarchy (specified using the `+dut+ <module_name>` option) are considered synthesizable, and partitioned to emulator. If a specified module does not pass the RTL checks, a warning is generated and the specified module and its entire hierarchy are partitioned to the software side. However, you can change this default behavior by specifying the value as `off` (completely switching off the RTL checks) or `error` (generating an error and stopping compilation if a module does not pass the RTL check).

Based on the value specified for the `-rtlchk` option (`warn`, `error`, or `off`), IXCOM takes the following actions:

- `-rtlchk warn`: Displays a message and pushes the non-synthesizable module to the software partition.

The non-synthesizable design constructs are reported with the `RTL_IGNORE_` prefix in the message, but the compilation process is not interrupted. The module containing the non-synthesizable constructs along with the hierarchy of modules under it is pushed to the software simulator side.

Note: The compilation aborts if the top-level DUT module (specified using the `+dut+` option) is pushed to the software simulator side.

- `-rtlchk error`: Displays a message and aborts the compilation.

The non-synthesizable design constructs are reported with the `RTL_UNSUPPORTED_` prefix in the message and compilation is aborted.

Note: If you intend to run `tbrun` or `autorun`, use the `-rtlchk error` option so that design violations are addressed at the compile time, pushing design modules to the software partition by using the `-rtlchk warn` option could lead to incorrect results in the simulation.

- `-rtlchk off`: Disables the automatic partitioning of the design.

When the `-rtlchk off` option is specified, no design partitioning takes place. IXCOM passes all the modules under the `+dut+<mod_name>` hierarchy to HDL compiler

without conducting any RTL synthesis checks. Then, if HDL Compiler detects any non-synthesizable modules, it generates an error message and aborts compilation.

If required, you can direct the partitioning of the designs manually:

- Entire module hierarchy partitioned to the hardware side by using the `+dut+<module_name>` option
- Entire module partitioned to the software side by marking it as non-synthesizable
To mark a module appearing in the hierarchy under the DUT module as non-synthesizable, specify the non-synthesizable module with the `+dutexcl+<module_name>` option. Then, this module, and its entire hierarchy will be partitioned to the software side, while the rest of the hierarchy under the DUT module will be on the emulator.
- Mark parts of the synthesizable module to be run on the software side by modifying the design code and adding the required testbench callback statements (See [Executing Non-Synthesizable Code in the Emulator](#) on page 268).
- Use the `+no_rtlchk+<module_name>` option of the `ixcom` command to skip automatic partitioning for a particular module or set of modules of the design. Use this option for a module or set of modules that might work correctly on the emulator but the automatic design partitioning pushes the module to the software partition.

Compilation Log Files

A number of log files are generated during compilation. All of these files are created in the current directory by default, but you can use the `-l <logfile>` option with any of these commands to specify a different filename and/or directory path.

- `vhan.log` (VHDL only)

Created by the VHDL analyzer program, `vhan`, this file contains messages relating to the syntax checking of your VHDL design.

- `vlan.log` (Verilog only)

Created by the Verilog analyzer program, `vlan`, this file contains messages relating to the syntax checking of your Verilog design.

- `ixcom.log`

Created by the `ixcom` command, this file contains a log of the complete compilation process along with the list of options used for the compilation process. The `ixcom.log` file also contains the following information if you are using the `-ua` option with the `ixcom` command for compiling:

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

- ❑ Messages produced by the Synthesis Policy Checker
- ❑ Signal Analysis report

Compiling SystemC Designs

IXCOM supports SystemC with any combination of Verilog and VHDL files. You can also simulate SystemC-only designs using this process.

SystemC supports a limited set of data types at the boundary between a Verilog or VHDL component and a SystemC model. Supported type combinations are listed in [Table 4-1](#) on page 219.

Supported compilers are GCC G++ 4.1.2 on Linux.

Supported Types

[Table 4-1](#) on page 219 lists SystemC, Verilog and VHDL types supported by SystemC at the boundary. Any SystemC type from this list can be connected to any Verilog or VHDL type from the list. The only requirement is that the bit size of the connected ports must be the same.

Note: User-defined VHDL types are not supported on the SystemC boundary.

Table 4-1 SystemC, Verilog, and VHDL types supported by SystemC level 2

SystemC Types	Verilog Types	VHDL Types
bool	net (scalar and vector)	bit
sc_bit	reg (scalar and vector)	std_[u]logic
sc_logic	integer	boolean
[unsigned] char		bit_vector
[unsigned] short		std_[u]logic_vector
[unsigned] int		array of boolean
[unsigned] long		
sc_[u]int<N>		
sc_big[u]int		
sc_bv<N>		

Table 4-1 SystemC, Verilog, and VHDL types supported by SystemC level 2, *continued*

SystemC Types	Verilog Types	VHDL Types
sc_lv<N>		

Developing and Simulating SystemC Designs

This section describes a procedure to develop and simulate different kinds of mixed-HDL/SystemC designs.

Cadence recommends that you write a makefile to automate this flow. You will find examples of makefiles in `<UXE_installation_directory>/share/uxe/gift/tutorial/systemc/`.

Note: For using SystemC, only the standard IXCOM environment is required; you do not need to do any special environment settings.

Designing and Coding Recommendations

Here are some recommendations when you are designing and coding for SystemC.

- Define cross-boundary ports using any of the types listed in [Table 4-1](#).
- Add the `NCSC_MODULE_EXPORT` macro for modules that are going to be instantiated in Verilog or VHDL.
- Define an instance of a Verilog or VHDL component in SystemC as a `sc_foreign_module`.
- Do not use global variables of type `SC_MODULE`.
- You do not need any special markers for SystemC modules instantiated in Verilog or VHDL.
- If you need to pass Verilog parameters or VHDL generics to the instantiated SystemC module, use the method described in the SystemC User Guide in the INCISIV documentation set.
- Create a SystemC wrapper for each Verilog or VHDL component instantiated in SystemC. Details about the wrapper coding can be found in [Instantiating Verilog/VHDL under SystemC](#) on page 223 .

Compiling and Linking SystemC Components

To compile and link your HDL and SystemC components:

- Choose the design libraries with which your SystemC modules will be associated.
- Use `ncsc_run` to analyze SystemC files that contain SystemC modules to be instantiated in Verilog or VHDL.
- Separately analyze each mixed-HDL component. For VHDL files use `vhan` and for Verilog files use `vlan`.

Compiling SystemC Designs

To compile a SystemC design, follow the same process as for standard mixed-HDL flow elaboration, with the addition of the option, `-loadsc`. Here is an example command line:

```
ixcom -top <top_module> -loadsc ./libncsc.so
```

You must specify all SystemC top modules (not instantiated in HDL) with the `-top` option. The constructors of SystemC modules are started twice: once during elaboration, and again at run time.

If you specify `sc_main` as a *top_module*, the `sc_main()` global function is started; otherwise, `sc_main()` is not used.

Using the `sc_main()` Global Function

If you pass the `-top sc_main` option to the elaborator, as described in the preceding section, the `sc_main()` global function is started. Here is the syntax of the function:

```
sc_main(int argc, char *argv[]) {  
  
    ...  
}
```

The `sc_main` function is called before any other SystemC modules instantiated in HDL or specified as `-top` modules.

To pass the arguments to the `sc_main` function, it is necessary to use the `+systemc_args+<args>` option of the `ixcom` command. During elaboration, `argv` contains all these options specified for `ixcom`. During run time, `argv` contains all plus options specified during elaboration and all options passed to *irun*.

All elaborator arguments specified with a + option are accessible in `sc_main()` by means of `argc` and `argv`, both during elaboration and at run time. It is not necessary to repeat these options at run time.

SystemC and Mixed Modules as Top Modules

A SystemC module can be the topmost module of the design. To enable it, declare this module using the `NCSC_MODULE_EXPORT` macro. The second argument of the macro is the name of the top module that should be specified with the `-top` option on the `ixcom` command line.

You can have multiple top modules, some in Verilog/VHDL and others in SystemC.

Simulating SystemC Designs (*irun*)

SystemC designs are simulated just like standard mixed-HDL flow designs with *irun*.

To pass the arguments to the `sc_main` function, it is necessary to use the `+systemc_args+<args>` option of the `ixcom` command.

Instantiating SystemC under Verilog/VHDL

Verilog and VHDL instantiate a SystemC model using the same syntax used to instantiate a corresponding HDL instance.

On the SystemC side, declare the instantiated module types using the following special macro:

```
NCSC_MODULE_EXPORT(<systemc_module_name>,  
  <verilog_module_name>|<vhdl_module_name>);
```

The first argument to the macro is the class name of the `sc_module` that you want to instantiate. The second argument specifies the HDL module name.

There are two reasons for showing two different arguments (`<verilog_module_name>|<vhdl_module_name>`) in the macro:

- VHDL is not case sensitive, while Verilog and SystemC are case sensitive. Therefore, the case of VHDL component and port names (for named connections) does not need to match the case of the SystemC counterparts. The case of Verilog and SystemC identifiers, however, must be identical.
- Not all SystemC module (type) names can be used in Verilog. For example, templated module names (such as `my_and<4>`) cannot be used in Verilog.

Some SystemC module names cannot be used in the `XSC_EXPORT_MODULE` macro. For example, if a SystemC module name is templated, it can use the comma separator. The comma character (,) is considered by the macro to be a separator. To work around this restriction, you might need to add a special `typedef` declaration to alias the SystemC module type name to one that can be used in the macro.

Every SystemC module you instantiate under Verilog or VHDL must have a constructor that takes only one argument of type `sc_module_name`. Alternatively, you can just use the standard macro `SC_CTOR()`, which declares such a constructor.

Note: You must specify `NCSC_MODULE_EXPORT` in the `.cpp` file.

A warning is issued if the symbolic name is not specified for the cross-boundary SystemC port, or if the symbolic name is specified, but does not match the corresponding port name on the HDL-side. The symbolic name must match even if you are using positional binding of ports. The symbolic name is required for visibility of the ports in waveform viewers. If a mismatch exists, a warning is issued by the elaborator.

Instantiating Verilog/VHDL under SystemC

To instantiate a Verilog module or VHDL component in the SystemC module, you need to create a shell. Use the `ncshell` utility to create this shell or follow the method given below.

Declare a new wrapper class derived from the class `sc_foreign_module`. This class is derived, in turn, from the standard SystemC `sc_module` class and is a Cadence extension to the SystemC library `libxsc`. The name of the wrapper class must match the Verilog module name or VHDL entity name. The wrapper class definition can be written manually or by using the `ncshell` utility.

The rules for writing wrappers are:

- Define the corresponding C++ class derived from the predefined class `sc_foreign_module`. The macro `SC_FOREIGN_MODULE` can be used to achieve the same functionality.

For example:

```
SC_FOREIGN_MODULE(Mult)
```

- Declare the ports of the wrapper as follows:

The order of the ports must match exactly the order of the formal ports in the HDL module (or entity) declaration.

The wrapper port names must be exactly the same as the names of the formal ports.

Port length in bits must be the same as in HDL.

- Define a constructor of the wrapper module as follows:

```
SC_CTOR(Mult): port-names-list {}
```

where *port-names-list* looks like this:

```
port-field-name("actual-port-symbolic-name"), ...
```

For example:

```
SC_CTOR(Mult) : a("a"), b("b"), clk("clock") {}
```

Support for Multiple SystemC Shared Libraries

The IXCOM flow supports multiple SystemC shared libraries that are created by the `ncsc` command. This support ensures that the SystemC libraries obtained from multiple sources are also supported. Another benefit of supporting multiple SystemC libraries is that it enables you to partition the SystemC code in multiple libraries. These SystemC shared libraries are created using the `ncsc` command and are passed to IXCOM by using the `-loadsc` option.

For example, to load multiple SystemC shared libraries, `libncsc_model1.so`, `libncsc_model2.so`, and `libncsc_model3.so`, the IXCOM command is:

```
ixcom -loadsc libncsc_model1.so -loadsc libncsc_model2.so <other ixcom options>
```

Using Specman Elite with IXCOM

You can use IXCOM to compile both Verilog and mixed-language DUT in the Specman environment, with a Specman **e** testbench.

This section discusses the following:

- [Compiling and Running a Verilog Design in the Specman Environment](#)
- [Compiling and Running a Mixed-Language Design in the Specman Environment](#)

Compiling and Running a Verilog Design in the Specman Environment

In the Specman environment, use IXCOM to compile a Verilog or mixed-language DUT and *irun* to compile the Specman **e** testbench and simulate the DUT. To compile a Verilog design in the Specman environment, you need a Specman-to-DUT reference file and a Specman stub file. The Specman-to-DUT reference file, typically named as `specman_hdl_ref.v`, is required by IXCOM whereas the Specman stub file, typically named as `specman.v`, is required by *irun*.

For example, suppose you have a Verilog DUT, `xor.v`, a Specman **e** testbench, `xor_verify.e`, and a sample script file, `xc.t` that contains the simulation acceleration commands.

A sample script file, `xc.t`, contains the following commands:

```
run 200ns
xc on -run
run
exit
```

Perform the following steps to compile and run the DUT with the Specman **e** testbench:

1. Create a Specman-to-HDL reference file manually based on how each DUT signal is used in Specman testbench. Refer to [Accessing DUT Signals from Non-HDL Interfaces](#) for details about creating such a file. For example:

```
$export_read(xor_try.clk);
$export_deposit(xor_try.inp_xor);
$export_read(xor_try.out_xor);
```

Note: You can get the list of HDL-referenced signals from Specman testbench by using the `dump_hdl_ref` command in the Specman tool. Specify the `specman` command to generate the Specman-to-DUT reference file and the Specman stub file, as shown:

Use the following command to generate the Specman-to-DUT reference file, `specman_hdl_ref.v`.

```
specman -command "load xor_verify.e; dump hdl_ref -verilog_top"
```

A sample `specman_hdl_ref.v` file is given below:

```
module specman_hdl_ref;
    initial begin
        $export_spexsim(xor_try.clk);
        $export_spexsim(xor_try.inp_xor);
        $export_spexsim(xor_try.out_xor);
    end
endmodule
```

Be sure to modify the `specman_hdl_ref.v` based on the way each signal is used in the Specman testbench before continuing the next step. Modify the `specman_hdl_ref.v` file if the port bindings for the DUT signals are `e` string variables instead of the tick (```) notations, because some of the DUT access signals might not be extracted from the testbench.

2. Generate the Specman stub file, `specman.v` for *irun*, as shown:

```
specman -command "load xor_verify.e; write stubs -ncvlog"
```

3. Specify the `ixcom` command to compile the Verilog DUT and the Specman-to-DUT reference file, `specman_hdl_ref.v`, as shown:

```
ixcom -ua +dut+xor_try +iscDelay+clk_gen xor.v specman_hdl_ref.v
```

4. Specify the `irun` command to compile the Specman testbench and the Specman stub file. This command also elaborates the DUT objects with Specman, as shown:

```
irun -elaborate xor_verify.e specman.v -f xc_work/irun.f
```

5. Specify the `irun` command to run the design, as shown:

```
irun +dpvcl -R -input xc.t
```

In the above command, the `+dpvcl` option is required to enable IXCOM to capture changes in signal values by Value Change Callback (VCL). This is because Specman modifies the DUT signal values directly in the *irun* space.

Compiling and Running a Mixed-Language Design in the Specman Environment

In the Specman environment, use IXCOM to compile a mixed-language DUT, which can contain SystemVerilog, Verilog, and VHDL portions. Also, use *irun* to compile the Specman `e` testbench and simulate the DUT. For a mixed-language design, you need a Specman-to-DUT reference file and a Specman stub file. The Specman-to-DUT reference file, typically

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

named as `specman_hdl_ref.v`, is required by IXCOM whereas the Specman stub file, typically named as `specman.v`, is required by *irun*.

For example, suppose you have a VHDL DUT, `<xor>.vhd`, a Verilog file, `clk_gen.v`, and a Specman **e** testbench, `xor_verify.e`. You also have a sample script file, `xc.t` that contains the following simulation acceleration commands:

```
run 200ns
xc on -run
run
exit
```

Perform the following steps to compile and run the VHDL DUT with the Specman **e** testbench:

1. Create a Specman-to-HDL reference file manually based on how each DUT signal is used in Specman testbench. Refer to [Accessing DUT Signals from Non-HDL Interfaces](#) for details about creating such a file. For example:

```
$export_read(xor_try.clk);
$export_deposit(xor_try.inp_xor);
$export_read(xor_try.out_xor);
```

Note: You can get the list of HDL-referenced signals from Specman testbench by using the `dump_hdl_ref` command in the Specman tool. Specify the `specman` command to generate the Specman-to-DUT reference file:

Use the following command to generate the Specman-to-DUT reference file, `specman_hdl_ref.v`.

```
specman -command "load xor_verify.e; dump_hdl_ref -verilog_top"
```

Be sure to modify the `specman_hdl_ref.v` based on the way each signal is used in the Specman testbench before continuing the next step. Modify the `specman_hdl_ref.v` file if:

- ☐ The target signal is only forced or released but not read or written in the Specman testbench, then change `$export_spexsim` to `$export_frcrel`.
- ☐ If the port bindings for the DUT signals are E string variables instead of the tick (‘) notations, then some of the DUT access signals might not be extracted from the testbench.

2. Generate the Specman stub file, `specman.v` for *irun*, as shown:

```
specman -command "load xor_verify.e; write stubs -ncvlog"
```

3. Specify the `vlan` and `vhan` commands to analyze the Verilog and VHDL source files, as shown:

```
vlan specman_hdl_ref.v
vhan clk_gen.v
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
vhan xor.vhd
```

4. Specify the `ixcom` command to compile the VHDL DUT and the Specman-to-DUT reference file, `specman_hdl_ref.v`, as shown:

```
ixcom -ua +dut+xor_try +iscDelay+clk_gen -top xor_try -top specman_hdl_ref
```

5. Specify the `irun` command to compile the Specman testbench and the Specman stub file. This command also elaborates the DUT objects with Specman, as shown:

```
irun -elaborate xor_verify.e specman.v -f xc_work/irun.f
```

6. Specify the `irun` command to run the design, as shown:

```
irun +dpvcl -R -input xc.t
```


Compiling Testbenches Separately

When compiling designs with IXCOM, you have the option to partition the design in such a manner that you can compile the testbench portion and elaborate the entire design separately using IES tools, while the DUT is compiled using IXCOM. Depending on the design hierarchy, it is possible that the testbench instantiates the DUT or the testbench itself is instantiated within the DUT.

Accordingly, you can use the following options of the `ixcom` command:

- If the DUT is instantiated within the testbench, you can use the `+xcDesignTop` option to enable you to compile the testbench separately using IES tools and compile the DUT using IXCOM. Refer to the section titled [Compiling Top-Level Testbenches Using `+xcDesignTop`](#) on page 230 for more information.
- If the testbench is instantiated within the DUT, you can use the `+xcEmbeddedTb` option to enable you to compile the testbench separately using IES tools and compile the DUT using IXCOM. Refer to the section titled [Compiling Embedded Testbenches Using `+xcEmbeddedTb`](#) on page 232 for more information.

Linking DUT with Multiple Testbenches

IXCOM enables you to compile the testbenches separately from the DUT, thereby, enabling you to link a DUT with multiple testbenches, without needing to recompile the DUT with each testbench. For this, use the `+xcDesignTop` and `+xcEmbeddedTb` options of the `ixcom` command.

In this case, you compile the DUT using the `ixcom` command and the testbenches using the `irun` command.

Following are the advantages of using the `+xcDesignTop` and `+xcEmbeddedTb` options:

- Sharing the DUT with multiple testbenches, reducing compilation time and improving the efficiency of the compilation process:

Compiling the DUT is a time-consuming process. You might want to compile the DUT with multiple testbench environments without recompiling the DUT every time. If there are multiple testbenches that need to be compiled with the DUT, the compilation time increases. Using the `+xcDesignTop` and `+xcEmbeddedTb` options, you can compile the DUT and the testbench separately, and later link the separately compiled DUT and testbench. Therefore, in case of multiple testbenches, you only need to compile the different testbenches, whereas the DUT needs to be compiled only once.

- Using `irun` to compile HVLs that cannot be mapped to the emulator:

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOT

Modules written in certain HVLs, such as SystemC, e, and Vera cannot be mapped to the emulator, and are, therefore, used for testbenches only. Using the `+xcDesignTop` and `+xcEmbeddedTb` options, you can direct IXCOT to skip the compilation of the testbench modules written in these HVLs.

Following is the syntax of the `ixcom` command with the `+xcDesignTop` option:

```
ixcom [ixcom_options] \  
      +xcDesignTop+<hier_ref_to_DUT_instance_in_testbench>=<DUT_module_name>
```

Here, `<hier_ref_to_DUT_instance_in_testbench>` is the instance name of the DUT in the testbench and `<DUT_module_name>` is the name of the top-level module in the DUT.

Following is the syntax of the `ixcom` command with the `+xcEmbeddedTb` option:

```
ixcom [ixcom_options] +xcEmbeddedTb+<module_name>[+<module_name>...]
```

Here, `<module_name>` is the shell module manually created for the testbench module embedded in the DUT hierarchy to be compiled by `irun`. It can be written in Verilog or VHDL.

Note: The `+xcEmbeddedTb` option is used with the `ixcom` command and not with the `vhan` or `vlan` commands.

Compiling Top-Level Testbenches Using `+xcDesignTop`

Consider an example design that includes a DUT with the top module as, `adder` and a testbench, `mytb`.

The DUT is instantiated in the testbench with the name, `mydut` as shown below:

```
module mytb;  
    <statements>;  
    dutmodule dutinst(a, b, z);  
        <statements>;  
        <statements>  
endmodule
```

To compile using the `+xcDesignTop` option, perform the following steps:

1. Specify the `ixcom` command with the following options to compile the DUT, `adder`:

```
ixcom -ua +dut+dutmodule dutmodule.v +xcDesignTop+mytb.dutinst=dutmodule
```

2. Specify the `irun` command with the following options to compile the testbench separately:

```
irun -elaborate mytb.v -top mytb -f xc_work/irun.f
```

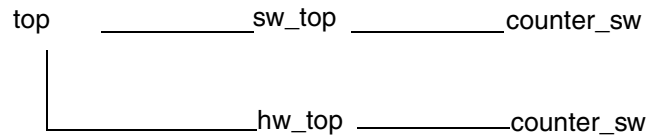
UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

In the above command, you can replace the testbench, `mytb.v` with a testbench written in any other HVL if the name of the testbench module remains the same as `mytb`, and the hierarchical path from the testbench to the DUT instance remains the same as `mytb.dutinst`.

Linking DUT with Multiple Testbenches Using INCISIV Tools

The IXCOM compiler generates `ncelab.f` and `ncsim.f` option files in the `xc_work` directory for elaboration and simulation, respectively. These option files are useful when instead of using *irun*, the individual INCISIV tools - *ncvlog* (or *ncvhdl*), *ncelab*, and *ncsim* are used to compile and elaborate the testbench and other non-synthesizable modules. For example, consider a design with the following hierarchy:



Here, `counter_sw` and `counter_hw` modules are described in VHDL while the other modules are described in Verilog. IXCOM compiles the `hw_top` and `counter_hw` modules for hardware. The `top`, `sw_top`, and `counter_sw` modules are compiled separately using *ncvlog* and *ncvhdl*. Then, the entire design is elaborated together using *ncelab*, followed by simulation using *ncsim*.

Following are the steps of this use flow:

1. Create a library mapping file `mylibmaprc` to be used by IXCOM, as shown below.

```
include <UXE_installation_directory>/share/uxe/packages/libmaprc
dut_vhdl_work => ./dut_vhdl_work
dut_vlog_work => ./dut_vlog_work
```

For more information, see [Associating Multiple Logical Libraries to Same Physical Directory](#) on page 212.

2. Analyze the DUT modules using *vhan* and *vlan*.

```
vhan counter_hw.vhd -libmaprc mylibmaprc -work dut_vhdl_work
vlan hw_top.sv -libmaprc mylibmaprc -work dut_vlog_work
```

3. Compile the DUT modules using *ixcom*.

```
ixcom +xcDesignTop+top.hw_top=hw_top -nccompile +dut+hw_top -top hw_top -ua -
libmaprc mylibmaprc -work dut_vlog_work
```

4. Create *irun* library file `uxc_cds.lib` to be used by INCISIV tools.

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
DEFINE tb_work ./tb_work
```

Here, `tb_work` is the logical library name and `./tb_work` is physical library directory name.

Note: The filename `uxc_cds.lib` cannot be changed because it is included automatically in the elaboration and simulation steps.

5. Analyze the testbench modules using `ncvhdl` and `ncvlog`.

```
mkdir tb_work
ncvhdl counter_sw.vhd -f xc_work/ncsim.f -work tb_work
ncvlog sw_top.sv top.v -f xc_work/ncsim.f -work tb_work
```

6. Use `ncelab` to combine and elaborate the testbench and DUT modules, and generate the snapshot.

```
ncelab top -f xc_work/ncelab.f -timescale 1ns/1ns
```

7. Use `ncsim` to simulate the snapshot.

```
ncsim top -f xc_work/ncsim.f -input hw_run.tcl
```

Limitations When Using `+xcDesignTop`

- The `+xcDesignTop` option does not support bidirectional signals, which require instrumentation of both the testbench and the DUT. The IXCOM compiler quits the compilation when it finds a potential bidirectional signal. You can use the `+dump_bidir` option of the `ixcom` command to dump such bidirectional signals that cause IXCOM to quit compilation.
- The `+xcDesignTop` option does not support use of the configuration name in the component instantiation statement in the testbench if the top-level DUT module is written in VHDL. You must use an entity name instead of the configuration name. The `<DUT_module_name>` in `+xcDesignTop` must also refer to the entity name.

Compiling Embedded Testbenches Using `+xcEmbeddedTb`

You can compile testbenches instantiated in the DUT hierarchy using the `+xcEmbeddedTb` option of the `ixcom` command so that the testbench module can be compiled by `irun` and the DUT is compiled by IXCOM.

Note: The `+dutexcl` and `+xcEmbeddedTb` options of the `ixcom` command are different. Use `+dutexcl` when a module is to be partitioned to the software simulator but can be processed in its entirety by IXCOM. Typically such a module is non-synthesizable, but does not contain any language constructs that are not supported by IXCOM, so the original module source is input to IXCOM with `+dutexcl`. Use `+xcEmbeddedTb` when the module contains

language constructs that are not recognized by IXCOM (for example, constructs written in an HVL).

For `+xcEmbeddedTb` you need to manually create a shell module for the testbench embedded in the DUT hierarchy and pass this shell module into IXCOM instead of the original embedded testbench. The shell module should contain only the parameters and port declarations of the actual testbench module. The functionality defined in the actual module is not required in the shell module. IXCOM uses these parameters and port declarations to generate the instrumentation logic to model the primary inputs and outputs.

Example

Consider an example of compiling a testbench module, `tb`, that is embedded within the DUT hierarchy, `dut`, by using the `+xcEmbeddedTb` option.

The testbench, `tb`, to be embedded in the DUT hierarchy is shown below:

```
// tb.sv
module tb(<port_formals>);
    <parameter_formals>;
    ...
    // The testbench contains features that
    // IXCOM cannot yet handle natively, but IES can.
    // Therefore, it cannot be passed to IXCOM.
    ...
endmodule
```

The testbench, `tb`, instantiated in the DUT, `dut`, is shown below:

```
// dut.sv
module dut;
    ...
    tb #(<param_actuals>) TB(<port_actuals>);
    ...
endmodule
```

The shell module, `tb`, (file `tb_shell.v`) should contain the parameters and port declarations as shown below:

```
module tb(<port_formals>);
    <parameter_formals>;
    // Nothing else needed here in the shell module
endmodule
```

To compile using the `+xcEmbeddedTb` option, perform the following steps:

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

1. Compile the DUT modules using the following `ixcom` command:

```
ixcom -clean +dut+dut -ua dut.sv tb_shell.v +xcEmbeddedTb+tb
```

2. Replace the existing `cds.lib` and `hdl.var` files with the files generated by the IXCOM compiler in the `xc_work` directory using the following commands:

```
rm -f cds.lib hdl.var
echo "softinclude ./xc_work/cds.lib" >> cds.lib
echo "softinclude ./xc_work/hdl.var" >> hdl.var
```

3. Analyze the testbench modules using the following `ncvlog` command:

```
ncvlog tb.sv
```

4. Elaborate the design using the following `irun` command:

```
irun -elaborate -f xc_work/irun.f
```

Limitations When Using `+xcEmbeddedTb`

- The `+xcEmbeddedTb` option is not supported with the `irun -hw` compilation command and gives an error.
- The `+xcEmbeddedTb` option does not support instantiation of the testbenches in the DUT hierarchy for which a shell module is not specified, and gives an error on compilation.
- The `+xcEmbeddedTb` option does not support instantiation of the testbenches in the DUT written in VHDL. It requires the DUT hierarchy to be in Verilog, and gives an error when the DUT hierarchy is not in Verilog. The testbenches instantiated in the DUT hierarchy can be in any language.

Determining File Type by Source Filename Extension

In the absence of Verilog mode options (`+v1995`, `+v2000`, or `+sv`), IXCOM determines the file type for a design element at the file level, based on the filename extension. Each recognized file type has a built-in, predefined set of file extensions. For each file type, there is a command-line option that you can use to change, or add to, the list of file extensions mapped to a given language.

Options for Verilog Design Compilation Mode

- ☐ `+sv`: Forces SystemVerilog compilation
- ☐ `+v1995`: Turns off new Verilog-2001 keywords

- ❑ `+v2000`: Allows new Verilog-2001 keywords

Options for File-Extension-Based Compilation Mode

- ❑ `-default_ext <file_type>`: Specifies Verilog file type or language version for unrecognized file extension(s)
- ❑ `-sysv_ext [+]<extns>`: Adds or overrides extensions for SystemVerilog sources
- ❑ `-vlog95_ext [+]<extns>`: Adds or overrides extensions for Verilog95 sources
- ❑ `-vlog_ext [+]<extns>`: Adds or overrides extensions for Verilog sources

The optional `+` symbol is used to add new extensions to the previous list of options. `<extns>` indicates one or more file extensions. Use a comma to separate multiple extensions.

For example:

```
-vlog_ext .v2k,.vLog  
-vlog_ext +.v2k,.vLog
```

If any of the Verilog mode options are specified, it means that you have explicitly specified the language (subset) to be used. Then, file-extension based compilation is not used.

The following table shows the recognized file types, the set of built-in, predefined file extensions for each language type, and the command-line option you can use to change, or add to, the list of file extensions mapped to a given language.

File Type	File extension	Compiler option
Verilog	.v, .V, .vp, .VP, .vs, .VS	-vlog_ext
Verilog 1995	.v95, .V95, .v95p, .V95P	-vlog95_ext
SystemVerilog	.sv, .SV, .svp, .SVP, .svi, .svh, .vlib, .VLIB	-sysv_ext

For example, the default file extensions for Verilog files are `.v`, `.vp`, `.vs`, `.V`, `.VP`, and `.VS`. For Verilog files with other extensions (for example, `.rtl` and `.vg`), it is essential to specify that these are valid extensions for Verilog files. Do this with the `-vlog_ext` option.

Additionally, the `-default_ext` option can be used to specify the file type for the respective filename extensions that are not specified by the built-in or user-specified extensions.

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

Note: By default, if no corresponding file type is set for a filename extension, the file type is set to Verilog-2001.

The argument to the `-default_ext` option is a string that represents a file type. For example, if the string specified with the `-default_ext` option is `systemverilog`, the file type is taken as SystemVerilog HDL.

Consider the following example that illustrates the filename-based compilation in IXCOM:

```
// File: top.sv
    reg [3:0] r;
    module Top; // sv module
        initial $unit::r = 11;
        Mid mid();
    endmodule // Top

// File: mid.v
    //
    module Mid;
        localparam p=1;
        Bot bot();
    endmodule // Mid

// File: bot.v95
    //
    module Bot;
        reg [3:0] int;
        initial int = 2;
    endmodule // Bot

// File: tip.sv
    // shares same compilation-unit
    module Tip;
        initial
            #1 display ($unit::r);
    endmodule // Tip
```

The following `irun` command is used for the above example:

```
irun top.sv mid.v bot.v95 tip.sv
```

Then, the Verilog version for each of the modules in the above example is determined as follows:

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

- Top - Verilog version set to SystemVerilog
- Mid - Verilog version set to Verilog-2000
- Bot - Verilog version set to Verilog-1995
- Tip - Verilog version set to SystemVerilog

Details of setting file types based on filename extensions are provided in the *Changing the Default Set of File Extensions* section of the *Customizing irun* chapter of the *irun User Guide* that is provided as a part of the INCISIV documentation set.

Compiling Behavioral Constructs

This section describes how IXCOM handles compilation of behavioral constructs.

During behavioral compilation, IXCOM maps the behavioral constructs in the DUT, specifically, behavioral BFM (Bus Functional Model) extensions to the RTL Verilog subset to the emulator for acceleration. You need to explicitly specify the modules that should undergo behavioral compilation.

A Verilog process is considered a behavioral process if any of the following conditions hold true:

- The `always` procedure contains any behavioral Verilog constructs.
- The `always` procedure is an asynchronous process that does not reference any objects in its sensitivity list.

The behavioral modules that you have specified are transformed one at a time and each behavioral process is mapped into an RTL-compliant process.

The IXCOM-generated RTL-compliant code has the following basic characteristics:

- All @ events in a single `always` procedure are identical and synthesizable.
- All processes and operations are synthesizable.
- The number of operations between consecutive @ events are bounded and can be completed in a single internal behavioral clock cycle.

Enabling Behavioral Compilation

To enable behavioral compilation for a module in the DUT, use either of the following options:

- ❑ Specify the module with the `+bc+<mod_name>` option of the `ixcom` command. The `<mod_name>` refers to the name of the DUT module that you have defined.

For example:

```
ixcom tpytha.v +dut+test_pytha +sv +bc+test_pytha ...
```

In the above command, `test_pytha` is the name of the module you have defined.

- ❑ Add the following `map_bc` directive in an `initial` statement within the required module:

```
$ixc_ctrl("map_bc");
```

Handling of Behavioral Constructs during Behavioral Compilation

Certain behavioral constructs are supported in the DUT when behavioral compilation is enabled. This section describes the supported constructs and their restrictions.

Non-RTL and Multiple Events

Behavioral compilation supports `always` procedure with general non-RTL sensitivity list and multiple events as listed below:

■ Event control (@) statements

An event control statement can have any number of event expressions, edge-sensitive events, level-sensitive events, or both.

- ❑ For edge-event expressions, the operand of `posedge` or `negedge` should be a 1-bit scalar or a 1-bit constant bit select of a simple bit vector.
- ❑ For level-sensitive events, the following conditions must hold true:
 - The event expression should not contain any function calls.
 - The data type of the expression should be either a 1-bit scalar or a packed bit-vector type.

Example:

```
wire wclk;           // 1-bit scalar
reg rclk;            // 1-bit scalar
reg [3:0] clks;      // simple bit vector
always begin         // multiple event controls supported
    @(wclk)...        // 1-bit scalar supported
    @(clks[3:1]) ...  // simple bit vector type supported
    @(posedge rclk) ... // 1-bit scalar supported
    @(negedge clks[0] // 1-bit const bit-select of packed bit vector
                        //supported
    or wclk)...       // mixed edge- and level-sensitive events supported
end
```

■ Wait statements

The `wait` expression must not contain any function calls.

```
integer x;
function integer f1; ...
always begin
    ...
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
wait(f1(x)) ... // wait expression containing function call not supported
end
```

Note: The `always wait(c) ...` blocks are supported. However, these can cause race conditions leading to hardware-software mismatches. Avoid the use of such `always` procedures in DUT.

■ Multiple events

- ❑ An `always` procedure can have any number of event-control statements. Event expressions can be identical or distinct.
- ❑ An `always` procedure can have any number of `wait` statements

Example:

```
always begin
    wait(x) ...
    if (!done) @(negedge y);
    @(posedge clk) ...
end
```

■ An asynchronous `always` procedure that does not reference any objects in its sensitivity list is transformed as a behavioral process to honor the specified event expressions.

- ❑ If any of the event expressions are referenced in a pure-RTL asynchronous `always` procedure, the `always` procedure is not transformed, and follows combinational logic behavior during synthesis. This means that the event expressions would be inferred from actual references.
- ❑ If none of the event expressions are referenced in the `always` procedure, the `always` procedure follows the LRM semantics in accordance to the specified event expressions.

Example:

```
always @(a, b, e) // event expressions referenced in always
    d = a + b + c; // no transformation: combinational logic behavior with
                  // inferred @(a,b,c)
always @(x)      // event expression not referenced in always
    h = f & g;    // will be transformed: no change to sensitivity list @(x)
```

Timing Controls

The following kinds of timing control statements are supported:

■ Non-blocking assignments with intra-assignment delay control

Note: The RTL synthesis rule is followed where the delay is ignored.

- Blocking assignments with intra-assignment delay control specified with the `+iscDelay` option or `map_delays` directive.

By default, time delay (`#delay`) and intra-assignment delays for blocking assignments are not supported. To ensure that non-zero RTL and behavioral time delays in specific modules are honored during compilation, add the following `map_delays` directive in an `initial` statement within the module containing the delays:

```
$ixc_ctrl("map_delays");
```

Alternatively, you can specify the modules in which delays should be honored by using the `+iscDelay+<mod_name>` option of the `ixcom` command. For more information, refer to [Enabling Interface Compilation in IXCOM](#) on page 254.

- Blocking assignments with intra-assignment event control
Note: Non-blocking assignments with intra-assignment event control are not supported.
- The `#delay` control statement with the `+iscDelay` option or `map_delays` directive

Multiple Drivers

Assignments to the same register in multiple `always` procedures are supported when these are either all blocking or all non-blocking assignments.

Passing a register as an output or inout argument to a subroutine call is considered as a blocking assignment to the register.

Avoid mixing blocking and non-blocking assignments for the same register because it might result in compilation errors during the synthesis phase and/or lead to simulation and emulation mismatch.

```
task T1(output x);
    always begin
        ...
        A = d1; // (S1); blocking assignment to A
        B <= d2; // (S2); non-blocking assignment to B
        T1(C); // (S3); blocking assignment to C via output argument passing
    end
    always begin
        ...
        A = d3; // (S4); all blocking assignments (S1 & S4)
        B <= d4; // (S5); all non-blocking assignments (S2 & S5)
        C <= d5; // (S6); mixed assignments (S3 & S6), might result in HDL
                // compilation error
    end
end
```

Named Event

Named event variable declarations in the DUT are supported if:

- It is a named event variable or a one-dimensional event array declared in the module scope of a behavioral module (declaration in all others scopes are not supported)
- The only operations for the named event are:
 - Triggering a named event via the `->` operator
 - Triggering a named event via the `->>` operator without any delay or event control
 - Waiting for a named event via the `@` event control operator

Triggering a Named Event

- A named event trigger (`->`) is supported in DUT if the named event is locally declared in the behavioral module, and the named event is referenced directly and not via hierarchical reference.
- Non-blocking event trigger (`->>`) without delay or event control is supported in DUT as a blocking event trigger (`->`), and the same restrictions apply.
- Triggering a named event declared in DUT from testbench procedures is supported without restrictions. This applies for both the (`->>`) and (`->`) operators.

Note: It is important that the behavior of the named event should be “glitch-free”. This means that a named event should not be triggered more than once (via `->` or `->>`) in a single time step. Behavior is unspecified in DUT for multiple zero-delay triggers.

Example

```
module top; // TB
  dut d(clk);
  ...
always begin
  ...
  -> d.e; // named event trigger supported in TB for DUT named event
  ->> d.ev[0]; // nonblocking event trigger supported in TB for DUT named event
  ->> @(clk) d.ev[1]; // nonblocking event trigger with @event supported in TB
  ->> #1 d.ev[2]; // nonblocking event trigger with #delay supported in TB
end
endmodule
module dut(input clk); //DUT
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
event e; // event variable supported
event ev[4]; // one-dimensional event array supported
always begin
    ...
    ->e; // trigger of locally declared named event supported in DUT
    ->top.d.ev[0]; // named event trigger via hierarchical ref not
                  // supported in DUT
    ->> ev[1];      // nonblocking event trigger supported in DUT,
                  // treated as ->ev[1]
    ->> @clk ev[2]; // nonblocking event trigger with @event not supported
                  // in DUT
    ->> #1 ev[3]; // nonblocking event trigger with #delay not supported
                  // in DUT
end
endmodule
```

Waiting for a Named Event

- Waiting for a named event is supported in DUT if:
 - The named event is locally declared in the behavioral module.
 - The named event is referenced directly, not via a hierarchical reference.
- Waiting for a non-constant select of a named event array is not supported.
- Waiting for a named event declared in DUT from testbench procedures is supported.

```
module top; // TB
dut d(clk);
...
always @(top.d.e) begin // waiting on DUT named event supported in TB
    ...
end
endmodule

module dut(input clk); // DUT
event e; // event variable supported
event ev[3]; // one-dimensional event array supported
always begin
    ...
    @ev[0]; // waiting on locally declared event supported in DUT
    @top.d.e; // waiting on event via hierarchical ref not supported in DUT
end
endmodule
```

Control Flow

Following is the support provided for dynamic loops, fork-join statements, and disable statements:

- Loops with dynamic loop bounds are supported.

```
always begin
    ...
    while (a > b) ... // dynamic loop bound
end
```

- Verilog fork-join statements are supported when there is no delay or disable inside the fork-join. Only the control option `join` is supported. Parallel statement containing disable statements to a scope external to that parallel statement is not supported. Parallel statement containing delay control is not supported. All variables declared inside a fork-join block would be treated as if these are declared with static lifetime.

```
begin:outer
    fork
        begin:inner
            if (...) disable outer; // disable scope outside parallel stmt not
                                    // supported
            if (...) disable inner; // disable scope internal to parallel stmt
                                    // supported
            ...
        end
    #2 ...; // delay control not supported
    join // join supported (join_none or join_any not supported)
end
```

- Only disable statements of a parent named scope are supported. Further restrictions for fork-join parallel statements are listed in the above bullet point.

```
module dut;
    always @(posedge clk) begin
        ...
        begin:x
            ...
            if (...) disable x; // disable parent scope supported
            ...
        end
    end
    always @(negedge clk) begin
        ...
        if (...) disable dut.x; // disable non-parent scope not supported
    end
end
```



```
        ...  
    end  
endmodule
```

Subroutine Calls

Following points highlight the way subroutine calls are handled and the restrictions associated with subroutine calls:

- Subroutine calls from an `always` procedure in a behavioral module shall be inlined into the `always` procedure. The inlined task or function and its locally declared variables would be treated as these are declared with automatic lifetime.
- The data type of the formals, the arguments, and the locally declared variables must be packed and have static width.
- Recursive subroutine (task/function) calls are not supported. A recursive subroutine is one that can call itself directly or transitively via calling other subroutines.

```
always begin  
    ...  
    t1(...);  
end  
  
task t1(...); // task reachable by always procedure  
begin  
    ...  
    if (...) t1(...); // recursive task call not supported  
end  
endtask
```

Other Restrictions

Following are other miscellaneous restrictions on a behavioral module:

- The enumerated data type is not supported if declared in a block name space in an `always` procedure, or in a block name space in a subroutine that is reachable from an `always` procedure through subroutine calling.

```
always begin  
    ...  
    t1(...);  
    begin:x // block name space in always procedure  
        typedef enum {r, g, b} color; // enum declaration not supported here
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
        color c;
        ...
    end
end

task t1(...); // task reachable by always procedure (reachable task)
begin
    ...
    begin:y    // block name space in reachable task
        typedef enum {r, g, y} color; // enum declaration not supported here
        color c;
        ...
    end
end
endtask
```

- **Named blocks and variables that are declared in an `always` procedure or in a subroutine that is reachable from an `always` procedure in a behavioral module, are not supported if there are any hierarchical references to them.**

```
module dut;
    always begin
        ...
        t1(...);
        begin:x    // named block in always procedure
            reg a;
            ...
        end
    end
end

task t1(...); // task reachable by always procedure (reachable task)
begin
    ...
    begin:y    // named block in reachable task
        reg b;
        ...
    end
end
endtask
always begin
    ...
    if (dut.x.a) disable dut.x; // hierarchical ref to x or x.a not supported
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
        if (dut.t1.y.b) disable dut.t1.y; // hierarchical ref to y or y.b not
                                         // supported
        ...
    end
endmodule
```

- **Labeled statements that are declared in an `always` procedure or in a subroutine that is reachable from an `always` procedure in a behavioral module, are not supported if there are any hierarchical references to them.**

```
module dut;
always begin
    ...
    x:begin // labeled statement in always procedure
        t1(...);
        ...
    end
end
task t1(...); // task reachable by always procedure (reachable task)
begin
    y:begin // labeled statement in reachable task
        ...
    end
end
endtask
always begin
    ...
    if (...) disable dut.x; // hierarchical ref to x not supported
    if (...) disable dut.t1.y; // hierarchical ref to y not supported
    ...
end
```

- **Synopsys `translate_on` / `translate_off` statements in a behavioral module:**

- **`translate_off` statements are not supported in an `always` procedure and subroutines that are reachable from an `always` procedure through subroutine calling when:**

- The `translate_off` statement is time-consuming

Or,

- The `translate_off` statement affects the control flow of `translate_on` region, for example, a `translate_off` jump statement such as a `continue` statement of a `translate_on` loop statement

- ❑ Cadence recommends that the `translate_on/off` statement should be used for the entire `always` procedure.

```
always @(posedge clk) begin
    while (a > b) begin
        //synopsys translate_off
        if (...) @(posedge clk) ... // time-consuming translate-off not
                                   // supported
        if (...) continue;          // jump stmt affecting control flow not
                                   // supported
        //synopsys translate_on
        ...
    end
end
```

Using Compiler Directives to Guide Behavioral Compilation

Use compiler directives to guide the behavioral compilation when the default behavior is not desirable or optimal for a design.

Specify the compiler directives by using the following syntax:

```
$ixc_ctrl("directive_name"[, <directive_object>]*);
```

The compiler directives are used for:

- ❑ Specifying Properties of Wires and Registers
- ❑ Specifying Loop Unrolling
- ❑ Specifying Loop Stepping

Specifying Properties of Wires and Registers

The following directives are used for specifying properties of registers and wires in order to guide behavioral compilation of the module specified with the `+bc` option or `map_bc` directive:

- `$ixc_ctrl("beh_var"[, <var_name>]+);`

Use the "beh_var" directive in an `initial` statement of a behavioral module to specify the variables that should be considered as behaviorally driven variables. For these variables, the current values should always be used.

- `$ixc_ctrl("rtl_var"[, <var_name>]+);`

Use the `rtl_var` directive in an `initial` statement of a behavioral module to specify variables that should be considered as RTL-driven variables. For these variables, the old values are used when referenced in a transformed behavioral process.

■ `$ixc_ctrl("rtl_clk" [, <var_name>] +);`

Use the `rtl_clk` directive in an `initial` statement of a behavioral module to specify the variables that should be considered as RTL clock. This would enable certain performance optimization during behavioral compilation.

■ `$ixc_ctrl("async_var" [, <var_name>] +);`

Use the `async_var` directive in an `initial` statement of a behavioral module to specify the variables that are asynchronous event variables. These variables can be defined as:

- Variables that are used in an event expression of a behavioral process. For example, `@(event_expr)`
- Variables whose value can be potentially changed by an asynchronous testbench event at run time. Asynchronous testbench events include the following:
 - Memory write operations through commands or `margin` interface
 - CLI-based deposit and force operations of signals that are NOT instrumented at compile time through explicit `$export_deposit` and `$export_frcrel` export directives

Asynchronous events require special instrumentation that might reduce performance. Memory write or CLI-based deposit and force operations are rarely used to trigger behavioral events in BFMs. Therefore, during behavioral compilation, all event variables are regarded as NOT asynchronous.

In some special cases where asynchronous events are used, but the BFMs are fairly small compared to the RTL portion of the design, you can choose to compile the design with the `+bcAsyncEvent` option of the `ixcom` command to mark all behavioral events as asynchronous.

Important

In each of the above directives, specify at least one or more variables.

Specifying Loop Unrolling

The `"bc_unroll_loop"` directive is used for specifying that all loops or a subset of the loops in a behavioral module should be unrolled prior to any behavioral transformation. With

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

this directive, loops that have static constant loop bound will be forced to be unrolled, even if the loop body includes time-consuming statements, such as event control (@). Named scopes that are declared inside unrolled loops would be renamed, when necessary. This is to avoid name conflicts between duplicate copies of the named block, one for each unrolled loop iteration.

To specify this directive for a particular behavioral module, include the following line in an `initial` statement in the module:

```
$ixc_ctrl("bc_unroll_loop");
```

For example:

```
module dut;
...
initial $ixc_ctrl("bc_unroll_loop"); // unroll all loops with static constant
                                     // loop bounds in this behavioral module
always begin
...
    for (i = 0; i < 10; i = i+1) begin // loop will be unrolled regardless of
                                     // time-consuming @event
        begin:xx // named block will be renamed, so duplicate copies of the named
                // block will have unique name
            @(count) ...;
        end
    end
    for (j = 0; j < 10; j = j + 1) begin // loop will be unrolled
        ...
    end
end
endmodule
```

To specify this directive only for a particular loop in the module, add the following line right before the loop:

```
//ixc bc_unroll_loop
```

For example:

```
module dut;
...
always begin
...
    //ixc bc_unroll_loop
    // preceded by bc_unroll_loop directive, force loop unrolling
    for (i = 0; i < 10; i = i+1) begin
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
// named block will be renamed, so duplicate copies of the named block
// will have unique names
begin:xx
    @(count) ...;
end
end
// not preceded by bc_unroll_loop directive, loop unrolling not forced
for (j = 0; j < 10; j = j + 1) begin
    ...
end
end
endmodule
```

Specifying Loop Stepping

The "bc_step_loop" directive specifies that the loop following the directive should be executed one iteration per behavioral evaluation. This directive is used to prevent loop unrolling that would cause resource explosion.

To specify this directive for a particular loop in the module, add the following line right before the loop:

```
//ixc bc_step_loop
```

For example:

```
module dut;
    ...
    always begin
        ...
        //ixc bc_step_loop
        // preceded by bc_step_loop directive, force loop stepping
        for (i = 0; i < 100; i = i+1) begin
            // each behavioral evaluation would execute only one iteration
            // of the loop
            // resource is shared between different iterations
            ...
        end
        // not preceded by bc_step_loop directive, loop unrolling is allowed
        for (j = 0; j < 10; j = j + 1) begin
            ...
        end
    end
end
```

```
endmodule
```

Process-Level Locking for Dependent and Conflicting Behavioral Processes

All behavioral processes are executed concurrently, unless there are resource conflicts or dependencies between these processes. Two behavioral processes performing read and blocking write operations on the same register are regarded as *dependent* processes. Behavioral processes writing or reading the same memory are also regarded as processes having *resource conflicts*.

For dependent behavioral processes, process-level locking or arbitration can be utilized to ensure that the dependent or conflicting processes are executed in a sequential manner, thus displaying serialized atomic execution behavior.

Enabling Behavioral Concurrent Compilation (+bcc Option)

If it is certain that reading and writing of same registers from different behavioral processes would not happen at run time, turning on the behavioral concurrent mode can boost performance. Under the concurrent mode, behavioral processes are executed concurrently even with read-write conflicts between them.

To enable behavioral concurrent compilation for a specific module in the DUT, use one of the following methods:

- Use the `+bcc+<mod_name>` option of the `ixcom` command instead of the `+bc+<mod_name>` option. The `<mod_name>` refers to the name of the DUT module that you have defined. For example:

```
ixcom tpytha.v +dut+test_pytha +sv +bcc+test_pytha ...
```

- Add the following `map_bcc` directive in an `initial` statement within the module:

```
$ixc_ctrl("map_bcc");
```

An example of dependent behavioral processes due to the read-write reference of the variable `x` is shown below. Arbitration would be utilized to ensure serialized behavior by default. If it is known that reading `x` and writing `x` would not happen at the same time, say `posedge clk1` and `posedge clk2` would never coincide, or `en1` and `en2` would never be both true, then using the behavioral concurrent mode can boost performance without altering functionality.

```
always @(posedge clk1)
begin
    if (en1) begin
        ...
    end
end
```


UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
        while (...) begin
            x = x + 1;
            ...
        end
    end
end
always @(posedge clk2)
begin
    if (en2) begin
        while (...) begin
            y = y + x;
            ...
        end
    end
end
end
```

Compiling Behavioral Interface Constructs

IXCOM maps the behavioral interface constructs in Verilog and SystemVerilog into the emulator and automatically generates connections for interface constructs crossing the testbench and hardware partitions. These interface constructs include Verilog and SystemVerilog constructs for:

- Connecting objects between different module instances
- Connecting objects with external (foreign language) components

The standard RTL interface constructs, such as module ports and Cross-Module Reference (XMR) wires are supported directly.

Interface compilation focuses on the behavioral interface constructs like:

- Verilog XMR function and task calls
- Verilog system-defined or user-defined task (`$task`) calls
- C and SystemC calls to exported DPI tasks
- SystemVerilog calls to C and SystemC imported tasks and functions
- `#delay` scheduling and `$time` support in emulator (implicit interface to scheduler)

The above list includes both explicit and implicit interface constructs. All task and function calls listed above, except the `#delay` and `$time` are explicit behavioral calls. For example, `#delay` is an implicit interface call to the simulation time scheduler that needs to consider delay events from all the module instances.

Enabling Interface Compilation in IXCOM

Interface compilation is enabled automatically, by default, if the `-ua` option is specified with the `ixcom` command or if the design includes behavioral modules, driven by the explicit interface constructs and compiler directives specified with the `ixcom` command.

However, to enable support for `#delay` scheduling and `$time` in the emulator, use one of the following options:

- Specify the `+iscDelay` option with the `ixcom` command
- Add the following `map_delays` directive in an `initial` statement in the module containing the delays:

```
$ixc_ctrl("map_delays");
```

The implicit interface call to the global scheduler must be explicitly controlled by using the `+iscDelay` option because the fine-grain delay control (`#1`) can have significant impact on the emulator performance. In most cases, the fine-grain delay control is used to avoid race conditions between the testbench and DUT in the sequential software simulation engine. The parallel emulation engine, on the other hand, is designed to be race free and the fine-grain delay control is not necessary and should be ignored.



Caution

The use of `#delay` in the DUT is not compatible with the `axis_clkgen` primitive.

With interface compilation, all the DUT `#delay` events are scheduled within the hardware in `tbrun` and `autorun` modes. Synchronization with the host simulator only happens when a host simulator delay event expires or a host interface call event occurs in the DUT.

Rules for `#delay` Support during Interface Compilation

The `#delay` statements are mapped to hardware according to the following rules:

- Only the blocking `#delay` statements in the `+iscDelay` modules are mapped; all `#delay` values in the default RTL modules are ignored.
- Only blocking intra-assignment delays `#delay` control are considered; all non-blocking intra-assignment delays are ignored, even in the modules specified with the `+iscDelay` option.
- Blocking delays in `initial` statements that meet the forever-loop clock generation style are supported in the modules specified with the `+iscDelay` option. For more details, refer to [Modeling Clocks in SA Mode](#) on page 1293.
- Both constant and variables delay values are supported.
- The `#0` delays, from `#0` statements and `#d` statements with `d=0` at run time, are ignored.
- When the `+iscDelay` option is used, the clock generation must be a part of the DUT to avoid errors. No `axis_clkgen` primitive can be used in the DUT.
- If behavioral transformation is needed for a module specified with the `+iscDelay` option, because `#delay` is behavioral in nature or because other constructs in the module require behavioral transformation, this module needs to be explicitly specified as behavioral by using the `+bc+<mod_name>` option or the `("map_bc")` directive.

Considerations for Using Interface Compiler for \$time and \$stime Functions

The `$time` and `$stime` functions are supported through interface compilation in the hardware with the following considerations:

- The `$time` and `$stime` functions in the `always` procedures or tasks/functions must be in the form of procedural assignment statements when the `+iscDelay` option is used.

```
var = $time;
```

The `$time` value is a 64-bit object and it consumes a lot of hardware resources to deliver them from the scheduler. Typical applications of `$time` and `$stime` functions in the hardware partition include adding timestamp in transaction data recording and time logging and does not need a 64-bit time stamp. To reduce resource requirements and to achieve better performance, the use of the assignments guide the compiler to reduce the bit needed to present the time in the application.

- The `$time` function used in `$display` and `$fdisplay` statements are supported as-is.
- All transformed `$time` value is scaled to the minimum time unit of the design unless it is used in `$display` or `$fdisplay` statements.

Compiler Directives for Interface Compilation

The Interface compiler uses the following compiler directives:

- `tb_import` compiler directive

A `tb_import` compiler directive indicates a task that is being executed in the simulator, and is to be called by the DUT running on the emulator. Following are the different ways in which the `tb_import` compiler directive can be used, where `<tf_name>` is the target task or function name.

```
❑ $ixc_ctrl("tb_import", "<tf_name>");
```

The above syntax declares an import task or function to run in the blocking mode and uses primary I/Os to transfer input and output.

```
❑ $ixc_ctrl("tb_import", "<tf_name>", "tb_mem");
```

The above syntax declares an import task or function to run in the blocking mode and uses memory to transfer input and output.

- `tb_export` compiler directive:

A `tb_export` compile directive indicates a task that is being executed on the emulator, and is to be called by the testbench running on the simulator. Following are the different

ways in which the `tb_export` compiler directive can be used, where `<tf_name>` is the target task name.

❑ `$ixc_ctrl("tb_export", "<tf_name>");`

The above syntax declares an export task to run in the blocking mode and uses primary I/Os to transfer input and output.

❑ `$ixc_ctrl("tb_export", "<tf_name>", "tb_mem");`

The above syntax declares an export task to run in the blocking mode and uses memory to transfer input and output.

❑ `$ixc_ctrl("tb_export", "<tf_name>", "threads=<num_threads>");`

The above syntax declares an export task to run in concurrent mode using pre-allocated static threads, and uses primary I/Os to transfer input and output. In this case, simultaneous multiple (less than or equal to `num_threads`) calls to the same target are allowed. This static concurrent mode usually consumes more hardware resources but runs faster than dynamic concurrent mode.

❑ `$ixc_ctrl("tb_export", "threads=<num_threads>", "tb_mem");`

The above syntax declares an export task to run in concurrent mode using pre-allocated static threads, and uses memory to transfer input and output. In this case, simultaneous, multiple (less than or equal to `num_threads`) calls to the same target are allowed. This static concurrent mode usually consumes more hardware resources but runs faster than dynamic concurrent mode.

■ **tb_task compiler directive**

A `tb_task` compiler directive indicates a task that is being executed in the simulator, and is to be called by the DUT running on the emulator. It is the same as the `tb_import` compiler directive, except that a `tb_task` directive is used for a local task or function in the DUT module while a `tb_import` directive is used for a task or function in another module or a DPI or PLI task or function.

❑ `$ixc_ctrl("tb_task", "<tf_name>");`

The above syntax declares a local task or function to run in the simulator in blocking mode and uses primary I/Os to transfer input and output.

❑ `$ixc_ctrl("tb_task", "<tf_name>", "tb_mem");`

The above syntax declares a local task or function to run in the simulator in blocking mode and uses memory to transfer input and output.

Interface Compilation Limitations

- The export functions are not supported. For export functions to be supported, convert the function to equivalent export task.
- Import of PLI and system functions is not supported. For such functions to be imported, add a local task wrapper to call that function, declare the task using the `tb_import` compiler directive, and then call this new task from the DUT instead of the function that was to be called initially.
- Cyclic Verilog cross module (XMR) task and function call chains within the hardware partition are not supported.

Using Cross Module Task and Function Calls

IXCOM supports Verilog cross module (XMR) task and function calls within the hardware partition and across the DUT and testbench partitions through interface compilation.

Consider, for example, a basic behavioral FIFO module:

```
module mFIFO;
reg [31:0] FIFO [31:0];
reg [4:0]  FIFOPutPtr = 0;
reg [4:0]  FIFOPutNxt;
reg [4:0]  FIFOGetPtr = 0;
  initial begin
    $ixc_ctrl("tb_export", putD);
    $ixc_ctrl("tb_export", getD);
  end
  task putD (input v1);
    reg [31:0] v1;
    begin
      FIFOPutNxt = FIFOPutPtr+1;
      wait(FIFOPutNxt != FIFOGetPtr);
      FIFO[FIFOPutPtr] = v1;
      FIFOPutPtr = FIFOPutNxt;
    end
  endtask
  task getD (output v1);
    reg [31:0] v1;
    begin
      wait(FIFOPutPtr != FIFOGetPtr);
      v1 = FIFO[FIFOGetPtr];
    end
  endtask
endmodule
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
        FIFOGetPtr = FIFOGetPtr + 1;
    end
endtask
initial begin
    $ixc_ctrl("tb_export", "putD");
    $ixc_ctrl("tb_export", "getD");
end
endmodule
```

This module provides two interface tasks for putting and getting data from the FIFO when the FIFO becomes empty or full. The `tb_export` directive is used to enable the two interface tasks `putD` and `getD` to be called from a testbench module. Both interface tasks are time consuming in nature. With this basic FIFO module, a DUT that has one input FIFO and one output FIFO can be built, using a basic transfer process for connecting the two FIFOs:

```
module DUT;
    reg clk1 = 0;
    always #2 clk1 = ~clk1;
    reg [31:0] xdata;
    mFIFO fifoI();
    mFIFO fifoO();
    always@(posedge clk1) begin
        fifoI.getD(xdata);
        fifoO.putD(xdata);
    end
endmodule
```

In the above DUT module, the data transfer process connects the two FIFOs by XMR calls to the FIFO interface tasks. These XMR task calls are within the hardware partition.

In parallel to the DUT, there can be a basic testbench module that applies tests to the input FIFO and gets the result data from the output FIFO.

```
module TB;
    integer i, j;
    reg [31:0] IV, OV;
    initial begin
        #1;
        for(i = 1; i <= 100; i = i+1) begin
            IV = $random;
            $display($time, " Put  ", IV);
            DUT.fifoI.putD(IV);
        end
    end
end
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
initial begin
    #1;
    for(j = 1; j <= 100; j = j+1) begin
        DUT.fifoO.getD(OV);
        #0;
        $display($time, " ----->Get  ", OV);
    end
    $finish;
end
endmodule
```

This basic testbench module connects to the DUT module by XMR interface task calls to the input and output FIFO. These XMR task calls are crossing the TB (running in host simulator) and DUT (running in hardware emulator) partitions.

Interface compilation enables the DUT to be compiled in the hardware by using the following `ixcom` command. For this `ixcom` command, it is assumed that all three modules are defined in a single Verilog file `demo.v`.

```
ixcom demo.v -ua +sv +dut+DUT +bc+DUT+mFIFO +iscDelay+DUT
```

Here, the `+iscDelay` option is used to map the `#delay` statements in the DUT module on to the emulator.

Interface Task Mapping Control

The tasks defined in hardware partition are not mapped to the hardware emulator unless such DUT tasks are explicitly marked as `tb_import`. All software tasks under the DUT are automatically treated as `tb_import`, even if the pragma is not provided. All the tasks defined in the testbench partition are run in the host software simulator. It is a good coding style to define the tasks in the right partition. However, this might not be very easy when porting from a simulation environment to an acceleration environment. In many cases, such a coding style requires dividing an interface module into a testbench module and DUT module.

To avoid changing the design hierarchy, you can use basic directives to assign tasks defined in a DUT module to be executed in the testbench domain.

```
$ixc_ctrl("tb_import", "<task>");
```

The `tb_import` directive assigns the `<task>` in a DUT module to the testbench domain in order to execute it in the host simulator.

Once the `<task>` is marked as a `tb_import`, an XMR call from a testbench module to such a task is within the testbench partition and will not trigger Interface Compilation.

If the `<task>` calls another task defined in a DUT module, the task must remain in the hardware partition. This local DUT task becomes an implicit cross partition interface task and requires Interface Compiler mapping, even though an explicit XMR task call has not been identified. Interface Compiler requires that such a DUT interface task should be marked by using the `tb_export` directive.

```
$ixc_ctrl("tb_export", "<task>");
```

In some cases, `tb_import` tasks (for example, DPI task calls) can be automatically detected, and the use of `tb_export` and `tb_import` directives could be redundant. Whenever marking a DUT task or function as `tb_import`, ensure that the side-effects are considered and taken care of. Random marking of `tb_import` can have a serious impact on performance.

Only DPI tasks/functions can be automatically marked as “`tb_import`” or “`tb_export`”. Other tasks/functions need to be marked as “`tb_import`” or “`tb_task`” by user. If a software task/function is not treated as “`tb_import`” or “`tb_task`”, the DUT module cannot be synthesized. The difference between “`tb_import`” and “`tb_task`” is that “`tb_task`” usually refers to a local software task/function in the current module, while “`tb_import`” refers to a non-local software task or function in other modules or C code.

Using DPI and \$task Calls

If interface compilation is enabled, XMR task calls, DPI calls, and `$tf_task` calls are unified into a common framework, using the same protocol. This unified solution extends the IXCOM coverage scope of external language interfaces. If interface compilation is not enabled, the support for SystemVerilog DPI and `$tf_task` calls is limited support.

Use the interface compilation directives to explicitly mark these DPI and `$tf_task` calls for generating the new interface.

For DPI export tasks, use the `tb_export` directive to map the task to hardware or use `tb_import` to keep the task running in host simulator. For example:

```
export "DPI-C" task putD;
export "DPI-C" task getB;
    initial begin
        $ixc_ctrl("tb_export", "getD");
        $ixc_ctrl("tb_import", "putB");
    end
```

If you do not mark a DPI-C exported task as `tb_export` task, the Interface compiler will treat it as a software task and map the task to execute in the software simulator, unless the task body contains an event wait statement to a variable, which is a named event, for example, `@(posedge clk)`.

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

For DPI import task, use the `tb_import` directive for interface generation.

For example:

```
import "DPI-C" context task dpi_report_result (input int p);
  initial begin
    $ixc_ctrl("tb_import", "dpi_report_result");
  end
```

When `$tf_task` is a PLI task, use `tb_import` directive and an I/O direction string for the `$tf_task`. Also, the `+no_uaictd` option needs to be specified with the `ixcom` command to enable handling of PLI tasks.

```
initial begin
  $ixc_ctrl("tb_import", "$myPliTask", "iobbb");
end
```

The format of the second string specifies the I/O direction as follows:

- ❑ The character “i” is used for INPUT
- ❑ The character “o” is used for OUTPUT
- ❑ The character “b” is used for INOUT

For example, “iobbb” means that the first argument to the `$tf_task` is INPUT only, the second one is OUTPUT only, and all the others are INOUT type.

The I/O directive is optional. If it is not specified, all arguments are treated as INOUT except for constant or expression arguments.

Note: The DUT module cannot be synthesized if a PLI or user-defined system task is called from the hardware partition, but not marked as a `tb_import` task.

Fine-Tuning Behavioral Performance

This section describes the levels of behavioral evaluation reported in the working directory and the steps to identify the source of behavioral evaluations.

Reporting Behavioral Evaluation

The following levels of behavioral evaluation are reported in the working directory for any hardware run:

1. The total number of behavioral evaluations is reported in `irun.log` and `tmp/xc_run.log` as `bevals` in the `HW execs` statistics. For example:

```
--- HW execs 1000 evals 300 bevals 100 tbsyncs
```
2. Behavioral evaluations for individual module instance is reported in `tmp/xc_run.log` in decreasing order of the number of evaluations. For example:

```
=====
Behavioral evals statistics:
-----
213 dut.trans2
153 dut.trans3
21 dut.trans1
=====
```

Multiple module instances might be active during a single behavioral evaluation. So, the sum of these behavioral evaluations can be greater than the total number of behavioral evaluations reported in the `HW execs` statistics.

3. When the design is compiled with the `+bcDebug` option of the `ixcom` command, detailed breakdown of behavioral evaluations for individual behavioral events are provided in the statistics in `tmp/xc_run.log`. For example:

```
=====
Behavioral evals statistics:
-----
213 dut.trans2
106 _zyL86_pbcCapOut2
213 _zyL74_pbcCapOut1
153 dut.trans3
153 _zyL99_pbcCapOut1
21 dut.trans1
10 _zyL86_pbcCapOut2
21 _zyL74_pbcCapOut1
```

=====

In the sample report shown above, the second level signals `_zyL86_pbcCapOut2`, `_zyL74_pbcCapOut1`, and so on, correspond to behavioral events on line 86 and 74, respectively.

Identifying the Source of Behavioral Evaluations

The following are the steps to identify the source of behavioral evaluations:

1. From the "Behavioral evals statistics" in `tmp/xc_run.log`, identify the module instance that incurs large number of behavioral evaluations.

2. Identify the module corresponding to the module instance with large evals.

3. Look up the module in `xc_work/bc.log` to find a list of line numbers of behavioral processes. For example:

```
Beh Procs found at lines 66, 90
```

Make sure the listed `always` procedure is behavioral and cannot be rewritten as RTL to improve performance.

4. When necessary, compile with the `+bcDebug` option of the `ixcom` command to get detailed statistics for individual behavioral events for each module instance. This helps in isolating the source of behavioral evaluations.

System Task Mapping in IXCOM flow

In the IXCOM mode, the behavior of system tasks can be emulated by issuing testbench callback to map the system tasks. You can use one of the following ways to map general system tasks or functions in IXCOM mode:

- Using \$ixc_ctrl Pragma within the Module
- Using ixcom Command-Line Option to Map System Task Calls

Using \$ixc_ctrl Pragma within the Module

To map a system task in a DUT module, write an `ixc_ctrl` pragma in an `initial` statement in that DUT module, as follows:

```
$ixc_ctrl("tb_import", "<system_task_function_name>"[, "tb_mem"]);
```

Here, "tb_mem" is an optional argument to denote that the input/output arguments are passed through the memory buffer.

Using ixcom Command-Line Option to Map System Task Calls

Use the following option of the `ixcom` command to map specific system task calls in specific modules:

```
+tb_import_systf+[<mod_name>.<sys_tf_name>[:mem] [+...]
```

The above syntax specifies the system task call `<sys_tf_name>` in DUT module `<mod_name>` to get mapped in blocking mode.

Note: The `<sys_tf_name>` does not have the \$ symbol as a prefix to its name. Without `<mod_name>`, all the system task or function calls of `<sys_tf_name>` in all DUT modules would be mapped. The optional argument of `:mem` denotes that the mapping uses the memory buffer to transfer input and output. Otherwise, primary I/O resources are used, by default.

For example, `+tb_import_systf+dut.display:mem` denotes mapping of `$display` function in DUT module `dut` using memory buffer to transfer the input arguments. This is equivalent to adding the following `initial` statement to module `dut`:

```
initial $ixc_ctrl("tb_import", "$display", "tb_mem");
```

No change needs to be made to the RTL source code, if the command-line option is being used.

Mapping of \$display-Family Tasks

The `$display` task is an important system task call, which is widely used in RTL code. The IXCOM mode provides the following mapping methods to map the `$display`-family system tasks, including:

- `$display`
- `$displayb`
- `$displayh`
- `$displayo`
- `$fdisplay`
- `$fdisplayb`
- `$fdisplayh`
- `$fdisplayo`

The following mapping methods are sorted by descending precedence and ascending performance or speed.

1. `$ixc_ctrl("tb_import", "<sys_task_name>"[, "tb_mem"])`

You can put a `tb_import` pragma in an `initial` statement in a DUT module, and map the specific `$display` task calls as `tb_import` task calls. The optional argument specifies using the memory buffer for argument transfer.

2. `+iscdisp+[<dut_mod_name>[+...]]`

You can use this command-line option to map all `$display`-family task calls in the specified DUT modules, in blocking mode and using primary I/O resources for argument transfer. Without the specification of `<dut_mod_name>`, all `$display`-family task calls in all DUT modules would get transformed.

3. `+bufdisp+<dut_mod_name>[+...]`

You can use this command-line option to map all `$display`-family task calls in a special fast non-blocking mode using memory buffers. The output cannot be ordered using this method.

The mapping method of higher precedence overrides the one of lower precedence. For example, `$display` is mapped in blocking mode if there is an `$ixc_ctrl("tb_import", "$display")` pragma in the module `dut`, even if the `+bufdisp+dut` option of the `ixcom` command is used.

Using System Tasks and Procedures at Run time

Many IEEE-standard system tasks can be added to the Verilog code and, in most cases entered at the Command-Line Interface (CLI). In addition, many equivalent VHDL procedures can be added to the VHDL code.

- Other system tasks can be entered using the `$xc_cmd()` Verilog system task, which lets you execute a CLI command from your source code. Refer to the `$xc_cmd()`; section in the *Commands and System Tasks for SA Mode* chapter of the *UXE Command Reference Manual*.
- If your design contains system tasks, you can get a syntax error when compiling with another simulator. To avoid this problem, surround such code with the AXIS compiler directive. The AXIS compiler directive is automatically recognized by the IXCOM compiler.

For example:

```
initial
begin
`ifdef AXIS
    $axis_readmemh("init.dat",top.design_mod.submod1.axis_mem_instance);
`else
    $readmemh("init.dat",top.design_mod.submod1.usermem);
`endif
end
```

Supported Verilog System Tasks and VHDL Procedures

To use the following Verilog system tasks and VHDL procedures you can:

- Add the system task directly to the Verilog code
- Add the procedure directly to the VHDL code
- Enter the system task at the CLI (Command-Line Interface)

To use the VHDL procedures, you must add the following library clause to your code:

```
library axis;
use axis.axis.all;
```

Note: Instance paths to signals or Verilog primitives are specified using period delimiters, that is:

```
instance.instance.signal_or_primitive
```

Executing Non-Synthesizable Code in the Emulator

The majority of typical testbench code involves the monitoring and manipulation of signals in the Device Under Test (DUT) to direct simulation or to detect error conditions occurring during simulation. This code is usually non-synthesizable and typically represents the largest performance bottleneck in simulation. There are several options to deal with this type of monitor code:

- Exclude the code completely from your simulation. This option provides the fastest performance, but sacrifices some debugging capabilities.
- Use the Testbench Callback technology to automatically execute the non-synthesizable code in the emulator hardware. Refer to the [Compiling DUTs with Non-Synthesizable Code by Using Testbench Callback Technology](#) on page 273 for more detail.
- Partition the monitor code into a synthesizable error-condition check and a non-synthesizable code segment that executes when the error is detected. This technique is described later in this section.
- Set the monitor code to be active in software simulation mode only. The design is swapped to software simulation time at specific time intervals, or after specific error conditions are detected. This technique is described later in this section.

Partitioning Monitor Code

The technique for partitioning monitor code into synthesizable and non-synthesizable segments is based on the premise that much of the monitor code is needed only after one or more error conditions have occurred during simulation.

If an error does occur, the ability to debug it becomes more important than simulating the design very quickly. This technique involves several steps, as illustrated in this example:

```
module testbench ();  
..  
monitor mon1 (..);  
..  
endmodule  
  
module monitor (..);  
..  
reg [0:31] local_mem [0:3];  
always @(posedge sys_clk)  
begin  
    if (packet_status==3'b0 && !packet_received) // error check; synthesizable
```


UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
begin
  for (i=0;i<3;i=i+1)
    local_mem[i] = testbench.chip.sram.mem[i];  // non-synthesizable
  if (local_mem[0] == 1'b0)
    $display("Invalid header; Packet Number: %d", packet_num);
  end
end
endmodule
```

In this example, the first `if` statement checks for the presence of an error condition, and if encountered, executes the remaining code in the `always` procedure. Unmodified, this code is unable to be accelerated, and the entire module will run in the software simulator only. To improve the run-time performance of this module, follow these steps:

1. Remove the non-synthesizable code segment that follows the `if` statement and replace it with a special `trigger` signal indicating that the error occurred. The remainder of the `always` procedure and the `if` statement itself must be written in a *synthesizable* coding style.

In the previous example, you would modify the code, as follows:

```
module monitor (...);
..
reg monitor_dummy_reg;          // declare this trigger register
always @(posedge sys_clk)
begin
  if (packet_status==3'b0 && !packet_received)
    // error check must be synthesizable
    monitor_dummy_reg = 1'b1;
    // error encountered; trigger signal set to 1
  else
    monitor_dummy_reg = 1'b0;    // not an error condition: no trigger
  end
endmodule
```

2. Create a *dummy* module and instantiate it in the testbench. In this module, check for the triggering of the dummy signal and add the removed non-synthesizable code, as follows:

```
module testbench ();
..
dummy_mon d_mon ();             // instantiate this dummy module here
monitor mon1 (...);
chip chip1 (...);
..
endmodule
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
module dummy_mon ();
reg [0:31] local_mem [0:3];
always @(posedge testbench.monitor.monitor_dummy_reg)
    // check trigger signal
begin
    // execute monitor code
    for (i=0;i<3;i=i+1)
        local_mem[i] = testbench.chip.sram.mem[i];
    if (local_mem[0] == 1'b0)
        $display("Invalid header; Packet Number: %d", packet_num);
end
endmodule
```

3. When compiling a mixed-HDL design, add the modified module `monitor` to the DUT. Similarly, when compiling a Verilog-only design, add the modified module `monitor` to the DUT and exclude the new module `dummy_mon` from the DUT:

For a mixed-HDL Design:

```
ixcom -top top_level_design_unit other_options +dut+chip+monitor
```

For a Verilog-only Design:

```
ixcom Verilog_sources other_options +dut+chip+monitor +dutexcl+dummy_mon
```

Activating Monitor Code Only in Software Simulation

There are two techniques for making monitor code active only in the software simulation mode:

- Marking Monitor Code to Simulate Only in Software Simulation
- Swapping to Software Simulation Mode at the Appropriate Time

Marking Monitor Code to Simulate Only in Software Simulation

IXCOM simulator lets you *mark* either a whole module in the DUT, or just a portion of it, to be active in software simulation mode only; in software acceleration mode the code is totally inactive and has no affect. After you swap to software simulation mode, the code becomes active again from the next simulation time-tick. This feature is especially useful for minimizing the impact of monitor code in testbenches. There are two ways to mark modules in this fashion:

- To *mark* an entire module to simulate while in software simulation mode only, specify the `+dutignore+module_name` compiler option. All instances of the module *and its sub-hierarchy* will not simulate at all while running in simulation acceleration mode. This

functionality differs from that of the `+dutexcl+module_name` compiler option, which causes the module sub-hierarchy to simulate in the software simulator at all times, regardless of the run-time mode.

- To *mark* just a portion of a module to be simulated while in software simulation mode, use the `$axis_rccoff` system task and the `//ixc rtl_off` pragma. The `$axis_rccoff` task takes no arguments, returning a boolean `true` if the design is currently running in software simulation mode or `false` if running in co-emulation mode.

Here is an example:

```
always @(posedge error_flag)
begin
    if ($axis_rccoff)
        // code executed only if design running in software simulation mode
    end
endmodule
```

Note: This system task is non-synthesizable and thus cannot be used in a DUT module.

Swapping to Software Simulation Mode at the Appropriate Time

To simulate the monitor code, you must initiate the swap from simulation acceleration mode to software simulation mode:

- You can control the swapping feature at the command line, using the commands described in the *SA Run-Time and Debug Commands* chapter of the *UXE Command Reference Manual*. If you wish to break on a signal in simulation acceleration mode, that signal must be exported at compile time, as previously explained.
- *Verilog testbench:* You can use the `$xc_cmd` system task to execute a CLI command directly from your testbench, for example:

```
module testbench ();
...
always @(posedge sys_clk)
begin
    if (packet_status==3'b0 && !packet_received)
        $xc_cmd("verbose 2");    // enable xc verbose messages
    end
endmodule
```

Note: The following commands are not supported through the `$xc_cmd` system task if you are using the `xeDebug` tool:

on, off, stop, run, tbrun, autorun, nbrun, match, bmatch

Monitoring Signals and Displaying Messages

The `$display` and `$monitor` system tasks, which can be used in the testbench to display message strings and signal values during simulation, are fully supported in the software simulator and produce expected results if used in non-accelerated modules.

However, if used in a DUT module, by default, these tasks will not work as expected while running the design in simulation acceleration modes, because they are not synthesizable and are thus ignored by the emulator hardware. If you are trying to add a testbench module to the DUT so it can be accelerated, this can be a problem, for example:

```
module dut (...);

always @(posedge sys_clk)
begin
    if (start_mem_cycle && write_en && read_en)
        $display("Invalid memory cycle at time %t", $time);
end
```

In this example, the `$display` system task is ignored if running the design in simulation acceleration mode.

Any of the preceding techniques for speeding up monitor code can also be used to display these messages. If using the partitioning technique, you would modify the previous example, as follows:

```
module dut (...);

reg dummy_reg;
always @(posedge sys_clk)
begin
    if (start_mem_cycle && write_en && read_en)
        dummy_reg = 1'b1; //register toggles when error condition detected
    else
        dummy_reg = 1'b0;
end
```

Now add the following code to a testbench module:

```
always @(posedge testbench.dut.dummy_reg)
    $display("Invalid memory cycle at time %t", $time);
```

In the previous example, only the `$display` message itself is simulated in software simulation; actual error checking is accelerated by the emulator hardware.

Compiling DUTs with Non-Synthesizable Code by Using Testbench Callback Technology

The Testbench Callback technology lets you embed non-synthesizable code in a DUT or call a task containing non-synthesizable code from a DUT design unit. The software simulator automatically executes this code while running.

The unique Testbench Callback technology lets you enable other types of non-synthesizable code, as well; this code is then executed.

Testbench Callback technology gives you the ability to:

- Improve the performance of designs with testbench code, such as bus protocol monitors and code that detects violations or fatal conditions.
- Display messages while running at hardware speeds.
- Execute non-synthesizable code, such as PLI C/C++ routines, Operating System calls, or other *testbench* code, when certain DUT conditions occur.
- Inject data into, and extract data from, your design at high speed, using `axis_smem` memory buffers and direct memory access routines.
- Load new data files, load memories, or dump contents of memories, and so on, on demand.
- Control (stop, start, and so on) Palladium XP Engine.
- Report assertion violations.

There are two ways to use Testbench Callbacks in Verilog designs:

- Testbench Callback Automated Method: In the automated method of using Testbench Callbacks, blocks of non-synthesizable code in the DUT are enabled using a special pragma. No other intervention is required, but some restrictions apply.
- Testbench Callback Primitive (`axis_tbcall`): The Testbench Callback primitive (`axis_tbcall`) lets you directly call a user-written task containing non-synthesizable code when a particular condition occurs. This method provides additional flexibility over the automated method and gives you greater control.

Testbench Callback Automated Method

Use IXCOM compiler to mark a region of statements inside a Verilog module that has been marked for mapping to hardware (under DUT hierarchy by using the `+dut` flag) to be executed in software. These set of statements are enclosed within a `begin..end` block. This

allows a very flexible way to execute a monitoring or error-reporting code, which otherwise would not be mapped to hardware. This feature is provided through the Testbench Callback automated method.

The automated method of using Testbench Callbacks lets you enable non-synthesizable code, such as `$display` statements embedded in a DUT module, typically located within the `//synopsys translate_off` blocks. To enable such code, add the following comment pragma to a `begin` statement in your `always` procedure:

```
// ixc tbcall_region
```

The code enclosed by the commented `begin` statement to the next `end` statement will not be checked by the Synthesis Policy Checker and will be automatically executed by the software simulator at the appropriate time. No user intervention is required. The comment pragma is ignored by all other tools.

This is illustrated by the following example.

```
module DFF_1(clk, D, Q,)
  input clk;
  input [3:0] D;
  output [3:0] Q;
  reg [3:0] Q;

  always @(posedge clk) begin
    Q <= D;
    begin //axis tbcall_region
      $display("Q= %d", $time, Q);
    end
  end
end
```

In this example, the block of code specified within the `tbcall_region` pragma is scheduled to execute after the non-blocking assignment in the simulator scheduling queue. This is done to simulate the behavior of the hardware. The `$display` statement should display the old value of signal, `Q`. However, in this example, the `tbcall_region` is executed after the non-blocking assignment, and the new value of signal `Q` is displayed in both the hardware and the software modes, when the `+dut` switch of the `ixcom` command are used. This happens when the signal value that is used, is assigned using the non-blocking assignment before the `axis tbcall_region` pragma.

To see the *old values* of VHDL signals and of Verilog signals that are the target of non-blocking assignments within the same `always` procedure, use the `+tbcnba` switch. With the `+tbcnba` switch, IXCOM analyzes the dependency of the signals that are read in the `axis tbcall_region` pragma and are assigned using the non-blocking assignment. IXCOM, then, generates the old values of such signals that are to be used in the `axis`

`tbcall_region` pragma. However, this applies to signals that are written and read within the same `always` procedure. If the signals are written in a different `always` procedure and read within the `axis tbcall_region` pragma, which is within another `always` procedure, new values of these signals are used.

VHDL Example

In this example, at the rising edge of `clk`, `writeline` will display the new value of `Q_tmp`.

```
library ieee;
use ieee.std_logic_1164.all;
use IEEE.std_logic_textio.all;
library axis;
use axis.axis.axis;
library std;
use std.textio.all;

entity DFF_1 is
port (
D : in std_logic_vector(3 downto 0);
clk : in std_logic;
Q : out std_logic_vector(3 downto 0 )
);
end DFF_1;

architecture DFF of DFF_1 is
signal Q_tmp : std_logic_vector(3 downto 0 );
begin

a1: process(clk)
begin
    if(clk'event and clk = '1') then
        Q_tmp <= D;
    end if;
end process a1;
    Q <= Q_tmp;

a2 :  process (clk)
variable l :line;
begin
if (clk'event and clk = '1') then
```

```
-- axis tbcall_region_on
write(1, NOW);
write(1, Q_tmp);
writeline(output, 1);
-- axis tbcall_region_off
end if;
end process a2;

end DFF;
```

Verilog Example

In this example, at the rising edge of `clk`, `$display` will display the new value of `Q`.

```
module DFF_1 (clk, D, Q);
input clk;
input [3:0] D;
output [3:0] Q;
reg [3:0] Q;
always @(posedge clk) begin
Q <= D;
end
always @(posedge clk) begin //axis tbcall_region
$display("Q= %d", $time, Q);
end
endmodule
```

Restrictions and Usage Suggestions for Testbench Callback Automated Method

Please be aware of the following restrictions and usage suggestions for the automated method of using Testbench Callbacks:

- The pragma can be added only to `always` procedures, not tasks or functions. However, you can call a task or function from the pragma code body.
- Additional restrictions apply to the use of delayed timing events and cross-module references within the pragma code body (Refer to Restrictions on Testbench Callback Primitive (`axis tbcall`) on page 278).
- Remember that all code within the comment pragma block is executed by the software simulator at a slower speed. For best performance, minimize the code within these pragmas. For example, if you have a conditional statement that checks for a particular condition, put the `axis tbcall_region` pragma after the condition-check code. That

way the check is performed in the hardware at full speed and only switches to the software simulator if the condition actually occurs, for example:

```
always @(posedge clk)      // condition checked in hardware
begin
    if (AddrW == AddrR)    // condition checked in hardware
    begin                  // axis tbcall_region switches to software simulator
        $display(" Write through operation not allowed!\n");
        $display(" Write Addr, Read Addr = %h", AddrW);
    end                    // switches back to Palladium XP here
end
```

For additional usage suggestions, refer to [Usage for Testbench Callback Primitive \(axis_tbcall\)](#) on page 281.

- When two `always` processes wake up at a clock edge, the correct behavior is to display the old value of the signal, that is, the value of the signal before it is updated.

For example, in the following code, the behavior should be to display the old value of signal `a` before it is incremented by 1:

```
always @(posedge clk)
begin
    a <= a + 1;
end
always @(posedge clk)
begin // ixc tbcall_region
    $display(a);
end
```

However, by default, when you use the `ixc tbcall_region` pragma, the updated value of the signal is displayed. In the above code, the value of signal `a` that is passed to the `$display` statement is read from the software side. By this time, the value of `a` is already updated, which is incorrect. To ensure that the generated code reads the value of the variables before these are updated, use the `+tbcnba` option of the `ixcom` command. The `+tbcnba` option enables reading of variables that can be assigned in a non-blocking way from outside of the process.

Testbench Callback Primitive (axis_tbcall)

IXCOM flow provides a special primitive, `axis_tbcall`, that lets you call a Verilog task while running in emulation modes. The task typically contains non-synthesizable code and is automatically executed in the software simulator. This mechanism allows you to perform operations not typically allowed in the DUT, such as testbench code, PLI C/C++ code, file I/O, display messages, and so on.

To call a testbench task from a Verilog module, call the built-in primitive `axis_tbcall` from within a DUT design unit, using the following syntax:

```
axis_tbcall(callback_signal, "callback_task");
```

The *callback_signal* is a 1-bit wire declared in the DUT module—ports cannot be used for this purpose. The *callback_task* is a local Verilog task with the testbench code to execute. To cause the *callback_task* to be called, create a transition from 0 to 1 on the *callback_signal*.

Restrictions on Testbench Callback Primitive (axis_tbcall)

The following restrictions apply when using the `axis_tbcall` primitive:

- The `callback_signal` should be triggered only by a sequential `always` procedure. Combinational blocks can have unexpected results like glitches. For example:

```
reg callback_trig, A, B;
always @(A or B)    // combinational: NOT recommended for Callback
begin
    if (A > B) callback_trig <= 1'b1;
    else callback_trig <= 1'b0;
end
```

```
axis_tbcall(callback_trig, "callback_task");
```

```
reg callback_trig, A, B;
always @(posedge clock) // sequential: Recommended for Callback
begin
    if (A > B) callback_trig <= 1'b1;
    else callback_trig <= 1'b0;
end
```

- Statements that generate delayed simulation events such as `wait`, `@event`, and delay assignments (`A <= #1 B;`) can be executed only after swapping into software simulation mode (refer to [Controlling Palladium XP from within Testbench Callback Tasks](#) on page 282). You should not use `forever` statements since these will prevent you from swapping back to emulation modes.
- The *callback_task* can call another task only if that task is declared in a testbench or other module running in the software simulator.
- Only the following signals can be referenced by the *callback_task*:

- ❑ Nets or registers declared in a DUT module can be referenced only if they are read. You can write to signals that represent edge-triggered flip-flop registers using the `$axis_rwrite` task (refer to [Writing to DUT Signals Using Testbench Callback Tasks](#) on page 282).

Note: Reading signals creates special Palladium XP I/Os, which impact performance.

- ❑ Remote signals using cross-module references can be referenced.

Note: Reading signals in the DUT creates special Palladium XP I/Os, which impact performance.

- ❑ Variables declared locally in the task can be read or written without restriction. You can reference variables in other tasks if the remote tasks are located in the testbench or other modules running in the software simulator.

Note: No Palladium XP I/Os are created.

- ❑ Memories mapped to the Verilog `axis_smem` primitive can be read or written using system tasks.
- ❑ If you read a DUT signal in an `axis_tbcall`, you will get the value at the end of current timestep, that is, after flip-flops have been updated, for example:

```
integer bcount;
always @(posedge clk)
begin
    bcount <= bcount + 1;           // bcount incremented to 1
    d_bcount <= bcount;           // change in d_bcount calls
                                // Callback Routine
end

axis_tbcall(d_bcount, "my_cb");

task my_cb; // called when d_bcount changes from 0 to 1
begin
    $display("bcount:%b", bcount); // displays updated value(1)
    $display("d_bcount:%b", d_bcount);
                                // displays updated flip-flop value!
end
endtask
```

- Some mismatches between the software and hardware runs of design can be observed when sampled value functions are used in continuous assignments, before calling the `axis_tbcall` primitive.

Example of Testbench Callback Primitive (`axis_tbcall`)

This example shows how to use `axis_tbcall` to display text from the DUT during emulation modes.

```
'timescale 1ns/100ps

module dut;
  reg [63:0] dL;
  reg [6:0]  dS;
  reg [1:0]  cnt;
  reg  enE;
  wire  enP;

  wire clk;

  axis_clkgen #(1, 20) (clk);

  always@(posedge clk) begin
    dL <= dL + 64'h0001000100010001;
    dS <= dS + 1;
    cnt <= cnt + 1;
    if(cnt == 0) begin
      enE <= ~enE;
    end
  end

  initial begin
    dL = 0;
    dS = 0;
    enE = 0;
    cnt = 0;
    $axis_set_clkgen("clk", 0, 5, 10, 10);
  end

  ixc_pulse(enP, enE);
  axis_tbcall(enP, "t1");
```

```
task t1;
    reg [32:0] t;

    begin
        t = $time;
        $display(t, "L: %h", dL);
        $display(t, "S: %h", dS);
        if(t > 1024)
            $finish;
    end
endtask

endmodule
```

Usage for Testbench Callback Primitive (*axis_tbcall*)

Here are some usage suggestions for the Testbench Callback primitive (*axis_tbcall*):

- As indicated earlier, the Testbench Callback task runs on the software simulator at a slower speed. For best performance, minimize both the amount of code in the task as well as the number of times the task is called. The condition that triggers the Testbench Callback task should be checked in Palladium XP and should occur infrequently for best performance.
- Avoid reading DUT signals from the Testbench Callback task, since this causes special Palladium XP I/Os to be created. The most efficient method for transferring data into and out of the Testbench Callback task is by using *axis_smem* memories. Instantiate the memory in the DUT and write the desired data to it using synthesizable code. You can then read/write the contents of the memory in the Testbench Callback task using special commands.
- Minimize disk I/O in the Testbench Callback task. One suggestion is to use a local memory declared in the Testbench Callback task and load all input stimulus at once into it. You can then use the system tasks to transfer that data to *axis_smem* memories in the DUT.
- Minimize C/C++ calls in the Testbench Callback task. PLI C/C++ is often used to read input stimulus from files and to process output data to disk. A more efficient method is to use *axis_smem* memory input and output buffers and two standalone C/C++ programs (that is, not linked to the simulator): one program to read the input stimulus and generate a memory image; the other to read the output buffer data and post-process it.

Writing to DUT Signals Using Testbench Callback Tasks

Testbench Callback tasks can be used to write to Verilog registers *that are edge-triggered flip-flops* in the DUT using the `$axis_rwrite` system task, for example:

```
$axis_rwrite(tb.dut.reg, 1);
```

However, to avoid unexpected results, the registers and signals should not be written to in the same timestep in which the DUT itself writes them. For example, if a signal is updated on the negative-edge of a clock signal in the DUT, the testbench callback task should write to it on the positive-edge.

Controlling Palladium XP from within Testbench Callback Tasks

The following commands enable you to execute most CLI commands from within your Testbench Callback task. The following commands are most commonly used for controlling Palladium XP:

■ `xc off`

This command first swaps the design at the end of the current timestep to software simulation mode from either emulation mode, then continues running in software simulation mode. This swap causes ignored DUT design units (those specified with the `+dutignore+module_name` option) and code within `translate_off` blocks to become active and begin executing. You must execute this command in the Testbench Callback before executing any `wait`, `@event`, or assignments with delays.

■ `xc stop`

This command swaps the design into software simulation mode and stops the running of the design at the end of the current timestep. You are also returned to the CLI.

■ `xc on`

This command swaps the design immediately to emulation mode from software simulation mode. The DUT is downloaded into Palladium XP, which continues running in an emulation mode. You must specify this command at the bottom of your Testbench Callback if you used either the `xc stop` or `xc off` commands earlier.

■ `xc autorun`

This command swaps the design immediately to emulation mode from software simulation mode. The DUT is downloaded into Palladium XP, which continues running in emulation mode. You must specify this command at the bottom of your Testbench Callback if you used either the `xc stop` or `xc off` commands earlier.

Assertion-Style Callbacks

If your design uses many assertion-style checkers that produce messages only when an error condition occurs (infrequently), you can get better performance if you use the assertion-style version of the Testbench Callback primitive: pass a # (1) parameter to the regular primitive:

```
axis_tbcall #(1) (callback_signal, "callback_task");
```

Assertion-style primitives work the same way as regular primitives except they do not create Palladium XP I/Os, so you should *not* use them for displaying DUT signal values because they will be incorrect. To read these signals, first swap to software simulation mode using `xc off`.

```
axis_tbcall #(1) (assert_signal, "assert_task");
task assert_task;
    begin
        $display("design data %h", design_data);
    end
```

Compiling Designs in Stages using IXCOM Compiler

The IXCOM compilation process comprises the following compilation stages (or steps):

- ❑ Design analysis and elaboration
- ❑ Compilation of the testbench and behavioral portion of the design for software simulation using IES
- ❑ Compilation of the design for the Palladium XP hardware

Note: Using the `-target` option of the `ixcom` command, you can control the compilation process to start from, stop at, or rerun from a particular compilation stage. To control the compilation, use the various arguments of the `-target` option as described in the following sections. Each argument can be used to compile the design up to that stage, or to rerun the stage.

Specifying the Design Compilation Stages

You can specify the arguments with the `-target` option of the `ixcom` command as per your compilation requirements.

Specifying the Software Compilation Stage

Use the `swmodel` argument to stop at or rerun the software compilation stage. During this stage, the software model for the design is generated to be used for software simulation.

The design compilation process depends on the arguments you specify with the `swmodel` argument:

- ❑ If you specify all the design source files, the design is **analyzed, elaborated**, and the appropriate **IES** utilities are invoked to generate the **simulation snapshot**:

```
ixcom -target swmodel -ua +dut+<DUT_top_name>  
      <testbench_src_files>  
      <design_src_files>
```

- ❑ Alternatively, if you include the `-top` option, it is assumed that the design files have been already analyzed, and design compilation proceeds from the **elaboration** stage. After elaboration of the specified design source files, the appropriate **IES** utilities are invoked to generate the **simulation snapshot**:

```
ixcom -target swmodel -ua +dut+<DUT_top_name>  
      -top <testbench_src_files>  
      <design_src_files>
```


- ❑ If, however, you do not specify any of the design source files, nor include the `-top` option with the `ixcom` command, **only the simulation snapshot** is generated by invoking the appropriate **IES** utilities:

```
ixcom -target swmodel
```

Important

In order to compile the complete design so that it can be run on hardware, you must compile the design using the `hdlice` and `xecompile` arguments of the `-target` option separately.

Re-running the Software Compilation Stage

To rerun the `swmodel` stage, simply issue the following command:

```
ixcom -target swmodel
```

For the above command to be successful, the database file (`./xc_work/xc_make`) must exist. Else, the compilation process aborts, and the error message, that the previous design compilation stage was not successfully completed, is issued.

Specifying the Design Synthesis Stage (-target hdlice)

Use the `hdlice` argument to stop at or rerun the design synthesis stage. During this stage, the design is synthesized using HDL Compiler, and the netlist is generated, which can be used to accelerate the design on the Palladium XP hardware.

The design compilation process depends on the arguments you specify with the `hdlice` argument:

- ❑ If you specify all the design source files, the design is **analyzed, elaborated**, the appropriate **IES** utilities are invoked to generate the **simulation snapshot**, and **HDL Compiler** is invoked to generate the **design netlist**:

```
ixcom -target hdlice -ua +dut+<DUT_top_name>  
      <testbench_src_files>  
      <design_src_files>
```

- ❑ Alternatively, if you include the `-top` option, it is assumed that the design files have been already analyzed and elaborated, and design compilation proceeds from the simulation snapshot generation stage. The appropriate **IES** utilities are invoked to generate the **simulation snapshot**, and **HDL Compiler** is invoked to generate the **design netlist**:

```
ixcom -target hdlice -ua +dut+<DUT_top_name>  
      -top <testbench_src_files>
```

Note: To re-run the synthesis stage using the above command, the design must have been compiled using the `-ua` option in the previous `ixcom` command.

 Important

In order to compile the complete design so that it can be run on hardware, you must compile the design using the `xecompile` argument of the `-target` option separately.

Re-running the Design Synthesis Stage

To re-run the `hdllice` stage, simply issue the following command:

```
ixcom -target hdllice
```

Specifying the Hardware Compilation Stage (-target xecompile)

Use the `xecompile` argument to run or rerun the hardware compilation stage. During this stage, the design is compiled using the Palladium XP back-end compilers and mapped to the Palladium XP emulator.

The `xecompile` stage does not apply to any other use model except for being run separately, without any other options and arguments:

```
ixcom -target xecompile
```

For the above command to be successful, the database file (`./xc_work/xc_va.ver`) must exist. Else, the compilation process aborts, and the error message, that the previous design compilation stage was not successfully completed, is issued.

Generating Pulses

The following three primitives are provided for generating pulses and clocks:

- `ixc_pulse` (for Testbench Callback, Synchronous Elements)
- `ixc_pulse#(0)` (for Combinational Logic and Latches)
- `ixc_pulse#(1)` (for Palladium XP eval or Simulation Timestep Clock)

`ixc_pulse` (for Testbench Callback, Synchronous Elements)

The `ixc_pulse` primitive generates a pulse when an input *signal* changes. Here is how to instantiate it:

```
ixc_pulse (clk, signal);
```

This pulse goes from 0 to 1 when the signal changes, then back to 0 at the end of the simulation timestep or Palladium XP evaluation cycle.

This feature is useful for creating pseudo-timed events, such as creating a pulse on any edge of a clock. It is also used in conjunction with the Testbench Callback technology when you need a 0 to 1 transition to occur on the Testbench Callback signal.

Note: The output *clk* can be used only to directly drive synchronous elements, such as flip-flops or synchronous memories. Use only the *posedge* of the pulse signal *pc/k* in the design, and never use the pulse value in combinational logic or latches.

`ixc_pulse#(0)` (for Combinational Logic and Latches)

The `ixc_pulse#(0)` primitive also generates a pulse (*ps*) when an input *signal* changes value. Here is how to instantiate it:

```
ixc_pulse#(0) (ps, signal);
```

However, this primitive has different delta cycle semantics: it changes from 0 to 1 and holds the value 1 to the last delta cycle, End-of-Time (EOT), of the current simulation time. Then it goes back to 0 immediately after EOT.

You can use this pulse signal in combinational logic and latches to do special modeling.

For example, consider the following flip-flop model:

```
always @(posedge c) q <= d;
```

To guarantee deterministic results in parallel simulation, Palladium XP forces all flip-flops to latch the old value of *d* into *q* and does not allow special flip-flop modeling that could capture the new value of *d* on the posedge of *c*. But `ixc_pulse#(0)` provides a way to achieve this function, as follows:

```
ixc_pulse #(0) (pc, c);  
always@(pc or c or d) if(pc & c) q = d;
```

In this way, you can build an equivalent flip-flop model that will latch in the new value of *d*.

Important

Do not use the edge of the `ixc_pulse#(0)` signal because edge detection logic in Palladium XP is based on the EOT value of the previous eval. Since the EOT value of this new pulse will be 1, not 0 as in the regular `ixc_pulse` model, the edge detector will fail if the input signal changes in every simulation time step. To use the positive edge of the pulse, use the regular `ixc_pulse` primitive to generate the pulse. And never use the negative edge of the pulse signal because it will not work in Palladium XP.

ixc_pulse#(1) (for Palladium XP eval or Simulation Timestep Clock)

The `ixc_pulse#(1)` primitive generates a Palladium XP eval clock or the simulation timestep clock.

The only way to use this primitive is as follows:

```
wire VCC = 1'b1;  
ixc_pulse#(1) (evalClk, VCC);
```

Again, use only the positive edge of *evalClk* and never use its value in logic or latches.

Note: `ixc_pulse (evalClk)` is no longer supported.

Pragma Handling in IXCOM

By default, the code within the IXCOM-supported pragmas is excluded from synthesis when compiling for hardware, that is, the Simulation Acceleration mode, but retained when compiling for software simulation mode. You can modify this behavior by using the following options with the `ixcom` or `vlan/vhan` commands for `translate_on`, `translate_off`, and Synopsys `full_case` and `parallel_case` pragmas:

- Handling `translate_on` and `translate_off` Pragmas in IXCOM: Use the following options for `translate_on` and `translate_off` pragmas:
 - `+rtlIgnPragma[+mod]`: Using this option implies that the `translate_off` and `translate_on` pragmas are ignored in the design or the specified design units and the code within the pragma blocks is executed in both the hardware and software partitions. If the optional `<mod>` parameter is not specified, it implies that this option is valid for all the modules or design units in the design, else it implies that it is applicable only to the specified modules or design units. This option needs to be specified with the `vlan` and `ixcom` commands to take effect in the Mixed-language designs. For more information, refer to description of `+rtlIgnPragma` in the *Commands and System Tasks for SA Mode* chapter of the *UXE Command Reference Manual*.
 - `+rtlCommentPragma[+mod]`: Using this option implies that the pragma itself is honored and the code within the `translate_on` and `translate_off` pragmas is treated as comment and ignored in both hardware and software partitions. This option needs to be specified with the `vlan` command. If the optional `<mod>` parameter is not specified, it implies that this option is valid for all the modules or design units in the design, else it implies that it is applicable only to the specified modules or design units. For more information, refer to the description of `+rtlCommentPragma` in the *Commands and System Tasks for SA Mode* chapter of the *UXE Command Reference Manual*. Using this option helps in ignoring the unnecessary code, such as syntactically incorrect code, without modifying the source, provided the code is inside pragma. If this option is provided with the `ixcom` command, the code inside the pragma must be syntactically correct for the file to be parsed.

Note: The `+mod` argument is not supported with the `+rtlCommentPragma` option when used with the `vhan` command. For VHDL designs, only the code inside the pragma is treated as commented out and removed.
- Handling `full_case` and `parallel_case` Pragmas in IXCOM: Use the following option Synopsys `full_case` and `parallel_case` pragmas:
 - `+rtlHonorCasePragma`: Using this option implies that the Synopsys `full_case` and `parallel_case` pragmas within a Verilog `case` statement are honored during

synthesis. This option needs to be specified with the `vlan` and `ixcom` command. For more information, refer to the description of `+rtlHonorCasePragma` in the *Commands and System Tasks for SA Mode* chapter of the *UXE Command Reference Manual*.

- `full_case` pragma: In a Verilog `case` statement, the case item expression might not cover all the possible values that the case expression can assume. In such a case, the RTL compiler creates additional logic to handle the missing conditions. Specifying the `full_case` pragma allows RTL compiler to generate more efficient logic by eliminating the logic for the missing case conditions. This might result in synthesis/simulation mismatch if case expression assumes a value that does not match with any case item expression.
- `parallel_case` pragma: In a Verilog `case` statement, the case item expression can be a dynamic expression. In such cases, LRM requires that the case item expression be evaluated sequentially as more than one case item expression might match with the specified case expression. The statements in the first matching case item expression needs to be executed. For RTL compiler, this implies synthesizing the priority logic. Using `parallel_case` pragma instructs the RTL compiler that the case item expression are all mutually exclusive and there will *only* be one match. RTL compiler can eliminate the priority logic and synthesize a more efficient logic. This, can, however result in synthesis/simulation mismatch if more than one case item expression matches the specified case expression.

Using `translate_off` and `translate_on` Pragma

The following rules are followed in IXCOM for determining whether or not the `translate_off` and `translate_on` pragma usage is suitable for hardware:

- ❑ An object written in the `translate_off` region should not be read inside in a non-`translate_off` region.
- ❑ The code inside the `translate_off` region should not be written to module ports.

The above usages result in a hardware-software mismatch, and modules containing such code are partitioned to the software side. Reading an object in the `translate_off` region is considered a legal usage for hardware. If the IXCOM compiler detects an assertion in the DUT that references a signal in a `translate_off` region in the design, the assertion is ignored with a warning.

Ignoring Pragmas in VHDL Designs

Use `-ignore_pragma <name>` option with the `vhan` command to ignore all the pragmas in the VHDL source that are prefixed with the specified `<name>`.

For example, `-ignore_pragma synopsys` will not honor pragmas that have `synopsys` as their prefix, such as, `synopsys translate_off/on`. The ignored pragmas are treated as comments.

Using `//ixc` Pragmas

The `//ixc` pragmas can be used for passing compiler directives to IXCOM. Some of the `//ixc` pragmas are used:

- ❑ To enable non-synthesizable code, such as `$display` statements embedded in a DUT module, typically located within the `//synopsys translate_off` blocks.

```
// ixc tbcall_region
```
- ❑ To move the action blocks of assertions (in DUT) to hardware or software partition, instead of the default behavior, which is to run the pass action block on hardware and fail action block on software.

```
// ixc hw_action_block  
// ixc sw_action_block
```
- ❑ To specify that all loops or a subset of the loops in a behavioral module should be unrolled prior to any behavioral transformation.

```
// ixc bc_unroll_loop
```
- ❑ To *mark* a portion of a module that should be simulated while in software simulation mode.

```
// ixc rtl_on  
// ixc_rtl_off
```

Earlier, these pragmas were prefixed with `//axis` instead of `//ixc`. Support for `//axis` will be deprecated in future releases.

Enabling `irun` to Handle Sparse Memories

Add the following pragma `/* sparse */` in the original HDL, to enable *irun* to recognize and handle the memory declaration for sparse memories. This is especially useful in the case of large memories that would otherwise cause a compilation error.

For example:

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
---- test.v ----  
module dut;  
    reg [575:64] mem [0:16777215];  
    /* sparse */  
endmodule // dut
```

To compile the above design, the following command is used:

```
ixcom -clean +dut+dut -ua test.v
```

Compiling the above design to run using *irun* will generate an error if this pragma is not added.

This pragma indicates that the memory declaration immediately preceding the pragma is to be implemented as a sparse memory during simulation (that is, physical memory is allocated dynamically for it at run time in the simulation). IXCOM recognizes this pragma and passes it to *irun* for processing. This pragma has no effect on how the memory is implemented in hardware. Refer to the *Sparse Arrays* section in the *NC-Verilog Simulator Help* manual available in the INCISIV documentation set for details of this pragma.

Reading and Writing Signals in DUT Modules

IXCOM lets you cross-reference nets and registers in DUT modules from your testbench for the purpose of using them in PLI calls, assuming the following conditions are met:

- DUT nets and registers can be read only; you cannot write to these objects.
- DUT nets and registers can be read at run time, only if they are *exported* at compile-time. Exporting a signal from the DUT allows you to read its value during simulation. Signals are automatically exported if:

- You reference the DUT net or register in your testbench or other module running in software simulation mode, for example:

```
always @(posedge testbench.chip.sys_clk)
    // read reference to DUT signal
```

- For Verilog, you explicitly export the signal using either the `$export_read` or `$export_frcrel` Verilog system tasks from a testbench module. Refer to *Command and System Tasks for SA Mode* chapter of the *UXE Command Reference Manual* for detailed information on these tasks.
 - For VHDL, you explicitly export the signal using either the `export_read` or `export_frcrel` VHDL procedures from a testbench module. Refer to *Command and System Tasks for SA Mode* chapter of the *UXE Command Reference Manual* for detailed information on these procedures.
- You can use VHDL `nc_mirror` procedures and Verilog `$nc_mirror` system tasks to trace the value of any VHDL or Verilog code in the design from a VHDL architecture or Verilog module. These procedures and system tasks are especially useful for monitoring the values of signals in the DUT during simulation and processing them in the testbench. Refer to [Using nc_mirror in IXCOM](#) for detailed information on these procedures and system tasks.
- If you use access routines in your PLI code, and they attempt to access signals that were not exported, you will get the following run-time error:

```
Error! Cannot fetch value|set value|add vcl because the handle passed
to acc_routine_name corresponds to an object signal_name_and_scope
in hardware
```

- If you use `tf` routines, and you attempt to pass a reference to a DUT signal as a parameter to the PLI call, you will get an error during compile-time by default, for example:

```
initial
    $my_pli(testbench.chip.my_reg);
--
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

Error: XMR defined in DUT cannot be assigned in testbench

To address this issue, you must enable read access to the PLI routine. This is done with the `+tfReadOnly+pli_routine` compiler option. To enable read access for several PLI routines, append their names to this option:

```
+tfReadOnly+<pli_routine1>+<pli_routine2>+...
```

If you do not specify any routine names, read access will be enabled for all routines in the design.

You can specify the `+tfReadOnly` option with the `ixcom` command.

For a Mixed HDL Design:

```
ixcom -top top_level_design_unit other_options +tfReadOnly+<my_pli>
```

For a Verilog-only Design:

```
ixcom Verilog_sources other_options +tfReadOnly+<my_pli>
```

Accessing DUT Signals from Non-HDL Interfaces

In order to access DUT signals from non-HDL interfaces like PLI, VPI, VHPI, Systemc, or Tcl, you need to write certain directives (`$export_read`, `$export_deposit`, and `$export_frcrel`). You can choose to generate these directives automatically by using the `-genhwaccessfile <file_name>` option with the `irun` or `ncelab` command at the time of elaborating or compiling the design in pure simulation mode.

For example:

```
irun [other_irun_options] \  
-genhwaccessfile <hwaccess_file_name> \  
-input <sim.tcl>
```

During simulation, the objects that are accessed by the non-HDL interfaces are monitored along with the types of access required for each object. When you exit the simulation, a hardware access file is created with the specified filename.

The hardware access file contains the export directives for the signals to be read, written, and forced, enclosed within an `initial` statement. The `$export_read`, `$export_deposit`, and `$export_frcrel` directives are used to specify the signals to be read, written, and forced, respectively. Following is an example of a hardware access file:

```
(* ixcom_instrumented_mod *)  
module dummy;  
  `ifdef AXIS  
    initial  
      begin  
        $export_read(<pathname1>, <pathname2>, ..., <pathname10>);
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
$export_read(<pathname1>, <pathname2>, ..., <pathname20>);
$export_deposit(<pathname1>, <pathname2>, ..., <pathname10>);
$export_read( <pathname21>, <pathname22>);
$export_frcrel( <pathname1>, <pathname2>, ..., <pathname8>);
$export_deposit(<pathname11>);
end
`endif
endmodule
```

The signals are qualified by their full path and the directives can appear in any sequence. If a signal is read, written, and forced, all the three directives, `$export_read`, `$export_deposit`, and `$export_frcrel` are generated for that signal qualified by its full path.

Then, at the time of compilation, specify this hardware access file with the `ixcom` or `irun -hw` command as an additional source file. No other changes are needed to your source files:

- For using `ixcom` to compile the designs:

```
ixcom -top dummy \
      [other_ixcom_options] <hwaccess_file_name>
```

- For using `irun` to compile designs for Simulation Acceleration mode:

```
irun -64 -hw -ua -top dummy \
      [other_ixcom_options] <hwaccess_file_name> \
      [other_irun_options]
```

Considerations for Using the -genhwaccessfile Option

Keep in mind the following points when using the `-genhwaccessfile` option:

- The `-genhwaccessfile` option should be used to generate an access file in pure simulation environment only. This option should not be used on an IXCOM-compiled snapshot by running `irun` on it because this adds certain unwanted signals (like those in `xc_top` and other internally instrumented signals) in the hardware access file, which can degrade performance.
- Avoid use of `irun` options, such as `-ncinitialize` that can dump redundant signals in the hardware access file, thereby impacting performance.
- The `-genhwaccessfile` option is not supported with Specman flow.
- For SystemVerilog compilation unit scope names, the names are generated in the following manner in the hardware access file:

```
$export_read(worklib.$unit_0x7189c4db:<var_name>)
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

So, in order to compile this file with IXCOM, you need to manually modify such names as follows:

```
$export_read($unit::<var_name>)
```

Supported Interfaces with Hardware Access Files

IXCOM adds instrumentation to ensure proper accessibility for signals read, written, and forced through non-HDL interfaces, such as PLI/VPI, VHPI, SystemC, and Tcl. The generated directives corresponding to the functions of PLI/VPI, VHPI and TCL interfaces are listed in the following table:

Interface	Directive	Functions
PLI 1.0	\$export_read	acc_vcl_add() acc_fetch_value() acc_fetch_paramval() acc_fetch_paramval_mtm()
	\$export_deposit	acc_set_value() with delay model : accAssignFlag, accNoDelay, accInertialDelay, accTransportDelay, accPureTransportDelay
	\$export_frcrel	acc_set_value() with delay model : accForceFlag

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

Interface	Directive	Functions
VPI	\$export_read	vpi_get_value() vpi_register_cb() cbValueChange cbForce/ cbRelease (on the expression) cbAssign/ cbDeassign (on the register being assigned)
	\$export_deposit	vpi_put_value() with the vpiAssignFlag, vpiNoDelay, vpiInertialDelay, vpiTransportDelay, and vpiPureTransportDelay flags
	\$export_frcrel	vpi_put_value() with the vpiForceFlag flag
VHPI	\$export_read	vhpi_get_value() vhpi_register_cb() for vhpiCbValueChange
	\$export_deposit	vhpi_put_value() with the vhpiDeposit and vhpDepositPropagate flags
	\$export_frcrel	vhpi_put_value() with the vhpiForcePropagate flag Note: The vhpiForce is not supported.
SystemC	\$export_read	observe_foreign_signal() control_foreign_signal()
	\$export_deposit	control_foreign_signal(<signal-name>, false)
	\$export_frcrel	control_foreign_signal(<signal-name>, true)

For the Tcl command interface, the export directives are dumped into the hardware access file according to the information available in the *Access Requirements for Tcl Interactive Commands* section of the *Elaborating Your Design* document in the INCISIV documentation set.

Using nc_mirror in IXCOM

The `nc_mirror` procedure or system task enables you to trace the value of a VHDL signal, port, or variable or a Verilog variable or wire lying anywhere in the design from a VHDL architecture or Verilog module. These procedures and system tasks can also be used to mirror the value of a source signal to a destination signal.

With an `nc_mirror` call, whenever the value of the specified source object is modified, the modified value is copied to the specified destination object. An `nc_mirror` call does not create additional drivers between the source and destination objects.

The procedure or system task performs syntax and error checks on the "`<source>`" and "`<destination>`" parameters to ensure that the two signals are of a compatible data type, same size, and valid range. If the two do not match, it reports an error message for the unsupported data types.

Note: In *irun*, mirroring of the value starts after `nc_mirror` is executed. In UXE, mirroring starts at the beginning of simulation regardless of when `nc_mirror` is actually executed.

VHDL nc_mirror() procedure

To use the `nc_mirror` procedure in the VHDL code, add the following lines before the entity declaration:

```
library AXIS;  
use AXIS.axis_util.all;
```

Syntax:

```
nc_mirror ("<destination>", "<source>", "verbose");
```

Or,

```
nc_mirror (destination => "<destination>",  
source => "<source>",  
verbose => "verbose");
```

Verilog `$nc_mirror()` system task

To use the `$nc_mirror` system task in the Verilog code, you can use the following syntax:

Syntax:

```
$nc_mirror ("<destination>", "<source>", "verbose");
```

Parameters

Table 4-2, “Parameters for `nc_mirror`,” on page 299 lists the parameters for the `nc_mirror` procedures and system tasks.

Table 4-2 Parameters for `nc_mirror`

Parameter	Description
<code>"<destination>"</code>	Specifies the full hierarchical path or a relative path of a VHDL signal, port, or variable or a Verilog variable or wire to which the value of the source signal is to be mirrored. You must specify either the destination signal, or pass a null string using " ". If you do not specify a destination signal, the new value of the source signal is displayed.
<code>"<source>"</code>	Specifies the full hierarchical path or a relative path of a VHDL signal, port, or variable or a Verilog variable or wire that is to be mirrored or traced. This parameter is mandatory.
<code>"verbose"</code>	Optional parameter used for status reporting. It displays extra information.

- **Full Hierarchical Path:** Full hierarchical paths begin with the top-level module in the hierarchy.

The paths begin with the top-level module name in the Verilog code.

For example:

```
$nc_mirror("top_mod.dest", "top_mod.I1.src", "verbose");
```

The paths begin with a colon (:) in the VHDL code.

For example:

```
nc_mirror(":dest", ":I1:src", "verbose");
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

- **Relative Path:** Relative paths begin with a module instance name in the current module.

```
nc_mirror("dest", "I1:src", "verbose");
```

The upscope operator (^) allows you to move one level up in the hierarchy.

For example:

```
nc_mirror("dest", "^..I2.signal1", "verbose");
```

Supported Data Types

Table 4-3, “Supported data types for nc_mirror,” on page 300 lists the supported data types for the `nc_mirror` procedures and system tasks.

Table 4-3 Supported data types for nc_mirror

Language	Signal ¹		Supported Data Types	
	Source	Destination	Source	Destination
VHDL-to-VHDL	Testbench	Testbench	Scalar types and one dimensional array ²	Scalar types and one dimensional array
	DUT	Testbench	std_ulogic, std_logic, std_ulogic_vector, or std_logic_vector	std_ulogic, std_logic, std_ulogic_vector, or std_logic_vector
	DUT	DUT	std_ulogic, std_logic, std_ulogic_vector, or std_logic_vector	std_ulogic, std_logic, std_ulogic_vector, or std_logic_vector
VHDL-to-Verilog	Testbench	Testbench	std_ulogic, std_logic, std_ulogic_vector, or std_logic_vector	Packed array
	DUT	Testbench	std_ulogic, std_logic, std_ulogic_vector, or std_logic_vector	Packed array
	DUT	DUT	std_ulogic, std_logic, std_ulogic_vector, or std_logic_vector	Packed array
Verilog-to-Verilog	Testbench	Testbench	Packed array	Packed array
	DUT	Testbench	Packed array	Packed array
	DUT	DUT	Packed array	Packed array

Table 4-3 Supported data types for nc_mirror

Language	Signal ¹		Supported Data Types	
	Source	Destination	Source	Destination
Verilog-to-VHDL	Testbench	Testbench	Packed array	std_ulogic, std_logic, std_ulogic_vector, or std_logic_vector
	DUT	Testbench	Packed array	std_ulogic, std_logic, std_ulogic_vector, or std_logic_vector
	DUT	DUT	Packed array	std_ulogic, std_logic, std_ulogic_vector, or std_logic_vector

1. Testbench-to-DUT signal: Not Supported. nc_mirror does not support mirroring a value from the source signal in testbench to the destination signal in DUT.
2. Record and array data types: Record types group elements of many types and each element can be accessed by its field name. You can access an element of a record that has a supported data type. Array types group elements of same types. You can access an array that has a supported data type.

Limitations

Note the following restrictions on the use of nc_mirror:

- When both the "<destination>" and "<source>" signals are VHDL, the data-types *must* be assignment compatible.
- When copying from Verilog to VHDL or vice versa, the Verilog bit width *must* match that of VHDL.
- Mirroring a value from the "<source>" signal in testbench to the "<destination>" signal in DUT is not supported for nc_mirror.
- When the "<source>" signal is in DUT and the "<destination>" signal is in testbench, the primary output must be created for the "<source>" signal.

Using Plus Arguments

IXCOM supports the use of *plus* arguments at run time, which are checked with the \$test\$plusargs or \$value\$plusargs system task in the Verilog part of your design

only. By using this task and by specifying different plus arguments, you can cause different code, such as different tests, to run when you simulate your design. All plus options passed to ixcom are passed to run-time software.

For example, you can use the `$test$plusargs` system task to selectively enable VCD-on-Demand dumping at run time, as follows:

```
initial
begin
if ($test$plusargs("dumpon"))
begin
    $dumpfile("design.vcd");
    $dumpvars;
end
end
```

Specifying `+dumpon` at run time gives you the flexibility of enabling or disabling dumping without recompiling:

```
irun -R +dumpon /*runs with dumping enabled*/
irun -R /*runs with dumping disabled*/
```

Support for Encryption in SA Mode

IXCOM supports the following encryption methods, by default:

- Verilog-2001 standard encryption by `ncprotect`
- Verilog-XL encryption
- Verilog-2005 standard encryption by `ncprotect`

By default, the `ncprotect` encryption is used. You can import all files with any encryption style to the IXCOM together without using any additional option.

IXCOM also supports a combination of these encryption styles.

For example, suppose:

- ❑ `a.v` is encrypted to `a.vp` using Verilog-XL style
- ❑ `b.v` is encrypted to `b.vp` using `ncprotect` style

Then, you can analyze these files separately:

```
vlan +sv a.vp
vlan +sv b.vp
```

Reading Encrypted Files

You can use the Cadence `ncprotect` utility to encrypt files for both Verilog and VHDL by using the following ways:

- Use the `-autoprotect` option to encrypt the entire source file.

```
ncprotect -autoprotect <source_file>
```

- Specify a source file that contains the `protect begin` and `protect end` pragmas to mark the regions for encryption.

```
ncprotect <source_file>
```

Or,

```
ncprotect -language {vlog|vhdl} <source_file>
```

For example, the following `<source_file>` contains a protection pragma:

```
//pragma protect
//pragma protect begin
module Add1 (sum, cout, ain, bin, cin);
input ain, bin, cin;
output sum, cout;
//pragma protect end
```

By default, when the `-autoprotect` option is not used, `ncprotect` encrypts the text between `//pragma protect begin` and `//pragma protect end`.

For the Verilog-2005 standard, `ncprotect` encrypts the text between `'pragma protect begin` and `'pragma protect end`.

For the Verilog-XL encryption, `ncprotect` is not used. However, Verilog-XL encrypts the text between `'protect` and `'endprotect`.

```
verilog loopcnt_comps.xl.v +protect
```

For more information on using the `ncprotect` utility, refer to *IP Protection* available in the INCISIV documentation set.

Encryption with Three Levels of Debug Capability

You can use the Cadence `ncprotect` utility to encrypt files and specify the levels of debug capability by using the following ways:

- Use the `-autoprotect` and `-simulation {portview|debugall|none}` options to encrypt the entire source file and specify the level of debug capability.

```
ncprotect -autoprotect -simulation {portview|debugall|none} <source_file>
```

You can specify the following levels of debug capability:

- ☐ `portview`: Provides access to module and entity names, ports, generics, and parameters. Access to internal registers, wires, local parameters, signals, and variables is not allowed.
- ☐ `debugall`: Provides the ability to traverse the complete design.
- ☐ `none`: Denies debug capability.
- Specify a source file that contains `rights` pragma to specify the level of debug capability as `portview`, `debugall`, or `none`.

```
ncprotect <source_file>
```

For example, a source file can contain the following `rights` pragma:

```
//pragma protect rights=(simulate=(viewers="portview"))
```

If two pragmas with different protection rights are specified in the same region, then the first pragma is used to encrypt the source file.

Application of Protection Rights by the IXCOM Compiler

A protection right is applied to a source file or a module based on the following conditions:

- When a module is protected with the same protection right, it is dumped with the specified protection right in the following cases:
 - ☐ The entire source file or module is protected with one pair of `rights` pragma and the `-autoprotect` option is not specified to `ncprotect`.
 - ☐ A source file is protected by specifying the `-autoprotect` and `-simulation{portview|debugall|none}` options to `ncprotect`.
- When a module has multiple protected regions with different protection rights, then the IXCOM Compiler will honor all protection rights and the protected regions of the module are dumped with the same protection right.

For example, the following module is protected with the `ncprotect -lang vlog test.v` command:

```
//pragma protect begin
//pragma protect rights=(simulate=(viewers="portview"))
module m (p1, p2);
//pragma protect end
wire w1;
//pragma protect begin
//pragma protect rights=(simulate=(viewers="debugall"))
```

```
assign p2 = p1;
//pragma protect endendmodule
```

It is dumped as the following module:

```
//pragma protect begin
//pragma protect rights=(simulate=(viewers="portview"))
module m (p1, p2);
//pragma protect end
wire w1;
//pragma protect begin
//pragma protect rights=(simulate=(viewers="debugall"))
assign p2 = p1;
//pragma protect endendmodule
```

- When a source file or a module is protected with multiple protection rights, but the `-autoprotect` and `-simulation{portview|debugall|none}` options are specified to `ncprotect`, then it is dumped with the protection right specified with the `-autoprotect` option.

For example, the following module is protected with the `ncprotect -autoprotect -simulation debugall test.v` command:

```
//pragma protect begin
//pragma protect rights=(simulate=(viewers="debugall"))
module m (p1, p2);
//pragma protect end
wire w1;
//pragma protect begin
//pragma protect rights=(simulate=(viewers="none"))
assign p2 = p1;
//pragma protect endendmodule
```

It is dumped as the following module:

```
//pragma protect begin
//pragma protect rights=(simulate=(viewers="debugall"))
module m (p1, p2);
wire w1;
assign p2 = p1;
endmodule
//pragma protect end
```

Support for Strengths in SA Mode

IXCOM supports the following strengths on the primitives and continuous assignments:

- ☐ SUPPLY
- ☐ STRONG
- ☐ PULL
- ☐ WEAK

The following strengths are mapped to WEAK:

- ☐ HIZ
- ☐ SMALL
- ☐ MEDIUM
- ☐ LARGE

The strengths are supported on primitives that can be mapped to hardware. The bus-resolution logic will incorporate strength information during the value resolution. Any conflicts that should result in a value of `x` in simulation will be mapped to `0` in hardware.

Handling of UDPs by IXCOM

Specify the `{+mapudp[<filename>]}` option of the `ixcom` command to direct IXCOM to map all the UDPs in the design to modules. Each mapped module has the same name and functionality as that of the corresponding UDP. If IXCOM fails to map a UDP to a corresponding module, a `MAPUDP_FAILED` warning message is issued. Compilation continues and the UDP (which IXCOM failed to map to a module) continues to be treated as a UDP.

If the `<filename>` optional parameter is specified, the original UDP and the module it is mapped to, are both output in the `<filename>`. This helps debug and ensure that the mapped module has the same functionality as the original UDP.

Instantiations of UDPs are not allowed in the DUT or hardware partition, and if found, IXCOM, by default, maps those UDPs to modules, even if the `+mapudp` switch was not specified with the `ixcom` command. If IXCOM fails to map a particular UDP to a module, the `MAPUDP_FAILED` warning is issued, followed by the `DUT_UDPCHILD` error message.

Support for Verilog Configuration Blocks

IXCOM supports Verilog configuration blocks to facilitate sharing of designs.

Verilog configuration blocks are a set of rules that specifies the exact source description to bind to each instance in a design. The following configuration blocks are supported by IXCOM:

- To design the global parameters of the configuration:
 - The `design` statement
 - The `default` clause
- To specify an instance or a module:
 - The `instance` clause
 - The `cell` clause
- To specify what to use for an instance or a module:
 - The `liblist` clause
 - The `use` clause
- To specify hierarchical configurations use the `config` tag.

Note: IXCOM reports an error when Verilog configuration blocks are used for binding VHDL design units.

Following is the syntax of supported Verilog configuration blocks:

```
config <config_name>;
    design [<library>.<cell>
    default liblist [{<libraries>}];
    cell [<library>.<cell> liblist [{<libraries>}];
    cell [<library>.<cell> use [<library>.<cell>[:config];
    instance <full_hierarchical_path> liblist [{<libraries>}];
    instance <full_hierarchical_path> use [<library>.<cell>[:config];
endconfig
```

You can use a library map file to specify libraries. The library map file is used to specify which file goes to which library. The files are analyzed and then the top-level configuration is used to elaborate the design. You can use the format specifier %l or %L to display the actual library binding information for module instances during simulation.

Examples

Following is an example of Verilog configuration blocks:

```
config cfg1;
    design rtlLib.top ;
    default liblist aLib rtlLib;
    instance top.a3 liblist gateLib;
    cell myaddr use rtlLib.adder1;
endconfig
```

Following is an example of hierarchical configurations:

```
config cfg1;
    design rtlLib.top ;
    default liblist aLib rtlLib;
    instance top.a3 use worklib.foo:config;
endconfig
```

In this example, the instance `top.a3` binds with the design hierarchy specified by the configuration `worklib.foo:config`.

Compiling Designs with Verilog Configuration Blocks

Perform the following steps to compile designs with Verilog configuration blocks:

1. Use the `vlan` tool to analyze the configurations and the Verilog modules into respective logical libraries.

```
vlan -work <logical_library> {<Verilog_files>}
```

2. Use IXCOM to elaborate the analyzed Verilog design, specifying the relevant `ixcom` command options. With the `-top` option, specify a previously compiled Verilog configuration, where the configuration design statement specifies the top-level module.

```
ixcom -top [<library>.]<Verilog configuration name> [other_ixcom_options]
```

Support for specify Blocks in SA Mode

The `specify` block is used to describe the module path delays and timing checks in a Verilog design. To ensure that the `specify` block is supported, use the `+specblk` option at the command line. To enable support for `specify` block in the SA mode, use the `+sv` option with the `ixcom` command. For example:

```
ixcom +sv +specblk test.v
```


The `specify` blocks are supported only for testbench modules. IXCOM ignores the `specify` blocks for modules that are mapped to the hardware.

Timing checks can be specified within the `specify` block to ensure that critical events, such as transitioning of the clock and data signals, occur with the specified time duration. Timing checks, if included in a `specify` block are used to verify the timing performance of the DUT.

The syntax of the `specify` block is:

```
specify_block ::=
    specify { specify_item } endspecify
specify_item ::=
    specparam_declaration |
    pulsestyle_declaration |
    showcanceled_declaration |
    path_declaration |
    system_timing_check
```

The `specify` block consists of the `specify..endspecify` block, within which you can provide various declarations and timing checks. The following declarations can be specified within the `specify..endspecify` block:

- Specparam
- Pathpulse Specparam
- Pulsestyle and Showcancelled
- Module Path
- System Timing Checks

The `specify` block and delayed signals can be included either in DUT or testbench modules or in both.

Using specify Blocks in a DUT Module

If a `specify` block is included in a module that has been specified as a DUT module using the `+dut+<module name>` option, this `specify` block (except `specparams`) is ignored even if the `+specblk` option is provided and an appropriate warning message is generated. If the `+specblk` option is not specified, the `specify` block construct along with `specparams` is ignored.

Following is the command that needs to be used when a `specify` block is included in the DUT:

```
ixcom +sv +specblk +dut+<mod_name> <testbench.v files>
```

If the module contains extended timing checks, continuous assignments driving the delayed signal using the original signal are created and appropriate information is generated. In the following example, the delayed signals, `dCLK` and `dip` will be driven by `CLK` and `ip`, respectively.

```
$setuphold (posedge CLK, posedge ip, -5, 10, , , , dCLK, dip);
```

Using specify Block in a Testbench Module

If a `specify` block construct is included in a testbench module and the `+specblk` option is specified, the `specify` block is processed. If the `+specblk` is not specified, the `specify` block is ignored with a warning.

If the module contains extended timing checks, and the `+specblk` option is not specified, continuous assignments are created to drive the delayed signal using the original signal. If the `+specblk` option is specified, the `specify` block is processed and no continuous assignments are created.

Deviations from Verilog 2005 LRM for specify Blocks

The `specify` block constructs are supported in the SA mode, but with certain deviations from the Verilog 2005 LRM. The following semantic checks or constraints as specified by the LRM are ignored by:

- Pulseparam declaration:
 - ☐ The error limit (if specified explicitly) should be equal to or greater than the reject limit.
 - ☐ The error limit (if specified explicitly) should be less than the delays specified in a module path.
- Module path declaration:
 - ☐ The module path destination can have only one driver (or no driver) inside the module.
 - ☐ The pulsestyle declaration (if specified) should precede the module path declaration.
- Showcancelled declaration:

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

The showcanceled declaration for a particular destination should always precede its module path declaration. If the module path appears before a showcanceled declaration, an error message is displayed.

Direct Access of DUT Memory Array and Register

IXCOM provides a built-in `marg` library to enable direct access to the following objects in the DUT:

- 2-D memory arrays
- Scalar registers
- Memories modeled using the `axis_smem` primitive

The following sections outline:

- [Using the `marg` Library](#)
- [C and Verilog Interface Functions](#)
- [SystemVerilog DPI Functions](#)
- [Specman Functions](#)

Using the `marg` Library

Following are some of the key points about the `marg` library objects:

- A `marg` object handle is created dynamically by path.
- For C-based testbenches, built-in C functions are used to create, delete, and directly access a `marg` object.
- For Verilog-based testbenches, built-in `$ixc_t` and `$ixc_f` interface functions are used.

For each C interface function, there is an equivalent Verilog task or function.

Using `marg` Interface in Pure Simulation

When using the `marg` interface in pure simulation, the `$UXEHOME/share/uxe/etc/ixcom/marg_sim.c` file must be compiled as a DPI application to the *irun* simulation.

The following are the two ways to do this:

- Provide the `marg_sim.c` file and the design files to the `irun` command.

For example:

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
irun -access \  
+rw test.sv \  
-I $UXEHOME/share/uxe/etc/ixcom \  
-cpost \  
$UXEHOME/share/uxe/etc/ixcom/marg_sim.c test.c \  
-end
```

- Create a shared library and then simulate with the shared library.

For example:

```
gcc -I $UXEHOME/share/uxe/etc/ixcom \  
-I $IESHOME/tools/include \  
-fPIC -shared -Wall \  
-o libmarg.so \  
$UXEHOME/share/uxe/etc/ixcom/marg_sim.c
```

```
irun -R \  
-sv_lib ./libmarg.so
```

Linking Both marg and SCE-MI Application in One Pure Simulation

The pure simulation library for SCE-MI is `$UXEHOME/tools.lnx86/lib/64bit/libuatba_dc.so`, which already contains the `marg_sim.c` file. If the design uses both `marg` and SCE-MI, do not include the `marg_sim.c` file. Just use the `libuatba_dc.so` file in simulation.

C and Verilog Interface Functions

This section describes the `marg` interface functions provided for use in C and Verilog testbenches. The `marg` C interface functions are located at `$UXE_HOME/etc/ixcom/marg_user.h`. In general, most `marg` functions return 0 if the function call is successful. A returned value of non-zero indicates that an error has occurred during the function call. However, the returned value 0 from all the `marg` "get handle" functions is not an error. Rather, a negative returned value from these "get handle" functions indicates an error. These "get handle" functions are `marg_new`, `marg_vmem_handle`, `marg_deposit_handle`, and `marg_value_handle`.

Tasks for Memory Array Buffer

The C and Verilog functions of the `marg` interface can be used to perform the following tasks for memory array buffer:

Creating a Memory Array Buffer

To create a memory array buffer of the given width and depth, use the following functions:

- C function:

```
int ma_handle = marg_new(int width, int depth);
```

The `marg_new` function returns a negative number if there is an error.

- Verilog function:

```
int ma_handle = $ixc_f("marg_new", width, depth);
```

Deleting a Memory Array Buffer

To delete a memory array buffer, use the following functions:

- C function:

```
void marg_delete(int ma_handle);
```

- Verilog function:

```
$ixc_t("marg_delete", ma_handle);
```

Writing a Memory Array Buffer at a Given Address

To write to a memory array buffer at the given address, use the following functions:

- C function:

```
int marg_write(int ma_handle, int addr, unsigned *value);
```

- Verilog function:

```
int errno = $ixc_f("marg_write", ma_handle, addr, value);
```

Reading a Memory Buffer at a Given Address

To read from a memory buffer at the given address, use the following functions:

- C function:

```
int marg_read(int ma_handle, int addr, unsigned *value);
```

Note: The address for `marg_write` and `marg_read` is the address of the `marg` buffer starting from 0. You have not associated this `marg` address with any DUT memory yet. A single `marg` buffer can be used to transfer data to multiple DUT memories at multiple locations.

- Verilog function:

```
int errno = $ixc_f("marg_read", ma_handle, addr, value);
```

Note: The address for `marg_write` and `marg_read` is the address of the `marg` buffer starting from 0. You have not associated this `marg` address with any DUT memory yet. In fact, one `marg` buffer can be used to transfer data to multiple DUT memories at multiple locations.

Writing a Block of Data to a Memory Buffer

To write a block of data to a memory buffer, use the following functions:

■ C function:

```
int marg_write_block(int ma_handle, int addr, int len, unsigned* value);
```

Where,

- ❑ `ma_handle` identifies the `marg` buffer used for writing or reading. The width of the `marg` buffer is already initialized when `marg_new` is used to get the `marg` buffer handle `ma_handle`.
- ❑ `len` specifies the number of `marg` words to be transferred.
- ❑ `value` for `marg_write_block` is expected to be packed to the byte boundary of a `marg` word. The second word of `value` should be packed to the next byte immediately following the first word. Likewise, the returned `value` from `marg_read_block` is packed with the same fashion.

In the following example, the first `marg` word is filled with content "0" from the first three bytes of `value`, The second `marg` word is filled with content "1" from the next three bytes of `value`.

```
unsigned char value[24];
int mah = marg_new(20, 8); // each marg word is 20 bits (3 bytes)
int i;
for(i = 0; i < 8; i++)
    value[i*3] = i;
marg_write_block(mah, 0, 8, (unsigned*)value);
```

■ Verilog function:

```
int errno = $ixc_f("marg_write_block", ma_handle, addr, len, value);
```

Reading a Block of Data from a Memory Buffer

To read a block of data from a memory buffer, use the following functions:

■ C function:

```
int marg_read_block(int ma_handle, int addr, int len, unsigned* value);
```

Note: The address for `marg_write_block` and `marg_read_block` is the address of the `marg` buffer starting from 0. You have not associated this `marg` address with any DUT memory yet. A single `marg` buffer can be used to transfer data to multiple DUT memories at multiple locations.

■ Verilog function:

```
int errno = $ixc_f("marg_read_block", ma_handle, addr, len, value);
```

Obtaining a DUT Memory Handle by Path

To obtain a DUT memory handle by path, use the following functions:

■ C function:

```
int vm_handle = marg_vmem_handle(char *vmem_path);
```

The `marg_vmem_handle` function returns a negative number if there is an error.

■ Verilog function:

```
int vm_handle = $ixc_f("marg_vmem_handle", vmem_path);
```

Getting the Width and Depth of DUT Memory

To get the width (in bits) and depth of a DUT memory, use the following functions:

■ C function:

```
int marg_vmem_size(int vm_handle, int* width, int* depth);
```

■ Verilog function:

```
int errno = $ixc_f("marg_vmem_size", vm_handle, width, depth);
```

Transferring Memory Array Buffer to DUT Memory

To transfer a memory array buffer (`ma_handle`) to a DUT memory (`vm_handle`), use the following functions:

■ C function:

```
int marg_put(int ma_handle, int ma_addr, int len, int vm_handle, int vm_offset);
```

■ Verilog function:

```
int errno = $ixc_f("marg_put", ma_handle, ma_addr, len, vm_handle, vm_offset);
```


Transferring DUT Memory to Memory Array Buffer

To transfer a DUT memory (`vm_handle`) to a memory array buffer (`ma_handle`), use the following functions:

■ **C function:**

```
int marg_get (int ma_handle, int ma_addr, int len, int vm_handle, int
vm_offset);
```

Note: For `marg_put` and `marg_get`, the starting address of the `marg` block is `ma_addr` and the destination offset of the DUT memory is `vm_offset`, which is the normalized DUT memory address. For example, if the DUT memory is `reg[31:0] mem[4:16]` or `reg[31:0] mem[16:4]`, the `vm_offset` for `mem[4]` is 0.

■ **Verilog function:**

```
int errno = $ixc_f("marg_get, ma_handle, ma_addr, len, vm_handle, vm_offset);
```

Note: For `marg_put` and `marg_get`, the starting address of the `marg` block is `ma_addr` and the destination offset of the DUT memory is `vm_offset`, which is the normalized DUT memory address. For example, if the DUT memory is `reg[31:0] mem[4:16]` or `reg[31:0] mem[16:4]`, the `vm_offset` for `mem[4]` is 0.

Loading a DUT Memory from a File

To load a DUT memory from a data file, use the following functions:

■ **C function:**

```
extern int marg_load_vmem(int vm_handle, const char* file, int offset, int len,
const char* format, int flags);
```

■ **Verilog function:**

```
$ixc_t("marg_load_vmem", vm_handle, file, offset, len, format, compress);
```

Dumping a DUT Memory to a File

To dump a DUT memory to a data file, use the following functions:

■ **C function:**

```
extern int marg_dump_vmem(int vm_handle, const char* file, int offset, int len,
const char* format, int flags);
```

Note: For `marg_load_vmem` and `marg_dump_vmem`, the offset is the normalized starting address of the DUT memory. The file format can be `%readmemh`, `%readmemb`, `%h`, `%b`, `%o`, `%pd_raw`, and `%pd_raw2`, `%pd_b`, `%pd_o`, `%pd_d`, `%pd_h`, or `%pd_memtran`. The `flags` argument can be 0 or OR'ed (|) of `MARG_Z`, `MARG_SKIPZERO`, `MARG_APPEND`, or `MARG_DONTPRINTOFFSET`:

- MARG_Z: generate compressed PD raw or raw2 format.
- MARG_SKIPZERO: skip all-zero words in PD ASCII format.
- MARG_APPEND: append to the dump file in PD raw2 format.
- MARG_DONTPRINTOFFSET: don't print the @ offset line in readmem format

For example, to append to an existing compressed pd_raw2 file named my_file:

```
marg_dump_vmem(id, "my_file", 0, 32, "%pd_raw2", MARG_Z | MARG_APPEND);
```

■ Verilog function:

```
$ixc_t("marg_dump_vmem", vm_handle, file, offset, len, format, compress);
```

Note: For `marg_load_vmem` and `marg_dump_vmem`, the offset is the normalized starting address of the DUT memory. The file format can be `%readmemh`, `%readmemb`, `%h`, `%b`, `%o`, `%pd_raw`, and `%pd_raw2`, `%pd_b`, `%pd_o`, `%pd_d`, `%pd_h`, or `%pd_memtran`. The flags argument can be 0 or OR'ed (|) of MARG_Z, MARG_SKIPZERO, MARG_APPEND, or MARG_DONTPRINTOFFSET:

- MARG_Z: generate compressed PD raw or raw2 format.
- MARG_SKIPZERO: skip all-zero words in PD ASCII format.
- MARG_APPEND: append to the dump file in PD raw2 format.
- MARG_DONTPRINTOFFSET: don't print the @ offset line in readmem format

Setting the Same Value to a DUT Memory

To set the same value to a DUT memory, use the following functions:

■ C function:

```
int marg_set_vmem(int vm_handle, int offset, int len, unsigned* value);
```

■ Verilog function:

```
$ixc_t("marg_set_vmem", vm_handle, offset, len, value);
```

Tasks for DUT Register Access

The C and Verilog functions of the `marg` interface can be used to perform the following tasks for DUT register access:

Obtaining a Handle of an Exported Register for Deposit

To obtain a handle of an exported register for deposit, use the following functions:

- C function:

```
int marg_deposit_handle(const char* vsig_path);
```

Note: `vsig_path` must be exported by `$export_deposit` in IXCOM. The `marg_deposit_handle` function returns a negative number if there is an error.

- Verilog function:

```
int dp_handle = $ixc_f("marg_deposit_handle", vsig_path);
```

Note: `vsig_path` must be exported by `$export_deposit` in IXCOM. `marg_deposit_handle` returns a negative number if there is an error.

Depositing a Value to an Exported Register

To deposit a value to an exported register, use the following functions:

- C function:

```
int marg_deposit(int dp_handle, unsigned* value);
```

- Verilog function:

```
int errno = $ixc_f("marg_deposit", dp_handle, value);
```

Obtaining a Handle of an Exported Signal for Read

To obtain a handle of an exported signal for read, use the following functions:

- C function:

```
int marg_value_handle(const char* vsig_path);
```

Note: `vsig_path` must be exported by `$export_read` in IXCOM. The `marg_value_handle` function returns a negative number if there is an error.

- Verilog function:

```
int handle = $ixc_f("marg_value_handle", vsig_path);
```

Note: `vsig_path` must be exported by `$export_read` in IXCOM. `marg_value_handle` returns a negative number if there is an error.

Getting the Value of an Exported Signal for Read

To get the value of an exported signal for read, use the following functions:

■ **C function:**

```
int marg_value(int handle, unsigned* value);
```

Note: The marg functions `marg_new`, `marg_vmem_handle`, and `marg_deposit_handle` return an integer. A handle of zero is a legal handle. If there is an error during the function call, a negative number is returned.

■ **Verilog function:**

```
int errno = $ixc_f("marg_value", handle, value);
```

Note: The marg functions `marg_new`, `marg_vmem_handle` and `marg_deposit_handle` return an integer. A handle of zero is a legal handle. If there is an error during the function call, a negative number is returned.

Examples: Passing Memory Data from Testbench to DUT

Sample code for the DPI-exported task in DUT to which the testbench needs to pass the memory data:

```
module DUT
    reg [31:0] lm[511:0];
    task dpi_export_task;
        begin
            <
                Use lm[addr] to process TB data;
            >
        end
    endtask
endmodule
```

Sample code for passing memory data from a C-based testbench to the DPI-exported task in DUT (DUT.lm):

```
#include "marg_user.h"
int ma = -1;
int vm = 0;
void c_task(){
    int i;
    if(ma < 0){
        ma = marg_new(32, 512);
        vm = marg_vmem_handle("DUT.lm");
    }
    for(i=0; i< 512; i++){
        < generate some value and fill the marg>
    }
}
```

```
        marg_write(ma, i, value);
    }
    marg_put(ma, 0, 512, vm, 0);
    dpi_export_task(); //call dpi_exported task;
    marg_get(ma, 0, 512, vm, 0);
}
```

Sample code for passing memory data from a Verilog testbench to the DPI-exported task in DUT (DUT.lm):

```
module TB;
    integer ma, vm;
    Initial begin
        ma = $ixc_f("marg_new", 32, 512);
        vm = $ixc_f("marg_vmem_handle", DUT.lm);
    end
    task v_task;
        begin
            for(i=0; i< 512; i++)
                begin
                    < generate some values and fill the marg>
                    $ixc_t("marg_write", ma, i, value);
                end
            $ixc_t("marg_put", ma, 0, 512, vm, 0);
            DUT.dpi_export_task();
            $ixc_t("marg_get", ma, 0, 512, vm, 0);
        end
    endtask
endmodule
```

SystemVerilog DPI Functions

This section describes the `marg` interface functions provided for use in SystemVerilog testbenches. The `marg` SV DPI functions are located at `$AXIS_HOME/etc/ixcom/marg_user.svh`.

The value parameter for the `marg_deposit`, `marg_value`, `marg_write`, `marg_read`, and `marg_set_vmem` functions cannot be an open array (as required by SystemVerilog DPI). So, you must generate these import functions by ``MARG_IMPORT` before using them. You can generate these import functions by ``MARG_IMPORT(N)`, where `N` can be a Verilog parameter or a constant integer. Then, you can call these functions by appending `"_N"` to the function name.

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

For example:

```
parameter vecw = 64;
parameter memw = 32;
`MARG_IMPORT(vecw)
`MARG_IMPORT(memw)
bit [vecw-1: 0] dp_value;
bit [memw-1: 0] ma_value;
if( marg_deposit_vecw(dp_id, dp_value)!=0 ) $display("failed in
marg_deposit");
if( marg_read_memw(ma_id, offset, ma_value)!=0 ) $display("failed in
marg_read");
void'( marg_write_memw(ma_id, offset, ma_value) );
```

Following is a description of these functions:

- To create a `marg` object of the given width and depth, use:

```
int ma_handle = marg_new(int width, int depth);
```

The `marg_new` returns a negative number if there is an error.

- To delete a `marg` object, use:

```
void marg_delete(int ma_handle);
```

- To obtain a Verilog memory handle by path, use:

```
int vm_handle = marg_vmem_handle(string vmem_path);
```

The `marg_vmem_handle` returns a negative number if there is an error.

- To get the width (in bits) and depth of a Verilog memory, use:

```
int marg_vmem_size(int vm_handle, output int width, output int depth);
```

- To write to a `marg` object at the given address, use:

```
int marg_write_N(int ma_handle, int addr, bit[N:1] value);
```

- To read from a `marg` object at the given address, use:

```
int marg_read_N(int ma_handle, int addr, output bit[N:1] value);
```

Note: The address for `marg_write` and `marg_read` is the address of the `marg` buffer starting from 0. This `marg` is not associated with any Verilog memory. One `marg` buffer can be used to transfer data to multiple Verilog memories at multiple locations.

- To transfer a block of data of the specified length (`len`) from a `marg` object (`ma_handle`) to a Verilog memory in DUT (`vm_handle`), use:

```
int marg_put (int ma_handle, int ma_addr, int len, int vm_handle, int
vm_offset);
```

Note: The starting address of the `marg` block is `ma_addr` and the destination offset of

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

the Verilog memory is `vm_offset`, which is the normalized Verilog memory address. For example, if the Verilog memory is `reg [31:0] mem [4:16]` or `reg [31:0] mem [16:4]`, the `vm_offset` for `mem[4]` is 0.

- To transfer a block of data of the specified length (`len`) from a Verilog memory in DUT (`vm_handle`) to a marg object (`ma_handle`), use:

```
int marg_get (int ma_handle, int ma_addr, int len, int vm_handle, int vm_offset);
```

Note: For `marg_put` and `marg_get`, the starting address of the marg block is `ma_addr` and the destination offset of the Verilog memory is `vm_offset`, which is the normalized Verilog memory address. For example, if the Verilog memory is `reg [31:0] mem [4:16]` or `reg [31:0] mem [16:4]`, the `vm_offset` for `mem[4]` is 0.

- To load a Verilog memory from a data file, use:

```
int marg_load_vmem(int vm_handle, string file, int offset, int len, string format, int compress);
```

- To dump a Verilog memory to a data file, use:

```
int marg_dump_vmem(int vm_handle, string file, int offset, int len, string format, int compress);
```

Note: For `marg_load_vmem` and `marg_dump_vmem`, the offset is the normalized starting address of the Verilog memory. The file format can be `%readmemh`, `%readmemb`, `%h`, `%b`, `%o`, `%pd_raw`, and `%pd_raw2`. The `compress` argument is 0 or 1 which you can use to specify the compression used in the raw data file.

- To uniformly set the same data to a Verilog memory, use:

```
int marg_set_vmem_N(int vm_handle, int offset, int len, bit[N:1] value);
```

- To obtain a handle for depositing an exported hardware signal, use:

```
int marg_deposit_handle(string vsig_path);
```

Note: The `vsig_path` must be exported by `$export_deposit` in IXCOM. The `marg_deposit_handle` returns a negative number if there is an error.

- To deposit a value to a deposit handle, use:

```
int marg_deposit_N(int dp_handle, bit[N:1] value);
```

- To obtain a handle for getting an exported hardware signal value, use:

```
int marg_value_handle(string vsig_path);
```

Note: The `vsig_path` must be exported by `$export_read` in IXCOM. The `marg_value_handle` returns a negative number if there is an error.

- To get a value of a value handle, use:

```
int marg_value_N(int handle, output bit[N:1] value);
```

Note: The `marg` functions `marg_new`, `marg_vmem_handle`, and `marg_deposit_handle` return an integer. A handle of zero is a legal handle. If there is an error during the function call, a negative number is returned. All other `marg` functions return 0 if the function call is successful. A returned value of non-zero indicates that an error has happened during the function call.

Specman Functions

The `marg` Specman interface functions are located at `$UXE_HOME/etc/ixcom/marg_user.e`.

`marg_t`

`marg_t` is an *e* macro that defines a parameterized struct. You must define an *e* type by specifying the width of the `marg` object you are going to use. For example, to define a type for 64-bits width `marg` objects, use:

```
type marg64_t: marg_t of 64 bits;
```

You can then declare a `marg` object variable using that type, for example,

```
margW: marg64_t;
```

The following methods can be used on a `marg` object:

- To create a `marg` object of the specified depth and bind this `marg` object to a Verilog memory:

```
new(depth: int, hdl_path_name: string);
```

- To delete a `marg` object:

```
delete();
```

- To write data from `uint` (of 64-bits, in this example) to the `marg` object at the specified address:

```
write_uint(addr: int, data: uint (bits: <width'exp>));
```

- To read data in `uint` (of 64-bits, in this example) from the `marg` object at the specified address:

```
read_uint(addr: int, data:* uint (bits: <width'exp>));
```

- To write data in list of bit to the `marg` object at the specified address:

```
write_lob(addr: int, data: list of bit);
```


For performance reasons, this method does not resize the data to align to the `marg` object width.

- To read and return data in list of bit (64 bits, in this example) from the `marg` object at the specified address:

```
read_lob(addr: int): list of bit;
```

- To transfer a block of data from the `marg` object to its associated Verilog memory:

```
put(addr: int, len: int);
```

- To transfer a block of data from the associated Verilog memory to the `marg` object:

```
get(addr: int, len: int);
```

marg_deposit_t

`marg_deposit_t` is an *e* macro that defines a parameterized struct. First, define an *e* type by specifying the width of the `marg_deposit` object you are going to use. For example, to define a type for 64-bits width `marg_deposit` objects, use:

```
type marg64_deposit_t: marg_deposit_t of 64 bits;
```

You can then declare a `marg_deposit` variable.

For example:

```
data_in: marg64_deposit_t;
```

The following methods can be used on a `marg_deposit_t` object:

- To bind the `marg_deposit` object to a Verilog variable:

```
bind(hdl_path_name: string);
```

Note: The width of the `marg_deposit` object must be greater than or equal to the width of the Verilog variable.

- To write data in `uint` (of 64-bits, in this example) to the associated Verilog variable:

```
write_uint(data: uint (bits: <width'exp>));
```

- To write data in list of bit to the associated Verilog variable:

```
write_lob(data: list of bit);
```

Note: The number of bits in the data must be greater than or equal to the width of the Verilog variable.

marg_value_t

`marg_value_t` is an *e* macro that defines a parameterized struct. First, define an *e* type by providing the width of the `marg_value` object you are going to use. For example, to define a type for 64-bits width `marg_value` objects, use:

```
type marg64_value_t: marg_value_t of 64 bits;
```

You can then declare a `marg_value` variable.

For example:

```
data_out: marg64_value_t;
```

The following methods can be used on a `marg_value` object:

- To bind the `marg_value` object to a Verilog variable:

```
bind(hdl_path_name: string);
```

Note: The width of the `marg_value` object must be greater than or equal to the width of the Verilog variable.

- To read data in `uint` (of 64-bits, in this example) from the associated Verilog variable:

```
read_uint(data:* uint (bits: <width'exp>));
```

- To read and return data in list of bit (64-bits, in this example) from the associated Verilog variable:

```
read_lob(): list of bit;
```

Enabling and Disabling Features at Compilation Time

When compiling the design with the IXCOM compiler, you can specify the various features to be enabled or disabled for a particular instance hierarchy, design unit, or HDL file names by using the `-featureFile <feature_file>` option of the `ixcom` command.

Using this option, you can enable or disable:

- Instance hierarchy - for example, `top.F001`, and optionally, the entire hierarchy under it.
- Module hierarchy - for example, module `boo`, and optionally, all the modules instantiated by it.
- HDL file - for example, all the modules in the `file2.v` file.

 Important

The `-featureFile <feature_file>` option can be used only with the `ixcom` command. It does not work with the `vlan` or `vhan` command. This option is not the same as the `-covfile` option of the `irun` command.

To enable assertions for all instances of the `foo` module in the example illustrated in the [Examples of <feature_spec_lines>](#) section, you can specify the `+sva+foo` option with the `ixcom` command. However, the `+sva` option does not allow you to enable assertions only for a specific instance hierarchy, such as `top.F001`. In such situations, you can add the following line in a `<feature_spec_file>`, and specify the name of this file with the `-featureFile <feature_spec_file>` option of the `ixcom` command:

```
sva disable
sva enable instance=top.F001+0
```

Syntax of the <feature_file> File

In the `<feature_file>` file, you need to list the set of features that you want to enable or disable, along with the configuration options in the form of `<feature_spec_lines>`. Each line identifies the `<feature>`, the corresponding `<control_option>`, and the configuration to which this control is applied based on the `<configuration_option_opt>`.

Following is the syntax of the `<feature_spec_line>`:

```
<feature> <control_option> <configuration_option_opt>
```

See the following sections to know the list of valid values for each of the above-mentioned parameters:

- [Specifying the <feature> Parameter](#)
- [Specifying the <control_option> Parameter](#)
- [Specifying the <configuration_option_opt> Parameter](#)

Depending on your requirements, you can specify the features to be enabled for the following types of configurations:

- [Instance-Based Configuration](#)
- [Design-Unit-Based Configuration](#)
- [File-Based Configuration](#)

Specifying the <feature> Parameter

Following are the valid values for the <feature> parameter:

<feature>	Denotes
sva	The SystemVerilog Assertions feature.
psl	The PSL Assertions feature.
assert	Both the PSL and SystemVerilog Assertions features. Note that <code>assert</code> implies both the <code>sva</code> and <code>psl</code> features. So, for example, you need not use multiple lines as follows: <pre>sva <control_option> <configuration_option_opt> psl <same_control_option_as_above> \ <same_configuration_option_opt_as_above></pre> Instead, you can specify a single line as follows: <pre>assert <control_option> <configuration_option_opt></pre>
coverage_functional	The SystemVerilog covergroups feature.
coverage_block	The Block coverage feature.
coverage_toggle	The Toggle coverage feature.

Any values other than those listed above are considered invalid and result in compilation error.

Following is a sample <feature_spec_line> that specifies that the Block Coverage feature should be enabled in the `top.DUT` instance hierarchy:

```
coverage_block enable instance=top.DUT
```

Note: You can include empty lines (lines with only whitespaces) and comments (lines beginning with # sign) in the <feature_file> file. Leading and trailing whitespaces are allowed, and ignored.

To continue a <feature_spec_line> on the next line add a \ <backslash> character immediately before the <newline> character on the current line.

Alternatively, you can use specific `ixcom` command options to specify the features, such as assertions, coverage, etc to be enabled or disabled in a particular module or analyzer session.

Specifying the `<control_option>` Parameter

Following are the valid values for the `<control_option>` parameter:

<code><control_option></code>	Denotes
<code>enable</code>	The feature should be enabled.
<code>disable</code>	The feature should be disabled.

The above values can be specified for all features. An invalid value for the `<control_option>` parameter results in compile error.

Specifying the `<configuration_option_opt>` Parameter

The `<configuration_option_opt>` parameter is optional. When it is not specified, the corresponding `<control_option>` is applied to the corresponding `<feature>` for the entire design (that is, globally).

Following are the valid values for the `<configuration_option_opt>` parameter:

<code><configuration_option_opt></code>	Denotes
<code>instance=<instance_spec>{,<instance_spec>}</code> <code><instance_spec> ::=</code> <code><hierarchical_instance_path>[+<depth>]</code>	Instance-based configuration
<code>du=<design_unit_spec>{,<design_unit_spec>}</code> <code><design_unit_spec> ::=</code> <code><design_unit_name>[+<depth>]</code>	Design-unit-based configuration
<code>file= <filename>{,<filename>}</code>	File-based configuration
<code><nothing></code>	If no configuration option is specified, the <code><control_option></code> parameter for the specified feature is applied to the entire design.

An invalid value for the `<configuration_option_opt>` parameter results in compilation error.

Instance-Based Configuration

Instance-based configuration is defined as a cone of hierarchy starting at the instance specified as the `<hierarchical_instance_path>` having the depth as specified by the optional `<depth>` parameter.

Specify an integer value greater than or equal to zero as the `<depth>` parameter.

The interpretation for the value of the `<depth>` parameter is as shown below:

<depth>	Indicates that the feature will be enabled for
0	The instance at the beginning of the <code><hierarchical_instance_path></code> and the entire hierarchy under it.
1 (default)	The instance at the beginning of the <code><hierarchical_instance_path></code> .
2	The instance at <code><hierarchical_instance_path></code> and one level of hierarchy under it.
<code><N></code> (a positive integer <i>N</i> greater than 2)	The instance at <code><hierarchical_instance_path></code> and (<i>N</i> -1) levels of hierarchy under it.
<code><any_other_values></code>	Compilation error

Note:

- ❑ If the specified `<hierarchical_instance_path>` cannot be resolved in the design hierarchy, or an invalid value is specified for the `<depth>` parameter, a compilation error is generated.
- ❑ Use of wildcards and environment variables for specifying the `<hierarchical_instance_path>` parameter is not supported.
- ❑ If the optional `<depth>` parameter is not specified, the `<depth>` is assumed to be 1.

Design-Unit-Based Configuration

The design-unit-based configuration is defined as a cone of hierarchy starting at each of the instances of the specified `<design_unit>` having the depth specified by the optional `<depth>` parameter.

The `<depth>` parameter takes an integer value that is greater than or equal to zero.

The interpretation for the value of the `<depth>` parameter is as shown below:

<depth>	Indicates that the feature will be enabled for
0	The instance at the beginning of the <code><design_unit></code> and the entire hierarchy under each instance.
1 (default)	All the instances of the <code><design_unit></code> .
2	All the instances of the <code><design_unit></code> and one level of hierarchy under each of the instances.
<code><N></code> (a positive integer <i>N</i> greater than 2)	All the instances of the <code><design_unit></code> and (<i>N</i> -1) levels of hierarchy under each of the instances.
<code><any_other_values></code>	Compilation error

Note:

- ❑ If any of the instances of the specified `<design_unit>` cannot be found in the design hierarchy, or an invalid value is specified for the `<depth>` parameter, a compilation error is generated.
- ❑ Use of wildcards and environment variables for specifying the `<design_unit>` parameter is not supported.
- ❑ If the optional `<depth>` parameter is not specified, the `<depth>` is assumed to be 1.

File-Based Configuration

The file-based configuration is defined as all of the design units defined in the specified file. The hierarchy under those instantiations is not included in this configuration. Unlike the instance and module based configurations, the *<depth>* parameter is not specified with file-based configuration.

Following is the syntax of file-based configuration:

```
<feature> enable|disable file=<filename>
```

The *<filename>* parameter must at least be a base filename, which means that it should at least contain a filename and extension(s), such as `test.v`, `flop.v`, `adder.sv`. In this case, IXCOM will look for the file only in the current compilation directory.

You can provide an absolute or relative (relative to the current working directory) path while specifying the *<filename>*. There should be no ambiguity between files with the same name in different directories or libraries.

You can also use a symbolic link to a file or an environment variable while specifying the file-based configuration.

For example:

```
file=/home/test_path1/test.v
file=../test.v
file=$PWD/test.v
file=$DESIGNFILE1
```

Note:

- Files included by using the `'include` directive in the *<filename>* file are not considered a part of the corresponding configuration.
- Use of wildcards is not supported in *<filename>*.
- If a module begins in one file and ends in another file, the enabling and disabling of such modules by using file-based configuration is defined by the file in which the module begins.
- Packages are not considered a part of the corresponding configuration. However, features in a package are disabled or enabled if any module refers to it. For example:

Assume that a package named `pack1` is defined in the file named `test.v`. Then, the following configuration does not apply to `pack1`:

```
sva disable file=test.v
```


If the `pack1` package is defined in `test.v`, and referred to by the `dut` module in the `dut.v` file, the following configuration applies to the `dut` module, and to all the references to `pack1` package from the `dut` module. Note that the feature is still enabled for any calls to `pack1` from modules in `test.v` file.

```
sva disable file=dut.v
```

- Compilation unit scopes are treated like packages as defined above.
- The file-based configuration is the equivalent of specifying design-unit-based configuration for each of the design units specified in the particular file.

This means that:

```
sva enable file=test.v
```

Is equivalent to the following line for each of the design units defined in the `test.v` file:

```
sva enable du=<design_unit_name>
```

Precedence Rules

You can specify multiple `<feature_file>` files by using this option multiple times, and each corresponding `<feature_file>` file for each option is processed in the order in which the option appears with the `ixcom` command. If the same `<feature_file>` file is specified multiple times, an informational message is issued that the previous specifications of this `<feature_file>` file are redundant and will have no effect.

Note: If required, you can improve your compile script by removing the previous specifications of this `<feature_file>` file.

The `<feature_files>` are processed or applied in the order in which they are specified on the command line and with a given `<feature_file>`, the `<feature_spec_lines>` are processed or applied in the order in which they appear. As a result of this, if two or more `<feature_spec_lines>` specify conflicting `<control_option>` for a given instance in the design hierarchy, the last specified `<feature_spec_line>` will be the one that takes precedence.

For example, consider the following command:

```
ixcom -assert top.sv block.sv chip1.sv chip2.sv -featureFile ff1 -featureFile ff2
```

In this example, the `ff1` file has the following `<feature_spec_lines>`:

```
sva enable
instance=top.block+0
sva disable file=chip1.sv
```

The `ff2` file has the following `<feature_spec_lines>`:

```
sva enable du=chip1_tester
```

In this example, the first line in the `ff1` file enables all the assertions in the `top.block` instance of the `block` design unit and all the instances within this hierarchy at all levels. The assertions in design units named `chip1` and `chip2` that are instantiated in `block` are also enabled.

However, the second line in `ff1` disables all the assertions that are present in modules within `chip1.sv`. Therefore, even though all the assertions in `chip1` should be enabled as per line 1, the assertions in `chip1` are disabled because of line 2.

Next, the `ff2` feature file is considered. It contains a line that enables assertions in the design unit called `chip1_tester`. This design unit might be in the `chip1.sv` file. So, now, even though the assertions in file `chip1.sv` have been disabled because of line 2 of the `ff1` file, the assertions in the `chip1_tester` design unit in the `chip1.sv` file are now enabled.

Refer to [“Enabling the Block and Toggle Coverage Features for Instance Hierarchies”](#) on page 456 and [“Enabling and Disabling Assertions at Compile Time”](#) on page 506 for more details about how existing options for enabling the coverage and assertion features behave when the `-featureFile <feature_file>` option is specified.

Examples of <feature_spec_lines>

Consider the following example:

```
// file1.v                //----- file2.v -----                // file3.v
module top;                module foo;                module boo;                module coo;
    foo F001();              boo B001();              cooC001();              endmodule
    foo F002();              coo C001();              endmodule
endmodule                  endmodule
```

Some sample `<feature_spec_lines>` and their interpretation for the above example are listed in the following sections:

- [No Operation Example](#)
- [Example with No <configuration_option_opt> Parameter](#)
- [Examples with Instance-Based, Design-Unit-Based, and File-Based Configurations](#)

No Operation Example

In the above example, the `sva` and `coverage_toggle` features are assumed to be enabled, by default.

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

If a feature is enabled by default, adding `<feature_spec_lines>` to enable the same feature in some design units, instances, files is regarded as a No Operation and the IXCOM compiler issues a corresponding informational message.

The following `<feature_spec_line>` specifies that the Toggle Coverage feature is enabled, but this is already enabled, by default, in the example. So, if this line is not preceded by a `<feature_spec_line>` to disable the `coverage_toggle` feature, this line is regarded as a No Operation and the IXCOM compiler generates a corresponding INFO message.

```
coverage_toggle
```

Similarly, the SystemVerilog Assertions feature was already enabled, by default, in the example. Without a preceding `<feature_spec_line>` to disable the `sva` feature, the following line results in a No Operation and the IXCOM compiler generates a corresponding INFO message.

```
sva enable
```

Example with No `<configuration_option_opt>` Parameter

The following `<feature_spec_line>` disables the Toggle Coverage feature in the entire design:

```
coverage_toggle disable
```

The following `<feature_spec_line>` disables the SystemVerilog assertions feature in the entire design:

```
sva disable
```

Examples with Instance-Based, Design-Unit-Based, and File-Based Configurations

The following <code><feature_spec_lines></code> enable Toggle Coverage	For the following instances
Example: Instance-based Configuration	
<code>coverage_toggle disable</code> <code>coverage_toggle enable instance=top.F001+0</code>	<code>top.F001</code> <code>top.F001.B001</code> <code>top.F001.B001.C001</code> <code>top.F001.C001</code>
<code>coverage_toggle disable</code> <code>coverage_toggle enable instance=top.F001+0</code> <code>coverage_toggle disable instance=top.F001.B001</code>	<code>top.F001</code> <code>top.F001.B001.C001</code> <code>top.F001.C001</code>

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

The following <i><feature_spec_lines></i> enable Toggle Coverage	For the following instances
Example: Design-Unit-Based Configuration	
coverage_toggle disable coverage_toggle enable du=boo+0	top.F001.B001 top.F001.B001.C001 top.F002.B001 top.F002.B001.C001
coverage_toggle disable coverage_toggle enable du=boo	top.F001.B001 top.F002.B001
coverage_toggle disable coverage_toggle enable du=coo	top.F001.C001 top.F001.B001.C001 top.F002.C001 top.F002.B001.C001
coverage_toggle disable coverage_toggle enable du=foo+0 coverage_toggle disable du=coo+0	top.F001 top.F001.B001 top.F002 top.F002.B001
coverage_toggle disable coverage_toggle disable du=coo+0 coverage_toggle enable du=foo+0	top.F001 top.F001.B001 top.F001.B001.C001 top.F001.C001 top.F002 top.F002.B001 top.F002.B001.C001 top.F002.C001
Example: File-Based Configuration	
coverage_toggle disable coverage_toggle enable file=file2.v	top.F001 top.F001.B001 top.F002 top.F002.B001

Using Conformal to Verify the IXCOM-Compiler-Generated Output

Encounter Conformal (commonly called LEC) is the Cadence Equivalence Checker tool that is used by to validate the pre- and post-synthesis netlists. When compiling the designs for the

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

Palladium XP emulator, you can choose to use the LEC tool to validate the two netlists either while compiling the design with the IXCOM compiler, or at the stage of HDL compilation. The use model for comparing the netlists at the HDL Compiler stage is described in the section titled Using Conformal to Verify HDL Compiler-Generated Netlist.

The current section, on the other hand, describes the same functionality that is offered at the IXCOM compilation stage. This use model replaces the need for you to validate the HDL-Compiler synthesized output against the pre-synthesis netlist. Still, the two use models are very similar to each other.

Following are the steps to be taken for validating the netlist generated by the IXCOM compiler with the Conformal netlist:

Generate the LEC script from the IXCOM compiler by using the `-conformal` option of the `ixcom` command. This script closely matches the script that is generated by HDL Compiler. This script includes the LEC commands customized for the user design and is used to invoke LEC after the HDL Compiler has finished synthesizing the design.

The IXCOM compiler automatically includes most of the necessary LEC-specific options in the LEC script that it generates. If required, you can also customize this generated script according to your requirements.

Invoke the LEC tool by executing the following command:

```
make -f xc_work/lec/lec.mak
```

Note: To use Conformal, you must purchase its licenses from Cadence. Refer to the *Conformal Equivalence Checking User Guide* for installation and platform support of Conformal software.

The `lec.mak` script generated by the IXCOM compiler invokes the Conformal Ultra tool to ensure better performance. Therefore, the LEC tool is invoked as follows:

```
lec -xl
```

This requires a Conformal “Ultra” license.

The IXCOM compiler creates the `xc_work/lec/lec.mak` file in addition to the following scripts in the `xc_work/lec` directory for running the LEC tool.

Filename / location	Description
Makefile	

Filename / location	Description
----------------------------	--------------------

<code>lec.mak</code>	The Makefile includes the target to run the LEC tool by using the generated script and the (current) path that you have defined.
----------------------	--

LEC Script and Related Files

<code>lec.scr</code>	The primary script for running the LEC tool that creates the log files, and other secondary script files after the LEC tool is run.
----------------------	---

Files Generated by the LEC Tool

After the LEC tool has been invoked by the IXCOM-generated makefile as mentioned above, the LEC tool generates the following files in the same directory as the main script - `xc_work/lec`:

- ☐ Log file: `lec.scr.log`
- ☐ Intermediate `do` file
- ☐ File of black box definitions

These files are not removed after Conformal finishes. If required, you can use these for analysis and debugging.

HDL-Compiler Options Necessary for Invoking the LEC Tool from the IXCOM Compiler

When you use the IXCOM compiler to invoke the LEC tool, at the HDL compilation stage, it internally switches on the following options for the `hdlSynthesize` command and `vaelab` utility:

- `-keepAllFlipFlop`: This option ensures that certain registers are not optimized during synthesis. If this option is not enabled, the LEC tool might not be able to map all the key points, and it might, subsequently, report false non-equivalence. Note that this option, however, slightly increases the gate count of the HDL output netlist.
- `-keepRtlSymbol`: This option ensures that Conformal-compatible names are generated when the output Verilog netlist is generated. This option is needed to improve the Conformal-mapping process and accuracy of comparison.

By default, both of these options are included in the makefile generated by the IXCOM compiler. However, if you have disabled these options in a compilation script, you would need to create a separate target for the LEC tool with these options enabled.

Customizing the LEC Tool Invocation

For better performance, and as per your requirements, you can customize the scripts generated by the IXCOM compiler for invoking and running the LEC tool in the following ways.

Excluding Modules from Equivalence Comparison

To avoid long verification times, you can choose to exclude certain modules from equivalence comparison by specifying the module names in a text file named `lec_script.moduleSkipEC`. Each module name should be on a separate line. The Conformal script created by IXCOM will read this file, and apply it to the modules to be excluded. Place this script in the `xc_work/lec` directory.

Customizing LEC Script File Using a Configuration File

You can customize the default LEC script that is generated by the IXCOM compiler by including a set of commands in a configuration file. The configuration file should be a text file named `lec_script.config`, and it should be located in the design directory from where the LEC tool is being invoked.

In this file, you can include a set of LEC commands, with each command appearing on a separate line. The original LEC script generated by IXCOM sources this configuration file.

For example, you can use this configuration file to enable the multi-threading feature in the LEC tool. This feature is not set, by default, in the LEC script.

```
set parallel option -threads 4
```

The above command in the configuration file sets the maximum number of threads to be used to 4.

Alternatively, in the configuration file, you can specify additional attributes to certain modules as follows:

```
add module attribute <module_name> -compare_effort low
```

or

```
add module attribute <module_name> -cpu_limit 3600
```

Such attributes help in limiting the time spent in the comparison of these modules.

Classifying Memories as Black-boxes

In certain cases, the HDL Compiler infers memories from the RTL arrays and uses UXE memory primitives (MPR and MPW), which are not understood by the LEC tool. So, either the LEC tool takes a long time for large memories to finish, or reports that the modules are non-equivalent in many instances.

The IXCOM compiler attempts to infer the target map for the memories in the design, and back-annotates the memory information after reading the HDL-compiler-generated files. Correspondingly, it excludes all those modules that infer UXE memories by writing the required black-boxing commands in the script that it generates for the LEC comparison.

Importing External Netlists into IXCOM Compiler

In the conventional Simulation Acceleration flow, the IXCOM compiler performs design analysis, elaboration, synthesis checks to map portions of the design that can be accelerated in emulator hardware. For synthesizable netlist files, these checks and processes are unnecessary, and the netlist can be directly presented to the hardware-mapping process of the compiler.

If the source files contain netlists either generated by HDL Compiler or produced by third-party tools, the above-mentioned compile process is very expensive - both in terms of memory consumption and time to compile the design. Also, such compile process is not productive for netlists because unlike the behavioral and RTL modules that require analysis, elaboration and synthesis, the gate-level netlists are pre-synthesized and can be passed to the back-end compiler as-is.

This section describes the process for importing external netlists in the SA mode using the IXCOM compiler. This use model allows you to bypass the conventional Simulation Acceleration compilation flow. An alternative mechanism to import netlists as blackboxes into the IXCOM compiler is described in [Appendix P, “Importing Netlists as Blackboxes in SA Mode.”](#) However, this mechanism is not recommended, and is likely to be obsolete in future release.

This mechanism provides support for:

- ❑ Instantiation of external netlist modules in RTL hierarchy
- ❑ Instantiation of RTL modules in external netlist hierarchy
- ❑ Hot-swap with external netlists
- ❑ Enhancements to address the following limitations in the alternative use model for importing netlists as black-boxes (See [Importing Netlists as Blackboxes in SA Mode](#)):
 - Certain options like Q_ASSIGN and bus resolution options used for netlists do not comply with the IEEE standard semantics.
 - The external nets need to be specified using files in the compilation directory.
 - External netlist object reference and binding specifications require usage of `$ixc_ctrl` statements in the shell module.
 - Additional hierarchy is introduced for the netlist hierarchy due to shell module for the netlist module. This limitation requires all hierarchical references to netlist objects to be modified in the compile and run-time scripts for including the IXCOM-generated hierarchy.

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

The IXCOM compiler does not analyze (or import) external netlists. So, it cannot infer cross-domain (RTL and external netlist) module instantiations directly from the external netlists. You need to specify all cross-domain bindings to the IXCOM compiler by using a configuration specification file.

To use this mechanism, specify the `-externalNetlists <file>` option of the `ixcom` command. The file is the configuration file in which you need to specify the following information:

- ❑ List of netlist files
- ❑ List of netlist design units to import (bind to RTL design hierarchy)
- ❑ List of RTL design units to export (bind to netlist design hierarchy)
- ❑ List of references to external netlist objects
- ❑ Hot-swap for the netlist hierarchy
- ❑ Netlist module definition to be kept if the definition is present in both the netlist and RTL domains

Use the following text format and syntax in the configuration file to specify the above-listed information:

```
// NET-LIST Configuration specification syntax
NL_IMPORT <design_unit> [: <hierarchical path DU instatnce>[,...]]
NL_EXPORT <design_unit> [: <hierarchical path to DU instance>[,...]]
NL_FILE <netlist_file(s)> // Verilog netlist files
NL_BIND <SV bind_file(s)> // SV bind directive files
NL_HOTSWAP on | off // default: off
NL_DEF_KEEP ixcom | external // default: external
NL_REF <wire_or_reg> [ <packed_dimension> ] \
                <hierarchical_path_to_external_netlist_object> \
                [<unpacked_dimension> ]
<wire_or_reg> ::= wire | reg
<packed_dimension> ::= <dimension>
<unpacked_dimension> ::= <dimension>
<dimension> ::= [ <integer_value> ] | [ <integer_value> : <integer_value> ]
```

The following syntax and semantics rules apply for the configuration specification file:

- The following words are recognized as keywords:

NL_IMPORT, NL_EXPORT, NL_FILE, NL_BIND, NL_REF, NL_HOTSWAP, NL_DEF_KEEP

Note: This document uses uppercase names for keywords in the configuration

specification file. However, these are case-insensitive keywords.

- A configuration file must have at least one `NL_IMPORT` specification item. All other specification items are optional depending on the use model requirements for the design.
- `NL_IMPORT` and `NL_EXPORT` specification can have optional information about the hierarchical path to the `<design_unit>` instance.

Important

You cannot compile both the netlist and RTL module with the same name into two different hierarchies.

- All configuration items can be specified multiple times.
- With the `NL_FILE` keyword, you can specify absolute or relative paths of the files. You can also use environment variables, but only with the file notation as `-f <file>`.
- The `NL_HOTSWAP` keyword retains the last value if it has been specified multiple times. If it has not been specified, by default, it is set to `off`.
- The `NL_DEF_KEEP` keyword retains the last value if it has been specified multiple times. If it has not been specified, by default, it is set to `external`.
- You can specify line comments in the XEL-style (using `#`) and SystemVerilog style (using `//`).

The following sections describe the use-model cases for importing netlist module(s) or exporting RTL module(s) in the IXCOM compiler.

Importing Netlist Modules and Exporting RTL Modules

This section describes the steps for:

- ❑ [Importing Netlist Modules into the RTL Hierarchy](#)
- ❑ [Exporting RTL Module in the Netlist Hierarchy](#)
- ❑ [Binding RTL Modules Using Direct Instantiation](#)
- ❑ [Using Explicit Binding to Export RTL Modules](#)
- ❑ [Importing Netlists Using the XEL Scripts for ICE-Mode Compilation](#)

Importing Netlist Modules into the RTL Hierarchy

A netlist module can be imported into RTL hierarchy as:

- ☐ A top-level DUT module
- ☐ A top-level DUT module instantiated in the testbench (software partition)
- ☐ A non-top DUT module instantiated in DUT (hardware partition)

To import an external netlist module into RTL hierarchy, you must specify the following information in the configuration file:

- The netlist module(s) using the `NL_IMPORT` keyword

Note: The netlist modules that you are importing into the IXCOM compiler must not include any ATB set of constructs. Refer to the [Synthesizing Designs with Acceleratable Testbench \(ATB\) Constructs](#) section for details about ATB constructs.

- The netlist file(s) using the `NL_FILE` keyword

This specification is optional, and you need to specify it only if you need:

- ☐ Hot-swap for the netlist
- ☐ The IXCOM compiler to generate the XEL import script for use by the Palladium XP back-end compilers.

- Hot-swap and module re-definition specification

This specification is optional.

You must specify an empty shell module with the complete interface (port I/O) declaration for the imported netlist module to the IXCOM compiler in RTL. This shell module is required for IXCOM compiler to build the accurate I/O interface. The interface declarations provided in the shell module must match original interface in the netlist module, including the order of port connection and declaration.

For the RTL hierarchy, the design units can be written in any of languages (Verilog, System Verilog and VHDL) supported by the IXCOM compiler. The IXCOM compiler presents the netlist files specified using `NL_FILE` to the back-end compiler (`xeCompile`) using the `netlistFile { verilog <file> } import` command.

This section uses the following example to illustrate the netlist module binding into the RTL hierarchy.

```
// File: top.sv
module Top;
    //...
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
Dut dut(clk, din, dout);
//...
endmodule // Top

// File:dut-nl.v
module Dut(clk, din, dout)
    //...
    // netlist
    //...
endmodule // Dut

// File: dut-shell.v
module Dut(clk, din, dout);
    input clk;
    input [3:0] din;
    output [3:0] dout;
endmodule // Dut

// File: nl-config.spec
NL_FILE ./dut-nl.v
NL_IMPORT Dut
```

To import `Dut` as the top-level DUT module (or VHDL entity), use the following command:

```
ixcom dut-shell.v -ua +dut+Dut -externalNetlists nl-config.spec
```

Or,

```
vlan dut-shell.v
ixcom -ua +dut+Dut -top Dut -externalNetlists nl-config.spec
```

To import `Dut` as the top-level DUT module (or VHDL entity) instantiated in the testbench (`Top`), use the following command:

```
ixcom top.sv dut-shell.v -ua +dut+Dut -external nl-config.spec
```

Or,

```
vlan top.sv dut-shell.v
ixcom -ua +dut+Dut -top Top -external nl-config.spec
```

To import `Dut` as a non-top DUT module (or VHDL entity) within the DUT hierarchy (`Top`), use the following command:

```
ixcom top.sv dut-shell.v -ua +dut+Top -externalNetlists nl-config.spec
```

Or,

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
vlan top.sv dut-shell.v
ixcom -ua +dut+Top -top Top -externalNetlists nl-config.spec
```

You must provide an empty RTL shell module with the complete interface (port I/O) declaration for the netlist module to the IXCOM compiler. The hot-swap feature for the netlist hierarchy is supported only when the netlist module is imported into a DUT hierarchy.

Exporting RTL Module in the Netlist Hierarchy

This section uses the RTL export terminology to define the following:

- ❑ The Netlist hierarchy that includes the RTL (behavioral) module instantiation
- ❑ The RTL module(s) that need to be compiled using the IXCOM compiler to elaborate the RTL hierarchy with the netlist hierarchy during the back-end compile process.

To allow the RTL module or hierarchy within the netlist hierarchy, you must specify the following information in the configuration file:

- Netlist module(s) using the `NL_IMPORT` keyword
- RTL module(s) using the `NL_EXPORT` keyword
- You can also specify a comma-separated list of hierarchical paths to module instance with the `NL_EXPORT` keyword. This information is optional, but it is necessary for the module(s) that generate primary I/O signals. The IXCOM compiler generates primary I/O signals if the exported RTL module needs to interact with the software partition (or the testbench).
- netlist file(s) using the `NL_FILE` keyword
- Hot-swap and module re-definition specification (optional)
- An empty shell module with complete interface (port I/O) declaration for the imported netlist module (to be specified to IXCOM compiler in RTL)
- RTL instance binding into the netlist hierarchy using the `NL_BIND` keyword

This specification is required only if you want to insert or replace the RTL-module instantiation in the netlist hierarchy. Use the SystemVerilog language syntax to specify the RTL binding. This file is not processed by the IXCOM compiler, and it is imported into the design by the back-end compile process.

For this use model, the back-end compiler (xeCompile) binds the RTL module instance(s) by using the following methods:

- Direct instantiating the RTL module within the netlist hierarchy

- Using the explicit binding that you have specified to insert the RTL-module instantiation or overwrite the existing RTL instantiation in the netlist hierarchy

The following sub-sections describe the use model for the above two scenarios.

Binding RTL Modules Using Direct Instantiation

This section describes how you can export the RTL module instantiated within the netlist hierarchy.

The following example is used to illustrate this use model.

```
// File: top.sv
module Top;
    // ...
    Dut dut(clk, din, dout);
    // ...
endmodule // Top
```

```
// File: dut-nl.v
module Dut(clk, din, dout)
    // ...
    Chk chk(clk, din, dout);
    // ...
endmodule // Dut
```

```
// File: dut-shell.v
module Dut(clk, din, dout);
    input clk;
    input [3:0] din;
    output [3:0] dout;
endmodule // Dut
```

```
// File: chk.sv
module Chk(clk, din, dout);
    // RTL module
endmodule // Chk
```

```
// File: nl-config.spec
NL_FILE ./dut-nl.v
NL_IMPORT Dut
NL_EXPORT Chk : Top.dut.chk
```

In the above example, the `Chk` module is exported as an RTL module to be compiled using the IXCOM compiler. The back-end compiler binds the RTL module compiled by the IXCOM compiler to the direct instantiation of this module found in netlist hierarchy.

The IXCOM compiler is invoked as follows:

```
ixcom top.sv dut-shell.v -ua +dut+Top chk.sv +dut+Chk \  
-externalNetlists nl-config.spec
```

Or,

```
vlan top.sv  
vlan dut-shell.v  
vlan chk.sv  
ixcom -ua +dut+Top+Chk -top Top -top Chk -externalNetlists nl-config.spec
```

This use model is supported in all the three netlist-import scenarios mentioned in the [Importing Netlist Modules into the RTL Hierarchy](#) section.

For this use model, the following design compile requirements apply:

- All netlist module requirements specified in the [Importing Netlist Modules into the RTL Hierarchy](#) section.
- The RTL module must have an HDL instance within the netlist hierarchy. HDL instances cannot be parameterized.
- The RTL module file(s) must be analyzed and elaborated using IXCOM compiler.

The RTL modules can have any levels of instance hierarchy and design constructs. The `generate` constructs are allowed within the RTL hierarchy, but restrictions apply for the compiler generated scope names. Refer to the [Using Hot Swap with Imported Netlists](#) section for more details on these restrictions.
- The RTL module(s) must be specified as top-level modules (using the `-top <du>` option) when using separate analyze and elaboration steps.
- The netlist module or hierarchy cannot be imported into the exported RTL hierarchy.

Using Explicit Binding to Export RTL Modules

This section describes the use model details for exporting RTL module instantiated within the netlist hierarchy using explicit binding.

The following example is used to illustrate this use model.

```
// File: top.sv  
module Top;
```


UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
//...
Dut dut(clk, din, dout);
//...
endmodule // Top

// File:dut-nl.v
module Dut(clk, din, dout)
    //...
    // Chk chk(...);
    //...
endmodule // Dut

// File: dut-shell.v
module Dut(clk, din, dout);
    input clk;
    input [3:0] din;
    output [3:0] dout;
endmodule // Dut

// File chk.sv
module Chk(clk, din, dout);
    // RTL module
endmodule // Chk

// File: nl-config.spec
NL_FILE dut-nl.v
NL_IMPORT Dut
NL_EXPORT Chk
NL_BIND nl-bind-rtl.sv
// File: nl-bind-rtl.sv
bind Top.dut Chk chk(clk, dout); // NOTE: Uses SystemVerilog bind syntax
```

In the above example, the `Chk` module is exported as an RTL module to be compiled using the IXCOM compiler. The IXCOM compiler uses the explicit binding that you have specified with the `NL_BIND` keyword to bind the RTL module into the netlist hierarchy.

Use the following command(s) to invoke the IXCOM compiler:

```
ixcom top.sv dut-shell.v -ua +dut+Top chk.sv +dut+Chk \
    -externalNetlists nl-config.spec
```

Or,

```
vlan top.sv dut-shell.v chk.sv
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
ixcom -ua +dut+Top+Chk -top Top -top Chk \  
-externalNetlists nl-config.spec
```

You can use this use model for all the three netlist import scenarios mentioned in the [Importing Netlist Modules into the RTL Hierarchy](#) section.

While compiling the design with this use model, keep the following requirements in mind:

- All the netlist module requirements listed in the [Binding RTL Modules Using Direct Instantiation](#) section.
- In addition to the SystemVerilog language rules, the following additional restrictions apply for the `bind` directives specified using the `NL_BIND` keyword:
 - ❑ These directives must be used only to bind RTL module(s) to be exported in the netlist hierarchy.
 - ❑ Parameterized instances are not supported.
 - ❑ Module instantiation and port-connection syntax is limited to Verilog syntax. SystemVerilog features like dot-start (. *) port connection are not allowed
 - ❑ Only instance-based binding is supported by the back-end compiler. So, the target scope in the `bind` directive must be an instance path.
 - ❑ The back-end compiler applies the following bind rules for the `bind` instances specified using the `NL_BIND` keyword:
 - The RTL-module instance(s) specified in the `bind` directive are inserted in the target hierarchy if no instances with the same name are found in the target hierarchy.
 - The RTL-module instance(s) specified in the `bind` directive overwrite existing instance(s) with the same name in the target hierarchy. The compiler issues a warning message in this case.
 - The RTL-module-instance name collision with the other (non-instance) objects in the target hierarchy might result in an error.

Importing Netlists Using the XEL Scripts for ICE-Mode Compilation

If you have previously written compilation scripts for ICE mode, you can easily edit these scripts to import the netlists generated by the IXCOM compiler.

For this, use the `-target` option of the `ixcom` command to compile the design in stages:

```
ixcom [ options ] -target hdllice
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOT

The above `ixcom` command ensures that the IXCOT compiler stops after the HDL Compiler stage and generates an import script in the compilation directory:

```
xc_work/nl/import.qel
```

This compile script only includes the step to import the netlist files specified using the `NL_FILE` keyword together with the `xc_va.ver` file generated during the IXCOT compilation. In the `xcva_top` file, the IXCOT compiler includes the top-level DUT modules specified with the `NL_IMPORT` keyword.

To enable the import of the IXCOT-generated netlist, the IXCOT compiler writes the following commands in the compilation script for the ICE mode:

- A list of reference libraries:

```
refLib qtref generic
```

- Netlist import options:

```
importOption { library hdlb }
```

- A list of net-list file (HDL-ICE or third-party net-list files), using:

```
netlistFile {...}
```

Source the above-generated import script:

```
source xc_work/nl/import.qel
```

Import the design using the following command:

```
designImport
```

Specify the top-level design name as follows:

```
design hdlb xcva_top
```

Issue the remaining precompile and compile commands as required for the ICE compilation.

Following is a sample ICE compilation script:

```
# ICE compilation script
# Reference libraries
# NOTE: The following reference libraries must be specified in addition
# to the reference libraries required to import other netlist file(s)

refLib qtref generic

#...
# Import options
# NOTE: The following library option must be specified
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
importOption {library hdlldb}

#...
# Specify netlist files using netlistFile
# netlistFile {...}
# Import the netlist file(s) generated by the IXCOM compiler

source xc_work/nl/import.qel
designImport
# Specify the top-level design module
# NOTE: IXCOM uses "xcva_top" as top level design unit name.
design hdlldb xcva_top
```

Continue and complete the back-end compile stage (xeCompile) using the next stage with the `ixcom` command:

```
ixcom -target xecompile
```

For the above command, you need to specify the ICE-compilation script as required by the IXCOM compiler by using one of the following ways:

- ☐ Use `./compile.qel` in the compilation directory
- ☐ Use the `ixcom -xeCompile compile=<file>` command

It is highly recommended to use the `ixcom` command above to invoke `xeCompile`.

You need to include all the `xeCompile` options (if specified earlier with the `ixcom` command to elaborate the design) with the above `ixcom` command for invoking `xeCompile`. These options include:

```
+lxua, +keepregs, -lps <file>, -xeCompile <arg>, -xecompileargs <string>
```

If you have invoked `xeCompile` directly from the ICE-compilation script, you need to specify the IXCOM options listed above explicitly by using the native XEL user data or commands. The IXCOM compiler also updates the design-partition information after the back-end compile stage. So, you need to invoke the following update stage prior to running the design on hardware:

```
ixcom -target postXeCompile
```

To continue and complete the software-model compilation using IES, use the following command:

```
ixcom -target swmodel
```

With the netlist configuration file, you cannot specify additional libraries for the third-party netlist file(s), if any. You must specify the reference in the ICE-compilation script (See the

`refLib` command description in the *UXE Command Reference Manual* for details on specifying reference libraries).

Note: You cannot use the compilation stages of the IXCOM compiler if you are compiling the design using the *irun* tool. For staged compilation, use the IXCOM compiler.

Using Hot Swap with Imported Netlists

The hot-swap feature provides the ability to move the run-time database from hardware (run using emulator) to software (run using IES) and vice-versa during the SA run. If you request a hot-swap capability for netlist, the IXCOM compiler needs to pass the netlist model to IES.

By default, the hot-swap feature is disabled for the netlist hierarchy. To enable it, include the following specification in the configuration file:

```
NL_HOTSWAP on
```

If hot-swap is enabled, the IXCOM compiler presents the netlist file(s) to IES with the required binding specification and HDL Compiler reference file.

For the example in the [Using Explicit Binding to Export RTL Modules](#) section, this information look like the following:

```
=> cat xc_work/nl/netlist-files.f
dut-nl.v
    -v ${AXIS_HOME}/etc/ixcom/ixcom_hdllice_ref.vp
    -define CBV=1
=> cat xc_work/nl/netlist-binds.f
nl-bind-rtl.sv
```

Using the hot-swap feature with netlist import has the following limitations:

- The netlist file(s) presented to the IXCOM compiler must have simulation-compatible model as supported by IES tools.

The RTL-module-instance binding using the SystemVerilog bind feature that requires *instance overwrite* is not allowed. For this limitation, you need to manually edit the netlist file and make it compliant with the SystemVerilog language requirements for IES compilation.

- The `generate` construct within the exported RTL hierarchy is not supported with the hot-swap feature. This limitation is because of the incompatibilities between the scope names in the hardware and software partitions.

With the netlist configuration file, you cannot specify additional libraries for the third-party netlist file(s), if any. You must specify the reference libraries and files for the IES compilation by using the `NCVLOGOPTS` and `NCELABOPTS` options in the `./uxc_hdl.var` file.

Accessing Netlist Objects from RTL Hierarchy

The IXCOM compiler recognizes the hierarchical references to objects in external netlist hierarchy as scalar nets. For such references, you do not need to provide any additional information to bind references to RTL domain.

For non-scalar nets, you must specify both hierarchical reference and object data-type information by using the `NL_REF` keyword:

- ❑ The `wire` keyword must be specified for hierarchical reference to *net* (or *signal*) in the netlist hierarchy.
- ❑ Packed dimension must be specified for hierarchical reference to `bit vector net` in the net-list hierarchy.
- ❑ Both packed and unpacked dimension must be specified for hierarchical reference to `memory` in the netlist hierarchy.
- ❑ Both the packed and unpacked dimension specifications must exactly match the original declaration for the netlist object.
- ❑ The `reg` keyword must be specified only in reference to the `memory` object as mentioned above. The hierarchical reference to `variable` in the netlist hierarchy is not supported.
- ❑ The hierarchical reference is limited to `net` and `memory` objects. References to other objects are not supported.
- ❑ The IXCOM compiler issues an error message for hierarchical references that do not refer to netlist objects.
- ❑ The IXCOM compiler issues a warning if hierarchical references to a `net` object are detected in the design unit partitioned to the software side.

The following example shows the configuration specification for the non-scalar reference to external netlist objects:

```
// File: top.sv
module Top;
    //...
    Dut dut(clk, din, dout); Chk chk(clk, dout);
    always @(*)
```

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
        dut.r = dut.chk.n[1];
    endmodule // Top

// File: dut-nl.v
module Dut(clk, din, dout)
    wire [3:0] a;
    reg [7:0] mem[1023:0];
endmodule // Dut

// File: chk.sv
module Chk(clk, dout);
    wire [3:0] n[32];
endmodule // Chk

// File: nl-config.spec
NL_FILE dut-nl.v
NL_IMPORT Dut
NL_EXPORT Chk

// Specify reference to external netlist objects
NL_REF wire [3:0] Top.dut.a
NL_REF reg [7:0] Top.dut.mem[1023:0];
NL_REF wire [3:0] Top.dut.chk.n [32];
```

For designs compiled with the `NL_REF` keyword references, the references have the following functional behavior:

- ❑ The reference to the netlist is always functional (holds accurate value) when the design is compiled with the hot-swap feature enabled.
- ❑ The reference to the netlist is only functional in the hardware run when the hot-swap feature is not enabled.

Generating Clocks and Binding Clock signals in the Netlist Hierarchy

Following features are supported in addition to the support described in the [Importing External Netlists into IXCOM Compiler](#) and [Importing Netlists as Blackboxes in SA Mode](#) sections for clock generation and binding:

- ❑ Support for CAKE 0.5x mode
- ❑ Simplified use model for binding clock signals into external netlist hierarchy

UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

The RTL hierarchy (including clock generation module) can refer to clocks in the external netlist hierarchy if you are using the configuration file (`-externalNetlists <file>`). This also means that the `ixclkgen` command can be used without the `-external <string>` option to generate the clock module. Alternatively, you can include `<module>-shell.inc` file with the shell module.

The following example illustrates how you can drive clock signals into imported and exported netlist hierarchies:

```
// File: top.sv
module Top;
    //...
    Dut dut(clk, din, dout);
    //...
endmodule // Top

// File: dut-nl.v
module Dut(clk, din, dout)
    //...
    // Chk chk(clk2, dout);
    //...
endmodule // Dut

// File: chk.sv
module Chk(clk, dout);
    output reg clk;
    // RTL module
    //...
endmodule // Chk

// File: nl-config.spec
NL_FILE dut-nl.v
NL_IMPORT Dut
NL_EXPORT Chk
NL_BIND nl-bind-rtl.sv
NL_REF reg Top.dut.sub.chk.clk // required for non-scalar nets

// File: nl-bind-rtl.sv
bind Top.dut Chk chk(clk, dout); // NOTE: Uses SV bind syntax

// File: clock-ud.qel
clockOption -add { technology CAKE 2 }
```


UXE User Guide

Compiling Designs for Simulation Acceleration Using IXCOM

```
clockFrequency -add { Top.dut.sub.clk 400 MHz }  
clockFrequency -add { Top.dut.sub.chk.clk 500 MHz }
```

In the above example, the `Dut` module is imported into the external hierarchy with the `Chk` hierarchy exported to the RTL domain. The clocks in the external netlist hierarchy can be driven as follows:

```
ixclkgen -input clock-ud.qel -output clk.sv -module Clk
```

And

```
ixcom top.sv -ua +dut+Top+Chk+Clk chk.sv clk.sv\  
-externalNetlists nl-config.spec
```

Or

```
vlan top.sv chk.sv clk.sv  
ixcom -ua +dut+Top+Chk+Clk -top Top -top Chk -top Clk \  
-externalNetlists nl-config.spec
```

The user is not required to specify external net-list information to `ixclkgen` utility using the `-external <string>` option. However, reference to the external netlist hierarchy must be specified in configuration file using the `NL_REF` keyword.

The `NL_REF` keyword is not required if the hierarchical reference to the external netlist object is a scalar net of `wire` type.

To drive a clock signal from an external module into the top-level design unit imported as a netlist, you need to specify the `+targetTop+<du>` option for disconnecting port signals marked as input or inout ports.

Import Options for Design Units Defined in Both Netlist and RTL Domains

In large designs, a design unit with the same name can exist in both netlist and RTL domains. The back-end compiler can only import one of the definitions. The following sections provide details on specifying import options for this purpose.

Keeping Design Unit Definition in the Netlist Domain

For design units defined in both domains, you can specify the following configuration to keep the definition from the netlist domain:

```
NL_DEF_KEEP external // optional
```

This is the default behavior in the absence of the above configuration specification.

The IXCOM compiler instruments the PI/PO signals for the RTL definition to support features like SVA, Coverage, `$export tasks`, etc. For such RTL definitions, you must specify the `+moduleStub+<du>` option with the `ixcom` command to ignore the RTL definition, and keep the design-unit definition from the netlist domain. This helps avoid the name-mapping error during back-end compiler stage.

Keeping the Design Unit Definition in the RTL Domain (Compiled Using IXCOM)

For the design unit defined in both domains, you can specify the following configuration to keep the definition from the RTL domain:

```
NL_DEF_KEEP ixcom
```

Re-Defined Design Units in a Given Domain

The import options mentioned in the earlier sections allow definitions to be chosen from one of the domains. Both the netlist and RTL domains can have the design units re-defined as well. For such re-defined design unit.

- ❑ In both the IXCOM and IES compilations, the last definition that is found is kept
- ❑ The back-end compiler (xeCompile) keeps the first non-empty definition

You can modify this behavior to match the IES compilation by using the `importOption { keep last }` option.

Note: Design unit re-definitions within a given domain cannot be resolved by the compilers. You need to resolve such conflicts manually by editing the design or by using language features, such as configuration.

Compiling and Running Simulation Acceleration Designs with *irun*

The *irun* utility enables compilation and simulation of designs using the INCISIV simulator by specifying all the input files and options on a single command line. *irun* simplifies the invocation process by invoking multiple tools using a single command.

The traditional use models for simulation acceleration (SA) mode using Palladium XP require a mix of different tools to compile and simulate a design. The *irun* use model for simulation has been designed to invoke the IXCOM tools (vlan, vhan, and ixcom) to compile a design for simulation acceleration, and simulate the hardware-acceleratable snapshot. Thus, *irun* facilitates a smooth transition from the pure IES-based software simulation mode to SA mode using Palladium XP.

This chapter covers the following topics:

- [Overview](#)
- [SA-Specific Options of the *irun* Command](#)
- [Compiling Designs](#)
- [Simulating Designs](#)
- [Limitations](#)



The content of this chapter pertains only to the SA mode.

Overview

The *irun* interface for compiling and running designs in the SA mode is similar to the *irun* interface used in IES-based software simulation mode.

For SA mode, in addition to using the typical software-simulation options of the *irun* command, you need to use the following command-line options with the *irun* command:

- Command options for enabling SA mode
- Compilation options for SA mode that need to be passed to IXCOT tools
- Simulation options that need to be passed to INCISIV tools or xeDebug

irun internally invokes the INCISIV and IXCOT tools to compile the source files, generate the snapshot for SA mode, and invoke the simulator. *irun* dispatches the specified command-line options to the appropriate INCISIV and IXCOT tools. At run time, the INCISIV simulator, *ncsim*, is invoked as the default simulation tool. Alternatively, you can invoke xeDebug as the run-time and debug environment by using the `-xedebug` option with the *irun* command. For more information about the new command options, see [SA-Specific Options of the *irun* Command](#) on page 361.

Setting Up the Environment to Use *irun* for Simulation Acceleration

You must set the required environment variables to use *irun*. See the *Configuring Your Environment for the 64-Bit Version* section in the *Setting Up Your Environment* document in the INCISIV documentation set for more information.

Additionally, set the required environment variables for IXCOT before you compile and run the design in SA mode. See [Setting Up the Environment for Palladium XP](#) on page 43 for more information.

Compiling and Simulating Designs

Use *irun* to compile and simulate designs for simulation acceleration either in a single step or multiple steps.

Single-Step Compilation and Simulation

In the single-step compilation and simulation process, use *irun* to:

- Compile and elaborate using IXCOT and INCISIV tools, and simulate using *irun*:

```
irun -hw [ixcot_options] [ies_options] <source_files>
```
- Compile and elaborate using IXCOT and INCISIV tools, and simulate using xeDebug:

```
irun -hw -xedebug [ixcot_options] [ies_options] <source_files>
```

Multi-Step Compilation and Simulation

In the multi-step compilation and simulation process, use *irun* to:

1. Compile and elaborate the design in multiple steps:

```
1 irun -hw -compile [ixcom_options] [ies_options] <source_files>
```

Note: This step can be invoked multiple times to compile different sets of input source files.

```
2 irun -hw -elaborate [ixcom_options] [ies_options]
```

2. Simulate and debug the design:

- ❑ Simulate the design using *ncsim*:

```
irun -R
```

- ❑ Simulate and debug the design using xeDebug:

```
irun -R -xedebug
```

The `-hw` option enables the compilation of design for SA mode. *irun* internally invokes the IXCOM tools (vlan, vhan, and IXCOM) to compile the design files with the IXCOM options specified on the command line. The IXCOM tools work according to their existing specifications. By default, *ncsim* is invoked for simulation. If the `-xedebug` option is specified on the *irun* command line, the xeDebug tool is invoked for simulation. The run-time environment using xeDebug is discussed in [Simulating Designs](#) on page 376.

Important

irun has been enhanced for SA mode to facilitate the transition from pure IES-based simulation to hardware-accelerated simulation using Palladium XP. You can still directly use the IXCOM tools (vlan, vhan, and ixcom) for design compilation. Unlike the INCISIV tools, IXCOM tools do not support incremental compilation. When using *irun* for SA mode, incremental compilation is supported only for the portion of the design compiled using INCISIV tools.

Note: Using *irun -hw* command to compile the design for SA mode so that it can be run on the Palladium XP emulator is a Beta-level feature. Beta-level features are those that have been introduced in one of the recent releases but have not yet reached the production stage. The stability and usability of these features is not well established. Beta-level features are released to you to obtain your feedback before the features reach the production stage.

SA-Specific Options of the *irun* Command

To use *irun* for compiling and running the design for SA mode, specify the following categories of options with the *irun* command:

- [Compilation Options to Enable SA Mode](#)
- [Run-Time Options for Using xeDebug](#)

Compilation Options to Enable SA Mode

This section lists the options of the *irun* command to compile the design files for creating a hardware-acceleratable simulation snapshot.

Table 5-1 Compilation Options for Simulation Acceleration

Compilation Option	Description
-hw	Note: Specifies that the IXCOT tools should be used to compile and process the design for simulation acceleration. All the <i>ixcom</i> command options that are specified with the <i>irun</i> command, without specifying the -hw option are ignored.
-compile_sw <filename1> <filename2> ... -end	Specifies the files that should be compiled using the INCISIV tools, irrespective of the HDL used to write these files. Use this option for using <i>irun</i> to compile code written in Hardware Verification Languages (HVLs) that cannot be mapped to the emulator.
-ixcomargs <string>	Directly dispatches options indicated by <string> to IXCOT (without any parsing by <i>irun</i>). The -<ixcom_tool>args <string> options are provided to make the use of <i>irun</i> for SA mode compatible with the use of INCISIV tools in the pure simulation invocation using <i>irun</i> .
-vlanargs <string>	Directly dispatches options indicated by <string> to the vlan tool (without any parsing by <i>irun</i>).
-vhanargs <string>	Directly dispatches options indicated by <string> to the vhan tool (without any parsing by <i>irun</i>).
-log_ixcom <logfile>	Displays output from the <i>ixcom</i> command into the specified <i>logfile</i> . The -log_<ixcom_tool> <logfile> options are provided to make the use of <i>irun</i> for SA mode compatible with the use of INCISIV tools in the pure simulation invocation using <i>irun</i> .

UXE User Guide

Compiling and Running Simulation Acceleration Designs with irun

<code>-log_vhan <logfile></code>	Displays output from the <code>vhan</code> command into the specified <i>logfile</i> .
<code>-log_vlan <logfile></code>	Displays output from the <code>vlan</code> command into the specified <i>logfile</i> .
IXCOM tools command options	All the options for IXCOM tools (<code>vlan</code> , <code>vhan</code> , and <code>ixcom</code>) are recognized by the <code>irun</code> command and passed to the appropriate tools. Refer to the <i>Commands and System Tasks for SA Mode</i> chapter in the <i>UXE Command Reference Manual</i> for a list of all these command options.

Run-Time Options for Using xeDebug

This section lists the options of the `irun` command used for running and debugging the simulation by using the xeDebug tool. By default, the *irun* tool is used at run time in the absence of these options.

Table 5-2 Options to Use xeDebug at Run Time

Option	Description
<code>-xedebug</code>	Specifies that xeDebug should be used at run time.
<code>-host <name></code>	Specifies the host name for the emulator. By default, the current machine is identified as the host machine.
<code>-xedebugargs "[options]"</code>	Specifies the arguments to be passed on to xeDebug at run time. For example, the following command enables you to use dynamic probes when running xeDebug from <i>irun</i>. This option is particularly useful because when you invoke xeDebug from <i>irun</i>, you do not have any control over the emulator. <pre>irun -R -xedebug -xedebugargs "-dynp"</pre>
<code>-disxedebug</code>	Specifies that <i>irun</i> should be used at run time. This is the default option.

Accessing Help Information for IXCOM Tools from *irun*

The help infrastructure of *irun* provides usage information for the options for IXCOM tools.

- To display help for options specific to a tool, use the following command syntax:

```
irun -helpsubject <topic>
```

For example:

- ❑ *irun* -helpsubject *vlan* - Displays a list of *vlan* command options and their descriptions.
- ❑ *irun* -helpsubject *vhan* - Displays a list of *vhan* command options and their descriptions.
- ❑ *irun* -helpsubject *ixcom* - Displays a list of *ixcom* command options and their descriptions.
- ❑ *irun* -helpsubject *hwaccel* - Displays a list of all the supported command options for the IXCOM tools (*vlan*, *vhan*, and *ixcom*) and their usage descriptions.

- To display help for a specific option of an IXCOM tool, use the following command syntax:

```
irun -helpargs <option>
```

- ❑ Executing the *irun* -helpargs <*ixcom_option*> command displays the description for the specified *ixcom* command option.

- ❑ Executing the *irun* -helpargs -ua command displays the following output:

```
irun(64): 10.20-s040: (c) Copyright 1995-2011 Cadence Design Systems, Inc.
```

```
-ua                                Compile design to UA hardware engine
```

irun provides additional help options for getting verbose descriptions for command options and retrieving a list of tools to which the specified option belongs. For more information, see the *irun User Guide* in the INCISIV documentation set.

Compiling Designs

irun invokes the IXCOM and INCISIV tools to compile input source files when the *-hw* option is specified on the command line. IXCOM and INCISIV tool invocation mechanism is described below:

- Verilog (Verilog-1995, Verilog-2001, SystemVerilog) and VHDL (87 and 93) source files are compiled using the **IXCOM** tools (*vlan*, *vhan*, and *ixcom*). The INCISIV tools support

all these languages supported by the IXCOM tools as well. But, priority is given to using IXCOM tools for these languages.

- All the source files written in languages other than the languages specified above, such as SystemC, Specman *e*, C and C++ (for PLI, VPI, VHPI, and DPI) are compiled using **INCISIV** tools.

Exception to the above points: Source files specified within the `-compile_sw` and `-end` options are always compiled using INCISIV tools irrespective of the HDL used to write these files.

irun dispatches the command-line options to the tools it invokes. For more information, see [Parsing IXCOM and *irun* Command Options with the *irun* Tool](#) on page 370.

The following sub-sections describe how *irun* supports designs using different languages, in simulation acceleration mode.

- [Verilog-Only Designs](#)
- [Mixed-Language \(Verilog and VHDL\) Designs](#)
- [Designs with PLI/VPI and VHPI](#)
- [Designs with SystemVerilog DPI](#)
- [Designs with SystemC](#)
- [Designs Using Specman *e* Testbenches](#)

Note: For the use model examples in this chapter, only the required options are listed with the *irun* command. You can include additional *irun* and/or IXCOM options for each of the following use models.

Other compilation aspects are described in the following sub-sections:

- [Parsing IXCOM and *irun* Command Options with the *irun* Tool](#)
- [User-Partitioned Testbench and DUT Compilation](#)
- [Logical Library Support](#)

Verilog-Only Designs

Verilog-only design files are compiled using the *irun* command as shown below:

```
irun -hw [<ixcom_options>] [<ies_options>] <source_files>
```

UXE User Guide

Compiling and Running Simulation Acceleration Designs with *irun*

If the `-top` option is not specified on the *irun* command line, single-step compilation using *ixcom* is invoked. IXCOM automatically determines the top-level design unit names. For example, the following command invokes *ixcom* to analyze the source files and elaborate the design in a single step:

```
irun -hw -ua +dut+Dut Tb.sv Dut.v
```

If you specify the top-level design unit names using the `-top` option on the *irun* command line, multi-step IXCOM compilation using *vlan* and *ixcom* is invoked. For example, the following command invokes *vlan* to analyze the source files and *ixcom* to elaborate the design.

```
irun -hw -ua +dut+Dut Tb.sv Dut.v -top Tb
```

Using the `<tool> args` Options

In single-step invocation using the *ixcom* command, *irun* ignores the `-vlanargs` option specified on the command line. `-ixcomargs` must also have context-sensitive (single-step or multi-step) options for correct compilation of the design.

Mixed-Language (Verilog and VHDL) Designs

Mixed-language (Verilog and VHDL) design files are compiled using *irun* command as shown below:

```
irun -hw [ixcom_options] [ies_options] <verilog_and_vhdl_source_files>
```

For mixed-language designs, *irun* invokes IXCOM tools to compile the design in multiple steps. First, design files are analyzed using language specific analyzers, *vlan* for Verilog files and *vhan* for VHDL files. In the final step, analyzed design units are elaborated using *ixcom* to generate the simulation snapshot. For example, the following command invokes *vlan* and *vhan* to analyze source files and *ixcom* to elaborate the design.

```
irun -hw -ua +dut+Dut Tb.sv Dut.vhd -top Tb
```

Important

To compile mixed-language designs that include VHDL source files, you must specify the top-level design unit names using the `-top` option. This requirement is applicable even for designs in which a Verilog top-level design unit instantiates a VHDL design unit.

Designs with PLI/VPI and VHPI

PLI/VPI and VHPI applications and design files are compiled using the *irun* command, as shown below:

```
irun -hw [ixcom_options] [ies_options] <design_files_&_PLI/VPI/  
      VHPI_application_source_files/library>
```

For example:

```
irun -hw -ua +dut+Dut +sv test.v test.c
```

The PLI/VPI/VHPI application files are compiled using *ncsc*. *irun* invokes IXCOM tools to compile the design files using either a single step or multiple steps, as described in [Verilog-Only Designs](#) and [Mixed-Language \(Verilog and VHDL\) Designs](#) sections. The design is elaborated using *ixcom* to generate the simulation snapshot with the application code library.

Only the *-afile*, *-loadpli1*, and *-loadvpi* options of *irun* are mapped to the corresponding options of the *ixcom* command.

- Use the *-afile* option to specify the PLI map file. The *-p|P* options of *ixcom* can also be used, depending on the map file format.

```
irun -hw -ua +dut+Dut +sv -afile PLI_map_file test.v test.c
```

Note: If the PLI map file is not specified, *irun* generates this mapping file using the *ncpli2tab* utility. Support for the *-afile* option is limited to PLI/VPI specific content in the access file. Other access specifications in the file result in IXCOM tool error.

- The support for PLI/VPI and VHPI is similar to the way *irun* reads the PLI/VPI-specific content in the file specified using the *-afile* option. Refer to INCISIV documentation on “*Using a PLI map file*” for details on PLI/VPI-specific-content-allowed access file. The *-afile* option is mapped to the *ixcom* command option. IXCOM reports an error for non-PLI/PVI specific content in the access file. Following is an example of an *irun* command using the *loadpli1* option:

```
irun -hw -ua +dut+Dut +sv Tb.sv Dut.sv +loadpli1=".<path>/<library_name>.so"
```

- Use the *+loadvpi* option for VPI applications.

VHPI support is handled through attributes in the source language, and does not require any explicit command-line option to load the application library.

Designs with SystemVerilog DPI

The SystemVerilog DPI application code and design files are compiled using the *irun* command, as shown below:

UXE User Guide

Compiling and Running Simulation Acceleration Designs with `irun`

```
irun -hw [<ixcom_options>] [<ies_options>]  
      <design_files_&_DPI_application_source_files/library>
```

SystemVerilog DPI application source files are compiled using `ncsc`. *irun* invokes IXCOM tools to compile the design files using single or multiple steps, as described in [Verilog-Only Designs](#) and [Mixed-Language \(Verilog and VHDL\) Designs](#) sections. The design is elaborated using `ixcom` to generate the simulation snapshot with the application code library.

For example, the following commands are used for multi-step compilation with pre-compiled application DPI application library:

```
irun -hw -ua +dut+Dut Tb.sv Dut.sv Dpi.cpp  
irun -hw -ua +dut+Dut Tb.sv Dut.sv -sv_lib ./<path>/<library_name>.so
```

irun provides the `-cpost` and `-end` options to specify the order of compilation of DPI C/C++ libraries. *irun* orders internal invocations of IXCOM tools to correctly handle the functional requirements of these options, as it is handled in pure software simulation mode. See the *irun User Guide* in the INCISIV documentation set for more information about these options and their functional requirements, and other DPI-specific-elaboration options.

Designs with SystemC

SystemC application code and design files are compiled using `irun` command, as shown below:

```
irun -hw [ixcom_options] [ies_options] \  
      <design_files_and_SystemC_application_source_files_or_library>
```

SystemC application source files are compiled using `ncsc`. The *irun* tool invokes the IXCOM tools to compile the design files using single or multiple steps, as described in [Verilog-Only Designs](#) and [Mixed-Language \(Verilog and VHDL\) Designs](#) sections. The design is elaborated by using the `ixcom` command to generate the simulation snapshot with the application code library.

Important

To compile SystemC designs, you must specify the top-level design unit names using the `-top` option. Otherwise, an error is reported.

For example:

- One-step compilation and simulation using the shell interface:

```
irun -hw -sysc Tb.v Dut.sv Dut_shell.sv Sysc.cpp
```

- One-step compilation and simulation using the auto-generated shell file:

```
irun -hw Tb.v Dut.sv -loadsc ./<path>/<library_name>.so
```

Automatic generation of SystemC shell file support is enabled for Verilog and VHDL with the `-scautoshell` option. An unsupported error message is reported for “`-scautoshell SystemC`” invocation with “`-hw`” options.

The *irun* tool passes the SystemC library (created by `ncsc_run`) to IXCOM elaboration invocation with the `-loadsc` option.

Designs Using Specman *e* Testbenches

Specman *e* testbench and design files are compiled using *irun* command, as shown below:

```
irun -hw [ixcom_options] [ies_options] <design_files_&_testbench_e_files>
```

For example:

```
irun -hw -ua +dut+Dut Dut.sv Tb.e
```

Keep the following considerations in mind when using *irun* to compile Specman *e* designs for hardware using the `-hw` option of the *irun* command:

- The Specman reference files for VHDL are not compiled automatically (unlike the pure-simulation flow). You must specify the reference files explicitly in the *irun* command line to enable VHDL design files to be compiled using the IXCOM tools.
- The Specman package files for SystemVerilog are not compiled automatically (unlike the pure-simulation flow). You must specify the package files explicitly in the *irun* command line to import package objects into SystemVerilog modules compiled using IXCOM tools. IXCOM tool limitations apply for the package file.
- Combination of Specman *e*, SystemC, and Verilog/VHDL compilation is not supported.
- The timescale information must be specified for IES elaboration.

Note: This limitation exists for the IXCOM-based compilation flow as well.

Important

Use the `-sntimescale <arg>` option on the *irun* command line to specify the timescale for Specman delays. In a pure software-simulation run, even if the timescale for Specman is not specified using the `-sntimescale` option, for a design with a consistent timescale definition, the timescale from *irun* is automatically used for Specman, along with a warning. When using *irun* for simulation acceleration, you need to explicitly use the `-sntimescale <arg>` option to specify the timescale.

Parsing IXCOM and *irun* Command Options with the *irun* Tool

irun passes the options specified with the *irun* command to the relevant IXCOM and INCISIV tools. The functionality of the option is defined by the tool that is invoked by *irun*. Thus, all the options of the *ixcom* command work in the same manner when using *irun* to compile and simulate designs in the simulation acceleration mode.

If you specify an INCISIV-tool option on the *irun* command line that has the same syntax and functionally equivalent behavior as an existing *ixcom* option, the INCISIV-tool option is mapped to the identical *ixcom* option. See [Table H-1](#) on page 1324 in the *IES - IXCOM Option Mapping* Appendix for a list of such options.

For example, both the *irun* and *ixcom* commands have the `-clean` option to remove the compiler-generated database files.

- ❑ If the `-hw` option is included, as in the following *irun* command, the database files generated by both the IES and IXCOM tools are cleaned up:

```
irun -hw -clean
```

- ❑ If the `-hw` option is not included, only the IES database files, `INCA_libs`, are cleaned up.

If you specify an IES tool option on the *irun* command line, for which the syntax is different but the functional behavior is equivalent to an option of the *ixcom* command, the IES option is mapped to the corresponding *ixcom* command option. See [Table H-2](#) on page 1326 in the *IES - IXCOM Option Mapping* Appendix for a list of such options.

You can use the `-debugscript <filename>` option of the *irun* command to generate a script for capturing the environment variables and commands related to the executables that were invoked by *irun*. This script helps to identify how each of the IES and IXCOM tools was invoked by *irun* and debug the problems encountered during the execution of the *irun* command.

Using the `-debugscript` option with the *irun* command, generates two files:

- A script with the specified `<filename>` that contains the commands that you can use to run the executables that were invoked by *irun*. Each executable is invoked with the `-f <argument_file>` option, which specifies the argument file created by *irun* when that executable was invoked. The argument file contains the command-line arguments that were passed to the executable.
- A file called `<filename>.env` that you can source to reproduce the same environment as the *irun* run.

More information about the `-debugscript <filename>` option is available in the *irun User Guide* in the *irun-Specific Command-line Options* section.

Handling Unsupported Options Ignored by *irun*

As stated above, when *irun* is invoked with the `-hw` option to compile designs for SA mode, all the *ixcom* command options are supported on the *irun* command line. However, all IES options cannot be used in the same manner. Only some IES options are mapped to IXCOM options, and are passed to the appropriate IXCOM tools at the time of invocation.

The rest of the IES options that are not mapped to any of the IXCOM options are ignored. However, you need to be careful about the usage of such options because this might result in incorrect functionality when compared to pure-IES simulation. In order to determine the ignored or unmapped IES options, use the `-checkargs` option of the *irun* command. When this option is used, the list of ignored options is reported as per the invocation of the appropriate IXCOM tool as follows:

- ❑ All ignored options of the *ncvlog* tool (IES) are reported at the time of invocation of *vlan* tool (IXCOM).
- ❑ All ignored options of the *ncvhdl* tool (IES) are reported at the time of invocation of *vhan* tool (IXCOM).
- ❑ All ignored options of the *ncvlog* and *ncelab* tools (IES) are reported at the time of single-step invocation of *ixcom* tool (IXCOM) for compiling and elaborating Verilog files.
- ❑ All ignored options of the *ncelab* tool (IES) are reported at the time of multi-step invocation of *ixcom* tool (IXCOM).

In order to see the list of all the options that are either ignored or processed, use the following command:

```
irun -hw -checkargs
```

IES-Compatible Support for Testbench Compilation

In the SA mode, the verification environment, typically, includes a non-synthesizable testbench and synthesizable design units. The *irun* utility invokes the IES tools to compile testbench source languages, which are not supported by the IXCOM tools (HVLs such as SystemC, *e* and Vera). The SystemVerilog and VHDL language portions can also contain non-synthesizable constructs intended for testbench. The *irun* command options `-compile_sw` and `-end` allow you to manually partition the testbench portion of the design.

Use the `-compile_sw` and `-end region` option to enable a pure-IES-like compilation for the testbench. However, this compilation is not exactly like the IES compilation, and has the limitation that you must specify the `-top <du>` option to compile the testbench portion of the design.

Refer to the sections [User-Partitioned Testbench](#) and [DUT Compilation](#) on page 372 and [Designs Using TBA and UVCs](#) on page 373 for details about supported testbench compilation use models.

User-Partitioned Testbench and DUT Compilation

You can specify explicit partitioning information to improve compilation efficiency. You can specify explicit IES compilation for testbench files (non-synthesizable HVLs and unsupported constructs in IXCOM). Use the `-compile_sw` and `-end` options with the *irun* command to specify these files. *irun* invokes IES tools to process the testbench files specified within the `-compile_sw` and `-end` command options (the testbench marker region). The design files are compiled using IXCOM tools. The final design is elaborated using IES with the IXCOM-compiled design units. The following *irun* command is used.

```
irun -hw -ua +dut=Dut +xcDesignTop=Tb.dut=Dut [ixcom_options] [ies_options]
    <design_source_files> -compile_sw <testbench_source_files> -end
```

Specify the top-level design units explicitly within the `-compile_sw` and `-end` command-line regions. The top-level design units specified using the `-top <du>` option within the `-compile_sw` and `-end` regions are compiled by IES. To specify top-level design units for IXCOM compilation, the `-top <du>` option must be specified outside of the `-compile_sw` and `-end` regions.

Example 1: IES testbench compilation and elaboration of pre-compiled design units using IXCOM (one-step compilation)

```
irun -hw -ua +dut=Dut +xcDesignTop=Tb.dut=Dut \
    Dut.sv Chk.vhd -compile_sw Tb.v -end
```

Example 2: IES testbench compilation and elaboration of pre-compiled design units using IXCOM (multi-step compilation)

```
irun -hw -compile dut.sv
irun -hw -elaborate -top Dut +xcDesignTop=Tb.dut=Dut ...
irun -hw -compile_sw tb.sv -top Tb -end ...
```

Limitations

IES limitations apply for options specified within the `-compile_sw` and `-end` option marker region on the command line. For more information, see *irun User Guide* in the INCISIV documentation set.

Designs Using TBA and UVCs

The *irun* support for these features is similar to use model described in the [User-Partitioned Testbench and DUT Compilation](#) section. Automation support to include header file and linking use-model-specific libraries is not available. Synthesizable design units are compiled using IXC tools and non-synthesizable constructs are compiled using IES tools. The `-compile_sw` and `-end` options in the command line are used to manually partition testbench and design source compilation. The following examples illustrate the compilation of TBA designs with SystemVerilog testbenches.

Example 1: IES testbench compilation and elaboration of pre-compiled design units using IXC (one-step compilation):

```
irun -hw dut.sv -ncompile ... -compile_sw tb.sv -top Tb ... -end
```

Example 2: IES testbench compilation and elaboration of pre-compiled design units using IXC (multi-step compilation):

```
irun -hw -compile dut.sv
irun -hw -elaborate -nccompile -top Dut ...
irun -hw -compile_sw tb.sv -top Tb -end ...
```

Note: You can compile non-synthesizable testbenches written in HVLs such as SystemC, e, and Vera by using the same use model.

Refer to [IES-Compatible Support for Testbench Compilation](#) on page 371 for details.

Logical Library Support

With the *irun* command, you can choose to analyze the design units into multiple logical libraries by using the `-makelib ... -endlib` option set as follows:

```
irun -hw [<ixcom_options>] [<ies_options>] \
    -makelib <path_to_library>[:<logical_name>] <source_files> -endlib | \
    -end> ...
```

Multiple logical libraries are supported in the same manner as pure-IES. Use the `-makeLib` and `-endlib` options to specify the libraries.

Refer to the section titled [Handling cds.lib and hdl.var Files](#) on page 374 for details about logical library specification.

Handling cds.lib and hdl.var Files

You can use the `-work` option of the `irun` command to analyze a given set of files into a single logical library, as follows:

```
irun -hw -compile -work <library> [<ixcom_options>] <source_files>
```

For IES tools, the library mapping information specified with the above commands is recorded in the `cds.lib` file. The IXCOM tools, on the other hand, use the `libmaprc` file for this library-mapping information. The syntax of the `libmaprc` file used by IXCOM tools is similar, but not the same as the `cds.lib` file used by the IES tools. When the `irun` tool is used for SA mode, this `libmaprc` file is automatically generated, based on the following:

- ❑ Information in the `cds.lib` file
- ❑ Arguments specified with the `-makeLib` option of the `irun` command

Note: The content in the `hdl.var` file is not considered for the generation of the `libmaprc` file. Also, any other specification in the `cds.lib` file other than the logical-library mapping using the `DEFINE` statement is ignored.

With **IES**, the `cds.lib` file has entries like the following:

```
define L1 ./Lib1
```

Here, `L1` is the logical name and `./Lib1` is the physical directory. It is essential to specify the logical name with the `irun` command as follows:

```
irun -work L1
```

With **IXCOM**, only the `libmaprc` file is used, and its syntax is as follows:

```
L1 => ./Lib1
```

Note: The `irun` tool creates the physical directories that are mapped to the specified logical libraries inside the `INCA_libs` directory. The IXCOM tools create the physical directories in a directory named `ixcom` within the `INCA_libs` directory. This means that when you use the `-clean` option with the `irun` command to remove the IES-generated directories, the IXCOM-generated directories are also removed.

In order to handle such automatic generation of the `libmaprc` file, and/or to force the IXCOM-style-library-mapping method of using the `libmaprc` file, you can disable the automatic conversion of `cds.lib` file by using the `-libmaprc <file>` option with the `irun` command.

For more information about the `libmaprc` files, see [Associating Logical Libraries to Physical Directories Using libmaprc File](#) on page 210. For details on handling `cds.lib` file, see [Handling cds.lib and hdl.var Files](#).

UXE User Guide

Compiling and Running Simulation Acceleration Designs with *irun*

The following table presents the search order for `cds.lib` and `libmaprc` files and the `-cdslib` and `-libmaprc` options.

Table 5-3 Search Order for `cds.lib` and `libmaprc` Files and Options

-cdslib Option	-libmaprc Option	cds.lib File	libmaprc File	used	Comment
Present	Present	Present	Present	<code>-libmaprc</code>	If both the <code>-cdslib</code> and <code>-libmaprc</code> options are present and the <code>cds.lib</code> and <code>libmaprc</code> files are present, the <code>-libmaprc</code> option takes precedence.
Present	-	Present	Present	<code>-cdslib</code>	The <code>-cdslib</code> option takes precedence over the <code>cds.lib</code> file in the current directory.
-	-	Present	<code>-libmaprc</code>	<code>cds.lib</code>	The <code>cds.lib</code> file takes precedence, and it is searched in the order as specified above.
-	-	-	-	-	The <code>libmaprc</code> file is never used; instead the <code>cds.lib</code> file, which is internally generated by the <i>irun</i> tool, is used.

Analyzing into Multiple Logical Libraries Using *irun* in Single- and Multi-Step Modes

irun supports design compilation into multiple logical libraries with both single-step and multi-step compilation processes.

Single-Step Compilation Process

Following are the examples of single-step compilation:

- One step compilation and simulation (with compilation into a logical library)

```
irun -hw -makelib Lib1 Dut.v -endlib Tb.sv
```
- One step compilation and simulation (with compilation into a logical library with explicit logical name)

```
irun -hw -makelib ./Chip/Dut:Lib1 Dut.v -endlib Tb.sv
```

Multi-Step Compilation Process

In the multi-step process, specify the logical library name using the `-reflib` option in the elaboration step. This is consistent with IES and IXCOM compilation use models for elaborating design units compiled into different logical libraries. Following is an example of compilation and elaboration using multiple steps:

```
irun -hw -compile -makelib ./Chip/Dut:Lib1 Dut.v -endlib
irun -hw -compile -makelib ./Chip/Sub:Lib2 Sub.v -endlib
irun -hw -elaborate Tb.sv -reflib Lib1 -reflib Lib2 -top Tb
```

Cadence recommends that you use the `-makelib` and `-reflib` options on the *irun* command line to compile and elaborate the design.

Note: IES limitations apply for options specified within the `-makelib` and `-endlib` options on the *irun* command line. For IES limitations and/or unsupported options when using this use model, use the multi-step compilation process as described above.

Simulating Designs

This section provides information on simulating a hardware-acceleratable design using *irun*. A prerequisite for simulating such a design is compilation using the `-hw` option with the *irun* command.

Simulating Using *ncsim*

By default, *irun* invokes *ncsim* to simulate the IXCOM-generated snapshot. This behavior is similar to pure IES-based simulation. When using *irun* for simulation acceleration, use the following steps to invoke *ncsim* as the run-time and debug environment:

1. Compile the design using `-hw` option with the *irun* command:

```
irun -hw [<ixcom_options>] [<ies_options>] <source_files>
```

Alternatively, use two-step compilation, as described in [Multi-Step Compilation and Simulation](#) on page 361.

2. Simulate the design:

```
irun -R [-gui]
```

For more information, see [Invoking *irun* Tool \(for SA Mode\)](#) on page 1409.

Simulating Using xeDebug

xeDebug provides a unified access to testbench in software and design mapped to the hardware. A unified interface eases transitioning from software simulation to the simulation acceleration mode. Minor modifications are required to the native simulation scripts, mostly to handle swap in and swap out of the design to and from emulator.

To ensure *irun* invokes xeDebug as the run-time and debug environment, specify the `-xeDebug` option with the *irun* run-time command. For example:

1. To compile the design, use the following command:

```
irun -hw [<ixcom_options>] [<ies_options>] <source_files>
```

2. To simulate the design, use the following command:

```
irun -R -xeDebug [-gui]
```

xeDebug is invoked as the run time. When the `-gui` option is specified, two sets of windows are displayed, one for SimVision and the other for xeDebug.

Note: You can still use the standard method of invoking the xeDebug tool for running and debugging a simulation snapshot. For more information, see [Invoking xeDebug Tool](#) on page 1408. For general information about xeDebug, see [Debugging Designs on Palladium XP](#) on page 865. Also see the [Limitations](#) section of this chapter.

Compilation and Run-Time Logs

irun collects standard output from the IXCOM tools and xeDebug to the default *irun.log* file. Alternatively, you can specify a log file of your own using the `-l <logfile>` option.

You can redirect the output from *vlan*, *vhan*, and *ixcom* to a separate log file using the `-log_<tool_name> <file_name>` option.

To append the output to an existing log file, use the existing *irun* option `-append_log`.

Limitations

- The Hardware Model Interface (HMI) use model is not supported.
- Incremental compilation support is limited to the IES portion of the *irun* compilation. Thus, *irun* does not support incremental compilation of design files (or database) compiled using *ixcom*.
- IXCOM supports only `-top <design_unit_name>` and `-top <library>.<design_unit_name>` for the `-top` option. IES tools support more sub-options. See INCISIV 11.2 documentation for more information.
- xeDebug user sessions are not supported. In the offline debugging mode, you need to start xeDebug directly.
- The Universal Verification Methodology (UVM) use model is not supported.

Compiling Designs with SystemVerilog Constructs in SA Mode

This release natively supports mapping HDL written in IEEE 1800-2005 SystemVerilog to the hardware. The constructs of the SystemVerilog language are divided into the following categories for the purposes of compilation and hardware mapping:

- SystemVerilog Design Constructs
- SystemVerilog Testbench Constructs (SVTB)

The constructs that are not supported are listed in Support for SystemVerilog Constructs on page 1149.



The content of this chapter pertains only to the SA mode.

SystemVerilog Design Constructs

IXCOM partitions the design into two partitions, one that can be run on the emulator, and the other that cannot be mapped to hardware. Any constructs that cannot be mapped to the emulator are partitioned on the software side with the testbench. During acceleration, the design modules that are in the testbench, execute on irun while the modules that are mapped to the emulator are run on emulator. For support of the SystemVerilog Design constructs by IXCOM, use the `+sv` option of the `ixcom` command. Alternatively, you can use the `vlan` executable to analyze the SystemVerilog design constructs, which can be, then, handled by IXCOM.

The SystemVerilog design needs to be compiled using the following steps.

Compiling a Pure SystemVerilog Design

1. Analyze and compile the SystemVerilog Design constructs using the following command:

```
ixcom +sv <module1> <module2>... +dut+<dut_module> -ua
```

Alternatively, use the following command sequence to analyze and compile the design:

- a. Use `vlan` to analyze the design containing SystemVerilog constructs:

```
vlan +sv <SystemVerilog_files> <vlan options>
```

- b. Compile the SystemVerilog Design constructs using the `ixcom` command:

```
ixcom -top <top_modulename> +dut+<dut_module> -ua
```

Compiling a Mixed-Language Design

1. Compile the design written in the various languages, such as VHDL, SystemVerilog, and Verilog using the following language compilers:

- c. For VHDL designs, use the following command:

```
vhan <VHDL primary or secondary unit> <vhan options>
```

- d. For SystemVerilog designs, use the following command:

```
vlan +sv <SystemVerilog_files> <vlan options>
```

2. Compile the SystemVerilog design constructs using the `ixcom` command:

```
ixcom -top <top_module> +dut+<dut_module> -ua
```

For a list of unsupported SystemVerilog design constructs, refer to [Support for SystemVerilog Design Constructs](#) on page 1157.

SystemVerilog Testbench Constructs (SVTB)

Certain SystemVerilog constructs, which are typically a part of the testbench, need to be separately linked by using the `irun`, and these cannot be directly handled by IXCOM. If these constructs are passed directly to the IXCOM compiler, it generates an error message to indicate that these constructs are unsupported. Most of these constructs belong to the testbench, such as SystemVerilog Classes, randomization and so on. The list of SVTB constructs is provided in [Support for SVTB Constructs](#) on page 1149.

Compile the testbench modules that contain these SVTB constructs with `irun`, and compile the DUT (that is, the RTL) with the `+xcDesignTop` option of the `ixcom` command. For more information on the `+xcDesignTop` option, refer to [Linking DUT with Multiple Testbenches](#) on page 229.

Compiling SVTB Constructs Supported by IXCOM

The SVTB constructs need to be compiled using the following process:

Compiling a Pure SystemVerilog Design with SVTB Constructs

1. Analyze and compile the SystemVerilog SVTB constructs using the following command:

```
ixcom +sv <module1> <module2> +dut+<dut_module> \  
+xcDesignTop+<hier_ref_to_DUT_instance_in_testbench>=<DUT_module_name> \  
-ua
```

Alternatively, use the following command sequence to analyze and compile the design:

- a. Use `vlan` to analyze the design containing SystemVerilog constructs:

```
vlan +sv <SystemVerilog_files> <vlan_options>
```

- b. Compile the SystemVerilog Design constructs using the `ixcom` command:

```
ixcom -top <top_modulename> +dut+<dut_module> \  
+xcDesignTop+<hier_ref_to_DUT_instance_in_testbench>=<DUT_module_nam  
e> \  
-ua
```

Compiling a Mixed-Language Design with SVTB Constructs

1. For mixed language designs, compile the design written in the various languages, such as VHDL, SystemVerilog, and Verilog by using the language compiler for each language

- c. Compile VHDL primary and secondary units using the following command:

```
vhan <vhdl_primary_or_secondary_unit> [vhan_options]
```

- d. Compile SystemVerilog and Verilog modules using the following command:

```
vlan +sv <SystemVerilog_files> [vlan_options]
```

2. Compile using IXCOM to generate the compilation databases.

```
ixcom -top <top_level_module> -top <another_top_level_module> \  
+xcDesignTop+<hier_ref_to_DUT_inst_in_tb>=<DUT_mod_name> ...
```

Support for Covergroups and Mailbox Constructs

IXCOM supports the following SystemVerilog testbench constructs:

- Support for Covergroups
- Support for Mailbox

Support for Covergroups

Covergroups are supported in the testbench and DUT. Refer to [Coverage Support](#) on page 395 for more information.

Support for Mailbox

Mailbox is implemented as a template class in SystemVerilog package implementation. The standard package is imported into a design when the `+sv` option is used.

Following datatypes are supported for mailbox:

- `shortint`, `int`, `longint`, `byte`, `integer`, `bit`, `logic`, `reg`
- class handles
- packed arrays of types `bit`, `logic`, and `reg`
- typedefs of the above datatypes

Compile the modules that contain these SVTB constructs by using the `+xcDesignTop` option. For more information on the `+xcDesignTop` option, refer to [Linking DUT with Multiple Testbenches](#) on page 229.

For a list of supported SVTB constructs, refer to [Support for SVTB Constructs](#) on page 1149.

Support for SystemVerilog Interface Constructs

IXCOM supports SystemVerilog interfaces as SVTB constructs (in the software partition) and design constructs (in the hardware partition or the DUT). For a list of supported SystemVerilog interface constructs, refer to [Support for SVTB Constructs](#) on page 1149 and [Support for SystemVerilog Design Constructs](#) on page 1157.

When a SystemVerilog interface is used as a design construct, reference to an interface object through a port connection is limited to:

- net or variable of integral datatype or packed one-dimensional array data type
- task or function
- interface instance

Any interface port connections that cross the software (testbench) and hardware (DUT) partition are not supported, and compilation is aborted with an error.

When the SystemVerilog interfaces are used as design constructs, unsupported references to interface objects or port connections are checked during RTL synthesis (controlled by the `-rtlchk` option). For more information on the design partitioning using RTL synthesis checks, refer to [Compiling for Hardware Using IXCOT](#) on page 202.



The current implementation does not warn or issue error for the following interface object references, and can cause simulation mismatch in some cases:

- ☐ Hierarchical reference to interface objects
- ☐ Multiple writes to an interface object from different modules

For steps to compile SystemVerilog interfaces as SVTB constructs, refer to [Compiling SVTB Constructs Supported by IXCOT](#). For steps to compile SystemVerilog interfaces as design constructs, refer to [SystemVerilog Design Constructs](#).

Support for SystemVerilog Packages in SA Mode

SystemVerilog packages are declared using the `package` keyword. An example of a SystemVerilog package declaration is given below.

```
package myPkg;  
    typedef int myType;  
    myType myVar;  
endpackage
```

Objects declared within a SystemVerilog package can be referenced using one of the following ways:

- Explicitly by using the `<pkg_name>::<object_name>` syntax.
- Implicitly by using the `<object_name>` variable, after the `<object_name>` variable is made visible in a given scope through a package `import` declaration.

```
import <pkg_name>::<object_name>;
```

Or,

```
import <pkg_name>::*;
```

Refer to the SystemVerilog LRM for further details about these declaration methods.

The SystemVerilog package name has a global namespace, which is different from the namespace for the name of a module, interface, or program declaration. This means that two

SystemVerilog packages should not have the same name, but a SystemVerilog package can have the same name as a module (or interface or program) declaration.

Note: As per the LRM, the following references are not allowed for SystemVerilog packages:

- Forward references to SystemVerilog package objects (that is referring to the package before the SystemVerilog package declaration)
- References to Compilation Unit Scope objects from within SystemVerilog packages.

The IXCOM compiler supports the SystemVerilog package declaration as well as all the forms of references to SystemVerilog package objects. It also supports package import statements written as a part of the module header as per Section 23.2 of the 1800-2009 SystemVerilog LRM.

Use the `+showSvPkgAccess` option of the `ixcom` (when using single-step compilation) or `vlan` (when using multi-step compilation) command to enable display of compilation-time information messages to show the package objects being accessed.

The following INFO message is displayed for every access to SystemVerilog package object:

```
Info in file test.sv at line 9           [FOUND_SV_PKG_ACCESS]
    Access to a SystemVerilog package object found
        (pkg: pkg1, obj: pkg1Type, file: test.sv, line: 5)
```

The IXCOM compiler supports references to a SystemVerilog package that was not compiled in the current compilation session (that is, it was part of a previous `vlan` command). The SystemVerilog package being referenced could have been compiled into the same logical library or a different logical library. Refer to [“Support for Packages Referenced Across Multiple Logical Libraries”](#) on page 387.

Accessing SystemVerilog Package Objects from Library Modules

Access to SystemVerilog packages that are declared in the source HDL is allowed from modules that are compiled from library files (using the `-v` option of the `vlan` or `ixcom` command). However, the SystemVerilog package declarations contained within library files are ignored, and, therefore, these cannot be accessed from modules that were compiled from library files.

Consider the following example:

```
// file1.sv
package p1;
    typedef int myInt1;
```

UXE User Guide

Compiling Designs with SystemVerilog Constructs in SA Mode

```
endpackage
```

```
// Note: Compiling the file2.sv as a -v library file as shown below in the ixcom  
// command causes a problem.
```

```
// file2.sv
```

```
package p2;  
    typedef int myInt2;  
endpackage
```

```
module top;  
    dut DUT();  
endmodule
```

```
module dut;  
    p1::myInt1 v1;          // OK  
    p2::myInt2 v2;          // Not OK : reference to myInt2  
endmodule
```

To compile the above example, the following command is used:

```
ixcom -clean file1.sv -v file2.sv
```

In the above example, the module `dut` is picked from the library file `file2.sv`. The package `p2` is declared in a library file and, hence, it is not seen by the IXCOM compiler. Subsequently, any reference to package `p2` will result in an error. References to the packages declared in source files (that is, package `p1`) are allowed from modules declared in library files (that is, module `dut`).

List of Supported Constructs

The following table lists the SystemVerilog package constructs supported by the INCISIV compiler, the Palladium HDL Compiler, and the IXCOM compiler according to the following legend:

- ☐ **Yes:** Yes, this feature is supported.
- ☐ **No:** No, this feature is not supported.

Note: The constructs that are not supported by the IXCOM compiler, but are supported by the INCISIV compiler can be separately compiled as a part of the testbench by using the INCISIV compiler. For this, use the `+xcDesignTop` option of the `ixcom` command. Refer to the section titled [SystemVerilog Testbench Constructs \(SVTB\)](#) on page 380 for details about separate compilation of testbench.

Table 6-1 Support for SystemVerilog Package Constructs by Various Compilers

Package Constructs	INCISIV Compiler	Palladium HDL Compiler	IXCOM Compiler
net_declaration	Yes	Yes	Yes
data_declaration	Yes	Yes	Yes
task_declaration	Yes	Yes	Yes
function_declaration	Yes	Yes	Yes
checker_declaration	No	No	No
dpi_import	Yes	No	Yes
dpi_export	Yes	No	Yes, but in testbench only
extern_constraint_declaration	No	No	No
class_declaration	Yes	No	No
class_constructor_declaration	Yes	No	No
local_parameter_declaration	Yes	Yes	Yes
parameter_declaration	Yes	Yes	Yes
covergroup_declaration	No	No	No
overload_declaration	No	No	No
property_declaration	Yes	No	Yes
sequence_declaration	Yes	No	Yes
let_declaration		No	No
anonymous_program	No	No	No
package_export_declaration	No	No	No
timeunits_declaration	Yes	Yes	Yes

Support for SVA and PSL Assertions with SystemVerilog Packages

Property and Sequence declarations in SystemVerilog packages and references to these by modules in the hardware partition is supported.

Support for Coverage Constructs with SystemVerilog Packages

Code coverage and functional coverage constructs are not supported with SystemVerilog packages.

Support for DPI Declarations with SystemVerilog Packages

DPI import and DPI export declarations contained within a SystemVerilog package are supported by the IXCOM compiler. However, only DPI import declarations within a SystemVerilog package can be called by the modules in the hardware partition.

DPI import declarations contained within a SystemVerilog package are supported in the hardware partition. Therefore, DPI import tasks and functions declared within a SystemVerilog package can be called from modules within the hardware partition.

DPI export declarations contained within a SystemVerilog package are not supported in the hardware partition. So, DPI export tasks and functions declared within a SystemVerilog package are marked using the `translate_on` and `translate_off` pragmas, and those modules in the hardware partition that call the DPI export tasks or functions declared in the SystemVerilog package, are partitioned to the software side.

Support for Packages Referenced Across Multiple Logical Libraries

The IXCOM compiler supports module references to packages that were analyzed in a logical library different from the one in which the modules were analyzed.

Consider the following example.

```
//pkg.v
package pkg
    typedef int MYINT;
endpackage

//top.v
module top;
    reg clk,a;
    dut dutInst(clk,a);
endmodule

//dut.v
module dut (input reg clk, a);
    import pkg::MYINT;
```

UXE User Guide

Compiling Designs with SystemVerilog Constructs in SA Mode

```
MYINT b;  
endmodule
```

The above package and module can be analyzed and compiled into multiple logical libraries as follows:

```
vlan +sv -work L1 pkg.v top.v  
vlan +sv -work L2 dut.v
```

To resolve the package reference, the vlan executable first searches the current logical library for the package referenced in the module. If it does not find the package, it searches other logical libraries specified in the libmaprc file(s).

If multiple packages are found, an error message is issued because the LRM states that the packages must have unique names.

If the referenced package cannot be found, an error message is issued because the LRM states that the packages to be compiled before being referenced.

Handling of Re-declaration of Packages

You cannot analyze two *different* packages with the same name into the same logical library. The *same* package can, however, be re-analyzed. Two packages of the same name, or the same package) can be analyzed into two different logical libraries. However, the IXCOM compiler generates an error if the design is found to have references to two *different* packages with the same name. Note that the IXCOM compiler allows multiple definitions of the *same* package.

Refer to Definition of Same Package as per vlan and Definition of Same Package as per the IXCOM Compiler sections for the definition and handling of *same* package.

If your compilation script results in the *same* package being specified in multiple `vlan` sessions, all subsequent compilations of the *same* package (in the same logical library) are ignored, and `vlan` re-uses only the original definition.

Definition of Same Package as per vlan

During analysis, `vlan` uses the following algorithm to identify if the package being analyzed is the same as a previously analyzed package in the same logical library:

Package Definitions Being Compared	Same or Different
If the HDL filename containing the previously analyzed package definition is <i>different</i> from the The HDL filename containing the package definition being currently analyzed	Packages are considered to be <i>Different</i> and an error message is issued.
If the location of the HDL filename containing the previously analyzed package definition is <i>different</i> from the The location of the HDL filename containing the package definition being currently analyzed Note that <code>vlan</code> traverses symbolic links to identify if it was the same file.	Packages are considered to be <i>Different</i> and an error message is issued.
If the last-modified time of the HDL file (containing the package definition being currently analyzed) Is more recent than The created time of the database file (with the previous definition of the package in the same logical library)	The previous definition is considered <i>outdated</i> and <code>vlan</code> issues an error message.
If the last-modified time of the HDL file (containing the package definition being currently analyzed) Is older than The created time of the database file (with the previous definition of the package in the same logical library), then the two package definitions are now considered <i>same</i> .	Packages are considered to be <i>same</i> .

Definition of Same Package as per the IXCOM Compiler

During compilation, the IXCOM compiler uses the following algorithm to identify if the two analyzed definitions of a package with the same name in different logical libraries are for the *same* package:

Package Definitions Being Compared	Same or Different
If the HDL filename stored in the two package analyzed databases is <i>different</i>	Packages are considered to be <i>Different</i> and an error message is issued.
If the location of the HDL filename stored in the two package analyzed databases is <i>different</i> Note that IXCOM compiler traverses symbolic links to identify if it was the same file.	Packages are considered to be <i>Different</i> and an error message is issued.
All other packages are considered to be <i>same</i> .	

Handling of Stale Package References

The IXCOM compiler ensures that the module that is referencing a package is analyzed *after* the package is analyzed. This ensures database consistency. If it is found that the package has been re-analyzed at a later time, and the referencing module has not been re-analyzed, the IXCOM compiler issues an error message.

Partitioning of SystemVerilog Packages by the IXCOM Compiler

The IXCOM compiler determines the partition (hardware or software) for the SystemVerilog packages. By default, all SystemVerilog packages get partitioned to the software partition. Then, the IXCOM compiler moves the following types of SystemVerilog packages to the hardware partition:

- ❑ SystemVerilog packages that are referenced by modules in the hardware partition
- ❑ SystemVerilog packages that are referenced by other SystemVerilog packages in the hardware partition

All non-synthesizeable constructs that are contained within the SystemVerilog packages that are partitioned to the hardware, are marked with the `translate_on` and `translate_off` pragma. If a module (in the hardware partition) references a non-synthesizeable SystemVerilog package construct, it is partitioned to the software side.

Support for Compilation Unit Scopes in SA Mode

The SystemVerilog language allows objects to be declared outside of the module, interface, program, or package declaration. The scope for such objects is called the **Compilation Unit Scope**. In accordance with the LRM, the IXCOT compiler decides the boundary of the *Compilation Unit Scope*. It also mirrors the IES behavior.

Use the `+showSvPkgAccess` option of the `ixcom` (when using single-step compilation) or `vlan` (when using multi-step compilation) command to enable display of information messages when compilation unit scope objects are referenced.

In single-step compilation (only the `ixcom` command), IXCOT creates just one *Compilation Unit Scope*. In multi-step compilation (`vlan`, `vhan`, and `ixcom` commands), IXCOT creates a separate *Compilation Unit Scope* for each `vlan` session that you invoke. IXCOT denotes the beginning of the first file passed to the `vlan` session as the starting point and the end of the last file passed to the same `vlan` session as the ending point of the *Compilation Unit Scope* corresponding to that `vlan` session.

This means that a design can have multiple *Compilation Unit Scopes* - one for each `vlan` session. The IXCOT mirrors the IES implementation in which a separate *Compilation Unit Scope* is created for each `ncvlog` session.

Objects declared within a *Compilation Unit Scope* are visible to all module, interface, program, and/or package declarations contained within that *Compilation Unit Scope*. These objects can be referenced directly by using `<object_name>` or explicitly by using `$unit::<object_name>`. The explicit form of reference is needed when an object is declared in the local scope with the same name as the *Compilation Unit Scope* object, thus hiding the visibility of the *Compilation Unit Scope* object.

Consider the following example:

```
// file test.sv
parameter int myVar = 32;

module top;
    int myVar;
    initial
        $display("The value of parameter myVar is %h", $unit::myVar);
endmodule
```

Objects declared within a *Compilation Unit Scope* are **NOT** visible outside of it. This means that the *Compilation Unit Scope* objects declared in one `vlan` session cannot be syntactically accessed in another `vlan` session.

The Palladium HDL Compiler supports a single *Compilation Unit Scope* as starting at the beginning of the first file in the `vavlog` session and ending at the end of the last file in the same `vavlog` session. Only one `vavlog` session can have objects declared at the *Compilation Unit Scopes* level.

Each `vlan` session has its own distinct *Compilation Unit Scope*. Therefore, there is no notion of a *Compilation Unit Scope* being *re-declared*.

Access to Compilation Unit Scope from Library Modules

Just like with SystemVerilog packages, access to *Compilation Unit Scope* objects that are declared in the source HDL is allowed from modules that are compiled from library files (using the `-v` option of the `vlan` or `ixcom` command). However, *Compilation Unit Scope* objects contained within the library files are ignored, and consequently, there is no access to such *Compilation Unit Scope* objects from modules that are compiled from library files.

Supported Constructs

The list of supported constructs in the *Compilation Unit Scope* is the same as the list of supported constructs in the SystemVerilog package. Refer to the [Support for SystemVerilog Package Constructs by Various Compilers](#) table.

Assertions (SVA and PSL)

Just like with SystemVerilog packages, Property and Sequence declarations in the *Compilation Unit Scope* and references to them by modules in the hardware partition are supported.

Coverage

Code Coverage and Functional Coverage features are not supported with *Compilation Unit Scopes*.

DPI Declarations

Just like SystemVerilog packages, DPI import and DPI export declarations contained within a *Compilation Unit Scope* are supported by the IXCOM compiler. However, just like with SystemVerilog packages, only DPI import declarations within a *Compilation Unit Scope* can be called by the modules in the hardware partition.

UXE User Guide

Compiling Designs with SystemVerilog Constructs in SA Mode

During partitioning, the *Compilation Unit Scopes* are treated in the same manner as SystemVerilog packages.

UXE User Guide

Compiling Designs with SystemVerilog Constructs in SA Mode

Coverage Support

Coverage is defined as the percentage of verification objectives that have been met. It is used as a metric for evaluating the progress of a verification project in order to reduce the number of simulation cycles spent in verifying a design.

Coverage metrics in verification fall under two categories:

- ❑ Code coverage: those that can be automatically extracted from the design code.

In this chapter, the code coverage support is detailed in [Code Coverage](#) on page 397.

- ❑ Functional coverage: those that are user-specified in order to tie the verification environment to the design intent or functionality.

In this chapter, the functional coverage support is detailed in [Functional Coverage Support with Palladium XP](#) on page 403.

The IEEE 1800 - 2009 standard for SystemVerilog provides a detailed description of these terms and their differences.

 Important

The content of this chapter is applicable only for the designs compiled using the IXCOM compiler. Details of using the IXCOM compiler are provided in [Chapter 4, “Compiling Designs for Simulation Acceleration Using IXCOM.”](#)

INCISIV Versions for Coverage Features

The coverage features are tightly integrated with the Incisive simulator. As a result, certain features are not available in the older IES releases. The matrix below is used to indicate such features and the base version of INCISIV that is required to enable those features.

Coverage	Feature	IES 11.1	IES 12.1	IES 12.2
Functional Coverage	Non-Uniform distribution of fixed bins	No	Yes	Yes
Toggle and Block Coverage	Unicov Database Using IMC Tool for Reporting Coverage Data	No	No	Yes

Code Coverage

In code coverage, the information monitored is inferred from the design. The IEEE standard focuses only on functional coverage. Therefore, in the absence of an industry standard for code coverage, the Cadence's Incisive Comprehensive Coverage (ICC) solution is used as a basis. With respect to the ICC definitions, Palladium XP supports toggle and block forms of code coverage.



It is necessary to use the IES 12.2 version for the code coverage support described in this section. If older versions of IES are used, the coverage database will not be created, and an error message will be issued at run time.

Palladium XP supports the following types of code coverage:

- ❑ **Toggle Coverage** - Provides information about the change (toggle) of signals and ports, during a simulation run. It measures activity in the design, such as unused signals, signals that remain constant, or signals that have too few value changes. This coverage metric tracks the 0->1 and 1->0 transitions of signals and ports.
- ❑ **Block Coverage** - Identifies the lines of code that get executed during a simulation run. It helps you determine if the various testbenches exercise the statements in a block. This coverage metric tracks the HDL blocks that have been executed.

The block coverage support with Palladium XP is detailed in [Toggle Coverage Support with Palladium XP](#) on page 398, and toggle coverage support with Palladium XP is described in [Toggle Coverage Support with Palladium XP](#) on page 398.

For more information on the other types of code coverage, refer to the following chapters of the *ICC User Guide* available in the INCISIV documentation set:

- *Chapter 2, “Block, Branch, and Statement Coverage”* for an overview of Block Coverage in ICC.
- *Chapter 4, “Toggle Coverage”* for an overview of Toggle Coverage in ICC.

Example: Code Coverage

Refer to the example provided in the `<$UXE_RELEASE>/gift/tutorial/code_coverage` directory to see how designs with code coverage are compiled and run on Palladium XP, and for a sample of the coverage reports.

Code Coverage Limited to DUT on Palladium XP

The primary purpose of code coverage is to ensure that there are no logically unreachable or untested areas of the design. In UXE, code coverage applies only to the design units that are targeted for the emulator. The design units that comprise the testbench or extracted hierarchies are not candidates for code coverage.

Note: If required, you can use the `+xcDesignTop` option of the `ixcom` command and compile the testbench to run in a separate simulation environment. Refer to the section titled Compiling Top-Level Testbenches Using `+xcDesignTop` on page 230 for details.

Toggle Coverage Support with Palladium XP

Toggle coverage is used to track the changes in the value of a signal from 0 to 1 or from 1 to 0.

■ For toggle coverage, the following terms have a specific meaning:

- ❑ Track - indicates that some HDL variable will be monitored at run time and included in the coverage database and coverage report.
- ❑ Scored - is the act of marking a coverage item as being covered.

Toggle coverage is only tracked for the following:

- ❑ Verilog (scalar and vector) nets, registers, and ports
- ❑ VHDL boolean (and vector versions)
- ❑ VHDL `std_ulogic`, `std_logic` (and vector versions)

The following items are not scored during toggle coverage on Palladium XP, and no messages are issued for these items:

- ❑ VHDL bit and vector versions
- ❑ VHDL records

Note: IES supports toggle coverage of the above two types.

- ❑ Variables that are not at the module scope (generates, named blocks, and so on)
- ❑ Real numbers or integers
- ❑ `supply0` and `supply1` nets
- ❑ SystemVerilog-specific data types that do not directly map into the supported subset mentioned above

- ❑ SystemVerilog aggregate types - struct, union, and so on.
- ❑ SystemVerilog package variables
- ❑ SystemVerilog interfaces
- ❑ Items within ``gtrefcells` primitives

For a given simulation run, the number of counter bits is fixed to one for code coverage items. In other words, if a signal toggles from 0 to 1 seven times in a given simulation, the actual count will reflect only one toggle. For toggle coverage, there are three one-bit counters: `rise`, `fall`, and `full`.

Differences in Toggle Coverage Support on Palladium XP with Respect to IES

Certain differences exist in coverage because IES, which is an event-based simulator and the Palladium XP, which is a cycle-based emulator, do not behave in a similar fashion. Following are the scenarios where Palladium XP coverage results might differ from IES.

■ Scenario 1: Transitions to X or Z Values can Result in a Toggle on Palladium XP Emulator

It is a good practice to resolve all the `x` and `z` values before swapping to the emulator. The `xc` command set allows you to specify how `X` and `Z` values should be handled (`xc xt0`, `xc zt0`, `xt1`, `zt1`). The conversion from `x` and `z` values to a constant 1 or 0 does not cause a toggle at swap time, but it does result in a toggle the next time that the signal changes. Take the following waveform as an example:



In the above waveform, the top `en` signal represents a simulation run where `en` remains un-initialized in a simulation environment. At time 20ns, `en` is driven to a 1. In a software simulator, the transition at time 20ns does not count as a rising toggle. The bottom `en` signal represents a Palladium XP run where the design is swapped into hardware at time 10ns. When the design is swapped to the hardware mode, all `x` values that you specify are converted to 0 on the emulator. As a result, the transition at time 20ns is seen as a rising toggle on Palladium XP. To avoid this behavior, try to delay the swap to the hardware (emulator) until all the `x` and `z` values have been resolved.

This situation might also happen when a bus signal is driven to an `x` or `z` value in pure simulation due to a multi-driver bus contention. On the emulator, the multi-bus-driver conflict does not resolve the value to `x`.

Note: Avoid using the IES configuration switches set like `set_toggle_includex|z`, to get IES to match Palladium XP. These switches are not intended to match IES runs with Palladium XP.

■ **Scenario 2: Event-based Glitches Result in a Toggle in Simulation**

Due to the cyclic nature of the Palladium XP hardware, 0-delay glitches do not result in a toggle. In an event-based simulator, these glitches are treated as toggles.

Note: Some features of Palladium XP allow certain logic to run in a more behavioral fashion. When these scenarios occur, some glitches are counted as a toggle. Common examples of this include enabling the behavioral compiler, driving the design through DPI calls, and so on. When these features are used, Palladium XP does not regard anything as being toggled if it was not toggled in the simulator. In other words, these will not result in false positives.

IES provides a configuration option called `set_glitch_strobe` that can be used to define a glitch window where toggles should be ignored. If glitching is a problem in your simulation runs, use this option to get a more Palladium-XP-like behavior. However, due to the different nature of these engines, this option cannot be used to guarantee an exact match with hardware.

Block Coverage Support with Palladium XP

Block coverage is used to track the HDL blocks in the code that have been executed and the ones that have not, and it identifies whether or not the test scenarios exercise the statements in a block. A block is a statement or sequence of statements in Verilog or VHDL that executes with no branches or delays. Either none or all of the statements in a block are executed.

This section describes a mechanism to gather and report block coverage statistics using Palladium XP.

Block coverage on Palladium XP emulator supports only sequential blocks for coverage in both Verilog and VHDL. A block is considered sequential if it falls into one of the following categories:

- ❑ A process with a single clock event that is edge sensitive.
- ❑ A process with multiple clock events, where one event is the true clocking event and all other events are asynchronous reset events.

Note: All clocking events must be edge sensitive.

The blocks or processes that do not meet these criteria are ignored for block coverage without warning.

Specifically, on Palladium XP, block coverage is tracked only for the following:

- Verilog `always` blocks (sequential only)
- VHDL `process` and `generate` statements
- Verilog and VHDL processes within `generate if`, `for`, and `case` blocks
- Verilog and VHDL explicit case default

The following items are not scored during block coverage on Palladium XP, and no messages are issued for these items:

- Verilog `initial` blocks, `final` blocks, tasks, and functions
- Non-sequential Verilog `always` blocks
- Statements in a TBcall region
- VHDL concurrent signal assignments
- Functions and procedures in VHDL subprograms inside packages
- Gate-level or user-defined primitives

- Protected Verilog and VHDL statements
- Event constructs @, or, and event (**Note:** Supported by IES with the `-coverage block` option.)

For a given simulation run, the number of counter bits is fixed to one for code coverage items. In other words, if a signal toggles from 0 to 1 seven times in a given simulation, the actual count will reflect only one toggle. For block coverage, each block is represented by a one-bit counter that is set when the block is executed.

Differences in Block Coverage Support on Palladium XP with Respect to IES

Certain differences exist in coverage support provided by Palladium XP and IES because IES, which is an event-based simulator and the Palladium XP, which is a cycle-based emulator, do not behave in a similar fashion.

IES supports block coverage of the following types:

- ☐ Verilog/VHDL implicit else
- ☐ Verilog implicit case default
- ☐ Verilog continuous assignments as blocks
- ☐ Verilog ternary assignments and VHDL conditional and selected signal assignments
- ☐ Verilog and VHDL protected statements

Following are the scenarios where Palladium XP coverage results might differ from IES.

■ Scenario 1: Multiple Asynchronous Clocking Events on Palladium XP

Logically speaking, the asynchronous clocking events are evaluated in a specific order. However, this order is ignored on Palladium XP, when evaluating block coverage for asynchronous blocks.

Consider the following example:

```
always @(posedge clk or posedge r0 or posedge r1)
begin
    if(r0)
        q <= 0;
    else if(r1)
        q <= 0;
    else
```

```
        q <= d;  
    end
```

If `r0` is set to `1'b1` for all time, the block associated with `r1` should logically never fire. On Palladium XP, the block coverage for asynchronous regions is tied to the clocking events. Even if `r0 == 1'b1`, a `posedge r1` clocking event causes the block associated with `r1` to be considered as covered.

■ Scenario 2: Asynchronous Regions not Covered on Palladium XP

Like the previous scenario, the block coverage for an asynchronous region is tied to the clocking event associated with that region. If `r0`, in the above example, is initialized to `1'b1` as in the following code, and the value remains unchanged, the block associated with the `posedge r0` clocking event is not marked as covered, even if other clocking events wake up the process:

```
reg r0 = 1'b1;
```

Functional Coverage Support with Palladium XP

Functional coverage requires you to specify the information that needs to be tracked. Palladium XP supports aspects of functional coverage through SystemVerilog and PSL. Functional coverage helps identify the non-tested functionality in the design, and provides a view of verification completeness from a functional point of view. Functional coverage can be performed on user-defined coverage points, specified using PSL and SystemVerilog assertions, or covergroup statements. These coverage points specify scenarios, error cases, corner cases, and protocols to be covered and also specify analysis to be done on different values of a variable. By exercising the important areas of the design with the help of user-defined functional coverage points, you can optimize the testing effort. Functional coverage can either be control-oriented by using PSL or SVA statements or data-oriented by using SystemVerilog covergroups.

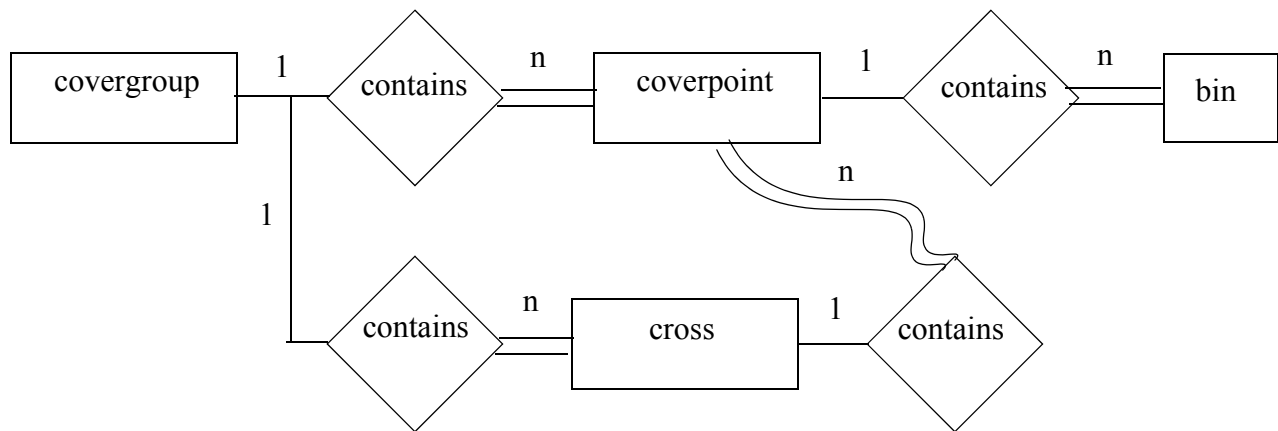
The IXCOM compiler supports functional coverage using assertions and covergroups.

This section describes the mechanisms to gather and interact with the functional coverage statistics for a SystemVerilog design by using the covergroup construct of SystemVerilog language. Covergroup support is based on the 2009 SystemVerilog LRM. All references to the SystemVerilog LRM in this chapter are specific to the 2009 version. For information on functional coverage support provided by IXCOM using assertions, refer to [Functional Coverage Support](#) on page 507.

The terms related to functional coverage support provided by SystemVerilog are defined below. This chapter uses the following terms:

- A *bin* is a value or set of values that have a corresponding hit counter. The counter increments when these values are sampled.
- A *coverpoint* is a collection of bins that relate to a particular expression.
- A *cross* is a collection of bins that represent the cross product of the referred coverpoints.
- A *covergroup* is a collection of coverpoints, crosses, and the options that relate to them.

The following entity-relationship diagram explains how the bin, coverpoint, cross, and covergroup relate to each other. For more information, refer to the *IEEE-1800-2009 SystemVerilog Language Reference Manual*.



Covergroup Support in IXCOM

The SystemVerilog functional coverage support for IXCOM is a subset of the covergroup support provided by the base IES release. IXCOM supports functional coverage for SystemVerilog designs as well as mixed-language designs and references. IXCOM supports the use of covergroups in the LA mode.

The following sections summarize support information working from the inner-most level, the bin, to the outer-most level, the covergroup. Additionally, support information for where and how a covergroup can be instantiated has also been provided.

UXE User Guide

Coverage Support

Bin Support

The following table lists the different types of bins and support level provided by IXCOM. See the *Functional coverage* chapters of the *SystemVerilog LRM* and the Cadence document *ICC User Guide* in the INCISIV documentation for more information about bins.

Name	Description	Example	Supported
Scalar	Single bin for a set	<pre>bins a = {5, 6}; bins trana = (0 => 1);</pre>	Yes
Vector	One bin per value in set	<pre>bins b[] = {[0:9]}; bins tranb[] = (0 => 1), (2 => 3);</pre>	Yes
Fixed	Distribute set to fixed number of bins	<pre>bins c[2] = {[0:9]};</pre>	Yes
Automatic	All possible values are distributed to auto_bin_max number of bins	N/A, but implicitly equivalent to: <pre>bins auto[auto_bin_max] = {[\$: \$]};</pre>	Yes, for evenly distributed bins
Transition	Sequence drives the bin	<pre>bins d = {1 => 2 => 3};</pre>	Yes, see Considerations for Bins
Default	Convenient way to group all non-specified values	<pre>bins e = default;</pre>	Yes, see Considerations for Bins
Default Sequence	Convenient way to group all non-specified sequences	<pre>bins f = default_sequence;</pre>	No
Wildcard	Convenient way to wildcard bit positions of a specified value	<pre>wildcard bins g = { 4'b11?? }; wildcard bins t = (2'b0? => 2'b1?);</pre>	Yes, see Considerations for Bins
Ignore	Mechanism to ignore values from coverage computation	<pre>ignore_bins h = {5};</pre>	Yes, state bins only.

UXE User Guide

Coverage Support

Name	Description	Example	Supported
Illegal	Mechanism to generate an error message if specified values are sampled during coverage sampling	<code>illegal_bins i = {5};</code>	Yes, state bins only.

Considerations for Bins

- The use of \$ as a range value is supported in the upper or lower bound of a range. Additionally, the non-standard usage of [\$:\$] is supported to indicate all valid values.
- *Default* bins are supported, but their counts are not tracked due to capacity reasons. The default bins, however, do not influence the coverage computation, according to the *SystemVerilog LRM*. The illegal default bins cause a simulation error.

Note: IES reports default bin counts as a separate item in the IMC report.

- Transition bins are supported except for:
 - ❑ `Illegal_bins`
 - ❑ `Ignore_bins`
 - ❑ `Default_sequence`
 - ❑ Wildcards with more than eight wildcard characters
 - ❑ When the transition sequences are too large and would use an excessive amount of capacity.

For the above-mentioned exceptions, a warning is issued and the entire covergroup is discarded.

- Formal argument to specify bin array index is not supported.
- `iff` expressions specified at the bin level are supported for state, transition, and wildcard bins. These are not supported for ignore, illegal and default bins, or default sequences. If an unsupported bin `iff` is encountered, the entire covergroup is discarded. Each line of the bin can have its own guard.

For example:

```
covergroup cg @ (posedge clk);  
  A: coverpoint expr1 {
```

```
bins scalar1 = { [0:3] } iff (en[0]);
bins scalar2 = { [2:5] } iff (en[1]);
bins vector1[] = { [4:7] } iff (en[2]);
bins vector2[] = { [8:9], 11 } iff (en[3]);
bins fixed1[2] = { 10, [12:13] } iff (~en[0]);
bins fixed2[2] = { [12:15] };
bins trana = (0 => 1), (2 => 3) iff (en[0]);
bins tranb[] = ( [0:1] => [1:2]) iff (en[1]);
}
B: coverpoint expr2 {
wildcard bins b_01 = {16'b????????????1?} iff (en[0]);
wildcard bins b_02 = {16'b????????????1??} iff (en[1]);
wildcard bins b_03 = {16'b????????????1???} iff (en[0]);
wildcard bins b_04 = {16'b????????????1????} iff (en[1]);
wildcard bins b_05 = {16'b????????????1?????} iff (en[0]);
wildcard bins b_06 = {16'b?????????1??????} iff (en[1]);
wildcard bins b_07 = {16'b?????????1???????} iff (en[0]);
}
endgroup // cg
```

- Error messages for illegal bins are not generated in LA mode.
- Wildcard bins are supported except for:
 - ❑ Vector or fixed bins
 - ❑ Transition bins with more than eight wildcard characters
 - ❑ The state bins that contain more than ten wildcard characters, and are part of a cross under some conditions.

See [Coverpoints Part of a Cross and Containing Wildcard Bins](#) section for more details.

Considerations for Coverpoints

- Coverpoints are supported on integral expressions, in compliance with the *SystemVerilog LRM*.
- You can explicitly state the name associated with a coverpoint. When a name is not explicitly supplied, the *SystemVerilog LRM* rules for naming coverpoints are used. When the LRM naming rules cannot be implemented, the name follows the IES convention, as described in the [Name Generation Algorithm for Coverpoint and Cross](#) on page 415 section.

- `iff` expressions specified at the coverpoint level are supported.

Considerations for Crosses

- Crosses are supported on coverpoints or variable identifiers, in compliance with the *SystemVerilog LRM*.
- You can explicitly state the name associated with a cross. When a name is not explicitly supplied, the LRM rules for naming crosses are used. When the LRM naming rules cannot be implemented, the name follows the IES convention as described in the [Name Generation Algorithm for Coverpoint and Cross](#) on page 415 section.
- `iff` expressions specified at the cross level are supported.
- The `select` expressions and conditions are supported.

Coverpoints Part of a Cross and Containing Wildcard Bins

If a coverpoint is part of a cross and contains wildcard bins, it is supported if one of the following is true:

- The number of state wildcard characters is ten or less or the number of transition wildcard characters is eight or less.
- The number of state wildcard characters is more than ten and:
 - ❑ There is at most one such coverpoint in the cross.
 - ❑ The number of non-wildcard coverpoints does not exceed 2048 bins.
 - ❑ There are at most 16 scalar wildcard bins.

For example:

```
covergroup cg @(posedge clk);
  option.auto_bin_max = 8192;
  A : coverpoint expr1 {
    ignore_bins b_toplevel = {4'h5, 4'h9, 4'hA, 4'hD, 4'hE};
  }
  B : coverpoint expr2 {
    wildcard bins b_00 = {16'b????????????1};
    wildcard bins b_01 = {16'b????????????1?};
    wildcard bins b_02 = {16'b????????????1??};
    wildcard bins b_03 = {16'b????????????1???};
    wildcard bins b_04 = {16'b????????????1????};
  }
```

```

wildcard bins b_05 = {16'b????????1?????};
wildcard bins b_06 = {16'b????????1?????};
wildcard bins b_07 = {16'b????????1?????};
wildcard bins b_08 = {16'b?????1????????};
wildcard bins b_09 = {16'b?????1????????};
wildcard bins b_10 = {16'b????1?????????};
wildcard bins b_11 = {16'b????1?????????};
wildcard bins b_12 = {16'b???1?????????};
wildcard bins b_13 = {16'b??1?????????};
wildcard bins b_14 = {16'b?1?????????};
wildcard bins b_15 = {16'b1?????????};
}
AxB : cross A, B;
endgroup // cg

```

Considerations for Covergroup Declarations

- If the total number of bins within a covergroup instance exceeds 2^{16} , a warning is issued and that covergroup instance is discarded.
- Most of the instance-specific coverage options (as listed in *Table 19-1 of the SystemVerilog LRM*) are supported, but must be specified in the covergroup declaration. The `get_inst_coverage` and `detect_overlap` options are not supported, and covergroups with these options are ignored with a warning.
- Covergroup type options (as listed in *Table 19-3 of the SystemVerilog LRM*) are supported, but must be specified in the covergroup declaration. The `goal`, `strobe`, and `merge_instances` type options are not supported, and covergroups with these options are ignored with a warning.
- Formal arguments for covergroup declarations are supported in compliance with the rules specified in the *Sections 13.5.2 and 19.3 of the SystemVerilog LRM*. For example, consider the following covergroup declaration:

```

covergroup CG (ref logic[3:0] X, input int C, string commentStr,
    input int low, input int high) @(posedge clk);
    coverpoint X
    {
        option.at_least = C;
        bins b1[] = {[low:high]};
    }
    option.comment = commentStr;
endgroup
logic[3:0] bus;

```

```
CG cg1 = new(bus, 3, "bottom values", 0, 7);
CG cg2 = new(bus, 2, "upper values", 7, 15);
```

Violations of the LRM rules generate errors at compilation time.

Additionally, the following types are not supported for the arguments:

- ❑ An element of an unpacked array
- ❑ A member of an unpacked structure
- ❑ A class property

Violations results in the offending covergroup instances to be discarded with a warning. Other supported instances of the same covergroup are, however, compiled.

For example, when compiling the following code within a design the covergroup instance, `cg3` is discarded with a warning:

```
logic[3:0] cnt[3:0];
CG cg3 = new(cnt[0], 3, "bottom values", 0, 7);
CG cg4 = new(bus, 3, "bottom values", 0, 7);
```

Coverage information for `cg4` is still created. Thus, type coverage summary contains data for only the supported instances.

- Overriding the built-in `sample` method is not supported. Covergroups that specify a built-in `sample` method are ignored with a warning.
- Sampling is supported only by specifying an explicit clocking event in the covergroup declaration. If a covergroup is specified without a clocking event, the covergroup is ignored with a warning. Block event expressions are not supported as a sampling mechanism. Use of block event expressions generates an illegal-use error, and the compilation process terminates.

Considerations for Covergroup Instances

A covergroup can be instantiated in an `initial` block provided the following conditions are met:

- ❑ The covergroup is defined at the module level.
- ❑ The covergroup variable is defined at the module level.
- ❑ The covergroup variables are instantiated only once. For example, instances that are determined statically to be exclusive via if-case branching are supported; however, instances within looping constructs are not supported.

If any of the above conditions is not met, the covergroup is discarded with a warning at compilation time.

Multiple instances of the same covergroup are supported; however, the same covergroup variable cannot be instantiated more than once. For example:

```
CG1 cg1 = new;
initial
begin
    #10 cg1 = new; // same covergroup variable instantiated again
end
```

All the instances of such a covergroup variable are discarded with a warning during compilation.

In the hardware partition, covergroup instances are only supported within modules or interfaces. The covergroup instances are not supported within `always` blocks, `final` blocks, `generate` blocks, function calls, or task calls. These unsupported instances are ignored with a warning.

Covergroup instances are only supported in variable declarations and in `initial` blocks such as:

```
CG cg1 = new();
CG cg2;
initial begin
    cg2 = new;
end
```

Accelerating Testbench Covergroups in the Hardware

Covergroups in the software partition can be accelerated in the UXE hardware under certain conditions without user intervention. IXCOM automatically identifies the covergroups that can be accelerated and processes these in the hardware. It scans all the covergroups in the software partition and checks all the signals accessed inside the covergroup.

- ❑ When the signals covered in a covergroup reside in the hardware partition, that covergroup is automatically moved to the hardware partition and processed by the PalladiumXP emulator. IXCOM still preserves the covergroup in the original hierarchy.

Note: IXCOM does not analyze the covergroups embedded in classes. Embedded covergroups remain in the software partition.

- ❑ When the signals covered in a covergroup reside in the software partition, that covergroup remains in the software partition and is processed by IES.

Refer to Criteria for Partitioning Covergroups to the Hardware section for details and examples of covergroups that are moved to hardware partition or retained in software partition.

Therefore, a software module can have a mixture of covergroups that are processed by IES software and UXE hardware. When a mixture of IES and UXE covergroups exist, all covergroup statistics are captured in the same database.

There is no difference in the coverage reports when a software covergroup is accelerated. IXCOM preserves the covergroup labels that you have defined and the coverage results are displayed in the original module in the software partition.

Note: Most of the features supported by IES can be used in the software partition except when a coverage method, call, or reference operates on the design level or refers to the DUT from the software partition.

Hierarchical references to covergroup instances are not supported. When present, an error is issued and `ixcom` exits non-zero.

Criteria for Partitioning Covergroups to the Hardware

The following criteria must be satisfied for a covergroup in the software partition to be accelerated, that is to be moved to hardware:

- ❑ Usage must be within the feature subset supported by IXCOM.
- ❑ Sampling clock must be a signal declared in the hardware partition.
- ❑ All coverpoint variables and guard conditions must be signals declared in the hardware partition.
- ❑ Covergroup options and type options assigned in the procedural context are ignored with a warning.
- ❑ Covergroup methods such as `start()`, `stop()`, and `sample()` are not allowed.
- ❑ Coverpoint variable data types are limited to packed and integer data types. Enum types are not allowed.

IXCOM will issue an informational message to you when a covergroup is successfully moved to the hardware partition.

Following is an example of the covergroups that are moved into the hardware:

Sampling clock, coverpoint variable, and guard condition are in the hardware partition.

```
module tb_monitor_1();
```



```
// sampling clock, coverpoint variable, guard condition
// are inside DUT
covergroup CG1 @(top.<DUT_scope>.clk);
    cp1: coverpoint top.<DUT_scope>.bus iff(top.<DUT_scope>.en);
endgroup
CG1 cg_inst = new();
endmodule
```

Following are examples of the covergroups that are NOT moved into the hardware:

- Sampling clock is in the hardware partition, and a mixture of signals is in the hardware partition and on the testbench - DUT interface.

```
module tb_monitor_3();
    // sampling clock and the coverpoint variable are inside DUT
    // guard condition is TB signal and is on the
    // TB<->DUT interface
    covergroup CG3 @(top.<DUT_scope>.clk);
        cp1: coverpoint top.<DUT_scope>.bus iff(top.en);
    endgroup
    CG3 cg_inst = new();
endmodule
```

- Sampling clock is in the software partition, and the coverpoint variable is in the hardware partition:

```
module tb_monitor_2();
    covergroup CG2 @(top.clk);
        cp1: coverpoint top.<DUT_scope>.bus;
    endgroup
    CG2 cg_inst = new();
endmodule
```

- Sampling clock is in the hardware partition, and a mixture of signals is in the hardware and software partitions:

```
module tb_monitor_4(en);
    covergroup CG3 @(top.<DUT_scope>.clk);
        cp1: coverpoint top.<DUT_scope>.bus iff(en);
    endgroup
    CG3 cg_inst = new();
endmodule

module top();
    bit en;
    dut DUT();
    tb_monitor_4 CHK(en);
endmodule
```

```
endmodule
```

■ **Covergroup method calls are in the procedural code:**

```
module tb_monitor_5();
    covergroup CG5 @(top.<DUT_scope>.clk);
        cpl: coverpoint top.<DUT_scope>.bus;
    endgroup
    CG5 cg_inst = new();
    always @(posedge _start) begin
        cg_inst.start();
    end
    always @(posedge _done) begin
        cg_inst.stop();
    end
endmodule
```

Support for Coverage Methods

The following table lists the predefined coverage methods and the support level provided by IXCOM. See *Section 19.8 of the 2009 SystemVerilog LRM* for more information about the predefined coverage methods.

Method Name	Support Level
<code>sample()</code>	Not supported Calls to <code>sample()</code> are ignored.
<code>get_coverage()</code>	Not supported When used, an error is generated and IXCOM exits non-zero.
<code>get_inst_coverage()</code>	Not supported When used, an error is generated and IXCOM exits non-zero.

UXE User Guide

Coverage Support

Method Name	Support Level
set_inst_name()	Supported The <code>set_inst_name()</code> method is supported only from within an <code>initial</code> block. When used outside an <code>initial</code> block, a warning is displayed and the name is ignored. The covergroup is not ignored. <pre>CG cg1; initial begin cg1 = new; cg1.set_inst_name("inst1"); ... end</pre> <code>set_inst_name()</code> can also be called for covergroups that are instantiated via variable declarations.
start() and stop()	Supported Note: If called in the same time slot in which the covergroup is sampled, a race can occur between the method and the evaluation of the covergroup. This should be avoided, because it can cause different behavior when swapped to hardware versus swapped to software versus regular simulation.

Name Generation Algorithm for Coverpoint and Cross

The following points describe how names are generated for coverpoints and crosses.

- If a coverpoint or a cross has a label, that label is taken as the name of corresponding coverpoint or cross. For example:

```
CP1 : coverpoint a; // Name is "CP1"
CROSS1 : cross a, b; // Name is "CROSS1"
```
- If a coverpoint has no label and the coverpoint is declared on a single variable, the name of the variable is taken as the name of the coverpoint. For example:

```
coverpoint a; // Name is "a"
```
- If a coverpoint has no label and the coverpoint is declared on an expression, the name is internally generated. For example:

```
coverpoint a+b; // Name is "__coverpoint_0"
coverpoint c&b; // Name is "__coverpoint_1"
```

The integer at the end of the name is the value of an integer starting from 0 that increments with every use. The integer is reset to 0 for each covergroup as the names are generated.

- If a coverpoint has no label and it uses a reference argument variable, the name of the variable is used as the name of the coverpoint. For example:

```
covergroup cg (ref logic [3:0] f1) @(posedge clk);
    coverpoint f1; // Name is "f1"
endgroup
```

- If a cross has no label and coverpoints are simple variables, the names of variables are used in generating the name of cross. For example:

```
cross a , b; // Name is "__a_X_b_0"
```

- If a cross has no label and coverpoint labels are used in cross, the coverpoint label names are used for creating the generated name of cross. For example:

```
A: coverpoint a;
B: coverpoint b;
cross A, B; // Name is "__A_X_B_0"
```

- If a coverpoint has no label and contains an OOMR variable reference, all names in the OOMR are used in forming the name of the coverpoint. For example:

```
coverpoint il.xx; // Name is "il_xx"
```

Support for SystemVerilog Functional Coverage Constructs

The following table contains the complete support information for the SystemVerilog covergroups with respect to the syntax specified in the *IEEE-1800-2009 SystemVerilog Language Reference Manual*. The table lists the following information:

- SystemVerilog LRM Reference, with reference numbers quoted inside the square brackets [] (*Outer* column)
- Construct description (*Mid-level* column)
- Scope unit (*Inner* column)
- Support in UXE (*UXE* column)
- Any limitations or points to be noted (*Remarks* column)

Outer	Mid-Level	Inner	UXE	Remarks
LRM [19.3]	covergroup instance in	package	No	

UXE User Guide

Coverage Support

Outer	Mid-Level	Inner	UXE	Remarks
LRM [19.3]	covergroup decl in	package	No	
LRM [19.3]	covergroup instance in	compilation unit scope	No	
LRM [19.3]	covergroup decl in	compilation unit scope	No	
LRM [19.3]	covergroup decl in	module	Yes	
LRM [19.3]	covergroup instance in	module - var decl	Yes	Covergroups in DUT only are accelerated.
LRM [19.3]	covergroup instance in	module - initial statement	Partial	An instance can be assigned only once.
LRM [19.3]	covergroup instance in	module - always statement	No	
LRM [19.3]	covergroup instance in	module - final statement	No	
LRM [19.3]	covergroup instance in	module - generate	No	
LRM [19.3]	covergroup decl in	module - generate	No	
LRM [19.3]	covergroup instance in	module - function call	No	
LRM [19.3]	covergroup decl in	module - function call	No	
LRM [19.3]	covergroup instance in	module - task call	No	
LRM [19.3]	covergroup decl in	module - task call	No	
LRM [19.3]	covergroup instance in	program	N/A	Cannot exist in the DUT
LRM [19.3]	covergroup decl in	program	N/A	Cannot exist in the DUT
LRM [19.3]	covergroup instance in	interface	Yes	
LRM [19.3]	covergroup decl in	interface	Yes	
LRM [19.3]	covergroup instance in	class	N/A	Cannot exist in the DUT

UXE User Guide

Coverage Support

Outer	Mid-Level	Inner	UXE	Remarks
LRM [19.3]	covergroup decl in	class	N/A	Cannot exist in the DUT
LRM [19.3]	covergroup instance in	checker	No	
LRM [19.3]	covergroup decl in	checker	No	
BNF Syntax 19-1	covergroup_declaration	arguments	Yes	
BNF Syntax 19-1	covergroup_declaration	Trailing: covergroup_identifier	Yes	
BNF Syntax 19-1	covergroup_declaration	coverage_event	Partial	See event items below for detailed support information.
BNF Syntax 19-1	covergroup_declaration	coverage_spec_or_option	Yes	
BNF Syntax 19-1	coverage_spec_or_option	attributes - coverage_spec	Yes	
BNF Syntax 19-1	coverage_spec_or_option	attributes - coverage_option	Yes	
BNF Syntax 19-1	coverage_option	option	Yes	
BNF Syntax 19-1	coverage_option	type_option	Partial	See type_option items section below for detailed information about this support.
BNF Syntax 19-1	coverage_spec	cover_point	Partial	No transition bins, limit to 2^16 bins
BNF Syntax 19-1	coverage_spec	cover_cross	Partial	Not allowed with transition bins

UXE User Guide

Coverage Support

Outer	Mid-Level	Inner	UXE	Remarks
BNF Syntax 19-1	coverage_event	clocking event	Partial	■ Event variables not supported. ■ Sequences not supported. ■ iff not supported.
BNF Syntax 19-1	coverage_event	@ @(block_event_expression)	No	
BNF Syntax 19-1	block_event_expression	or operator	No	
BNF Syntax 19-1	block_event_expression	begin hierarchical_btf_identifier	No	
BNF Syntax 19-1	block_event_expression	end hierarchical_btf_identifier	No	
BNF Syntax 19-1	hierarchical_btf_identifier	hierarchical_tf_identifier	No	
BNF Syntax 19-1	hierarchical_btf_identifier	hierarchical_block_identifier	No	
BNF Syntax 19-1	hierarchical_btf_identifier	hierarchical_identifier	No	
BNF Syntax 19-1	variable_decl_assignment	new operation	Yes	
BNF Syntax 19-1	variable_decl_assignment	Optional arguments	Yes	

UXE User Guide

Coverage Support

Outer	Mid-Level	Inner	UXE	Remarks
Semantics Syntax 19-1	class	omitted covergroup_varia ble_identifier	N/A	Cannot exist in the DUT
LRM [19.3]	new operator	ref arguments	Partial	Only simple variables are allowed.
BNF Syntax 19-2	cover_point	identifier	Yes	When the LRM naming rules cannot be implemented, IES naming conventions are used for unnamed bins.
BNF Syntax 19-2	cover_point	expression	Yes	
BNF Syntax 19-2	cover_point	iff (expression)	Yes	
BNF Syntax 19-2	cover_point	bins_or_empty	Yes	
BNF Syntax 19-2	bins_or_empty	attribute_instance	N/A	Attribute instances are not supported by IXCOM.
BNF Syntax 19-2	bins_or_empty	bins_or_options	Yes	
BNF Syntax 19-2	bins_or_options	coverage_option	Yes	

UXE User Guide

Coverage Support

Outer	Mid-Level	Inner	UXE	Remarks
BNF Syntax 19-2	bins_or_options	wildcard	Partial	Fixed, automatic, transition bins with more than eight wildcard characters, and vector bins are not supported.
BNF Syntax 19-2	bins_or_options	bins_keyword	Yes	
BNF Syntax 19-2	bins_or_options	bin_identifier	Yes	When the LRM naming rules cannot be implemented, IES naming conventions are used for unnamed bins.
BNF Syntax 19-2	bins_or_options	bin automatic array	Yes	■ Illegal and ignore bins are not supported when present in an automatic coverpoint where the number of values is greater than <code>auto_bin_max</code>. ■ The number of values must be less than or divisible by <code>auto_bin_max</code>.

UXE User Guide

Coverage Support

Outer	Mid-Level	Inner	UXE	Remarks
BNF Syntax 19-2	bins_or_options	bin vector array	Yes	
BNF Syntax 19-2	bins_or_options	bin fixed array	Yes	Illegal and ignore bins are not supported when a fixed array is present in the coverpoint.
BNF Syntax 19-2	bins_or_options	open_range_list	Yes	
BNF Syntax 19-2	bins_or_options	iff (expression)	Yes	See Considerations for Bins for limitations.
BNF Syntax 19-2	bins_or_options	trans_list	Yes	See Considerations for Bins for limitations.
BNF Syntax 19-2	bins_or_options	default	Yes	Default bins are not tracked.
BNF Syntax 19-2	bins_or_options	default sequence	No	
BNF Syntax 19-2	bins_keyword	bins	Yes	

UXE User Guide

Coverage Support

Outer	Mid-Level	Inner	UXE	Remarks
BNF Syntax 19-2	bins_keyword	illegal_bins	Yes	■ Illegal bins within coverpoints do not generate a non-zero exit status at run time. ■ Error messages for illegal bins within a cross are delayed until the end of simulation. ■ Error messages for illegal bins within a cross report one value from that bin set (which might not represent the actual value that was sampled).
BNF Syntax 19-2	bins_keyword	ignore_bins	Yes	
BNF Syntax 19-2	open_range_list	value_range	Yes	
LRM [19.5]	clocking blocks	coverpoint sampling	No	Clocking block signals are not supported.

UXE User Guide

Coverage Support

Outer	Mid-Level	Inner	UXE	Remarks
LRM [19.5]	open_value_range	[\$:expression]	Yes	Additionally, the IES proprietary [\$:\$] syntax is supported.
LRM [19.5]	open_value_range	[expression:\$]	Yes	Additionally, the IES proprietary [\$:\$] syntax is supported.
BNF Syntax 19-3	trans_list	(trans_set)	Yes	See Considerations for Bins for limitations.
BNF Syntax 19-3	trans_set	trans_range_list	Yes	See Considerations for Bins for limitations.
BNF Syntax 19-3	trans_range_list	trans_item	Yes	See Considerations for Bins for limitations.
BNF Syntax 19-3	trans_range_list	trans_item[* repeat_range]	Yes	See Considerations for Bins for limitations.
BNF Syntax 19-3	trans_range_list	trans_item[-> repeat_range]	No	Goto repeats are not supported.
BNF Syntax 19-3	trans_range_list	trans_item[= repeat_range]	No	Non-consecutive repeats are not supported.
BNF Syntax 19-3	repeat_range	“constant or constant range”	Yes	See Considerations for Bins for limitations.
BNF Syntax 19-4	cover_cross	cross_identifier	Yes	
BNF Syntax 19-4	cover_cross	list_of_coverpoint s	Yes	

UXE User Guide

Coverage Support

Outer	Mid-Level	Inner	UXE	Remarks
BNF Syntax 19-4	cover_cross	iff (expression)	Yes	
BNF Syntax 19-4	cover_cross	select_bins_or_empty	Yes	
BNF Syntax 19-4	list_of_coverpoints	“cross_item,{cross_item}”	Yes	
BNF Syntax 19-4	cross_item	cover_point_identifier	Yes	
BNF Syntax 19-4	cross_item	variable_identifier	Yes	
BNF Syntax 19-4	select_bins_or_empty	{bins_selection_or_options}	Yes	
BNF Syntax 19-4	bins_selection_or_options	attribute_instance	N/A	Attribute instances are not supported by IXCOM.
BNF Syntax 19-4	bins_selection_or_options	coverage_option	Yes	See option and type_option section for specific support set.
BNF Syntax 19-4	bins_selection_or_options	bins_selection	Yes	
BNF Syntax 19-4	bins_selection	bins_keyword	Yes	
BNF Syntax 19-4	bins_selection	bins_identifier	Yes	
BNF Syntax 19-4	bins_selection	select_expression	Yes	

UXE User Guide

Coverage Support

Outer	Mid-Level	Inner	UXE	Remarks
BNF Syntax 19-4	bins_selection	iff (expression)	No	Gating conditions at the bin level are not supported.
BNF Syntax 19-4	select_expression	select_condition	Yes	
BNF Syntax 19-4	select_expression	! select_condition	Yes	
BNF Syntax 19-4	select_expression	select_expression && select_expression	Yes	
BNF Syntax 19-4	select_expression	select_expression , select_expression	Yes	
BNF Syntax 19-4	select_expression	(select_expression)	Yes	
BNF Syntax 19-4	select_condition	binsof(bins_expression)	Yes	
BNF Syntax 19-4	select_condition	intersect {open_range_list}	Yes	
BNF Syntax 19-4	bins_expression	variable_identifier	Yes	
BNF Syntax 19-4	bins_expression	cover_point_identifier	Yes	
BNF Syntax 19-4	bins_expression	cover_point_identifier.bins_identifier	Yes	
LRM [Table 19-1]	options	weight	Yes	Options allowed within declaration only.

UXE User Guide

Coverage Support

Outer	Mid-Level	Inner	UXE	Remarks
LRM [Table 19-1]	options	goal	Yes	Options allowed within declaration only.
LRM [Table 19-1]	options	name	Yes	Options allowed within declaration only.
LRM [Table 19-1]	options	comment	Yes	Options allowed within declaration only.
LRM [Table 19-1]	options	at_least	Yes	Options allowed within declaration only.
LRM [Table 19-1]	options	detect_overlap	No	
LRM [Table 19-1]	options	auto_bin_max	Yes	Number of bins must be divisible by value.
LRM [Table 19-1]	options	cross_num_print_missing	Yes	Options only allowed within declaration
LRM [Table 19-1]	options	per_instance	Yes	
LRM [Table 19-1]	options	get_inst_coverage	No	
LRM [Table 19-2]	type-options	weight	Yes	Options only allowed within declaration
LRM [Table 19-2]	type-options	goal	No	
LRM [Table 19-2]	type-options	comment	Yes	Options only allowed within declaration
LRM [Table 19-2]	type-options	strobe	No	

UXE User Guide

Coverage Support

Outer	Mid-Level	Inner	UXE	Remarks
LRM [Table 19-5]	coverage methods	sample	No	Calling <code>sample</code> explicitly is not supported.
LRM [Table 19-5]	coverage methods	<code>get_coverage</code>	No	Getting coverage mid-simulation is not supported.
LRM [Table 19-5]	coverage methods	<code>"get_coverage(ref int, ref int)"</code>	No	
LRM [Table 19-5]	coverage methods	<code>get_inst_coverage</code>	No	Getting coverage mid-simulation is not supported.
LRM [Table 19-5]	coverage methods	<code>"get_inst_coverage(Ref int, ref int)"</code>	No	
LRM [Table 19-5]	coverage methods	<code>set_inst_name(string)</code>	Yes	This is supported only in <code>initial</code> blocks.
LRM [Table 19-5]	coverage methods	<code>start()</code>	Yes	
LRM [Table 19-5]	coverage methods	<code>stop()</code>	Yes	
LRM [19-9]	Task/Function	<code>\$set_coverage_db_name(name)</code>	No	
LRM [19-9]	Task/Function	<code>\$load_coverage_db(name)</code>	No	
LRM [19-9]	Task/Function	<code>\$get_coverage()</code>	No	

Additional Limitations of Coverage Support

Note the following limitations when using the coverage features supported by IXCOM:

- Coverage databases dumped before the end of simulation will not contain all the accumulated hardware data.
- Clocking blocks are not supported by IXCOM. Thus, using clocking block signals in `covergroup` constructs is not supported.

Reporting for Functional Coverage Constructs

UXE covergroup reporting includes separate type-based and instance-based reports. Keep in mind the following considerations for these reports:

- The instance-based report does not display type-based report.
- By default, the default value for `option.per_instance` is set to zero (0), not one (1) as per the LRM.
- In UXE, by default, covergroup instance names are auto-generated, if you do not explicitly specify them. You can specify the instance name by using the `set_inst_name()` method or `option.name` setting. The instance-based report displays the user-specified names.
- The `type_option.goal` setting is supported, and used for presenting the type-coverage calculations in the report.
- Following are some points that are true for UXE covergroup reporting only when you use the INCISIV 12.1 version:
 - ❑ Fixed bin matches are reported with the fixed index instead of the bin values.
 - ❑ Non-uniform distribution bins are handled as per the LRM only when you use INCISIV 12.1 and later versions. These are not supported with earlier versions.

Error Handling for Functional Coverage Constructs

The following table describes how errors are categorized and handled when using SystemVerilog coverage constructs with IXCOM.

Category	Result
Illegal HDL	IXCOM issues error message, and exits non-zero.
Not supported in UXE	IXCOM issues warning, and covergroup might or might not be ignored. See below.
Not supported in IES	IXCOM issues warning, and covergroup is ignored.
Covergroup exceeds capacity limitations	IXCOM issues warning, and covergroup instance is ignored.
Unsupported run time commands	<code>xeDebug</code> issues a warning.
Illegal bin hit	An error message is displayed.

Illegal Bins

The *SystemVerilog LRM* allows you to specify illegal bins. In UXE, when an illegal bin is encountered, an error similar to the following is displayed:

```
ncsim: *E,EILLVS: Sampled value (3) for coverpoint (top.DUT.cg1.cnt) is an
illegal value.
```

Illegal bins can also be used in the covergroup cross statement. The following restrictions exist for illegal bins used in the cross statement:

- For crosses, the error messages are batched up and displayed during final block processing.
- For crosses, the error messages are not printed in chronological order.
- For crosses, the illegal value displayed will not be representative of a particular illegal value, but of just one optimized illegal value.

For example:

```
covergroup cg @ (posedge clk);
    A : coverpoint din1 {
        bins statea = { [0:3] };
        bins stateb = { [4:7] };
    }

    B : coverpoint din2 {
        bins statea = { [8:11] };
    }
    AxB : cross A, B
    {
        illegal_bins illegal = binsof (A.stateb);
    }

endgroup // cg
```

A suggested workaround is to write a SystemVerilog assertion to get comparable behavior.

In this case the IXCOM compiler uses 4 as a representative value for `A.stateb` and 8 as a representative value for `B.statea`. So, all illegal messages are printed with those values. To get accurate illegal cross values, the bins must use the vector form.

General Flow for Using Coverage

To use the coverage feature with Palladium XP, follow these steps:

1. Enable coverage at compile time by using the `ixcom` or `irun -hw` command. Refer to the compile-time options listed in Table 7-1, “Compile-Time Options for Enabling Coverage,” on page 432.
2. Generate the coverage information at run time using the `irun`, `xeDebug`, or `irun -xeDebug` command. Refer to [Generating a Coverage Database at Run Time](#) on page 441.
3. View the coverage information by using the IMC tool. Note that to use the IMC tool for code coverage analysis, you must use INCISIV 12.2 or higher versions. For details, refer to the [Using Incisive Metrics Center for Analyzing Coverage Data](#) section.

To enable support for coverage at compile time, you need to use the following options with either the `ixcom` or `irun -hw` command:

Table 7-1 Compile-Time Options for Enabling Coverage

Option	IES Equivalent	Description
<code>-coverage [block toggle functional all]</code>	<code>-coverage <target></code>	Enables coverage. ■ block - enables block coverage ■ toggle - enables toggle coverage ■ functional - enables functional coverage ■ all - enables all types of coverage You can use shorthand notation specified in bold font. For example, you can use t for <code>toggle</code> and u for <code>functional</code>. You can specify multiple options in the same line using colon (:) as the separator, such as <code>-coverage b:t</code>. By default, all modules compiled for hardware are included under code coverage. To specify particular design portions to be covered, use the <code>-featureFile</code> and <code>-covdut</code> options.

UXE User Guide

Coverage Support

Option	IES Equivalent	Description
<code>-covdut</code> <code><dut_module></code> (Only for toggle and block coverage)	<code>-covdut</code> <code><dut_module></code>	Specifies the top-level design unit for code coverage. If the design unit does not exist in the design, an error message is generated. It does not apply to functional coverage. All of the design units instantiated within the <code><dut_module></code> are marked for coverage. You can specify the <code>-covdut</code> options multiple times to specify different top-level design units. It is better to use the more powerful <code>-featureFile</code> option instead the <code>-covdut</code> option. Mixing the <code>-featureFile</code> and <code>-covdut</code> options in the same compile is not recommended.
<code>-featureFile</code> <code><feature_file></code>	<code>-covfile <file></code>	Used in conjunction with the <code>-coverage</code> option, the <code>-featureFile</code> option supports enabling and disabling coverage at the instance, design unit, or file level. For details, refer to the Enabling and Disabling Features at Compilation Time section.

Option	IES Equivalent	Description
<code>+tcovTypes+<type></code> (Only for toggle coverage)	None	Enables you to specify the types of variables to be tracked: ■ <code>p</code> - covers all the module and entity ports ■ <code>v</code> - covers all the Verilog and VHDL variables ■ <code>n</code> - covers all the Verilog nets and VHDL signals You can specify mutiple types of variables together, like <code>+tcovTypes+v:p:n</code>. The default value is <code>p</code>. If you specify this switch more than once, all the specified types are honored.

UXE User Guide

Coverage Support

Option	IES Equivalent	Description
<code>+ccovModelSummary</code> (Only for toggle and block coverage)	None	Enables creation of two comma-separated value (<code>.csv</code>) files to help you identify the portions of the design that should be considered for code coverage. The files are created as follows: ■ <code><design_dir>/ccov_model_summary_inst.csv</code> ■ <code><design_dir>/ccov_model_summary_type.csv</code> You can import these <code>.csv</code> files into a spreadsheet tool for browsing through the design to see how code coverage items are distributed. This is particularly useful when the available space on the emulator is limited and you need to prioritize the portions of the design to be covered. Refer to Instance-Based Summary and Type-Based Summary sections for details.

UXE User Guide

Coverage Support

Option	IES Equivalent	Description
<code>+ccovProfile</code> (Only for toggle and block coverage)	None	Enables tracking of statistics related to code-coverage processing. For example the time spent in processing code coverage at compile time is reported, such as: <pre>Code Coverage - End: cpu time = 1.24 sec, elapsed time = 2 sec heap memory size = 0.00M bytes</pre> This output is included in the <code>ixcom.log</code> file. Note: The counter used by this feature impacts other profiling counters used in the IXCOM compilation.
<code>-covfile <file></code> (Only for functional coverage)	<code>-covfile <file></code>	Specifies the file that lists the coverage-specific options sent directly to IES. This option is recognized only for functional coverage.

Instance-Based Summary

The instance based summary is useful for identifying how to partition a given design for code coverage. A sample of the `.CSV` file is shown below.

Figure 7-1 Sample Instance-Based Summary

	A	B	C	D	E	F	G
		Number of Local Toggles	Number of Local Blocks	Number of Local Coverage Items	Number of Local Toggles + Number of Sub- Instance Toggles	Number of Local Blocks + Number of Sub- Instance Blocks	Number of Local Coverage Items + Number of Sub- Instance Items
1	Instance						
2	top	14	6	20	5077699	60151	5137850
3	top.DUT	835	0	835	4836120	57414	4893534
4	top.DUT.A_CORE	844	0	844	1215183	7644	1222827
5	top.DUT.B_CORE	844	0	844	1215183	7644	1222827
6	top.DUT.RCVR	206	0	206	343975	4107	348082
7	top.DUT.A_CORE.FPU	368	0	368	294278	1302	295580
8	top.DUT.B_CORE.FPU	368	0	368	294278	1302	295580
9	top.DUT.XMITR	1522	0	1522	265014	5321	270335
10	top.DUT.B_CORE.IUNIT	840	0	840	263113	734	263847
11	top.DUT.A_CORE.IUNIT	840	0	840	263113	734	263847
12	top.DUT.B_CORE.FPU.SUB	1041	0	1041	232139	880	233019
13	top.DUT.A_CORE.FPU.SUB	1041	0	1041	232139	880	233019
14	top.DUT.B_CORE.IUNIT.IPROC	1257	0	1257	227635	576	228211
15	top.DUT.A_CORE.IUNIT.IPROC	1257	0	1257	227635	576	228211
16	top.DUT.GDI	925	0	925	162554	2268	164822

Start from the last column that contains the cumulative coverage contained within the specified instance. The file is sorted by the last column so the instances with the most cumulative coverage are reported first. The other columns are useful for advanced analysis and sorting.

For example, according to cell G2, this design has approximately 5.1 million coverage items. If the main intent is to get coverage on the two CORE instances (A_CORE and B_CORE), this spreadsheet indicates that the number of coverage items will be reduced by about half if coverage is restricted to `top.DUT.A_CORE` and `top.DUT.B_CORE`. You can use the `-featureFile` option for controlling coverage for these instances.

Note: Actual capacity usage may be different based on internal optimizations. The toggle values listed in the above table refer to the number of variable bits being toggled.

Type-Based Summary

The type-based summary is useful for identifying the high-capacity library cells that are not interesting for code coverage. A sample `.csv` file is shown below

Figure 7-2 Sample Type-Based Summary

	A	B	C	D	E	F	G
					Number of Coverage Items per Design Unit	Instances of Design Unit	Number of Coverage Items per Design Unit Times Instances of Design Unit
1	Design Unit	File	Number of Toggles	Number of Blocks			
2	INV	prims.v	2	0	2	423345	846690
3	MUX2	prims.v	5	0	5	52407	262035
4	NAND2	prims.v	3	0	3	57415	172245
5	NOR2	prims.v	3	0	3	36509	109527
6	LATCH01X	prims.v	3	0	3	34485	103455
7	MUXE202X	prims.v	4	0	4	20850	83400
8	NAND202X	prims.v	3	0	3	23065	69195
9	INV02X	prims.v	2	0	2	33778	67556
10	MUXE201X	prims.v	4	0	4	15212	60848

Start from the last column that contains the total number of coverage items associated with a specific design unit. Often, the largest consumers of coverage space are low-level library cells. Depending on your requirements, you can decide whether or not coverage on these is important. Identify the non-interesting high-capacity design units, and remove these from the coverage consideration.

For example, the previous spreadsheet indicated that there are approximately 5.1M coverage items in the design. This spreadsheet tells us that 846,690 coverage items (roughly 17% of the coverage items in the design) are related to a low-level inverter primitive. A primitive such as this is not typically interesting with respect to coverage reporting. In fact, mostly, all the primitives listed would not be interesting for code coverage. Then, you can use the `-featureFile` option to remove all the design units declared in `prims.v` from code-coverage consideration. This would lower the 5.1M coverage items reported down to 3.4M for just the nine design units shown.

Note: Due to optimizations, the capacity change will not be directly proportional, but it will be a noticeable improvement.

These compile-time options must be specified either with the `ixcom` command or with the `irun -hw` command. The `irun -hw` command enables you to use the `irun` utility with the Palladium XP emulator. This `irun` use model is described in [Chapter 5, “Compiling and Running Simulation Acceleration Designs with irun.”](#)

Examples for Enabling Coverage with the IXCOM Compiler

Following is an example of the `ixcom` command that tracks the coverage for each instance of the `dut`, `alu`, and `cpu` design units, and tracks all the ports and variables:

```
ixcom [ixcom_switches] -coveragetoggle \  
                        -covdut dut -covdut alu -covdut cpu +tcovTypes+pv
```

Following is the corresponding sample `irun -hw` command that tracks the same toggle coverage metrics as the above `ixcom` command:

```
irun -hw [other_switches] -coverage toggle \  
                        -covdut dut -covdut alu -covdut cpu +tcovTypes+pv
```

Compile-Time Output for Toggle and Block Coverage

During compilation, the information is generated with respect to code coverage in the `ixcom.log` file. Following is an example of this information. `%d` indicates a numeric placeholder for actual values that are calculated during compilation.

```
Code Coverage User Summary  
Block Coverage Items:      %d  
Toggle Coverage Items:    %d      (Rise + Fall)  
Total Coverage Items:    %d
```

The *User Summary* section details the information about the number of items that are tracked for coverage. These numbers should match the total number of coverage items in the run-time coverage report. Refer to [Using Incisive Metrics Center for Analyzing Coverage Data](#).

Compiling Designs with Functional Coverage Constructs for IXCOM Mode

To enable the parsing of the SystemVerilog covergroup constructs in the design, ensure that either the `+sv` option is included with the `ixcom` command, or the file has the `.sv` extension.

If you have not enabled functional coverage via the `-coverage functional` option, but you have specified the `+sv` option, the covergroup statements in both the hardware and software partitions are silently ignored.

Controlling Functional-Coverage Counter Sizes

For performance reasons, coverage counters for synthesizable covergroups are stored on the emulator. The number of bits in these counters can impact capacity and in some cases the overall simulation performance. The `+fcovCoverCnt+<N>` option of the `ixcom` command enables some control over the bit size of the internal counter.

The `+fcovCoverCnt+N` option specifies the minimum number of bits [default: `N=8`] to be used for the bin counters of covergroups in the DUT. Valid values for `N` include 1, 4, 8, 16, and 32. When values other than these are specified, IXCOM generates a compile-time error. Once an individual counter reaches its maximum value, the counter stays fixed at that value.

Counter values beyond the specified size can occur under the following circumstances:

- If the `+fcovCoverCnt` value cannot represent the `at_least` value for the coverpoint, cross, or covergroup, the local counter is increased as needed, to the smallest valid `+fcovCoverCnt` value that can contain the number. For example, suppose you specify `+fcovCoverCnt+1`, but has a local covergroup option of `at_least=5`, the maximum value that can be stored in 1 bit is the number 1. For correct coverage, IXCOM increases the size of that covergroup's counter to 4 bits. An information message is also displayed.
- If a coverpoint is feeding a cross. IXCOM implementation of crosses always maintains the minimum number of bits for each crossed combination. As a result, the coverpoint counts could go beyond the specified minimum in some scenarios.

Consider the following example.

Cross Example

```
typedef enum int {UP, DOWN} vertical;
vertical y;

typedef enum int {LEFT, RIGHT} horizontal;
horizontal x;

covergroup CG @(posedge clk);
    cross x,y;
endgroup

CG cg1 = new();
```

Assuming `+fcovCoverCnt+1` has been specified, IXCOM maintain a 1 bit counter for each combination in the cross. These are:

x	y	Counter Size	Example Counts
LEFT	UP	1	1
LEFT	DOWN	1	0

x	y	Counter Size	Example Counts
RIGHT	UP	1	1
RIGHT	DOWN	1	1

From the *Example Counts* column, all the combinations except for {LEFT, DOWN} are covered. The side effect of maintaining this granularity is that the coverpoint can exceed the counter size. In the above example, the coverpoint for `x==RIGHT` will be incremented twice in the database. Once for {RIGHT, UP} and once for {RIGHT, DOWN}. This results in an end counter value of 2 even though only 1 bit for the minimum counter size is expressed.

Generating a Coverage Database at Run Time

The following table describes run-time options specific to code-coverage collection. These options can be used with the any of the following commands:

- `irun`
- `irun -xeDebug`
- `irun -hw`

Coverage statistics are written the database during the end-of-simulation processing. The output data is stored in the Cadence UniCov format. This format is used by IES, Specman, IFV, and other utilities for generating coverage data. Use the IMC tool for analyzing coverage results. Note that for code coverage in UXE, the vManager and vPlanner tools are not supported.

Run-time Option	Description
<code>-covmodeldir <arg></code>	Specifies the path to the coverage model or scope directory that should be used in place of the default model or scope directory. By default, this is set to <code><cov_work>/<scope></code> .
<code>-covnomodeldump</code>	Specifies that the coverage design database (model) should not be dumped.
<code>-covoverwrite</code>	Enables overwriting of existing coverage output files and directories.
<code>-covscope <arg></code>	Specifies the scope name for the coverage-results databases as <code><covworkdir>/<covscope>/<covtest></code> . By default, this is set to <code>scope</code> .

UXE User Guide

Coverage Support

Run-time Option	Description
<code>-covtest <arg></code>	Specifies the test name for the coverage-results databases as <code><covworkdir>/<covscope>/<covtest></code> . By default, this is set to <code>test</code> .
<code>-covworkdir <arg></code>	Specifies the base name for the coverage-results databases as <code><covworkdir>/<covscope>/<covtest></code> . By default, this is set to <code>./cov_work</code> .

Note: These run-time options must be specified on the command line directly to be parsed properly. Do not specify any files to be parsed with these options (for example the `-f` option).

The options with arguments control the directory structure for storing the coverage information. In general, the format is as follows:

`<covworkdir>/<covscope>/<covtest>`

Keep the following points in mind about the run-time options:

- If you specify an option with an argument more than once, the first occurrence of the option is used and others are ignored with a warning.
- If you specify an option without an argument more than once, the extraneous occurrences are ignored without a warning.
- If the `-covmodeldir` option is used with the `-covnomodeldump` option, the `-covnomodeldump` option has higher precedence.
- If the `-covmodeldir`, `-covscope`, `-covtest`, or `-covworkdir` option is specified multiple times, the first instance is honored, and the others are ignored.

Coverage Run-time Output

By default, the model files specific to the topology of the design are stored in the `<covscope>` directory. If already present from a previous run, the model files are reused. The `<covtest>` directory contains test-specific data. Individual test results must be kept in

UXE User Guide

Coverage Support

separate `<covtest>` directories to that the data from each test is maintained. The following table describes the file types that are created at the end of simulation for code coverage:

File Extension	Description	Notes
.cmf	Coverage Model File	Used by the simple reporting tool called IXCC (Toggle and block coverage only)
.ucm	Unicov Model File	Used by IMC (Code coverage requires INCISIV 12.2 or higher)
.cdf	Coverage Data File	Used by the simple reporting tool called IXCC (Toggle and block coverage only)
.ucd	Unicov Data File	Used by IMC (Code coverage requires INCISIV 12.2 or higher)

Specific file information is reported at the end of simulation.

Using xeDebug to Generate Coverage Information

You can also use the xeDebug tool in place of using the *irun* utility to generate the coverage information at run time. The only difference in the usage is that the xeDebug command does not support the use of IES-related options directly on the command line. Therefore, these IES-related options must be passed through the xeDebug tool for run-time processing by using the `--ncsim` option of the xeDebug command.

For example:

```
xeDebug --ncsim -covworkdir /lan/cov_data \  
-covtest test1 -- [other_xeDebug_options]
```

Here, the `/lan/cov_data` and `test1` settings are specified, and for the other settings, the default values are used. The ncsim-specific arguments are encompassed within the `--` (hyphens) grouping. It is very important that the `--` (hyphens) are carefully placed. Refer to the xeDebug command line usage for more information in the *UXE Command Reference Manual*.

Run-Time Support for Coverage Constructs

The following table lists the options of the IES `coverage` command that apply to covergroups and how these operate in UXE at run time.

Options of the IES coverage Command	Functional (Assertions)	Functional (Covergroups)	Toggle	Block
<code>-code</code>	N/A	N/A	Not Supported	Not Supported
<code>-dump</code>	Supported	Testbench Only	Not Supported	Not Supported
<code>-fsm</code>	N/A	N/A	N/A	N/A
<code>-functional</code>	Supported	N/A	N/A	N/A
<code>-off</code>	Supported	Supported	Not Supported	Not Supported
<code>-setup</code>	Supported	Supported	Not Supported	Not Supported
<code>-toggle</code>	N/A	N/A	Not Supported	N/A
<code>-analyze</code>	Supported	Testbench Only	Not Supported	Not Supported

Note: The column for Functional (Assertions) in this table is provided only for completeness.

The only valid way to view the correct covergroup information is to wait for the end-of-simulation coverage database to be created. Hence, the use of `-dump` and `-analyze` options results in a database that is not up to date.

Running the Designs with Functional Coverage Constructs

When an IXCOT-compiled design including coverage constructs is run on the hardware by using the xeDebug run-time tool, the testbench covergroups are tracked by IES, and in some scenarios, even by UXE. The DUT covergroups are tracked by UXE. See [Run-Time Support for Coverage Constructs](#) for more details.

Use `finish`, `exit`, or `$finish` statements to trigger the extraction of coverage counters from the hardware at the end of simulation.

Important

At times, the `run` and `finish` commands of IES can generate recoverable errors. These errors do not terminate the simulation, but can cause xeDebug to exit with a non-zero exit status. To prevent this problem, encapsulate the `run` and `finish` simulation commands within a `catch` statement.

At the end of simulation, a coverage database is automatically generated. Following is an example of the default output at the end of simulation.

```
coverage setup:
workdir   : ./cov_work
dutinst   : top(top)
dutinst   : _ixc_isc(_ixc_isc)
dutinst   : xc_top(xc_top)
design     : design
testname  : test
coverage files:
model(design data) : ./cov_work/design/00000000_696b.ucm
data              : ./cov_work/design/test/icc.ucd
```

Running Designs with Coverage in ICA and LA Modes

The In-Circuit Acceleration (ICA) mode allows mixing uncontrolled clocks with a simulation environment.

The following restrictions apply while running covergroups in an uncontrolled domain or in the Logic Analyzer (LA) mode:

- ☐ Messages for illegal bins are suppressed.
- ☐ Covergroups where the number of bins is greater than 2^{11} are not fully sampled.

Using Incisive Metrics Center for Analyzing Coverage Data

Incisive Metrics Center (IMC) can be used to analyze and operate on coverage data. Typical operations of IMC include:

- ☐ Displaying coverage results
- ☐ Merging coverage data
- ☐ Ranking tests based on coverage amounts

❑ Refining tests

INCISIV tools and Incisive Comprehensive Coverage (ICC) must be installed to create and interact with the coverage data. Refer to the *IMC User Guide* available in the *INCISIV documentation set* for details on these and other operations. For more information about ICC, see the *ICC User Guide*, also available in the *INCISIV documentation set*.

The rest of this section covers the way that IMC interacts with the Palladium XP for coverage collection.

Important

The Palladium XP code-coverage solution generates an IMC-compatible database **only** when you are using INCISIV 12.2 or above version. Using older INCISIV releases limits you to using the simple reporting tool called IXCC for generating coverage reports. The IXCC tool is described in [coverage_toggle disable file=prims.vManaging Code Coverage Data and Generating Reports with the IXCC Tool](#) on page 458. Note that there is no such restriction related to functional-coverage databases and IMC.

Important

To utilize the functional coverage feature support described in this chapter, use the INCISIV version recommended in [INCISIV Versions for Coverage Features](#).

Managing Data with IMC

The following table enumerates the basic features of IMC and their compatibility with a UXE database.

Batch Command	Support Information
load show unload	Supported
report -list report -summary report -detail	Supported for toggle and block coverage

UXE User Guide

Coverage Support

Batch Command	Support Information
merge_config merge	Supported
rank_config rank	Supported
exclude unexclude load -refinement save -refinement source	Supported
convert_icf	Supported
preferences alias unalias sourcemap history help exit	Supported

Keep the following points in mind:

- Merging code-coverage data with IES Runs is not supported.
- Merging functional-coverage data with code-coverage data: Palladium functional coverage uses a completely different mechanism to create the UniCov database. As a result, the functional coverage information cannot be merged or viewed at the same time in IMC. A separate instance of IMC must be run to view results at the same time.
- Type-Based and Instance-Based Reports: The difference between type-based reporting and instance based is very crucial to interpreting code coverage results. Type-based reporting accumulates the coverage results from each instance of a particular design unit into a single line item. Instance-based reporting lists each instance as a separate line item. Both type-based and instance-based reports are available via IMC.

- **Type-Based Reports:** Due to parameterization and other aspects that make design units unique, modules might appear multiple times in the type-based report. Each appearance represents a different compile-time configuration of that design unit. IMC does not summarize this information automatically when this occurs.
- **Initial and Union resolutions:** When two different coverage models are merged, IMC resolves the differences based on the specified initial model file or creates a union of the two model files. IMC is a powerful tool, which provides many options for merging data. Refer to the *IMC user guide* for more information.

Differences When Using IMC for UXE versus IES

Following are the expected differences when using IMC with UXE as compared to its usage with IES.

- The UXE statistics are submitted to IES at the end of simulation during the final block processing. If you purposefully bypass the end-of-simulation processing, or abruptly exit the simulation, the coverage information will not be submitted to IES. You must end the simulation with the `finish` or `exit` command, or by hitting a `$finish()` in the HDL.
- Sometimes, IXCOM splits a module into multiple variants to satisfy parameter uniquification and testbench versus DUT variations, for example. Under these circumstances, the internally generated names are used for reporting. The internally generated names start with the original name, but are suffixed with `_xcm` followed by a unique number.

Note: IES does something similar, but only for modules where the parameters directly affect covergroups.

- **Functional Coverage Only:**
 - ❑ When using the IMC Console, the source line points to the `xc_work/v` information. The IXCOM compiler places the original HDL as a comment at the start of every covergroup line. If required, use the `-sourceoff` option to format the output so that the source HDL line is not displayed.
 - ❑ The IMC GUI source browser points to the `xc_work/v` HDL.
 - ❑ For performance reasons, IXCOM optimizes away coverage items that do not influence coverage calculations. Examples of such coverage items include empty covergroups and default bins. When these items are optimized away, an information message is generated during compilation.

Performance Considerations for Coverage Support

Coverage is defined by the following:

$$\text{coverage} = \{ \text{number of items covered} \} / \{ \text{number of items} \}$$

To achieve emulation run-time speeds, coverage instrumentation logic is used to track these values on the hardware. Before deciding if coverage can be used, it is important to determine the unused hardware capacity when your design is run on the Palladium XP emulator. In order to compile, enough emulator capacity needs to be available to fit the design under test, and any desired coverage instrumentation logic. You need to ensure that the number of items does not exceed emulator capacity. This is required for both functional and code coverage.

Performance Considerations for Code Coverage

When code-coverage instrumentation is inserted by the compiler into the design code:

1. The code that is to be tracked for code coverage cannot be optimized by the compiler.
2. The code-coverage instrumentation logic can possibly double the capacity of the area being covered.
3. The code-coverage instrumentation logic is also added to the software model. This can cause a noticeable slowdown in software-simulation performance if the code coverage area is large.

The Palladium XP compiler attempts to optimize and keep this coverage-related overhead as low as possible, but in general it is still very expensive. Therefore, Cadence recommends that code coverage be limited to as small an area as possible.

Cost of Using Code Coverage

Code coverage is very expensive because:

- ❑ The instrumentation logic can prohibit certain compiler optimizations.
- ❑ The instrumentation logic required to be inserted for the code coverage can almost double the capacity of the area being covered.

Optimizations are done to keep this overhead as low as possible, but in general, it is still very expensive. For system-level verification, the most efficient use of code coverage resources is to focus the coverage on system-level interconnect logic, sub-system interfaces, and specific blocks of interest.

Before deciding to use code coverage, it is important that you ascertain the unused capacity while running the design on the Palladium XP emulator. Then, you can use the code-coverage options to control the portions of the design to be covered.

The best way to measure the space available on the emulator is by domains. For example, if the Palladium XP emulator you are using has 16 domains, and the design requires eight domains, then up to eight domains can be used for toggle coverage. The projected minimum number of domains is reported in the `ixcom.log` file.

To determine the capacity associated with code coverage, consider the following table that lists expected coverage cost for a design that has coverage enabled across the entire DUT. A number is given for the typical capacity overhead up to a maximum value.

Coverage Type	Typical Increased Capacity as Compared to No Coverage	Maximum Increased Capacity compared to No Coverage
Toggle - ports (default)	1.5x	2x
Toggle - ports, variables	2x	3x
Toggle - ports, variables, nets	3x	4x
Block	1.1x	1.3x

For a typical eight-domain design, for example, the expected cost of basic toggle coverage (ports only) will be around 4 - 8 more domains.

Caveats:

- The numbers in the table are based on the UXE 12.1 release. Values may change in future releases as the supported subset increases (negative impact) and as more optimizations are added (positive impact).
- Some toggle scenarios can result in higher than expected results if:
 - ❑ The design is a gate-level design. For this scenario, it is recommended that you remove the gate-level primitives that are from coverage consideration by using the `+ccovModelSummary` option.
 - ❑ There is a high amount of local buffering of port signals.
 - ❑ There is a high amount of concatenation and other expression operations in the port signals.
 - ❑ The design contains a high amount of very large vectors.

If there is not enough capacity on the emulator, you can reduce the capacity cost of code coverage by limiting the area of the design being covered. You can explicitly mark the required modules and design units for code coverage to only track the areas of highest concern.

Managing Coverage Data

Cadence recommends that coverage data should be placed in its own directory away from the design. This is a common practice in simulation environments to ensure that all the coverage results can be easily accessed. Even if you only have one Palladium XP emulator, it is still recommended that you follow this practice.

This can be done by using the `-covworkdir` and `-covtest` options, which are natively included with the `irun` command.

If you do not specify these options, the compiler works with the default values. The default directory in the case of both options is "." or the current directory. Using these options is useful for managing the data generated over multiple simulation runs.

You should explicitly specify these commands if you are trying to collect data over multiple runs, and do not want the data to be overwritten. By using these options, you can ensure that the files are not accidentally deleted if another user cleans up the design directory.

Note: The data will not be overwritten if you have not used the `-covOverwrite` switch.

Considerations Related to Coverage Verification on Palladium XP Emulator

Here are certain points to be kept in mind when running designs with coverage enabled.

Performance

The major attraction of enabling coverage on Palladium XP is to leverage the high capacity and run-time performance of the emulator. Minimal degradation in hardware performance at run time is expected when coverage is enabled. However, this is not true when the design is swapped out of the emulator. With all the instrumentation monitors that are introduced after coverage is enabled, the software-simulation environment is expected to be slower than with coverage disabled. Therefore, Cadence recommends that when you intend to enable coverage, swap to the software mode only for design initialization and debugging.

Merging Code Coverage and Functional Coverage

Code coverage can be used in conjunction with the functional coverage features provided by Palladium XP. Both functional- and code-coverage databases are generated in the Unicov format that can be read by IMC. However, the underlying technologies to create those databases use different mechanisms. As a result, code-coverage and functional-coverage results cannot be combined into a single coverage database, and the respective databases must be opened separately.

Code Coverage for Testbench Signals

The toggle coverage support with Palladium XP applies only to those design units that are targeted for the emulator. Design units that comprise the testbench or extracted hierarchies are not considered for code coverage. Compile-time options that include such design units are ignored by the IXCOM compiler with a warning message. Passing Coverage Information Directly to IES without IXCOM Processing

To pass the coverage instrumentation control information directly to the IES simulator, *irun*, use the `-covfile <file>` option of the `ixcom` command. Then, the IXCOM compiler parses this option and directly passes it to IES compile. The IXCOM compiler does not analyze the contents of the file that you have specified.

With the `-covfile` option, you can use the following switches:

```
set_covergroup -per_instance_default_one  
set_covergroup -svcg_expand_trans
```

Avoid using any other options of the `set_covergroup` command because the resulting behavior might be unexpected.

Note: The `-type_rep_in_inst` and `-no_default_name` options have been deprecated in the IES 12.1 release, and are no longer supported by the IXCOM compiler.

Deprecated Options

This section details the options that have been deprecated but are still supported for backward compatability.



Usage of these options is not recommended.

Deprecated Compile-Time Options

The following table describes the deprecated compile-time options specific to code coverage. These options have been replaced by IES-equivalent options.

Table 7-2 Deprecated Compile-time Options Used for Enabling Code Coverage Support with Palladium XP

Option	Description
Options Specific to Block Coverage	
<code>+bcov[+<design_unit_or_file_name>]</code>	Enables block coverage and indicates the design units or source HDL files that are to be tracked for block coverage. You can specify the names of multiple design units or files by delimiting each name with a '+' character with a single <code>+bcov+</code> option, or by using multiple <code>+bcov+</code> options.
<code>+bcovExclude+<design_unit_or_file_name></code>	Excludes the specified design unit or file from block coverage collection. You can specify the names of multiple design units or files by delimiting each name with a '+' character with a single <code>+bcovExclude+</code> option, or by using multiple <code>+bcovExclude+</code> options.

UXE User Guide

Coverage Support

Option	Description
Options Specific to Toggle Coverage	
<code>+tcov[+<design_unit_or_file_name>]</code>	Enables toggle coverage and specifies the design units to be tracked for toggle coverage. Enabling toggle coverage on the entire design is not recommended because of performance issues. Refer to Performance Considerations for Coverage Support on page 449 for details.
<code>+tcovExclude+<design_unit_or_file_name></code>	Excludes the specified <code><design_unit></code> or <code><file_name></code> from the toggle-coverage calculation.

The following table maps the deprecated and replacement options. For backward compatibility, the deprecated options can be used but not combined with the new options.

Deprecated Option

New Option

<code>+tcov</code>	<code>-coverage toggle</code>
<code>+tcov+<design_unit file_name></code>	<code>-coverage toggle</code> <code>-featureFile <file></code>
<code>+tcovExclude+<design_unit file_name></code>	<code>-coverage toggle</code> <code>-featureFile <file></code>
<code>+bcov</code>	<code>-coverage block</code>
<code>+bcov+<design_unit file_name></code>	<code>-coverage block</code> <code>-featureFile <file></code>
<code>+bcovExclude+<design_unit file_name></code>	<code>-coverage block</code> <code>-featureFile <file></code>

Deprecated Run-Time Options

The following table describes the deprecated run-time options specific to code-coverage collection. These options have been replaced by IES-equivalent options. Run-Time Options for Code Coverage

Deprecated Option	New Option
+ixccWorkDir+<workdir>	-covWorkDir <workdir>
+ixccTest+<testname>	-covTest <testname>
+ixccOverwrite	-covoverwrite

Deprecated Use Models

The following sections describe the use models that have been deprecated, but are maintained only for backward compatibility. These are not recommended.

Enabling the Block and Toggle Coverage Features for Instance Hierarchies

It is recommended that you use the `-featureFile` option described in the [Enabling and Disabling Features at Compilation Time](#) section, to enable or disable portions of the design for functional, toggle, and block coverage.

For those users who use the old `+tcov` and `+bcov` options, this section describes how you can modify the `ixcom` compile command to replace older `+bcov` and `+tcov` options with the `-featureFile` option. The `-featureFile` option enhances the control to enable or disable features at instance-hierarchy level instead of just at the module level.

You can replace the following options with the `-featureFile` option for better control over enabling and disabling features like coverage and assertions in the design:

```
+bcov[+<du|file_name>]
+bcovExclude[+<du|file_name>]
+tcov[+<du|file_name>]
+tcovExclude[+<du|file_name>]
```

Using the `-featureFile` Option of the `ixcom` Command to Control Coverage

The `-featureFile` option of the `ixcom` command provides a mechanism to control coverage based on names of instances , design units, and source files.

The following table provides a comparison of the functionality supported by the `+bcov` and `+tcov` options and the `-featureFile` option.

Features can be Enabled or Disabled for	Using the <code>+bcov</code> and <code>+tcov</code> Options	Using the <code>-featureFile</code> Option
Design-Unit-Based Configuration	Yes	Yes
Design-Unit-Based Configuration (with Depth)	No	Yes

Features can be Enabled or Disabled for	Using the +bcov and +tcov Options	Using the -featureFile Option
Instance-Based Configuration	No	Yes
Instance-Based Configuration (with Depth)	No	Yes
File-Based Configuration	Yes	Yes

Example - Explicit Control

The following example shows the old method of using the `+tcov` option to enable toggle coverage for the `dut`, `alu`, and `cpu_core` modules.

```
+tcov+dut +tcov+alu +tcov+cpu_core
```

Alternatively, you can use the `-featureFile` option as follows:

```
ixcom -coverage toggle -featureFile toggles.ff
```

The `-coverage toggle` option enables toggle-coverage instrumentation for the entire design.

Following are the `<feature_spec_line>` lines that you need to have in the `toggles.ff` file:

```
// The following line removes toggle consideration across the entire design.
coverage_toggle disable
// The following lines enable toggle coverage for particular modules: dut, alu, and
// cpu_core.
coverage_toggle enable du=dut
coverage_toggle enable du=alu
coverage_toggle enable du=cpu_core
```

Example - Implicit Control

You can choose to mark the entire design for coverage consideration, but remove low-level sections that are not interesting. The following set of options illustrate the case where toggle coverage is enabled for the entire design, but disabled for the gate-level primitives defined in a file called `prims.v`.

```
+tcov +tcovExclude+prims.v
```

A comparable option set that uses the feature file would be:

```
ixcom -coverage toggle -featureFile toggles.ff
```

The following `<feature_spec_lines>` in the `toggles.ff` file enable toggle-coverage instrumentation for the entire design, and disable for the gate-level primitives in the `prims.v` file:

```
coverage_toggle disable file=prims.vManaging Code Coverage Data and Generating  
Reports With the IXCC Tool
```

A utility called `ixcc` is provided to manage code-coverage data and generate basic run-time reports for users who have not yet upgraded to the INCISIV 12.2 release. The `ixcc` utility consumes one or more coverage counter files and generates either a text-based cumulative coverage counter report file, and/or a binary cumulative coverage counter file depending on options specified to the `ixcc` command.



Tip

The IMC tool provides a number of advanced features for analyzing coverage results. It is highly recommended that you use the IMC tool to get maximum benefit out of the coverage data.

Following is the syntax of the `ixcc` command:

```
ixcc [-inputDir <work_dir> ] \  
    [-input <test_name>] | [-inputAll] \  
    [-outputDir <work_dir>] \  
    [-output <test_name>] \  
    [-format <format>] \  
    [-overwrite] \  
    [-summary] \  
    [-listInst <file>] \  
    [-matchInst <file>]
```

Refer to the description of the `ixcc` command in the *UXE Command Reference Manual* for details of the options of the `ixcc` command.

Generating Code Coverage Reports for Design Unit Instances

The code-coverage report is generated when the “`stdout`” format is specified with the `ixcc` command.

- The signal name is the name of the signal as it would be locally referred to within the design unit. For example, the signal “`top.dut.cpu2.en`” would appear as “`en`” in this report. The full hierarchical path to the instance of the design unit, “`top.dut.cpu2`”, is reported in the *Design Instance Section*.

Block Coverage Section

If a particular design unit has block-coverage data, the following information is printed for each design unit instance:

Block Coverage Report

count	block	line	origin
-----	-----	-----	-----
%d	%s	%d	%s
.	.	.	.
.	.	.	.
-----	-----	-----	-----
%f			

- The Block Coverage report contains one line item for each block being tracked.
- The count represents the number of times this block was executed.
- The counter is only 1 bit wide for each simulation run.
- The %f below the dashed line indicates the percentage of the blocks in that design hierarchy that have been covered.
- The block field represents the kind of block that is being tracked. The following table describes the possible block type values. The rules for defining blocks can be found in *Chapter 2* of the *ICC User Guide* available with the INCISIV documentation set.

Block Text	Description	Verilog Example
“ “	Generic block	<code>begin</code>
“true part of”	True statement	<code>if (en)</code>
“else part of”	Else statement	<code>else</code>
“case item”	Case item statement	<code>5: state = 2'b11;</code>

- The line column is used to indicate the line of the source code where the block starts.

- The origin field represents the first 38 characters of the line of the block. Any preceding whitespace is removed from the line. If the source file cannot be opened or the line number does not exist, no information is displayed.

Controlling Coverage Output

The report generated for `stdout`, by default, lists all the instances. To limit the instances that should be reported, the `-listInst` and `-matchInst` switches can be used as follows:

1. Run the `ixcc` command with the `-listInst` option.

When the `-listInst` switch is used, no coverage reports are generated. The `-output` and `-summary` switches are ignored, if specified.

2. Create a text file based on the `-listInst` output to specify only those hierarchies that should appear in the report.

To create this text file, you can either remove the lines manually or by using custom scripts (generated using the `sed` command or `perl`, and so on). Lines beginning with a `#` character are ignored by the `ixcc` utility.

3. Run the `ixcc` command again with the `-matchInst` option, specifying the text file.

When the `-matchInst` switch is used, only the matched instances are presented in the summary table.

If an instance name is not found in the coverage database, the following text is printed in a warning message:

```
Instance %s not found in the coverage database.
```

Summary Report for Code Coverage

A summary report is printed when the `-summary` option is specified with the `ixcc` command. This summarization helps in identifying the uncovered sections of the design.



Tip

Cadence recommends that you first examine this summary report to get a general idea of the sections of the design that currently lack coverage. After identifying the instances of your interest, you can review the detailed coverage report associated with those instances.

The summary information is printed regardless of output format types. If multiple input files are specified, the summary information consists of the merged results.

UXE User Guide

Coverage Support

The following information is printed in the summary report:

```
LOG = Local Overall Grade
LOC = Local Overall Covered
OG = Overall Grade
OC = Overall Covered
Instance      LOG      LOC      OG      OC
-----
      %s      %f%      %d / %d      %f%      %d / %d
```

Local Overall Grade represents the percentage of the coverage items in this instance that are covered. It is the result of dividing the two values in the Local Overall Covered field. The numerator represents the number of coverage items that have been covered. The denominator represents the total number of coverage items. For block coverage, each block is considered one coverage item. For toggle coverage, the full toggle is considered one coverage item. For a full toggle to be covered, the toggle net must have a rising and falling transition in the same test run. If only one transition occurs, the toggle is not considered as being covered in the summary report.

The Overall Grade and Overall Covered columns are mathematically the same as their local counterparts, but these also include the coverage items from sub-hierarchies. A “-” is used to represent a grade value if the number of coverage items is zero.

Instance Output Order

You cannot control the output order of instances. However, the `-listInst` switch can be used to generate a file of instance names in the order printed by IXCC. With some basic scripting, this information can be fed into Incisive Metric Center (IMC) to ensure that the output comes in the same order by using the following command:

```
report -detail -metrics all -inst %s
```

Replace `%s` in the above command with the actual instance name.

Here is an example script:

```
foreach inst (`cat match_inst.txt`)
    echo "report -detail -metrics all -inst %s" >> imc.cmds
end
```

Source the `imc.cmds` file from the IMC utility after loading the test files required for comparison.

Compiling Designs with SystemVerilog DPI

IXCOM supports a subset of the IEEE-P1800 version of the *SystemVerilog Direct Programming Interface (DPI)*.

This chapter contains the following topics:

- [Overview of SystemVerilog DPI](#)
- [SystemVerilog Layer](#)
- [Foreign Language \(C/C++\) Layer](#)
- [import Functions and Tasks](#)
- [export Functions and Tasks](#)
- [Using SystemVerilog DPI](#)
- [Support for Open Arrays in DPI](#)

Note: Please review standard SystemVerilog DPI documentation for a more complete description of the SystemVerilog DPI. This chapter will focus primarily on how SystemVerilog DPI has been implemented for the environment.



The content of this chapter pertains only to the SA mode.

Overview of SystemVerilog DPI

The SystemVerilog Direct Programming Interface (DPI) is an interface between SystemVerilog and any foreign language, although C/C++ is the only one supported at this time. SystemVerilog DPI greatly simplifies connecting a Verilog-based design or test environment to C/C++.

Earlier interfaces, such as the Programming Language Interface (PLI), required you to set up an explicit table to link Verilog with C/C++. This table was usually created as a `veriusertf` table in user C/C++ code or using a `TAB` file. You also had to access Verilog design objects from within C/C++ using access functions provided by the PLI, such as `tf_getp()`, `acc_set_value()`, or `tf_putp()`, and so on.

In a significantly more efficient manner, SystemVerilog DPI allows the HDL compiler to create the interface and data access, using two *layers*:

- **SystemVerilog Layer** which can call `import` functions/tasks that have been defined in the Foreign language (C/C++) layer.
- **Foreign Language (C/C++) Layer** which can call `export` functions/tasks that have been defined in the SystemVerilog layer.

SystemVerilog Layer

The SystemVerilog layer of the DPI is the layer from which you `import` C/C++ tasks or functions. It is also the layer in which you define `export` tasks or functions that can be called by the C/C++ layer.

All import DPI tasks and functions have to be executed on the simulator. All export DPI tasks and functions, by default, are also executed on the simulator unless the `tb_export` compiler directive is used to map the export DPI task to the emulator.

Foreign Language (C/C++) Layer

The foreign language, or C/C++, layer of the DPI is the layer in which you call an exported Verilog task or function from C/C++ code. The task or function must have an `extern` declaration on the C/C++ side. (Refer to [Supported/Unsupported C/C++ Functions](#) on page 465.)

This layer specifies how:

- actual arguments are passed
- arguments are accessed from the foreign (C/C++) code
- Verilog data types are represented
- data types are translated to and from some predefined C/C++-like types

IXCOM allows SystemVerilog DPI only in the context of the Verilog-2001 subset. The following table maps Verilog data types to C/C++ data types:

Table 8-1 Verilog-2001 Data Types Mapped to C/C++ Data Types

Verilog-2001 Data Type	C/C++ Data Type
four-value scalar	unsigned char
bit	unsigned char
four-value vector (packed array, time, integer)	svLogicVec32, svLogicPackedArrRef
bit vector (packed array)	svBitVec32, svBitPackedArrRef
real	double

Return types of the DPI `import` functions are limited to the following Verilog types:

- bit (two-value, vector up to 32 bits)
- logic (scalar four-valued logic)
- real

Supported/Unsupported C/C++ Functions

This section contains lists of:

- [Supported C/C++ Functions](#)
- [Unsupported C/C++ Functions](#)

Supported C/C++ Functions

IXCOM supports the following functions, whose `extern` declarations are found in the `svdpi.h` file (the standard file supplied with SystemVerilog DPI):

```
extern int svSizeOfBitPackedArr(int width);
extern int svSizeOfLogicPackedArr(int width);
extern void svPutBitVec32(svBitPackedArrRef d, const svBitVec32* s, int w);
extern void svPutLogicVec32(svLogicPackedArrRef d, const svLogicVec32* s, int w);
```

UXE User Guide

Compiling Designs with SystemVerilog DPI

```
extern void svGetBitVec32(svBitVec32* d, const svBitPackedArrRef s, int w);
extern void svGetLogicVec32(svLogicVec32* d, const svLogicPackedArrRef s, int w);
extern svBit svGetSelectBit(const svBitPackedArrRef s, int i);
extern svLogic svGetSelectLogic(const svLogicPackedArrRef s, int i);
extern void svPutSelectBit(svBitPackedArrRef d, int i, svBit s);
extern void svPutSelectLogic(svLogicPackedArrRef d, int i, svLogic s);
extern void svGetPartSelectBit(svBitVec32* d, const svBitPackedArrRef s, int i, int w);
extern svBitVec32 svGetBits(const svBitPackedArrRef s, int i, int w);
extern svBitVec32 svGet32Bits(const svBitPackedArrRef s, int i);
/* 32-bits */
extern unsigned long long svGet64Bits(const svBitPackedArrRef s, int i);
extern void svGetPartSelectLogic(svLogicVec32* d,
const svLogicPackedArrRef s, int i, int w);
extern void svPutPartSelectBit(svBitPackedArrRef d, const svBitVec32 s,
int i, int w);
extern void svPutPartSelectLogic(svLogicPackedArrRef d,
const svLogicVec32 s, int i, int w);
extern svScope svGetScope();
extern const char* svGetNameFromScope(const svScope);
extern svScope svGetScopeFromName(const char* scopeName);
extern int svPutUserData(const svScope scope, void *userKey,
void* userData);
extern void* svGetUserData(const svScope scope, void* userKey);
extern int svGetCallerInfo(char **fileName, int *lineNumber);
extern svScope svSetScope(const svScope scope);
```

Unsupported C/C++ Functions

IXCOM does not currently support the following functions, whose `extern` declarations are *not* included in the `svdpi.h` file:

```
extern int svLeft(const svOpenArrayHandle h, int d);
extern int svRight(const svOpenArrayHandle h, int d);
extern int svLow(const svOpenArrayHandle h, int d);
extern int svHigh(const svOpenArrayHandle h, int d);
extern int svIncrement(const svOpenArrayHandle h, int d);
extern int svLength(const svOpenArrayHandle h, int d);
extern int svDimensions(const svOpenArrayHandle h);
extern void *svGetArrayPtr(const svOpenArrayHandle);
extern int svSizeOfArray(const svOpenArrayHandle);
extern void *svGetArrElemPtr(const svOpenArrayHandle, int indx1, ...);
extern void *svGetArrElemPtr1(const svOpenArrayHandle, int indx1);
```

UXE User Guide

Compiling Designs with SystemVerilog DPI

```
extern void *svGetArrElemPtr2(const svOpenArrayHandle, int indx1,
int indx2);
extern void *svGetArrElemPtr3(const svOpenArrayHandle, int indx1,
int indx2, int indx3);
extern void svPutBitArrElemVec32(const svOpenArrayHandle d,
const svBitVec32* s, int indx1, ...);
extern void svPutBitArrElem1Vec32(const svOpenArrayHandle d,
const svBitVec32* s, int indx1);
extern void svPutBitArrElem2Vec32(const svOpenArrayHandle d,
const svBitVec32* s, int indx1, int indx2);
extern void svPutBitArrElem3Vec32(const svOpenArrayHandle d,
const svBitVec32* s, int indx1, int indx2, int indx3);
extern void svPutLogicArrElemVec32(const svOpenArrayHandle d,
const svLogicVec32* s, int indx1, ...);
extern void svPutLogicArrElem1Vec32(const svOpenArrayHandle d,
const svLogicVec32* s, int indx1);
extern void svPutLogicArrElem2Vec32(const svOpenArrayHandle d,
const svLogicVec32* s, int indx1, int indx2);
extern void svPutLogicArrElem3Vec32(const svOpenArrayHandle d,
const svLogicVec32* s, int indx1, int indx2, int indx3);
extern void svGetBitArrElemVec32(svBitVec32* d,
const svOpenArrayHandle s, int indx1, ...);
extern void svGetBitArrElem1Vec32(svBitVec32* d,
const svOpenArrayHandle s, int indx1);
extern void svGetBitArrElem2Vec32(svBitVec32* d,
const svOpenArrayHandle s, int indx1, int indx2);
extern void svGetBitArrElem3Vec32(svBitVec32* d,
const svOpenArrayHandle s, int indx1, int indx2, int indx3);
extern void svGetLogicArrElemVec32(svLogicVec32* d,
const svOpenArrayHandle s, int indx1, ...);
extern void svGetLogicArrElem1Vec32(svLogicVec32* d,
const svOpenArrayHandle s, int indx1);
extern void svGetLogicArrElem2Vec32(svLogicVec32* d,
const svOpenArrayHandle s, int indx1, int indx2);
extern void svGetLogicArrElem3Vec32(svLogicVec32* d,
const svOpenArrayHandle s, int indx1, int indx2, int indx3);
extern svBit svGetBitArrElem(const svOpenArrayHandle s, int indx1, ...);
extern svBit svGetBitArrElem1(const svOpenArrayHandle s, int indx1);
extern svBit svGetBitArrElem2(const svOpenArrayHandle s, int indx1,
int indx2);
extern svBit svGetBitArrElem3(const svOpenArrayHandle s, int indx1,
int indx2, int indx3);
extern svLogic svGetLogicArrElem(const svOpenArrayHandle s,
int indx1, ...);
extern svLogic svGetLogicArrElem1(const svOpenArrayHandle s, int indx1);
extern svLogic svGetLogicArrElem2(const svOpenArrayHandle s, int indx1,
int indx2);
```

```
extern svLogic svGetLogicArrElem3(const svOpenArrayHandle s, int indx1,
int indx2, int indx3);
extern void svPutLogicArrElem(const svOpenArrayHandle d, svLogic value,
int indx1, ...);
extern void svPutLogicArrElem1(const svOpenArrayHandle d, svLogic value, int
indx1);
extern void svPutLogicArrElem2(const svOpenArrayHandle d, svLogic value, int
indx1,int indx2);
extern void svPutLogicArrElem3(const svOpenArrayHandle d, svLogic value, int indx1,
int indx2, int indx3);
extern void svPutBitArrElem(const svOpenArrayHandle d, svBit value,
int indx1, ...);
extern void svPutBitArrElem1(const svOpenArrayHandle d, svBit value,
int indx1);
extern void svPutBitArrElem2(const svOpenArrayHandle d, svBit value,
int indx1, intindx2);
extern void svPutBitArrElem3(const svOpenArrayHandle d, svBit value,
int indx1, int indx2, int indx3);
extern int svIsDisabledState();
extern void svAckDisabledState();
```

import Functions and Tasks

`import` functions or tasks are defined in the C/C++ layer and are called in the SystemVerilog layer.

To use `import` functions or tasks, you must declare them in your Verilog code using an *import declaration*, for example:

```
import "DPI-C" context task          c_rand (output int x, output int y);
import "DPI-C" context function int pytha_check (input int inx, input int iny, input
int mod, input int angle);
```

This section has the following subsections:

- [Definition of Import Declaration](#)
- [Example: SystemVerilog Layer Calling C/C++ Function](#)
- [Accessing Two-Value Bit Values in C/C++](#)
- [Accessing Four-Value Logic Values in C/C++](#)

Definition of Import Declaration

- task or function: A DPI call can be of the type task or function. The main difference between a task and function is:

- ❑ A DPI function is equivalent to a normal C or Verilog function – it has a return value and appears on the RHS of the expressions. Conversely, a DPI task does not return any value to the RHS – it returns a value from C to HDL using an argument.
- ❑ A function can have arguments only of type `input`; a task can have arguments of type `input`, as well as `output` and `inout`.

■ function return type: A function's return type can be one of the following:

- ❑ `bit` [signed|unsigned] ['[x:y]']: two-state, optional signed or unsigned (default) specification, optional range specification for a vector (limited up to only 32 bits)
- ❑ `int` [signed|unsigned]: two-state, 32 bits, optional signed (default) or unsigned specification
- ❑ `logic`: default if unspecified, four-state scalar value
- ❑ `real`: 64 bit floating point value

■ task or function arguments: Task or function arguments have a mandatory direction specification and an optional type.

- ❑ Direction: Direction can be of type `input`, `output` or `inout`. The direction is defined from the C/C++ interface perspective.

`input` type arguments are set from the HDL and read by C/C++.

`output` type arguments are set from C/C++ and read by the HDL.

`inout` type arguments can be read or set from either the HDL or C/C++.

The mechanism for passing values for `output` or `inout` type arguments to/from C/C++ is in the form of a reference on the C/C++ side.

- ❑ Type: The following types are supported for arguments:

`[logic] ['[x:y]']`: This is the default type if unspecified, 4-state unsigned value, an optional range specification for a vector.

`bit` [signed|unsigned] ['[x,y]']: two-state value, an optional signed or unsigned specification (default unsigned); an optional range specification for a vector.

`int` [signed|unsigned]: two-state, 32-bit value, an optional signed or unsigned (default signed).

`real`: 64 bit floating point value.

- Here are some more examples of import declarations:

```
import "DPI" context function myDpi(input [0:5] first); // function returning
scalar 4 value, one input 6 bits wide logic type argument
```

```
import "DPI" context function int unsigned myDpi(input bit signed [0:7] first,
input second); // function returning an unsigned int, one input 8 bits wide
signed bit type argument, second input 4-value logic scalar
```

```
import "DPI" context task myDpi(input bit [0:7] first, output int second); /
/ task, one input 8 bits wide unsigned bit type argument, one output signed int
type argument
```

- SystemVerilog has two types of imported functions: `pure` and `context`. Tasks can be only `context`. To be `pure`, the result of a function must depend only on the values of its inputs, and it must not change the value of any variable directly or indirectly (by calling other functions), that is, it must not have any *side effects*.

In contrast, a `context` task or function accesses SystemVerilog data (other than arguments passed to the function) by the PLI, VPI, and so on. You must identify `context` functions in the declaration to optimize the code.

Example: SystemVerilog Layer Calling C/C++ Function

In the following DPI-based implementation of a Pythagorean processor testbench taken from the tutorial (`<UXE_installation_directory>/share/uxe/gift/tutorial/dpi`), the DPI function `c_rand` provides two random data values to test code.

The test process then applies the inputs to the DUT and sets up a control signal `do_pytha` to trigger a computation. The test process waits for the status signal `done_pytha` to be set, which indicates that results are available from the DUT. Then, the DPI function call `pytha_check` checks the results against those computed using C/C++ and reports errors exceeding the limit set in the verification environment.

In addition to DPI functions, Behavioral Verilog code, including `wait` conditions and dynamic `while` loops, is used as part of the test process.

As previously mentioned, you must declare any `import` functions in your Verilog code using an *import declaration*:

```
import "DPI-C" context task          c_rand (output int x, output int y);
import "DPI-C" context function int pytha_check (input int inx, input int iny, input
int mod, input int angle);
```

After these `import` functions have been declared, they can be used as regular functions or tasks in Verilog:

```
always @(posedge clk)
```

UXE User Guide

Compiling Designs with SystemVerilog DPI

```
begin : test
    while(!reset_)
        @(posedge clk);
    @(negedge clk);
    if(count < 10)
        begin
            c_rand(inx,iny);
            do_pytha <= 1;
            @(posedge clk)
                do_pytha <= 0;
            while(done_pytha)
                @(posedge clk);
            count = count+1;
            while(!done_pytha)
                @(posedge clk);
            i=pytha_check(inx, iny, mod, angle);
            errors = errors+i;
        end
    else
        begin // axis tbcall_region
            $display("Done with %0d computations, %0d errors", count, errors);
            $finish;
        end
    end
end
```

For this example, the DPI functions are defined as:

```
#include "veriusers.h"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Function corresponding to SystemVerilog DPI declaration
// import "DPI-C" context task          c_rand (output int x, output int y);
void c_rand(int* x, int* y)
{
    // "output" type values are passed by reference
    *x=(1+rand())%(1<<15));
    *y=(1+rand())%(1<<15));
}

// Function corresponding to SystemVerilog DPI declaration
```

UXE User Guide

Compiling Designs with SystemVerilog DPI

```
// import "DPI-C" context function int pytha_check (input int inx, input int iny,
input int mod, input int angle);
int pytha_check(int x, int y, int mod, int ang) {
    int inx, iny, imod, iang;
    if ((x>>15) & 1) inx = -(x&0x7fff); else inx = x; // sign extension from 16 bits
to int
    if ((y>>15) & 1) iny = -(y&0x7fff); else iny = y; // sign extension as above
    imod = (int) sqrt((double) (inx*inx+iny*iny)); // C computed modulus
    iang = 180 - (int) (180*atan2(iny, inx))/(M_PI); // C computed angle
    if(iang < 0) iang -= 360;
    if((abs(imod-mod)>2) | (abs(iang-ang)>15))
    {
        // DUT generated result deviates too much from C computed
        io_printf("PYTHA-DPI Inx: %d Iny: %d Mod: %d Ang: %d C-Mod: %d C-Ang:
%d\n", inx, iny, mod, ang, imod, iang);
        return(1);
    }
    else
        return(0);
}
```

To compile this code, use the following command:

```
ixcom tpytha.v +dut+test_pytha +sv +bc+test_pytha ...
```

The above command includes the following options:

■ +sv

Enables the SystemVerilog DPI in IXCOM.

Note: Refer to the *+sv* section in the *Commands and System Tasks for SA Mode* chapter of the *UXE Command Reference Manual* for more detail.

■ +bc+<module>+<module>...

Enables behavioral compilation of the specified module(s). This option is required when the module contains either SystemVerilog DPI, behavioral Verilog, or both (as in this case).

Note: Refer to the *+bc* section in the *Commands and System Tasks for SA Mode* chapter of the *UXE Command Reference Manual* for more detail.

Accessing Two-Value Bit Values in C/C++

Up to 32-bit values are directly accessed as signed or unsigned `int`. When a task or function specifies a two-value bit vector type argument of width > 32 bits, its values are accessed as an array of 32-bit values on C/C++. In this case, the argument in C/C++ should be declared as `svBitVecVal *`, which is defined as a reference to an array of 32-bit unsigned `int` in `svdpi.h`.

Each element of the array starting from 0 represents the next 32 bits of the bit vector. The number of elements in the array is dependent on the size of the vector, such that it is split into 32-bit elements. The bit wise values of an argument of type `svVecVal` can also be accessed using utility functions such as `svGetBitselLogic()`, `svPutBitselLogic()`, `svGetPartselLogic()`, `svPutPartselLogic()`, and so on. These and other such functions are defined in `svdpi.h`.

For example:

Verilog:

```
import "DPI" context function bit equal (input bit [47:0] ina, input bit [47:0]
inb);
```

C:

```
// Implement logical equality function in C
int equal (svBitVecVal *ina, svBitVecVal *inb)
{
    unsigned int p[2]; // two unsigned ints to store 48 bits of vector
                        //information
    p[0] = ina[0]; // lower 32 bits
    p[1] = ina[1]; // upper 16 bits
    unsigned int q[2];
    q[0] = inb[0];
    q[1] = inb[1];
    return((p[0] == q[0]) & (p[1] == q[1])); // return value of function
}
```

Accessing Four-Value Logic Values in C/C++

When a task or function specifies a four-value logic type argument, its four values are accessed as 2-bit information (known as "aval", "bval") per bit on C/C++.

- For a scalar input argument in HDL, the argument in C/C++ should be declared as unsigned `char` or predefined `svLogic` type. The four values of the scalar are

enumerated in `svdpi.h` as `sv_0`, `sv_1`, `sv_z`, and `sv_x` corresponding to unsigned values of `0x0`, `0x1`, `0x2`, and `0x3`, respectively.

- For a scalar output argument of a task or scalar return value of a function in HDL, the argument in C/C++ should be declared as `unsigned char*` or `svLogic *`. The value is then assigned into the dereferenced variable using the same four values' system or enumerated types, as shown previously.
- For a vector input/output argument in HDL, the argument in C/C++ should be declared as `svLogicVecVal *`, which is defined as a reference to an array of `p_vpi_vecval` structures in `svdpi.h`. Each element of the array starting from 0 represents the 32 bits of vector four-value with the structure member *a* having *avalues* and member *b* with *bvalues* to make the four-value logic. The number of elements in the array is dependent on the size of the vector such that it is split into 32 bits of the vector per element. The bit wise values of an argument of type `svLogicVecVal` can also be accessed using utility functions such as `svGetBitselLogic()`, `svPutBitselLogic()`, `svGetPartselLogic()`, `svPutPartselLogic()`, and so on. These functions are defined in `svdpi.h`.

For example:

Verilog:

```
import "DPI" context task mult (input [3:0] ina, input [3:0] inb, output [4:0] out);
```

C:

```
// implement multiplier task in C
void mult (svLogicVecVal *ina, svLogicVecVal *inb, svLogicVecVal *out)
{
    unsigned int p = ina->a;    // Assume logical 0,1 value in aval
    unsigned int q = inb->a;
    out->a = p*q;               // sets aval of return value
    out->b = 0;                 // make sure bval is zero (no z, x)
}
```

export Functions and Tasks

`export` functions or tasks are defined in the SystemVerilog layer and called in the C/C++ layer.

Example: C/C++ Layer Calling SystemVerilog Function

The following example shows an exported task, that is, a task declared in SystemVerilog and then called in C/C++.

Here is the declaration:

```
export "DPI-C" task      rom_read;
export "DPI-C" task      rom_write;
```

Note: The arguments of exported functions or tasks are not part of the declaration; the arguments are compiled directly based on task definitions in Verilog, for example:

```
// exported tasks
task rom_read(input int addr, output int data);
begin
    $axis_mread(test_pytha.ul angl.roml.rom, addr, data);
    $display("%m: ROM Read from DPI addr=%x, data=%x", addr, data);
end
endtask

task rom_write(input int addr, input int data);
begin
    $display("%m: ROM Write from DPI addr=%x, data=%x", addr, data);
    $axis_mwrite(test_pytha.ul angl.roml.rom, addr, data);
end
endtask
```

When these tasks have been exported from Verilog, they can be called from C/C++ as follows:

```
extern void rom_read(int addr, int *data);
extern void rom_write(int addr, int data);

void setup_atan_rom()
{
    int i;
    unsigned romd;
    // for a 15 bit (inx/iny) division result of x & y values, calculate atan ROM values
    for(i=1;i<=31;i++)
    {
        romd = 90-180*(atan(i))/M_PI;
        io_printf("PYTHA-DPI Rom Setup @ addr=%x data=%x\n",i, romd);
        rom_write(i,romd);
    }
}
```

```
}  
}
```

In this example, the C/C++ function `setup_atan_rom()` calls the `rom_write()` task exported from Verilog to perform a write into a design memory using a direct cross-module reference in Verilog. The `setup_atan_rom()` is used in the Pythagorean example to set up the lookup ROM for converting from X/Y ratio to angle in degrees.

The C/C++ function `setup_atan_rom()` itself is called from Verilog at initial time as an SV-DPI task using `import`:

```
import "DPI-C" context task      setup_atan_rom;  
  
initial  
begin  
    $axis_set_clkgen("clk", 0, 50, 100, 100);  
    setup_atan_rom;  
    count = 0; errors = 0; i=0; reset_=0;  
    reset_=0;#1812 reset_=1;  
end
```

Using SystemVerilog DPI

Using the SystemVerilog DPI is similar to using the Programming Language Interface (PLI). As with the PLI, you first compile your C/C++ code using your preferred C/C++ compiler; then, you pass the generated object file to the compiler. Unlike the PLI, you do not need the PLI table file (`veriusr.c`).

You do need to pass a new option, `+sv`, when you start `ixcom`.

```
ixcom options verilog_files -pl "dpi_object_files_or_archives" +sv
```

Typically, a module with export/import DPI scopes is not considered for behavioral compilation unless it includes other constructs requiring behavioral compilation. If such a module needs to undergo behavioral compilation, explicitly specify its name with the `+bc` option of the `ixcom` command, or include the `$map_bc` directive in that module.

Following are the steps to run a DPI example for 32-bit mode:

1. Ensure that your environment has been set up for 32-bit compilation. Set the library path variable `LD_LIBRARY_PATH` to the 32-bit path for IXCOM run-time library, for example:

```
/tools.lnx86/lib/32bit:/grid/cva/test_ius/ius.9.2.s14/tools.lnx86  
/tools.lnx86/lib
```

2. Compile the C/C++ code and pass the generated object file to the compiler.


```
gcc -m32 -c -o counter.o counter.c
gcc -m32 -shared -o libdpi.so counter.o
```

3. Invoke `ixcom`.

```
ixcom -f list.f +dut+counter_t_ff +timescale+1ns/10ps -32 +sv
```

Note: By default, the compilation occurs in 64-bit mode. You must explicitly pass `-32` flag to force the 32-bit compilation.

4. Invoke `irun`, passing the `libdpi.so` file using the `-sv_lib` option.

```
irun -R -TCL -sv_lib libdpi.so
```

SystemVerilog DPI Keywords in Verilog

SystemVerilog DPI introduces the following keywords to Verilog:

- `DPI`
- `import`
- `export`
- `pure`
- `context`

Using SystemVerilog DPI with Emulator

When you are mapping your design for the emulator, the method you use depends on the type of SystemVerilog DPI function you are using — `import` or `export`.

SystemVerilog DPI Import Functions with Emulator

You can call a SystemVerilog DPI import function from the module under the DUT. A special flag is not required to map the design, and the module is automatically treated as behavioral module. This is same as using the `+bc+<module>` option. The DPI import functions are always executed on the simulator. Refer to [+bc](#) in the Tabular Reference for more details about the `+bc` option.

SystemVerilog DPI Export Functions with Emulator

By default, compiler treats a SystemVerilog DPI export function in the same way as a DPI import function. A special flag is not required to map the design, and the module is executed

on the simulator. To map an export function or task to the emulator, use the `tb_export` compiler directive. In this case, SystemVerilog DPI export functions and tasks have the same language restrictions as normal tasks and functions in DUT.

Improving Performance in DPI Environment Using SDPI Mode

For pthread-HMPI, a special mode, named SDPI mode for DPI environment is provided to improve performance. In the SDPI mode, it is assumed that Client Verification Process (CVP) has a single testbench thread, the main program, which repeatedly does the following test cycles:

1. CVP calls DPI export tasks to initialize the test, and then monitor Hardware Model Server (HMS) activity.
2. HMS only calls the import DPI functions (no tasks are allowed) to access CVP after the above step.
3. HMS notifies the end of test from an import DPI function call.

In the SDPI mode, the best performance can be achieved by using the following SDPI interface functions defined in the `${UXE_HOME}/etc/ixcom/hm_top_pt.h` file:

```
#define DPI_EXPORT_ONLY 1
#define DPI_IMPORT_ONLY 2
extern void hm_initialize_sdpi(); // configure to run in SDPI mode
extern void hm_sdpi_halt(); // halt HMS from DPI import between tests.
extern void hm_sdpi_mode(int mode); // switch mode of execution to
                                     DPI_IMPORT_ONLY or DPI_EXPORT_ONLY
```

Here is a basic code skeleton that you should follow:

```
int main(int argc, char *argv[]){
    hm_initialize_sdpi(); //configure to run in SDPI mode

    do{
        <C-code for test preparation>
        hm_sdpi_mode(DPI_EXPORT_ONLY); // calling DPI export to configure
                                         a test

        <call DPI export tasks here>;
        hm_sdpi_mode(DPI_IMPORT_ONLY); // enable DPI import call then
                                         monitoring test progress

        while(test_done){
            <Monitor test progress and events here>;
        }

        <post test processing here>
```

```
        } until(all_done);
        hm_exit();
    }
    /* A DPI import C function to notify test_done *.
    int dpi_notify () {
        hm_sdpi_halt(); // Halting HMS not to continue.
        test_done = 1;
    }
}
```

Support for Open Arrays in DPI

This section describes the support for open arrays for packed and unpacked dimensions in import DPI tasks or functions. This support is particularly useful in TBA (Transaction-Based Analysis) applications because the SCE-MI standard requires the use of open arrays for supporting SystemVerilog adaptors.

When you use IXCOM to compile the designs, you can use DPI scopes in HMI and non-HMI modes. Refer to [Chapter 13, “IXCOM Hardware Model Interface”](#) for details about the HMI mode.

Example: Using Open Arrays in DPI

Following is a basic example to illustrate the use of open arrays in an import DPI function. The second parameter, `v`, of the function `foo` has an open-packed dimension. So, the SystemVerilog code can call the import function with an arbitrary-sized bit vector as its argument. Notice the two function calls inside the initial process to pass arguments of different sizes.

```
/* SystemVerilog code */
module top;
    import "DPI-C" function void foo(int id, bit [] v);
    bit [31:0] b1 = 32'h1234;
    bit [63:0] b2 = {32'h5678, 32'h1234};
    initial begin
        foo(1, b1); // call foo with 32-bit vector
        foo(2, b2); // call foo with 64-bit vector
    end
endmodule
```

The C function `foo` should be defined so that the variable-sized open-array argument is properly handled. This can be achieved by the use of query functions, such as `svGetArrayPtr`, `svLength`, `svLeft`, `svRight`, and so on.

```
/* C code */
#include "stdio.h"
#include "svdpi.h"
int foo(int id, svOpenArrayHandle h)
{
    int i = 0;
    unsigned *v = svGetArrayPtr(h);
    int sz = svLength(h, 0);
    printf("Call DPI %d %d ", id, sz);
    while(sz > 0){
        printf(" %x ", v[i]);
        sz -= 32; i++;
    }
    printf("\n");
    return 0;
}
```

Querying and Accessing Open Arrays from the C Code

To support the use of open-array dimensions in import DPI scopes, methods for querying information about open dimensions from C code must be provided. Otherwise, the C code cannot handle variable-sized arguments adaptively.

As an example, consider a DPI import function that is declared as follows:

```
import "DPI-C" function void foo(bit [] v);
```

Given this declaration, the SystemVerilog code can call the function `foo`, with bit vectors of different sizes (for example, `bit [3:0] arg1`, `bit [31:0] arg2`). To support the actual arguments of different sizes, the definition of the function `foo` needs to figure out the size of the open-packed dimension. For this, a query function `svLength` can be used.

In the non-HMI mode, the actual call from the SystemVerilog code to the import C function is fully controlled by IES, no IXCOM support is required in the implementation of import DPI function calls. IES, at the time of function call, it will transfer control from the SystemVerilog code to the body of the called C function, marshal or unmarshal all arguments to and from the C function, and support any query functions that can be used inside the C function. So, you can use any query functions as defined in the `svdpi.h` file supported by IES.

However, in the HMI mode, the DPI task call is handled by IXCOM-generated code. Given an import DPI function call, where the caller and callee sits in different processes, the IXCOM-generated code is responsible for implementing the inter-process procedure calls and any

query functions that can be used in CVP code should be provided by IES. The following query and access functions are supported. So, the CVP-side code can query about the open arrays.

```
void *svGetArrayPtr(svOpenArrayHandle h)
```

The above function returns the pointer to the array associated with the handle. This pointer can be used to access the element of arrays. [Example: Using Open Arrays in DPI](#) shows how to use the `svGetArrayPtr` function to access the elements of a bit vector. When the array does not have an unpacked dimension, the C code can directly access the array elements as shown in the example.

To access the elements of open unpacked arrays, a different function, called `svGetArrElemPtr` needs to be used:

```
void *svGetArrElemPtr(const svOpenArrayHandle h, int idx);
```

To access the elements of open unpacked arrays, a different access method is required because the actual layout of the SystemVerilog array passed as an argument for open unpacked arrays is different from the C layout. This function returns the pointer to the element at the given index of the open-unpacked dimension. Only one unpacked dimension is supported.

```
// import "DPI-C" function void foo(input bit [3:0] p []);
int foo(const svOpenArrayHandle p)
{
    int p_low = svLow(p, 1); // lower bound of unpacked dimension
    int p_high = svHigh(p, 1); // higher bound of unpacked dimension
    for (int i = p_low; i <= p_high; i++) {
        unsigned *elem = (unsigned *) svGetArrElemPtr(a, i);
        printf("p[%d] = %x\n", i, *elem);
    }
}
```

When a pointer to the array element is obtained, it is possible to both read and write to the memory location of the array element.

```
void svPutBitArrElemVecVal(const svOpenArrayHandle h, const svBitVecVal*,
                           int idx);
```

This function allows the packed element of an open array to be updated. Note that only one dimension is required because only one open-unpacked dimension is supported.

Given a handle and a dimension, the following function returns the left bound of the actual argument in the given dimension. When dimension `d` is 0, the query refers to the packed part of the array. When dimension `d` is 1, it refers to the unpacked part of an array.

```
int svLeft(svOpenArrayHandle h, int d)
```

Given a handle and a dimension, the following function returns the right bound of the actual argument in the given open dimension.

```
int svRight(svOpenArrayHandle h, int d)
```

Given a handle and a dimension, the following function returns the lower value of the left and right bound of the actual argument in the given dimension.

```
int svLow(svOpenArrayHandle h, int d)
```

Given a handle and a dimension, the following function returns the higher value of the left and right bound of the actual argument in the given dimension.

```
int svHigh(svOpenArrayHandle h, int d)
```

Given a handle and a dimension, the following function returns the size of the range in the given open dimension. The return value is equivalent to $|svLeft(h, d) - svRight(h, d) + 1|$.

```
int svSize(svOpenArrayHandle h, int d)
```

Limitations in Using Open Arrays in DPI

Due to language constraints, as specified in SystemVerilog LRM, limitations in IES or IXCOM, only a subset of all possible open arrays can be used as formal parameters in DPI scope declarations. This section summarizes such restrictions assuming that DPI scope is used inside a DUT module.

- ❑ Open arrays can be only used in imported DPI scopes.
- ❑ Only formal parameters of imported DPI functions or tasks can be specified as open arrays.
- ❑ An exported function or task cannot have open arrays as its formal parameters.

Only Pthreads and SystemC CVPs are supported in HMI-mode. When using open arrays in import DPI calls, the C function definitions need to use query functions, such as `svLeft` and `svRight` to determine the range of the open dimension in actual arguments.

In the non-HMI code, these query functions are supported by IES. However, in the HMI-mode, the caller and the callee are in different processes (one in HMS and the other in CVP) and the interprocess procedure calls are implemented using IXCOM-generated instrumentation.

For Pthreads and SystemC CVPs, IXCOM provides its own query functions which thread functions or SystemC functions can use. However, When the CVP is in a third-party SystemVerilog code, there is no way to override the DPI functionality that is embedded in the third-party SystemVerilog simulator. In HMI-mode, there is no way to control over third-party software simulators, such as VCS or MTI.

Using Open Arrays in Packed Dimensions

The SystemVerilog LRM states that when an open dimension is used in a packed dimension, the open dimension must be the only packed dimension in the formal parameter. Therefore, it is not possible to have two or more open-packed dimensions. Also, IES further constrains that such packed dimension must be either a bit vector or a logic vector.

```
bit [] p;           // allowed;
reg [] p;           // allowed
logic [] p;         // allowed
byte [] p;          // not allowed; illegal as per the LRM
bit [3:0][] p;      // not allowed;
                    // open-packed dimension must be the only packed dimension
```

Using Open Arrays in Unpacked Dimensions

IES allows up to one open unpacked dimension in formal parameters of import DPI scopes.

```
int p []; // allowed
shortint p []; // allowed
longint p []; // allowed
byte p []; // allowed
bit p []; // not allowed; use bit [] p
reg p []; // not allowed; use reg [] p
logic p []; // not allowed; use logic [] p
bit [3:0] p []; // allowed
reg [3:0] p []; // allowed
logic [3:0] p []; // allowed
bit [] p []; // not allowed
reg [] p []; // not allowed
logic [] p []; // not allowed
int p [][]; // not allowed; only one unpacked dimension is allowed
logic [] p [][]; // not allowed
```

Supported Set of DPI Functions

SystemVerilog LRM defines the set of DPI functions that C code can use. The list of such functions can be found in the `svdpi.h` file in the IES product installation hierarchy. In the non-HMPI mode, the following DPI functions are supported.

```
void *svGetArrayPtr(svOpenArrayHandle h);
void *svGetArrElemPtr(svOpenArrayHandle h, int idx);
```

UXE User Guide

Compiling Designs with SystemVerilog DPI

```
void svPutLogicArrElemVecVal(const svOpenArrayHandle h,
                             const svLogicVecVal *v, int indx1, ...);
void svGetLogicArrElemVecVal(svLogicVecVal *v, const svOpenArrayHandle h,
                             int indx1, ...);
void svPutBitArrElemVecVal(const svOpenArrayHandle h, const svBitVecVal *v,
                             int indx1, ...);
void svGetBitArrElemVecVal(svBitVecVal *v, const svOpenArrayHandle h,
                             int indx1, ...);
int svLeft(svOpenArrayHandle h, int d);
int svRight(svOpenArrayHandle h, int d);
int svLow(svOpenArrayHandle h, int d);
int svHigh(svOpenArrayHandle h, int d);
int svLength(svOpenArrayHandle h, int d);
```

In HMI-mode, when the CVP is written in Pthreads, SystemC, SystemVerilog, only the following subset of functions is supported. These functions are declared in the following file:

\$AXIS_HOME/etc/ixcom/{hm_top_pt.h, hm_top_sc.h}

```
svScope svGetScope();
svScope svSetScope(const svScope scope);
const char* svGetNameFromScope(const svScope);
svScope svGetScopeFromName(const char* scopeName);
int svPutUserData(const svScope scope, void *userKey, void* userData);
void* svGetUserData(const svScope scope, void* userKey);
svBit svGetBitselBit(const svBitVecVal* s, int i);
svLogic svGetBitselLogic(const svLogicVecVal* s, int i);
void svPutBitselBit(svBitVecVal* d, int i, svBit s);
void svPutBitselLogic(svLogicVecVal* d, int i, svLogic s);
void svGetPartselBit(svBitVecVal* d, const svBitVecVal* s, int i, int w);
void svGetPartselLogic(svLogicVecVal* d, const svLogicVecVal* s, int i, int w);
void svPutPartselBit(svBitVecVal* d, const svBitVecVal s, int i, int w);
void svPutPartselLogic(svLogicVecVal* d, const svLogicVecVal s, int i, int w);
```

To support open arrays in HMPI mode, in addition to the above set of functions, the following functions are provided so that Pthreads and SystemC CVPs can be use in HMI mode. The meanings of the functions are the same as defined in SystemVerilog LRM. Open arrays when the CVP is written in SystemVerilog are not supported. So, SystemVerilog CVPs must not use the following functions:

```
void *svGetArrayPtr(svOpenArrayHandle h);
void *svGetArrElemPtr(svOpenArrayHandle h, int idx);
void svPutLogicArrElemVecVal(const svOpenArrayHandle h,
                             const svLogicVecVal *v, int idx);
void svGetLogicArrElemVecVal(svLogicVecVal *v, const svOpenArrayHandle h,
                             int idx);
```


UXE User Guide

Compiling Designs with SystemVerilog DPI

```
void svPutBitArrElemVecVal(const svOpenArrayHandle h, const svBitVecVal *v,  
                           int idx);  
void svGetBitArrElemVecVal(svBitVecVal *v, const svOpenArrayHandle h,  
                           int idx);  
int svLeft(svOpenArrayHandle h, int d);  
int svRight(svOpenArrayHandle h, int d);  
int svLow(svOpenArrayHandle h, int d);  
int svHigh(svOpenArrayHandle h, int d);  
int svSize(svOpenArrayHandle h, int d);  
int svLength(svOpenArrayHandle h, int d);
```

UXE User Guide

Compiling Designs with SystemVerilog DPI

PLI/VPI and VHPI Support

IXCOM supports both PLI/VPI standard for Verilog designs and VHPI standard for the VHDL designs. This chapter discusses the following:

- PLI/VPI Support in IXCOM
- VHPI Support in IXCOM



The content of this chapter pertains only to the SA mode.

PLI/VPI Support in IXCOM

IXCOM supports the Programming Language Interface 1.0 (PLI 1.0) and Programming Language Interface 2.0 (PLI 2.0 or VPI) standards for linking C or C++ code with the simulation snapshot. The PLI standards are specifically used for Verilog designs, and VHPI is the corresponding standard for VHDL designs.

Following steps describe the use-flow for linking the C/C++ code with the simulation snapshot:

- 1. Step 1:** Compile the C/C++ source files of the PLI application into a shared library.

For example, you have a `veriusers.c` file, compile this C file and the other PLI source files to create a PLI application, which is a C/C++ shared library, such as `libpli.so`. The PLI application uses the same C/C++ compiler setup as used for IXCOM.

- 2. Step 2:** Use IXCOM to compile the design and generate the snapshot. The IXCOM compilation can be a single-step or a multi-step process, as shown:

- a.** In single-step compile mode, specify the `+loadpli1` or `+loadvpi` option with the `ixcom` command, specifying the library name and (optionally) any bootstrap routine names, along with other options.

- b.** In multi-step compile mode, perform the following steps. The multi-step compilation is used when the design contains some VHDL or SystemC portions.
- Specify the `vlan` command. The `vlan` command does not accept a shared library as input for PLI information.
 - Generate a PLI table file containing PLI definitions. This file can be created using the `ncpli2tab` utility. Then, provide this file to `vlan` using the `-p` option to parse the design. For more information on creating a PLI table file, refer to the section on [Creating a PLI Table File](#) on page 488.
 - In the elaboration step, run the `ixcom` command with the `+loadpli1` or `+loadvpi` option, specifying the library name and any bootstrap routine names, apart from the usual `-top <top module name>` and other options.

3. Step 3: Use `xeDebug` to simulate the design. During simulation, the `xeDebug` command initiates the calls to relevant VPI access and tasks and functions of the PLI application.

Note: For more information on PLI 1.0 standard, refer to the *PLI 1.0 User Guide and Reference* manual in the INCISIV documentation set.

For more information on PLI 2.0 standard, refer to the *VPI User Guide and Reference* manual in the INCISIV documentation set.

Creating a PLI Table File

In the multi-step compilation mode using IXCOM, `vlan` requires a PLI table file. The PLI table file is required to successfully parse the Verilog part of the design that can contain tasks or functions defined in the PLI application.

Following two types of schemes are supported for PLI table files:

- [Using Cadence Verilog-Compatible Scheme](#)
- [Using the Synopsys VCS-Compatible Scheme](#)

Using Cadence Verilog-Compatible Scheme

Perform the following steps to use the Cadence Verilog-Compatible PLI table file with IXCOM:

1. Compile the `veriusers.c` file and the other PLI source files to create a PLI application (a C/C++ shared library, such as `libpli.so`), using the same C/C++ compiler setup for IXCOM.
2. Use the PLI application in the following manner:

- a. In single-step compile mode, specify the `+loadpli1` or `+loadvpi` option with the `ixcom` command, specifying the library name and (optionally) any bootstrap routine names, along with other options.
- b. In multi-step compile mode, perform the following steps:
 - Specify the `vlan` command.
 - Generate the special IXCOM table file that contains the PLI definitions in the Verilog-XL compatible format, using the `ncpli2tab` utility. Specify the `+loadpli1` or `+loadvpi` option, with the `libpli.so` or `libvpi.so` library and any bootstrap routine names. Specifying the bootstrap routine names is optional.
 - Specify the name of the PLI table with the `vlan` command using the `-p` option.
 - Specify the `ixcom` command with the `+loadpli1` (for PLI1.0 shared library) or `+loadvpi` option (for VPI shared library), `-top <top module name>`, and other options.
 - Specify the `xeDebug` command to run the design.

Following table lists the Verilog-XL compatible syntax for a user-defined task and function entry in the PLI table file:

PLI Table Entry	Verilog-XL Compatible Syntax
user-defined task	<code>\$myplitask</code>
user-defined function	<code>\$myplifunc type=unsigned size=32</code>

Note: The Cadence Verilog-compatible scheme is the default PLI scheme for IXCOM.

Using the Synopsys VCS-Compatible Scheme

Perform the following steps to use the Synopsys VCS-Compatible PLI table file with IXCOM:

Note: This scheme can only be used if you have a VCS style table file containing PLI definitions.

1. Compile the `veriuser.c` file and the other PLI source files to create a PLI application (a C/CPP shared library, such as `libpli.so`), using the same C/C++ compiler setup for IXCOM.

2. Use the PLI application in the following manner:

- a.** In single-step compile mode, you do not need to create a PLI table file. In this case, IXCOM internally creates and uses a PLI table file when you specify the `+loadpli1`(for PLI 1.0 shared library) or `+loadvpi` option (for VPI shared library) with the `ixcom` command.
- b.** In multi-step compile mode, perform the following steps:
 - Specify the `vlan` command.
 - Create a special IXCOM table VCS-compatible file that contains the PLI definitions. You can also manually create this table file.
 - Specify the name of the PLI table with the `vlan` command using the `-P` option.
 - Specify the `ixcom` command with the `+loadpli1`(for PLI 1.0 shared library) or `+loadvpi` option (for VPI shared library), `-top <top module name>`, and other options.
 - Specify the `xeDebug` command to run the design.

Following table lists the VCS compatible syntax for a user-defined task and function entry in the PLI table file:

PLI Table Entry	VCS Compatible Syntax
user-defined task	<code>\$myplitask</code>
user-defined function	<code>\$myplifunc type=unsigned size=32</code>

Using the `ncpli2tab` Utility

The `ncpli2tab` utility takes a shared library, and optional bootstrap routine(s) information as input and provides information about the system tasks and functions that you have defined in a tabular format. This output can be saved in a file, which is called as the PLI table file, and is provided to IXCOM. The `ncpli2tab` utility generates the table file in the Verilog-XL format for the Verilog Compatible Scheme. IXCOM uses this generated tab file for information about user defined system tasks and functions. This utility is also internally used with IXCOM when you specify the `+loadpli1` or `+loadvpi` options and provide a PLI application, such as shared library.

ncpli2tab Options for Using PLI/VPI Features

Following options related to PLI files are supported by `ncpli2tab` utility in this release:

Note: The `ncpli2tab` utility only supports Verilog-XL format. The `-p` and `-P` options are not required.

- `+loadpli1=<arg>`: Used to specify a shared library containing the PLI 1.0 application.
- `+loadvpi=<arg>`: Used to specify a shared library containing the VPI application.
- `-o <arg>`: Used to specify the output file. If nothing is specified, `ncpli2tab` writes output to `stdout`.

Following is the syntax of using the `ncpli2tab` for PLI 1.0:

```
ncpli2tab +loadpli1="./libpli"  
ncpli2tab +loadpli1="./libpli:bootstrap_routine(s)"
```

Following is the syntax of using the `ncpli2tab` for VPI:

```
ncpli2tab +loadvpi="./libvpi"  
ncpli2tab +loadvpi="./libvpi:bootstrap_routine(s)"
```

PLI/VPI Options to IXCOC

IXCOM supports the following PLI/VPI options:

- `-p <arg>`: Provides information about PLI tasks in Verilog-XL format.
- `-P <arg>`: Provides information about PLI tasks in VCS format
- `+loadpli1=<arg>`: Provides a shared library containing the PLI 1.0 application. However, IXCOC internally generates a tab file in the Verilog-XL format for its usage.
- `+loadvpi=<arg>`: Provides a shared library containing the VPI application. However, IXCOC internally generates a tab file in the Verilog-XL format for its usage.

Combinations of Bootstrap Routines Supported by IXCOC

You can specify the bootstrap routines with the IXCOC command as values of the `+loadpli1` and `+loadvpi` options. These options are supported by both `ixcom` and `irun` commands.

Following combinations of bootstrap routines are supported by IXCOC:

- `ixcom +loadpli1="lib1:boot_routine1.boot_routine2"`

In the above command, the elaborator loads the library, *lib1*, and executes the bootstrap routine, *boot_routine1*. The simulator loads the library, *lib1*, and executes the bootstrap routine, *boot_routine2*. You use this command format when you want to run a PLI application only during simulation. Such a PLI application can perform tasks, such as recording the duration of a simulation or establishing communication with another simulation session.

Note: The bootstrap routine, *boot_routine2* cannot register new system tasks in addition to the ones registered by the bootstrap routine, *boot_routine1*.

- `ixcom`
`+loadpli1="lib1:boot_routine1,boot_routine2.boot_routine2"`

In the above command, the elaborator loads the library, *lib1*, and executes the bootstrap routines, *boot_routine1* and *boot_routine2*. The simulator loads the library, *lib1*, and executes the bootstrap routine, *boot_routine2*.

- `ixcom +loadpli1="lib1:boot_routine1"`

In the above command, the elaborator loads the library, *lib1*, and executes the bootstrap routine, *boot_routine1*. The simulator loads the library, *lib1*, and executes the bootstrap routine, *boot_routine1*.

- `ixcom +loadpli1="lib1:boot_routine1."`

In the above command, the elaborator loads the library, *lib1*, and executes the bootstrap routine, *boot_routine1*. The simulator just loads the library, *lib1*, and performs no other action.

- `ixcom +loadpli1="lib1:.boot_routine1"`

In the above command, elaborator loads the library, *lib1*. The simulator loads the library, *lib1*, and executes the bootstrap routine, *boot_routine1*.

Note: The above command format is typically used when you need to load and run PLI applications that do not register any system tasks.

- `ixcom +loadpli1="lib1:"`

In the above command, the elaborator loads the library, *lib1*. The simulator loads the library, *lib1*. Both use the `veriusertfs` array or `vlog_startup_routines`, if available in absence of any bootstrap routines or functions.

- `ixcom +loadvpi="vpilib:boot_routine1.boot_routine2"`

In the above command, the elaborator loads the library, *vpilib*, and executes the routine, *boot_routine1*. The simulator loads the library, *vpilib*, and executes the routine, *boot_routine2*.

- `ixcom`
`+loadvpi="vpilib:boot_routine1,boot_routine2.boot_routine3"`

In the above command, the elaborator loads the library, *vpilib*, and executes the routines, *boot_routine1* and *boot_routine2*. The simulator loads the library, *vpilib*, and executes the routine, *boot_routine3*.

- `ixcom +loadvpi="vpilib:boot_routine1"`

In the above command, the elaborator loads the library, *vpilib*, and executes the routine, *boot_routine1*. The simulator loads the library, *vpilib*, and executes the routine, *boot_routine1*.

- `ixcom +loadvpi="vpilib:boot_routine1."`

In the above command, the elaborator loads the library, *vpilib*, and executes the routine, *boot_routine1*. The period [*.*] at the end of the command line indicates that the simulator does not perform any action.

- `ixcom +loadvpi="vpilib:.boot_routine1"`

In the above command, the elaborator loads the library, *vpilib*. The simulator loads the library, *vpilib*, and executes the routine, *boot_routine1*. You typically use this command format to load and run VPI applications that do not register any system tasks.

- `ixcom +loadvpi="vpilib:"`

In the above command, both the elaborator and the simulator load the library, *vpilib*. No functions or routines are executed.

Restrictions on Using PLI/VPI Applications

While working with PLI applications, you need to take care of certain restrictions related to:

- Using a PLI/VPI Application in a Mixed-HDL Design
- Using a Verilog PLI Application in a Testbench
- Using a PLI/VPI Application in a DUT Module
- Statically Linking PLI/VPI Applications
- Naming PLI/VPI Applications

Using a PLI/VPI Application in a Mixed-HDL Design

If the design contains Verilog and VHDL portions, PLI/VPI tasks can only be called from the Verilog portion of the design. In addition, the PLI/VPI tasks can only be used to traverse and `set`, `force`, or `read` Verilog signals and variables.

PLI and VPI routines cannot be used to traverse the hierarchy of a VHDL design, and to `set`, `force`, and `read` VHDL signals and variables. You should make use of VHPI routines to perform these actions on the VHDL part of the design.

Using a Verilog PLI Application in a Testbench

When using a Verilog PLI application, consider the following:

- [Using Task/Function Routines Instead of Access Routines](#)
- [Referencing Nets and Registers Across the Hierarchy from a PLI Routine](#)
- [Referencing Nets and Registers in DUT Modules from the Testbench](#)

Using Task/Function Routines Instead of Access Routines

Following guidelines are recommended when writing Verilog PLI applications. This is because, Verilog PLI applications are run on the software side, and should be written efficiently.

- Use task/function routines instead of access routines in the PLI code. The task/function routines simulate more efficiently on IES and other compiled-code simulators.
- Use the `+tfReadOnly` option with the `ixcom` command, when the arguments of the PLI/VPI tasks and functions are read-only and not read-write.

Referencing Nets and Registers Across the Hierarchy from a PLI Routine

IXCOM compilation generates a configuration file, `hdl.var` in the `xc_work` directory. The `hdl.var` file contains the `-access +rw` argument, which gives read and write access to all signals in the entire design. This allows PLI/VPI application to access/update signals throughout the design hierarchy.

Cadence recommends that you do not change the values of `-access` argument as it might make the signals inaccessible to PLI/VPI application. If a PLI/VPI routine attempts to read/write to signals that were not accessible, an appropriate run-time error is generated, as shown below:

UXE User Guide

PLI/VPI and VHPI Support

Error from IUS:

```
ERROR:          ACC          VISNOR
Attempting to read from <full name of signal> which does not have read
access.
ERROR:          ACC          VISNOW
Attempting to place a value into <full name of signal> which does not
have write access.
```

Referencing Nets and Registers in DUT Modules from the Testbench

You can cross-reference nets and registers in DUT modules from the testbench for the purpose of using them in PLI calls, if these nets and registers are exported during compilation. You can export the nets and registers in following two ways:

- Signals are automatically exported if, you reference a DUT net or register in the testbench or other module running in software simulation mode.

For example, the following is a read reference to a DUT signal:

```
always @(posedge testbench.chip.sys_clk)
```

- Signals are explicitly exported using either the `$export_read` or `$export_frcrel` Verilog system tasks from a testbench module.

Using a PLI/VPI Application in a DUT Module

A DUT module containing PLI/VPI calls is run on the software side. This is because the emulator cannot directly call a PLI/VPI routine.

In case the DUT module containing the PLI/VPI calls is to be run on the emulator, all the PLI/VPI calls of the DUT module should be ignored. To ignore all the PLI/VPI calls from a DUT module, specify the `+rtlIgnTask+<pli_call_name>` argument with the `ixcom` command. The `pli_call_name` argument indicates the names of all the PLI/VPI tasks and functions that are called from the DUT module. This allows the DUT module to be run on the emulator.

Statically Linking PLI/VPI Applications

IXCOM does not support statically linked PLI/VPI applications. It is supported by IES for backward compatibility.

Naming PLI/VPI Applications

A boot-strap function is not necessary if the `veriusertfs` (for PLI) or `vlog_startup_routines` (for VPI) array of function pointers is initialized within the PLI or VPI application. However, if the shared library is not named as `libpli.so` (for PLI) or `libvpi.so` (for VPI), IES requires a boot-strap function.

Example: Using PLI and VPI Applications

Following example demonstrates how to use a PLI or VPI function. This example uses following three source files:

- `veriuser.c`: Contains the registration of the task that you have defined using a `veriusertfs` array. Therefore, PLI bootstrap routine is not required in this case.
- `myplitest.c`: Contains the implementation of the PLI task's call function.
- `design_file.v`: Contains HDL design that calls the PLI/VPI task.

veriuser.c

```
.....
.....
.....
.....
s_tfcell veriusertfs[] =
{
    /*** Template for an entry:
    { usertask|userfunction, data,
      checktf(), sizetf(), calltf(), misctf(),
      "$tfname", forwref?, Vtool?, ErrMsg? },
    Example:
    { usertask, 0, my_check, 0, my_func, my_misctf, "$my_task" },
    ***/
    /*** add user entries here ***/
    { userfunction, 0, 0, size1, mypli1, mypli1_misc, "$mypli1", 1 },
    { userfunction, 0, 0, size32, mypli2, mypli2_misc, "$mypli2", 1 },
    { userrealfunction, 0, 0, size64, mypli3, mypli3_misc, "$mypli3", 1 },
    { usertask, 0, 0, 0, mypli4, mypli4_misc, "$mypli4", 1 },
    {0} /*** final entry must be 0 ***/
};
.....
.....
```

```
.....
.....
```

myplitest.c

```
.....
.....
.....
.....
int mypli1()
{
    tf_putp(0, 1);
}
int mypli2()
{
    tf_putlongp(0, 12, 0);
}
int mypli3()
{
    tf_putrealp(0, 123.456);
}
int mypli4()
{
    io_printf("In mypli4 at time %d\n", tf_gettime());
}
.....
.....
.....
.....
```

design_file.v

```
module myTopMod;
    integer mypliretval;
    initial begin
        mypliretval = $mypl2;
        $mypli4;
    end
endmodule
```

Perform the following steps to use the PLI/VPI application:

- 1. Step 1:** Compile the C/C++ source files of the **PLI application** into a shared library.

Specify the following commands to compile the C or C++ source files of the PLI application into a shared library.

```
cc -m32 -fpic -c -g -I$CDS_INST_DIR/tools/include -DVXL myplitest.c -o myplitest.o
cc -m32 -fpic -c -g -I$CDS_INST_DIR/tools/include -DVXL veriusers.c -o veriusers.o
cc -m32 -shared -o libpli.so myplitest.o veriusers.o
```

Similarly, C/C++ source files for a **VPI application** can be:

- *vpi_user.c*: Contain the registration of the task that you have defined using a `vlog_startup_routines` function. Therefore, VPI bootstrap routine is not required in this case.
- *myvpitest.c*: Contains the implementation of the task's call function.
- *design_file.v*: Contains HDL design that calls the PLI/VPI task.

Specify the following commands to compile the C or C++ source files of the VPI application into a shared library:

```
cc -m32 -fpic -c -g -I$CDS_INST_DIR/tools/include -DVXL myvpitest.c -o myvpitest.o
cc -m32 -fpic -c -g -I$CDS_INST_DIR/tools/include -DVXL vpi_user.c -o vpi_user.o
cc -m32 -shared -o libvpi.so myvpitest.o vpi_user.o
```

Note: The above CC compilation commands are to compile a shared library on 32-bit linux platform and are only indicative. These commands vary according to the compiler and the operating system used.

2. Step 2: Run the `ixcom` command to compile the design and generates the snapshot. Then, run the `irun` command to simulate the design and initiate the calling function of the PLI or the VPI task.

- a. Single-step Compilation for a **PLI** application:** In this mode, specify the following commands to compile and run the design:

```
ixcom +loadpli1="./libpli" design_file.v other_options
xeDebug <input_tclfile>
```

- b. Multi-step Compilation for a **PLI** application:** In this mode, specify the following commands to compile and run the design. Multi-step compilation is used when the design contains some VHDL or SystemC portions.

- Run the `ncpli2tab` command with the PLI application to generate a Verilog-XL style table file, *mytfs.pli*, as shown:

```
ncpli2tab +loadpli1="./libpli" -o mytfs.pli
```

UXE User Guide

PLI/VPI and VHPI Support

- Run the `vlan` command with the Verilog-XL style table file, `mytfs.pli`, generated in the previous step, as shown. This step parses the HDL design.

```
vlan -p mytfs.pli design_file.v other_options
```

- Run the `vhan` command, as shown:

```
vhan design_file.vhd other_options
```

- Run the `ixcom` command to compile the design and generate the snapshot, as shown:

```
ixcom +loadpli="./libpli" -top myTopMod other_options
```

- Run the `xeDebug` command to simulate the design and initiate the calling function of the PLI task, as shown:

```
xeDebug <input_tclfile>
```

- c. Single-step compilation for a **VPI** application:** In this mode, specify the following commands to compile and run the design:

```
ixcom +loadvpi="./libvpi" design_file.v other_options
```

- d. Multi-step compilation for a **VPI** application:** In this mode, specify the following commands to compile and run the design. Multi-step compilation is used when the design contains some VHDL or SystemC portions.

- Run the `ncpli2tab` command with the VPI application to generate a Verilog-XL style table file, as shown:

```
ncpli2tab +loadvpi="./libvpi" -o mytfs.pli
```

- Run the `vlan` command with the Verilog-XL style table file generated in the previous step, as shown. This step parses the HDL design.

```
vlan -p mytfs.pli design_file.v other_options
```

- Run the `vhan` command, as shown:

```
vhan design_file.vhd other_options
```

- Run the `ixcom` command to compile the design and generate the snapshot, as shown:

```
ixcom +loadvpi="./libvpi" -top myTopMod other_options
```

- Run the `xeDebug` command to simulate the design and initiate the calling function of the VPI task, as shown:

```
xeDebug <input_tclfile>
```

VHPI Support in IXCOM

VHDL Programming Language Interface or VHPI is a standard interface that enables you to link C/C++ programs with the VHDL design.

A C/C++ application using VHPI interface can be integrated with a VHDL design in the following cases:

- To access various design elements, such as design hierarchy, connectivity information, and so on. This type of access is called static access.
- To monitor various states of a design, such as current simulation time, signal value, and so on. This type of access is called dynamic access.
- To call a C/C++ VHPI function/procedure from a VHDL design, and is termed as calling a foreign subprogram.
- To create the body of the VHDL architecture using C/C++ VHPI routines, and is termed as creating a foreign architecture.

For more information on VHPI use model and its features, refer to the *VHPI User Guide* in the INCISIV documentation set.

Restrictions on Using VHPI Applications

Consider the following limitations, while working with the VHPI applications:

- A foreign architecture or foreign subprogram must not be part of the DUT.
- Any VHPI application should not access the signals/variables falling in the DUT. This implies that VHPI application should not try to get the handle of any object present in the DUT.

Compiling and Running Designs with Assertions

Assertions represent the executable specification of a design and are primarily used to validate the behavior of a design. UXE supports assertions written in SystemVerilog and PSL, in addition to the Incisive Assertion Library (IAL) and Open Verification Library (OVL) components.

For a complete list of the SVA supported constructs, refer to [Support for Specific SVA Constructs](#) on page 565.

For a complete list of the PSL supported constructs, refer to [Support for Specific PSL Constructs in a Mixed-HDL Design](#) on page 579 and [Support for Specific PSL Constructs in a Verilog Design](#) on page 595.

The IAL components components are listed in the section titled [Support for Incisive Assertion Library Components](#) on page 606.

The OVL components are listed in the section titled [Support for Open Verification Library Components](#) on page 609.

 Important

The content of this chapter pertains only to the SA mode.

Assertions can be mapped to the Palladium XP engine for Verilog and VHDL designs by using the IXCOM compiler. This mapping allows powerful debugging features, including hot-swapping, to be extended to assertions.

 Caution

The performance of the software simulator is adversely affected as the number of assertions in the design increases. Therefore, mapping the assertions into the emulator provides an order of magnitude performance improvement over the software simulator.

Assertions can be included either as a part of the design (as embedded PSL/SVA) or by using external files (in verification units or vunits). Using vunits enables verification engineers to add assertions without modifying the design. Also, in this manner, assertions can be reused. For example, assertions written for one version of the design unit can be linked to another version of the design.

Methodology guidelines for compiling and running assertions for hardware are provided in [Appendix N, “Methodology Guidelines for Compiling Assertions with the IXCOM Compiler.”](#)

Assertion Examples

Examples of assertions are located in the following directory:

```
<UXE_installation_directory>/share/uxe/gift/tutorial/assertions
```

The `assertions` directory has the following two sub-directories:

- `PSL` - This directory includes examples of PSL assertions written within VHDL and Verilog modules.
- `sva` - This directory includes examples of SystemVerilog assertions written within Verilog modules.

Each example has an associated README file that describes the example, lists the related files in the directory, and provides instructions on how to compile and run the design.

Enabling the Use of SVA

SystemVerilog Assertions (SVA) are part of the SystemVerilog language; therefore, to compile assertions, you must use the `+sv` (SystemVerilog) option during compilation with the `ixcom` and `vlan` commands. (For more about compilation, refer to [Chapter 4, “Compiling Designs for Simulation Acceleration Using IXCOM.”](#))

The `+sv` option indicates that the SystemVerilog constructs and keywords should be honored. The assertions under the DUT will be mapped to the emulator, and the assertions in the testbench will run in software simulator. No additional option is required to map SVA constructs to the emulator. All assertion-containing modules that are under the hierarchy of the module specified with the `+dut` option will be mapped to the emulator.

For more information on the `+sv` option, refer to the option description in the *Commands and System Tasks for SA Mode* chapter in *UXE Command Reference Manual*.

Enabling the Use of PSL Assertions

To ensure that the PSL assertions are recognized, specific options need to be passed to the `vhan`, `vlan`, and the `ixcom` commands.

- `-psl` option enables parsing of PSL assertions in the DUT. Parsing of PSL assertions is *not enabled* by default. You can specify `-psl` option with `vhan`, `vlan`, and `ixcom` commands.
- `-propfile <file>` option ensures that PSL assertions specified in external files (other than the design file) are recognized and parsed. The `<file>` option refers to the external files containing PSL assertions, such as `vunits`. This option can be specified with `vhan`, `vlan`, and `ixcom` commands.
- `+no_psl` option ensures that the PSL assertions are not elaborated in the design. Also, you can specify selected modules with this option to ensure that PSL assertions in the specified modules are not elaborated. This option can be specified *only* with the `ixcom` command. (See [Disabling PSL Assertions in Selected Modules](#).)

Both the VHDL and Verilog flavors of PSL assertions are supported. Therefore, depending on the language, PSL assertions can be parsed and elaborated in following two modes:

■ Two-step compile mode

In this mode, `-psl` option is specified with the `vlan` command to parse PSL assertions in a Verilog design, and with the `vhan` command to parse PSL assertions in a VHDL design. After the PSL assertions are parsed, the `ixcom` command elaborates the PSL assertions at compile time.

■ Single-step compile mode

In this mode, `-psl` option is passed to the `ixcom` command that parses and elaborates the PSL assertions in a design. The single-step compile mode is applicable to Verilog designs only.

Disabling PSL Assertions in Selected Modules

You can disable elaboration of PSL assertions in specific modules of the DUT by using the `+no_psl+<mod_name_list>` option of the `ixcom` command. This option disables processing of PSL assertions in the modules specified in the `<mod_name_list>` list. To specify multiple modules, use the format, `+<mod_1>+<mod_2>`.

Valid combinations can be the following:

<code>+no_psl+mod_1</code>	Disables elaboration of PSL assertions in module, <i>mod_1</i> .
<code>+no_psl+mod_1+mod_2</code>	Disables elaboration of PSL assertions in modules, <i>mod_1</i> and <i>mod_2</i> .
<code>+no_psl</code>	Disables elaboration of PSL assertions in the entire design.

No additional option is required to map PSL constructs to the emulator. All the modules that have been specified with the `+dut` option, and have assertions will be mapped to the emulator.

Labeling PSL Assertions

For unique identification, Cadence recommends that each PSL assertion is appropriately labeled. A label enables easy identification of an assertion in all forms and reports. If you do not provide a label to an assertion, a system-generated label is used.

The syntax for labeling PSL assertions is:

```
[ Label : ] Verification_Directive    (IEEE Std. 1850-2005, Section 7.1)
```

For example:

For PSL assertions that are embedded in comments in the HDL, use:

PSL assertions in Verilog HDL:

```
// psl my_label: assert <assertion>
```

Or, PSL assertions in VHDL:

```
-- psl my_label: assert <assertion>
```

For PSL assertions that are defined within a vunit, use:

```
my_label: assert <assertion>
```

Command Options for Compiling Designs with Assertions

The following table lists the options for compiling designs with assertions. These options are syntactically and functionally the same for both `ixcom` and `irun -hw` commands.

Option	Description
<code>+sv</code>	Enables SystemVerilog assertions.
<code>-assert</code>	Enables PSL features in HDL files.
<code>-assert_vlog</code>	Enables PSL features in Verilog files.
<code>-assert_vhdl</code>	Enables PSL features in VHDL files.
<code>-noassert</code>	Disables PSL and SystemVerilog assertions.
<code>-propfile_vlog</code>	Enables all the PSL assertions found in the file specified with the <code>-propfile_vlog</code> option, and all the assertions in any other HDL files specified on the command line.
<code>-propfile_vhdl</code>	Enables all the PSL assertions found in the file specified with the <code>-propfile_vhdl</code> option, and all the assertions in any other HDL files specified on the command line.

The following table lists the advanced options that you should use only if you face any issues when compiling designs using the IXCOC compiler:

Option	Description
<code>+assertDisableCtrlOpt</code>	Disables optimization of assertion-control signals.
<code>+assertDumpPlaceholder[+<module>]</code>	Allows IXCOC compilation to proceed without dumping original assertions for these Verilog module(s).

UXE User Guide

Compiling and Running Designs with Assertions

Option	Description
<code>+assertDumpTable</code>	Dumps meta-information about all the assertions present in the design.
<code>+assertIgnoreActionBlock</code>	Ignores user-written Pass and Fail action blocks in concurrent assertions.
<code>+assertMaxIter +<N></code>	Enables concurrent assertions with the allowed <N> number of repetitions. By default, the value of <N> is 32.
<code>+noProcAsrtCtrl</code>	Disables creation of control signals for concurrent assertions in procedural blocks.

Enabling and Disabling Assertions at Compile Time

Use the `-featurefile` option of the `ixcom` command to enable or disable assertions as a feature at compile time. You need to specify the following keywords in the feature file to describe the assertions.

- ☐ `sva` - Enables or disables the SystemVerilog Assertions
- ☐ `psl` - Enables or disables the PSL assertions.
- ☐ `assert` - Enables or disables the SVA and PSL assertions.

The `-featurefile` option can be used only with the `ixcom` command. It cannot be used with the `vlan` and `vhan` commands. Using the `-featurefile` option controls the assertion synthesis. Therefore, only the assertions available after analysis can be disabled or enabled by using the `-featurefile` option.

Following is a brief description of how the assertion-related options can be specified in conjunction with the `-featureFile` option:

Analysis-specific Assertion Options: The `-psl`, `+no_sva` and `+sva` options of the `vhan` and `vlan` commands control the parsing or analysis of assertions. The assertions that are analyzed are available during elaboration for synthesis, and these can be controlled by using the `-featureFile` option.

Elaboration-specific Assertion Options: The `-featureFile` option can be used during elaboration for disabling or enabling synthesis of assertions. By default, all the analyzed assertions are synthesized. You can override this behavior by using the `-featureFile` option. Note that you can achieve the same objective by using the `+no_psl`,

`+no_sva` and `+sva` options, and override the default behavior; But you cannot use these options together with the `-featureFile` option to enable or disable SystemVerilog assertions. Use only one of these options, and recompile.

All other assertion-related options, such as `+assertCover`, `+assertStatus` etc. are still required and applicable to all the assertions selected for synthesis.

For a detailed description of the usage of the `-featureFile` option, refer to [Enabling and Disabling Features at Compilation Time](#).

Functional Coverage Support

The PSL and SystemVerilog Assertion (SVA) languages provide functional coverage information about the designs and are integrated with the Incisive Comprehensive Coverage Tool.

- [Concurrent Cover Statement](#)
- [Integration with Incisive Comprehensive Coverage Tool](#)

Concurrent Cover Statement

IXCOM uses a `concurrent cover` statement to give you the following counts:

- `Checked count` is the number of times the antecedent of the implication becomes true.
- `Finished count` is the number of times the property succeeded.
- `Failed count` is the number of times the property failed.

By default, `ixcom` keeps these coverage counters for SVA cover statements only. If you want coverage data for SVA `assert` and `assume` statements, use the `+assertCover` option with `ixcom`. (Refer to `+assertCover` section in the *Commands and System Tasks for SA Mode* chapter of the *UXE Command Reference Manual* for more detail.) IXCOM then keeps track of coverage counters for all concurrent assertion statements.

By default, all of these counters are one-bit counters; but you can increase their size using the following option of the `ixcom` command:

```
+assertCoverCnt+number
```

The *number* is the size of the counters. The larger the size of the counters, the larger the resources required in hardware to keep track of the counters. Refer to `+assertCoverCnt`

section in the *Commands and System Tasks for SA Mode* chapter of the *UXE Command Reference Manual* for more detail. The value of these counters is printed at the end of the simulation as the ASSERTION COVERAGE SUMMARY.

Integration with Incisive Comprehensive Coverage Tool

You can view functional coverage information using the Incisive Comprehensive Coverage tool. IXCOM writes out the checked, failed, and finished counts for each assertion in the coverage database, at run time. To enable this option, specify the following option with the `ixcom` command:

```
-coverage functional
```

The `-coverage functional` option creates the functional coverage database to store the functional coverage information. IXCOM records the coverage data in `cov_work/scope/test/icc_<design-checksum>.ucd` file, at run time. The `icc_<design-checksum>.ucd` file contains the functional coverage information that can be viewed through the ICC tool.

To invoke the ICC tool, specify the following command:

```
iccr -g
```

The Incisive Comprehensive Coverage viewer is located in the IES release tree.

SystemVerilog Bind Support

There are two ways to use SystemVerilog assertions in designs:

- You can instantiate the module that models the assertions. This requires you to make a change to your design. The advantage of this method is that the assertions travel with the design and can also be used by other tools, for example, formal verification tools.
- You can bind the assertion modules to the design modules or the instances of the design module using the SystemVerilog `bind` construct. The biggest advantage of this approach is that verification teams do not have to make any modification to the design RTL.

The SV `bind` construct is fully supported. Using the `bind` construct, you can, in effect, instantiate a module containing SVA in the design RTL. This construct also allows the flexibility of instantiating the assertion module inside only the selected instances of the target design module.

Using Assertion Control Tasks (ACTs) in HDL

IXCOM supports the following three assertion control tasks:

- `$asserton`
Re-enables the execution of all specified assertions.
- `$assertoff`
Stops checking of all specified assertions until a subsequent `$asserton`.
- `$assertkill`
Aborts execution of any currently executing specified assertions and then stops the checking of all specified assertions until a subsequent `$asserton`.

For IXCOM to be able to parse, elaborate, and dump these assertion control tasks, specify the `+assertStatus` option with the IXCOM command. If the `+assertStatus` option is not specified, the assertion control tasks are ignored.

Specifying Varying Widths for Assertion Counters

You can specify the widths of the assertion counters according to your requirements and control the resource usage on the emulator. The IXCOM compiler enables you to specify different widths for the Checked, Finished, and Failed counters.

Use the following options of the `ixcom` command to specify the different widths:

- ☐ `+assertCheckedCnt+<N>` - Use this option to set the width of the checked counter to the value N.
- ☐ `+assertFinishedCnt+<N>` - Use this option to set the width of the finished counter to the value N.
- ☐ `+assertFailedCnt+<N>` - Use this option to set the width of the failed counter to the value N.

Alternatively, you can set the widths of all the three types of counters to a single value by using the `+assertCoverCnt+<N>` option of the `ixcom` command. Using the above-mentioned options overrides the value set by the `+assertCoverCnt+<N>` option.

Refer to the *Commands and System Tasks for SA Mode* chapter in the *UXE Command Reference Manual* for details of these options.

Following are some examples of setting the widths of assertion counters:

UXE User Guide

Compiling and Running Designs with Assertions

- ❑ In the following example, the widths of the checked and finished counters are set to 8, and the width of the failed counter is set to 32-bit, which is the default setting.

```
ixcom +assertCover +assertCheckedCntr+8 +assertFinishedCntr+8 +sv test.v
```

- ❑ In the following example, the widths of the checked and finished counters are set to 16, and the width of the failed counter is set to 1 because the `+assertFailCounter+hw` option is included with this `ixcom` command.

```
ixcom +assertCover +assertCheckedCntr+16 +assertFinishedCntr+16 \  
      +assertFailCounter+hw +sv test.v
```

Note that 1-bit is the default setting with the `+assertFailCounter+hw` option.

- ❑ In the following example, the widths of all the counters are set to 12.

```
ixcom +assertCover +assertCoverCnt+12 +sv test.v
```

- ❑ In the following example, the widths of the checked and finished counters are set to 12. The width of the failed counter, however, is set to 32 because the `+assertFailedCntr+32` option is included in this `ixcom` command.

```
ixcom +assertCover +assertCoverCntr+12 +assertFailedCntr+32 +sv test.v
```

Running and Debugging Designs with Assertions

This section describes the procedure for exercising and debugging designs with assertions on Palladium XP systems in the SA mode. For compiling assertions in any of the other supported modes, for example, Dynamic Target or Synthesizable TestBench (STB), you need to compile the design with assertions using IXC.COM.

You can use the emulator to test the assertions that are expected to be exercised at the end of long verification runs, within reasonable *'real'* time, which would not be so achievable in traditional simulation. Running designs with assertions does not require any emulator resources in addition to those used during a normal run, except for the additional hardware gates used for assertion instrumentation. This solution enables you to see the assertion activity on the emulator as they do during simulation.

Examples of SVA Usage on page 529 provides some examples of compiling designs with SVA in various modes and debugging them using xeDebug tool.

Examples of PSL Assertion Usage on page 538 provides some examples of compiling designs with PSL in various modes and debugging them using xeDebug tool.

The *Assertions* tab allows you to perform the following tasks:

- Browsing Assertions

Browse the assertion hierarchy and enable and/or disable assertions at the module level by specifying the depth. The *Path* field specifies the current scope. You can filter the current displayed hierarchy by specifying the filtering condition in the *Filter* field. Select the assertions to be displayed with their depth, and click *To Display*.

■ Selecting Assertions

View the assertions in the selected module and enable and/or disable assertions individually. The selected assertions are then displayed. Additionally, filter the assertion list - *by type* and *by state* using the *Name Filter* field. The options in the *Name Filter* field are used to provide rapid specification of the assertions that are listed in the assertion table, based on the directive type, current state, and name patterns.

To operate on a subset of the compiled assertions, directly select one or more adjacent assertions from the Selection Filters.

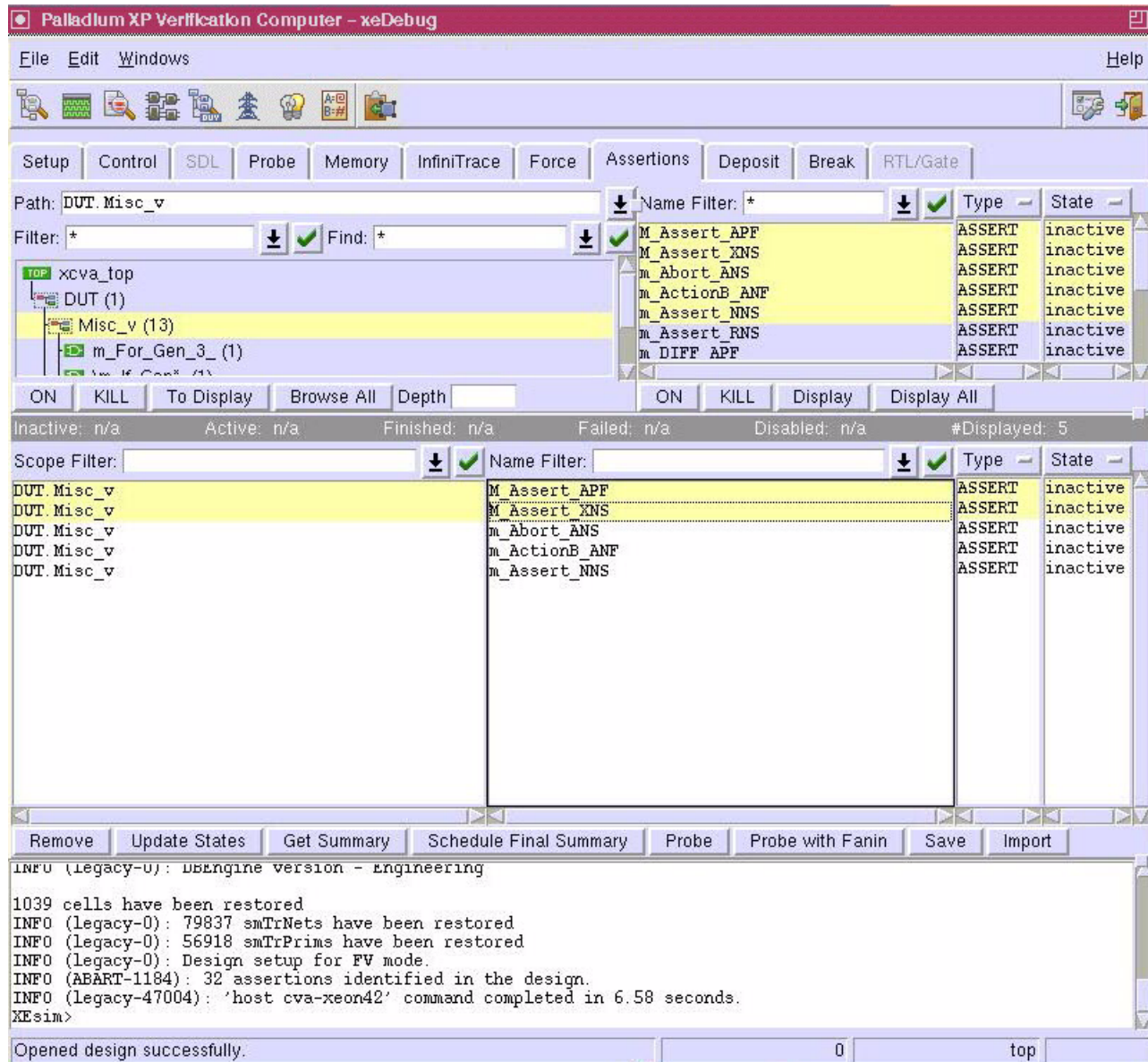
■ Displaying Assertions

The selected list of assertions are displayed in the assertion table. The selected assertions can be removed from the display and you can update states or choose to do so automatically after every run. Anytime the emulator is run, you must click the *Update States* button in order to see accurate assertion state status. The assertion states displayed in the assertion table read *n/a* when they are first entered to the *Assertion* tab, and whenever a `run` command is executed. They show *n/a* status until the *Update States* button is selected. The *Update States* button is also highlighted to indicate that it is required to refresh the state values.

UXE User Guide

Compiling and Running Designs with Assertions

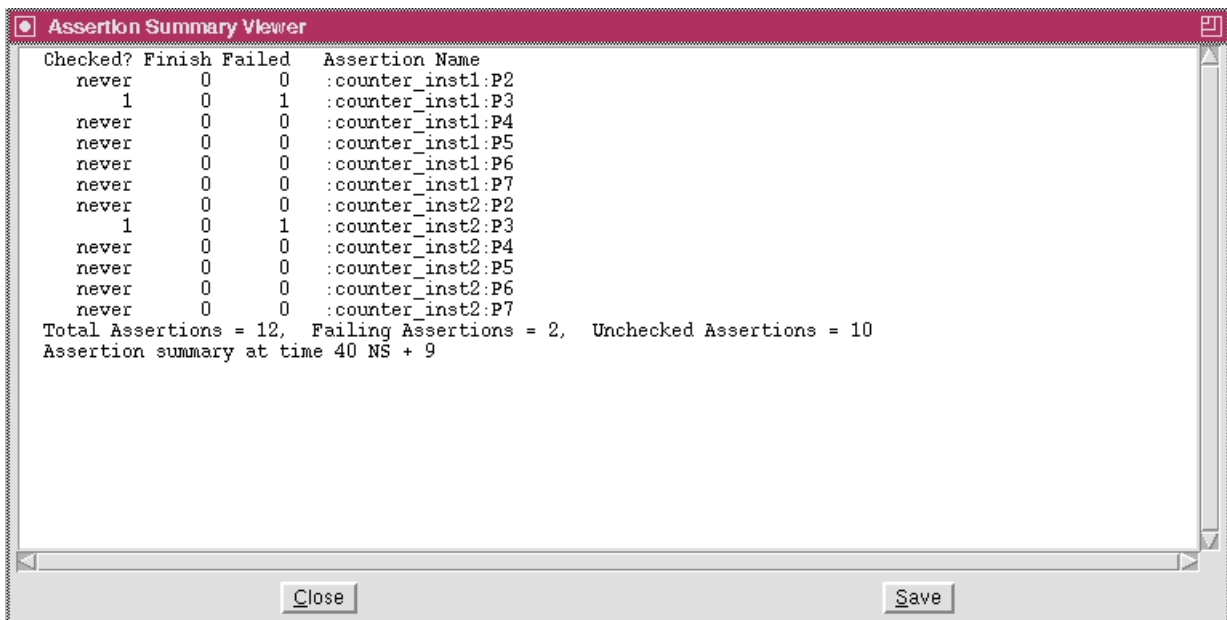
Figure 10-1 Assertions Tab



You can get a summary by clicking *Get Summary* button. The *Get Summary* button launches a window displaying the Assertion Summary report. You can get a summary or schedule a final summary by clicking the *Schedule Final Summary* button. The *Schedule Final Summary* button specifies that the summary report should not be generated immediately; instead the report is generated only on exiting from the xeDebug tool. Probe the selected assertions from the assertion table by clicking the *Probe* button. Save/import the list of displayed assertions by clicking the *Save* and *Import* buttons respectively.

UXE User Guide

Compiling and Running Designs with Assertions



Checked?	Finish	Failed	Assertion Name
never	0	0	:counter_inst1:P2
1	0	1	:counter_inst1:P3
never	0	0	:counter_inst1:P4
never	0	0	:counter_inst1:P5
never	0	0	:counter_inst1:P6
never	0	0	:counter_inst1:P7
never	0	0	:counter_inst2:P2
1	0	1	:counter_inst2:P3
never	0	0	:counter_inst2:P4
never	0	0	:counter_inst2:P5
never	0	0	:counter_inst2:P6
never	0	0	:counter_inst2:P7

Total Assertions = 12, Failing Assertions = 2, Unchecked Assertions = 10
Assertion summary at time 40 NS + 9

Close Save

Status bars above and below the assertion table provide status information. The status bar above the assertion table provides information on the number of unique assertion instances currently compiled into the Palladium XP DUT, the number of valid cover points in the compiled assertion set, the number of assertions currently enabled, and the number of assertions filtered to be displayed in the assertion table. The assertion table itself presents the status for each assertion displayed with respect to directive type, current status with respect to enabled/disabled and coverage database selection, and current state. The status bar below the assertion table reflects how many assertions are currently in each state. For this status to be accurate you should be sure to use the *Update States* button after each run to force updated state retrieval. Additionally, the console window provides visibility to the stdout stream and, thus, to any command success and error status.

Debugging Assertion Failures

At CLI prompts, you can disable or enable assertions (by default, assertions are enabled), request an assertion summary, or specify other assertion-related commands. At the time of running the `ixcom` compilation command, you can exclude instrumentation of selected assertions. Additionally, due to implementation restrictions, it might not be feasible to instrument some assertions and those will be ignored. There are no other means of including or excluding assertions.

All assertion failures are reported by the instrumentation. Assertion passes (finishes) are reported only when the SVA assertion has a pass-related action block. Pass-related action blocks are processed on the emulator.

If assertion failures are reported, by default, emulation continues until the specified run interval has been completed.

By default, the instrumented assertion model is tuned to use low or moderate capacity. If you need to debug assertions, specify the `+assertStatus` option with the `ixcom` command at the time of compiling the design. This debug choice of using the `+assertStatus` option entails the use of additional hardware resources and sacrifices some runtime performance.

If required, capacity and performance can be sacrificed to get more debug features by specifying the `+assertStatus` switch to `ixcom` command. This includes:

- Querying the status of the assertion and inspecting the fanins of the assertion
- Debugging using SimVision, Debussy
- Enabling/disabling assertions
- Other assertion run-time features supported by IES

When the `+assertStatus` option of the `ixcom` command is used, the assertions can be turned off and on at any point of time during simulation.

IES Simulator Required for Exercising Assertions

To enable the exercising of assertions on Palladium XP system, it is necessary to have the IES simulator running concurrently with the emulator for all the supported use models. **An IES license is always required to run designs with assertions.** Assertions are not supported in a non-simulator (ICE) mode. Therefore, for designs with assertions, even the emulation you need to compile using `IXCOM` and load the design into the simulator. A distinct advantage of the presence of the IES simulator is that it enables support for enhanced Hot Swap capabilities when running designs with assertions. Identical assertion support is provided for both the DUT design partitions in the emulator and simulator. The run-time engine transparently determines the active engine and makes the required adjustments accordingly.

The run-time functionality is defined by the compile-time instrumentation decisions, such as presence of counters and their widths. These decisions are specified by using the relevant `ixcom` command options.

For details of compiling designs with IXCOM, corresponding assertion compilation options, and the supported assertions set, refer to [Chapter 4, “Compiling Designs for Simulation Acceleration Using IXCOM.”](#)

The supported verification packages (OVL, IAL, VIP), languages (PSL, SVA) and various constructs are listed in the following sections:

- [Support for Specific SVA Constructs](#)
- [Support for PSL Constructs](#)
 - [Support for Specific PSL Constructs in a Mixed-HDL Design](#)
 - [Support for Specific PSL Constructs in a Verilog Design](#)
- [Support for Incisive Assertion Library Components](#)
- [Support for Open Verification Library Components](#)

Support for Assertions on Emulator

When assertions fail or pass in these modes, the emulator is stopped, and the control is passed to the simulator to process the event. When done, the changes are updated in the emulator and emulation continues.

Assertions are disabled or enabled by invoking the corresponding IES Tcl assertion commands. These will handle the setting/clearing of the value of the assertion-specific disable signal.

In the waveforms, the assertion state values and counters in the simulator databases or hardware-instrumented counters in the emulator databases can be traced. Use the `assertion -get` command, to access the assertion state value. Use the `assertion -summary` or `assertion -counter` command to access the counter values.

Note: In the SA mode, the testbench assertions reside only in the simulator partition.

The `reset` command in xeDebug can generate the following warning if assertions are disabled before specifying the `reset` command:

```
ncsim: *W,FRCACC: Potentially forced object, object does not have read and connectivity access
```

When this warning is issued, assertions are not re-enabled as expected. However, this warning is not generated when using *irun* tool.

Using Assertions with InfiniTrace

In the `infiniTrace -prepare` sessions, complete assertion support is provided. In the `infiniTrace -observe` session, the assertion support is limited to providing waveform support of assertion states, hardware-instrumented counters, and assertion state values. Tracing is supported only for assertion state values and hardware-instrumented counters.

Also, assertion run-time and `assertion -summary` commands are not supported in the `infiniTrace -observe` mode.

The following assertion-related commands are supported in the `infiniTrace -observe` mode:

- `assertion -get`
- `assertion -list`
- `assertion -check`
- `assertion -counter`
- `probe`

The following assertion-related commands are not supported in the `infiniTrace -observe` mode:

- `assertion -disable`
- `assertion -enable`
- `assertion -summary`

SDL Support for Assertions

SDL supports conditional expressions that reference assertions.

Specifying the assertion name implies a reference to its state value. For example, the following statement results in an SDL trigger if the `DUT.AssertAlwaysReady` assertion enters a `finished` state.

```
If ( DUT.AssertAlwaysReady == "finished" )
```

Note: All five legal assertion states are supported. Refer to [Assertion States](#) on page 526.

SDL expressions referencing assertion counters are not supported.

Invalid assertion names are treated the same as the signal names that cannot be found and result in the following kind of error messages when the `sdl -verify` command is executed:

```
Error in expression 'assert1: signal 'assert1' does not exist, file <sdlFilename>,
line <sdlFileLineNum>.
```

The pre-defined signal, `ASSERTION.FAILURE` enables triggering an action on an assertion failure.

For example:

```
if (ASSERTION.FAILURE)
{
    exec "puts \"Assertion Failure Detected\"";
    trigger;
}
```

This condition evaluates to true when any assertion fails. You can specify an action when this condition is met. For example, you can:

- Capture waveforms related to the context of assertion failure.
- Use the `assertion -list -failed` command to identify failed assertions. The returned list is not limited to the assertions that caused the trigger, but all the failing assertions from time 0.

Static Assertions

Static assertions are assertions for which all fanins are either Verilog parameters or VHDL generics. When detected, these assertion inputs do not vary in simulation and emulation and these failures are reported during compilation. No instrumentation for run-time failure or summary reporting of these assertions is created.

At compile time, the failure messages are generated in the same format as other compile errors; not in the run-time assertion failure format.

End-of-Sim Assertion Checks

Some assertions, such as a `strong eventually!`, if in the `active` state, are required to complete before the end of the session. If not satisfied, these are considered as failures, and must be reported. These end-of-sim evaluation checks are automatically performed when the session terminates.

These checks are not initiated when the entire design is swapped to the simulator using Hot Swap feature. When the design is swapped to the simulator, the emulator is released, but the session is not terminated.

The end-of-sim failure counts are included only in the `-final` assertion summaries.

probe Command

The `probe` command can be used with the following options to include assertions in the trancelist. This command can be used to trace both testbench and DUT assertions in SST2 format. For more details of this command, refer to the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

```
probe -create {<object> | <scopeName>}  
      -depth { n | all}  
      -assertions -state -signals
```

Pertaining to assertions, the above options perform the following actions:

<code>-create</code>	Creates a probe for the specified assertion(s) or the assertions within the specified scopes.
<code>-depth</code>	Selects assertions contained in instances within <code><n></code> or all levels of the specified scope.
<code>-assertions</code>	Specifies that only assertion objects should be selected from specified scopes. Note: Specific assertion pathnames can be probed without the <code>-assertion</code> switch.
<code>-state</code>	Captures and displays assertion state values. This is the default option.
<code>-signals</code>	The <code>-signals</code> option adds the fanin signals for the specified assertions (that you have either enumerated or included in the specified scope).

Breakpointing Assertions

Assertion breakpointing is supported using the existing native IES `stop` command and `assert_stop_level` and `assert_output_stop_level` Tcl variables. These variables are described in the *INCISIV Assertion Checking in Simulation* manual.

Support for Assertion Fanin Signals

To debug assertion failures, it is usually necessary to identify the assertion inputs and their values leading up to the failure. In addition to knowing the assertion state and counters, knowledge of the fanin signals is required to probe these without probing the entire scope.

To address this requirement, the `-signals` option is provided with the `probe` command and the `assertion -list` command. The syntax of the command options to enable support for assertion fanin signals browsing support is consistent with the syntax used in the `irun` tool.

In general, the options of the `probe` and `assertion` commands apply to all those objects that have been named in the command and for which these options make sense. For example, the `-depth` option applies to all the scopes and the `-signals` option applies to all assertions, and so on, regardless of their position in the command. These options do not apply only to the immediately preceding or following objects.

When you use the `-signals` option with either of the `probe` or `assertion -list` commands, the assertion fanin signals are automatically selected, provided any assertions are specified. You can use this option with any other options of the `probe` and `assertions -list` commands, and even with the Dynamic Target mode.

The `-signals` option of the `probe` command adds the fanin signals only for the specified assertions (that you have either enumerated or included in the specified scope):

```
probe [[-assertions] [-signals]]... <pathnames> ...
```

You can specify *locator* arguments including multiple pathnames and scopes with the `-signals` option.

```
assertion -list -signals [-depth <level>] \  
[-directive <directiveList>] <pathnames>...
```

The list of fanins is returned as a Tcl output in the form of a Tcl list of Tcl lists. Each top-level list identifies one selected assertion, and its corresponding list of fanins:

```
{<assertionName1> {<faninSig1> <faninSig2> ... <faninSigN>}  
{<assertionName2> {<faninSig1> <faninSig2> ... <faninSigN>}  
...  
{<assertionNameN> {<faninSig1> <faninSig2> ... <faninSigN>}}
```

The pathnames of all the assertions and fanin signals include the full hierarchy.

You can, for example, use the following code snippet to see a listing of all the assertions with fanin signals:

```
foreach assertEntry [assertion -list -signals -depth all top] {  
    puts "Assertion:  [lindex $assertEntry 0] "  
    foreach faninSig [lindex $assertEntry 1] {
```

UXE User Guide

Compiling and Running Designs with Assertions

```
        puts "$faninSig"
    }
}
```

When you use the `-signals` option with other options of the `assertions -list` command, the Tcl output uses the above-mentioned nested-list formatting.

Keep the following points in mind when probing fanins of the assertions:

- ❑ The `-signals` option is not applicable to non-assertion objects. If you include only non-assertion pathnames, and have not specified any scope arguments, a warning is generated. So, if you use the `-signals` option with the `probe` command, you must specify the pathnames of the assertions.
- ❑ The `-assertions` option is applicable only for scopes. If you include the `-assertions` option, but no assertions are included in the specified scope(s), a warning is generated. If an assertion pathname is explicitly specified, the assertion and its fanin signals are probed. The `-all` option also implies the `-assertions` option. In the above cases, the probe action will still be processed.

Examples of Using the -signals Option

Here are a few examples of using the `-signals` option:

The following command probes all the assertions in the DUT hierarchy and all of their fanin signals. It also probes the `top.Assert1` assertion and its fanin signals.

```
probe -create -depth all -assertions -signals top.DUT top.Assert1
```

The following command probes all objects (signals and assertions) in the `top.DUT.Inst1` instance. It also probes the `top.Assert1` assertion and its fanin signals. The `top.Assert1` is not a scope. So, the `-depth` and `-all` options do not apply to it. In this case, the fanin signals for any assertions within the `top.DUT.Inst1` instance are already probed because of the use of the `-all` option. These will not be duplicated.

```
probe -create -all -signals -depth 1 top.DUT.Inst1 top.Assert1
```

The following command probes the `top.DUT.Assert_Always` assertion and its fanin signals, and the `top.clk` signal. Because the assertion pathname is already specified, the `-assertion` option is not required. Here, `top.clk` is not an assertion. So, the `-signals` option does not apply to it.

```
probe -create -signals top.DUT.Assert_Always top.clk
```

Using the Assertions Tab of the xeDebug GUI to View Fanin Signals

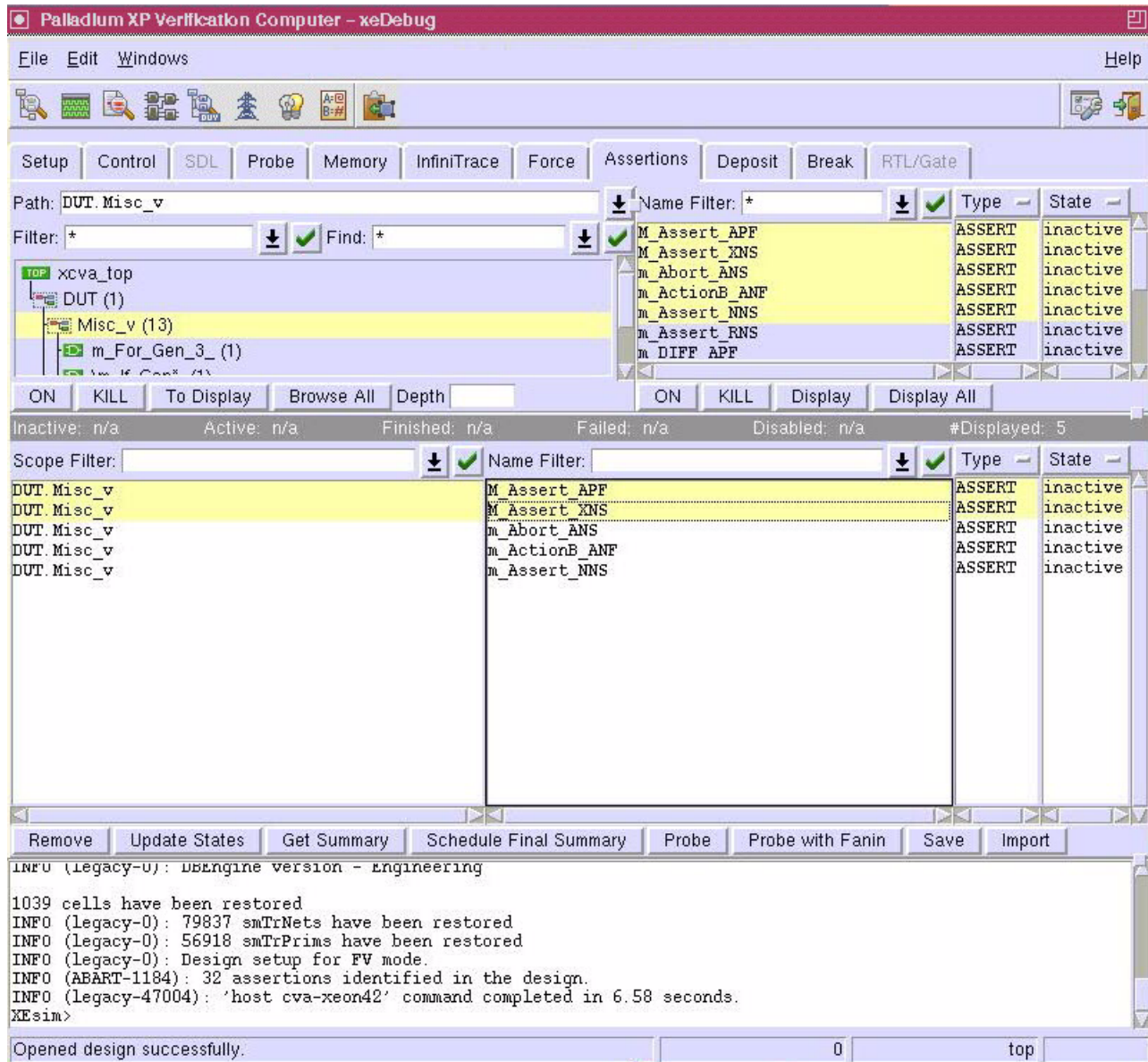
To view the values of the assertion fanin signals, in addition to the `probe` and `assertion` XEL commands, you can use the *Probe with Fanin* button on the *Assertions* tab of the xeDebug GUI as shown below. You do not explicitly specify the assertions in the *Probe* tab of the xeDebug GUI. You only select from the Assertions tab by selecting their parent scope. Then, the fanin signals will be probed through the scope, without needing any other action.

These fanin signals are included in the probe list, but these are not directly associated with the assertion.

UXE User Guide

Compiling and Running Designs with Assertions

This means that unlike the assertion counters, these signals are not displayed *within* the assertion object in the waveforms. Also, like other objects, even if the same fanin signals are probed for multiple assertions, these are displayed only once.



Assertions within Protected IP

Assertions within protected IP are supported. If a protected assertion fails, the failure message will only indicate the time at which the assertion failed, and not provide any other

details, such as assertion name. Assertion disabling will not be supported for these protected assertions.

Protected assertions are not included in the assertion summary. However, when these exist in the design, the summary will be followed by this message:

```
Part of the design was protected. Affected assertions are not shown.
```

For details on handling of protected assertions, refer to the “*Writing Assertions for Protected IP*” in the “*Assertion Writing Guide*” available in the INCISIV documentation set.

Coverage Support

If the design has been compiled with the `-coverage functional` option of the `ixcom` command, assertion coverage is supported at run time.

Assertion Reporting

Assertion status information is provided to you in three forms:

- ☐ Waveforms
- ☐ Messages provided for failed and passed assertions
- ☐ Assertion summary report

This information is written to stdout, the simulator log file (`ncsim.log`), and the emulator run-time log file (`xeDebug.log`).

The following table summarizes the assertion-related output.

Table 10-1 Assertion Reporting Output

	stdout	xeDebug.log	xe.msg	ncsim.log
Failure/Pass	Yes	Yes	Yes	Yes
Summary Report	Yes	Yes	No	Yes
Echoing of Commands	No	No	Yes	No

Assertion Summary Report

The assertion summary report is of the following form:

Checked?	Finish	Failed	Assertion Name
31	17	14	test.TB_A1
31	22	5	test.inst1.A1
0	0	0	test.inst1.ABORT1 (./dut.vp:45)
24	24	0	test.inst1.Cover2 (./dut.vp:40)
52	0	0	test.inst1.EVENT1 (./dut.vp:42)
28	22	6	test.inst1.MP_PROT_ASSERT
1	0	1	test.inst1.StaticAlways (./dut.vp:20)

Total Assertions = 7, Failing Assertions = 3, Unchecked Assertions = 1
Assertion summary at time 200 NS + 0

Contents of the Assertion Summary Report

Each entry displays the checked, finished, and failed counts (or subset specified by the `-show` option) with full hierarchical assertion path.

The count for any of the assertion counters listed in the summary report freezes on the maximum value for the counter bit width after that value has been achieved. There is no way to differentiate between a counter which has attained or surpassed the maximum value.

A footnote summary briefly indicates the total numbers of assertions — those that have failed at least once and those that were never exercised.

In the summary, the protected assertions that share the same ‘visible’ hierarchical pathname are consolidated into a single entry.

Both testbench and DUT assertions are presented in the same summary report.

Debugging Assertions Using Waveforms

You can choose between SimVision and Debussy browser tools for viewing the assertion waveforms. Correspondingly, you need to specify the database format as SHM (default) for SimVision or FSDB for Debussy waveform tool. SimVision integration with the xeDebug tool enables display of assertion waveforms and the Assertion Browser data from both the emulator and simulator engines.

UXE User Guide

Compiling and Running Designs with Assertions

If the Debussy waveform tool is to be used, you will have to use the licenses for Debussy. A loose integration with Debussy, in terms of compatible database formats for debugging capabilities using Debussy is provided by UXE. This support does not include the Property Result Table feature (Debussy Assertion Browser). However, if you independently license the Property Result Table, it will display correct results. For more information on integrating Novas Tools, refer to [Appendix 20, “Using Novas Tools.”](#)

The VBF and VCD trace formats do not support assertion objects and using these settings will result in only the raw instrumentation signals.

Important

To be able to debug assertions, it is essential that at compilation time, the `+assertStatus` option should have been specified with the `ixcom` command.

Assertions are supported in both testbench (software simulator) and DUT (emulator). Emulator assertions can be viewed using Debussy and SimVision. For the simulator assertions, waveforms are dumped only in the SST2 format and only the SimVision browsers are supported. Simulator assertions include the testbench assertions and all assertions after Hot Swap to simulator.

When a portion of the trace interval is simulated after a Hot Swap, the simulator waveforms will be dumped to a separate SST2 database. At viewing, as with other traceable objects, the assertion waveforms can be included in both a simulator and emulator trace database. The relevant waves will be in the database for which that assertion was processed during the desired interval.

When the Assertion Browser window is opened, the displayed values are associated with the waveform time selected. This functionality is analogous to the IES-only simulation. All of the assertions and related data are presented in the same Assertion Browser window regardless of which engine is currently processing the assertion.

No assertions or corresponding fanin signals are automatically added to the tracelists – you must add these signals explicitly.

Assertions can be added to tracelists and included in subsequent uploads analogous to other trace objects. The assertion trace-related attributes include:

- ☐ Assertion state value
- ☐ Tracing of assertion counters, if instrumented with the `+assertCover` option of the `ixcom` command

Adding assertions to the tracelist is accomplished using the `probe` command. If assertions are explicitly selected or contained within the selected scope, these are included in the tracelist.

Assertion States

The assertion strobe provides visibility into the state of the assertions, which include:

<code>off</code>	Indicates that the instrumentation for the particular assertion has been made non-active by an <code>assertion -disable</code> command.
<code>failed</code>	Indicates that the fulfilling condition of the assertion has evaluated to false and represents an assertion failure.
<code>finished</code>	Indicates that the fulfilling condition of the assertion has evaluated to true or has discharged without failing. In the case of a cover directive, this signals a coverage completion.
<code>active</code>	Indicates that the assertion is enabled. The first sub-element of the enabling condition is satisfied and the assertion has not finished or failed.
<code>inactive</code>	Indicates that the assertion is enabled. Currently, there are no matches with the sub-elements of the enabling condition for the assertion.

The above list also indicates the priority of the assertion states. This means that **OFF** has the highest priority and **INACTIVE** has the lowest priority. If multiple attempts of a given assertion are running simultaneously, the state probe indicates the state with the highest precedence. For example, if an assertion simultaneously passes and fails, the state probe will display **FAILED**.

Assertions-Related Compile-Time Command Options

Certain `ixcom` command options related to assertions are mandatory at compile time for correct run-time assertion behavior.

- If the `+assertStatus` option has not been specified with the `ixcom` command, assertion values and waveforms will not be available at run time. A warning message will be produced and the command for getting the assertion state values will be ignored.
- If the `+assertCover` option has been specified with the `ixcom` command, the coverage information (passed, failed, and finished coverage counters) is added for all `assert/assume` statements in the assertion summary report. If this option is not specified, counters are provided only for `cover` statements..

- If the `-coverage functional` option is used, assertion coverage will be supported.

Assertion Run-Time Commands

At run time, you can use various run-time interface commands to:

- Obtain detailed assertion summary reports
- Debug any unexpected results or assertion failures using assertion logs and waveforms
- Modify the testbench based on coverage results
- Control assertions by enabling or disabling assertions
- Manage probes and uploads
- Terminate or interrupt conditional emulation based on assertion failure

The assertion command set is primarily used to configure the assertion environment and produce the required analysis data. These XEL commands only affect the state of the assertion instrumentation and its settings in the XEL environment. These commands do not cause any simulation time to be run.

Following is the list of assertion run-time commands:

```
assertion -check
assertion -counter
assertion -get
assertion -off
assertion -list
assertion -on
assertion -summary
```

For a description of the assertion run-time commands, refer to *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

These commands are available at any time in the CLI after the design has been loaded. These are case-insensitive and can be abbreviated to minimum unique characters. These command invocations are echoed to the `xe.msg` log file.

Most of the assertions-related behavior is the same irrespective of the DUT being run in the emulator or simulator.

All assertions are initially active. Use `assert_stop_level` variable of *irun* to specify the stop severity level.

Assertion Tcl Variables

The following table identifies the IES assertion-related Tcl variables. Only some of these are supported in xeDebug and some of those only partially. These variables can be set or queried in the xeDebug Tcl environment as well as the nested IES Tcl interpreter using the `ncsim` prefix.

Refer to the IES documentation for further description of these variables.

Table 10-2 Assertions-Related Tcl Variables

Variable	Supported (Y/N)	Comments
<code>assert_1164_warnings</code>	Y	Full support
<code>pack_assert_off</code>	Y	Full support
<code>severity_pack_assert_off</code>	Y	Full support
<code>assert_stop_level</code>	Y	Supported in all modes except Dynamic Targets
<code>assert_output_stop_level</code>	Y	Supported in all modes except Dynamic Targets
<code>assert_report_level</code>	N	
<code>assert_stop_reason</code>	Y	Supported in all modes except Dynamic Targets
<code>assert_count_attempts</code>	N	Limited compile-time support
<code>assert_count_traces</code>	N	Limited compile-time support
<code>assert_report_incompletes</code>	N	Limited compile-time support

Examples of SVA Usage

This section contains different examples of SystemVerilog Assertion (SVA) usage:

- [Example 1: Using Concurrent Assert Statements](#)
- [Example 2: Using Concurrent Cover Statements](#)
- [Example 3: Using SV Bind](#)
- [Example 4: Functional Coverage Information](#)

Example 1: Using Concurrent Assert Statements

In this example of how to use a concurrent assert statement, the property `P1` tests for signal *a* to be high in the first clock cycle, followed by signal *b* to be high, followed by *c* to be high in the next cycle.

```
`timescale 1ns/100ps
```

```
module top;
```

```
    reg [3:0] in1;
    reg [3:0] in2;
    reg clk;
    wire [4:0] out1;
```

```
    initial
    begin
        clk = 0;
        #100 $finish;
    end
```

```
    always
    begin
        #10 clk = ~clk;
    end
```

```
    always
    begin
        #9;
        in1 = $random;
        in2 = $random;
```

UXE User Guide

Compiling and Running Designs with Assertions

```
#11;
end // always begin

assert_model
inst1(clk,in1[3],in1[2],in1[1],in1[0],in2[3],in2[2],in2[1],in2[0]);

always @(posedge clk) begin
    $strobe("%t strobe: abcdefgh = %b%b ", $time, in1, in2);
end // always @ (posedge clk)

endmodule

module assert_model (clk,a,b,c,d,e,f,g,h);

    input a,b,c,d,e,f,g,h;
    input clk;

    property P1;
        a ##1 b ##1 c;
    endproperty

    A1:assert property (@(negedge clk) P1)
    $display("Assertion A1 has passed: ",$time);

endmodule
```

To run this design, perform the following steps:

1. Save it in a file as `sva_example.v`.

2. Run the following `ixcom` command:

```
ixcom +sv -ua +dut+assert_model sva_example.v
```

3. Run the following `xeDebug` command:

```
xeDebug <input_tclfile>
```

The default action when the assertion fails is to call `$error`. In this example, the assertion fails at time 0, time 20ns, and time 80ns.

Example 2: Using Concurrent Cover Statements

To see how to use a concurrent cover statement, change the module `assert_model` from Example 1 to the following:

UXE User Guide

Compiling and Running Designs with Assertions

```
module assert_model (clk,a,b,c,d,e,f,g,h);

    input a,b,c,d,e,f,g,h;
    input clk;

    property P1;
        a ##1 b ##1 c;
    endproperty

    A1:cover property (@(negedge clk) P1)
    $display("Assertion A1 has passed: ",$time);

endmodule
```

To run this design, perform the following steps:

1. Save it in a file as `sva_example2.v`
2. Run the following `ixcom` command. By default, only 1 bit of counters is kept. To increase the size of the counters, specify `+assertCoverCnt+10` as an option to the `ixcom` command:

```
ixcom -ua +dut+assert_model +sv +assertCoverCnt+10 sva_example2.v
```

3. Run the following `xeDebug` command

```
xeDebug <input_tclfile>
```

Example 3: Using SV Bind

In examples 1 and 2, you instantiated module `assert_model` inside module `top`. This requires a change in module `top`.

To keep from having to change module `top`, you can create a `bind_module` with the following information in the file `bind.v`:

```
module bind_model;

bind top assert_model inst1(clk,
in1[3],in1[2],in1[1],in1[0],in2[3],in2[2],in2[1],in2[0]);

endmodule
```

The `bind` construct lets you effectively instantiate module `assert_model` inside module `top`.

UXE User Guide

Compiling and Running Designs with Assertions

Your module *top* will now be:

Note: There is now no instantiation of the module *assert_model* in the module *top*.

```
`timescale 1ns/100ps

module top;

    reg [3:0] in1;
    reg [3:0] in2;
    reg clk;
    wire [4:0] out1;

    initial
    begin
        clk = 0;
        #100 $finish;
    end

    always
    begin
        #10 clk = ~clk;
    end

    always
    begin
        #9;
        in1 = $random;
        in2 = $random;
        #11;
    end // always begin

    always @(posedge clk) begin
        $strobe("%t strobe: abcdefgh = %b%b ", $time, in1, in2);
    end // always @ (posedge clk)

endmodule

module assert_model (clk,a,b,c,d,e,f,g,h);

    input a,b,c,d,e,f,g,h;
    input clk;
```


UXE User Guide

Compiling and Running Designs with Assertions

```
property P1;
  a ##1 b ##1 c;
endproperty

A1:cover property (@(negedge clk) P1)
$display("Assertion A1 has passed: ", $time);

endmodule
```

To run this design, perform the following steps:

1. Save it in a file as `sva_example3.v`.

2. Run the following `ixcom` command:

```
ixcom +sv sva_example3.v bind.v
```

3. Run the following `xeDebug` command:

```
xeDebug <input_tclfile>
```

You will get the same output from the simulator as in examples 1 and 2.

Example 4: Functional Coverage Information

In this example, you will see NC-Sim dumping assertions in `sst2` format and related functional coverage information for the Incisive Comprehensive Coverage tool.

```
`timescale 1ns/100ps

module tb(clk,a,b,c,d,e,f,g,h);
  output reg a,b,c,d,e,f,g,h;
  output reg      clk;

  initial
  begin
    clk = 0;
    #100 $finish;
  end

  always
  begin
    #10 clk = ~clk;
  end
end
```

UXE User Guide

Compiling and Running Designs with Assertions

```
always
  begin
    #9;
    {a,b,c,d} = $random;
    {e,f,g,h} = $random;
    #11;
    end //always begin

always @(posedge clk) begin
  $strobe("%t strobe: abcdefg = %b%b%b%b%b%b%b%b ", $time, a,b,c,d,e,f,g);
end
endmodule // tb

module top;
  wire a,b,c,d,e,f,g,h;
  wire clk;

  tb TB(clk,a,b,c,d,e,f,g,h);
  assert_model inst1(clk,a,b,c,d,e,f,g,h);

endmodule

module assert_model (clk,a,b,c,d,e,f,g,h);
  input wire a,b,c,d,e,f,g,h;
  input wire clk;
  property P1;
    a ##1 b ##1 c;
  endproperty
  property P2;
    b ##1 d;
  endproperty
  A1:cover property (@(negedge clk) P1)
    $display("Assertion A1 has passed: ",$time);
  A2:cover property (@(negedge clk) P2)
    $display("Assertion A2 has passed: ",$time);
endmodule
```

To run this design, perform the following steps:

1. Save it in a file as `sva_example4.v`.

UXE User Guide

Compiling and Running Designs with Assertions

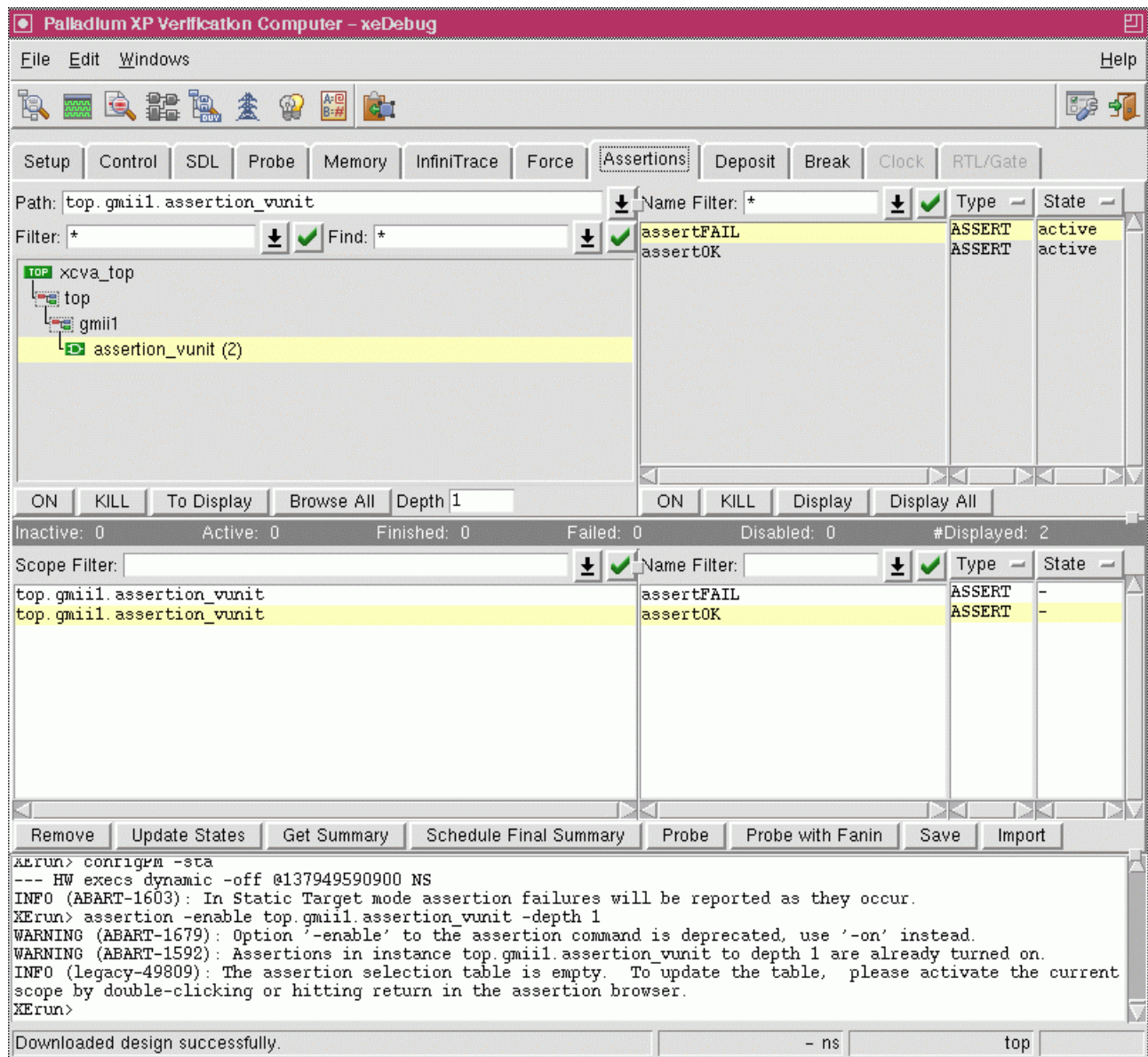
2. Run the following `ixcom` command:

```
ixcom +dut+assert_model +sv +assertStatus +assertCoverCnt+10 -coverage  
functional -ua +dut+assert_model sva_example4.v
```

3. Run the following `xeDebug` command to connect to the emulator:

```
xeDebug -gui
```

Figure 10-2 Assertions Tab



4. Run the following commands:

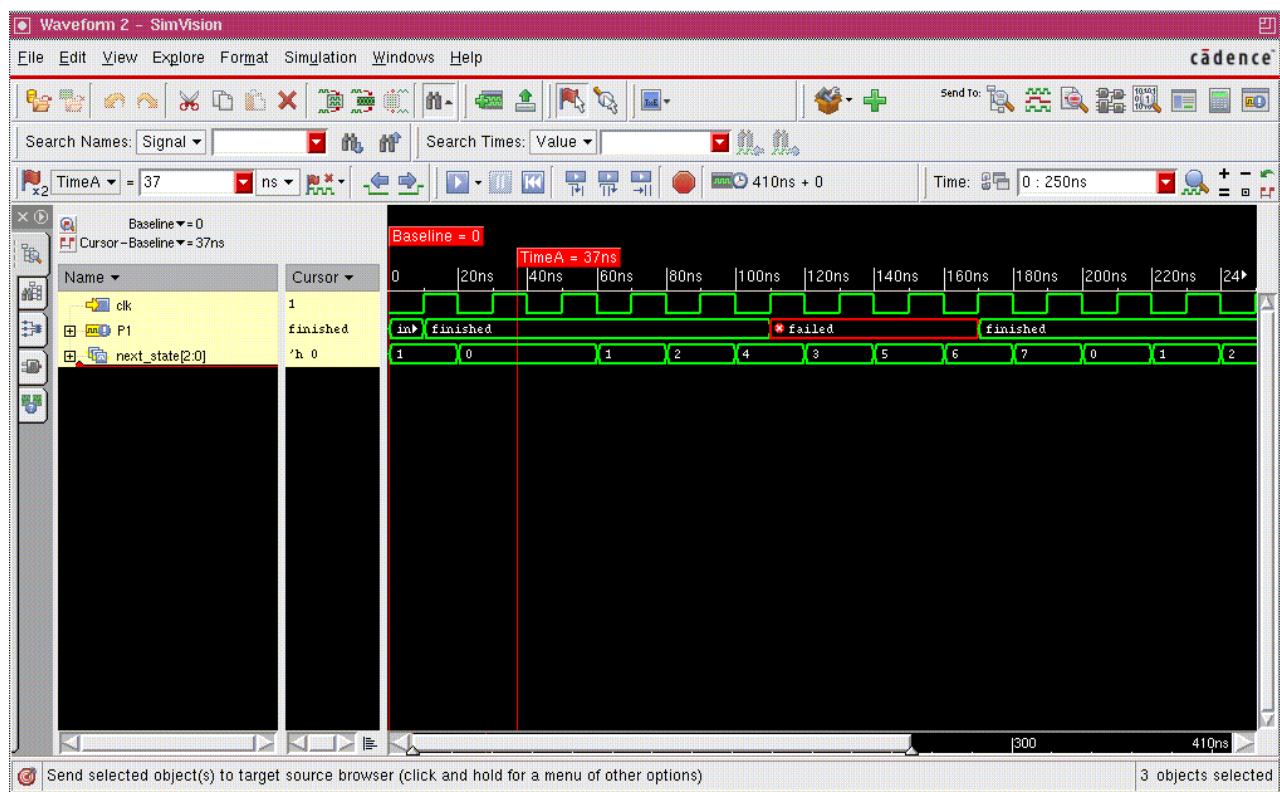
UXE User Guide

Compiling and Running Designs with Assertions

```
xc on -xt0 -zt0 -run  
run
```

To view the simulation waveforms in SimVision, open the simulation database inside SimVision. Highlight the assertion signals, and send them to target waveform windows. In the waveform in [Figure 10-3](#), two assertions are shown. Their counters (that is, checked count, finish count, and failure count) can also be shown by expanding their corresponding assertion signals.

Figure 10-3 SimVision Waveform Window

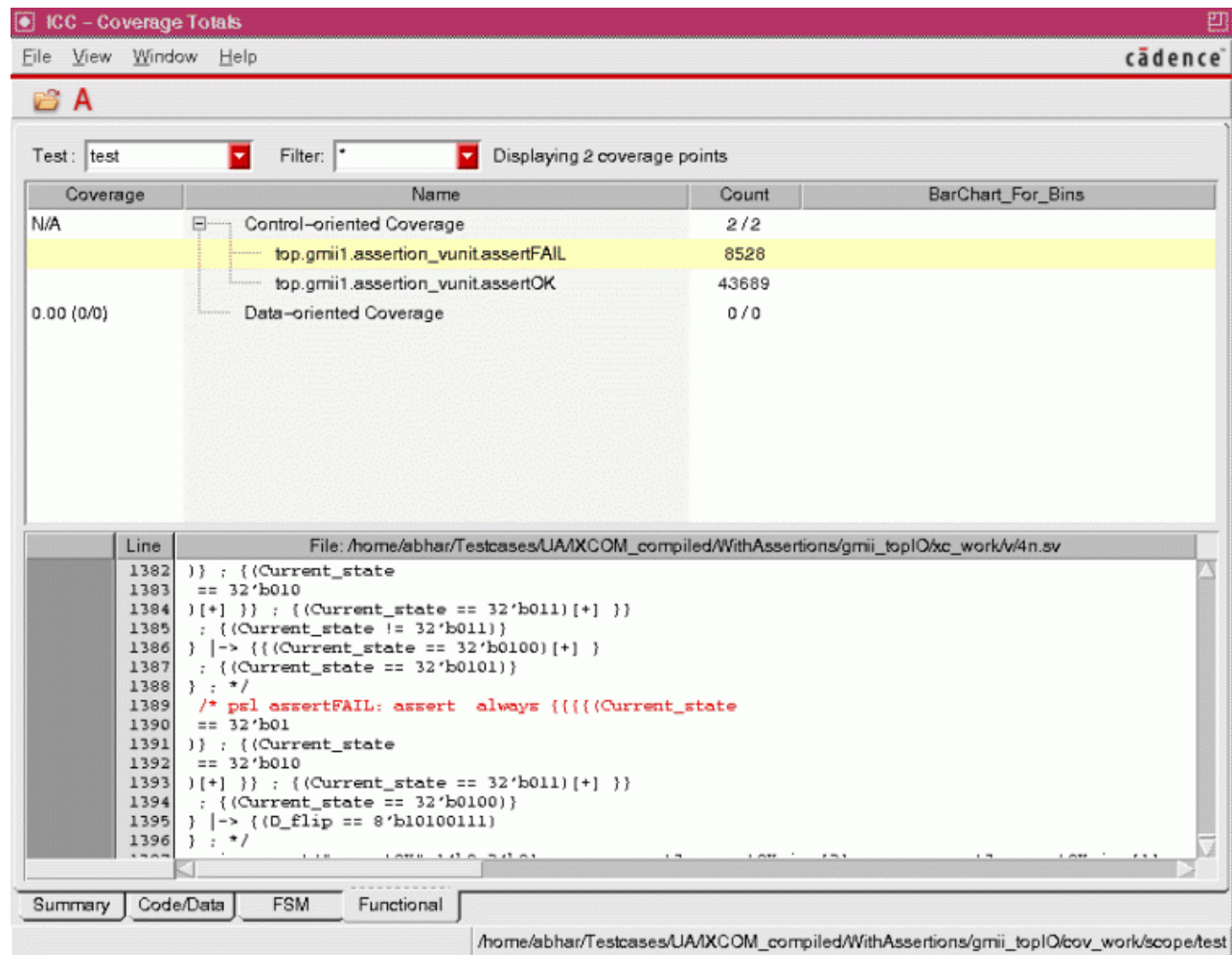


To view functional coverage information using the Incisive Comprehensive Coverage tool (ICC), launch the application in GUI mode and then open the functional coverage file. In ICC, only the finished count is captured. See [Figure 10-4](#) and [Figure 10-5](#). Only one of the assertion signals out of two is covered. So, the control-oriented coverage is 50%.

UXE User Guide

Compiling and Running Designs with Assertions

Figure 10-4 Incisive Comprehensive Coverage Tool



Examples of PSL Assertion Usage

This section contains two examples of using PSL assertions in a Verilog or a VHDL design:

- [Example 1: Using Embedded PSL Assertions \(VHDL Only\)](#)
- [Example 2: Using PSL Assertions in vunits \(VHDL Only\)](#)
- [Example 3: Using Embedded PSL Assertions \(Verilog Only\)](#)
- [Example 4: Using PSL Assertions in vunits \(Verilog Only\)](#)

Example 1: Using Embedded PSL Assertions (VHDL Only)

This example consists of VHDL files, `test.vhd` and `dut.vhd`, and demonstrates the use of PSL assertions embedded with the design. The PSL assertions are embedded in the `dut.vhd` file.

For designs with PSL assertions, perform the following steps:

- [Compiling VHDL Designs with PSL Assertions](#)
- [Debugging Designs with PSL Assertions](#)

The `test.vhd` and `dut.vhd` files used in this example are given below.

test.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;
entity tb is
end;
architecture a of tb is
    signal clk : std_logic := '0';
    signal reset : std_logic := '0';
    signal stim : bit_vector(5 downto 0); -- stimulus
begin
    inst1: entity work.dut(a) port map ( clk => clk,
                                         rst => reset,
                                         a => stim(5),
                                         b => stim(4),
                                         c => stim(3),
                                         d => stim(2),
                                         e => stim(1),
```

UXE User Guide

Compiling and Running Designs with Assertions

```
        f => stim(0),
        out1 => open
    );

clock_generator: process (clk)
begin
    if clk = '0' then
        clk <= '1' after 5 ns, '0' after 10 ns;
    end if;
end process clock_generator;
reset_generator: process
begin
    reset <= '1' after 301 ns, '0' after 321 ns;
    wait; -- forever
end process reset_generator;
stimulus_reader: process
    use STD.TEXTIO.all;
    file DATA : text open read_mode is "data";           -- stimulus data
    variable l : LINE;
    variable l2 : LINE;
    variable ok : boolean := true;
    variable tmp : bit_vector(5 downto 0);
begin
    while (not endfile(DATA)) loop
        readline(DATA, l);
        read(l, tmp, ok);
        if (ok) then
            stim <= tmp;
            write(l2, now, right, 8);
            write(l2, tmp, right, 8);
            writeline(OUTPUT, l2);
        else
            report "Error reading from data file" severity NOTE;
        end if;
        wait until falling_edge(clk);
    end loop;
    wait; -- forever
end process stimulus_reader;
monitor: process
begin
    wait for 500 ns;
    assert false report "Test completed" severity failure;
```

UXE User Guide

Compiling and Running Designs with Assertions

```
end process monitor;
end;
```

dut.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;
use STD.TEXTIO.ALL;

entity dut is
  generic (repcnt: integer := 2 );
  port ( clk: in std_logic := '0';
        rst: in std_logic := '1';
        a: in bit := '0';
        b: in bit := '0';
        c: in bit := '0';
        d: in bit := '0';
        e: in bit := '0';
        f: in bit := '0';
        out1: out bit := '0'
        );
end;

architecture a of dut is
  constant foo : integer := 3;
begin

  -- psl default clock is rising_edge(clk);

  -- Sequences are covered
  -- psl IMPL: assert [*2 to 3] |-> {a;b};
  -- psl gen_cover2_S1: cover {a;b;c};

  process (clk)
  begin
    if rising_edge(clk) then
      out1 <= a or b or c or d or e or f;
    end if;
  end process;
```


UXE User Guide

Compiling and Running Designs with Assertions

```
process (rst)
begin
  if (rst = '1') then
    report "Reset occurred." severity NOTE;
  end if;
end process;
```

end;

Following is the data file used for this example:

data

```
001001
110110
000010
100000
000000
100010
011000
001011
011100
```

Compiling VHDL Designs with PSL Assertions

To compile the VHDL design and testbench with PSL assertions, perform the following steps:

1. Run the `vhan` command with the `-psl` option to analyze the VHDL design and testbench.

```
vhan -psl dut.vhd test.vhd
```

2. Run the `ixcom` command to elaborate and compile the VHDL design.

```
ixcom -ua -coverage functional +assertStatus +assertCover \  
+assertCoverCnt+32 -top tb
```

The above command compiles the design with the assertions and creates the IES snapshot. The PSL assertions in the design are elaborated by the `ixcom` command.

3. Run the `xeDebug` command to view the Assertion Coverage Summary and record the functional coverage data.

```
xeDebug -gui
```

Log Files Generated at Compilation Time

The following log files are generated at the compilation time:

- `vhan.log`:

This file contains the output of the `vhan` command.

- `ixcom.log`:

This file contains the output of the `ixcom` command.

- `xeDebug.log`:

This file contains the output of the `xeDebug` command.

At run time, the error messages pertaining to assertion failure are displayed in the following format:

```
ncsim: *E,ASRTST (dut.vhd,27): (time 25 NS) Assertion :inst1:IMPL has failed
```

In the above format, `:inst1:IMPL` represents the path of the PSL assertion. `IMPL` is the label of the assertion directive and `inst1` is the path of the module from which the PSL assertion is fired. The filename in which the assertion directive is stored along with the line number is `dut.vhd` and 27, respectively.

Note: The waveforms for checked and finished counts of the PSL assertions are dumped only if you have specified the `+assertCover` option of the `ixcom` command at the time of design elaboration. To dump and view the assertion status, use the `+assertStatus` option of the `ixcom` command.

Example 2: Using PSL Assertions in vunits (VHDL Only)

This example consists of VHDL files, `test.vhd` and `dut.vhd`, and demonstrates the use of PSL assertions in vunit. The PSL assertions are specified in vunit, which is the `prop.psl` file.

For designs with PSL assertions included as vunit, compile the design in the same manner as given in [Example 1: Using Embedded PSL Assertions \(VHDL Only\)](#) on page 538. The steps to compile the design with vunit are given in [Compiling VHDL Designs with vunit](#) on page 546. In contrast to compilation of designs with embedded assertions, when compiling designs with vunits, you must include the additional `-propfile` option with the `vhan` command. Rest of the compilation and debugging processes remain the same for both flows.

The `test.vhd`, `dut.vhd`, and `prop.psl` files used in this example are given below.

test.vhd

UXE User Guide

Compiling and Running Designs with Assertions

```
library IEEE;
use IEEE.std_logic_1164.all;

entity tb is
end;

architecture a of tb is
    signal clk : std_logic := '0';
    signal reset : std_logic := '0';
    signal stim : bit_vector(5 downto 0); -- stimulus
begin

    inst1: entity work.dut(a) port map ( clk => clk,
                                         rst => reset,
                                         a => stim(5),
                                         b => stim(4),
                                         c => stim(3),
                                         d => stim(2),
                                         e => stim(1),
                                         f => stim(0),
                                         out1 => open
                                         );

    clock_generator: process (clk)
    begin
        if clk = '0' then
            clk <= '1' after 5 ns, '0' after 10 ns;
        end if;
    end process clock_generator;

    reset_generator: process
    begin
        reset <= '1' after 301 ns, '0' after 321 ns;
        wait; -- forever
    end process reset_generator;

    stimulus_reader: process
    use STD.TEXTIO.all;
    file DATA : text open read_mode is "data"; -- stimulus data
    variable l : LINE;
    variable l2 : LINE;
```

UXE User Guide

Compiling and Running Designs with Assertions

```
variable ok : boolean := true;
variable tmp : bit_vector(5 downto 0);
begin
  while (not endfile(DATA)) loop
    readline(DATA, l);
    read(l, tmp, ok);
    if (ok) then
      stim <= tmp;
      write(l2, now, right, 8);
      write(l2, tmp, right, 8);
      writeline(OUTPUT, l2);
    else
      report "Error reading from data file" severity NOTE;
    end if;
    wait until falling_edge(clk);
  end loop;
  wait;                                -- forever
end process stimulus_reader;
```

```
monitor: process
begin
  wait for 2000 ns;
  assert false report "Test completed" severity failure;
end process monitor;
end;
```

dut.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;
use STD.TEXTIO.ALL;

entity dut is
  port ( clk: in std_logic := '0';
        rst: in std_logic := '1';
        a: in bit := '0';
        b: in bit := '0';
        c: in bit := '0';
        d: in bit := '0';
        e: in bit := '0';
        f: in bit := '0';
        out1: out bit := '0'
```

UXE User Guide

Compiling and Running Designs with Assertions

```
        );  
end;  
  
architecture a of dut is  
begin  
  
    process (clk)  
    begin  
        if rising_edge(clk) then  
            out1 <= a or b or c or d or e or f;  
        end if;  
    end process;  
  
    process (rst)  
    begin  
        if (rst = '1') then  
            report "Reset occurred." severity NOTE;  
        end if;  
    end process;  
  
end;
```

prop.psl

```
vunit A1 (dut(a)) {  
-- Sequential repetition:  
-- psl default clock is rising_edge(clk);  
property P1 is never {a[*2]; b;          c[*3]};  
property P2 is always {a;a;b;a};  
assert P1;  
A2: assert P2;  
}
```

Following is the data file used for this example:

data

```
001001  
110110  
000010  
100000  
000000  
100010
```

```
011000
001011
011100
```

Compiling VHDL Designs with vunit

To compile the VHDL design and testbench with assertions in vunit, perform the following steps:

1. Run the `vhan` command with the `-propfile` option to analyze the VHDL design and testbench, and the file containing the vunit.

```
vhan -propfile prop.psl -psl dut.vhd test.vhd
```

The `-profile prop.psl` option enables the reading of the assertions within the verification unit called `A1`, linked to design unit with the entity named `dut` and architecture named `a`. The `-psl` option is required if the DUT contains PSL assertion constructs.

2. Run the `ixcom` command to elaborate and compile the design.

```
ixcom -ua -coverage functional +assertStatus +assertCover +assertCoverCnt+32 -  
top tb
```

The above command compiles the design with the assertions.

3. Run the design in simulation acceleration mode using the `xeDebug` command:

```
xeDebug -gui
```

Example 3: Using Embedded PSL Assertions (Verilog Only)

This example consists of Verilog files, `test.v` and `dut.v`, and demonstrates the use of PSL assertions embedded with the design. The PSL assertions are embedded in the `dut.v` file.

For designs with PSL assertions, compile the Verilog design with PSL Assertions.

The `dut.v` and `test.v` files used in this example are given below.

dut.v

```
module dut (clk,rst,in1,in2,out1);  
  
    input [3:0] in1;  
    input [3:0] in2;  
    input rst;  
    input clk;  
    output [4:0] out1;
```

UXE User Guide

Compiling and Running Designs with Assertions

```
reg [2:0] state;
reg [2:0] next_state;
reg [4:0] result[7:0];
reg [4:0] out1;
integer idx;

always @(negedge clk) begin
  if (rst) begin // synch reset
    next_state = 3'b000;
    for (idx = 0; idx < 8; idx = idx + 1)
      result[idx] = 5'b000000;
    end
  else
    case (state)
      3'b000: begin
        next_state = 3'b001;
        result[next_state] = in1 + in2;
        end
      3'b001: begin
        next_state = 3'b010;
        result[next_state] = in1 - in2;
        end
      3'b010: begin
        next_state = 3'b100;
        result[next_state] = in1 - in2;
        end
      3'b011: begin
        next_state = 3'b101;
        result[next_state] = in1 + in2;
        end
      3'b100: begin
        next_state = 3'b011;
        result[next_state] = in1 + in2;
        end
    end
    3'b101: begin
      next_state = 3'b110;
      result[next_state] = in1 - in2;
      end
    3'b110: begin
      next_state = 3'b111;
      end
  end
```

UXE User Guide

Compiling and Running Designs with Assertions

```
        result[next_state] = in1 + in2;
    end
    3'b111: begin
        next_state = 3'b000;
        result[next_state] = in1 + in2;
    end
    endcase // case(state)
    out1 = result[state];
end // always @ (negedge clk)

always @(posedge clk) begin
$display("State %b, next_state %b", state, next_state);
    state = next_state;
end

// psl property A1 = always {next_state == state + 1 ||
//                               next_state == 3'b000} @(posedge clk);
// psl P1: assert (A1);
endmodule
```

test.v

```
module test ();

    reg [3:0] in1;
    reg [3:0] in2;
    reg clk;
    reg reset;
    wire [4:0] out1;

    initial begin
        clk = 0;
        #20;
        reset = 1;
        #30;
        reset = 0;
        #500 $finish;
    end

    always begin
        #10 clk = ~clk;
    end
```


UXE User Guide

Compiling and Running Designs with Assertions

```
end

always begin
    #20;
    in1 = $random;
    in2 = $random;
end

dut inst1(clk, reset, in1, in2, out1);

always @(posedge clk) begin
    $strobe("strobe: Reset = %b, In1 = %b, In2 = %b, Output = %b",
            reset, in1, in2, out1);
end

endmodule
```

Compiling Verilog Designs with PSL Assertions

To compile the Verilog design and testbench with PSL assertions, perform the following steps:

1. Run the `vlan` command with the `-psl` option to analyze the Verilog design and testbench with PSL assertions.
2. Run the `ixcom` command with the required options to elaborate and compile the Verilog design and the testbench.
3. Run the `xeDebug` command to view the Assertion Coverage Summary and record the functional coverage data.

```
vlan -psl dut.v test.v
```

```
ixcom -ua -top test -coverage functional +assertCover +assertCoverCnt+32
```

```
xeDebug -gui
```

Log Files Generated at Compilation Time

The following log files are generated at the compilation time:

- `vlan.log`

This file contains the output of the `vlan` command.

- `ixcom.log`:

This file contains the output of the `ixcom` command.

■ `xeDebug.log`:

This file contains the output of the `xeDebug` command.

At run time, the error message pertaining to assertion failure are displayed in the following format:

```
ncsim: *E,ASRTST (dut.v,70): (time 10 NS) Assertion test.inst1.P1 has failed
```

In the above format, `test.inst1.P1` represents the path of the PSL assertion. `P1` is the label of the assertion directive and `test.inst1` is the path of the module from which the PSL assertion is fired. The filename in which the assertion directive is stored along with the line number is `dut.v` and `70`, respectively.

Note: The waveforms for checked and finished counts of the PSL assertion are dumped only if the `+assertCover` option of the `ixcom` command at the time of design elaboration. To dump and view the assertion status, use the `+assertStatus` option of the `ixcom` command.

Example 4: Using PSL Assertions in vunits (Verilog Only)

This example consists of Verilog files, `test.v` and `dut.v`, and demonstrates the use of PSL assertions in vunit. The PSL assertions are specified in vunit, which is the `prop.psl` file.

For designs with PSL assertions included as vunit, compile the design in the same manner as given in [Example 3: Using Embedded PSL Assertions \(Verilog Only\)](#) on page 546. The steps to compile the design with vunit are given in [Compiling VHDL Designs with vunit](#) on page 546. The only difference between compilation of designs with embedded assertions and compilation of designs with vunits is that in the latter flow, the `vlan` command includes the additional `-propfile` option for compiling designs with vunits. Rest of the compilation and debugging processes remain the same for both flows.

The `test.v`, `dut.v`, and `prop.psl` files used in this example are given below.

dut.v

```
module dut (clk,rst,in1,in2,out1);

    input [3:0] in1;
    input [3:0] in2;
    input rst;
    input clk;
    output [4:0] out1;

    reg [2:0] state;
```

UXE User Guide

Compiling and Running Designs with Assertions

```
reg [2:0] next_state;
reg [4:0] result[7:0];
reg [4:0] out1;
integer idx;

always @(negedge clk) begin
  if (rst) begin                                // synch reset
    next_state = 3'b000;
    for (idx = 0; idx < 8; idx = idx + 1)
      result[idx] = 5'b000000;
    end
  else
    case (state)
      3'b000: begin
        next_state = 3'b001;
        result[next_state] = in1 + in2;
        end
      3'b001: begin
        next_state = 3'b010;
        result[next_state] = in1 - in2;
        end
      3'b010: begin
        next_state = 3'b100;
        result[next_state] = in1 - in2;
        end
      3'b011: begin
        next_state = 3'b101;
        result[next_state] = in1 + in2;
        end
      3'b100: begin
        next_state = 3'b011;
        result[next_state] = in1 + in2;
        end
      3'b101: begin
        next_state = 3'b110;
        result[next_state] = in1 - in2;
        end
      3'b110: begin
        next_state = 3'b111;
        result[next_state] = in1 + in2;
        end
    end
  end
```

UXE User Guide

Compiling and Running Designs with Assertions

```
    3'b111: begin
        next_state = 3'b000;
        result[next_state] = in1 + in2;
    end
    endcase // case(state)
    out1 = result[state];
end // always @ (negedge clk)

always @(posedge clk) begin
$display("State %b, next_state %b", state, next_state);
    state = next_state;
end

// psl property A1 = always {next_state == state + 1 ||
//                               next_state == 3'b000} @(posedge clk);
// psl P1: assert (A1);
endmodule
```

test.v

```
module test ();

    reg [3:0] in1;
    reg [3:0] in2;
    reg clk;
    reg reset;
    wire [4:0] out1;

    initial begin
        clk = 0;
        #20;
        reset = 1;
        #30;
        reset = 0;
        #500 $finish;
    end

    always begin
        #10 clk = ~clk;
    end

    always begin
```

UXE User Guide

Compiling and Running Designs with Assertions

```
#20;
in1 = $random;
in2 = $random;
end

dut inst1(clk, reset, in1, in2, out1);

always @(posedge clk) begin
    $strobe("strobe: Reset = %b, In1 = %b, In2 = %b, Output = %b",
           reset, in1, in2, out1);
end

endmodule
```

prop.psl

```
vunit demo (dut) {

// psl default clock = negedge clk;
// psl property A1 = always {next_state == state + 1 || next_state == 3'b000};
//
// psl L1 : assert A1;
}
```

Compiling Verilog Designs with vunit

To compile the Verilog design and testbench with PSL assertions in vunit, perform the following steps:

1. Run the `vlan` command with the `-propfile` option to analyze the Verilog design file and testbench with PSL assertions in vunit:

```
vlan dut.v test.v -propfile prop.psl -psl
```

2. Run the `ixcom` command with the required options to elaborate and compile the Verilog design and the testbench.

```
ixcom -ua -top test -coverage functional +assertStatus +assertCover
+assertCoverCnt+32
```

3. Run the `xeDebug` command to view the Assertion Coverage Summary and record the functional coverage data.

```
xeDebug -gui
```

Debugging Designs with PSL Assertions

To debug designs with PSL assertions, use SimVision. In this section, the design and testbench of [Example 1: Using Embedded PSL Assertions \(VHDL Only\)](#) on page 538 and [Example 3: Using Embedded PSL Assertions \(Verilog Only\)](#) on page 546 have been used.

Perform the following steps to use xeDebug for viewing PSL assertion information:

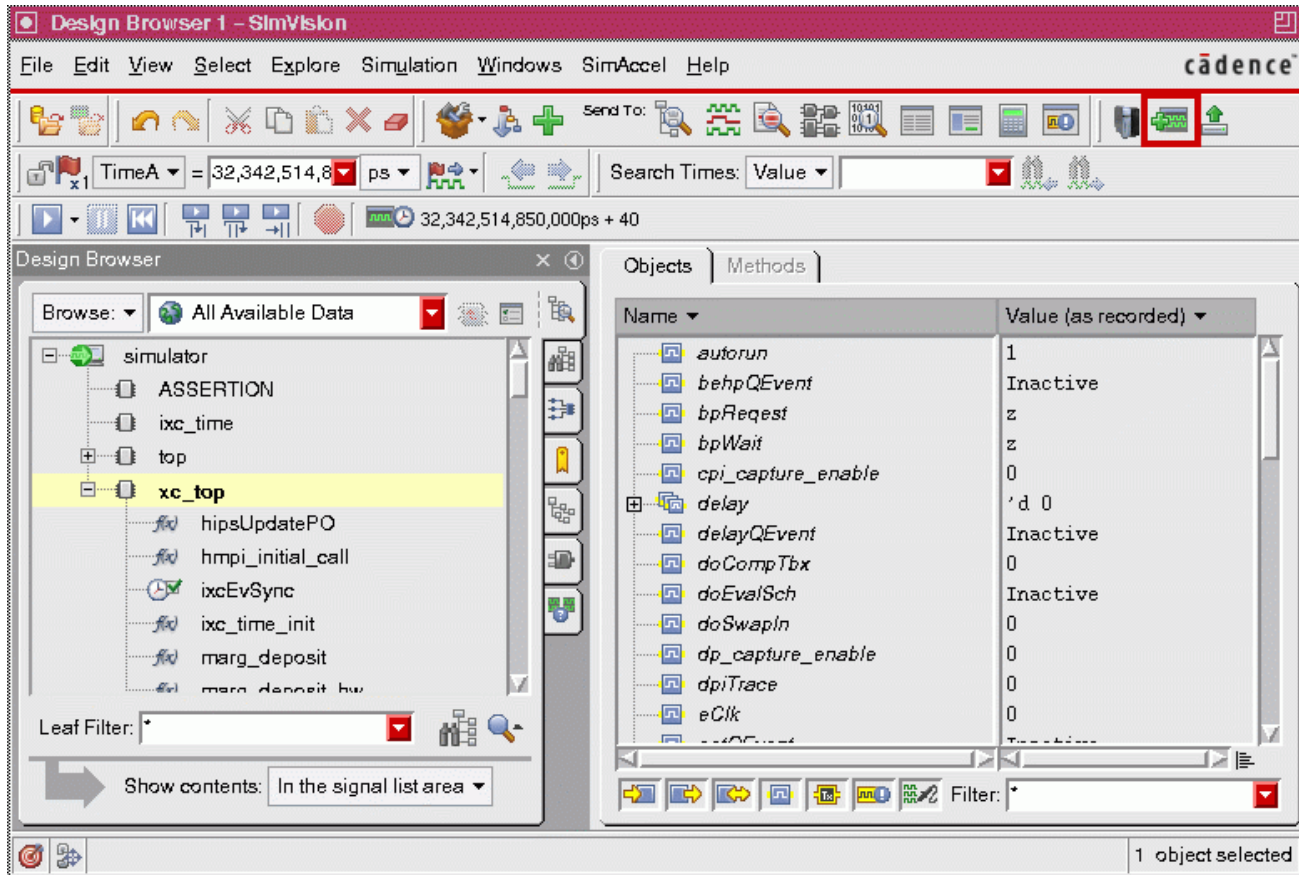
1. Specify the following command to invoke xeDebug:

```
xeDebug -gui
```

2. Open the design, and start the session after specifying the required configuration settings. After you click the Start Session button, the icons in the toolbar at the top of the xeDebug GUI window get activated.
3. Click the  icon on the toolbar to display the SimVision *Design Browser* window.

The *Design Browser* window displays the hierarchy view of the testbench and the DUT. The left pane displays the design browser and the right pane displays various signals. Select the required signals including the assertion signals from the right pane of the *Design Browser* window, as shown in the following figure.

Figure 10-5 SimVision Design Browser



4. Select the required signals from the right pane of the *Design Browser* window.
5. Click the *Probe and Display* icon highlighted in [Figure 10-5](#) from the *Design Browser* window to display the *Waveform Viewer* window illustrated in [Figure 10-3](#).
6. Specify the following command to invoke ICC to view the coverage information in the ICC database generated for this design.

```
iccr -g
```

The *ICC - Coverage Totals* window is displayed, as shown in the following figure.

UXE User Guide

Compiling and Running Designs with Assertions

Figure 10-6 ICC - Coverage Totals

ICC - Coverage Totals

File View Window Help

cadence

Threshold 100 %

Test: Include: b e t

Type	Coverage	Passing Ratio
Module/Unit	--%	0 / 0
Instance	--%	0 / 0

FSM Coverage :

Type	Coverage	Passing Ratio
State	--%	0 / 0
Arc	--%	0 / 0

Functional Coverage :

Type	Coverage	Passing Ratio
Control-oriented	--%	0 / 0
Data-oriented	--%	0 / 0

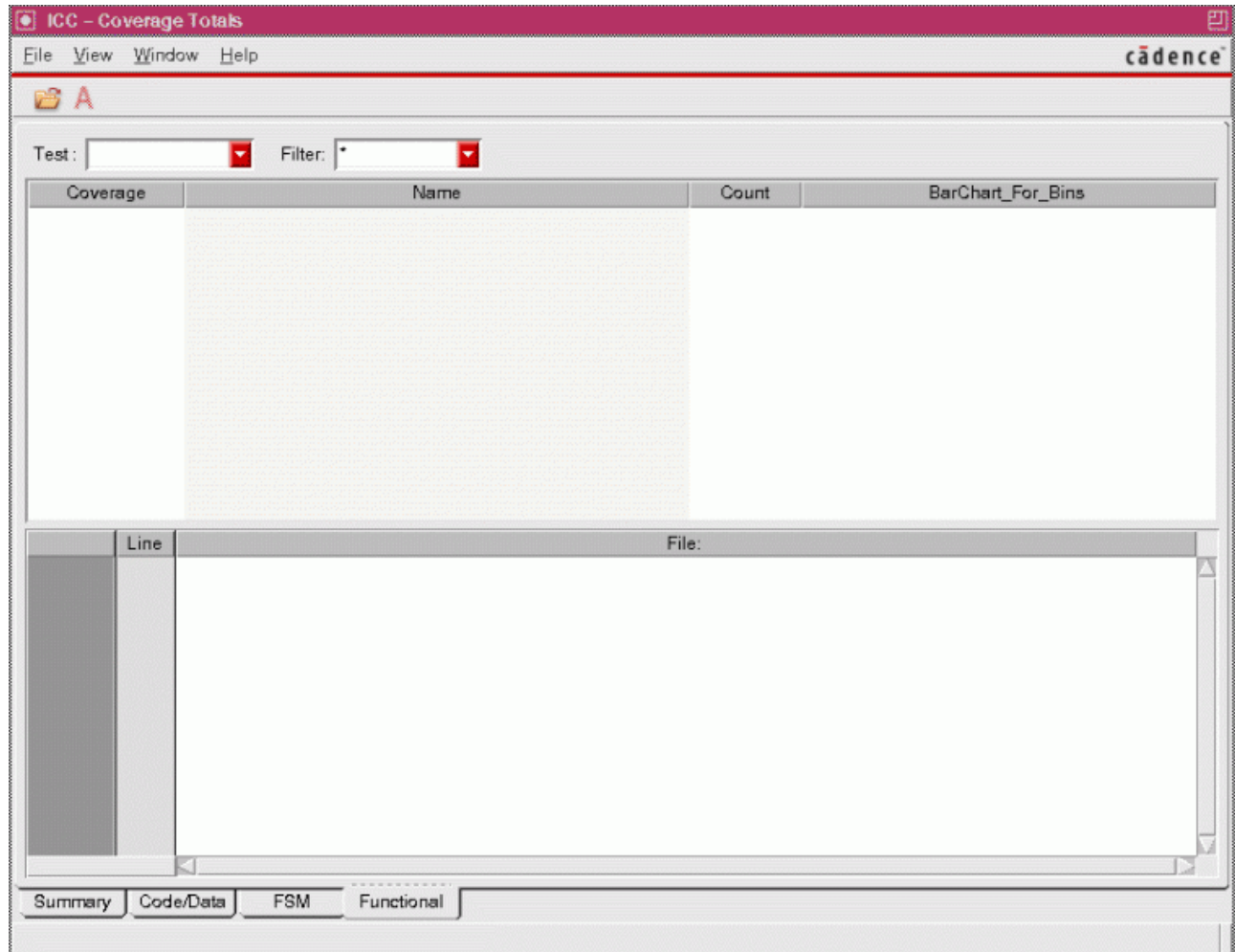
Summary Code/Data FSM Functional

7. Select the *Functional* tab, as shown in the following figure, to load the functional coverage database.

UXE User Guide

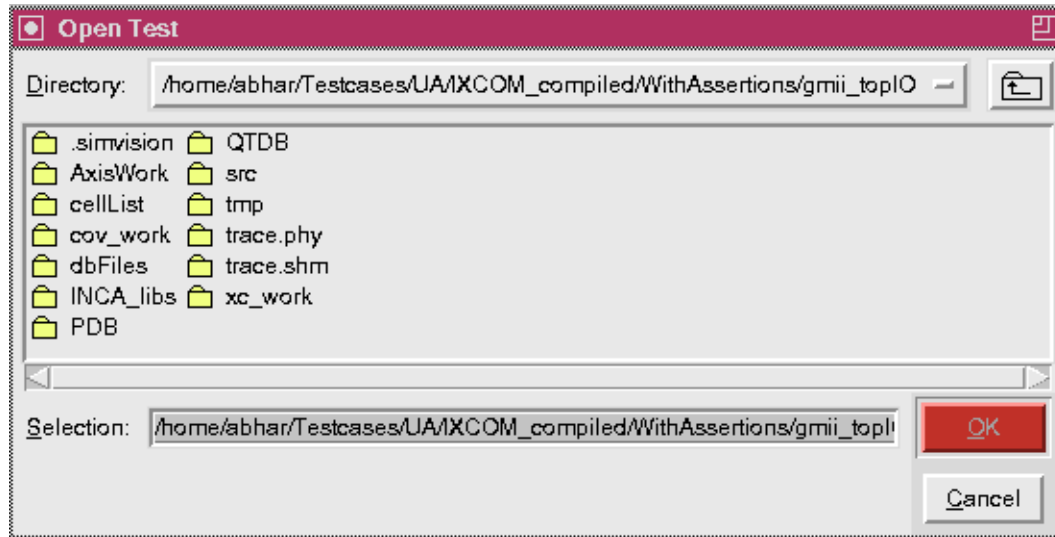
Compiling and Running Designs with Assertions

Figure 10-7 ICC - Functional Tab



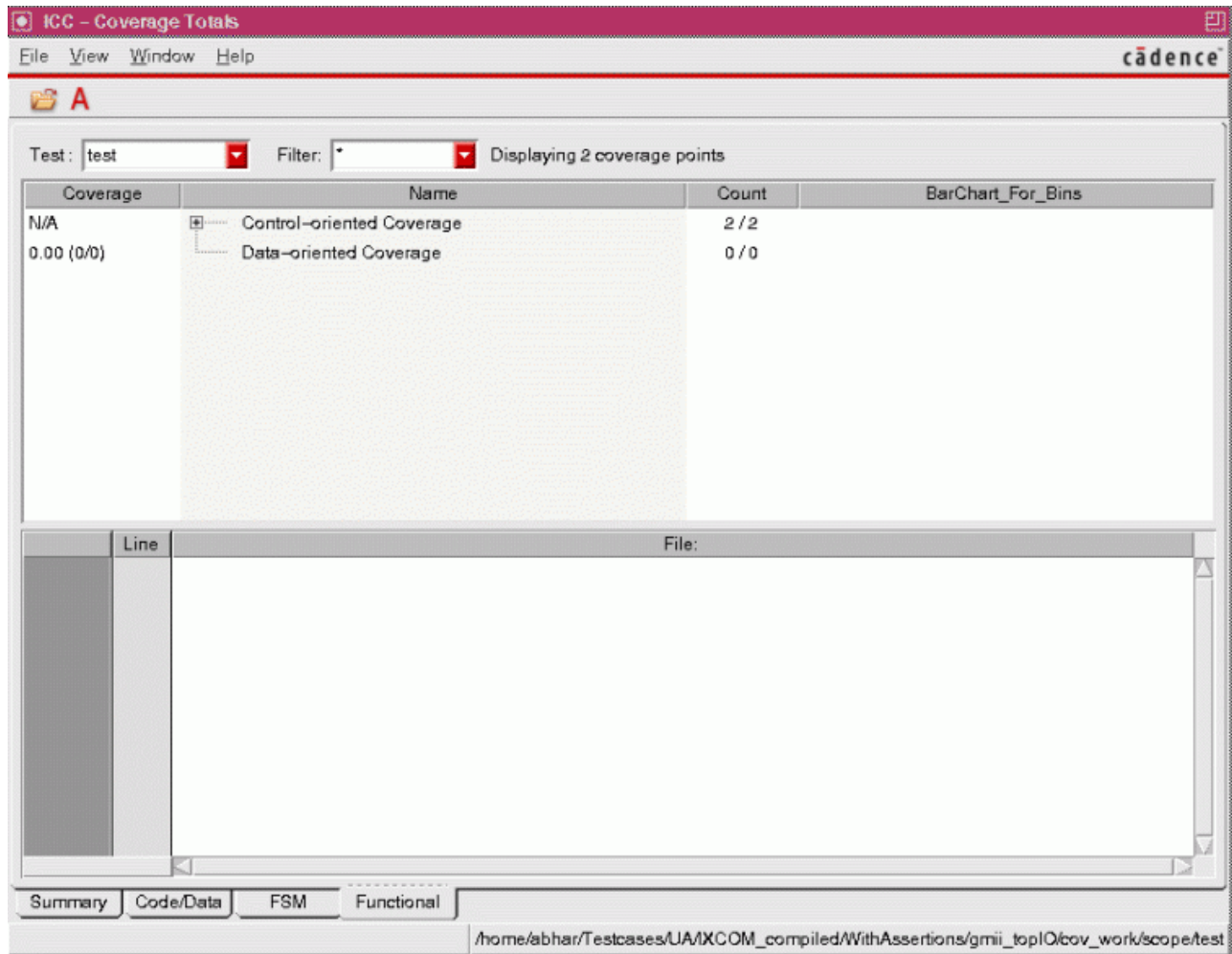
8. Select the *File – Open Test* menu to display the *Open Test* dialog, as shown in the following figure.

Figure 10-8 Open cov_work Directory



9. Select the *cov_work* directory and click the *OK* button.
10. Select the subdirectory of the *cov_work* directory that has the coverage data. The contents of the select coverage data directory will be displayed within the *ICC - Coverage Totals*, as shown in the following figure.

Figure 10-9 ICC - Coverage Totals with Data

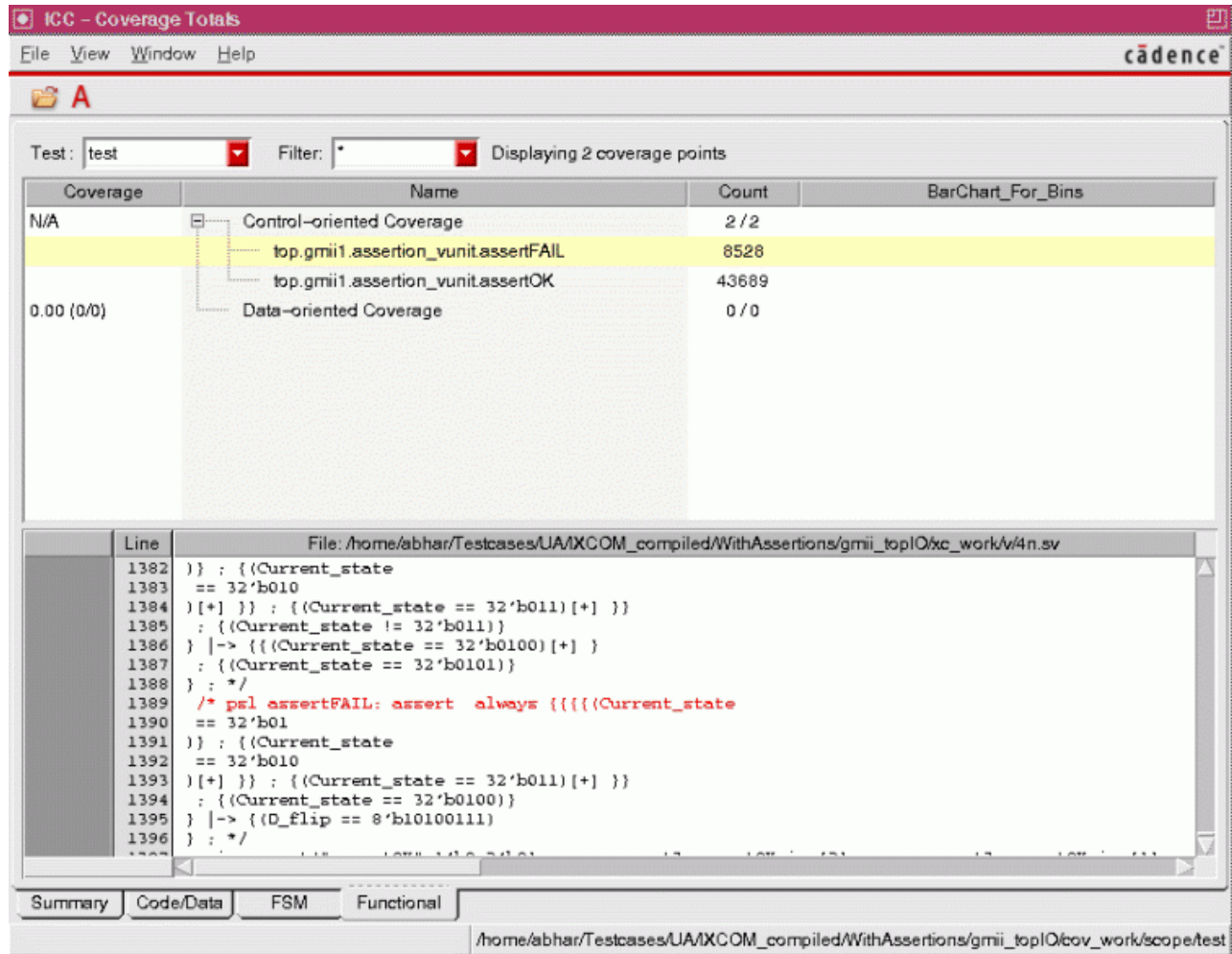


11. Click the plus sign beside the *Control-oriented Coverage* option to display the functional coverage data, as shown in the following figure.

UXE User Guide

Compiling and Running Designs with Assertions

Figure 10-10 ICC - Coverage Totals Window with Displayed Data



Using Assertions in Dynamic Target Mode

In the SA mode, the assertion failure counters are implemented on the software simulator to reduce hardware resource utilization. The assertion failure action blocks are also executed by the simulator. But, in the Dynamic Target mode, because design clocks never stop, the emulator cannot stop, and hence simulator is not available. So, assertion failure counters need to be implemented on the emulator side in the Dynamic Target mode, which is achieved by using `+assertFailCounter+hw` option of the `ixcom` command. This option enables the failure counters to be tracked on the emulator instead of the software simulator.

The failure action blocks, however, reside on the software simulator side only. The failure action blocks are not executed when the design is run in the Dynamic Target mode. Switch to the static target mode to run the failure action blocks. Then, appropriate messages are displayed for the failing assertions.

Important

If the `+assertCover` option is not specified with the `ixcom` command, the fail counters are not generated. Thus, if you specify the `+assertFailCounter+hw` option with the `ixcom` command, but not the `+assertCover` option, an error is generated and compilation is aborted.

Running Assertions in Dynamic Target Mode

1. Compile the design using the `+assertFailCounter+hw` and `+assertCover` options with the `ixcom` command.

```
ixcom <ixcom_options> +assertFailCounter+hw +assertCover
```

2. Use the simulator to initialize the design.

Note: Assertions can also be disabled at this stage, if required.

3. Swap to emulator and start the Dynamic Target mode.

For example, for steps 2 and 3 described above, you can execute the following set of commands:

```
debug .
host .
xc piok
xc on -autorun
configPM -dynamicTarget
```

In the Dynamic Target mode, the pre-defined SDL signal `ASSERTION.FAILURE` enables triggering an action on any assertion failure. The trace clocks can be freezed to provide waveforms of assertion failure intervals.

For example:

```
if (ASSERTION.FAILURE)
{
    Some code;
    trigger;
}
```

- ❑ The SDL can be written to trigger either on specific assertions or any assertion failure.
- ❑ At trigger time, you can list assertions that either failed or did not get covered.

To disable triggering of SDL at assertion failures, load a new SDL command file which does not contain the assertions-sensitive text.

4. Use the `assertion -summary -final` command to request the assertions summary report generation at any time when running in the Dynamic Target mode or at the time of session termination.

Note: To generate an assertion summary report at session termination, you must first switch back to the static target mode.

```
configPM -staticTarget
```

Support for Assertions Commands when Using Dynamic Targets

In the Dynamic Target mode, the simulator is available before the design is swapped to the emulator and after switching to the static target mode. Support for assertions in the Dynamic Target mode is limited to the features that do not require simulator processing.

For more information, see [Assertions Commands Supported with Dynamic Targets](#) and [Assertions Commands not Supported with Dynamic Targets](#).

Assertions Commands Supported with Dynamic Targets

The following assertions commands are supported in the Dynamic Target mode although the output from the `assertion -counter` command is from the time that the counter value on the emulator was last accessed.

- `assertion -check`

UXE User Guide

Compiling and Running Designs with Assertions

- `assertion -counter`
- `assertion -list`
- `assertion -summary -final`

The use of `assertion -summary -final` causes generation of a summary report at session termination.

The `assertion -list` options are supported to enable examining a subset of design assertions. Omitting an assertion locator defaults to selecting all the assertions.

The `assertion -list` command provides the assertion information while the design clocks are running. This information is gathered by examining the assertion counter signals on the emulator. Following are the relevant command options:

- `assertion -list -failed`
Returns a cumulative list of all the assertions that have failed since time 0.
- `assertion -list -uncovered`
Returns a list of assertions that are still uncovered at the time of the emulator state access.
- `assert -list -failed -uncovered`
Returns a set of assertions that have either failed since time 0 or are still uncovered at the time of the emulator state access.

The output of the above commands is returned as a Tcl result list. These command options are supported in all run modes.

Important

The design clocks continue to run while processing the `assertion -list` command; therefore, the assertion counters can change before the results are reported. The output always presents the counter values at the time they were examined and not any particular simulation time.

Assertions Commands not Supported with Dynamic Targets

The following assertions commands are not supported in the Dynamic Target mode, and are ignored with a warning. These can be used before the design is swapped to the emulator and at session termination, although some of these commands might not be relevant. For example, the `-off` and `-on` options are not relevant at session termination, when no

UXE User Guide

Compiling and Running Designs with Assertions

simulation time is available; the disable or enable actions for assertions, therefore, have no effect.

- `assertion -get`
- `assertion -off`
- `assertion -on`
- `assertion -summary` without the `-final` switch

Support for Specific SVA Constructs

The following tables show the current implementation status of specific SVA Constructs.

Table 10-3 Support for SVA Constructs - LRM 1800 - 2005

SVA Construct	LRM 1800-2005 Reference	Supported	Notes
Immediate Assertion	17.2	yes	Not supported in <code>always_comb</code> , <code>always_latch</code> , <code>always@(*)</code> processes.
<code>cycle_delay_range</code> <code>sequence_expr</code>	17.5	yes	Only elaboration time constants are allowed in the cycle delay range expression. The cycle delay range that exceeds 32 and appears in one of the following expressions is not supported: ■ Assertion instantiation ■ Sequences on which methods or <code>first_match</code> is called ■ Multiclocked sequence and properties
<code>\$</code> (end of simulation expression)	17.5	yes	
sequence declaration	17.6	yes	
Typed formal argument in sequences	17.6.1	yes	

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-3 Support for SVA Constructs - LRM 1800 - 2005

SVA Construct	LRM 1800-2005 Reference	Supported	Notes
Sequence repetition: Consecutive	17.7.2	yes	
Sequence repetition: Goto	17.7.2	yes	
Sequence repetition: non-consecutive	17.7.2	yes	
Sampled value functions	17.7.3	yes	Supported in the context of assertions as well as procedural code. Additionally, these functions can be used as part of the actual argument expressions in a module instantiation statement.
<code>\$sampled</code>	17.7.3	yes	Optional clock must be the same as the assertion clock.
<code>\$sampled in action block</code>	17.7.3	yes	Optional clock must be the same as the assertion clock.
<code>\$rose</code>	17.7.3	yes	Optional clock can differ from the clock of the context as per the 1800 - 2009 LRM.
<code>\$past</code>	17.7.3	yes	Optional gating expression is supported. Optional clock can differ from the clock of the context as per the 1800 - 2009 LRM.

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-3 Support for SVA Constructs - LRM 1800 - 2005

SVA Construct	LRM 1800-2005 Reference	Supported	Notes
<code>\$fell</code>	17.7.3	yes	Optional clock can differ from the clock of the context as per the 1800 - 2009 LRM.
<code>\$stable</code>	17.7.3	yes	Optional clock can differ from the clock of the context as per the 1800 - 2009 LRM.
Sequence operator: <code>AND</code>	17.7.4	yes	
Sequence operator: <code>INTERSECT</code>	17.7.5	yes	
Sequence operator: <code>OR</code>	17.7.6	yes	
Sequence operator: <code>first_match</code>	17.7.7	yes	
Sequence operator: <code>throughout</code>	17.7.8	yes	
Sequence operator: <code>within</code>	17.7.9	yes	
Clocking event in sequence expression	17.5	yes	
Detecting sequence endpoint (<code>.ended</code>)	17.7.10	yes	
Detecting sequence endpoint in multiclock context (<code>.triggered</code> , <code>.matched</code>)	17.12.5	yes	<code>.triggered</code> must have the same clock as the assertion clock. The <code>triggered</code> method is supported in sequence expressions as per the 1800-2009 LRM.

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-3 Support for SVA Constructs - LRM 1800 - 2005

SVA Construct	LRM 1800-2005 Reference	Supported	Notes
Sequence match item	17.8	yes	
Sequence: local variables	17.8	yes	<code>first_match</code> is not supported with local variables. Sequence methods cannot modify local variables.
Local variables of complex data types	17.8	yes	
Calling subroutine on match of a sequence	17.9	yes	
System function: <code>\$onehot</code>	17.10	yes	
System function: <code>\$onehot0</code>	17.10	yes	
System function: <code>\$isunknown</code>	17.10	yes	In the hardware mode, it evaluates to 0. In the software mode, it follows the LRM.
System function: <code>\$countones</code>	17.10	yes	
Property declaration	17.11	yes	
Property operator: <code>not</code>	17.11	yes	
Property operator: <code>and</code>	17.11	no	

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-3 Support for SVA Constructs - LRM 1800 - 2005

SVA Construct	LRM 1800-2005 Reference	Supported	Notes
Property operator: <code>or</code>	17.11	no	The Property <code>OR</code> operator is replaced with the Sequence <code>OR</code> operator if the following conditions are met: ■ The Property <code>OR</code> is used with cover directives ■ The Property operands of the <code>OR</code> operator are actually sequence expressions.
Property operator: <code>if-else</code>	17.11	yes	
Property operator: <code> -></code>	17.11	yes	
Property operator: <code> =></code>	17.11	yes	
Property: <code>disable iff</code>	17.11	yes	
Typed formal argument in property	17.11.1	yes	
Recursive properties	17.11.4	no	

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-3 Support for SVA Constructs - LRM 1800 - 2005

SVA Construct	LRM 1800-2005 Reference	Supported	Notes
Multiclock support	17.12	yes	■ Cannot change clock within <code>first_match</code> ■ No support for local variables ■ multiclock sequence methods are supported
<code>assert</code> statement (concurrent assertion)	17.13.1	yes	
<code>assume</code> statement	17.13.2	yes	
<code>cover</code> statement	17.13.3	yes	
Concurrent assertion in procedural code	17.13.5	yes	Assertion controls and counters are supported for procedural assertions. Concurrent assertions are not supported in <code>always_comb</code> , <code>always_latch</code> , <code>always@(*)</code> processes.
Default clocking	17.14	yes	

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-3 Support for SVA Constructs - LRM 1800 - 2005

SVA Construct	LRM 1800-2005 Reference	Supported	Notes
Multiclock properties	17.14.1	yes	■ Cannot change clock within <code>first_match</code> ■ No support for local variables ■ Multi-clocked sequence methods are supported
Clock resolution	17.14.1	yes	
Binding properties to scope or instances	17.15	yes	
<code>expect</code> statement	17.16	yes	

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-3 Support for SVA Constructs - LRM 1800 - 2005

SVA Construct	LRM 1800-2005 Reference	Supported	Notes
<code>\$asserton</code>	22.8	yes	Supported if used in one of the following manners: ■ These can exist in any part of a testbench module. ■ These can exist within the <code>initial</code> block of a DUT module. ■ These can exist within the tasks called using <code>axis</code> <code>tb_call</code> region in a DUT module. ■ These can exist within the <code>fail</code> <code>action</code> block of the concurrent assertions.

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-3 Support for SVA Constructs - LRM 1800 - 2005

SVA Construct	LRM 1800-2005 Reference	Supported	Notes
<code>\$assertkill</code>	22.8	yes	Supported if used in one of the following manners: ■ These can exist in any part of a testbench module. ■ These can exist within the <code>initial</code> block of a DUT module. ■ These can exist within the tasks called using <code>axis</code> <code>tb_call</code> region in a DUT module. ■ These can exist within the <code>fail</code> <code>action</code> block of the concurrent assertions.

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-3 Support for SVA Constructs - LRM 1800 - 2005

SVA Construct	LRM 1800-2005 Reference	Supported	Notes
<code>\$assertoff</code>	22.8	yes	Supported if used in one of the following manners: ■ These can exist in any part of a testbench module. ■ These can exist within the <code>initial</code> block of a DUT module. ■ These can exist within the tasks called using <code>axis</code> <code>tb_call</code> region in a DUT module. ■ These can exist within the <code>fail</code> action block of the concurrent assertions.

The following table lists the support for some of the SVA constructs that are listed in the LRM 1800-2009.

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-4 Support for SVA Constructs - LRM 1800 - 2009

SVA Construct	LRM 1800-2009 Reference	Supported	Notes
Repetition operators <code>[*]</code> , <code>[+]</code>	16.7	yes	Consecutive repetition operators: ■ <code>[*]</code> is an equivalent representation of <code>[*0:\$]</code> ■ <code>[+]</code> is an equivalent representation of <code>[*1:\$]</code>
delay operators <code>##[+]</code> , <code>##[*]</code>	16.9.2	yes	Cycle delay ranges: ■ <code>##[*]</code> is used as an equivalent representation of <code>##[0:\$]</code> ■ <code>##[+]</code> is used as an equivalent representation of <code>##[1:\$]</code>
default disable iff	16.13	yes	
<code>\$changed</code>	16.15.8	yes	

Support for PSL Constructs

The following sections outline:

- [Default Clock Implementation](#)
- [Extended Support for Built-in Methods \(VHDL Only\)](#)
- [Support for Specific PSL Constructs in a Mixed-HDL Design](#)
- [Support for Specific PSL Constructs in a Verilog Design](#)

For identifying PSL assertions support in this release, refer to the PSL 1850-2005 LRM.

Default Clock Implementation

Default clock declarations can be done in both design and verification units. However, there are certain aspects that need to be considered while implementing default clock declarations in design and verification units.

- [Default Clock Declaration in a Design Unit](#)
- [Default Clock Declarations in a Verification Unit](#)

Default Clock Declaration in a Design Unit

A design unit can have multiple default clock declarations. However, the clock declaration that is placed nearest to the assertion instance and appears before it within the same scope is considered the default clock declaration for that assertion. Following are the various scenarios for declaring and handling a default clock in a design unit:

- If an assertion instance contains an explicit clock declaration, then the default clock declaration is not applicable to this assertion instance.
- If there are multiple clock declarations before an assertion instance, then the nearest default clock declaration in the design is applied to this assertion instance.
- If the default clock declaration that is applicable to an assertion instance is unsupported, then the assertion instance is ignored and a warning message is displayed.
- If the current scope does not contain any default clock declarations before the assertion is instantiated, then the assertion is considered to be unlocked.
- If an assertion is instantiated in a sub-scope (generates or named block [for Verilog-only]) and does not have a default clock declaration before it in the sub-scope, then the parent scope is used to find the applicable default clock declaration.

- ❑ If the parent scope contains a default clock declaration that is nearest to the sub-scope and appears before it, then this default clock declaration becomes the default clock for the assertion in the sub-scope.
- ❑ If the parent scope does not contain any default clock declaration before the sub-scope, then the default clock declaration is located in the design hierarchy. The default clock declaration in the design hierarchy that is nearest to the sub-scope becomes the default clock for the assertion in that sub-scope.

The search for a default clock declaration is terminated in the following two cases:

- If the default clock declaration is located while moving up in the design hierarchy.
 - The parent architecture (in a mixed-HDL or VHDL design) or module (in a Verilog-only) is hit while moving up in the design hierarchy. In this case, if the parent architecture or module is hit and no default clock declaration is found, then the assertion in the sub-scope is considered to be unclocked.
- For a mixed-HDL or a VHDL Design: If an entity contains a default clock declaration, then the last default clock declaration in an entity becomes the first default clock declaration for the architecture.
 - For a mixed-HDL or a VHDL Design: If an entity contains assertions but no default clock declarations, then the assertions in the entity are considered to be unclocked. Default clock declarations are not derived from any architecture to the entity.

Note: All these above considerations are applicable to assertions in an entity. This behavior of declaring and handling default clocks in IXCOM is consistent with the behavior of IES.

Default Clock Declarations in a Verification Unit

In IXCOM, the behavior of the default clock in a verification unit is compliant with PSL LRM 1850-2005. The following are various scenarios for declaring and handling a default clock in a verification unit:

- If there are multiple default clock declarations in the verification unit, it is considered as an error.
- If there is only one default clock declaration in the verification unit, it is applicable to all assertion instances within that verification unit.
- If there is no default clock declaration in a verification unit, the default clock declaration from the inherited verification unit is applicable to all the assertion instances in the current verification unit. However, if multiple clock declarations are derived from the

inherited verification units, it results in an error and the assertions in the current verification unit are ignored with a warning.

- If a default clock is not declared in a verification unit and is also not derived from inherited verification units, the default clock declaration in the default verification unit (default `vmode`) is applied to the assertions within the current verification unit. However, if there are multiple default clock declarations in a default verification unit, then this results in an error.
- If there is no default clock declaration in the current verification unit and if the default clock has not been derived from any inherited verification unit, the assertions in the verification unit are considered to be unlocked.

Note: IES displays a different behavior for default clock declarations in a verification unit.

Extended Support for Built-in Methods (VHDL Only)

The built-in methods, `prev`, `rose`, `fell`, and `stable` are supported in unlocked expressions if these methods are used as boolean terms and are not used in HDL expressions.

But, if these built-in methods appear in HDL expressions of an unlocked expression, then the unlocked assertion is ignored with a warning message. This is illustrated in the following examples.

```
-- psl property always (a -> b or prev (c))
```

This assertion is ignored because `-prev` is used in the HDL expression.

```
-- psl property always (a -> prev (c))
```

This assertion is supported.

The clock expression argument to built-in methods and endpoint is supported if the clock is same as that of the assertion in context. If the assertion is clocked and the built-in methods and endpoints used in the assertion have their own clock expression argument, then both the clocks need to be same. However, if the clock of built-in methods is different from the clock of the assertion in context, then the assertion is ignored with a warning message. In addition, if the built-in methods have clock expression argument but is used in an unlocked assertion, then the assertion is ignored with a warning message.

Differences between IES and IXCOM for PSL Assertions

Following are the main differences between the handling of PSL constructs in IES and IXCOM:

UXE User Guide

Compiling and Running Designs with Assertions

- PSL within entity and architecture declarative regions is not supported in IXCOM. PSL within a process statement is not allowed in IXCOM. IES supports PSL in declarative regions.
- IES does not increment the failure counts for the PSL failing at end of the simulation, for example, strong properties, while IXCOM does that. Therefore, the failure counts for strong properties are different in IES and IXCOM.
- When an assertion fails and finishes simultaneously, IES only increments the failed count and not the finished count, while IXCOM increments both.
- IES ignores the `assume_guarantee` verification directive, whereas IXCOM treats it same as the `assert` directive.

Refer to the *PSL Quick Reference Guide* in the IUS documentation set for details of the constructs discussed in this section.

Support for Specific PSL Constructs in a Mixed-HDL Design

The following table shows the current status of support for specific PSL Constructs.

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
Boolean Layer			
Bit expressions	5.1.1	yes	
Boolean expressions including <code>true</code>	5.1.2	yes	
BitVector expressions	5.1.3	yes	
Numeric expressions	5.1.4	yes	
String expressions	5.1.5	yes	

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
HDL expressions	5.2.1	yes	<i>Limitations (Verilog-only):</i> ■ HDL signals with names that are PSL keywords, such as Clock, A, or E are not supported within PSL statements. ■ Concatenation expressions with single term is not supported.
PSL expressions	5.2.2	limited	Support as per simple subset. Use of <code>-></code> and <code><-></code> in the context of boolean expression is not supported. For example, <code>always {a->b; c->d};</code>
Built-in Functions			

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
<code>prev(Any_type[, number[, clock_expr]])</code>	5.2.3.1	yes	Limitations (VHDL only): Optional clock expression is supported if it is the same as the clock of the assertion in context. Limitations (Verilog only): ■ Only integral types are supported. ■ Optional clock should be same as the clock of the sequence/property expression in context.
<code>next(Any_type)</code>	5.2.3.2	no	

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
<code>stable(Any_type[,clock_expr])</code>	5.2.3.3	yes	Limitations (VHDL only): ■ Optional clock is supported if it is same as the clock of the assertion in context. ■ Only variable reference, record select, bit select and part-select expressions are supported as first arguments. Limitations (Verilog-only): ■ Only integral types are supported. ■ Optional clock should be same as the clock of the sequence/property expression in context.

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
<code>rose(bit[,clock_expr])</code>	5.2.3.4	yes	Limitations (VHDL only): ■ Optional clock is supported if it is same as the clock of the assertion in context. ■ Only variable reference, record select, bit select and part-select expressions are supported as first arguments. Limitations (Verilog-only): ■ Only integral types are supported. ■ Optional clock should be same as the clock of the sequence/property expression in context.

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
<code>fell(bit[,clock_expr])</code>	5.2.3.5	yes	Limitations (VHDL only): ■ Optional clock is supported if it is same as the clock of the assertion in context. ■ Only variable reference, record select, bit select and part-select expressions are supported as first arguments. Limitations (Verilog-only): ■ Only integral types are supported. ■ Optional clock should be same as the clock of the sequence/property expression in context.

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
<code>ended(sequence[,clock_expr])</code>	5.2.3.6	yes	Limitation (VHDL-only): Optional clock is supported if it is same as the clock of the assertion in context. It is not supported if used in HDL expressions and unclocked properties. Limitation (Verilog-only): Optional clock argument is supported only if it is the same as the clock of enclosing sequence/property.
<code>isunknown(BitVector)</code>	5.2.3.7	yes	Returns <code>false</code> for assertions running on the emulator.
<code>countones(BitVector)</code>	5.2.3.8	yes	Limitation (Verilog-only): Only integral types are supported.
<code>nondet(Value_List)</code>	5.2.3.9	no	
<code>nondet_vector(Number, Value_List)</code>	5.2.3.10	no	
<code>onehot(BitVector), onehot0(BitVector)</code>	5.2.3.11	yes	Limitation (Verilog-only): Only integral types are supported.

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
named endpoint (single cycle)	N/S	yes	Limitation (VHDL-only): Optional clock is supported if it is same as the clock of the assertion in context. It is not supported if used in HDL expressions and unclocked properties. Limitation (Verilog-only): Optional clock is supported only if it is the same as the clock of enclosing sequence/property.
named endpoint (full temporal multi-cycle)		yes	Limitation (VHDL-only): Optional clock is supported if it is same as the clock of the assertion in context. It is not supported if used in HDL expressions and unclocked properties. Limitation (Verilog-only): Optional clock is supported only if it is the same as the clock of enclosing sequence/property.
union	5.2.4	no	
clock expression	5.3	yes	

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
Bit and std_logic types (VHDL)		yes	
Multiple sensitivity		no	Not supported for VHDL
Edge sensitive clock		yes	
rising_edge(), falling_edge() (VHDL)		yes	
default clock	5.4	yes	If there are multiple clock declarations within a scope, IXCORE accepts the textually closest declaration. Limitation (VHDL-only): Only limited expression types are supported as clock expressions. For example, level-sensitive clock expressions are not supported. For more information on default clock behavior, refer to Default Clock Implementation on page 576.
multiple clock support	4.3.2.3	no	Not supported for VHDL

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
Unlocked assertions	4.4.1	limited	Limitation (VHDL-only): Properties that contain built-in PSL functions, such as <code>ended</code> , <code>prev</code> , <code>rose</code> , <code>fell</code> , and <code>stable</code> , with clock arguments are not supported. The built-in PSL functions, <code>prev</code> , <code>rose</code> , <code>fell</code> , and <code>stable</code> are supported in unlocked assertions only if they are used as Boolean terms and do not appear in HDL expressions.
Temporal Layer			
SERE concatenation (SERE; SERE)	6.1.1.1.1	yes	
SERE sequence fusion (SERE: SERE)	6.1.1.1.2	yes	
SERE or (SERE SERE)	6.1.1.2.1	yes	
SERE and (non-lgth match) (SERE & SERE)	6.1.1.2.2	yes	
SERE and (lgth match) (SERE && SERE)	6.1.1.2.3	yes	
SERE <code>within</code>	6.1.1.2.4	yes	
Parameterized sequence (<code>for..</code>)	6.1.1.2.5	yes	

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
SERE consecutive repeat [*],[+]	6.1.2.1	yes	IES does not support function operator in repeat count expression, such as *1 to MAX+1.
SERE non-consecutive repeat [=]	6.1.2.2	yes	
SERE goto repeat [->]	6.1.2.3	yes	
Braced SERE	6.1.2.4	yes	
Clocked SERE	6.1.2.5	yes	Limitation (VHDL-only): This is not supported if it is multi-clocked.
FL Property Bool Property	6.2.1	yes	
Sequential FL Property Sequence!	6.2.1.1	yes	
Clocked FL Property FL_prop@clk_expr	6.2.1.2	yes	
FL always	6.2.1.3.1	yes	
FL never	6.2.1.3.2	yes	Supported as per simple subset with argument being boolean or sequence.
FL eventually!	6.2.1.3.3	yes	
FL next [n]	6.2.1.3.4	yes	

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
FL next! [n]	6.2.1.3.4	no	
FL next_a [finite_range]	6.2.1.4.1	yes	
FL next_a! [finite_range]	6.2.1.4.1	no	
FL next_e [finite_range]	6.2.1.4.2	yes	The argument property has to be a boolean.
FL next_e! [finite_range]	6.2.1.4.2	no	
FL next_event[!] [n]	6.2.1.4.3	no	
FL next_event_a[!] [n]	6.2.1.4.4	no	
FL next_event_e[!] [n]	6.2.1.4.5	no	
FL abort (implemented as async)	6.2.1.5.1	yes	
FL sync_abort, FL async_abort	6.2.1.5.1	yes	
FL before, FL_before_	6.2.1.5.2	yes	Both arguments should be boolean.
FL before! FL_before!_	6.2.1.5.2	yes	Both arguments should be boolean.
FL until FL_until_	6.2.1.5.3	yes	RHS has to be a boolean.
FL until! FL_until!_	6.2.1.5.3	no	
Sequence implication (->, =>)	6.2.1.6.1	yes	

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
Parameterized property (for..)	6.2.1.7.1	no	
Logical implication (->)	6.2.1.7.2	yes	LHS should be a boolean. For example, always (a->b) ->c is not supported since LHS is a property.
Logical iff (<->)	6.2.1.7.3	no	
Boolean AND_OP FL_Property	6.2.1.7.4	no	Supported by IES.
Boolean OR_OP FL_Property	6.2.1.7.5	no	Supported by IES.
NOT_OP Boolean	6.2.1.7.6	yes	
LTL operators	6.2.1.8	no	Not supported by IES.
OBE operators	6.2.2	no	
Replicated properties (forall)	6.2.3	no	Supported by IES.
PSL Formal parameter types: const, boolean, sequence, property	6.3.1.1	yes	
PSL Formal parameter types: bit, bitvector, numeric, string	6.3.1.1	yes	
HDL Formal types	6.3.1.2	yes	
Sequence Declaration	6.3.2.1	yes	
Property Declaration	6.3.2.2	yes	

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
Verification Layer			
support of labels	7.1	yes	
support of report	7.1	yes	
support of severity	N/S	yes	
assert property [report] [severity]	7.1.1	yes	
assume property [report] [severity]	7.1.2	yes	assume is mapped to assert, report and severity is supported in assume directive
assume_guarantee property [report][severity]	7.1.3	no	Ignored
restrict sequence [report]	7.1.4	yes	
restrict_guarantee sequence [report]	7.1.5	no	Ignored.
cover sequence [report]	7.1.6	yes	
fairness	7.1.7	no	Ignored.
strong fairness	7.1.7	no	Ignored.
'vmode' & 'vprop'	7.2	yes	
vunit binding (module based)	7.2.1	yes	
vunit binding (to instances)		yes	Not supported for VHDL

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
vunit inheritance	7.2.2	yes	
vunit scoping rules (cross language binding)	7.2.3	no	
properties in <code>for-generate</code> statements	N/S	yes	
properties in <code>if-case-generate</code> statements	N/S	yes	
support opt clk expr in built-in functions	5.2.3	yes	Supported if the clock expression is same as that of the assertion.
use of <code>ended/endpoints</code> in boolean expressions	5.2.3.6	yes	Limitation (VHDL-only): <code>ended/endpoints</code> are not supported if these occur within an HDL expression. For example: <code>a and ended (b)</code> .
instantiation of props in other props	6.2.1	yes	
parameterized declaration of seq & props	6.3.2.1, 6.3.2.2	yes	
HDL in a vunit	7.2	yes	
pragma support in vunits (protection pragma,..)	7.2	no	
<code>`define</code> & <code>`ifdef</code> in external files	7.2	yes	

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-5 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
<code>%if</code> and <code>%for</code> PSL macro	4.2.5	yes	Support restricted to vunits only
vunits should not be able to see others	7.2.1	yes	
binding to vunit instances	7.2.1	no	
inheritance, vmode, vprop	7.2.1	yes	
Modeling layer	8	limited	Certain features, such as re-declaration of object with the same name are not allowed.
support of complex report args	N/S	yes	
properties in VHDL packages	N/S	yes	
<code>endpoints/ended</code> in unlocked properties		no	
<code>clocked endpoints/ended</code>		yes	Supported only if the clock expression of the endpoint is same as that of the entire property.
<code>endpoints/ended</code> on RHS of <code>sync_abort/async_abort</code>		no	
complex first argument of <code>prev</code>	N/S	yes	
Assertions with large repeat count	N/S	no	Not supported, by default, but by providing high value for <code>+assertMaxIter</code>

For the purpose of this table:

N/S or Non-Std means Not Specified or included as part of the IEEE 1850 Standard.

Support for Specific PSL Constructs in a Verilog Design

The following table shows the current status of support for specific PSL Constructs in a Verilog design.

Table 10-6 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
Boolean Layer			
Bit expressions	5.1.1	yes	
Boolean expressions including "true" (1'b1)	5.1.2	yes	
BitVector expressions	5.1.3	yes	
Numeric expressions	5.1.4	yes	
String expressions	5.1.5	yes	
HDL expressions	5.2.1	yes	■ HDL signals with names that are PSL keywords, such as Clock, A, or E are not supported within PSL statements. ■ Concatenation expressions with single term is not supported.

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-6 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
PSL expressions	5.2.2	limited	Support as per simple subset. Use of <code>-></code> and <code><-></code> in the context of boolean expression is not supported. For example, <code>always {a->b; c->d}</code> .
Built-in Functions			
<code>prev(Any_type[, number[, clock_expr]])</code>	5.2.3.1	yes	■ Only integral types are supported. ■ Optional clock should be same as the clock of the sequence/property expression in context.
<code>next(Any_type)</code>	5.2.3.2	no	
<code>stable(Any_type[, clock_expr])</code>	5.2.3.3	yes	■ Optional clock is supported if it is same as the clock of the sequence/property expression in context. ■ Only integral types are supported.
<code>rose(bit[, clock_expr])</code>	5.2.3.4	yes	■ Optional clock is supported if it is same as the clock of the sequence/property expression in context. ■ Only integral types are supported.

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-6 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
<code>fell(bit[,clock_expr])</code>	5.2.3.5	yes	■ Optional clock is supported if it is same as the clock of the sequence/property expression in context. ■ Only integral types are supported.
<code>isunknown(BitVector)</code>	5.2.3.7	yes	Returns false if running on the emulator.
<code>ended(sequence[,clock_expr])</code>	5.2.3.6	yes	■ Optional clock is supported if the clock is same as the clock of the enclosing sequence/property in context. In HDL expressions, <code>ended</code> is treated as SVA <code>ended</code>. ■ <code>endpoints/ended</code> on RHS of <code>sync_abort/async_abort</code> are not supported.
<code>countones(BitVector)</code>	5.2.3.8	yes	Only integral types are supported.
<code>nondet(Value_List)</code>	5.2.3.9	no	
<code>nondet_vector(Number, Value_List)</code>	5.2.3.10	no	
<code>onehot(BitVector), onehot0(BitVector)</code>	5.2.3.11	yes	Only integral types are supported.

UXE User Guide
 Compiling and Running Designs with Assertions

Table 10-6 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
named endpoint (single cycle)	N/S	yes	Optional clock is supported if the clock is same as the clock of the enclosing sequence/property.
named endpoint (full temporal multi-cycle)		yes	Optional clock is supported if the clock is same as the clock of the enclosing sequence/property.
union	5.2.3.12	no	
clock expression	5.3	yes	
Built-in functions rose, fell		yes	
Multiple sensitivity		yes	
Edge sensitive clock		yes	
HDL_CLOCK_EXPR	4.3.2.3	yes	
default clock	5.4	yes	If there are multiple clock declarations within a scope, IXCORE/IES accepts the textually closest declaration.
multiple clock support	4.2.3.2.2	yes	
Unlocked assertions	4.4.1	yes	
Temporal Layer			

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-6 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
SERE concatenation (SERE; SERE)	6.1.1.1.1	yes	
SERE sequence fusion (SERE: SERE)	6.1.1.1.2	yes	
SERE or (SERE SERE)	6.1.1.2.1	yes	
SERE and (non-lgth match) (SERE & SERE)	6.1.1.2.2	yes	
SERE and (lgth match) (SERE && SERE)	6.1.1.2.3	yes	
SERE <code>within</code>	6.1.1.2.4	yes	
Parameterized sequence (<code>for..</code>)	6.1.1.2.5	yes	
SERE consecutive repeat <code>[*],[+]</code>	6.1.2.1	yes	IES does not support function operator in repeat count expression, such as <code>*1 to MAX+1</code> .
SERE non-consecutive repeat <code>[=]</code>	6.1.2.2	yes	
SERE goto repeat <code>[->]</code>	6.1.2.3	yes	
Braced SERE	6.1.2.4	yes	
Clocked SERE	6.1.2.5	yes	
FL Property Bool Property	6.2.1	yes	
Sequential FL Property Sequence!	6.2.1.1	yes	

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-6 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
Clocked FL Property FL_prop@clk_expr	6.2.1.2	yes	
FL always	6.2.1.3.1	yes	
FL never	6.2.1.3.2	yes	Supported as per simple subset with argument being boolean or sequence.
FL eventually!	6.2.1.3.3	yes	
FL next [n]	6.2.1.3.4	yes	
FL next! [n]	6.2.1.3.4	no	
FL next_a [finite_range]	6.2.1.4.1	yes	
FL next_a! [finite_range]	6.2.1.4.1	no	
FL next_e [finite_range]	6.2.1.4.2	yes	The argument property has to be a boolean.
FL next_e! [finite_range]	6.2.1.4.2	no	
FL next_event[!] [n]	6.2.1.4.3	no	
FL next_event_a[!] [n]	6.2.1.4.4	no	
FL next_event_e[!] [n]	6.2.1.4.5	no	
FL abort (implemented as async)	6.2.1.5.1	yes	

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-6 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
FL sync_abort, FL async_abort	6.2.1.5.1	yes	
FL before, FL_before_	6.2.1.5.2	yes	Both arguments should be boolean.
FL before! FL_before!_	6.2.1.5.2	yes	Both arguments should be boolean.
FL until FL_until_	6.2.1.5.3	yes	RHS has to be a boolean.
FL until! FL_until!_	6.2.1.5.3	no	
Sequence implication (->, =>)	6.2.1.6.1	yes	
Parameterized property (for..)	6.2.1.7.1	no	
Logical implication (->)	6.2.1.7.2	yes	LHS should be a boolean. For example, always (a->b) ->c is not supported since LHS is a property.
Logical iff (<->)	6.2.1.7.3	no	
Boolean AND_OP FL_Property	6.2.1.7.4	no	Supported by IES.
Boolean OR_OP FL_Property	6.2.1.7.5	no	Supported by IES.
NOT_OP Boolean	6.2.1.7.6	yes	
LTL operators	6.2.1.8	yes	Not supported by IES.
OBE operators	6.2.2	no	

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-6 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
Replicated properties (<code>forall</code>)	6.2.3	no	Supported by IES.
PSL Formal parameter types: <code>const</code> , <code>boolean</code> , <code>sequence</code> , <code>property</code>	6.3.1.1	yes	
PSL Formal parameter types: <code>bit</code> , <code>bitvector</code> , <code>numeric</code> , <code>string</code>	6.3.1.1	yes	
HDL Formal types	6.3.1.2	yes	
Sequence Declaration	6.3.2.1	yes	
Property Declaration	6.3.2.2	yes	
Verification Layer			
support of <code>labels</code>	7.1	yes	
support of <code>report</code>	7.1	yes	
support of <code>severity</code>	N/S	yes	
<code>assert property</code> <code>[report] [severity]</code>	7.1.1	yes	
<code>assume property</code> <code>[report] [severity]</code>	7.1.2	yes	<code>assume</code> is mapped to <code>assert</code> ; <code>report</code> and <code>severity</code> is supported in <code>assume</code> directive.
<code>assume_guarantee property</code> <code>[report] [severity]</code>	7.1.3	no	Ignored

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-6 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
<code>restrict sequence [report]</code>	7.1.4	yes	
<code>restrict_guarantee sequence</code>	7.1.5	no	Ignored.
<code>cover sequence [report]</code>	7.1.6	yes	
<code>fairness</code>	7.1.7	no	Ignored.
<code>strong fairness</code>	7.1.7	no	Ignored.
<code>'vmode' & 'vprop'</code>	7.2	yes	
vunit binding (module-based)	7.2.1	yes	
vunit binding (to instances)	7.2.1	yes	
vunit inheritance	7.2.2	yes	
vunit scoping rules (cross language binding)	7.2.3	no	
properties in <code>for-generate</code> statements	N/S	yes	
properties in <code>if-case-generate</code> statements	N/S	yes	
pragma support in vunits (protection pragma,..)	7.2	no	
<code>%if</code> and <code>% for</code> PSL macro	4.2.5	yes	Support restricted to vunits only.

UXE User Guide

Compiling and Running Designs with Assertions

Table 10-6 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
support opt clk expr in built-in functions	5.2.3	yes	Supported if the clock expression is same as that of the property in context.
use of <code>ended</code> in boolean expressions	5.2.3.6	yes	
instantiation of props in other props	6.2.1	yes	
parameterized declaration of seq & props	6.3.2.1, 6.3.2.2	yes	
HDL in a vunit	7.2	yes	
vunits should not be able to see others	7.2.1	yes	
Modeling layer	8	limited	
support of complex report args	N/S	yes	
<code>endpoints/ended</code> in unlocked properties		yes	
<code>clocked endpoints/ended</code>		yes	Supported only if the clock expression of the endpoint is same as that of the entire property.
<code>endpoints/ended</code> on RHS of <code>sync_abort/async_abort</code>		no	
complex first argument of <code>prev</code>		yes	

UXE User Guide
Compiling and Running Designs with Assertions

Table 10-6 Support for Specific PSL Constructs

PSL Construct	PSL1850-2005 LRM Section	Supported	Notes
Assertions with large repeat counts		no	It is not supported by default. Supported by providing a high value for <code>+assertMaxIter</code> .
Multiclocked properties/sequence		yes	Optional clock argument in PSL-builtin functions should be same as that of the sequence/property expression. The sequence fusion, sequence and (&), sequence or (), sequence length matching and (&&), and sequence within operators are not supported. The repetition of multiply clocked sequence is not supported. The property operator until, until_, before, before_, abort, sync_abort, async_abort, property and, property or, logical if, logical iff, and overlapping implication are not supported to have multiply clocked operands.

Support for Incisive Assertion Library Components

The Incisive Assertion Library (IAL) components can be compiled to use SVA or PSL for the underlying assertion language. Refer to the IAL documentation for the required command-line arguments to enable IAL.

The Incisive Assertion Library is named `ial.v` and is installed in the following location:

```
<IES_install_dir>/tools/ial/verilog
```

Examples of the IAL components are located in the following directory:

```
<IES_install_dir>/tools/ial/examples/verilog/simulation
```

Supported Basic Types

```
ial_active_bits
ial_always
ial_always_on_edge
ial_bitcnt
ial_compare
ial_constant
ial_cover
ial_gray_code
ial_hamming_distance
ial_implication
ial_mutex
ial_never
ial_never_unknown
ial_never_unknown_async
ial_next
ial_no_overflow
ial_no_underflow
ial_one_cold
ial_one_hot
ial_proposition
ial_quiescent_state
ial_range
ial_time
ial_timeout
ial_tristate
```

ial_zero_one_hot
ial_value

Supported Interface Types

ial_change
ial_cycle_sequence
ial_follower
ial_frame
ial_handshake
ial_leader
ial_no_transition
ial_stall
ial_together
ial_transition
ial_unchange
ial_width
ial_win_change
ial_win_unchange
ial_window

Supported Control Types

ial_eq_frame
ial_eq_seq
ial_req_gnt_ack
ial_stack
ial_var_time_seq
Interface and Control Types
ial_arbiter
ial_case_check
ial_fifo
ial_fifo_index
ial_mclk_mport_fifo
ial_mux
ial_outstanding_id

Supported Datapath Types

ial_arith_overflow

`ial_crc`
`ial_decoder`
`ial_decr`
`ial_delta`
`ial_encoder`
`ial_even_parity`
`ial_incr`
`ial_max`
`ial_min`
`ial_odd_parity`
`ial_parallel2serial`
`ial_serial2parallel`

Unsupported Incisive Assertion Library Components

`ial_tristate`

Comparison with `z` is always treated as `false` on the Palladium XP system.

`ial_mem_access`

`ial_mport_mem_access`

Depending on the parameters used, `ial_mem_access` and `ial_mport_mem_access` can have a detrimental effect on the performance of the Palladium XP system.

Support for Open Verification Library Components

The Open Verification Library (OVL) components can be compiled to use SVA or PSL for the underlying assertion language.

Supported OVL Components

IXCOM supports the following OVL 2.6 version components.

```
ovl_code_distance
ovl_hold_value
ovl_no_contention
ovl_reg_loaded
ovl_stack
ovl_always
ovl_always_on_edge
ovl_bits
ovl_change
ovl_cycle_sequence
ovl_decrement
ovl_delta
ovl_even_parity
ovl_fifo_index
ovl_frame
ovl_handshake
ovl_implication
ovl_increment
ovl_mutex
ovl_never
ovl_next
ovl_no_overflow
ovl_no_transition
ovl_no_underflow
ovl_odd_parity
ovl_one_cold
ovl_one_hot
ovl_proposition
ovl_quiescent_state
ovl_range
ovl_req_requires
```

UXE User Guide

Compiling and Running Designs with Assertions

ovl_time
ovl_transition
ovl_unchange
ovl_valid_id
ovl_value
ovl_width
ovl_win_change
ovl_win_unchange
ovl_window
ovl_zero_one_hot
ovl_never_unknown
ovl_never_unknown_async
ovl_next_state
ovl_fifo
ovl_arbiter
ovl_multiport_fifo
ovl_memory_async
ovl_memory_sync
ovl_req_ack_unique

Using the Dynamic Target Mode

A target system (or just a target) is an external board-level electronic system you provide that communicates with the emulated design through special connectors on the Palladium XP system's external panel. A static target is a target system whose clocks can be stopped and restarted without corrupting its state. A dynamic target is a target whose clocks cannot be stopped and restarted.

Dynamic Target mode is suited for running a design while using dynamic targets in the IXCOM flow. This chapter covers the following topics:

- [Overview of Dynamic Target Mode](#)
- [Assertions Support in Dynamic Target Mode](#)
- [Limitations of Dynamic Target Mode](#)

Overview of Dynamic Target Mode

The Dynamic Target mode is based on the autorun mode available in the IXCOM flow. When you invoke the autorun mode using the command `xc on -autorun`, the system is configured to run with static targets.

Dynamic Target mode in the IXCOM flow corresponds to the Logic Analyzer (LA) mode in the ICE flow. For example, the rules for reading and writing memories and signal values in the Dynamic Target mode are exactly the same as in LA mode. It is possible to switch between the Dynamic Target mode and the Static Target mode during a debug session. This is useful when dealing with a target system that behaves both as dynamic target and static target. In such a case, some segments of the debug session are run in the Dynamic Target mode as the clock cannot be interrupted, while other segments can take advantage of the better debug capabilities of the Static Target mode. By switching between the Dynamic Target mode and the regular autorun mode (which supports running with a static target), features of IXCOM environment, such as hot swap using IES can be used.

This section provides steps for running and stopping designs in the Dynamic Target mode (both non blocking and blocking), and switching between the Dynamic Target mode and Static Target mode.

Running Designs in Dynamic Target Mode

By default, a design run in the Dynamic Target mode is non blocking, which means that you get the prompt to run other commands immediately. You can also run in the blocking mode, which means that the running of other commands (or the availability of command prompt in an interactive session) is suspended until the `run` command terminates.

To run a design in the Dynamic Target mode:

1. Compile the design.
2. Invoke the autorun mode.

```
xc on -autorun
```

The above command configures the system for Static Target mode.

3. Switch to the Dynamic Target mode using the following command:

```
configPM -dynamicTarget
```

4. Use the following command to run designs in the Dynamic Target mode:

```
run
```

The above command runs the design in non-blocking mode, by default. To run the design in the blocking mode, use the following command:

```
run -wait
```

The `-nowait` option is supported only for compatibility with the ICE mode. If this option is used with the `run` command, it is silently ignored.

The first `run` command executed after the `configPM -dynamicTarget` command starts both the design clocks and the trace clocks. Subsequent `run` commands only start the trace clocks because design clocks are already running.

To start the design clocks only, without starting the trace clocks, use the following command:

```
run -emulation
```

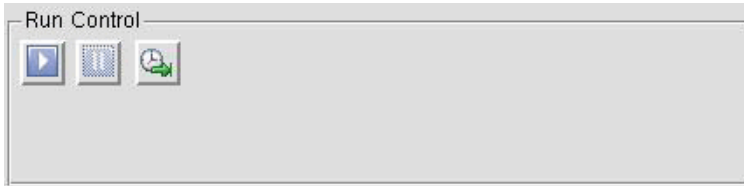
Stopping a Design

- To stop a run in the non-blocking mode, use the `stop` command without any arguments.

- To stop a run in the blocking mode, use `Ctrl-C`.

In the xeDebug GUI, you can start and stop a run using the buttons in the *Run Control* frame on the *Control* tab, as shown in [Figure 11-1](#) on page 613.

Figure 11-1 Run Controls on Control Tab



Switching between the Different Modes

The `configPM` command can be called any number of times during a session to switch between the Dynamic and Static Target modes.

- To determine the current mode, run the `configPM` command without any argument:

```
configPM
```

- To switch from Dynamic Target mode to the Static Target mode, use the following command:

```
configPM -staticTarget
```

The above command stops any design clocks (if running) and configures the system back to the Static Target mode.

Note: When switching from Dynamic Target mode to Static Target mode, the waveform data that was captured in the emulator but not uploaded to the workstation will be lost. To avoid the loss of waveform data, you must upload the waveform using the `database -upload` command before switching to Static Target mode.

For more information about the `configPM` command, see the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

When running in the Dynamic Target mode, the testbench callbacks are not executed. When you switch to the Static Target mode, the following actions are taken at run time:

- All pending testbench callbacks are triggered.

Note: When there are multiple instances of testbench callback trigger conditions, only the last one will be processed. It is your responsibility to handle the design requirements that require every trigger condition to be processed.

- The IES software time is synchronized with the hardware time.
- The co-simulation interfaces get activated.

In the xeDebug GUI, the *Setup* tab has two radio buttons corresponding to the *Debug Mode* field within the *Configuration* frame (refer to the highlighted area in [Figure 11-2](#) on page 614). Select the appropriate radio button to switch between the Dynamic and Static Target modes.

Figure 11-2 Switching Controls on Setup Tab



Assertions Support in Dynamic Target Mode

Although assertions handling in UXE mostly depends on a simulator, the Dynamic Target mode provides some support for handling assertions without using a simulator.

For information on how assertions are supported in the Dynamic Target mode, see [Using Assertions in Dynamic Target Mode](#) section in [Chapter 10, “Compiling and Running Designs with Assertions.”](#)

Limitations of Dynamic Target Mode

Following are the limitations when using the Dynamic Target mode:

- The `force/release/deposit/value` requests and memory `read/write` operations are executed without suspending the design clocks; therefore, time coherency is not guaranteed.
- The following run-time features are not available in the Dynamic Target mode:
 - ❑ InfiniTrace
 - ❑ Saving and restoring design state
- The software simulator does not execute any commands in the dynamic target and non-blocking run modes, even though the command prompt is available.

UXE User Guide

Using the Dynamic Target Mode

- A number of assertion features require simulator support and hence are not available or do not function during a design run in the Dynamic Target mode. Following is a list of such features:
 - ❑ Failure action blocks
 - ❑ Reporting of assertion failures
 - ❑ Enabling or disabling of assertions
 - ❑ Assertion Control Tasks (ACTs)
 - ❑ `assertion -kill -onfailure` command
 - ❑ Assertion summary is not available until the end of design run

UXE User Guide

Using the Dynamic Target Mode

Implementing SCE-MI Pipes

The UXE TBA solution provides an implementation of the SCE-MI pipes interface. A SCE-MI pipe is defined as a construct that enables streaming of transactions between the testbench partition (C-side in SCE-MI definition) running on the simulator and hardware partition (HDL-side in SCE-MI definition) running on the Palladium XP emulator.

UXE supports SCE-MI 2.0 and 2.1 standards for SCE-MI Pipes. The SCE-MI 2.1 implementation is not completely backward-compatible with the SCE-MI 2.0 implementation because of variations in the two standards. UXE provides two sets of libraries for the two SCE-MI implementations, as discussed in [“Software Components of SCE-MI Pipes Interface”](#) on page 621.

This chapter contains the following sections:

- [Overview of Transaction Pipes](#)
- [New Features in SCE-MI 2.1 Pipes Interface](#)
- [Software Components of SCE-MI Pipes Interface](#)
- [Testbench-Side Thread-Neutral C Interfaces](#)
- [DUT-Side HDL Interface](#)
- [Executing the Simulator Thread](#)
- [Reset of SCE-MI Pipes](#)
- [Using Language-Based Adaptors](#)
- [Generating DUT Clocks with SCE-MI Pipes](#)
- [Compiling and Running Designs with SCE-MI Pipes](#)
- [Examples: Implementing SCE-MI Pipes](#)

Overview of Transaction Pipes

UXE provides support for the SCE-MI 2.1 and SCE-MI 2.0 implementations of transaction pipes as defined by:

- Section 5.8, *Pipes-Based Interface: Transaction Pipes* of the *Accellera Standard Co-Emulation Modeling Interface (SCE-MI) Reference Manual Version 2.1*.
- Section 5.7, *Pipes-based Use Model: Transaction Pipes* of the *Accellera SCE-MI Reference Manual Version 2.0*.

Refer to these manuals for a complete description of Pipes functionality and the SCE-MI API.

All the features described in the relevant sections of these manuals are implemented in UXE, exactly as specified, unless otherwise stated in this chapter.

SCE-MI Transaction Pipes are unidirectional, which means that the transactions flow only in one direction. This ensures that the transactions are received in the same order as they were sent when the receiver accesses these transactions through function calls. SCE-MI pipes are of the following two types:

- **Input pipes:** SCE-MI pipes that pass one-way transactions from the testbench (running on the simulator) to the DUT (running on the emulator).
- **Output pipes:** SCE-MI pipes that pass one-way transactions from the DUT (running on the emulator) to the testbench (running on the simulator).

New Features in SCE-MI 2.1 Pipes Interface

This section discusses the UXE implementation for the following new features of SCE-MI 2.1 Pipes Interface:

- Deferred and Immediate Visibility Modes
- Notification Threshold
- Callback Threshold
- New and Enhanced Functions of SCE-MI 2.1

Deferred and Immediate Visibility Modes

UXE supports both the `immediate` and `deferred` visibility modes defined by SCE-MI 2.1.

Set the visibility mode for each pipe by setting an appropriate value for the HDL-side `VISIBILITY_MODE` parameter of the pipe interface. For details about setting the visibility mode in the pipes-based interface, refer to *Section 5.8.5.1.1* of the *Accellera Standard Co-Emulation Modeling Interface (SCE-MI) Reference Manual Version 2.1*.

SCE-MI defines the default value for the `VISIBILITY_MODE` parameter as 0, which denotes an illegal pipe interface with unspecified mode of operation. If you do not set the `VISIBILITY_MODE` parameter, UXE sets this parameter to `immediate` and a warning message is generated at run time.

Notification Threshold

A pipe supports a static HDL-side notification threshold. The threshold can be:

- The minimum number of elements that must be added to an empty pipe for consumer notification, or
- The minimum number of elements that must be removed from a full pipe for a producer notification.

Specify the notification threshold by using the `NOTIFICATION_THRESHOLD` parameter.

When the notification threshold is set to 1, notifications due to send and receive operations occur immediately, that is, after adding at least one element to an empty pipe or removing at least one element from a full pipe. Notifications due to flush operations are not affected by the notification threshold setting.

The valid values for the `NOTIFICATION_THRESHOLD` parameter depend on the visibility mode. In the `deferred` visibility mode, the `NOTIFICATION_THRESHOLD` will be set to `BUFFER_MAX_ELEMENTS` regardless of the value you set. In the `immediate` visibility mode, `NOTIFICATION_THRESHOLD` will be set to `BUFFER_MAX_ELEMENTS` unless it is explicitly set to 1. Any value for `NOTIFICATION_THRESHOLD` other than 1 or `BUFFER_MAX_ELEMENTS` results in an error.

For details about notification threshold in the pipes-based interface, refer to *Section 5.8.5.1.3* of the *Accellera Standard Co-Emulation Modeling Interface (SCE-MI) Reference Manual Version 2.1*.

Callback Threshold

The `callback_threshold` can be set to a value between 1 and `NOTIFICATION_THRESHOLD`. For an input pipe, dynamic callback can occur whenever `scemi_pipe_c_can_send() >= callback_threshold`. For an output pipe, dynamic

callback can occur whenever `scemi_pipe_c_can_receive()` $\geq$ `callback_threshold`. A warning message is generated if an invalid value is specified.

To ensure identical implementation-independent callback timing, `callback_threshold` should be set to the `notification_threshold` of the pipe.

For details about callback threshold in the pipes-based interface, refer to *Section 5.8.5.3.3* of the *Accellera Standard Co-Emulation Modeling Interface (SCE-MI) Reference Manual Version 2.1*.

New and Enhanced Functions of SCE-MI 2.1

Following is a list of new and enhanced functions for SCE-MI 2.1. The reference section number in the *Accellera Standard Co-Emulation Modeling Interface (SCE-MI) Reference Manual Version 2.1* has been specified in square brackets.

- `void *scemi_pipe_get_user_data(void *pipe_handle, void *user_key); [5.8.5.3.4]`
- `void scemi_pipe_put_user_data(void *pipe_handle, void *user_key, void *user_data); [5.8.5.3.4]`
- `svBit scemi_pipe_get_direction(void *pipe_handle); [5.8.5.3.3]`
- `svBit scemi_pipe_c_in_flush_state(void *pipe_handle); [5.8.5.3.2]`
- `typedef void *scemi_pipe_notify_callback_handle; [5.8.5.3.3]`
- `scemi_pipe_notify_callback_handle
scemi_pipe_set_notify_callback(
 void *pipe_handle,
 scemi_pipe_notify_callback notify_callback,
 void *notify_context,
 int callback_threshold); [5.8.5.3.3]`
- `void scemi_pipe_clear_notify_callback(
 scemi_pipe_notify_callback_handle
 notify_callback_handle); [5.8.5.3.3]`

Software Components of SCE-MI Pipes Interface

The core components for implementing the SCE-MI pipes interface are a set of encrypted HDL and pre-compiled shared libraries that implement the non-blocking `send` and `receive` operations of SCE-MI Pipes interface.

Encrypted HDL and Pre-Compiled Shared Libraries

■ Encrypted HDLs:

- These Verilog files are used to implement the core functions of SCE-MI pipe and interfaces on the HDL-side:

- `scemi_input_pipe.vp`
- `scemi_output_pipe.vp`
- `scemi_pipe.vp`

(Includes `scemi_input_pipe.vp` and `scemi_output_pipe.vp` files)

Note: The same set of files is used for implementing SCE-MI 2.0 and SCE-MI 2.1 pipes. At compilation time, however, use the `+define+SCEMI21` option for a design that utilizes SCE-MI 2.1.

■ Pre-compiled shared libraries:

- For SCE-MI 2.0 pipes, use the `libuatba.so` and `libuatba_dc.so` shared libraries and the `uatba.h` C header file to implement the core functions of the pipe and thread-neutral C-interface for the testbench partition.

The UXE SCE-MI 2.0 libraries (`libuatba.so` and `libuatba_dc.so`) contain only non-blocking SCE-MI interface functions. To access the blocking interface functions, you need to compile the blocking interface source code provided with the UXE TBA solution, such as the `uatba_scemi.c`, `uatba_event.cpp` and `uatba_sc_event.cpp` files.

- For SCE-MI 2.1 pipes, use the `libscemi21.so`, `libscemi21_hmpi_so`, and `libscemi21_dc.so` shared libraries and the `uatba.h` header file to implement the core functions of the pipe and thread-neutral C-interface for the testbench partition.

The blocking interface functions have been built into the SCE-MI 2.1 libraries (`libscemi21` and `libscemi21_dc`). You do not need to add the blocking interface source code in the compilation.

UXE User Guide

Implementing SCE-MI Pipes

The HMPI library for SCE-MI 2.0 (`libuatba_hmpi.so`) and HMPI library for SCE-MI 2.1 (`libscemi21_hmpi.so`) also have built-in blocking interface functions.

The “_dc” in the names of the libraries mentioned above refers to Direct C or direct access to C code.

The “_dc.so” libraries are used only for pure software simulation.

These core components are used with language-based adaptors that are provided in source form to implement blocking `send` and `receive` operations of the SCE-MI pipe. The following table lists the language adaptors.

Table 12-1 Language Adaptors

Language Adaptor	Description
<code>uatba_event.cpp</code> and <code>uatba_event.sv</code>	These source files are used to implement simulation thread suspension and continuation control from C-side. Include these files with all language adaptors that are based on the native simulator threads. See Executing the Simulator Thread on page 626 for more information.
<code>uatba_scemi.c</code> and <code>uatba_scemi.h</code>	These source files are used to implement the SCE-MI blocking C-interfaces. See SCE-MI Pipes Blocking C Interfaces on page 630 for more information.
<code>uatba_cpp.h</code>	This source file is used to implement the C++ interface adaptor. See SCE-MI Pipes C++ Interface on Simulator Thread on page 630 for more information.
<code>uatba_sc.h</code> and <code>uatba_sc.cpp</code>	These source files are used to implement the SystemC interface adaptor. See SCE-MI Pipes SystemC Interface on SystemC Thread on page 631 for more information.
<code>uatba_sv.sv</code> and <code>uatba_class.svh</code>	These source files are used to implement the SystemVerilog interface adaptor. See SCE-MI Pipes SVTB Interface on Simulator Thread on page 632 for more information.

Language Adaptor

uatba_sn.e and
svtb.sv

Description

These source files are used to implement the Specman interface adaptor.

See [SCE-MI Pipes Specman Interface on Specman Thread](#) on page 633 for more information.

Testbench-Side Thread-Neutral C Interfaces

Based on the current SCE-MI pipes definition, following are the thread-neutral C-interface functions that are supported on the testbench side:

Common Configuration and Query Functions:

- `void *scemi_pipe_c_handle(const char *endpoint_path );`
- `typedef void (*scemi_pipe_notify_callback)(void *context );`
- `void *scemi_pipe_get_notify_context(void *pipe_handle );`
- `int scemi_pipe_get_bytes_per_element(void *pipe_handle );`
- `int scemi_pipe_get_depth(void *pipe_handle );`

SCE-MI 2.0 Functions:

- `void scemi_pipe_set_notify_callback(void *pipe_handle, scemi_pipe_notify_callback notify_callback, void *notify_context);`
- `int scemi_pipe_get_direction(void *pipe_handle);`
- `void scemi_pipe_set_eom_auto_flush(void *pipe_handle, svBit enabled);`

SCE-MI 2.1 Functions:

- `typedef void *scemi_pipe_notify_callback_handle;`
- `scemi_pipe_notify_callback_handle
scemi_pipe_set_notify_callback(void *pipe_handle,
scemi_pipe_notify_callback notify_callback, void
*notify_context, int callback_threshold);`

UXE User Guide

Implementing SCE-MI Pipes

- `void
scemi_pipe_clear_notify_callback(scemi_pipe_notify_callback_handle callback_handle);`
- `svBit scemi_pipe_set_eom_auto_flush(void *pipe_handle, svBit enabled );`
- `svBit scemi_pipe_get_direction(void *pipe_handle);`
- `void scemi_pipe_put_user_data(void *pipe_handle, void *user_key, void *user_data);`
- `void *scemi_pipe_get_user_data(void *pipe_handle, void *user_key);`
- `svBit scemi_pipe_c_in_flush_state(void *pipe_handle);`

SCE-MI Reference Manual 2.0 defines the `set_depth` function shown below, but this function has no effect in the UXE implementation of SCE-MI pipes because the depth of the buffer is defined at the HDL-side.

```
void scemi_pipe_set_depth(void *pipe_handle, int num_elements );
```

A warning message is printed out when this function is used.

SCE-MI input pipe non-blocking interface:

```
int scemi_pipe_c_try_send(void *pipe_handle, int byte_offset, int  
                        num_elements, const svBitVecVal *data, svBit eom );  
int scemi_pipe_c_can_send(void *pipe_handle );  
void scemi_pipe_c_try_flush(void *pipe_handle);
```

SCE-MI output pipe non-blocking interface:

```
int scemi_pipe_c_try_receive(void *pipe_handle, int byte_offset, int  
                           num_elements, svBitVecVal *data, svBit *eom );  
int scemi_pipe_c_can_receive(void *pipe_handle);
```

In addition to the `svBitVector`-based interfaces that are defined in the SCE-MI standard, IXCOM supports the unpacked open byte array interfaces. To directly transfer unpacked open byte array into the pipe, use the `uatba_try_send_ba` function. This function has the same semantics as the `scemi_pipe_c_try_send` function except that the C-char array is used as the data type.

```
int uatba_try_send_ba(void *pipe_handle, int byte_offset,  
                    int num_bytes, const char *data, int eom);
```

This function returns the number of requested bytes that are actually sent. Similarly, the `uatba_try_receive_ba` function returns the number of bytes that are actually received. The syntax of the `uatba_try_receive_ba` function is given below:

UXE User Guide

Implementing SCE-MI Pipes

```
int uatba_try_receive_ba(void *pipe_handle, int byte_offset,
                        int num_bytes, char *data, int *eom);
```

DUT-Side HDL Interface

Based on the current SCE-MI pipe definitions, the DUT-side interfaces are implemented as Verilog modules as given below:

```
interface scemi_input_pipe;
    parameter BYTES_PER_ELEMENT = 1;
    parameter PAYLOAD_MAX_ELEMENTS = 1;
    parameter BUFFER_MAX_ELEMENTS = 16;
    `ifdef DEFER_MODE
        parameter VISIBILITY_MODE = 2; // 1:immediate 2:deferred
    `else
        `ifdef SCEMI21
            parameter VISIBILITY_MODE = 0;
        `else
            parameter VISIBILITY_MODE = 1;
        `endif
    `endif
    `ifdef SCEMI21
        parameter NOTIFICATION_THRESHOLD = BUFFER_MAX_ELEMENTS;
    `endif
    `ifdef HMPI
        parameter hmpi = 1;
    `else
        parameter hmpi = 0;
    `endif

    event sn_notify;
    task receive(input int num_elements, output int num_elements_valid,
                output bit [PAYLOAD_MAX_BITS-1:0] data, output bit eom );
    function int try_receive( int byte_offset, int num_elements,
                             output bit [PAYLOAD_MAX_BITS-1:0] data, output bit eom );
    function int can_receive(); // return: #elements that can be received
endinterface

interface scemi_output_pipe;
    parameter BYTES_PER_ELEMENT = 1;
    parameter PAYLOAD_MAX_ELEMENTS = 1;
```

UXE User Guide

Implementing SCE-MI Pipes

```
parameter BUFFER_MAX_ELEMENTS = 16;
`ifdef DEFER_MODE
    parameter VISIBILITY_MODE = 2; //2:deferred
`else
    `ifdef SCEMI21
        parameter VISIBILITY_MODE = 0;
    `else
        parameter VISIBILITY_MODE = 1;
    `endif
`endif

`ifdef SCEMI21
    parameter NOTIFICATION_THRESHOLD = BUFFER_MAX_ELEMENTS;
`endif

`ifdef HMPI
    parameter hmpi = 1;
`else
    parameter hmpi = 0;
`endif

event sn_notify;
task send(input int num_elements, input bit [PAYLOAD_MAX_BITS-1:0] data,
input bit eom );
task flush;
function int try_send( int byte_offset, int num_elements,
bit [PAYLOAD_MAX_BITS-1:0] data, input bit eom );
function int can_send();
endinterface
```

UXE exports a special event signal, `sn_notify` in the above modules for Specman.

Executing the Simulator Thread

Implementation of the SCE-MI pipes blocking interface needs to be based on the thread system running on the testbench side. For a simulation environment, the simulator thread is the thread system that is common to all the hardware verification languages supported by the simulator. Using the SystemVerilog DPI export task mechanism, the common simulation thread functions can be accessed from all the hardware verification languages through their C-interfaces.

The `uatba_event.sv` defines the SystemVerilog DPI module function that is used by the SCE-MI Pipes blocking interface to implement thread execution control.

UXE User Guide

Implementing SCE-MI Pipes

```
module uatba_event;
    class uatba_ev;
        static uatba_ev eArr[];
        export "DPI-C" function int uatba_new_event(); // return event_id
        export "DPI-C" task uatba_trigger(int event_id);
        export "DPI-C" task uatba_wait(int event_id);
    endmodule
```

Specify this module as the top-level module during the elaboration phase. This module contains an open array of SystemVerilog events. To wait for a SCE-MI Pipes event, a blocking interface uses the following sequence:

1. The `uatba_new_event()` function is called to allocate an event and set the returned `event_id` as the *notifiy_context*.
2. The `uatba_trigger()` function is set as the `scemi_pipe_notify_callback` function.
3. The `uatba_wait_event(event_id)` function is called to suspend its execution and wait for the callback notification event.

Reset of SCE-MI Pipes

In the software-simulation and simulation-acceleration environments, at times you might need to reset the simulator, for example when you have encountered many errors and need to rerun a test. Also, you might need to reset only the SCE-MI pipes, but not reset the simulator when errors like protocol errors are detected. Correspondingly, there are two kinds of SCE-MI Reset in SCE-MI pipes-based software-simulation and simulation-acceleration environments:

■ Simulator reset

You issue the simulator-reset manually by using the `reset` command. With the execution of the `reset` command, the hardware modules are re-instantiated and registers are set to known values, the SCE-MI simulator-reset software function is called to clean up and release the SCE-MI resources. When you issue the `run` command on the CLI to rerun the test, the software modules are re-generated or re-constructed on rerun. Therefore, the SCE-MI pipes are re-bound and re-registered by user code.

■ Protocol reset

The protocol-reset is a C function to be called to reset SCE-MI pipes when the protocol errors have been detected. You can call the protocol-reset function, `scemi_reset()` in your code either from the testbench side when protocol errors are detected, or from the DUT-side when a hardware-reset signal is asserted. To

initiate the protocol-reset from the hardware side, you need to call the imported DPI function `scemi_reset()` when the hardware-reset signal is asserted.

In order to support both kinds of reset, the UXE software has provided an extension to the SCE-MI 2.0 and 2.1 versions. This extension provides the required reset capability, while ensuring that the SCE-MI implementation in UXE is fully compatible with the SCE-MI standard.

Note: It is your responsibility to force BFM's attached to input or output pipes to an idle state after the use of `scemi_reset`. A state machine attached to an input pipe could use the `num_elements_valid == 0` to test for a `scemi_reset`; however, an output pipe does not have a similar indicator. Therefore, the proxy model of the output pipe will need to force the BFM to an idle state, or set a flag prior to the `scemi_reset`, which can be tested when the `send()` function returns.

The SCE-MI simulator-reset function is automatically called by the CLI `reset` command. This section focuses on the requirements for SCE-MI protocol-reset function in order to help you understand the functionality and how to use it.

SCE-MI Protocol-Reset Functionality

The SCE-MI protocol-reset feature is available with both SCE-MI 2.0 (`libuatba.so`) and SCE-MI 2.1 (`libscemi21.so`) software libraries that are provided with the UXE software installation.

The SCE-MI protocol-reset function is implemented as a C function that you can call from the testbench or from a DUT module by using an imported DPI call.

```
int scemi_reset()
```

This function always returns 0.

When the `scemi_reset()` function is called, all the SCE-MI pipe registers in the hardware and the variables in the software are initialized to known values. All buffers are emptied, data is discarded, and all blocked functions are terminated.

Note: If the simulator is reset, IES needs to regenerate or reconstruct the software modules at the time of the rerun. Therefore, it cannot be supported with the hardware model interface (HMI) feature because the CVP software modules of the HMI feature are not a part of IES modules and cannot be re-generated or re-constructed by IES. Refer to [Chapter 13, “IXCOM Hardware Model Interface”](#) for details.

Input Pipes

Following blocked functions are terminated:

- ❑ `scemi_pipe_c_send()` returns. Sent data discarded.
- ❑ `scemi_pipe_c_flush()` returns. Flush action aborted.
- ❑ `receive()` returns with `num_elements_valid = 0`.

Output Pipes

Following blocked functions are terminated:

- ❑ `scemi_pipe_c_receive()` returns with `num_elements_valid = 0`.
- ❑ `send()` returns. Sent data discarded.
- ❑ `flush()` returns. Flush action aborted.

Input-Output Pipes

- ❑ All callback registrations are maintained.
- ❑ All autoflush states are maintained.
- ❑ All User Data is maintained.
- ❑ All pending callbacks are discarded.
- ❑ All data buffered in the pipes is discarded.

Using Language-Based Adaptors

The UXE TBA solution supports the following language-based adaptors:

- SCE-MI Pipes Blocking C Interfaces
- SCE-MI Pipes C++ Interface on Simulator Thread
- SCE-MI Pipes SystemC Interface on SystemC Thread
- SCE-MI Pipes SVTB Interface on Simulator Thread
- SCE-MI Pipes Specman Interface on Specman Thread

SCE-MI Pipes Blocking C Interfaces

This interface uses the `uatba_event.sv` function for thread execution control. See [Executing the Simulator Thread](#) on page 626 for more information. The caller C-thread must be a native thread under simulator management. For example, the C thread can be created by calling a DPI import function from HDL-side or calling a DPI export function to `fork-join_none` thread from C.

Blocking input pipe interface:

```
void scemi_pipe_c_send(void *pipe_handle, int num_elements,
                      const svBitVecVal*data, svbit eom );
void scemi_pipe_c_flush(void *pipe_handle );
void uatba_send_ba(void *pipe_handle, int num_bytes,
                  const char *data, int eom);
```

Blocking output pipe interface:

```
void scemi_pipe_c_receive(void *pipe_handle, int num_elements, int
                        *num_elements_valid, svBitVecVal *data, svbit *eom );
void uatba_receive_ba(void *pipe_handle, int num_bytes, int *num_bytes_valid,
                     char *data, int *eom );
```

SCE-MI Pipes C++ Interface on Simulator Thread

The C++ interface provides an object-oriented interface shell on top of the native SCE-MI C interface. This interface also assumes that the caller thread is a simulator-native thread.

```
class tba_c_pipe{
public:
    tba_c_pipe(const char *path_name);
    void set_eom_auto_flush(int enabled ); // for SCE-MI 2.0
    svBit set_eom_auto_flush(svBit enabled );// for SCE-MI 2.1
    int get_bytes_per_element();
    int get_depth();
};

class tba_c_in_pipe : public tba_c_pipe {
public:
    tba_c_in_pipe(const char *path_name);
    int can_send();
    int try_send(int byte_offset, int num_elements,
                const svBitVecVal *data, svBit eom );
    int try_send_ba(int byte_offset, int num_bytes,
                   const char *data, int eom );
    void try_flush();
};
```

UXE User Guide

Implementing SCE-MI Pipes

```
void send(int num_elements, const svBitVecVal *data, svbit eom );
void send_ba(int num_bytes, char *data, int eom);
void flush();
}
class tba_c_out_pipe : public tba_c_pipe {
public:
    tba_c_out_pipe(const char *path);
    int can_receive();
    int try_receive(int byte_offset, int num_elements,
                    svBitVecVal *data, svbit *eom );
    int try_receive_ba(int byte_offset, int num_bytes,
                       char *data, int *eom );
    void receive(int num_elements, int *num_elements_valid,
                 svBitVecVal *data, svbit *eom );
    void receive_ba(int num_bytes, int *num_bytes_valid,
                    char *data, int *eom );
}
```

SCE-MI Pipes SystemC Interface on SystemC Thread

This interface is identical to the C++ interface except for the class name. However, for implementation, this interface uses only the SystemC event classes and wait functions and does not use the event objects from `uatba_event.sv`. This interface library can be used with OSCI-based SystemC verification environment without an HDL simulator. For simulator-based SystemC verification environment, you can use either the C++ interface on simulator thread or this SystemC interface on SystemC thread.

```
class tba_sc_pipe {
    tba_sc_pipe(const char *path_name);
    <Same_as_tba_c_pipe>;
}
class tba_sc_in_pipe : public tba_sc_pipe {
public:
    tba_sc_in_pipe(const char *path_name);
    <Same_of_tba_c_in_pipe>;
}
class tba_sc_out_pipe : public tba_sc_pipe {
public:
    tba_sc_out_pipe(const char *path_name);
    <Same_as_tba_c_outpipe>;
}
```

SCE-MI Pipes SVTB Interface on Simulator Thread

The SystemVerilog Testbench (SVTB) interface to the SCE-MI pipes channel is implemented as the SystemVerilog interface with similar interface functions and tasks as defined in C++.

IES does not support `string` and `open` array in interface task calls. Therefore, two wrapper classes for `path_name` and `byte_array` should be specified in a separate SystemVerilog header file, `uatba_class.svh`.

```
class path_name;
    string s;
    function
        new (string path);
        s = path;
    endfunction
endclass
```

```
class byte_array;
    byte a[];
    function new(int size);
        a = new[size];
    endfunction
endclass
```

The SystemVerilog interfaces are defined in the `uatba_sv.sv` file:

```
`include uatba_class.sv
interface tba_sv_in_pipe;
    parameter PAY_LOAD_MAX_BITS = 256;
    function void connect(path_name pname);
    function void set_eom_auto_flush(int enabled);
    function int get_bytes_per_element();
    function int get_depth();
    function int can_send();
    function int try_send(input int byte_offset, input int num_elements,
                        input bit [PAY_LOAD_MAX_BITS-1:0] data, input bit eom);
    function int try_send_ba(input int byte_offset, input int num_bytes, input
                        byte_array data, input int eom);
    function int try_flush();
    task send(input int num_elements, input bit [PAY_LOAD_MAX_BITS-1:0] data,
            input bit eom);
    task send_ba(input int num_bytes, input byte_array data, input int eom);
    task flush();
endinterface
```

UXE User Guide

Implementing SCE-MI Pipes

```
interface tba_sv_out_pipe;
    parameter PAY_LOAD_MAX_BITS = 256;
    function void connect(path_name pname);
    function void set_eom_auto_flush(int enabled);
    function int get_bytes_per_element();
    function int get_depth();
    function int can_send();
    function int can_receive();
    function int try_receive(input int byte_offset,
                            input int num_elements,
                            output bit [PAY_LOAD_MAX_BITS-1:0] data,
                            output bit eom);
    function int try_receive_ba(input int byte_offset, input int num_bytes,
                               input byte_array data, output int eom);
    task receive(input int num_elements, output int num_elements_valid,
                output bit [PAY_LOAD_MAX_BITS-1:0] data, output bit eom );
    task receive_ba(input int num_bytes, output int num_bytes_valid,
                   input byte_array data, output int eom );
endinterface
```

SCE-MI Pipes Specman Interface on Specman Thread

The Specman interface is implemented as an **e** unit with similar interface functions and tasks as defined in C++. The implementation of this interface is completely based on Specman SN-C interface to call the SCE-MI thread-neutral interface. The Specman interface also uses an external event port to get notification callbacks indirectly. It does not require any special foreign language adaptors.

```
unit tba_in_pipe {
    run() is also { create binding get_unit().full_hdl_path(); }
    tba_notify: in event_port is instance;
    keep tba_notify.hdl_path() == "sn_notify";
    set_eom_auto_flush(enabled: int) is { <implementation> };
    get_bytes_per_element(): int is { <implementation> };
    get_depth(): int is { <implementation> };
    can_send(): int is { <implementation> };
    try_send(byte_offset: int, num_elements: int,
            data: list of bit, eom: bit ):
        int is { <implementation> };
    send(byte_offset: int, num_elements: int,
        data: list of bit, eom: *bit)
        @sys.any is { <implementation> };
    flush() @ sys.any is {};
};
```

UXE User Guide

Implementing SCE-MI Pipes

```
unit tba_out_pipe {
  run() is also { create binding get_unit().full_hdl_path(); }
  tba_notify: in event_port is instance;
  keep tba_notify_hdl_path() == "sn_notify";
  set_eom_auto_flush(enabled: int) is {<implementation> };
  get_bytes_per_element(): int is {<implementation> };
  get_depth(): int is {<implementation> };
  can_receive(): int is {<implementation> };
  try_receive(byte_offset: int, num_elements: int,
              data: list of bit, eom: *bit ): int is {<implementation> };
  receive(num_elements: int, num_elements_valid: *int,
          data: list of bit, eom: *bit)@sys.any is {<implementation> };
};
```

Note: Due to the Specman `event_port` limitation, the `uatba_sn.e` blocking functions add one extra clock delay to the end of the blocking wait operation.

Using Specific Libraries for Different SCE-MI Versions in Specman Designs

The Specman adaptor requires certain text macros to be defined in order for the adaptor to function properly with the SCE-MI 2.0, SCE-MI 2.1, and Fast pipes versions.

For SCE-MI 2.1 version with both regular pipes and Fast pipes, use the following compilation options:

- ❑ The `+define+SCEMI21` option of the `ixcom` and `irun` commands
- ❑ The `-sndefine "SCEMI21=1"` option of the `ixcom` and `irun` commands
- ❑ The `libscemi21.so` and `libscemi21_dc.so` libraries

For SCE-MI 2.0 version, use `libuatba.so` and `libuatba_dc.so` libraries for both regular and Fast pipes.

To instantiate regular pipes, use `scemi_input_pipe` and `scemi_output_pipe` interface names. The regular pipe interfaces are defined in `scemi_input_pipe.vp`, `scemi_output_pipe.vp`, and `scemi_pipe.vp` files.

To instantiate Fast pipes, use `scemi_input_fast_pipe` and `scemi_output_fast_pipe` interface names. The Fast pipe interfaces are defined in `scemi_input_fast_pipe.vp`, `scemi_output_fast_pipe.vp`, and `scemi_fast_pipe.vp` files.

Note: The Fast pipes feature was introduced in SCE-MI 2.1 version. However, it can be used in SCE-MI 2.0 version with UXE also because of the support provided for Deferred mode in UXE for the SCE-MI 2.0 version. So, you can mix regular pipes with Fast pipes,

provided the `+define+SCEMI21` option and `libscemi21*.so` libraries are used for SCE-MI 2.1, and the `libuatba*.so` library without the `+define` option is used for SCE-MI 2.0.

Generating DUT Clocks with SCE-MI Pipes

To avoid race conditions between the testbench and DUT, non-blocking assignments should be used to generate clocks for DUT and blocking assignments should be used to generate clocks for the testbench.

For example, use the following code to generate clocks for the DUT:

```
reg clk = 0;
always #1 clk <= ~clk;
```

Use the following code to generate clocks for the testbench:

```
reg clk = 0;
always #1 clk = ~clk;
```

Compiling and Running Designs with SCE-MI Pipes

The UXE implementation of SCE-MI pipes supports two utilities for compiling and running designs. These utilities are `IXCOM` (invoked using the `ixcom` command) and `irun` (invoked using the `irun` command). Correspondingly, at a high-level you can compile and run designs in two flows:

- Compile and run a design using `IXCOM` for software and hardware.
- Compile the design using `irun`, that is without using `IXCOM`, and run the design in pure software-simulation mode.

The major differences between using `IXCOM` and `irun` are the compile options and the shared libraries used with these two compilers.

- With `irun`, the shared libraries used are:
 - For SCE-MI 2.1 - `libscemi21_dc.so`
 - For SCE-MI 2.0 - `libuatba_dc.so`
- With `ixcom`, the shared libraries used are:
 - For SCE-MI 2.1 - `libscemi21.so`
 - For SCE-MI 2.0 - `libuatba.so`

UXE User Guide

Implementing SCE-MI Pipes

- None of the compilation options that are specific to `ixcom` should be used when using `irun` to compile a design targeted for the pure software-only mode.
- Additionally, to compile test cases using TBA Specman `e` adaptor (as described in [SCE-MI Pipes Specman Interface on Specman Thread](#) on page 633):

Specify the `-sndefine "NO_IXCOM=1"` option with the `irun` command.

- For SA mode only, use the 1X mode for faster simulation. See [Synchronizing the Fastest Design Clock with Emulator FCLK](#) on page 1301 for more information about the implementation of 1X mode in SA mode.

Compiling and Running Designs in 32-bit and 64-bit Modes

This section describes the settings required for compiling and running TBA designs in 32-bit and 64-bit modes.

To compile and run designs for 32-bit mode:

- Add `${AXIS_HOME}/bin` to the `PATH` variable
- Add `${AXIS_HOME}/lib` to the `LD_LIBRARY_PATH` variable

Note: For Linux, set `${AXIS_HOME}` to `${UXE_INSTALL_DIR}/tools.lnx86`.

To compile and run designs for 64-bit mode:

- Add `${AXIS_HOME}/bin/64bit` to the `PATH` variable
- Add `${AXIS_HOME}/lib/64bit` to the `LD_LIBRARY_PATH` variable

Sample Makefiles

Some sample Makefiles are provided in the UXE TBA solution to help you compile and run a design using `IXCOM` and `irun`. These Makefiles can be accessed from the examples available in the `${AXIS_HOME}/gift/tutorial/uatba` directory.

Makefile Targets

The following Makefile targets are used:

- `comp_hw`
- `comp_sw`

UXE User Guide

Implementing SCE-MI Pipes

- `run_hw`
- `run_sw`
- `comp_irun`
- `run_irun`

Makefile Variables

The following variables are used to define your source code and the UXE TBA source code in the sample Makefile:

- `V_SRC` - defines all user `*.v` and `*.sv` modules
- `C_SRC` - defines all user `*.c` and `*.cpp` files
- `VPLIB` - defines all UXE TBA `*.vp` files
- `SVLIB` - defines all UXE TBA `*.sv` files, UXE TBA `*.e` files, user testbench `*.sv` files, and user testbench `*.e` files
- `TBA_SRC` - defines all UXE TBA `*.cpp` and `*.c` files required for the design
- `CLIB` - defines the UXE TBA shared library, `libuatba.so` to be used with `IXCOM`
- `CLIB_DC` - defines the UXE TBA shared library, `libuatba_dc.so` to be used with `irun`

Compiling and Running Designs

- **To compile and run in hardware and software:**

```
make comp_hw
make run_hw
make run_sw
```

- **To compile and run in the software simulation mode only:**

```
make comp_sw run_sw
```

- **To compile and run in the software mode using `irun` without using `IXCOM`:**

```
make comp_irun run_irun
```

Compiling Designs

- To compile a SystemC based testbench **using `ixcom` flow**, first use `vlan` to compile Verilog and VHDL source code. Then, use the `ixcom` command for compilation. Lastly, use the `irun` command to elaborate:

UXE User Guide

Implementing SCE-MI Pipes

```
comp_hw: clean comp_rtl_hw comp_tb
comp_sw: clean comp_rtl_sw comp_tb

comp_rtl_hw:
    vlan -32 +sv $(V_SRC) $(VPLIB)
    ixcom -32 -nccompile +sv -ua -top dut +dut+dut

comp_rtl_sw
    vlan -32 +sv $(V_SRC) $(VPLIB)
    ixcom -32 -nccompile +sv -top dut

comp_tb
    irun -elaborate -TIMESCALE 1ns/1ns \
    -f xc_work/irun.f $(CINC) \
    $ (C_SRC) \
    -sysc -DPI -top sc_main $(SC_SRC) \
    $(CLIB)
```

- To compile a SystemC based testbench for pure software-only mode using *irun*:

```
comp_irun:
    irun -access +rw -sysc +sv -c +define+NO_IXCOM -top sc_main -top \
    dut $(V_SRC) $(VPLIB) $(CINC) $(C_SRC) $(SC_SRC) $(CLIB_DC) -end
```

Note: Some test cases using SystemC testbenches might hang if the `+scAllowSCFromPLI` option is not passed to the `irun` command at run time. Cadence recommends that the `+scAllowSCFromPLI` option is passed to the `irun` command when running a SystemC test case.

Note: To use void functions that do not return a zero as tasks, and call these as DPI tasks, it is necessary to use the `-dpi_void_task` option of the `irun` command to avoid errors by *irun* compiler.

- To compile a non-SystemC based Testbench using **ixcom**, use the `ixcom` command to compile the Verilog modules. Then, use the `irun` command to elaborate the design.

```
comp_hw: clean comp_rtl_hw comp_tb
comp_sw: clean comp_rtl_sw comp_tb

comp_rtl_hw:
    ixcom -32 -nccompile +sv $(V_SRC) $(VPLIB) -ua +dut+dut

comp_rtl_sw:
    ixcom -32 -nccompile +sv $(V_SRC) $(VPLIB)
```

UXE User Guide

Implementing SCE-MI Pipes

```
comp_tb:
    irun -elaborate -TIMESCALE 1ns/1ns \
    -f xc_work/irun.f $(CINC) \
    $(C_SRC) \
    -top uatba_event $(SVLIB) $(TBA_SRC) \
    $(CLIB)
```

- To compile a non-SystemC based Testbench for pure software-only mode **using** *irun*:

```
comp_irun:
    irun -access +rw +define+NO_IXCOM -c +sv $(V_SRC) $(VPLIB) \
    $(SVLIB) \
    $(CINC) $(C_SRC) $(TBA_SRC) $(CLIB_DC)
```

Sample Makefile to Compile a Non-SystemC Testbench for 32-bit Mode

```
V_SRC = top.v
C_SRC = test.cpp

SRC_DIR = ${AXIS_HOME}/etc/tba

VPLIB = ${SRC_DIR}/scemi_input_pipe.vp ${SRC_DIR}/scemi_output_pipe.vp
SVLIB = ${SRC_DIR}/uatba_event.sv

TBA_SRC = ${SRC_DIR}/uatba_scemi.c ${SRC_DIR}/uatba_event.cpp

CLIBDIR = ${AXIS_HOME}/lib
CLIB = $(CLIBDIR)/libuatba.so
CLIB_DC = $(CLIBDIR)/libuatba_dc.so

CINC = -I${AXIS_HOME}/etc/tba

comp_hw: clean comp_rtl_hw comp_tb
comp_sw: clean comp_rtl_sw comp_tb
comp_irun: clean comp_irun_all

comp_rtl_hw:
    ixcom -32 -nccompile +sv $(V_SRC) $(VPLIB) -ua +dut+dut
comp_rtl_sw:
    ixcom -32 -nccompile +sv $(V_SRC) $(VPLIB)
```

UXE User Guide

Implementing SCE-MI Pipes

```
comp_tb:
    irun -elaborate -TIMESCALE 1ns/1ns \
    -f xc_work/irun.f $(CINC) \
    $(C_SRC) \
    -top uatba_event $(SVLIB) $(TBA_SRC) \
    $(CLIB)
comp_irun_all:
    irun -access +rw +define+NO_IxCOM -c +sv $(V_SRC) $(VPLIB) \
    $(SVLIB) \
    $(CINC) $(C_SRC) $(TBA_SRC) $(CLIB_DC)
run_sw:
    irun -R -input sw.run
run_irun:
    irun -R -input sw.run
run_hw:
    irun -R input hw.run
clean:
    rm -rf INCA* WORK AxisWork xc * *.o *.log *.so cds.lib hdl.var .design .key
    .log .xe* CLOCK_SOURCES DBEngine.msg PDB QTDB cellList dbFiles tmp xe.msg
    xeCompile.key
```

Using Makefile to Compile a Non-SystemC Testbench for 64-bit mode

Following are the differences between the 32-bit and 64-bit Makefile:

- The CLIBDIR environment variable is set to the 64-bit directory:
CLIBDIR = \${AXIS_HOME}/lib/64bit
- The ixcom command is replaced with ixcom -64.
- The vlan command is replaced with vlan -64.
- The irun command is replace with irun -64.

Using Makefile to Compile SCE-MI 2.1 Files

To simplify the UXE-TBA compilation flow, certain compilation option files are provided for the different compilation flows. These compilation option files are located in the `/etc/tba` directory and have `.args` as the filename extension.

Following is an example of the Makefile using the compilation option files to compile a basic testcase that contains only one DUT file named `top.v` and one C testbench file named `test.cpp`.

UXE User Guide

Implementing SCE-MI Pipes

```
comp_rtl_hw:
    ixcom -nccompile -F ${AXIS_HOME}/etc/tba/ua.args -ua \
    top.v +dut+dut
comp_tb:
    irun -f ${AXIS_HMOE}/etc/tba/tb.args \
    test.cpp
```

Using the compilation options file eliminates the need for typing out the following options:

```
comp_rtl_hw:
    ixcom -nccompile +sv -ua \
    +define+SCEMI21 \
    ${AXIS_HMOE}/etc/tba/scemi_input_pipe.vp \
    ${AXIS_HMOE}/etc/tba/scemi_output_pipe.vp \
    top.v +dut+dut
comp_tb:
    irun -elaborate \
    -DSCEMI21 \
    -TIMESCALE 1ns/1ns \
    -f xc_work/irun.f \
    -I${AXIS_HOME}/etc/tba \
    test.cpp
    -top uatba_event \
    ${AXIS_HMOE}/etc/tba/uatba_event.cpp
    ${AXIS_HMOE}/etc/tba/uatba_event.sv
    ${AXIS_HOME}/lib/${BITS}bit/libscemi21.so
```

The sample Makefile files for different language-based testcases using SCE-MI 2.1 are available in the `${UXE_INST_DIR}/gift/tutorial/uatba/scemi21` and `${UXE_INST_DIR}/gift/tutorial/uatba/scemi21_hmpi` directories. As evident from the Makefiles in these examples, these are identical except that the compilation option file used for compiling the testbench is different. To disable compilation for hardware, you can remove the `-ua` option from the original `ixcom` or `irun` command.

Usage of the Compilation Option Files

The names of the compilation option files and their usages are summarized in the following tables.

Table 12-2 Compilation Option Files for IXCOM Flow

Filename	Used with
ua.args	ixcom command to compile DUT files
ixcom.args	irun command to invoke ixcom command for compiling the DUT files
vlan.args	irun command to invoke vlan command for compiling the mixed-language DUT files
tb.args	irun command to compile testbench files using C/C++/SC and elaborate the design
tb-sv.args	irun command to compile testbench files using SystemVerilog and elaborate the design
tb-e.args	irun command to compile testbench files using Specman e and elaborate the design
tb-hmpi.args	irun command to compile the HMPI testbench files and elaborate the design

Table 12-3 Compilation Option Files for Software Simulation Flow Using the irun Command

Filename	Used with
ncsim.args	irun command to compile C/C++/SC based testbench files and all DUT files
ncsim-sv.args	irun command to compile SystemVerilog based testbench files and all DUT files
ncsim-e.args	irun command to compile Specman e based testbench files and all DUT files

Examples: Implementing SCE-MI Pipes

To illustrate the implementation of the SCE-MI Pipes, the following examples are provided in the UXE installation directory.

- ❑ [SCE-MI 2.0 Examples](#)
- ❑ [SCE-MI 2.1 Examples](#)

SCE-MI 2.0 Examples

- ❑ [SystemC Testbench Using C Interface Examples](#)
- ❑ [C++ Testbench Using C Interface Examples](#)
- ❑ [C Testbench Using C Interface Examples](#)
- ❑ [SystemC Testbench Using SystemC Interface Examples](#)
- ❑ [C++ Testbench Using C++ Interface Examples](#)
- ❑ [SystemVerilog Testbench Using SVTB Interface Examples](#)
- ❑ [Specman e Testbench Using Specman e Adaptor Interface Examples](#)
- ❑ [HMPI Examples: C, OSCI SystemC, and SystemVerilog Testbenches Using HMPI Thread](#)

SystemC Testbench Using C Interface Examples

- Blocking Interface of Input Pipe with SystemC Testbench
`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/sysc/
input_pipe_blk_dpi_sysc`
- Non-blocking Interface of Input Pipe with SystemC Testbench
`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/sysc/
input_pipe_nb_dpi_sysc`
- Blocking Interface of Output Pipe with SystemC Testbench
`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/sysc/
output_pipe_blk_dpi_sysc`
- Non-blocking Interface of the Output Pipe with SystemC Testbench
`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/sysc/
output_pipe_nb_dpi_sysc`
- Blocking Interface of Input Pipe with SystemC Testbench and VPI Functions

UXE User Guide

Implementing SCE-MI Pipes

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/sysc/  
input_pipe_blk_export_vpi_sysc
```

■ marg API Functions with SystemC Testbench for Input Pipe

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/sysc/  
input_marg_export_sysc
```

■ marg API Functions with SystemC Testbench for Output Pipe

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/sysc/  
output_marg_export_sysc
```

The above examples use the following components:

Pipes

SCE-MI Input pipe, blocking interface	input_pipe_blk_dpi_sysc
SCE-MI Input pipe, non-blocking interface	input_pipe_nb_dpi_sysc
SCE-MI output pipe, blocking interface	output_pipe_blk_dpi_sysc
SCE-MI output pipe, non-blocking interface	output_pipe_nb_dpi_sysc

DPI

Import DPI	input_pipe_blk_dpi_sysc
Export DPI	input_pipe_blk_dpi_sysc

Cross module variable access API

export_read, export_deposit	input_pipe_blk_export_vpi_sysc
marg C-interface functions	input_marg_export_sysc, output_marg_export_sysc

C++ Testbench Using C Interface Examples

■ Non-blocking Interface of Input Pipe with C++ Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/cpp/  
input_pipe_nb_dpi_cpp
```

■ Blocking Interface of Input Pipe with C++ Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/cpp/  
input_pipe_blk_dpi_cpp
```

■ Non-blocking Interface of the Output Pipe with C++ Testbench with DPI

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/cpp/  
output_pipe_nb_dpi_cpp
```


UXE User Guide

Implementing SCE-MI Pipes

■ Blocking Interface of the Output Pipe with C++ Testbench with DPI

`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/cpp/
output_pipe_blk_dpi_cpp`

■ marg API Functions with C++ Testbench for Input Pipe

`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/cpp/
input_marg_export_cpp`

■ marg API Functions with C++ Testbench for Output Pipe

`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/cpp/
output_marg_export_cpp`

■ Blocking Interface of Input Pipe with C++ Testbench and VPI Functions

`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/cpp/
input_pipe_blk_vpi_cpp`

The above examples use the following components:

Pipes

SCE-MI Input pipe, blocking interface	<code>input_pipe_blk_dpi_cpp</code>
SCE-MI Input pipe, non-blocking interface	<code>input_pipe_nb_dpi_cpp</code>
SCE-MI output pipe, blocking interface	<code>output_pipe_blk_dpi_cpp</code>
SCE-MI output pipe, non-blocking interface	<code>output_pipe_nb_dpi_cpp</code>

DPI

Import DPI	<code>input_pipe_blk_dpi_cpp</code>
Export DPI	<code>input_pipe_blk_dpi_cpp</code>

Cross module variable access API

<code>export_read</code> , <code>export_deposit</code>	<code>input_pipe_blk_export_vpi_cpp</code>
marg C-interface functions	<code>input_marg_export_cpp</code> , <code>output_marg_export_cpp</code>

C Testbench Using C Interface Examples

■ Non-blocking Interface of Input and Output Pipe with C Testbench

`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/c/
input_output_pipe_nb_c`

■ Blocking Interface of Input and Output Pipe with C Testbench

UXE User Guide

Implementing SCE-MI Pipes

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_c/c/  
input_output_pipe_blk_c
```

The above examples use the following components:

Pipes

SCE-MI Input pipe and Output_pipe blocking interface `input_output_pipe_blk_c`

SCE-MI Input pipe and Output_pipe non-blocking interface `input_output_pipe_nb_c`

SystemC Testbench Using SystemC Interface Examples

■ Non-blocking Interface of Input and Output Pipe with SystemC Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_sc/  
input_output_pipe_nb_sysc
```

■ Blocking Interface of Input and Output Pipe with SystemC Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_sc/  
input_output_pipe_blk_sysc
```

The above examples use the following components:

Pipes

SCE-MI Input pipe and Output_pipe blocking interface `input_output_pipe_blk_sysc`

SCE-MI Input pipe and Output_pipe non-blocking interface `input_output_pipe_nb_sysc`

C++ Testbench Using C++ Interface Examples

■ Non-blocking Interface of Input and Output Pipe with C++ Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_cpp/  
input_output_pipe_nb_cpp
```

■ Blocking Interface of Input and Output Pipe with C++ Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_cpp/  
input_output_pipe_blk_cpp
```

The above examples use the following components:

Pipes

SCE-MI Input pipe and Output_pipe blocking interface `input_output_pipe_blk_cpp`

SCE-MI Input pipe and Output_pipe non-blocking interface `input_output_pipe_nb_cpp`

SystemVerilog Testbench Using SVTB Interface Examples

- Non-blocking Interface of Input and Output Pipe with SystemVerilog Testbench

`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_sv/
input_output_pipe_nb_sv`

- Blocking Interface of Input and Output Pipe with SystemVerilog Testbench

`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_sv/
input_output_pipe_blk_sv`

The above examples use the following components:

Pipes

SCE-MI Input pipe and Output_pipe blocking interface `input_output_pipe_blk_sv`

SCE-MI Input pipe and Output_pipe non-blocking interface `input_output_pipe_nb_sv`

Specman e Testbench Using Specman e Adaptor Interface Examples

- Blocking Interface of Input and Output Pipe with Specman **e** Testbench

`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_e/
input_output_pipe_blk_e`

- Non-blocking Interface of Input and Output Pipe with Specman **e** Testbench

`${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/uatba_e/
input_output_pipe_nb_e`

The above examples use the following components:

Pipes

UXE User Guide

Implementing SCE-MI Pipes

SCE-MI Input pipe and Output_pipe blocking interface `input_output_pipe_blk_e`

SCE-MI Input pipe and Output_pipe non-blocking interface `input_output_pipe_nb_e`

HMPI Examples: C, OSCI SystemC, and SystemVerilog Testbenches Using HMPI Thread

- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/hmpi/hm_c_pt`
- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/hmpi/hm_c_sv`
- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/hmpi/hm_concurrent_pt`
- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/hmpi/hm_osci_sc`
- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/hmpi/hm_osci_scemi_1_2`
- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/hmpi/hm_sv_sv`
- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/hmpi/hm_sv_sv_no_clk`
- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/hmpi/hm_sv_sv_schd`

SCE-MI 2.1 Examples

- ❑ [C Testbench Using C Interface Examples - SCE-MI 2.1](#)
- ❑ [C++ Testbench Using C++ Interface Examples - SCE-MI 2.1](#)
- ❑ [SystemVerilog Testbench Using SVTB Interface Examples - SCE-MI 2.1](#)
- ❑ [SystemC Testbench Using SystemC Interface Examples - SCE-MI 2.1](#)
- ❑ [Specman e Testbench Using Specman e Adaptor Interface Examples - SCE-MI 2.1](#)
- ❑ [HMPI Examples: C, OSCI SystemC, and SystemVerilog Testbenches Using HMPI Thread - SCE-MI 2.1](#)

C Testbench Using C Interface Examples - SCE-MI 2.1

- Non-blocking Interface of Input and Output Pipe with C Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21/  
input_output_pipe_nb_c
```

- Blocking Interface of Input and Output Pipe with C Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21/  
input_output_pipe_blk_c
```

C++ Testbench Using C++ Interface Examples - SCE-MI 2.1

- Non-blocking Interface of Input and Output Pipe with C++ Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21/  
input_output_pipe_nb_cpp
```

- Blocking Interface of Input and Output Pipe with C++ Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21/  
input_output_pipe_blk_cpp
```

SystemVerilog Testbench Using SVTB Interface Examples - SCE-MI 2.1

- Non-blocking Interface of Input and Output Pipe with SystemVerilog Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21/  
input_output_pipe_nb_sv
```

- Blocking Interface of Input and Output Pipe with SystemVerilog Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21/  
input_output_pipe_blk_sv
```

SystemC Testbench Using SystemC Interface Examples - SCE-MI 2.1

- Non-blocking Interface of Input and Output Pipe with SystemC Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21/  
input_output_pipe_nb_sysc
```

- Blocking Interface of Input and Output Pipe with SystemC Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21/  
input_output_pipe_blk_sysc
```

Specman e Testbench Using Specman e Adaptor Interface Examples - SCE-MI 2.1

- Blocking Interface of Input and Output Pipe with Specman **e** Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21/  
input_output_pipe_blk_e
```

UXE User Guide

Implementing SCE-MI Pipes

■ Non-blocking Interface of Input and Output Pipe with Specman **e** Testbench

```
${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21/  
input_output_pipe_nb_e
```

HMPI Examples: C, OSCI SystemC, and SystemVerilog Testbenches Using HMPI Thread - SCE-MI 2.1

- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21_hmpi/
hm_c_pt`
- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21_hmpi/
hm_c_sv`
- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21_hmpi/
hm_osci_sc`
- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21_hmpi/
hm_sv_sv_no_clk`
- `${UXE_INSTALL_DIR}/share/uxe/gift/tutorial/uatba/scemi21_hmpi/
hm_sv_vpi_sched`

IXCOM Hardware Model Interface

IXCOM provides a Hardware Model Interface to connect a standalone verification process with the IES process controlling the emulator. In this mode of operation, the IES process is called the Hardware Model Server (HMS) and the standalone verification process is called the Client Verification Process (CVP).

In most common cases, the CVP is an OSCI SystemC process. In other cases, the CVP can also be a third-party HVL simulator or a custom pthread-based simulator.

Communication through the Hardware Model Interface occurs at the following levels:

- Linux Process Level
- User Application Level

Linux Process Level

At the Linux process level, the inter-process communication (IPC) between the HMS and CVP happens through a proprietary shared memory technology. For this communication to happen seamlessly and at the required performance, the following conditions are required:

- The HMS and CVP processes must run on the same host computer that is connected to an emulator.
- The host computer must have at least dual cores to run both HMS and CVP concurrently in real time to achieve top emulation performance.
- A quiet host machine dedicated to HMS and CVP execution is desired for best performance.

Note: The IPC will still be functional with a single core or a busy host computer. However, the performance might suffer depending on the traffic pattern between the CVP and HMS processes.

User Application Level

At the application level, the basic programming interface between CVP and HMS is SystemVerilog DPI-C (Direct-C Programming Interface). Using the IXCOM compiler directives, you can select the following:

- The import DPI tasks and functions to be executed on the CVP side
- The export DPI tasks (functions not allowed) to be called from the CVP side

These selected DPI tasks and functions define the Hardware Model Programming Interface (HMPI).

IXCOM supports execution of a mix of local DPI and remote HMPI tasks and functions. DPI tasks and functions can be executed in the HMS process if they are not selected to be HMPI tasks and functions.

The IXCOM “*marg*” interface enables CVP to directly access the HMS memory - the DUT memory objects in the emulator. This is the recommended method for passing large data objects through HMPI calls.

On top of the HMPI tasks and functions, the SCE-MI Macro-based and Pipes-based interfaces are built for the CVP applications that can take advantage of these standard co-emulation interfaces.

At the application level, the CVP application uses one function to initialize the hardware model and another function to control its execution. The Hardware Model Interface supports both of the following execution models for synchronization:

- ❑ The Alternating execution model in which CVP and HMS synchronize in simulation time
- ❑ The Concurrent execution model in which CVP and HMS run in their own simulation time

Selecting HMPI Tasks and Functions for IXCOM Compilation

There are two ways to map DPI-C tasks and functions into HMPI tasks and functions.

- ❑ Using the `+hmpi` option of the `ixcom` command:

The `+hmpi` option enables selection of all DPI tasks and functions in DUT to be HMPI task and functions.

The `+hmpi[+<mod_name>]*` option enables selection of the DPI tasks and functions in the specified module(s) to be HMPI tasks and functions.

- ❑ Including `$ixc_ctrl("hmpi", "<task_or_function_name>" [, "tb_mem"])` compiler directives in the source code to select a specific DPI task or function to be a HMPI task or function. By default, it uses primary I/O resource to transfer input and output. With `tb_mem`, it uses memory to do I/O transfer.

Note: The DPI tasks and functions that are not mapped to HMPI tasks and functions will remain on the HMS side.

Building the HMPI Adaptor for CVP

After IXCOM compile, you need to build a custom HMPI shared library using one of the supported thread systems and link this shared library to the CVP executable. This shared library is called the HMPI adaptor. The HMPI adaptor is tied to the thread system of the CVP process.

To build a specific adaptor, you need to run the `make` executable in the IXCOM build directory, which is created after the execution of the `ixcom compile` command. IXCOM supports solutions to the following three standard thread systems:

- OSCI SystemC adaptor: `libhmsc.so`
- Linux pthread adaptor: `libhmpt.so`
- Third-Party SystemVerilog adaptor: `libhmsv.so`, `libhmsv.sv`

Following are the commands for building the HMPI adaptor using these thread systems:

OSCI SystemC Adaptor

```
setenv HMSC_INCL_DIR <path_to_OSCI_include>
make -f xc_work/xc_make hmsc
```

Or

```
make -f xc_work/xc_make HMSC_INCL_DIR=<path> hmsc
```

Linux pthread Adaptor

```
make -f xc_work/xc_make hmpt
```

Third-Party SystemVerilog Adaptor

```
setenv HMSV_INCL_DIR <path_to_third_party_SV_DPI_include>
make -f xc_work/xc_make hmsv
```

Or

```
make -f xc_work/xc_make HMSV_INCL_DIR=<path> hmsv
```

After the make, the adaptor shared library is built in the following location:

```
xc_work/c/<adaptor.so>
```

The third-party SystemVerilog adaptor has both a shared library and a SystemVerilog source file in:

```
xc_work/v/libhmsv.sv
```

The SystemVerilog source file needs to be compiled and elaborated to build the CVP.

CVP Run-Time Control of HMS

Before simulation, run the following script to set up the environment variables: XC_WORK_DIR and XC_HM_PORT. The HMPI adaptor needs to use the XC_WORK_DIR to find the design database and to use the XC_HM_PORT number to connect the IPC channel.

```
setenv XC_WORK_DIR <path_to_xc_work>
setenv XC_HM_PORT <number>
```

1. Start the HMS in the background.
2. Start the CVP.

After the script is run, the file /tmp/ipc_\${USER}_\${XC_HM_PORT} is created to initialize the IPC channel and it is removed by HMS after the CVP finishes execution. If the CVP or HMS crashed during the previous run, this file cannot be deleted from the /tmp directory, and it would prevent the initialization of the next run. Therefore, it is a good practice to clean up the /tmp/ipc_\${USER}_* area before beginning the next run.

Sharing the Hardware Model in Concurrent Tests

For hardware model applications, Cadence recommends that the hardware model is compiled in a different directory from the one in which CVP is run. By doing so, you can share the same hardware model in multiple concurrent tests. For such an environment, use a setup similar to the one suggested below:

```
setenv XC_WORK_DIR <path_to_xc_work>
# Set port number to nano second time stamp
```

UXE User Guide

IXCOM Hardware Model Interface

```
setenv XC_HM_PORT `date +%N`
# start the HMS from pwd
irun -R -xeDebug +nclibdirpath+${XC_WORK_DIR} -input xct -nostdout &
# start the CVP
cpv_executable
```

Hardware Model Initialization

The CVP must first call `hm_initialize()` to connect to HMS before starting any threads that make any DPI/SCE-MI calls to the HMS. For example, `hm_initialize()` must be called before `sc_start()` in the OSCI SystemC simulation.

- ❑ C/C++/SystemC:
`extern "C" void hm_initialize();`
- ❑ SystemVerilog:
`hm_top.hm_initialize();`

When `hm_initialize()` returns, the CVP is connected with HMS. Connection to the CVP from HMS is completed at the end of time (EOT) of simulation time 0.

You can start the CVP and HMS in any order for a Linux shell.

After establishing the IPC connection, HMS can run in either the software simulation mode or the hardware acceleration mode with hot-swap controls. All the IXCOM `xc` emulation execution control commands, *irun* simulation control commands, and the `xeDebug` emulation debugging commands are supported, as is, in HMS. HMS is just an instrumented *irun* that can take events both locally within, and remotely from CVP, to control the execution of the emulator.

Hardware Model Execution Control

Use the following threads to control the execution on the HMS side.

- C/C++ pthread:
`typedef unsigned long long ullong;`
`ullong hm_run(ullong hmTargetT, void (*cvpWaitFunc)(ulong T));`
`ullong hm_run_nblk(ullong hmTargetT, void (*monWait)());`
- OSCI SystemC:
`ulong hm_run(ullong hmTargetT);`
`ulong hm_run_nblk(ullong hmTargetT, void (*monWait)());`
- SystemVerilog:

```
module hm_top;
    task run(input longint hmTargetT, output longint hmTime);
        class waitC; virtual task do_wait; endclass
    task run_nblk(input longint hmTargetT,
        input waitC monWait,
        output longint hmTime);
endmodule
```

Synchronization between CVP and HMS

The CVP and HMS follow their own separate simulation times. Some applications require strict synchronization of the simulation time; other applications allow the two simulation times to run independent (concurrent) of each other.

Strict Synchronization Execution Model

In the strict synchronization model, it is very difficult to run two independent time schedulers and synchronize. For practical applications, one of the time schedulers can run as the master and the other scheduler can be the follower. The follower schedules its next timing events with the master scheduler.

The design clocks run in the emulator. Therefore, it would be too expensive to make CVP as the master scheduler. It is only practical to make HMS the master time scheduler and CVP the follower.

The `hm_run()` blocking execution control function is provided for this execution model. A call to this function runs HMS up to the specified target simulation time (`hmTargetT`) or when any HMPI event occurs in HMS. After this function returns, the CVP simulation time is advanced to the HMS current time. For simplicity, we require that both CVP and HMS are run in the same timescale.

```
hm_run(hmTarget) {
    Call HMS with CVP events to run to hmTargetT or HMS events.
    Wait for HMS return and get the current hmTime;
    wait(hm_time_unit*hmTime); // advance CVP time
    If(any HMPI event from HMS){
        Process HMS events;
    }
    wait(SC_ZERO_TIME);
    return hmTime;
}
```

For the OSCI system C and third-party SystemVerilog thread systems, a well-defined method is provided to advance the CVP simulation time by using the `wait()` function of the language. On the other hand, for the pthread-based C/C++ system, the simulation time is custom-designed and you provide the `wait()` function.

You need to write a dedicated scheduling thread in CVP for control the execution of HMS in strict-synchronization mode. Following is the standard code template for such a scheduler thread:

```
SC_THREAD(hm_scheduler);
void hm_scheduler() {
    while(1) {
        sc_wait(SC_ZERO_TIME);
        Tnext = <find next time event>;
        Tnow = hm_run(T_next);
    }
}
```

Calling the `hm_run()` function blocks the entire CVP process until the HMS ends. The strict-synchronization execution model demands that HMS and CVP should be executed alternately.

There is no (standard) way for getting the next simulation time event in CVP. But many practical application solutions are available for you, such as the following:

- In most cases, CVP is completely driven by the DUT events after initialization and does not use any “wait for time” to synchronize with the DUT. For these cases, “Tnext” should be set to the maximum possible simulation time (-1).
- For cases that do use delay to synchronize with HMS, a user-level `wait()` function can be written to collect and sequence all the active wait times into an HMS next-time queue for use by the scheduler thread. For example, instead of direct calling `sc_wait(sc_time& t)`, a user-level `wait()` function can be written as follows:

```
void cvp_wait(sc_time &t) {
    Tnow = sc_time_stamp().value();
    Tdelay = t.value();
    Push (Tnow+Tdelay) to the cvp_next_time_Queue;
    sc_wait(t);
}

ulong cvp_next_time_event() {
    Tnext = pop the smallest value from cvp_next_time_Queue;
    return Tnext;
}
```

Concurrent execution model

In this mode of execution, CVP uses a dedicated thread to monitor HMPI events from HMS and CVP. If an HMPI event is detected, HMS is stopped first, and then the event is processed before the system continues.

To support this execution model, the `hm_run_nblk(hmTargetT, monWait)` non-blocking execution control function is provided. A call to this function runs HMS up to the given target simulation time (`targetT`) or up to the HMPI event from both HMS and CVP. This function does the following:

```
hm_run_nblk(hmTargetT, void(*monWait)()) {  
    Call HMS with CVP events to run to hmTeagetT or HMS events.  
    while(1) {  
        Call (*monWiat)(); //advance time before check again.  
        Check any HMPI events;  
        If(any_event) {  
            If(any CVP events) stop HMS;  
            If(any HMS events) process HMS events;  
            Return HMS_time;  
        }  
    }  
}
```

This task returns when an HMPI event is detected and returns the current HMS time.

You can use `monWait()` function to poll the HMPI events. This function can be as basic as a `wait()` for a constant delay:

```
monWait() {  
    wait(sc_time(2, NS));  
}
```

For more sophisticated applications, the `wait()` function can be based on time delays and the CVP events that need the immediate attention of HMS.

In SystemVerilog, a task cannot be passed as an argument to another task. So, a class with a virtual task is used instead to achieve the same purpose. For example, to create a constant delay class for SystemVerilog, use the following code:

```
class waitT extends waitC;  
    task do_wait; #2 endtask  
endclass
```

To use this execution model, you need to write dedicated scheduler threads to monitor the HMPI events and to control the execution of the hardware model. Following is the standard code template for such a monitor thread:

```
SC_THREAD(hm_scheduler);  
void hm_scheduler() {  
    while(1) {  
        hmTime = hm_run_nblk(hmTargetT, CVPMonT);  
    }  
}
```

In most cases, the CVP is completely driven by the DUT events after initialization without using any “wait for time” construct to synchronize with the DUT. The `hmTargetT` parameter can be set at the maximum simulation time (-1) and the basic constant delay function can be used as described.

CVP Programming Notes for Pthread Adaptor

To build CVP with HM pthread adaptor, please include the following components:

■ **Header file:**

```
${UXE_HOME}/etc/ixcom/hm_top_pt.h
```

■ **Adaptor lib:**

```
${XC_WORK_DIR}/xc_work/c/libhmpt.so
```

The emulator system is a shared resource. To get predictable results, all HMPI events must be processed sequentially. The thread systems of SystemVerilog and OSCI SystemC are synchronous cooperating threads. This ensures that the HMPI events are processed sequentially. On the other hand, the pthread system is an asynchronous concurrent thread system. To get predicable results, the pthread adaptor uses a mutex to serialize the execution of HMPI events. The pthread adaptor also provides extra interface functions to control the HMPI mutex.

Extra Interfaces

```
void hm_sync_on();  
void hm_sync_off();  
pthread_t hm_pthread_create( void (*f)(void *), void* context);  
void hm_initialize(int import_dpi_mode=0);
```

To better understand the working of HMPI mutex, consider the two special functions that control the locking and unlocking of the HMPI mutex:

- `hm_sync_on()`: Locks the HMPI mutex.
- `hm_sync_off()`: Unlocks the HMPI mutex.

These functions can be nested called without incurring a deadlock.

The pthread adaptor uses these functions internally for all the HMS interface functions that generate and consume HMPI events:

- Export DPI tasks calls
- SCE-MI send and receive calls
- SCE-MI service loop call
- `hm_run()`;

The pthread adaptor will call `hm_sync_on()` at the start of these interface function calls and `hm_sync_off()` at the end of these interface functions. Within these interface functions, the HMPI mutex will be unlocked when waiting for an HMS event and relocked when coming out of the HMS event.

The HMPI mutex control function is exposed to reduce the cost of frequently locking and unlocking the mutex.

For example, suppose CVP calls a DPI task in a loop:

```
for(int i = 0; i < 1000; i++){  
    dpi_export_task_call(i);  
}
```

This loop will lock and unlock the HMPI mutex 1000 times at the entry and exit of the interface function to the exported Verilog task. Also, it will unlock and lock the HMPI mutex 1000 times when waiting for HMS to execute the Verilog task. Consider a basic code change:

```
hm_sync_on();  
for(int i = 0; i < 1000; i++){  
    dpi_export_task_call(i);  
}  
hm_sync_off();
```

The HMPI mutex is now locked/unlocked for the entire loop. This eliminates all the locking and unlocking overheads of calling the DPI interface function. Overall, this reduces the HMPI mutex locking and unlocking from 2000 to 1002, which can be significant for some tight loops, as above.

If a pthread is created for controlling the HMS and does not do any wait on conditional variables, the thread can be created with `hm_sync_on()` and there is a special function `hm_pthread_create()` for such a purpose.

The pthread adaptor automatically locks the HMPI mutex before calling a DPI import function because a DPI import function is not allowed to do any wait operations.

The pthread adaptor creates a pthread for each DPI import task call based on the assumption that the import task is time consuming and does wait operations. A special argument, `import_dpi_mode`, is used to control the HMPI mutex locking when creating DPI import task pthread. Following are the valid values for this argument:

- `import_dpi_mode = 0`: Create DPI import task pthread with the HMPI mutex locked.
- `import_dpi_mode = 1`: Create DPI import task pthread without locking HMPI mutex.

By default, a DPI import task pthread is created with HMPI mutex locked to match the DPI semantics. Based on the SystemVerilog standard, the only legal, time-consuming operation of a DPI import task is that of calling DPI export tasks. Also, the import DPI tasks should be executed sequentially. For applications that go beyond the scope defined in the SystemVerilog standard, `import_dpi_mode` needs to be configured to 1 to execute the import DPI task in parallel and use other pthread time-consuming wait operations.

CVP Programming Notes for OSCI SystemC Adaptor

To build CVP with HM OSCI-SC adaptor, include the following components:

- Header file:
`${UXE_HOME}/etc/ixcom/hm_top_sc.h`
- Adaptor Lib:
`${XC_WORK_DIR}/xc_work/c/libhmssc.so`

Notice that the adaptor `libhmssc.so` is for OSCI System C use only. You should not use it with NCSC or any third-party SystemC solution integrated with an HDL simulator. To build a SystemC CVP into an HDL simulator, use the SystemVerilog adaptor, not the OSCI System C adaptor. The `libhmssc.so` has DPI `svScope` related utility functions defined and will conflict with HDL simulators' definitions.

CVP Programming Notes for OSCI SystemVerilog Adaptor

To build CVP with SV adaptor, please include the following components:

- Adaptor SV file:
`${XC_WORK_DIR}/xc_work/v/libhmssv.sv`
- Adaptor DPI lib:
`${XC_WORK_DIR}/xc_work/c/libhmssv.so`

UXE User Guide

IXCOM Hardware Model Interface

In both the OSCI SystemC and pthread adaptors, we provide the SystemVerilog DPI scope utility functions such as `svGetScope` because the CVP is not a SystemVerilog simulator. However, we cannot redefine these scope functions in the SystemVerilog adaptor for the third-party SystemVerilog simulator. Instead, we provide the DUT module instance tree skeleton in the adaptor (`libhmsv.sv`) so that all the DUT scopes will be visible in the CVP SystemVerilog simulator to make all the scope functions work.

If the third-party SystemVerilog elaborator requires explicit specification of the top modules, you must include the DUT top in the elaboration process.

The SV adaptor uses scope translation to automatically translate `svScope` from SystemVerilog simulator into scopes in HMS.

To call the HM scheduler, you should use `hm_top.hm_init_with_schd()` instead of using `hm_top.hm_run()`. Also, the `hm_top.hm_init_with_schd()` scheduler automatically calls `hm_top.hm_initialize()`, if it is not already called. When compiling the CVP, your SystemVerilog testbench can just include the following:

```
initial hm_top.hm_init_with_schd();
```

There are two run-time execution modes: Alternate and Concurrent, when you use the `hm_init_with_schd()` scheduler, and the Alternate mode is the default mode of execution. The CVP and HMS process take turns in execution. Only one process is active at a time. This is similar to the TBrn mode when the hardware model interface is not used.

You can activate Concurrent mode by invoking the CVP process by using the `+hmschd=` option of the `irun` command. Both CVP and HMS are active at the same time, and the communication between these occurs periodically. This is similar to the NBrn mode when the hardware model interface is not used. Specify the synchronization intervals by providing a testbench (CVP) delay with the `+hmschd=` option. For example, `+hmschd=100ns`.

The DPI-C interface is defined between SystemVerilog and C. Therefore, you need to add a C-bridge to forward calls between two SystemVerilog systems running on the HMS and CVP. Without a good methodology, it is possible to get confused and make mistakes in adding the C-bridge.

The following methodology for SystemVerilog process/thread calling remote SystemVerilog task/functions through the C-bridge is suggested:

- Declare export DPI-C for all tasks to be called by the other side.
- Declare import DPI-C for all remote tasks to be called locally using `<name>_c` as the task name.
- On the testbench side, define a SV interface as the proxy for a DUT module.

- Within the SV interface, define an import DPI-C function to create a scope binding to a specific DUT instance.
- Define the C-bridge that does the call forwarding.

For example, we have a DUT module that exports its `dutTask` and calls a remote `tbTask` using the name `tbTask_c`;

```
module Dut;
    import "DPI-C" context task tbTask_c(<params>);
    export "DPI-C" task dutTask;
    task dutTask(<params>) begin <body> end endtask
endmodule
```

We have a testbench module that exports its `tbTask` and calls a remote `dutTask` using the name `dutTask_c`;

```
interface Dut_If;
    import "DPI-C" void function bindDut_c(string path);
    import "DPI-C" context task dutTask_c(<params>);
    export "DPI-C" task tbTask;
    task tbTask(<params>) begin <body> end endtask
endinterface
module Tb;
    Dut_If dut();
    initial
        dut.bindDut_c("<path_to_dut>");
        call remote task by dut.dutTask_c(<args>);
endmodule
```

The C-bridge code to connect the two is the following:

```
extern "C" void bindDut_c(char *dutPath){
    svScope tbScp = svGetScope();
    svScope dutScp = svGetScopeFromName(dutPath);
    svPutUserData(tbScp, (void *) &bindDut_c, dutScp);
    svPutUserData(dutScp, (void *) &bindDut_c, tbScp);
}
extern "C" int tbTask(<params>);
extern "C" int tbTask_c(<params>){
    svScope dutScp = svGetScope();
    svScope tbScp = svGetUserData(dutScp, (void *) &bindDut_c);
    svSetScope(tbScp);
    tbTask(<params>);
    return 0;
}
```

```
}  
extern "C" int dutTask(<params>);  
extern "C" int dutTask_c(<params>){  
    svScope tbScp = svGetScope();  
    svScope dutScp = svGetUserData(tbScp, (void *) &bindDut_c);  
    svSetScope(dutScp);  
    dutTask(<params>);  
    return 0;  
}
```

SCE-MI Macro with HMPI

For the DUT side, there are two SystemVerilog files for the HDL-side interface:

- `${UXE_REL}/etc/tba/SceMiHmpi.sv`
- `${UXE_REL}/etc/tba/ScemiHmpiLib.sv`

If you do a single-step IXCOM compile to build HMS, use the following script.

```
ixcom <other_sources_and_options> \  
    +dut+SceMi \  
    ${UXE_REL}/etc/tba/SceMiHmpi.sv \  
    -v ${UXE_REL}/etc/tba/ScemiHmpiLib.sv
```

For compiling mixed-language designs using IXCOM to build HMS, use the following script.

```
vlan ${UXE_REL}/etc/tba/SceMiHmpi.sv \  
    ${UXE_REL}/etc/tba/ScemiHmpiLib.sv  
ixcom <other options> \  
    -top SceMi +dut+SceMi
```

To build the CVP side, you need to include the standard SCE-MI header file and the shared library.

- SCE-MI header:
`${UXE_REL}/etc/tba/scemi.h`
- SCE-MI library:
`${UXE_REL}/lib/<32|64>bit/libscemihmpi.so`

The semantics of SCE-MI macro is concurrent by definition, and the HM scheduler thread only needs to call `ServiceLoop`. The entire execution control is embedded in `ServiceLoop`. You should not call `hm_run()` or `hm_run_nblk()`.

The following is the scheduler thread code template with SCE-MI Service loop included.

```
void hm_scheduler()
{
    while(!termical condition) {
        wait(1, SC_NS);
        scemi->ServiceLoop();
    }
}
```

SCE-MI Pipes with HMPI

For the DUT side, there are two Verilog files for the HDL-side interface.

```
${UXE_REL}/etc/tba/scemi_input_pipe.vp
${UXE_REL}/etc/tba/scemi_output_pipe.vp
```

We allow some SCE-MI Pipes connected transactors running on HMS (that is, DPI-pipe) and some other SCE-MI Pipes connected to the remote transactors running on CVP (that is, HMPI-pipe). By default, all SCE-MI Pipes are DPI-pipes. You can set the 5th parameter (parameter `hmpi`) of HDL interface module `scemi_input_pipe` and `scemi_output_pipe` to non-zero during module instantiation to configure a SCE-MI Pipe into an HMPI-pipe. Alternatively, you can use `+define+HMPI` in `ixcom` compile command line to set the default value of the `hmpi` parameter to 1 and make all SCE-MI Pipes to HMPI-pipes. Notice that the use of global `+hmpi` will force all `scemi_pipe` to HMPI-pipe regardless of the parameter. However, this will also force all DPI to HMPI which are not related to pipe.

If you do single-step IXCOM compile to build HMS, use the following script.

```
ixcom <other sources and options> \
${UXE_REL}/etc/tba/scemi_input_pipe.vp \
${UXE_REL}/etc/tba/scemi_output_pipe.vp
```

For compiling mixed language designs using IXCOM to build HMS, use the following script.

```
vlan <other sources and options> \
${UXE_REL}/etc/tba/scemi_input_pipe.vp \
${UXE_REL}/etc/tba/scemi_output_pipe.vp
ixcom -top <dut_top>
```

To build the CVP side, include the following adaptor interfaces headers for blocking interface and the common shared library for the non-blocking interface.

- **Header path:**

```
-I ${UXE_REL}/etc/tba
```

- **Header files for blocking interface:**

- ❑ For pthread C-adaptor:
 `#include "uatba_scemi.h"`
- ❑ For pthread C++-adaptor:
 `#include "uatba_pt_cpp.h"`
- ❑ For OSCI SC-adaptor:
 `#include "uatba_osci_sc.h"`
- ❑ For SV adaptor:
 ``include "uatba_sv.sv"`

■ **SCE-MI Pipes Library:**

`${UXE_REL}/lib/<32|64>bit/libuatba_hmpi.so`

For the CVP side, the HMPI-pipe library `libuatba_hmpi.so` contains implementation of both blocking and non-blocking interface functions. Therefore, you should NOT include any SCE-MI Pipe blocking interface implementation source files for CVP as these are needed on the HMS side for various adaptors to DPI-pipes.

Using Makefiles to Compile HMPI Files

To simplify the UXE-TBA compilation flow, certain compilation option files are provided for the different compilation flows. These compilation option files are located in the `/etc/tba` directory and have `.args` as the filename extension.

Following is an example of the Makefile using the compilation option files to compile a basic HMPI test case that contains only one DUT file named `top.v` and two testbench files named `test.c` and `tb.sv`:

```
comp_rtl:
    (cd svr; ixcom -nccompile -F ${AXIS_HOME}/etc/tba/ua.args -ua \
        top.v +dut+dut)
comp_adpt:
    (cd svr; make -f xc_work/xv_make \
        HMSV_INCL_DIR=`ncroot`/tools/include hmsv)
comp_tb:
    (cd clnt; irun -f ${AXIS_HMOE}/etc/tba/tb-hmpi.args \
        tb.sv test.c)
```

Using the compilation options file, eliminates the need for typing out the following options:

```
comp_rtl:
    (cd svr; ixcom -nccompile +sv -ua \
        +define+SCEMI21 \
```

UXE User Guide

IXCOM Hardware Model Interface

```
    ${AXIS_HMOE}/etc/tba/scemi_input_pipe.vp \
    ${AXIS_HMOE}/etc/tba/scemi_output_pipe.vp \
    top.v +dut+dut)
comp_tb:
    (cd clnt; irun -elaborate \
    -DSCEMI21 \
    -TIMESCALE 1ns/1ns \
    -I${AXIS_HOME}/etc/tba \
    tb.sv test.c \
    -top uatba_event \
    ${XC_WORK_DIR}/xc_work/v/libhmsv.sv \
    ${XC_WORK_DIR}/xc_work/c/libhmsv.so \
    ${AXIS_HOME}/lib/${BITS}bit/libscemi21-hmpi.so)
```

The sample Makefile files for different language-based testcases using SCE-MI 2.1 are available in the `${UXE_INST_DIR}/gift/tutorial/uatba/scemi21` and `${UXE_INST_DIR}/gift/tutorial/uatba/scemi21_hmpi` directories. As evident from the Makefiles in these examples, these are identical except that the compilation option file used for compiling the testbench is different. To disable compilation for hardware, remove the `-ua` option from the original `ixcom` or `irun` command.

The tables in the Usage of the Compilation Option Files section provide a summary of the compilation option files.

UXE User Guide
IXCOM Hardware Model Interface

Design Profiling for Palladium XP Acceleration

Profiling a design in the context of hardware acceleration using Palladium XP involves measuring the time spent in running the DUT on the hardware and remaining parts of the design running on the software simulator. A run-time profile file that contains the simulation-run-time performance information is generated. This information enables you to identify bottlenecks in the non-acceleratable testbench environment, which can then be optimized to enhance performance with the Palladium XP system.

The IES profiler is used to estimate the potential speed up before an emulator is available. To analyze the performance speed up possible with the use of the Palladium XP emulator, use the XC Profiler.

This chapter covers the following topics:

- [Design Profiling on Software Using IES Profiler](#)
- [Profiling Design Run-Time Performance on Hardware with XC Profiler](#)

Design Profiling on Software Using IES Profiler

You can use the IES profiler on Palladium XP to:

- ❑ Estimate the expected speedup/performance on a given design when it will be run in COSIM (SA) mode on Palladium XP when compared to the current pure simulation run.
- ❑ Identify the bottlenecks in the non-acceleratable testbench (non-DUT HDL) environment or embedded software modules/functions (TBCalls) that can be optimized to achieve better acceleration performance with the Palladium XP system.

When you enable the IES profiler, you can obtain cumulative statistics of the CPU time spent within portions of the design. These statistics are divided into the following categories:

- **DUT** - Design Units or scopes (including complete hierarchies below a scope) of HDL that you have marked as `DUT`.
- **OTHER** - Other Units or the scope outside of DUT scopes or those that you have explicitly marked as `OTHER`.
- User C applications in the form of shared object libraries communicating with the simulator using PLI/VPI/VHPI or DPI methods.

The DUT Summary section gives a cumulative CPU time by adding the CPU time of each module according to their hierarchy (DUT or OTHER). In addition to this cumulative information, the IES profiler presents the modules that are top CPU-time consumers.

The IES profiler works by interrupting the simulation at regular intervals and noting what was executing at that time. It tracks the number of “hits” on different activities, and this number indicates the approximate amount of CPU time spent in these activities.

The IES profiler is easy to run and has minimal impact on simulation performance and memory usage. Cadence recommends that for an apples-to-apples comparison with final hardware-based performance (that offers full waveform debug capabilities all the time), a full debug-enabled (`+access+rw`) snapshot should be created.

Commands for Design Profiling Using the IES Profiler

You can invoke the IES-based design profiler by using one of the following commands:

- Using the `-profile` Option of the `irun` Command
 - Following are the elaboration-time profiling options of the `irun` command that are related to the `ncelab` command:
 - `-dut_prof_elab`
 - `-cycle_prof`
 - Following are the run-time profiling options of the `irun` command related to the `ncsim` command:
 - `-profile`
 - `-dut_prof <dut_spec_file>`
- Using the `+profile` Option of the `ixcom` Command
- Using the `Tcl profile` Command

Using the -profile Option of the irun Command

You can use the `-profile` option with the `irun` command to generate profiling statistics for a design, as shown below:

```
% irun -profile [other_irun_options] <source_files>
```

When the simulator exits, the profiler creates an output file. By default, the file is created in the current working directory and is called `ncprof.out`.

If you start a simulation with the `-profile` option, profiling can be stopped only by exiting or resetting the simulation. Resetting the simulation with a `reset` command switches off the profiling automatically.

For more information, see *Running the Profiler* section in the *Maximizing Simulation Performance* document in the INCISIV documentation set.

Using the -dut_prof Option for Reporting DUT Profiling Information

Accurately identifying the time spent in the DUT is the most important step in determining if a design is suitable for hardware acceleration. Use the `-dut_prof` option to make the IES Profiler aware of the DUT in the design environment, and generate a run-time profile file, which, by default, is named `ncprof.out`. This file includes a table called *Design Unit Time Summary* that reports the percentage of time spent simulating a specified design under test (DUT) and the amount of time spent in the rest of the design.

```
irun -dut_prof <dut_prof_spec> -profile
```

Format of the DUT Specification Information File

At run time, you need to provide a DUT specification information file, `<dut_prof_spec>` to the IES profiler. This file must specify the name of at least one design unit that is considered to be top level of the DUT RTL/Gate hierarchy, and is assumed to be acceleratable on the Palladium XP system.

The format of the DUT specification in this file is:

```
# comments can be also be added
DUT: <lib>.<cell>:<view>
OTHER: <lib>.<cell>:<view>
```

A sample `<dut_spec_file>` is provided below:

```
-----
# This is a sample dut specification file
DUT: LIB1.E1:A1
```

UXE User Guide

Design Profiling for Palladium XP Acceleration

```
OTHER: LIB2.E2:A2
DUT: LIB3.E3:A3
-----
```

In the `<dut_spec_file>`, the DUT and OTHER keywords have the following meaning:

- DUT: Implies that all the specified instances of `L.C:V (<lib>.<cell>:<view>)` should be considered as DUT `top`. All the instantiation below DUT `top` (defined recursively) including the top-level module of the DUT, is considered as a part of DUT, and the rest is considered as a part of the testbench.
- OTHER: None of the instances of the specified `L.C:V` should be considered as a part of DUT. If an instance of `L.C:V`, which is specified in OTHER, is under the DUT hierarchy of DUT, the whole instance hierarchy below it (`OTHER: L.C:V`) is not considered as DUT. All embedded modules must be specified in the OTHER category.

Specify the required module and all its sub-hierarchies to mark these for profiling as DUT design units. You can specify multiple DUT design units, and their sub-hierarchies. For example:

```
# multiple top level DUT modules
dut: sontop.sontop:v
dut: worklib.bidi:v
dut: chip.data_grab:v
OTHER: worklib.mc28m256x:v
```

Note: It is mandatory to specify the `<library>` and `<view>` names in IES format. To find out the names of the library and view of the module, use the `ncls` utility. The `ncls` utility lists the compiled objects stored in the library system and displays various attributes and information about those objects. For example:

```
% ncls -64 bidi
ncls: 06.20-s004: (c) Copyright 1995-2008 Cadence Design Systems, Inc.
      module worklib.bidi:v (VST)
      module worklib.bidi:v (SIG) <0x661d9fc9>
```

In the above example, the `bidi` design unit resides in `worklib` library with a single Verilog view `v`.

With the `ncls` command, the `-64` option is required because the default setting for the `ncls` utility is 32 bit, while the IXCOM compiler uses 64 bit by default.

If a module exists in multiple libraries or has multiple views, choose the required library and view.

All HDL scopes outside of those marked as DUT are implicitly considered in the `OTHER` category to be interpreted as the testbench or verification-related HDL, which cannot be run on an HDL accelerator like Palladium XP.

In addition to the implicit `OTHER` scopes, any particular design unit can be marked explicitly as `OTHER`. This is useful when a DUT hierarchy marked with the top-level DUT module contains a non-synthesizable design unit (including the sub-hierarchy below it), and, therefore, those scopes should not be counted as acceleratable design unit in the profiling information. In such cases, the units at the top of the non-synthesizable scopes should be added to `dut_spec_file` using the `OTHER` marking.

In the above case, the IES profiler includes CPU time for `mc28m256x` also under the `OTHER` category.



Tip

Profiling information should be generated out of several simulation runs with different test conditions, each of which should run for a duration of at least 5-10 minutes. Very short simulation runs are likely to give inaccurate profile numbers because the design cannot be stimulated enough.

For more information, see *Using the Profiler to Identify and Eliminate Simulation Bottlenecks* in the *Maximizing Simulation Performance* document available in the INCISIV 11.1 documentation.

Using the `+profile` Option of the `ixcom` Command

You can use the `+profile` option of the `ixcom` command to generate profiling statistics for a design using the basic or advanced IES profilers, as shown below:

```
% ixcom {+profile=[ncprof|iprof]} [other_ixcom_options] <source_files>
```

if you specify the `ncprof` argument to the `+profile` option, the basic IES profiler is invoked. If you specify the `iprof` argument to the `+profile` option, the advanced IES profiler is invoked. If you do not specify any argument with the `+profile` option, the default argument is `ncprof`.

For more information, see *Using the Advanced Profiler* and *Using the Basic Profiler* sections in the *Maximizing Simulation Performance* document in the INCISIV documentation set.

Using the `+profile` option implies the following:

- ❑ At elaboration time, IXCOM invokes the `ncelab` utility is invoked with the `-dut_prof_elab` option. The IXCOM compiler automatically generates the DUT

profile specification file in the `xc_work` directory, and specifies this file as an argument to the `-dut_prof` option when invoking *irun*. If you have specified the `+profile` option, IXCOT generates this file even if you have not specified the `-ua` option with the `ixcom` command.

- ❑ The IXCOT compiler enables IES profiling by adding `-profile -dut_prof <generated_dut_spec_file>` with the `NCSIMOPTS` option in the `xc_work/hdl.var` file. The IXCOT compiler also enables hardware profiling by adding the `+xcprof` argument with the `NCSIMOPTS` option in the `xc_work/hdl.var` file. The hardware profiling is enabled only when the design is run on hardware, else it is not invoked.

Using the Tcl profile Command

The `profile` command provides more control over the profiler behavior than the `-profile` option.

With the `profile` command, you can:

- Start or stop profiling at any time (`-on` and `-off`)
- Dump the profiled data to a file at any time (`-dump`)
- Clear the currently collected profiled data (`-clear`)

If profiling is enabled with the `profile` command, it remains on even after you issue a `reset` command. The `reset` command clears the profile data, but does not disable profiling.

Use one of the following options on IES Tcl command line:

```
ncsim> help profile
profile.....Profile Command
  [-clear].....Clear profile data.
  [-dump] [-overwrite].....Dump profile data to file. Use the overwrite
                           option to replace an existing file.
  [-off].....Disable profiling of user designs.
  [-on].....Enable Profiling of user designs.
```

For example:

```
ncsim> profile -dump myprof.out -clear
```

Note: The `-dut_prof` option is not supported with the Tcl `profile` command.

For more information about the `profile` command, see the *profile* section in the *Using the Tcl Command-line Interface* chapter of the *Incisive Simulator Tcl Command Reference* document in the INCISIV documentation set.

Analyzing and Interpreting the IES Profiler Output in ncprof.out

The profiler output file, `ncprof.out` contains the following sections:

- Design Unit Time Summary
- Most Active Modules (Other Units)
- Stream Counts
- Most Active Modules (Behavioral)
- IXCOM Run-Time Profiler Report (hardware-run only)
- Stream Type Summary Counts

Design Unit Time Summary

In an `ncprof.out` file generated using the `-dut_prof` option, the Design Unit Time Summary section summarizes the % of hits and number of hits for the following categories:

- DUT
- HDL corresponding to non-DUT hierarchy.
- HDL that is not attributed to either 1 or 2 above, under the Aggregated Simulation Overhead category
- HDL C interface
- User Code
- IXCOM (Emulator + Overhead), if design profiled in IXCOM hardware mode
- Gate Evaluation Methods
- Support for VPI callbacks (or UI)
- Engine support
- Outside engine
- File I/O

Design Unit Time Summary (473684 hits total)

%hits	#hits	Set
-------	-------	-----

UXE User Guide

Design Profiling for Palladium XP Acceleration

66.3	314075	Design Unit
2.6	12545	Other Units
2.5	11731	Aggregated Simulation Overhead
12.9	60985	Specman
2.1	9739	HDL C interface
2.4	11321	User Code
10.0	47835	Support for VPI callbacks (or UI)
0.1	235	Engine support
0.9	4608	Outside engine
0.2	608	File I/O

Potential Speed-Up Estimation in the SA Run Mode

This section provides a range of achievable speed-up in the SA run mode estimated on the basis of the various factors that affect the speed-up.

This section includes the following sub-sections:

- Best-Case Performance Estimate
- Average-Case Performance Estimate

Following is a brief description of these sub-sections.

Best-Case Performance Estimate

This section provides the performance estimate for the best case, and is based on following equation:

$$\text{Estimated speedup} = 100 / (100 - (\% \text{DUT CPU time}))$$

This is the best-case performance estimate of the SA run mode, and is based on the following assumptions:

- ❑ The time spent in hardware-software synchronization and in executing the DUT on Palladium XP is assumed to be negligible as compared to the time spent in testbench.
- ❑ The time taken by the testbench (software) portion of the design to execute on the simulator remains identical in the co-simulation run and the present pure-simulation run.

In this section, the following data is generated:

```
--- Time spent in "Design Unit (DUT)" = 66.3%
```


UXE User Guide

Design Profiling for Palladium XP Acceleration

--- Time spent in "Aggregated Simulation Overhead (ASO)" = 2.5%

Dividing the ASO in proportion between the DUT and OTHER categories for a more realistic estimate:

--- Estimated Time spent in DUT = 66.3% + 66.3% of 2.5 (ASO %) = 68.0%

--- Estimated Best-Case Speedup = $(100/(100-(\text{DUT time}\%))) = 3.12\text{X}$

The `<dut_spec_file>` specifies the DUT hierarchy and the profiler adds the time for that design hierarchy into the time spent in DUT.

The `-dut_prof_elab` option must be included with the `irun` command for the above-mentioned information to be generated. Otherwise, the following message is displayed at the top of the section in `ncprof.out` file:

```
### Please elaborate the design snapshot using '-dut_prof_elab' NCELAB/IRUN
elaboration option for improved accuracy in Time spent in DUT ####
```

Average-Case Performance Estimate

While calculating the best-case performance estimate, the time spent in hardware-software synchronization and executing the DUT on Palladium XP is assumed to be negligible as compared to the time spent in the testbench. However, this assumption is not completely valid. Therefore, to calculate a reasonable average-case performance estimate of the speed-up, it is necessary to include the time spent in hardware-software synchronization and executing the DUT on Palladium XP. Generally, each synchronization incurs a penalty in the range of 10 us to 15 us. So, a typical value of 13 us penalty per hardware-software synchronization is assumed. The average-case performance estimate can be determined by estimating the number of hardware-software synchronizations required in the co-simulation run.

To enable this calculation, you need to use the `-cycle_prof` and `-dut_prof_elab` options of the `irun` command. When you use these options, the following data is dumped in the `ncprof.out` file.

```
--- SW simulator executes @ 29487.78 cycles/sec.
--- SW simulation runtime = 463.65 sec.
--- Time spent in SW SIM of non-DUT portion = (100 - %DUT)% of SW simulation
runtime = 148.37 sec.
--- Estimated number of HW-SW synchronizations required in COSIM run = 4391604
--- Estimated COSIM runtime = DUT runtime on HW (incl. HW-SW Sync Overhead) +
non-DUT runtime on SW = 57.09 + 148.37 = 205.46 sec.
--- Estimated Average Case speedup = (SW SIM runtime)/(COSIM runtime) =
463.65/205.46 = 2.26 X.
```

UXE User Guide

Design Profiling for Palladium XP Acceleration

This sub-section is displayed only when you specify the `-cycle_prof` option while elaborating the design hierarchy. Otherwise, a message like the following appears in the `ncprof.out` file.

```
### Please elaborate the design snapshot using '-cycle_prof' NCELAB/IRUN
elaboration option for Average Case Speedup Estimation ###
```

You should use the following `irun` command options for IES design profiling if you need the average-case performance estimate section in the `ncprof.out` file:

```
% irun -dut_prof_elab \
      -cycle_prof -profile \
      -dut_prof <dut_specification_file> \
      [other_irun_options] <source_files>
```

Sample Report File with the Average-Case Performance Estimate Section

Following is an extract of the header part of a sample report file, when both the `-dut_prof_elab` and `-cycle_prof` elaboration options have been specified.

```
-----
Design Unit Time Summary (473684hits total)
-----
```

%hits	#hits	Set
66.3	314075	Design Unit
2.6	12545	Other Units
2.5	11731	Aggregated Simulation Overhead
12.9	60985	Specman
2.1	9739	HDL C interface
2.4	11321	User Code
10.0	47835	Support for VPI callbacks (or UI)
0.1	235	Engine support
0.9	4608	Outside engine
0.2	608	File I/O

```
-----
Potential Speed-up Estimation in IXCOM 'Run Mode'
-----
```

```
Best Case Performance Estimate
-----
```

```
--- Time spent in "Design Unit (DUT)" = 66.3%
```

```
--- Time spent in "Aggregated Simulation Overhead (ASO)" = 2.5%
```

```
Dividing the ASO in proportion between DUT and OTHER to get more realistic
estimate
```

UXE User Guide

Design Profiling for Palladium XP Acceleration

```
--- Estimated Time spent in DUT = 66.3% + 66.3% of 2.5 (ASO%) = 68.0%
--- Estimated Best-Case Speedup = (100/(100-(DUT time%))) = 3.12X
```

Average Case Performance Estimate

```
--- SW simulator executes @ 29487.78 cycles/sec.
--- SW simulation runtime = 463.65 sec.
--- Time spent in SW SIM of non-DUT portion = (100 - %DUT)% of SW simulation
    runtime = 148.37 sec.
--- Estimated number of HW-SW synchronizations required in COSIM run = 4391604
--- Estimated COSIM runtime = DUT runtime on HW (incl. HW-SW Sync Overhead) +
    non-DUT runtime on SW = 57.09 + 148.37 = 205.46 sec.
--- Estimated Average Case speedup = (SW SIM runtime)/(COSIM runtime) =
    463.65/205.46 = 2.26 X.
```

Most Active Modules (Other Units)

The `ncprof.out` file also provides the top CPU-time consumers in `OTHER` scopes in the **Most Active Modules** section. This section carries detailed information about `OTHER` time consumers. This is useful for identifying and prioritizing the HDL-verification modules that can be targeted for improving performance of acceleration by mapping them to hardware or less CPU-time-consuming methodologies.

Most Active Modules (Other Units)

```
%hits #hits #inst name
  0.8  5131 [    1] worklib.SonTopTB:v (file: <uxe_install>son/verification/chip/
SonTopTB.v line: 21)
....
```

Stream Counts

This section indicates the proportion of simulation time spent in different parts of the design (processes, subprograms), in the kernel, in housekeeping, in user C code, and in interacting with the operating system.

The following example shows a part of a Stream Counts section from a profile.

```
Stream Counts (90056 hits total)
```

```
%hits #hits #inst name
```

UXE User Guide

Design Profiling for Palladium XP Acceleration

```
36.2 32568 [ 1] Always stmt (file: ./hdl/hsp11.v, line: 54 in worklib.hsp11
[module])
32.9 29654 [    ] specman engine
 8.2  7372 [    ] Method SSS_KM_CL2TA (method)
 7.8  7067 [    ] Method SSS_KM_FINDRFT (method)
 6.1  5462 [    ] /<specman_install>/specman/6.20.s3/components/sn/linux/
libncvlog_sn_adapter.so
 1.3  1165 [    ] Logic primitive 'timing output', Module 'BUFX20', Port 'Y', 1
Timing Inputs (process)
```

The Specman verification engine is a common case of an external application, and in the above example, it appears in the Stream Counts section as two separate entries - one for the Specman engine and the other for Specman interface (`libncvlog_sn_adapter`) to the simulator. Here, the Specman engine is consuming 32.9% time and the adapter and interface to simulator 6.1%. This separation of engine time and interface time in Stream Counts is only done for Specman case. For a general case of C application, all of the time will be reported in Stream Counts, and will be associated with the `.so` file.

Most Active Modules (Behavioral)

This section summarizes the contribution from each of the design units in descending order of the simulation time. This can be useful when a design unit is taking a lot of time due to the combined time in a lot of streams that are below the threshold for being listed in the Stream Counts section.

Following is an example of the *Most Active Modules* section.

```
-----
Most Active Modules (behavioral)
-----
```

```
%hits #hits #inst name
7.3    57    [1] xc_ncwork.xc_top:module (file:./xc_work/v/xc_top.v line: 7)
0.9     7    [1] xc_ncwork.nw_driver_xcm23:module (file:./xc_work/v/23n.v95 line:
4)
0.9     7    [1] xc_ncwork.nw_driver_xcm25:module (file:./xc_work/v/25n.v95 line:
4)
0.6     5    [1] xc_ncwork.nw_monitor_xcm20:module (file:./xc_work/v/20n.v95 line:
4)
0.6     5    [1] xc_ncwork.nw_monitor_xcm26:module (file:./xc_work/v/26n.v95 line:
4)
0.5     4    [1] xc_ncwork.clock:module (file:./xc_work/v/2n.v95 line: 4)
0.5     4    [1] xc_ncwork.nw_driver_xcm19:module (file:./xc_work/v/19n.v95 line:
4)
0.5     4    [1] xc_ncwork.nw_monitor_xcm24:module (file:./xc_work/v/24n.v95 line:
4)
```

UXE User Guide

Design Profiling for Palladium XP Acceleration

IXCOM Run-Time Profiler Report

This section presents the IXCOM run-time profiling information for the entire design. this section is generated only when the design is run on the hardware. For more information, see [Profiling Design Run-Time Performance on Hardware with XC Profiler](#) on page 684.

IXCOM Runtime Profiler Report (760 hits, 97.4% of total)

%hits	#hits	%xcRun	name	comments
2.8	22	2.9	xc_runtime	IXCOM runtime Control (Overhead).
24.2	189	24.9	IUS	IUS runtime during HW Run (TB, tbcalls, VPI/VHPI/DPI).
2.2	17	2.2	decode	HW O/P change decode (Overhead).
68.2	532	70.0	emu	Time spent in HW evaluation.

Note: This section is reported in `ncprof.out` only if the design is also run on the hardware. For a pure software simulation, the `ncprof.out` file does not include the IXCOM Run-Time Profiler Report section.

Stream Type Summary Counts

This final section of the `ncprof.out` file summarizes the overall contribution by different activity units toward simulation time. This section includes streams with counts too low to appear separately in the first section and gives a general idea of where simulation time is being spent.

The summary makes it easier to identify widespread inefficiencies in the simulation.

Following is an example of *Stream Type Summary Counts*.

Stream Type Summary Counts (780 hits total)

%hits	#hits	#inst	name
73.2	571	[]	IXCOM (Emulator + Overhead)
6.7	52	[6029]	Always statements
4.2	33	[179]	Verilog tasks
4.1	32	[]	Simulation Engine Time
2.7	21	[]	User defined system tasks/functions
1.9	15	[2199]	Initial statements
1.7	13	[66]	Verilog functions
1.3	10	[]	Engine support
1.3	10	[]	HDL C interface

UXE User Guide

Design Profiling for Palladium XP Acceleration

1.2	9	[]	VCD/SHM variable dumping
0.8	6	[6603]	Non-blocking assignments
0.4	3	[4926]	Continuous assignments
0.4	3	[]	CAG stream
0.3	2	[831]	Anonymous continuous assignments

For more information, see *Using the Profiler to Identify and Eliminate Simulation Bottlenecks* in the *Maximizing Simulation Performance* document available in the INCISIV documentation set.

Assertions Reporting in ncprof.out

For a design compiled and run in pure software simulation (using IES only), assertions are profiled separately. The profiling information related to assertions is reported as follows:

- An `Assertions` category is reported in the `Design Unit Time Summary` section.
- A separate `Assertion Summary Counts` section is reported in the `ncprof.out` file.

The IXCOM compiler, however, converts assertions to HDL. Therefore, for a design compiled using the IXCOM compiler, the assertions are not profiled separately and reported in either of the above two sections in `ncprof.out`.

Improving Profiling Accuracy and Caveats

Following sections provide some tips and caveats for improving the profiler accuracy related to:

- [Using the -dut_prof_elab Option in Pure Software Simulation](#)
- [Using -cycle_prof to Improve the Potential Speed-Up Estimation](#)
- [Updating Embedded Modules Information in <dut_spec_file>](#)
- [Using IES Run-Time Profile Command to Control Profiling](#)

Using the -dut_prof_elab Option in Pure Software Simulation

To ensure accuracy in the statistics reported in the `Design Unit Time Summary` section of `ncprof.out` for a design compiled and run using IES, use the `-dut_prof_elab` option during design elaboration.

Using `-cycle_prof` to Improve the Potential Speed-Up Estimation

Use of the `-cycle_prof` option is costly, and can incur 5-10% of simulation-run-time penalty. Use this option only when profiling the design for a potential speed-up estimation.

Updating Embedded Modules Information in `<dut_spec_file>`

In a typical profiling exercise, it is customary to declare just the top design units of the DUT hierarchy in `<dut_spec_file>`. This makes an assumption that the entire hierarchies are synthesizable and is a good first order assumption to get an estimate of acceleratability of the environment.

However, for a more precise profile, you must check if there are any significant non-synthesizable blocks under the DUT hierarchy that need to run under IES and, therefore, must be explicitly marked as `OTHER` in the `<dut_spec_file>`. Examples of such design blocks include: embedded C model of IPs, analog-mixed signal blocks, and complex behavioral CAMs.

To update the `OTHER` list in the `<dut_spec_file>`, first generate a list of embedded software modules from the compiler's RTL mapping phase in `ixcom.log`. For this step, the design should be run through IXCOM compiler with the `-ua` option.

This produces the `ixcom.log` file containing the list of non-synthesizable embedded modules. For the above list, add the following line in the `<dut_spec_file>` (after verifying library and view name in IES database as explained in [Format of the DUT Specification Information File](#)) as follows:

```
other: worklib. FXPLL110HC0H_APGD:v
```

Multiple modules in the form of multiple `OTHER` specification lines can be added.

Using IES Run-Time Profile Command to Control Profiling

To start profiling from a non-zero time or to reset profile data during simulation, instead of using the `-profile` option at the start of simulation, use the Tcl `profile` command. For more information, see [Using the Tcl profile Command](#).

Profiling Design Run-Time Performance on Hardware with XC Profiler

In order to analyze the performance of the design run on the Palladium XP emulator, and to identify the bottlenecks to be removed to improve performance, you can use the run-time profile information and design statistics, generated at the end of a simulation run, specifically for SA mode. For this, you need to enable the XC profiler.

Enabling the XC Profiler

You can use one of the following ways to enable the XC profiler:

- Specify the `+xcprof` option of the `irun` or the `xeDebug --ncsim` command.

Generating Hardware-Run Profiling Information Using +xcprof Option

- Use the TCL based `xc profon` command on the `ncsim>` prompt when using the `xeDebug` tool. (Generating Hardware-Run Profiling Information Using xc profon Command)

Note: Using the `+xcprof` option supersedes the `xc profon` command. When you use the `+xcprof` option, the hardware-run-time profiler is enabled for the entire session, and subsequent `xc profon` and `xc profoff` commands do not have any effect.

Generating Hardware-Run Profiling Information Using +xcprof Option

Specify the `+xcprof` option with any of the following commands to generate hardware-run-time profiling information for SA mode:

```
irun -R [other_options] +xcprof
```

When you use the `-profile` option of the `irun` command to enable IES profiling, the verbose hardware-run-time profiler (`+xcprof` option) is not enabled by default. You need to explicitly specify the `+xcprof` option to enable the hardware-run-time profiling.

Or,

```
xeDebug --ncsim +xcprof [other_options]
```

The use of the `+xcprof` option does not incur a run-time penalty of more than 1-2% of the total run time of the design.

UXE User Guide

Design Profiling for Palladium XP Acceleration

The output of this option is generated in the `xcprof.out` file in the current working directory. It indicates the time spent in running the design on the hardware and software sides during the previous run.

Categories of Profiling Information in the XC Profiler Report (+xcprof Option)

The hardware-run-time profiling information for the design running on Palladium XP is generated in a detailed profiling report named `xcprof.out` in the current working directory (by default) when you enable the XC profiler.

You can modify the location and name of the profiler-output file by using the `+xcprofoutput=<filename>` option when you specify the `+xcprof` option as follows:

```
% irun -R [other_irun_options] +xcprof +xcprofoutput=prof_work/xcprof_1.out
```

The profiling information in the `xcprof.out` file is sorted into various categories of statistics as listed in the following table.

Each category is reported only if the count in the given run is non-zero.

Profiling Category	Profiling Information
Behavioral evals statistics	Reports the number of behavioral evaluations done per module. These per-module behavioral-evaluations occur concurrently. So, it should not be aggregated by summation to match with the global behavioral evaluations done during the simulation of the design.

UXE User Guide

Design Profiling for Palladium XP Acceleration

Profiling Category	Profiling Information
TB/DPI/SVA/SysTask calls	Reports the number of TBCalls, DPI-Calls, SVA, and System Task calls that happened during the run along with the line and file information for these calls. Also reports the time taken in executing the DPI calls, including the time required to pass the parameters between the software and hardware portions. You can use this information to determine if: ■ The parameters to the DPI calls should be passed using PIO or MEM ■ The DPI calls should be mapped to execute in only hardware or software. Note: Time-based profiling is not supported for VHDL TBCalls, HMPI, and those TBCalls that have not been processed by interface compilation.
Output event statistics	Reports the number of events on the DUT outputs
Input event statistics	Reports the number of events on the DUT Inputs
Force Release statistics	Reports the number of force-release operations on the DUT signals during the hardware run.
Deposit statistics	Reports the number of deposits on DUT signals during the hardware run
Memory Read statistics	Reports the number of memory-read transactions issued to the hardware
Memory Write statistics	Reports the number of memory-write transactions issued to the hardware.

Trade-Off Between the Run-Time Impact and Accuracy of Time-Based DPI-Profiling

You can use the `+dpiprofsamplecnt=<N>` option of the `ixcom` command to change the number of samples that the profiler should consider for computing the average time taken by the DPI calls. By default, the value of the sample count `<N>` is set to 100.

The higher the sample count, the more accurate are the results.

UXE User Guide

Design Profiling for Palladium XP Acceleration

The time spent in the first N samples of the DPI calls is averaged to get the time spent per call, and based on this number, the total time spent in executing the DPI calls is extrapolated. This extrapolation is based on the assumption that typically each invocation of the same DPI call consumes a similar amount of time.

Total Time = No. of DPI calls * Time spent in execution of each DPI call computed by averaging first N samples.

However, the execution of the DPI call might depend on the parameter that is being passed, or on a global flag. Consequently, the time required for each invocation might be different. In such cases, increasing the sample count to a larger number (greater than or equal to) the total number of calls for this DPI routine would help calculate the actual time spent in the execution of the call, and estimation or extrapolation would be unnecessary.

The Profiler denotes the extrapolated value with $\approx$ (approximately equal to) symbol instead of the $=$ (equal to) sign.

Note: If the `+xcprof` option is not used, or the sample count is set to 0, no sampling is done.

For example:

```
% irun -R [other_options] +xcprof +dpiprofsamplecnt=5000
```

The Profiler generates the following information per DPI call:

```
<Total_Count> <DPI_call> (Mapped to hardware or software, PIO-based or MEM-based)
```

```
Profile: <Total_time> (<time_percall>, %  
<time_spent_in_input_transfer,_execution_and_output_transfer>  
        <Location_of_the_DPI_call_i.e_File,_Line_info>
```

Example:

```
500000 top.dut.dpicalls_0_.m0.c_task_multiple (Type = tb_import:pio)  
Profile: Total time  $\approx$  4.34 sec (8.68 usec/call, input-args=8.53%, exec=79.26%,  
output-args=12.21%)  
Loc: (top.v, 92)
```

Example: DPI Profiling Sample Count

--- DPI Profiling Sample count set to default 25. Use option `+dpiprofsamplecnt=<N>` to change.

```
-----  
DPICALL statistics: 6238 dpi-calls  
-----
```

```
3119 sys.timer.timer_interrupt (Type = tb_import:pio)  
Profile: Total time  $\approx$  0.21 sec (66.48 usec/call, input-args=1.50%,  
exec=96.78%, output-args=1.71%)
```

UXE User Guide

Design Profiling for Palladium XP Acceleration

```
Loc: (tokens.v, 407)
1560 sys.uart4.uart_interrupt (Type = tb_import:pio)
  Profile: Total time =~ 0.07 sec (42.30 usec/call, input-args=2.22%,
    exec=95.08%, output-args=2.70%)
Loc: (tokens.v, 214)
1559 sys.uart3.uart_interrupt (Type = tb_import:pio)
  Profile: Total time =~ 0.07 sec (43.76 usec/call, input-args=2.06%,
    exec=96.02%, output-args=1.92%)
Loc: (tokens.v, 214)
```

Analysis and Interpretation of Report Generated with the +xcprof Option

This section provides some example reports followed by the analysis of the information provided in these reports.

Example 1:

```
xcprof.out for (RUN Mode)
=====
IXCOM Performance and Design Profile
-----
--- Maximum HW operating speed (fclk freq): 3571.00 KHz
--- Clocking Mode: Default (2X).
--- HW executed 4927145 evals(or sim-steps) in 52.89 sec (52.85 CPU sec)
--- Acceleration speed achieved: 93.16 KHz (93157.21 evals/sec)
--- Number of HW-SW Synchronizations executed: 4927144
--- Channel Latency per Synchronization: 8.00 us/sync.
--- xc Profile: (%)
    78.05    HW-EMU (Elapsed: 41.28 sec; CPU: 41.25 sec)
    18.71    SW-SIM (Elapsed: 9.89 sec; CPU: 9.89 sec)
    3.24     SYNC-OH (Elapsed: 1.71 sec; CPU: 1.71 sec)

--- HW-EMU : HW evaluations and Synchronization Latency.
--- SW-SIM : TB, TBCalls, DPICalls, VPI/VHPI/PLI/E/SystemC.
--- SYNC-OH: Synchronization Overhead of IXCOM Runtime.
=====
Output event statistics: 410565 output events
-----
    410564 top.PD0.clrOut
        1 top.PD0.vldOut
=====
```

UXE User Guide

Design Profiling for Palladium XP Acceleration

Input event statistics: 31203242 Input events

```
4105954 top.PD0.clk
821190 top.PD0.slowclk
410564 top.PD0.clrIn(0)
410564 top.PD0.clrIn(1)
```

Analysis of the above report for Example 1 is as follows:

- The maximum operating FCLK frequency for this design is ~3.5 MHz. However, the speed-up that is achieved in this run is ~ 93KHz.
- The time of 52.89 sec in the above report is the elapsed time during the simulation run after the design is swapped in to the hardware until the end of the particular hardware-run session. This could be at the end of simulation or at the time of swapping out of hardware. Out of this 52.89 sec, 78.05% has been spent on the emulator, 18.71% on the software simulator, that is for executing the TBCalls or while running the testBench portion, and 3.24% in transferring data between hardware and software sides.
- Channel Latency per synchronization or the I/O latency is the latency for sending input values to hardware, evaluation, and getting output values from hardware. I/O latency excludes any time used for output decoding and TB-runtime communication through DPI calls.

This I/O latency is obtained through peak I/O test at the end of simulation. However, if the end-of-simulation occurs on the software side, and the emulator cannot be locked again at the end to compute this value, a warning message is generated.

In such cases, you can use the `xc peaktest -io Tcl` command to determine the latency.

Repeat the `xc peaktest` command three-five times to get an average value:

```
xc peaktest -io -n 1000000
xc peaktest -io -n 1000000
xc peaktest -io -n 1000000
```

- Based on the above results, the break-up of time spent on the hardware can be displayed.
 - Time spent in hardware_software synchronization = Channel Latency * #synchronizations = 8.0 us * 4927144 = 39.41 s.
 - Time spent on hardware-Emu = DUT execution + hardware_software synchronization.
 - Time spent in execution of DUT on emu = 41.28 – 39.41 = 1.87 s.

UXE User Guide

Design Profiling for Palladium XP Acceleration

- In *RUN MODE*, the number of synchronizations is the primary contributor to performance.
- In *RUN MODE*, the hardware stops at all the `tb_delays` to synchronize the DUT inputs, deposit, force, and memory write operations.
 - From the input-output event section, it can be inferred that the hardware has to stop to synchronize on the clock signals, `clk` and `slowclock`, `clk` (4105954) + `slowclock` (821190) = 4927144 times.
- The primary cause of synchronization is the testbench fast clock (`top.PD0.clk`).

Example 2:

xcprof.out for (Optimized SA -TBRUN Mode)

```
=====
IXCOM Performance and Design Profile
-----
--- Maximum HW operating speed (fclk freq): 4032.00 KHz
--- Clocking Mode: +1xua is enabled.
--- HW executed 4105955 evals(or sim-steps) in 17.46 sec (17.43 CPU sec)
--- Acceleration speed achieved: 235.13 KHz (235128.79 evals/sec)
--- Number of HW-SW Synchronizations executed: 821191
--- Channel Latency per Synchronization: 8.43 us/sync.
--- xc Profile: (%)
    55.06    HW-EMU (Elapsed: 9.61 sec; CPU: 9.60 sec)
    39.17    SW-SIM (Elapsed: 6.84 sec; CPU: 6.83 sec)
     5.78    SYNC-OH (Elapsed: 1.01 sec; CPU: 1.01 sec)
--- HW-EMU : HW evaluations and Synchronization Latency.
--- SW-SIM : TB, TBCalls, DPICalls, VPI/VHPI/PLI/E/SystemC.
--- SYNC-OH: Synchronization Overhead of IXCOM Runtime.
=====
Output event statistics: 410565 output events
-----
    410564 top.PD0.clrOut
         1 top.PD0.vldOut
=====
Input event statistics: 27097288 Input events
-----
    821190 top.PD0.slowclk
    410564 top.PD0.clrIn(0)
```

UXE User Guide

Design Profiling for Palladium XP Acceleration

You can change the testbench and the DUT in the SA mode so that instead of the testbench sending the faster clock (clk) to DUT, the DUT generates the faster clock, thereby improving the number of required synchronizations from 4927144 to 821191.

Analysis of the above report for Example 2 is as follows:

- The synchronizations happen only due to slowclock (821190 top.PD0.slowclk input events).
- The speed-up that is achieved in this run is ~ 235KHz. This result is because of the following reasons:
 - Only 55% of total runtime is spent on hardware (9.61 sec out of 17.46 sec).
 - Of this 55% of the time spent on hardware, the hardware needed to execute 821191 synchronizations with each synchronization having a latency of 8.43 us.
- Following is the break up of time spent on hardware:
 - Time spent in hardware_software synchronization = Channel Latency * #synchronizations = 8.43 us * 821191 = 6.92 s.
 - Time spend on hardware-Emu = DUT execution + hardware_software synchronization.
 - Time spent in execution of DUT on emu = 9.61 – 6.92 = 2.69 s.

As can be seen, the bottleneck is still the number of hardware_software synchronizations. So, to further improve the performance, the DUT-testbench communication can be reduced by using TBA as shown in the following example.

Example 3:

xcprof.out for (TBA using SCEMI pipes)

```
=====
IXCOM Performance and Design Profile
-----
--- Maximum HW operating speed (fclk freq): 4167.00 KHz
--- Clocking Mode: Default (2X).
--- HW executed 8212015 evals(or sim-steps) + (821193 behav evals) in 7.70 sec (7.65 CPU sec)
--- Acceleration speed achieved: 1172.95 KHz (1172952.11 evals/sec)
--- Number of HW-SW Synchronizations executed: 53
--- Channel Latency per Synchronization: 8.28 us/sync.

--- xc Profile: (%)
```

UXE User Guide

Design Profiling for Palladium XP Acceleration

```
68.46      SW-SIM (Elapsed: 5.27 sec; CPU: 5.24 sec)
26.81      HW-EMU (Elapsed: 2.06 sec; CPU: 2.05 sec)
 3.12      HW-MEM (Elapsed: 0.24 sec; CPU: 0.24 sec)
 1.60      SYNC-OH (Elapsed: 0.12 sec; CPU: 0.12 sec)
--- SW-SIM : TB, TBCalls, DPICalls, VPI/VHPI/PLI/E/SystemC.
--- HW-EMU : HW evaluations and Synchronization Latency.
--- HW-MEM : Memory Access (read/write) from/to HW.
--- SYNC-OH: Synchronization Overhead of IXCORE Runtime.
```

```
-----
--- Synchronizations due to TB/DPI/SVA/SysTask calls: 53.
--- Total number of TB/DPI/SVA/SysTask calls: 53.
-----
```

```
TBCALL statistics: 53 tb-calls
-----
```

```
27 <protected>.<protected> (/vobs/ua/share/uxe/etc/tba/
   scemi_input_pipe.vp, 232)
26 <protected>.<protected> (/vobs/ua/share/uxe/etc/tba/
   scemi_output_pipe.vp, 271)
-----
```

```
Memory Read statistics: 26 mem reads
-----
```

```
26 top.PD0.bfm_ins.my_output_pipe.buffer
```

```
--- Memory Read Transactions via Orion 26 times, transferred 4926996 bytes in 0.1174
sec (41.976 MB/sec)
-----
```

```
Memory Writes statistics: 26 mem writes
-----
```

```
26 top.PD0.bfm_ins.my_input_pipe.buffer
```

```
--- Memory Write Transactions via Orion 26 times, transferred 4926996 bytes in
0.1473 sec (33.452 MB/sec)
-----
```

```
--- Memory Access during Swapin 1 times, transferred 196608 bytes in 0.0109 sec
(18.006 MB/sec)
```

Analysis of the above report for Example 3 is as follows:

- Maximum hardware-operating speed (fclk freq): 4167.00 KHz.
- The speed-up that is achieved in this run is ~ 1172.95 KHz. This result is because of the following reasons:
 - Only 26.81% of total run time is spent on hardware (2.06 sec out of 7.7sec).

- ❑ Of this 26.81% of the time spent on hardware, the hardware had to execute 53 synchronizations with each synchronization having a latency of 8.28 us.
- The breakup of time spent on hardware is as follows:
 - Time spent in hardware_software synchronization = Channel Latency * #synchronizations = 8.28 us * 53 = 438 us.
 - Time spend on hardware-Emu = DUT execution + hardware_software synchronization.
 - Time spent in execution of DUT on emu = 2.06 – 0.000438 = 2.06 s

As can be seen in this TBA testcase, the synchronization is no longer the bottleneck in performance. The bottleneck is that 68% of total run time is in software simulation to service the 53 tbcalls. You can identify these tbcalls from the report and optimize them to further improve the performance.

Generating Hardware-Run Profiling Information Using `xc profon` Command

The `xc profon` command profiles the simulation time during the hardware run. You can use the `xc profon` command at any time before executing the `xc run` and `xc tbrun` commands. The hardware run-time profiling process begins only after the design states have been swapped into the hardware.

Here is a comparison of the use of `xc profon` command with the use of the `+xcprof` option:

- The `xc profon` command is a Tcl command that enables you to switch the profiler on or off using the TCL interface **during** the run-time session. This is different from the use of the `+xcprof` option, which must be specified **before** the run to enable the profiler.
- The `xc profon` command provides a basic-level profiling of **smaller sessions** within the whole simulation, and generates a basic report in the `irun.log` file to indicate the time spent in various categories. This is unlike the `+xcprof` option that works as a more advanced profiler for the **complete run-time session**, and generates a comprehensive performance report in a separate `xcprof.out` file.

To end the profiling session, use the `xc profoff` command. If you do not issue the `xc profoff` command, the profiler automatically stops when the simulation finishes.

Analyzing and Interpreting the Profiling Output of the `xc profon` Command

When the profiler stops, results like the following are displayed in the `irun.log` file:

UXE User Guide

Design Profiling for Palladium XP Acceleration

```
--- HW execs 951077 evals 0 bevals 951076 tbsyncs in 30.53 sec (32.10 us/eval;  
31147.95 eval/sec; 30.52 CPU sec)  
--- HW execs 0 force/deposit 31512984 input events  
--- HW execs 12311846 output events  
--- Accessed Memory with Orion 2 times, transferred 65824 bytes in 0.0035 sec  
(18.695 MB/sec)  
--- Accessed Memory with DBEngine 2 times, transferred 786944 bytes in 0.1220  
sec (6.452 MB/sec)  
--- HW execs Memory access during Swap-In 4 times in 0.1255 sec  
--- xc Profile: (%)  
50.61      emu (Elapsed: 15.45 sec; CPU: 15.45 sec)  
35.01      IUS (Elapsed: 10.69 sec; CPU: 10.69 sec)  
10.75      xc_runtime (Elapsed: 3.28 sec; CPU: 3.28 sec)  
3.63       decode (Elapsed: 1.11 sec; CPU: 1.11 sec)
```

Here's a brief description of the values in this report:

- The `emu` value is the percentage of time spent in hardware evaluation. This includes the time spent in sending the DMA input to the hardware, initiating hardware evaluation, polling the hardware for stop conditions, and sending the DMA output back to the software simulator.
- The `IUS` value is the percentage of time spent in IES space during the profile time. This includes the simulation events in testbench modules, the `tbcall` tasks triggered by hardware `tbcall` calls, and any overheads imposed by the VPI, VHPI, and DPI calls to the IES simulator.
- The `decode` value is the percentage of time spent in decoding the hardware output changes.

If many DUT outputs change very frequently, this time can be significant.

- The `xc_runtime` value is the percentage of time spent in the rest of the run-time execution.

When a design is profiled using the `-profile` option of the `irun` command, the profiling information is automatically reported in `ncprof.out`, as shown below.

The categorization of information in the above-mentioned buckets indicates that:

- The time reported under the `emu` category is the time spent when the design was running on the hardware. It includes the time spent in executing the DUT on hardware and the I/O latency delays in the hardware-software synchronizations. This is the same time as is reported in the `HW-Emu` category of the XC Profiler report, which is generated with the use of the `+xcprof` option.

UXE User Guide

Design Profiling for Palladium XP Acceleration

- ❑ The times reported in the `xc_runtime` and `decode` categories are the typical overhead times for accessing and transferring the DUT between the Palladium XP emulator and the IES simulator.
- ❑ The time reported in the `IUS` category is the time spent in the non-DUT portion of the design like the testbench or the `tbcalls` tasks, and IXCOM-instrumentation overhead (like the `xc_top` directory).

Example: Sample output of XC Profiler in `ncprof.out`

```
-----  
IXCOM Runtime Profiler Report (985 hits,  97.5% of total)  
-----
```

%hits	#hits	%xcRun	name	comments
3.2	32	3.2	<code>xc_runtime</code>	IXCOM runtime Control (Overhead).
28.8	291	29.5	<code>IUS</code>	IUS runtime during HW Run (TB, <code>tbcalls</code> , VPI/VHPI/DPI).
3.0	30	3.0	<code>decode</code>	HW O/P change decode (Overhead).
62.6	632	64.2	<code>emu</code>	Time spent in HW evaluation.

UXE User Guide

Design Profiling for Palladium XP Acceleration

Analyzing Power Consumption of the Designs

You can analyze and feed power information back into the design at every step. The underlying principle of power analysis is to calculate the power consumed by a circuit that has the following three contributing parts:

- **Leakage power** that reflects the *logic state* of the device. It is dependent on the leakage current that flows between the source and drain.
- **Internal power** that is the measure of power used by the logic gates (cells) when a driver is switching, that is, *toggle rate* * *cell transition power* (which is the power used by a cell when there are transitions on the input pins).
- **Net power** that shows the impact of net loading when a driver is switching, that is, $1/2 * \text{capacitive loading} * (\text{voltage level})^2 * \text{toggle rate}$.

Dynamic power is the sum of *internal power* and *net power*; while total power is the sum of *dynamic power* and *leakage power*.

So, when performing Dynamic Power Analysis (DPA), you estimate the total power consumed by a design to identify the power peaks that can break the functionality of the design and segregate the highly active modules. There are following two basic approaches to DPA:

- Toggle Count Analysis (TCA) that generates a toggle count profile capturing the design activity and analyzes the histograms of the activity over time using the xeDebug DPA GUI.

Palladium XP supports analysis of normal and weighted toggle counts to enable identification of areas for detailed power analysis. For details, refer to [RTL Toggle Count Analysis](#) (that uses normal toggle counts) and [Weighted Toggle Count \(WTC\) Analysis](#).

- Detailed power analysis using power profiles generated by the RTL Compiler that in turn uses Toggle Count Format (TCF) files generated by Palladium XP. For details, refer to [RTL Power Analysis](#) section (that discusses the flow for performing power analysis for RTL designs) and [Gate-Level Power Analysis](#) section (that discusses the flow for performing power analysis for gate-level netlists).

Palladium XP also enables you to generate Switch Activity Interchange Format (SAIF) files for analyzing power using third party tools. This chapter describes the process of generating SAIF files in the [Generating Backward SAIF Files for Power Analysis by Third-Party Tools](#) section, but does not covers the process of reading SAIF files using third party tools because that depends on the tool being used. Cadence does not recommend using the RTL Compiler to read SAIF files.

Considerations when Using DPA

- DPA operations are supported in both ICE and SA modes. However, in the SA mode, DPA support is limited to the DUT part; the testbench part is not supported. Consequently, you can perform DPA operations in SA mode only after the design has been swapped to the hardware.
- DPA does not work for designs compiled in 0.5X mode.
- DPA can be used only with the FullVision vision mode.
- DPA is not supported with uncontrolled clocks.
- You require a special license to use the Palladium XP DPA functionality. This license includes access to five licensed features for different DPA functions, an RTL Compiler license for power estimation capability, and a SimVision license. Contact the Cadence representative to obtain the necessary licenses. For more information, refer to the [DPA Licenses](#) section given later in this chapter.

Identifying Activity-Based Power Analysis Components

For DPA, you need the following three components:

- A netlist that describes the interconnectivity of cells. The netlist can be in either of the following two forms:
 - ❑ RTL files that are generated after synthesizing the RTL designs using the Palladium XP HDL Compiler or IXCOM compiler
 - ❑ Gate-level netlist that also has the associated device library

The following sections of the chapter discuss the different methods of DPA based on the type of netlist you have:

- ❑ [Analyzing Power Consumption of RTL Designs](#)
 - [RTL Toggle Count Analysis](#)

UXE User Guide

Analyzing Power Consumption of the Designs

- RTL Power Analysis
- Analyzing Power Consumption of Gate-Level Netlists
 - Normal Toggle Count (NTC) Analysis
 - Weighted Toggle Count (WTC) Analysis
 - Gate-Level Power Analysis

Note: In the SA mode, a gate-level netlist generated using a synthesis tool other than Palladium XP HDL Compiler is not supported. Therefore, analysis of power consumption of gate-level netlists is not supported in SA mode.

- An activity file that describes the toggle rate and logic state of nets.

The RTL Compiler accepts the activity file in TCF format. The TCF files contain toggle count, high count, and net name for every net.

Note: You can also generate SAIF files for third party tools used for power analysis.

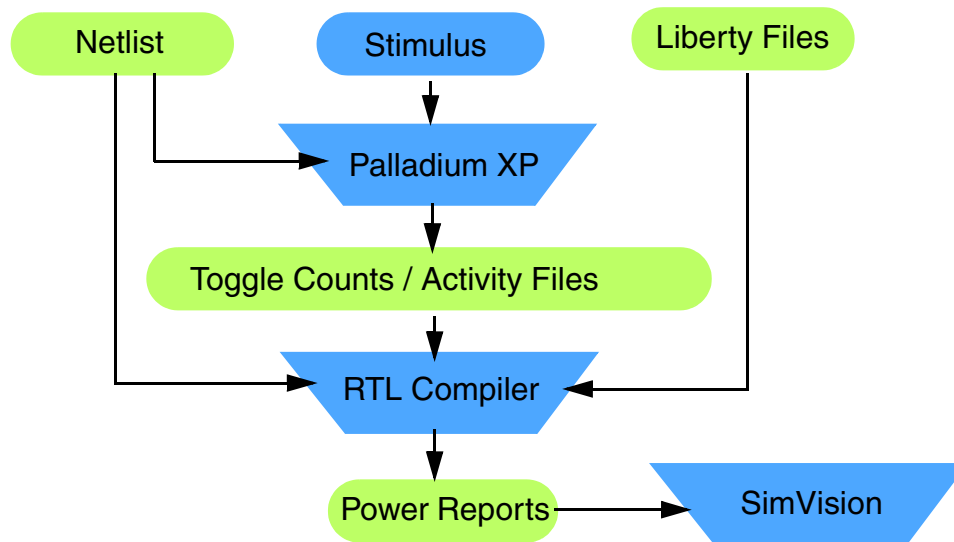
- A library that describes how much power is consumed when an input causes a cell to switch its outputs.

The information about power consumed by a cell when inputs and outputs change state is recorded in device-specific library (`lib`) files that are often in Liberty file format. The library file contains detailed information, such as leakage power for every combination of input to output, and internal power consumed by each type of transition on every input pin.

Important

If you do not have device-specific library files, use the power estimation techniques discussed under the Analyzing Power Consumption of RTL Designs section.

Figure 15-1 Components Needed for Power Analysis



Palladium XP and the RTL Compiler operate concurrently to generate the final power estimation reports.

Palladium XP emulator runs at a high speed compared to simulation. It uses the real world stimulus to capture the activity information, and eventually generate either TCF or SAIF format files over a specific time range. You can also use Palladium XP for direct analysis of switching activity with Toggle Analysis tools.

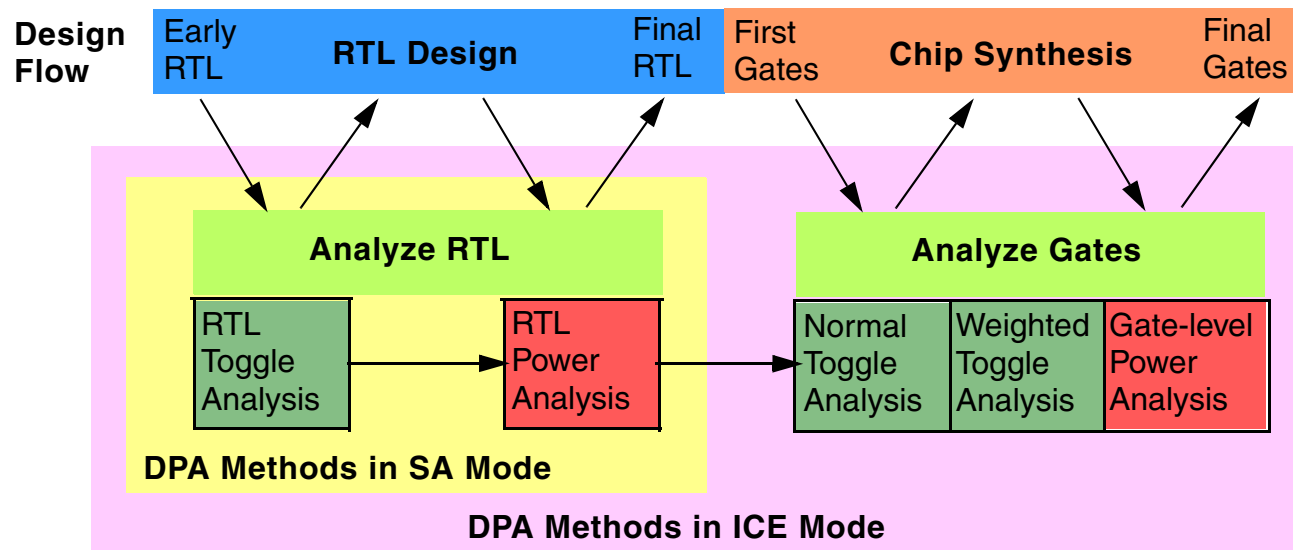
The RTL Compiler uses the activity information captured by Palladium XP and the switching information contained in the device-specific library files to estimate power.

Using DPA Methods in Design Flow

You can follow a multi-step approach to analyze power consumption, starting from initial results derived through RTL toggle count analysis to final results derived through gate-level power analysis.

At each phase of design flow, you can analyze the power information and feed it back into the design. As you progress through the design flow, the accuracy of results derived during power analysis and the processing time involved increase.

Figure 15-2 Design and Power Analysis Flow



Performing DPA in Offline Mode

DPA operations are generally time consuming. Therefore, to free up the emulator hardware for other designs or users, performing DPA operations in offline session is recommended.

You can create a *.phy file in online session using the database `-prepareoffline` command or the *Prepare Offline* button on the *Control* tab of xeDebug GUI. The *.phy file contains FullVision information that can be used to get waveforms for all signals in the design and perform DPA operations.

Note: You can still perform DPA operations in online session. However, you need to first select an existing *.phy file using the `dpa -phyfile` command or the *Setup* tab of the xeDebug GUI. For example, the following set of commands enable you to create and then open the t1.phy file:

```
run 1000
# Specify the name of the .phy file as t1
database -open t1
# Create the t1.phy file
database -prepareoffline
# Open the t1.phy file
dpa -phyfile t1.phy
# other dpa commands
```

You can save more data to a *.phy file even after opening it. However, the DPA operations will not be able to examine the new data. You will have to issue the `dpa -phyfile` command again to save the new data in the *.phy file.

Analyzing Power Consumption of RTL Designs

Starting with an RTL design saves time by getting results earlier during the design development life cycle. However, RTL design analysis generates less accurate estimates as compared to gate-level netlists. This is because the netlist examined by Palladium XP might not match the synthesized netlist.

Power consumption of RTL designs can be analyzed using either of the following methods:

- [RTL Toggle Count Analysis](#)
- [RTL Power Analysis](#)

RTL Toggle Count Analysis

Toggle Count Analysis (TCA) is the most direct method for toggle analysis. This is possible because you can avoid the time-consuming synthesis step and do not require inputs, such as Liberty (.lib) format files that might not be available. The toggle count captured during TCA shows the total number of nets in the design that toggle between the `logic1` and `logic0` states at a specific cycle. Each cycle during a time window has a toggle count number.

TCA also captures the total number of nets in the design that have high values at a specific cycle. The high values represent a high count.



Only **“normal” toggle counts**, which show the raw number of internal nets switching at a time unit, are derived from RTL designs. Weighted toggle counts can be derived from gate-level netlists. For detailed information about the weighted toggle counts, refer to the [Weighted Toggle Count \(WTC\) Analysis](#) section.

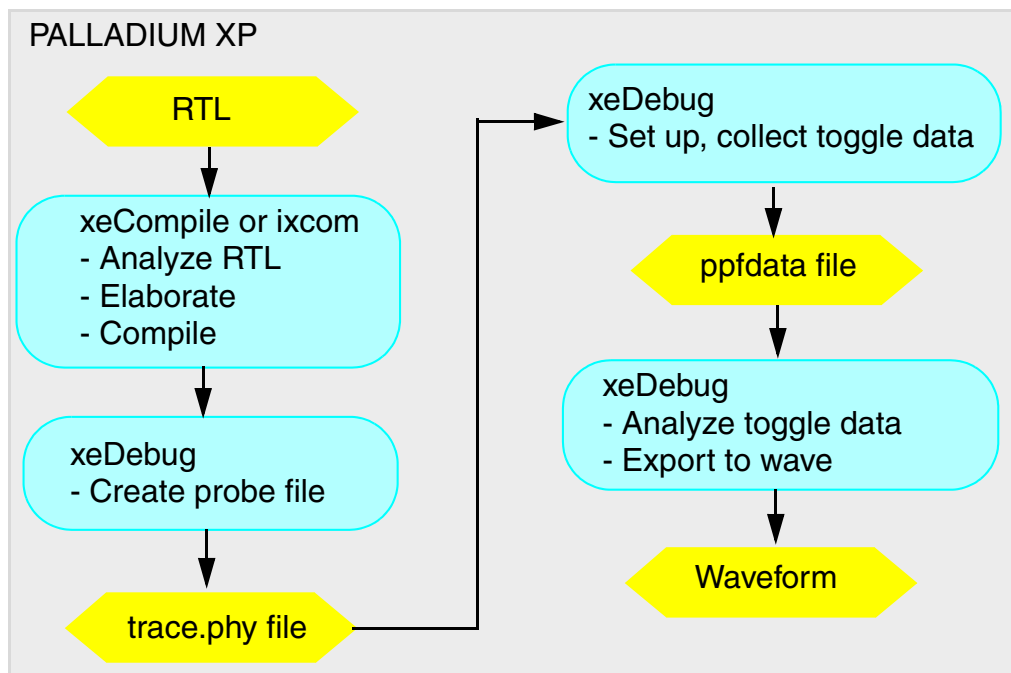
The captured toggle count and high count numbers are plotted as a histogram that represents the switching activity over a certain time. These histograms let you identify the areas of potential high power consumption. Both toggle count and high count correlate to the power consumption of the design. Large toggle count and high count values generally show a high power consumption.

When you perform TCA before doing a full power analysis using the RTL Compiler, you must isolate logic blocks or time regions with potential power problems. You can trim out the time

windows with low design activity and identify windows with high activity out of millions of design cycles. You then zoom into time segments of high activity for a detailed power analysis. Using toggle counts to identify active time windows is faster than using waveforms or VCD files. However, it is less accurate than estimating power consumption with the RTL Compiler.

Figure 15-3 shows the general flow of how an RTL design moves through the TCA process.

Figure 15-3 Process of Toggle Count Analysis for RTL Design



After compiling the design using the normal compile flow, invoke xeDebug in online session to run the emulation and create the *.phy file. Then, open an offline session of xeDebug to do toggle count analysis. The basic steps required for TCA are:

1. Selecting Instances to Collect Toggle Data
2. Collecting Toggle Count Data in an Offline Session
3. Analyzing Toggle Count Data in an Offline Session

Selecting Instances to Collect Toggle Data

Toggle counts are collected on a per instance basis. On each clock cycle, the following information is available in a set of bins:

- Number of nets in an instance that are high

UXE User Guide

Analyzing Power Consumption of the Designs

■ Number of nets in an instance that toggled

The software can automatically select a default set of instances for toggle count analysis or you can explicitly specify a set of instances using the following `dpa` command:

```
dpa -setupToggleCount -addinst [-top <n>] [-depth <n>] [-min <n>] <insts>
```

This command accepts a list of instance names, and adds the specified instances and all their children. If a dot (.) is specified for the instance name parameter, then toggle count data is collected for the entire design.

Use the `-top`, `-depth`, and `-min` options to filter children of each specified instance.

For example, the following command adds all the instances within the top seven levels below `dut`, the top instance:

```
dpa -setupToggleCount -addinst dut -depth 7
```

If no instances were selected before starting toggle count data collection, by default, the top 1000 largest instances are selected from the entire design. These instances can be at any level of the hierarchy, and the size of the instances should be more than one percent of the whole design. In most cases, this is adequate to cover all modules that might be of interest.

Collecting Toggle Count Data in an Offline Session

Use the following `dpa` command to generate toggle count data from the *.phy file and save the data in the <name>.ppfdata file that is required for performing toggle count analysis:

```
dpa -getToggleCount [-append] [-weighted] [-file <name>]
```

This command can be issued multiple times. For example, the following commands get 1000 cycles of the toggle count data and save it into the file named `ntc.ppfdata`:

```
dpa -tracewindow -start 0 -end 1000
dpa -getToggleCount -file ntc -append
```

Then, issue the following commands:

```
dpa -tracewindow -start 1000 -size 10000
dpa -getToggleCount -file ntc -append
```

The commands above get 10000 cycles of toggle count data and append it into the `ntc.ppfdata` file that now records 11000 cycles of toggle count data.

If this command is given without previously executing a `dpa -setToggleCount` command, the default values are used.

By default, the existing file data is overwritten every time new data is uploaded. To retain the toggle count data from previous uploads, specify the `-append` option.

UXE User Guide

Analyzing Power Consumption of the Designs

When analyzing power for RTL designs, only normal toggle count data is collected. Therefore, the `-weighted` option can be used only with gate-level netlists. For more information on weighted toggle counts, refer to the [Weighted Toggle Count \(WTC\) Analysis](#) section.

Analyzing Toggle Count Data in an Offline Session

Different instances in the design have separate histograms for toggle count and high count. You can compare the histograms of the different instances for better estimates of power patterns and identify the instances that consume the maximum power. You can also request either the average number of nets toggling for a time range or the cycle with largest number of toggling nets to find peak counts for peak analysis.

The `*.ppfdata` file and the histogram data it contains can be analyzed using the xeDebug GUI or the command prompt.

If you opt to analyze the data from the command prompt, open the `*.ppfdata` file using the following `tca` command:

```
tca -setFile <name>.ppfdata
```

The filename should use the absolute path if it is not in the current directory and should include the `.ppfdata` extension.

Then, you can use the following `tca` command options for the analysis:

```
tca -getAllInstanceNames [-maxDisplay <number>]
tca -getTimeRange
tca -getValues <list_of_inst_names> -start <start_time> [-end <end_time>]
[-type toggle|high] [-count ave|max]
tca -getTopNInst <number> [-start <start_time>] [-end <end_time>]
[-type toggle|high] [-count ave|max]
tca -getTopNPeak <number> [-start <start_time>] [-end <end_time>]
[-type toggle|high] [<list_of_instance_names>]
tca -export <waveform_name> [-start <time>] [-end <time>] [-format sst2]
[-filter <filtername><filter_arguments>}] <list_of_instance_names>
```

For detailed information about the syntax and options of the `tca` command, refer to the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

The waveform can be exported for viewing in SimVision using the `tca -export` command.

Note: In the SA mode, the waveform shows the time advances in unevenly steps. Also, when there are multiple toggle samples captured for the same point of time, all except the last

UXE User Guide

Analyzing Power Consumption of the Designs

sample are ignored. For example, consider the following table that shows the data captured in a waveform:

Time	0ns	0ns	0ns	5ns	10ns	10ns	30ns	30ns	35ns	50ns	80ns	100ns
Value	0	1	1	1	0	1	0	0	1	1	0	1

Here, only the following values are considered: 1 (at 0ns), 1 (at 5ns), 1 (at 10ns), 0 (at 30ns), 1 (at 35ns), 1 (at 50ns), 0 (at 80ns), and 1 (at 100ns). This shows that there were four toggles during the time window starting from 0ns to 100ns.

RTL Power Analysis

The RTL power analysis process depends on synthesizable RTL design, switching activity file and device-specific library files.

For RTL designs compiled in the SA mode, the IXCOM compiler inserts the instrumentation logic in the synthesized RTL files. This instrumentation logic is not readable by the RTL Compiler. Therefore, while analyzing power consumption of such designs, the RTL files that you provide to the RTL Compiler should be the original files that were specified to the IXCOM compiler.

The switching activity files for RTL power analysis can be in either TCF or SAIF format. Palladium XP supports generation of files in both these formats. However, if you want to use the RTL Compiler for power analysis, you must generate TCF files. Else, you can generate backward SAIF files to use with any third party tool for power analysis. For details related to SAIF files, see [Generating Backward SAIF Files for Power Analysis by Third-Party Tools](#) on page 739.

For generating the TCF files, Palladium XP needs the data file that contains input and output information dumped by the RTL Compiler after elaborating the RTL design, Palladium XP reads the data file and creates probes for collecting the toggle data. Generating the TCF files using the data file from the RTL Compiler ensures the following:

- Sets of all primary inputs and sequential outputs are complete
- Names created by Palladium XP match the names in the RTL Compiler
- Sequential outputs are collected from the correct level of hierarchy and not from the lowest level of cells in Palladium XP which do not exist in the RTL Compiler

Then, the RTL Compiler uses the TCF files for generating a power report. The information in the power reports is in terms of watts consumed by each hierarchical instance. As the power

report is based on TCF files, the power is reported over the time interval of each of the TCF files.

The data in the power report can be further processed using the `genPowerProfile` program that converts it into waveform format. This allows you to see the power reported for all TCF files in a single waveform using SimVision. There are waveforms for all the categories of power reported by the RTL Compiler – total, dynamic, internal, net, and leakage.

To use the DPA extension license, you must run the `dpawave` or `dpa_rc` scripts instead of directly running SimVision or RTL Compiler, respectively. For more details on the DPA licenses, refer to [DPA Licenses](#).

Note: RTL power analysis is not as accurate as gate-level power analysis because each tool must synthesize the RTL design and each synthesis results in a different collection of gates due to different algorithms.

In the RTL power analysis process, you can use the RTL Compiler either directly from the xeDebug tool or as a standalone application. Based on the way you choose to access the RTL Compiler, the steps for RTL power analysis vary. The following sections discuss the specific steps you need to perform in both the cases:

- [RTL Power Analysis Using RTL Compiler as Standalone Application](#)
- [Analyzing Power by Invoking RTL Compiler from xeDebug](#)

RTL Power Analysis Using RTL Compiler as Standalone Application

You can perform RTL power analysis by invoking the RTL Compiler as a standalone application to generate the power reports. The basic steps you need to follow in this case are discussed below and illustrated in the flowchart given in [Figure 15-4](#) on page 709.

1. Invoke the RTL Compiler for [Using DPA_RC Program to Read and Elaborate the RTL Design](#).
2. [Dumping Nets for DPA Probes Using DPA_RC and dump_dpa_info Command](#).
3. [Reading, Elaborating, and Compiling the RTL Design Using Palladium XP](#).
4. Invoke the xeDebug tool in **online** session and perform the following steps:
 - a. Download and run the design.
 - b. Create the `*.phy` file using the `database -prepareOffline` command.
 - c. Exit the xeDebug tool.

UXE User Guide

Analyzing Power Consumption of the Designs

5. Invoke the xeDebug tool in *offline* session and load the files from the *.phy subdirectory using the following command:

```
dpa -phyfile <phy_filename>
```

6. Using the following command, Adding DPA Probes for TCF Generation in RTL Flow based on the data file generated by the RTL Compiler in step 2 above:

```
dpa -readNetNames <rc_dpa_info_file_name> [<instanceName>]
```

7. Run Palladium XP to Generating TCF Files.



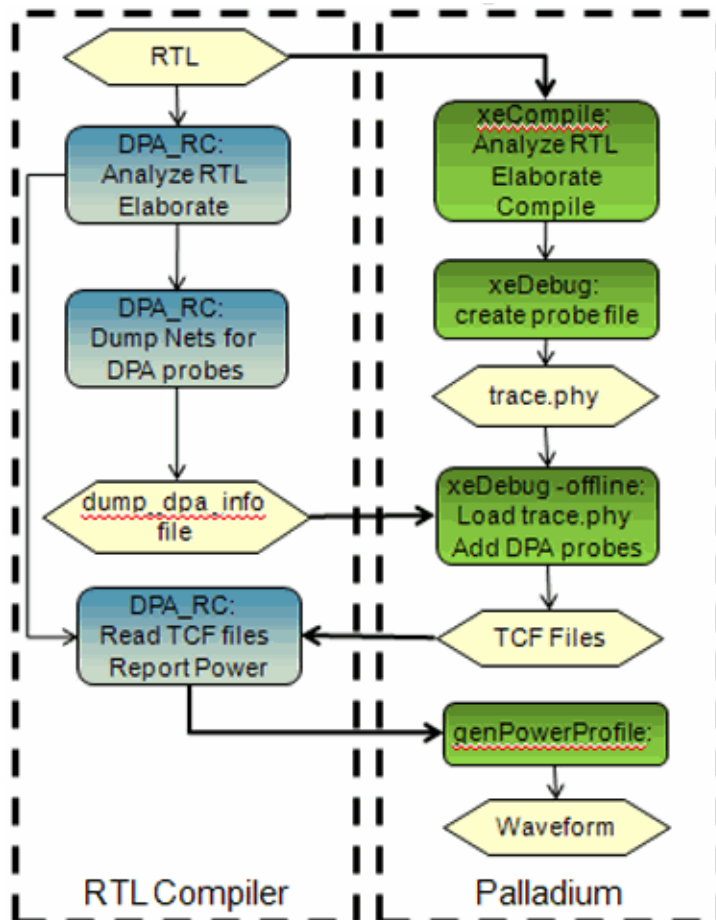
Tip

Timescale in TCF Files

The TCF files generated by Palladium XP use the fastest design clock timescale.

8. Run the RTL Compiler to Reading TCF Files and Generating Power Report of RTL Designs. The power report shows a profile of all library cells inferred during synthesis.
9. Generating Power Consumption Profiles in SST2 waveform format using the `genPowerProfile` program.
10. Viewing Waveforms Using SimVision.

Figure 15-4 RTL Power Analysis Flow Using RTL Compiler as Standalone Application



Using DPA_RC Program to Read and Elaborate the RTL Design

Use the `dpa_rc` command to start RC with the ability to read and elaborate the RTL design, do power estimation, and report power consumption.

Note: The DPA_RC program does not have full synthesis capability.

To use the `dpa_rc` command, you must first set the following environment variables:

```
setenv RC_HOME <rc_install_root>
setenv CDN_SYNTH_ROOT <rc_install_root>
```

The `dpa_rc` command uses the DPA_RC license by default. Alternatively, if you prefer to use an RTL Compiler license instead of the DPA_RC license, set the following environment variable:

```
setenv RC_LIC
```

UXE User Guide

Analyzing Power Consumption of the Designs

Then, use the following command:

```
dpa_rc {-f <Tcl_script>} [options]
```

Here, `{-f <Tcl_script>}` is a required parameter that expects the name of the Tcl script. You can set the following attributes in the Tcl script:

- `set_attribute lib_search_path`
- `set_attribute hdl_search_path`
- `set_attribute hdl_record_naming_style "%s.%s"`

Note: The `hdl_record_naming_style` argument is required for VHDL or mixed language designs.

- `set library`
- `set_attribute library`

For the `[options]` parameter, you can specify the following RTL Compiler command-line arguments:

- `read_hdl` - Enables the RTL Compiler to read the RTL design.
- `elaborate` - Enables the RTL Compiler to elaborate the RTL design.
- `synthesize -to_generic` - Enables the RTL Compiler to synthesize the RTL design to generate a generic netlist.

Dumping Nets for DPA Probes Using DPA_RC and dump_dpa_info Command

For accurate power estimations at RTL-level of design abstraction, the RTL Compiler should be able to read the Palladium XP generated TCF files readily. To accomplish this, use the following command in the RTL Compiler and generate a side file consisting of a list of primary inputs and sequential outputs with full hierarchical names in the RTL Compiler format:

```
dump_dpa_info -design <designTopInstance> -to_file <dump_dpa_info_filename>
```

Note: The hierarchy does not include the testbench path.

The `dump_dpa_info` command must be used after the `read_hdl` and `elaborate` commands.

All these commands are supported by the `dpa_rc` command.

The side file, which is a dump of the proper set of probe points, has the following format:

```
#nets for <designTopInstance>
<net1>
```

`<net2>`

The `<designTopInstance>` specified in the side file is the RTL Compiler DUT name. Also, the net names in the side file are full hierarchical net names of the RTL Compiler format. This is because unlike Palladium XP that stores both synthesizable testbench and DUT, the RTL Compiler stores only the DUT part.

Palladium XP can use this file with the `dpa -readNetNames` command while adding probes for collecting toggle data.

Reading, Elaborating, and Compiling the RTL Design Using Palladium XP

To read, elaborate, and compile the RTL design in ICE mode, use the normal Palladium XP compile flow discussed in [Chapter 3, “Compiling Designs for In-Circuit Emulation with xeCompile.”](#) To use the SA mode instead, refer to the process discussed in [Chapter 4, “Compiling Designs for Simulation Acceleration Using IXCOM.”](#)

Adding DPA Probes for TCF Generation in RTL Flow

Use the following command in Palladium XP to add into the probe list the nets/pins that appear in the RTL Compiler-generated side file:

```
dpa -readNetNames <rc_dpa_info_file_name> [<instanceName>]
```

The above command requires you to specify the name of the side file generated using the `dump_dpa_info` command in the RTL Compiler. You can also specify the top instance name.

Since different tools present RTL names differently, the names in the side file generated by the RTL Compiler follow the tool-specific naming convention. While reading the net and pin names from this side file, Palladium XP finds a corresponding match in its database.

To reduce the probability of name mismatch issue, you should use the following Palladium XP and RTL Compiler settings for naming conventions:

- In Palladium XP, at the time of HDL synthesis, use the `-keepRtlSymbol` and `-keepVhdlCase` command options with the `hdlSynthesize` command.
- In the RTL Compiler, specify the VHDL record name style as `%s.%s` using the following command:

```
set_attr hdl_record_naming_style "%s.%s"
```

Based on the side file generated by the RTL Compiler, Palladium XP collects the toggle count and high value count information of the nets and pins, and dumps out the activity files in TCF format.

Generating TCF Files

A TCF file can exist in either of the following forms:

- Regular TCF file - It contains the net/pin names and toggle counts.
- Master/slave TCF file - This format uses the following types of files:
 - A single master file to map long design net/pin hierarchical names to shorthand names.
 - Multiple slave files that contain only the activity information comprising the shorthand name, toggle count, and high count.

During TCF file generation, the definition of high counts differ in both ICE and SA modes. In the ICE mode, high count is the count of FCLK cycles when a signal has value 1. However, in the SA mode, it is the summary of time intervals in which the signal has value 1.

Using the master/slave TCF files results in better performance (less storage space and faster writing/reading speed) than regular TCF files. If you only want one time period power consumption value, use a regular TCF file; if you want to have multiple time periods power consumption values, use master and slave TCF files. The RTL Compiler can accept the TCF files in either form to calculate power consumption for a specified time period.

The process of generating TCF files involves the following steps:

1. Specifying the File Format: This step includes the following sub-steps:
 - a. File Formats of TCF Files — Regular, Master, and Slave
 - b. Generating TCF Files Based on Design or Module
 - c. Generating TCF Files for Specific Segment Windows
2. Uploading the Generated TCF Files

Specifying the File Format

To generate TCF files, first specify the file format using the `dpa -outfile -tcf` command. Following is the full syntax of this command:

```
dpa -outfile -tcf <tcfFileName> [-instance <instanceName>] [-masterTCF]
[-slaveTCF] [-noregularTCF] [-segment <n>% | <cycleNumber>] [-start <startnum>]
[-end <endnum>]
```

File Formats of TCF Files — Regular, Master, and Slave

By default when `-outfile -tcf` is used, the regular TCF is chosen. Thus, if you want to generate master/slave files, you need to explicitly specify the `-mastertcf` and/or

UXE User Guide

Analyzing Power Consumption of the Designs

`-slavetcf` options. If you do not want to generate regular TCF files, you must disable the default setting by using the `-noregulartcf` option.

To generate only a regular TCF file:

```
XE> dpa -outfile -tcf <tcfFileName>
```

To generate only a master TCF file:

```
XE> dpa -outfile -tcf <tcfFileName> -mastertcf -noregulartcf
```

To generate only a slave TCF file:

```
XE> dpa -outfile -tcf <tcfFileName> -slavetcf -noregulartcf
```

To generate regular, master, and slave TCF files:

```
XE> dpa -outfile -tcf <tcfFileName> -mastertcf -slavetcf
```

The master, slave, and regular TCF files can be generated at the same time if `-mastertcf`, and `-slavetcf` options are specified with the `dpa` command, and no `-noregulartcf` option is found. The order of specifying the `-mastertcf` and `-slavetcf` options does not matter.

Generating TCF Files Based on Design or Module

A TCF file can be design-based or module-based.

If the `-instance` option is not specified with the `dpa` command, a design-based TCF file is generated. Such TCF files use full hierarchical names.

A module-based TCF file is generated if the `-instance` option is specified with the `dpa` command as following:

```
dpa -outfile -tcf <tcfFileName> -instance <instanceName>
```

Module-based TCF files use the names starting from the instance name specified with the `-instance` option.

Generating TCF Files for Specific Segment Windows

Each TCF file covers a time period, which is called a segment window. The segment window must be described as the number of cycles to capture in each TCF slave or regular file.

Specify the segment window using the following command:

```
dpa -outfile -tcf <tcfFileName> -segment <cycleNumber>
```

The `-segment` option generates:

- slave or regular TCF files with one file per segment. Each file contains activity information about `<cycleNumber>` cycles.
- a TCF log file that records the start and end time information in text format for each file or segment. It also generates a `segment.log` file which contains the headers from all of the TCF log files.
- a toggle/high count profile in SST2 format.

 Important

A small `<cycleNumber>` value results in more TCF files and finer granularity of power measurement.

Use the `-start` and `-end` options to specify a time window for which the TCF file should be generated. If these options are not specified, TCF file will be generated for the whole time window captured in the `*.phy` file.

Uploading the Generated TCF Files

Use the following command to actually generate and upload the TCF files:

```
dpa -upload
```

When TCF files are generated, corresponding TCF log files are also created. The TCF log files record the percentage of time the net is in the logic 1 or high state and the number of toggles which occur within the time range. The TCF files contain a header which gives the start and end time of the segment, the number of samples and the number of signals. The header also records the `togglesPerCycle`, `highsPerCycle`, `toggleRate`, and `highRate` parameters.

Assume the following parameters in a TCF file:

`<length>` is the duration

`<nSignals>` is the signals(pins or nets) number

`<nToggles>` is the total toggle counts of all signals in a given duration

`<nHighs>` is the total high count of all signals in a given duration

Then,

```
<toggleRate> = <nToggles>/(<length>*<nSignals>)  
<togglesPerUnitTime> = <nToggles>/<length>  
<highRate> = <nHighs>/(<length>*<nSignals>)  
<highsPerUnitTime> = <nHighs>/<length>
```

UXE User Guide

Analyzing Power Consumption of the Designs

The `*_segment.log` file contains the headers from all of the TCF files. The `segment.log` file is used in the generation of the power profile database. It is also possible to use the `genPowerProfile` command to generate a waveform from the `*_segment.log` file which shows the `toggleRate`, `togglesPerUnitTime`, `highRate`, and `highsPerUnitTime` parameters as waves.

```
genPowerProfile -tcfLogFile <*_segment.log> -databaseName <SHM database name>
```

Reading TCF Files and Generating Power Report of RTL Designs

TCF files are generated in compressed format and the RTL Compiler reads the compressed files directly, without expanding them.

The RTL Compiler reads the TCF files with the `read_tcf` command, and creates power reports for each time segment using the `report power` command and options.

The sample script given below illustrates the usage of these commands:

read_tcf \$tcfDir/dpa_Instance.tcf.m.gz

```
for { set i 1 } { $i <= $count } { incr i 1 } {
    reset_power_analysis
    read_tcf $tcfDir/dpa_inst.tcf.s.$i.gz
###--- one time only execution
    if { $i == 1 } {
        synthesize -to_generic
        build_rtl_power_models -clean_up_netlist // needs to be done only once
        report power -verbose -depth $pwrpt_depth >> $tcfDir/dpa_inst.tcf.s.$i.log
        report power -tcf_summary
        set rtlpe_factors [get_attr lp_rtlpa_scale /des*/*]
        //calculate scaling factor with first tcf file to be used with the rest
        of the reports
    }
###--- all remaining
    if { $i > 1 } {
        set_attr -quiet lp_rtlpa_scale $rtlpe_factors /des*/*
###--- Reporting hierarchical power
        report power -verbose -depth $pwrpt_depth >> $tcfDir/dpa_inst.tcf.s.$i.log
        //append to slave segment log file
        report power -tcf_summary
    }
```

Generating Power Consumption Profiles

The power report can be further processed and analyzed into a power consumption profile of waveform format with the `genPowerProfile` program. For the syntax and description of `genPowerProfile`, refer to the *Run-Time Commands* chapter in the *UXE Command Reference Manual*.

This program reads multiple report files created by the RTL Compiler and the TCF segment log files, and converts the individual power reports into a single waveform covering all of the time ranges in the segment log files. The waveform shows values for leakage, internal, net, dynamic, and total power for each hierarchical block in the design. The values are derived from the power report file and the time range comes from the segment log files.

The power consumption profile provides information in watts consumed by each hierarchical instance. It is based on the TCF files; therefore, the power is reported over the time interval of each of the TCF files. This profile can be viewed in SimVision.

The `genPowerProfile` command can also be used to generate a waveform directly from the TCF files. Use the `-tcfLogFiles` option as described in the *UXE Command Reference Manual*. The waveform will show high probabilities and average toggle counts across time in SST2 format.

Figure 15-5 shows the waveform generated based on the following command:

```
genPowerProfile -tcfLogFiles <logFileName> -databaseName <db_name>
```

You will see two elements in each profile - the average toggle count per time segment; and the average high count per time segment. As shown in the SimVision display below, data was captured for 240,000ns. In the top display, where `-segment 10%` was used, the information is divided into 10 segments, each showing the average toggle count over a 24,000ns period. In the lower display, the information is divided into segments of length 1,000ns, resulting in 240 toggle count measurements, each averaging over a period of 1,000ns.

UXE User Guide

Analyzing Power Consumption of the Designs

Each power report file generated by the RTL Compiler contains the power consumption data of the corresponding time segment. For each time segment, a log file containing toggle information is also generated. When using the separate file use model, specify the following `genPowerProfile` command to generate a power profile for each time segment from the power report files that match the specified filename pattern:

```
genPowerProfile -segmentLogFile <segment_log_file_from_TCF_generation>  
                -reportFileNamePattern <fileNamePattern> [-honorTimeScale]  
                [-honorFrequency] [-databaseName <db_name>]
```

Viewing Waveforms Using SimVision

Use the `dpawave` command to open the SST2 database with SimVision waveform viewer.

Following is the syntax of the `dpawave` command:

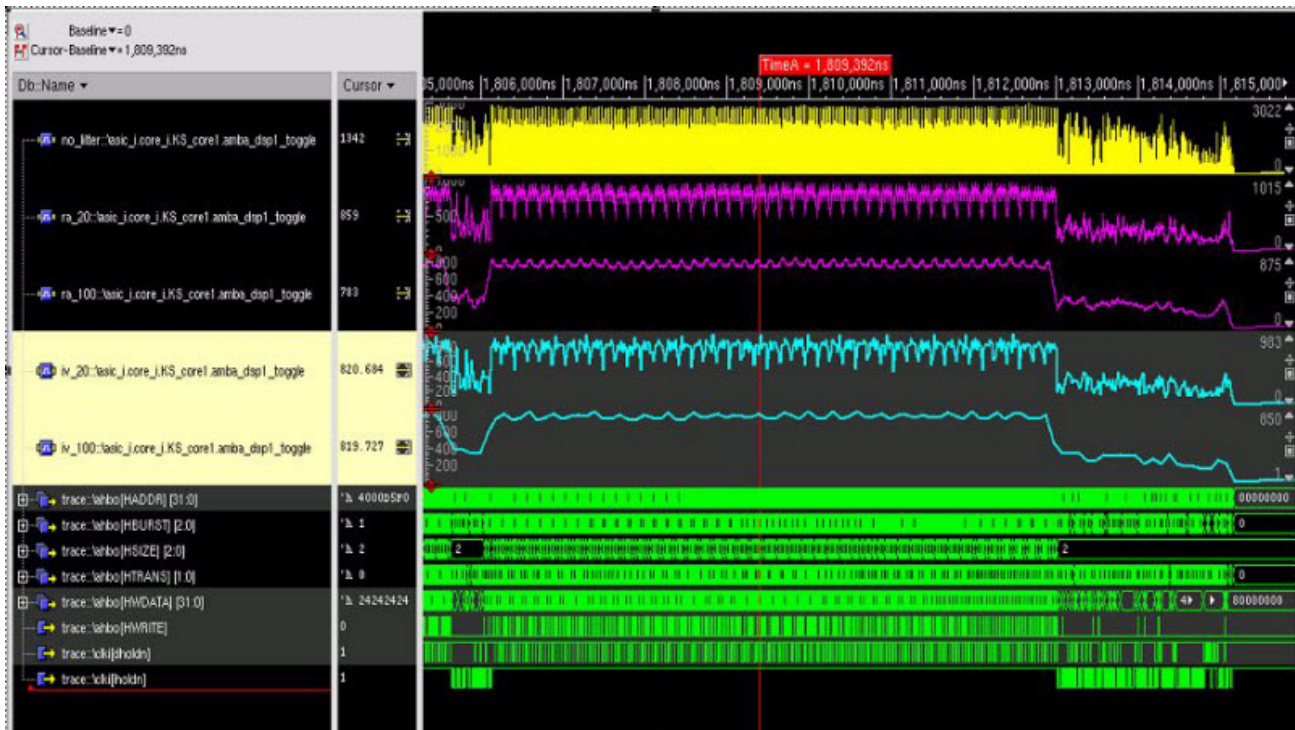
```
dpawave [database <databaseName>] -input <svcf_file> | -waves <database>]
```

Here,

- `database <databaseName>` argument requires the name of the directory that contains the SST2 database.
- `-input <svcf_file>` argument specifies the name of the `svcf_file` that contains the SimVision Command Script to restore waveform signals.
- `-waves <database>` argument opens specified SST2 database.

Note: To use the `dpawave` command, you must use the DPAPXP_SV license.

Figure 15-6 Power Consumption Profile in SimVision



Analyzing Power Consumption of Gate-Level Netlists

Generally a multi-step approach is followed in analyzing power consumption from gate-level netlists. The gate-level netlist can be created by the RTL Compiler or any other synthesis tool.

Note: SA mode does not support analysis of power consumed by gate-level netlists.

You can achieve early results by using toggle analysis that involves collection of toggle rates over different time ranges for different logic blocks. Analyzing both normal and weighted toggle counts help you to identify areas that require detailed power estimation. You can do a final power analysis on the post-layout structural netlist with complete clock trees, wire load models, parasitic, and layout related timing.

Gate-level netlists enable you to achieve various levels of accuracy. As you progress through the sequence, the accuracy of results increases as does the processing time.

This section covers the following methods of power analysis that you can perform with gate-level netlists:

- Normal Toggle Count (NTC) Analysis

- [Weighted Toggle Count \(WTC\) Analysis](#)
- [Gate-Level Power Analysis](#)

Normal Toggle Count (NTC) Analysis

Analysis of normal toggle counts can be used to get an early look at the design activity. It provides the most direct method for analyzing the power consumption. It can be done using just Palladium XP and does not require the RTL Compiler.

Gate-level normal toggle count analysis is the same as RTL toggle count analysis. The only difference is that it begins with a gate-level netlist. For detailed information about the process, refer to the [RTL Toggle Count Analysis](#) section.

Weighted Toggle Count (WTC) Analysis

The **weighted toggle count** (WTC) is the sum of the weights on internal nets switching at a time unit. WTC gives a higher toggle rate to a net transition that consumes more power. For instance, a state transition on a memory address pin consumes more power than a transition on a NAND gate.

In WTC analysis, each internal net that is driven by a gate is assigned a weight based on its power arc. The assignment of weights is done using the power profile database that comprises the power reports derived from the Liberty format (`lib`) files by the RTL Compiler and the `powerdb.index` file. If a net driven by gate is not described in any RTL Compiler-generated power report, the net is assigned a default base weight 1 for WTC analysis. You can change the power characteristics of a specific net to affect its weight by defining either `cell_power_factor`, `gate_power`, or `power_file` keyword in the `powerdb.index` file. For information about these keywords and usage details, refer to the following sections:

- [Adding cell_power_factor Keyword to the powerdb.index File](#)
- [Adding gate_power Keyword to the powerdb.index File](#)
- [Adding power_file Keyword to the powerdb.index File](#)

The basic steps for WTC analysis are:

1. Generate power reports from the Liberty format files using the RTL Compiler and create the `powerdb.index` file using a text editor. The `powerdb.index` file labels each of the Liberty files as either gate or blackbox. For procedural details, refer to the [Creating a Power Profile Database](#) section.

UXE User Guide

Analyzing Power Consumption of the Designs

2. Analyze, elaborate and compile the gate-level netlist. Refer to the [Applying the Power Database](#) section.

3. Start the emulation and create the probe file.

4. Select the modules for which toggle data needs to be collected:

```
dpa -setupToggleCount -addinst [-top <n>] [-depth <n>] [-min <n>] <insts>
```

5. Collect toggle count data, using the following command:

```
dpa -getToggleCount -weighted [-append] [-file <name>]
```

This creates a file named `<name>.ppfdata`.

Note: When the UXE compiler implements the ASIC cells in the design, it can create internal nets that do not exist in the design, below the ASIC level. When collecting normal toggle count data, the toggle count includes those internal nets. However, when collecting weighted toggle counts, those internal nets will be excluded.

6. Use the xeDebug GUI or `tca` command to analyze the toggle count data as with normal toggle counts. Refer to the [Analyzing Toggle Count Data in an Offline Session](#) section.

7. [Viewing Waveforms Using SimVision](#).

Creating a Power Profile Database

Palladium XP uses the information from the power profile database directory to add weights and netlist information to the emulator database. To create a power profile database, you must have the following:

- Power models (Liberty format `*.lib` files) for ASIC cells and macros in the design
- Palladium XP models for the ASIC cells and macros

Note: Macros are complex cells, such as memories, which have complex power models.

Then, synthesize the design for power profile database using a tool like RTL Compiler. This process differs from synthesizing to Palladium XP primitives using xeCompile because a power profile does not include the testbench. When you have a testbench, you must first create a power table, and then use xeCompile to synthesize your testbench, ASIC cell and macro Palladium XP models, followed by compile.

Note: A power table needs to be generated only once for every Liberty file that is used in the design.

Generating a Power Table File

For generating a power table file, first collect all power models (Liberty format `*.lib` files) for the ASIC cells and complex cells in the design. The power models describe the power characteristics of the cells that can be of two types: simple and complex. Simple cells include logic gates such as AND, OR, XOR, INV, BUF, and so on.

Then, sort the `.lib` files based on cell type. A file can contain multiple cell descriptions, but simple cells and complex cells must not be mixed in a file.

If a cell is marked as a simple cell, the weighted toggle count of that cell is computed in a faster manner. If a cell is marked as a complex cell, the weighted toggle count of that cell is computed in a slower but more accurate manner. Cadence recommends that you mark simple combinatorial gates as simple cells and mark memory as complex cells.

When the cells are sorted, execute the `create_power_table` command as following to generate the power table file from the `*.lib` file:

```
create_power_table <*.lib_file_name> <power_table_file_name>
```

The generated power table is encrypted.

Creating a Power Database File (powerdb.index)

Save all power table files in a single directory that becomes your power profile database. In that same directory, create a `powerdb.index` file that describes how the Palladium XP compile process should treat the cells in each power table (`*.rpt`) file, that is gates or blackbox. Each power table file must be described in the `powerdb.index` file by using the following format for the entries:

- `gates=<power_table_file_for_simple_cells>`
- `black_box=<power_table_file_for_complex_cells>`

Note: Cadence recommends to mark power table files containing complex cells or memories as blackbox, and power table files containing combinatorial and simple sequential cells as gates.

Following is an example of a `powerdb.index` file:

```
# power database
gates = standard_cell.rpt
black_box = memory_cell1.rpt
black_box = another_macro.rpt
```

UXE User Guide

Analyzing Power Consumption of the Designs

You can also add the `cell_power_factor`, `gate_power`, and `power_file` keywords into the `powerdb.index` file for changing the power characteristics of a specific pin of a cell to affect its weight during WTC analysis. The sections below describe the syntax of these keywords.

Adding cell_power_factor Keyword to the powerdb.index File

You can use the `cell_power_factor` keyword to adjust the scaling ratio of power numbers in a specified power table file. This keyword helps you to define the power values of both simple gate and blackbox cells.

The `cell_power_factor` keyword can be specified using either of the following syntax:

- `cell_power_factor=<Power_Table_File>:<Cell_Name>:<Value>`

Here, `<Power_Table_File>` is either `<power_table_file_for_simple_cells>` or `<power_table_file_for_complex_cells>` which is included in the `powerdb.index` file.

The syntax indicates that all power numbers of the cell, `<Cell_Name>`, defined in the power table file, `<Power_Table_File>`, will be multiplied with the specified `<Value>`.

For example, you have a power table file `tutorial.rpt` that includes cells `nand2`, `nor2`, and `xor2`. Now, you want to reduce the power number of these cells to half of the original value while collecting the WTC data. You can define the following in the `powerdb.index` file for this purpose:

```
# power database
gates=tutorial.rpt
cell_power_factor=tutorial.rpt:nand2:0.5
cell_power_factor=tutorial.rpt:nor2:0.5
cell_power_factor=tutorial.rpt:xor2:0.5
```

An error is reported in either of the following conditions:

- ❑ The specified power table file does not exist.
- ❑ The specified cell does not exist in the specified power table file.
- `cell_power_factor=<Power_Table_File>:*:<Value>`

This is similar to the syntax explained above. However, the `*` (asterisk) between `<Power_Table_File>` and `<Value>` in this syntax means ***all*** cells. Therefore, when this syntax is used, the `cell_power_factor` acts on all simple gate and blackbox cells defined in the specified power table file.

UXE User Guide

Analyzing Power Consumption of the Designs

For example:

```
# power database
gates=tutorial.rpt
cell_power_factor=tutorial.rpt:*:0.5
```

The above example definition enable you to reduce the power numbers of all cells defined in `tutorial.rpt` to half of the original value while collecting WTC data.

If the specified power table file does not exist, an error is reported.

Note:

- If you define multiple `cell_power_factor` keywords in the `powerdb.index` file, the definition with the specified `<Cell_Name>` will be given the highest priority and be implemented. For example, the `powerdb.index` file contains the following definitions:

```
# power database
gates=tutorial.rpt
cell_power_factor=tutorial.rpt:nand2:3.0
cell_power_factor=tutorial.rpt:*:2.5
```

Here, the definition `tutorial.rpt:nand2:3.0` will be implemented for `nand2` because it specifies the cell name `nand2`. Consequently, the power numbers of `nand2` will be multiplied with `3.0`, not `2.5`.

Instead of definitions illustrated in above example, if you define the following, the power numbers of `nand2` will be multiplied with `2.5` because `nand2` is included in `tutorial.rpt`:

```
# power database
gates=tutorial.rpt
cell_power_factor=tutorial.rpt:*:2.5
```

- If you define multiple `cell_power_factor` keywords for the same cell in the same power table file, the last definition will be implemented.

For example:

```
# power database
gates=tutorial.rpt
cell_power_factor=tutorial.rpt:nand2:0.5
cell_power_factor=tutorial.rpt:nand2:3.0
```

Here, the last statement, `cell_power_factor=tutorial.rpt:nand2:3.0`, will be implemented.

Adding gate_power Keyword to the powerdb.index File

You can use the `gate_power` keyword to define the power values of only simple gate cells for WTC analysis.

The `gate_power` keyword can be specified using either of the following syntax:

- `gate_power=<Lib_Name>:<Cell_Name>:<Pin_Name>:<PowerValue>`

This means, assign the `<PowerValue>` as the power value for the pin, `<Pin_Name>`, of the cell, `<Cell_Name>`, in the library, `<Lib_Name>`.

For example, to assign power value 15.0 to the pin Y of the cell nand2 in the library ASIC_LIB, you can define the following in `powerdb.index` file:

```
gate_power=ASIC_LIB:nand2:Y:15.0
```

An error is reported in either of the following conditions:

- ☐ The specified pin does not exist in the specified cell.
- ☐ The specified cell does not exist in the specified library.
- ☐ The specified cell is a blackbox cell defined in an imported power table file.

- `gate_power=<Lib_Name>:<Cell_Name>:*:<PowerValue>`

This means, assign the `<PowerValue>` as the power value for **all** pins (as defined using the asterisk) of the cell, `<Cell_Name>`, in the library, `<Lib_Name>`.

For example, the following definition will assign 15.0 as power value for every pin of the cell nand2 in the library ASIC_LIB:

```
gate_power=ASIC_LIB:nand2*:15.0
```

In the two formats discussed above, you have to specify `<Lib_Name>`. Therefore, you should know the `<Lib_Name>` in your compilation script, for example the library name specified in the `refLib` command in the compilation script. `<Lib_Name>` is required to resolve the ambiguity when a `<Cell_Name>` appears in multiple libraries. If a cell does not have a duplicate in other libraries, you can use an asterisk (*) as `<Lib_Name>` to avoid specifying a real name, as shown in the syntax below:

- `gate_power=*<Cell_Name>:<Pin_Name>:<PowerValue>`

This means, assign the `<PowerValue>` as the power value for the pin, `<Pin_Name>`, of the cell, `<Cell_Name>`.

An error is reported in either of the following conditions:

- ☐ The specified pin does not exist in the specified cell.

UXE User Guide

Analyzing Power Consumption of the Designs

- ❑ The power value is zero or negative.
- ❑ The specified cell is not a simple gate cell.
- ❑ The specified cell exists in multiple libraries. In this case, you need to specify the `<Lib_Name>`.

■ `gate_power=*<Cell_Name>:<PowerValue>`

This means, assign the `<PowerValue>` as the power value for **all** pins of the cell, `<Cell_Name>`.

An error is reported in either of the following conditions:

- ❑ The power value is zero or negative.
- ❑ The specified cell is a blackbox cell defined in an imported power table file.
- ❑ The specified cell exists in multiple libraries. In this case, you need to specify the `<Lib_Name>`.

Note: If you define multiple `gate_power` keywords in the `powerdb.index` file, the definitions that specify the `<Pin_Name>` along with the `<Cell_Name>` are given the highest priority as compared to those that specify only the `<Cell_Name>`. For example, when you define the following statements, the pin `Y` of `nand2` will have power value `15.0`, and other pins in `nand2` will have power value `10.0`:

```
gate_power=ASIC_LIB:nand2:Y:15.0
gate_power=ASIC_LIB:nand2:*:10.0
```

Adding power_file Keyword to the powerdb.index File

The `power_file` keyword can import a text-format file that includes power values of blackbox cells and simple gate cells for WTC analysis.

■ `power_file=<Power_File_Name>`

Here, `<Power_File_Name>` is a text-format file.

For the content of the `<Power_File_Name>`, you can refer to the `powerdb.dump` file in the `dbFiles` directory. This is a text-format file that includes the entire power data collected in the internal database after running `dbgPrepare` using the following command:

```
dbgPrepare <lib_name> <proto_name> -checkpowerdb
```

You can copy the contents of the `powerdb.dump` file to a new file, modify the content, and then, import the new file using the `power_file` keyword in the `powerdb.index`

UXE User Guide

Analyzing Power Consumption of the Designs

file. Run the `dbgPrepare <lib_name> <proto_name>` command to overwrite the old values in the internal power database with the power data from the imported file.

You can also write your own power file. However, make sure that the following mandatory content is added to your power file in the same order:

```
cell name : RAM_4096x32
QTDB library name : MACRO_LIB
simple gate : false
arc # : A[0]      A[0]      10.000000
arc # : CLK      CLK      4430.000000 (!PGEN & RETN) & (!CEN & ! (WEN) ) & (!EMA[2])
& (!EMA[1]) & (!EMA[0])
...
```

Note: The keywords in bold are case-sensitive. Use `arc` to describe a power characteristics for a pin, which is equivalent to a module port. The specified pin should exist in the port list of a module definition, but should not be a wire or register.

For a simple gate cell, one `arc` has two elements: the *pin* name and *power(nW)*, which is the power for the specified *pin*.

For example:

```
cell name : cell buf1
...
...
arc(s) : pin      power(nW)
arc #0 : Y        5.033000
```

For a blackbox cell, one `arc` can have four elements: *pin(from)*, *pin(to)*, *power(nW)*, and *when*. However, *when* is optional, and the other three are mandatory.

For example:

```
cell name : RAM_4096x32
...
...
arc(s) : pin(from)  pin(to)    power(nW)    when
arc #13 : CLK      CLK      4430.000000    (!PGEN & RETN) & (!CEN &
! (WEN) ) & (!EMA[2]) & (!EMA[1]) & (!EMA[0])
```

An error is reported in either of the following conditions:

- ☐ The specified cell does not exist in the specified library.
- ☐ The specified library is not imported by the design.
- ☐ The specified pin does not exist in the specified cell.
- ☐ The power value is zero or negative.

- ❑ Cell type (described by simple gate) is inconsistent with the definition in the imported power tables.

Important Considerations When Using Keywords in powerdb.index File

Keep the following considerations in mind when using either `cell_power_factor`, `gate_power`, or `power_file` keyword in the `powerdb.index` file:

- After compiling the design, if you change the definitions of any of these keywords in the `powerdb.index` file, you need to run the `dbgPrepare` command to bring the changes into affect. Following is the syntax of the `dbgPrepare` command:

```
dbgPrepare <lib_name> <proto_name>
```

To find the correct values for `<lib_name>` and `<proto_name>`, access the PDB subdirectory in the design directory. The PDB subdirectory contains files with names of the format `<lib_name>.<proto_name>.<file_extension>`. For example, in the file named `LIB.testbench.dadb`, `LIB` is the `<lib_name>` and `testbench` is the `<proto_name>`. So, you should run the following command:

```
dbgPrepare LIB testbench
```

The `dbgPrepare` command re-computes the weights for all nets and saves them into a power profile database that can be used for WTC analysis later.

- When you use either the `gate_power` or `power_file` keyword for a cell, if the cell is directly included in an imported power table file, its cell type (simple gate or blackbox gate) should not be changed in the definition of `gate_power` or `power_file`. If the cell is not directly included in any imported power table, you must choose the correct cell type based on the design; otherwise, the WTC data will be incorrect.
- In most cases, only NTC analysis is applied to RTL designs because Liberty format files might not be available, and so, power table files cannot be used. However, you can apply WTC analysis for RTL designs as well by adding either the `gate_power` or `power_file` keyword to the `powerdb.index` file. Following are the necessary steps:

- a. Add the following `compilerOption` command to your compilation script:

```
compilerOption -add {powerdb <powerdb_directory>}
```

The `<powerdb_directory>` parameter in the above syntax is the power profile database directory name specified with either the full path or path relative to the working directory. This power profile database directory will contain the `powerdb.index` file.

- b. Create the `powerdb.index` file in the power profile database directory specified using the `<powerdb_directory>` parameter in the above step.

UXE User Guide

Analyzing Power Consumption of the Designs

Add the `gate_power` or `power_file` keyword definitions to this `powerdb.index` file.

- c. Compile the design.
- d. At run time, use the following `dpa` command to collect the WTC data:

```
dpa -getToggleCount -file <ppfdata_file> -weighted
```

Applying the Power Database

To define your power database to the UXE compiler, use the `compilerOption` command to specify the power database:

```
compilerOption -add {powerdb <powerdb_directory>}
```

The directory name can be either the full path or relative to the working directory. This directory must contain all power reports and the `powerdb.index` file. Along with specifying this `compilerOption` parameter, you can include standard compilation options, and specify the precompile and compile options as normal.

With a gate-level netlist and the power database, you can collect and view weighted toggle counts to get a more accurate view of when the design is likely to be using high amounts of power. You can then generate TCF or SAIF files for these times.

The TCF files are input to the RTL Compiler along with your gate-level netlist and descriptions of Liberty format cells to determine actual power consumption for those periods. The SAIF files are input for third party tools used for power analysis. For details related to SAIF files, see [Generating Backward SAIF Files for Power Analysis by Third-Party Tools](#) on page 739.

Gate-Level Power Analysis

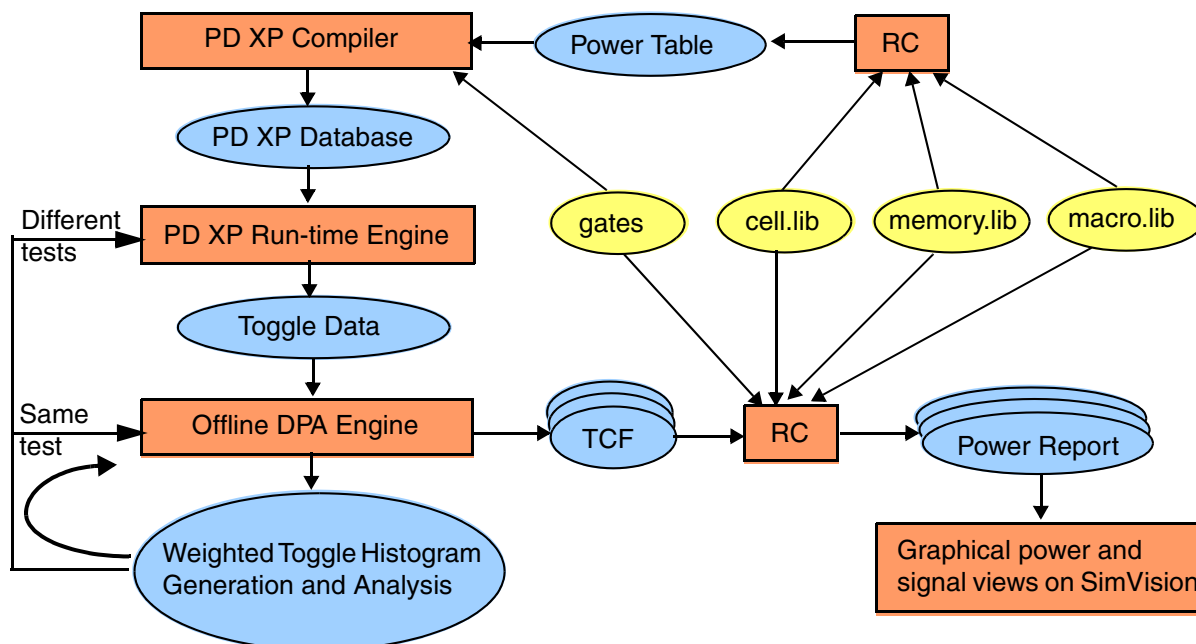
Gate-level power analysis improves accuracy over RTL by starting all tools with the same post-synthesis netlist. This form of power analysis benefits from using a netlist derived from the physical layout.

When starting with a gate-level netlist, use the RTL Compiler to create a gate report. The Palladium XP uses this gate report to determine the crucial probe points. The probes are used in a debug session to create activity files in either TCF or SAIF format.

The RTL Compiler reads the TCF format activity files and generates a power report for each one. The `genPowerProfile` program converts the individual power reports into a single waveform showing how internal, net, and dynamic power change over the full time range.

To prepare for DPA using gate-level netlist, you need to perform a few steps differently at compile time in Palladium XP than for the standard compile. [Figure 15-7](#) shows the entire design flow, starting with the power models, through the RTL Compiler synthesis, in the upper portion of the flow chart.

Figure 15-7 Process of DPA for Gate-Level Netlists



To generate a power profile, you can invoke the RTL Compiler either directly from the xeDebug tool or as a standalone application. Based on the way you choose to access the RTL Compiler, the steps for gate-level power analysis vary. The following sections discuss the specific steps you need to perform in both the cases:

- Gate-Level Power Analysis Using RTL Compiler as Standalone Application
- Analyzing Power by Invoking RTL Compiler from xeDebug

Gate-Level Power Analysis Using RTL Compiler as Standalone Application

You can perform gate-level power analysis by invoking the RTL Compiler as a standalone application to generate the power reports. The basic steps you need to follow in this case are discussed below and illustrated in the flowchart given in [Figure 15-8](#) on page 732.

1. Read gates, elaborate, and compile using Palladium XP.

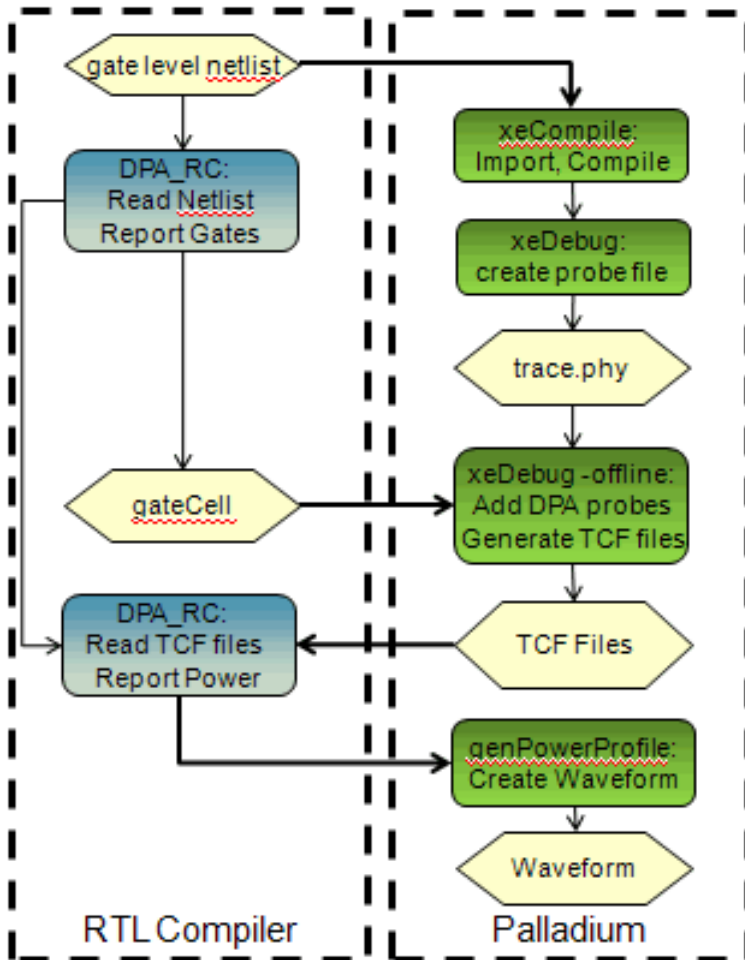
UXE User Guide

Analyzing Power Consumption of the Designs

2. Run the RTL Compiler to generate a gate report that shows a profile of all library cells inferred during synthesis. Refer to the [Generating Low-Power Key Information Using the RTL Compiler](#) section.
3. Invoke the xeDebug tool in **online** session and perform the following steps:
 - a. Download and run the design.
 - b. Create the *.phy file using the `database -prepareOffline` command.
 - c. Exit the xeDebug tool.
4. Invoke the xeDebug tool in offline session and load the files from the *.phy subdirectory using the following command:

```
dpa -phyfile <phy_filename>
```
5. During TCF-file generation for gate-level netlists, after specifying the file format ([Specifying the File Format](#)), you must specify the probe set using the method described in the [Adding DPA Probes for TCF Generation in Gate-Level Flow](#) section.
6. Run Palladium XP to [Generating TCF Files](#). Also, when using a third party tool for power analysis, you can generate backward SAIF files instead of generating TCF files. For detailed information, refer to the [Generating Backward SAIF Files for Power Analysis by Third-Party Tools](#) section.
7. Run the RTL Compiler to [Reading TCF Files and Generating Power Report of Gate-Level Netlist](#). The power report shows a profile of all library cells inferred during synthesis.
8. [Generating Power Consumption Profiles](#) in SST2 waveform format using the `genPowerProfile` program.
9. [Viewing Waveforms Using SimVision](#).

Figure 15-8 Gate-Level Power Analysis Flow Using RTL Compiler as Standalone Application



Generating Low-Power Key Information Using the RTL Compiler

Use the following RTL Compiler command to generate the low-power gates information:

```
report gates
```

Note: This command should be used after the Palladium XP compile process, which creates a `dbFiles` directory to store the `gateCell` file. Confirm that the `dbFiles` directory has write permissions.

Executing this command generates a report (`gateCell` file) listing all the gates, the number of instances in the design, and the total area for all these instances is generated. At the end of the gate report, the RTL Compiler shows the total number of instances and the area for all

the sequential cells, inverters, buffers, logic, and timing-models, if any. To get a report on the total combinational area, add the logic, inverter, and buffer area numbers.

Adding DPA Probes for TCF Generation in Gate-Level Flow

Use the `dpa -addinstoutputpin` command to probe the I/O pins listed in the `gateCell` file.

```
dpa -addinstoutputpin <instanceName>
```

For a gate-level netlist, the `dpa -addinstoutputpin <instanceName>` command adds the I/O pins of even the innermost sub-instances of this instance. If the power database is established (refer to the [Creating a Power Profile Database](#) section), it also adds output pins of ASIC cells, and I/O pins of hardmacro of this instance.

Reading TCF Files and Generating Power Report of Gate-Level Netlist

The RTL Compiler reads all TCF files with the `read_tcf` command, and creates power reports for each time segment by using the `report power` command and options.

The sample script given below illustrates the usage of these commands. It assumes that you have a single master and 5 slave TCF files.

```
read_tcf $tcfDir/dpa_inst.tcf.m.gz

for { set i 1 } { $i <= $count } { incr i 1 } {
    reset_power_analysis
    read_tcf $tcfDir/dpa_inst.tcf.s.$i.gz
    ###--- Reporting hierarchical Power
    report power -verbose -depth $pwrpt_depth >> dpa_inst.tcf.s.$i.log
    report power -tcf_summary
}
```

Analyzing Power by Invoking RTL Compiler from xeDebug

The xeDebug tool has the capability to invoke and use the RTL Compiler for necessary power analysis. You can do this by specifying the relevant `dpa` command options on the xeDebug command line or by using the [PowerProfile Tab](#) of the xeDebug GUI.

Note: This is an alternative to using the `genPowerProfile` UNIX command described in the [Generating Power Consumption Profiles](#) section.

If you choose to use the xeDebug command line, perform the steps described below.

1. Invoke the xeDebug tool in **online** session and perform the following steps:

- a. Download and run the design.

 Important

When the xeDebug tool is invoked and CPF data is found in the `./QTDB/cpf` directory, CPF-aware DPA is applied automatically. For detailed information about CPF-aware DPA, refer to the [Analyzing Power Consumption with CPF](#) section.

- b. Create the `*.phy` file using the database `-prepareOffline` command.
- c. Exit the xeDebug tool.

2. Invoke the xeDebug tool in **offline** session and load the files from the `*.phy` subdirectory using the following command:

```
dpa -phyfile <phy_filename>
```

3. Specify a time range to work on using the following command:

```
dpa -tracewindow -start <start_time> -end <end_time>
```

Note: If this command is not issued, the whole time range in the `*.phy` file will be used.

4. Specify the required RTL Compiler libraries along with the access path using the `dpa -setupRC -libList` command.

Note: If you want to specify the list of RTL Compiler libraries and path to access those using separate command options, you can use the `-libPath` option along with the `-libList` option.

The `dpa -setupRC` command has multiple options. You can use the command with the required options either one by one in separate command lines, or combine more than one option in a single command line. For the syntax and description of the various `dpa -setupRC` command options, refer to the `dpa` command section in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*. For command usage, you can refer to the examples given later in this section.

5. Specify whether the RTL Compiler must be invoked for RTL design flow or gate-level netlist flow using either of the following commands:

```
dpa -setupRC -rtlFlow
```

or

```
dpa -setupRC -gateFlow
```

By default, the RTL Compiler is invoked for gate-level netlist flow.

For a sample script file used in RTL design flow, refer to the [Example: RTL Power Analysis Using RTL Compiler from xeDebug Tool](#) section for a sample script file.

6. Specify the RTL design filename(s) to the RTL Compiler if `-rtlFlow` option was specified in the previous step, or gate-level netlist name(s) if `-gateFlow` option was specified. Use the following command to specify the complete path to the design files/netlist:

```
dpa -setupRC -designFile {<file_name_list>}
```

Note: If you want to specify the design filename and path to access those using separate command options, you can use `-designPath` option along with the `-designFile` option.

7. Specify the top-level module name using the following command:

```
dpa -setupRC -topDesign <top_module_name>
```

This is an optional command in the ICE mode, but mandatory in the SA mode. The `<top_module_name>` is directly passed to the RTL Compiler. The RTL Compiler needs the name of the top module in various situations such as following:

- When you specify the `dpa -setupRC -servers {<machine_list>}` command to use the Super Threading feature of the RTL Compiler.
- When you specify the `dpa -setupRC -rtlFlow` command so that the RTL Compiler elaborates the design in RTL flow.

In the ICE mode, for most designs, if you do not specify the `<top_module_name>`, the UXE software will try to search a top module and pass it to the RTL Compiler. If you get an RTL Compiler error during the execution of the `dpa -genPowerProfile` command, you must examine the error messages in the log file generated by the RTL Compiler.

Note: The RTL Compiler error messages are not listed here assuming that you know about the RTL Compiler.

If the RTL Compiler error message indicates that the error is related to top module name, you should specify the `dpa -setupRC -topDesign` command and re-run the `dpa -getPowerProfile` command.

8. Specify the instance name for which you want to do the power calculations using the following mandatory command:

```
dpa -setupRC -instance <instance_name>
```

The RTL Compiler uses the specified instance name to create probe nets for TCF generation.

9. If you are analyzing power for a gate-level netlist, you can also use the Super Threading feature of the RTL Compiler to compute multiple power reports in parallel. For enabling this feature, which is *off* by default, specify the following command:

```
dpa -setupRC -servers {<machine_list>}
```

UXE User Guide

Analyzing Power Consumption of the Designs

Refer to the [Example: Gate-Level Power Analysis Using Super Threading Feature of RTL Compiler from xeDebug](#) section for a sample script file.

10. Specify the `dpa -setupPowerProfile -database <name>` command to identify the name of the power profile.

For the syntax and description of the various `dpa -setupPowerProfile` command options, refer to the `dpa` command section in the *Run-Time Commands* chapter in the *UXE Command Reference Manual*.

11. Specify the subdirectory name in which the TCF files generated during power profile computation will be saved. This can be done using the following command:

```
dpa -setupPowerProfile -tcfDir <name>
```

This is an optional step. By default, the directory for the TCF files is named as `tcf` and is created within the design directory. However, if the power profile name specified with the `-database <name>` option contains a directory name, like `test1/topInst`, the `tcf` subdirectory will be created within the specified root directory, like `test1/tcf`.

12. Set the number of segment windows for which toggle counts need to be calculated using the `dpa -setupPowerProfile {-segment <n>% | <cycleNumber>}` command.

The `<n>%` parameter specifies the percentage of the total analysis time period that is to be covered in one segment window. Else, the `<cycleNumber>` parameter specifies the number of emulation cycles to be covered in one segment window. If the `-segment` option is not set, the segment window is set to 100 percent by default. This means the whole time range in the `*.phy` file will be computed as one segment window.

One master TCF file be generated for the total analysis time period. In addition, one slave TCF file will be generated for each identified segment window. For example, if you set the segment window to 20 percent, 5 slave TCF files and 1 master TCF file will be generated for the complete time range specified in the `*.phy` file. For more information about TCF files, refer to section [Generating TCF Files](#) on page 712

13. Specify the `dpa -genPowerProfile` command to generate the power profile waveform based on the parameters specified using the `dpa -setupRC` and `dpa -setupPowerProfile` command options.

While generating power profile waveform in ICE mode, you must use the same time unit as the signal waveform to ensure that both waveforms are synchronized. To use a common time unit, set the `timeUnit` run-time parameter using the `xeset` command. In SA mode, you need not be concerned about this difference because the tool can automatically get the correct time scale.

To get more accurate power consumption data, you can refine the data reported for a specific time range in the previous run by giving the following command:

UXE User Guide

Analyzing Power Consumption of the Designs

```
dpa -genPowerProfile -refine
```

If refinement process needs to be run on existing power consumption data, the `-refine` option should be explicitly specified for a time range that is less than the previous time range. Also, the design file and `*.phy` file must be the same as the last run. Usually, only two commands, `dpa -tracewindow -start <start_time> -end <end_time>` and `dpa -setupPowerProfile -segment`, need to be changed.

For example, user *A* got unexpectedly low-power consumption data for time range 1000 to 2000ns. As the power consumption data is a average value, user *A* does not know what happened during this time range. So, to get more accurate results for this time range, user *A* can first specify the time range with the `dpa -tracewindow` command as 1000 and 2000. Then, specify the segment window as 20% using the `dpa -setupPowerProfile -segment` command. Finally, execute the `dpa -genPowerProfile -refine` command to generate power consumption data with smaller granularity, such as [1000 1199], [1200 1399] and so on. As a result, more accurate data will be merged to the previously generated data for time range 1000 to 2000 in the waveform database.

Refer to the [Example: Gate-Level Power Analysis with Refinement of Power Consumption](#) section for a sample script file.

14. View the power profile waveform in SimVision.



Tip

When you repeat the power analysis process for the same design, some steps might be skipped if the values of the corresponding command options were not changed during the second or subsequent run.

Example: RTL Power Analysis Using RTL Compiler from xeDebug Tool

Assume, user *A* has a file `trace.phy` that has been generated for a whole time range of 0 to 999. Using the `ppf.qel` script illustrated below, user *A* can generate 5 slave TCF files and one master TCF file under the `./pwrpt/tcf` directory. Each slave file will cover 160 cycles time range. The RTL Compiler script and log file, and the list of probed nets used for TCF generation will be automatically generated under the `./pwrpt/tmp` directory. The power profile data will be created in the `./pwrpt/dut.shm` directory and can be viewed using SimVision.

ppf.qel

```
debug .  
dpa -phyfile trace.phy  
dpa -tracewindow -start 20% -end 100%
```

UXE User Guide

Analyzing Power Consumption of the Designs

```
dpa -setupRC -libList {./lib/*.lib} -designFile src/dut.v -rtlFlow -topDesign dut
dpa -setupRC -instance dut -depth 10
dpa -setupPowerProfile -database pwrpt/dut -segment 20%
dpa -genPowerProfile
```

Example: Gate-Level Power Analysis Using Single Thread RTL Compiler from xeDebug

Assume, user *B* has a file `trace.phy` that has been generated for a whole time range of 0 to 999. Using the `ppf.qel` script illustrated below, user *B* can generate 50 slave TCF files and one master TCF file under the `./pwrpt/tcf` directory. Each slave file will cover 20 cycles time range. The RTL Compiler script and log file, and the list of probed nets used for TCF generation will be automatically generated under the `./pwrpt/tmp` directory. The power profile data will be created in the `./pwrpt/dut.shm` directory and can be viewed using SimVision.

ppf.qel

```
debug .
host -offline trace.phy
dpa -tracewindow -start 0 -end 100%
dpa -setupRC -libPath {./lib} -libList {tutorial.lib RAM.lib}
dpa -setupRC -designFile src/dut.vg -gateFlow -topDesign dut
dpa -setupRC -instance dut -depth 10
dpa -setupPowerProfile -database pwrpt/dut -segment 2%
dpa -genPowerProfile
```

Example: Gate-Level Power Analysis Using Super Threading Feature of RTL Compiler from xeDebug

Assume, user *C* has a file `trace.phy` that has been generated for a whole time range of 0 to 999. Using the `ppf.qel` script illustrated below, user *C* can generate 50 slave TCF files and one master TCF file under the `./pwrpt/tcf` directory. Each slave file will cover 16 cycles time range. The RTL Compiler script and log file, and the list of probed nets used for TCF generation will be automatically generated under the `./pwrpt/tmp` directory. The power profile data will be created in the `./pwrpt/dut.shm` directory and can be viewed using SimVision.

Note the `dpa -setupRC -servers` command in the `ppf.qel` script below. The RTL Compiler will use the machines named `srv1`, `srv2`, `srv3`, and `srv4` for computing the power reports in parallel.

ppf.qel

```
debug .
```

UXE User Guide

Analyzing Power Consumption of the Designs

```
dpa -phyfile trace.phy
dpa -tracewindow -start 20% -end 100%
dpa -setupRC -libList {./lib/*.lib} -designFile src/dut.vg -gateFlow -topDesign dut
dpa -setupRC -instance dut -depth 10
dpa -setupRC -servers {srv1 srv2 srv3 srv4}
dpa -setupPowerProfile -database pwrpt/dut -segment 2%
dpa -genPowerProfile
```

Example: Gate-Level Power Analysis with Refinement of Power Consumption

Assume, user *D* has a file `trace.phy` that has been generated for a whole time range of 0 to 999. Using the `ppf.qel` script illustrated below, user *D* can generate 15 slave TCF files and one master TCF file under the `./pwrpt/tcf` directory. The slave TCF files start with index 1 to 5 covering 200 cycles time range in each file. The slave TCF files starting with index 6 to 15 cover 10 cycles time range in each file. The RTL Compiler script and log file, and the list of probed nets used for TCF generation will be automatically generated under the `./pwrpt/tmp` directory. The power profile data will be created in the `./pwrpt/dut.shm` directory and can be viewed using SimVision.

ppf.qel

```
debug .
host -offline trace.phy
dpa -tracewindow -start 0 -end 100%
dpa -setupRC -libPath {./lib} -libList {tutorail.lib RAM.lib}
dpa -setupRC -designFile src/dut.vg -gateFlow -topDesign dut
dpa -setupRC -instance dut -depth 10
dpa -setupPowerProfile -database pwrpt/dut -segment 20%
dpa -genPowerProfile
dpa -tracewindow -start 30% -end 40%
dpa -setupPowerProfile -database pwrpt/dut -segment 10%
dpa -genPowerProfile -refine
```

Generating Backward SAIF Files for Power Analysis by Third-Party Tools

Two types of SAIF files are supported – library forward SAIF and backward SAIF. Palladium XP reads the library forward SAIF file containing the state-dependent and path-dependent (SDPD) directives and generates the backward SAIF file containing the SDPD switching activity for the third party tool used for power analysis.

The process of generating backward SAIF files involves the following steps:

1. Loading the Library Forward SAIF File into Palladium XP.
2. Specifying the Signals That Need to be Inserted in the Backward SAIF File.
3. Defining the Scope of Design Hierarchy Search for Instances.
4. Specifying the Time Windows for Backward SAIF File Generation.
5. Generating the Backward SAIF File.

Loading the Library Forward SAIF File into Palladium XP

Use the following command to load the SDPD directives from the library forward SAIF file into Palladium XP:

```
dpa -saif -load <saif_file_name> [-module <module_name_pattern>]
```

The library forward SAIF file defines a set of modules. Palladium XP runs a search for all instantiations of these modules in the entire design hierarchy and performs calculations on those instances to identify the toggle counts. Therefore, select the modules carefully to avoid any negative impact on the performance.

Inclusion of memory cells and IP cells is recommended. You can also include flip-flops because they are important to make accurate power estimation. However, inclusion of flip-flops can cause additional overhead because the number of flip-flops are usually large.

Including simple gates, such as AND, OR, and XOR, in the library forward SAIF file is not recommended. This is because they have too many instances and do not improve the power estimation accuracy significantly.

Specifying the Signals That Need to be Inserted in the Backward SAIF File

Signals inserted in the backward SAIF file include the set of instances that are selected based on the module names in the loaded library forward SAIF file.

In addition, if required, specify a set of normal signals selected using the following `dpa` commands:

```
dpa -addinstoutputpin <instance>
dpa -addinstff <instanceName> [-asicLibrary <.llib name> ... <.llib name>] /
    [-hardmacroLibrary <.llib name> .... <.llib name>]
dpa -addinstReg <instanceName> [-asicLibs <.libName> ....<.libName> ] /
    [ -hardmacroLibs <.libName> ....<.libName>]
```


These normal signals are written as `NET` in the backward SAIF file.

When generating the backward SAIF file, the two types of signals will be combined into the same design hierarchy. If an instance matches a module specified in the library forward SAIF file, then that instance is written to the backward SAIF file. It is possible that some of the internal nets also get selected as normal signals. In such cases, the internal nets will not be written to the backward SAIF file.

Specifying Instances to Generate Module-Based Backward SAIF Files

To generate module-based SAIF files, use the following command:

```
dpa -outfile -saif <saif_file_name> -instance <instanceName>
```

The `<instanceName>` specified in this command must be a subset of the instances selected in the library forward SAIF file. Module-based SAIF files use the names starting from the instance name specified with the `-instance` option.

When the `-instance` option is used, the third party tool can read from the netlist only that part which corresponds to the specified instance. Also, the generated backward SAIF file works with the partial netlist.

The signal names saved in the SAIF file are trimmed at the beginning to remove the instance name. For example, if you specified `-instance Top.A` option, the signal named `Top.A.n1` will be saved with name `n1` and the `Top.A` part will be removed.

Defining the Scope of Design Hierarchy Search for Instances

The signals are searched through the entire design hierarchy by default. To define a scope to restrict the search operation to only the specified signals, use the following command:

```
dpa -saif -addscope <instance_name>
```

Setting of scope saves the processing time and significantly reduces the size of the backward SAIF file.

Specifying the Time Windows for Backward SAIF File Generation

One backward SAIF file is generated for each time window. So, to generate multiple SAIF files, evenly split the time window into segments using the following command:

```
dpa -outfile -saif <saif_file_name> -segment {<cycleNumber> | <percent>}
```

Segments can be split based on either the number of cycles captured in the backward SAIF file or the percentage of the whole time window. For example, 10% means to split the time window into 10 segments.

You can also specify `-start` and `-end` options with the `dpa -outfile -saif` command to generate multiple SAIF files.

Generating the Backward SAIF File

Use the following command to generate the backward SAIF files:

```
dpa -upload
```

When SAIF files are generated, corresponding SAIF log files are also created. These log files record a summary of toggles and high probabilities, and the start/end time.

Parallel Computing for DPA Using Multiple CPUs

To improve the performance of DPA, the UXE software uses multithreading in its DPA computation engine to make use of multiple CPUs on a workstation.

The core computation engine uses variable number of CPUs, while the other part of the DPA computation engine uses fixed number of CPUs. The UXE software can use multiple hosts to do computation as described in the [Running DPA with Multiple Hosts](#) section. When using multiple hosts, the xeDebug process on the local host distributes the computation tasks to the fvCompute processes on multiple remote hosts. The core computation engine exists in the xeDebug process on the local host and the fvCompute process on the remote hosts.

In UXE 12.1 release, the xeDebug process can use up to five more CPUs in addition to the CPUs used by the core computation engine. While the fvCompute process can use up to 1 more CPU in addition to the CPUs used by the core computation engine. The number can change in future releases if the software architecture changes.

The core computation engine can detect the number of CPUs installed on a workstation and decide how many CPUs to use. If a workstation has eight or fewer CPUs, the core computation engine uses all of them. Otherwise, the core computation engine uses only eight CPUs. The number 8 is a preselected number that you can change by setting the following environment variable:

```
setenv CDN_FV_MAX_THREAD <number>
```

Note: In addition to the core computation engine, the UXE software uses multithreading for other features as well, such as computation of FullVision probes.

Running DPA with Multiple Hosts

DPA operations need a huge amount of computation. The computation work can be distributed to multiple hosts to speed up the process. You only need to specify a set of hosts to use. The software will then automatically start the fvCompute process on those hosts. The fvCompute process creates a socket connection to the xeDebug process on local host after start up. Then, the tasks are distributed from xeDebug to fvCompute through this socket connection. xeDebug detects load on the remote hosts and accordingly distributes the tasks. If there are no tasks to compute, the fvCompute process remains idle.

There are two ways to specify the set of hosts to use. You can use either the `rsh` remote execution command or LSF, which is described later.

Requirement on Remote Hosts

The remote hosts must satisfy the following conditions:

- The remote hosts must have UXE software installed on the same path as on the local host.
- The remote hosts must be able to access the design directory by the same path as on the local host.
- As the remote hosts are expected to carry out intensive computation, the CPU speed is critical. Generally, workstations with Intel/AMD CPU are much faster.

Specifying Multiple Hosts Using Commands

Use either of the following commands to specify multiple hosts:

- Specify the following `xeset mtHost XEL` command:

```
xeset mtHost "multi_host_specification_string"
```

- Specify the following UNIX environment variable:

```
setenv CDN_MT_HOST "multi_host_specification_string"
```

Note: The `xeset mtHost` command is given preference over the `CDN_MT_HOST` environment variable.

Specifying Multiple Hosts with rsh

To use `rsh`, the `multi_host_specification_string` is a list of host names separated with colon (:). Also, make sure that you can do `rsh` to the specified hosts without being asked for a password.

Note: If your IT environment does not allow remote login (`rsh`) without password, this method cannot be used.

Specifying Multiple Hosts with LSF

You need to define a dedicated queue for DPA purpose. There should be no task per host, and task per user limitation on the queue. The `fvCompute` processes will not exit after they are started and will remain idle if there is no computation task. If there is a limitation on the queue, other users will not be able to start `fvCompute` after there are some `fvCompute` processes running in the queue. The `fvCompute` process will exit automatically when their parent `xeDebug` process exits.

To use LSF in DPA on the Palladium XP platform, both LSF and `xeDebug` need to be set up. The configuration of LSF is typically done by the system administrator. Therefore, this document assumes that the basic LSF settings are already in place, and concentrates mainly on the setting up of `xeDebug` to use LSF.

The `multi_host_specification_string` should follow the following format that consists of three parts separated by a colon punctuation mark:

`LSF:<number_of_hosts>:<bsub_command>`

where,

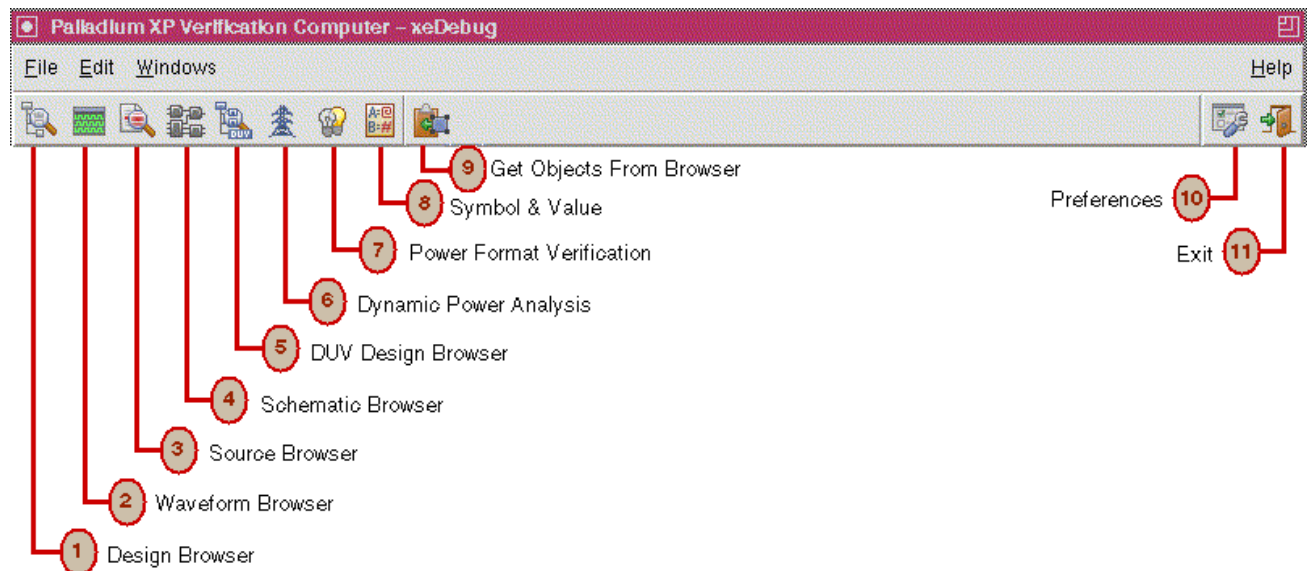
- The first part of the string is the text `LSF` which is used as the keyword.
- The second part is a numeric value that indicates the number of hosts that will be used for the computation work.
- The third part is a command string for the `bsub` command. It can include any character except for ':', which is used as a separator for the LSF string. You can add any options/arguments in this command string. For example, add the `bsub -q my_dpa_que` command string to specify the `bsub` queue name. You can also specify the complete path for the `bsub` command using the `/<LSF_install_dir>/bsub -q my_dpa_queue` command string.

Analyzing Power Consumption Using xeDebug GUI

The xeDebug GUI can be used to collect information during a design run, and analyze the results for the dynamic power consumed by the design. The xeDebug tabs and windows used for DPA are discussed in the following sections.

After completing the design compile, start an xeDebug session and download the design database using the *Setup* tab. If you have a DPA license and the compiled design has power information, the *Power Tower* icon (as shown in [Figure 15-9](#)) is enabled on the toolbar. Click this icon to bring up the DPA GUI (as shown in [Figure 15-10](#)).

Figure 15-9 Toolbar with Power Tower Icon for Dynamic Power Analysis



Note: In addition to needing a separate DPA license to access the *Power Tower* icon, if the license server is not running, or insufficient DPA licenses are available, the *Power Tower* icon will not be enabled.

The DPA GUI has the following tabs:

- Setup Tab - Enables you to select the `.phy` file on which the DPA operations will be performed.
- TCA Preparation Tab - Enables you to set up the parameters for collecting toggle count data. This is equivalent to using the `dpa -setupToggleCount` command.
- TCA Tab - Enables you to analyze the toggle count data once a `*.ppfdata` file has been generated. This is equivalent to using the `tca` command.

- DPA Probe Tab - Enables you to add DPA probes for generating TCF or SAIF files.
- TCF Tab - Enables you to generate TCF files independently.
- SAIF Tab - Enables you to load forward SAIF files and/or generate backward SAIF files.
- PowerProfile Tab - Enables you to generate power profile based on TCF files.

Each tab in the DPA GUI has a set of three general buttons that function as *minimize window*, *maximize/unmaximize window*, and *close window*. These additional buttons are provided because some operating systems do not provide these on the top-right of the DPA window.

Setup Tab

When using the xeDebug GUI, Cadence recommends that you invoke an offline session for the DPA operations. An offline session requires selection of a `.phy` file in the *Offline Debugging* group box of the xeDebug *Setup* tab (refer to [Figure F-25](#) and [Figure F-29](#) of [Appendix F, “Using the xeDebug GUI”](#)). This `.phy` file is used for waveform generation and DPA operations.

When you open the *Dynamic Power Analysis* window by clicking the *Power Tower* icon from the xeDebug toolbar, the *Setup* tab for DPA operations is displayed (as shown in [Figure 15-10](#)). This tab allows you to select a `.phy` file if you do not want to work on the `*.phy` file that was selected when opening the offline session.

Note: Selecting a `.phy` file on the DPA *Setup* tab is equivalent to using the `dpa -phyfile <phyFileName>` command on the xeDebug command prompt. If no `.phy` file is opened for DPA operations, the remaining tabs in the *Dynamic Power Analysis* window remain disabled.

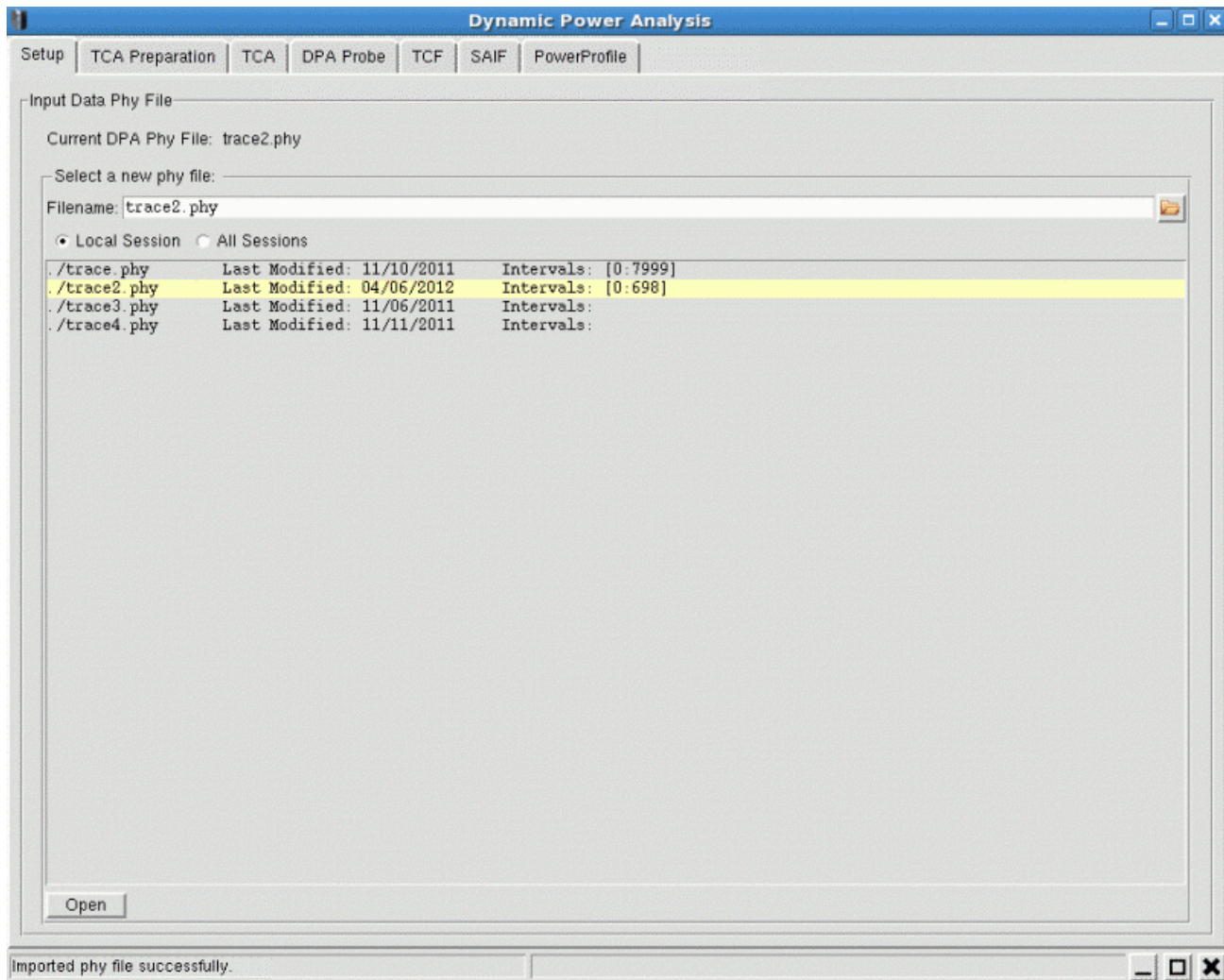
1. Initially, the non-editable *Current DPA Phy File* field displays the name of the `.phy` file selected on the xeDebug *Setup* tab by default. The value adjacent to this field changes when you select a separate `.phy` file for the DPA operations.
2. You can browse to select a separate `.phy` file for the DPA operations by using the button with directory icon adjacent to the *Filename* text field.
3. Below the *Filename* text field, a list of all `.phy` files under the current directory and subdirectories is displayed based on the selection of the *Local Session* or *All Sessions* radio button. The list displays the path where the `.phy` file exists, date when it was last modified, and intervals of cycle which were capture in it.
4. Select the required `.phy` file from the list and click *Open*. This action will update the filename corresponding to the *Current DPA Phy File* field.

UXE User Guide

Analyzing Power Consumption of the Designs

Note: Alternatively, to open the required .phy file, you can select the .phy file and double-click the filename.

Figure 15-10 DPA Setup Tab

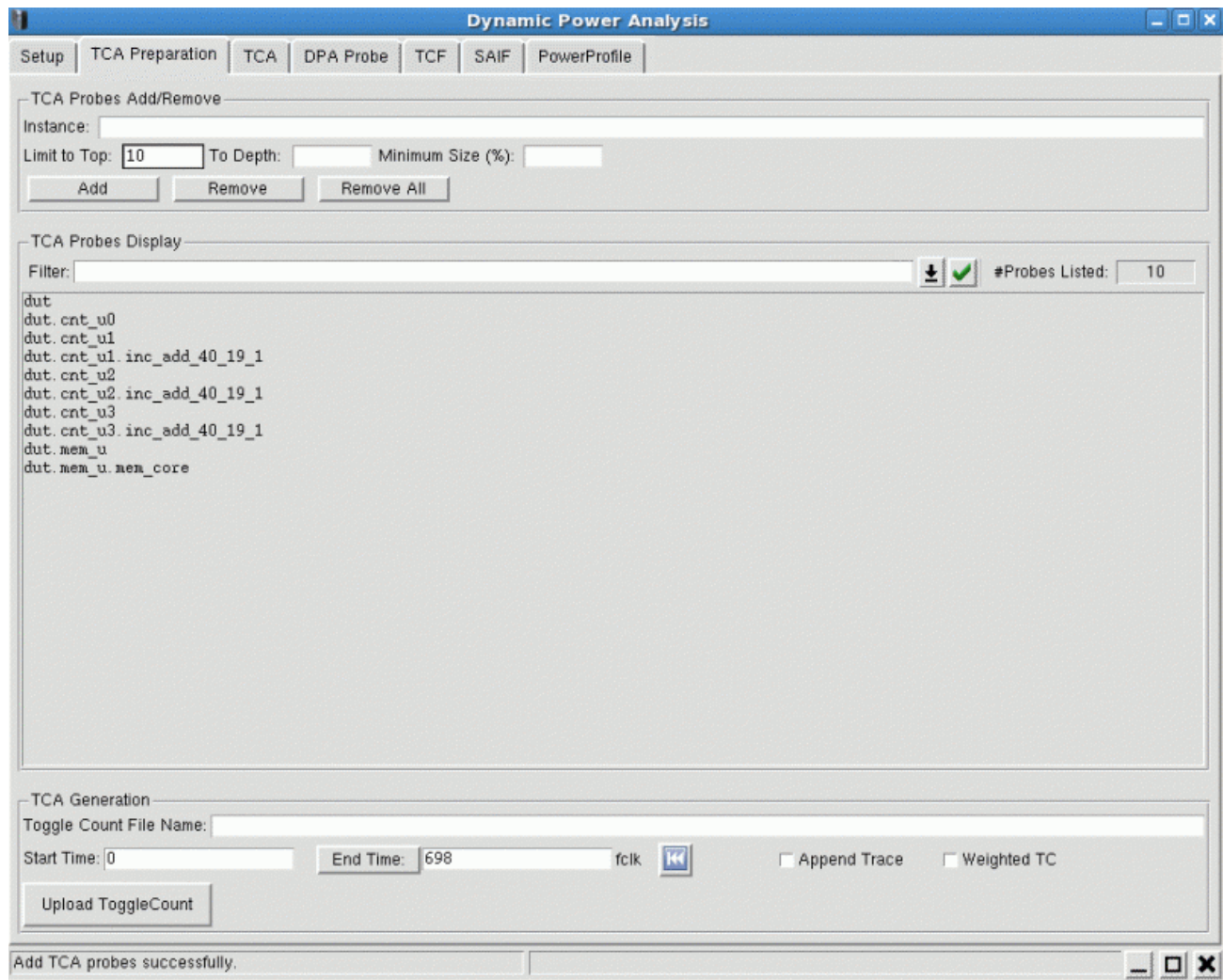


Note: You can also perform DPA operations in an online session. In this case, explicitly open a .phy file before starting DPA. You can continue to update this .phy file with new data during the online session even after opening it for the DPA operations. However, the new data will not be visible for DPA. For example, you opened a .phy file for DPA when it contained data for cycles 0 to 1000. Then, the .phy file is updated with cycles 1000 to 2000. Despite the update, the DPA operations will continue to see only cycles 0 to 1000.

TCA Preparation Tab

The *TCA Preparation* tab (as shown in [Figure 15-11](#)) enables you to select the instances to be included for analysis.

Figure 15-11 TCA Preparation Tab



Use the *TCA Preparation* tab to do the following:

1. Use the *TCA Probes Add/Remove* group box to add and remove instance names for TCA by using the *Instance* field and *Add*, *Remove*, and *RemoveAll* buttons.

You can limit the number of instances by using the parameter fields below the *Instance* field, taking the top instances, listing how deep you wish to probe, and giving a minimum size in terms of percent of the design.

2. Use the *TCA Probes Display* group box to view the name list of the currently added modules (instances). Module names can be filtered to match a pattern specified in the *Filter* field.
3. Use the *TCA Generation* group box to generate and upload a toggle count file by performing the following steps:

- a. Specify the *Toggle Count File Name*.
- b. Set the *Start Time* and *End Time*.

When a `.phy` is opened for DPA operations, the *Start Time* and *End Time* fields get initially set to the entire time range of the opened `.phy` file.

The *End Time* field name is clickable. You can switch this field to *Size* by clicking it.

These fields are equivalent to using the following command:

```
dpa -tracewindow [{-start <time>} -end <time> | -size <num_cycles>]
```

Note: The button next to the *End Time* or *Size* field can be used to reset the trace window to the entire time range of the opened `.phy` file.

- c. Select the *Append Trace* check box to show trace data captured during the first run and add to it the data for the current run.
- d. Select the *Weighted TC* check box to collect weighted toggle counts. Use this only if you compiled using a power database; otherwise, normal toggle counts are collected.
- e. Click *Upload ToggleCount*. The toggle count data will be collected and placed in a file with the suffix `.ppfdata`.

Note: Multiple hosts can be used to speed up the collection of the toggle count data by setting the `mtHost` run-time parameter or the `CDN_MT_HOST` environment variable.

TCA Tab

The *TCA* tab is used for examining the toggle count data.

This tab allows you to open multiple `.ppfdata` files in the same session. This feature is useful for analyzing multiple `.ppfdata` files generated for the same design to compare the power consumption of the same module at different run times.

In addition, the local graphing feature adds support to plot both the high count data and toggle count data for the same instance at the same time.

The *TCA* tab allows the data analysis for the current working files and provides a quick way to identify instances and times of interest before exporting the data to waveform. By default, **all** currently opened `.ppfdata` files are treated as working files. To close an opened `.ppfdata` file, in the Import TCA Data group box, specify the name of the file in the *TCA File Name* field and then click **X** (cross) icon button next to the text field. Else, change the selection to any one of the opened files by selecting the name of the specific file using the down-arrow button next to the *File* field, which is the first field in the left pane. Consequently, the subsequent operations will only use that selected file.

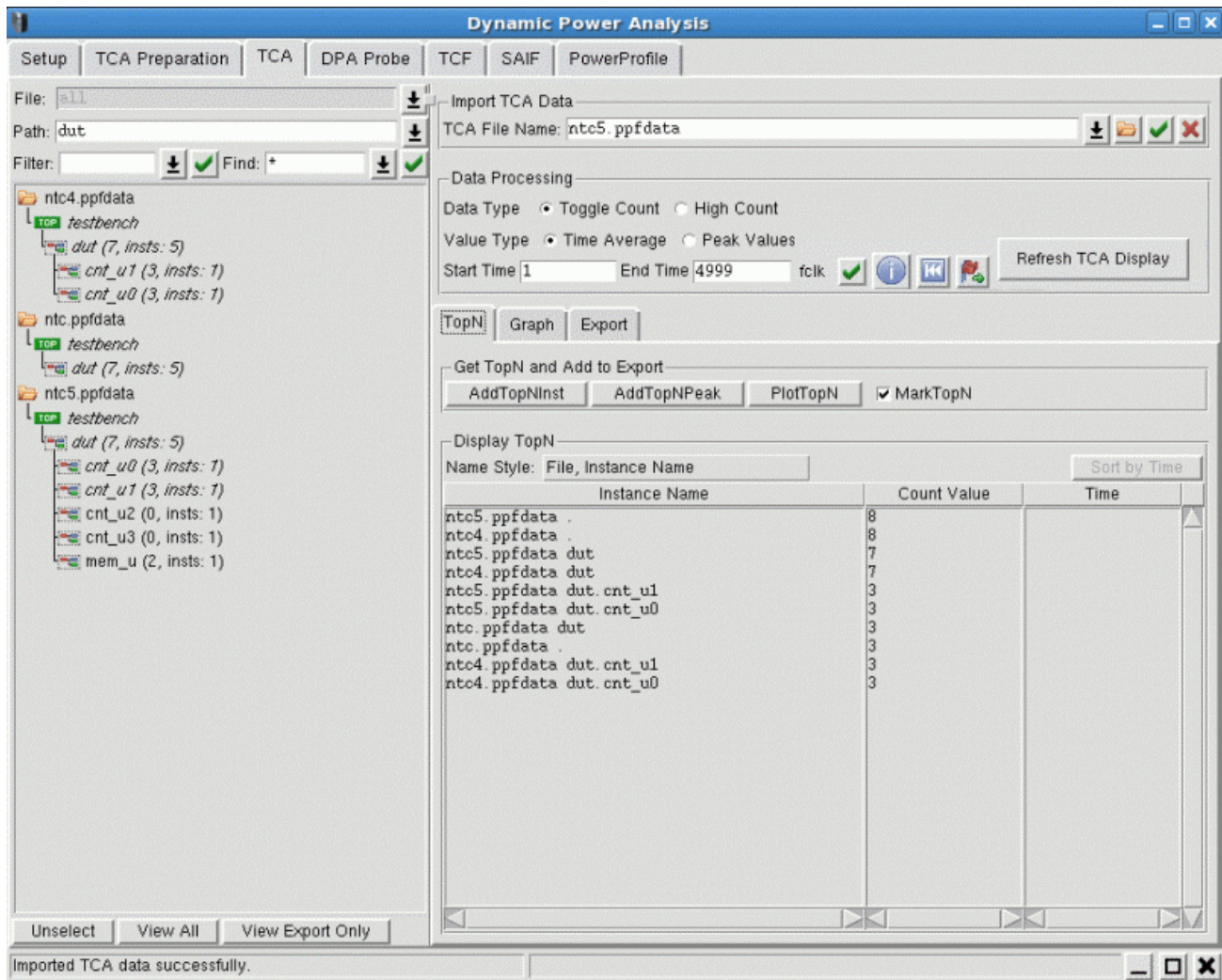
The following sections describe the use of each group box on the *TCA* tab:

- Import TCA Data on page 751
- Data Processing on page 752
- TopN Graph Export Tabs on page 753
- Tree Structure in Left Pane on page 759

UXE User Guide

Analyzing Power Consumption of the Designs

Figure 15-12 TCA Tab Showing Top Peaks



Import TCA Data

Specify the *TCA File Name* in the *Import TCA Data* group box to open the specified toggle data file for analysis. For example, [Figure 15-12](#) shows an opened a data file named `ntc5.ppfdata`.

Note: If no files are opened, all fields except *TCA File Name* remain disabled.

The left side of the tab is a hierarchy viewer that shows all the instances saved in the data file in a tree-like organization. Double click on a node in the tree to expand and collapse that node. The number associated with the name of each node is the average or peak (depending

on the *Value Type*) toggle count or high count (depending on the *Data Type*) over the specified period of time.

Data Processing

Use the *Data Processing* group box to specify the following:

1. **Data Type:** Click to select whether you wish to use toggle counts or high counts in DPA. Toggle count is the number of nets within a specific instance at any given FCLK cycle that change value. High count is the number of nets within a specific instance at any given FCLK cycle that have a high value.

2. **Value Type:** Click to select whether you want to use averaged data or examine peak data.

Time Average is the count number (toggle or high) averaged over the given time range. *Peak Values* is the count number at individual FCLK cycles.

This parameter affects the *GetTopNInst* and the browser display (values). *GetTopNPeak* always uses the peak number.

In addition, this parameter affects the results on the *Graph* display because of the resolution of the canvas, independent of the time range; the graph uses either the time average or the peak value for each pixel. *Time Average* means that, on the *Graph* display, the data is divided into time intervals with each data point on the graph being an average of its interval. *Peak Value* indicates to use the maximum value within each time interval. For a smoother waveform, you should use the *Time Average*.

3. **Time Range:** The *Start Time* and *End Time* fields limit the time range to gain finer resolution. This affects the *GetTopNInst* and *GetTopNPeak*, graphing, the browser display (values), and the export function. The GUI is refreshed upon clicking the  button (same as clicking the *Refresh TCA Display* button).

- a. **Reset Time Range Icon:** Clicking this button resets the time range to the minimum and maximum time ranges specified by all open files. The open files might have overlapping time ranges. Also, the average counts calculated for a time range that includes a point without any data are the same as the ones calculated when the time range is limited to the point until which data exists. Clicking this button is the same as clicking the *Refresh TCA Display* button. Consequently, the GUI gets refreshed.

- b. **Get Time Range from Graph Markers Icon:** If the start and end time markers are set in the graph, the time range within those markers is written into the *Start Time* and *End Time* fields.

4. Refresh TCA Display: Clicking this button updates the browser, graphs, and export list. It does not update the topN data. Following are some examples to illustrate situations when refresh is useful:

- ❑ If some graphs are deleted and you want to rescale the remaining plots to increase the vertical resolution.
- ❑ When the window is resized, the plots are not resized automatically. So, click this button after manually resizing the window.

TopN Graph Export Tabs

The lower right hand side of the *TCA* tab contains the following tabs:

- The *TopN* tab allows you to identify peaks or instances for analysis. For more information, refer to the [Finding Peaks](#) section.
- The *Graph* tab displays waveforms for the toggle data. For more information, refer to the [Displaying Toggle Count Data Graphically](#) section.
- The *Export* tab allows you to create toggle profiles to be viewed in SimVision or Debussy waveform viewer, or in a text file. For more information, refer to the [Exporting Data to Waveform Viewer](#) section.

Finding Peaks

The *TopN* tab (as shown in [Figure 15-12](#)) has the following buttons:

- **AddTopNInst:** Clicking this button finds the top N instances by examining the value calculated based on the selection of *Data Type*, *Value Type*, and specified *Time Range*.

It selects N instances having the largest values. The values in the *Count Value* column correspond to the values in the hierarchy viewer. If *Time Average* is selected, the *AddTopNInst* button returns the top N number of instances with the highest average toggle or high count. If *Peak Values* is selected, the top N number of instances with the highest peak values are returned.

It also automatically adds these to the export list.

Note: You can select any one or all of the opened .ppfdata files.

- **AddTopNPeak:** Clicking this button examines the toggle data over a time domain to figure out the peak values. First select one or more instances in the hierarchy viewer and click the *AddTopNPeak* button. The selected value(s) will be searched for among the selected instances. If no instances are selected before clicking the button, all instances

will be searched. The peaks are arranged in decreasing order, according to data value. Each line shows instance name, peak value, and the time for the peak value.

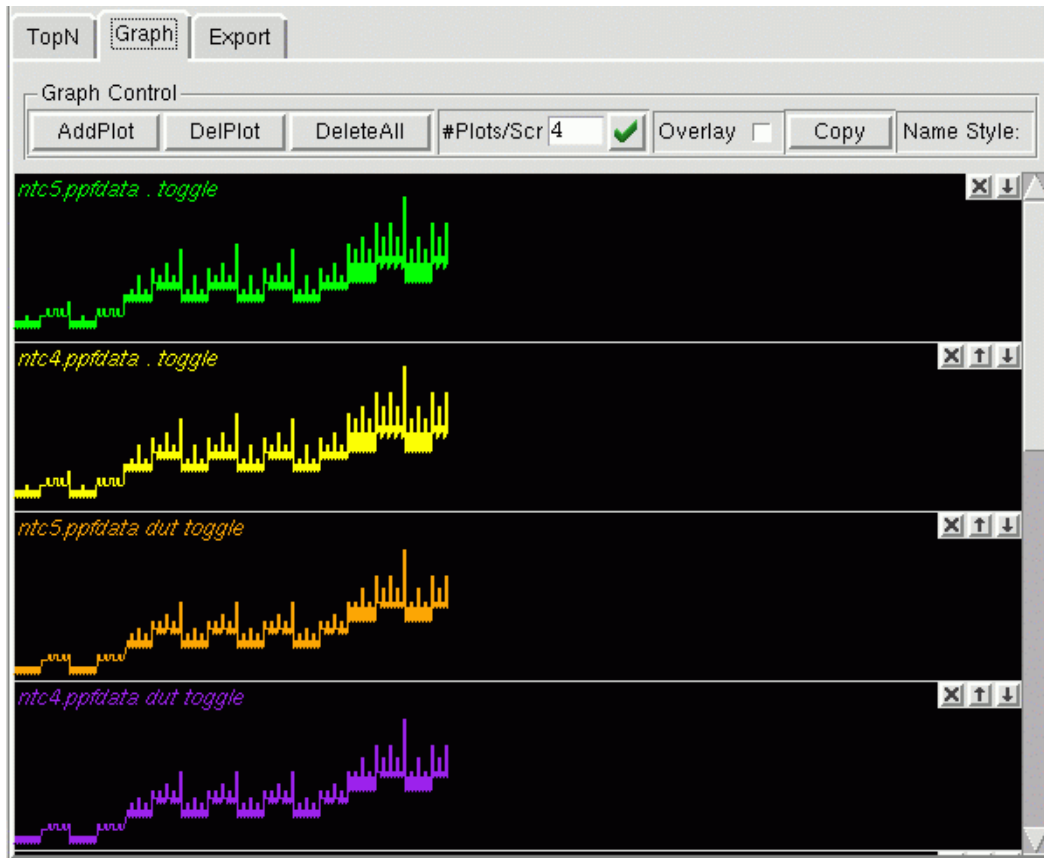
Note: More than one peak value can be found from the same instance. The *AddTopNPeak* value is not affected by the *Time Average* or *Peak Values* settings. You can select any one or all of the opened *.ppfdata* files to use, and also highlight in the browser a subset of the instances between all the opened *.ppfdata* files.

- ***PlotTopN:*** On clicking this button, the GUI graphs all the instances from the displayed list. If selected and applicable, that is, the times of the peaks are available because you selected the *AddTopNPeak* button, the peak values are also marked on the graphs with a red-colored **X** (cross mark).
- ***Name Style:*** You can select this check box to view the instance names with the additional information of filename and/or data type.

Displaying Toggle Count Data Graphically

On the *Graph* tab (as shown in [Figure 15-13](#)), you can view the toggle count data (normal or weighted, depending on how they were collected at run time) graphically over a time range and zoom in on areas of interest.

Figure 15-13 Toggle Count Histograms



The *Graph* tab will show the graphs for multiple instances for the specified time window.

1. **AddPlot:** Select one or more instances in the hierarchy viewer and click the *AddPlot* button to add instances to histogram viewer. You can also use drag-and-drop to add instances from the hierarchy viewer to the histogram viewer.
2. **DelPlot:** Clicking this button brings up a pop-up window that shows all the instance names on the graph and allows you to delete instances from the graph. There is also a delete button beside each graph, which can be used to remove an instance from the graph.
3. **DeleteAll:** Clicking this button clears the entire canvas.
4. **#Plots/Scr:** Sets the number of plots per screen. It determines the height of each plot given the fixed canvas size. The default is 4.
5. **Overlay:** Maps all the plots to the same shared y-axis for comparison purposes. The names of the plots and control buttons are removed to avoid cluttering the plot. The

individual plots can be identified by matching the color to the name in the pop-up window invoked by clicking *DelPlot*.

6. *Copy*: Copies the current screen to another pop-up canvas.

7. *Name Style*: You can select to view the instance names with the additional information of filename and/or data type.

8. *Buttons on the Plots*:

Each plot has its own set of control buttons. In general, there are three buttons:

- a. X (Cross mark): This deletes the plot for a specific instance from the canvas.
- b. Up arrow - This swaps positions with the plot above it.
- c. Down arrow - This swaps positions with the plot below it.

The graphs can be very sensitive to cursor position because each pixel can represent thousands of samples.

Clicking once in the graph viewer draws a red vertical line on the plot, called a time marker, and also updates the left bottom status field with the corresponding time.

Dragging the mouse with the left button pressed draws two red vertical lines on the plots, one at the point of button press and another at the point of button release. The status field on the left bottom of the graph viewer shows the time corresponding to the two time markers.

A right click on a specific instance plot, only updates the status bar on the right, showing the following:

- Time
- Real data value and instance name

You can also zoom in and out to view the information displayed on the graph. Use the time marker icon (red flags with a small green arrow) to grab the times from the graph (lower left status bar) and put them in the time range fields, *Start Time* and *End Time*. Then, click the *Refresh TCA Display* button to zoom in on the specified time range.

Note:

- While performing the zooming operation, the values shown on hierarchy view are also updated according the current time range.
- The graph viewer is a global view of the toggle count data. It has limited ability to perform zooming and scrolling by specifying the time range. Toggle data can be exported to a

waveform viewer for greater viewing capabilities. Use the *Export* tab (shown in [Figure 15-14](#)) to perform this operation.

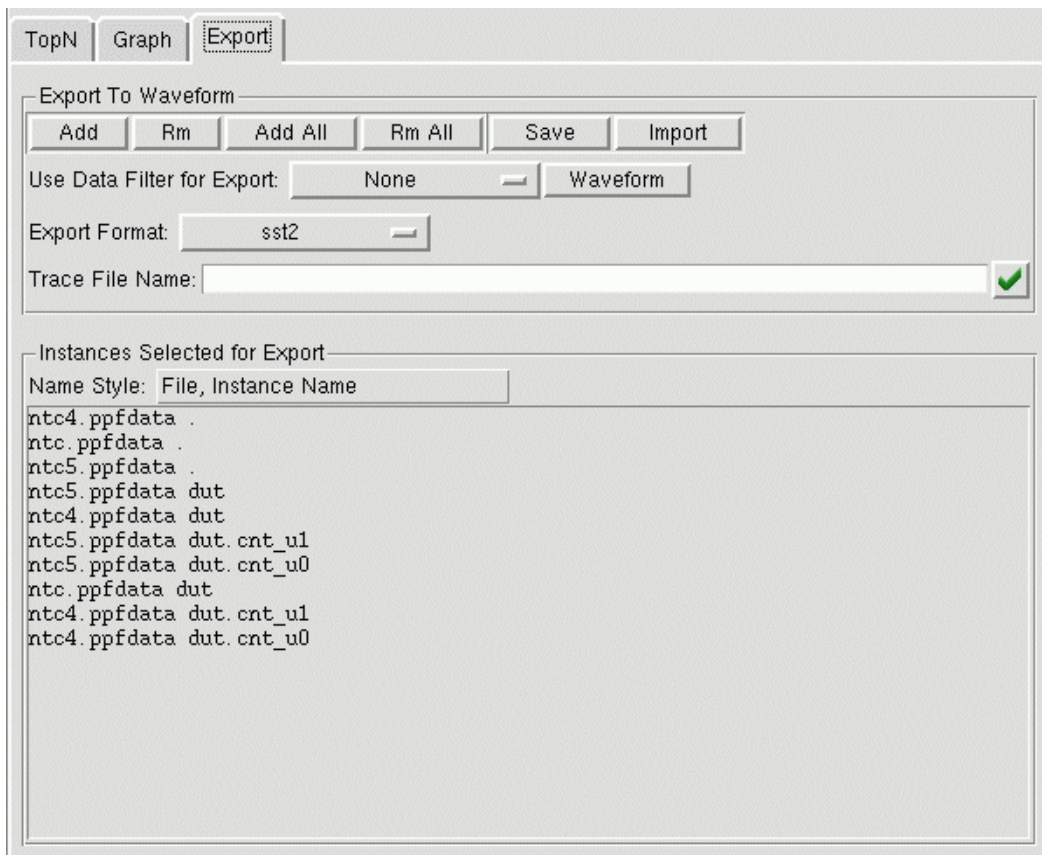
Exporting Data to Waveform Viewer

Use the *Export* tab (as shown in [Figure 15-14](#)) to select a set of instances to be exported to a waveform. This set of instances is displayed on the *Export* tab and can also be viewed in the browser.

You can perform the following operations on this tab after selecting an instance or set of instances from the hierarchy viewer in the left pane:

1. **Add:** Clicking this button adds the selected instance(s) from the browser to the export list for exporting to a waveform. The browser is updated to display this item as italicized to indicate that it is for export.
2. **Rm:** Clicking this button removes the selected instance from the displayed export list. If nothing is selected from the export list and something is highlighted in the browser, then the browser item is removed. The browser updates the style to be not italicized, indicating the selected item is not for export.
3. **Add All:** Clicking this button adds to the export list all the instances from either all opened `.ppfdata` files or any one of the opened files. The browser is updated to display all added instances as italicized.
4. **Rm All:** Clicking this button removes from the export list all the instances from either all opened `.ppfdata` files or any one of the opened files. The browser updates the style to be not italicized for all removed instances.
5. **Save:** Clicking this button saves the *current* list of instances selected for export to a file. This enables you to restore the selection later by using the *Import* button.

Figure 15-14 Export to Waveform Viewer



6. Import: Clicking this button enables you to import a saved file and all the instances listed in that file, which are also found in the current working files. The export list and the style in the browser will change accordingly.

7. Data Filter: You can define different types of filters while performing the export. The raw data will first be processed by the filter before it is saved to the SST2 file. Currently, there are two types of filters. The first filter is for *IntervalAverage*. You must first specify a segment size by clicking the *Properties* button, which is displayed after you select a *User Data Filter for Export*. The specified time range is split into segments according to the specified segment size. An average number is calculated from all data points inside a segment. Then, the average number is saved for each segments. The intention of this filter is to reduce the number of data points in the SST2 waveform.

The second filter is the *RunningAverage*. It calculates an average number for each sample of the raw data. The time window for the average is centered on the specified sample. The window size can be set by clicking the *Properties* button. The running average does not reduce data size, but produces a smoother waveform by transforming each raw data point into an average with its neighboring data points; you can specify the number of neighboring data points as a time window.

8. **Waveform:** Invokes SimVision or Debussy, and displays the exported waveform.
9. **Export Format:** This list box allows you to select the format for exported data, i.e either SST2 for SimVision or FSDB for Debussy.
10. **Trace File Name:** Specify the name of the trace file to be created and click the  button to export the data.
11. **Name Style:** This list box in the Instances Selected for Export group box allows you to select the way you want to view the instance names in the export list.

Below this list box, the *Export* tab displays a list of the instance names that are currently selected.

Note: You must specify a time window for export by entering values in the *Start Time* and *End Time* fields in the *Data Processing* group box. Both toggle count and high count will be exported for each of the selected instances.

Tree Structure in Left Pane

The left pane of the *TCA* tab displays the instances in a hierarchal format. Next to each instance in parentheses is the count and number of immediate child instances. The count depends on the current user selections of data type, value type of average or peak, and time range. The count is updated when you click *Refresh TCA Display* from the *Data Processing* group box.

1. **File:** Shows the current working file selection, which might be name of any of the opened .ppfdata files or *all*. By default, its value is *all*, which indicates that instances of all opened .ppfdata files will be listed in a hierarchal format.
2. **Path:** Specify an instance name and press the *Enter* or *Return* key. Consequently, the left pane gets updated to display all instances in the specified entry and its antecedents. This works on the current working file selection, which can be *all* files.
3. **Filter:** Clicking the  button next to this field or pressing the *Enter* or *Return* key when the cursor is placed in this field, sets the filter on expanding children of the instances in the left pane. It also filters the current display in the left pane to all instances which match the filter; it does not expand instances that are not already shown. It works on the current working file selection, which can be *all* files. Filtering follows the preferences that you set globally for *regex* or *glob*, and case sensitivity. Default is *glob* style case-sensitive filtering.
4. **Find:** Clicking the  button next to this field or pressing the *Enter* or *Return* key when the cursor is placed in this field, highlights in the current left-pane display all instances which match the *find* filter. It works on the current working file selection, which can be *all* files. It does not change the tree structure. Filtering the searched items follows the

preferences that you set globally for *regex* or *glob*, and case sensitivity. Default is *glob* style case-sensitive filtering.

5. **View All:** Clicking this button expands the entire hierarchy for the current working file selection, which can be *all* files.
6. **View Export Only:** Clicking this button shows in the left pane only instances selected for export and all their antecedents. It works on the current working file selection, which can be *all* files.

Note:

- If you click once on any instance, it is displayed in the *Path* filter.
- If you click twice on any instance, it is displayed in the *Path* filter. Also, if no children of that instance were displayed, the tree expands by displaying its children by one level. If children are already displayed, it will collapse to hide the children of that instance.
- You can select a range of instances by selecting an entry from the tree structure and dragging with the left mouse button clicked till the entry you want to select.
- You can add to the current selection of instances by keeping the *Ctrl* or *Control* key pressed while clicking a particular instance to select.
- You can drag the selected instances while keeping the right mouse button clicked and drop them to the graph.

DPA Probe Tab

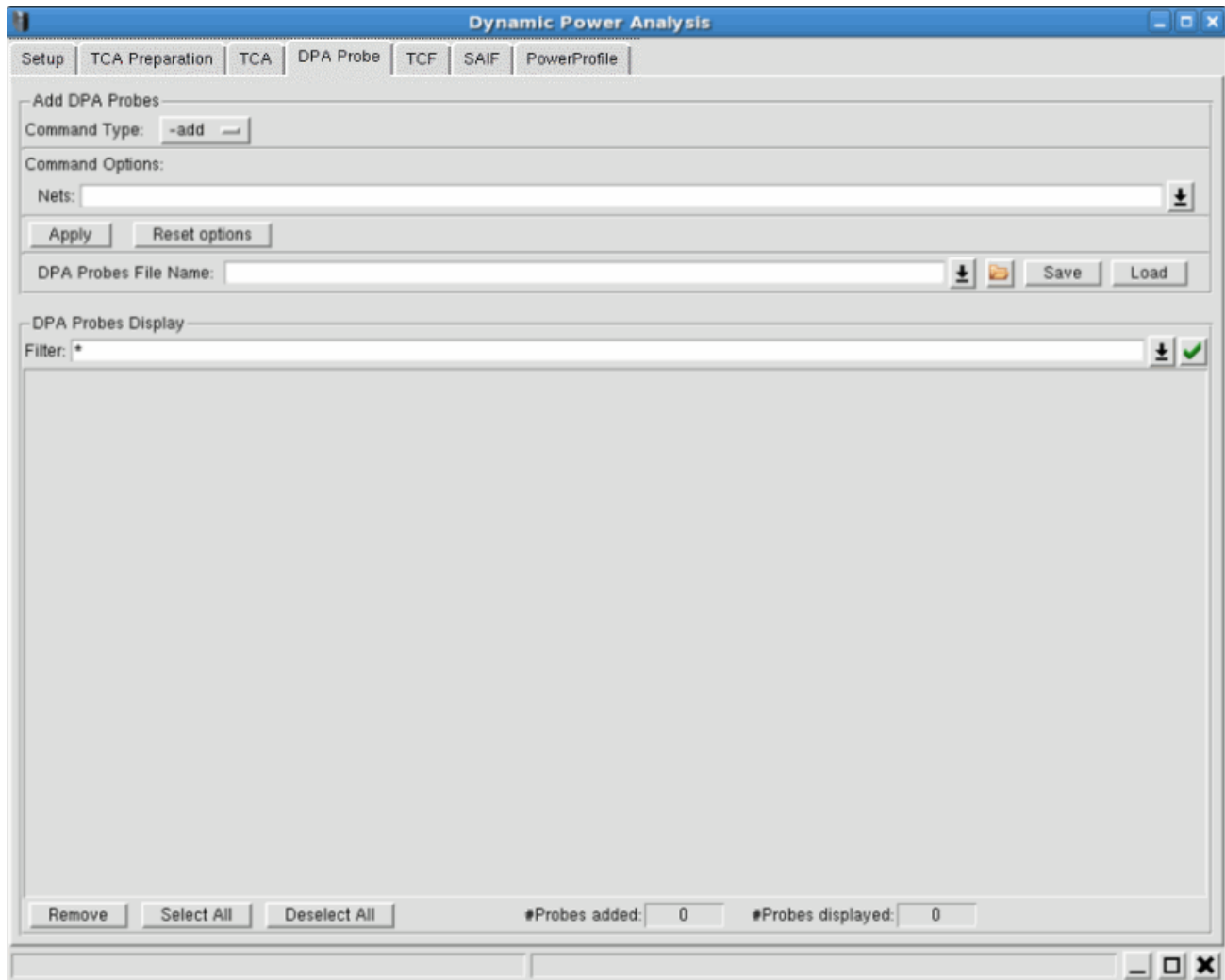
The *DPA Probe* tab (as shown in [Figure 15-15](#)) allows you to add DPA probes before generating TCF or SAIF files.

Note: The *DPA Probe* tab is enabled only after you open a *.phy* file for DPA operations using the *Setup* tab of the *Dynamic Power Analysis* window.

The following sections describe the use of each GUI element on the *DPA Probe* tab:

- [Adding DPA Probes](#)
- [Saving or Loading DPA Probes](#)
- [DPA Probes Display](#)

Figure 15-15 DPA Probe Tab



Adding DPA Probes

Perform the followings steps in the *Add DPA Probes* group box:

1. Click the menu button adjacent to the *Command Type* field to select one of the `dpa` command options listed below. The fields displayed in the *Add DPA Probes* group box change according to the selected command option to enable you to set the related parameter values.

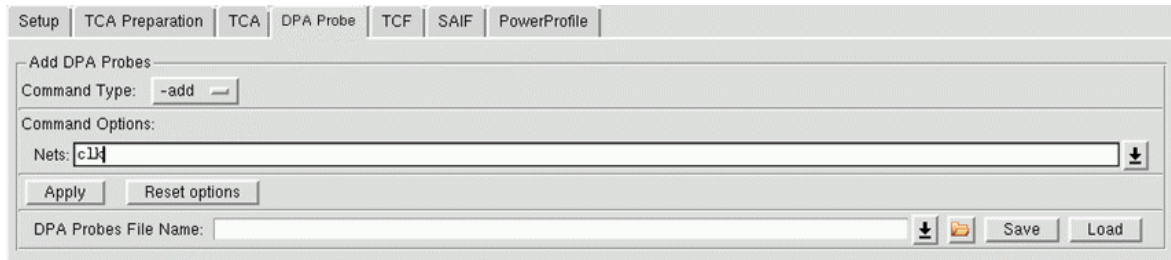
☐ -add

Selecting `-add` is an equivalent to the `dpa -add <nets>` command. You need to specify net(s) name. When you want to specify multiple net names, separate each

UXE User Guide

Analyzing Power Consumption of the Designs

name by a space. If any net name includes a backslash or space, use a pair of curly braces to enclose it.

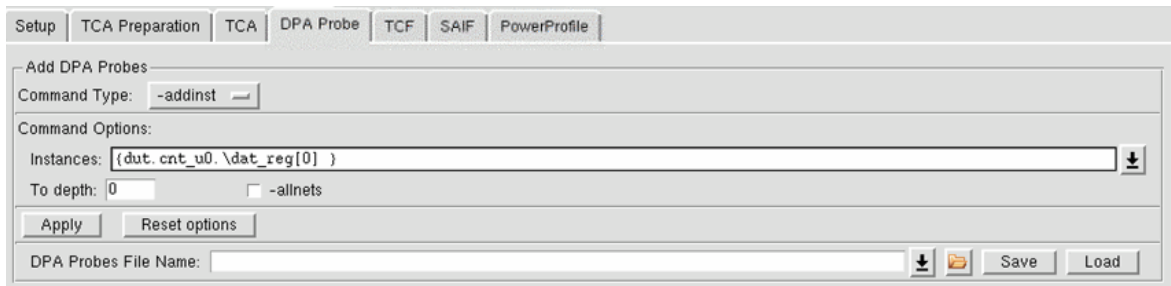


You can drag and drop net names from other browsers, like Design Browser, into the *Nets* text field.

☐ -addinst

Selecting *-addinst* is an equivalent to the `dpa -addinst [-allnets] -depth <n> <instances>` command. You need to specify instance name(s) and depth value. When multiple instance names need to be specified, separate each name by a space. If any instance name includes a backslash or space, use a pair of curly braces to enclose it.

If `-allnets` check box is selected, the nets starting with `n#` will also be added, which are otherwise filtered and not added by default.



You can drag and drop net names from other browsers, like Design Browser, into the *Instances* text field.

☐ -readNetNames

Selecting *-readNetNames* is an equivalent to the `dpa -readNetNames <rc_dpa_info_file_name> [<instances>]` command. This command adds the nets or pins that appear in the specified file, `<rc_dpa_info_file_name>`, which is the RC-generated side file. You can

UXE User Guide

Analyzing Power Consumption of the Designs

specify the [*<instances>*] argument if the required nets or pins are not under the top cell.

The screenshot shows the 'Add DPA Probes' dialog box. The 'Command Type' dropdown is set to '-readNetNames'. Under 'Command Options', the 'rc_dpa_info_file_name' field is empty, and the 'Instance' field contains 'dut'. There are 'Apply' and 'Reset options' buttons. At the bottom, there is a 'DPA Probes File Name' field with 'Save' and 'Load' buttons.

You can click the button with directory icon adjacent to the *rc_dpa_info_file_name* text field to bring up a window that lists all directories and allows you to select the required file.

You can drag and drop net names from other browsers, like Design Browser, into the *Instance* text field.

❑ -addpath

Selecting *-addpath* is an equivalent to the `dpa -addpath -depth <n> <net1> <net2>` command. You must specify *Net1* and *Net2* to trace back from *Net1* to *Net2* and add all nets on the path. The value specified in the *To Depth* text box indicates the number of levels that are to be traced back.

The screenshot shows the 'Add DPA Probes' dialog box. The 'Command Type' dropdown is set to '-addpath'. Under 'Command Options', the 'Net1' and 'Net2' fields are empty, and the 'To depth' field contains '10'. There are 'Apply' and 'Reset options' buttons. At the bottom, there is a 'DPA Probes File Name' field with 'Save' and 'Load' buttons.

You can drag and drop net names from other browsers, like Design Browser, into the *Net1* and *Net2* text fields.

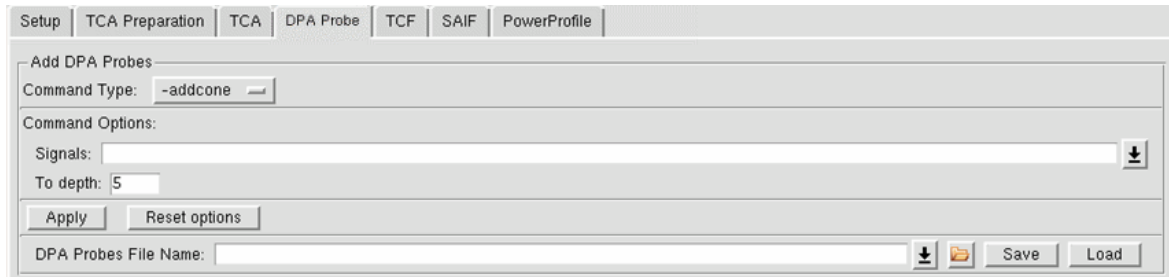
❑ -addcone

Selecting *-addcone* is an equivalent to the `dpa -addcone -depth <n> <signals>` command.

UXE User Guide

Analyzing Power Consumption of the Designs

This command allows you to add all signals in the input cone of logic of the given nets.



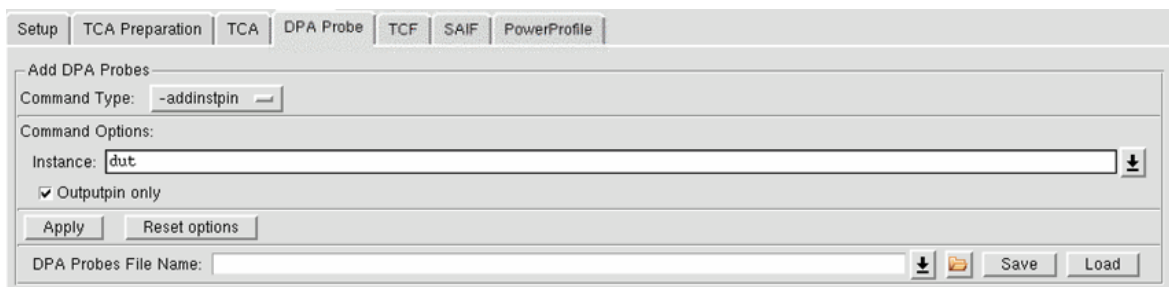
The screenshot shows the 'Add DPA Probes' dialog box. The 'Command Type' is set to '-addcone'. The 'Command Options' section has 'Signals' and 'To depth: 5'. There are 'Apply' and 'Reset options' buttons. At the bottom, there is a 'DPA Probes File Name' field with 'Save' and 'Load' buttons.

You can drag and drop net names from other browsers, like Design Browser, into the *Signals* text field.

☐ -addinstpin

Selecting *-addinstpin* is an equivalent to the `dpa -addinstpin <instance>` command. Specify an instance name. All input and outputs of the specified instance, and all instances within it are added to the DPA probe list.

If the *Outputpin only* check box is selected, the *Command Type* is an equivalent to the `dpa -addinstoutputpin <instance>` command which is used to add only outputs of the specified instance and all instances within it.



The screenshot shows the 'Add DPA Probes' dialog box. The 'Command Type' is set to '-addinstpin'. The 'Command Options' section has 'Instance: dut' and the 'Outputpin only' checkbox is checked. There are 'Apply' and 'Reset options' buttons. At the bottom, there is a 'DPA Probes File Name' field with 'Save' and 'Load' buttons.

You can drag and drop net names from other browsers, like Design Browser, into the *Instance* text field.

2. Click *Apply* to add DPA probes based on the parameters specified for the selected *Command Type*.

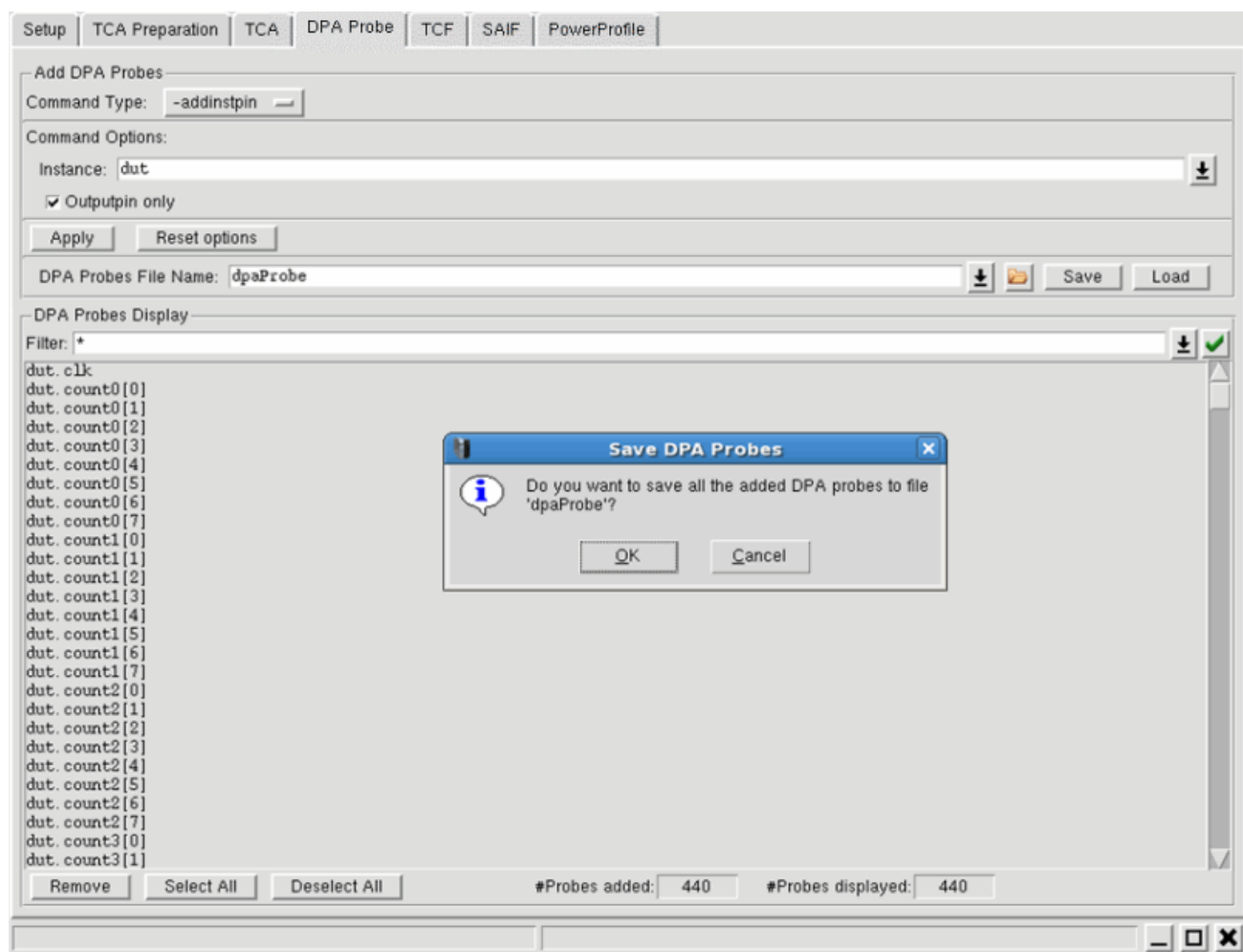
Note: Clicking *Reset options* enables you to reset the parameter values of the selected command option to default values. You can then specify new parameter values for the selected command option.

Saving or Loading DPA Probes

At any time, you can either save DPA probes to a file or load DPA probes from a file.

To save DPA probes into a file that can be loaded later, first specify a filename in the *DPA Probes File Name* text field and then click *Save*. The *Save DPA Probes* dialog box (as illustrated in [Figure 15-16](#)) will be displayed to confirm whether or not you want to save all the added DPA probes to the specified file. Click *OK* to continue.

Figure 15-16 Probe Tab - Save DPA Probes to a File



To load DPA probes from a file, you can do either of the following:

- Enter the name of the file in the *DPA Probes File Name* text field.

OR

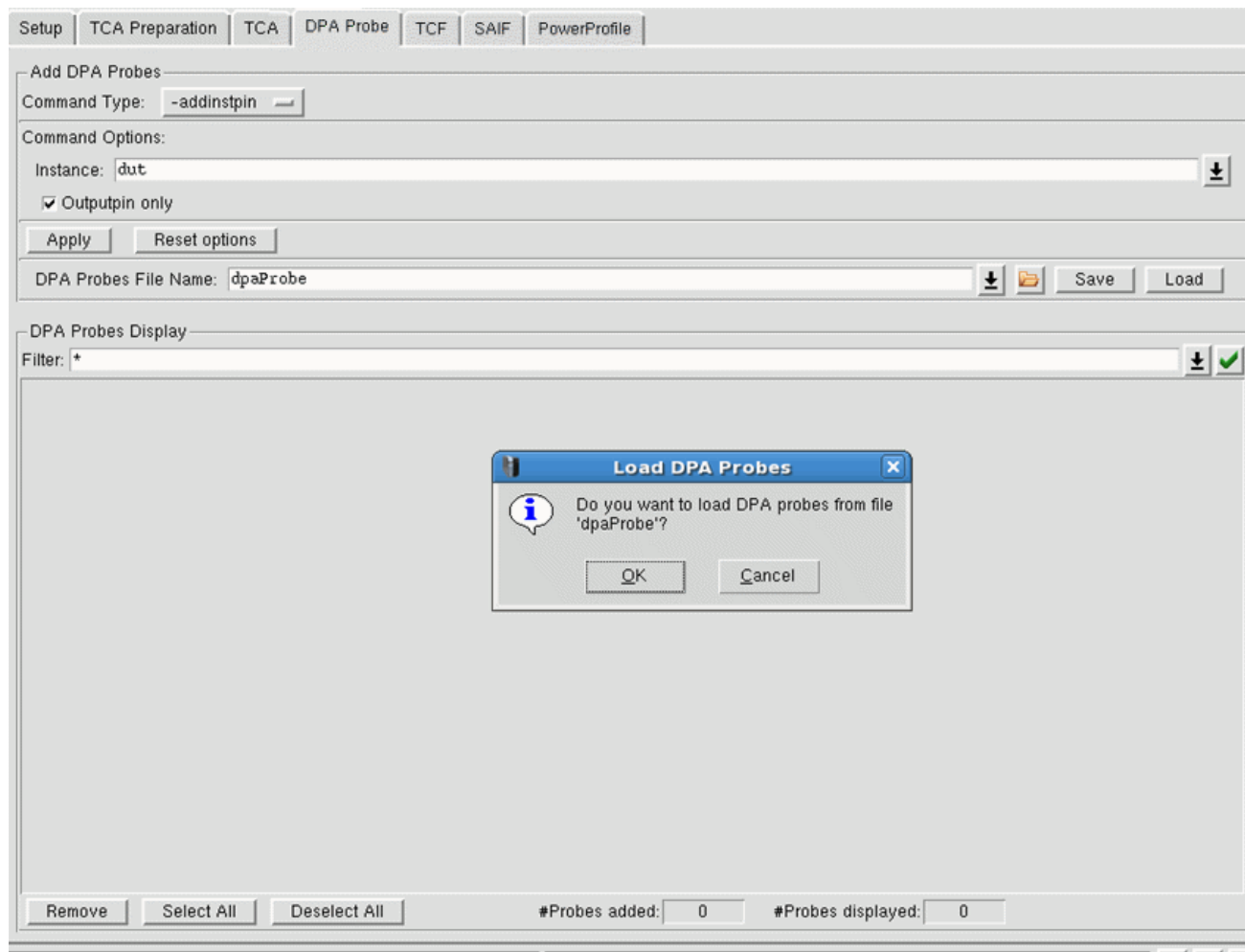
UXE User Guide

Analyzing Power Consumption of the Designs

- Select the required file from the window invoked by clicking the button with directory icon adjacent to the text field.

Then, click *Load* to list the DPA probes from the selected file in the *DPA Probes Display* group box. The *Load DPA Probes* dialog box (as illustrated in [Figure 15-17](#)) will be displayed to confirm whether or not you want to load the DPA probes from the specified file. Click *OK* to continue.

Figure 15-17 Probe Tab - Load DPA Probes from a File



DPA Probes Display

The *DPA Probes Display* group box enables you to view the added DPA probes according to the specified filter. You can also select the probes that are not required and remove those from the display list.

UXE User Guide

Analyzing Power Consumption of the Designs

By default, the *Filter* is set to an asterisk (*), which indicates to list all added DPA probes. You can specify a new filter pattern and click the  button to bring it into effect. This is equivalent to the `dpa -list -glob <pattern>` command.

The content list can show only the top 1000 DPA probes. The purpose of this limitation is to allow you to browse the DPA probes easily. Therefore, if the number of added DPA probes is more than 1000, a warning message will be displayed at the bottom of the probe list box (refer to [Figure 15-18](#) for an example). In this case, you can use a more accurate filter pattern to display only specific probes that you want to see.

Figure 15-18 Probe Tab - Display Added DPA Probes



There are two non-editable text boxes at the bottom of the list to show the total number of *Probes added* and *Probes displayed*.

You can select any probe from the DPA probe listbox.

You can use the following buttons as well:

- Click *Remove* to remove the selected DPA probes.
- Click *Select All* to select all probes that match the specified filter pattern (including even those that are not displayed in the probes listbox).
- Click *Deselect All* to deselect all probes.

UXE User Guide

Analyzing Power Consumption of the Designs

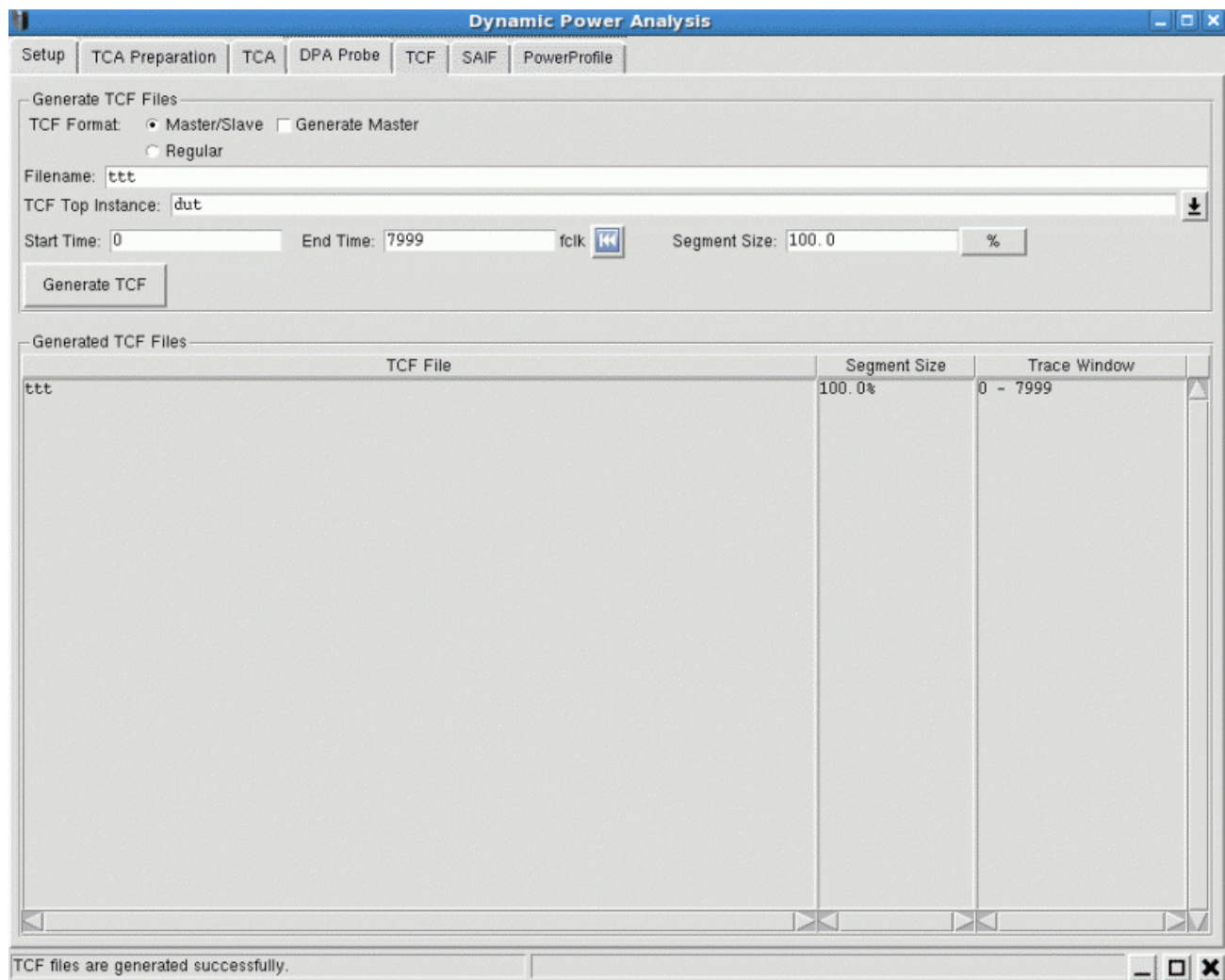
TCF Tab

The *TCF* tab (as shown in [Figure 15-19](#)) enables you to independently generate TCF files. This is equivalent to the following command:

```
dpa -outfile -tcf <tcfFileName> [-instance <instanceName>] \  
[-masterTCF] [-slaveTCF] [-noregulartcf] \  
[-segment <n>% | <cycleNumber>] \  
[-start <time>] [-end <time>]
```

Note: The *TCF* tab is enabled only after you open a .phy file for DPA operations.

Figure 15-19 TCF Tab



Perform the following steps to generate the TCF files using the *TCF* tab:

UXE User Guide

Analyzing Power Consumption of the Designs

1. In the xeDebug console, enter any one of the following commands to add probes for generating TCF files:

```
dpa -addinstoutputpin <instanceName>
dpa -addinsttff <instanceName> [-asicLibrary <.llib name> ... <.llib name>] \
                               [-hardmacroLibrary <.llib name> .... <.llib name>]
dpa -addinstReg <instanceName> [-asicLibs <.libName> ....<.libName> ] \
                               [-hardmacroLibs <.libName> ....<.libName>]
```

2. On the TCF tab, specify the *TCF Format* for the files to be generated by selecting either the *Master/Slave* or *Regular* radio button.

If the *Master/Slave* radio button is selected, the *Generate Master* check box is enabled for selection. This is equivalent to giving the [-masterTCF], [-slaveTCF], and [-noregularartcf] options to the dpa -outfile -tcf command.

3. Specify the name of the required TCF file in the *Filename* field. This is a mandatory field, which is equivalent to giving the <tcfFileName> option to the dpa -outfile -tcf command.

4. Specify the top instance name in the *TCF Top Instance* field. This is an optional field. Selecting this field is equivalent to specifying the [-instance <instanceName>] option.

Note: This field allows you to drag and drop instance name from other browsers, such as the Design Browser, into the *TCF Top Instance* field.

5. Specify the required cycle time in the *Start Time* and *End Time* fields. The time unit will be displayed adjacent to these fields. The button next to these fields can be used to reset to the entire range. This is equivalent to giving the [-start <time>] [-end <time>] options.

Note: If the selected .phy file has data for a specific interval of time, the start and end time is by default displayed in these fields. You will be allowed to specify only a time that falls within that range. For example, trace.phy has data for interval 0-7999, the *Start Time* and *End Time* fields will display 0 and 7999, respectively. You will be allowed to change the *Start Time* and *End Time* to 100 and 500, but changing to 7000 and 8500 will generate an error.

6. Specify the *Segment Size*. You can switch the unit as % (percentage) or cycles/time (number) by clicking the button adjacent to the *Segment Size* field. This field is equivalent to giving the [-segment <n>% | <cycleNumber>] option.



Segment Size: 100ns

cycles/time

cycles/time

%

7. Click *Generate TCF* to generate the TCF file. The TCF files generated in the current session are listed in the table within the *Generated TCF Files* group box.

Note:

- ❑ Before clicking *Generate TCF*, add the probes manually (refer to [step 1](#) in this section); otherwise, nothing will be uploaded into the TCF files. Also, a warning message will be displayed in the xeDebug console and status bar of the DPA GUI window.
- ❑ If you entered a filename which already exists, a dialog box will be displayed to confirm whether or not you want to overwrite it.

SAIF Tab

The *SAIF* tab (as shown in [Figure 15-20](#)) contains the following group boxes that enable you to perform the SAIF-related operations that are equivalent to using the `dpa -saif` and `dpa -outfile -saif` commands:

- [Forward SAIF](#)
- [Scope](#)
- [Backward SAIF](#)

Note: The *SAIF* tab is enabled only after you open a `.phy` file for DPA operations.

Forward SAIF

The *Forward SAIF* group box provides fields and tabs for loading and filtering the forward SAIF files that provide the description to the tool to generate the backward SAIF files. Perform the following steps in this group box:

1. Enter the *Forward SAIF File Name* in the *Load Forward SAIF File* group box. You can also use the button with directory icon to browse and select a forward SAIF file. This is a mandatory field.
2. Enter the *Module Patterns*. Each pattern should be separated by a blank space. This is an optional field.

If specified, the load operation looks in the loaded forward SAIF file for only those modules whose names match any of the specified patterns.

Note:

UXE User Guide

Analyzing Power Consumption of the Designs

- ❑ If the module pattern includes space, enclose the pattern within a pair of curly braces.

For example:

```
{M *}
```

- ❑ If the module pattern includes a backslash, enclose the pattern within a pair of curly braces and add an extra backslash before each original backslash.

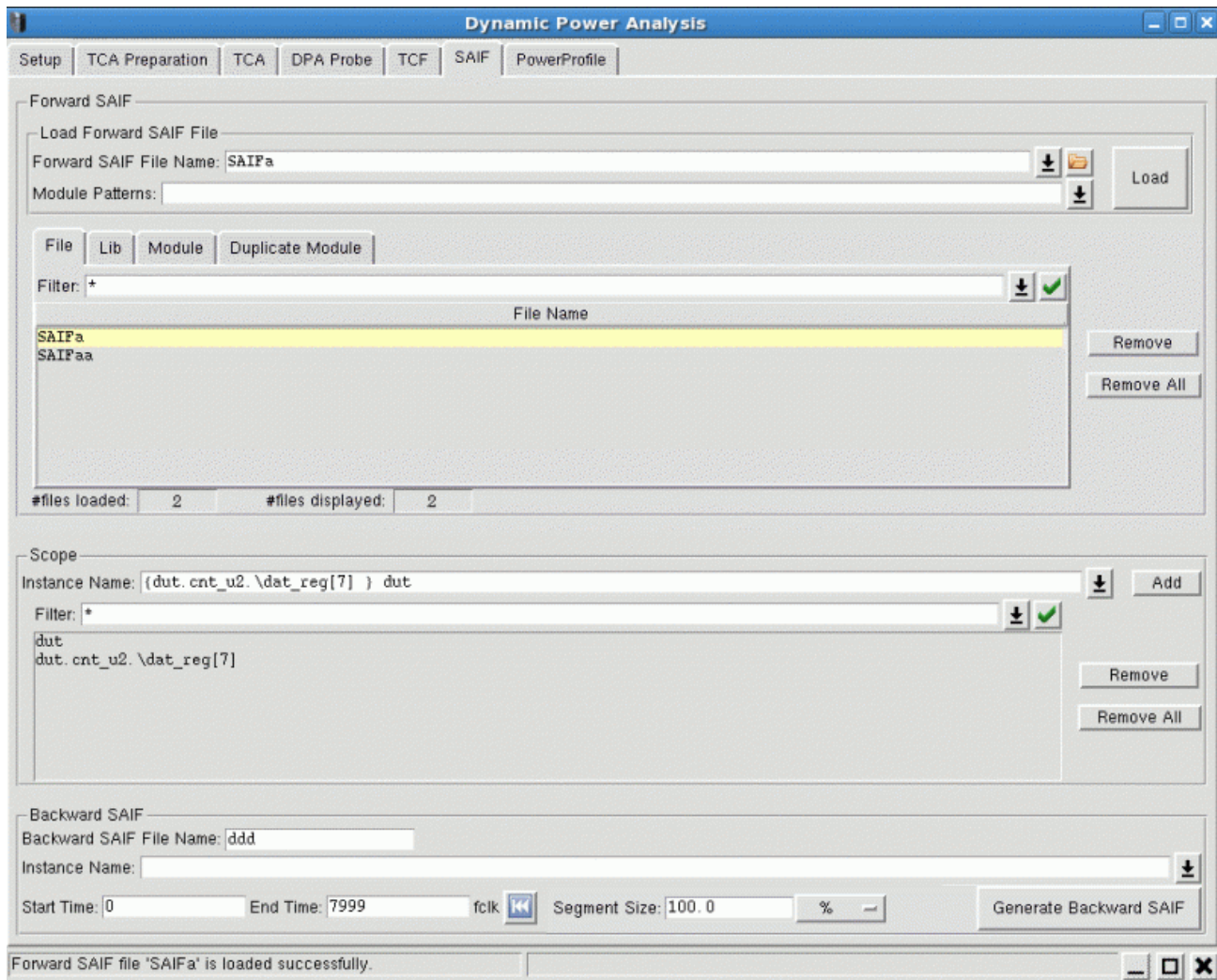
For example:

```
{M *} {MX. \\A*}
```

3. Click *Load* to load the specified forward SAIF file(s). This is equivalent to using the following command:

```
dpa -saif -load <saif_file_name> [-module <module_name_pattern>]
```


Figure 15-20 SAIF Tab - Forward SAIF Group Box with File Tab Enabled



The *Forward SAIF* group box also has the following four tabs to view the content of the loaded forward SAIF files:

■ **File:**

Lists all the loaded forward SAIF files. This is equivalent to using the `dpa -listfile` command. Refer to [Figure 15-20](#).

You can use *Filter* to show only those files whose name match the specified pattern. By default, the *Filter* field has an asterisk (*) that means display *all*.

At the bottom of the results table, you can see the *#files loaded* and *#files displayed* in non-editable fields.

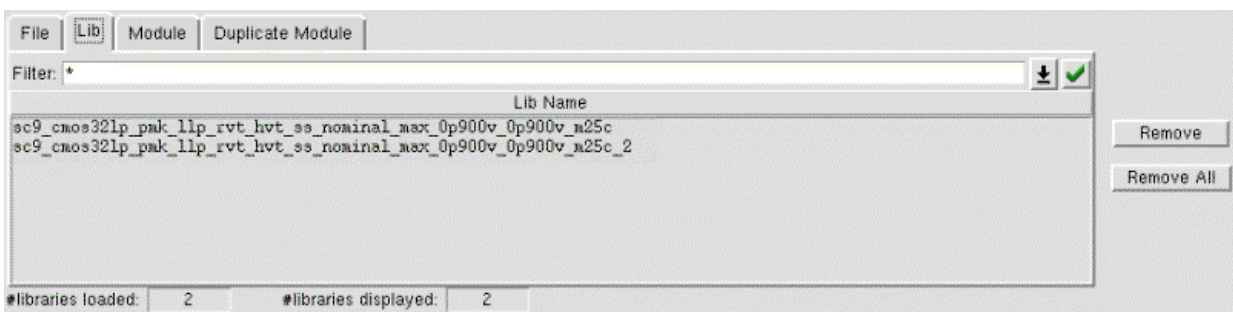
Clicking *Remove* removes the selected forward SAIF file(s). This is equivalent to using the `dpa -saif -rmfile <file_name>` command.

Clicking *Remove All* removes all displayed files that match the specified filter.

■ **Lib:**

Lists all libraries that have been loaded from the forward SAIF files. This is equivalent to using the `dpa -listlib` command.

Figure 15-21 SAIF Tab - Forward SAIF Group Box with Lib Tab Enabled



You can use *Filter* to show only those libraries whose name match the specified pattern. By default, the *Filter* field has an asterisk (*) that means display *all*.

At the bottom of the results table, you can see the *#libraries loaded* and *#libraries displayed* in non-editable fields.

Clicking *Remove* removes the selected library. This is equivalent to using the `dpa -saif -rmlib <lib_name>` command.

Clicking *Remove All* removes all displayed libraries that match the specified filter.

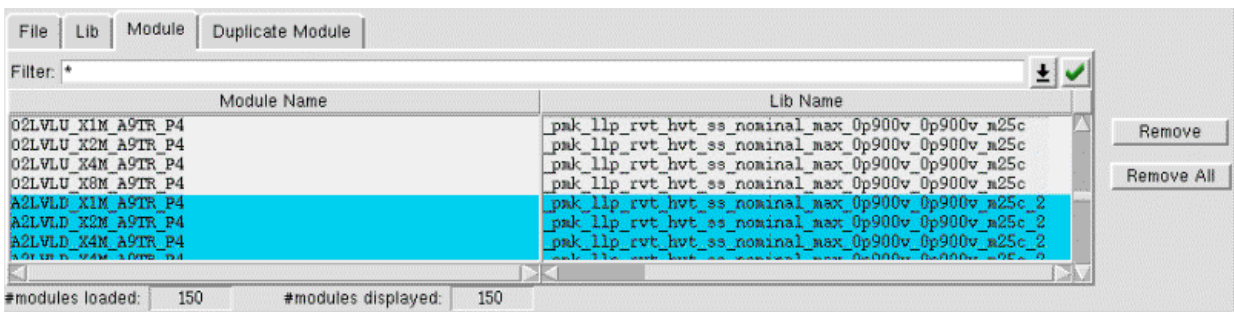
■ **Module:**

Lists all modules that have been loaded from the forward SAIF files. Additionally, the library name in which the listed module is included is displayed. This is equivalent to using the `dpa -saif -listmodule [-lib]` command.

UXE User Guide

Analyzing Power Consumption of the Designs

Figure 15-22 SAIF Tab - Forward SAIF Group Box with Module Tab Enabled



You can use *Filter* to show only those modules whose name match the specified pattern. By default, the *Filter* field has an asterisk (*) that means display *all*.

At the bottom of the results table, you can see the *#modules loaded* and *#modules displayed* in non-editable fields.

Clicking *Remove* removes the selected modules. This is equivalent to using the `dpa -saif -rmmodule <module_name_pattern> [<lib_name>]` command.

Clicking *Remove All* removes all displayed modules that match the specified filter.

Note:

- ❑ The modules are grouped by library name and the modules that belong to the same library are sorted by the module name.
- ❑ If more than one libraries are loaded, the *Module* tab uses colored background to distinguish the modules in different libraries (that is, the modules from the same library are displayed with the same background color).
- ❑ If the *Filter* text field on the *Module* tab has a pattern that includes a backslash, add one extra backslash before each original backslash. For example, `\\MX*`.

Note: Unlike the *Module Pattern* field within the *Load Forward SAIF File* group box (refer to [step 2](#)), you need not enclose the pattern within a pair of curly braces even if the pattern includes a backslash or space.

- ❑ To deselect, press the `Esc` key on the keyboard. To select multiple modules from the list, you can use the `Ctrl` (Control) and `Shift` keys.

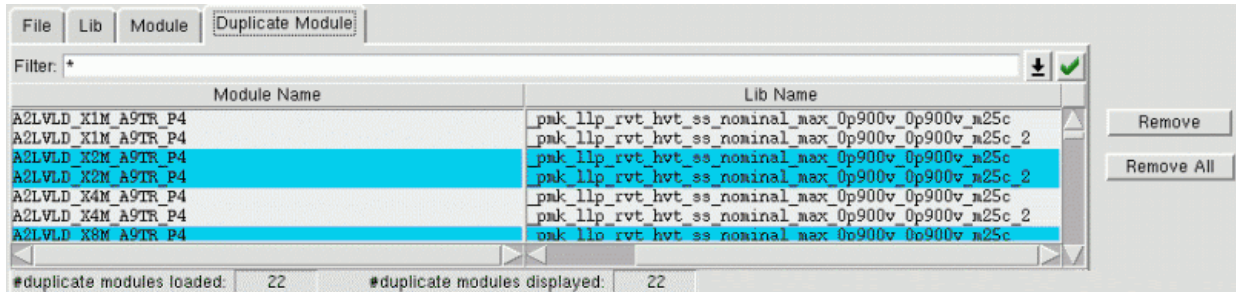
■ Duplicate Module:

Lists only those modules that appear in multiple libraries. The content of the *Duplicate Module* tab is a subset of the content of the *Module* tab. This is equivalent to using the `dpa -saif -listduplicatemodule` command.

UXE User Guide

Analyzing Power Consumption of the Designs

Figure 15-23 SAIF Tab - Forward SAIF Group Box with Duplicate Module Tab Enabled



You can use *Filter* to show only those modules whose name match the specified pattern. By default, the *Filter* field has an asterisk (*) that means display *all*.

At the bottom of the results table, you can see the *#duplicate modules loaded* and *#duplicate modules displayed* in non-editable fields.

Clicking *Remove* removes the selected modules. This is equivalent to using the `dpa -saif -rmmodule <module_name_pattern> [<lib_name>]` command.

Clicking *Remove All* removes all displayed modules that match the specified filter.

Note:

- On the *Duplicate Module* tab, if some modules are removed, the modules that are left might no longer have duplicates. In this case, the modules whose duplicates have been removed will disappear automatically from the *Display Duplicate Module* list.
- Lists on all four tabs – *File*, *Lib*, *Module*, and *Duplicate Module*, will be updated as soon as you load a new forward SAIF file.
- Removing some elements from one list will also update the other three lists. For example, if you remove one file, other three lists will also be updated by automatically removing libraries or modules within that file.
- You can select multiple elements from a list by clicking the required items while keeping either the `Ctrl` or `Shift` key pressed on the keyboard.
- To deselect the item that was selected last, press the `ESC` key on the keyboard.

Scope

The *Scope* group box (as shown in [Figure 15-24](#)) enables you to specify only those instances that should be included in the backward SAIF file. This group box is equivalent to using the `dpa -saif -addscope <instance_name>` command.

Figure 15-24 SAIF Tab - Scope Group Box



Perform the following operations to define the scope:

1. Enter an *Instance Name*(s). Each instance name should be separated by a blank space.

Note:

- ☐ If the instance name includes a space or backslash, enclose the instance name within a pair of curly braces.

For example:

```
{dut.cnt_u2.\dat_reg[7]} dut
```

- ☐ You can drag and drop instance name from other browsers, such as Design Browser, into the *Instance Name* field.

2. Click *Add* to add the specified instance into scope that needs to apply Forward SAIF. Alternatively, you can press the *Enter* or *Return* key on the keyboard.

All added instances are displayed in the scope listbox.

You can use *Filter* to show only those instances whose name match the specified pattern. By default, the *Filter* field has an asterisk (*) that means display *all*. This field is an equivalent to using the `dpa -saif -listscope` command.

Clicking *Remove* removes the selected instances from the scope. This is equivalent to using the `dpa -saif -rmscope <instance_name>` command.

Clicking *Remove All* removes all displayed instances from scope that match the specified filter.

Note: You can select multiple instances from the list by clicking the required instances while keeping either the `Ctrl` or `Shift` key pressed on the keyboard.

UXE User Guide

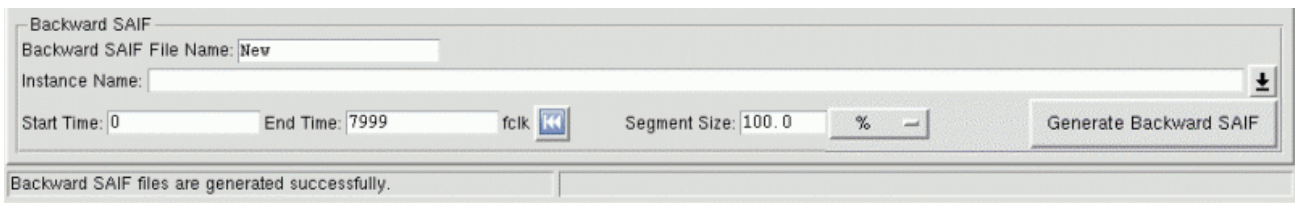
Analyzing Power Consumption of the Designs

Backward SAIF

The *Backward SAIF* group box (as shown in [Figure 15-25](#)) is an equivalent to using the following command to generate the backward SAIF file:

```
dpa -outfile -saif <saif_file_name> [-instance <instance_name>] \  
[-segment <cycleNumber> | <percent>] \  
[-start <time>] [-end <time>]
```

Figure 15-25 SAIF Tab - Backward SAIF Group Box

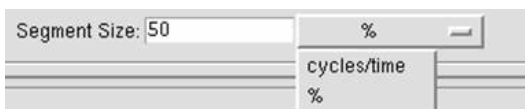


Perform the following operations using this group box:

- Enter the output filename in the *Backward SAIF File Name* field. This is equivalent to using the `<saif_file_name>` option. This is a mandatory field.
- Enter the *Instance Name*. This is an optional field. Selecting this field is equivalent to using the `[-instance <instance_name>]` option.

Note: You can drag and drop instance name from other browsers, such as Design Browser, into the *Instance Name* field.

- Enter the *Start Time* and *End Time*. The time unit is displayed next to these fields. Also, there is a button next to these fields that can be used to reset to the entire range. These fields are equivalent to giving the `[-start <time>] [-end <time>]` option.
- Enter the *Segment Size*. This is equivalent to using the `[-segment <cycleNumber> | <percent>]` option. You can switch % (percent) and cycles/time (number) as unit by clicking the button adjacent to the *Segment Size* field.



- Click *Generate Backward SAIF File* to generate the output file according to the currently specified settings.

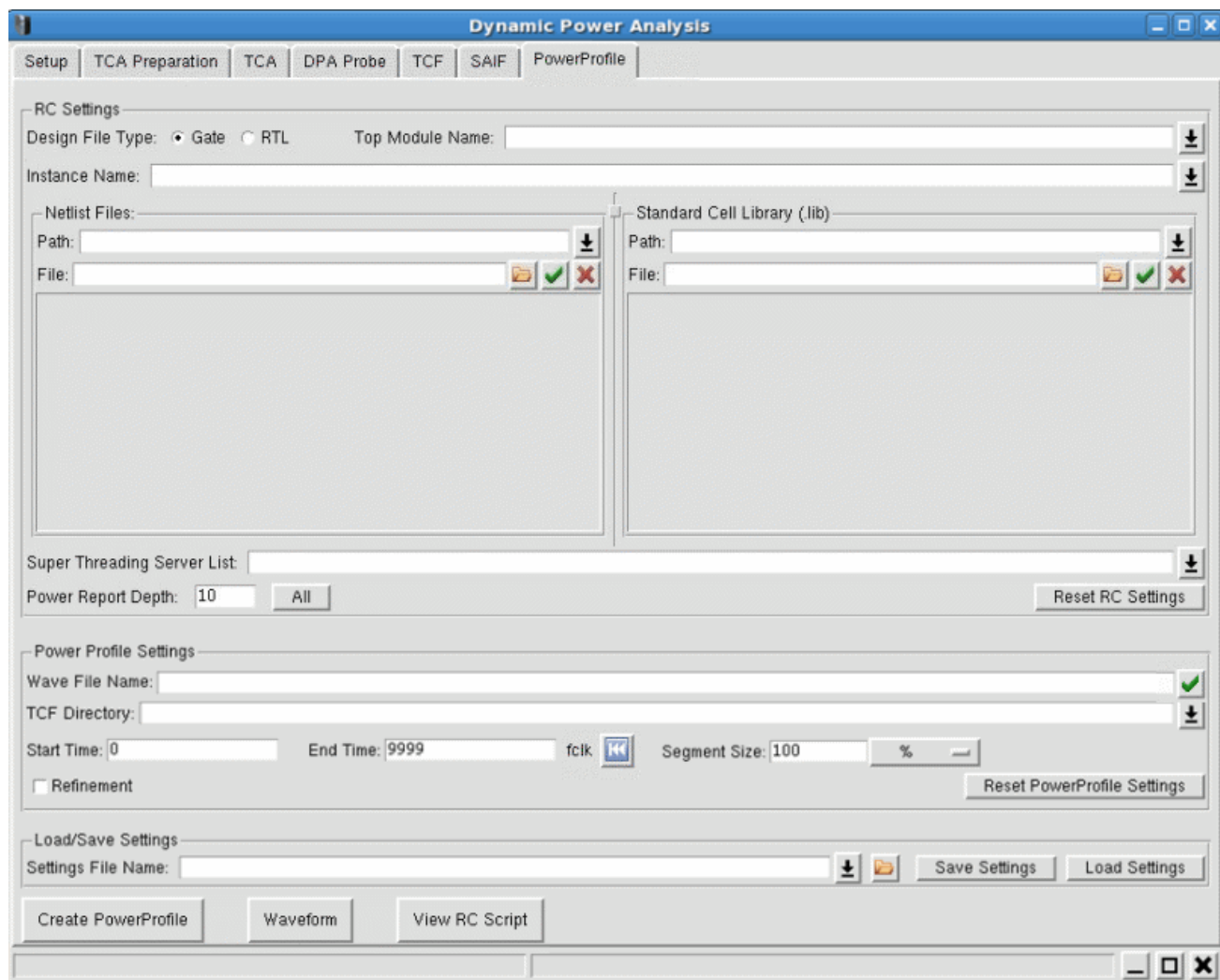
Note: If you entered a backward SAIF filename which already exists, a dialog box will be displayed to confirm whether or not you want to overwrite it.

PowerProfile Tab

The *PowerProfile* tab (as shown in [Figure 15-26](#)) allows you to generate TCF files, power report files, and power profile in automation mode. It provides controls for the following actions:

- [RC Settings](#)
- [Power Profile Settings](#)
- [Load/Save Settings](#)
- [Generate Power Profile](#)

Figure 15-26 PowerProfile Tab in ICE Mode



RC Settings

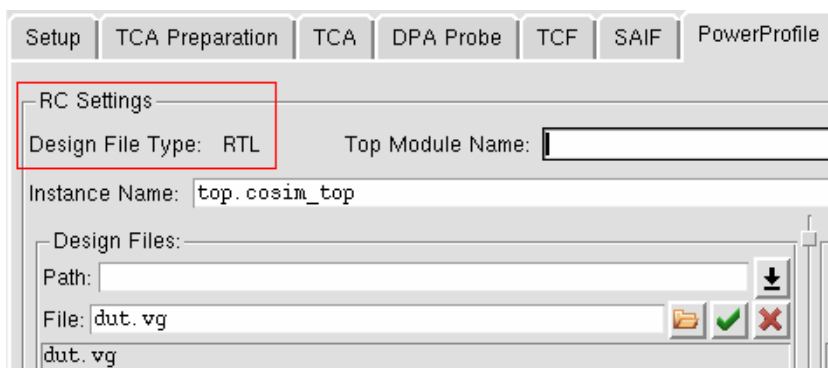
Use the *RC Settings* group box to set the parameters for invoking the RTL Compiler as following:

1. Select the *Design File Type* as either *Gate* or *RTL*. This field is an equivalent to specifying the `dpa -setupRC {-gateFlow | -rtlFlow}` command. Based on your selection, the RTL Compiler will be invoked for either gate-level netlist flow or RTL design flow.

Important

In SA mode, only RTL design type is supported for creating power profile. Therefore, on the *PowerProfile* tab, the *Design File Type* field of the *RC Settings* group box displays RTL as a static text (as shown in [Figure 15-27](#)).

Figure 15-27 PowerProfile Tab - RC Setting for RTL Design Files in SA Mode



2. Specify the *Top Module Name*, which is an equivalent to the `dpa -setupRC -topDesign <top_module_name>` command. This action will set the top-level module name that the RTL Compiler should use to:
 - elaborate the design when the *Design File Type* field is selected as *RTL*.
 - utilize the Super Threading feature when the *Design File Type* field is selected as *Gate*.
3. Specify the *Instance Name*, which is an equivalent to the `dpa -setupRC -instance <instance_name>` command. This action sets the instance name that the RTL Compiler uses to create the probe nets for TCF generation.

You can drag and drop instance names from other browsers, like Design Browser, into the *Instance Name* text field.

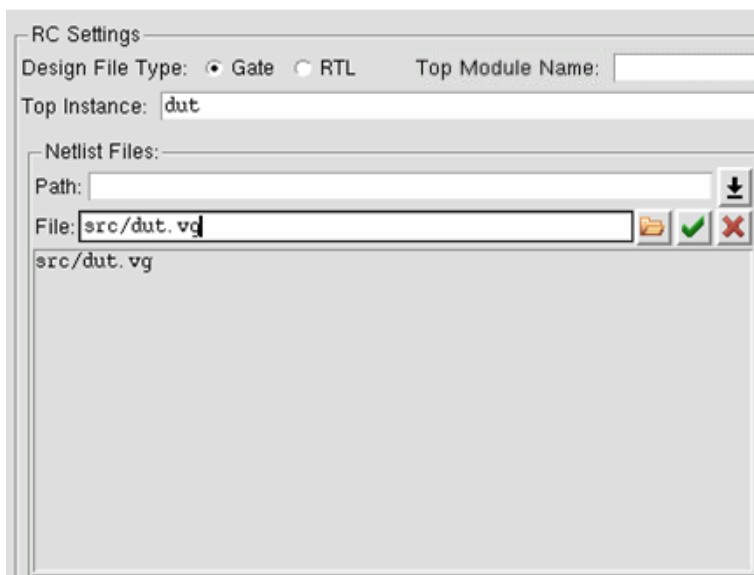
4. When the *Design File Type* field is selected as *Gate*, you should specify the gate-level netlist files in the *Netlist Files* group box. This is equivalent to the `dpa -setupRC -designFile {<file_name_list>}` command. Specify the *File* name or click the button with directory icon to browse and select a file. Then, click the  button to add the specified file into the listbox below the *File* text field.

Note: To remove a file from the listbox, select the specific file and click the  button.

If you do not enter the path to the specified netlist in the *Path* text field, enter the full path with the netlist filename in the *File* text field. The path should be specified with respect to the current directory. For example, refer to [Figure 15-28](#) in which the filename is specified as `src/dut.vg` because no input has been defined in the *Path* text field.

Note: You can specify multiple paths in the *Path* text field. Ensure that each path is separated by a space.

Figure 15-28 PowerProfile Tab - RC Setting for Gate-Level Netlist Files in ICE Mode



When the *Design File Type* field is selected as *RTL*, the *Netlist Files* group box is instead displayed with the label *Design Files* (as shown in [Figure 15-29](#)). Specify the RTL design files in the same way as discussed above.

You can add or remove multiple netlist or RTL design files using either of the following ways:

- ❑ ***Global patterns***

UXE User Guide

Analyzing Power Consumption of the Designs

- a. Specify global patterns in the filename in the *File* text field. All netlist or RTL design files with name that matches the specified pattern will be either added or removed, as the case is.

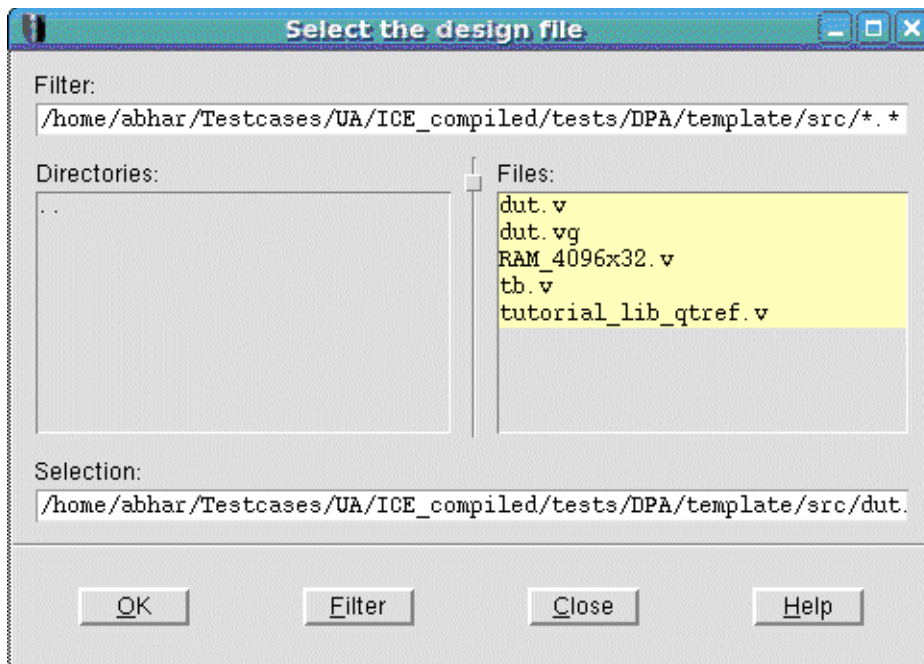
For example, if you specify `src/dut*.v` and click the  button, all files with name matching the `dut*.v` pattern under the `src` directory will be added at the time of power profile creation.

Note: All files matching the specified pattern will not be listed in or removed from the listbox. However, when you click the *Create PowerProfile* button after completing the mandatory specifications, the related action will be performed at the back end.

Mouse pointer

You can select multiple files using the mouse only when the required files are listed consecutively. Perform the following steps:

- a. Open the *Select the design file* dialog box by clicking the directory icon next to the *File* text field to add multiple files.

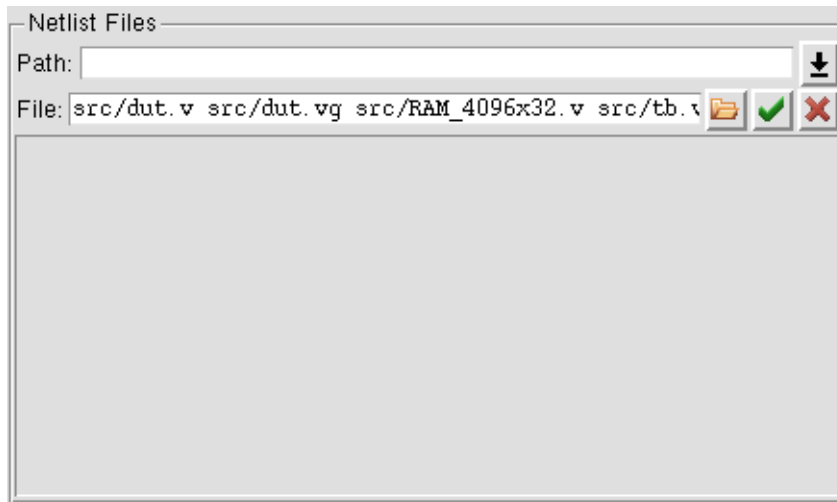


- b. Access the directory in which your netlist or RTL design files are located.
- c. Click the first file and drag the pointer keeping the left button of the mouse pressed until the last file you want to select. All selected files will be highlighted with yellow-colored background. Refer to the above figure.

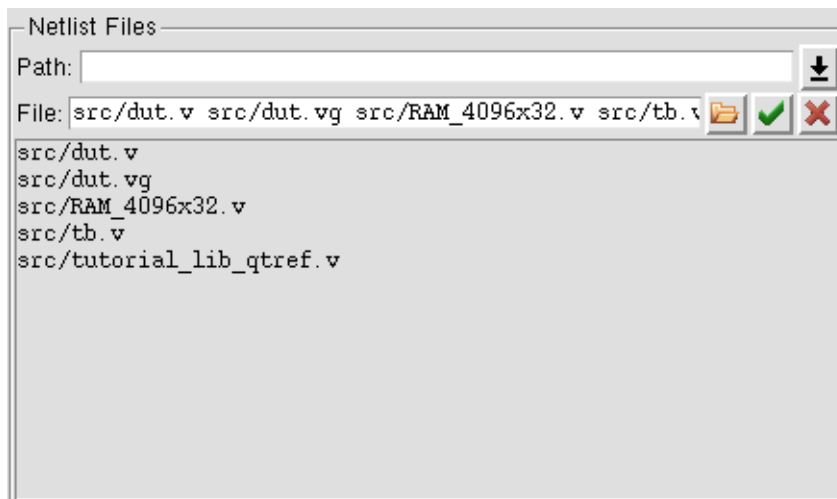
UXE User Guide

Analyzing Power Consumption of the Designs

- d. Click *OK*. The names of the selected files along with the path will be displayed in the *File* text field as shown in the figure below.



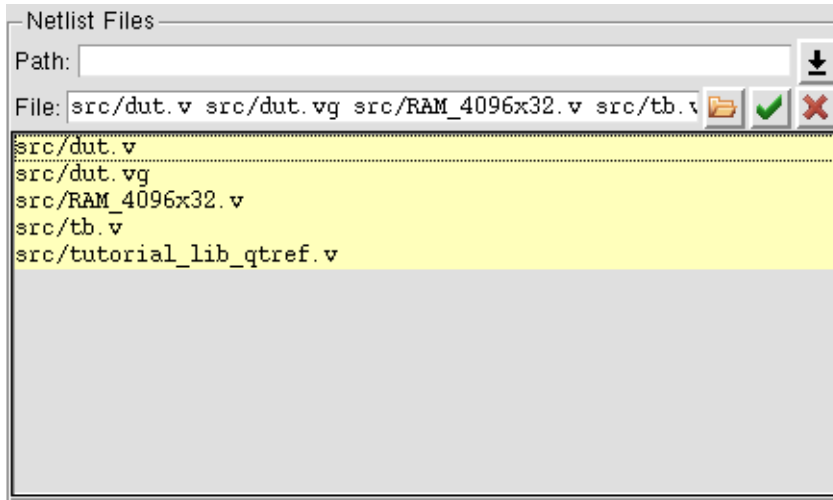
- e. Click the  button to add the selected files into the listbox as shown in the figure below.



UXE User Guide

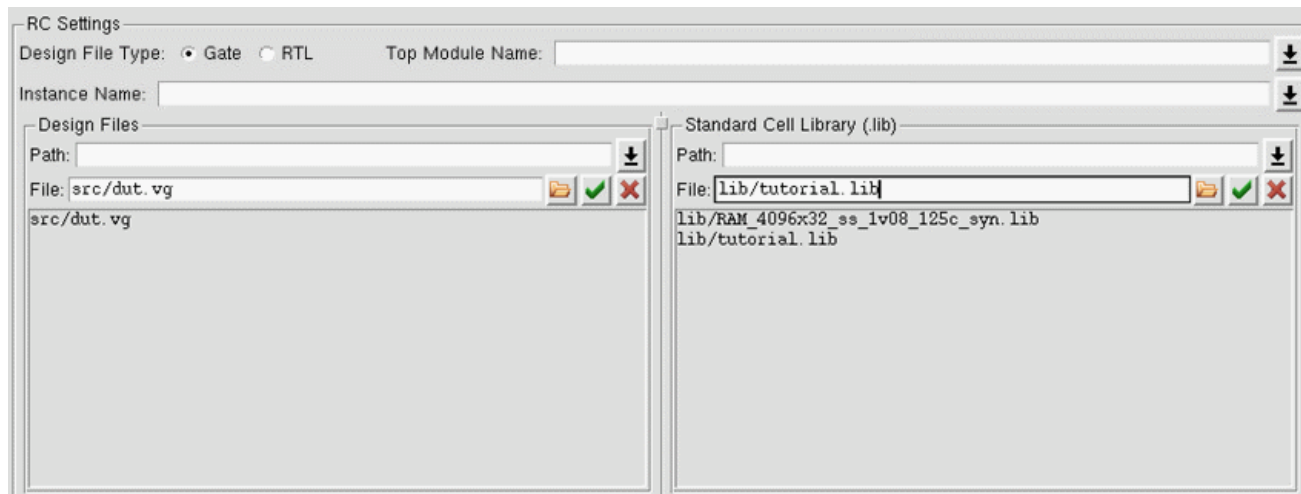
Analyzing Power Consumption of the Designs

Similarly, to remove multiple files, you can select the files from the listbox (as shown in the figure below) and click the  button.



5. Set the target technology library used by the RTL Compiler in the *Standard Cell Library (.lib)* group box (illustrated in Figure 15-29). This is an equivalent to the `dpa -setupRC -libList {<lib_name_list>}` command.

Figure 15-29 PowerProfile Tab - RC Setting for Standard Cell Libraries



Specify the library name in the *File* text field or click the button with directory icon to browse and select the required file. Then, click the  button to add the specified file into the listbox below the *File* text field.

- You can specify multiple library names in the *File* text field. Ensure that each library name is separated by a space.

- To remove a file from the listbox, select the specific file and click the  button.

If you do not enter the path to the specified library in the *Path* text field, enter the full path with the library name in the *File* text field. The path should be specified with respect to the current directory. For example, refer to [Figure 15-29](#) in which the filename is specified as `lib/tutorial.lib` because no input has been defined in the *Path* text field.

6. Specify a list of servers if you want to use the Super Threading feature of the RTL Compiler. Multiple server names should be separated with space. This text field is an equivalent to the `dpa -setupRC -servers {<machine_list>}` command.

Note: This feature is available only when you select the *Design File Type* field as *Gate*. Consequently, the *Super Threading Server List* text field is not displayed when running the design in SA mode or when *RTL* is selected as the *Design File Type* in ICE mode.

7. Specify the number of hierarchy levels to descend in the power report data in the *Power Report Depth* text field. This is an equivalent to the `dpa -setupRC -depth <N>` command. The default value is set to 10. Clicking the *All* button will set the depth to 999999, which means *all levels*.

Important

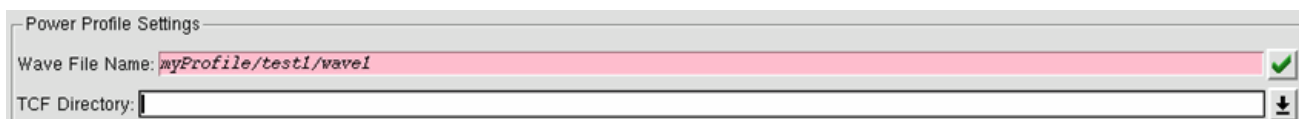
Clicking *Reset RC Settings* enables you to reset the settings defined for the RTL Compiler to default values.

Power Profile Settings

Use the *Power Profile Settings* group box to set the parameters for generating a power profile as following:

1. Specify the name of the power profile in the *Wave File Name* text field. A power profile is generate as a waveform of SST2 format. This field is an equivalent to the `dpa -setupPowerProfile -database <name>` command.

When you are entering the waveform filename, the text field is displayed in pink (refer to the image below).

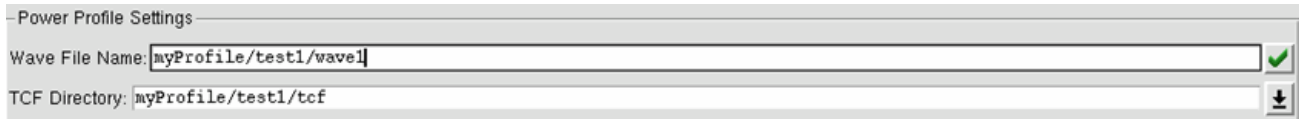


Either click the  button next to the text field or press the *Enter* key to confirm the entry. Based on the text entered in the *Wave File Name* text field, the *TCF Directory* text field

UXE User Guide

Analyzing Power Consumption of the Designs

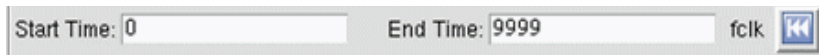
gets automatically populated with the full path to the default directory (refer to the image below).



2. Specify the *TCF Directory* for saving the TCF files generated during power profile computation. This is an equivalent to the `dpa -setupPowerProfile -tcf <directory>` command.

You can either use the default directory generated after entering the *Wave File Name* or set another directory. If the *TCF Directory* text field is empty, it will get automatically populated with the full path to the default directory when the *Wave File Name* is set.

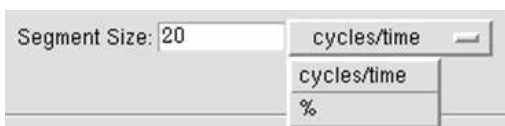
3. Specify the required cycle time in the *Start Time* and *End Time* fields. These fields are an equivalent to the `dpa -tracewindow -start <time> -end <time>` command.



The current time unit is displayed next to the two fields. For example, `fclk` is displayed in the above figure.

Note: When you define a new time period, use the  button to apply the changes.

4. Specify the *Segment Size*. You can switch the unit to either % (percent) or cycles/time (number). This is an equivalent to the `dpa -setupPowerProfile -segment {<n>% | <cycleNumber>}` command.



5. Select the *Refinement* check box if you want to refine the power consumption data that was reported last time.

Note: If you have already generated power profile without *Refinement*, you can re-generate power profile with *Refinement* based on the previous result. To do *Refinement*, you must make sure that either the time range or segment size is less than the previous power profile. Also, the design file and `.phy` file must be the same as in the last run. Usually, only two options need to be changed, which are the *Start/End time* and *Segment Size*.

6. Click the *Reset PowerProfile Settings* button to reset the settings for generating TCF files.

Load/Save Settings

You can save all RTL Compiler and Power Profile settings to a file so that these settings can be reloaded later.

In the *Load/Save Settings* group box, enter a *Settings File Name* to which the settings should be saved and click *Save Settings*.

To load the settings from an existing file, specify the filename in the same text field and click *Load Settings*. You can use the button with directory icon to browse and select the required settings file.

Note: When the RTL Compiler and Power Profile settings are loaded from the specified file, the current values will be overwritten.

Generate Power Profile

After specifying the RTL Compiler and Power Profile settings, click the *Create PowerProfile* button to generate a power profile. The cursor remains in *busy* state until the process is finished.

If any problem occurs during the process, a failure message is shown on the status bar of the Dynamic Power Analysis window. You can see more information in the commands console in the xeDebug GUI.

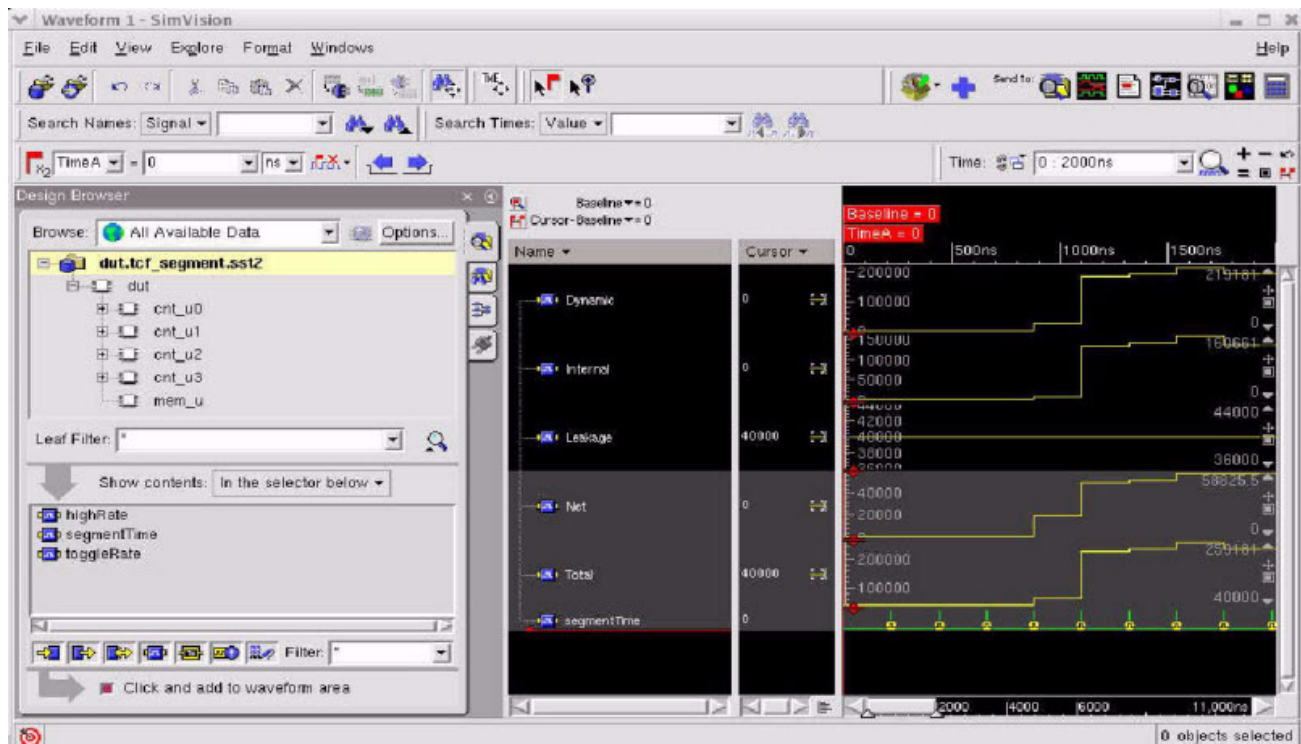
After the power profiles has been generated, you can click the *Waveform* button to bring up the SimVision application to show the SST2 format waveform file.

Figure 15-30 shows an example of the SimVision viewer displaying an expanded power profile. A second view, in Figure 15-31 shows the power profile waveform in more detail.

UXE User Guide

Analyzing Power Consumption of the Designs

Figure 15-30 SimVision Displaying Power Waveforms



UXE User Guide

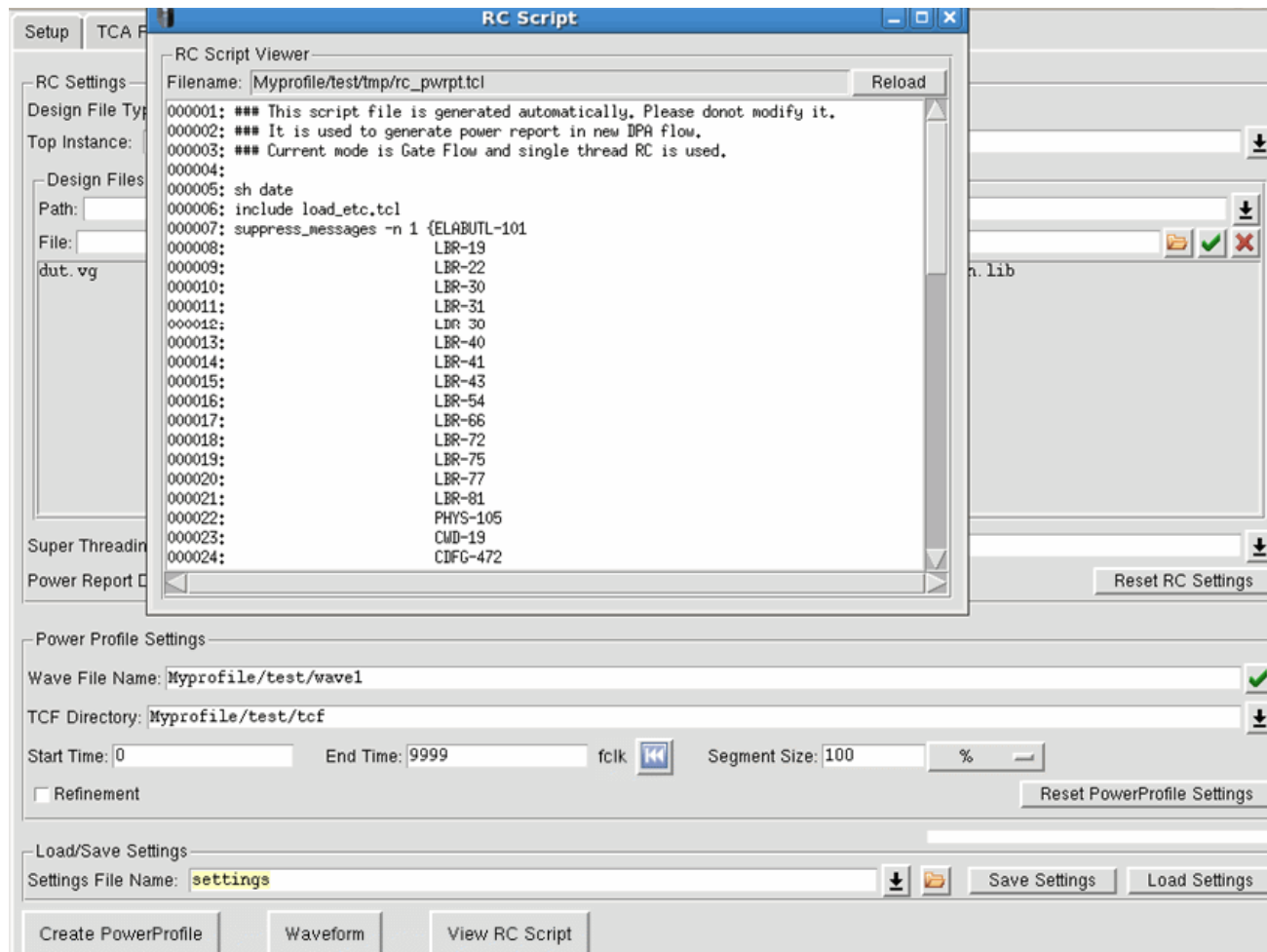
Analyzing Power Consumption of the Designs

Figure 15-31 Power Profile



You can click the *View RC Script* button to see the RTL Compiler script generated in the latest processing. The script file is always named as `rc_pwrpt.tcl` and is saved in the `<wave_file_directory>/tmp` directory. The script viewer displays the script in read-only mode, this means you cannot edit or save this script. You can only reload the file by clicking the *Reload* button.

Figure 15-32 RC Script Window



Analyzing Power Consumption with CPF

DPA recognizes the power constraints defined for low-power applications using the Common Power Format (CPF). CPF-aware DPA provides the mechanism to identify the logic applied to the power domains and adjust the power calculations to reduce the dynamic power.

For detailed information on the CPF implementation, refer to [Chapter 16, “Specifying Low-Power Intent on Palladium XP.”](#)

DPA supports CPF while performing the following operations:

- Calculating NTC or WTC

The NTC or WTC of a power domain can be calculated by summing up toggle count data and high count data from the instances that are included in the power domain.

- Generating power profile in TCF or RTL Compiler flow

Working with Power Domain in DPA GUI

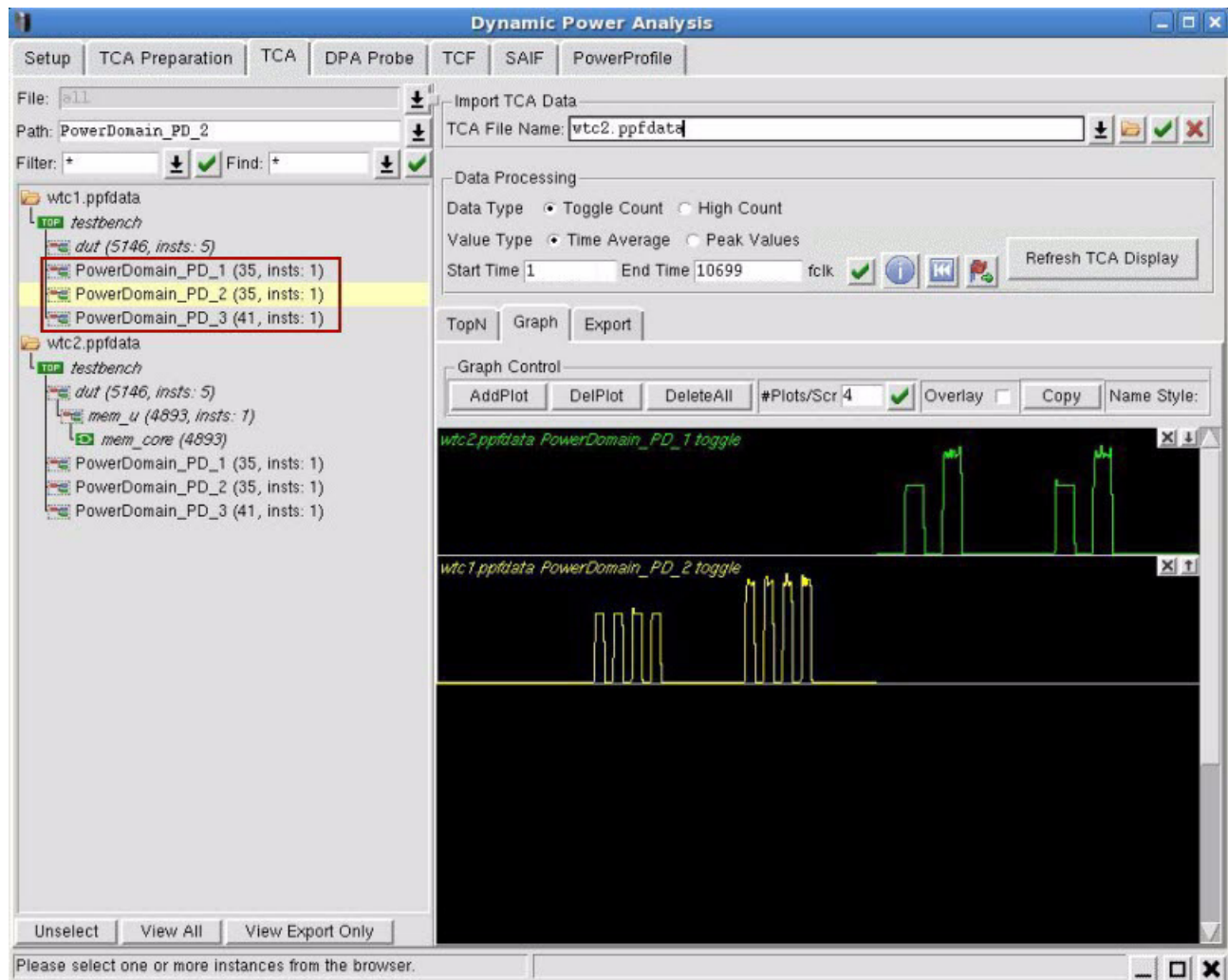
You can define instances on the *TCA* tab of the *Dynamic Power Analysis* GUI window. To add a power domain into *.ppfdata file, add the power domain name in the DPA GUI.

In the DPA GUI, power domain is displayed as *virtual instance* which is integrated into the hierarchical tree of normal instances in the design, displayed in [Figure 15-33](#). Cadence recommends to prefix the name of the power domain with `PowerDomain` in order to avoid confusing power domains with normal instances. For example, in [Figure 15-33](#), the names of the power domains are `PowerDomain_PD_1`, `PowerDomain_PD_2`, and `PowerDomain_PD_3`. According to the CPF standard, a power domain can have the same name as the normal instance in the design. The hierarchy for normal instances is shown under *dut* and power domains are shown below normal instances under the Power Domain icon.

On double-clicking the power domain the hierarchical tree is expanded, displaying the instance names for that power domain. This allows you to explore instance's hierarchy starting from a power domain.

Power domains are treated as *virtual instance* and you can perform any operations on the power domain as they perform on normal instances. For example, in [Figure 15-33](#), the power profile graph for power domain `PowerDomain_PD_1` and `PowerDomain_PD_2` is displayed.

Figure 15-33 DPA Power Domain



Creating *.ppfddata File for Power Domain

Specify the instances to be included in the *.ppfddata file for power domain using the following command:

```
dpa -setupToggleCount
```

The power domain information is saved in the *.ppfddata file. Use the following command to create the *.ppfddata file:

```
dpa -getToggleCount
```

If the `dpa -setupToggleCount` command is not specified, automatically a default set of instances is included in the `*.ppfdata` file. All the power domains are included in the default set.

Both the power domains and the normal instances name can be specified with the `dpa -setupToggleCount` command option. For example:

```
dpa -setupToggleCount -addinst normal_inst1 \  
normal_inst2 power_domain_1
```

Generating TCF and Power Profile Using Power Domain and Power Mode

The toggle count and high count is zero if a power domain is shut off. The toggle count and high count for power domain along with all the sub-instances under power domain will have zero data.

The software reports power consumption for power domains and when a power domain is shut down its power consumption is set to zero. The RTL Compiler report is post-processed to achieve this.

First, the power number from all the sub-instances is summed-up to get the power number of the power domain.

Second, the power number is set to zero, if a power domain is shut off. TCF generation takes CPF into account for this. Usually, multiple TCF files are generated for a time window. Each TCF file represents a small time window. You can specify the number of cycles one TCF file should cover. Therefore, the whole time window is partitioned into smaller segments. When creating TCF file for one segment the event of power shut-off state is monitored. Whenever a power domain is switched-on or shut-off, a new partition is created at that point. This ensures that each power domain is either on or off for one TCF file. As a result, the number of TCF files generated is larger with CPF enabled compared to when CPF is not enabled.

When using CPF-aware DPA, you will get more TCF or SAIF files than specified with the `-segment` option. The reason is that a new segment (TCF or SAIF file) is created when a power domain is either turned on or off.

Using Sparse Sampling with DPA

The sparse sampling mechanism allows you to sample once every `N` cycles instead of sampling every `FCLK` cycles. This improves the run-time performance and increases the DPA coverage in LA mode.

Note: Sparse sampling is not supported when performing DPA operations in the SA mode and Vector Debug mode.

The DPA features of capturing the toggle count data (both NTC and WTC) and generating the TCF/SAIF files take time. Consequently, when DPA needs to be performed on large trace windows with millions of FCLK cycles, it takes a long time to upload each cycle. In such situations, using sparse sampling for DPA reduces the amount of data that needs to be uploaded and thus, improves the performance.

In LA mode, Data Capture Card (DCC) can capture only once. Therefore, capturing multiple times and merging them into a large trace window is not possible. This limits DPA in LA mode to the maximal trace depth, which is typically 1M cycles. With sparse sampling, the trace window can cover more cycles.

The sparse sampling is useful to analyze average power; but it is not good at peak power. Select an appropriate sample rate according to the performance and power analysis requirements. With higher N value, you can get larger improvement on performance and trace window size for LA mode. However, there is a higher risk of missing some import power usage change, such as peak.

Sparse sampling is transparent when the NTC or WTC feature is used. You can run all XEL commands and GUI as if sparse sampling was not enabled.

Sampling rate is set at multiple of frame size, which is generally 32 cycles for most designs.

Note: This document uses 32 in the following sub-section. However, in UXE, the frame size need not be always equal to 32.

Enabling and Disabling Sparse Sampling

Use the following XEL command to enable or disable sparse sampling:

```
dpa -sparse_sampling [<N>]
```

In the above command, the numeric value $<N>$ specifies the sample rate at which a sample should be captured after every $<N>$ FCLK cycle. If $<N>$ is not a multiple of 32, then it will be automatically rounded-off to the nearest multiple of 32. For example, if $<N>$ is specified as 1000, it will be rounded-off to 992. When sparse sampling is enabled, a sample is continuously captured for 32 FCLK cycles, from which 31 toggle counts and 32 high counts are generated. Since 32 FCLK cycles are captured for each sample, the actual benefit of using sparse sampling is not $<N>$ times, instead it is $<N>/32$ times. When capturing NTC or WTC data, an average is calculated and only one toggle count number and one high count number is provided for those 32 FCLK cycles.

Specifying $<N>$ as 0 disables sparse sampling.

When the `dpa -sparse_sampling` command is used without specifying a value for `<N>`, the current sample rate is returned.

After enabling sparse sampling, user can start the emulation using the `run` command. However, after running the emulation process, any conditional acquisition statements in your SDL program will be ignored.

Note: When using the xeDebug GUI for DPA, before running the emulation, specify a sample rate on either the *Control* tab or the *InfiniTrace* tab. For information about these two tabs, refer to the [Control Tab](#) and [InfiniTrace Tab](#) sections in [Appendix F, “Using the xeDebug GUI.”](#)

Estimating Power Using TCF/SAIF Files with Sparse Sampling

You can perform TCF and SAIF-based power estimation even when using sparse sampling for DPA. This approach speeds up the power estimation by sampling only a part of the run activity at a fixed rate.

The steps involved are the same as normally followed in RTL or gate-level power analysis. Refer to the [RTL Power Analysis](#) and [Gate-Level Power Analysis](#) sections.

The only difference is that, before generating the `*.phy` file during run time, you should specify the sparse sampling ratio using the `dpa -sparseSampling <N>` command.

Improving Accuracy of Power Estimation

Though gate-level netlists provide more accurate power estimates than netlists generated from RTL designs, there are some considerations and commands that can help you improve accuracy.

Generating High Quality TCF Files

To generate high quality TCF files, consider the following:

- Ensure that the time taken for generating a particular TCF file is the same as expected. The segment time can be retrieved from the TCF file header or TCF log file.
- Always specify the real clock frequency because Palladium XP uses cycle-based method for calculating toggle information.
- Take into account the mode of clocking that you are using, that is, CAKE 2X or CAKE 1X.
- Adjust the `traceMemSize` run-time parameter to cover the number of samples needed. This helps because the physical trace memory available in Palladium XP limits the time

range covered by TCF files. If the time range is larger than what the trace buffer can cover, then run just enough cycles to fill the trace memory, and perform upload for each run.

Estimating Accurate Power Consumption of RTL Designs

Use the RTL Compiler to perform the following operations to improve accuracy of the RTL-level power estimation:

- Build an RTL power model that puts additional information into the RTL Compiler database for calculating power. Do this operation only for one TCF file because the model remains the same with the activity. After power has been reported once based on the model, acquire a scaling factor that is used in subsequent power calculations as shown in the following example:

```
build_rtl_power_models -clean_up_netlist // Enhance database with model
information
report_power -tcf_summary // report TCF summary
set rtlpe_factors [get_attr lp_rtlpa_scale/des*/* ]// Grab scaling factor
```

- Move to a generic gate-level netlist using the following command:

```
synthesize -to_generic
```

- Include wire load models from the Liberty files, as shown below. This can result in multiple net name mismatches, but there will also be a lot of nets that are recognized.

```
set wlm [find / -wireload_selection *]
set_attr wireload_selection [lindex $wlm 1]
```

Estimating Accurate Power Consumption of Gate-Level Netlists

The following ways can help you improve accuracy for gate-level power analysis:

- Include clock tree and physical transmission delays. Always use this information, if available. Reporting flat power instead of hierarchical power allows you to analyze the local contribution of clock buffers in a better way. The hierarchical power report will likely place all clock tree power in the top level instance.
- Use the `read_spf` RTL Compiler command to add SPEF file information, especially when using wire load models. The SPEF files contain resistance, capacitance, and inductance for wires based on the physical layout.
- Use the `read_sdc` RTL Compiler command to retrieve timing and clocking information from an SDC file.

For more information on the `read_spf` and `read_sdc` RTL Compiler commands, refer to *Command Reference for Encounter® RTL Compiler, Version 10.1*, available on the Cadence Online Support website -> *Digital IC Design -> Encounter® RTL Compiler 10.1* set of documentation.

Cadence recommends that you always add wire load models, SPEF files, and SDC files when available.

DPA Licenses

The DPA license control requires 29SDPAT.

When using the xeDebug tool, you have the choice of using DPA licenses or other licenses while running SimVision and RTL Compiler:

- For viewing waveforms, either use the `DPA_SimVision` license or the `IES Affirma_sim_analysis_env` license.
- For RTL Compiler usage, either use the `DPA_RC_PowerEstimation` license or the `RC` license.

This product contains the following five licensed features for different DPA functions:

- `DPA_ToggleCountAnalysis`
- `DPA_TCAExport`
- `DPA_ToggleCountFormat`
- `DPA_SimVision`
- `DPA_RC_PowerEstimation`

These licensed features provide the following functionalities:

- By default, the *Power Tower* icon is set to ON in the xeDebug GUI, irrespective of the availability of the licenses. Therefore, you can simultaneously work on the different DPA functions from different xeDebug processes.
- Both `DPA_RC_PowerEstimation` and `RC` licenses are supported by the `DPA_RC` program. The xeDebug tool invokes the `DPA_RC` program using either of these two licenses depending on the specified user settings. The license is released after the RTL Compiler library has been generated or the power calculation has been done, without requiring you to exit from the xeDebug tool.

Note: To use the `RC` license, you can invoke the RTL Compiler directly instead of using

the DPA_RC program.

- The xeDebug tool supports invoking SimVision directly. The `DPA_SimVision` license is released when the SimVision process exits, without requiring you to exit from xeDebug.
- If the `DPA_SimVision` license is available, it is used for the `dpawave` program. Otherwise, the `IES Affirma_sim_analysis_env` license is used. To use the IES license, use the `simvision -waves` command instead of using `dpawave`.

In addition to the licensed features, the xeDebug tool provides the following two user-defined Tcl global variables that enable you to set the preferred license keys:

- `dpaRClic`

By default, the value of this variable is set to 1 and it means that the `DPA_RC_PowerEstimation` key will be used. You can set it to 0 by using the following command to use the RTL Compiler license key:

```
XE> set dpaRClic 0
```

- `dpaSVlic`

By default, the value of this variable is set to 1 and it means that the `DPA_SimVision` key will be used. You can set it to 0 by using the following command to use the SimVision license key:

```
XE> set dpaSVlic 0
```

DPA Tutorial Examples

A tutorial example for DPA can be found in the `<installation_dir>/gift/tutorial/dpa` directory. The `README` file in this directory provides the instructions on the use of the tutorial.

UXE User Guide

Analyzing Power Consumption of the Designs

Specifying Low-Power Intent on Palladium XP

Common Power Format (CPF) is the standard format for specifying design- and technology-related power constraints for low-power applications. CPF provides the infrastructure to combine the RTL design files with CPF commands and simulate operations of a power saving circuit.

The supported low-power verification techniques are Power Shut-Off (PSO), Multiple Supply Voltage (MSV), and Dynamic Voltage Frequency Scaling (DVFS). For details, refer to the [Selecting Low-Power-Aware Verification Method Based on Design Type](#) section.

Palladium XP supports a limited set of CPF 1.1 and 2.0 standards. The [Design Considerations for Low-Power Verification](#) section discusses the deviations from the supported CPF standards and restrictions that need to be observed while verifying a low-power-aware design on Palladium XP. For the list of supported and unsupported CPF commands, refer to [Table 16-1](#) in the [Support for CPF Commands on Palladium XP](#) section.

For information on the CPF 1.1 standard for the supported low-power verification techniques, refer to the *Common Power Format User Guide, Version 1.1* and *Common Power Format Language Reference, Version 1.1*, available on the *Cadence Online Support* website -> *Product Pages* menu -> *Product Manuals* sub-menu -> *Digital IC Design* section -> *Encounter® RTL Compiler 11.1* -> *RC 11.1* -> *Product Documentation for RC 11.1* web page.

For information on the CPF 2.0 standard, refer to *Si2 Common Power Format Specification™, Version 2.0* document.

Important

The following are Beta-level features:

- ❑ Support for CPF 2.0 standard

For details, refer to the [CPF 2.0 Features Supported on Palladium XP \(Beta-Level Feature\)](#) section.

UXE User Guide

Specifying Low-Power Intent on Palladium XP

- ❑ Support for generating CPF event logs

For details, refer to the [Logging of CPF Events \(Beta-Level Feature\)](#) section.

Beta-level features are those that have been introduced in one of the recent releases but have not yet reached the production stage. The stability and usability of these features is not well established. Beta-level features are released to you to obtain your feedback before the features reach the production stage.

Support for CPF Commands on Palladium XP

UXE does not support the entire set of functionality available in the *Cadence Common Power Format Language Reference, version 1.1* and *version 2.0*. For option-wise details and syntax of the supported CPF commands, refer to the *Common Power Format Commands* chapter in the *UXE Command Reference Manual*.

The following table lists the supported and unsupported CPF commands:

Table 16-1 Supported and Unsupported CPF 1.1 and 2.0 Commands

Category	Supported CPF Command(s)	Unsupported CPF Command(s)
Version command	set_cpf_version	-
Modeling commands	end_design get_parameter set_design set_instance update_design ¹	set_pad_ports
Power mode commands	assert_illegal_domain_configurations create_mode ^a create_mode_transition create_power_mode	set_power_mode_control_group end_power_mode_control_group update_power_mode

UXE User Guide
Specifying Low-Power Intent on Palladium XP

Category	Supported CPF Command(s)	Unsupported CPF Command(s)
General-purpose commands	<code>find_design_objects^a</code> <code>include</code> <code>set_hierarchy_separator</code>	<code>set_power_unit</code> <code>set_time_unit</code> <code>set_array_naming_style</code> <code>set_register_naming_style</code>
Macro support commands	<code>set_macro_model</code> <code>end_macro_model</code>	<code>set_floating_ports</code> <code>set_input_voltage_tolerance</code> <code>set_wire_feedthrough_ports</code>

UXE User Guide

Specifying Low-Power Intent on Palladium XP

Category	Supported CPF Command(s)	Unsupported CPF Command(s)
Design constraint commands	create_isolation_rule create_nominal_condition create_power_domain create_state_retention_rule set_sim_control ^a	create_analysis_view create_bias_net create_global_connection create_ground_net create_level_shift_rule create_operating_corner create_pad_rule create_power_nets create_power_switch_rule define_library_set identify_power_logic identify_secondary_domain set_equivalent_control_pins set_power_target set_switching_activity update_level_shifter_rules update_nominal_condition update_power_switch_rule update_isolation_rules update_state_retention_rules

UXE User Guide

Specifying Low-Power Intent on Palladium XP

Category	Supported CPF Command(s)	Unsupported CPF Command(s)
Library-related commands	-	define_always_on_cell define_isolation_cell define_state_retention_cell define_level_shifter_cell define_open_source_input_pin define_power_clamp_cell define_power_switch_cell define_related_power_pins
Verification support commands	-	create_assertion_control

1. This command has been introduced in the CPF 2.0 standard.

Note: When modeling low-power intention of hard-custom IPs in a CPF file, you can use only a specific set of CPF commands and options in the different parts of the CPF file. For more information, refer to the sub-sections of the Modeling Hard-Custom IP Low-Power Intention in a CPF File section.

CPF 2.0 Features Supported on Palladium XP (Beta-Level Feature)

The section lists the CPF 2.0 features supported on Palladium XP in the table below. For detailed information about these features, refer to the *Enhancements and New Capabilities for CPF 2.0* chapter of the *Si2 Common Power Format Specification, Version 2.0*.

Enhancements and new capabilities for CPF 2.0	
Design/instance semantic change	■ set_design semantic change ■ Append power intent ■ Virtual port expend ■ Instance port mapping

UXE User Guide

Specifying Low-Power Intent on Palladium XP

Enhancements and new capabilities for CPF 2.0

Simulator-related

- Specify simulation action to control PSO behavior
- Test bench CPF model
- Power off corruption model

Exclude instances or pins in power domain

Additional operators for expression

Note: Only logical operators ! (unary logical negation), && (logical AND), and || (logical OR) are supported with operands as a port or pin. However, bitwise XOR operator (^) is not supported.

Design and Instance Semantic Differences in CPF 1.1 and CPF 2.0

Following are the important differences in semantic of design and instance related commands in context of CPF 1.1 and CPF 2.0 standards:

- The semantics of the `set_design` command have been changed as following:
 - CPF 1.1 specified the power intent for one *module*.
 - CPF 2.0 specifies an entity, referred to as a *power design*, that defines power intent. A power design may be applied to
 - a top design, or
 - instantiations of any module, or
 - instantiations of one or more explicitly specified modules
- The rules for appending power intent to an existing design are different as following:
 - CPF 1.1 appends the power intent to an existing design using additional pairs of the `set_design` and `end_design` commands.
 - CPF 2.0 appends the power intent to an existing power design using the new `update_design` command paired with the `end_design` command.
- The semantics for virtual ports have been expanded as following:
 - In CPF 1.1, the `-ports` option of the `set_design` command specified input ports only.

- In CPF 2.0, the new `-input_ports`, `-output_ports`, and `-inout_ports` options of the `set_design` command explicitly specify the input, output, and inout ports.

Note: The `-ports` option still exists for backward compatibility.

- In CPF 2.0, the semantics of the `-port_mapping` option of the `set_instance` command have been clarified to include the specification of real ports in addition to virtual ports.

Processing CPF Information on Palladium XP

Perform the following steps to use CPF for analyzing low-power-aware designs on Palladium XP:

1. Create a CPF file to specify the power intent for the design and the technology information. Refer to the [Creating CPF Files](#) section.
2. Add the power information recorded in the CPF files to your design (RTL design or gate-level netlist) by using the `readCPFfile` command. Refer to the [Adding CPF Information to a Design](#) section.
3. Compile your low-power-aware design that comprises the design files and the CPF files. Refer to the [Compiling a Design with Low-Power Intent](#) section.

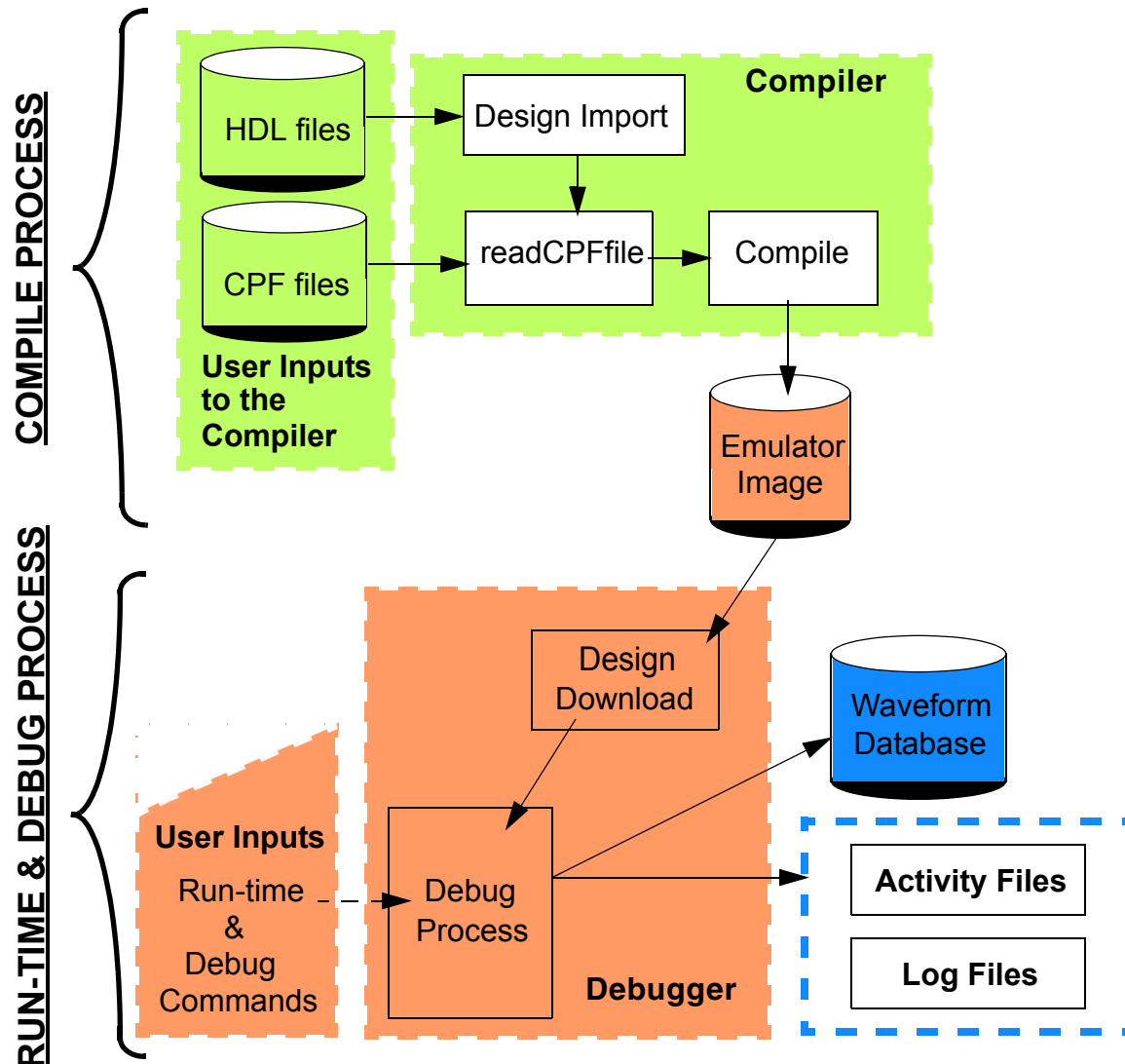
The compiler processes these files to generate the emulator executable image.

Note: On Palladium XP, the behavior of the CPF files is based on the synthesized gate-level netlist, instead of the original RTL design.

4. At run time, download the emulator executable image on the debugger. Refer to the [Running and Debugging a Compiled Design with Low-Power Intent](#) section.
5. Check the signal values in the SST2 or FSDB format waveforms by using the SimVision or Debussy waveform viewer tool.

[Figure 16-1](#) illustrates the flow for processing CPF information in UXE.

Figure 16-1 Flow for Processing CPF Files in UXE



Creating CPF Files

Create a CPF file that contains the following information:

- CPF design model defined using the `set_design` and `end_design` CPF commands
Note: Multiple embedded `set_design` and `end_design` command pairs are not supported.
- Power domains created using the `create_power_domain` CPF command

- Nominal conditions and power modes defined using the `create_nominal_condition` and `create_power_mode` CPF commands, respectively
- Rules created for isolation using the `create_isolation_rule` CPF command
- Rules created for state retention using the `create_state_retention_rule` CPF command

To use multiple CPF files, you must create a master CPF file and use the `include` command to source into it the supporting CPF files, which contain the additional CPF information.

For example, create a `design_TOP.cpf` master CPF file containing the basic information used by the tool. Also, create two supporting CPF files, `library_1.cpf` and `impl_1.cpf`, that contain the additional power information. To add the content of the supporting files in the master CPF file, specify the following two commands in `design_TOP.cpf`:

```
include library_1.cpf
include impl_1.cpf
```

While running and debugging your low-power-aware design using the xeDebug GUI, you can view and edit the content of the associated CPF files in the *Power Format Verification* window. For details, refer to the [Using the xeDebug GUI for Low Power Verification](#) section.

Adding CPF Information to a Design

To read the CPF file listing the CPF commands into the database, use the `readCPFfile` command that has the following syntax:

```
readCPFfile [-checkSyntaxOnly | -checkOnly | -noLinter | -force] <cpfFileName>
readCPFfile -clear
readCPFfile [-help]
```

The `readCPFfile [options] <cpfFileName>` command should be issued after the design is imported, but before the `precompile` command; otherwise, the precompile and compile processes do not take in the content of the CPF file. For command syntax and description of the `readCPFfile` command, refer to the *Compiler Commands* chapter of the *UXE Command Reference Manual*.

Note: Execute the `checkCPFfile` command before the `readCPFfile` command to internally invoke the Conformal Low-Power (CLP) integrator for checking the CPF commands for semantic errors. The CLP integrator translates the hierarchical-CPF files into flattened-CPF files, which are then accepted by the `readCPFfile` command.

The `readCPFfile` command performs the following functions:

UXE User Guide

Specifying Low-Power Intent on Palladium XP

- Translates the CPF information into User Data
- Issues warning messages for unsupported CPF commands
- Checks the CPF commands for grammar errors
- Checks the UXE-supported CPF commands for semantic errors

The CPF-related settings can be viewed at both compile and run time by using the following `lp` command:

```
lp [assert_illegal_domain_configurations |  
    create_nominal_condition |  
    create_isolation_rule |  
    create_power_domain |  
    create_power_mode |  
    create_state_retention_rule |  
    set_cpf_version |  
    set_design |  
    set_instance |  
    set_macro_model |  
    set_hierarchy_separator |  
    set_sim_control]
```

Note: At compile time, you can execute the `lp` User Data command at any point after executing the `readCPFfile` command.

Compiling a Design with Low-Power Intent

In the ICE mode, use the normal compile flow discussed in [Chapter 3, “Compiling Designs for In-Circuit Emulation with xeCompile”](#) to compile a design with low-power intent saved in CPF file(s).

During compilation, the compiler checks the CPF files, interprets the CPF specification, and inserts the derived instrumentation logic in the emulator executable image.

If you decide to not model the low-power behavior in the design, but have already specified the CPF file using the `readCPFfile` command, use the `precompile -noCPF XEL` command to turn off the CPF support during compilation. Consequently, the design is compiled without the CPF information. Alternatively, you can use the `precompileOption -add {noCPF} XEL` command.

Running and Debugging a Compiled Design with Low-Power Intent

In the ICE mode, perform the steps listed below to run and debug a compiled design with low-power intent.

1. Download the emulator executable image on the debugger.

UXE User Guide

Specifying Low-Power Intent on Palladium XP

Note:

- ❑ By default, CPF support is enabled at run time. However, if required, use the `lp lpDisable true` command to disable the CPF support. This command is useful when you do not want to keep different design databases, such as one with CPF and the other without CPF support. If you run into loop-breaking issues, you do not need to fix the loop-breaking issues in both the databases. Therefore, using the ability to enable and disable CPF support at run time, you can generate only one database that is with CPF during compilation.
- ❑ After download, use the `lp -value -<lpObjectType> <lpObjectName>` command at any time to get the actual value of the CPF objects.

2. Add power domains, power modes, general modes, and rules to the probes set using the following CPF-specific options of the `probe` run-time command:

- ❑ `-addPowerDomain <powerDomainName>` - probes control signals and instance signal values

You can use the `-addPowerDomain` option to add the `cpf_power_domains` object into the waveform database at the top hierarchy.

Each power domain is its own instance under the `cpf_power_domains` object. Each power domain has the following properties, shown as instances:

- `active_state_conditions`
- `base_domains`
- `instances`
- `interfaces`
- `shutoff_condition`
- `state_retention` (if applicable)

- ❑ `-addRetentionRules <stateRetentionRuleName>` - probes control signals and secondary domain

You can use the `-addRetention` option to add the `cpf_state_retention_rules` object into the waveform database at the top hierarchy. Each retention rule is its own instance.

- ❑ `-addIsolationRules <isolationRuleName>` - probes control signals and attributes

You can use the `-addIsolation` option to add the `cpf_isolation_rules` object into the waveform database at the top hierarchy.

UXE User Guide

Specifying Low-Power Intent on Palladium XP

- ❑ `-addPowerMode <powerModeName>` - probes the power mode transition waves and attributes

You can use the `-addPowerMode` option to add the `cpf_power_modes` object into the waveform database at the top hierarchy.

- ❑ `-addPowerModeTransition <ModeTransitionName>`
- ❑ `-addMode <modeName>`

Note: With each of the above-listed options, you can use the (*) wildcard character.

The following are added into the waveform database at the top hierarchy if any CPF probes are added:

- `cpf_controls` containing the `lpDisable` signals.
- `cpf_power_mode_transition` containing the `power_mode` information.

3. View the selected power information in the SimVision waveform viewer.

Note: Debussy also provides support for reading in a CPF file and applying the shutdown logic to waveforms. However, there is no integrated support for this with our software.

In xeDebug, probed CPF information is directly generated into the FSDB database. Therefore, after opening the FSDB database, you can view the probed CPF information.

For each power domain, the waveform displays the power-domain states to indicate the times when the power is up, down, or standby; and when the state retention happens. Additionally, for each isolation rule specified in the CPF files, the waveform displays the isolation on and off status.

In the waveforms, the values of the potential interface pins in the power domain are displayed as 0, 1, or X to depict the power-loss state of the pins. The value X is displayed for signals in power-loss state. The potential interface pins are the pins of those instances that are declared in the listed power domain instances using the `create_power_domain` command.

Note: To display the original emulator value of the signals in power-loss state instead of displaying the value X, use the `lp powerLossXDisplay [true|false|1|0]` command.

When you specify the `-all` option with the `probe` command, the value of signals in power-loss state is displayed as 1 or 0 instead of X in the waveform even if the `lp powerLossXDisplay` command has been set to `true`. The disabling of the feature of displaying the value of signals in power-loss state as X happens automatically because the `-all` option might initiate probing of the internal instrumentation nets too.

UXE User Guide

Specifying Low-Power Intent on Palladium XP

Palladium XP cannot guarantee the correctness of the power-loss information corresponding to the internal instrumentation nets.

If you want to enable the feature again after it has been automatically disabled, perform the following steps:

- a. Execute the `lp powerLossXDisplay` command.
- b. Execute the `probe` command without using the `-all` option.
- c. Execute the `database` command to create a new waveform database.

Suppose, a user executes the following set of commands including the `probe -all` command:

```
XE> lp powerLossXDisplay true
XE> probe -all dut
XE> database t
XE> database -upload
```

In waveform database `t`, if the driver of a signal is in power-loss state, the value of the signal will be displayed as either 0 or 1, instead of `X`.

To enable display of value `X` for signals in power-loss state, you should instead give the following set of commands:

```
XE> lp powerLossXDisplay true
XE> probe dut
XE> database tX
XE> database -upload
```

In waveform database `tX`, the signals with the driver in power-loss state will be displayed with value `X`.

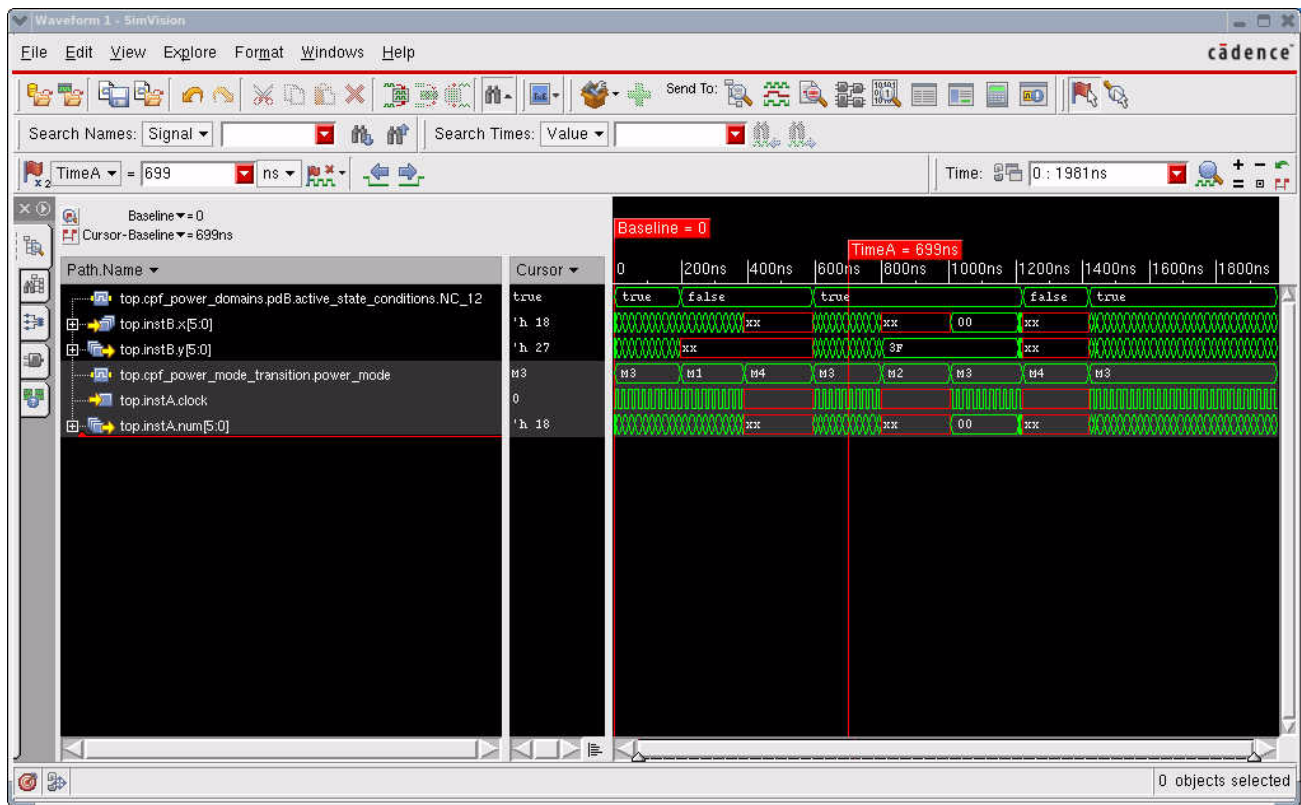
At the `XE>` prompt, you can also execute the `value` run-time command to know the power-down condition. The value returned will be either 0, 1, or `X`; where `X` implies that the net is in shut-off power domain.

Refer to [Figure 16-2](#) that illustrates a waveform depicting power-loss display in `X` value.

UXE User Guide

Specifying Low-Power Intent on Palladium XP

Figure 16-2 Waveform Displaying Power Loss in 'X' Value



Using `-power_up_states` and `-power_down_states` with Palladium XP

The `-power_up_states` and `-power_down_states` options of the `create_power_domain` command affect the values of state devices that do not use retention. The `-power_up_states` option determines the values of state devices when power shut-off condition is removed. When `lp powerLossXDisplay` is used, the values will change from x to the value specified in `-power_up_states`. If `-power_down_states` is used with `lp powerLossXDisplay`, the power down values will only show x until power is back. Then, as long as there is no `-power_up_states` and no state retention, the state devices will contain the values determined by `-power_down_states`. If `lp powerLossXDisplay` is false, the state devices will show the `-power down states` immediately on power shut-off.

The purpose of `-power_down_states` is to check the effect on the design if power down happens and isolation is not complete. The purpose of `-power_up_states` is to check if any non-retained sequential elements during power off period causes design chaos when power is back.

UXE User Guide

Specifying Low-Power Intent on Palladium XP

IES does not support the `-power_down_states` and `-power_up_states` options. IES uses `x` to model power loss corruption in the design. IES handles power back re-initialization by calling initial blocks. For more information on IES solution on power down and power back, refer to the IES documentation.

Timing considerations related to `-power_up_states` and `-power_down_states`

In silicon, state devices require some stabilization time after power is applied before they can be clocked with new values. In Palladium, this stabilization time is in the form of `FCLKS`. It can take up to four `FCLKS` for state devices to accept new values when `-power_up_states` is used and up to three `FCLKS` when `-power_down_states` is used. The number of `FCLKS` is less if the value of the state device does not change, that is, 0 remains 0 or 1 remains 1.

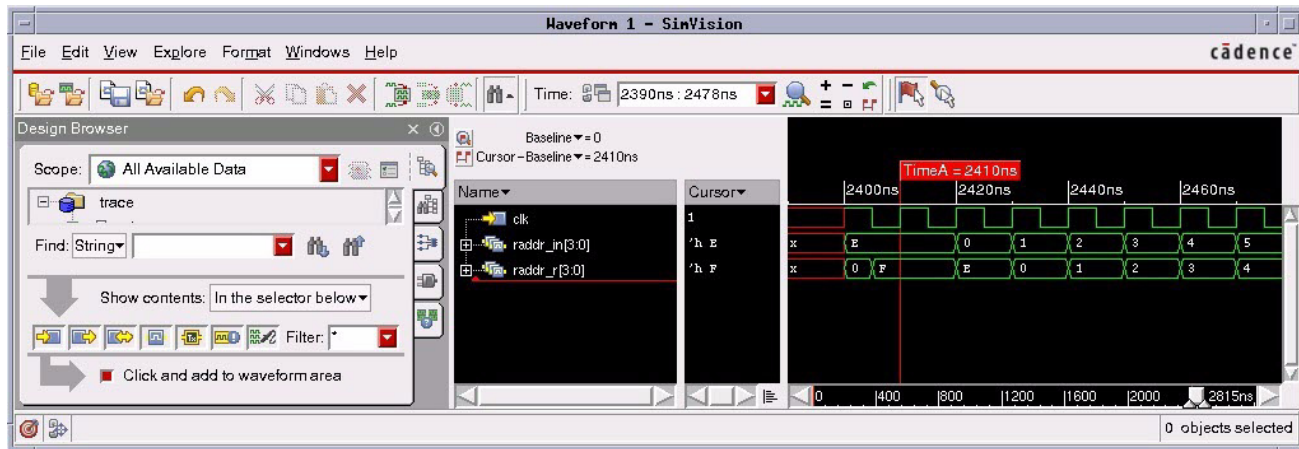
CPF does not specify the delay related to this power back. It depends on each individual tool to determine how to implement the power back re-initialization, either with delay, or without delay.

The waveform in [Figure 16-3](#) shows the clock changing. But this is misleading because the state devices within the shut-off power domain are not allowed to clock until the required `FCLK` delay has passed. This behavior does not match simulation and is a by-product of instrumentation compiled into the emulator for CPF.

In [Figure 16-3](#) the design has used CAKE 2X clocks. (two `FCLKS` cycles per design `CLK` cycle) You can see that at time 2410 ns, the rising edge of the clock does not change the output of the `FF` and `raddr_r` to match the input. The clock pin on the `FF` is effectively disabled until just before 2420 ns. The clock edge at 2420 ns, which is four `FCLKS` after power on, is allowed to clock the `FF` and `raddr_r` changes to match `raddr_in`. If this was a CAKE 1X design, `raddr_r` would not update until 2440 ns. If the `-power_up_states` option is not used with CAKE 2X, then `raddr_r` does follow `raddr_in` at 2410 ns.

If you want to ignore any delay due to `-power_down_states` and `-power_up_states`, CAKE 4X can be used. This puts all four `FCLK` cycles within a single user clock.

Figure 16-3 Waveform Displaying CAKE 2X Clocks



Displaying PSO Status

To probe the power domain PSO conditions, execute the following `probe` command before using a `run` or `database -upload` command:

```
probe -addPowerDomain <powerDomainName>
```

For example:

```
XE> database -open testCPF
XE> probe -addPowerDomain PD1
XE> run 1000
XE> database -upload
```

Here, an SST2 database is generated in `testCPF`. This database stores the power domain PD1 related information including the PSO condition, save/restore condition, nominal conditions status, and interface pins waveforms. These conditions and pins can be viewed using SimVision.

To probe the *on* and *off* status of the save/restore condition in the specified state retention rule, use the following `probe` command:

```
probe -addRetention <stateRetentionRule>
```

Important

The `probe -addRetention` command should be used before the `run` and `database -upload` commands.

For example:

```
XE> database -open testCPF0
```

UXE User Guide

Specifying Low-Power Intent on Palladium XP

```
XE> probe -addRetention rt1
XE> run 1000
XE> database -upload
```

The above set of commands, first generate an SST2 database in `testCPF0`. Then, the state retention rule, `rt1`, records two state-retention conditions, `rt1_save` and `rt1_restore`. Use SimVision waveform viewer to view these conditions from the `testCPF0` SST2 database.

Execute the following command to probe the `on` and `off` isolation conditions of a specific isolation rule:

```
probe -addIsolation <isolationRuleName>
```

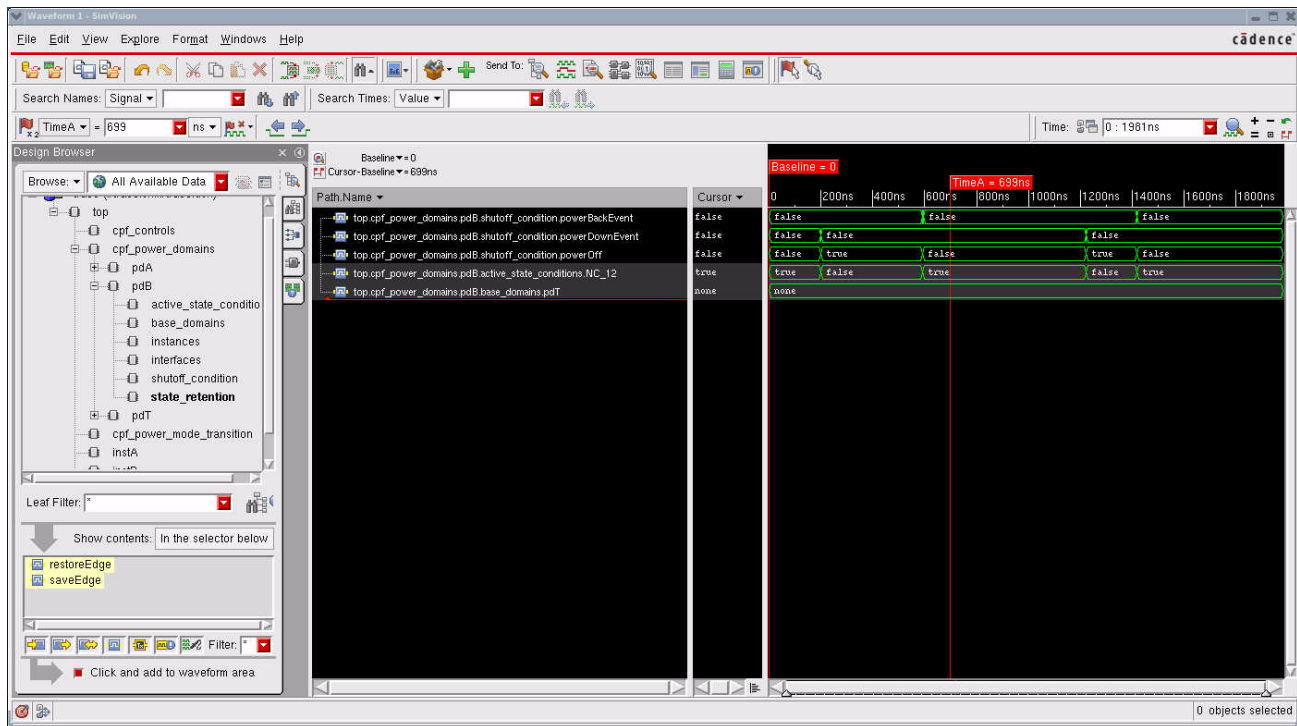
The above command should be used only before using a `run` or `database -upload` command.

For example:

```
XE> database -open testCPF1
XE> probe -addisolation isol
XE> run 1000
XE> database -upload
```

An SST2 database is generated in `testCPF1`. Using SimVision waveform viewer, the special isolation signal, `isol`, can be viewed from the `testCPF1` file as illustrated in [Figure 16-4](#).

Figure 16-4 Waveform Displaying Power Domains in SimVision



Profiling Power Mode Transition

A power mode is a static state of a design in which each power domain operates on a specific state: up, down, or standby.

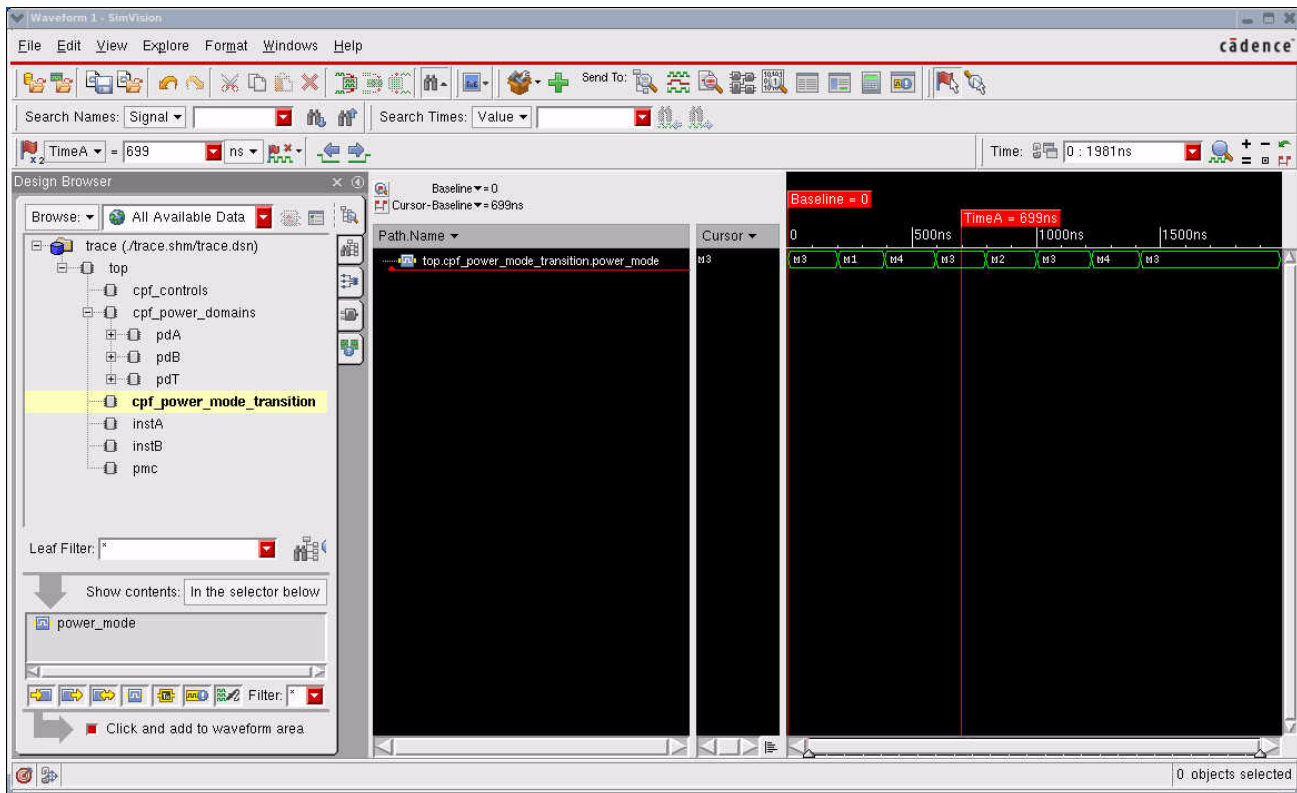
If the `create_power_mode` commands are present in the CPF file, whenever SST2 or FSDB waveforms are generated for probes at run time, the power mode transitions are automatically generated in the same waveform database. Consider the following set of commands:

```
XE> database -open testCPF
XE> run 10000
XE> database -upload
```

In the above example, a special signal, `power_mode_transition`, will be generated in the `testCPF` database. Using the `database -upload` command after the `run` command enables you to view the power mode transitions in the SimVision waveform viewer as illustrated in [Figure 16-5](#).

Note: Use the `probe -addPowerMode <powerModeName>` command to display the status of a specific power mode in the SST2 or FSDB waveform database.

Figure 16-5 Waveform Displaying Power Mode Transition in SimVision



Specifying CPF Conditions in SDL Expressions

Palladium XP emulator allows you to specify CPF conditions in SDL expressions for better verification and debugging of the low-power intent in your designs.

The CPF conditions can be used to do the following:

- Identify whether the state of a CPF object is true or false.
- Identify whether any control signals are corrupted due to power loss. The scope of control signals includes any power shut-off control signals, isolation conditions, or state retention control signals (defined using options such as `-save_edge`, `-restore_edge`, `-save_precondition`, `-restore_precondition`, `-retention_precondition`, or `-active_state_conditions`).
- Define the CPF illegal power control sequences and then check at run time to see whether the sequence happens or not.

Following is the basic format of CPF conditions in an SDL expression:

```
[ power :: ] <cpfObject>
```

UXE User Guide

Specifying Low-Power Intent on Palladium XP

```
[ power :: ] <cpfObject>@<cpfObject>
[ power :: ] <cpfObject>@<cpfOption>
[ power :: ] <cpfObject>@corrupted.<cpfObject>
[ power :: ] <cpfObject>@corrupted.<cpfOption>
[ power :: ] <cpfObject>@corrupted@<nominalCondition>
```

Here, `power ::` indicates that the condition is related to a CPF file. This is optional. The purpose of using it is to avoid having a signal with the same name in the design database.

`<cpfObject>` is a CPF object defined using the `create_...` CPF commands, that is nominal condition name, power domain name, retention rule name, isolation rule name, power mode name, general mode name, or illegal domain configuration name.

`<cpfOptions>` is an option of the `create_...` CPF command, such as `-shutoff_condition` is an option of the `create_power_domain` command.

Using CPF Conditions in SDL Expressions

The following are the basic CPF commands:

- If the `<powerMode>` defined with the `create_power_mode` command is true
`power :: <powerMode>`
- If the `<illegalDomainConfiguration>` defined in the `assert_illegal_domain_configurations` command is true
`power :: <illegalDomainConfiguration>`
- If the `<mode>` defined in the `create_mode` command is true
`power :: <mode>`
- Assume `<powerDomain>` is defined with the `create_power_domain` command, `<nominalCondition>` is defined with the `create_nominal_condition` command. To check whether `<powerDomain>` is in `<nominalCondition>`
`power :: <powerDomain>@<nominalCondition>`
- If the `<powerDomain>` is in power-off state
`power :: <powerDomain>@shutoff_condition`
- If the retention rule `save_edge` is true
`power :: <retentionRule>@save_edge`
- If the retention rule `restore_edge` is true
`power :: <retentionRule>@restore_edge`
- If the retention rule `save_precondition` is true

UXE User Guide

Specifying Low-Power Intent on Palladium XP

`power :: <retentionRule>@save_precondition`

- If the retention rule `restore_precondition` is true

`power :: <retentionRule>@restore_precondition`

- If the retention rule `retention_precondition` is true

`power :: <retentionRule>@retention_precondition`

- If the isolation rule `isolation_condition` is true

`power :: <isolationRule>@isolation_condition`

- If the power shut-off control signals in the given `<powerDomain>` are in power-loss state

`power :: <powerDomain>@corrupted.shutoff_condition`

- If the given `<nominalCondition>` corresponding power active_state control signals in given `<powerDomain>` are in power loss

`power :: <powerDomain>@corrupted@<nominalCondition>`

- If the retention rule `save_edge` control signals are in power loss

`power :: <retentionRule>@corrupted.save_edge`

- If the retention rule `restore_edge` control signals are in power loss

`power :: <retentionRule>@corrupted.restore_edge`

- If the retention rule `save_precondition` control signals are in power loss

`power :: <retentionRule>@corrupted.save_precondition`

- If the retention rule `restore_precondition` control signals are in power loss

`power :: <retentionRule>@corrupted.restore_precondition`

- If the retention rule `retention_precondition` control signals are in power loss

`power :: <retentionRule>@corrupted.retention_precondition`

- If the isolation rule `isolation_condition` control signals are in power loss

`power :: <isolationRule>@corrupted.isolation_condition`

You can use CPF conditions to define CPF illegal sequence, and then do CPF illegal sequence checking during run time. To check the illegal sequence, enable the isolation rule ISO, before power off in power domain PD1, by adding the following statement to the SDL file:

```
if (PD1@shutoff_condition == 'bP && !(ISO@isolation_condition))
    trigger;
```

Using the xeDebug GUI for Low Power Verification

For power verification in the ICE mode, you need to invoke the *Low Power Verification* window by performing the following steps:

1. Invoke the xeDebug GUI using the following command:

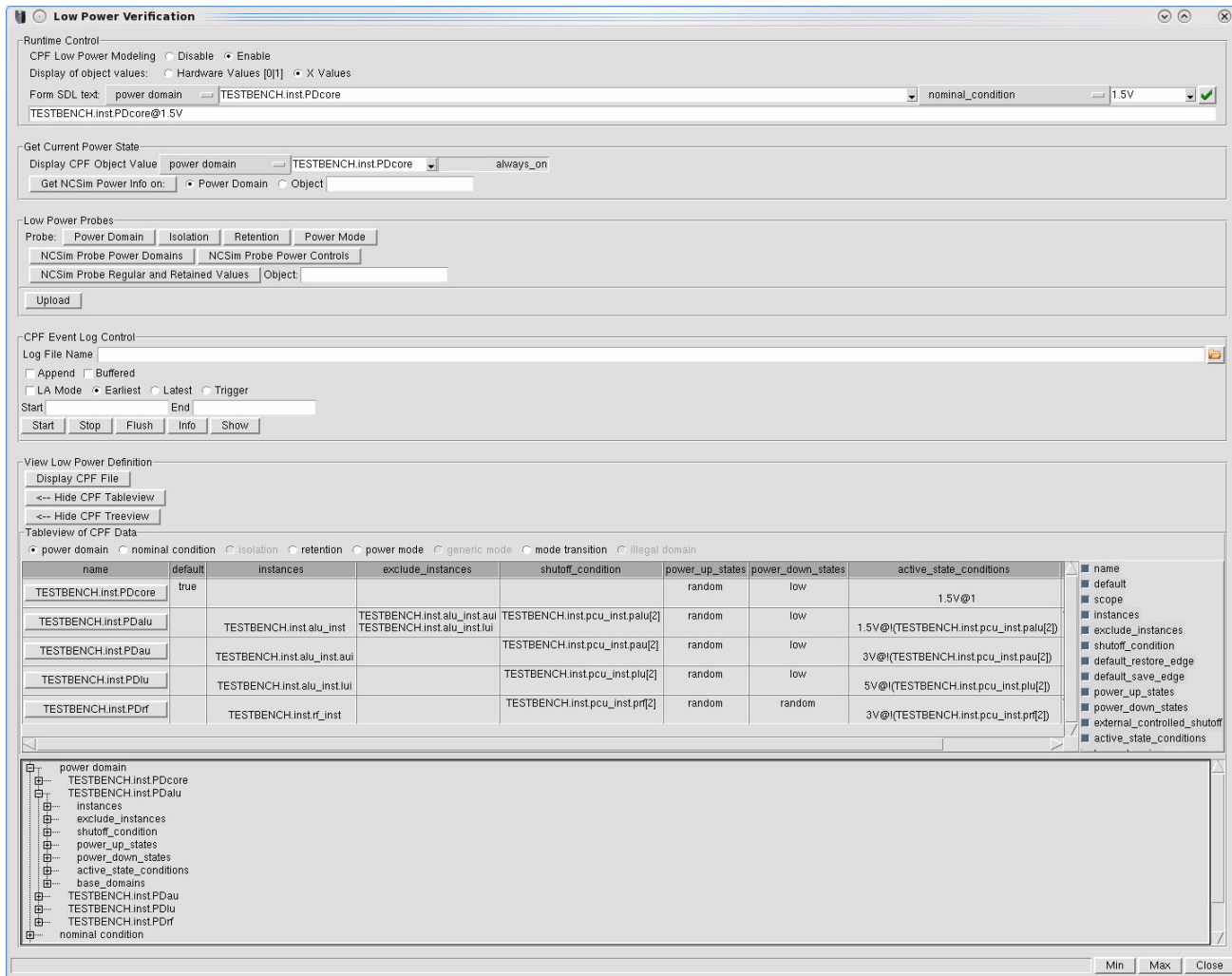
```
xeDebug -gui
```

2. Click the *Open Design* button.
3. After specifying the emulator host configuration, click the *Download* button. After the processing completes, the toolbar at the top of the xeDebug GUI window is activated.
4. Click the lightbulb icon () from the toolbar. The *Low Power Verification* window that helps in debugging low-power-aware designs is displayed (see [Figure 16-6](#)).

UXE User Guide

Specifying Low-Power Intent on Palladium XP

Figure 16-6 Low Power Verification Window



The *Low Power Verification* window provides the options for browsing the Palladium CPF implementation, and run-time operations like *enable*, *disable*, *probe*, and viewing and selecting instrumentation logic for probing and SDL. These options are displayed within different group boxes that are discussed in the steps given below.

5. In the *Runtime Control* group box, you can perform the following actions:
 - a. The *Enable* radio button adjacent to the *Disable CPF* field is selected by default. If you want to disable the use of CPF in the design at runtime, select the *Disable* radio button.
 - b. If you choose to keep the CPF logic enabled for the design, you can specify your choice to display the power-loss values either as *X* or real emulator values. The *Display X or hardware value* field is set to *False* by default, which indicates that

UXE User Guide

Specifying Low-Power Intent on Palladium XP

you want to display the actual emulator values. This is equivalent to using the `lp powerLossXDisplay false` command.

To display the power-loss values as X, select the *True* radio button. This is equivalent to using the `lp powerLossXDisplay true` command.

- c. The *Form SDL text* list boxes allow you to construct SDL statements for a selected CPF object.

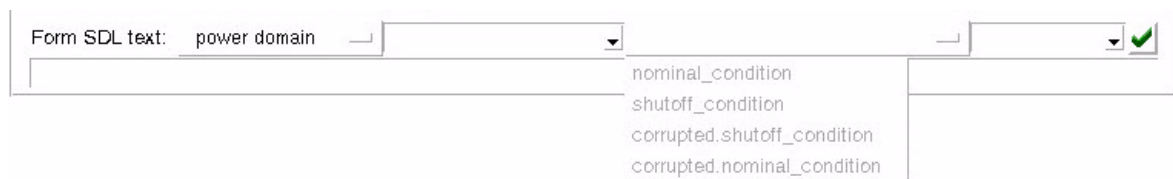


To construct SDL statements, perform the following steps:

Select a CPF object type. The list box adjacent to the object type box is automatically populated with a list of objects based on the selected type. Select a CPF object from the list box.



Select a condition type. The list box adjacent to the condition type box is automatically populated with a list of conditions based on the selected type. Select a condition from the list box.



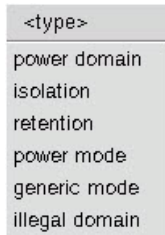
Click the  button to construct an SDL statement for the selected CPF object.

- 6. In the *Get Current Power State* group box, you can perform the following actions:

UXE User Guide

Specifying Low-Power Intent on Palladium XP

- a. To get the current value of the various CPF objects, click the *<type>* box to display a list and select the required object type.

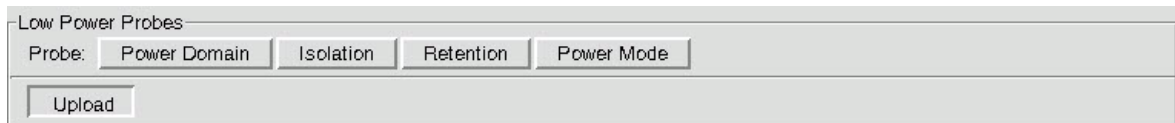


- b. The list box adjacent to the *<type>* box is automatically populated with a list of objects based on the selected type. The corresponding value of the object is displayed in a non-editable text field.



7. In the *Low Power Probes* group box, you can perform the following actions:

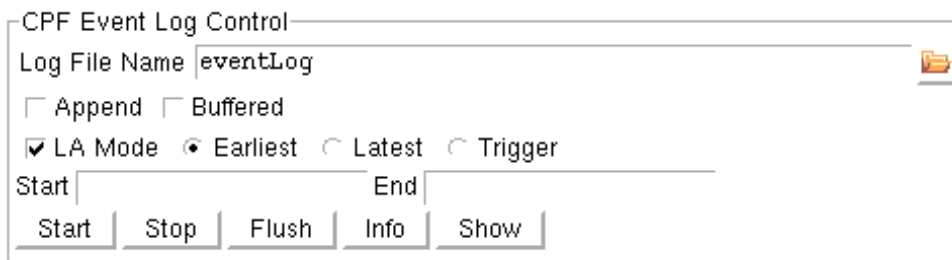
- a. Click the type of CPF object for which you want to create a probe. Four types of CPF objects that can be probed are—*Power Domain*, *Isolation*, *Retention*, and *Power Mode*. This is equivalent to using the `probe` commands.



- b. Click the *Upload* button. This is equivalent to using the database `-upload` command.

8. In the *CPF Event Log Control* group box, you can perform the following actions:

- a. Specify the *Log File Name*. You can also browse to a specific directory and select an already existing log file by clicking the  button next to the text box.



UXE User Guide

Specifying Low-Power Intent on Palladium XP

- b.** Select the type of log you want to create—*Append* (that is, instant log), *Buffered*, or *LA Mode*. Note only one of these check boxes can be selected at a time.
- c.** If you specify the type of log as *LA Mode*, select either *Earliest*, *Latest*, or *Trigger* to identify the way in which the CPF events should be captured.
- d.** Click the *Start* button to begin event logging. You also have buttons to *Stop*, *Flush*, *Info*, or *Show* the event log.

For conceptual details, refer to the Logging of CPF Events (Beta-Level Feature) section.

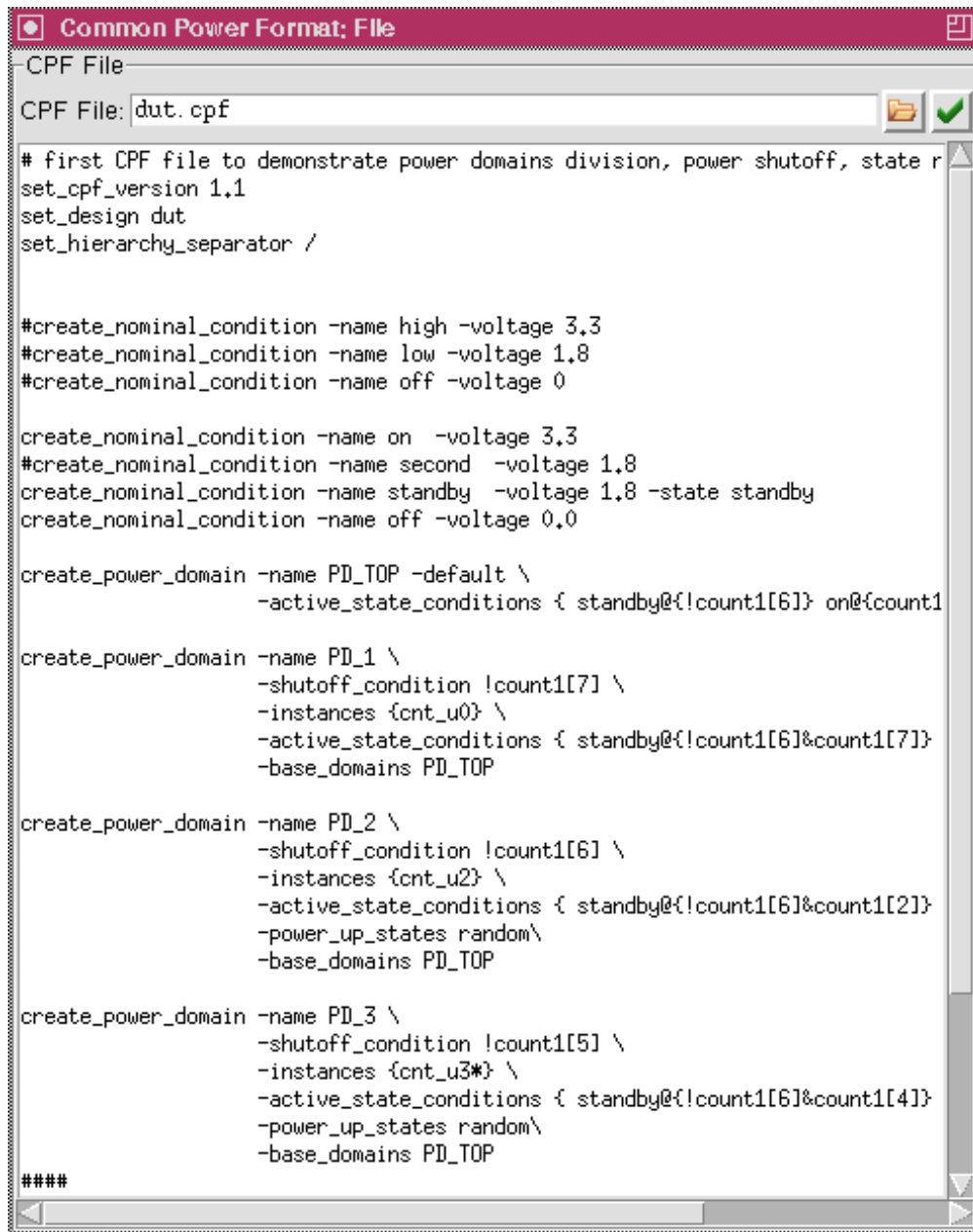
- 9.** In the *View Low Power Definition* group box, you can perform the following actions:
 - a.** Click the *Display CPF File* button to view the contents of the CPF files associated to the low-power-aware design.

The *Common Power Format: File* text window illustrated below is displayed. To view and edit a specific file, you can either enter the name of the required *CPF File*

UXE User Guide

Specifying Low-Power Intent on Palladium XP

and click the  button or select it using the *Select CPF file* dialog box invoked by clicking the  button next to the *CPF File* text box.



Note: The contents of the CPF file displayed in the *Common Power Format: File* text window are editable.

UXE User Guide

Specifying Low-Power Intent on Palladium XP

- b. To display the CPF data (from a Palladium XP emulator point of view) in a tabular format, click the -->Show CPF Tableview button. A table as illustrate in the following figure is displayed:

View Low Power Definition

Display CPF File

<-- Hide CPF Tableview

--> Show CPF Treeview

Tableview of CPF Data

☒ power domain ☐ nominal condition ☐ isolation ☐ retention ☐ power mode ☐ generic mode ☐ mode transition ☐ illegal domain

name	default	instances	shutoff_condition	power_up_states	power_down_states	base_domains
tb.dut.Base1		tb.dut.DOM_base1		random	low	
tb.dut.Base2		tb.dut.DOM_base2		random	low	
tb.dut.PD_in1		tb.dut.DOM_in1	tb.dut.pso1	high	low	tb.dut.Base1
tb.dut.PD_in2		tb.dut.DOM_in2	tb.dut.pso2	high	low	tb.dut.Base1
tb.dut.PD_out1	true	tb.dut.DOM_out1		random	low	
tb.dut.PD_out2		tb.dut.DOM_out2		random	low	

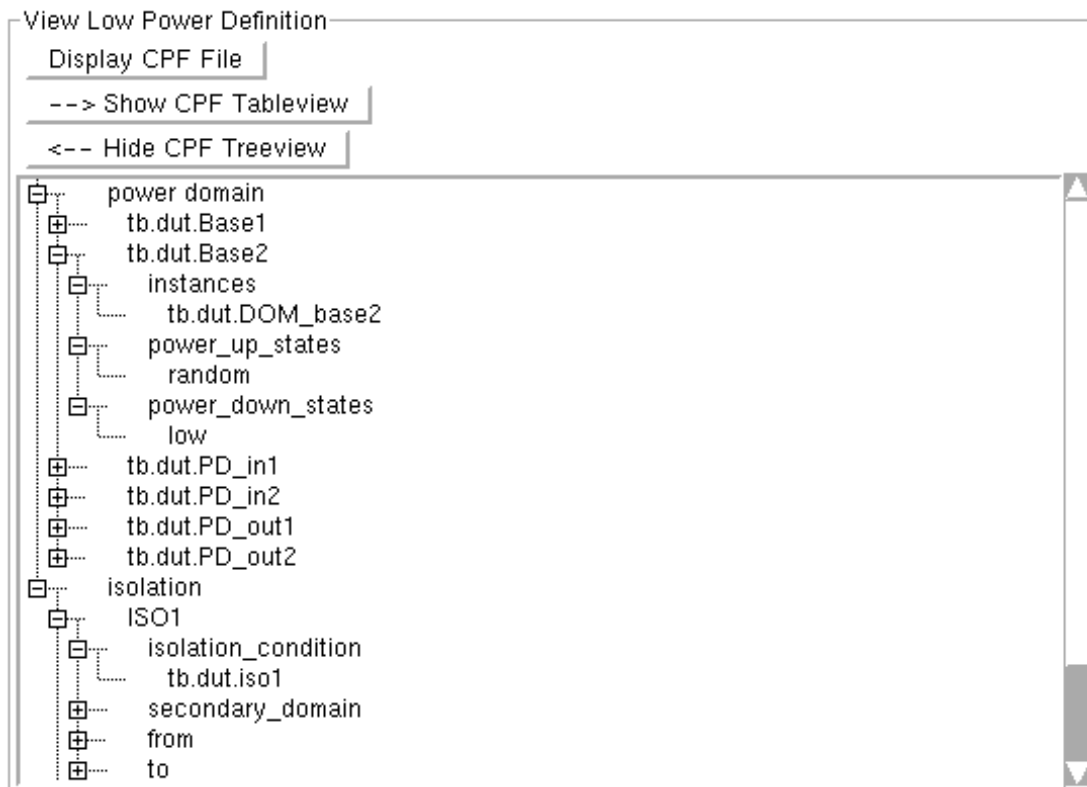
name
default
instances
exclude_instances
shutoff_condition
default_restore_edge
default_save_edge
power_up_states
power_down_states
external_controlled_shutoff
active_state_conditions
base_domains
boundary_ports
exclude_ports
mapped_domains
Redraw

To change the table layout, select or deselect the check boxes next to the CPF objects listed on the right of the table. Then, click the *Redraw* button to bring the changes into affect. The columns in the table will be displayed according to your CPF object selections.

The CPF object name is displayed as a button. Clicking these button enables you to probe the corresponding object. If a CPF object type displayed in the row above the table is grayed out, it means that the design does not contains CPF objects of the disabled type.

Note: After you click the -->Show CPF Tableview button, its label changes to -->Hide CPF Tableview button. When you click the -->Hide CPF Tableview button, the displayed table is closed.

- c. To display the CPF data in a standard hierarchical browser format, click the -->*Show CPF Treeview* button. The following figure illustrates the CPF objects of a design arranged hierarchically in a tree view:



Note: After you click the -->*Show CPF Treeview* button, its label changes to -->*Hide CPF Treeview* button. When you click the -->*Hide CPF Treeview* button, the hierarchical tree display is closed.

Design Considerations for Low-Power Verification

Keep the considerations discussed in the following sections in mind when verifying low-power-aware designs using UXE:

- [Creating Power Domains](#)
- [Using Verilog Registers as Power-Domain Instances](#)
- [Defining Power-Domain States](#)
- [Resolving Inconsistencies between the Designs](#)
- [Handling Power Shut-Off Designs](#)

- [Handling Designs with Multiple Supply Voltages \(MSV\)](#)
- [Handling Dynamic Voltage Frequency Scaling \(DVFS\)](#)
- [Verifying Hard-Custom IPs Using Macro CPF Modules](#)

Creating Power Domains

- A power domain is a collection of instances that share the same power distribution network and can be switched on or off simultaneously. The instances can be either module instances in RTL designs and gate-level netlists, or Verilog registers in gate-level netlists.

An instance can exist in only one power domain. However, a sub-instance can exist in a power domain that is different from its parent instance.

In UXE, a register can be modeled as a combinational logic, flip-flop, or memory instance.

- CPF commands support the use of expressions for defining `True` conditions. In UXE, these expressions can only be specified using terminals, not registers. For example, to use the following command, the `<expression>` parameter must specify the terminals:

```
create_power_domain -default -shutoff_condition <expression>
```

Therefore, if you had the following module, specify terminals `a` and `c` in the `<expression>` parameter and ignore `b` because it is a register:

```
module (a,c)
input a;
output c;
reg b;
endmodule
```

If you specify registers in the `<expression>` parameter, the `readCPFfile` command will generate an error. But if you need to use a register to control a power domain, a workaround is to create a virtual terminal. Use the following CPF commands in a CPF file to create a virtual terminal mapped to the required register:

```
set_instance u_3 -port_mapping {<virtual_terminal> <register>}
set_design counter -ports <virtual_terminal>
end_design
```

- Enclose the special character “!” (exclamation mark), “&” (ampersand), or “|” (pipe separator) in quotation marks when creating complex expressions. For example:

```
"!power1 & (power2 | power3)"
```


UXE User Guide

Specifying Low-Power Intent on Palladium XP

- For each power domain, the power can be either *on* or *off*. At run time, signals in the powered-off domains are marked with the *x* value.
- Unlike CPF 1.1 standard, in UXE, a net or pin must be connected to a power domain for waveform purpose. For feedthrough nets and pins, the value of the signal is the same as that of their driver. For example, if a pin is in a powered-on domain, but its driver is in a powered-off domain, the pin value is *x*. Conversely, if a pin is in a powered-off domain, but its driver is in a powered-on domain, it will have a value of 0 or 1. In UXE, a signal's driver is based on its connected pins, and the software checks the input-output direction.
- To properly restore the state of a power domain, the `restore_edge` condition must occur at least 1 `FCLK` cycle after the power is restored to that domain. Otherwise, the power domain will not be restored to the correct saved state from the `save_edge` condition.

In the following example, module `IPBlock` is instantiated as instance `IPInst` in the design `Top`. Instance `IPInst` is part of a switchable power domain `X`.

```
set_design Top
create_power_domain -name X -instance IPInst -shutoff_condition switch_en \
-restore_edge restore_en
set_instance IPInst
source IPBlock.cpf
```

In this example, all instances of `IPInst` are converted to state-retention flops, using the restore condition specified for domain `X` at the top-level design.

- All the net segments that comprise one physical net should be in the same power domain unless honor boundary ports or isolation is used. Generally, the power domain of a net is the same as that of its driver. If the net connected to a power domain's driver is isolated, the pin's power domain can change. For example, if instance `A` belongs to power domain `PD1`, instance `B` belongs to power domain `PD2`, output pin `A.a` connects to input pin `B.b`. Without isolation, `A.a` and `B.b` are in power domain `PD1`. With isolation between `A.a` and `B.b`, `B.b` is in power domain `PD2` while `A.a` is in `PD1`.

Using Verilog Registers as Power-Domain Instances

In UXE, you can specify Verilog registers as power-domain instances in CPF files.

Use the `-instances <instance_list>` option of the `create_power_domain` and `create_state_retention_rule` commands to specify the RTL-register variable in the instance list.

In RTL designs, the RTL-register variable should be the variable name. If during synthesis, the compiler retains the register names, the register array in the instance list of the `create_power_domain` and `create_state_retention_rule` commands will be

recognized. However, if the compiler changes the register names to respective internal names, the register array in the instance list of these two commands will not be recognized.

For example, consider that you have the following register in the RTL design:

```
reg [1023:0] rf[0:15];
```

Here, it is acceptable to specify the following command syntax in the CPF file:

```
create_power_domain -name PDMem -instance {rf }  
create_state_retention_rule -name srMem -instance {rf }
```

However, you cannot use the following command syntax:

```
create_power_domain -name PDMem -instance {rf[0] rf[2] }  
create_state_retention_rule -name srMem -instance {rf[2] rf [6] }
```

The format of a register name in the RTL design and the corresponding flip-flop or latch instance names in the netlist can be different. If a CPF file is written for an RTL design, use the RTL object names. If a CPF is written for a gate-level netlist, use the gate-level netlist object names.

Defining Power-Domain States

A power domain has three possible states, namely *power-up*, *power-down*, and *standby*.

A power domain in *power-up* state can have different power voltage supply levels at different periods of time. A power regulator will output the different voltage supplies in different conditions. When a pin-related expression is true, the power domain uses the voltage supply in the corresponding nominal condition.

The *standby* and *power-down* states have been designed to save power consumption, where *standby* does not need state retention, but *power-down* needs state retention. The *standby* state is entered by using the `create_power_domain -active_state_conditions` command.

Note: The *power-down* state saves more leakage power than the *standby* state. Palladium XP treats a power domain in the *standby* state as if it were in the *power-up* state.

In UXE, a power domain cannot be in *power-down* state at one time and in *standby* state at another. The *standby* state is specified with the `-active_state_conditions` option to the `create_power_domain` command. The *power-up* state should also be specified with the `active_state_conditions` option when *standby* is used. As *standby* and *power-down* state should not be mixed, there should be no `-shutoff_conditions` when `-active_state_conditions` are used.

A power domain can be in either of the following *combination of states*:

■ *Power-down or power-up*

Before power is off, sequential elements in the power domain should be saved; and after the power is back, sequential elements will be restored. During *power-down*, the interface from the powered-down domain to other powered-up domain should be isolated.

The `-power_up_states` and `-power_down_states` options of the `create_power_domain` command enable specification of the values for the sequential elements (such as registers modeled as flip-flops or memory instances) on power-up and power-down. These options accept the values `low`, `high`, `random`, or `inverted`. If these options are not provided with the `create_power_domain` command, by default, the `-power_up_states` option is set to `random` and the `-power_down_states` option is set to `low`.

Note: The upload and download of memories is not affected by their power status. If the `precompileOption -add randomizeMemories` command is specified at compile time, the `-power_down_states` and `-power_up_states` options allow the CPF memories to mimic the behavior of the *real* memories when they are powered down and/or up. For detailed information, refer to the [Handling of CPF Memory Power Up or Down Behavior](#) section.

Handling of CPF Memory Power Up or Down Behavior

To cause the memories to behave in a way that mimics the behavior of *real* memories when they are powered up and/or down, an emulated circuitry is added to the inputs and outputs of the memory that responds to the power-condition signal for the enclosing block. When the signal returns that the block is powered-down or powered-up, the additional circuitry causes the outputs (memory reads) to behave according to the specified value of the `-power_up_states` and `-power_down_states` options. It preserves the ability to upload or download the memories, but the effects of uploading or downloading are affected by the power state of the volatile and non-volatile memories.

Note: A memory is considered to be volatile unless it is identified as a ROM that lacks the circuitry for writing to the memory.

On a power-down event, both volatile and non-volatile memories behave as following, depending upon the value provided:

high	The output of a memory is high when power is off. Writes to the memory are disabled and reads always return all ones.
low	The output of a memory is low when power is off. Writes to the memory are disabled and reads always return all zeros.

UXE User Guide

Specifying Low-Power Intent on Palladium XP

random	The content of a memory is randomized on power-down. Writes to the memory are disabled and reads return randomized data.
inverted	The content of a memory is inverted on power-down. Writes to the memory are disabled and reads return inverted data.

On a power-up event, volatile memories behave as following, depending upon the value provided:

high	Same as random
low	Same as random
random	The content of a volatile memory is randomized on power-up. Then, the addresses might be read and written normally. Addresses that are read before being written, return random data.
inverted	The content of a volatile memory is inverted on power-up. Then, the addresses might be read and written normally. Addresses that are read before being written, return the inverse of the content.

Whether powered-up or powered-down, you can access volatile memories through the debugging tool. However, the existing content of a randomized volatile memory appears as random, and an inverted volatile memory appears as inverted. Also, the content loaded into a randomized or an inverted volatile memory during power-down state appears as random or inverted to the circuit.

On a power-up event, the content of a non-volatile memory is restored to its state before power-down, regardless of the value provided. When powered-down, the content of the non-volatile memory is modified according to its power option, and it can be downloaded to the xeDebug tool. A non-volatile memory is the same as ROM, it retains its value during power-down.

If a memory identified as a retain-state element is within a power domain that has up or down states specified, its contents are preserved when state-retention is active. If the power state of the power domain changes, the memory responds according to the specifications of the domain (*high*, *low*, *random*, or *inverted*). When state-retention state is restored, the memory reverts to the way it was when the retain-state began; the content appears to be unchanged by any intervening power up or down events.

Enabling Power State Handling of CPF Memories Manually

The mechanism of handling the CPF memories based on the defined power states is enabled by default. However, if you had disabled the feature earlier, you can enable it again manually by giving the following command along with other User Data commands either in the XEL script or on the command line at compile time:

```
precompileOption -add randomizeMemories
```

Disabling Default Power State Handling Mechanism of CPF Memories

To disable the default mechanism for handling the CPF memories based on the defined power states, give the following command along with other User Data commands either in the XEL script or on the command line at compile time:

```
precompileOption -add dontRandomizeMemories
```

When such memories are modeled in the Palladium XP emulator, the power never really goes off. Instead, the power-down state is modeled according to the behavior specified by the CPF specification and the memories are allowed to retain their state, regardless of the power status of the enclosing block. This is at variance with the real-world behavior.

Resolving Inconsistencies between the Designs

In emulator systems like Palladium XP, both the testbench and DUT parts of the design are considered. However, RTL Compiler (RC), a Cadence tool that you can use for power analysis, considers only the DUT part of the design. Therefore, to share the same CPF files between the Palladium XP and RC tools, first resolve the inconsistencies between these two definitions of the design.

If the `set_design` CPF command in a CPF file uses a non-top cell name, it means the CPF file contains only the DUT part. So, to run such a design on Palladium XP, iterate the current top cell to find out the instance, which belongs to the non-top cell name. Then, use that instance for the scope of the current CPF objects.

For example, consider a design top cell named `testbench`, a DUT module named `DUT`, and a DUT instance named `dut`. The CPF file named `dut.cpf` refers to the top-level module `DUT`, and specifies a power domain `pd1` with instances `A` and `B`. The `readCPFfile dut.cpf` command will translate the power domain `pd1` as `pd1` and its instances as `dut.A` and `dut.B`, instead of just `A` and `B`.

This solution chooses one instance that belongs to the CPF top design module because Palladium XP does not support multiple top-level modules. If the current design-top cell has

multiple CPF top-level module instances, the `readCPFfile` command will fail with an error message.

Handling Power Shut-Off Designs

A design with Power Shut-off (PSO) conditions contains some portions that can be switched on or off to save both leakage and dynamic power. PSO is one of the most effective low-power techniques.

There are three types of PSO behaviors: power switching, state retention, and interface isolation. In Palladium emulators, instrumentation logic is used to mimic these three PSO behaviors. This model might increase design capacity, but the emulation speed is not affected. The change depends on the number of sequential elements and isolated pins.

When a power domain is off, all instances, pins, and ports, nets of the power domain are in a power-loss state (also called corrupted state). Before the power-down, sequential element states in the power domain will be saved; after the power is back, the sequential element states will be restored.

The pins of a powered-down domain should be isolated so that other power domains (which can be still on) are not affected. The isolation should be done before the power-down happens. Remove the isolation once the power is back. UXE mimics this PSO behavior.

However, the state retention behavior of a PSO design needs to be handled differently. For details, refer to the [Handling State Retention PSO Behavior](#) section below.

As emulation systems are two-logic systems, 0 or 1, the `-power_down_state` and `-power_up_state` options are used to specify power-loss values and power back initialization values; thus, mimicking the power switching behavior in the design.

In a power-aware design, the PSO intention is given in the CPF file instead of the HDL files. During compilation, the PSO specifications are read from the CPF files. At run time, you can view the power domain states, power modes, occurrence of state retention, and values of the isolation pins in the waveforms. To view the powered-down state, probe the power domain using the `probe -addPowerDomain <domainNameList>` command. The value of a signal in power-loss state is shown as X in the waveform.

Note: If an instance in the HDL files has built-in PSO functions and the CPF file also specifies the PSO intent, use the `set_sim_control` command in the CPF file to disable PSO for that instance. This will enable you to avoid duplicate PSO instrumentation logic in the emulator executable image of the design.

The simplest PSO use model involves specification of PSO conditions without nominal condition. In this case, the defined PSO behaviors are modeled in the instrumentation logic,

but power mode is not displayed in the waveform. The other PSO use model involves specification of PSO conditions with nominal conditions, but without power mode definitions. Both these use models are not recommended when you want to use power mode to analyze the power intent.

Handling State Retention PSO Behavior

A PSO design supports the following three kinds of state-retention cells:

- Clock Low Retention Flip-Flop (CLRFF)
- Clock High Retention Flip-Flop (CHRFF)
- Clock Free Retention Flip-Flop (CFRFF)

By default, UXE only supports the CFRFF kind of state-retention cells that retain the value of the flip-flop during power shut-off only if the flip-flop is in retention mode. Such flip-flops are not dependent on the clock.

To specify a state-retention cell of type other than CFRFF, execute the following command:

```
stateRetentionControl -add {<stateRetentionRuleName> low|high}
```

Specifying the value as `low` in the `stateRetentionControl` UXE command indicates use of CLRFF state-retention cells. Such state-retention cells retain the value on power down only when the retention condition is `true`, and the clock is `LOW` while the flip-flop is going to retention mode.

Specifying the value as `high` in the `stateRetentionControl` UXE command indicates use of CHRFF retention cells. Such state-retention cells retain the value on power down only when the retention condition is `true`, and the clock is `HIGH` while the flip-flop is going to retention mode.

For each `create_state_retention_rule` CPF command, you must execute a `stateRetentionControl` UXE command to specify the corresponding flip-flop type which will be applied to that retention rule. If a value is not set using the `stateRetentionControl` UXE command, then CFRFF is used by default.

Debugging a PSO Power-Aware Netlist

To read PSO specifications for a netlist that was generated using a tool other than Palladium XP, generate the power-domain information based on the top-level signal connected to the power supply pins of each instance using the following UXE command:

```
identify_power_domain {[-name <powerDomainName>] [-modules <moduleNameList>]  
                      [-shutoff_condition <expression>]}
```

The power-control net is specified in the PSO condition.

If an instance belongs to a module existing in the specified *<moduleNameList>* and its input pin is connected to the power-control net, that instance is considered as a part of the specified power domain.

If an instance is not within the specified power domain, the instance will be always in power-up state and no instrumentation logic will be added during compilation for PSO operations.

Handling Designs with Multiple Supply Voltages (MSV)

A pure MSV design without *standby* power domains has the following characteristics:

- It uses multiple supply voltages for the core logic, but no other low-power techniques like PSO and DVFS.
- It does not need any instrumentation logic.
- It has only one power mode and each power domain can be associated with only one nominal condition.

To check the MSV level shifter rules, use CLP integrator. UXE does not support level shifter rule checking directly.

A pure MSV design has only one state of design (power mode).

At run time, the downloaded emulator executable image includes the power supply information, such as power domains, nominal condition, instances list, and pin/port list in each power domain.

Handling Dynamic Voltage Frequency Scaling (DVFS)

In UXE, a power domain of DVFS type can be in either *standby* or *power-up* state. However, a power domain cannot have a mix of *powered-down* and *standby* states.

At compile time, the DVFS specifications within the CPF files are checked. Consequently, the PSO instrumentation logic is added to mimic the power switching, state retention, and isolation behaviors. The *-active_state_conditions* option is used to specify in which condition the given power domain will be in which nominal condition.

At run time, the emulator executable image for DVFS designs includes the power mode transition and power domain signal values, that is, *on*, *off*, and *standby*.

Verifying Hard-Custom IPs Using Macro CPF Modules

Palladium XP emulator supports three types of IPs—hard non-custom IPs, hard-custom IPs, and soft IPs. Hard non-custom IPs and soft IPs can allow changes. However, hard-custom IPs do not allow changes, and cannot have logical and physical views.

In the ICE mode, the power behaviors for hard-custom IPs can be described using macro CPF modules written in either Verilog or VHDL.

Modeling Hard-Custom IP Low-Power Intention in a CPF File

In Palladium XP emulator, to model a low-power-aware hard-custom IP, the CPF file must contain CPF commands that define the following parts:

- Testbench
- Top-Level Module of DUT
- Macro CPF Module

Testbench

In the testbench portion of the power-aware hard-custom IP, you can specify only the following CPF commands and options:

```
set_cpf_version <2.0 | 1.1>
set_hierarchy_separator <character>
set_design <instance> -testbench
```

Note: Use the `set_design` command with the `-testbench` command option to specify the top-level module.

```
set_instance <instance>
```

Note: In the testbench CPF, a `set_instance <instance>` command can be used only to change the scope.

```
include <Top_DUV_CPF_filename>
end_design <power_design_name>
```

Important

Power intentions specified in the testbench CPF are ignored with a warning message.

UXE User Guide

Specifying Low-Power Intent on Palladium XP

Top-Level Module of DUT

In the top-level module of the DUT part of the CPF file, you must specify CPF commands to create the power domains, state retention rules, and isolation rules; and define the power modes and nominal conditions. In addition, you can specify CPF commands to update the isolation rules.

In the top-level module of the DUT, you can specify only the following commands and options:

```
set_cpf_version <2.0 | 1.1>
set_design <module>
set_hierarchy_separator <character>
create_power_domain -name -default -instances -shutoff_condition
                   -power_up_states -power_down_states
                   -active_state_conditions -default_isolation_condition
                   -default_save_edge_condition
                   -default_restore_edge_condition -base_domains
create_isolation_rule -name -from -to -pins -isolation_output
                    -isolation_condition -secondary_domain -exclude
create_state_retention_rule -name -domain -instances -exclude -restore_edge
                           -save_edge -save_precondition
                           -restore_precondition -secondary_domain
set_instance <instance> -domain_mapping -model
set_sim_control {-action {disable_corruption | disable_isolation |
                        disable_retention}} {-type {reg | module | instance}} \
              [-targets [-exclude]][-domains | -instances ] \
              [-modules | -lib_cells]
include <CPF_filename>
create_nominal_condition -name -voltage -state
create_power_mode -name -domain_conditions
create_mode_transition -name {-start -to}
get_parameter <parameter_name>
find_design_objects <pattern> [-pattern_type {name | cell | module}] \
                    [-scope][-object][-direction][-leaf_only | -non_leaf_only] \
                    [-hierarchical][-exact | -regex][-ignore_case]
end_design <power_design_name>
```

Use the `set_design <module>` command to assign CPF description to the specified module.

Note: Palladium XP emulator supports specification of an instance name in the first `set_design` command of the top-level module of the DUT if it has not been already specified in the testbench. In addition, all commands supported in the top-level module of the DUT part of the CPF file can be specified in block-level CPF command files.

Use the following command to define a macro model:

```
set_instance <instance> -domain_mapping -model
```

UXE User Guide

Specifying Low-Power Intent on Palladium XP

Consequently, the macro CPF is loaded and the power domains are mapped.

If the macro CPF module has been defined in a separate CPF file, specify the name of that file in the top-level module of the DUT by using the `include <CPF_filename>` command.

Macro CPF Module

Use a macro CPF module to define the power domains, power modes, and nominal conditions for the hard-custom IP. It is also used to create state retention and isolation rules.

A macro CPF can contain only the following commands and options:

```
set_macro_model <macro>
create_power_domain -name -default -instances -boundary_ports \
    -shutoff_condition -power_up_states -power_domain_states \
    -active_state_conditon -default_isolation_condition \
    -default_restore_edge_condition \
    -default_save_edge_condition -base_domains
create_isolation_rule -name -from -to -pins -isolation_output_condition \
    -exclude -secondary_power_domain
create_state_retention_rule -name -domain -instances -exclude -restore_edge \
    -save_edge -save_precondition -restore_precondition \
    -secondary_domain
create_nominal_condition -name -voltage -state
create_power_mode -name -domain_conditions
end_macro_model <macro>
```

Handling Hard-Custom IP Behavioral Models on Palladium XP Emulator

Palladium XP emulator expects that the behavioral model of the hard-custom IP has power-aware behaviors. Palladium XP emulator handles a low-power-aware hard-custom IP as a blackbox. The macro CPF module for such hard-custom IP should describe its implementation. Instrumentation logic for power corruption, state retention, and isolation of registers is not required. However, instrumentation logic for power corruption of the boundary ports can be modeled after domain mapping.

At run time, if the upper-level power domain mapped to the hard-custom IP is off, the IP's boundary ports and the output of the registers are displayed in power-loss state.

Mapping Power Domains of Macro CPF Modules

The macro CPF domain mapping process on Palladium XP emulator deviates from the CPF standard because of the following behavioral conflicts:

- When a macro-level switchable domain is mapped to a top-level switchable domain, the top-level shut-off condition is used and there is no structural equivalency checking.
- The `-active_state_conditions` of macro-level mapped power domains are ignored with warning messages. Instead, the top-level `-active_state_conditions` are applied to the macro-level mapped power domains.
- In macro CPF module, registers can be specified with the `create_power_domain` CPF command. Palladium XP emulator can synthesize a register as a memory instance, a flip-flop or latch, or a wire connecting to an input or output of a combinational logic. Registers that appear in a power domain are recognized only when their names are reserved by the compiler.
- After domain mapping, macro-level power modes are updated to use top-level domain conditions. Palladium XP emulator does not conduct a top-level versus macro-level power mode consistency check. Palladium XP emulator supports probing of top-level power modes during debug time.

Logging of CPF Events (Beta-Level Feature)

While a design is running in the ICE mode, you might want to see a message when a particular CPF event happens. Palladium XP emulator supports logging of messages related to the power mode transitions that happen during a design run and the power domain status. This section discusses the types of logs that enable recording of the messages, and the format of the logged messages.

Note: When the design is re-compiled, the CPF event log database generated in the previous design run is overwritten by new log data. Therefore, in both online and offline sessions, the CPF event log of the previous runs becomes inaccessible.

Types of CPF-Event Logs

The following two types of logs can be used to track the occurrence of CPF events in a design run:

- [Instant Log](#)
- [Buffered Log](#)

Instant Log

An instant log enables you to view a message related to a CPF event as soon as it happens. Such type of logging can be done only in the STB mode that allows you to pause the design

UXE User Guide

Specifying Low-Power Intent on Palladium XP

run when required. Whenever a CPF event happens, the design run is paused, the message is printed on the screen, and then the design run is resumed. As an instant log needs to pause the design run, it cannot be used in the LA mode in which the design run cannot be stopped.

Important

You can dynamically change between the LA mode and the STB mode using the `configPM` command. However, when you change the mode, the current CPF event logging will be stopped.

To use an instant log, execute the following command before giving the `run` command to start running the design:

```
lp log -start [<log_file_name> [-append]]
```

If you specify a log file name, the messages will also be saved in a log file, which you can study later. You can save the log file in either the current directory or any other directory. If you specify to save the log file in a directory other than the current one, you should provide the correct path with the log file name. If you specify the `-append` option, the log of new CPF events is appended into the existing log file.

You can use the `lp log -stop` command to stop the CPF event log. If you exit from the xeDebug tool without stopping the event log, the UXE software automatically uploads the remaining logs and saves those into the log file.

The log file is saved in binary format. Therefore, you need to execute the `lp log -show` command to decrypt the binary data and get the CPF event log messages. For detailed information, refer to the [Show CPF-Events Log Messages from a Log File](#) section.

The instant log impacts the run-time performance if there are too many messages to log. This is because the pausing and resuming of the design process incurs some overheads. If performance is a critical requirement and you do not need to see the logging message right away, you can use the buffered logs instead.

Buffered Log

Palladium XP uses a memory buffer to capture the CPF events. The size of the memory buffer is limited. You can set the depth of the memory buffer using the following command at compile time:

```
compilerOption -add {cpfLogBufferDepth <n>}
```

Note: The feature of generating buffered logs can be disabled by setting the value of the `cpfLogBufferDepth` parameter to 0.

UXE User Guide

Specifying Low-Power Intent on Palladium XP

In the STB mode, whenever the memory buffer is full, the design run is paused to upload the memory buffer and then resume design run. In the LA mode, the emulation cannot be paused. Therefore, there is a different usage model which is described later in the [Buffered Log in LA Mode](#) section.

Use the `lp log -start -buffered <log_file_name>` command to start using a buffered log. Then, you can start the design run. The logged data is automatically saved to the specified log file while the design is running. After executing a `run` command, the memory buffer might contain data that has not been uploaded. You can execute the `lp log -flush` command to force the data to be uploaded. If you use the `lp log -stop` command to stop the logging, the memory buffer is also flushed.

When using a buffered log, you will not see any logging message while the design is running. Use the `lp log -show` command to see the logging messages. For detailed information, refer to the [Show CPF-Events Log Messages from a Log File](#) section.

Buffered Log in LA Mode

In the LA mode, the design run cannot be stopped. Therefore, the maximum number of CPF events that can be logged is limited by the depth of the memory buffer. There are following three different strategies to control the logging of the CPF events:

- **Earliest**

The CPF event log is stopped once the memory buffer is full. Therefore, the earliest events are logged in the memory buffer.

- **Latest**

The CPF event log never stops. When the memory buffer is full, the latest event overwrites the earliest event.

- **Trigger**

The CPF event log is stopped once a trigger happens. This is the most preferred strategy because it allows you to get a set of CPF events before each trigger occurs.

Use the following command to start CPF event logging in LA mode:

```
lp log -start -lamode { earliest | latest | trigger } <log_file_name> \
[-posttrigger <num>]
```

The `-lamode` option indicates that the buffered log is for LA mode. You can execute the `lp log -stop` command to stop the log. Using the `lp log -flush` command enables to upload the memory buffer after automatically stopping the log. If you execute the `lp log -start` command again, the current log will be stopped and a new log will be set up.

Format of CPF-Events Log Messages

The CPF-events log messages include the following:

- Time when the event happened
- Current power modes
- Current illegal power modes

Note: An illegal power mode is defined in the CPF file to indicate the power mode state in which the design should never enter.

- Status change for power domains

Design could be in multiple power modes. Following is an example of logging message:

```
INFO CPFLOG: At cycle 1299 design enter power mode M1 and M3. Design enter illegal power mode M5. Power domain D2 changed from Off to On. Power domain D4 changed from On to Standby.
```

The logged messages only contain power domains for which the status changed. The messages are also saved in the `xe.msg` file whenever it is printed on the screen. The message is prefixed with text string `INFO CPFLOG:` so that you can identify the messages from `xe.msg` file.

Show CPF-Events Log Messages from a Log File

The log file is saved in binary format. So, to list the messages from the log file, you can execute the `lp log -show <log_file_name>` command. This command shows all messages in the log file.

You can also specify a time range as following to show only the messages logged during the specified time range:

```
lp log -show [-start <time>] [-end <time>] [<log_file_name>] [-outfile <out_file_name>]
```

You can execute the following command to get some basic information of a log file, such as the time range for logged messages and number of logged messages:

```
lp log -info <log_file_name>
```

If you execute the `lp log -show` command without a log file name, the currently open log file is used, that is, the one specified with the `lp log -start` command. If you do not specify a log file name and there is no open log file, for example you executed `lp log -stop` command, an error message is displayed.

Using Dynamic Power Analysis with Low-Power-Aware Designs

Dynamic Power Analysis (DPA) can be combined with a low-power-aware design to get the following main benefits:

- Fast running of the designs
- Support for very large capacity designs with real software application environment

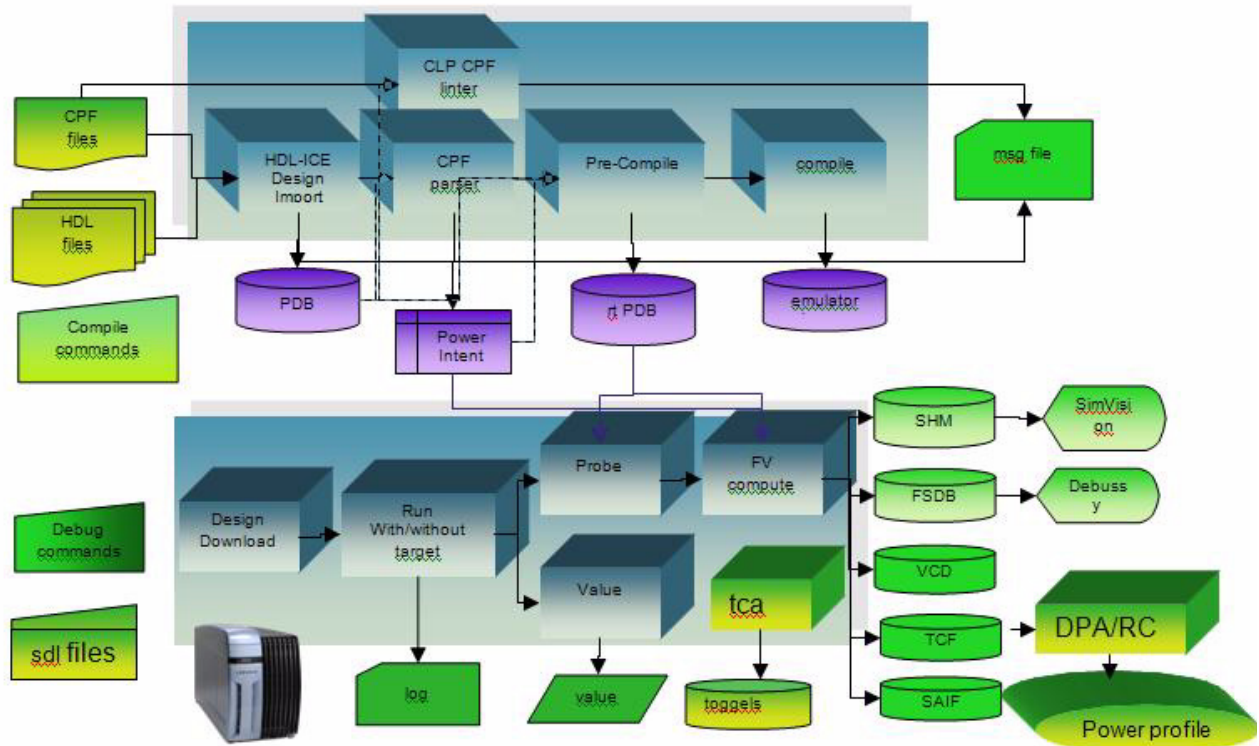
The power estimation and power profile are impacted when parts of the logic are in power-off condition. This allows you to get an estimate of the power reduction the design will experience from low-power design.

Note: The `-power_up_states` or `-power_down_states` option of the `create_power_domain` command impacts the power estimation by creating switching transitions to the new state.

For detailed information, refer to the [Analyzing Power Consumption with CPF](#) section of Chapter 15, “Analyzing Power Consumption of the Designs.”

The following diagram presents the basic architecture for low-power-aware designs:

Figure 16-7 Basic Architecture - Low-Power-Aware Designs



To verify the low-power-aware design with emulation systems:

- The inputs are HDL files, CPF files, compiler commands, debug commands, SDL files;
- The environments are either running with target systems or without target systems, either running with emulation online, or debugging offline without emulator running;
- The outputs of the specified HDL and CPF files are as follows
 - ❑ Message files
 - ❑ Log files in STB and LA modes
 - ❑ SST2 with values marked as X for power loss
 - ❑ SST2 with emulator values
 - ❑ FSDB with values marked as X for power loss
 - ❑ FSDB with emulator values
 - ❑ TCF with emulator values for pins or nets

- ☐ SAIF with emulator values for pins or nets
- ☐ VCD with emulator values for pins or nets
- ☐ Value for signals
- ☐ Value for CPF objects
- ☐ TCA with power domains
- ☐ Power consumption profile with power domains

Selecting Low-Power-Aware Verification Method Based on Design Type

The following sections discuss low-power-aware verification methods for each type of design:

- Emulating RTL Low-Power-Aware Design
- Emulating Gate-Level Low-Power-Aware Design

Emulating RTL Low-Power-Aware Design

System-level designs can include RTL designs without any low-power intention, and some IP with or without low-power intention.

For RTL and IP part without low-power intention, use CPF files to specify the low-power intentions. For the hard IP part with low-power intention, use macro CPF modules to model the intent in a CPF file.

Table 16-2 covers the type of low-power techniques you can use for different parts of system-level designs.

Table 16-2 Low-Power Techniques for Different Parts of System-Level Designs

Parts of System-Level Design	Possible Low-Power Techniques	Parts of CPF File
RTL without low-power intent	■ PSO ■ DVFS ■ Standby	■ Power intention with RTL names ■ Flatten CPF

UXE User Guide

Specifying Low-Power Intent on Palladium XP

Parts of System-Level Design	Possible Low-Power Techniques	Parts of CPF File
Soft IP without low-power intent	■ PSO ■ DVFS ■ Standby	■ Power intention with name in Soft IP simulation model ■ Flat CPF
Hard IP with low-power intent	■ PSO ■ DVFS ■ Standby	■ Macro CPF before DUT CPF and flat CPF

Emulating Gate-Level Low-Power-Aware Design

A gate-level netlist may or may not have power-intention cells in the netlist. You can describe the power intention either in the CPF files or within the netlist itself. In the CPF files, you can use names from the gate-level netlist or use original RTL names.

This section discusses all the possible cases of emulating a gate-level low-power-aware design.

Emulating Gate-Level Netlist without Low-Power Intention

If a gate-level netlist does not have any low-power cells (such as switching cells, isolation cells and retention cells), you can use CPF files to specify the low-power intention. Palladium XP adds instrumentation logic into the netlist to mimic the low-power behaviors. Use Palladium XP to run the design and debug the low-power intention.

Table 16-3 covers the type of low-power techniques you can use for different parts of gate-level netlists.

Table 16-3 Low-Power Techniques for Different Parts of Gate-Level Netlists

Parts of Gate-level Netlist	Possible Low-power Techniques	Parts of CPF file
Gate-level netlist without low-power intent	■ PSO ■ DVFS ■ Standby	■ Power intention with gate-level names from the netlist ■ Flatten CPF
Soft IP without low-power intent	■ PSO ■ DVFS ■ Standby	■ Power intention with name in Soft IP simulation model ■ Flat CPF
Hard IP with low-power intent	■ PSO ■ DVFS ■ Standby	■ Macro CPF before DUT CPF and flat CPF

Note: Emulating gate-level netlist with low-power cells and gate-level CPF files is not supported.

Improving Emulation Speed

This chapter describes the factors that influence emulation speed in Palladium XP emulators, and the actions you can take to improve emulation speed. It covers in-circuit emulation and emulation with a synthesizable testbench in ICE mode.

The following topics are discussed in detail in this chapter:

- [Using Design Retiming](#)
- [Improving Oversampling](#)
- [Improving Step Count](#)

Using Design Retiming

Use the design retiming technique to increase the design emulation speed. This technique is based on the fact that the emulation processors are almost never assigned with an even amount of work. While some processors are busy with calculations, the other processors might be idle, waiting for their inputs to become ready. The idle processors can be used to pre-calculate the signal values for the next emulation cycle, thus shortening the subsequent cycles.

Enabling or Disabling Design Retiming

To enable design retiming, use the `retiming` option of the `compilerOption` command:

```
compilerOption -add{retiming ON}
```

With `retiming OFF`, no design retiming is attempted. Use the following command to disable design retiming:

```
compilerOption -add{retiming OFF}
```

To revert to the default retiming setting, use the following command:

```
compilerOption -rm {retiming}
```

UXE User Guide

Improving Emulation Speed

The default value for the `retiming` option is `ON`. Hence, design retiming is enabled, by default.

Important

Even if you enable design retiming using the above instructions, it is not guaranteed that the design retiming will be performed. The compiler performs design retiming only if preliminary estimations predict some performance benefits for the design.

The preliminary estimations to evaluate performance benefits of design retiming are heuristic and cannot guarantee 100% accurate results. For some designs, retiming might have no effect or even an adverse effect on the design performance. Thus, switch off design retiming if you suspect it has an adverse impact on the design performance.

Tip

To determine if design retiming was performed, search the `tmp/et3compile.msg` file for a line similar to the following:

```
00 ET5NLOPT | Retimed 457 keynodes, 946 gates, created 679 muxes and 820  
dupflops
```

If a multiple compile process is used (for example, when using the `compileFind -first` command), look for the above line in `tmp/et3pass1.msg` instead of `tmp/et3compile.msg`.

Improving Oversampling

A Palladium XP emulator has its own clock called FCLK. Designs are usually compiled with CAKE clock generation. With CAKE clock generation, at compile time you specify an oversampling ratio, N :

```
clockOption -add {technology CAKE  $N$ }
```

The oversampling ratio is the number of FCLK cycles per cycle of the fastest design clock. The oversampling ratio is usually 4, 2, or 1, but 0.5 can be used with certain designs. A design compiled with oversampling ratio 4, 2, 1 or 0.5 is said to be compiled in 4X, 2X, 1X or 0.5X mode.

The fastest design clock frequency is N times slower than the FCLK frequency. Therefore, the higher the oversampling, the slower the design runs. Higher oversampling also adversely affects the effective Logic Analyzer trace depth and waveform upload speed, since higher oversampling means that fewer design clock cycles are captured in a given Logic Analyzer trace depth.

To obtain the maximum possible emulation speed, trace depth, and waveform upload speed, you should use the smallest possible oversampling. Refer to the following sub-sections for more information.

Upgrading from 4X Mode to 2X Mode

If a design functions correctly in 4X mode but fails in 2X mode, the reason is usually loop-breaking. When a design contains combinational loops, the compiler's loop-breaking algorithm automatically inserts delays to break the loops. If these delays prevent signals from propagating from one register to another within one user clock cycle, the design will function incorrectly. The design might fail in 2X mode, but function correctly in 4X mode because 4X mode increases the period of the clock compared to the loopbreaking delay.

In [Loop-Breaking](#), sections [Proactive Steps to Avoid Loop-Breaking Problems](#) and [Avoiding the Need for Oversampling](#) describe the steps you can take to improve oversampling from 4X to 2X.

Upgrading from 2X Mode to 1X Mode

This section describes the steps to upgrade from 2X mode to 1X mode. There are three things to consider:

- For in-circuit emulation with a target system, you might need different compilation parameters related to the target interface in 1X mode than in 2X mode.
- For some designs, loop-breaking causes functional problems in 1X mode but not in 2X mode. When loop-breaking prevents the use of 1X mode, the sections of [Loop-Breaking](#) mentioned earlier can help you improve oversampling from 2X to 1X.
- Some designs have a capacity cost for 1X mode. However, you might be able to eliminate the capacity cost using techniques described in the *Using 1X Mode* chapter.

Upgrading from 1X Mode to 0.5 Mode

0.5X mode is a special mode that allows the fastest design clocks to run at twice the frequency of FCLK.

Clocks running faster than FCLK are called double-speed clocks. The emulator capacity requirement is doubled for parts of the design clocked by double-speed clocks. Thus, 0.5X mode is useful primarily for designs where the fastest design clocks are used by only a small fraction of the design. 0.5X mode imposes additional restrictions on designs. For example, storage elements cannot change state on the falling edge of a double-speed clock.

For more information about 0.5X mode, refer to the [Modeling Clocks](#) chapter.

Improving Step Count

FCLK frequency is design- and compile-dependent, but for most designs, is between 0.5Mhz and 1.5Mhz. Underlying FCLK is a much faster clock: the step clock. The step clock frequency is fixed at 500Mhz.

Each FCLK cycle takes a number of steps M , determined by the compiler for a given compile of a given design. The step count, M , is affected both by long combinational paths in the design, and by the availability of hardware resources in the emulator.

You can sometimes improve step count by eliminating unnecessary long paths in the design, or by using compilation options that reduce the need for a particular hardware resource. Refer to the following sub-sections for more information.

Factors Contributing to Step Count

High step count can be caused by several factors.

- Long combinational paths in the design can cause a high step count. Paths from flip-flop or latch clock, set or reset to output, or from latch D to Q are considered combinational in this context. A long path can cause high step count even when the path is not needed for functionality.
- Scheduling of accesses to memories can require a high step count. Sometimes, the compiler is forced to pack multiple memories, or memory slices, into a single physical memory with multiple accesses per FCLK. Successive accesses to an XDRAM memory (the memory in a User Memory Card) must be at least 30 steps apart.
- Designs compiled in 1X mode or 0.5X mode usually need I/O timing constraints on inputs from an external target. I/O timing constraints (especially on inputs) can contribute to a high step count.
- A shortage of routing resources within the emulator can contribute to high step count.
- Vision mode FV can increase the step count compared to DYNP.
- The fraction of the emulator's logic primitives that are used by a design imposes a lower limit to the step count, and therefore, an upper limit to the FCLK frequency. The step count can be no better than $512 \text{ steps} \times P$, where P is the fraction of the logic primitives that are used. However, most designs have a higher step count than the minimum imposed by utilization.

General Techniques to Improve Step Count

When you compile a design repeatedly, the back-end compiler (et3compile, invoked by the `compile` XEL command) automatically uses a different *random seed* for each compile, unless you specify a specific seed. Usually, different compiles will show a range of about 10% variation in step count due to the different random seeds used. You can automatically try multiple compiles with different seeds using the `compileFind` command with different options (such as `-first`, `-best`, `-opt`, `-all`, and `-lsf`). For a complete list of options, refer to the description of `compileFind` command in *Compile-time Commands in ICE Mode* chapter of the *UXE Command Reference Manual*.

When studying the effects of user actions on step count, you should always do at least 3 compiles with each condition, to avoid being misled by random variations.

For best results, always use the latest IXE software release.

Usually, to improve step count significantly, you must first analyze the critical path report as described in the following sections, to find the reasons for a poor step count.

Note: During design compilation, the critical path report can be generated and saved in the `tmp/et3compile.msg` file by specifying the `compile -crit_paths <number>` command. To generate the critical path report after the design has been compiled, specify the `compile -criticalPathOnly [<number>]` command. For details about these command options, refer to the description of the `compile` command in the *Compile-time Commands in ICE Mode* chapter of the *UXE Command Reference Manual*.

Finding the Reason for a Poor Step Count

After compiling, the file `tmp/et3compile.msg` includes two different critical path reports. The ET3CRIT section lists the *pre-scheduling critical paths* with the greatest number of logic levels in the design. The ET5ANAL section shows the *post-scheduling critical paths* that actually take the maximum number of steps to execute. A path that uses scarce resources might cause a high step count, even though some other path has more logic levels. If the pre- and post-schedule paths are different, usually the post-schedule path report gives more insight into the reason for a high step count.

Critical paths in `et3compile.msg` show encrypted signal names rather than actual signal names from the design. To see the actual names, run the `getCriticalPathDesignNetNames` command after compile. It creates a file `tmp/criticalPathReport.msg` with critical paths showing actual signal names.

At times, you might see some net names in the `criticalPathReport.msg` file in the following form:

UXE User Guide

Improving Emulation Speed

`trNet_<number>`

Such names, typically, appear for those nets that were added to the design by the compiler. In some cases, you can get more meaningful net names instead of `trNet` names by setting the `SMDB_SAVE_MC` environment variable to 1 before precompile.

To be able to see readable net names that are similar to the original net names in the `criticalPathReport.msg` file, set the `SMDB_SAVE_MC` environment variable to 1 before precompile.

The `SMDB_SAVE_MC` setting is not done by default because it can greatly increase the size of the database size and compilation time. Set the `SMDB_SAVE_MC` environment variable only if it is needed.

In a Palladium XP compile with the default (high) effort, some nets in the critical path report have no names. Names are absent when logic optimization rearranges combinational logic to create nets that are not equivalent to specific user nets. If you wish to see names for all nets, compile with low effort (using `compilerOption -add {compilerEffort LOW}`) and assume that the resulting critical path is similar to the path with default effort.

Here is an example of a post-schedule critical path report in the `criticalPathReport.msg` file. To save space, only the beginning and end of the path are shown here.

```
00 ET5ANAL* | Post-scheduling critical path
00 ET5ANAL* |
00 ET5ANAL* | Location legend: All values are in decimal.
00 ET5ANAL* | [brd, chp, cls, ep]
00 ET5ANAL* | | | |
00 ET5ANAL* | | | | +-- emulation processor index within the cluster
00 ET5ANAL* | | | +----- processor cluster index within chip
00 ET5ANAL* | | +----- chip index within board
00 ET5ANAL* | +----- board index
00 ET5ANAL* |
00 ET5ANAL* | Critical path 1 sinks
00 ET5ANAL* | Count Location Step Jump Type ID ins:cr Name
00 ET5ANAL* |
-----
00 ET5ANAL* | Critical net starts from an actual critical location.
00 ET5ANAL* | . [00,002,032,0] 366 + 0 NET 480826 1
uPlatform.n16 **
00 ET5ANAL* | 1 [00,002,032,0] 366 + 0 LOGICx 645864 3: 3
00 ET5ANAL* | . [00,002,032,3] 290 + 76 NET 690875 1
uPlatform.n16$inv782 **
```

UXE User Guide

Improving Emulation Speed

```

00 ET5ANAL* | 2 [00,002,032,3] 290 + 0 LOGIC 645137 3: 3
00 ET5ANAL* | . [00,002,032,1] 284 + 6 NET 690874 1
00 ET5ANAL* | 3 [00,002,011,2] 281 + 3 LOGIC 645136 4: 3
00 ET5ANAL* | . [00,002,010,3] 230 + 51 NET 690102 232
uPlatform.rstack_pop **
00 ET5ANAL* | 4 [00,002,032,3] 141 + 89 LOGIC 643735 3: 1
...
00 ET5ANAL* | 71 [00,019,028,0] 6 + 0 LOGIC 111169 4: 3
00 ET5ANAL* | . [00,019,028,3] 5 + 1 NET 152904 96
trNet_2312478
00 ET5ANAL* | 72 [00,019,028,3] 5 + 0 LOGIC 138820 2: 1
00 ET5ANAL* | . [00,019,028,0] 4 + 1 NET 54669 44
TCK_w_158681@QTdelay
00 ET5ANAL* | 73 [00,019,007,2] 0 + 4 FD01 38737 4: 0
00 ET5ANAL* |

```

```

-----
00 ET5ANAL* | Maximum jump of 89 steps occurs at step 141
00 ET5ANAL* | 5 inter-chip-pair hops, 61 steps inter-chip-pair delay
00 ET5ANAL* |
=====

```

The report contains two types of lines called *box* and *net* lines. A *box* is a logic primitive, such as a combinational gate, memory port primitive, or delay cell. A *net* connects the output of one logic primitive to inputs of other primitives. The top of the report corresponds to the *downstream* end of a combinational path.

Box and net lines have the following fields:

- *Count* is the number of logic levels from the downstream end of the path to the current point.
- *Location* is the location of a given logic primitive in the emulator. A location consists of a board number, chip number, cluster number, and processor number. (Each domain has 12 chips. In the most common compile mode, each chip has 64 clusters and each cluster has 4 processors.)
- *Step* is the step count used for the path, from the upstream end to the current point.
- *+ Jump* is the change in step count from the previous line to the current line.
- *Type* is either NET, or a box type. The box types LOGIC and LOGICx are combinational primitives. XMPRx, XMPRe, XMPWx, XMPRe are memory port primitives; the lower-case x and e stand for *external* (XDRAM) and *embedded* (EDRAM) memory,

respectively. `FD01` is a 1-FCLK delay primitive that is a part of the implementation of a design flip-flop.

- *net* lines usually end with the name of a net. If the name is from the design, it is marked with `**`.

At the end of the path is a summary, including the maximum jump in step count for any single pair of lines. Large jumps in step count usually occur when a resource is heavily utilized, so that the scheduler needs to wait some steps for the resource to become available.

What to Look for in the Critical Path

To find the reason for a high step count, examine the post-schedule critical path, considering the following:

- Does the path contain many logic levels? If the number of levels is more than about 200, it is likely that the large number of levels contributes significantly to the step count.
- If the number of levels is less than 100, it is likely that a high step count is caused by lack of resources in the emulator, rather than by long paths. In such a case you can often improve the step count by compiling for a larger number of domains or boards.

If the path contains a large number of logic levels, consider the following:

- Does the design logic for part of the long path have a regular, repeating structure?

If so, it is likely that you can eliminate this part of the long path. For example, in one design, a memory model written for emulation contained an 1024-to-10 encoder written using a basic “for” loop, which produced a chain of 1024 muxes. When the “for” loop was replaced by a tree-structured encoder, the long path was eliminated and step count improved. In another design, a long path was part of a testability circuit which did not need to be tested in emulation. The path contained a certain cell repeated over and over. When this cell was modified to eliminate the long path, the step count improved.

To find a repeating structure in the critical path, it is usually helpful to view the post-schedule path in the schematic viewer, using the method described in the next section.

- What RTL signal is being generated by this path?

A critical path can contain only net names like `n1234` that are generated by synthesis and are not a part of the RTL. Usually, the most downstream net connects to the D input of a flip-flop, and the Q output of the flip-flop generates an RTL signal. Use the DUT design browser, the `verigen` program, or a text editor, to inspect the gate-level view of the design and find the flip-flop at the end of the path. This technique can help you identify an RTL construct whose implementation results in a long path.

- Is most of the long path in a single module of the design?

Using a synthesis tool other than xeCompile, you can re-synthesize the specific module, optimizing for speed rather than area. Some synthesis tools have an option for *retiming* (`balance_registers`) which can redistribute flip-flops into combinational paths to reduce the path length.

If the module is a Synopsys DesignWare component, consider using DesignWare models implemented specially for Palladium XP (available from Cadence). Alternatively, try using Synopsys Design Compiler to synthesize it. A library is available that allows Design Compiler to target the Palladium XP `qtref` library.

You can get an upper-bound estimate of the possible improvement by changing a particular module or instance. To do so, use the `moduleStub` or `instanceStub` command to replace the module or instance by an empty module or instance. Use the `precompileOptions` `keepStubInputs` and `keepStubOutputs` to prevent extra optimization due to undriven module outputs or loadless module inputs.

- Does the long path include paths from D to Q of latches?

To find latches in the critical path, it is usually helpful to view the post-schedule path in the schematic viewer, using the method described in the next section.

If the latches are in master/slave pairs, replace the master/slave pairs, if possible, with flip-flops from the `qtref` library.

If the latches use a non-overlapping clock scheme, provide information about the non-overlapping clocks using the `noOverlapClock` command.

You can use the `precompile -syncData` option to insert a delay on the D input of every latch in the design. The inserted delay will eliminate paths going from D to Q of latches. However, in most designs, using `-syncData` increases the CAKE oversampling number required for the design, so that `-syncData` has no overall benefit and is not recommended.

- Does the long path include paths from flip-flop or latch set or reset to Q?

You can eliminate paths from flip-flop or latch set or reset to Q by using the `precompile -syncReset` option. In some designs, using `-syncReset` increases the CAKE oversampling ratio required for the design, so that the option has no overall benefit. However, in some designs, especially with CAKE 2 or higher, the option improves step count without requiring extra oversampling.

- Is the starting point for the path a primary input?

If a one-FCLK delay on this primary input will not cause a functional problem, you can usually improve the step count by about 20 steps by using `breakNet` to insert a break at this primary input.

If a one-FCLK delay will cause a problem, you should not use a `breakNet`, but if there is a `terminalTiming` constraint on the given input, consider reducing the size of the constraint.

- Does the path include one or more I/O pads in the middle of the path?

An emulated design sometimes contains a “top-level wrapper” that instantiates a “design under test” (DUT) and other logic or memories. Sometimes the critical path might go through several bidirectional I/O pads of the DUT block. Such paths are often not needed for functionality, and can be broken using `breakNets` without harming functionality.

If the I/O signals of the DUT are made I/O of the top-level wrapper, and these I/O are not connected to a target, breaks will be added automatically unless you specify the `delayBidir off` compilerOption. If the I/O signals are connected to a target, then unless you use `terminalTiming` to constrain input components to be scheduled later than outputs, the paths from output to input of an I/O pad will in effect be broken automatically.

- Does the path include nets with the string `QTshadow` in the name?

Nets with `QTshadow` in the name are shadow nets. Shadow nets occur in 1X-mode compiles, and in compiles that use the `precompile -breakHalfCycle` option. Consider whether the shadow nets appearing in the critical path can be eliminated, using techniques described in [When to Use 1X Mode](#) on page 1264.

If you use the multisampled I/O feature, multisampled inputs and outputs can appear in the critical path with `QTshadow` as part of the name. If the given input or output does not need to be multisampled, make it non-multisampled by not specifying it as a `multiSampledTerminal`, or by using `enableMultiSampledIO 2` instead of 1.

- Are there large jumps in step count (jumps of more than 20 steps for a single logic level) along the path, and if so, where do they occur?

If large jumps occur when the path goes from one board to another, then adding more board-to-board cables can improve the step count.

If large jumps occur at memory ports (shown in the report as boxes of type XMPW or XMPR rather than LOGIC) see the [Effects of Memory on Step Count](#) section, below.

If large jumps in step count occur for no apparent reason, it could be due to a lack of logic primitives or routing resources within an emulation board. In such a case, increasing the number of domains or boards can improve the step count. In general, increasing the number of domains or boards can have two contrary effects on step count: it can reduce the step count by making more resources available, but it can increase the step count by increasing the number of hops between different domains or different boards.

A large jump where the ending step number for the jump is less than 512 is more unusual than a jump where the ending step count is more than 512. When the step count for a

path goes beyond 512, large jumps sometimes occur because of the nature of the scheduling algorithm, even when emulator utilization is low.

Viewing the Critical Path in the Schematic Viewer

The reports in `criticalPathReport.msg` show net names, but do not show the type of gate driving each net. To see the gate driving each net, you can use the schematic viewer.

Using the schematic viewer can be very helpful in finding a repeating structure within the critical path, or in identifying latches within the critical path. Usually, a repeating structure shows up as a very noticeable “diagonal line” in the path displayed in the schematic viewer.

To view the critical path in the schematic viewer, perform the following steps:

- If you are compiling for Palladium XP, compile with `compilerEffort LOW` (or with `optimizationEffort LOW`); otherwise many nets in the path report will have no names.

- Issue the `getCriticalPathDesignNetNames XEL` command.

- Issue the following commands:

```
source <install_dir>/share/uxe/gift/misc/critical.gel
critical crit.v
```

The `critical crit.v` command does the following:

- Runs the verigen program to generate a Verilog file `crit.v` containing the (overlapping) cones of logic obtained by tracing back five levels from each net in the post-schedule critical path.
- Creates a Debussy session file `crit.v.ses` which assigns color to each net that is in the reported post-schedule critical path, or is connected through buffers or inverters to such a net. (If the `criticalPathReport.msg` contains two or more critical paths, then colors green, yellow and blue are used for nets in the first path, the second path, or in both paths respectively.)
- Invokes the Debussy schematic viewer to display a schematic of the post-schedule critical path. If you do not wish to invoke Debussy, unset the UNIX DISPLAY variable before invoking `xeDebug`.

You can display the schematic later by giving the `qtTurboDebug crit.v` command from the UNIX command line, and then restoring the `crit.v.ses` session.

The resulting schematic has some confusing aspects:

UXE User Guide

Improving Emulation Speed

1. The paths in `criticalPathReport.msg` are based on the netlist after it is mapped to Palladium XP primitives, while the `crit.v` schematic is based on the design netlist. When several user gates are mapped to a single emulator primitive, the nets between these gates do not appear in `criticalPathReport.msg`, and these nets are not assigned any color in the Debussy schematic. Therefore, there are “gaps” in the coloring of the critical path.
2. If a sequence of more than 5 gates is assigned to a single emulator primitive, not only will there be a gap in the coloring of the path, there can be a gap in the path itself that is displayed in the schematic. Fortunately, such gaps are rare.
3. Ideally, coloring should be shown only on the single fanout of a net that is part of the critical path. Unfortunately, the tool currently shows color on an entire net, rather than just on one fanout. This can be confusing, especially for clocks, where all fan-outs are shown in color rather than just the single fanout that is on the critical path.

If the schematic is confusing, compare it carefully with the post-schedule critical path report in `criticalPathReport.msg`.

Effects of Memory on Step Count

The synthesis step of compilation can implement a multi-dimensional register array using "hard-macro memories" (indicated by MPR and MPW primitives in the synthesized netlist) or using flip-flops and latches. Later, the `et3compile` step can convert some hard-macro memories to flip-flops and latches.

Hard-macro memories are implemented using Palladium XP EDRAM and XDRAM memory resources. Each domain has 72 MBytes of EDRAM. If the domain has a User Memory Card, the User Memory Card provides 4 GByte of XDRAM. Multiple user memories can be packed into a single EDRAM or XDRAM memory, which must then be accessed multiple times per FCLK. EDRAM accesses take fewer steps than XDRAM accesses. Therefore, the compiler uses EDRAM whenever possible, and uses XDRAM only for deep memories, or when the EDRAMs are already used up by shallower memories.

The `et3compile` program generates the `dbFiles/<top>_et5mpart` file. This file lists all the memories that will be implemented in EDRAM or XDRAM. The file contains one line for each design memory, as shown in the following example:

Addr	Data	Port	Name
3	8	2	uPlatform.RFifo
4	38	3	uPlatform.TestRows
6	38	3	uPlatform.Fusemem

UXE User Guide

Improving Emulation Speed

The line for each memory shows the number of address bits, the number of data bits, and the number of ports for that memory.

The post-schedule critical path report sometimes includes memory port primitives, shown as boxes of type XMPRe or XMPWe (for EDRAM) or XMPRx or XMPWx (for XDRAM), as in the following example:

```

00 ET5ANAL* | Count  Location      Step  Jump  Type      ID      ins:cr      Name
00 ET5ANAL* | -----
...
00 ET5ANAL* |    . [00,022,025,0]  338 +   0  NET      527963      1
mem.CHAIN2_PORT4
00 ET5ANAL* |   10 [00,022,000,0]  266 +  72  XMPRx     502888  29:29
ET5_V1_H1.mem.CHAIN2_PORT4
00 ET5ANAL* |    . [00,022,000,0]  232 +  34  NET      185858      1
ET5_V1_H1.mem.CHAIN2.sync0_1to2
00 ET5ANAL* |   11 [00,022,000,0]  232 +   0  XMPWx     503094  66:43
ET5_V1_H1.mem.CHAIN2_PORT3
...

```

If memory port primitives appear in the critical path report, it is possible that the scheduling of memory accesses is contributing to the step count for the design; especially if ports of a particular memory consistently appears in the report and this memory is implemented in XDRAM.

When memory accesses contribute to a high step count, you can sometimes improve the step count by one of the following actions:

- Turn off *write-through behavior* for the given memory, or for all memories.

By write-through behavior, we mean that if an address *A* is both written and read in the same FCLK cycle, the value read will be the newly written value. xeCompile normally synthesizes memories to have write-through behavior. The `memWriteThru` userdata command and the `memoryWriteThru` compiler option can be used to turn off write-through behavior for a given memory, or for all memories in the design. With write-through off, a value written to an address *A* at some FCLK cycle cannot be read until the next FCLK cycle.

- Provide `shadowsData` information, if appropriate.

If nets before or after the XMPR or XMPW have QTshadow in the name, the ports are shadow ports that allow memory read or write to happen at the falling edge of the fastest clock in 1X mode. If the memory read or write data cannot change at the falling edge of the fastest clock, consider using supplying `shadowsData` information as described in [When to Use 1X Mode](#) on page 1264.

■ Re-model the memory

Sometimes, carelessly-written RTL can cause xeCompile to synthesize a memory with unexpectedly many ports. If the `tmp/<top>_et5mpart` file shows an unexpectedly large number of ports for a memory that appears in the critical path, review the RTL to see if the large number of ports is really needed for the memory function.

■ Specify the memory as a sparsely-used memory, if appropriate.

A synthesizable testbench can contain a very deep memory that is used in a “sparse” fashion: a given test writes only a small fraction of the locations in the memory. To reduce the XDRAM requirement for such a memory, Palladium XP provides the `memoryTransform userdata` command. The `memoryTransform -add {inst SPARSE n p}` command specifies that memory *inst* should be implemented using a virtual memory scheme, with a set of 2^n pages, each of depth 2^p . At run time, if the design writes more than the available number of pages, an error message is generated (and if you are using STB mode, the emulation stops). The run-time `memory -sparse_mem_info` command gives status information for sparse memories at run time, and the command `memory -reset inst` clears the specified memory, freeing all its pages.

■ Specify memory compaction, if appropriate.

The memory compaction step of `et3compile` can replace deep memories less than 64 bits wide, with shallower 64-bit-wide memories. Compaction of memories is done automatically if the compile would fail without it.

Memory compaction can either increase or decrease step count. Compaction can increase step count because compacting a memory increases its port count, so that more memory accesses must be scheduled within each FCLK. Compaction can decrease step count by reducing the amount of the emulator’s memory required for user memories, so that the compiler need not pack as many user memories into a given emulator memory.

Sometimes you can improve step count by forcing more compaction than occurs by default. To force extra compaction, use the `COMPACT` option of the `memoryTransform` command.

Using breakNet and breakPin

You can use the `breakNet` and `breakPin` userdata commands to break long paths at specific locations. In some designs, it is possible to find a small number of breakNets which break long paths and improve step count.

However, for most designs, use of breakNets should be considered only as a last resort. Use of breakNets has the following drawbacks:

- It is usually difficult to find a small number of breakNets that significantly improve the step count.
- The required breakNets can change as the design changes
- The added breaks can cause functional problems or require extra oversampling.

If you decide to use breakNets, the following guidelines can be helpful:

- Try to delay nets whose value never changes, or for which a one FCLK delay will not affect functionality.
- Avoid adding multiple delays on one combinational path.
- Avoid delaying clocks.
- Avoid delaying nets whose names can change with resynthesis.
- When delaying a bit of a bus, it is sometimes helpful to delay all bits of the bus.
- If the post-schedule path always ends in a certain module, it is sometimes helpful to put breaks on all inputs of that module.

The delay added by breakNet and breakPin is normally 1 FCLK cycle. For certain nets in 1X or 0.5X mode, or for all nets if the precompile -breakHalfCycle option is used, the default delay added by breakNet and breakPin is only one-half FCLK. A one-half FCLK break will not give any improvement in step count; in such a case, you can specify a breakNet delay of *2 units*, which is one FCLK cycle.

A Possible Shortcoming of the Post-Schedule Path Report

In one design, a long combinational path appeared in the pre-schedule critical path but not in the post-schedule critical path. Eventually it was determined that the long combinational path had caused a high step count, although it did not appear in the post-schedule path.

To obtain the most reliable guidance from the post-schedule path report, you should do multiple compiles (for example, five or 10 compiles) and look for similarity in the resulting post-schedule critical paths. You can also consider the pre-schedule critical path, although usually, if the pre- and post-schedule paths are different, you should give most attention to the post-schedule path.

Effect of Vision Mode

In some designs, your choice of vision mode can significantly affect step count. Usually, DYNP has the best step count, followed by FV.

Use of `compilerEffort`, `optimizationEffort` and `placementEffort`

In Palladium XP, you can specify a non-default value `LOW` for any of the compiler options `compilerEffort`, `optimizationEffort` and `placementEffort`. (`compilerEffort LOW` is equivalent to the combination of `optimizationEffort LOW` and `placementEffort LOW`). Usually, the default value `HIGH` for all these options gives the best step count, but in some designs a `LOW` value for one of the options gives a better result.

Files to Send to Customer Support

Palladium XP Customer Support engineers can sometimes help improve emulation speed. When requesting assistance from Customer Support, always send the complete design database (actually, `QTDB` directory, `et3config` file and `xe.msg` from a compile are enough) so that compile experiments can be performed at Cadence. If you cannot send the design directory, then send at least the following:

- complete `tmp` directory after running `getCriticalPathDesignNetNames`
- dump of `userdata`
- `xe.msg`
- `dbFiles/*et5mpart` file
- `dbFiles/*ctl` file
- `crit.v`, `crit.v.path`, and `crit.v.ses` files after running the `critical crit.v` command.

Debugging Designs on Palladium XP

This chapter discusses how the design database, derived after compilation, can be run and debugged using the xeDebug and *irun* tools in the SA and ICE modes using script or command mode.

If you can want to use the xeDebug GUI for debugging designs, refer to [Appendix F, “Using the xeDebug GUI”](#) for information.

This chapter includes the following sections:

- [Basics of Starting Debug Session](#)
 - ❑ [Invoking the Debugger Tool\(s\)](#)
 - ❑ [Command Sequence for ICE Mode](#)
 - [Debugging Process for Vector Debug](#)
 - [Debugging Process for LA/STB](#)
 - ❑ [Debugging Process for SA Mode with xeDebug](#)
- [Tracing Signals Using Probes](#)
 - ❑ [Setting Probes](#)
 - ❑ [Uploading Probes](#)
 - ❑ [Probing Modes](#)
 - ❑ [Changing the Vision Mode at Run time](#)
- [Tracing Signals Using InfiniTrace](#)
 - ❑ [InfiniTrace Sessions](#)
 - ❑ [Using InfiniTrace in ICE Mode](#)
 - ❑ [Using infinTrace Command Options in SWIT and HWIT Modes](#)
 - ❑ [Comparing SWIT and HWIT Modes](#)

UXE User Guide

Debugging Designs on Palladium XP

- ☐ [Using InfiniTrace in SA Mode Using xeDebug](#)
- ☐ [Using InfiniTrace in SA Mode Using irun](#)
- ☐ [Comparing the InfiniTrace Feature in SA and ICE Modes](#)
- ☐ [Bookmarking InfiniTrace Session](#)
- [Tracing and Uploading Waveforms](#)
 - ☐ [Tracing and Uploading Waveforms in ICE Mode Using xeDebug Tool](#)
 - ☐ [Tracing and Uploading Waveforms in SA Mode Using xeDebug Tool](#)
 - ☐ [Tracing and Uploading Waveforms in SA Mode Using irun](#)
- [Dynamic Swap between Hardware and Software in SA Mode](#)
 - ☐ [Using Hot Swap Options](#)
 - ☐ [Using Hot Swap Run Modes](#)
 - ☐ [Software Simulation Mode](#)
 - ☐ [Run Mode](#)
 - ☐ [TBrun Mode](#)
 - ☐ [Autorun Mode](#)
 - ☐ [Match Mode](#)
- [Accessing Signal Values](#)
 - ☐ [Forcing Values](#)
 - ☐ [Releasing Values](#)
 - ☐ [Depositing Values](#)
- [Debugging in Offline Session](#)
 - ☐ [Generating Trace Data and Preparing for Offline Session](#)
 - ☐ [Debugging in Offline Session to Save Emulator Time](#)
 - ☐ [Using Offline Debugging](#)
 - ☐ [Additional Debugging Features in Offline Session](#)
 - ☐ [Examples of Using Offline Session](#)
- [Files Generated during Debugging](#)

■ Running Designs with Uncontrolled Clocks Using xeDebug in SA Mode

Basics of Starting Debug Session

Overall, before starting to debug the design, you need to set up the debug session and the conditions for launching the Debugger tool. Then, you can launch the Debugger tool to begin the debugging operations. The set-up part of the debugging operations includes setting probes, specifying the conditions that will cause a trigger, setting vector files, and so on. After the set up, you can run the design to obtain test results of the run. These test results can be viewed and analyzed using the Debugger browser tools. You can use the browser tools to traverse the design hierarchy and select design objects such as nets, memories, instances or HDL source lines for debugging purposes.

Note: During a debug session, the set-up and run steps can be repeated in any order. For example, you can set the probes and trigger, then run the design, then change probes and trigger, run again, view waveforms, and so on.

Designs can be debugged in an online session (while the emulator connection is available) and offline session (when the connection to the emulator is not available). In both type of sessions, the test results can be viewed in the browsers.

The process of debugging and its steps change slightly, depending on whether you are using LA, STB or VD mode, or whether your design contains memories.

Invoking the Debugger Tool(s)

For running and debugging the designs in the SA mode, you can use either *irun* or xeDebug tool. In the ICE mode, you can only use the xeDebug tool.

For information about invoking either of these tools, refer to Appendix O, “Invoking Palladium XP Compilers and Debugging Tools.”

Command Sequence for ICE Mode

To run a design in the ICE mode, you need to perform the following steps:

1. Open a debug session and specify the directory path to the design database:

```
debug <dbPath>
```

For example:

```
debug .
```

 Important

Different directories can be used for the Compiled database and for running the Debugger tool so that the Debugger files that are created during emulation, are separated from the design database files.

Only one debug session can be run from a single directory. However, multiple debug sessions can be run from different debug directories using the same design directory.

2. Specify the emulation host and download the design:

```
host .  
download
```

Note: The emulation host can be alternatively specified using the `xeDebug -host` command.

3. Specify the debugging mode as VD, STB, or LA (for ICE designs only):

```
configPM -la|-vd|-stb
```

For example:

```
configPM -la
```

4. Set up the target connection parameters, and clock parameters, if required.

5. Optionally, if the design contains memories, initialize the memories before starting any testing.

The steps mentioned above are mandatory. After these steps, you can execute various set of commands for debugging operations, as documented in the *UXE Command Reference Manual*.

Debugging Process for Vector Debug

1. Open the design. Specify the emulation host and download the design to the emulator.

2. Specify the Vector file (the file can be in any one of the accepted vector formats).

Add probes, if desired.

Specify break condition through SDL, or number of cycles to run, if desired.

Run the design, and observe data on the waveform display. The `run` command also reports back whether or not the actual results match expected results.

3. The Waveform browser automatically displays the comparison between expected and actual results, highlighting any mismatches.

Debugging Process for LA/STB

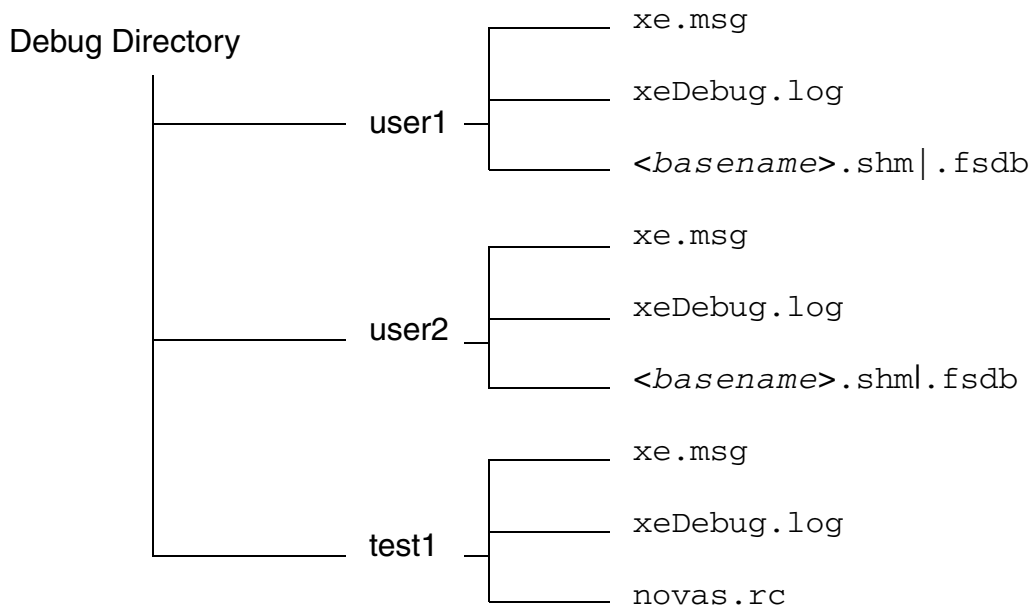
1. Open the design. Specify the emulation host and download the design to the emulator.
2. Add probes.
3. Using SDL, define a trigger for stopping the clocks in STB mode or for stopping data collection in LA mode.
4. Initialize contents of all memory instances to 0 or 1, or load contents of specific memory instances from files.
5. Force some nets to specific constant values.
6. Run the design.
7. Inspect values of specific nets.
8. Inspect contents of memory instances, or edit their contents.
9. Upload probed data for viewing in the waveform browser.
10. Inspect the design using the waveform tools and/or by reading memories.

Multiple User Sessions

In all modes, the xeDebug tool allows you to set up multiple user sessions. Each session is associated with a directory where the results are stored. These sessions are stored in automatically-created subdirectories under the debug directory (which is the directory from which you started the debugger). These subdirectories store all the files associated with that particular user session. Thus, you can have multiple output files, but each will be stored in the specific directory associated with the session.

In multi-user sessions, opening a user session switches the `xeDebug.log` to the session directory. If the log file was already opened by another user, then the log is created with a temporary name and will become `xeDebug.log` after opening a session in a subdirectory. This log file contains an exact copy of your terminal or console window. The file is incremental over multiple debug sessions.

Figure 18-1 Sample Multiple Session Hierarchy



You can specify any text as the session name. The session directory is named `<session_name>.session` file. All output files for these user sessions are stored in the appropriate session directory. Sessions must be released when moving from the online debugging session to the offline debugging session (or vice versa).

Note: From this point onwards, the online debugging session will be referred as online session and the offline debugging session will be referred as offline session

Debugging Process for SA Mode with xeDebug

To run a design in the SA mode with xeDebug, you can perform the following steps:

1. When you invoke the xeDebug tool, you must explicitly start a debug session using the following command:

```
debug <dbPath>
```

For example:

```
debug .
```

2. Specify the emulation host to which the design should be downloaded:

```
host <SA_host>
```

The specified emulation host must have a SA card to enable its interaction with IES. In the SA mode, the `download` and `configPM` commands are executed implicitly by the

UXE software during swap-in (when switching from pure simulation to SA mode). Therefore, you need not execute these commands separately as required in the ICE mode.

3. When you have finished running the design on the simulator and want to run it on the emulator, you can *hot swap* by using the `xc on` command to specify the run mode (`run`, `tbrun`, `autorun`). Then, use the `run -swap` command to complete the *hot swap*.

For example:

```
xc on -run
run -swap
```

4. If running the design under *autorun* mode, you can use the `configPM` command to switch between the Static Target and Dynamic Target debugging modes. Use the following command after *hot swap* for this action:

```
configPM -staticTarget|-dynamicTarget
```

5. Set up the target connection parameters, and clock parameters, if required.
6. Add probes.
7. Using SDL, define triggers for stopping the clocks.
8. If the design contains memories, initialize contents of all memory instances to 0 or 1, or load contents of specific memory instances from files.
9. Force some nets to specific constant values.
10. Run the design. You can dynamically swap the design from the hardware to software and back again while running a design. For details, refer to the Dynamic Swap between Hardware and Software in SA Mode section.
11. Inspect values of specific nets.
12. Inspect contents of memory instances, or edit their contents.
13. While running the design on the hardware, upload probed data for viewing in the waveform browser.
14. Inspect the design using the waveform tools and/or by reading memories.
15. To run a design in SA mode using `irun`, the steps are similar to using `xeDebug`

Debugging Process for SA Mode with *irun*

To run a design in the SA mode with `irun`, you can perform the following steps:

1. When you invoke *irun* without the `-xedebug` option, it is not necessary to specify a debug session or a host. The tool implicitly executes the following commands:

```
debug .  
host .
```

In addition, the tool implicitly executes the `configPM` and `download` commands.

2. The rest of the steps are just like running a design in the SA mode with xeDebug. Refer to the [Debugging Process for SA Mode with xeDebug](#) section for the remaining steps.

For example:

```
xc on -run  
run -swap  
probe -create dut -depth all  
sdl -enable  
sdl -verify  
force dut.sig1 1  
value dut.sig2  
run  
database -upload
```

Also, refer to [Chapter 5, “Compiling and Running Simulation Acceleration Designs with *irun*”](#) for details specific to running SA designs with the *irun* tool.

Tracing Signals Using Probes

Before the first run of the design, either at compile or debug time, you need to set probes on the signals for which you are interested in viewing run-time values. This is the initial default list of probes. The results can then be viewed in the browsers. After viewing results, more probes can be added, without re-running emulation, either by requesting the new probes to be uploaded through the xeDebug and SimVision tools, or by issuing the relevant `probe` and `database` commands.

Probes can be either virtual or physical. Physical probes are based on actual data obtained via emulation. Virtual probes are probes that were not obtained directly from emulated logic, but instead calculated in software from physical probe data.

When data tracing stops, either due to a trigger or exhausting the requested cycle count, the physical (raw) probe data can optionally be uploaded from the hardware to a disk file (by default called `trace.phy`, and can be renamed by using the xeDebug GUI or command options). After additional transformations that calculate the virtual probes, the data is written to the output file, called waveform database, which is displayed in the waveform tool. The default format of the output file is SST2. By using the run-time menus or commands options,

you can specify a different name for the output file. You can choose to view the waveforms in the FSDB format by specifying the `-fsdb` option of the `database` command at the time of invoking `xeDebug`. After invoking `xeDebug`, you cannot change the waveform file format. The FSDB format is supported to enable easy debug of Palladium-generated databases when using Novas debugger tools like Debussy, Verdi, and Siloti. For more information on using Novas Tools, refer to [Chapter 20, “Using Novas Tools.”](#)

The `database` command supports only one active database at a time. A request to open a database closes the current database before opening a new one. At the same time, `database` command directly passed to NC-Sim has no limitation on the number of databases.

Setting Probes

Probes can be set in both ICE and SA modes, and in any part of the design.

If the design is compiled for FV (Full Vision) mode, after viewing results, more probes can be added and uploaded for viewing without re-running the emulation, either by requesting the new probes to be uploaded through the Debugger menus or by issuing a `database -upload` command. If the design is compiled for DYNP (Dynamic Probes) mode, after adding new probes, the design must be run before the trace for these probes can be uploaded and viewed in the waveform browser. In DYNP mode, adding the new probes only affects future runs. There is no way to retrieve the traced data for a previous run. In FV mode, you can add probes after a run and still retrieve your traced data from that run.

Table 18-1 Probing Options According to Vision Modes in ICE and SA Modes

Debug Mode	FullVision (FV)	Dynamic Probes (DYNP)	Dynamic Target	InfiniTrace	
				Hardware InfiniTrace (HWIT)	Software InfiniTrace (SWIT)
Vector Debug (VD)	Yes	Yes	No	No	No
Synthesizeable TestBench (STB)	Yes	Yes	No	Yes	Yes
Logic Analyzer (LA)	Yes	Yes	Yes	No	No
Simulation Acceleration (SA)	Yes	Yes	Yes, but limited	Yes	No

Uploading Probes

The uploaded probed data can be used for:

- Working in online session on the emulator and viewing emulation results online immediately.
- Working in online session, but not viewing the results immediately; instead, the data is saved to a physical probe file for later analysis in offline debug mode.
- Working in offline session (without an emulator) and viewing the results from a physical probe file.

Uploading data from a design with numerous probes can be time consuming. It might be desirable to only use a subset of the total probes.

You can also find signal values for the current emulation time (a single time point) by issuing the `value` command from the `xeDebug` command line.

Probes can be set using either the GUI or the CLI. Refer to [Memory Tab](#) on page 1228 for using the `xeDebug` GUI options. Steps to set probes and view the waveforms using commands are documented in the following section.

Setting Probes through Commands

1. Assign probes to the specified nets in the design using the `probe` command.

Note: When debugging the design using *irun*, do not use the `probe` command to set probes in the DUT. *irun* does not access the DUT correctly unless the simulation control is swapped into the simulator. For more information on setting probes using *irun*, refer to the *Run-Time Commands* chapter in the *UXE Command Reference Manual*.

2. Create a probe during run time using the following command:

```
probe -create <options>
```

3. Upload the trace results from the emulator into the SHM or FSDB waveform database using the following command:

```
database -upload
```

For more information on the above mentioned commands and their options, refer to *UXE Command Reference Manual*.

Probing Modes

The required mode of probing is selected at the compile time. The following probing modes are the supported in UXE:

- FullVision Mode (FV mode)
Note: This is the default mode.
 - Selective FullVision (FV)
- Dynamic Probes Mode (DYNP mode)

FullVision Mode

FullVision mode allows you to probe and view the waveform of any design signal at run time, without recompiling or re-running. With this mode, you can:

- specify probes to be uploaded before running the design
- add and upload additional probes, after running the design
- add and upload probes as often as needed, without re-running the design.

You can still define signals to probe at compile time, but that is merely as an initial default list.

FV mode provides full visibility to the design. Waveform of any net in the design can be viewed using FV mode. In FV mode, the compiler automatically adds enough probes to ensure the full visibility of all signals. Unlimited number of probes can be added either before or after running the emulation. All the nets in the design can be probed.

With FV mode, offline session can be used. When waveform is uploaded during online session, all the data which is necessary for FV mode is uploaded from emulator and is saved in the *.phy subdirectory. An offline session can be started afterwards in which the *.phy subdirectory is used to generate waveform for any nets in the design. The *.phy subdirectory is generated by `database -upload` command or `database -prepareOffline` command during online session.

The FV overhead is the resource required to capture signal values into Data Capture Card (DCC). The number of signals that can be captured per domain per step is fixed. If there are more signals to be captured, consequently either more domains or more steps are required. Trace depth is also affected by the number of signals. The fewer the signals the larger the trace depth.

The **advantage** of using the FV mode is:

- An unlimited number of signals can be probed. The signals to be probed do not need to be chosen before running.

The **disadvantages** of using the FV mode are:

- The FV mode has a larger capacity cost than DYNP. The cost is due to a larger step count which means slower emulation speed and larger capacity because it requires more DCC resources.

The time it takes to upload data from the emulator to the disk depends on the size of the design and can be significant for large designs. This is true even if the number of probes requested to be viewed on the waveform is small.

Maximal trace buffer depth and upload time can be adversely affected when using conditional acquisition due to the frame-based tracing in FV mode. That is because a whole trace frame (typically 32 consecutive cycles) is captured at any time SDL requests to trace any of the cycles in the frame.

Selective FullVision (FV)

When you need to debug only specific instances/modules in the design, FV can be applied only on those specific instances/modules. With Selective FV mode, you can enable or disable specific instances and modules and set probes on the enabled region. Selective FV can be used in both ICE and IXCOM modes.

FullVision will be applied only to the enabled region in the design.

Note: To enable selective FV, `compilerOption -add visionMode` option must be set to FV.

To enable or disable specific instances and specific modules, use the `instanceFV` command and the `moduleFV` command at compile time respectively. By default, FV is enabled on the entire design if nothing is specified with the `instanceFV` and `moduleFV` User Data commands.

The following command enables or disables FV on modules:

```
moduleFV -add {<LIB cell> <enable | disable>}
```

Use the following command to enable or disable FV on instances:

```
instanceFV -add { <instance_name> <enable | disable>}
```

For more information on `moduleFV` command and the `instanceFV` command, refer to the *User Data Commands* chapter of the *UXE Command Reference Manual*.

The **advantages** of using selective FV mode are:

UXE User Guide

Debugging Designs on Palladium XP

- Faster upload speed can be achieved at run time.
- Smaller *.phy subdirectory size because fewer nets are probed.

The **limitation** of using selective FV mode is:

- Depending on how the design is mapped to domains, all available DCCs might not be used because of limited domain-to-domain and board-to-board connections. Therefore, selective FV does not ensure a deeper trace depth.

The following [Table 18-2](#) marks out the differences between selective FV and FV mode.

Table 18-2 Differences between Selective FV mode and FV mode

	Selective FV	FV
Full visibility	No, only on enabled instances/modules	Yes
Adding probes and uploading waveforms	Yes, only on enabled instances/modules	Yes
Offline debugging	Yes, only on enabled instances/modules	Yes
InfiniTrace	Yes	Yes
Capacity and speed overhead	Less	More
Maximum trace depth	Same or Larger	Same or Smaller

Parallel Computation for FullVision using Multiple CPUs and Multiple Hosts

The UXE software has the same computation engine for FullVision probes and Dynamic Power Analysis (DPA). Therefore, computation of FullVision probes can also make use of multiple CPUs and hosts as DPA does. For detailed information, refer to the following two sections of [Chapter 15, “Analyzing Power Consumption of the Designs”](#):

- [Parallel Computing for DPA Using Multiple CPUs](#)
- [Running DPA with Multiple Hosts](#)

Dynamic Probes Mode

DYNP mode allows the dynamic probing of any net in the design. In DYNP mode, you must define probes before running the emulation as waveforms of any nets in the design cannot be generated. Dynamic probes need to be configured on the emulator hardware.

The number of nets that can be probed depends on the number of emulator domains for which the design is compiled, and `numCapturedNetsPerDomain`.



Caution

If the resources are not sufficient for probing, adding a set of probes can fail.

To increase the number of probes that can be added:

- Increase the number of domains for compiling the design
- Increase `numCapturedNetsPerDomain`

Following are the **advantages** of using the DYNP mode:

- Enables fastest waveform upload.
- Avoids the costs related to emulator capacity and emulation speed that occur in some designs when large number of nets need to get probed in FV mode.
- Provides a larger maximum possible trace depth.

Following are the **disadvantages** of using the DYNP mode:

- Provides limited number of signals that can be probed in a specified run.
- Probes need to be defined before running the emulation.

Supported Probing Strategies for Each Probing Mode

Different probing strategies are supported under different debug modes. [Table 18-3](#) shows what functionality is available for all emulation debug modes.

Table 18-3 Palladium XP Probing Options by Debug Strategy

Debug Mode	Dynamic Probes (DYNP)	FullVision (FV)	InfiniTrace
Vector Debug (VD)	Yes	Yes	No
Synthesizable TestBench (STB)	Yes	Yes	Yes
Logic Analyzer (LA)	Yes	Yes	No
Simulation Acceleration (SA)	Yes	Yes	Yes

For designs compiled with the STB mode in ICE flow, both hardware InfiniTrace (HWIT) and software InfiniTrace (SWIT) are supported. However, in the SA mode, only hardware InfiniTrace (HWIT) is supported. The support for InfiniTrace is turned on by default. If not required, you can disable it during compilation.

Comparison between FV and DYNP Modes

Following table summarizes the comparison between DYNP and FV modes.

Table 18-4 Comparison between DYNP and FV mode

Differences	DYNP	FV
Full visibility	No	Yes
Probes can be added after run and upload waveform	No	Yes
Offline debugging support	No	Yes
InfiniTrace support	Yes	Yes
Speed overhead	Less	More
Maximum trace depth	Larger	Smaller
Conditional acquisition	Cycle-based	Frame (32 cycles) based

Differences	DYNP	FV
Full visibility	No	Yes
Probes can be added after run and upload waveform	No	Yes
Offline debugging support	No	Yes
InfiniTrace support	Yes	Yes
Speed overhead	Less	More
Maximum trace depth	Larger	Smaller
Conditional acquisition	Cycle-based	Frame (32 cycles) based

The following sections describe some of the main differences between FV mode and DYNP mode in detail:

- Capacity and Emulation Speed Overheads
- Trace Buffer
- Conditional Acquisition

Capacity and Emulation Speed Overheads

FV mode has **more** overhead on capacity and emulation speed as compared to DYNP. Ideally, designs should be compiled in the FV mode. However, if the capacity and emulation speed is critical, compile the design in DYNP mode with a relatively small number of probes.

In Palladium XP hardware, each MCM has one Data Capture Card (DCC), which is the memory buffer to capture waveform data. Each DCC can capture a fixed number of probes, called probing channels. The capacity and emulation speed overhead comes from the cost of probing channels.

FV mode needs to capture much more probes than DYNP mode, therefore, it requires more probing channels. There are two ways to increase the number of probing channels:

- Increase the number of DCC, i.e the number of domains. DCC is the source of capacity overhead.
- Increase the number of probing channels on each DCC by increasing the step count. In each step, Palladium XP allows only a limited number of probes to be transformed from MCM to DCC. The bandwidth from MCM to DCC is small, so the step count needs to be increased in order to transform more data from MCM to DCC. This is the source of emulation speed overhead.

In the DYNP mode, you need to specify the number of probing channels. The number of probing channels specified in DYNP mode is usually much smaller than what is required in FV mode. This results in a **higher** overhead than DYNP mode. If the number of probing channels specified is very large, the overhead in DYNP mode can be almost as much as the overhead in FV mode.

Trace Buffer

During emulation, the trace buffer is used for capturing the trace data. If the trace buffer is filled up, the oldest data in trace buffer is replaced by the newest data. The trace buffer always contains only the latest trace data.

When uploading a waveform, you can specify a time window ending at the last emulation cycle, this time window is called trace window. The trace window contains the newest captured data. The trace window size can be specified by using the `xeset traceMemSize` command. The trace window size must be smaller than trace depth. The maximum size of trace window that you can specify is the trace buffer depth.

Conditional Acquisition

In DYNP mode, the decision to capture/discard is taken at each fclk cycle.

In FV mode, the decision to capture/discard is taken per unit of frame, which is 32 fclk cycles. If a FCLK cycle within a frame needs to be captured, then the whole frame is captured. This has the disadvantage that a lot of unwanted fclk cycles are captured. The extra cycles can also be seen in the waveform, and the maximal trace depth is reduced because of these extra cycles.

When using conditional acquisition, the requested trace window size as specified by `xeset traceMemSize` refers to the cycles that are requested to be acquired by SDL, and not to the actual number of traced cycles. When using FV probing mode, the actual number of traced cycles can be higher than the number requested because the acquisition is frame based, so a whole frame (typically 32 cycles) is captured whenever any of the cycles in that frame is requested to be captured.

Controlling Maximal Trace Depth

The maximal trace depth can be controlled only in the DYNP mode.

Following are the different scenarios by the combination of `traceDepth` parameter and `numCapturedNetsPerDomain` parameter, which are specified through the `compilerOption` command during the compilation process:

- `traceDepth` and `numCapturedNetsPerDomain` both are not specified

In this case you do not need to know the number of probes that can be added and the trace depth. Here, the default value depending on the design is used. For large designs, the default value of `traceDepth` is 1M and the default `numCapturedNetsPerDomain` is 16K. For small designs, the default value of `traceDepth` is larger because the design does not need larger number of probes and `numCapturedNetsPerDomain` can use a smaller value. For example, a very small design can have default `traceDepth` of 4M.

- Only `numCapturedNetsPerDomain` is specified

In this case you might want to add more probes at run time because the default value of `numCapturedNetsPerDomain` is too small for the design. `numCapturedNetsPerDomain` value can be specified up to 307200.

- Only `traceDepth` is specified

In this case you might want more trace depth because the default value of `traceDepth` is too small for the design. For example, if in a huge design only 100 I/O signals are needed for a long period of time, the available trace depth can be 20M cycles (`numCapturedNetsPerDomain` is 768 in this case).

- Both `traceDepth` and `numCapturedNetsPerDomain` are specified

In this case both long trace depth and large number of probes are required. This requirement is difficult to meet. The compiler calculates the product of `traceDepth` and `numCapturedNetsPerDomain`. If the product is larger than 8*4G, then compiler errors out. Otherwise, the compiler accepts `numCapturedNetsPerDomain` value and ignores `traceDepth`. The trace depth results in $8*4G/\text{numCapturedNetsPerDomain}$.

Specifying traceMemSize

Use `xeset traceMemSize` at run time to specify how many cycles to upload. `traceMemSize` must be smaller than the maximum trace depth.

You can set `traceMemSize` in the unit of simulation time.

For example:

```
run 1000ns
xeset traceMemSize 10ns
database -upload
```

The above example results in a waveform from 990ns to 1000ns.

`traceMemSize` can be set in the following ways:

- Set value and unit. This value is a floating point value. For example:

```
xeset traceMemSize 0.012s
xeset traceMemSize 1.2e-2s
```
- Set value only. In this case the specified unit uses the current value for `xeset timeUnit`.
- Set `fclk` cycles. Use `fclk` as the unit. In this case, the `database -upload` command is used to upload the specified number of samples, instead of a time window. For example:

```
xeset traceMemSize 128000fclk
```

Note: In the SA mode, the `fclk` time unit is not supported; instead, `ns` is used as the default time unit.

Changing the Vision Mode at Run time

A design compiled for FV mode can be changed to DYNP mode at run time without recompiling the design either using the CLI or the GUI. Use the following command at run time to change the vision mode:

```
host -dynp <emulator_host>
```

Switching to the DYNP mode has the following advantages:

- Faster waveform upload performance
- Larger trace depth

DYNP mode has different trace depth than FV mode. The compiler selects a default trace depth for DYNP mode, which is generally larger than the trace depth of FV mode. You can also explicitly specify a trace depth for DYNP at compile time.

Use the following command at compile time to set the trace depth in DYNP mode:

```
compilerOption -add {traceDepthDYNP <value>}
```

For more information on the syntax and values of the `traceDepthDYNP` option refer to *User Data Commands* chapter of the *UXE Command Reference Manual*.

However, switching to DYNP mode does not improve the emulation speed because the step count is not changed. Step count is defined at compile time to accommodate probes required by FV mode.

The vision mode can also be changed when running the design in the xeDebug GUI. Before downloading the design, select the *DYNP* check box adjacent to the *Vision Mode* field on the *Setup* tab to change the mode. For detailed information on the fields available on the *Setup* tab in the SA and ICE modes, refer to the [Starting a Debug Session](#) section in Appendix F, “Using the xeDebug GUI.”

Tracing Signals Using InfiniTrace

InfiniTrace allows you to access a virtually unlimited trace window. During the emulation run, data that includes emulation history and status is periodically dumped to the disk by stopping the emulation clocks at regular time intervals. When the emulation run is stopped, the saved data can be restored to any time point in the period where it was saved, and observe the emulation status. Different parts of the run can be replayed, using different SDL trigger conditions to search a specific time point within the saved emulation history.

Note: InfiniTrace does not support wall clock time. If a number does not follow an emulation time, it is assumed to be in emulation cycles.

You must download the design on an emulator to be able to use the InfiniTrace feature. InfiniTrace is available in STB and SA emulation modes, with or without a static target system, using either FV or DYNP probing mode. It is not supported in LA mode.

In the SA mode, InfiniTrace is supported with both xeDebug and *irun*. In the SA mode, InfiniTrace replays the emulation history of DUT part, but it cannot replay the history of the testbench part. It can replay only the testbench inputs.

There is no additional gate capacity overhead associated with the use of InfiniTrace. It is enabled by default. It can be enabled or disabled at compile time by using the following command:

```
compilerOption -add {infiniTrace on | off}
```

Cadence recommends to keep the InfiniTrace feature enabled. In rare cases, where designs have large numbers of primary inputs, you can disable it to decrease the overhead.

InfiniTrace Sessions

The InfiniTrace mode involves the *Prepare* and *Observe* sessions. Additionally, designs can be run in the non-InfiniTrace mode, which is termed as the *normal* run. Following is a brief description of these types of sessions:

■ **Prepare** session:

In the *Prepare* session, the system collects data while the emulation is in progress. A special memory buffer records the data necessary to record the history of emulation. Emulation run is periodically paused to upload data from the memory buffers to the workstation's disk once the buffer is full. This can slow down the effective emulation speed. The target also pauses during the data upload. Except for the slow down, there is no difference between the normal run (without InfiniTrace) and the *Prepare* session. All operations that can be performed during a normal run can be performed during a *Prepare* session. The *Prepare* session data is saved in the `<session_name>.infiSession` sub-directory.

■ **Observe** session:

In the *Observe* session, the emulation status is restored to a specific time point by using the data collected during the *Prepare* session. In an *Observe* session, you can only perform operations that *observe* the simulation acceleration or emulation status, such as dumping memory content, getting values of nets, defining probes, and uploading waveforms and so on. You cannot perform operations that change emulation status and thus alter the history of the emulation, such as writing of memories, setting, forcing, or releasing nets, or running emulation. During *Observe* sessions, the output from the emulator to the target is frozen, so that the operations on emulator do not affect the target. In the SA mode, only the DUT is observed, and the testbench running on the software simulator is frozen.

To use InfiniTrace, you must first create a *Prepare* session to record the emulation history. Each session is identified by a specified name. The data for the session is saved in a subdirectory that derives its name from the session name. Then, in the *Observe* session you can replay the emulation history recorded in the *Prepare* session.

Using InfiniTrace in ICE Mode

InfiniTrace can be used in the following modes:

- Software-Mode InfiniTrace (SWIT)
- Hardware-Mode InfiniTrace (HWIT)

Software-Mode InfiniTrace (SWIT)

In SWIT, the software replays the emulation history without the need to occupy the hardware in the *Observe* session.



SWIT is supported only in the ICE mode. Designs compiled in the SA mode do not support it.

Preparing a Session in SWIT Mode

Add the `-sw` option to `infiniTrace -prepare` command to specify that the *Prepare* session should be run in SWIT mode. If you do not specify the `-sw` option, by default, the *Prepare* session runs in HWIT mode. [Figure F-43](#) on page 1233 displays the button in the GUI using which you can select between software and hardware.

```
infiniTrace -prepare -sw <session_name>
```

Emulation can be run after opening the SWIT *Prepare* session. The waveform data file is automatically uploaded during emulation. At the time of emulation, the trace memory in the hardware captures waveform data. When the trace memory is full, emulation is paused, and the trace memory is uploaded into the *Prepare* session data file. Emulation is resumed after uploading the waveform data. Therefore, emulation pauses for small time intervals.

With SWIT, you can create a `.infiSession` file that covers a large time window. Use `infiniTrace -on|-off` command to enable or disable data capturing. Uploading data can slow down emulation speed. So, Cadence recommends that data capturing should be enabled only when necessary.

The following commands illustrate how the `.infiSession` file can be generated for a large time window.

1. Use the following command to open a *SWIT Prepare* session:

```
infiniTrace -prepare -sw my.phy
```

2. Switch off the data capturing using the following command:

```
infiniTrace -off
```

3. Use the SDL trigger to find a start point of trace window by specifying the following commands:

```
sdl -expression <sdl_expression>  
run
```

4. Capture the specified number of cycles after the trigger point by using the following commands:

```
infiniTrace -on  
run 10000000  
infiniTrace -off
```

SWIT runs slower than HWIT in the *Prepare* session because more data needs to be uploaded. Also, the session directory occupies more disk space than in HWIT. So, choose the time window carefully, and do not indiscriminately capture every cycle during emulation.

Observing a Session in SWIT Mode

In SWIT, the *Observe* session is run in offline debug mode. Use the following command to open an *Observe* session:

```
infiniTrace -observe <session_name>
```

Note: Opening a hardware-mode session in offline session generates an error.

Any operations related to waveform generation and DPA operation, creating Toggle Count (NTC) or Weighted Toggle Count (WTC), or creating TCF and power profile can be performed in the SWIT *Observe* session.

SWIT *Observe* session works in offline session. Therefore, the operations that need access to the emulator cannot be performed. For example, dumping memory content or getting a signal value using `value` command and some other actions cannot be performed.

Hardware-Mode InfiniTrace (HWIT)

In HWIT mode, the emulator hardware is required for replaying the emulation history during the *Observe* session.

With HWIT, you can enter a temporary *Observe* session from the *Prepare* session, and replay a part of the recorded history while in *Prepare* session. End the temporary *Observe* session and switch back to *Prepare* session by using the `infiniTrace -goto end` command. The temporary *Observe* session is not available in SWIT mode.

Using infiniTrace Command Options in SWIT and HWIT Modes

This section lists the commonly used `infiniTrace` command options. For the corresponding steps when using the xeDebug GUI, refer to [InfiniTrace Tab](#) on page 1233.

1. Start a *Prepare* session and assign a name for this session by using the following command:

```
infiniTrace -prepare -sw <session_name>
```

The `-sw` option specifies that the *Prepare* session opens for SWIT.

Note: This command cannot be used in an *Observe* session.

2. Enable or disable the data capture by using the following command:

```
infiniTrace -on | -off
```

Data capture is enabled by default, after you start and name a *Prepare* session.

3. Remove the specified data set, including the directory and its contents by using this command:

```
infiniTrace -rm <name>
```

The session must be closed before issuing this command.

4. Open an *Observe* session by using the following command:

```
infiniTrace -observe <name>
```

If a session is already open, it is closed.

5. Use the following command to specify the number of cycles to be uploaded:

```
xeset traceMemSize <size>
```

The `traceMemSize` value can be infinite, and it is not limited by maximum trace depth. You can specify any value for it. The `traceMemSize` run-time parameter must be smaller than the maximum trace depth. Trace depth is specified at compile time. Larger trace depths require more hardware resources. For `infiniTrace`, there is no limit for trace depth.

6. Restore simulation acceleration status to the specified time point by using the following command:

```
infiniTrace -goto <time_specification>
```

In this command, the time is specified in the following format:

- ❑ `t` - An absolute time specification, where `t` specifies the time
- ❑ `+t` - Advances from the current time, where `t` specifies the time
- ❑ `-t` - Decrements from the current time, where `t` specifies the time
- ❑ `trigger` - Searches forward from the current time until a trigger point is encountered or reaches the current *Observe* session. The trigger condition is defined the same way as it is defined in normal run.
- ❑ `begin` - Goes to the first cycle
- ❑ `end` - Goes to the end cycle

7. Specify the time interval between automatic capture points by using the following command:

```
infiniTrace -stateSaveFrequency <value>
```

This command only applies to HWIT.

8. Display information about the automatic saves during the *Prepare* session by using the following command:

```
infiniTrace -getStateSavePoints
```

9. Close a session by using the following command:

```
infiniTrace -close
```

10. List the name of all available data sets that can be used for *Observe* sessions by using the following command:

```
infiniTrace -list
```

11. List the time intervals where data capture is enabled in the current session by using the following command:

```
infiniTrace -intervals
```

This command can be used in either a *Prepare* session or an *Observe* session.

Note: The time intervals returned on execution of the `infiniTrace -intervals` command in SWIT mode are one cycle shorter than those returned in HWIT mode.

Comparing SWIT and HWIT Modes

The [Table 18-5](#) marks out the differences between the SWIT and HWIT.

Table 18-5 Differences between SWIT and HWIT

	SWIT	HWIT
Prepare Session		
Emulation Speed	Slow	Fast
Disk space coverage	More	Less
Observe Session		
Emulator needed	No	Yes
Offline session availability	Yes	No
Memory content dump	No	Yes

Using InfiniTrace in SA Mode Using xeDebug

InfiniTrace helps to save the state of the design during the run, so that, this state can be restored from any saved time points later to observe the simulation acceleration status. In the SA mode, InfiniTrace only saves/restores the DUT part of the design. After restoring the state, you can dump out memory content, add probes and upload waveform of any net in the DUT.

In the *Prepare* session, the system collects data for DUT part of the design which is running in SA mode.

In the *Observe* session, the DUT can be restored at any specified time point by applying the data collected during *Prepare* session.

The testbench is not replayed in *Observe* session. An error is reported if you try to access the testbench while in *Observe* session.

During *Observe* session, the simulator is not used and the following command sets are disabled:

- `xc` commands
- NC-Sim commands
- Commands related to testbench. For example, memory read, value, probe on nets in testbench.

To exit the *Observe* session, you must issue the `goto end` command so that the DUT and testbench are again synchronized with each other.

Using InfiniTrace in SA Mode Using *irun*

This section describes the support for the InfiniTrace feature with the use of the *irun* tool at run time. The InfiniTrace feature functions quite similarly to the manner in which it functions with the xeDebug tool as described above, except for the following differences:

- ❑ When you use the *irun* tool, you cannot use the *Observe* session; only the *Prepare* session is available.

When you are using the *irun* tool, the *Prepare* session is automatically closed if you issue any of the commands:

- `reset`
- `restart`

- ❑ You must swap in to the hardware run-time mode before opening the *Prepare* session when using the *irun* tool at run time. For example:

```
xc run
xc infinixTrace -prepare <session_name>
```

When you swap from the hardware to the software run-time mode, the InfiniTrace feature is automatically disabled. If any *Prepare* session is open, it is closed before the swap to the software mode.

InfiniTrace Commands Supported with the *irun* Tool

All InfiniTrace commands EXCEPT for the following are supported when you use the *irun* tool:

- ❑ `xc infinixTrace -observe`
- ❑ `xc infinixTrace -goto`
- ❑ `xc infinixTrace -bookmark -goto`

Specifically, the following *Prepare* session commands are supported when you use the *irun* tool:

```
xc infinixTrace -prepare <session_name> [-overwrite]
xc infinixTrace -on | -off
xc infinixTrace -close
xc infinixTrace -list
xc infinixTrace -intervals
xc infinixTrace -bookmark -add | -rm <bookmark_name>
xc infinixTrace -bookmark -list
xc infinixTrace -isCaptureEnabled
```

```
xc infiniTrace -getCurrentMode [-text]
xc infiniTrace -getSessionName
```

Like all SA run-time commands that you use with the *irun* tool, the commands specific to the infiniTrace feature must be executed with the `xc` prefix.

To observe the infiniTrace session, you must exit the *irun* tool, and start the xeDebug tool. Then, download the debug database, and open the *Prepare* session that you had created while running the *irun* tool.

Comparing the InfiniTrace Feature in SA and ICE Modes

The InfiniTrace feature functions with the following differences in the SA and ICE modes:

- In the SA mode, the time specification is in the unit of simulation time when using the `infiniTrace -goto <time>` command. In the ICE mode, the time is specified in FCLK cycles.
- When specifying trace window size, the unit is simulation time in the SA mode. You can specify any trace window size as long as the trace window is within the range where infiniTrace is turned on during *Prepare* session.
- In the SA mode, the `infiniTrace -goto <xtime>` command cannot be used to go to the exactly specified time. The data is replayed only for the simulation times saved during emulation. The replay can be for the simulation time less than or equal to the `<xtime>` specified.

For example, if time steps saved in *Prepare* session are 5ns, 10ns, 15ns

```
infiniTrace -goto 7ns will stop at 5ns
infiniTrace -goto 10ns will stop at 10ns
infiniTrace -goto 11ns will stop at 10ns
```

All other operations are the same as in the ICE mode.

Note: In the SA mode, the simulation runs slower because of the need to save design snapshot and simulator inputs.

For more information on the commands used in InfiniTrace and its modes, refer to the description of the `infiniTrace` command and its options in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

Bookmarking InfiniTrace Session

You can add bookmarks in the InfiniTrace session to mark specific time-points. Then, in the *Observe* session, you can quickly reach those time-points using the the pre-defined bookmarks.

Following are the common `infiniTrace` commands used for bookmarking:

1. Define a bookmark using the following command:

```
infiniTrace -bookmark -add <book_mark_name>
```

The current cycle is defined as a bookmark with specified name. Any text string can be used as the bookmark name.

2. Use the following command to remove a bookmark:

```
infiniTrace -bookmark -rm <book_mark_name>
```

3. List the defined bookmarks using the following command:

```
infiniTrace -bookmark -list
```

The bookmark names and the corresponding cycle numbers are displayed.

4. Use the following command to go to a specific bookmark:

```
infiniTrace -bookmark -goto <book_mark_name>
```

A bookmark can be specified for both *Prepare* session and *Observe* session.

Tracing and Uploading Waveforms

Palladium XP supports waveforms of SST2 (also known as SHM) and FSDB formats. By default, waveforms are generated in SST2 format.

The traced waveform database can be viewed by uploading it to the appropriate waveform viewer that is selected based on the waveform format. The following sections cover information about tracing waveforms in different modes using either `xeDebug` or `irun`:

- [Tracing and Uploading Waveforms in ICE Mode Using `xeDebug` Tool](#)
- [Tracing and Uploading Waveforms in SA Mode Using `xeDebug` Tool](#)
- [Tracing and Uploading Waveforms in SA Mode Using `irun`](#)

Minimum Run Cycles and Intervals before Uploads

Before uploading the debug data, the design must be run for at least one cycle for the upload to be successful. Also, a minimum interval of one cycle should be observed before doing the next upload of the debug data. The data is available from the 0th cycle until one cycle before the time of uploading the data.

Tracing and Uploading Waveforms in ICE Mode Using xeDebug Tool

Following is an example of a command sequence to generate and view waveforms in ICE mode:

1. Set the waveform format at the time of invoking the xeDebug tool as following:

```
xeDebug [-shm | -fsdb | -vcd]
```

If you do not specify the waveform format while executing the `xeDebug` command, by default the format is set to SHM.

2. Create the waveform database to store the waveform output files by using the following command:

```
database -open <database_name>
```

In the Vector Debug mode, this command is not required because the name of the waveform file is automatically set according to the stimulus vector filename. The vector name is set using the `vector` command.

3. Use XEL command `xeset`, to set the `traceMemSize` run time parameter in order to determine the number of samples for upload from the trace buffer into a file in SA, Logic Analyzer (LA), and STB modes. Setting this run time parameter affects all subsequent `run` and `database -upload` commands.

4. Add or delete probes during the run time using the `probe` XEL command. To create new probes, execute the following command:

```
probe -create <probe_name>
```

For detailed information on various options of the `probe` command, refer to the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

5. Run the design by executing the `run` XEL command.

For example:

```
run 1000
```

6. Upload the trace results to a trace file of the specified file format using the `database -upload` XEL command. If there is no data for upload, gaps are inserted into the

waveform database. For example, if you run the following commands when the design was compiled for DYNP mode:

```
probe -create netA
run 1000
database -upload
probe -create netB
run 1000
database -upload
```

You will see the waveform of netA from cycle 1 to 1998 (the last 2 cycles cannot be uploaded). But for netB, you will see the waveform for only cycles 1001 to 1998. The first half of the waveform will be blank.

After using the `database -upload` command, you can use the `probe -waveform` command to add the objects within a probe to the waveform display.

If you are using the xeDebug GUI instead of command-line interface, you can use either of the following ways to upload trace results:

- ☐ Click the *Upload* button on the *Probe* tab in xeDebug GUI.
- ☐ Turn on the *Auto Upload* option on the *Control* tab to automatically trace data, upload it and display the waveforms after each run. *Auto Upload* is set to *off* by default.

7. View the waveforms in the waveform viewer depending upon the selected waveform file format, that is, SST2 or FSDB.

By default, the waveforms are generated in the SST2 or SHM format. Such waveforms can be opened in SimVision by using the following set of commands:

From the UNIX/Linux shell:

```
%      simvision -waves <sst2_database_name> &
%      simvision -waves &
```

From XEL command-line interface:

```
XE>      exec simvision -waves <sst2_database_name> &
XE>      exec simvision -waves &
```

If `<sst2_database_name>` is not specified on the command line, the SST2 format waveforms can be opened from the SimVision GUI. For more information on SimVision, refer to the INCISIV documentation set.

For FSDB format waveforms, you need to use the Novas waveform tools, such as Debussy, Siloti, and Verdi. To open the waveforms in Verdi waveform tool installed on your machine, use the following set of commands:

UXE User Guide

Debugging Designs on Palladium XP

From the UNIX/Linux shell:

```
%      verdi -nWave -showtrigger -ssf <fsdb_database_name> &
%      verdi -nWave -showtrigger &
```

From XEL command-line interface:

```
XE>      exec verdi -nWave -showtrigger -ssf <fsdb_database_name> &
XE>      exec verdi -nWave -showtrigger &
```

If *<fsdb_database_name>* is not specified on the command line, the FSDB format waveforms can be opened from the GUI. For more information on using Novas tools to view waveforms of FSDB format, refer to [Chapter 20, “Using Novas Tools.”](#)

When running xeDebug GUI mode, the waveform can be displayed by clicking the waveform icon at the top of the GUI. The waveform can also be opened automatically if the *Auto Wave* option in the *Control* tab is turned on.

Note: If you zoom Novas nWave to 100% while the xeDebug is either generating the waveform data or sending the commands to Novas nWave, you might see the nWave waveform displayed incorrectly.

The xeDebug tool supports the VCD format. For VCD databases, probes cannot be incrementally added once transitions are written in a VCD file and it has no live waveform display. Also, you must load trace data into waveform window manually and this requires VCD to SST2 or FSDB translation on every database reload request. VCD mode is preferred for offline debugging.

To use VCD format with xeDebug tool specify the *-vcd* option with the xeDebug command:

```
xeDebug -vcd
```

Tracing and Uploading Waveforms in SA Mode Using xeDebug Tool

Following is an example of a command sequence to generate waveforms with xeDebug:

1. Set the waveform format at the time of invoking the xeDebug tool from the UNIX command prompt. Use the following command:

```
xeDebug [-shm | -fsdb | -vcd]
```

If you do not specify the waveform format while executing the xeDebug command, by default the format is set to SHM.

2. Create the waveform database to store the waveform output files by using the following command:

```
database -open <database_name>
```

3. Create a probe during run time using the following command:

UXE User Guide

Debugging Designs on Palladium XP

```
probe -create cpu_test.RISC.control -all -depth all
```

4. Run the design in the software simulator using the following command:

```
run 100ns
```

5. Swap the DUT to hardware and run using both the simulator and emulator:

```
xc on -xt0 -zt0 -run
```

```
run 100ns
```

6. Add probes to the DUT and run more with new probes:

```
probe -create cpu_test.RISC.registers
```

```
run 300ns
```

7. Upload the trace results from both the simulator and the emulator and create a waveform database using the following command:

```
database -upload
```

Accessing SST2 Format Waveform Databases in SA Mode

In the online session for SA mode, the waveform data is stored into a pair of linked databases. One database contains the data from the simulator and the files, by default, are named as `trace.dsn` and `trace.trn`. Another database contains the data from the emulator and the files are named as `trace_hw.dsn` and `trace_hw.trn`. Note the suffix `_hw` in the names of the emulator files. Both the databases are located in the same waveform database directory named, by default, `trace.shm`, as shown in the directory structure below:

```
trace.shm/  
  trace.dsn  
  trace.trn  
  trace_hw.dsn  
  trace_hw.trn
```

When you generate a new hardware database, xeDebug creates a new SHM directory, and links to the online software database. So, the new directory includes the online software data and the new hardware data.

The structure of the new directory is as shown below:

```
trace2.shm/  
  trace.dsn -- hard link --> ../trace.shm/trace.dsn  
  trace.trn -- hard link --> ../trace.shm/trace.trn  
  trace2_hw.dsn  
  trace2_hw.trn
```

Any previous files in the new database directory are moved into a subdirectory named `bak`. The hard links enable the waveforms of the software and hardware data to be merged or

integrated. See the [Viewing Integrated SST2 Format Waveforms in SA Mode in Offline Session](#) section for information about the merged waveforms functionality.

To ensure that the online and offline waveform databases and trace files match, during the online session, xeDebug saves signatures in the files.

During an offline session with xeDebug tool, if you additionally open a hardware database that was generated in online session, it is shown as a separate hierarchy.

During offline debugging and infiniTrace observe sessions, the SST2 waveform database created in online session can only be viewed and not modified.

When the SimVision tool is invoked by a command like `simvision -snapshot tbench trace.shm` or from the xeDebug tool, the snapshot and the database files of both types (software and hardware) are shown under a single hierarchy, and the Waveform Viewer displays both the waveforms merged. You need to specify the snapshot name in order to view the merged waveforms.

Viewing Integrated SST2 Format Waveforms in SA Mode in Offline Session

In the SA mode, you can use the SimVision waveform viewer to view SST2 format waveform databases to get an integrated view of the design objects of both software and hardware portions. You can also use the `xeDebug -offline` option to start an offline session.

This helps you to view, at the same time, the signal values written by the simulator and the emulator, and get a comprehensive look into the complete design.



For the integrated view, ensure that the software portion of the design is running on INCISIV11.10.009 or higher version.

Essentially, the following two features are offered through this integrated view:

- ❑ Access to testbench data in an offline session, which was not available in previous releases
- ❑ Merged waveform functionality in an offline session, which means that the data from the simulator and emulator is merged into one line

Note: The merged waveform functionality is not supported in an InfinitiTrace *Observe* session in SA mode.

In the SimVision waveform window, you can differentiate the hardware data from software data by its cross-hatched background. When the design is in swap-in state and the signals

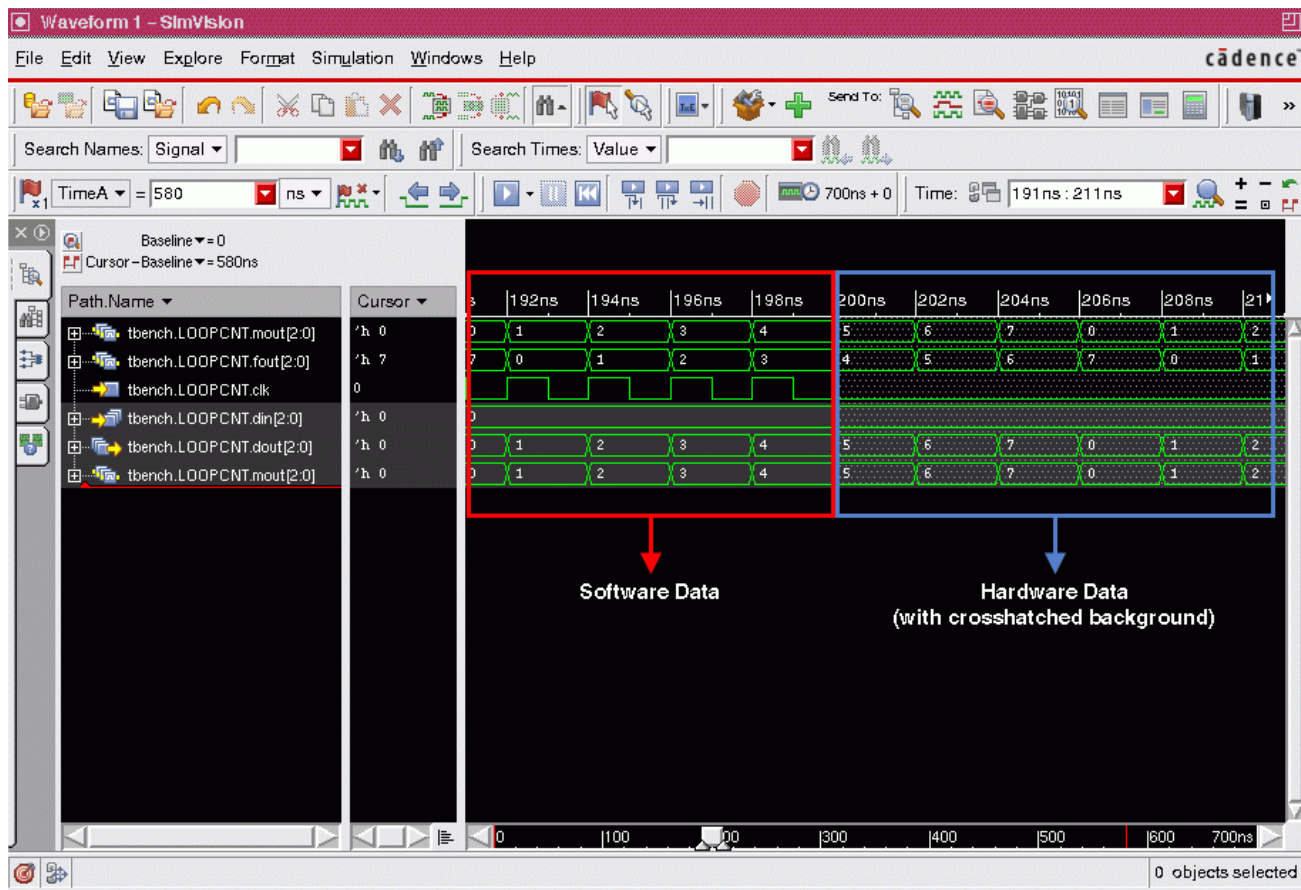
UXE User Guide

Debugging Designs on Palladium XP

are from DUT, then the background will be crosshatched. Refer to [Figure 18-2](#) for a sample SimVision waveform window.

Note: During data collection on the emulator, the crosshatched areas might not have any data. This means that the design ran for a time more than the specified trace window size.

Figure 18-2 SimVision Window with Software and Hardware Data Integrated View



Note: If you open the waveform database using the `-captureMode uncontrolled` option of the database command, the probed data from the NC-sim simulator and the data from the Palladium XP emulator do not merge in the waveform browser. This is because both use a different time scaling, and there is no way to align the transitions correctly along the time axis.

Steps:

1. Start an offline session of xeDebug and specify the *.phy subdirectory using the following command:

```
xeDebug -offline <session_name> <phyfile_name>
```

UXE User Guide

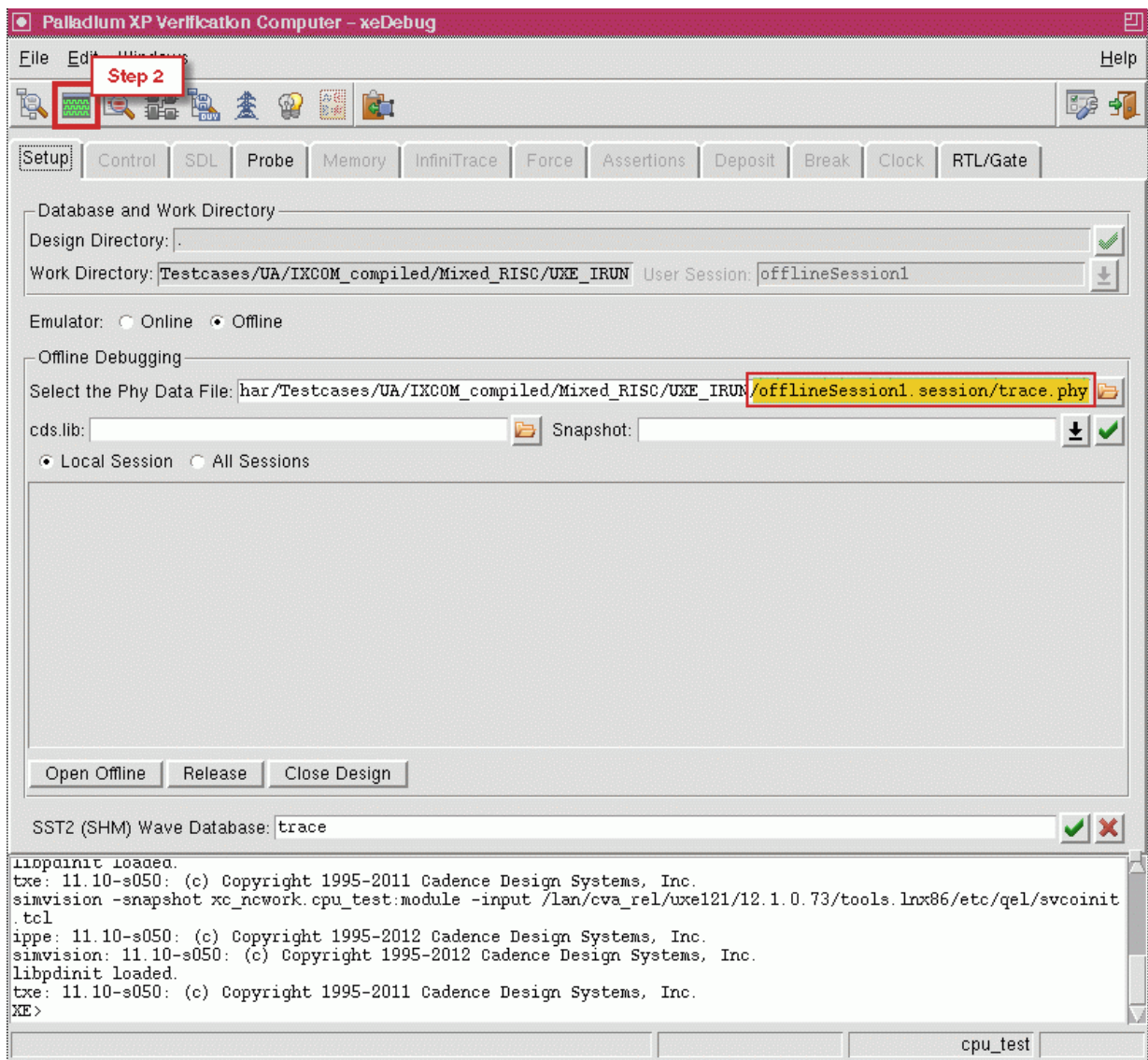
Debugging Designs on Palladium XP

For example:

```
xeDebug -offline . trace.phy
```

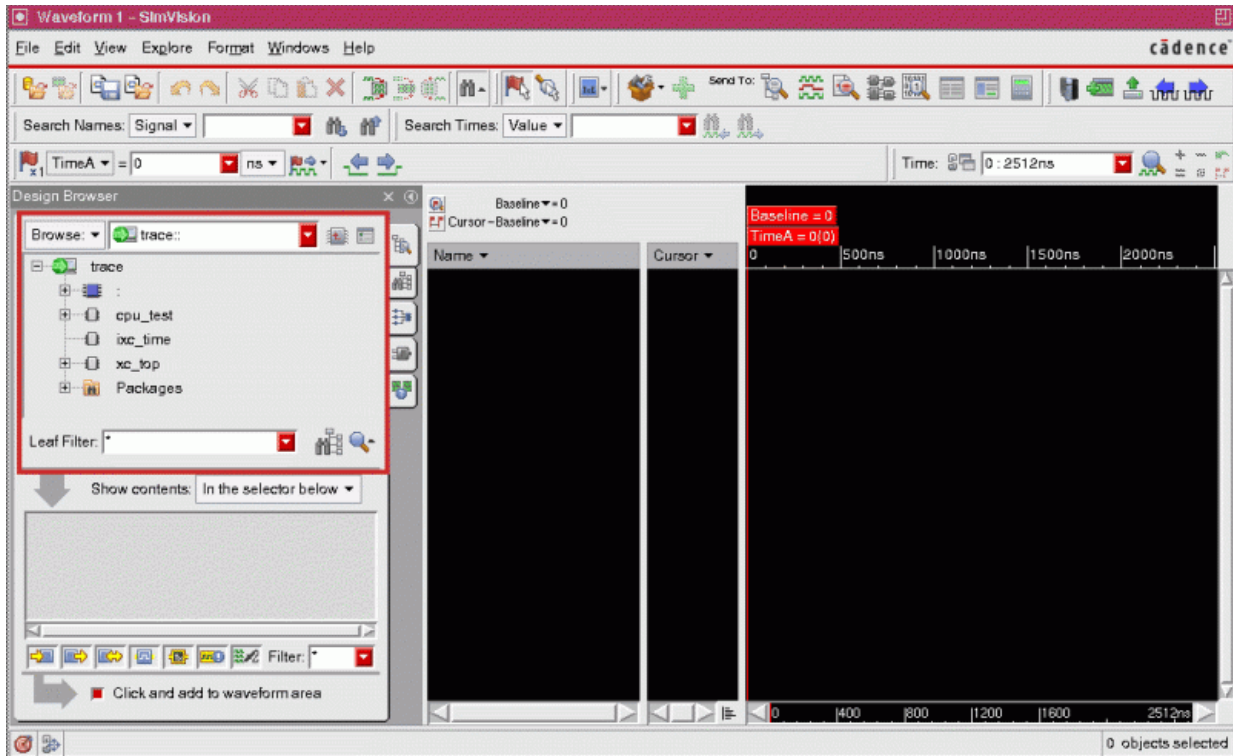
The dot (.) in the above command indicates to use the trace.phy subdirectory from the current design directory while opening the xeDebug GUI window (refer to [Figure 18-3](#)).

Figure 18-3 xeDebug GUI Window - Offline Session



2. Click the SimVision Waveform icon () from the toolbar at the top of the xeDebug GUI window. The SimVision browser illustrated in [Figure 18-4](#) is displayed.

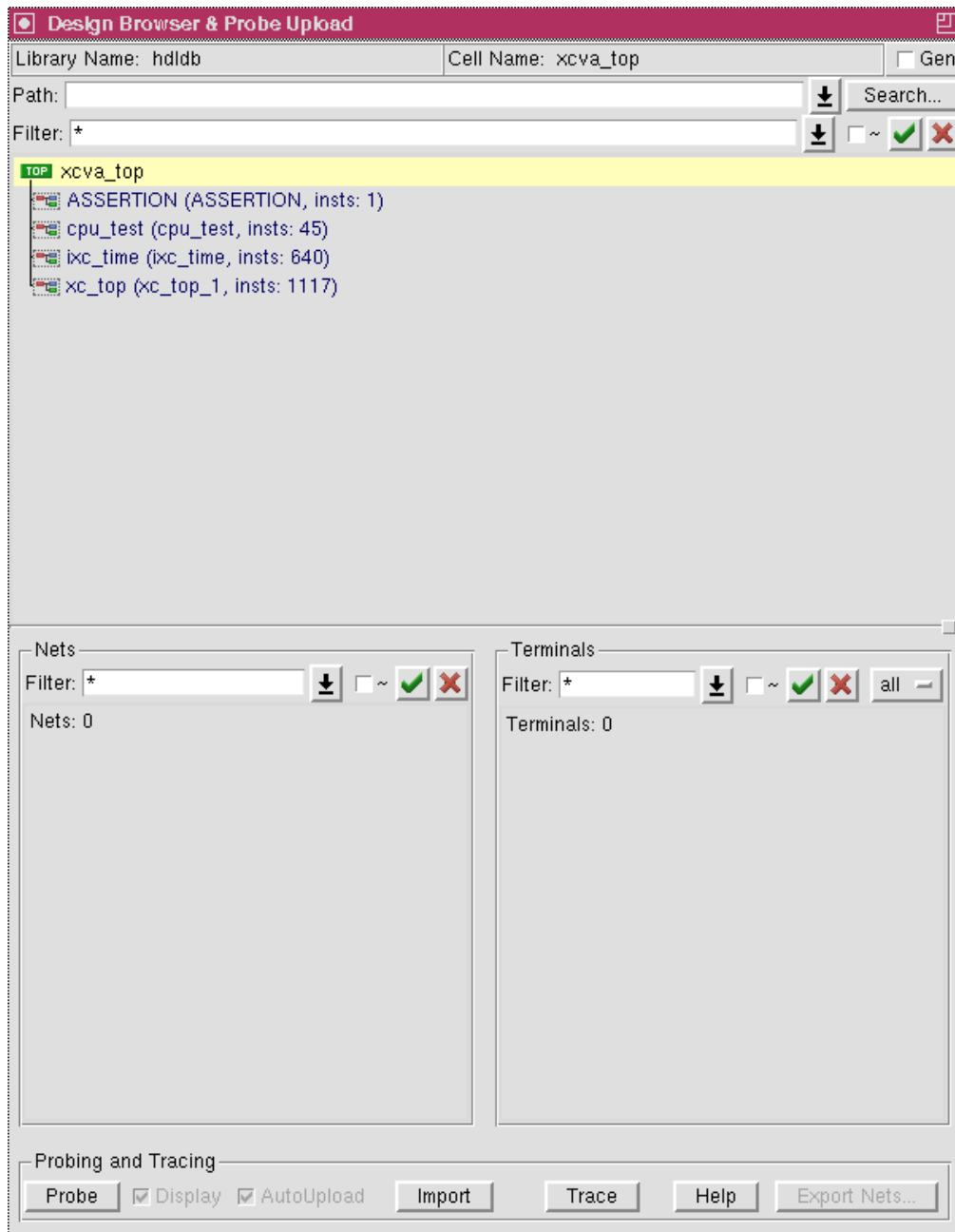
Figure 18-4 SimVision Browser



When this window opens, at first, the name of the file is displayed as *trace* (refer to red highlighted in [Figure 18-4](#)).

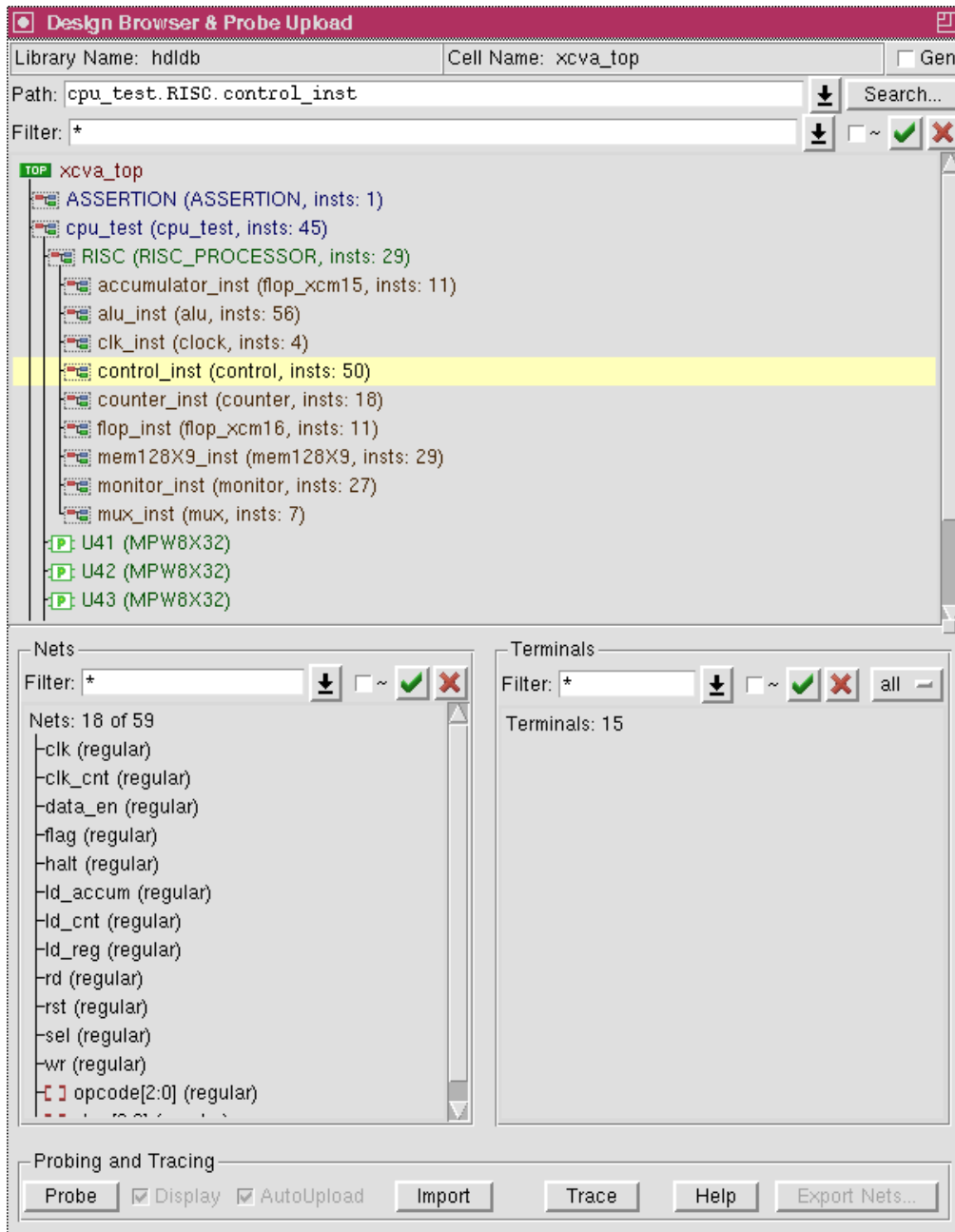
3. Click the *Probe and Display* button () from the toolbar at the top of the SimVision browser. The *Design Browser & Probe Upload* window illustrated in [Figure 18-5](#) is displayed.

Figure 18-5 Design Browser & Probe Upload Window



4. From the design hierarchy, select the required design file. For example, select `xcva_top.cpu_test.RISC.control_inst` as shown in [Figure 18-6](#).

Figure 18-6 Design Browser & Probe Upload Window - Design Hierarchy and Nets



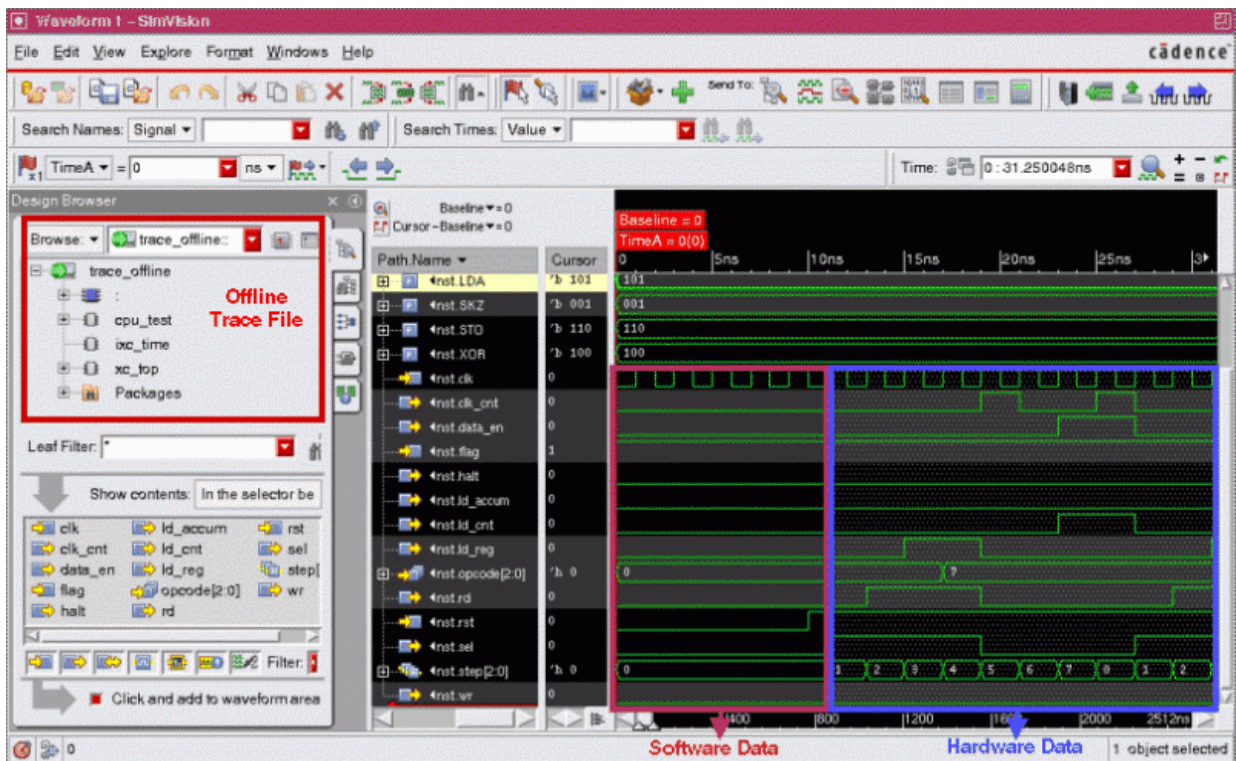
5. Specify the *Filter* criteria in the *Nets* group box and click the  button. The nets that match the specified criteria are displayed within the group box.
6. Click the *Probe* button in the *Probing and Tracing* group box at the bottom of the *Design Browser & Probe Upload* window. After you add the required probes, the database name displayed in the SimVision browser changes from *trace* to *trace_offline*

UXE User Guide

Debugging Designs on Palladium XP

as shown in [Figure 18-7](#). This database provides an integrated view of the design, which means that there is a single design hierarchy (`cpu_test`) displayed for the testbench and all the design components appear under it. Unlike the previous releases, you will not see two separate blocks for the testbench in which only one block displayed the probes.

Figure 18-7 SimVision Browser with Software and Hardware Data Captured during Online Session



After the database name changes to `trace_offline`, you can add more probes in both the testbench and DUT.

Note: Executing the database `-upload` command is necessary after adding DUT probes. The values for the DUT probes are available for both the runs on the software simulator and the emulator. However, the values for the run on the software simulator are available only if you had probed the net during the online session.

Creating the probe by using the following command displays the values during the offline session on the software simulator (with no grey dots in background) if the probes were added before swapping into the emulator during the online session:

```
probe -add cpu_test.RISC.control_inst -all
```

However, if you did not add this probe during the online session, you can see only the values from the hardware run during the offline session.

UXE User Guide

Debugging Designs on Palladium XP

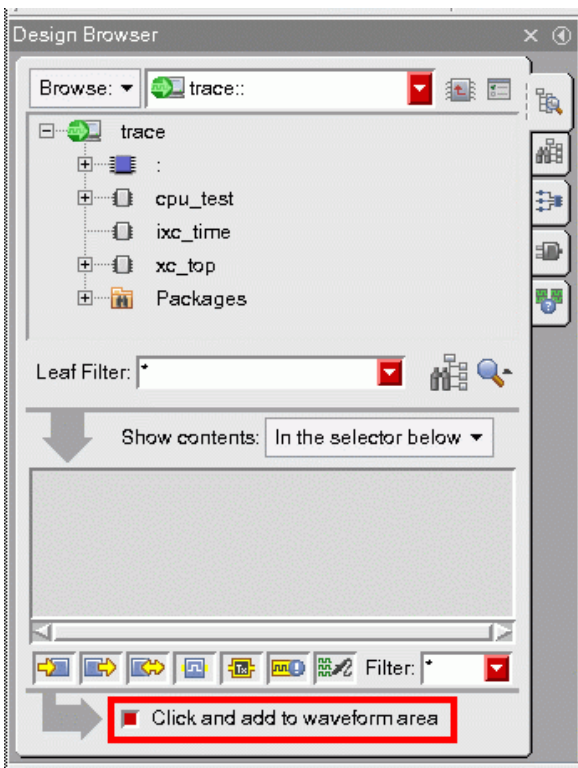
In the offline session, while using the SimVision browser invoked from the xeDebug GUI, you need to click the *Send-to Waveform* button to get the value from the run on the software simulator. Refer to the figure given below.



To get the values from the run on the hardware, you must use the *Probe and Display* button followed by the *Upload DUV Probes* button.

Alternatively, use the buttons available on the *Probe* tab of the xeDebug window to view the values from the hardware run. For more information, refer to the [Probe Tab](#) section of [Chapter F, “Using the xeDebug GUI.”](#)

By using the options in the SimVision browser for adding a signal to the waveform, such as the *Click and add to waveform area* check box at the bottom of the SimVision *Design Browser*, you can get only the values from the run on the software simulator.



Tracing and Uploading Waveforms in SA Mode Using *irun*

Following is an example of a command sequence to generate waveforms with *irun* using interactive command-line input mode:

1. Invoke *irun* from the UNIX/Linux command shell in command-line mode:

```
irun -R -64bit -tcl
```

2. Add simulation probes to be used only by the software simulator using the following command:

```
probe -add -shm cpu_test.RISC.control -depth all
```

3. Run the design using the following command:

```
run 100ns
```

4. Swap the DUT portion of the design to hardware and run the simulation, using the following commands:

```
xc on -xt0 -zt0 -run  
run 1ns
```

5. Set the name and format of the DUT waveform database using the following command:

```
xc sigtrace -outfile -sst2 wave1
```

6. Add DUT emulation probes using the following command:

```
xc sigTrace -addinst -depth 0 cpu_test.RISC.control
```

7. Run the design using the following command:

```
run 100ns
```

8. Upload trace results from the emulator into a trace file using the following command:

```
xc sigTrace -upload
```

Dynamic Swap between Hardware and Software in SA Mode

The presence of the IES simulator enables support for enhanced Hot Swap capabilities in the use models where the design clocks can be stopped. During Hot Swap, the simulator takes full control and the emulator is released. Later, the design can be swapped back to the emulator (as available) and the emulator takes control.

 Important

As a deviation from the traditional SA mode, hot swap is not supported when an ICE target is present. This means that you have to start running the design on the hardware from the beginning of the simulation time.

Using Hot Swap Options

The `xc on -xt0` and `xc on -xt1` commands specify how the signals with simulation values of X, W, -, or U (VHDL `STD_LOGIC` or `STD_ULOGIC`) or X (Verilog nets or registers) are to be handled in either of the following situations:

- Anytime during simulation acceleration modes, if an input to the DUT goes to any of these values, or if an attempt is made to write these values to a DUT signal **or**
- At the time the design is swapped from software mode to simulation acceleration modes, if a VHDL signal or Verilog register signal happens to be at one of these values.

The `xc on -xt0` command sets the hardware to translate their values to 0 if any of these values exist in the preceding situations. DUT inputs and DUT signals being written to with these values will be assigned 0 during simulation acceleration modes. VHDL signals or Verilog registers with any of these values will get 0 values instead when swapped to simulation acceleration modes.

If a signal has simulation value of Z in either of the situations defined above for `xt0` and `xt1`, the `xc on -zt0` command sets the hardware to translate the signal values to 0 and the `xc on -zt1` command translates the values to 1.

There is a limitation while probing signals when using hot-swap. At the time of hot-swap, xeDebug must stop probing or remove the probes located in the DUT and create new probes in hardware. However, you can save probes as Tcl script and restore probes right after swap or any time later.

Figure 18-8 Hot Swap Options on Setup Tab of xeDebug GUI



Using Hot Swap Run Modes

xeDebug tool starts in software mode, and then you can switch to the four available hardware modes. To switch between the available hardware modes, you must first swap back to the software mode by using the `xc stop` or `xc off` commands, and then switch to the new hardware mode.

For running a design in the SA mode, the following run modes are available:

- Software Simulation Mode
- Run Mode
- TBrn Mode
- NBrn Mode
- Autorun Mode
- Match Mode

Before switching to the TBrn, NBrn, or Autorun mode, ensure a clock generator is created in the DUT module. For details, refer to the [Using userData to Create a Clock-Generation Module for SA Mode](#) section of [Appendix G, “Modeling Clocks.”](#)

Software Simulation Mode

In this mode the whole design runs in software; nothing is accelerated with the emulator. This is the default mode, in which the xeDebug tool starts.

If your design contains `initial` blocks in DUT modules, you should run in software simulation until all events generated by those blocks are simulated. If you attempt to swap to simulation acceleration modes while events are still pending, unless you use the `xc on -initok` option, you get an error by default.

Run Mode

The parts of the design that meet certain requirements are accelerated with the emulator; the other parts stay in the software.

To run in this mode use the following command:

```
xc on -run
```

This mode is the general purpose acceleration execution mode. The complete successfully compiled design can be accelerated in this execution mode. Both testbench and DUT events are processed and executed in this mode. Although no event is ignored, the accelerated parallel event simulation algorithm can have different scheduling order than the software and thus have different results. For most designs that are race free and compatible with two-state simulation, the `xc on -run` mode should produce the same result as software simulation.

Alternatively, execute the following command before specifying the `xc on -run` command:

```
xc checkxon
```

The `xc checkxon` and `xc checkxoff` commands can be specified interchangeably to disable or enable X checking. However, by default X checking is disabled.

TBrun Mode

In the TBrun mode, the hardware and software schedulers always block each other. These are synchronized by the nearest delays in each domain. While advancing the hardware time, the software time is blocked until the hardware stops. Likewise, while advancing software time, the hardware is blocked until the nearest software delay elapses. At any point in time, either the hardware or the software can advance time; both cannot advance together. This lock-step mechanism ensures that all the software-to-hardware and hardware-to-software events are transferred immediately as these happen based on the same time wheel. This is called simulation congruency.

The TBrn mode only works for designs with the following restrictions:

- Design compiled with clock generator in DUT (`+iscDelay`) can use the `xc on -tbrun` mode.
- The output events generated from DUT are processed and propagated into the testbench only at the following testbench synchronization events:
 - a. `tbcall` (`tbcall_region`) or import DPI call events.
 - b. Time delay events scheduled by testbench.
 - c. Special output events explicitly defined using `$export_event` compiler directives.

The `xc on -tbrun` mode can run close to peak simulator speed if the testbench synchronization events happen at low frequency and requires low processing overhead.

In `xc on -tbrun` mode, only the next scheduled time event in the testbench module (including SystemC- and PLI-based time events) is considered. Then, fast emulation speed like `xc on -autorun` mode is used to run the hardware from the current time to this next scheduled event.

You can have PI and PO signals in your design, but they are not used to control the hardware execution in `xc on -tbrun` mode. PI and PO signals are *enabled-on-sync*, which means they are enabled whenever *hardware and software synchronize*, for example:

- at `axis_tbcall`
- at a DPI-call
- at scheduled `#delay` events before `xc on -tbrun` or from an `axis_tbcall` or DPI-call task during `xc on -tbrun`.

If you schedule testbench events in very short intervals, then `xc on -tbrun` mode is executed like the `xc on -run` mode automatically. If you schedule testbench events with a long delay, then `xc on -tbrun` provides the same performance as `xc on -autorun`.

In `xc on -tbrun` mode, you can have both `#delay` and mode swap control from the CLI just as in `xc on -run` mode.

To run a design in `xc on -tbrun` mode:

1. Compile the entire design as the DUT, leaving no module out.
2. Use `#delay` statement to model clock generation and use `ixcom switch +iscDelay` to compile the clock generator.

3. Use the `axis_tbcall` primitive/procedure for file I/O operations (`$readmemb`, `$readmemh`, `$display`) or system control functions (`xc stop` and `xc off`).

To stop `xc on -tbrun` mode and return to software simulation mode:

```
ncsim> run 500000
ncsim> xc on -xt0 -tbrun
```

You not only get back to the CLI at time 500000, but you also keep the design in the hardware. You have full control of the system from the CLI just like `xc on -run` mode.

Note:

- Only the hardware output events are transferred to the testbench at testbench-scheduled time events. You must wait for time, not output events, to *wake up* testbench activity. If you want to use a DUT output event to wake up the testbench, you must use `axis_tbcall` or a DPI call for the task.
- Do not use the `xc on -tbrun` command with an embedded software module in the DUT because it requires P/I-P/O events to work. Therefore, you should only use the `xc on -run` command with such modules.

Example

For the following code:

```
initial
do some PI activities with short delay;
#1000000;
do some other PI activities with short delay;
```

If you are using the `xc on -tbrun` command, when there are PI activities, the emulator will run the same way as when the `xc on -run` command is issued. During the long-delay period of #1000000, the emulator will run the same way as when the `xc on -autorun` command is issued.

NBrun Mode

In the NBrun mode, both software and hardware domains advance the time concurrently instead of alternately. Therefore, this run mode is called the non-blocking run mode or NBrun mode. This mode is in complete contrast to the TBrn mode.

After the DUT is swapped into the emulator, the NC-Sim process spawns a hardware control thread, which advances the hardware time immediately until the first TBCall happens. The software testbench also advances the time in parallel after the DUT state is swapped. In the

NBrun mode, the timestamps in the software testbench and hardware DUT are not synchronized and can vary randomly from one session to another. It will not function correctly if the communication between the testbench and DUT is based on relative times. Therefore, the NBrun mode is not simulation congruent.

The software testbench can also send DUT events asynchronously. These asynchronous events force the hardware control thread to stop the hardware DUT time immediately to accept these new DUT events.

Once the hardware time is stopped for TBCalls or asynchronous software events, the hardware waits until the hardware control thread detects that the hardware time has stopped. When this is detected, it will do the following:

1. Advance the software time by a zero-delay delta cycle to update DUT outputs, execute the TBCall service functions, and send DUT inputs.
2. Then, advance the hardware time and software time again in parallel.

You should be careful when writing a multi-thread testbench. A multi-thread testbench might not function as expected in the NBrun mode if a thread is waiting for something to be returned from DUT while another thread has entered a critical section that is supposed to happen only after the first thread has finished.

NBrun versus TBrun

- The NBrun mode is not necessarily faster than the TBrun mode, however, it can reduce the number of synchronizations for the designs in which the software clock is running frequently and faster in real time than the hardware clock. For example, you might have a small software clock generating a sequence of data to be sent to the hardware after filling up a FIFO. There is no need to synchronize hardware at each small software clock edge.
- The NBrun mode is useful when the software has small delay events most of which are not dependent on the DUT. In the TBrun mode, the hardware stops at every software delay event for synchronization. In the NBrun mode, the hardware stops only when the hardware needs service or when the software has to send some data to the hardware.
- The NBrun mode allows emulation and simulation times to proceed concurrently, which means that the simulation times of DUT and testbench are independent and do not synchronize with each other.
- In both the TBrun and NBrun modes, the TBCall service cannot be executed in parallel with the hardware. For example, in the SCE-MI pipe applications, the pipe-message transfer is done in zero-delay delta simulation cycles while holding the hardware time.

The hardware time cannot proceed while the software is filling or emptying the hardware pipe.

Running a Design in NBrun Mode

To run a design in this mode, use the following command:

```
xc on -nbrun <delay>
```

You can specify *<delay>* as a number with or without time units, such as `100ns` or just `100`. When you specify only a number, the minimum precision of the design is used. If you specify the *<delay>* as 0 or a negative number, an error message is generated.

Cadence recommends that you starts from a minimum *<delay>* to get the design functionality correct. Then, gradually increase it to a maximum value to get the best performance in the NBrun mode. To determine the *<delay>*, consider the following guidelines:

- At the minimum, *<delay>* should be the smallest of the following:
 - ❑ The smallest delay in the testbench
 - ❑ The fastest clock delay in DUT because the DUT state will not change within the fastest clock delay
- At the maximum, *<delay>* should not be larger than the time between two consecutive testbench-to-DUT events.

Using xeDebug to Run a Design

You can use the *NBrun* button from the *Setup* tab of the xeDebug GUI to run a design in the NBrun mode.

To specify *<delay>*, use the *nbrun delay* text field, which becomes visible on clicking the --> *Show Hot Swap Options* button from the *Configuration* group box (refer to [Figure 18-8](#)).

You can use the *Save* and *Restart* buttons to save and restart a session in the NBrun mode. When you restart a session, the time at the swap-in needs to be synchronized; thus, the time after the restart is the time in software. You can reset a session in the NBrun mode, which implicitly changes the run mode to software; thus, the time is 0 at reset. Similar to the TBrun mode, in the NBrun mode, you can use other features, such as SDL, dual-clock domain, DYNP mode, and FV mode. However, the hardware waveforms and the SDL time appear according to the time in hardware. For more information, refer to the [Using Setup Tab in SA Mode](#) section of [Chapter F, “Using the xeDebug GUI.”](#)

Viewing Timestamps in NBrun Mode

In the NBrun mode, the timestamps in the software and hardware domains are not synchronized. Therefore, the timestamps displayed in different channels might be different.

The following channels display the software timestamps:

- The time in the run-time log files, such as `irun.log` or `xeDebug.log`
- The time in the command prompt window
- Waveforms for the software signals

The following channels display the hardware timestamps:

- *Current Time* in the *System Status* group box in the *Control* tab of the xeDebug GUI
- Waveforms for the hardware signals including controlled and uncontrolled clocks

Viewing Waveform Database Generated in NBrun Mode

In the NBrun mode, the software and hardware signals cannot be viewed in the same waveform viewer window. You should open separate waveform viewer windows to see the hardware signals and software signals. For example, if `trace.shm` is your database, you need to perform the following steps:

1. In the SimVision *Design Browser* window, open the `trace.shm` database.
2. Select the *Windows* menu of the *Design Browser*.
3. From displayed sub-menu, click *New > Waveform* to open a new waveform viewer window. Repeat this step to open the second waveform viewer window as well.
4. In both the waveform viewer windows, click the *Browse the design hierarchy* tab to display the *Design Browser* pane on the left.
5. In the first waveform viewer window, select *trace* instead of *All Available Data* from the *Browse* drop-down list. From the displayed list of software testbench signals, select the required signals for viewing.
6. In the second waveform viewer window, select *trace_hw* instead of *All Available Data* from the *Browse* drop-down list. From the displayed list of hardware DUT signals, select the required signals for viewing.

Note: When you trace the waveform on hardware in an NBrun session, the waveform shows the hardware timestamps. The hardware might be running faster or slower than the software. If the hardware time is later than the software time, and you swap the design out to software,

UXE User Guide

Debugging Designs on Palladium XP

and swap back to hardware, the waveform that was captured in the first NBrn session will be deleted. This is because the starting time in hardware at the time of the second swap-in needs to be synchronized with the time in software. But it is possible that the time in software is lesser than the ending time of the first NBrn session in hardware. Thus, an error similiar to the following is generated:

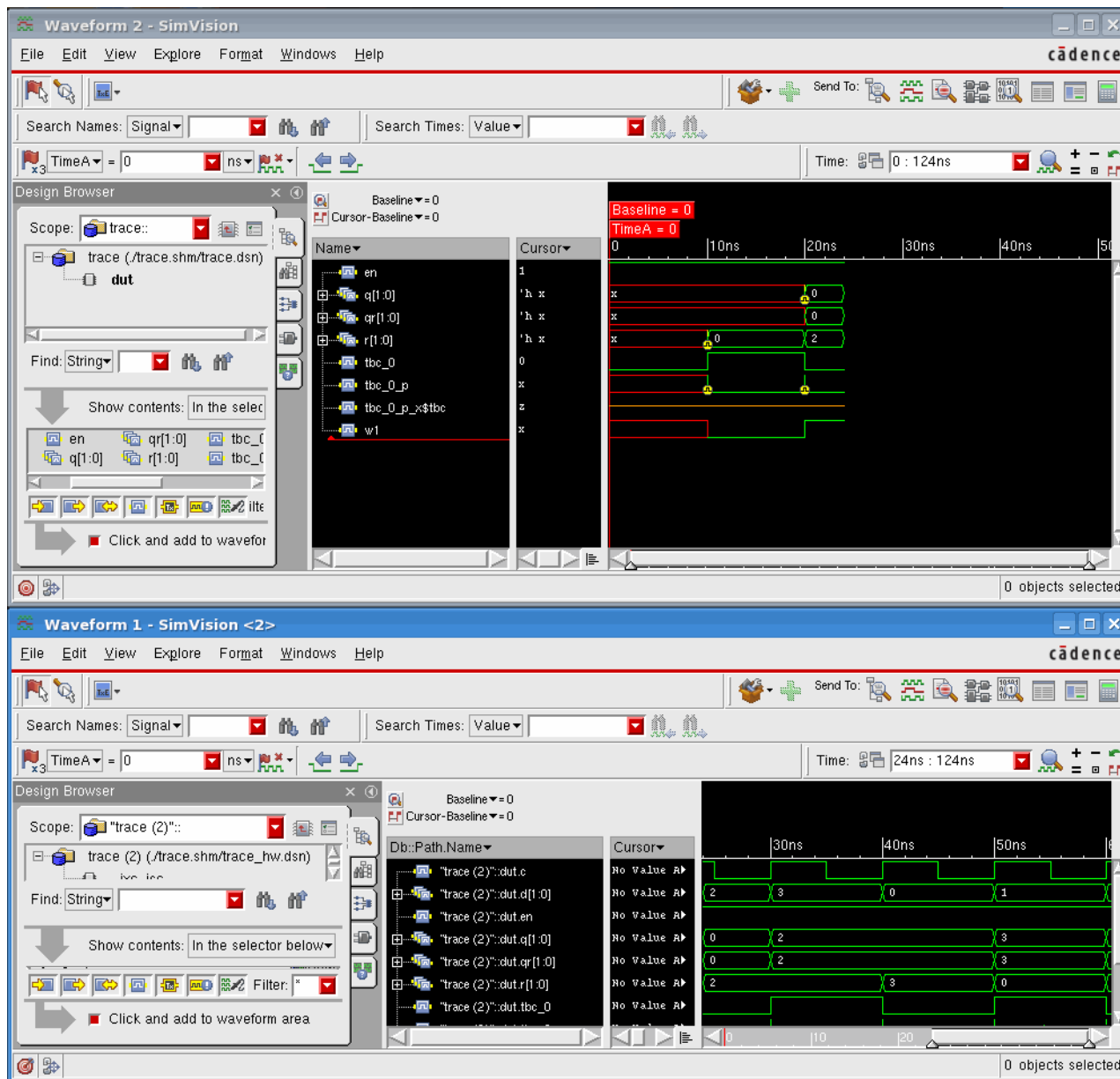
```
ERROR (legacy-44643): upload: start cycle 585124 is larger than end cycle 18000.
```

Refer to [Figure 18-9](#) that illustrates two separate waveform viewer windows.

UXE User Guide

Debugging Designs on Palladium XP

Figure 18-9 Viewing Software and Hardware Signals in NBrn Mode



Limitation

InfiniTrace is not supported in the NBrn mode.

Autorun Mode

The Autorun mode uses the most restricted execution model with the following features:

- Designs compiled with clock generator in DUT (`+iscDelay`) can only use `xc on -autorun mode`.
- Only the `tbcall` (`tbcall_region`) and import DPI calls from DUT are processed in the `xc on -autorun mode`.
- Only non-time consuming `tbcall` and import DPI calls are allowed in the `xc on -autorun mode`.
- All pending testbench time delay events scheduled before entering the `autorun mode` are executed after exiting the `autorun mode`.
- In most cases, the `tbrun` mode has the similar performance as the `autorun mode` but has fewer restrictions than the `autorun mode`, and is therefore recommended for use.
- Only in special cases when you want to ignore all testbench time delay events, such as testbench local generated clocks, `autorun mode` has its special applications for better performance.
- The testbench input events are ignored to get faster speed.
- An error is generated if the design contains any input ports referencing to DUT. Specify `xc on -piok -autorun` command to ignore the input port error. This ensures that `autorun` ignores the DUT input (PI) changes completely.

When using `xeDebug` tool, specify `run` command to advance hardware time in `autorun mode`. Specifying `run` command without a time argument runs the hardware indefinitely. `run 100ns` command advances hardware time by `100ns` and then stops.

When using `irun` tool in `autorun mode`, specify `xrun` command to advance hardware time. Specifying `xrun` command without a time argument runs the hardware indefinitely. `xrun 100ns` command advances hardware time by `100ns` and then stops.

After swapping back to simulator from the `autorun mode`, only the DUT states in IES are uploaded by the emulator. The testbench states remain the same as at the time of swap-in. To get best performance in `autorun mode` use a design which has no testbench and only uses `tbcall` with no delays in the `tbcall` tasks.

For example, if you have a testbench clock rising at every `10ns` and the `autorun` starts at `100ns` and ends at `414ns`; since a rising edge event is scheduled to execute at `+10ns` when `autorun` starts the pending rising edge event takes place at `424ns` which is `10ns` after `autorun` stops at `414ns`.

Running a Design in Autorun Mode

To run a design in autorun mode, perform the following steps:

1. Compile the entire design as the DUT, leaving no module out.
2. Use `#delay` statement to model clock generation and use `ixcom switch +iscDelay` to compile the clock generator.
3. Use the `axis_tbcall` primitive/procedure for file I/O operations (`$readmemb`, `$readmemh`, `$display`) or system control functions (`xc stop` and `xc off`).

Stopping a Design Running in Autorun Mode

Stop a design running in autorun mode by using the following commands:

1. Use `Ctrl-C` from the CLI to interrupt the operation.
2. Add `$stop` and `$ixc_ctrl (tb_import, $stop)` in a DUT module. The simulator stops when hardware executes the `$stop` statement.

Note: To ignore all PI events during autorun, use the `xc on -pio -autorun` command.

Match Mode

When an unexpected behavior is noticed while running in the emulator, run in match mode to debug the issue.

In the Match mode both simulation and emulation run concurrently and the DUT values from the emulator model are checked at each time step to make sure that they match with the simulator DUT model.

A typical reason for mismatch can be a race condition in your RTL code, and the solution can be to rewrite the source code to avoid this race condition. If there is a mismatch while running in match mode, an error message is generated and printed out and the run is stopped.

For example:

```
sample error message:Error: PO top.mem_address(0) mismatch hw = 1, sw = 0 at 230
NS Memory Usage - 20.2M program + 296.0M data = 316.2M total CPU Usage - 0.1s system
+ 0.3s user = 0.4s total (0.1% cpu) Simulation stopped via $stop(2) at time 230 NS
+ 7
```

Accessing Signal Values

You can force, release, deposit values to the design objects.

❑ **Forcing and Releasing Nets:**

You can force and release values of the nets in the design. The value can be entered using Verilog notation. You can also save current list of forced nets and restore forced nets from a file. There is no need to specify whether the net is located in the testbench or DUT.

❑ **Depositing Values to Design Objects:**

When running in SA mode, the values can be deposited to testbench objects with the following options:

- With respect to absolute/relative time
- Repeat/cancel time period
- Release force prior to deposit
- With inertial, transport or no delay

Values can be deposited to DUT objects with optionally releasing force prior to depositing values. You can deposit values to the nets and memories in the design.

This section provides information on forcing, releasing, and depositing values to design objects. For details of the commands mentioned in this section, refer to the *System Tasks and Procedures* section in the *Compile-time Commands in SA Mode* chapter in the *UXE Command Reference Manual*. The steps and concepts for these are described in [System Task Mapping in IXCOM flow](#) on page 265.

Forcing Values

You can set a given value for specific design objects, and force them to retain it until explicitly released. For information on releasing the forced values from objects, refer to the [Releasing Values](#) section.

The design objects to which values can be forced include signals, symbols, and buses. A signal can be forced only if it is specified as a `keepNet` at the compile time.

The method for forcing values varies based on the run-time tool being used, that is, `xeDebug` or `irun`. The following sub-sections discuss this in detail.

Note: If you force a partial vector that is mapped to the hardware in the IES simulator, the whole vector is forced. For example, if the current value of a vector is 'b1101 and the `bit[0]` is forced to 0, then the vector is forced with 'b1100.

Forcing Values Using the xeDebug Tool

In both ICE and IXCOM flows, execute the following XEL command to force a value:

```
force <object_name> <value>
```

When a signal represents a single bit, *<value>* is either 0 or 1. When forcing a value to a vector that represents several bits, the *<value>* can be specified as a binary, octal, decimal, or hexadecimal radix, prefixed by the appropriate letter following Verilog syntax. For example, 'b011, 'o1237, 'd089, 'hffcd. A number without a radix prefix is assumed to be decimal, for example 89 is the same as 'd89.

Refer to [“Force Tab”](#) on page 1236 for details about using the xeDebug GUI for forcing signals.

Forcing Values Using the irun Tool

When using the *irun* run-time tool in SA mode, executing the `force` XEL command does not access the hardware correctly. Therefore, follow either of the steps given below before you proceed:

- Modify the HDL design to include the `$export_frcrel` system task on the target signals and then re-compile the design.

OR

- At run time, to dynamically export the target signals to force the values, execute the following command:

```
xc export_frcrel <signal>
```

Note: The `xc export_frcrel` command might not work for certain DUT signals that have been optimized by the compiler.

These steps create a special force logic for the specified signals in the hardware and cause these signals to be exported from the emulator, which is required if you wish to force and release the signal in the SA mode. After executing either of the above-mentioned steps, you can use the `force <signal> <value>` command to force values to the DUT signals. Else, the `force` command will work only for the testbench objects.

Releasing Values

A signal retains a forced value until it is explicitly released. The method for releasing a previous force varies based on the run-time tool being used, that is, xeDebug or *irun*. The following sub-sections discuss this in detail.

Releasing Values Using the xeDebug Tool

To release any previous force (that was executed using the `force` command) on the specified signal or symbol, execute the following command:

```
release {<signal> ... | <symbol> ...}
```

Consequently, the value of the specified signal is recalculated at the next cycle. However, the sourceless nets that were declared as `keepNets` will retain their values (state).

Note: If the `release` command is operated on a bus in LA mode while the clocks are running, time coherency between the values of the signals in the bus is not guaranteed.

When using the xeDebug GUI, the *Release* button on the *Force* tab enables you to release individual signals or nets (See [Figure F-46](#) on page 1238).

Releasing Values Using the *irun* Tool

Similar to forcing values, follow either of the two steps discussed in the [Forcing Values Using the *irun* Tool](#) section to correctly access the hardware for releasing values.

Note: The `xc export_frcrel` command might not work for certain DUT signals that have been optimized by the compiler.

Depositing Values

You can set the state of a signal, symbol, bus, or memory word to the specified value. For a net type object (not a memory) that exists in the emulator, the deposit operation can be done reliably only if the object had been specified as a `keepNet` during compile. The deposit operation will force the value of the object to the specified value, and then release it. The number of cycles through which the value remains forced on the object depends on the operation mode.

The method for depositing values varies based on the run-time tool being used, that is, xeDebug or *irun*. The following sub-sections discuss this in detail. Refer to [Deposit Tab](#) on page 1241 for the usage of xeDebug GUI for depositing values.

Depositing Values Using the xeDebug Tool

In both ICE and SA modes, execute the following XEL command to deposit a value:

```
deposit <object_name> <value>
```

When a signal represents a single bit, *<value>* is either 0 or 1. When forcing a value to a vector that represents several bits, the *<value>* can be specified as a binary, octal, decimal, or hexadecimal radix, prefixed by the appropriate letter following Verilog syntax. For example, 'b011, 'o1237, 'd089, 'hffcd. A number without a radix prefix is assumed to be decimal, for example 89 is the same as 'd89.

In the SA mode, to release an active force and allow to deposit a specified value using the `deposit` command, execute the following command:

```
deposit <object_name> <value> -release
```

In the ICE mode (STB, LA, and VD modes), the `-release` option is superfluous because a previous force is always released before applying the deposit.

For testbench objects in the SA mode, following additional options of the `deposit` command can be used:

```
deposit <object_name> [=] <value>
    [-after <time_spec> [value -after <time_spec> ...]]
    [{-relative | -absolute}]
    [-generic]
    [-inertial]
    [-release]
    [-repeat period [-cancel period]]
    [-cancel 0]
    [-transport]
```

For syntax details, refer to the *NC-Verilog Simulator Help* in the INCISIV documentation set.

Depositing Values using the irun Tool

When using the *irun* run-time tool in SA mode, execute the following command:

```
xc deposit <signal> <value>
```

This command deposits a value to a DUT signal. It does not work for a testbench signal. The DUT signal does not have to be pre-compiled with `$export_deposit`. If the DUT signal is

not pre-compiled with `$export_deposit` and it is optimized by the compiler, the `xc deposit` command will report an error.

Using the Palladium Xref Browser of SimVision GUI in SA Mode

When you compile the designs for SA mode, the IXCOC compiler generates an instrumented snapshot for the IES simulator, NC-Sim. At the time of debugging, this snapshot is used, and it appears to be quite different from the original RTL design sources in the Source browser windows of the SimVision GUI. So, you might experience some difficulties while watching active annotation during simulation or setting breakpoints.

To resolve these issues, you can choose to use a side-by-side browsing tool, called the *Palladium Xref Browser*, which consists of essentially two source browsers that cross-link the instrumented snapshot back to the corresponding lines in the original design files.

To enable access to this browser, you need to include the `+xref_browser` option of the `ixcom` command at the time of compiling the design. When you include this option, the IXCOC compiler ensures that the mapping information from the original source browser to the instrumented sources is automatically generated. When you invoke the *Palladium Xref Browser*, it searches for the mapping file and returns an error if the file is not found.

This browser is available whether you invoke `xeDebug` independently by using the `xeDebug -gui` UNIX command, or by using the `irun -xeDebug -gui` command. It is provided in the SimVision GUI under the *New* menu option. It supports both online and offline sessions. This browser is available **only in the SA mode** from the SimVision GUI, and allows only object-level and file-level cross referencing.

At a time, you can open only one instance of the *Palladium Xref Browser*. When you invoke the *Palladium Xref Browser*, two source browsers open side by side, and display the current simulation point. These source browsers are:

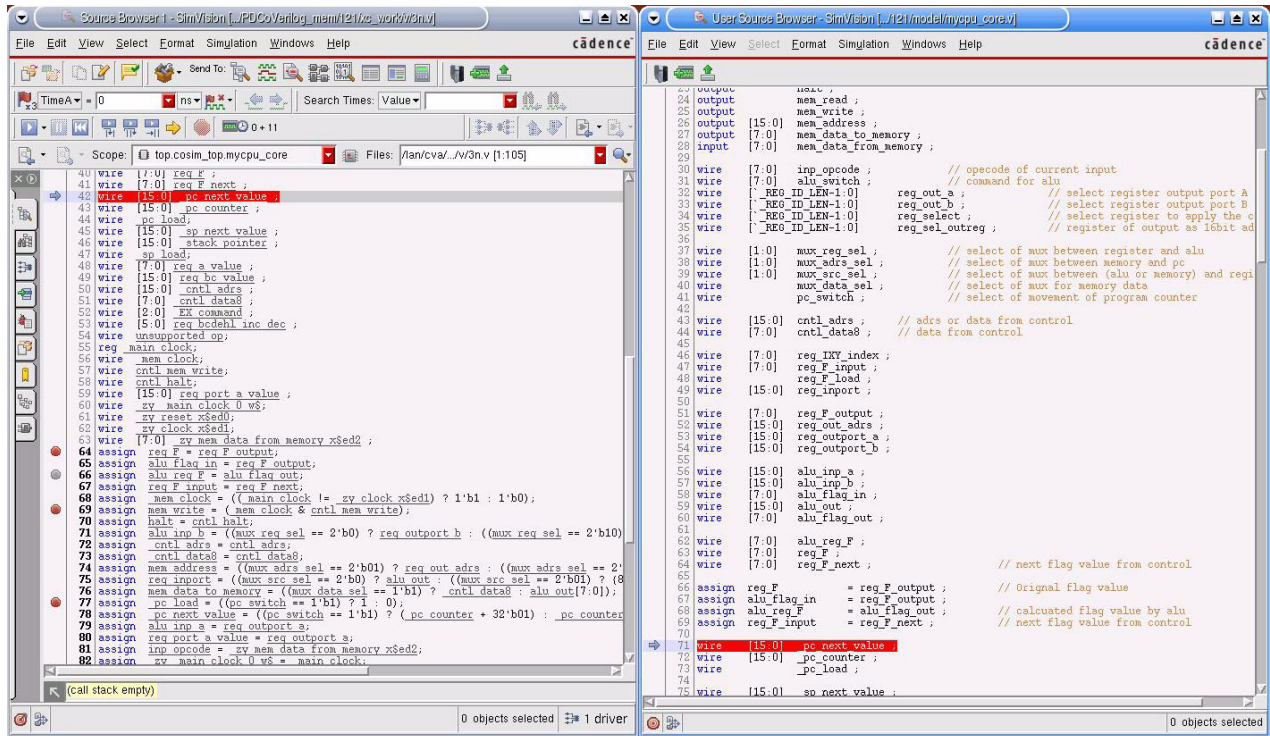
- The source browser in which the original user sources are displayed. It is titled as *User Source Browser*.
- The instrumented source browser in which the instrumented sources are displayed. This is the normal source browser window, which is titled *Source Browser #*. The `#` suffix in the source browser window title indicates the number of source browser windows that are currently open, for example *Source Browser 1*.

Figure 18-10 illustrates the two source browser windows of the *Palladium XRef Browser*. The instrumented source browser is displayed on the left and the *User Source Browser* on the right. In the following figure, the corresponding assertion is highlighted in both browsers.

UXE User Guide

Debugging Designs on Palladium XP

Figure 18-10 Palladium Xref Browser



Using the Source Browsers of the Palladium Xref Browser

Both windows in the *Palladium Xref Browser* are standard SimVision source browser windows and, therefore, have the standard menus: *File*, *Edit*, *View*, *Select* (disabled in the *User Source Browser*), *Format*, *Simulation*, *Windows*, and *Help*. The user source browser has neither the *Palladium action* toolbar, nor the *Scope* selector.

The *Palladium Xref Browser* automatically starts a processing loop to detect the changes in the instrumented source files, as these occur, either as a result of simulation advancing, or because of user interaction, such as changing the scope. If the instrumented source browser is blank, the *User Source Browser* is blank. You can initialize the instrumented browser by selecting an object in the design browser.

If the *User Source Browser* is closed, the *Palladium Xref Browser* feature terminates, but the instrumented source browser remains open. If the instrumented source browser is closed, the *Palladium Xref Browser* feature terminates, but the *User Source Browser* remains open. To re-enable the *Palladium Xref Browser*, re-invoke it from the SimVision pull-down menu; this will re-open its closed browsers.

You can manually open a different file using the menus, or change the scope by using the scope dropdown box. Changes in the instrumented source browser scope or file affect the *User Source Browser*. But if no mapping information available, there is no change in the *User Source Browser*.

In the *Palladium Xref Browser*, the objects, for example signals, can be highlighted in either of the browsers. Simultaneously, the corresponding objects, are highlighted in the other browser. However, you can perform object-manipulation operations, like probing, forcing, or value annotation only in the instrumented source browser. Selecting objects in the other SimVision browsers, even another Design Browser has no effect on the *Palladium Xref Browser*.

Offline Debugging

This feature is available when you start an offline session of xeDebug, that is, without the simulator. During offline debugging, you do not have access to simulator-related features, such as breakpoints and pointer advancing. However, active annotation from the waveform in the instrumented source browser is available.

Assertions

Assertion failures also display the relevant assertions in the original source code. You can invoke the SimVision Assertion browser to display the failed assertion, and select the assertion to be sent to the SimVision source browser. Then, you can invoke the *Palladium Xref Browser* to display the original sources.

Limitations

The *Palladium Xref Browser* does not support the following functionality:

- ❑ Mapping original user design objects to the equivalent instrumented design objects to be used for probing and other standard functions is not supported. In the source browser, you can highlight strings in the text window, but you need to rely on mapping data, including column information, to find the corresponding instrumented object. Automated probing of user design objects is not supported.
- ❑ The IXCOC compiler does not provide line-to-line mapping information to allow correspondence points within the file other than objects. So, breakpoints in user sources and live position updates are not possible.
- ❑ Certain standard SimVision operations work orthogonally to this feature. So, the reset, restart, and save operations do not disable the *Palladium Xref Browser*.

Debugging in Offline Session

Offline debugging can be performed only in FullVision mode, and not in DYNP mode. This functionality allows you to run the design and save the trace results for later debugging without needing to run an emulator session. This frees the emulator for the use of others.

To perform offline debugging, you must first run an online debug session to generate the trace data. Then, release the emulator, open the trace data in offline session, and generate waveforms for the additional probes.

In the offline debug mode, you only have limited functionality available. You can add probes and examine signals in the waveform viewer, but cannot actually debug the design using other debug features, such as inspecting memory contents, as you would while running the design on the emulator in online session.

You can specify a new waveform database to save the existing waveform file. If a new waveform database is not specified, the existing waveform file is overwritten. Before bringing up the waveform viewer, you must add (or import) probe signals and upload the trace data in the xeDebug tool. You can then use the *File* menu of the waveform viewer to view a previously generated waveform file.

Generating Trace Data and Preparing for Offline Session

In an online session, open a database using the following command to record the trace data:

```
database -open <database_name>
```

After running the design for the desired number of cycles, use the `database -prepareOffline` command to generate the trace data for offline debugging. Execution of this command writes the traced data to an output subdirectory named as `<database_name>.phy` without generating the SST2 and FSDB waveform file. You can generate multiple trace output subdirectories for different offline debug sessions. For example, results for different test runs can be captured in subdirectories named `trace1.phy`, `trace2.phy`, and `trace3.phy`.

You can then use the trace data from the `<database_name>.phy` subdirectory during an offline session started through xeDebug GUI or command-line interface.

The `database -prepareOffline` command generates the `*.phy` subdirectory for the time window ending at current cycle. The size of the time window can be specified by using the `xeset traceMemSize` command.

Preparing for offline session helps running the design faster and you spend less time on the emulator. However, the `*.phy` subdirectory needs a preprocessing step in order to make

offline debug session run faster. The data saved in *.phy subdirectory is organized into blocks. Each block is about 2000 cycles. Each block has a flag to indicate whether it has been pre-processed. When a block is first accessed in offline session, it needs to carry out a pre-process step, therefore waveform generation is slower. But, when the second time the block is accessed, the block has already been pre-processed so, the waveform generation is faster. For example, the first time to create waveform for time 0 to 2000 is slower because of the pre-processing. But, if more signals are added in the same time window, the waveform generation is faster.

Optimizing Trace Data for Faster Interactive Offline Debugging

To achieve faster interactive offline debugging, pre-processing on the *.phy subdirectory created in online session is required. Pre-processing computes waveforms for all the physical probes and saves the waveform data back to *.phy subdirectory.

Launching the fvCompute process starts pre-processing. Specify the following command before database -prepareoffline command to automatically launch fvCompute process in background after prepare offline is carried out:

```
xeset fvPreprocess on
```

Alternatively, fvCompute process can be launched from UNIX command line after creating the *.phy subdirectory in online session using the following command at UNIX prompt:

```
fvCompute <base_name_of_phy> [-nice <n>]
```

User can specify a nice value to assign priority of the fvCompute process. The default nice value is 20 meaning lowest priority. For more information on specifying the nice value, refer to UNIX help for details.

Pre-processing on the *.phy subdirectory should be performed in offline session. However, performance is slow the first time the waveform is created. An offline session can be opened when fvCompute is still running, and multiple fvCompute processes can pre-process the *.phy subdirectory to save time.

Starting an Offline Session

When using the xeDebug command-line interface, use the following command to open the different trace output subdirectories in offline session:

```
host -offline <*.phy>
```

Then, you can use the probe -create command to add more probes to debug the design and the database -upload command to upload the trace data to generate waveforms.

Debugging in Offline Session to Save Emulator Time

Using FV and InfiniTrace, you can view waveforms of any signals in the design over a longer time window. Cadence recommends to run an InfiniTrace *Prepare* session in online session and then perform interactive waveform generation in offline session, so that the emulator is free. Avoid generating waveforms in online session to reduce the time needed to occupy emulator.

Cadence recommends to view waveforms for a limited time window and limited set of signals so that the waveform can be generated within a few seconds. You can then switch to a different set of signals or a different time window. For example, the *.phy subdirectory contains waveform data for a large time window, for example a few million cycles. You can generate waveforms for smaller time windows, such as a few thousand cycles. Use the `database -tracewindow` command to specify a time window to generate the waveform.

Using Offline Debugging

The emulator is required for generating the *.phy subdirectory in the online session. After the *.phy subdirectory is created switch to the offline debug session. In offline session, use the following commands to select a small time window and observe waveform for small set of probes:

1. Specify the following command to start an offline session:

```
host -offline <*.phy>
```

2. Set a small trace window using the `database -tracewindow` command.

For example:

```
database -tracewindow -start 6000 -end 10000
```

3. Generate the waveform using the `database -upload` command. For example, the waveform from 6,000 to 10,000 cycles is generated.

You can also add and upload more probes. If you do not use the `database -tracewindow` command to specify a smaller time window, the waveform is generated for the whole time range saved in the *.phy subdirectory.

Additional Debugging Features in Offline Session

The following debugging features can be used in offline session:

- [Setting Trigger Points Using SDL Simulator with InfiniTrace](#)
- [Analyzing Power Consumption Using DPA](#)

Setting Trigger Points Using SDL Simulator with InfiniTrace

SDL simulator can be used when tracing signals using FV and InfiniTrace. A SWIT session is created for a long time window during online session. SDL simulator can be run on the complete time window of the SWIT session file to identify a trigger point in the time window. The SDL simulator uses FV engine to get the signal values that are used in SDL program.

If pre-processing has not been performed for the huge SWIT session, to get signal values required by SDL simulator, FV engine needs to be invoked to find the signal values. But, that is a time-consuming process.

When running SDL simulator in a SWIT session, there is no need to invoke FV engine.

Search a trigger point in *Observe* session using the following command:

```
infiniTrace -goto trigger
```

In HWIT, SDL logic running inside emulator searches the trigger point. Though in SWIT *Observe* session a software module of SDL simulator searches the trigger point in offline debug mode.

You can use the `sdl` command to set up a trigger condition and check the trigger status in a similar way as you do in the online session.

Analyzing Power Consumption Using DPA

Normal toggle count (NTC) generation and weighted toggle count (WTC) generation can be performed in offline session, thus saving the time spent on the emulator.

Set the size of the time window in which NTC/WTC are generated using the following command:

```
dpa -tracewindow -start <start_time> -end <end_time>
```

Alternatively, the following command can also be used:

```
dpa -tracewindow -start <start_time> -size <size_of_time_window>
```

If a time window is not specified, the DPA operation is performed on the whole time range of the *.phy.

Use `dpa -setupToggleCount` command to specify the set of instances for NTC or WTC.
Use `dpa -getToggleCount` command to get NTC or WTC.

Sparse Sampling

In the offline session, the sparse data in the *.phy subdirectory can be used to generate NTC, WTC and TCF. The waveform too can be generated, but since the waveform is also sparse its not very useful.

Cadence recommends to perform sparse sampling in online session. Specify a sample rate before running the emulation, so that when the trace memory is uploaded, the data saved in the *.phy subdirectory is already sparse.

Examples of Using Offline Session

The following three example scenarios show running under the xeDebug script mode or GUI interface mode. The first example shows a normal run in online debug session to prepare trace data for an offline session. The second shows offline debug session using xeDebug commands in script mode. The third describes an offline debug session using xeDebug GUI interface.

Example 1: Online debug session to prepare trace data for offline debugging

1. Specify the directory path to the design database using the following command:

```
debug <dbPath>
```

For example:

```
debug .
```

2. Specify the emulation host name using the following command:

```
host <hostname>
```

3. Download the design to the hardware using the following command:

```
download
```

4. You can choose the operation modes to be used while debugging the designs. Specify the following command to select the debug mode:

```
configPM -la|-vd|-stb
```

For example:

```
configPM -stb
```

5. Create a new database using the following command:

```
database [-open] <database_name>
```

For example:

```
database -open mytrace
```

6. Run the design using the following command:

```
run
```

Specify the emulation cycles for which the design needs to be run, for example:

```
run 20000
```

7. Save data for offline session in a *.phy subdirectory using the following command:

```
database -prepareOffline
```

This option is supported in the FullVision mode, but not in DYNP (Dynamic Probes) mode.

8. Exit the debug session using the following command:

```
exit
```

Example 2: Offline debug session using xeDebug command-line interface

1. Specify the directory path to the design database using the following command:

```
debug <dbPath>
```

For example:

```
debug .
```

2. Specify the name of the database or the *.phy subdirectory that will be used in the offline session using the following command:

```
host -offline {<*.phy_database_name>}
```

For example:

```
host -offline mytrace.phy
```

3. Create a new database using the following command:

```
database [-open] <database_name>
```

For example:

```
database -open newwave
```

4. Create a probe during run time using the following command:

```
probe -create
```

For example:

```
probe -create -ports .
```

5. Specify time window to generate the waveform:

```
database -tracewindow -start <time> [-end <time> | -size <size_of_window>]
```

For example:

```
database -tracewindow -start 1000 -end 2000 or  
database -tracewindow -start 1000 -size 1000
```

6. Upload the trace results from the emulator into the SHM or FSDB waveform database, so that it can be used by the waveform browser using the following command:

```
database -upload
```

7. Exit the debug session using the following command:

```
exit
```

Example 3: Offline debug session using xeDebug GUI

1. Open design in xeDebug GUI.
2. Select the *Offline* radio button.
3. Select a `<database_name>.phy` trace output subdirectory to be used.
4. Click *Open Offline* button to start the offline debug session.
5. Add and display probes using the Design Browser tool.
6. Exit the offline debug session.

Files Generated during Debugging

Here are the log and message files that the xeDebug tool generates during the debug process.

xeDebug.log

When a debugging session is started, xeDebug makes a main log named `xeDebug.log` in the design directory. This log file contains an exact copy of your terminal or console window. This file is incremental, that is, multiple debug sessions get sequentially recorded within the `xeDebug.log` file.

xe.msg

A file named `xe.msg` is also generated which supports recursive logging, that is, nested command calls are also logged in the file. The `xe.msg` file contains a complete log of all previous compile and run-time sessions. This file includes the output displayed on the screen (same as `xeDebug.log`), as well as additional messages that otherwise are not recorded in the console log file.

The `xe.msg` file is a superset of `xeDebug.log`. It is also incremental like the `xeDebug.log` file. Therefore, many commands are logged twice; first as an echo to the screen and second from inside the commands. Echoes are shown with a prompt in upper case, like on the screen, and logging from inside commands are shown with the prompt in lower case.

xeDebug.key

The `xeDebug` tool generates another log file named `xeDebug.key` that contains all the commands that you have typed or GUI has generated. It is a record of the debug session, and if sourced, produces exactly the same results as the original session with a few exceptions.

- ❑ The first exception is for time-sensitive features. For example, consider executing the `run -nowait` command followed by delay of one second, and then execute the `stop` command. The result in this case depends on the performance of the emulator, host, and host load.
- ❑ The second exception is `Ctrl+c` that you have entered. If you start a blocking run and then press `Ctrl+c`, it cannot be recorded correctly.

There could be other reasons for the replayed session to produce different results. For example, if during the debug session you modify contents of files that are used by the debugger, such as memory initialization files or SDL files, then the replayed session will only use the latest version of the files, which might not be the same as the contents used in the original debug session.

Run-Time Command Support in ICE and SA Modes

This section provides information about the XEL run-time commands supported or not supported in the ICE and SA modes.

For syntactical details of the listed commands, refer to the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

Table 18-6 XEL Run-Time Commands Support Matrix

Run-Time Commands	ICE Mode	SA Mode
ATB commands: hdlConfig hdlRunLogfile	Supported	ATB feature is not supported.
clockConfig	Supported	This command is typically not supported in the SA mode because of the differences in the modeling of the clocks. However, some options of the <code>clockConfig</code> command are partially supported if there are uncontrolled clocks.
configPM	Use switches: <code>-stb</code> and <code>-la</code>	Different switch names need to be used: ■ <code>-staticTarget</code> instead of <code>-stb</code> ■ <code>-dynamicTarget</code> instead of <code>-la</code>
lp	Supported	Supported when using the <code>xeDebug</code> tool and not supported in the <code>irun</code> tool.
deposit	Supported	The <code>-allff</code> option is not supported. Subtle difference in behavior by default, when compared to ICE mode. The behavior in ICE mode can be configured to be the same as SA mode by setting the <code>depositNCsimMode</code> run-time parameter.
download	Supported	Not supported. Instead, execute the <code>xc xt0 zt0 autorun</code> command.
force	Supported	The <code>-allff</code> option is not supported.

UXE User Guide

Debugging Designs on Palladium XP

getCycleNum	Supported	This command is supported, but the result might be different from ICE mode due to difference in modeling.
getTime	Supported	The <code>dclk</code> and <code>fclk</code> units of time are not supported
memory	Supported	Some command options not supported.
release	Supported	The <code>-allff</code> option is not supported.
resetCycleNum	Supported	Not supported
resultWaveform	Supported	Not supported
resume	Supported	Not supported. Use the swap-in feature instead.
run	Supported	Supported except: ■ Amount to run cannot be specified using the number of clock cycles (only accepts simulation time).
suspend	Supported	Not Supported. Use the swap-out feature instead.

Running Designs with Uncontrolled Clocks Using xeDebug in SA Mode

At the time of compiling designs for SA mode using the IXCOS compiler, you can use a free-running or uncontrolled clock-generator module that can be enabled or disabled independently of the controlled clock of the simulator for the purpose of driving a dynamic target. By default, this is a free-running clock, but the system provides ways to stop it.

Refer to sections [Synchronizing the Fastest Design Clock with Emulator FCLK](#) on page 1301 and [Using userData to Create a Clock-Generation Module for SA Mode](#) on page 1305 for a description of how you can model uncontrolled clocks in SA mode manually and automatically by using the *ixclkgen* utility, respectively.

This section describes how you can run these designs in SA mode with uncontrolled clocks while using the xeDebug tool, and control the data capture and display at run time.

The features that are described in this section are supported only with the `xeDebug` and `irun -xeDebug` commands; the `irun` command without the `-xeDebug` option is not supported for uncontrolled clocks.

Capture and Display Control

The `-captureMode` option of the `database` command controls how the waveform display behaves with a design having uncontrolled clocks:

```
database -open <name> -captureMode [controlled | uncontrolled] [-event]
```

It is not possible to change capture mode on a database that is already open. This option can only be specified while the design is swapped-in to the hardware. It is applicable only for designs with uncontrolled clocks.

Refer to the description of the `-captureMode` option of the `database` command in the *Run-time Commands* chapter of the *UXE Command Reference Manual*.

Table 18-7 Effect of the `-captureMode` and `-event` Options

Options	Time Stamp	Sampling
<code>-captureMode controlled</code>	controlled (simulator time)	DYNP Mode: One sample per simulator time slot FV Mode: One FV frame per simulator time slot
<code>-captureMode controlled</code> <code>-event</code>	controlled (simulator time)	1x Mode: Two samples per FCLK 2x Mode: One sample per FCLK
<code>-captureMode uncontrolled</code>	uncontrolled (free-running)	
<code>-captureMode uncontrolled</code> <code>-event</code>	uncontrolled (free-running)	

Alternatively, you can select *Controlled*, *Uncontrolled*, or *Controlled with Event* from the *Capture Mode* box in the *Trace Data Capture* group box on the Control tab of the xeDebug GUI. For more information, refer to the [Control Tab](#) section of [Chapter F, “Using the xeDebug GUI.”](#)

The following limitations ensure that controlled and uncontrolled time stamps cannot be mixed in the same waveform database:

- If a database had been opened with the `-event` option while the design is swapped out to software mode, after the design is swapped in to the hardware mode, the sampling rate will honor the `-event` option as specified in [Table 18-7](#) on page 936.
- If the `-captureMode` option is set to `uncontrolled` while the design is swapped in to the hardware, then, if you choose to swap out to the software mode, the database is automatically closed.

Using the run Command with Uncontrolled Hardware Clocks

Issuing the `run` command starts the following:

- ☐ NC-Sim simulator
- ☐ Controlled clocks in the DUT
- ☐ Uncontrolled clocks in the DUT
- ☐ Capture clock

A non-blocking run allows the NC-Sim simulator to run in the background when running in swapped-in mode. You can enter the commands targeted for both the simulator and the emulator while the simulator is running.

To enable a non-blocking run, use the `-nowait` option of the `run` command.

```
run -nowait [other-run-options]
```

Note: To allow your run-time command scripts to be compatible with the ICE mode, the `-wait` option is accepted and ignored.

To stop a non-blocking run, you cannot use the combination of `CTRL+C` keys. But, you can stop it by using any of the other methods that are described in the [Stopping the Design Clocks and Data Capture](#) section.

You can switch back from a non-blocking run to a blocking run at any point by using the `waitWhileBusy` command.

Stopping the Design Clocks and Data Capture

The `clockConfig` XEL command determines how clocks are stopped. It helps in determining the clocks that will be stopped by the various events, such as SDL triggers or NC-Sim breakpoints, that usually cause the run to end.

```
clockConfig -stopMode  
            [controlled | *controlled | uncontrolled | both | none | *none]
```

UXE User Guide

Debugging Designs on Palladium XP

Alternatively, you can select the *Controlled*, *Uncontrolled*, or *Capture/SDL* check box adjacent to the *Stop Mode* field in the *Run Control* group box on the *Control* tab of the xeDebug GUI. For more information, refer to the [Control Tab](#) section of [Chapter F, “Using the xeDebug GUI.”](#)

The following tables specify the effect of the `clockConfig -stopMode` command on:

- ❑ Controlled clocks ([Table 18-8](#))
- ❑ Uncontrolled clocks ([Table 18-9](#))
- ❑ SDL and capture clocks ([Table 18-10](#))

In these tables, “no” means the clock does not stop, and “stops” means the specified clock stops when the event happens. When the design is running in blocking mode, the control returns to the command prompt when the controlled clocks are stopped.

Table 18-8 Effect of using the `clockConfig -stopMode` Command on Controlled Clocks

Type of Event	Option Used for <code>clockConfig -stopMode</code>			
	controlled *controlled	uncontrolled	both	none *none
SDL Trigger	stops	no	stops	no
NC-Sim Breakpoint \$stop System Task run expires	stops	stops	stops	stops
CTRL + C keys stop command	stops	stops	stops	stops
stop -emulation command	stops	stops	stops	stops

The following table specifies the effect of the `clockConfig -stopMode` command on uncontrolled clocks.

Table 18-9 Effect of of using the clockConfig -stopMode Command on Uncontrolled Clocks

Type of Event	Option Used for clockConfig -stopMode			
	controlled *controlled	uncontrolled	both	none *none
SDL Trigger	no	stops	stops	no
NC-Sim Breakpoint \$stop System Task run expires	no	stops	stops	no
CTRL + C keys stop command	no	stops	stops	no
stop -emulation command	stops	stops	stops	stops

The following table specifies the effect of the clockConfig -stopMode command on SDL and capture clock. The sdl -haltOnTrigger command is disabled in the designs that have uncontrolled hardware clocks.

Table 18-10 Effect of using the clockConfig -stopMode Command on SDL and Capture Clock

Type of Event	Option Used for clockConfig -stopMode					
	controlled	*controlled	Uncontrolled	both	none	*none
SDL Trigger	stops	stops	stops	stops	stops	stops
NC-Sim Breakpoint \$stop System Task run expires	stops	no	stops	stops	stops	no
CTRL + C keys stop command	stops	stops	stops	stops	stops	stops
stop -emulation command	stops	stops	stops	stops	stops	stops

Using Uncontrolled Clocks with Other Debugging Operations

The limitations and explanations in the following sub-sections apply only to the designs that have uncontrolled clocks; Other types of designs are not affected.

Dynamic Target Mode

If you switch to the Dynamic Target mode by using the `configPM -dynamicTarget` command, the clocks (whether controlled or uncontrolled) that are generated in the DUT become free-running clocks. All these design clocks will remain free running until stopped with the use of the `stop -emulation` or `configPM -staticTarget` command. Use of the `clockConfig -stopMode` command is not applicable, and, therefore, does not have any effect on the design running in the Dynamic Target mode.

InfiniTrace

InfiniTrace is not supported with designs that have uncontrolled clocks.

SDL

The SDL feature is associated with capture clock. Stopping SDL means stopping capture (and vice versa). The SDL state is always clocked using FCLK, which is the fastest clock in the system.

When SDL is enabled, it is also associated with the “busy” status. Which means that SDL and the system is considered busy as long as SDL is running.

The SDL EXEC feature is supported with all the settings of `-stopMode`. When `-stopMode` is selected as `controlled`, `*controlled`, `none`, or `*none`, the EXEC action is executed without pausing the uncontrolled clock domain. This means that the EXEC action might be executed with thousands of cycles after the condition that triggered it, possibly resulting in an EXEC overflow condition.

The SDL display feature is supported with all the `-stopMode` options, but a buffer overflow might happen when `-stopMode` is set as `controlled`, `*controlled`, `none`, or `*none`, thereby resulting in a loss of display messages.

UXE User Guide

Debugging Designs on Palladium XP

Controlling the Run with SDL

This chapter describes the State Description Language (SDL). SDL is a run-time debugging feature that allows, based on design signal values, to control several aspects of the run such as when to stop, or when to capture probed data (tracing). Certain run modes allow to schedule execution of Tcl or XEL commands during a run using the EXEC facility.

Topics discussed in this chapter include:

- SDL Concepts
- SDL Preparation, Compilation and Execution
- Formatting and General Syntax Rules
- SDL Language Constructs
- Examples: SDL Program
- Importing Breakpoints from the NC-Sim Simulator into SDL

SDL Concepts

SDL is the trace control and break mechanism used by UXE. It can control data capture in In-Circuit Emulation (ICE) modes (both STB and LA modes) and Simulation Acceleration (SA) mode. It can stop the running design in Vector Debug (VD), STB and SA modes, and it can schedule execution of XEL commands during a run in STB and SA modes. SDL is specified as a program in human-readable text.

In VD, STB and SA modes, SDL will stop the run when executing a TRIGGER statement.

In Dynamic Target and LA modes, executing a TRIGGER statement typically stops only data tracing (probed data capturing). It is possible to override the default behavior by using XEL commands, and make the trigger stop the design clocks as well.

Additionally, if using a run mode that supports *Conditional Acquisition*, SDL can control when to capture or not capture probed data, using `ACQUIRE` or `NO_ACQUIRE` statements.

UXE User Guide

Controlling the Run with SDL

In most run modes, an `EXEC` statement can be used to execute XEL commands while the run is in progress.

SDL can print out formatted messages into a file and/or the console using the `DISPLAY` statement. Although `EXEC` can also print out messages by scheduling the Tcl command `puts`, the `DISPLAY` statement has some advantages over using `puts` inside `EXEC`, which are discussed in the [Printing Messages from SDL with the DISPLAY Facility](#) section.

SDL provides a way to specify complex conditions, based on signal value patterns along the time axis. Unlike basic breakpoints (which are common in many simulators for stopping the run and are based on a single pattern of signal values) SDL can detect complex patterns along the time axis with relative ease. An example can be: trigger when signal X makes 1000 value transitions without staying at one value more than 2 consecutive cycles.

Since SDL programs can be quite complex by themselves, SDL comes with its own debugging facility called *SDL Trace Dump*, which allows to troubleshoot SDL programs using either the waveform display or text format. See the [Debugging SDL Operation with the SDL Trace Dump Facility](#) section for more details.

If you are running in SA mode (under IXCOM) and swapped the DUT from the software simulator into the Palladium hardware, breakpoints that were defined for the software simulator using the `stop` command will not function correctly after the swap if they use design objects from the DUT. With some limitations, you can import these breakpoints into an SDL program and use it on Palladium, provided that the breakpoints are of type `object` or `condition`, and use design objects only from the DUT. For more information, see the [Importing Breakpoints from the NC-Sim Simulator into SDL](#) section.

Multiple SDL Instances

SDL can independently track up to 64 different patterns along the time axis. Each pattern is tracked by a state machine (called an SDL instance), which has internal resources such as counters and expression evaluators. Each of the SDL instances operates independent of the others, and can independently stop emulation, control data capturing or invoke XEL commands through `EXEC` statements.

In simple cases, a single SDL instance is enough. A typical application of using multiple SDL instances is to trigger when any one of several possible sequences of events happen. Each SDL instance can be used to identify one of these sequences. Using this example (trigger when a signal makes 1000 value transitions without staying at one value more than 2 consecutive cycles), a set of three SDL instances can be used to detect this pattern independently on three different signals.

While each SDL instance can contain only one state machine (or none at all), it can also contain multiple number of global EXEC, TRIGGER, DISPLAY, or NO_ACQUIRE statements in the Instance Definition Section (See State Machine versus Global trigger on page 947).

The number of different SDL instances that are available for use during run time must be declared to the compiler using the `sdlInstances` compiler option. If not specified, the compiler will allocate the resources for a single SDL instance.

Enabling and Disabling Individual SDL Instances

Each SDL instance can be enabled or disabled at run time (even while the emulator is running) independent of other SDL instances through the `sdl -enable` and `sdl -disable` XEL commands.

The software maintains two lists of SDL instance names, which are referred to as the **Enabled Instances List** and the **Disabled Instances List**.

The Enabled Instances List is a list of all the SDL instances that are always enabled. An instance name can be appended to the list through the XEL command:

```
sdl -enable <SDL_instance_name>
```

The Disabled Instances List is a list of all the SDL instances that are always disabled. An instance name can be appended to it in a similar way using the `-disable` option.

We will refer to all the other SDL instances that do not appear in either the Enabled or Disabled Instances Lists as the **Uncommitted Instances List**.

The Uncommitted Instances List, as well as all the TRIGGER/DISPLAY/EXEC/ACQUIRE/NO_ACQUIRE constructs in the Global Definition Section section are enabled or disabled as a whole using one of the XEL commands:

```
sdl -enable  
sdl -disable
```

For further details, refer to the `-enable` and `-disable` options of the `sdl` command in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

When running under GUI, selecting *Enable* or *Disable* for the *SDL program* in the *Control* tab affects only the Uncommitted Instances List and the constructs in the Global Definition section, but does not affect instances in either the Enabled Instances List or the Disabled Instances List.

This mechanism was originally created to allow uninterrupted use of one SDL instance as a “system service” (for example, to unload a streaming memory at given intervals), while using

another SDL instance for “regular debugging” (for example, to stop the emulator when a specific condition happens).

Enabling and Disabling Individual Trigger, Display, or Exec Statements

In addition to enabling or disabling complete SDL Instances, it is also possible to disable individual `TRIGGER`, `DISPLAY`, or `EXEC` statements in any part of the SDL program, provided that they have a label. The system only maintains a single list of disabled labels, and any statement with a label that exists in that list is disabled. Labels are added or deleted from the list through `XEL`.

For more details and other command-line options related to this feature, refer to the `-disable -label` and `-enable -label` options of the `sdl` command in the *UXE Command Reference Manual*.

Conditional Acquisition

SDL syntax provides two methods to define conditional acquisition, one using `ACQUIRE` statements, the other using `NO_ACQUIRE` statements. The two methods cannot be mixed in the same SDL program.

In the first method, probed data is acquired into the trace buffer in every cycle in which any `ACQUIRE` statement is active (which can be either an `ACQUIRE` action or a `Global ACQUIRE`).

In the second method, probed data is acquired into the trace buffer in every cycle, except cycles in which any `NO_ACQUIRE` statement is active (which can be either a `NO_ACQUIRE` action or a `Global NO_ACQUIRE`).

Note: The conditional acquisition feature of SDL (defined by using a `NO_ACQUIRE` or `ACQUIRE` statement), may create time segments in which data capture is disabled. When running in the SA mode, the waveform will show the value of signals as flat during these time segments, instead of showing these segments as blank or empty.

If the SDL program does not specify any `ACQUIRE` or `NO_ACQUIRE` in any SDL instance, the probed data is acquired every cycle.

Some operation modes, such as Vector Debug and infiniTrace, do not support conditional acquisition.

State Machine versus Global trigger

TRIGGER, DISPLAY, EXEC, ACQUIRE, and NO_ACQUIRE statements can be specified in one of two methods. The two methods can be used together in the same SDL program.

The first method, called Global TRIGGER (or Global EXEC or Global ACQUIRE/Global NO_ACQUIRE) uses a basic boolean expression based on design signal values (or assertions) that is evaluated repeatedly every cycle. Such global expressions can appear either at the very top of the SDL program (Global Definition Section), or at the top of each SDL instance (Instance Definition Section).

The second method uses a state machine that can track complex patterns along the time axis. Each SDL Instance can contain at most one state machine. Trigger or other Actions are embedded inside the definition of the state machine. The state machine has two internal counters that can be used to count long sequences (up to $2^{40}-1$). Other statements within the state machine can be used to control the state machine, for example, decide when to switch from one state to another, or when to load or decrement one of the two counters. Any Action can be executed based on boolean conditions made from design signal values or the outputs of the counters.

SDL Counters

Each state machine has two 40-bit counters (named COUNTER1 and COUNTER2), which can be set to auto-decrement every clock cycle, or it can be incremented or decremented only in selected cycles. Each counter can be independently controlled and its output can be tested to compare the counter's value against any positive integer value between zero and $(2^{40}-1)$.

Each counter can be loaded with a new value, which can be different from one LOAD command to another.

Note: Values for SDL counters can also be loaded dynamically from the command line by using the following command:

```
sdl [-ldcnt1 | -ldcnt2] [-instance <instance>] <value>
```

For more information, refer to the `sdl` command description and syntax in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

Each of the counters decrements or increments by 1 each time when the statement DECREMENT or INCREMENT is executed. In addition, the counter can also be put in auto-decrement mode through the use of the START statement. When in auto-decrement mode, the counter will decrement by 1 each time when its Count Enable is high. The Count Enable is a boolean expression made of design signals.

Although the `DECREMENT` statement is honored also while in auto-decrement mode, the counter decrements at most by 1 each cycle.

Since the counters are state machine resources, they can only be used inside STATE Statements. This is true for both counter actions, such as DECREMENT, and Counter Conditions.

Statement Usage and Execution Semantics of an SDL State Machine

A basic state machine construct is the STATE Statement, which defines one state of the state machine. Within each state, input conditions (based on values of design signals and/or the internal SDL counters) can be tested by one or more IF - ELSE statement, and various actions can be taken, based on the results of these tests.

In each evaluation cycle, first all the conditions are evaluated, and then all the actions are executed concurrently. This means, for example, that the result of an action to decrement a counter in cycle N can be tested by a condition only in cycle N+1.

Some types of actions cannot be executed together. For example, two GOTO actions that both request to transfer control to two different states are conflicting, and cannot be executed together. Conflicts happen when any of the following types of actions are requested to be executed in the same cycle (within the same SDL instance):

- Two GOTO actions with different destinations.
- Any two different actions for the same counter, except the following two combinations. These two combinations are allowed and will be executed concurrently if encountered in the same cycle.
 - LOAD and START
 - LOAD and STOP

In case of a conflict between two actions, SDL picks the one that appears first in the SDL program (when scanning the text from top to bottom). The SDL compiler prints out warning messages about potential conflicts before execution begins.

If enabled, the SDL state machine continues to execute as long as the design in the emulator continues to run (or until the requested amount of trace buffer is filled, when running in LA mode).

SDL Clocking

When running in ICE mode, each SDL execution cycle is typically a single Fast Clock (FCLK) cycle. Design signals can toggle faster or slower than FCLK depending on the sampling ratio. This means that when running with sampling ratio of 1X or 0.5X, SDL is not guaranteed to detect all signal values. For more details, refer to the [Limitations when Running with 1x Sampling Ratio](#) section.

When running in SA mode, SDL evaluation cycles run every FCLK cycle in which any design signal makes a transition. While this guarantees that SDL will detect all signal transitions, there is no guaranteed correlation between SDL evaluation cycles and simulation time. Also, SDL might or might not detect transitional values on combinatorial nets, depending on the way the design was compiled. Additional limitations exist when running in SA mode with 1X clocking. See the [Limitations when Running with 1x Sampling Ratio](#) section for more information.

When a design is compiled in SA mode using IXCOM with an uncontrolled clock generator (free running), you can select at run time the clocks (controlled or uncontrolled) that need to be stopped by an event, such as SDL trigger or NC breakpoint. For more information, refer to the [Running Designs with Uncontrolled Clocks Using xeDebug in SA Mode](#) section in Chapter 18, “Debugging Designs on Palladium XP.”

File Inclusion

A single SDL program can span several different files through the use of [File Inclusion](#).

Aliases

The [Alias](#) facility allows to assign any boolean expression to a single name, and then use that name in different places in the SDL program instead of writing the whole expression.

Use of Tcl Variables

The value of any global Tcl variable can be used inside the SDL program using the `$var` or `${var}` syntax. The variable's value is substituted once at the beginning of the run. For more details, refer to the [Using Values of Tcl Variables Inside SDL](#) section.

Assertions

Expressions in SDL program can check not only values of design nets, but also whether an assertion has reached a specific state. See the [Assertion State Conditions](#) section for more details.

Auto Probing

The auto probing facility allows to request from XEL to automatically probe every net or assertion that is actively used in the SDL program. Nets or assertions that appear in alias definitions that are not used actively in the SDL program are excluded.

Fast versus Accurate

By default, the SDL instrumentation is sampling the design signal values with a delay of up to 2 fast clock (FCLK) cycles. In STB mode, the delay is almost always two FCLK cycles, except some rare cases (for example, on the first cycle of the run), where the delay can be less than two. Where possible, the system tries to hide the effects of this delay from you. For example, the delay does not affect requests to filter out probe samples in specific cycles through Conditional Acquisition (`ACQUIRE` or `NO_ACQUIRE`), thanks to special instrumentation that re-synchronizes the timing of the probe samples with SDL. Also, the cycle number of the trigger point as reported on the GUI or by XEL, or in an SDL trace dump is not showing the delay.

However, this delay becomes visible when the emulator stops because of a trigger, or when emulation clock are suspended during execution of XEL commands through the `EXEC` statement. If you request from XEL at that point to view any SDL status information (such as state name or SDL counter values) you will see that the status information matches the cycle in which the trigger happened according to the SDL program, but is lagging after the design signals.

Another possible anomaly that can be observed is when the run is stopped because of an event outside of SDL. For example, reaching the specified amount of time to simulate, or an assertion that stops the run. In that case, an `EXEC` or `TRIGGER` that was supposed to happen on the same cycle (or time slot in IXCOM) in which the run stopped, might not be processed.

It is possible to override this behavior and eliminate the delay (switching to what's called *accurate triggering*), but this comes with a speed penalty. In STB mode in such case, the emulator speed is only 1/3 of the regular speed. Switching between stopping the emulator with delay and without delay is performed by the XEL command:

```
xeset stopDelay 0|1
```

This command can only be executed while the emulator clocks are stopped (or suspended while executing an `EXEC` statement). A value of 0 means no delay between the trigger and the stop point (and running at 1/3 speed).

In some cases, one can avoid the delay without incurring the 1/3 speed penalty by switching from fast to accurate triggering only during strategic time segments in which triggering is expected. This is practically done by executing the `stopDelay` command itself from within SDL using the `EXEC` statements.

Under IXCOM, there is another twist. On top of the above delay, `TRIGGER` and `EXEC` actions are executed only at the end of the simulation time slot after all the behavioral processes finished evaluation. This means that even if `stopDelay` was specified as 0, the conditions that caused the `TRIGGER` or `EXEC` actions, might have already disappeared by the time the `TRIGGER` or `EXEC` are actually executed.

Debugging SDL Operation with the SDL Trace Dump Facility

SDL comes with a debugging facility called *SDL Trace Dump*, which allows to troubleshoot the behavior of SDL programs.

During the run, key information on the state machine component of each SDL instance is recorded into dedicated memory, which is arranged as a circular buffer. The buffer's depth is 16k samples, and newer data samples overwrite the oldest. The data is recorded only in *interesting clock cycles*. By default, an interesting clock cycle is one in which any SDL action (except `ACQUIRE` or `NO_ACQUIRE`) is executed. You can change the default through XEL and designate what cycles to record based on the type of actions that SDL executes. For example, you can request to record information only in cycles in which SDL executes a `GOTO` action, or on the other extreme, record continuously every cycle.

The recorded data includes information such as the emulation time, value of the two SDL counters, SDL state name, and what actions were executed by the state machine in that cycle.

The data is always collected during a run, but you can decide whether to dump it to a disk file or not, through XEL. When the data is printed in textual form, the information is most detailed and can include (based on XEL switches) even the filename and line number in the SDL file for each action being executed.

There is a limitation regarding SDL trace dump in text mode: if a label (for a `TRIGGER`, `DISPLAY`, or `EXEC`) is disabled or enabled in the middle of the run (for example, with `sd1 -disable -label -now ...`), the resulting SDL trace might not correctly show the actions associated with that label.

A waveform format is also available, but it shows only SDL state name and values for the two counters.

Recording of SDL activity into the SDL trace memory is stopped when a trigger happens. You can request that the tracing stop immediately upon trigger, or only when regular probing is stopped (as specified by the post trigger amount), depending on settings in the `sdl -traceOn` XEL command.

The trace is dumped to a file by executing the `sdl -traceDump` XEL command. When the command is executed, it dumps to the specified file SDL activity that was traced in the most recent run. If no trigger happened in the most recent run, the trace dump includes whatever data was captured so far.

By default, at the beginning of each new run, the SDL trace memory is reset, which means that any traced data from the previous run is discarded. However, you can request through XEL to reset the SDL trace memory not every run, but only when dumping the data to the disk, or you can even request to never reset the SDL trace memory.

Note that if a trace dump is requested while the emulator is running, SDL tracing is suspended while the data is being dumped into the file.

When running from GUI, you can request to automatically create an SDL trace dump every run by selecting the *Upload SDL Activity* in the *Control* tab or *Infinitrace* tab. The waveform will show SDL activity only for the last run, although you can override this behavior in the *Advanced SDL Control* window (which you launch from the *SDL* tab). This window allows you to control other advanced aspects of SDL and SDL trace dump in specific.

Saving and Restoring the Internal State of SDL

The internal state of the SDL program can be saved into a file and restored later, either in the same session, or in a new session. The saved file includes information for each SDL instance, such as the name of the SDL instance being saved, the name of the current SDL state, current values of SDL counters, and whether they were auto-decrementing or not.

Saving (or restoring) SDL state is typically done together with saving (or restoring) the emulated design state, in order to allow to return to a previously saved checkpoint, including the SDL state.

The save/restore mechanism only requires matching of state and instance names, therefore it can be used to restore the state of one SDL program into another SDL program. The only requirement is that the two SDL programs agree with names of instances and states.

Detection of Events without Triggering

If you only want to detect whether specific events occurred during a run, and do not want to stop the emulator (or data tracing) each time a specified event happens, use the `-onlyReport` switch of the `sdl` command. Here, the term event refers to a specified expression becoming true. When the `-onlyReport` switch is turned on, trigger conditions that are defined in the [Global Definition Section](#) or [Instance Definition Section](#) are disabled from stopping design or capture clocks, but they are still reported by the `sdl -getTriggers` and `sdl -report` commands. Triggers coming from the state machine portion of SDL are not disabled, and will cause the design and/or capture clocks to stop. For more details, refer to the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

Limitations when Running with 1x Sampling Ratio

If the design was compiled to run with sampling ratio of 1x (Refer to *Appendix Clocking* in the *UXE Compile User's Guide* concerning `clockOption` technology CAKE), it is possible that a signal will make more than one transition within the same sampling cycle (one FCLK cycle).

If requested to test whether a signal is 0 or 1, SDL will pick the value at the end of the FCLK cycle. This means that SDL might miss every second value on the fastest changing signal.

If requested to test for a special value such as a positive edge, SDL will scan the entire FCLK cycle to determine whether the condition is true. For example, in order to qualify for a positive edge, it is enough that the signal will make a low to high transition anywhere in the last FCLK cycle. In order to qualify for a “stable low” value, the signal must be low throughout the entire FCLK cycle. For general information on the use of special values in expressions, refer to the [Special Non-Numeric Constants](#) section.

Limitations when Running with 0.5x Sampling Ratio

If the design was compiled to run with sampling ratio of 0.5x, special values such as edge detection should not be used in the SDL program because SDL will not handle them correctly.

Testing for regular values (0 or 1) will sample signals only once at the end of each sampling period.

Interaction between SDL and XEL

SDL and XEL can interact in several ways.

UXE User Guide

Controlling the Run with SDL

First, in emulation modes that support this feature, SDL can issue any Tcl or XEL commands during emulation using the `EXEC` facility (See descriptions of `EXEC` and `Global EXEC`). From a syntax point of view, any string that can be executed by the Tcl interpreter, including calls to Tcl procedures is accepted. There are some XEL commands that are not allowed in an `EXEC`, irrespective of whether these commands are called directly from the `EXEC` or indirectly through a Tcl procedure. In run modes that allow stopping the clocks, the design clocks are suspended during the execution of the XEL commands to guarantee time coherency. When running in LA or Dynamic Target mode, the clocks are free running, time coherency is not guaranteed, which means that `EXEC` requests are executed with significant delay.

Second, XEL can control SDL, or retrieve from it information through the `sdl` XEL command.

Here is a quick summary of the most important facilities that are provided by the `sdl` command. You will find more details as well as command syntax in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

- Specify the name of the SDL file (the file that contains the SDL program).
- Verify the contents (for syntax errors) of an existing SDL file without actually requesting to run it.
- Enable or disable the SDL feature as a whole, as well as enable or disable individual SDL instances, or individual `TRIGGER`, `DISPLAY`, or `EXEC` statements.
- Retrieve current status information such as:
 - Current state name and counter values for each of the SDL instances.
 - Did a trigger happen, and if it did, in what cycle/time, and which statement (or statements) caused it.
 - Check whether specific trigger(s) happened by providing the identifying label.
- Generate a log of recent SDL activity, called SDL Trace Dump (for example, state transitions, actions, counter values and so on) in either text or waveform format, and specify what information should be logged.
- Turn on/off automatic probing of all design signals that are used by SDL.
- Save and restore SDL internal state to/from a file.
- Control destination of `DISPLAY` messages.

Several other XEL commands, variables, and run-time parameters also affect the SDL execution as follows:

- Symbols (representing vectors), defined through the `symbol` XEL command, can be used in SDL expressions.

UXE User Guide

Controlling the Run with SDL

- The value of any Tcl variable that is declared at global call level (that is, by the `set` Tcl command outside any proc definition, or by the `global` command), can be used inside an SDL program with the `$var` or `${var}` syntax, where `var` is the name of the Tcl variable. Any numeric value or a character string can be passed that way from the Tcl script to the SDL program.
- Default values for post trigger samples or trigger position can be set by the `xeset` XEL command, and will be used by the SDL instrumentation if not set inside the SDL program.

Each time before starting SDL, the SDL program is recompiled if any of the following occurs:

- The main SDL file or any other file that it includes has changed.
- The value of any Symbol that was defined by XEL and is used in the SDL program has changed.
- The value of any Tcl variable that is used by the SDL program has changed.
- Any one of the XEL commands that can affect the SDL run-time characteristics have been executed.

Typically, SDL is started by executing the `run` command. However, SDL will not be recompiled (regardless of any changes) if the run is started using the `run -continue` command.

These rules also apply to the independent trigger expression that is defined by the `sdl -expression` command (although the independent trigger expression is not considered part of the SDL program).

Printing Messages from SDL with the DISPLAY Facility

The DISPLAY facility allows to print out messages from an SDL program based on specified conditions. The message may optionally contain signal values and simulation time at the exact emulation clock cycle from which the `DISPLAY` statement was called. The message is formatted using a syntax similar to the Verilog `$display` system task.

You can also use EXEC to print message, however, there are some disadvantages to using `EXEC` and these may even render it useless for some situations. EXEC allows to execute any Tcl command; so, messages can be printed using the standard `puts` Tcl command. For example, printing the value of the signal `ABC`, and the current simulation time using `EXEC` may look like this:

```
if(some_condition)
    EXEC "puts \"value of ABC at time [getTime]is [value ABC]\"";
```

Using a `DISPLAY` statement to print the same information may look like this:

UXE User Guide

Controlling the Run with SDL

```
if(some_condition)
    DISPLAY ("value of ABC at time %t is %d", ABC);
```

Comparing DISPLAY and EXEC Methods

Use of `DISPLAY` statements has the following advantages over `EXEC`:

- **Simplicity**: The syntax of `DISPLAY` statement is simpler. Note that when using `EXEC`, you need to escape the inner set of double quotes with a backslash.
- **Speed**: Each `EXEC` statement causes the design clocks to pause while the Tcl interpreter executes the `puts` statement. This operation typically takes several hundred milliseconds, and may cause severe slow down when the number of `EXEC` requests is large.

`DISPLAY` messages are buffered during the run in a special *display buffer* memory, and therefore do not affect execution speed as long as the workstation can read the contents of the display buffer fast enough. Reading the display buffer is done in large blocks while the design is running, so in most cases, the `DISPLAY` statements have no effect on the execution speed once the run starts.

- **Use with Dynamic Target**: If running a design in Logic Analyzer or Dynamic Target mode, the design clocks are not allowed to stop. Therefore, `EXEC` requests can easily get overwritten and lost because they are not buffered.

Display requests are buffered. Therefore, `DISPLAY` can be used in Logic Analyzer or Dynamic Target mode without losing any messages. The only requirement for not losing `DISPLAY` messages is that the rate at which the workstation reads the trace buffer contents must be faster than the average rate at which the `DISPLAY` messages are generated by SDL. And if this is not possible, you have an option to request a large display buffer depth at compile time.

- **Time coherency**: Values of arguments of the `DISPLAY` statements are captured in the exact cycle that invoked the `DISPLAY` statement. Therefore, the printed values will match the condition that invoked them.

Not so with `EXEC`. The `EXEC` statements are typically executed with some delay after the condition that invoked them. For example, in STB mode, the delay is 2 cycles or less (See the [Fast versus Accurate](#) section). In LA mode, the delay can be thousands of cycles.

There are also some advantages for printing messages with `EXEC` over `DISPLAY`:

- **SDL program load time**: Arguments of the `DISPLAY` statements must be routed to the display buffer when loading the SDL program into the emulator. This might add to the time it takes to start a `run` command with a new SDL program. Therefore, if your SDL

program prints the message only a couple of times throughout the debug session, it might still be faster to use `EXEC` instead of `DISPLAY`.

- **Message Formatting:** `EXEC` statements can invoke the full power of the Tcl programming language for the purpose of generating the output string.

Emulation Resources Consumed by SDL

The typical overhead for each SDL instance that is requested during compile time is approximately 1MB RAM and 3k gates, irrespective of whether that instance is actually being used during the run time or not.

The actual overhead can be larger when increasing the default values for some of the compiler options that affect the size of the display buffer or the SDL trace buffer.

The total size of the display buffer (in bytes) is determined by the following formula:

$$D * (G/8 + W/8 + 2*N)$$

Here,

- `D` is the depth as specified by compiler option `sdlDisplayDepth`.
- `G` is the value specified by compiler option `sdlGlobalDisplay`.
- `W` is the value specified by compiler option `sdlDisplayWidth`.
- `N` is the value specified by compiler option `sdlInstances`.

For example, the display buffer for a display depth of 4k, 64 global `DISPLAY` statements, a display width of 1024, and 3 SDL instances, will consume 640kB of emulated memory:

$$(64/8 + 1024/8 + 8*3) * 4096 = 655,360 \text{ (equivalent to 640kB)}$$

The total size of the SDL trace dump buffer (in bytes) for all SDL instances is determined by the following formula:

$$T * N * 20$$

Here,

- `T` is the value specified by compiler option `sdlTraceDepth`.
- `N` is the value specified by compiler option `sdlInstances`.

For detailed information about the compiler options mentioned above, refer to the description of `compilerOption` command in the *User Data Commands* chapter of the *UXE Command Reference Manual*.

Summary of Basic SDL features

- Up to 64 SDL instances, each can be enabled or disabled individually.
- Each SDL instance can have a state machine with up to 64 states.
- If - else statements for testing conditions in any nested configuration.
- Each SDL instance has two decrementing 40 bits counters, each controlled independently.
- Ability to enable and suppress probed data acquisition in probing modes that support conditional acquisition.
- Ability to stop the run in some operation modes.
- Ability to execute Tcl or XEL commands during emulation in STB, LA or Dynamic Target modes (EXEC facility).
- Ability to generate an activity log (SDL trace dump) in either text or waveform format per SDL instance.
- Ability to print out formatted messages (DISPLAY facility).
- State-less TRIGGER, DISPLAY, and EXEC statements.
- Ability to request auto probing of all nets and assertions referenced in the SDL program.
- Ability to save and restore internal SDL state.
- Ability to disable individual TRIGGER, DISPLAY, and EXEC statements using their label.

SDL Preparation, Compilation and Execution

Setup during Design Compilation

The maximal number of instances that can be used by an SDL program can be specified during design compilation by the compiler directive:

```
compilerOption -add {sdlInstances <num-instances>}
```

<num-instances> must be an integer number between 1 and 64. This number sets the upper limit for the number of SDL instances that can be used at run time. If unspecified, a value of 1 is assumed.

The maximal number of samples that an sdl trace dump can have can be specified during design compilation by the compiler directive:

```
compilerOption -add {sdlTraceDepth <depth>}
```

<depth> must be a positive integer number. This number can use a postfix of **k** or **M** (for example, 4k means 4x1024). If unspecified, a default value of 16k is used. Values above 16M or below 1k are converted to the maximal value of 16M or the minimal value of 1k.

Other than that, all other aspects of SDL are programmable at run time.

Preparing the SDL Program

During run time, SDL is prepared by a text editor and has to be stored in a file (or a set of files) before it can be executed. The SDL text is known as an *SDL Program*. The editor can be any text editor of your choice, but the recommended method is to use the *SDL Editor* group box under the *SDL* tab in the main xeDebug GUI window.

Using the *SDL* tab has the following advantages:

- It has a dedicated set of predefined buttons and menus for each of the SDL keywords.
- It highlights SDL keywords in a different color.
- It automatically verifies the SDL program for any errors and save it to a file before each run.
- It allows you to iterate through any errors found in the SDL program, while highlighting the location of the offending text on the screen.
- It allows you to drag and drop signal names from other browsers (for example, the Design Browser or the Schematic Browser).

When running in the xeDebug GUI, specify the filename in the *Edit File* field in the *State Editor* group box and click the  button adjacent to it.

Before starting a run (either through the xeDebug GUI or by executing the `run XEL` command), the SDL program is compiled into a special binary image that is downloaded into the Palladium hardware. The SDL compiler, which performs that process, can produce error or warning messages. Any error conditions reported by the SDL compiler must be fixed before the run request can be executed.

SDL Enabling, Activation, and Execution

Once the file containing the SDL program is defined with the `sdl -setFile` command, and SDL (or any one of its components) is enabled by the `sdl -enable` command, SDL is ready for use (considered *enabled*). Assuming that the SDL program does not have any errors,

SDL is compiled into a binary image and loaded into the emulator when any one of the following XEL commands is executed:

- `sdl -load`
- `run`
- `infiniTrace -goto trigger`

`sdl -load` only loads the SDL image into the emulator, which allows you to query about basic properties of the SDL program (for example, what are the names of instances being used). This is done whether SDL is enabled or not.

`run` and `infiniTrace -goto trigger` load the SDL image then start both SDL and design clocks and/or capture clocks (depending on operation mode).

Once SDL is loaded into the emulator, it is considered **active**, whether it is enabled or not. Note that a successful `sdl -load` command will always load the SDL program into the emulator and make it active, while a successful `run` or `infiniTrace` command will load the SDL program and make it active only if it is enabled.

An active SDL program can be enabled during a run, even if the run started with the SDL program disabled, therefore if you want to start the run with an SDL program disabled, and then enable it while the run is in progress, you need to execute the `sdl -load` command before executing the `run` command. In the current release, this will work only if you do not make use of the independent trigger expression (with the `sdl -expression <expressions>` command)

You can query whether SDL is active or not through the `sdl -isActive` command. The return values of the `sdl -isActive` command only refer to the SDL program and do not consider the independent trigger expressions that are specified using the `sdl -expression <expressions>` command.

Formatting and General Syntax Rules

Format

SDL is a free-format language. You can break statements across multiple lines. An end of line is considered a white space. You can also put multiple statements on the same line.

The syntax of the File Inclusion facility is an exception. It must be contained on exactly one line.

Statements are typically terminated by a semicolon. However, where several statements are grouped into a single block using braces, then the right brace is used as the terminating token.

Comments

Insert comments by using `/* */` or `//` notation. For example:

```
/* This is a comment. This comment style
   can span multiple lines. */
// This is also a comment. This comment style
// restarts at each new line.
```

Comments cannot be inserted in the middle of tokens (for example, keywords, symbols, constants).

White Space

SDL ignores spaces, tabs, end of lines and empty lines (all considered as “white space”), except where used as separators between tokens. You must use white space between tokens in cases where it would otherwise be impossible to separate them.

Case Sensitivity

All SDL keywords are case insensitive; they are recognized in either upper, lower or mixed case.

For example, `Trigger`, `trigger`, `TRIGGER` are all recognized as the same keyword.

Names that represent design signals, or other user given names (aliases, instance names, labels, and state names) are case sensitive.

File Inclusion

SDL programs can include text from other files similar to the way this is done in Verilog through the use of an `include` directive.

The syntax of an include directive is as follows:

```
`include "<file-name>"
```

The line cannot contain any comments or substitution of TCL variables (with the syntax `$<var>` or `${<var>}`).

The first character is a backward single quote mark, while the filename must be enclosed in straight double quotes. The `include` directive is not a free format: It must be specified on a separate line, and the `include` keyword must be specified in lower case. On the other hand, the include directive can be inserted at any point in the SDL program.

Included files can include other files to any nesting level.

The string `<file-name>` can contain either an absolute directory path (starts with the character `/`) or it can contain a relative directory path, or just the filename with no path at all.

If `<file-name>` does not contain an absolute directory path, then it is searched in the current directory. If not found in the current directory, then it is searched in a list of directories that are specified by the run time parameter `sdlIncludePath`. The list must be space separated. For example:

```
xeset sdlInclude {/home/george/myfiles/sdl /home/ben/projectX}
```

In order to make this search path visible to all your sessions with debug tools, whether running in script or GUI mode, you can place such definition inside the `.xerc` or `.rtxerc` files. The `.xerc` file is an XEL script that is always executed (if it exists) during the software initialization phase, while `.rtxerc` is executed on the initial call to the `configPM` command. It can be placed either inside the current directory from where you start the debug tool, or under the home directory.

Using Values of Tcl Variables Inside SDL

Values of global Tcl variables can be used directly inside the SDL program using either the `$<tcl-variable-name>` or `${<tcl-variable-name>}` syntax.

Any value can be substituted, whether numeric or a string. The second form with the `{ }` is useful when the variable value has to be concatenated with another string without any intervening delimiter.

For example, suppose that a Tcl variable is assigned the value `top.alu`, which represents a hierarchy in the design, by executing the following Tcl statement (either from the XE prompt or from a script):

```
set path top.alu
```

Suppose that the SDL program contains the following construct:

```
state s1 {
    if(${path}.y[7:0] < ${path}.x[7:0]) trigger;
    ...
}
```

The above construct is equivalent to:

```
state s1 {  
    if(top.alu.y[7:0] < top.alu.x[7:0]) trigger;  
    ...  
}
```

Note that `path` must be declared in the Tcl script at global call level. This means that if the variable `path` is assigned inside the definition of a Tcl `proc`, then it must be declared explicitly as global as shown in the following example:

```
proc foo {args} {  
    global path  
    set path top.alu  
    ...  
}
```

The Tcl variables that are referenced with the above syntax inside an SDL program are substituted with their values only when the SDL program is loaded into the emulator (for example, at the beginning of each run). This does not apply to Tcl variables that appear inside **EXEC** actions. The construct `$var` or `${var}` inside **EXEC** actions are evaluated each time when the **EXEC** action is executed. See more details in the description of the **EXEC** action.

SDL Language Constructs

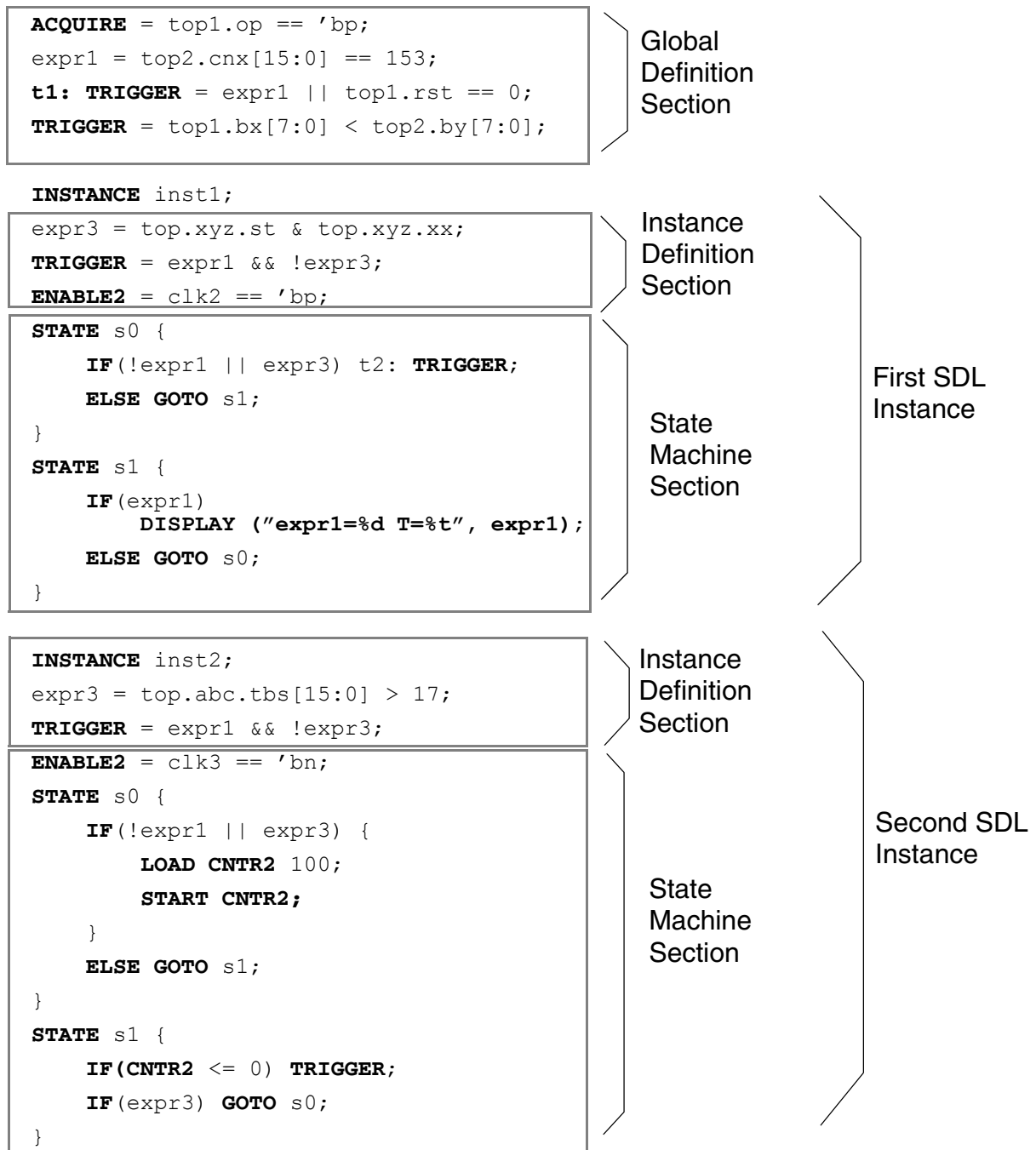
Any of the SDL constructs that is explained below is optional. An SDL program can be as simple as a single statement of the form:

```
TRIGGER = <signal-name>;
```

Or it can be a complex program made of several SDL instances spanning on multiple files (through the use of the File Inclusion facility).

Figure 19-1 depicts an SDL program with its various constructs. All SDL keywords appear in the example in uppercase bold letters.

Figure 19-1 SDL Program Example



An SDL program can have a Global Definition Section that is shared by all instances, followed by definitions of SDL Instances.

UXE User Guide

Controlling the Run with SDL

Each SDL instance starts with a INSTANCE Statement, and extends until the next Instance statement (or the end of the file).

Each SDL instance can have its own Instance Definition section (whose definitions are limited only to that instance), followed by a State Machine section.

If the SDL program has only one instance, then the Instance statement can be omitted, and the software will assign to it a default instance name.

The State Machine section is made of a series of STATE Statements.

SDL main constructs are grouped into the following categories:

- Expressions
- INSTANCE Statement
- Global or Instance Definition Sections
- STATE Statement
- IF - ELSE statement
- Actions

Table 19-1 lists the predefined keywords and operators that can be used in expressions. Expressions will typically use in addition names of design signals and assertions.

Table 19-1 Predefined Keywords and Operators for use in Expressions

Type	Keywords and Operators	Usage
Counter Names	Counter1, Cntrl	Used in one of the following ways: COUNTER1 <= 0 COUNTER1 > 1000 COUNTER2 <= 150 COUNTER2 > 0
	Counter2, Cntrl2	
special operators	{}	Concatenation operator. Create a vector from a list of scalars.
	[]	Bit-select or part-select of vectored signals.
	()	Parenthesis for changing evaluation order.

UXE User Guide
Controlling the Run with SDL

Table 19-1 Predefined Keywords and Operators for use in Expressions

Type	Keywords and Operators	Usage
Unary Boolean Operators	~	Unary bitwise negation
	!	Unary logical negation
	>	greater than
	<	less than
	>=	greater than or equal to
	<=	less than or equal to
	==	equal to
	!=	not equal to
	&	unary reduction AND, bitwise AND
	^	bitwise XOR
		Unary reduction OR, bitwise OR
	&&	logical AND
		logical OR
Assertion state values	"active"	Used as in the following way: <code><assertion-name> == "finished"</code>
	"inactive"	
	"finished"	
	"failed"	
	"off"	

UXE User Guide

Controlling the Run with SDL

Table 19-2 lists the predefined keywords that can be used in the Global Definition Section or the Instance Definition Section. The list does not include keywords that can be used in expressions, and are listed separately.

Table 19-2 Predefined Keywords in a Global or Instance Definition Section

Type	SDL Keyword	Usage
Definitions of Numeric Values	counter1, cntr1	Specify default load value for one of the two counters.
	counter2, cntr2	
	triggerPos	Specify trigger position in percent. Can appear only in Global Definition section.
	postTriggerSamples	Specify the number of FCLK cycles that will be captured into the trace buffer after trigger. Can appear only in Global Definition section.
Counter Decrement Enable	enable1	Specify Decrementing clock for each of the two counters.
	enable2	
Simple IF Block	if	Used for global trigger, display, acquire, no_acquire and exec
Trigger	trigger, trig	Global Trigger Definition
Command Execution	exec	Execute a Tcl/XEL command
Message Printing	display	Generate a formatted message
Conditional Acquisition	no_acquire, noAcquire, nacq	Specify when to acquire or not acquire probed data
	acquire, acq	

UXE User Guide

Controlling the Run with SDL

Table 19-3 lists the predefined keywords that can be used for all other SDL constructs. All the entries in this table, except INSTANCE are used for definitions in the State Machine section. The list does not include keywords that can be used in expressions, and are listed separately.

Table 19-3 Other Predefined Keywords

Type	SDL Keyword	Usage
Instance	instance	Beginning of a new SDL instance
Basic blocks of a State Machine	state	Definition of an SDL state
	if	Control execution based on values of expressions
	else	
Counter Names	counter1	Names of counters for use in counter actions
	counter2	
Actions	decrement, decr, count	Decrement counter by 1
	increment, inc	Increment counter by 1
	load	Load counter with initial value
	start	Start counter auto-decrement mode (Start to count every cycle)
	stop	Stop counter auto-decrement mode
	goto	Transfer control to another state
	trigger, trig	Stop design clocks or tracing
	no_acquire, noAcquire, nacq	Do not acquire traced data in current cycle
	acquire, acq	Acquire traced data in current cycle
	exec	Execute a Tcl or XEL command
	display	Generate a formatted message

Expressions

Topics discussed in this section include:

■ General Guidelines

- Names
- Counter Conditions
- Assertion State Conditions
- Numeric Constants
- Special Non-Numeric Constants

General Guidelines

Expressions are used in `IF` statements to define conditions, or in the Global Definition Section or Instance Definition Section for definitions of `TRIGGER`, `DISPLAY`, `EXEC`, `ACQUIRE`, `NO_ACQUIRE`, `ENABLE1`, and `ENABLE2`.

They can also be used as building blocks for other expressions through the use of Aliases.

Expressions are made of Names, Numeric Constants, Operators, Assertion State Conditions, and Counter Conditions. An expression can also be a single name, a single assertion condition, or a single counter condition.

The use of boolean operators and parenthesis in expressions is very similar to their use in C or Verilog.

When evaluated during run, the result of an expression can be either a scalar boolean value (0 or 1), or a vector of boolean values. As in Verilog, a vector can be interpreted as an integer. This interpretation becomes significant when applying an operator between the vector and an integer constant. For example:

```
sig1[7:0] > 23
```

In this example, an 8 element vector representing an 8 bit integer is compared to the integer 23.

The same conventions that are used in C or Verilog are used when evaluating an expression as a condition: the result of a scalar expression is considered `TRUE` if it is 1, and `FALSE` if it is 0. The result of a vector expression is considered `TRUE` if any of its bits is 1, and `FALSE` if all its bits are 0.

Names

A name appearing in an expression can be one of the following:

- *Signal name* - a name of a design signal.

UXE User Guide

Controlling the Run with SDL

- *Assertion name* - a name of an assertion from your design.
- *Alias* - a name representing a scalar or vector expression that was defined as an alias to an expression in the Global or Instance Definition Sections of the SDL program.
- *User Symbol* - a name defined externally to the SDL program through the `symbol` XEL command.
- *CPF condition* - a name of a CPF object or event. For detailed information, refer to the Specifying CPF Conditions in SDL Expressions section of Chapter 16, "Specifying Low-Power Intent on Palladium XP."

A User Symbol cannot represent arbitrary expressions. It is used by the UXE debugger for grouping unrelated scalar signals into vectors to streamline and facilitate some operations (such as reading or forcing signals).

An assertion name must contain the full hierarchical path. It can be used in Assertion State Conditions.

Signal Names must contain the full hierarchical path. If a signal name represents a vector, then it can be accompanied by an explicit index notation. If a name representing a vector appears without any index notation, then all its bits are used. An index notation can select a single bit of the vector (bit-select) as in:

```
sig1[3]
```

Or it can select a range of bits (part-select) in a vector as in:

```
sig2[7:2]
```

The order of bits is determined by the two indexes inside the `[]`. In the example above, bit 7 of sig2 is interpreted as the MSB, while bit 2 is interpreted as the LSB.

Aliases or User Symbols cannot be used with an explicit index notation.

If an Alias, User Symbol, or SDL keyword uses the same name as a design signal, then this does not cause an error condition, but the design signal becomes inaccessible to the SDL program. This is a potential issue only for top-level design signals that are specified without their hierarchy path.

Counter Conditions

Use the `>` and `<=` operators to compare the values of one of the two counters. Specify the counter name on the left side of the operator and the integer value on the right. The integer value must be between 0 and $(2^{40} - 1)$. Each SDL state can use only one unique integer value for comparing each counter.

Syntax:

```
COUNTER1 <= <value>
```

or:

```
COUNTER2 <= <value>
```

or:

```
COUNTER1 > <value>
```

or:

```
COUNTER2 > <value>
```

<value> is an integer value between 0 and $2^{40}-1$.

The first two cases that make use of the `<=` operator are evaluated as 1 (or TRUE) if the counter is less than or equal to the integer value specified by *<value>*.

The third and fourth cases are the inverse of the first two.

For example, `COUNTER1 <= 0 7` is equivalent to `!(COUNTER1 > 0 7)`.

The keywords `CNTR1` and `CNTR2` can be used instead of `COUNTER1` and `COUNTER2`.

Assertion State Conditions

The state of an assertion can be tested by an expression using the following syntax:

```
<assertion-name> == "<assertion-state>"
```

<assertion-name> is the full hierarchical name of the assertion from the RTL description.

<assertion-state> is one of the following five pre-defined strings:

```
active inactive finished failed disabled
```

These strings can be used in either lower, upper or mixed case.

For example here is a statement that will print out a message when an assertion gets to a finished state:

```
IF (top.ct1.my_assertion == "finished")  
    DISPLAY ("my_assertion arrived to a finished state");
```

It is also possible to check whether any assertion in the design failed, by testing the value of the net `ASSERTION.FAILURE`. This net is defined by the compiler only if the design has assertions in it.

For example:

```
IF (ASSERTION.FAILURE)
    DISPLAY ("some assertion failed");
```

See the [Assertion States](#) section of [Chapter 10, "Compiling and Running Designs with Assertions"](#) for more information on assertion states.

Numeric Constants

Constants used in expressions can be specified with either binary, octal, decimal or hexadecimal radix. The radix notation for regular numerical constants is very similar to the one used by Verilog. For example, a 6-bit constant that represents the decimal value of 14 can be written as any one of the following:

binary	6'b1110
octal	6'o16
decimal	6'd14
hexadecimal	6'hE

Most of the Verilog conventions for writing integer constants are followed:

- The radix letter as well as hexadecimal letters are case insensitive.
- If the number of bits is not specified, then it is inferred from the value. For example, 'b1110 is a 4 bit constant representing the value 14.
- A number without an explicit radix notation is assumed to be decimal.
- Underscores can be inserted anywhere in the string (except left most character) in order to improve readability, but they are otherwise ignored. For example, 'b1110_1111 is equivalent to 'b11101111.

Special Non-Numeric Constants

SDL also recognizes a set of *special values* that can be used only in conjunction with the comparison operator (==). These values are represented by the following letters, which can be in either upper- or lower-case:

P - Positive edge.
N - Negative edge.
T - Any transition (Positive or negative edge).
L - Value is stable at a low value (0 for two consecutive FCLK cycles).
H - Value is stable at a high value (1 for two consecutive FCLK cycles).

UXE User Guide

Controlling the Run with SDL

S - Value is stable at either low or high value for two consecutive FCLK cycles.

X - Don't care; any value is accepted as a match.

The only thing that can be done with these values is compare them to a signal or to an expression with the (==) operator. Expressions that are used in such comparisons cannot contain counter conditions.

With the exception of X (don't care), all other values represent a combination of two consecutive values. For example:

```
sig1 == 'bP
```

is testing for a positive edge on the signal `sig1`. The result is 1 (TRUE) if the signal `sig1` has the value 0 followed by a value of 1.

Consider another example:

```
(sig1 ^ sig2) == 'bT
```

Here, any transition occurring on the XOR result of two signals (the left operand) will make the result of the entire expression 1 (or TRUE). In other words, this expression is true if one of the signals `sig1` or `sig2` makes a transition, but not if both make a transition at the same time.

Constants using these values must be specified with an explicit radix notation of either binary, octal, or hexadecimal.

When specified as part of a multi digit constant, each of the letters can represent either 1 bit, 3 bits or 4 bits depending on the radix, and you can mix in the same constant both numeric digits and any of the special non-numeric constants. For example, the expression:

```
aa[11:0] == 12'h5PX
```

is equivalent to:

```
aa[11:0] == 12'b0101PPPPXXXX
```

Comparing anything to the value X always yields a result of 1. This is useful to mask out selected bits when comparing a vector to a constant.

The above expression can therefore be re-written also as:

```
(aa[11:8] == 5) && (aa[7:4] == 4'bPPPP)
```

Note that the least significant 4 binary digits are X (don't care), so the sub-expression (`aa[3:0] == 4'bXXXX`) became redundant and was omitted altogether.

A typical use of the S (stable) value is to test for a change (or transition) in the value of a vector (meaning change in any of its bits). This is done in practice by checking whether all of the bits are stable, and negating the result, as in the following example:

UXE User Guide

Controlling the Run with SDL

```
TRIGGER = !(myvec[16:0] == 'hSSSS); //trigger on any change
```

Note: When comparing the value of a design signal with any of the special values except X (that is, any one of: P,N,T,H,L,S), there are some limitations that must be observed:

- If the design was compiled to run in 1x or 0.5x sampling mode, it is possible that a signal will make two or more transitions within the same FCLK cycle. For details about how SDL handles this, refer to the [Limitations when Running with 1x Sampling Ratio](#) and [Limitations when Running with 0.5x Sampling Ratio](#) sections.
- Performing a deposit/force/release through XEL on a signal can cause SDL to evaluate incorrectly any condition with special values that depend on that signal or other signals that it drives through a combinatorial network.
- SDL assumes on the first cycle of each run (except a `run` command with the `-continue` option) that values of design signals did not change from the previous cycle, regardless of what the value in the previous cycle was. In other words, an expression such as `(signal == 'bP)` is always evaluated on the first cycle of the run as false and `(signal == 'bS)` as true. This only affects the `ACQUIRE` or `NO_ACQUIRE` actions and does not affect other types of actions because all other types of actions start to evaluate only on the second cycle of the run.
- Creating an expression that tries to nest conditions with special values except X (that is, any one of: P,N,T,H,L,S), might not evaluate correctly on the first cycle of the run (except the `run -continue` command) because it requires knowledge of signal values for more than one cycle in the past. For example, an expression such as: `((signal == 'bs) == 'bp)` might not evaluate correctly on the first evaluation cycle of SDL. This limitation affects all types of SDL actions.

Operators

Topics discussed in this section include:

- [Using Vector Operands with Different Widths](#)
- [Evaluation Order and Operator Precedence Rules](#)
- [Unary Bitwise Negation \(~\)](#)
- [Unary Logical Negation \(!\)](#)
- [Unary Reduction AND \(&\)](#)
- [Unary Reduction OR \(|\)](#)
- [Magnitude Comparison Operators \(> < >= <=\)](#)

- Equal To (==)
- Not Equal To (!=)
- Bitwise AND (&)
- Bitwise XOR (^)
- Bitwise OR (|)
- Logical AND (&&)
- Logical OR (||)
- Concatenation ({...})

The following types of operators can be used when constructing expressions:

- Boolean operators: `~ ! == != & ^ | && ||`.
- `==` operator: can also be used for specifying Assertion State Conditions.
- Arithmetic comparison operators: `> < >= <=`
- `>` and `<=` operators: can also be used for specifying Counter Conditions.
- Concatenation ({...}) operator.
- parenthesis `()` can be used to change default evaluation order.

All the boolean operators can be used on any names (for example, signal names, aliases and so on), constants or expressions, with the only exception that the special non-numeric constants can only be used with the `==` operator and only against expressions that do not contain Counter Conditions.

Using Vector Operands with Different Widths

SDL handles operations between two vectors that have different widths in a similar way to how Verilog handles unsigned operations, but with some minor exceptions, as explained below.

Verilog semantics say that when evaluating a bit-wise operation (such as `& | ^ == !=`) between two vectors that are unsigned, if the two operands have different widths, then the operand with the smaller width is extended by adding most significant zero elements.

For example, the following expression:

```
A[5:0] == 4'b1111
```

is evaluated by SDL as follows:

```
A[5:0] == 6'b001111
```

SDL has two exceptions to this rule.

- If both operands are not constants, then SDL prints out a warning message if the sizes do not match.

For example, the following expression is legal, and evaluated as explained above, but will produce a warning message anyway:

```
A[5:0] == B[3:0]
```

- The second exception can happen with the == operator when the operand with the smaller width is a constant. If its most significant bit is one of the special non-numeric constants (P,N,T,L,H,S,X), SDL will flag the expression as an error.

For example, the following will be flagged as an error:

```
A[5:0] == 4'bxx11
```

Evaluation Order and Operator Precedence Rules

When several different operators appear in the same expression, the evaluation order is done according to the same precedence rules that are used by C or Verilog. This is depicted by [Table 19-4](#). Operators in the same box have the same precedence. Parenthesis can be used to change evaluation order.

For example, the operator == has higher precedence than &, therefore:

```
A & B == C
```

Is equivalent to:

```
A & (B == C)
```

Table 19-4 Operators Precedence

priority	operator	
highest	~	Unary bitwise negation
	!	Unary logical negation
	&	Unary reduction AND
		Unary reduction OR
	>	greater than
	<	less than
	>=	greater than or equal to
	<=	less than or equal to
	==	equal to
	!=	not equal to
	&	bitwise AND
	^	bitwise XOR
		bitwise OR
	&&	logical AND
		logical OR
lowest		

Unary Bitwise Negation (~)

When applied to a scalar expression this operator performs a boolean NOT (inverse) operation.

When applied to a vector expression, the result is also a vector with each bit inverted individually.

Syntax:

`~ <Expression>`

Unary Logical Negation (!)

When applied to a scalar expression performs a boolean NOT (inverse) operation.

When applied to a vector expression, the result is a single bit with a value of 1 if all the bits in the vector are 0. If any of the bits in the vector are 1, the result is the single bit 0.

Syntax:

`! <Expression>`

Unary Reduction AND (&)

Performs a boolean AND operation between all the elements of a vector expression, producing a single bit.

Syntax:

`& <Expression>`

For example, the expression:

`&A[3:0]`

is equivalent to:

`A[3] & A[2] & A[1] & A[0]`

Unary Reduction OR (|)

Performs a boolean OR operation between all the elements of a vector expression, producing a single bit.

Syntax:

`| <Expression>`

For example, the expression:

`|A[3:0]`

is equivalent to:

`A[3] | A[2] | A[1] | A[0]`

Magnitude Comparison Operators (> < >= <=)

These operators compare between two scalar or vector expressions, and produce a single bit as a result. The result is 0 if the comparison fails, and 1 if the comparison succeeds. Each of the two operands is considered as an unsigned integer.

The number of bits in any one of the two operands is not limited by the SDL syntax, but keep in mind that magnitude comparison operators tax emulation resources more than basic equal/not equal operators, so a large number of bits can cause the SDL run time loader to fail.

Syntax:

Greater Than:

`<Expression> > <Expression>`

Less Than:

`<Expression> < <Expression>`

Greater Than or Equal To:

`<Expression> >= <Expression>`

Less Than or Equal To:

`<Expression> <= <Expression>`

Equal To (==)

Compares two scalar or vector expressions, and produces a single bit as a result.

The result is 1 if the two expressions match, and 0 if not.

Syntax:

`<Expression> == <Expression>`

The == operator can also be used to detect special time dependent conditions (for example, a positive edge). For details, see the [Special Non-Numeric Constants](#) section.

Not Equal To (!=)

Compares two scalar or vector expressions, and produces a single bit as a result.

The result is 0 if the two expressions match, and 1 if not.

Syntax:

<Expression> != <Expression>

Bitwise AND (&)

Performs a boolean AND operation between two expressions. If the operands are vectors, the result is also a vector with the same number of bits as the operands.

Syntax:

<Expression> & <Expression>

Bitwise XOR (^)

Performs a boolean Exclusive OR operation between two expressions. If the operands are vectors, the result is also a vector with the same number of bits as the operands.

Syntax:

<Expression> ^ <Expression>

Bitwise OR (|)

Performs a boolean OR operation between two expressions. If the operands are vectors, the result is also a vector with the same number of bits as the operands.

Syntax:

<Expression> | <Expression>

Logical AND (&&)

This operation produces a single bit as result whether the operands are vectors or scalars.

If the operands are scalars, performs a boolean AND operation.

If any (or both) of the operands is a vector, then it is first converted to a scalar by performing an OR operation between all its bits. The final result is therefore 0 only if all the bits are 0 in at least one of the operands.

Syntax:

<Expression> && <Expression>

Logical OR (||)

This operation produces a single bit as result whether the operands are vectors or scalars.

If the operands are scalars, performs a boolean OR operation.

If any (or both) of the operands is a vector, then it is first converted to a scalar by performing an OR operation between all its bits. The final result is therefore the boolean OR operation between all the bits of both operands.

Syntax:

```
<Expression> || <Expression>
```

Concatenation ({...})

A Verilog-style concatenation operator allows you to concatenate scalars and vectors to create new vectors for use in expressions.

Syntax:

```
{<name>, <name>, ...}
```

Example:

```
{a,b,c,d,f[3:0]}
```

The result is an 8 bit vector with signal `a` as the most significant bit, and `f[0]` as the least significant bit. It is equivalent to:

```
{a,b,c,d,f[3],f[2],f[1],f[0]}
```

Elements inside the concatenation can only be signal names and symbols that were defined by the `symbol` XEL command.

Numeric constants, Aliases, or other expressions are not allowed as elements inside concatenation.

Global or Instance Definition Sections

Topics discussed in this section include:

- [Global Definition Section](#)
- [Instance Definition Section](#)
- [Global Definitions](#)

- Alias
- Trigger Position
- Post Trigger Samples
- Global TRIGGER
- Global ACQUIRE
- Global NO_ACQUIRE
- Global EXEC
- Global DISPLAY
- Count Enable
- Counter Load Value

Global Definition Section

The Global Definition section appears at the top of the SDL program above the first INSTANCE Statement. Its definitions apply to all the SDL instances.

Instance Definition Section

The Instance Definition section appears between the INSTANCE statement and the first STATE statement of that instance. Its definitions apply only to that instance.

See Figure 19-1 on page 964 as an example.

Global Definitions

Global definitions in an SDL program are statements that appear either in the Global Definition section, or in the Instance Definition section.

If the SDL program has only one instance with no INSTANCE statement, then the Global Definition section and the Instance Definition section are the same thing.

Except Alias definitions, expressions used in all other global definitions cannot rely on any state machine resources. This means they cannot contain counter values.

Most definitions have the following syntax:

```
<name> = <value>;
```

UXE User Guide

Controlling the Run with SDL

`<name>` can be one of the keywords: `TRIGGER`, `ACQUIRE`, `NO_ACQUIRE`, `COUNTER1`, `COUNTER2`, `ENABLE1`, `ENABLE2`, `TRIGGERPOS`, `POSTTRIGGERSAMPLES`, or it can define a new symbol called Alias which can be used later as a substitute for an expression.

`<value>` is either a numeric value or an expression depending on what is being defined.

Global definitions also support `TRIGGER`, `DISPLAY`, `ACQUIRE`, `NO_ACQUIRE`, and `EXEC` statements with an `IF()` clause (similar to the syntax inside an SDL state machine).

Each of the possible global definitions is discussed below.

Alias

An alias defines a new name which can be used later instead of typing the whole expression. The expression used for defining an alias can itself contain names of other aliases that were already defined.

Syntax:

```
<name> = <Expression>;
```

`<name>` can contain only alphanumeric characters or an underscore (`_`), and cannot begin with a digit. Names of aliases are case sensitive.

An alias appearing in a Global Definition Section can be used anywhere below that definition, including all the SDL instances. An alias definition in an Instance Definition Section is private to that Instance.

The same alias name can be used to define a different alias value in different instances. However, if an alias was defined in the Global Definition Section, then it cannot be redefined in any Instance Definition Section.

Trigger Position

SDL can override the value of the `triggerPos` run-time parameter by providing a different value in the Global Definition Section.

The syntax is:

```
triggerPos = <value>;
```

The keyword `triggerPos` can appear in lower, upper, or mixed case. `<value>` is an integer number between 0 and 100. This value is assigned to the `triggerPos` run-time parameter (overwriting any previous value) when the `run` command is executed, and it is used during the run.

A `triggerPos` definition can appear only in the Global Definition Section, but not in the Instance Definition Section. Both `triggerPos` and `postTriggerSamples` definitions cannot appear together in the same SDL program.

For more information on the semantics of the `triggerPos` run-time parameter, refer to the description of the `xeset` command in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

Post Trigger Samples

SDL can override the value of the `postTriggerSamples` run-time parameter by providing a different value in the Global Definition Section.

The syntax is:

```
postTriggerSamples = <value>;
```

The `postTriggerSamples` keyword can appear in lower, upper or mixed case. `<value>` is a non-negative integer number. This value is assigned to the `postTriggerSamples` run-time parameter (overwriting any previous value) when the `run` command is executed, and it is used during the run.

A `postTriggerSamples` definition can appear only in the Global Definition Section, but not in the Instance Definition Section. Both `triggerPos` and `postTriggerSamples` definitions cannot appear together in the same SDL program.

For more information on the semantics of the `postTriggerSamples` run-time parameter, refer to the `xeset` command in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

Global TRIGGER

Defines a trigger condition. The trigger is executed whenever the specified expression becomes true regardless of any other trigger statements. Any number of global trigger statements can be declared in any of the Global Definition Section or Instance Definition Section, and each is evaluated independently every cycle. A true condition in any of them will cause a trigger. The condition can be constructed as explained in the Expressions section above, except that it cannot contain counter conditions (for example, something like `COUNTER1 <= 0`).

Syntax:

```
TRIGGER = <Expression>;
```

or:

UXE User Guide

Controlling the Run with SDL

```
<Label>: TRIGGER = <Expression>;
```

The above two constructs have an equivalent form using an IF () clause as follows:

```
IF(<Expression>)  
    TRIGGER;
```

or:

```
IF(<Expression>)  
    <Label>: TRIGGER;
```

The shorthand notation TRIG is also accepted instead of TRIGGER.

<Label> is a string made of letters, digits, or underscores, and must start with a letter or underscore. It can be used by XEL for the purpose of identifying, enabling or disabling any TRIGGER, DISPLAY, or EXEC statements that have a specific label. <Label> does not have to be unique within the SDL program.

Global ACQUIRE

Defines a condition which when true in any cycle, will cause probed data capturing in that cycle. Any number of global ACQUIRE statements can be declared in any of the [Global Definition Section](#) or [Instance Definition Section](#), and each is evaluated independently every cycle. A true condition in any one of them will cause data capturing in that cycle. The condition can be constructed as explained in the [Expressions](#) section above, except that it cannot contain counter conditions (for example, something like COUNTER1 <= 0).

ACQUIRE and NO_ACQUIRE statements (whether inside a STATE or a global/instance definition) cannot be used in the same SDL program.

Syntax:

```
ACQUIRE = <Expression>;
```

or an equivalent form:

```
IF(<Expression>)  
    ACQUIRE;
```

The shorthand notations ACQ is also accepted instead of ACQUIRE.

Global NO_ACQUIRE

Defines a condition which when true in any cycle, will suspend probed data capturing in that cycle. Any number of global NO_ACQUIRE statements can be declared in any of the [Global Definition Section](#) or [Instance Definition Section](#), and each is evaluated independently every cycle. A true condition in any one of them will suspend data capturing in that cycle. The

UXE User Guide

Controlling the Run with SDL

condition can be constructed as explained in the [Expressions](#) section above, except that it cannot contain counter conditions (for example, something like `COUNTER1 <= 0`).

`ACQUIRE` and `NO_ACQUIRE` statements (whether inside a `STATE` or a global/instance definition) cannot be used in the same SDL program.

Syntax:

```
NO_ACQUIRE = <Expression>;
```

or an equivalent form:

```
IF(<Expression>)
    NO_ACQUIRE;
```

The shorthand notations `NOACQUIRE` and `NACQ` are also accepted instead of `NO_ACQUIRE`.

Global EXEC

Defines a condition and an XEL command (or set of commands). The condition, when true, will initiate execution of the XEL command. In emulation modes that allow stopping the clocks, emulation clocks are suspended while the command executes. Refer to section [EXEC](#) for more details on the `EXEC` facility.

The condition can be constructed as explained in the [Expressions](#) section above, except that it cannot contain counter conditions (for example, something like `COUNTER1 <= 0`).

Syntax:

```
IF(<Expression>)
    EXEC "<command>";
```

or

```
IF(<Expression>)
    <Label>: EXEC "<command>";
```

`<Label>` is a string made of letters, digits, or underscores, and must start with a letter or underscore. It can be used by XEL for the purpose of identifying, enabling or disabling any `TRIGGER`, `DISPLAY`, or `EXEC` statements that have a specific label. `<Label>` does not have to be unique within the SDL program.

Global DISPLAY

Defines a condition and a message to be printed. The condition, when true, generates a message that is printed to a file and/or the console.

UXE User Guide

Controlling the Run with SDL

Syntax:

```
IF(<Expression>)  
    DISPLAY ("<format-specifier>", <Arg>, <Arg>, ...);
```

or

```
IF(<Expression>)  
    <Label>: DISPLAY ("<format-specifier>", <Arg>, <Arg>, ...);
```

<Expression> is a condition. The DISPLAY message is printed whenever this <Expression> is true. It can be constructed as explained in the [Expressions](#) section above, except that it cannot contain counter conditions (such as, COUNTER1 <= 0).

<Arg>, <Arg> ... are optional arguments to be printed. Each is an expression as explained in the [Expressions](#) section above, except that they cannot contain counter conditions.

<Label> is a string made of letters, digits or underscores and must start with a letter or underscore. It can be used by XEL for the purpose of identifying, enabling or disabling any TRIGGER, DISPLAY, or EXEC statements that have a specific label. <Label> does not have to be unique within the SDL program.

For more details on this statement, see [DISPLAY](#).

Count Enable

Defines a condition upon which one of the two counters will decrement when in auto-decrement mode (using the [START](#) and [STOP](#) actions). The [Global Definition Section](#) or [Instance Definition Section](#) can contain at most one count enable definition for each counter.

A count enable definition appearing in the Global Definition Section applies to all SDL instances. A count enable definition in an Instance Definition Section is private to that Instance.

Different enables can be defined in different instances. However, if an enable definition appears in the Global Definition Section, then it cannot be redefined in any Instance Definition Section.

If no enable definition exists for a specific counter, then that counter will decrement once every FCLK cycle (when in auto-decrement mode).

Note that VD and ICE run modes have clear semantics that define a linear relationship between FCLK cycles and simulation time.

This construct is therefore most useful for Simulation Acceleration mode which does not have such clear semantics.

Syntax:

```
ENABLE1 = <expression>;
```

or:

```
ENABLE2 = <expression>;
```

A typical use is decrementing on a specific clock edge from the design, as in the following example (count on positive edge of `mainclock`):

```
ENABLE1 = mainclock == 'bp;
```

Counter Load Value

Defines the default numeric value for a LOAD action if none was specified with the `Load` action itself. The Global Definition Section or Instance Definition Section can contain at most one counter load value definition for each counter.

A load value definition appearing in the Global Definition Section applies to all SDL instances. A load value in an Instance Definition Section is private to that Instance.

Different load values can be defined in different instances. However, if a load value was defined in the Global Definition Section, then it cannot be redefined in any Instance Definition Section.

Syntax:

```
COUNTER1 = <value>;
```

or:

```
COUNTER2 = <value>;
```

The shorthand notations `CNTR1` and `CNTR2` are also recognized.

`<value>` is an unsigned integer value between 0 and $2^{40}-1$. The value can be specified as a decimal number or in a different radix (as specified in the section Numeric Constants on page 972).

INSTANCE Statement

Defines the beginning of a new SDL instance, and assigns a name to the instance. The name is used for identification by XEL, but other than that is not being used inside the SDL program

itself. See [Setup during Design Compilation](#) on page 958 for information on how to use more than one instance.

Syntax:

```
instance <InstanceName>;
```

<InstanceName> is a string made of letters, digits or underscores and must start with a letter or underscore.

STATE Statement

Declares the start of a state definition. Each state performs actions based on conditions and tests.

You can declare up to 64 states. The first state in the SDL program is the starting state. Only one state is active in any given cycle.

Syntax:

```
STATE <StateName>
{
    <StateBody>
}
```

where <StateName> is a string, and *<StateBody>* is a set of IF-ELSE statements or Action statements. The *<StateName>* is a string made of letters, digits or underscores and must start with a letter or underscore.

IF - ELSE statement

A basic IF statement evaluates an expression. If the expression is TRUE, then it executes a statement, or a set of statements. An ELSE statement can also be specified to execute a different set of statements if the expression evaluated to FALSE.

Syntax:

```
IF ( <Expression> )
    <Statement>
```

or:

```
IF (<Expression> )
{
    <Statement>
    <Statement>
    . . .
}
```

An ELSE statement can immediately follow an IF statement.

Syntax:

```
ELSE
    <Statement>
```

or:

```
ELSE
{
    <Statement>
    <Statement>
    . . .
}
```

<Expression> can be scalar or vector. A scalar expression is considered TRUE if it has the value 1. A vector expression is considered TRUE if any of its bits is 1.

<Statement> can be another IF-ELSE statement or any Action statement (See [Examples: SDL Program](#) on page 1001). There are no restrictions on depth of nesting of IF-ELSE statements, although there are some limitations on the maximal number of IF statements that can be used in each state.

When an IF statement is the only statement nested inside an ELSE block, then no braces are needed for the ELSE block, and the nesting takes the well known structure of IF-ELSE-IF chain. A nested IF chain can be written as follows:

```
IF (<expression1>)
    <action1>
ELSE IF (<expression2>)
    <action2>
ELSE IF (<expression3>)
    <action3>
ELSE
    <action4>
```

The expressions are evaluated from top to bottom, and the action attached to the first expression that is true is executed.

The following truth table shows how SDL interprets the above example:

<expression1>	<expression2>	<expression3>	Action Executed
true	don't care	don't care	<action1>
false	true	don't care	<action2>
false	false	true	<action3>

UXE User Guide

Controlling the Run with SDL

<expression1>	<expression2>	<expression3>	Action Executed
false	false	false	<action4>

A state can contain more than one chain of `IF-ELSE-IF` constructs. Independent `IF` statements are executed concurrently. If there is a possibility for a conflict between actions from different `IF` statements within the same state (for example, `GOTO` actions to different destinations), then the SDL compiler will issue a warning message, and the first action from the top will be executed. Obvious cases, in which one of the actions has no chance of ever getting executed will be flagged as errors.

Limitations Related to Expressions and IF Statements

SDL has an inherent limit on the maximal number of `IF` statements in each state, which is unrelated to how these `IF` statements are nested. The limit depends on the presence of counter conditions in the expressions inside the `IF` statements. Without counter conditions, the rule is simple: there can be no more than 8 expressions (or `IF` statements) in each state. There is no limit on the number of expressions that are constructed exclusively of counter conditions. If an expression contains both counter conditions and signal names, the SDL compiler tries to convert the expression into several sub-expressions that are either counter conditions, or expressions based on signal names. The resulting number of sub-expressions that are based only on signal names add up towards the maximal limit of 8. These restrictions do not apply to `IF` statements in the global or instance definition sections.

The SDL compiler might also fail to download complex expressions into the emulator due to lack of sufficient emulation resources to emulate these expressions. When this happens, an error message will suggest to increase the step count per `FCLK` cycle. To change the step count, see description of the `-min_steps` option of the `compile XEL` command in the *Compile-time Commands in ICE Mode* chapter of the *UXE Command Reference Manual*.

Actions

The following actions can be called from inside STATE Statements:

- GOTO
- LOAD
- DECREMENT

- INCREMENT
- START
- STOP
- TRIGGER
- ACQUIRE
- NO_ACQUIRE
- EXEC
- DISPLAY
- TRACE_TSM

GOTO

Transfer execution from the current state to the specified state, in the next cycle.

Syntax:

```
GOTO <StateName> ;
```

where <StateName> is the name of a state.

LOAD

Loads one of the counters with an initial value. Each `LOAD` action can use a different load value. The `LOAD` action must be executed before executing a `START` or `DECREMENT` actions in order to guarantee that the counter starts from a known value.

Syntax:

```
LOAD COUNTER1 <value>;
```

or:

```
LOAD COUNTER2 <value>;
```

<value> is an integer value between 0 and $(2^{40}-1)$. The value can be specified as a decimal number or in a different radix (see section [Numeric Constants](#) on page 972).

If <value> is not specified, SDL uses a default value as explained in the section [Counter Load Value](#) on page 988.

The counter is loaded with the new value on the cycle that immediately follows the LOAD action.

If SDL previously executed a *START* action, and did not execute a *STOP* action, or if a *START* action was executed concurrently with the Load action, then the counter is first loaded with the new value, and begins decrementing in the cycle after that.

For example, if at cycle 100 the following two actions were executed:

```
START COUNTER1  
LOAD COUNTER1 20
```

then counter1 will show the value 20 in cycle 101, and the value 19 in cycle 102.

DECREMENT

Decrements the specified counter by 1. This is done unconditionally regardless of the Count Enable. This statement is meaningful only when the counter is in a stopped state. If a counter is auto-decrementing because a previous *START* statement defined it as free running and its enable control is true, the *DECREMENT* statement has no effect on the counter.

Syntax:

```
DECREMENT COUNTER1 ;
```

or:

```
DECREMENT COUNTER2 ;
```

When the counter reaches 0, decrementing stops.

The keywords *DECR* and *COUNT* are recognized as synonyms for *DECREMENT*.

INCREMENT

Increments the specified counter by 1 unconditionally. If a counter is auto-decrementing because a previous *START* statement defined it as free running, the *INCREMENT* statement overrides the default auto-decrementing and causes the counter to increment instead of decrementing in that cycle.

Syntax:

```
INCREMENT COUNTER1 ;
```

or:

```
INCREMENT COUNTER2 ;
```

When the counter reaches its maximum value of $(2^{40}-1)$, incrementing stops.

The keyword `INC` is recognized as a synonym for `INCREMENT`.

START

Puts the counter in auto-decrement mode. In this mode, the counter decrements by 1 each time when the expression represented by the Count Enable is true. If no count enable is specified, then the counter is decremented once every cycle. When the counter reaches 0, decrementing stops, but only the STOP action takes the counter out of the auto-decrement mode.

Syntax:

```
START COUNTER1 ;
```

or:

```
START COUNTER2 ;
```

STOP

Stops the specified counter from auto-decrementing.

Syntax:

```
STOP COUNTER1 ;
```

or:

```
STOP COUNTER2 ;
```

TRIGGER

The effect of the `TRIGGER` action depends on the operation mode.

When running in LA mode, or in Simulation Acceleration mode the `TRIGGER` action causes probed data tracing to stop after the specified amount of post trigger samples are captured into the trace buffer. After trigger the SDL program continues to run, so the `NO_ACQUIRE` action can still control what data samples should be traced into memory (if the number of requested post-trigger samples is non zero).

When running in Vector Debug, or STB modes, the trigger cause the design running on the emulator to stop (after the number of requested post-trigger samples is captured into the trace buffer).

Syntax:

TRIGGER;

or:

<Label>: TRIGGER;

<Label> is a string made of letters, digits or underscores and must start with a letter or underscore. It can be used by XEL for the purpose of identifying, enabling or disabling any TRIGGER, DISPLAY, or EXEC statements that have a specific label. <Label> does not have to be unique within the SDL program.

The effect of the TRIGGER action is independent of the Global TRIGGER. Each can cause a trigger independently.

When running in SA or STB mode, the SDL examines design signals with some delay. For precision debugging, you can eliminate the delay by running at a lower speed. See section Fast versus Accurate on page 950 for more details.

ACQUIRE

Allows probed data sampling in the current cycle. Use this action to more-selectively fill the trace buffer.

Syntax:

ACQUIRE ;

When using the ACQUIRE action, only cycles in which an ACQUIRE action (or a global acquire) is executed will capture a probe sample.

The use of the ACQUIRE action precludes the use of the NO_ACQUIRE action or the global NO_ACQUIRE definition.

NO_ACQUIRE

Temporarily prevents probed data sampling in the current cycle. Use this command to more-selectively fill the trace buffer.

Syntax:

NO_ACQUIRE ;

The effect of this action is independent of the Global NO ACQUIRE. Each can prevent data sampling independently.

The use of `NO_ACQUIRE` precludes the use of the `ACQUIRE` action or the global `ACQUIRE` definition.

EXEC

This action cause execution of a Tcl or XEL command, as if it was executed from the `XE` command line prompt. In emulation modes that allow stopping the clocks, emulation clocks are suspended while the command executes. This feature is available in STB and LA modes and also under IXCOM with either static or dynamic target. It is not available when running in infiniTrace or VD modes.

Syntax:

```
EXEC "<command>";
```

or:

```
<Label>: EXEC "<command>";
```

`<Label>` is a string made of letters, digits or underscores and must start with a letter or underscore. It can be used by XEL for the purpose of identifying, enabling or disabling any `TRIGGER`, `DISPLAY`, or `EXEC` statements that have a specific label. `<Label>` does not have to be unique within the SDL program.

`<command>` represents almost any string that can be executed by the XEL command interpreter during a debug session, including calls to Tcl procedures that were defined earlier, or even a sequence of commands separated by semicolons.

For example, suppose that `my_proc` is a Tcl procedure that takes a single numeric argument and was already defined. The following prints a short message, then calls the procedure with an argument 11.

```
EXEC "puts {executing my_proc} ; my_proc 11";
```

The following restrictions apply:

Since the whole command string needs to be enclosed in double quotes, it cannot contain itself any double quotes unless they are properly escaped with a `'\'` (Following normal Tcl escape rules). For example:

```
EXEC "puts \"time now is: [clock format [clock seconds] -format %c]\"";
```

Any Tcl variable reference that appears inside the `<command>` string (with the syntax `$var` or `${var}`) is substituted with its value at the time when the `EXEC` action is executed. For example consider the following snippet from an SDL program:

```
IF (signalx == 1)
    EXEC "puts \" xyz = $xyz\"; incr xyz";
```


UXE User Guide

Controlling the Run with SDL

If the Tcl variable `xyz` was initialized before the `run` command to the value 5 and the `EXEC` action is executed 3 times during the run, then the following output is produced:

```
5
6
7
```

`EXEC` actions are always executed at global Tcl call level. In other words, variables that are local to a Tcl proc are not accessible to `EXEC`.

Some XEL commands that are normally allowed in a debug session, cannot be executed by an `EXEC`, whether they are called directly from the `EXEC` statement, or indirectly from a Tcl procedure that was called from the `EXEC` statement. The list of XEL commands that are not allowed to be executed from an `EXEC` is:

- `configPM`
- `debug`
- `download`
- `host`
- All `infiniTrace` command options except:
 - `-prepare -on -off -close`
- `restart`
- `reset`
- `run`
- `save`
- `vector`
- `waitWhileBusy`
- `xc`
- The following options of the `sd1` command:
 - `-setFile -traceOn -verify -expression -restoreState -load -onlyReport`

`EXEC` requests are serviced whether running in blocking or non-blocking mode.

When running in emulation modes that allow to stop the clocks, SDL examines design signals with a delay of up to 2 Fast Clock (FCLK) cycles. This means that any `EXEC` that is based on values of design signals will be delayed by up to two FCLK cycles.

For precision debugging, you can eliminate the delay by running at 1/3 of the regular speed. See section [Fast versus Accurate](#) on page 950 for more details.

When running in emulation modes that do not allow to stop the clocks, such as LA mode, or Dynamic Target mode under IXCOM, clocks are not suspended while EXEC requests are executed, and the execution delay of the EXEC request can be thousands of cycles. Moreover, since the clocks do not stop and EXEC requests are not buffered, it is possible that SDL issues a new EXEC request before the previous one finished execution, and in such case, the new EXEC request is lost. This is called an EXEC overflow condition.

DISPLAY

Prints a message to the console and/or a file, using a syntax similar to the Verilog \$display system task.

Syntax:

```
DISPLAY ("<format-specifier>", <Arg>, <Arg>, ...);
```

or

```
<Label>: DISPLAY ("<format-specifier>", <Arg>, <Arg>, ...);
```

<Arg>, <Arg>, ... are optional arguments to be printed. Each is an expression as explained in the [Expressions](#) section above, except that they cannot contain counter conditions.

<Label> is a string made of letters, digits or underscores and must start with a letter or underscore. It can be used by XEL for the purpose of identifying, enabling or disabling any TRIGGER, DISPLAY, or EXEC statements that have a specific label. <Label> does not have to be unique within the SDL program.

<format-specifier> is a string containing the main text of the message. It may possibly contain escape sequences, which are substituted with other strings according to [Table 19-5](#) below. The sequences %h %d %i %o %b and the arguments <Arg>, <Arg>, ... are matched in sequence.

The format specifier may also contain some special characters using a leading backslash, as shown in [Table 19-5](#). A backslash followed by any other character will be ignored and removed from the output string.

Tcl variable substitution using \$name and \${name} is also supported inside the format specifier. The Tcl variable value substitution is performed at the beginning of each run.

Table 19-5 Escape Sequences in a Display Statement's Format Specifier

Escape Sequence	Explanation
%h or %H	Convert one <i><arg></i> using unsigned hexadecimal conversion.
%d or %D	Convert one <i><arg></i> using unsigned decimal conversion
%i or %I	Convert one <i><arg></i> using signed decimal conversion
%o or %O	Convert one <i><arg></i> using unsigned octal conversion
%b or %B	Convert one <i><arg></i> using unsigned binary conversion
%t or %T	Simulated time
\n	Newline
\t	Tab
\\	The character \
\"	The character "
\\$	The character \$
%%	The character %

The format specifier character can be either lower or upper case. The only difference between lower case and upper case is when using a hexadecimal conversion. The resulting string from %h will be constructed with only lower case characters, while the resulting string from %H will be constructed with only upper case characters.

The field size (number of characters) allocated when substituting an argument is set to the maximal possible number of characters that the argument may require. For example, a %b conversion for an 8 bit argument will take 8 characters, while a %d conversion will take 3 characters (because the maximum decimal value of an 8 bits argument is 255). A %i conversion will take 4 characters because it also needs to account for a sign character.

The value is right justified in the field. Binary, octal and hexadecimal conversions are padded with zeros on the left, while decimal values are padded with spaces. The padding can be eliminated and the result forced into the minimum required field by inserting the digit 0 immediately to the right of the %.

For example, suppose that the 8 bit argument `data[7:0]` represents the decimal value 12 (binary 1100):

UXE User Guide

Controlling the Run with SDL

```
DISPLAY ("DATA=:%d:%0d::%b:%0b:", data[7:0],data[7:0],data[7:0],data[7:0]);
```

Will produce the following printout:

```
DATA=: 12:12::00001100:1100:
```

Note that the conversion rules are similar to those used by the Verilog `$display()` system task, except that `%t` is not matched with any argument, and Verilog does not support `%i` conversion.

The time is formatted using default time units as was set by the `xeset timeUnit XEL` command.

Messages from `DISPLAY` statements are buffered in the emulator's memory. While the emulator is running, the buffer contents are dumped into the workstation several times each second, formatted and sent to the console and/or a file. In emulation modes that allow stopping the clocks, the emulator clock will pause if the buffer becomes full before the workstation has a chance to read it. In Logic Analyzer or Dynamic Target modes, if the rate of writing into the buffer is higher than the rate at which the workstation reads it, then some messages are lost. This is called an overflow condition. You can increase the depth of the buffer during compilation using compiler option `sdlDisplayDepth` in order to address the overflow issue.

The total number of different arguments (`<Arg>`, `<Arg>`, ...) that can be used in all `DISPLAY` statements depends on the width of the display buffer in the hardware. If the same signal appears as an argument in more than one `DISPLAY` statement in the SDL program, that signal is counted only once for the purpose of calculating the required buffer width. The width of the display buffer can be increased at compile time using compiler option `sdlDisplayWidth`.

Several aspects of the `DISPLAY` facility can be controlled from XEL through the `sdl` command. For example, you can direct the messages to a file instead of (or in addition to) the console.

TRACE_TSM

This action will cause the current clock cycle to be unconditionally traced into the SDL trace memory (which can be later dumped to a file using the `sdl -traceDump XEL` command).

The `TRACE_TSM` action itself is not logged into the SDL trace dump file, and unlike other actions, it cannot be disabled from XEL. (The `sdl -traceOn XEL` command can configure which SDL actions will cause SDL tracing, but it has no effect on `TRACE_TSM`).

Examples: SDL Program

Example 1: A State-Less SDL Program

The following SDL program compares two 8 bit vectors `A[7:0]` and `B[7:0]` as unsigned integer numbers and triggers if the number represented by the vector `A[7:0]` is greater than the number represented by the vector `B[7:0]`.

In addition, the program filters out from captured data all samples in which the low order 4 bits of the vector `A` equal the number 13.

```
if(A[7:0] > B[7:0]) trigger;
if(A[3:0] == 13) no_acquire;
```

An equivalent form of the above, using a different variant of the syntax is:

```
trigger = A[7:0] > B[7:0];
no_acquire = A[3:0] == 13;
```

Example 2: A Single SDL Instance

The following SDL program has two states. It triggers if a specific signal becomes high within 100 cycles or less after `sig1` is high. The search for this pattern is restarted each time when `sig1` remains low for 100 cycles or more.

Note: The name of the net being tested in the first `IF` statement inside `state Two` is taken from the value of a Tcl variable (called `signame`). This allows to change the name of the net being tested from XEL without modifying the SDL file.

```
/* Trigger on sig1 high followed by sig2 high, within 100 cycles.
   The number of cycles is counted by counter1
*/
State One          // wait until sig1 goes high
{
    if (sig1)
    {
        load counter1 99;
        start counter1;
        goto Two;
    }
}

State Two
{
    if ( $signame && Counter1 > 0 )
        trigger ; // trigger if $signame is high while counting
    if ( sig1 )
        load counter1 99; // restart the counter if sig1 high
    if ( Counter1 <= 0 )
        goto One ;      // start all over again
}
```

Example 3: SDL Program with Two Instances

The following SDL program demonstrates use of different SDL constructs. It has two SDL instances, and the program spans on two files.

The bodies of each of the two state machines are exactly the same text, derived from a single file called `instance.tdf`. However, the two state machines execute differently because they rely on the definition of the alias `expr`, which gets a different value in each of the two instances. Each of the two instances define also a global trigger and a global acquire.

```
// Global Definition Section
exp1 = counter1 > 0 || top.ctl.xyz; // an alias
exp2 = top.ctl.bus1[7:0] != 'b10011 || top.ctl.sig1 == 'bs; // another alias
address_plus_data = {address[15:0],data[15:0]}; // alias defined by concatenation
counter2 = 100; // define default load value for counter2
glob_trig1: trigger = top.cyx[7:0] == top.fsq[15:8]; // global trigger #1

// begin first instance definition
Instance INST_1;
expr = address_plus_data == 32'h874A_FFFF; // new alias using previous alias
`include "instance.tdf"

// begin second instance definition
Instance INST_2;
expr = address_plus_data == 0; // new alias using previous alias
`include "instance.tdf"

----- contents of file instance.tdf start below -----
glob_trig2: trigger = expr & !top.enbr; // global trigger (one for each instance)
acquire = !expr; // global acquire definition (one for each instance)
// Definition of States
state first {
    load counter2 $cnx; //load counter from a value defined in a Tcl variable
    load counter1 511;
    if (exp1 && !(top.vrx[3:0] || top.vx[7:0] >= top.vy[7:0]))
        goto second;
    else if (expr) {
        if (top.bxx != 1)
            goto second;
        else
            acquire;
    }
    else
    {
        acquire;
        display ("uploading probes. time=%t");
        exec "database -upload";
    }
}

state second {
    if( {ra,rb,ctrl,ctr2} == 5)
        tsm_trig: trigger;
    if(exp2 == 0 || exp1 && counter2 <= 0) {
        trigger;
        start counter2;
        decrement counter1;
    }
    if(!(top.ctl.bus2[15:8] == 'hps || top.xyz != top.vpprt))
```

```
        goto first;  
    }
```

Importing Breakpoints from the NC-Sim Simulator into SDL

In IXCOM flow, you can use the `stop` command to define on the NC-Sim simulator an arbitrary number of breakpoints that are independent of each other. These breakpoints do not function correctly after swapping the design into the Palladium XP emulator if they rely on design objects that reside in the DUT. It is possible to import some of these breakpoints into an SDL program that can run on the Palladium XP emulator. The only types of breakpoints that can be imported to SDL are `Object` and `Condition`.

The breakpoints import operation is initiated either by the XEL command:

```
sdl -ncbreak -import
```

or automatically during swap-in by setting up the following run-time parameter:

```
xeset sdlAutoNCbreakpoints 1
```

Importing breakpoints involves more than just creating an SDL file. If the imported breakpoints contain `-skip` or `-delete` options, then the skip or delete counts are imported into the xeDebug run-time environment as values of Tcl variables. A successful import operation also loads the imported SDL file into the emulator, as if the `sdl -load` command was executed.

Disabled breakpoints are not imported.

Theory of Operation

The `Object` or `Condition` component of the imported breakpoint is combined with the expression of the `-if` component if one was specified, and the result is used to create the `if(<expression>)` of a global SDL `TRIGGER` and/or `EXEC` statements.

The `skip`, `delete` and `execute` features are handled by SDL `EXEC` statements, while keeping track of the `skip` and `delete` counts is done by elements of a Tcl array. The values in the array represent `skip` and `delete` counters and their values are initialized by the `sdl -ncbreakpoints -import` command.

Expressions in Breakpoints

When breakpoints are imported to SDL, the following modifications are made to the expressions that come from the `if` component or the main condition of a `Condition` type breakpoint:

- References to signal values in the form `#<signal-name>` in expressions are translated to SDL by removing the `#`. The syntax `[value <signal-name>]` is not supported by the import operation.
- The operators `=` and `===` in the original NC-Sim breakpoints are translated to `==`.
- VHDL `STD_LOGIC` and integer constants are translated to SDL format (for example, `"10110"` is translated to `'b10110`).
- Not all operators that are allowed in NC-Sim breakpoints can be imported. For example, NC-Sim allows you to use a multiplication operator inside a breakpoint condition, but this cannot be imported to SDL. See [Table 19-4](#) for a list of operators that are allowed in SDL conditions.

Error Conditions

The following rules decide which breakpoints are to be imported, and which cause the operation to fail:

- Breakpoints that rely only on design objects that are located in the TB (or that do not rely on any design object) remain functional on the simulator, and are not imported.
- Breakpoints of type `Condition` or `Object` are imported only if they use signals from the DUT, and expression syntax that can be processed by SDL.
- In all other cases, the software makes an effort to determine whether the breakpoint in question continues to operate correctly under the NC-Sim simulator after the swap-in.
 - ❑ If it is determined that the breakpoint does not operate correctly, the import operation fails.
 - ❑ If the software cannot determine for sure, then only a warning is printed.

Examples

- If a breakpoint contains an `if` component that uses a mix of TB and DUT objects, an error condition will be generated.

- If a breakpoint uses an expression that compares a DUT signal with a value of `unknown` (`x`), an error is generated because Palladium XP does not support unknowns. Note that, in SDL `'bX` means `don't care`, while `'bX` in a breakpoint means `unknown`.
- An isolation rule breakpoint in the DUT does not function correctly after swap-in, but only generates a warning because the current implementation cannot determine the location of the isolation rule.
- A delta breakpoint almost always behaves differently after swap-in, unless the entire DUT was input in the form of a gate-level netlist, therefore it generates a warning message.

Error Recovery Methods

By default, any error condition during import of any of the breakpoints causes the whole import operation to fail. If the import is done automatically during swap-in (by using `xeset sdlAutoNCbreakpoints 1`), then the swap-in fails and the system reverts to simulation mode.

You can select to ignore breakpoints that generate error conditions during the import, and proceed with any breakpoints that can be imported successfully by using the following command:

```
sdl -ncbreakpoints -errorok 1
```

This affects all future requests to import breakpoints to SDL. In such case, the same error messages are reported (if any error conditions are found), but they do not cause the operation to fail and an SDL file with any breakpoints that could be imported successfully is generated and loaded into the emulator. This also prevents a swap-in from failing due to errors in automatic import.

Limitations and Differences in Behavior

Even breakpoints that were imported successfully to SDL might produce a behavior that is not exactly the same as running under pure simulation. Following are the differences:

- The import operation ignores the options `-silent` and `-noexecout`, although all other options are honored.
- When running with NC-Sim, a breakpoint is completely deleted when it reaches its `delete` count. Under SDL, the corresponding trigger is only disabled when it reaches the delete count.
- Tcl variables that are referenced as `$name` by an `execute` component in the breakpoint reference Tcl variables in the NC-Sim Tcl name space, while the imported SDL program will reference Tcl variables in `xeDebug`'s Tcl name space.

UXE User Guide

Controlling the Run with SDL

- NC-Sim breakpoints cause the simulator to stop immediately at the exact simulation time in which the breakpoint's condition becomes true. When using SDL, such behavior is not guaranteed, and the run might stop with some delay (See section [Fast versus Accurate](#) for more details).
- In order for a Condition type breakpoints to stop the NC-Sim simulator, the expression defined by the condition must be true, and an *event* must happen. The *event* is generated by a change in the value of any object that is used in the expression, and if the expression contains any bit select of a vector (for example `name[17]`), then any change in value of any bit in that vector will also generate an event, whether that bit is included in the expression or not.

The translated SDL program uses by default simpler semantics in order to reduce the amount of emulation resources. With the simplified semantics SDL will stop the run each time when the result of the condition makes a transition from false to true.

It is possible to request that the translated SDL use exactly the same event-driven behavior that NC-Sim uses for breakpoints with the `sdl -ncbreakpoints -exact 1` command.

- Using `uxc_hdl.var` file with xeDebug causes unpredictable behavior. Any breakpoints defined in the `uxc_hdl.var` file might not be imported correctly.

Using Novas Tools

In the ICE flow, Palladium XP allows you to view waveforms of FSDB format using standalone Novas tools, such as Debussy, Verdi, and Siloti. Before using these tools, you need to integrate them with the xeDebug tool as described in the following section.

The integrated standalone Novas tools support the following:

- RTL Verilog, VHDL, and mixed-HDL designs
- Gate-level designs (for backward compatibility)
- Verigen's trace output verilog files (for backward compatibility)
- HDL Compiler generated vericom/vhdlcom scripts
- FSDB waveform database for both RTL and Gate-level designs without setting the verilog delimiter rule or any additional compiler options

Note: VBF format is not supported.

Integrating Novas Tools with xeDebug

Before opening the waveform database generated using xeDebug, determine the installation of the Novas tools (Debussy, Verdi, or Siloti). Set the Novas license file and other Novas-related environment variables. In addition, specify the executable name (debussy, verdi, or siloti) by setting the `NOVAS_EXE` environment variable.

Setting the Environment Variables for Running Novas Tools

Ensure that the required environment variables have been set as following:

```
setenv NOVAS_HOME <novas_tool_installation_directory>
setenv TURBO_ETC_DIR $NOVAS_HOME/etc/access

# set Palladium schematic primitives
setenv TURBO_LIBS qds_u
```

UXE User Guide

Using Novas Tools

```
# set Palladium XP schematic symbols path for 64bit design
setenv TURBO_LIBPATHS $QTHOME/share/uxe/gift/novas_schematic/64
setenv NOVAS_LIBPATHS $QTHOME/share/uxe/gift/novas_schematic/64
```

```
#set Novas tool's license file
setenv LM_LICENSE_FILE $NOVAS_HOME/license/license.dat:$LM_LICENSE_FILE
set path = ($NOVAS_HOME/bin $path)
```

```
# set NOVAS_EXE for xeDebug
setenv NOVAS_EXE $NOVAS_HOME/bin/verdi (or debussy or siloti)
```

To run Simulation Acceleration (IXCOM) in fsdb mode, you need to include the following Novas VHPI libraries:

```
# set library path for $fsdb* system task for testbench waveform
setenv LD_LIBRARY_PATH ${LD_LIBRARY_PATH}:$NOVAS_HOME/share/PLI/ius8.1_vhpi/
LINUX64
```

To ensure Palladium XP `libcvanovas.so` can load Novas' `libcfc` and `libfmi` successfully, you need to make the following changes in NOVAS PLI directory (or create your own shared libraries directory):

- Rename the `libpli.so` to `libpli.so.orig` using the following UNIX command:

```
mv libpli.so libpli.so.orig
```

- Link `llibcfc_novas.so` to `libcfc.so` using the following UNIX command:

```
ln -s libcfc.so libcfc_novas.so
```

- Link `libfmi_novas.so` to `libfmi.so` using the following UNIX command:

```
ln -s libfmi.so libfmi_novas.so
```

The PLI directory is located at the following path:

```
$NOVAS_HOME/share/PLI/ius8.1_vhpi/${PLATFORM}
```

The `qtref` cells definition is needed if you use the HDL Compiler output gate file. The definition of `qtref` cells is installed at `$QTHOME/share/uxe/etc/qtref_ext.v`.

You can get `Q_*` schematic symbol database from:

```
$QTHOME/share/uxe/etc/gift/novas_schematic/{64, 32}
qds_l.lib++/
```

and

```
qds_u.lib++/
```

Functions Supported by Novas Tools

After integrating the standalone Novas tools with the xeDebug tool, following debug functions are supported:

- The Novas Browser/Schematic can be opened from the xeDebug tool by using the xeDebug icons or RTL/Gate tab.

Note: xeDebug does not support schematic compiler in the xeDebug RTL/Gate panel. Therefore, before running the xeDebug tool, you need to manually generate the Debussy OH database. The `hdlOutputFile -f debussy <file_name>` command can be used to generate the schematic compiler script. For more information, refer to the [Generating the Debussy OH Database](#) on page 1016.

- The drag and drop functionality is supported among the Novas Source Code Browser, Design Browser, Schematic, and Waveform tools.

Note: Drag and drop between the xeDebug tool and Debussy is not supported.

- Back annotation can be turned on in all Novas Browsers.

Note: Back annotation for flattened verilog files (the Verigen's net trace output) can be supported automatically if it is opened from xeDebug RTL/Gate GUI. If the flattened verilog file needs to be opened using UNIX command manually, use `-play` option to include the name translation function.

Example for the flattened verilog file, `trace_back_20.v`

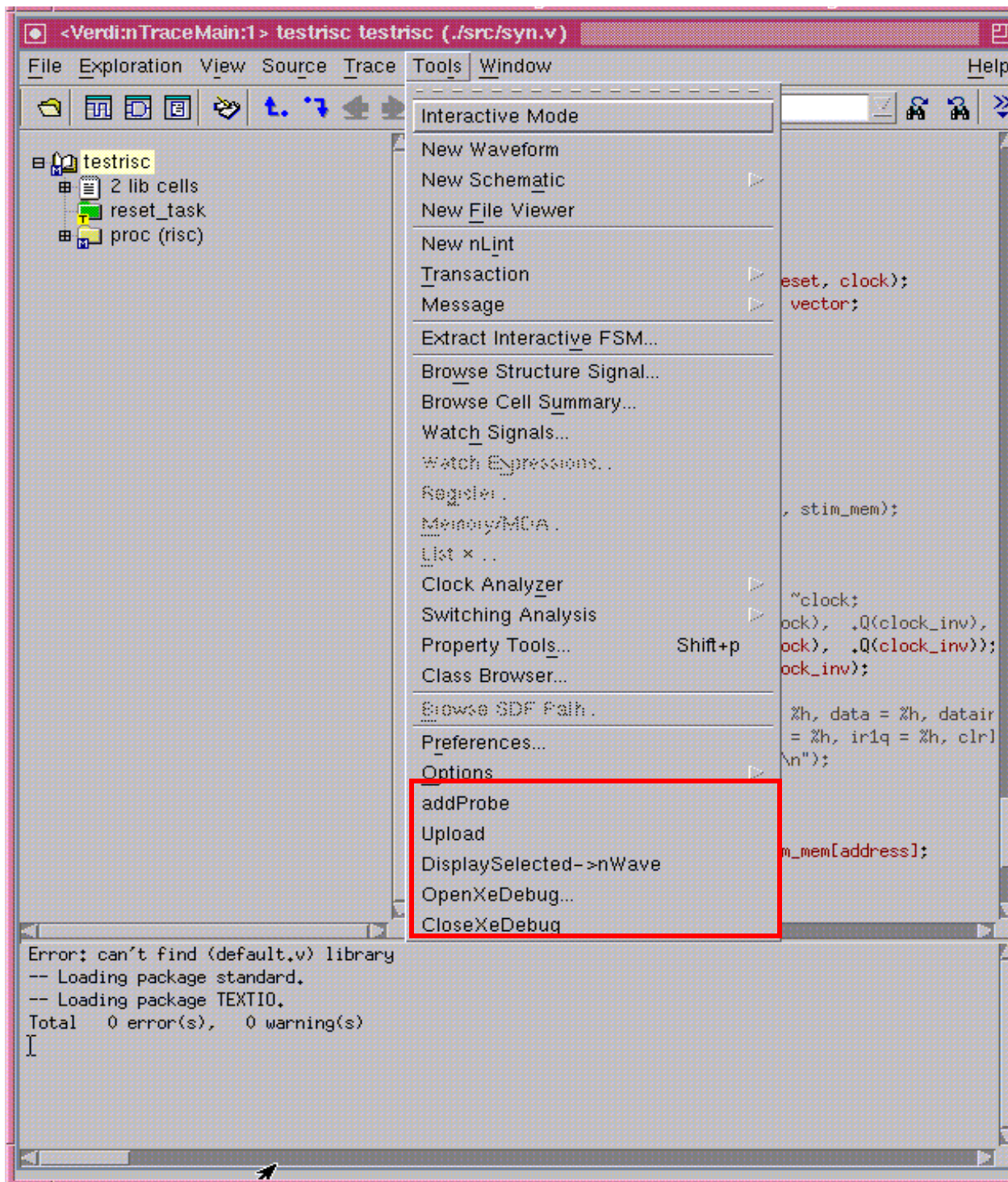
```
% verdi trace_back_20.v -play  
  
<UXE_installation_directory>/share/uxe/etc/xel/tdname.tcl &
```

- The Novas nWave waveform viewer can be integrated into the xeDebug ICE and Vector Debug modes.
- Result waveform databases are automatically compared with the stimulus database and the result mismatches are displayed.
- Following xeDebug run-time functions are available in the pull-down menus of the Novas Source/Schematic/Waveform windows:

UXE User Guide

Using Novas Tools

Figure 20-1 Novas Tool (Verdi) Source Code Browser with xeDebug Functions



- ❑ Novas Source Browser (refer to [Figure 20-1](#)):

UXE User Guide

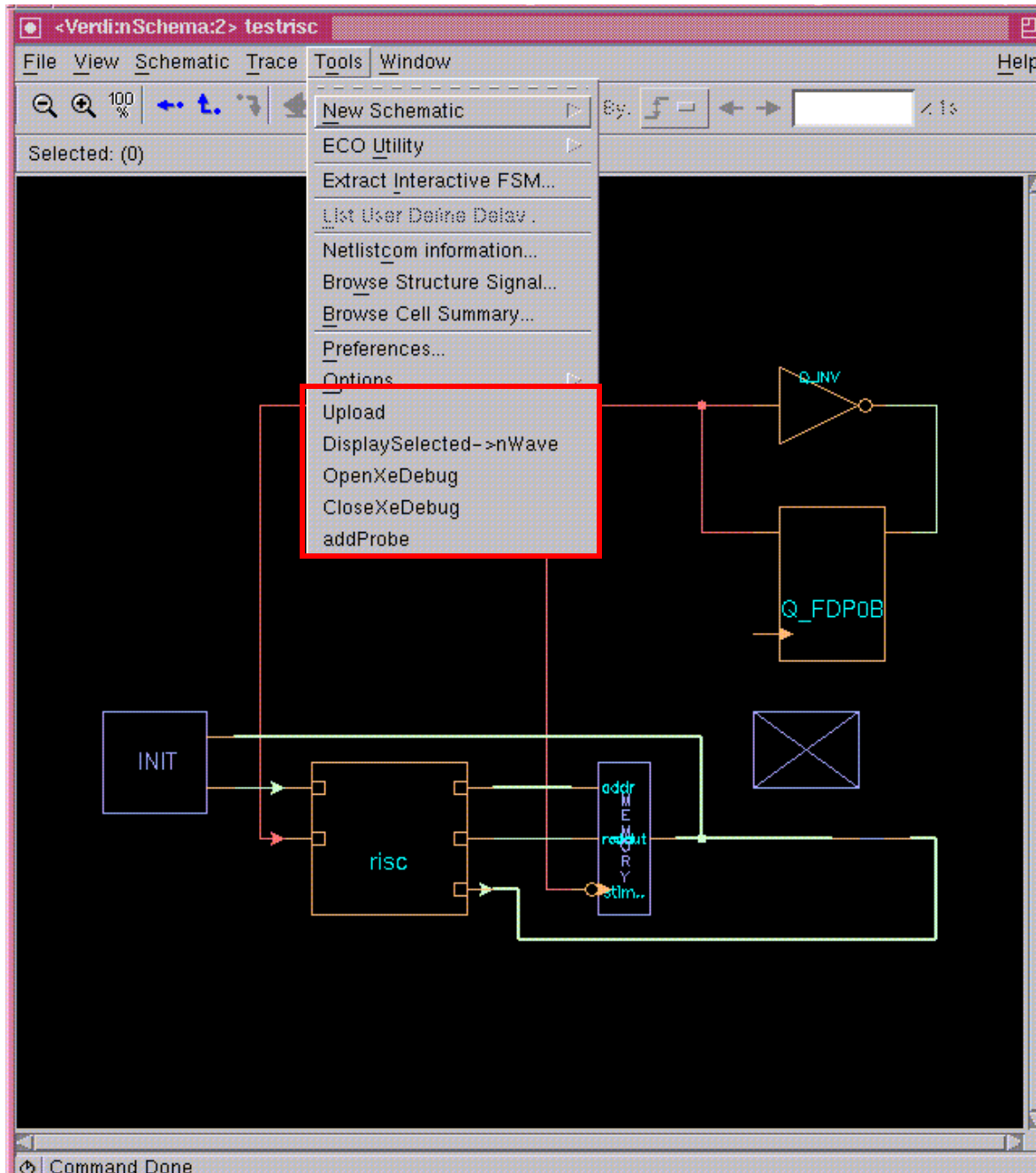
Using Novas Tools

- addProbe
- Upload
- DisplaySelected->nWave
- OpenXeDebug
- CloseXeDebug

UXE User Guide

Using Novas Tools

Figure 20-2 Novas Tool (Verdi) Schematic Browser with xeDebug Functions



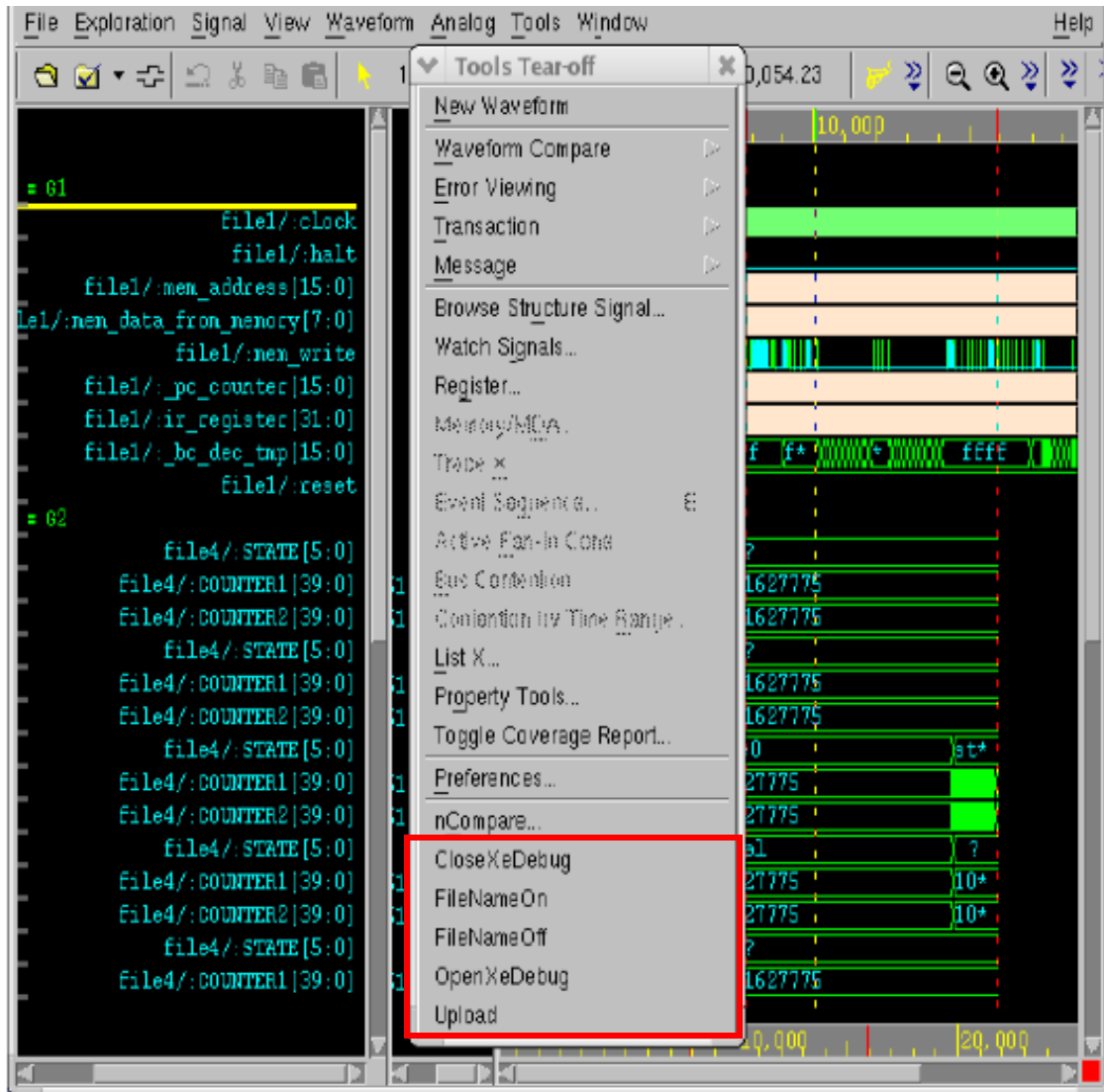
- ❑ Novas Schematic Browser (refer to [Figure 20-2](#)):
 - Upload
 - DisplaySelected->nWave

UXE User Guide

Using Novas Tools

- OpenXeDebug
- CloseXeDebug
- addProbe

Figure 20-3 Novas Tool (Verdi) nWave Browser with xeDebug Functions



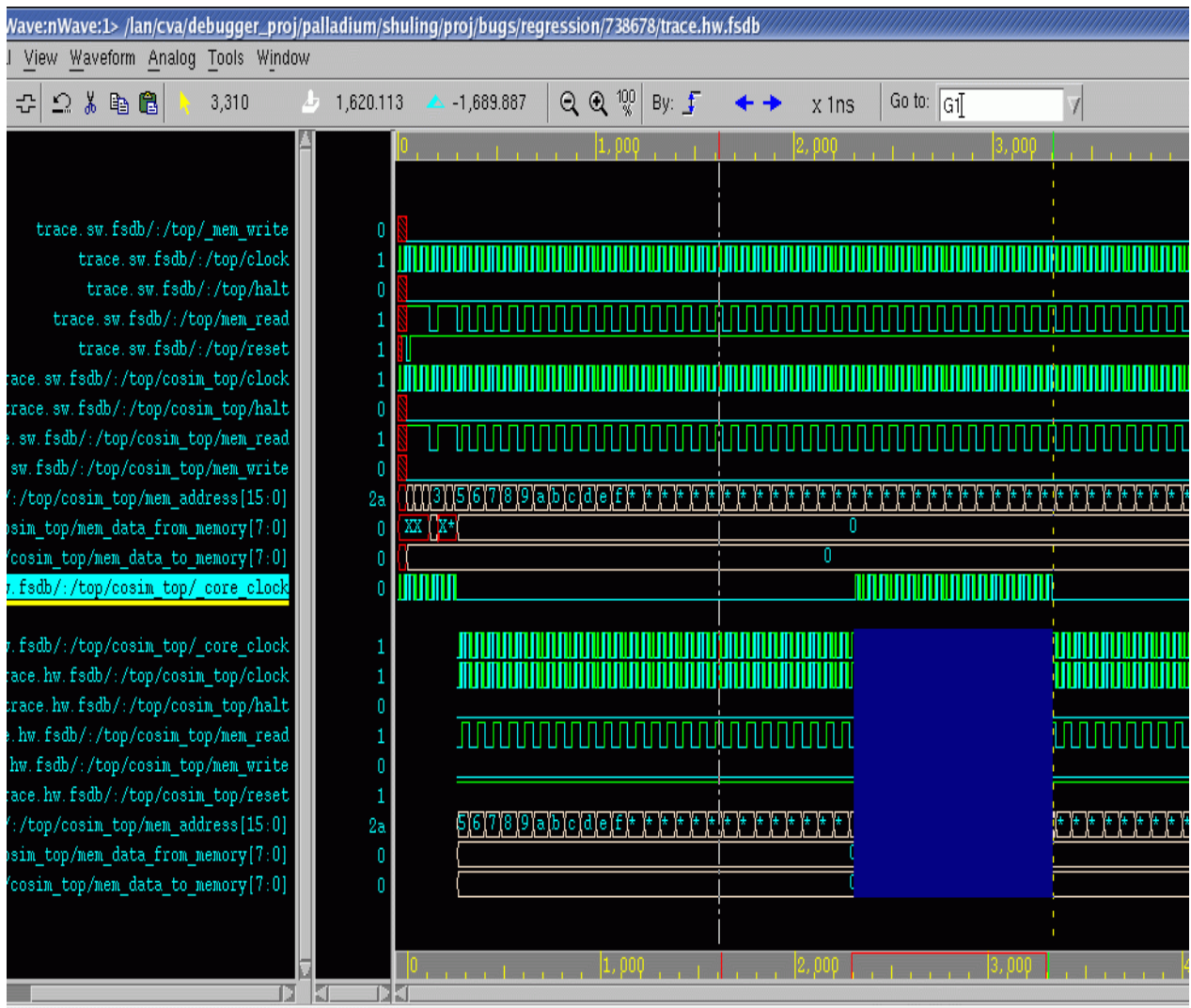
- Novas Waveform Browser (refer to [Figure 20-3](#)):
 - CloseXeDebug
 - FileNameOn

UXE User Guide

Using Novas Tools

- FileNameOff
- OpenXeDebug
- Upload

Figure 20-4 nWave Browser with xeDebug IXCOT Flow



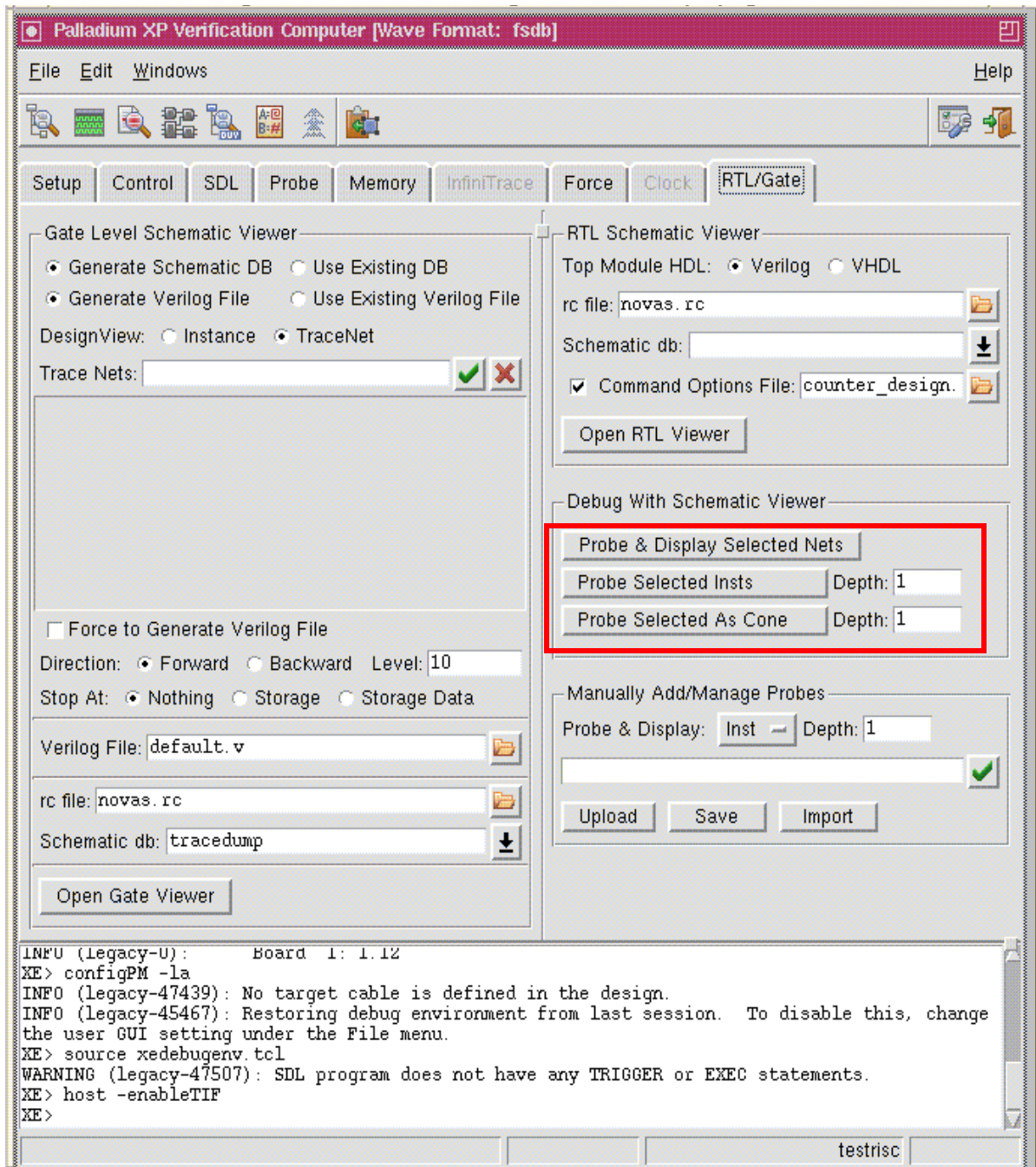
There are two database in IXCOT Flow. One database, named `<database>.sw.fsd` is for dumping all the testbench signals and can include DUT signals. The other database, named `<database>.hw.fsd` is for DUT signals only. If you have added the DUT signals into testbench database before swapping-in to hardware, then after the swapping-in to hardware, the DUT signals in `<database>.sw.fsd` do not change (you have to ignore the

UXE User Guide

Using Novas Tools

value in <database>.sw.fsdb and read the value from DUT database). After swapping-out, the DUT database <database>.hw.fsdb) stops dumping.

Figure 20-5 RTL/Gate Tab in the xeDebug Tool



- RTL and Gate-level schematic databases can be debugged using the *RTL/Gate* tab of the xeDebug tool (refer to [Figure 20-5](#)). The tab provides the following buttons for debugging such databases:

- ☐ Probe & Display Selected Nets
- ☐ Probe Selected Insts
- ☐ Probe Selected Cone buttons for debugging such databases

Before you start debugging an RTL schematic database, you must open it using either of the following ways:

- ☐ Select the check box adjacent to the *Command Options File* field within the *RTL Schematic Viewer* section and specify the name of the input command file, that is, `turbo.f` file. Then click the *Open RTL Viewer* button.
- ☐ Specify the name of the schematic OH database in the *Schematic DB* textbox. Then click the *Open RTL Viewer* button.

Note: The RTL schematic database must be created before running the xeDebug tool.

To open a gate-level schematic database for debugging, use either of the following ways:

- ☐ Select the *Use Existing DB* option button and then specify the name of the required gate-level schematic OH database in the *Schematic DB* textbox.
- ☐ Select the *Use Existing Verilog File* option button and then specify the name of the required verilog input file in the *Verilog File* textbox.
- ☐ Select the *Generate Schematic DB* and/or *Generate Verilog File* option buttons. Then, select the *Instance* option button to create and view instance dump of whole or partial design; or select the *TraceNet* option button to create and view flattened trace results.

Generating the Debussy OH Database

If you use the HDL Compiler tool to generate the Debussy compiler (`vericom` and `vhdlcom`) script for creating the schematic OH database, you need to add the `-f debussy <file_name>` option to the `hdlOutputFile XEL` command as follows:

```
XE> hdlOutputFile -f verilog <verilog_gate> -f debussy <file_name>
```

Alternatively, you can add the `-Debussy <file_name>` option to the `vaelab` HDL Compiler executable.

For example:

```
vaelab <top_module> -OutputVlog <file_name> -Debussy <filename>
```

Or, add `-f debussy <file_name>` option to the `vavlog` HDL Compiler executable.

For more information on `vaelab` and `vavlog`, refer to the *UNIX Command-Line Interface* chapter of the *UXE HDL Compiler and Modeling Guide*.

The following files are created after passing the above mentioned commands:

- A shell file named `debussyRTLImport.sh`
- `<file_name>` text file
- `<file_name>.libDebussy` text file

The `<file_name>.libDebussy` text file contains all the `logical_library` and `physical_library` mapping information. The `<file_name>` text file is the input file for the `debussyRTLImport.sh` script.

After the HDL Compiler has finished the compilation (as illustrated in the [Example of Generating Debussy OH Database Using HDL Compiler](#) section), on the UNIX command prompt, execute `debussyRTLImport.sh` with the `<file_name>` as follows:

```
% debussyRTLImport.sh <file_name> &
```

The above command generates the Debussy OH database and the `<file_name>turbo.f` file. The `<file_name>turbo.f` file is required for opening the waveform database from the RTL/Gate tab of the `xeDebug` tool.

You can open the Debussy OH database from either the RTL/Gate tab of the `xeDebug` tool or the UNIX prompt. In the *RTL/Gate* tab, specify `turbo.f` in the *Command Options File* textbox. The `turbo.f` file is required for opening the Debussy database.

```
# Generating Debussy database
UNIX> debussyRTLImport.sh <file_name> &
# Opening schematic from xeDebug RTL/Gate Panel or UNIX window
## either enter turbo.f to the entry of xeDebug RTL/Gate panel's command file entry
## or run Debussy executable from UNIX prompt as:
UNIX> verdi -f turbo.f &
```

Whether Novas tools are invoked from `xeDebug` or UNIX command, you always need to set the following:

```
NOVAS_HOME and PATH before running xeDebug. Example:
setenv NOVAS_HOME <novas_tool_installation_directory>
setenv TURBO_ETC_DIR $NOVAS_HOME/etc/access
# set Palladium schematic primitives
```

UXE User Guide

Using Novas Tools

```
setenv TURBO_LIBS qds_u
setenv LM_LICENSE_FILE $NOVAS_HOME/license/license.dat:$LM_LICENSE_FILE
set path = ($NOVAS_HOME/bin $path)
# set NOVAS_EXE for xeDebug
setenv NOVAS_EXE $NOVAS_HOME/bin/verdi (or debussy or siloti)
```

Example of Generating Debussy OH Database Using HDL Compiler

```
hdlLang verilog
hdlInputFile -add ./src/counter.v
hdlImport

# Following command generates the debussyRTLImport.sh, counter_design.txt,
# and counter_design.libDebussy files:
hdlOutputFile -f verilog counter_gate.v
hdlOutputFile -add -f debussy counter_design

hdlSynthesize TOP
importOption {mode full}
importOption {library LIB}
refLib xtref generic
netlistFile {verilog ./counter_gate.v}
designImport
design LIB TOP
userData -load src/user.udf
emulatorConfiguration -add {host my_eng2} {boards 0.8}
compilerOption -add {mode ice}
precompile -removeSourceless
compile

# Following command generates the Debussy OH database and the
counter_design.turbo.f file:
debussyRTLImport.sh counter_design &
```

Loop-Breaking

This appendix describes the compiler's loop-breaking process, the problems that might occur, and how you can solve the problems or avoid them.



This appendix is applicable only to the ICE mode.

Definition of Loop-Breaking

The UXE back-end compiler (invoked by the `compile XEL` command) does not tolerate any combinational loops in its input netlist. Therefore, precompile checks the design for combinational loops, and breaks any loop it finds by inserting a delay at a point within the loop. The delay is usually 1 FCLK cycle, but can sometimes be a *half* FCLK cycle as explained below.

A path from flip-flop clock, reset or set to Q, or latch D, enable, set or reset to Q is considered combinational.

Problems Caused by Loop-Breaking

Precompile's loop-breaking is done automatically, and usually you need not be concerned with it. However, there are two situations in which loop-breaking delay could cause a problem.

1. Loop-breaking delay could require extra oversampling.

Normally, for correct function, signal transitions on a path from a flip-flop Q output to another flip-flop D input must propagate within one clock cycle. Occasionally, transitions must propagate within a half clock cycle. Such paths are called single-cycle and half-cycle paths, respectively. If a clock cycle is $2N$ FCLKs long, then $2N$ inserted 1-FCLK delays on a single-cycle path, or N delays on a half-cycle path, will cause the design not to work. In such a case, you can make the design work by increasing the *oversampling ratio* (the number of FCLKs per cycle of the fastest user clock). For example, when clocks

are generated using CAKE technology, to increase the oversampling ratio from 2 to 4, you would specify

```
clockOption -add {technology CAKE 4}
```

to compile in 4X mode, instead of specifying

```
clockOption -add {technology CAKE 2}
```

to compile in 2X mode. Alternatively, without recompiling, you can use the command

```
XE> clockConfig -sample 4
```

at run-time to increase the oversampling ratio to 4.

2. Loop-breaking could cause clock skew.

A delay on a path from a clock source to a flip-flop or latch clock pin can cause clock skew, and cause the design not to work. A failure caused by clock skew cannot be solved by increasing the oversampling ratio. Fortunately, precompile uses heuristics that in most designs, ensure that loop-breaking delay will not be added on a clock path unless there is no other way to break a loop.

Proactive Steps to Avoid Loop-Breaking Problems

You can take the following steps to prevent loop-breaking problems:

- [Steps to Break Loops and Long Paths](#)
- [Miscellaneous Steps for Clock Delays](#)

Steps to Break Loops and Long Paths

You can use the following steps to eliminate or break long combinational paths in a design. These actions reduce the need for loop-breaking and improve the compilation step count, resulting in faster emulation speed.

- Tie the scan or test-mode control signals, if possible. Some designs have scan or test logic that does not need to be tested in emulation. In such designs, tying a scan or test-mode control signal to the non-test value can enable simplification of the design logic, eliminating multiple loops and long paths.
- If the design uses flip-flop library models that implement a flip-flop by using master-slave pairs of latches, remodel these flip-flops, if possible, by using flip-flops from the QTREF library.

- If parts of the design are latch-based, and use non-overlapping clocks, use the `noOverlapClock` User Data command to specify information about the non-overlapping clocks.
- By default, when a flip-flop set or reset input goes active, an immediate effect is seen on the flip-flop output. Use the `precompile -syncReset` option to delay the effect of set or reset going active by one FCLK. This effectively eliminates the combinational path from the flip-flop set or reset to the output. Typically, designs work correctly with the `precompile -syncReset` option, but might have fewer loops, and fewer long paths, thereby, causing a poor step count.
- Use the precompile options, especially the `-dontMinimizeDataBreaks` and `-syncData` options to add extra breaks on the latch D pins. These options reduce the need for loop-breaking, but the extra breaks often cause the design to need extra oversampling. So, it is possible that, overall, using these options does not provide any benefit.

Miscellaneous Steps for Clock Delays

At times, the `xe.msg` file contains a warning like the following:

```
WARNING (multclk-1121): Maximum analysis depth 128 exceeded.
```

This means that the precompiler judged that loopbreaking was taking too long, and abandoning its usual analysis, it used, instead, a simpler algorithm that tends to insert more loop-breaks than are really needed. The simpler algorithm might require extra oversampling, or it might delay clocks, causing incorrect functionality regardless of oversampling. To get the best loopbreaking results, you should use the `precompile -loopsDepth <n>` option, where `<n>` is a value large enough to eliminate the warning. (Usually, 1024 is large enough.) The drawback of using this option is that precompilation might take longer.

You can use the `precompile -recognizeClockDivider` option to reduce the possibility that loopbreaking will delay a clock in a design, which contains clock dividers. This option is not enabled by default, but its use is recommended. You can also try the `precompile -extraClockSources` option to avoid clock delays.

What to Do if a Loop-Breaking Problem is Suspected

If a loop-breaking problem is suspected, first try increasing the CAKE oversampling. If you compiled in 2X mode, you can increase to CAKE 4 or 8 at run time using the `clockConfig` command, without recompiling. If increasing the oversampling solves the problem, it is likely a loop-breaking problem, although it could be a problem caused by delays other than loop-

breaking. Refer to the [Avoiding the Need for Oversampling](#) section for suggestions about how to solve such problems without increasing oversampling.

Vector Debug does not support CAKE clock generation. However, you can increase oversampling in Vector Debug by doubling the number of vectors for each clock cycle. You can use the script `<install_dir>/share/uxe/gift/misc/double_cbc` to double the vectors in a cbc file. Comments in the script give more information.

If increasing the oversampling does not solve the problem, it still could be a loop-breaking problem in which loop-breaking delayed a clock. (Problems caused by clock delay are generally not solved by increasing the oversampling.) Refer to the [Solving Clock-Delay Problems Caused by Loop-Breaking](#) section for suggestions about how to solve clock delay problems.

Any time a loop-breaking problem is suspected, if `xe.msg` contains the following message, you should try to eliminate it using the `-loopsDepth` option:

```
WARNING (multclk-1121): Maximum analysis depth 128 exceeded.
```

If possible, apply the other proactive steps to avoid loop-breaking problems, mentioned earlier.

Delays in Palladium XP

If a design works only with extra oversampling (for example, it works in 4X mode but not 2X mode, or it works in 2X mode but not 1X mode), it is because increasing the oversampling increases the periods of the design clocks relative to the following fixed delays:

- Internal delays
- Target I/O delays
- Non-target I/O delays

Internal delays can be added automatically by precompile's loop-breaking, or these can be added by a user's `breakPin` or `breakNet` userdata commands, or by delay cells (`Q_FDP0B` cells) instantiated in the netlist.

Target I/O delays occur when the transition of a design primary output (for example, a clock output to the target) causes a transition of a design primary input (for example, an input from the target). Unless you use `delayBox` and `terminalTiming` commands to constrain the input to be scheduled at a later step than the output, there will normally be a one-FCLK delay from the output to the input. Also, for a bidirectional primary I/O signal connected to a target, unless you constrain the input component of the bidirectional signal to be scheduled at a later step than the output and enable components, there will normally be a one-FCLK delay from output

and enable components to the input component. In the *UXE Command Reference Manual*, section on the `terminalTiming` command explains how to use `delayBox` and `terminalTiming` commands to avoid target I/O delays.

If a bidirectional top-level I/O signal of the design is not connected to an external target using `terminalAssign`, precompile adds a one-FCLK delay on the load pins of the top-level signal, unless the compiler option `delayBidirIO OFF` is specified. This delay is called a *non-target I/O delay*.

Avoiding the Need for Oversampling

If internal delays (especially loop-breaking delays) cause extra oversampling to be needed, you can use one of the following methods to avoid the need for oversampling:

1. Debug the problem, and direct precompile to break the loop differently

Use Full Vision to debug the design without extra oversampling, and isolate the failure to a particular delay. Then, use the *.brk file (described below) to determine whether the delay is a loop-breaking delay. If it is a loop-breaking delay, you can direct precompile to break the loop differently, using one of the methods described in the section “Preventing a particular break”, below.

If you think precompile could have broken the loop in a more intelligent way, please provide the details to a Palladium XP Customer Support engineer. Feedback from you is the best guide to help Cadence improve the loop-breaking algorithm.

2. Use precompile -breakHalfCycleAll

The precompile -breakHalfCycleAll option makes all loop-breaking and user-added delays *one-half FCLK*. (Palladium XP hardware cannot directly implement a half-FCLK delay. Therefore, half-FCLK delays are implemented using full-FCLK delays and 1X-mode shadow nets.) A half-FCLK delay will break a loop as effectively as a 1-FCLK delay, but the delay is only half as long compared to the design clock cycle, so it is less likely than a full-FCLK delay to make the design fail.

The drawback of the precompile -breakHalfCycleAll option is that in some designs, it causes a large capacity penalty.

Precompile -breakHalfCycleAll was originally intended for use in CAKE 1X mode. However, the option is supported in 2X mode as well. If you use -breakHalfCycleAll with 2X mode, the waveform shows two samples per FCLK, as in 1X mode. However, the fastest clock source has two FCLKs per design clock cycle, as in normal 2X mode.

If your design works in 4X mode but internal delays are causing the design not to work in 2X mode, you can try `-breakHalfCycleAll`. If the capacity cost is not too much, the compile will pass, and the design will now work correctly.

Preventing a Particular Break

If precompile breaks a loop in an undesired way, and you know the location of the undesired break, there are several ways to direct precompile to break the loop at a different location:

- Specify a zero-frequency clock source.

If you are using CAKE or SCM technology for clock generation, you can specify nets as clock sources with frequency 0. When you specify a net as a clock source with frequency 0, precompile considers that net as a clock source for the purpose of loop-breaking, but leaves the design logic driving that net intact and does not insert clock generation logic to drive the net. In general, precompile marks clock sources, and all nets that can be reached from clock sources by tracing forward through combinational gates, as `(clk)` nets. When breaking loops, precompile tries never to delay a net marked as `(clk)`.

- Use the `noBreakPath` command.

The `noBreakpath -add {<startnet> <endnet>}` command directs loopbreaking not to insert any break on a path from `<startnet>` to `<endnet>`, nor any break on a fanout of `<endnet>`. For details, see the *UXE Command Reference Manual*.

- Specify a NOBREAKS property.

You can specify a NOBREAKS property for a module or instance in the RTL or gate level netlist. When you specify a NOBREAKS property for a module or instance, precompile will not insert breaks at the input pins of leaf-level cells within that module or instance. Breaks will be inserted instead at the input pins of leaf-level cells elsewhere in the design. To specify a NOBREAKS property for a module, insert the line

```
// pragma CVAINTPROP MODULE <module_name> NOBREAKS 1
```

anywhere between module and endmodule of the given module's definition. For example, in module `mmm` you can insert the line

```
// pragma CVAINTPROP MODULE mmm NOBREAKS 1
```

To specify a NOBREAKS property for an instance, insert the line

```
// pragma CVAINTPROP INSTANCE <instance_name> NOBREAKS 1
```

in the module that instantiates the given instance. Ensure that the NOBREAKS property for an instance is inserted below the line that has the instantiation definition. In the following example,

```
Q XOR2 U57 ( .A0(n8), .A1(n11), .Z(n12));  
//pragma CVAINTPROP INSTANCE U57 NOBREAKS 1
```

precompile will not insert breaks at the input pins of the Q_XOR2 gate U57.

- Manually break the loop at a different point.

You can get a listing of the loop that was broken by the undesired break from the *.loops report (described later in this Appendix), and use breakNet or breakPin commands (or instantiate a Q_FDP0B delay cell in the netlist) to break the given loop. A drawback of this approach is that often, a given break can break several different loops. Sometimes when you manually break a loop to avoid a particular break, precompile still inserts the undesired break, because it also breaks a different loop that was not broken by the manual break. In such a situation, you need to insert multiple breaks, consulting the *.brk and *.loops files after each precompile run, until precompile no longer inserts the undesired break.

Identifying (clk) Nets with Precompile

A *clock path* is a path taken by a clock transition from a clock source, through logic such as muxes, gates, and clock dividers, to the clock pin of a flip-flop or latch. If loopbreaking inserts delay on a clock path, the delay is likely to introduce skew between different clocks, causing the design to function incorrectly. Precompile attempts to identify likely clock paths by using an algorithm that labels certain nets as (clk) nets. Precompile inserts loop-breaking delay at a fanout of a (clk) net only if there is no other way to break a given loop.

Precompile uses a combination of *forward-tracing* and *backward-tracing* algorithms to identify nets to be labeled as (clk).

The forward-tracing algorithm starts from clock sources, and traces forward through combinational gates, labeling all nets reached as (clk). A limitation of the forward-tracing algorithm is that when a clock path goes through a flip-flop (such as a clock divider) or latch, the tracing algorithm will stop at the flip-flop or latch, and not label the flip-flop or latch output and nets downstream from it as (clk). If you know the output of a certain flip-flop or latch output net is likely to be part of a clock path, you can use the clockFrequency command to specify the net as a *zero-frequency clock source*. Precompile does not insert CAKE clock generation logic for zero-frequency clock sources, but does treat them as clock sources for the forward-tracing algorithm.

Two options, -recognizeClockDivider and -moreClockSources, affect forward tracing. The precompile -recognizeClockDivider option enables recognition of certain simple clock dividers. The forward-tracing algorithm traces from clock to Q of a recognized clock divider as though it were a combinational gate. The -moreClockSources option

causes any net connected directly, or through buffers and inverters, to a flip-flop or latch clock pin to be treated as a clock source by the forward-tracing algorithm.

During loopbreaking, if a loop contains no `(clk)` net, then precompile uses an additional backward-tracing algorithm to label `(clk)` nets: it traces backward from flip-flop and latch clock and data pins, marking each net reached as pure clock (not reached by tracing back from any data pin), pure data (not reached by tracing back from any clock pin) or both clock and data (reached by tracing back from both clock and data pins). Pure clock nets in the given loop are labeled `(clk)`.

Avoiding Clock Delay with Precompile

Precompile's loop-breaking uses the following heuristics that attempt to avoid delaying clocks. The heuristics are listed here in order of decreasing priority.

- When possible, insert delay on an input pin of a combinational gate (not on a flip-flop clock input) which has another input coming from outside the loop
- Precompile identifies `(clk)` nets as described earlier. If all nets in a loop are labeled `(clk)`, then precompile must insert a break at one of these nets. It prefers to break at an input of a gate which has another input which is a labeled `(clk)`, but which comes from outside the loop.
- When precompile can break a loop by at a non-`(clk)` input pin of a gate whose output is labeled `(clk)`, it prefers this break compared to an input of a gate whose output is not labeled `(clk)`. (This heuristic is not applied to a certain type of loop called a set/reset loop.)
- When a loop contains a flip-flop or latch followed by a chain of gates, insert delay on an input pin of a gate as far back in the chain as possible.

Solving Clock-Delay Problems Caused by Loop-Breaking

If you know, or suspect, that loop-breaking has delayed a clock, you can try using the precompile options `-recognizeClockDivider` and `-moreClockSources`. Try `-recognizeClockDivider` first, and then if necessary, try the combination of both options.

Clock delay problems cannot be solved by increasing the CAKE oversampling, but they can be solved using the `-safeLoops` precompile option. This option corrects the functional problems caused by loop-breaking, including clock delay problems, at the expense of emulation speed. For more information, refer to the `precompileOption` command description in the *User Data Command* chapter of *UXE Command Reference Manual*.

The `-safeLoops` option has the drawback of slower emulation speed. Alternatively, use the `precompile -verify` option (described in the [Detecting Clock Delay](#) section) to perform the following:

- Quickly determine whether a flip-flop or latch clock transition is ever delayed.
- Find the delayed clock.
- Find the loop-break that caused the delay.

If you know from the waveform that a certain clock is delayed, then you should use full vision waveforms to trace the delayed clock transition back through the design and find the loop-break which caused the delay.

Once the loop-break has been found, there are two possible cases:

- If the loop-break is on a fanout of a net N which is not marked as `(clk)` in the `*.loops` file, then precompile did not know this net was a clock. Use full vision waveforms to trace back the clock transition from net N gate by gate, until you come to a flip-flop or latch output. Specify the flip-flop or latch output as a zero frequency clock source so that precompile will consider it a clock.
- If the loop-break is on a net which is labeled as `(clk)` in the `*.loops` file, then check the entire loop in the `*.loops` file. All nets in the loop should be labeled `(clk)`. If not, contact Palladium XP Customer Support, since this is probably a bug. If all nets in the loop are labeled `(clk)`, you must manually choose a point in the loop to insert a break that will not delay a clock transition. To do this, use full vision waveforms to trace back the clock transition until it comes from a gate input which is not part of the loop (that is, this gate input is not marked with `@` symbol in `*.loops`). Now, manually insert a break at the input of this gate that is marked with `@`. This break will break the loop without delaying the clock transition.

Designs Containing Latches

For a latch, unlike a flip-flop, loop-breaking considers there to be a path from D to Q. As a result, precompile tends to find many loops in designs that have many latches.

To make loop-breaking manageable in designs containing latches, precompile uses a technique called exclusivity point analysis. If precompile can prove that a particular latch D never changes when G is active, it can safely add a delay on the latch D input pin. (The delay is one FCLK in 2X or higher mode, and one-half FCLK in 1X mode.) Such a latch D input is called an *exclusivity point*. Exclusivity point delays reduce the need for loop-breaking and can shorten the design critical path as well (thus reducing step count). An exclusivity point delay never affects the function of the design, nor the waveform of any signal.

UXE User Guide

Loop-Breaking

Precompile cannot recognize all possible exclusivity points, but it recognizes simple cases where, for some signals C, E1, and E2,

- A latch is enabled only when E1 and C is true, and
- The D input of this latch changes only when E2 and ~C is true

Some latch-based designs obey a design rule called *2-phase non-overlapping clocking*. A set of latches obeys this rule if it can be divided into two groups, where latches in each group receive data only from the other group, and latches from different groups are never enabled at the same time. In such set of latches, every latch ideally should be recognized as an exclusivity point. However, if precompile cannot prove the clocks of the two groups are non-overlapping, it will not recognize the exclusivity points.

In a latch-based design using 2-phase non-overlapping clocking, if there are many latches not recognized as exclusivity points, you might be able to increase the number of exclusivity points in the following ways:

- You can identify pairs (or even n-element sets) of clocks as non-overlapping. You can specify as many pairs (or sets) as you want. Specify them before precompile. Examples:

```
XE> noOverlapClocks -add {A B}
```

means A, B are never simultaneously high.

```
XE> noOverlapClocks -add {C D ~ E ~ U1__F}
```

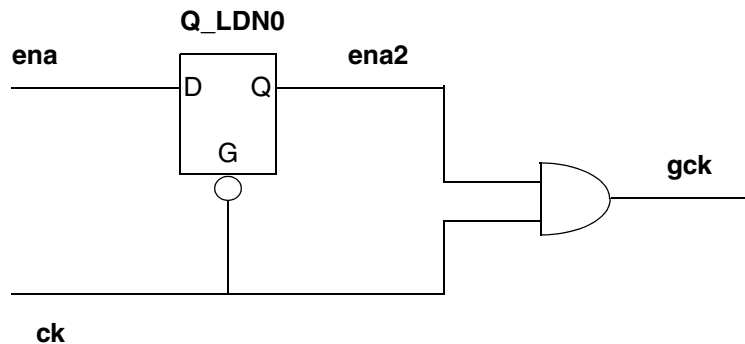
means no two of C, D, ~E, ~U1__F are ever simultaneously high.

- You might be able to simplify the clocking, or else define extra CAKE clock sources, in a way that allows precompile to prove that clocks are non-overlapping.

Using these techniques can increase the number of exclusivity points identified by precompile. This in turn can avoid the need for extra oversampling, and also reduce the design step count by breaking long combinational paths.

Clock Gating Cells

For clock gating, designs often use a cell with a negative-enabled latch and an AND gate, as in the following diagram:



In this circuit, the `ena2` input of the AND gate can change only when the `ck` input is low. Precompile recognizes the `ena2` input of the AND as a *gate-input exclusivity point*, and inserts a delay (one FCLK in 2X mode and one-half FCLK in 1X mode). Similarly, if an OR gate input can change only when the other input of the OR is high, it is also recognized. Like latch D input exclusivity point delay, gate-input exclusivity point delay cannot affect design functionality nor the waveform of any signal. By default, precompile only analyzes direct fanouts of latches (not counting buffers and inverters) and flip-flops as candidates for gate-input exclusivity points. However, if you specify precompile `-moreExclusive`, precompile does not limit the search for gate-input exclusivity points to direct fanouts of latches and flip-flops. Specifying `-moreExclusive` can increase the number of gate-input exclusivity points found. The drawback of this option is a possible increase in precompile execution time.

Classification of Loops

Precompile distinguishes between four types of loops.

- purely combinational loops
- clock loops
- data loops
- set/reset loops

These four types of loop are discovered by precompile in the following way:

First, *purely combinational loops* containing only combinational gates are found and broken. Then, each additional type of loop is found by considering one additional type of path, and loops of the new type are broken. *Clock loops* contain clock to Q paths of latches and flip-flops. *Data loops* contain paths from D to Q of latches. *Set/reset loops* contain paths from set or reset to Q of flip-flops or latches.

Some users incorrectly assume that a clock loop is broken by delaying a clock path (path from clock source to flip-flop clock pin). This is a misunderstanding. For most designs, loop-breaking delay is never added on a clock path.

Using the Precompile -breakHalfCycleAll Command

In 1X mode, by default, each internal delay (added by automatic loop-breaking, breakNet/breakPin, or by a Q_FDP0B instance) is either a full FCLK cycle, or one-half FCLK. A Q_FDP0B instance gives a half FCLK delay if the net being delayed has a shadow; otherwise it gives a full FCLK. A delay added to an input pin of a gate by loop-breaking, breakNet, or breakPin gives a half FCLK delay if the gate's output has a shadow; otherwise it gives a full FCLK. If there are loops in a part of the design clocked by the fastest design clock, loop-breaking delays are likely to make the design not work, since most nets do not have shadows and most delays will be a full cycle of the fastest design clock.

You can use the `precompile -breakHalfCycleAll` option to specify that internal delays on all nets should be only one-half FCLK rather than one FCLK. The option applies to all three types of internal delays: loop-breaking, breakNet/breakPin or Q_FDP0B

In some designs, `precompile -breakHalfCycleAll` has a large capacity cost (even doubling the capacity requirement) but in other designs, the option makes the design work without any significant capacity cost.

The following XEL commands give a half-cycle delay on a specific net or pin even when `precompile -breakHalfCycleAll` is not used:

```
XEC> breakNetHalfCycle -add <netname>
XEC> breakPinHalfCycle -add <pinname>
```

In a design where long combinational paths cause a high step count, you might wish to use delays to break the long paths and improve the step count. Because of the way a half-FCLK delay is implemented, it will break a loop, but it cannot break a long combinational path to improve step count. However, when `-breakHalfCycleAll` is specified, you can insert a full FCLK delay by specifying a breakNet of 2 units, as follows:

```
XEC> breakNet -add {<netname> 2}
```

Using the Precompile `-dontMinimizeDataBreaks` (or `-dataLoops101`) Command

In Palladium release 1.1, precompile's method of breaking of loops that go through latch D inputs (called *data loops*) was changed in the following way. Previously, precompile used a simple method of breaking data loops which often delayed every latch D input through which any loop went. Now, precompile uses a more complex algorithm which tends to minimize the number of delays added, and to delay combinational gate inputs in preference to latch D inputs.

The new algorithm allows some designs to run successfully in 2X mode that previously could only run in 4X mode or higher. However, contrary to expectation, there are a few designs that could run previously in 2X mode, that with the new algorithm can only run in 4X mode or higher. For such designs, the `precompile -dontMinimizeDataBreaks` option gives the old algorithm for data loops. Another name for the same option is `-dataLoops101` (this name reflects the fact that this option gives the Palladium 1.0.1 algorithm for data loops).

Similar to `-dataLoops101`, there are also options `-combinLoops101`, `-clockLoops101` and `-resetLoops101`. The `-loops101` option (also known as `-dontMinimizeLoops`) gives the complete old algorithm. These options are rarely used.

A problem with the new algorithm is that for some designs, it takes a long time. By default, precompile will break loops up to a certain *analysis depth* using the new algorithm, and then break the remaining loops using the old algorithm. A message like the following will be printed in `xe.msg`:

```
WARNING (MultiClock#44756): Maximum analysis depth 128 is reached for data loops.  
Faster technique will be applied.
```

If this message is printed, it is usually best to eliminate the message by using the `precompile -loopsDepth N` option, where `N` is a number larger than the default (128). However, increasing `N` can cause the precompile step of compilation to take longer.

If you wish to reduce the precompile time (at the risk of introducing functional problems due to loop-breaking) you can use `-dontMinimizeDataBreaks` or `-loopsDepth 0`. The `-dontMinimizeDataBreaks` option affects only the analysis of data loops, while `-loopsDepth` affects the analysis of all loops. Another possible problem with the new algorithm is that it usually yields a design with longer combinational paths. In some designs this could increase the step count, which could decrease the emulation speed.

Note: Setting `-dontMinimizeDataBreaks` or `loopsDepth 0` disables, for data loops, some of the heuristics that attempt to avoid delaying clocks.

The *.brk File

Precompile breaks loops by adding delays on input pins of Palladium XP *generic library* primitives (combinational gates, flip-flops (FDRSE) or latches (LDRSE)).

The file PDB/<lib>.<cell>.brk normally reports all the delays that were added by the compiler. For each primitive where a pin was delayed, the file contains an entry something like this:

```
MX2 SDRAM0.U636.mux_1.mux_1.U00
  <-- O SDRAM0.U636.n1y
  -->*I1 SDRAM0.U519.nor_5.U00.NET00
  =F>*I0 SDRAM0.data_out_9
  -->*S SDRAM0.U518.nor_5.U00.NET00
---
```

The symbols <-- and --> indicate output and input pins. Loop-breaking delays are always added on input pins. When delay is inserted on an input pin, a symbol =F> or =H> is used instead of -->. The letters F and H indicate Full and Half FCLK delays respectively. The letter X (as in XF> or XH>) is used instead of = for an exclusivity point delay. This makes it easier for a user to distinguish between safe delays X which cannot affect functionality, and unsafe delays = which can potentially affect functionality. The letter U (as in UF> or UH>) is used for a user-inserted delay.

In the above example, a full FCLK delay has been inserted at the I0 pin of the MX2 primitive whose instance name is SDRAM0.U636.mux_1.mux_1.U0 (since this pin is marked with =F>). This pin is connected to net SDRAM0.data_out_9. (The * characters at some pins indicate logical inversion.)

If you run precompile with the -keepBreaks option, precompile will use an already-existing PDB/*.brk file if there is one, in the following way. Precompile will first add all the non-exclusivity-point delays from this file, and re-calculate and add all exclusivity point delays. Then, if any loops still exist, precompile will add additional delays to break the remaining loops, and the *.brk will be updated to include the new delays as well as the original ones. Without -keepBreaks, precompile ignores and overwrites any existing PDB/*.brk file.

If you compile the same design with different software versions, sometimes the number of loop-breaking delays will be different in the different versions. This happens because the loop-breaking algorithms are often improved in a new release. A given delay can break more than one loop. Thus, the number of delays needed to break all the loops in the design can change.

In some extreme cases (usually latch-based designs) the number of breaks could be so high that writing *.brk file itself might cause a slowdown. In this case, xe.msg will contain a warning:

UXE User Guide

Loop-Breaking

```
Warning 44849 Number of breaking points exceeds 100000. File
PDB/*.brk will not be created. Option -logAllBreaks will force precompile to
create this file.
```

If you wish to examine *.brk and it does not exist, you might need to use `-logAllBreaks`.

The *.loops File

The PDB/*.brk file reports where delays were added, but does not show the actual loops that are broken. To see the actual loops which precompile identified and broke, you can use the file PDB/<lib>.<cell>.loops.

Often, a single loop-break breaks multiple overlapping loops which exist in a design. Precompile does not attempt to report all loops broken by a given break. It reports only one loop for each break.

In the *.loops file, by default, only certain types of loops (*combinational loops* and *clock loops*) are reported. To make the report show a loop for every delay in *.brk, use

```
precompile -logAllLoops N
```

where N is a number greater or equal to the number of delays reported in the *.brk file. The option `-logAllLoops N` tells precompile to log in the *.loops file, up to N *data loops* and *set/reset loops* in addition to the loops it would normally log.

Precompile limits the size of the *.loops file to 128MB and by default stops writing loops when this limit is reached. However, if you use the option `-logAllLoops` (with any $N > 0$), this limit is ignored and precompile will keep writing the file. So, `-logAllLoops 1` might be needed just to observe all combinational and clock loops regardless of the size of .loops file.

The special option `-logAllLoops 0` will stop writing .loops file at all (neither combinational nor clock loops will be written). This is not quite logical because for example `-logAllLoops 1` means "write all combinational and clock loops plus 1 more data or set/reset loop".

In some designs, when `-logAllLoops N` is used, the *.loops file might become huge, taking many hours to write and eventually filling the disk. To avoid this problem, run precompile first without `-logAllLoops`, to obtain the *.brk file. Then create in the design directory a file named `loop.request`, containing the entries from the *.brk file for one or more breaks whose loops you wish to investigate. For example, for a break on a mux input, the `loop.request` file can contain the following:

```
MX2 SDRAM0.U636.mux_1.mux_1.U00
  <-- O SDRAM0.U636.nly
  -->*I1 SDRAM0.U519.nor_5.U00.NET00
```

UXE User Guide

Loop-Breaking

```
=F>*I0 SDRAM0.data_out_9
-->*S SDRAM0.U518.nor_5.U00.NET00
---
```

Now run precompile again. When a loop.request file is present, precompile reports in the *.loops file only loops which are broken at locations specified in the loop.request file. If you use a loop.request file, you need not specify `-logAllLoops`.

During the loop-breaking process, sometimes a break is added, but later removed because a subsequent break made the first break unnecessary. When a break is removed in this way, the *.loops file still shows the corresponding loop with its break, even though the break has been removed. To know which breaks are retained, you must use the *.brk file.

The `-logAllLoops` option can have a side effect of reducing the number of data and set/reset breaks added by loop breaking. The reason for the reduction is that before printing the loop for a given break, precompile performs an extra check to verify that the break has not been made unnecessary by previous breaks. If the break is unnecessary, it is not set. Normally, setting fewer breaks is desirable, but if you wish to avoid this effect, use a loop.request file to report the loops for specific breaks, and do not use `-logAllLoops`.

In the *.loops file, each loop is reported as a sequence of gates between { } characters, as in the example below. Moving downward through the sequence of gates takes you forward along the combinational path. (This is counterintuitive to some users, who expect moving downward in the file to take you backward along the path.)

For each gate, an @ character shows the input pin on the gate which the loop goes through. An = character shows where the loop was broken. Some nets have the label (clk), as described in the earlier section “How precompile identifies (clk) nets”.

```
Feedback loop (clock) {
  MX2 3gio.hsync_pad.UI101.U4.g2.U00.U00
    <-- O 3gio.hsync_pad.UI101.U4.g2.S1HI
    --> I1 3gio.hsync_pad.rst_or_susp_en_pwr
    @=> I0 3gio.hsync_pad.vip_hbus_pad_en_
    --> S 3gio.hsync_pad.QT_func_sel[1] (clk)
  ---
  A2 3gio.hsync_pad.U3.u0.U01@QT1
    <-- O hsync
    @-->*I1 3gio.hsync_pad.UI101.U4.g2.S1HI (clk)
    --> I0 3gio.hsync_OB
  ---
  FDRSE 3gio.hsync_pad.pad_en__reg.map.u0.U01.U00.U00
    <-- Q 3gio.hsync_pad.vip_hbus_pad_en_
    --> D 3gio.hsync_pad.pad_en_79 gnd
}
```

```
@--> CK hsync
--> CE
--> R
-->*S reset_tri_
---
} // Feedback loop (clock)
```

Displaying a Loop in the Schematic Viewer

You can use the script `<install_dir>/share/uxe/gift/misc/loops.gel` to display a particular loop in the Debussy Schematic Viewer. Give the following commands:

```
xeDebug
  source <install_dir>/share/uxe/gift/misc/loops.gel
  debug .
  crt openPDB rt
  loops <net>
```

where `<net>` is the name of a net within the loop you wish to display. This command sequence brings up the Schematic Viewer and displays in color, any loops that are reported in `*.loops` and which contain the specified net. Press the *100%* zoom button to see the loop. Currently, the tool has the limitation that it shows entire nets in color, even though a particular loop actually involves only a single fanout of each net. This can make the schematic confusing.

The command sequence given above generates two files: `loop_nets.v` and `loop_nets.v.ses`.

Loops through Memory

Memories with byte-write-enable or bit-write-enable are sometimes modeled using a *read-modify-write* memory model. Such a model uses an extra read port to pre-read data from the address about to be written. The output from the extra read port is given through a mux (each bit controlled by the bit-write-enable) to the memory data input. If there is no flip-flop in this feedback path, it is a combinational loop.

When a memory model contains a combinational loop, the `tmp/<lib>.<cell>.MultiClock` file will contain a warning such as:

```
WARNING (38412):  Loops through memory detected. See tmp/<lib>.<cell>.MultiClock
for details.
```

In addition, in the loop-breaking section of this file will be an entry as shown below and the `*.loops` file will show loops involving this memory:

```
memory: MPW2X32 resetseq.resetmemcell.RAMBLK0_0.ET3_MEM_0
```

If the overall memory being modeled is synchronous (that is, it has a clock and registers its inputs) then it is usually possible to model the memory in a better way, without a combinational loop. For examples, refer to the [Synthesizing Designs with Memories](#) section of [Chapter 3, “Compiling Designs for In-Circuit Emulation with xeCompile.”](#)

Loops Discovered after Precompile

In rare cases, the `compile XEL` command could fail with a message reporting loops. This happens because, by default, the precompile loop-breaking does not consider any `terminalTiming` constraints. To avoid such failures, specify any `delayBox` and/or `terminalTiming` commands (except for `terminalTiming -edgeInfo` commands), then run `precompile` using the `-breakTerminalTiming` option. When you specify the `-breakTerminalTiming` option, precompile’s loop-breaking function takes into consideration the `terminalTiming` constraints.

Detecting Clock Delay

In most cases, if loop-breaking delays cause a design to function incorrectly, you can obtain correct function by increasing the CAKE oversampling ratio. However, if loop-breaking delay causes an internal clock net to be delayed, then usually the design will function incorrectly regardless of the CAKE oversampling ratio.

Compiling with the `precompile -verify` option creates a special database that you can use to quickly identify any internal clock net that was delayed by loop-breaking, and find the particular loop-breaking delay (or other delay) that caused the problem.

The `-verify` option is allowed with CAKE 2 only. If your design requires a higher CAKE oversampling ratio such as 4, a workaround is to add a dummy clock to your design with frequency double your fastest real design clock, and compile with CAKE 2. The fastest design clock (other than the dummy) effectively runs with CAKE 4, but `-verify` will work properly and not complain.

To add a dummy clock source, the dummy clock source net must exist in the design. You can use `iScopedNets` to add the dummy clock source to the design without modifying the netlist. For more information, see the section “Using a Dummy Fastest Clock Source” in Emulation User Guide Appendix D.

UXE User Guide

Loop-Breaking

To use the `-verify` option, specify this option in addition to any other precompile options you normally use.

For example:

```
precompile -ignoreEmptyCells -verify
```

or

```
precompileOption -add ignoreEmptyCells
precompileOption -add verify
precompile
```

With `-verify`, CAKE clock generation leaves an idle FCLK cycle after every FCLK cycle when some clock source toggled. This is close to technology CAKE 4 but somewhat different. With CAKE 4 there is an idle FCLK cycle after every FCLK cycle when the fastest clock source toggles, but slower clock sources might toggle at idle cycles of the fastest clock. The `-verify` option ensures that if *any* clock source toggles, then the next FCLK cycle is idle for *all* clock sources.

The term *clock net* is defined as follows: in general, precompile removes most buffers and inverters from the design, placing an inversion property on input pins of gates and storage elements when necessary. A clock net is any net that is connected directly (after buffer/inverter removal) to a clock pin of a flip-flop or enable pin of a D latch.

A clock net will never change value at an idle cycle unless a loop-breaking or other delay has been introduced on a path between a CAKE clock source and that clock net.

With `-verify`, precompile modifies the emulator's normal triggering mechanism in the following way. When running in STB mode with `sdl -enable`, if any clock net changes value at one of the idle FCLK cycles mentioned earlier, the emulation will stop immediately (without the normal two-FCLK trigger-to-stop delay of Enhanced SDL).

After compiling with `-verify`, run your emulation test almost as you normally would, but with the following changes:

- ☐ You must run in STB mode.
- ☐ You cannot use the `clockConfig` command.
- ☐ Normally, you must modify any `run n` commands, doubling the value of *n* for verify mode compared to normal CAKE 2 mode.
- ☐ You must make sure any run is done with `sdl -enable` if you wish to detect delayed clock transitions that occur during that run. If you use the following state machine,

```
State s0 { if (l==0) { TRIGGER ; } };
```

the emulator will stop only if a delayed clock is detected or the cycle count limit *n* in a `run n` command is reached.

- ❑ When using `-verify`, any `run` command in an XEL script must have the `-nowait` option specified. Without `-nowait`, a delayed clock will not cause the emulation to stop. Therefore, instead of the command `run 500`, you must give the command `run -nowait 500; waitWhileBusy`.

Whenever the emulator stops, you can determine whether the stop was due to the `-verify` feature by using the `value` command on the special signal `qt_verify_CAKE_edges_trigger`. If the value is 1, the stop was caused by the `-verify` feature. If the value is 0, the stop was not caused by the `-verify` feature.

The `-verify` feature stops the emulation if some clock has a transition at an idle cycle. You can easily find out which clock net(s) had a transition, as follows: Use the `value` command on all the signals listed in the file `PDB/clock.edges`. For each clock net *N* connected to the clock pin of a rising edge flip-flop or negatively enabled latch, the file has a signal with a name similar to `N@QTposedgeVerify`. This signal has value 1 only if net *N* had a rising edge at this idle cycle. Similarly, for each clock net *M* connected to the clock pin of a falling edge flip-flop or positively enabled latch, the file has a signal with a name similar to `M@QTnegedgeVerify`, which has value 1 only if net *M* had a falling edge at this idle cycle.

Once the `value` command has returned the clock net that was delayed, set `sdl -disable`, and give the `run 2` command to run two more FCLKs. Then upload waveforms in the usual way. You can now use waveforms from incremental upload or offline debug to trace the clock transition back through the design, to find the reason the clock was delayed. No knowledge of the design is required for this tracing; you can trace back the transition at the idle cycle, gate by gate. If two inputs of a gate both have a transition, pick one arbitrarily and continue tracing. You will eventually trace back to a net which is delayed even though the inputs of its driving gate are not delayed. If the delay is caused by loop-breaking, the `PDB/*.brk` file will show the delay on an input pin of the gate.

A current limitation of the `-verify` feature is that if somehow a clock is delayed by 2 FCLK cycles, the feature will not detect it.

Sometimes, a particular clock net *N* can be delayed, but you know that delay on this net does not cause a problem for the design. In such a case, at run time you can selectively disable the `-verify` triggering for net *N*. To disable the checking, force low the appropriate net listed in `PDB/clock.edges`.

Rarely-Used Options

With the option `-noLoopBreaks`, precompile breaks loops as it normally does (writing `.brk` and `.loops`), but then, if any loops were detected, it fails with a message:

```
Info 39539 Breaking feedback loops was disabled per user request.
```

This option was created for a user who wished to break all loops manually.

UXE User Guide

Loop-Breaking

Modeling Guide for Verilog Designs for ICE Mode

This appendix describes how to synthesize Verilog designs for emulation in ICE mode using the Palladium HDL Compiler. It also describes Verilog features that the HDL Compiler does not synthesize, and provides some caveats for special cases such as multi-language designs.



Important

This appendix is applicable only to the ICE mode. Refer to [Chapter 4, “Compiling Designs for Simulation Acceleration Using IXCOM”](#) for details about modeling guidelines for SA mode.

The following topics are discussed in detail in this chapter:

- [Creating Synthesizable Verilog Designs](#)
- [Considering Synthesis Assumptions](#)
- [Modeling Sequential Logic with always Blocks](#)
- [Synthesizing Tristate Logic](#)
- [Verilog -2001 Configuration](#)
- [SystemVerilog](#)

Creating Synthesizable Verilog Designs

This section describes how to create a Verilog design for RTL synthesis. The HDL Compiler synthesis procedure is performed using the `hdlSynthesize` XEL command. Refer to the *HDL Compiler Commands* chapter in the *UXE Command Reference Manual* and the [Synthesizing the RTL Design for Emulation](#) section.

This section assumes that you are already familiar with Verilog as a simulation language. So, this section describes only the differences between Verilog as a simulation language and as

a synthesis language. For a complete description of the Verilog language, see the Verilog documentation that is included with the Verilog simulation tool.

Using Verilog HDL

Verilog contains a rich set of syntax to describe a digital system design. A synthesizable Verilog circuit can be described in either of the following two ways:

- A structural description models a design as gate-level components, and the connections between them.
- A behavioral description (also called a functional description) models what a design does at the more abstract behavioral or functional level.

The structural and behavioral descriptions can be mixed when modeling a design. Design hierarchy is represented through module hierarchy, and each design module has one corresponding Verilog module. The following subsections explain these description styles.

Using Structural Descriptions

To model a design, a typical netlist description uses signal nets to connect instances of library-specific gates. A Verilog structural description also models the structural view of a design. In addition to library-specific gates, the structural description can also contain Verilog primitive logic and user-defined modules.

Moreover, a signal net can be driven by an arbitrary expression that models the logical value assigned to the net. The continuous assignment capability in Verilog uses an arbitrary expression to assign a value to a net.

The following is a structural description of a 4-bit adder.

```
module adder4 (in1, in2, sum, zero);
    input [3:0] in1;
    input [3:0] in2;
    output [4:0] sum
    output zero;
    fulladd u1 (in1[0], in2[0], 1'b0, sum[0], c0);
    fulladd u2 (in1[1], in2[1], c0, sum[1], c1);
    fulladd u3 (in1[2], in2[2], c1, sum[2], c2);
    fulladd u4 (in1[3], in2[3], c2, sum[3], sum[4]);
    nor u5 (zero, sum[0], sum[1], sum[2], sum[3], sum[4]);
endmodule

module fulladd (in1, in2, carryin, sum, carryout);
    input in1, in2, carryin;
    output sum, carryout;
    assign {carryout, sum} = in1 + in2 + carryin;
endmodule
```

A behavioral design model describes how the design works in abstract terms. Outputs of a module are described in terms of the inputs of that module, but the model does not describe how the module is implemented.

The key elements of a behavioral description are functions and always blocks. A function implies combinational logic, although it might appear to be sequential. An always block, however, can be sequential, and implies sequential logic.

Using behavioral modeling, the 4-bit adder from the previous example can be described as shown in the code sample below.

```
module adder4 (in1, in2, sum, zero);
    input [3:0] in1;
    input [3:0] in2;
    output [4:0] sum;
    output zero;
    reg [4:0] sum;
    reg zero;

    always @(in1 or in2) begin
        sum = in1 + in2;
        if (sum == 0)
            zero = 1;
        else
            zero = 0;
    end
endmodule
```

Mixing Descriptions

You can mix structural and behavioral descriptions within a Verilog module. In a mixed-level design, the combinational portions are typically described in Verilog functions, always blocks, and continuous assignments. You can use multiple always blocks, which infer flip-flops and latches, to maintain state values between different clock cycles.

The following code sample shows the 4-bit adder from the preceding examples, described in a mixed mode.

```
module adder4 (in1, in2, sum, zero);
    input [3:0] in1;
    input [3:0] in2;
    output [4:0] sum;
    output zero;
    reg zero;

    fulladd u1 (in1[0], in2[0], 1'b0, sum[0], c0);
    fulladd u2 (in1[1], in2[1], c0, sum[1], c1);
    fulladd u3 (in1[2], in2[2], c1, sum[2], c2);
    fulladd u4 (in1[3], in2[3], c2, sum[3], sum[4]);

    always @ (sum)
        if (sum == 0)
            zero = 1;
```

```
        else
            zero = 0;
        end
    endmodule

module fulladd (in1, in2, carryin, sum, carryout);
    input in1, in2, carryin;
    output sum, carryout;
    assign {carryout, sum} = in1 + in2 + carryin;
endmodule
```

Using Module Hierarchy

Verilog uses module blocks to describe a unit of logic in your design. Module hierarchy represents the design hierarchy—that is, an instance within a module represents another module in the design hierarchy. The HDL Compiler can synthesize hierarchical Verilog designs.

The mixed-level version of the 4-bit adder example uses a two-level hierarchy, where the top-level `adder4` module contains the `u1` instance, which is an instance of another design hierarchy, the 1-bit full adder.

The HDL Compiler supports a majority of Verilog constructs, including all structural elements and most behavioral (functional) statements. However, within an expression, the HDL Compiler synthesis does not support operators that are used exclusively for simulation.

Identifying Simulation Directives

The HDL Compiler synthesis process accepts, but ignores, Verilog simulation directives. Following are the examples of simulation directives:

- ``accelerate`
- ``celldefine`
- ``default_nettype`
- ``endcelldefine`
- ``endprotect`
- ``expand_vectornets`
- ``noaccelerate`
- ``noexpand_vectornets`
- ``nounconnected_drive`

- ``noremove_netnames`
- ``protect`
- ``remove_netnames`
- ``reset`
- ``timescale`
- ``unconnected_drive`

Identifying System Tasks

All Verilog system tasks begin with the dollar sign (\$), and are usually used for waveform monitoring and input/output generation during simulation. The HDL Compiler synthesis procedure accepts system tasks, but issues warnings when they occur. The following are examples of system tasks:

<code>\$finish</code>	<code>\$log</code>	<code>\$showvars</code>	<code>\$time</code>
<code>\$input</code>	<code>\$monitor</code>	<code>\$stop</code>	<code>\$write</code>
<code>\$list</code>	<code>\$output</code>	<code>\$strobe</code>	

Identifying Keywords

Keywords are pre-defined, non-escaped identifiers that define the Verilog language constructs. These keywords cannot be used as variable names, unless the backslash (\) escape character is used.

The following is a list of some Verilog keywords:

<code>always</code>	<code>and</code>	<code>assign</code>
<code>begin</code>	<code>buf</code>	<code>bufif0</code>
<code>bufif1</code>	<code>case</code>	<code>casex</code>
<code>casez</code>	<code>cmos</code>	<code>deassign</code>
<code>default</code>	<code>defparam</code>	<code>disable</code>
<code>else</code>	<code>end</code>	<code>endcase</code>
<code>endfunction</code>	<code>endmodule</code>	<code>endprimitive</code>

UXE User Guide

Modeling Guide for Verilog Designs for ICE Mode

endtable	endtask	event
for	force	forever
fork	function	highzo
highz1	if	initial
inout	input	integer
join	large	medium
module	nand	negedge
nmos	nor	not
notif0	notif1	or
output	parameter	pmos
posedge	primitive	pulldown
pullup	pull0	pull1
rcmos	reg	release
repeat	rnmos	rpmos
rtran	rtranif0	rtranif1
scalared	small	strong0
strong1	supply0	supply1
table	task	time
tran	tranif0	tranif1
tri	triand	trior
triereg	tri0	tri1
vectored	wait	wand
weak0	weak1	while
wire	wor	xnor
xor		

Identifying Unsupported Statements

The HDL Compiler synthesis process does not support the following Verilog statements, and reports error messages if it encounters them:

- `deassign`
- `defparam` (with upward reference in design hierarchy)
- `event` (control)
- `force`
- `fork`
- `join`
- `release`
- `repeat`
- `wait`

Note: During synthesis, most names in the design are preserved, but the names of blocks, tasks, or functions are not preserved.

Considering Synthesis Assumptions

The HDL Compiler assumes the following when synthesizing Verilog RTL designs.

Constant Expressions

The synthesis process evaluates constant expressions, and propagates them as far as practical, to reduce the amount of circuitry required. No circuitry is synthesized to compute the constant value of the expression.

Size of Constant Numbers and Parameters

The default size of an unsized constant or parameter is 32 bits.

Size of Integer Declaration

The default size for integers is 32 bits. the synthesis process calculates the maximum required bit-width, and assigns the appropriate size.

Part Select

A part-select expression selects a group of bits from a wire, register, or parameter vector. Unlike the bit-select operator, this expression must be constant. It must also follow the same ascending or descending index order as originally declared for the variable to which you apply the part-select.

Comparison to X or Z

The synthesis process ignores a comparison to x (unknown) or z (high impedance)—that is, the comparison always evaluates to false. A warning message is displayed, because such treatment might cause simulation results to mismatch synthesis results.

Delays and Drive Strengths for Module Statements

The HDL Compiler synthesizes weak, pull, and high drive strengths; but it does not distinguish between the weak and pull strengths. HDL Compiler synthesis accepts, but ignores, delay values or other drive strength values associated with module statements, or with constructs for simulation.

Undriven or Multiply Assigned Wire

The HDL Compiler synthesis process leaves undriven wires unconnected. For multiple connections or assignments to a wire, HDL Compiler either creates logic to resolve them correctly, or reports an error message.

Assignment to Tristate

All but one variable that drives the tristate should have a value of z (high-impedance). This variable determines the value of the tristate. You must ensure that this rule is followed, because the HDL Compiler synthesis does not check or enforce it.

Continuous Assignment

If the number of bits on the left side of an assignment is greater than the number on the right side, then the HDL Compiler synthesis adds zeroes to extend the right-side operands.

Parameters in Module Instantiations

The Verilog language lets you override parameter values in a module during instantiation, using either the `defparam` statement or the following syntax:

```
moduleName #(parameterValue, parameterValue, ... instanceName
```

Modeling Sequential Logic with always Blocks

Verilog models sequential logic through the `always` block. This section describes modeling guidelines, all of which are compatible with established industry practice.

Two types of `always` blocks are available:

- Combinational logic, such as:

```
always @ (a or b) begin
    f = a & b;
end
```

- Sequential logic, such as:

```
always @ (posedge ck) begin
    f = a & b;
end
```

The main difference between the above types is how they trigger events.

An `always` block contains logic that responds as follows:

```
always @ (event-expression) begin
    ... statements ...
end
```

The event-expression declares triggers, grouped by the word `or`. If any of the triggers in the event-expression occur, then the `always` block is executed.

Two types of triggers are used:

- A combinational `always` block triggers based on changes in the logic levels. Interpreting a combinational `always` block is straightforward, except while making an inference to latches (see [Latch Logic Inference](#) on page 1052).
- A sequential `always` block triggers based on changes of signal edges (rising or falling edges).

Register Inference

Any variable that is assigned a value from within a sequential always block, is implemented as a D-type edge-triggered flip-flop (DFF). The clock of the DFF is the trigger, even if only one such trigger exists, or one of the triggers is based on modeling guidelines.

If only one trigger exists, then the DFF is clocked using the signal and edge specified in the event-expression. In the following example of clock edges in a sequential always block, the DFFs that implement `f` and `g` are clocked at the rising edge of `ck1`, and `h` and `i` are clocked at the falling edge of `ck2`.

```
always @ (posedge ck1) begin
    f = a & b;
    g = x & y;
end

always @ (negedge ck2) begin
    h = a & b;
    i = x & y;
end
```

Asynchronously Reset Registers

To model a DFF that has an asynchronous reset, you must follow a modeling template. The following is a basic example showing register inference with a basic asynchronous reset. The DFF is clocked by `CK` on the rising edge, and has an asynchronous reset following the falling edge of `RST`.

```
always @ (posedge CK or negedge RST) begin
    if (!RST) begin
        ... statements ...
    end
    else begin
        ... statements ...
    end
end
```

The following example infers DFF registers with two asynchronous resets: `RSTa` on the rising edge and `RSTb` on the falling edge.

```
always @ ( posedge CK or posedge RSTa or negedge RSTb) begin
    if (RSTa) begin
        ... statements ...
    end
    else if ( !RSTb) begin
        ... statements ...
    end
    else begin
        ... statements ...
    end
end
```

UXE User Guide

Modeling Guide for Verilog Designs for ICE Mode

To model DFF registers that have asynchronous resets, use the following template as a modeling guideline:

```
always @ (posedge clock or posedge reset1 or negedge reset2) begin
  if (reset1) begin
    /* statements for asynchronous inputs to flip flops
     * triggered by reset1
     */
    ...statement1...
  end
  else if (!reset2) begin
    /* statements for asynchronous inputs to flip flops
     * triggered by reset2
     */
    ...statement2...
  end
  else begin
    /* no explicit event reference here */
    /* assignments to D-flip flop inputs */
    ...statement3...
  end
end
```

If the final `else` clause assigns a value to a variable, then the only triggering event that is not referenced in any `if-else` clause, clocks the DFF for that variable. Any assignment to a variable in an `if` or `else if` clause implies an asynchronous reset condition for the corresponding DFF.

The event-expression must list all reset conditions, and must list the clock, with all desired senses of edges. All conditions that the `if-else` clause tests, such as `reset1` and `reset2`, must match the declared sense of edges for the corresponding triggers in an event-expression. The following are examples:

```
else if (condition)
  // reset on rising edge; posedge appears in
  // event-expression
```

or

```
else if (! condition)
  // reset on falling edge; negedge appears in
  // event-expression
```

Each reset condition must have a single-bit value. Explicitly reference all but one trigger in the `if-else-if` clauses. A lone trigger, with its declared sense of edge in the event-expression, serves as the clock to the DFFs.

Asynchronously Set/Reset Registers

In the preceding template example, `statement1` and `statement2` are basic statements such as:

```
f = 1'b1;
```

This statement example describes a preset condition, rather than a reset condition. For the corresponding DFF, an asynchronous preset is created, rather than a reset.

A sequential `always` block can assign values to more than one variable. Each variable is subject to a different configuration of set and reset conditions. That is, each variable is synthesized as if each has its own dedicated if-else-if, and else specification.

The following is an example of registers with asynchronous set and reset:

```
always @ (posedge CK or posedge RST or posedge PR) begin
    if (RST) begin
        f = 1'b0;
        g = 1'b0;
    end

    else if (PR) begin
        f = 1'b1;
    end

    else begin
        f = a & b;
        g = x & y;
    end
end
```

The above example produces two DFFs, one for f and one for g. The DFF for f has both asynchronous set (PR) and reset (RST), but the DFF for g has only asynchronous reset (RST).

Latch Logic Inference

To maintain consistency between simulation and synthesis, latches are sometimes inferred and created in a combinational `always` block. However, if you do not intend such behavior, and you need to eliminate extra latches, then use the guidelines in this section to rewrite portions of the Verilog file.

Verilog register variables can hold their states over time in simulation. To maintain consistency between simulation and synthesis, synthesis must simulate such behavior in synthesized logic. the HDL Compiler synthesis inserts and implements qualifying variables, which require such behavior and use a D-type flow-through latch. This latch allows a variable to hold its value (state) until that value is reassigned.

A qualifying variable needs to hold its state if its previous value changes, but at the same time the current calculation requires that previous value. The following is an example of latch inference with an `if` statement:

```
always @ (cond or a or b) begin
    if (cond) begin
        x = a & b;
    end
end
```



```
    end  
end
```

In the above example, if `cond` is false, then the `x` variable should retain its previous value. The same combinational logic that implements the new value of `x` when `cond` is true (the normal condition), should be blocked when `cond` is false.

To block this logic when `cond` is false, the following occurs:

- To retain the previous value of `x`, a latch is created for `x`.
- The data input of the latch is connected to `(a & b)`.
- The latch output is connected to `x`.
- `cond` controls the latch gate.

The following example shows the behavior of the new implementation in the rewritten description. In the preceding example, the if statement controls the condition under which `x` must retain its previous value. A case statement can also control these conditions, as in the following example:

```
input [1:0] d;  
  
always @ (d or a or b) begin  
  case (d)  
    2'b00: x = a & b;  
    2'b01: x = a | b;  
  endcase  
end
```

To avoid unwanted latches, make sure that the variable is assigned a value under all circumstances. You can rewrite the two preceding examples to avoid creating latches, as shown in the following two examples:

```
always @ (cond or a or b) begin  
  if (cond) begin  
    x = a & b;  
  end  
  else begin  
    x = 1'b0;  
  end  
end  
  
input [1:0] d;  
  
always @ (d or a or b) begin  
  case (d)  
    2'b00: x = a & b;  
    2'b01: x = a | b;  
    2'b10: x = 1'b0;  
    2'b11: x = 1'b0;  
  endcase  
end
```

Synthesizing Tristate Logic

You can synthesize the tristate logic embedded in a Verilog description using either of two methods: the `hdlSynthesize XEL` command.

- Select the toolbar menu Command: *Design>HDLCompiler>Synthesize*

Examples of Verilog Tristate Logic

Tristate logic typically involves testing a condition under which a z (high-impedance) value should be assigned to a variable.

The following Verilog code segments require testing conditions:

- IF Statements
- Conditional Assignments
- Case Statements

The following is an example of synthesizing tristate logic with an *if statement*:

```
always @ (ENA or din) begin
    if (ENA)
        dout = 1'bz;
    else
        dout = din;
    end
end
```

The following is an example of synthesizing tristate logic with a *conditional assignment*:

```
always @ (ENA or din) begin
    dout = ENA ? 1'bz : din;
end
```

The following is an example of synthesizing tristate logic with a *case statement*:

```
always @ (d or a or b) begin
    case (d)
        2'b00: x = a & b;
        2'b01: x = a | b;
        2'b10: x = 1'bz;
        2'b11: x = 1'bz;
    endcase
end
```

Enabling or Disabling Memory Synthesis

HDL Compiler will automatically choose to use the `hardmacro` memories for constructs for which they are suitable. In addition, HDLICE provides fine granular control of memory synthesis with two synthesis directives.

❑ `no_hardmacro`

This command disables memory synthesis for the signal specified. For example, if there is a memory declared in the RTL:

```
reg [1024:9] mem [23:13];
```

You can add the synthesis directive after the declaration,

```
//quickturn no_hardmacro mem
```

❑ `use_hardmacro`

This command forces the signal to be synthesized into `hardmacro` memories. The usage is similar to `no_hardmacro`.

Both synthesis directives are effective when memory synthesis option is turned on. They can be used with either Verilog or VHDL designs.

Out of Module Reference

The HDL Compiler currently supports out of module references only if the objects are directly inside the module, and not in any nested code. OOMRs are supported only if the objects are nets or variables of a scalar or vector type. They are not supported if the objects are a parameter, function, or task. Multiple-dim arrays are also not supported. An OOMR that refers to a different object in a different instantiation is not supported. Out of module references to memory nets or variables is not supported, except when used in `$readmem` system tasks.

Import all objects referenced by the OOMR in the same `hdlImport` session: Cadence recommends that OOMRs should not be used with an incremental import.

If the object referenced by the OOMR is not imported in the same `hdlImport` session, it is assumed to be scalar or vector, with a range that minimally covers all indexes used in bit/part-select of all OOMR's referring to it. For example: if `a.b.c[2:4]`, `a.b.c[5]`, `a.b.c[1:3]` are used, `c` is assumed to be a vector of `[1:5]`. Only locally static indexes are taken into account. If no bit/part-select is being used, the OOMR is assumed to be scalar. If the OOMRs occur in different `hdlImport` sessions, a warning message is generated stating the assumptions made by the HDL Compiler.

Verilog -2001 Configuration

HDL Compiler provides limited support of Verilog-2001 configuration with the following restrictions:

- The configuration must be imported with the corresponding module in the same import session, although they can be in different files.
- The `design` statement can only contain a module name.

For example:

```
design <cell>;
```

The following yields an error from the HDL Compiler

```
design [<library>.<cell>;
```

- The `default` clause is fully supported. The libraries specified are linked in the same manner as in VHDL.
- In an `instance` clause, only instances under the current module (one level down) are supported, and only one library is allowed in the `liblist` clause.

For example:

```
instance top.a1 liblist mylib1;  
// this is ok
```

The following yields an error from HDL Compiler:

```
instance top.a1.a2 liblist mylib1;  
// 2-level hierarchy  
instance top.a1 liblist mylib1 mylib2;  
// 2 libraries in liblist
```

- In a `cell` clause, only one library is allowed in the `liblist` clause, similar to an `instance` clause.

For example:

```
cell adder liblist mylib1;  
// this is ok  
cell adder liblist mylib1 mylib2;  
// this yields an error
```

- `use` clauses are supported except those with the `:config` suffix. In other words, hierarchical configuration is not supported, but `use <lib.>cell` is supported.

For example:

```
instance top.mux2 use mux4;
```

```
// use mux4 in the same lib as top
cell add1 use add4;
// use mux4 in the same lib as top
instance top.mux2 use mylib3.mux4;
cell add1 use mylib3.add4;
```

The following yields an error from the HDL Compiler

```
instance top.mux2 use mylib3.mux4:config;
```

- The library map file (IEEE1364-2001 [13.2]) is not supported. Continue to use the existing method to specify libraries; `hdlDefinePath` in XEL or `va.lib` on the UNIX command-line interface.

SystemVerilog

By default, the IEEE Verilog-1995 standard is assumed for a Verilog input file. To enable recognition of the SystemVerilog keywords, use the `-sv` option with either the `hdlImport` XEL command or the `vavlog` UNIX command as follows:

```
hdlImport -sv
```

```
vavlog -sv
```

Supported SystemVerilog Features

The following section lists the supported SystemVerilog features with their corresponding LRM section numbers in brackets []. Refer to IEEE Standard 1800-2005 for these sections.

- Literal Values [3]
- Data Types [4]
- Arrays [5]
- Data Declarations [6]
- Operators and Expressions [8]
- Procedural Statements and Control Flow [10]
- Processes [11]
- Tasks and Functions [12]
- Assertions [17]
- Hierarchy [19]

- [Interfaces \[20\]](#)
- [System Tasks and System Functions \[22\]](#)
- [Compiler Directives \[23\]](#)
- [Packages \[26\]](#)
- [Formal Syntax \[Annex A\]](#)
- [Encryption/Decryption Flow \[Annex 0\]](#)

Literal Values [3]

- [unsized literals \[3.3\]](#)
- [shortreal literals \(parsed then ignored\) \[3.4\]](#)
- [time units in literals \(parsed then ignored\) \[3.5\]](#)
- [string literals \[3.6\]](#)
- [array literals \[3.7\]](#)

Data Types [4]

- Integer data types including `logic`, `integer`, `bit`, `byte`, `shortint`, and `longint` [4.3]
- `shortreal` data type (recognized but not synthesized) [4.4]
- user-defined type [4.9]

Limitation: Forward `typedef` is not supported

- enumeration type [4.10]

- `typedef enum` [4.10.1]

Limitation: Forward `typedef` is not supported

- `enum` type ranges [4.10.2]

enumerated data type shorthand (name [N]), [N]=C) (name [N:M]); (name [N:M]=C)

- `enum` type checking [4.10.3]

- `enum` type methods [4.10.4]

- `enum` type method `first()` [4.10.4.1]

- enum type method last() [4.10.4.2]

- enum type method next() [4.10.4.3]

```
typedef enum {e0,e1,e2,e3} t ;  
t x,y ;  
y = x.next() ;
```

- enum type method prev() [4.10.4.4]

```
typedef enum {e0,e1,e2,e3} t ;  
t x,y ;  
y = x.prev() ;
```

- enum type method num() [4.10.4.5]

Limitation: enum type method name() [4.10.4.6] is not supported.

- Unpacked structure [4.11]

Limitation: The initial value is ignored

- Packed structure [4.11]

Limitations:

- ☐ The initial value is ignored.
- ☐ Treated only as vector at run time.

```
struct packed unsigned {integer a ; logic [7:0] b ;} pack1 ;
```

- Packed union [4.11]

```
typedef union packed  
{  
    logic [15:0] bit_slice ;  
    logic [1:0][7:0] byte_slice ;  
} word_type ;
```

Arrays [5]

- Packed and unpacked arrays [5.2]

- Positive number for range of unpacked dimension [5.2]

Note: [size] is equivalent to [0:size-1]

```
logic [3:0] A[8] ; // that is, logic [3:0] A[0:7]
```

- Multiple dimensions array port [5.3]

UXE User Guide

Modeling Guide for Verilog Designs for ICE Mode

```
module top #(width=2,depth=4)
(
    input [depth:1][width:1] i,
    output [depth:1][width:1] o,
) ;
    ...
endmodule
```

■ Multiple dimensions array parameter [5.3]

```
parameter [3:0][7:0] p = '{8'h00,8'h01,8'h10,8'h33} ;
```

■ Indexing and slicing of arrays [5.4]

■ Array querying functions [5.5]

`$left`, `$right`, `$low`, `$high`, `$increment`, `$size`, `$dimensions`

■ Array assignment [5.7]

```
int A[10:1];
```

fixed-size array of 10 elements

```
int B [0:9];
```

fixed-size array of 10 elements

```
int C [24:1];
```

fixed-size array of 24 elements

```
wire [31:0] W [9:0];
```

```
A = B;
```

Correct usage: compatible type and same size

```
A =C;
```

type check error: different sizes

```
assign W = A;
```

Correct usage

■ Array as arguments [5.8]

```
task fun (int a[3:1][3:1])
```

Given the above declaration of `fun`, consider the following actual:

```
int b[3:1][3:1];
```

Correct usage: same type, dimension, and size

```
int b[1:3][0:2];
```

Correct usage: same type, dimension and size (different range)


```
int b[3:1][4:1];
```

Error: incompatible size (3 vs. 4)

Data Declarations [6]

■ type parameter [6.3.3]

```
parameter type node = logic ;
```

Limitations:

- ❑ The `type` parameter can only be used for built-in type and simple type name.
- ❑ The `type` parameter cannot be used with incremental import.

■ const constants [6.3.5]

```
const logic a = 1'b0;
```

■ net datatype [6.5]

```
wire integer n ;
```

■ Declarations in unnamed blocks [6.6]

■ Continuous assignments to variables [6.7]

In Verilog, continuous assignments can only drive nets. In SystemVerilog, continuous assignments are allowed to drive nets or variables.

Operators and Expressions [8]

■ Assignment operators as statements [8.3]

■ Assignment operators as expressions [8.3]

■ Postincrement/decrement statements [8.3]

■ Preincrement/decrement statements [8.3]

■ ++ and -- as expressions [8.3]

■ Concatenation [8.12]

■ Assignment pattern for unpacked array/struct [8.13]

```
var int B[4] = '{0,1,2,3} ;
```

Limitations:

- ❑ Will attempt to support the obsolete SV 3.1.a assignment pattern syntax. However, due to syntax conflict with concatenation, it might not work in all contexts.

- ❑ Assignment pattern should not use both IEEE 1800-2005 syntax and the obsolete SV 3.1.a syntax at the same time.
- ❑ Type key for assignment pattern is not supported.

- Aggregate expressions [8.15]

- Streaming operator : as expression [8.17]

Limitation: Only streaming operator used as expression will be supported and the objects used cannot be parameter-sized.

```
logic [3:0] A[3:0] ;
{<< {A}} ;      // pack array bits in right-to-left order
                // that is, {A[0][0],...,A[0][3],...,A[3][0],...,A[3][3]}
```

Procedural Statements and Control Flow [10]

- Blocking and non-blocking assignments (time units) [10.3]

SystemVerilog allows a time unit to be specified.

```
#1ns r = a;
r = #1 ns a;
r <= #1 a;
```

The time unit is parsed but ignored.

- Selection statements -- if [10.4]

- Selection statements -- case [10.4]

- do while loop [10.5.1]

- Enhanced for loop [10.5.2]

```
for (int i=0,j=0 ; i<n ; i++,j++)
```

Note: Multiple initialization and step statements are supported.

- Jump statements (return, break, continue) [10.6]

- Named block (matching end block name) [10.8]

- iff event control [10.10]

Processes [11]

- always_comb [11.2]

- `always_latch` [11.3]
- `always_ff` [11.4]

Tasks and Functions [12]

- Default direction and type for task/function formal arguments [12.2]

```
function int f (a,b,output c,d) ;  
// input logic a,b ;  
// output logic c,d ;
```

- Function output/inout port [12.3]

```
function int func (input x,output y) ;  
    ...  
endfunction
```

- void functions [12.3.1]

```
function void myfunc (input x) ;  
    ...  
endfunction
```

- Discarding function return values [12.3.2]

```
void'(f(i,o)) ; // used as statement
```

- Task/function default argument values [12.4.3]

```
function int f (input int n=3, input [3:0] a,b,c=4'hf) ;  
    ...  
endfunction  
// function call with  
f(,4'h3,4'h5)           //equivalent to f(3,4'h3,4'h5,4'hf)
```

- Task/function argument binding by name [12.4.4]

```
function int f (input int n, input [3:0] a,b,c) ;  
    ...  
endfunction  
//function call with  
f(3, .c(4'h3), .b(4'h5), .a(4'hc))
```

Assertions [17]

- Sampled value functions [17.7.3]

The following functions are supported:

❑ `$sampled(<expression>[,<clocking_event>])`

Return type: type of first argument

❑ `$rose(<expression>[,<clocking_event>])`

Return type: bit

❑ `$fell(<expression>[,<clocking_event>])`

Return type: bit

❑ `$stable(<expression>[,<clocking_event>])`

Return type: bit

❑ `$past(<expression>[, [<number_of_ticks>][, [<expression>][, [
[<clocking_event>]]]])`

Return type: type of first argument

■ System functions [17.10]

❑ `$onehot(<expression>)`

Return type: bit

❑ `$onehot0(<expression>)`

Return type: bit

❑ `$countones(<expression>)`

Return type: not specified in IEEE standard, implemented as int

❑ `$isunknown(<expression>)`

Return type: bit

Hierarchy [19]

■ package [19.2]

```
package pkg ;  
function id (input x) ;  
    return x ;  
endfunction  
endpackage  
module test (input a,output b) ;  
import pkg::* ;
```

```
assign b = id(a) ;  
endmodule
```

■ Compilation unit scope [19.3]

Limitations:

- ❑ Only parameter, typedef, function, net, variable declarations are supported in compilation unit scope.
- ❑ Only global compilation unit scope is supported, not file compilation unit scope.
- ❑ Compilation unit scope should not be used with incremental import. Only the one with latest HDL import will be supported.
- ❑ When using `vavlog` for importing the source files, all files containing the compilation unit scope should be compiled in the same `vavlog` session.

```
parameter int p = 2 ;  
typedef logic myBit ;  
module mod ( ... ) ;  
    ...  
endmodule
```

■ Variable ports [19.8]

With SystemVerilog, a port can be declared as a variable.

```
module mod ( input byte a, b );  
    ...  
endmodule
```

Only variable ports with basic data type are supported.

■ Array ports [19.8]

■ instance array [19.11]

Use of only single expression for range is supported.

```
sub s[4] (a[3:0],b[3:0]) ; // This is same as s[0:3]  
module sub (input x,output [3:0] y) ;  
// The instance array creates the following instances of the sub module:  
// sub \s[0] (a[3],b[3:0]) ;  
// sub \s[1] (a[2],b[3:0]) ;  
// sub \s[2] (a[1],b[3:0]) ;  
// sub \s[3] (a[0],b[3:0]) ;
```

■ Implicit .name port connection [19.11.3]

```
.name (name)
```

■ Implicit .* port connection [19.11.4]

Interfaces [20]

■ Interfaces [20]

Limitations:

- ❑ Interface instance is flattened and seen as a collection of objects at run time, objects are named as `<instance_name>_<obj_name>`.
- ❑ Nested interface is not supported, neither is nested module.
- ❑ Hierarchical reference (OOMR) into interface is not supported, only direct reference through interface or modport port is allowed.
- ❑ Instance array of interface is not supported.
- ❑ The interface declaration and all modules and interfaces that use that interface need to be in the same import session.

■ always block and continuous assignment in interface declaration [20]

```
interface itfc #(w=4) (input clk,en) ;  
  logic [w-1:0] din,dout ;  
  logic clk_out ;  
  assign clk_out = en ? clk : 1'b0 ;  
  always @(negedge clk) dout <= din ;  
endinterface
```

■ named bundle [20.2.2]

```
interface itfc ;  
  logic req,gnt ;  
  logic [7:0] addr,data ;  
  logic start,rdy ;  
endinterface : itfc  
module sub (itfc a,input bit clk,avail) ;  
  always @(posedge clk) a.gnt <= a.req & avail ;  
endmodule  
module cpu (itfc b,input bit clk) ;  
  ...  
endmodule  
module test (input logic clk,output logic gnt) ;  
  logic avail = 0 ;  
  itfc sb () ;
```

```
    sub s(sb,clk,avail) ;
    cpu c(.b(sb),.clk(clk)) ;
endmodule
```

■ **Ports in interfaces [20.3]**

```
interface itfc (input bit clk) ;
    logic req,gnt ;
    logic [7:0] addr,data ;
    logic start,rdy ;
endinterface : itfc
module sub (itfc a) ;
    ...
endmodule
module cpu (itfc b) ;
    ...
endmodule
module test (input logic clk) ;
    itfc sb1 (clk) ;
    itfc sb2 (clk) ;
    sub s1(sb1) ;
    cpu c1(.b(sb1)) ;
    sub s2(sb2) ;
    cpu c2(.b(sb2)) ;
endmodule
```

■ **Modports [20.4] (only 20.4.1 - 20.4.3)**

```
interface itfc ;
    wire a,b,c,d ;
    modport master (input a,b,output c,d) ;
    modport slave (output a,b,input c,d) ;
endinterface
module m (itfc.master i) ;
    ...
endmodule
module s (itfc.slave i) ;
    ...
endmodule
module test ;
    itfc i () ;
    m u1 (i) ;
    s u2 (i) ;
endmodule
```

■ **Parameterized interface [20.7]**

```
interface itfc #(WIDTH=1) (input clk) ;
    logic [WIDTH-1:0] data ;
    ...
endinterface
```

■ **System function : \$clog2(<expression>) [20.8.1]**

return type : int

Non-static argument is supported.

Note: This feature is not a part of IEEE 1800-2005 SystemVerilog standard; the section number is from the IEEE 1800-2009 SystemVerilog standard.

System Tasks and System Functions [22]

■ **\$bits() [22.3]**

Limitation: Argument of \$bits() can be either expression or type. Only expression argument is fully supported. For type argument, only simple type name (from typedef) is supported.

```
parameter w = 3 ;
wire [w:0] d1 ;
wire [w-1:0] d2 ;
assign d2 = d1[$bits(d2)-1:0] ;
```

Compiler Directives [23]

■ **`define macros [23.2]**

■ **`begin\_keywords and `end_keywords directives [23.4]**

```
`begin_keywords "1364-2001"
...
`end_keywords
```

The supported version specifiers include 1364-1995, 1364-2001, 1364-2001-noconfig, and 1800-2005.

Packages [26]

■ **Using package in module header [26.4]**

```
package pkg ; typedef struct {logic a,b ;} t ; endpackage
module m import pkg::* ; (input clk,input t d,output t q) ;
```


UXE User Guide

Modeling Guide for Verilog Designs for ICE Mode

```
// module m (input clk,input pkg::t d,output pkg::t q) ;
```

Note: This feature is not a part of IEEE 1800-2005 SystemVerilog standard; the section number is from the IEEE 1800-2009 SystemVerilog standard.

Formal Syntax [Annex A]

- local genvar declaration [A.4.2]

```
for (genvar i=0 ; i<4 ; i++)
```

Note: This feature appears in Annex A (formal syntax). Semantically, it is not defined in either IEEE 1800-2005 or 1800-2009 SystemVerilog standard.

Implementation treats the `genvar` as if it were declared in the `generate` block for loop body. There is no implicit scope created similar to procedural for-loop.

Encryption/Decryption Flow [Annex O]

- IP protection [Annex O.4]

```
`pragma protect
`pragma protect begin
... text to be protected ...
`pragma protect end
```

Note: This feature is not a part of IEEE 1800-2005 SystemVerilog standard; the section number is from the IEEE 1800-2009 SystemVerilog standard.

UXE User Guide
Modeling Guide for Verilog Designs for ICE Mode

Modeling Guide for VHDL Designs for ICE Mode

This chapter describes the features of the VHDL language that Palladium XP HDL Compiler can synthesize for emulation in ICE mode. It also describes VHDL features that the HDL Compiler does not synthesize.



This appendix is applicable only to the ICE mode. Refer to [Chapter 4, “Compiling Designs for Simulation Acceleration Using IXCOM”](#) for details about modeling guidelines for SA mode.

The following topics are discussed in detail in this chapter:

- [Entities and Architectures](#)
- [Objects](#)
- [Data Types](#)
- [Operators](#)
- [Operands](#)
- [Sequential Statements](#)
- [Concurrent Statements](#)
- [Subprograms](#)
- [Packages](#)
- [VHDL Synthesis Directives](#)
- [Syntax and Semantic Restrictions](#)
- [Modeling Styles](#)
- [Configuration Declaration](#)

Entities and Architectures

A digital system is usually designed as a hierarchical collection of blocks of logic. Each logic block corresponds to a design unit in VHDL.

Design units represent portions of a hardware design that have well-defined inputs and outputs and perform well-defined functions.

Each design unit has two parts: an entity declaration, and architecture bodies. An entity declaration describes the external interfaces of a component, and architecture bodies describe its internal implementations.

The following example describes an implementation of a 1-bit full adder:

```
entity FULL_ADDER is
  port( A, B, CIN : in  BIT ;
        SUM, COUT : out BIT ) ;
end;
architecture DATAFLOW of FULL_ADDER is
  signal S : BIT ;
begin
  S      <= A xor B ;
  SUM    <= S xor CIN;
  COUT   <= (A and B) or (S and CIN);
end;
```

The entity describes the boundary of the design unit. Each port is assigned a direction (in this example, *in* or *out*) and a data type (such as *BIT*).

The architecture describes the contents of the `FULL_ADDER` entity. The architecture starts with a declarative region; in this example, an internal signal *S* is defined.

The architecture content starts with `begin` and ends with `end`. The region between the `begin` and `end` is called the *dataflow* region. All statements in the dataflow region are executed concurrently, so the order of the statements is irrelevant.

An entity can have several architectures. A design can be modeled using different implementation methods, at different views, or at different levels of abstraction. Only one architecture is synthesized each time. By default, VHDL links the last defined architecture to the entity.

Use generic constants to control the structure of an entity. For example, in the following VHDL description, the `<N>` generic constant specifies the number of bits for the adder. The default value of the `<N>` generic constant is 4, until otherwise specified when you add an instance of the component.

```
entity ADDER is
  generic(N: INTEGER := 4 ) ;
  port( A, B : in  BIT_VECTOR (N-1 downto 0) ;
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
CIN  : in  BIT ;
SUM  : out BIT_VECTOR (N-1 downto 0) ;
COUT : out BIT ;

end;
architecture GEN_FOUR of ADDER is
  signal TMP : BIT_VECTOR(N downto 0) ;
  component FULL_ADDER
    port (A, B, CIN : in BIT;
          SUM, COUT : out BIT) ;
  end component ;
begin
  TMP(0) <= CIN ;
  G : for I in 0 to 3 generate
    FA : FULL_ADDER port map (
      A(I), B(I), TMP(I), SUM(I), TMP(I+1) ) ;
  end generate ;
  Cout <= TMP(N) ;
end
```

Objects

Data objects hold values of a specific type.

VHDL provides three standard classes of objects: constants, variables, and signals. It also supports three special object types: ports, generics, and loop variables. Ports and generics are associated with an entity declaration, and loop variables are associated with an iteration scheme.

Declarations list the data objects to use, state the object type for each data object, and optionally set the initial value of each object.

This section describes each of the standard and special object types.

Constants

A constant is an object that is initialized to a specific value when you create it, and that you cannot subsequently modify. You can use constant declarations in packages, entities, architectures, subprograms, blocks, and processes.

```
constant YES      : BOOLEAN := TRUE ;
constant CHAR7     : BIT_VECTOR(4 downto 0) := "00111" ;
constant MSB      : INTEGER := 5 ;
```

Variables

Variables hold temporary data. Declare variables only in a process or a subprogram.

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

A variable declaration must include a type. Its value can be either a range constraint or an initial value. By default, the initial value is the lowest (left-most) value of the range for that variable type.

```
variable STAND,BROKE : BIT ;
variable TEMP        : BIT_VECTOR(8 downto 0) ;
variable DELAY       : INTEGER range 0 to 15 := 0;
```

Signals

Signal declarations create new signals (wires) of a specified type. Signals can communicate between processes, or they can synchronize processes. Declare signals in package declarations (global signals), entities, architectures, and blocks. You can also use, but not declare, signals in processes and subprograms.

```
signal BEEP          : BIT := '0' ;
signal RESULT        : INTEGER range 0 to 1023 ;
signal COUNT         : STD_LOGIC_VECTOR(7 downto 0);
```

Ports

Ports specify the interface channels of an entity. Ports are signals that communicate with the outside world.

As in standard object types, port declarations must specify a type, and can assign an initial value.

Each port is associated with a mode or direction, either `in`, `out`, `buffer`, or `inout`.

- `In` ports can be read but not assigned a value.
- `Out` ports can be assigned a value but not read.
- `Buffer` and `inout` ports can be read and assigned a value.

```
entity ADDER is
  port( A, B : in  BIT_VECTOR (3 downto 0) ;
        CIN  : in  BIT ;
        SUM   : out BIT_VECTOR (3 downto 0) ;
        COUT  : out BIT ) ;
end ADDER ;
```

Generics

Generics are defined in an entity that controls the structure or behavior of a design.

Generic constants can be used to control the structure of an entity. For example, in the following VHDL description, the $\langle N \rangle$ generic constant specifies the number of bits for the adder.

```
entity ADDER is
  generic(N: INTEGER := 4 ) ;
  port( A, B      : in  BIT_VECTOR (N-1 downto 0) ;
        CIN      : in  BIT       ;
        SUM      : out BIT_VECTOR (N-1 downto 0) ;
        COUT     : out BIT       ) ;
end ADDER ;
```

$\langle N \rangle$ is defined as a generic constant with an initial value of 4. You can use a generic map statement to overwrite the value of $\langle N \rangle$ when you add an instance of the component.

For example, if you defined ADDER as a cell in your design, you can use the following statement to add an instance of an 8-bit adder:

```
U1: ADDER generic map(N=>8)
      port map( IN1, IN2, RESULT, CARRY);
```

Loop Variables

Loop variables are special objects that you do not have to declare. You can use them in loop statements or generate statements within a for loop.

```
G : for I in 0 to 3 generate
FA : FULL_ADDER port map (
      A(I), B(I), TMP(I), SUM(I), TMP(I+1) ) ;
end generate ;
```

In this example, the I variable is an integer with a range from 0 to 3. You can use a loop variable only inside the loop, and you cannot assign a value to the loop variable.

Data Types

All data object definitions in VHDL must include a data type. You can either use predefined data types, or define your own data types.

IEEE 1076 Predefined Types

IEEE 1076-1987 predefined a set of data types in the standard VHDL packages. You can synthesize the following standard data types:

```
type boolean is (false,true);
type bit is ('0', '1');
type character is (
  nul, soh, stx, etx, eot, enq, ack, bel,
  bs, ht, lf, vt, ff, cr, so, si,
  dle, dc1, dc2, dc3, dc4, nak, syn, etb,
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
can, em, sub, esc, fsp, gsp, rsp, usp,  
' ', '!', '"', '#', '$', '%', '&', ' ',  
'(', ')', '*', '+', '-', '.', '/',  
'0', '1', '2', '3', '4', '5', '6', '7',  
'8', '9', ':', ';', '<', '=', '>', '?',  
'@', 'A', 'B', 'C', 'D', 'E', 'F', 'G',  
'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O',  
'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W',  
'X', 'Y', 'Z', '[', '\', ']', '^', '_',  
'`', 'a', 'b', 'c', 'd', 'e', 'f', 'g',  
'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o',  
'p', 'q', 'r', 's', 't', 'u', 'v', 'w',  
'x', 'y', 'z', '{', '|', '}', '~', del);
```

```
type string is array (positive range <>) of character;  
type integer is range -2147483648 to 2147483647;  
subtype natural is integer range 0 to integer'high;  
subtype positive is integer range 1 to integer'high;  
type bit_vector is array (natural range <>) of bit;
```

The HDL Compiler treats `character` and `string` as an enumeration type, and `string` as an array. See [Modeling Styles](#) on page 1112 for restrictions on how the HDL Compiler synthesis uses these types of objects.

IEEE 1164 Predefined Types

The IEEE 1164 standard defines a nine-valued logic type for simulation. The meaning of the nine values are:

- 'U' —Uninitialized
- 'X'—Forcing Unknown
- '0' —Forcing Low
- '1' —Forcing High
- 'Z'—High Impedance
- 'W'—Weak Unknown
- 'L'—Weak Low
- 'H'—Weak High
- '-' —Don't Care

The HDL Compiler supports these types, but with some restrictions. Among the nine values, only '0', '1', 'X', '-', and 'Z' have well-defined meaning for synthesis.

Some useful data types in the package include:

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
type Std_ULogic is ('U','X','0','1','Z','W','L','H','-');
type Std_ULogic_vector is array (natural range <>) of std_ulogic;
subtype std_logic is resolved std_ulogic;
type std_logic_vector is (natural range <>) of std_logic;
SUBTYPE X01      IS resolved std_ulogic RANGE 'X' TO '1';
SUBTYPE X01Z     IS resolved std_ulogic RANGE 'X' TO 'Z';
SUBTYPE UX01     IS resolved std_ulogic RANGE 'U' TO '1';
SUBTYPE UX01Z    IS resolved std_ulogic RANGE 'U' TO 'Z';
```

To use the types defined in the `STD_LOGIC_1164` package, your VHDL design must include the following statements:

```
library IEEE ;
use IEEE.STD_LOGIC_1164.all ;
```

IEEE 1076.3 Predefined Types

The IEEE 1076.3 standard defines `SIGNED` and `UNSIGNED` data types.

A data object of `SIGNED` or `UNSIGNED` data type is treated as a `bit_vector` (or `std_logic_vector`), but it can be used in arithmetic computation.

```
type UNSIGNED is array (NATURAL range <>) of BIT/STD_LOGIC;
type SIGNED is array (NATURAL range <>) of BIT/STD_LOGIC;
```

The standard synthesis packages (1076.3) include two VHDL packages: `NUMERIC_BIT` and `NUMERIC_STD`. The `NUMERIC_BIT` package is based on the `BIT` type, and the `NUMERIC_STD` package is based on the `STD_LOGIC` type.

Both packages interpret the `UNSIGNED` type as an unsigned binary integer, and the `SIGNED` type as a two's complement binary integer.

Both packages are compiled into the IEEE library. Because the numerical array types in both packages are defined using the same name, you can use only one package at a time.

To use the definitions and functions of the `NUMERIC_BIT` package, your VHDL design must include the following statements:

```
library IEEE ;
use IEEE.NUMERIC_BIT.all ;
```

To use the definitions and functions of the `NUMERIC_STD` package, your VHDL design must include the following statements:

```
library IEEE ;
use IEEE.STD_LOGIC.all ;
use IEEE.NUMERIC_STD.all ;
```

Enumeration Types

An enumeration type is defined by listing all possible values of that type.

You define all values of an enumeration type. These values can be either identifiers or single-character literals.

- An identifier is a name, such as `white`, `monday`, or `lily`.
- Character literals are single characters enclosed in quotes, such as `'X'`, `'0'`, or `'1'`.

```
type COLOR is (RED, ORANGE, YELLOW, GREEN, BLUE, PURPLE);
type STD_ULOGIC is ('U', 'X', '0', '1', 'Z', 'W', 'L', 'H', '-');
```

Integer Types

Integer types (or subtypes) are for mathematical integers.

All normal predefined mathematical functions, such as add, subtract, multiply, and divide, apply to integer types.

```
type MY_INTEGER is range -64 to 64;
type TWO'S_COMPLEMENT_INTEGER is range -32768 to 32767;
```

Array Types

Array types group one or more elements of the same type together as a single object.

Array elements can be of any VHDL data type.

An array has one and only one index, whose value selects each element. The number of indexes (the dimension) of an array can be any positive number. The range of the index (or the *called index range*) determines the number of elements in the array, and their direction (low-to-high, or high-to-low). In VHDL, the index type can be either integer or enumeration.

For example, the following defines a 64-element array with type as `AR1`, in which the type of each element is `INTEGER`:

```
type AR1 is array (0 to 63) of INTEGER ;
```

An unconstrained array type is a type whose dimension is specified, but the exact range and direction of each dimension is left unspecified. This allows multiple subtypes to share a common base type.

For example, VHDL uses two predefined unconstrained array types: `BIT_VECTOR` and `STRING` types.

```
type BIT_VECTOR is array (NATURAL range <>) of BIT ;
type STRING is array (POSITIVE range <>) of CHARACTER ;
```

To use an unconstrained array type, you must specify the index range. For example:

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
subtype B4 is BIT_VECTOR (3 downto 0) ;
variable V5 : BIT_VECTOR(4 downto 0) ;
```

The HDL Compiler lets you use multiple dimensional arrays with arrays of arrays.

```
type TWO_D_AR is array(0 to 7, 0 to 3) of BIT ;
constant TWO_D_ROM : TWO_D_AR :=
  ( ( '0', '0', '0', '1'),
    ( '0', '0', '1', '1'),
    ( '0', '1', '0', '1'),
    ( '0', '1', '0', '1'),
    ( '0', '0', '1', '1'),
    ( '1', '0', '0', '1'),
    ( '1', '1', '0', '1'),
    ( '1', '1', '1', '1') ) ;
X := TWO_D_ROM(2, 3) ;
```

Record Types

Record types group one or more elements of different types together as a single object. the HDL Compiler can access each element of the record by its field name.

Record elements can include elements of any type, including *array* and *records*. The following is an example of a record type declaration:

```
type DATE_TYPE is (SUN, MON, TUE, WED, THR, FRI, SAT) ;
type HOLIDAY is
  record
    YEAR: INTEGER range 1900 to 1999 ;
    MONTH: INTEGER range 1 to 12 ;
    DAY: INTEGER range 1 to 31 ;
    DATE: DATE_TYPE ;
  end record ;
```

A selected name is used to reference an element of a record. A selected name consists of the name of the object, followed by a dot (.), followed by the name of the element.

The type of a selected name is the type of its element.

For example:

```
signal S: HOLIDAY ;
variable T1: INTEGER range 1900 to 1999 ;
variable T2: DATE_TYPE ;
  T1 := S.YEAR ;
  T2 := S.DATE ;
```

Subtypes

VHDL provides subtypes, which are subsets of other types. Anywhere that you can add a type definition, you can also add a subtype definition.

A subtype is a subset of a previously defined type or subtype. You can compare overlapping subtypes of a given parent type, and assign them to each other.

Subtypes are a powerful way to ensure valid assignments and meaningful data handling in VHDL. Subtypes inherit all operators and subprograms defined for their parent types.

For example, `NATURAL` and `POSITIVE` are subtypes of `INTEGER`, and you can use them with any `INTEGER` function. The HDL Compiler can add, multiply, compare and assign them to each other, if the values are within the appropriate range for the subtype.

The following are examples of subtype declarations:

```
subtype INT4 is INTEGER range 0 to 15 ;  
subtype BIT_VECTOR6 is BIT_VECTOR(5 downto 0) ;
```

Operators

Predefined VHDL operators that you can synthesize using the HDL Compiler meet one of three IEEE standards: IEEE 1076, IEEE 1164, or IEEE 1076.3.

IEEE 1076 Predefined Operators

VHDL provides six classes of operators. Each VHDL operator has a precedence level.

[Table C-1](#) lists the classes of the operators, from lowest to highest precedence.

Table C-1 Class Precedence for IEEE 1076 Predefined Operators

Class of Operator	Members of Operator Class
logic_operator	and, or, nand, nor, xor
relational_operator	=, /=, <, <=, >, >=
adding_operator	+, -, &
sign	+, -
multiplying_operator	*, /, mod, rem
miscellaneous_operator	**, abs, not

Logical Operators

The logical operators (`and`, `or`, `nand`, `nor`, `xor`, and `not`) accept operands of predefined type `BIT`, type `BOOLEAN`, or one-dimensional array type of `BIT` or `BOOLEAN`.

For the binary operators (`and`, `or`, `nand`, `nor`, and `xor`), the parent type of both operands must be the same. If the type of the operands is a one-dimensional array, then the operands must be arrays of the same length.

The binary operators designate the `bitwise` function on matching elements of the arrays, and the result is an array with the same index range as the left operand.

The unary operator (`not`) computes the bitwise negation of its operand. If the type of the operand is an array, then the result is an array with the same index range as the operand.

Relational Operators

The relational operators work on operands of the same type, and return a `BOOLEAN` value.

- The equality (`=`) and inequality (`/=`) operators are defined for all types.
- The ordering (`<`, `=`, `>`, `>=`) operators are defined for all enumerated types, integer types, and one-dimensional arrays of enumeration or integer types. The internal order of a type's values determines the result of the ordering operators. Integer values are ordered from negative infinity to positive infinity.

Adding Operators

Adding operators include adding `+`, `-`, and `&`.

The arithmetic operators (`+` and `-`) work on `INTEGER` operands.

The concatenation operator (`&`) groups element of the same type into an array of that type. Each operand of `&` can be an array, or an element of an array.

Unary Operators

All three unary operators take an object of numeric type, and return the same type.

- The unary plus operator (`+`) returns the value of its operand.
- The unary minus operator (`-`) computes the arithmetic negation of its operand.
- The unary absolute operator (`abs`) returns the absolute value of its operand.

Multiplying Operators

The multiplying operators (******, *****, **/**, **mod**, and **rem**) work on **INTEGER** types. the HDL Compiler restricts the supported values for the operands of the multiplying operators:

- For exponentiation (******), the left operand must be a power of 2.
- For division (**/**), modulus (**mod**), and remainder (**rem**) operations, the right operand must be a power of 2.
- Multiplication (*****) has no limitation.

IEEE 1164 Predefined Operators

The logical operators (**and**, **or**, **nand**, **nor**, **xor**, and **not**) are overloaded in the **STD_LOGIC_1164** package for **std_logic** and **std_ulogic** types, and for **std_logic_vector** and **std_ulogic_vector** array types.

IEEE 1076.3 Predefined Operators

The base type of **SIGNED** and **UNSIGNED** is either a **BIT** type or **STD_LOGIC** type. You can apply any operators that work on **BIT** type or **STD_LOGIC** type to these two data types.

Except for the exponentiation (******) operation, all adding operators and multiplying operators also work on **SIGNED** and **UNSIGNED** data types.

Operator Overloading

The predefined VHDL operators work only on data types that VHDL defines for the specific operator. To use an operator on data types that are not defined for that operator, use operator overloading. For example, in the HDL Compiler, the **+** operator is overloaded so it can add two operands with **bit_vector** types and return a result of integer type.

```
architecture op_ov5 of op_ov5 is
    signal in1, in2 : bit_vector ( 0 to 2);
    signal out1 : integer;
function v2i (in1 : bit_vector(0 to 2)) return positive is
begin
    case in1 is
        when "001" => return 1;
        when "010" => return 2;
        when "011" => return 3;
        when "100" => return 4;
        when "101" => return 5;
        when "110" => return 6;
        when "111" => return 7;
        when "000" => return 8;
    end case;
end;
```

```
end;  
function "+" (in1,in2: bit_vector(0 to 2)) return integer is  
begin  
    return v2i(in1) + v2i(in2);  
end;  
  
begin  
    out1 <= in1 + in2;  
end op_ov5;
```

Operands

In an expression, the operator uses the operands to compute the value of the expression. Operands can themselves be expressions.

The types of operands you can use in an expression include literals, identifiers, indexed names, slice names, attribute names, aggregates, qualified expressions, type conversions, and function calls.

Literals

Literals are fixed (constant) values that you cannot change. the HDL Compiler can synthesize numeric, character, enumeration, and string literals in your VHDL design.

Numeric Literals

Numeric literals are constant integer values, which can be either *decimal* or *based*. The numeric literal can be separated by underscores.

- A decimal literal is written in base 10.
- A based literal can be written in any base from 2 to 16, as a base number, a pound sign (#), the value in the given base, and another # sign. The following example is equivalent to the decimal number 27:

```
4#0_123#
```

Character Literals

Character literals are single characters enclosed in single quotation marks. For example:

```
'z'
```

Enumeration Literals

Enumeration literals list (enumerate) all possible values of a specified data type. The two types of enumeration literals are characters and identifiers. The following example uses enumeration identifiers:

```
type SOME_ENUM is (ENUM_ID_1, ENUM_ID_2, ENUM_ID_3);
```

String Literals

String literals represent one-dimensional arrays of characters. The two types of strings are *character strings* and *bit strings*.

- Character strings are a sequence of characters in double quotes, such as:

```
"DEMO"  
"10101"
```

- Bit strings are similar to character strings, but represent binary, octal, or hexadecimal values, such as:

```
B"11001"  
O"277"  
X"4C"
```

Identifiers

Identifiers are the names for constants, variables, signals, entities, ports, subprograms, or parameter declarations.

Names must begin with a letter (a to z), followed by letters, underscores, or digits. However, underscore (_) cannot be the last character of an identifier.

VHDL identifiers are not case-sensitive—that is, `name`, `Name`, and `NAME` are all the same identifier.

Some identifiers, such as `entity`, `port`, `is`, and `end`, are reserved words. Each reserved word has a fixed meaning, so you cannot use a reserved word for other purposes in VHDL.

Indexed Names

Indexed names identify elements of array objects or signals. The syntax of an indexed name is:

```
<identifier> (<expression>)
```


UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

where `<identifier>` is an array-type signal or variable, and `<expression>` returns a value within the array's index range. For example:

```
Data_Array(ADDR)
```

Slice Names

Slice names identify a sequence of elements of array objects. The slice direction can be either `to` or `downto`. However, the slice direction must be consistent with the direction of the identifier's declaration—for example:

```
bitvec(8 downto 2)
```

Attribute Names

Attributes are data attached to VHDL objects. VHDL attributes use variables or signals of a selected type, and they return values.

```
subtype IDX_RANGE is INTEGER range 10 downto 0 ;
variable VEC1: BIT_VECTOR (IDX_RANGE) ;

VEC1'left -- equal to IDX_RANGE'left == 10
VEC1'right -- equal to IDX_RANGE'right == 0
VEC1'high -- equal to IDX_RANGE'high == 10
VEC1'low -- equal to IDX_RANGE'low == 0
VEC1'range -- equal to IDX_RANGE == 10 downto 0
VEC1'reverse_range -- equal to == 0 to 10
VEC1'length -- 11
```

The `event` and `stable` attributes are associated with signals. These attributes indicate whether the signal value changes at the current simulation time. However, in the HDL Compiler synthesis, you can use a VHDL `event` or `stable` attribute only to specify a leading or falling clock edge. For example:

```
signal'event and signal='1' -- leading edge of signal
signal'event and signal='0' -- falling edge of signal
NOT signal'stable and signal='1' -- leading edge of signal
```

You can use the signal edge expression only as a condition within the `wait` and `if` statements.

Aggregates

Aggregates combine values into composite values of either *array* or *record* type. You can use aggregates to assign values to array type or record type objects during the initial declaration, or in an assignment statement.

You can use either *positional* or *named* association to specify the index of an element.

- If you use positional association, then the HDL Compiler assigns a value to each element in order, by calculating the expression for that element.
- If you use name association, then the HDL Compiler explicitly makes the correspondence by naming each element with a value.

For example:

```
type Color_List is (Red, Orange, Blue, White) ;
type Color_Array is array (Color_List) of BIT_VECTOR
(1 downto 0) ;
variable X : Color_array ;
X := ("00", "01", "10", "11") ; -- positional association
X := (Red => "00", Blue => "10", Orange => "01", White => "11") ;
```

Qualified Expressions

Qualified expressions are expressions or aggregates that resolve ambiguities in operand types. This type of expression explicitly states the type or the subtype of an operand.

The syntax of a qualified expression is:

```
<type_name>' (<expression>)
```

where <type_name> is the name of a defined operand type, and <expression> evaluates to an appropriate type.

For example:

```
signed' ("0000")
```

Type Conversions

Type conversion explicitly converts data types to related data types. Type conversions change the type of expressions; qualified expressions only resolve the type of expressions.

Function Calls

Function calls transfer execution to a specified function body (which can have parameters), and retrieve values.

The syntax of a function call is:

```
<function_name> ([<parameter_name> =>] <expression> {, [<parameter_name> =>]  
<expression>})
```

where <function_name> is the name of a defined function, the optional <parameter_names> are the names of formal parameters of the function, and <expression> returns a value that evaluates to an appropriate type for the parameter.

For example:

```
function FUNC(A, B, C : INTEGER) return BIT;
```

Sequential Statements

Sequential statements are statements within a process or a subprogram. VHDL uses the following sequential statements:

- Wait Statements
- Signal Assignment Statements
- Variable Assignment Statements
- If Statements
- Case Statements
- Loop Statements
- Next Statements
- Exit Statements
- Null Statements
- Assertion Statements
- Function and Procedure Calls

Wait Statements

The HDL Compiler can synthesize `wait` statements only if they include edge-detecting attributes (`event` and `stable`). This restriction assures correct compilation of a VHDL program into a logic circuit.

the HDL Compiler synthesizes the following three statements the same way:

```
wait until CLK = '1';  
wait until CLK'event and CLK='1';  
wait until not CLK'stable and CLK='1';
```

You can also include a conditional expression in the `wait` statement. For example, the following means “wait for the falling edge of the BCLK signal, and until the value of START is 1.”

```
wait until BCLK = '0' and start = '1';
wait until BCLK'event and BCLK='0' and start = '1';
wait until not BCLK'stable and BCLK='0' and start = '1';
```

The HDL Compiler can synthesize multiple `wait` statements in a process. However, multiple `wait` statements within a single process must not specify different clock sources, as this will produce an error similar to the following:

```
ERROR: design.vhd(line 49): Model error: More than one clock source (Done, CLK) in
a module.
```

Signal Assignment Statements

VHDL associates signals or ports with time. A signal assignment changes the value of a signal or a port.

In VHDL, when you assign a value to a signal, the value of a signal is not updated immediately. Instead, VHDL schedules it to update at a delta time (or a specified delay time).

The `<=` operator assigns values to signals or ports.

```
signal S1,S2: BIT;
signal S3,S4,S5: STD_LOGIC_VECTOR(7 downto 0);
process(S2,S3,S4)
begin
    S1 <= not S2;
    S5 <= S3 or S4;
end process ;
```

In a process, the value of the signal changes after a `wait` statement, or after the complete process is executed.

Variable Assignment Statements

A variable assignment changes the value of a variable. The value of a variable updates immediately after the assignment.

You can declare variables only in a process or a subprogram. VHDL uses variables for temporary storage; other concurrent statements cannot detect these variables.

The `:=` operator assigns values to variables.

The following is an example of using variable assignment statements:

```
subtype INT16 is INTEGER range 0 to 65535 ;
signal S1, S2: INT16 ;
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
signal GT: BOOLEAN ;
process(S1, S2)
  variable A, B : INT16 ;
  constant C : INT16 := 100 ;
begin
  A := S1 + 1 ;
  B := S2 * 2 - C ;
  GT <= A > B ;
end process ;
```

If Statements

An `if` statement creates a branch in the execution flow. Depending on how it evaluates the conditions, the HDL Compiler executes one or none of the enclosed sequences of statements.

The following example, which represents a multiplexer function, shows how to use `if` statements:

```
signal IN1, IN2, OU : STD_LOGIC_VECTOR(0 to 3);
signal SEL : STD_LOGIC_VECTOR(1 downto 0);
MUX : process (SEL, IN1, IN2)
begin
  if SEL = "00" then
    OU <= IN1;
  elsif SEL = "01" then
    OU <= IN2;
  elsif SEL = "10" then
    OU <= IN1 or IN2;
  else
    OU <= "0000";
  end if;
end process ;
```

Case Statements

A case statement is another form of conditional control that VHDL provides. It is a multi-way branch based on the value of a control expression.

A case statement is similar to an `if` statement, in that a condition is used to choose among a number of collections of statements. It is different, however, in that the HDL Compiler chooses case statements based on the value of an expression.

The following is an example of using case statements:

```
signal S1 : INTEGER range 0 to 7 ;
signal I1, I2, I3 : BIT ;
select_process :
process(S1, I1, I2, I3)
begin
  case S1 is
    when 0 | 2 =>
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
    OU <= '0' ;
  when 1 =>
    OU <= I1 ;
  when 3 to 5 =>
    OU <= I2 ;
  when others =>
    OU <= I3 ;
  end case ;
end process select_process ;
```

Loop Statements

A `loop` statement contains a loop body of statements to execute repeatedly, zero or more times.

The following example repeatedly executes the same statements enclosed in the loop, starting when the value of `Sec` is 0, until the value of `Sec` becomes 60.

```
process
  variable Sec : INTEGER range 0 to 60 ;
begin
  L1 : loop
    wait until CLK'event and CLK = '1' ;
    Sec := Sec + 1 ;
    exit L1 when (Sec = 60) ;
  end loop L1 ;
end process;
```

You can associate a loop statement with an iteration scheme, either `for` or `while`.

Using For Loops

A `for` loop is a sequential statement in a process that iterates over a number of values. For example:

```
for I in 1 to 10 loop
  I_SQUARED(I) := I * I ;
end loop ;
```

In this example, you do not need to declare the `I` loop index, and you cannot reassign a value within the loop. By default, the `I` index is an integer, and takes on the values 1 to 10. This expression returns the squares of all numbers from 1 to 10 (1, 4, 9, 16, 25, 36, 49, 64, 81, 100).

You can use a `downto` clause in a `for` loop to create a descending range. For example:

```
for I in 10 downto 1 loop
  I_SQUARED(I) := I * I ;
end loop ;
```

This example returns the same values as the preceding example, in reverse order (100, 81, 64, 49, 36, 25, 16, 9, 4, 1).

Using while Loops

Another iteration scheme is the `while` loop, where the statement is executed by first evaluating the `condition`.

If the result is `TRUE`, then the HDL Compiler repeatedly executes the statements in the loop body, until either the `condition` evaluates to `FALSE`, or an `exit` statement transfers control out of the loop body.

The following is an example of using a `WHILE` loop.

```
signal S      : INTEGER range 0 to 10 ;
process
  variable I : INTEGER range 0 to 10 ;
begin
  wait until (CLK'event and CLK = '0');
  I := 0 ;
  while (I < 10) loop
    wait until (CLK'event and CLK = '0');
    S <= I ;
    I := I + 1 ;
  end loop ;
end process ;
```

Each `while` loop statement must find at least one `wait` statement in each circular execution path. Otherwise, the HDL Compiler gives an error.

Next Statements

The `next` statement skips execution to the next iteration of an enclosing `loop` statement. If you nest loops in VHDL, then you can assign a label to each loop.

The `next` statement can include one or more `loop_label` parameters to specify particular loop labels to skip. If you do not use the `loop_label` parameter, then the `next` statement applies to the innermost enclosing loop.

The following example shows how to use the `next` statement.

```
L1 : while I < 10 loop
  L2 : while J < 20 loop
    Sum(I,J) := Sum(I,J) + DataIn;
    next L1 when Sum(I,J)<0;
  end loop L2 ;
end loop L1 ;
```

Exit Statements

The `exit` statement skips the remainder of an enclosing loop statement, and continues with the statement immediately after the exited loop.

As in the `next` statement, you can specify a particular enclosing loop to exit.

```
loop
B := B - A ;
exit when B > A ;
end;
```

Null Statements

The `null` statement in VHDL performs no action. the HDL Compiler ignores `null` statements, and proceeds to the next statement.

You can use this statement to explicitly state not to perform any action when a condition is true.

The following is an example of using a `null` statement:

```
variable SEL : INTEGER range 0 to 31;
variable V   : INTEGER range 0 to 31;
case SEL is
  when 0 to 15 => V := SEL;
  when others  => null;
end case;
```

Assertion Statements

The `assertion` statement indicates that you expect a specified condition to be true. If the condition is false, then this statement reports a diagnostic message.

The HDL Compiler ignores `assertion` statements.

Function and Procedure Calls

In behavioral design descriptions, subprograms are a convenient way to implement frequently used functions. A subprogram consists of sequential statements, just as a process does.

VHDL uses two types of subprograms:

- A procedure returns multiple values.
- A function returns a single value.

Procedure calls invoke procedures to execute in a process. You must declare the procedure in a declaration section of your VHDL design, before you can call the procedure. These procedures act as extensions to the behavior of the process statement.

Concurrent Statements

An architecture that describes the functionality of an entity consists of one or more concurrent statements. Each concurrent statement defines a unit of computation that reads signals, performs computations based on their value, and assigns computed values to signals.

Concurrent statements are executed in parallel. These statements can appear in any order in the architecture.

The types of concurrent statements in VHDL are:

- Process Statements
- Concurrent Signal Assignments
- Conditional Signal Assignments
- Selective Signal Assignments
- Block Statements
- Component Instantiations
- Generate Statements
- Functions and Procedures

Process Statements

A process consists of a set of sequential statements, but processes are themselves concurrent statements. To communicate with the rest of a design, a process reads values from, or writes values to, signals or ports that are declared outside the process.

You can use `process` statements to describe a combinational circuit. The following example of using a `process` statement describes a 2-to-1 multiplexer.

```
signal IN1, IN2, OUT : STD_LOGIC_VECTOR(0 to 3);
signal SEL : STD_LOGIC;

process (SEL, IN1, IN2)
begin
    if SEL = '0' then
        OUT <= IN1;
    else
        OUT <= IN2;
    end if;
end process ;
```

You can also use `process` statements to describe a sequential circuit. In the following example, on each 0-to-1 transition of the `CLK` signal, the process gates the output of the multiplexer into the `OUT` register.

```
signal CLK: BIT ;
signal IN1, IN2, OUT : STD_LOGIC_VECTOR(0 to 3);
signal SEL : STD_LOGIC;

process
begin
    wait until (CLK'event and CLK = '1') ;
    if SEL = '0' then
        OUT <= IN1;
    else
        OUT <= IN2;
    end if;
end process ;
```

When a `process` statement assigns a value to a signal, the value defines a driver of the signal. If more than one `process` statement, or other concurrent statements, assign a value to a signal, then that signal has multiple drivers.

In VHDL, if the signal is not a resolved data type, then you must define a resolution function for the signal:

```
signal OUT, IN1, IN2 : STD_LOGIC;
signal SEL1, SEL2: BIT;
P1: process (SEL1, IN1)
begin
    if SEL1 = '0' then
        OUT <= IN1;
    end if;
end process;
P2: process (SEL2, IN2)
begin
    if SEL2 = '0' then
        OUT <= IN2;
    end if;
end process;
```

In this example, the `OUT` signal has multiple drivers. Because `STD_LOGIC` is a resolved data type, it allows multiple drivers. See the [Resolution Functions](#) on page 1101 section for more information.

Concurrent Signal Assignments

A concurrent signal assignment statement represents an equivalent `process` statement that assigns values to signals.

The following example shows a concurrent signal assignment within a VHDL architecture.

```
architecture Description1 of EXAMPLE is
signal I1, I2, AND_OUT: BIT ;
begin
```

```
    AND_OUT <= I1 and I2;
end Description1 ;
```

This example is equivalent to the following:

```
architecture Description2 of EXAMPLE is
signal I1, I2, AND_OUT: BIT ;
begin
    process (I1, I2)
    begin
        AND_OUT <= I1 and I2;
    end process ;
end Description2;
```

Conditional Signal Assignments

A conditional signal assignment is a concurrent statement that has one target, but it can use more than one expression. If one condition evaluates to TRUE, then the HDL Compiler uses the corresponding expression; otherwise, the HDL Compiler uses the remaining expression. the HDL Compiler uses one and only one of the expressions at a time.

You can transform any conditional signal assignment into an equivalent `process` statement.

The following is an example of using conditional statements:

```
architecture Description1 of EXAMPLE is
    signal A, B, Z: BIT ;
    signal X: INTEGER ;
    constant C: BIT := '0' ;
begin
    Z <= A when ( X > 10 ) else
        B when ( X > 5 ) else
        C ;
end Description1 ;
```

This example is equivalent to the following process statement:

```
architecture Description2 of EXAMPLE is
    signal A, B, Z: BIT ;
    signal X: INTEGER ;
    constant C: BIT := '0' ;
begin
    process (A, B, X)
    begin
        if ( X > 10 ) then
            Z <= A ;
        elsif ( X > 5 ) then
            Z <= B ;
        else
            Z <= C ;
        end if ;
    end process ;
end Description1 ;
```

Selective Signal Assignments

A selective signal assignment can have only one target, and can have only one `with` expression. The HDL Compiler tests this value for a match, in a manner similar to the `case` statement. A selective signal assignment runs whenever the selected signal changes in any way.

You can describe any selected signal assignment using a `process` statement that contains a `case` statement.

The following is an example of a selective signal assignment:

```
architecture Description1 of EXAMPLE is
    signal A, B, Z: BIT ;
    constant C: BIT := '0' ;
begin
    with SEL select
        Z <= A when 0 | 1 | 2 ,
        B when 3 to 10,
        C when others ;
end Description1;
```

This example is functionally equivalent to the following:

```
signal A, B, Z: BIT ;
constant C: BIT := '0' ;
process (SEL, A, B)
begin
    case SEL is
        when 0 | 1 | 2 =>
            Z <= A ;
        when 3 to 10 =>
            Z <= B ;
        when others =>
            Z <= C ;
    end case ;
end process ;
```

Block Statements

A block statement consists of a set of concurrent statements. Blocks organize a set of concurrent statements (procedure calls) hierarchically.

```
architecture BHV of EXAMPLE is
    signal IN1 : INTEGER ;
begin
    B1 : block
        signal S : BIT ;
    begin
        B1_1 : block
            signal S : INTEGER ;
        begin
            S <= IN1 ;
        end block B1_1 ;
    end block B1 ;
end architecture BHV of EXAMPLE;
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
end block B1 ;
end BHV ;
```

Component Instantiations

Component instantiations describe a design hierarchy. At the point of instantiation, only the external view of the component (the names, types, and directions of its ports) is visible; signals internal to the component are not visible. The component instantiation statement adds an instance and connects components, to form a netlist of instances of a design.

```
architecture INST of STRUCTURE is
  component XOR_gate
    port (I0, I1 : in STD_LOGIC; O : out STD_LOGIC) ;
  end component ;
  component AND2_gate
    port( I0, I1 : in  STD_LOGIC; O : out STD_LOGIC) ;
  end component ;

  signal A, B, Tmp1, Tmp2 : STD_LOGIC ;
  -- use default binding
begin
  U1: XOR_gate
    port map( A, B, Tmp1); -- Positional Association
  U2: AND2_gate
    port map( I0=>A, I1=>B, O=>Tmp2);
    -- Named Association
end INST ;
```

You can use a component instantiation statement to add an instance of any component defined in a VHDL architecture. You can also pass generic constants to the lower-level components. For example:

```
library cs35 ;
use cs35.all ;
architecture instance of instance is
  component instancel
    generic ( N : integer; Y: integer);
    port ( in1,in2 : bit_vector( N downto Y);
          output : out bit_vector(N downto Y)
        );
  end component;

begin
  inst1: instancel generic map ( N => 7, Y=> 4)
    port map(in1( 7 downto 4),in2( 7 downto 4),
      output(7 downto 4));
  inst2: instancel generic map ( N => 3, Y=> 0)
    port map(in1( 3 downto 0),in2( 3 downto 0),
      output( 3 downto 0));
end;
```

Using Default Search Order

The preceding example has not yet defined the `instance1` component. You can use any of the methods described in this section to provide this information. When the HDL Compiler finds a component instantiation, it uses the following sequence to search for the entity for the component:

- Searches for a design in the working library. If an entity matches the name and port properties of the component, then the HDL Compiler binds the component to the entity.
- Searches for the entities in your design library, as specified in the VHDL description (the `cs35` library in this example). If an entity matches the name and port properties of the component, then the HDL Compiler uses the entity that it found.
- If the HDL Compiler cannot find an entity that matches the component, then it reports a warning.

Generate Statements

The `generate` statement is a concurrent statement that describes replicated structures. You must define the statement in an architecture.

VHDL uses two methods of generation: `for` and `if`.

Using FOR Generation

The `for` method describes a regular structure. The following, example uses four one-bit full adders to construct a four-bit adder.

```
architecture GEN_FOR of FULL_ADDER_4 is
  signal X, Y, Z : STD_LOGIC_VECTOR(3 downto 0) ;
  signal Cout    : STD_LOGIC;
  signal TMP     : STD_LOGIC_VECTOR(4 downto 0) ;
  component FULL_ADDER
    port ( A, B, C : in STD_LOGIC;
          S, Co   : out STD_LOGIC) ;
  end component ;
begin
  TMP(0) <= '0' ;
  G : for I in 0 to 3 generate
    FA : FULL_ADDER port map (
      X(I), Y(I), TMP(I), Z(I), TMP(I+1) ) ;
  end generate ;
  Cout <= TMP(4) ;
end GEN_FOR ;
```

In a `generate` statement, you do not need to declare the loop variable. You can describe any concurrent statement inside a `generate` statement.

Using IF Generation

The `if` method generates a regular structure that has some irregularity.

Use a `for-generate` statement to define the width of a design, and a set of `if-generate` statements to define the beginning, middle, and ending set of connections.

The following example describes a four-bit full adder, consisting of three one-bit full adders, and one half-adder:

```
architecture GEN_FOR of FULL_ADDER_4 is
    signal X, Y, Z : STD_LOGIC_VECTOR(3 downto 0) ;
    signal Cout    : STD_LOGIC;
    signal TMP     : STD_LOGIC_VECTOR(4 downto 1) ;
    component HALF_ADDER
        port ( A, B : in STD_LOGIC;
              S, Co : out STD_LOGIC) ;
    end component ;

    component FULL_ADDER
        port ( A, B, C : in STD_LOGIC;
              S, Co   : out STD_LOGIC) ;
    end component ;

begin
    G0: for I in 0 to 3 generate
        G1: if I = 0 generate
            HA: HALF_ADDER port map (
                X(I), Y(I),
                Z(I), TMP(I+1) ) ;
        end generate ;

        G2: if I >= 1 and I <= 3 generate
            FA: FULL_ADDER port map (
                X(I), Y(I), TMP(I),
                Z(I), TMP(I+1) ) ;
        end generate ;
    end generate ;
    Cout <= TMP(4) ;
end GEN_FOR ;
```

Unlike the sequential `if` statement, the `if-generate` statement cannot use `else` or `elsif` branches.

Subprograms

VHDL uses two types of subprograms: procedures and functions.

- A procedure runs as a statement, and can return one or more parameters, or no parameters.
- A function runs as an expression, and always returns just one parameter.

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

A special category of functions, called resolution functions, resolves multiple sources for a signal into a single value for that signal.

Functions and Procedures

Procedures can change the values of objects, if those objects are associated with the formal in, out, or inout mode parameters of the procedure.

Functions compute values, but unlike procedures, they do not change the value of any objects associated with the formal parameters of the function. A function must declare the type of value that the function returns.

Subprograms consist of subprogram declarations and subprogram bodies.

Subprogram declarations contain only interface information. The following are examples of subprogram declarations:

```
function Incrementer (Count: INTEGER) return INTEGER ;
    -- by default will be understood as:
    -- constant Count: in INTEGER
procedure Send ( signal CLK : in BIT ;
                Data      : INTEGER ;
                -- by default will be understood as:
                -- constant Data : in INTEGER;
                signal Ack   : BIT ;
                signal Ready : out BIT ;
                signal Wire  : out INTEGER ) ;
```

Subprogram bodies describe the functionality of the subprograms. The following examples are subprogram (procedure and function) bodies.

```
function Incrementer (Count : INTEGER) return INTEGER is
variable TEMP: INTEGER ;
begin
    if (Count >= 255) then
        TEMP := 0 ;
    else
        TEMP := Count + 1 ;
    end if ;
    return(TEMP) ;
end Incrementer ;

procedure Send ( signal CLK   : BIT ;
                Data      : INTEGER ;
                signal Ack   : BIT ;
                signal Ready : out BIT ;
                signal Wire  : out INTEGER ) is
begin
    wait until (CLK'event and CLK = '1' and Ack = '0') ;
    Wire <= Data ;
    Ready <= '1' ;
    wait until (CLK'event and CLK = '1' and Ack = '1') ;
    Ready <= '0' ;
end Send ;
```


Note: When a function is called, the function call must terminate by executing a return statement, which determines the value that the function call returns.

Resolution Functions

In VHDL, more than one source can drive a signal, but you must use a `resolution` function to resolve the multiple sources into a single value for the signal.

A resolved signal is created if either a signal declaration or its subtype declaration includes a resolution function.

The following is an example of such a declaration:

```
signal NODE : WIRE_AND BIT ;
subtype RESOLVED_STD is WIRE_OR STD_ULOGIC ;
```

The first declaration uses the `WIRE_AND` resolution function to resolve the `NODE` signal. Each time an event effects the `NODE` signal, the HDL Compiler calls the `WIRED_AND` function, and returns a `BIT`-type value.

The second declaration defines a resolved subtype named `RESOLVED_STD`. Signals declared using the `RESOLVED_STD` type are resolved signals.

For simulation, the resolution function can be any function, consisting of any VHDL program. But synthesis supports only a limited number of methods for handling resolution functions in hardware, such as a wired-or gate, a wired-and gate, or a tristate gate. In the HDL Compiler, the resolution functions must be either wired-and, wired-or, or tristate connections. You must specify a synthesis directive to indicate which method to use.

For example, a synthesis directive is inserted in the `wired_and` function. the HDL Compiler uses the `wire_and` gate to resolve the signal, regardless of the content of the function.

```
package pack_res_func is
    function wired_and ( input : bit_vector ) return bit ;
end ;
package body pack_res_func is
    function wired_and ( input : bit_vector )
        return bit is
        -- pragma resolution_method wired_and
        begin
            for i in input'range loop
                if ( input (i) = '0' ) then
                    return '0' ;
                end if;
            end loop ;
            return '1' ;
        end ;
    end ;
end ;
use work.pack_res_func.all;
entity res_func is
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
port ( clk      : bit;
      res_sig : out wired_and bit
    );
end ;
architecture res_func of res_func is
begin
    source_1 :
        process
        begin
            wait until clk'event and clk='1';
            res_sig <= '1' ;
            wait until clk'event and clk='1';
            res_sig <= '0' ;
        end process;
    source_2 :
        process
        begin
            wait until clk'event and clk='1';
            res_sig <= '1' ;
        end process;
end;
```

In the `STD_LOGIC_1164` package, the `STD_LOGIC` and `STD_LOGIC_VECTOR` data types are resolved types. A tristate function resolves multiple sources assigned to a signal of `STD_LOGIC` or `STD_LOGIC_VECTOR` type.

Packages

A VHDL package is a set of declarations and definitions for objects, functions, procedures, components, and attributes. This section describes the following types of packages:

- User-Defined Packages
- STANDARD and TEXTIO Packages
- IEEE STD_LOGIC_1164 Packages
- IEEE NUMERIC_BIT and NUMERIC_STD Packages

Note: The package name for the Cadence `qtref` library was formerly `qtref.qtref`, but is now `qtref.qtref_pkg`. If your VHDL source code uses the old Cadence package name, you need to change it.

User-Defined Packages

A package consists of two parts: a package declaration section and a package body.

Package Declarations

The package declaration defines the interface for the package. For example:

```
library IEEE ;
use IEEE.NUMERIC_BIT.all ;
package WATCH_PKG is
    subtype MONTH_TYPE is INTEGER range 0 to 12 ;
    subtype DAY_TYPE is INTEGER range 0 to 31 ;
    subtype BCD4_TYPE is UNSIGNED(3 downto 0) ;
    subtype BCD5_TYPE is UNSIGNED(4 downto 0) ;
    constant BCD5_1 : BCD5_TYPE := B"0_0001" ;
    constant BCD5_7 : BCD5_TYPE := B"0_0111" ;
    function BCD_INC (L : in BCD4_TYPE) return BCD5_TYPE ;
end WATCH_PKG ;
```

Package Bodies

A package body always has the same name as its corresponding package declaration, preceded by the reserved words `package body`.

The package body specifies the behavior of the package. Design and entities that use the package do not recognize the information in a package body. The package body inherits all declarations defined in the corresponding package declaration section.

The following example is a package body definition for the `WATCH_PKG` package:

```
package body WATCH_PKG is
    function BCD_INC (L : in BCD4_TYPE) return BCD5_TYPE is
        variable V, V1, V2 : BCD5_TYPE ;
    begin
        V1 := L + BCD5_1 ;
        V2 := L + BCD5_7 ;
        case V2(4) IS
            when '0' => V := V1 ;
            when '1' => V := V2 ;
        end case ;
        return (V) ;
    end BCD_INC ;
end WATCH_PKG ;
```

The `use` clause adds functionality from a package into a VHDL description. In the preceding example, if you compiled the package into the `WATCH_LIB` library, then you need to include the following two lines, so the design recognizes the information in the package.

```
library WATCH_LIB ;
use WATCH_LIB.WATCH_PKG.all;
```

STANDARD and TEXTIO Packages

The `STANDARD` package (IEEE 1076-1987) defines some useful data types, such as `INTEGER`, `BIT`, and `BOOLEAN` (see [Operators](#) on page 1080).

- The `STANDARD` package is automatically visible, and does not require a `use` clause.
- The `TEXTIO` package defines types and operations, for communication with a standard programming environment. This `TEXTIO` package is used mostly for simulation. Synthesis does not require it.

IEEE STD_LOGIC_1164 Packages

The IEEE Standard Logic Package defines the following useful data types:

```
type Std_ULogic is ('U','X','0','1','Z','W','L','H','-');
type Std_ULogic_vector is array (natural range <>) of std_ulogic;
subtype std_logic is resolved std_ulogic;
type std_logic_vector is (natural range <>) of std_logic;
SUBTYPE X01 IS resolved std_ulogic RANGE 'X' TO '1';
SUBTYPE X01Z IS resolved std_ulogic RANGE 'X' TO 'Z';
SUBTYPE UX01 IS resolved std_ulogic RANGE 'U' TO '1';
SUBTYPE UX01Z IS resolved std_ulogic RANGE 'U' TO 'Z';
```

The logic operators, such as `and`, `nand`, `or`, `nor`, `xor`, and `not`, are overloaded to apply to the data types.

[Table C-2](#) describes some useful functions that this type of package defines.

Table C-2 IEEE STD_LOGIC_1164 Functions

Function	Description
<code>to_bit(<S>, <XMAP>)</code>	Converts a data object of <code>STD_ULONGIC</code> or <code>STD_LOGIC</code> type to <code>BIT</code> type. If the value of <code><S></code> is not '0' or '1', then this function maps it into '0' or '1' according to the function specified in <code><XMAP></code> .
<code>to_StdULogic(<S>)</code>	Converts the value of the <code><S></code> signal from a <code>BIT</code> type to a data object of <code>STD_ULONGIC</code> type.
<code>rising_edge(<S>)</code>	Detects the leading edges of the <code><S></code> signal, which is a <code>STD_ULONGIC</code> type.
<code>falling_edge(<S>)</code>	Detects the falling edges of the <code><S></code> signal, which is a <code>STD_ULONGIC</code> type.

To use the definitions and functions of the `STD_LOGIC_1164` packages, your VHDL design must include the following statements.

```
library IEEE ;
use IEEE.STD_LOGIC_1164.all ;
```

IEEE NUMERIC_BIT and NUMERIC_STD Packages

The Standard Synthesis packages (IEEE 1076.3) include two VHDL packages: `NUMERIC_BIT` and `NUMERIC_STD`. Both packages are compiled into the IEEE library. The numerical array types in both packages use the same name, so you can use only one package at a time.

The `NUMERIC_BIT` package is based on the `BIT` type, and the `NUMERIC_STD` package is based on the `STD_LOGIC` type. Both packages interpret the `UNSIGNED` type as an unsigned binary integer, and the `SIGNED` type as a two's-complement binary integer.

This package defines two useful data types:

```
type UNSIGNED is array (NATURAL range <>) of BIT/STD_LOGIC;  
type SIGNED is array (NATURAL range <>) of BIT/STD_LOGIC;
```

Arithmetic operators (such as `+`, `-`, and `*`), and comparison operators, are overloaded to apply the two data types.

To use the definitions and functions of the `NUMERIC_BIT` packages, you must include the following statements in your VHDL design:

```
library IEEE ;  
use IEEE.NUMERIC_BIT.all ;
```

To use the definitions and functions of the `NUMERIC_STD` packages, you must include the following statements in your VHDL design:

```
library IEEE ;  
use IEEE.STD_LOGIC_1164.all ;  
use IEEE.NUMERIC_STD.all
```

[Table C-3](#) describes some useful functions that this type of package defines.

Table C-3 IEEE NUMERIC_BIT and NUMERIC_STD Functions

Function	Description
<code>to_integer(<arg>)</code>	Converts the signed or unsigned <code><arg></code> object into an object of integer type.
<code>to_signed(<arg>, <n>)</code>	Converts the <code><arg></code> integer into a signed number that is <code><n></code> bits long.
<code>to_unsigned(<arg>, <n>)</code>	Converts the <code><arg></code> integer into an unsigned number that is <code><n></code> bits long.

Function	Description
<code>resize(<arg>, <n>)</code>	Changes the size of the unsigned or a signed number <arg> to a number that is <n> bits long.
<code>rising_edge(<S>)</code>	Detects the leading edges of the <S> signal, which is a BIT data type.
<code>falling_edge(<S>)</code>	Detects the falling edges of the <S> signal, which is a BIT data type.

Arithmetic Operations

the HDL Compiler performs arithmetic operations on signed or unsigned numbers in your VHDL design, such as:

```
shift_left (<arg>, <n>)
shift_right (<arg>, <n>)
rotate_left (<arg>, <n>)
rotate_right (<arg>, <n>)
```

VHDL Synthesis Directives

The HDL Compiler provides several synthesis directives that you can use to do the following:

- Direct the process of translating behavioral (RTL) design descriptions into gate-level descriptions.
- Map functions to hardware components.
- Specify the asynchronous set/reset signal for cycle-fixed, multiple-wait descriptions.

For example, the HDL Compiler supports the following synthesis directives:

```
-- pragma translate_off
-- pragma translate_on
-- pragma synthesis_off
-- pragma synthesis_on
-- pragma resolution_method wired_and
-- pragma resolution_method wired_or
-- pragma resolution_method three_state
```

In addition, Cadence provides its own versions of these synthesis directives, that accomplish similar purposes:

```
-- quickturn translate_off
-- quickturn translate_on
-- quickturn synthesis_off
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
-- quickturn synthesis_on
-- quickturn wired_and
-- quickturn wired_or
-- quickturn three_state
```

A synthesis directive resembles a regular comment starting with two hyphens (--). However, if the word following these characters is `pragma` or `synopsys`, then the HDL Compiler interprets the remaining comment text as a directive, not a comment.

The HDL Compiler uses three groups of synthesis directives:

- Synthesis Off and On Directives
- Resolution Function Directives
- Asynchronous Set/Reset Directives

Synthesis off and on Directives

These directives stop and start synthesis of a gate-level netlist from the VHDL source design. You can use `synthesis_off` and `synthesis_on` anywhere in your design description.

The syntax is:

```
-- pragma synthesis_off
-- pragma synthesis_on
```

The HDL Compiler cannot synthesize some VHDL statements, such as `FILE` declaration and file-access subroutines in the `TEXTIO` package. These statements, however, are very useful for simulation and debugging purposes.

This directive masks out those segments which the HDL Compiler should not synthesize, but that a simulator should process. Translation into a gate-level description is enabled at the beginning of each VHDL source file.

The following example is a design description for a multiplexer and its test bench. A single architecture describes both processes. Synthesis does not need the test bench, so this example includes it in the VHDL code, but sets a `synthesis_off` directive, so the HDL Compiler does not synthesize it.

```
library IEEE ;
use IEEE.NUMERIC_BIT.all ;
entity MUX is
    port (X0i : buffer UNSIGNED(3 downto 0) ;
          X1i : buffer UNSIGNED(3 downto 0) ;
          Z2i : buffer UNSIGNED(3 downto 0) ;
          SEL : buffer BIT ) ;
end MUX;
architecture BHV of MUX is
begin
    MUX2 :
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
process(SEL,X0i,X1i)
begin
  case SEL is
    when '0' => Z2i <= X0i ;
    when '1' => Z2i <= X1i ;
  end case ;
end process ;

-- pragma synthesis_off
X0i <= "0011" ;
X1i <= "0101" ;
process
begin
  SEL <= '0' ;
  wait for 10 ns ;
  SEL <= '1' ;
  wait for 10 ns ;
  assert (FALSE)
    report "MUXs : Testing Complete "
    severity failure ;
end process ;

-- pragma synthesis_on
end BHV ;
```

Resolution Function Directives

These directives specify the resolution function for signals that have multiple drivers. The HDL Compiler supports the following three methods:

```
--pragma tri_state
--pragma wire_or
--pragma wire_and
```

The HDL Compiler uses the tristate buffer to implement the `tri_state` resolution function, OR gates to implement the `wire_or` function, and AND gates to implement the `wire_and` function.

The following example shows how to use a resolution function directive.

```
package pack_res_func is
  function wired_and ( input : bit_vector ) return bit ;
end ;

package body pack_res_func is
  function wired_and ( input : bit_vector )
    return bit is
    -- pragma resolution_method wired_and
  begin
    for i in input'range loop
      if ( input (i) ='0' ) then
        return '0' ;
      end if;
    end loop ;
    return '1' ;
  end ;
end ;
```


UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
use work.pack_res_func.all;
entity res_func is
    port ( clk      : bit;
          res_sig   : out wired_and bit
        );
end ;
architecture res_func of res_func is
begin
    source_1 :
        process
        begin
            wait until clk'event and clk='1';
            res_sig <= '1' ;
            wait until clk'event and clk='1';
            res_sig <= '0' ;
        end process;
    source_2 :
        process
        begin
            wait until clk'event and clk='1';
            res_sig <= '1' ;
        end process;
end;
```

Asynchronous Set/Reset Directives

These directives specify the asynchronous set/reset signal for cycle-fixed design descriptions, containing multiple wait statements. The syntax is:

```
--pragma reset_high
--pragma reset_low
--pragma reset <reset_signal_name>
```

The asynchronous reset signal either sets or resets initial values to selected signals after the circuit starts or resets.

A process can include only one asynchronous reset signal. The data type of the asynchronous reset must be either `BIT` or `STD_ULONGIC`, and you can specify the asynchronous reset only in the declaration section of your VHDL design.

- The pragma `reset_high` directive sets the signal to active HIGH. This is the default for the VHDL reset function.
- The pragma `reset_low` directive sets the signal to active LOW.

The pragma `reset <reset_signal_name>` directive resets all signals, from their initial value, to the current value of the specified `<reset_signal_name>` signal.

Using these directives, you can instruct the HDL Compiler to set or reset selected registers, after the asynchronous reset signal is set to `HIGH` or `LOW`.

The following VHDL description shows how to use an asynchronous reset directive, to specify an asynchronous reset signal (named `RESET`) at the algorithmic level. When `RESET` becomes HIGH, it sets all signals that have an initial value, to the current value of the `RESET` signal.

The following example sets the `CountOut` output signal to 0:

```
entity Counter is
  port ( CLOCK      : in      BIT ;
        RESET      : in      BIT ;
        CountOut    : buffer INTEGER range 0 to 15 := 0 ) ;
end Counter;
architecture Algorithm of Counter is
  -- pragma reset_high reset
begin
  process
  begin
    wait until CLOCK'event and CLOCK = '1' ;
    CountOut <= (CountOut + 1) mod 16 ;
  end process;
end Algorithm ;
```

The following is a functionally equivalent description:

```
architecture FSMD of Counter is
begin
  process (RESET, CLOCK)
  begin
    if (RESET = '1') then
      CountOut <= 0;
    elsif (CLOCK'event and CLOCK = '1') then
      CountOut <= (CountOut + 1) mod 16 ;
    end if;
  end process ;
end FSMD ;
```

Enabling or Disabling Memory Synthesis

HDL Compiler will automatically choose to use the `hardmacro` memories for constructs for which they are suitable. In addition, HDLICE provides fine granular control of memory synthesis with two synthesis directives.

❑ `no_hardmacro`

This command disables memory synthesis for the signal specified. For example, if there is a memory declared in the RTL:

```
reg [1024:9] mem [23:13];
```

You can add the synthesis directive after the declaration,

```
//quickturn no_hardmacro mem
```

❑ `use_hardmacro`

This command forces the signal to be synthesized into hardmacro memories. The usage is similar to `no_hardmacro`.

Both synthesis directives are effective when memory synthesis option is turned on. They can be used with either Verilog or VHDL designs.

Syntax and Semantic Restrictions

The HDL Compiler synthesis tool for VHDL designs is based on VHDL 1076-1993, which can be fully simulated, but not fully synthesized.

The HDL Compiler has the following restrictions on using the VHDL language in synthesis:

- Accepts, but cannot operate on (and therefore ignores), the `FILE`, `REAL`, `ACCESS`, and `PHYSICAL` data types.
- Does not support `REGISTER` and `BUS` signal declarations.
- Ignores assertion statements.
- Does not support the `BEHAVIOR`, `STRUCTURE`, `LAST_EVENT`, `LAST_ACTIVE`, and `TRANSACTION` attributes.
- Ignores the `after` clause in signal assignment statements.
- When you import your design into the HDL Compiler, the loop index must evaluate to constants.
- When you use `wait` or `if` statements, they must include the `event` or `stable` edge detection attributes. These attributes detect the leading or falling edge of a signal. See [Sequential Statements](#) on page 1087, for details.
- Ignores the initial value of a signal (or variable) in a signal (variable) declaration.
- Supports design descriptions containing multiple `wait` statements, but you must specify either a synchronous reset or asynchronous reset signal (see [Wait Statements](#) on page 1087 and [Asynchronous Set/Reset Directives](#) on page 1109). Otherwise, the HDL Compiler might not be able to determine the power-up state in the real circuit.
- Extended identifiers are not supported.

Modeling Styles

Although your VHDL design style can be flexible, some modeling (or coding) styles produce better synthesis results than other styles. This section describes the coding styles that produce the best results when synthesizing for emulation.

Combinational Logic

A process without edge detecting attributes (`event` and `stable`) in `if` or `wait` statements is called a *combinational process*. A combinational process is usually synthesized as a combinational circuit (except on latch inferences, as described in the next section, [Latches](#) on page 1112).

The following is an example of a combinational circuit:

```
process (A,B,C,D,Sel)
begin
    if (Sel = '1') then
        Z <= B + C ;
    else
        Z <= A + D ;
    end if ;
end process ;
```

Latches

Flip-flops and latches are two commonly used, one-bit memory devices. A flip-flop is an edge-triggered memory device. A latch is a level-sensitive memory device.

The following example describes a level-sensitive latch:

```
-- latch inference
signal S, Data_In, Data_Out: BIT ;
. . .
process (S, Data_In)
begin
    if (S = '1') then
        Data_Out <= Data_In ;
    end if ;
end process ;
```

The sensitivity list includes the `S` and `Data_In` signals, which are required for processes without `wait` statements. This list indicates that whenever the `S` and `Data_In` signals change, the process is evaluated.

Because the assignment to the `Data_Out` signal is hidden in a conditional clause, `Data_Out` does not change value (preserves its old value) if `S` is zero (0). `Data_Out` is updated with the value of `Data_In` if `S` equals one (1). This is the behavior of a level-sensitive latch.

In general, latches are synthesized from conditional expressions that are not completely specified, in a combinational description. Any signal or variable that is not driven under all conditions, becomes a latched element.

Latch inferences normally occur with `if` or `case` statements. To avoid having a latch inferred, assign a value to the signal under all conditions. For example, if you add an `else` part to the `if` statement, then the `if` statement becomes a completely specified statement, and is therefore synthesized into a combinational circuit.

Flip-Flops

A process containing edge-detecting attributes (`event` and `stable`) in `if` or `wait` statements is called a `clocked process`. the HDL Compiler infers an edge-triggered flip-flop for signals in signal assignment statements. By detecting clock edges, the HDL Compiler can locate where to insert flip-flops, so that the synthesized gate-level design matches the simulation behavior.

The `event` attribute on a signal is the most commonly used edge-detecting mechanism. It operates on a signal, and returns a Boolean value. The result is true if the signal shows a change in value.

The `stable` attribute is the Boolean inversion of the `event` attribute.

The following example shows a basic VHDL program that describes a flip-flop. The process contains an `if` signal leading-edge statement. At each leading edge of the `CLK` signal, the content of `Data_In` is assigned to `Data_Out`. The rising edge of the clock is signified by “`clk=1`.”

```
signal CLK, Data_In, Data_Out: BIT ;
. . .
process ( CLK )
begin
    if (CLK'event and CLK = '1') then
        Data_Out <= Data_In ;
    end if ;
end process ;
```

Synchronous Sets and Resets

A synchronous set (reset) input presets (clears) the output of a flip-flop when it is asserted. The assignments takes effect only while the clock edge is active; at all other times, the flip-flop does not detect changes on these inputs.

All signal assignments that are inside the edge detection clause of an `if` signal, cause the values to be assigned to flip-flops along the signal edge. The HDL Compiler can model a flip-

flop with synchronous reset, by detecting whether a synchronous input was set on the leading or falling edge of the signal.

```
signal CLK, Data_In, Data_Out, S_RST: BIT ;
...
process ( CLK )
begin
    if (CLK'event and CLK = '1') then
        if (S_RST = '1') then
            Data_Out <= '0' ;
        else
            Data_Out <= Data_In ;
        end if ;
    end if ;
end process ;
```

Note: The S_RST and Data_In signals need not be on the sensitivity list of the process. Their changes do not result in any action inside the edge-detection clause of the if signal.

Asynchronous Sets and Resets

D-type flip-flops sometimes require asynchronous set or reset inputs. Asynchronous set (reset) inputs clear the output of flip-flops, when they are asserted independently of the clock.

The following VHDL program example describes a D-type flip-flop, with an asynchronous reset input:

```
signal CLK, A_RST, Data_In, Data_Out: BIT ;
...
process (CLK, A_RST)
begin
    if (A_RST = '0') then
        Data_Out <= '0' ;
    elsif (CLK'event and CLK = '1') then
        Data_Out <= Data_In ;
    end if ;
end process ;
```

The entity now has an extra input, the A_RST port, which resets the D flip-flop asynchronously. The CLK and A_RST signals must be in the sensitivity list. Whenever an event occurs on either the CLK or A_RST signal, the statements inside the process are executed.

The HDL Compiler first tests whether the A_RST signal value is 1 (active-HIGH asynchronous reset). If so, then the flip-flop output is reset to 0. Otherwise, the value of Data_In is assigned to the Data_Out signal, at the leading edge of the CLK signal.

You can describe a flip-flop that has more than one asynchronous input, as in the following example:

```
signal CLK, RST, PRST, EN: BIT ;
signal Data_In, Data_Out: BIT ;
process (CLK, PRST, RST, EN)
begin
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
if (PRST = '1') then
    Data_Out <= '1' ;
elsif (RST = '1') then
    Data_Out <= '0' ;
elsif (CLK'EVENT and CLK = '1') then
    if (EN = '1') then
        Data_Out <= Data_In ;
    end if ;
end if ;
end process ;
```

This circuit example performs a prioritized testing of the `PRST` signal, with respect to the `RST` signal. The `PRST` signal is tested before the `RST` signal. If the `PRST` signal is active, then the flip-flop is preset to 1, regardless of the state of the `RST` input.

If the `PRST` signal is inactive, then the flip-flop resets to 0 when the `RST` input is asserted. Otherwise, the `Data_In` is assigned to `Data_Out` synchronously, when the `EN` signal is asserted.

Registers

Various types of registers are used in a circuit. The following VHDL program example infers a basic 4-bit register:

```
-- 4-bit basic register
signal CLK, ASYNC: BIT ;
signal Din, Dout: BIT_VECTOR(3 downto 0) ;
. . .
process (CLK, ASYNC)
begin
    if (ASYNC = '1') then
        Dout <= "1100";
    elsif (CLK'event and CLK = '1') then
        Dout <= Din ;
    end if ;
end process ;
```

The clock pulse enables all flip-flops, so that the information currently available at the four inputs, can be transferred into the 4-bit register. The register is preset to an initial value of 1100 asynchronously, when the `ASYNC` signal becomes 1.

You can use a `load` signal to load input data into a register. When the `LOAD` input is 1, the inputs transfer into the register on the next clock pulse. When the `LOAD` input is 0, inputs are inhibited, so the D flip-flops maintain their present value.

If the technology library contains flip-flops with enable, then the HDL Compiler can use the library to build the circuit. If it does not find these flip-flops, then the HDL Compiler uses flip-flops and multiplexers to synthesize a circuit that meets the behavior of the description.

```
-- 4-bit parallel load register
signal CLK, ASYNC, LOAD: BIT ;
signal Din, Dout: BIT_VECTOR(3 downto 0) ;
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
. . .
process (CLK, ASYNC)
begin
  if (ASYNC = '1') then
    Dout <= "1100";
  elsif (CLK'EVENT and CLK = '1') then
    if (LOAD = '1') then
      Dout <= Din ;
    END IF ;
  END IF ;
end process ;
```

If you declare a range of values for a register, such as:

```
reg [2:0] div
```

You cannot assign a register value that is outside of the declared range, such as:

```
wire div[3]
```

Asynchronous Counters

Counters are sequential logic circuits that proceed through a well-defined sequence of states. An asynchronous counter does not use a synchronizing clock pulse to control state changes. The most common type of asynchronous counter is the binary ripple counter. A binary ripple counter consists of a series connection of complementing flip-flops (T type), with the output of each flip-flop connected to the clock input of the next higher-order flip-flop. The flip-flop holding the least significant bit, receives the incoming count pulses.

```
-- 4-bit binary counter
signal CLK, RESET: BIT ;
signal COUNT: BIT_VECTOR(3 DOWNTO 0) ;
. . .
process (CLK, COUNT, RESET)
begin
  if RESET = '1' then
    COUNT <= "0000";
  else
    if CLK'event and CLK = '1' then
      COUNT(0) <= not COUNT(0) ;
    end if ;
    if COUNT(0)'event and COUNT(0) = '0' then
      COUNT(1) <= not COUNT(1) ;
    end if ;
    if COUNT(1)'event and COUNT(1) = '0' then
      COUNT(2) <= not COUNT(2) ;
    end if ;
    if COUNT(2)'event and COUNT(2) = '0' then
      COUNT(3) <= not COUNT(3) ;
    end if ;
  end if ;
end process ;
```


Synchronous Counters

If all flip-flops of a counter are controlled by a common clock signal, then it is a synchronous counter. In a synchronous counter, the storage elements simultaneously examine their inputs and determine new outputs. This is the preferred way to build counters.

For example, the following program example describes a synchronous counter with asynchronous reset, synchronous load, and up or down count. When the `Load` signal equals 1, then the count sequence is disabled, and the external input is transferred into the counter. If the `Load` signal is 0 and the `Count` signal is 1, then the circuit operates as a counter. Depending on the value of the `Updown` signal, the clock pulses cause the state of the register to either count up or count down. If both `Load` and `Count` are 0, then the clock pulses do not change the state of the register.

```
-- 4-bit synchronous counter
signal CLK, RESET : BIT ;
signal Load, Count, UpDown : BIT ;
signal DataIn : INTEGER range 0 to 15 ;
signal REG : INTEGER range 0 to 15 := 0 ;
. . .
process (CLK, RESET)
begin
    if RESET = '1' then
        REG <= 0 ;
    elsif CLK'event and CLK = '1' then
        if Load = '1' then
            REG <= DataIn ;
        else
            if Count = '1' then
                if UpDown = '1' then
                    REG <= (REG + 1) mod 16 ;
                else
                    REG <= (REG - 1) mod 16 ;
                end if ;
            end if ;
        end if ;
    end if ;
end process ;
```

Tristate Buffers

Besides 0 and 1, digital systems can have a third signal value: the *high-impedance* state, usually denoted by the *Z* symbol. When the output of a gate is in a high-impedance state, the gate acts as if it were disconnected from the output. Gates that can be placed in such a state are called *tristate* gates. They can produce outputs of the following three values: 0, 1, and *Z*.

In addition to its conventional inputs, a tristate gate has one more input called *output enable*. If this input is not asserted, then the output is in its high-impedance state, and the gate is effectively disconnected from the output wire.

If the output enable is asserted, then the data inputs of a gate determine its output.

The predefined types of the `STANDARD` package do not include a type that describes the high-impedance value. To specify the high-impedance value in VHDL, you must use the `STD_LOGIC` multi-value logic type in the `STD_LOGIC_1164` package. When a signal is assigned the `Z` value, the output of the tristate gate is disabled. The HDL Compiler synthesizes a tristate buffer between the `Din` and `Dout` signals. The `OE` signal controls the state of the tristate buffer. In the `if` clause, both `Din` and the `OE=0` condition can be full expressions.

```
library IEEE ;
use IEEE.STD_LOGIC_1164.all ;

architecture IMP of Tri_State_Buf is
signal Din, OE, Dout : STD_LOGIC ;
begin
    . . .
    process(OE, Din)
    begin
        if (OE = '0') then Dout <= 'Z';
        else Dout <= Din;
        end if;
    end process;
end IMP ;
```

Another handy notation of a tristate buffer is to use a conditional signal assignment, as in the following example:

```
architecture Implement of Tri_State_BUF is
begin
    Dout <= Din when OE = '1' else 'Z' ;
end Implement ;
```

Buses

You can use tristate gates, instead of multiplexers, to construct a bus system. The outputs of four buffers are connected, to form a single bus line. The control inputs to the buffers, `S(0)` and `S(1)`, determine which one of the four normal inputs (`R0-R3`) communicates with the bus line. The circuit designer must guarantee that no more than one buffer is in the active state at any time. Only one of the connected tristate buffers can have access to the bus line; all other buffers must be maintained in a high-impedance state.

```
library IEEE ;
use IEEE.STD_LOGIC_1164.all ;

entity BUS_EX is
    port ( S_ : in STD_LOGIC_VECTOR(1 downto 0) ;
           OE : buffer STD_LOGIC_VECTOR(3 downto 0) ;
           R0, R1, R2, R3 : in STD_LOGIC_VECTOR(7 downto 0) ;
           BusLine : out STD_LOGIC_VECTOR(7 downto 0) ) ;
END BUS_EX ;

architecture Implement of BUS_EX is
begin
    process (S)
    begin
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
case (S) is
  when "00" => OE <= "0001" ;
  when "01" => OE <= "0010" ;
  when "10" => OE <= "0100" ;
  when "11" => OE <= "1000" ;
  when others => null ;
end case ;
end process ;
BusLine <= R0 when OE(0) = '1' else 'ZZZZZZZ' ;
BusLine <= R1 when OE(1) = '1' else 'ZZZZZZZ' ;
BusLine <= R2 when OE(2) = '1' else 'ZZZZZZZ' ;
BusLine <= R3 when OE(3) = '1' else 'ZZZZZZZ' ;
end Implement ;
```

Normally, simultaneous assignment to a signal (such as `BusLine` in the preceding example) is not allowed at the architectural level, unless a resolution function is provided. However, in VHDL, the `STD_LOGIC` or `STD_LOGIC_VECTOR` data types are already resolved (a resolution function is provided in `STD_LOGIC_1164`). Therefore, signals declared with these two data types can have multiple drivers.

Moore Machines

In the Moore model of sequential circuits, the outputs are functions only of the present state. You can represent a Moore model of a sequential circuit as a state diagram, with the circles marked to show the current state and output, and directed edges marked with the input only.

The following example shows a VHDL description for the Moore machine. In this example, the asynchronous reset signal clears the content of the `O` register, and resets the finite state machine to its initial state. At each leading edge of the `CLK` signal, the value of the next state is assigned into the current state. The rest of the statements describe the combinational logic, which generates the next state and the outputs of the design. The outputs are computed based on the current state only, so the assignments of output signals cannot depend on the input signals.

```
entity MOORE is
  port ( CLK : in  BIT ;
         RST : in  BIT ;
         I   : in  BIT ;
         O   : out BIT ) ;
end MOORE ;

architecture Implement of MOORE is
begin
  process (CLK, RST, I)
    type STATE_TYPE is (ST0,ST1,ST2,ST3);
    variable State,Next_State : STATE_TYPE ;
  begin
    if (RST = '1') then
      O <= '0' ;
      State := ST0 ;
    elsif (CLK'event and CLK = '1') then
      State := Next_State ;
    end if ;
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
case State is
when ST0 =>
    O <= '0' ; -- output depends on present state
    if (I = '0') then Next_State := ST0 ;
    else Next_State := ST1 ;
    end if ;
when ST1 =>
    O <= '0' ;
    if (I = '0') then Next_State := ST0 ;
    else Next_State := ST2 ;
    end if ;
when ST2 =>
    O <= '1' ;
    if (I = '0') then Next_State := ST3 ;
    else Next_State := ST2 ;
    end if ;
when ST3 =>
    O <= '1' ;
    if (I = '0') then Next_State := ST3 ;
    else Next_State := ST0 ;
    end if ;
end case ;
end process ;
end Implement ;
```

Asynchronous Mealy Machines

In the Mealy model, the outputs are functions of both the present state and current inputs. The outputs can change if the inputs change during the clock pulse period. The outputs might momentarily show false values, because of the delay encountered from the time that the inputs change, to the time that the flip-flop outputs change. To synchronize an asynchronous Mealy machine, the inputs of the sequential circuit must be synchronized with the clock, and the outputs must be sampled only during the clock-pulse transition.

In a state transition diagram representation of a Mealy machine, both the input and output values are included along the directed lines between the circles. The following example shows the corresponding VHDL program:

```
entity ASYNC_MEALY is
port ( CLK      : in  BIT ;
      RST      : in  BIT ;
      I        : in  BIT ;
      O        : out BIT ) ;
end ASYNC_MEALY ;

architecture Implement of ASYNC_MEALY is
begin
    process (CLK, RST, I)
        type STATE_TYPE is (ST0,ST1,ST2,ST3,ST4) ;
        variable State,Next_State : STATE_TYPE ;
    begin
        if (RST = '1') then
            State := ST0;
        elsif (CLK'event and CLK = '1') then
            State := Next_State ;
        end if ;
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
case STATE is
  when ST0 =>
    if (I = '0') then O <= '0'; Next_State := ST1 ;
    else              O <= '1'; Next_State := ST0 ;
    end if ;
  when ST1 =>
    if (I = '0') then O <= '0'; Next_State := ST1 ;
    else              O <= '0'; Next_State := ST2 ;
    end if ;
  when ST2 =>
    if (I = '0') then O <= '0'; Next_State := ST0 ;
    else              O <= '0'; Next_State := ST3 ;
    end if ;
  when ST3 =>
    if (I = '0') then O <= '0'; Next_State := ST3 ;
    else              O <= '1'; Next_State := ST4 ;
    end if ;
  when ST4 =>
    if (I = '0') then O <= '1'; Next_State := ST2 ;
    else              O <= '0'; Next_State := ST0 ;
    end if ;
end case ;
end process ;
end Implement ;
```

At each leading edge of the CLK signal in this example, the value of the next state is assigned into the current state. The combinational section describes the combinational logic, for generating both the next state and the outputs of the design. The HDL Compiler generates the output signals, based on the value of the input signals.

Algorithmic State Machines

You can represent the behavior of your design as a flow-chart description, such as an Algorithmic State Machine (ASM). An ASM chart contains three basic elements: the state box, the decision box, and the conditional output box.

- A state in the control sequence is indicated by a state box (a rectangular box).
- The decision box, which describes the effect of an input on the system, is a diamond-shaped box that has two or more exit paths.
- The conditional output box is represented as an oval box. The input path of a conditional box must originate from one of the exit paths of a decision box. The operations listed inside the conditional box are executed during a specified state, if the input condition is satisfied.

The following example is a VHDL program for an Algorithmic State Machine that calculates the Greatest Common Divider (GCD) of two numbers.

```
architecture FSMD of GCD is
begin
  Main :
    process (RESET, CLOCK)
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
variable X, Y: INTEGER range 0 to 255 ;
type S_Type IS (ST0 , ST1 );
variable State : S_Type := ST0 ;
variable TMP_0, TMP_1 : BOOLEAN ;
begin
  if (RESET = '1') then
    OUT <= 0 ;
    State := ST0 ;
  elsif (CLOCK'event and CLOCK = '1') then
    case State is
      when ST0 =>
        if (START = '1') then
          X := XI ;
          Y := YI ;
          State := ST1 ;
        else
          State := ST0 ;
        end if ;
      when ST1 =>
        if (X /= Y) then
          if (X < Y) then
            Y := Y - X ;
          else
            X := X - Y ;
          end if ;
          State := ST1 ;
        else
          OUT <= X ;
          State := ST0 ;
        end if ;
      end case ;
    end if ;
  end process Main ;
end FSM_D ;
```

Cycle-Fixed Algorithm Using Multiple Waits

You can use sequential statements in VHDL to describe any algorithm. When modeling an algorithm, the HDL Compiler uses signal assignment statements, to communicate externally. It also uses `wait` statements, to synchronize the circuit execution with outside events.

You can use VHDL to describe an algorithmic behavior, if you properly insert the `wait` statements to delimit the clock boundaries.

The following VHDL example describes how the Greatest Common Divider (GCD) of two numbers is calculated:

```
entity GCD is
  port ( CLOCK : in  BIT ;
        RESET : in  BIT ;
        XI,YI : in  INTEGER range 0 to 255 ;
        READY : in  BIT ;
        OUT   : out INTEGER range 0 to 255 := 0 ) ;
end GCD ;
architecture BHV of GCD is
  -- mebs reset_high RESET
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
begin
Main : process
variable X, Y : INTEGER range 0 to 255 ;
begin
wait until(CLOCK'event and CLOCK = '1' and READY = '1');
X := XI ;
Y := YI ;
while (X /= Y) loop
wait until(CLOCK'event and CLOCK = '1');
if (X < Y) then
Y := Y - X ;
else
X := X - Y ;
end if ;
end loop ;
OUT <= X ;
end process Main ;
end BHV ;
```

The program includes two `wait` statements. When the `READY` signal is `HIGH` at a leading edge of the `CLOCK` signal, then the two eight-bit signals move into the `X` and `Y` local variables. The `while ... loop` statement computes the GCD of the two numbers.

Note: Because a `wait` statement is inserted in the `while` statement, every iteration executes in exactly one clock cycle. When the computation is over, the result moves to the `OUT` output port.

Synchronous Reset in Multiple-Wait Descriptions

When a synchronous reset signal is asserted (active-`HIGH` or active-`LOW`), the synthesized design is forced to the state that is the equivalent of the beginning of the process. Some of the registers are also either cleared, or preset to certain values. In VHDL, you need to test a synchronous reset signal after each `wait` statement.

```
entity GCD is
port ( CLOCK : in BIT ;
S_RESET : in BIT ;
XI,YI : in INTEGER range 0 to 255 ;
READY : in BIT ;
OUT : out INTEGER range 0 to 255 := 0 ) ;
end GCD ;

architecture BHV of GCD is
begin
Main : process
variable X, Y : INTEGER range 0 to 255 ;
begin
s_reset_loop: loop
wait until(CLOCK'event and CLOCK = '1' and READY = '1');
exit s_reset_loop when s_reset = '1';
X := XI ;
Y := YI ;
while (X /= Y) loop
wait until(CLOCK'event and CLOCK = '1');
exit s_reset_loop when s_reset = '1';
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
        if (X < Y) then
            Y := Y - X ;
        else
            X := X - Y ;
        end if ;
    end loop ;
    OUT <= X ;
end loop s_reset_loop;
OUT <= 0;
end process Main ;
end BHV ;
```

Asynchronous Reset in Multiple-Wait Descriptions

In a design description with multiple `wait` statements, you must use directives that specify the asynchronous reset signal. After detecting the directive, the HDL Compiler connects the asynchronous reset line to the state registers. This connection sets the state registers to the initial state, after the reset signal is asserted. An asynchronous reset directive starts with `pragma`, followed by `reset_high reset_low`, indicating that it is first an active-HIGH signal, and then the asynchronous reset signal.

The following is an example of using asynchronous reset in a multiple-wait description.

```
entity GCD is
    port ( CLOCK : in  BIT ;
          RESET : in  BIT ;
          XI,YI : in  INTEGER range 0 to 255 ;
          READY : in  BIT ;
          OUT   : out INTEGER range 0 to 255 := 0 ) ;
end GCD ;

architecture BHV of GCD is
    -- pragma reset_high RESET
begin
    Main : process
        variable X, Y : INTEGER range 0 to 255 ;
    begin
        wait until(CLOCK'event and CLOCK = '1' and READY = '1');
        X := XI ;
        Y := YI ;
        while (X /= Y) loop
            wait until(CLOCK'event and CLOCK = '1');
            if (X < Y) then
                Y := Y - X ;
            else
                X := X - Y ;
            end if ;
        end loop ;
        OUT <= X ;
    end process Main ;
end BHV ;
```


Configuration Declaration

Configuration declaration specifies binding information of component instances to design architectures, such as the architecture to use and/or generic constants of a specific instance, which could be inside a for-generate statement or if-generate statement. The HDL Compiler supports a subset of configuration declaration commonly used in RTL synthesis. (See also [Unsupported Constructs in Configuration Declaration](#) on page 1126.)

Configuration declaration could be used to specify a particular architecture of an instance, for example `ripple_adder` or `carry_save_adder`.

```
CONFIGURATION cfg1 OF loopcntTop IS
  FOR rtl
    FOR loopcnt_inst1 : loopcnt
      USE ENTITY work.loopcnt(ripple_adder);
    END FOR;
    -- all unspecified instances use default binding
  END FOR;
END CONFIGURATION;
```

Configuration declaration could associate a different value to a generic constant of an instance, for example the depth of a FIFO.

```
CONFIGURATION cfg1 OF loopcntTop IS
  FOR rtl
    FOR fifo_inst1, fifo_inst2 : fifo
      GENERIC MAP (DEPTH => 55);
    END FOR;
    FOR OTHERS: fifo
      -- applies to all other instances of fifo,
      -- except explicitly listed ones
      GENERIC MAP (DEPTH => 75);
    END FOR;
  END FOR;
END CONFIGURATION;

CONFIGURATION cfg2 OF loopcntTop IS
  FOR rtl
    FOR ALL : fifo
      -- applies to all instances of fifo
      GENERIC MAP (DEPTH => 55);
    END FOR;
  END FOR;
END CONFIGURATION;
```

Instances in a for-generate statement could be bound to different architectures or different values of generic constants.

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

For example:

```
CONFIGURATION cfg1 OF loopcntTop IS
  FOR rtl
    FOR genAnd(0 to 3)
      -- applies to a for-generate block labeled 'genAnd'
      FOR add_inst : add
        USE ENTITY add(add_archA);
      END FOR;
    END FOR;
    FOR genAnd(8)
      FOR add_inst : add
        USE ENTITY add(add_archB);
      END FOR;
    END FOR;
  END FOR;
END CONFIGURATION;
```

Nested configuration is supported.

```
CONFIGURATION cfg1 OF loopcntTop IS
  FOR rtl
    FOR fifo_inst1, fifo_inst2 : fifo
      USE CONFIGURATION fifo_cfg1;
    END FOR;
  END FOR;
END CONFIGURATION;
```

Restrictions in Configuration Declaration

- Generic constants of record types are not supported.
- A configuration identifier should be unique inside a library. It cannot be used by another entity or package.

Unsupported Constructs in Configuration Declaration

- Configuration specification inside a declarative portion of an architecture is not supported.
- The configuration declarative part, such as the USE clause and ATTRIBUTE clause, is not supported.

For example, the following is not supported.

```
LIBRARY mylib;
CONFIGURATION cfg1 OF loopcntTop IS
  -- This is the declarative part --
```

UXE User Guide

Modeling Guide for VHDL Designs for ICE Mode

```
USE mylib.all; -- this is not supported
FOR str
  FOR loopcnt_3 : loopcnt
    -- This is the component specification part --
    USE CONFIGURATION work.cfg3; -- this is supported
  END FOR;
END FOR;
END CONFIGURATION;
```

- **USE OPEN in component specification is not supported.**

For example, the following is not supported.

```
CONFIGURATION cfg1 OF loopcntTop IS
  FOR str
    FOR loopcnt_3 : loopcnt
      USE OPEN; -- this is not supported
    END FOR;
  END FOR;
END CONFIGURATION;
```

- **PORT MAP aspect in component specification is not supported.**

UXE User Guide
Modeling Guide for VHDL Designs for ICE Mode

Guidelines for Writing Testbenches

The simulation process consists of two components that together allow you to simulate almost any Verilog design:

- The emulator, which accelerates *synthesizable* Verilog code
- The IES software simulator, the co-simulation component, which simulates the *non-synthesizable* parts of your design, including C/C++ code linked through the Programming Language Interface (PLI)

In a typical design, the *chip*, or Device Under Test (DUT), contains mostly or entirely synthesizable code, while the *testbench* contains non-synthesizable, behavioral-style code that cannot be accelerated with the emulator.

This chapter provides you with guidelines for writing testbenches that can be accelerated by the emulator.

 Important

The guidelines provided in this chapter are applicable only to the SA mode.

Accelerating even a small portion of the testbench can provide significant overall performance improvement. The following aspects of testbench development are covered:

- Following a Synthesizable Coding Style
- Using Initial Blocks with (Verilog Only)
- Specifying Clocks
- Applying Stimulus to Your Design
- Avoiding Testbench Race Conditions (Verilog Only)
- Using Hierarchical References in Testbench (Verilog Only)
- Accessing Memories Using Hierarchical References
- Using Verilog PLI in the Testbench

■ Dealing with Testbench Monitoring Code

Note: While it might be a good idea to change your testbench to improve performance, Cadence does not recommend that you make changes to the chip itself for this purpose. You should always follow a methodology that will allow you to best realize the chip into actual hardware.

Following a Synthesizable Coding Style

To maximize simulation performance, you should write as much of your testbench as possible in the *synthesizable* coding style.

In addition to the Verilog and VHDL constructs allowed by the synthesis subset defined by Synopsys, IXCOM also lets you make use of Verilog cross-module references (XMRs) or VHDL global signals to read registers and wires inside the Device Under Test. This feature is especially useful for accelerating *monitor* code, which is contained by many testbenches. When writing the testbench, try to keep all non-synthesizable code in separate modules; these will not be accelerated by the emulator.

Using Initial Blocks with(Verilog Only)

IXCOM supports the use of Verilog `initial` blocks anywhere in your design, even in the Device Under Test (DUT). The DUT is automatically run in software simulation mode for at least one simulation time-tick in order to execute the `initial` block statements.

`initial` blocks are especially useful for initializing registers and memories and for applying basic stimulus to the design. If you are using an `initial` block in a module running in the software simulator only, then you can use any construct that the Verilog language allows to add to `initial` blocks.

However, if the `initial` block is used in a module in the DUT that can be accelerated, certain constructs will delay the time when you can swap your design to Simulation Acceleration mode or prevent it from happening altogether. This is because all events generated by `initial` blocks must be completed while in software simulation mode before swapping can occur. These constructs are described in the following section.

This section contains the following:

- Initial Block Statements That Delay Swapping to Hardware
- Initial Block Statements That Prevent Swapping to Simulation Acceleration Mode

Initial Block Statements That Delay Swapping to Hardware

This section tells you about `initial` block statements that *delay* swapping to Simulation Acceleration mode.

- **Delay specifications:** The design cannot be swapped to Simulation Acceleration mode until the delay period is over. If there are different delays in various `initial` blocks, the design cannot be swapped until the longest delay period is over, for example:

```
module dut(..);
  reg a;
  initial
  begin
    #10;           // cannot swap design to Simulation Acceleration
                  // or emulation mode until after #10

    a = 1'b0;
  end
endmodule
```

Note: Inserting delays in `initial` blocks does not typically reduce simulation performance if reasonable delay values are used.

- **Event control statements:** The design cannot be swapped to Simulation Acceleration mode until after the event is simulated, for example:

```
module dut(..);
  reg clk;
  initial
  begin
    @(posedge clk); // cannot swap to Simulation Acceleration mode until
    first posedge on clk
    a = 1'b1;
  end
endmodule
```

- **Wait statements:** The design cannot be swapped to Simulation Acceleration mode until all events are simulated.

Initial Block Statements That Prevent Swapping to Simulation Acceleration Mode

This section tells you about `initial` block statements that *prevent* swapping to Simulation Acceleration mode.

- **Forever statements:** Since events are generated continuously, the design can *never* be swapped to Simulation Acceleration mode .

```
module dut(..);
  reg clk;
  initial
  begin
    clk = 1'b0;
    forever #10 clk = ~clk;    // generates events forever--cannot swap
                                // design to Simulation Acceleration mode
  end
endmodule
```

Refer to [Specifying Clocks](#) on page 1132 for details on using `forever` statements for this reason.

- **PLI (Programming Language Interface) calls:** These calls persist throughout simulation. So, the design cannot be swapped to Simulation Acceleration mode.

```
module dut(..);
  reg a;
  initial
    $ARM7TDMI(...); // This PLI call persists throughout simulation;
                    // cannot swap design to Simulation Acceleration modes
endmodule
```

Note: The preceding `initial` block statements affect swapping to Simulation Acceleration mode *only* if used in a DUT module. Modules running exclusively in the software simulator (that cannot be accelerated) do not affect the swapping capability.

If you need to use any of the preceding constructs, move the `initial` block to a module that is not in the DUT, or explicitly exclude the module with the `+dutexcl+module_name` compiler option.

Specifying Clocks

There are two common methods for specifying design clocks.

Method 1

```
initial
begin
  clk = 1'b0;
  forever #10 clk = ~clk;
```


UXE User Guide

Guidelines for Writing Testbenches

```
end
```

Method 2

```
initial
    clk = 1'b0;
```

```
always
begin
    #10;
    clk = ~clk;
end
```

To accelerate the code for the above methods, Cadence recommends that a separate clock-generation module be added to the design. Then, specify this module with the `+iscDelay` option of the `ixcom` command.

Here is an example of a separate clock-generation module:

```
module synthesizable_testb ();
wire sys_clk;
```

```
clk_gen clk_gen1(sys_clk);
chip chip1 (sys_clk, ...);
..
<synthesizable code>
..
endmodule
```

```
module clk_gen(clk);
output clk;

initial
begin
    clk = 1'b0;
    forever #10 clk = ~clk;
end
```

```
endmodule
```

Here is the way you would run the compiler to exclude this clock-generation module from the DUT:

For a mixed-HDL or VHDL Design:

```
ixcom -top synthesizable_testb -ua +dut+synthesizable_testb +iscDelay+clk_gen
```

For a Verilog-only Design:

```
ixcom testbench.v chip.v -ua +dut+synthesizable_testb +iscDelay+clk_gen
```

Alternatively, you can include the `$ixc_ctrl` system task to ensure that the clock-generating module is mapped to the hardware.

```
initial $ixc_ctrl("map_delays");
```

Then, the module need not be specified with the `+iscDelay` option of the `ixcom` command.

Applying Stimulus to Your Design

This section describes how to apply stimulus to your design and contains the following sections:

- [Synchronous Testbenches](#)
- [Applying Test Vectors Using Memory \(\\$random\)](#)

Synchronous Testbenches

Many testbenches use delay specifications for applying stimulus to the design at specific intervals of time, for example:

```
initial
begin
    reset = 1'b0;
    #10 reset = 1'b1;
    #10 start_mem_cycle = 1'b1;
end
```

Since delay specifications are non-synthesizable, this coding style cannot be accelerated. Instead, use the edge of a clock to apply stimulus to the design. In the following synthesizable example, stimulus is applied on each negative edge of the clock. Since the design uses the positive edge, this ensures the timely application of stimulus before it is actually used by the design.

```
initial
    reset = 1'b0;

    always @(negedge clock)
```

UXE User Guide

Guidelines for Writing Testbenches

```
begin
    reset = 1'b1;
    @(negedge clock);    // Note: Must use same clock to be synthesizable
    start_mem_cycle = 1'b1;
    @(negedge clock);
    ...
end
```

Even if the intent is not to write a synthesizable testbench, Cadence recommends that you make use of clocks, instead of delays, to apply stimulus. The emulator simulates with zero delay and consequently might not exhibit the delay relationships between signals that you would expect. Applying stimulus on clock-edges results in much more predictable behavior.

Applying Test Vectors Using Memory (\$random)

A common method for applying stimulus is by using random test vectors generated with the `$random` system task (Verilog only). The test vectors are then applied to nets and registers in the design at regular intervals. However, the `$random` system task is non-synthesizable, so any module containing this task will not be accelerated.

An alternate method of applying randomly generated test vectors (or any other stimulus) that can be accelerated involves the use of special memories. Here is how this technique works:

1. Generate the test vectors outside of simulation, using the `rand` C function in a C program, or using a `perl` script, and write to an ASCII-formatted file.
2. Create a read-only memory structure.
3. Load the test vectors into the ROM using an `initial` block.
4. Read the ROM at specified intervals to extract the desired vectors.

Example

In this example, the 32-bit randomly generated vectors are in a file, `random.hex`, and are loaded into the ROM at initialization, then read into the circuit one vector at a time on the posedge of the system clock:

```
module testbench ();
    wire sys_clk;
    reg [31:0] address;
    reg [31:0] data;

    clk_gen clk_gen1(sys_clk);           // clock-generator in above example
```

UXE User Guide

Guidelines for Writing Testbenches

```
memory vmem (data, sys_clk, address);    // read vectors into
                                         // register "data"

initial
begin
    address = 0;                          // initialize ROM address counter
    $readmemh("random.hex",testbench.vmem); // load test-vectors into ROM
end

always @(posedge sys_clk)
    address <= address + 1;               // Increments ROM address counter

endmodule

module memory (data, clk, address);
parameter ADDRESS_WIDTH=32; // 2**32 test vectors supported by this memory
parameter DATA_WIDTH=32;  // 32-bit vectors
output [DATA_WIDTH-1:0] data;
input clk;
input [ADDRESS_WIDTH-1:0] address;

wire [DATA_WIDTH-1:0] axis_dout;
wire [ADDRESS_WIDTH-1:0] axis_addr = address;
wire axis_clk = clk;

assign data = axis_dout;

axis_smem #(ADDRESS_WIDTH,DATA_WIDTH, 1, 1'b0) mem // instantiates a
                                                    // read-only memory

({axis_dout},
 {{DATA_WIDTH{1'bz}}},
 {axis_addr},
 {1'b0},
 {1'b1},
 {axis_clk},
 {{8{1'bz}}});

endmodule
```

Avoiding Testbench Race Conditions (Verilog Only)

Some designers make use of `#delays` of various values to establish certain delay relationships between various signals, such as clock and data, in their designs. The emulator simulates with zero delay. Therefore, this strategy might not yield the desired results when using the emulator.

An alternate method that works as well as other simulators is to make use of blocking versus nonblocking assignment statements to establish relationships between signals. The most common race condition occurs between the data and clock signals to a flip-flop. In most cases, you would want the data signal to change value only after the clock edge used by the flip-flop, that is, the flip-flop would store the old value of the data signal. In this case, use blocking assignment statements when generating the clock signal and nonblocking assignment statements when assigning the data signal, for example:

```
module synthesizable_testb ();
  wire sys_clk;

  clk_gen clk_gen1(sys_clk);

  always @(posedge sys_clk)
    q <= d;           // nonblocking - old value of d assigned to q

endmodule

module clk_gen(clk);
  output clk;

  initial
  begin
    clk = 1'b0; // blocking assignment on clock - evaluates first
    forever #10 clk = ~clk;
  end

endmodule
```

Using Hierarchical References in Testbench (Verilog Only)

Many testbenches make use of hierarchical references (also known as cross-module references or XMRs) to reference nets and registers in the Device Under Test (DUT), as well

UXE User Guide

Guidelines for Writing Testbenches

as in other parts of your design. Hierarchical references can be used to initialize memories or to probe the values of signals during simulation.

When writing your testbenches, keep the following in mind, especially if you are attempting to accelerate parts of your testbench by adding them to the DUT:

- A special I/O signal is created for each DUT net or register bit that is referenced in a testbench module or other module running in the software simulator. Such I/Os incur a performance penalty for uploading and downloading their values through the PCI bus. To maximize simulation performance, keep hierarchical references of this type to a minimum.
- You can reference nets and registers in the DUT from your testbench only if they are being read. You can write to registers using system tasks, although this routine is not synthesizable and reduces simulation performance. Similarly, for a VHDL design, you can write to registers *that are edge-triggered flip-flops* in the DUT using system tasks. You cannot write to DUT nets.
- You can add testbench modules that are otherwise synthesizable, except for the usage XMRs, to the DUT to be accelerated. However, keep the following rules in mind:
 - ❑ You can reference nets and registers from one DUT module to another for both reading and writing.
 - ❑ You cannot reference nets and registers in the testbench or other modules running in the software simulator, for example:

- For a mixed-HDL or VHDL Design:

```
ixcom -top top_level_design_unit other_options +dut+chip+testb_in_dut
```

- For a Verilog-only Design:

```
ixcom Verilog_sources other_options +dut+chip+testb_in_dut
```

```
module test ();
  reg sys_clk;                      // testbench clock signal
  chip chip1 (...);
  testb_in_dut dut2 (...);
endmodule

module testb_in_dut(...);
  reg int_clk;
  initial
  begin
    int_clk = test.sys_clk; // reference to a testbench signal not
                          // allowed since module testb_in_dut is in the DUT
  end
endmodule
```

```
end  
endmodule
```

There is a workaround for this issue, assuming you are referencing the DUT net or register in an `initial` block, as in the previous example. The workaround is to use system tasks. This task, which has no parameters, returns a boolean `true` if the design is currently running in the software simulator or `false` if running in Simulation Acceleration mode. Since all designs are run at least 1 timestep in software simulation mode before swapping to Simulation Acceleration mode, `initial` blocks are always executed. For the previous example, make the following change:

```
module test ();  
    reg sys_clk;                // testbench clock signal  
    chip chip1 (...);  
    testb_in_dut dut2 (...);  
endmodule  
  
module testb_in_dut(...);  
    reg int_clk;  
    initial  
    begin  
        `ifdef AXIS  
            if ($axis_rccoff)      // code enabled in software simulation mode  
                int_clk = test.sys_clk; // XMR simulated as expected  
            `else  
            `endif  
        end  
    endmodule
```

The ``ifdef AXIS` compiler directive automatically instructs the simulator to include the `$axis_rccoff` system task, but it will be ignored by other simulators.

Some testbenches use Verilog `force` and `release` statements to set reference nets and registers to desired values. These statements are non-synthesizable, and modules containing them will not be accelerated.

Accessing Memories Using Hierarchical References

Many testbenches access embedded memories in the design for the purposes of monitoring the simulation for expected results, for example:

```
always @(packet_complete)  
    if (testbench.chip.sram.mem[0] == 1'b0)    // Using XMR to read memory  
        $display("Invalid header bit");
```

UXE User Guide

Guidelines for Writing Testbenches

If you wish to access your memories in this fashion, there are two things to keep in mind:

1. Cross-module references (XMRs) to memories are not synthesizable; modules containing this type of XMR will not be accelerated by the emulator.
2. Memory fetches in a hardware-based component, such as the emulator, can be expensive, especially if done frequently during simulation.

If you want to frequently access multiple addresses to the same memory, it is generally more efficient to upload the entire memory to a locally declared memory, perform the operation, then download the changed memory back to the -mapped memory. There are two system tasks for this purpose, `$axis_a2m` (Verilog) or `axis_a2m` (VHDL) and `$axis_m2a` (Verilog) or `axis_m2a` (VHDL). The `$axis_a2m` (Verilog) or `axis_a2m` (VHDL) task lets you upload an -mapped memory to a locally-declared Verilog memory, while the `$axis_m2a` (Verilog) or `axis_m2a` (VHDL) task downloads from the Verilog memory back to the -mapped memory.

For example, if your code is as follows:

```
reg [0:63] count;
always @(packet_complete)
begin
  for (i=0;i<64;i=i+1)
  begin
    testbench.chip.sram.mem[i] = count;    // Using XMR to write memory
    count = count + 1;
  end
end
```

You could change it to:

```
reg [0:31] local_mem [0:63];
reg [0:63] count;
always @(packet_complete)
begin
  $axis_a2m(testbench.chip.sram.mem, local_mem, 0, 63, 0, 63);
  // download 64 locations from -mapped memory to local memory
  for (i=0;i<64;i=i+1)
  begin
    local_mem[i] = count;
    count = count + 1;
  end
  $axis_m2a(local_mem, testbench.chip.sram.mem, 0, 63, 0, 63);
  // upload 64 locations from local memory to -mapped memory
end
```


Using Verilog PLI in the Testbench

Programs written using the Verilog Programming Language Interface (PLI) are run in software simulation. Since they are not accelerated, it is important to write them as efficiently as possible. Use the following guidelines when writing PLI calls.

Task/Function Routines Instead of Access Routines

Whenever possible, use task/function routines instead of access routines in your code. These simulate more efficiently in the simulator, as well as in other compiled-code simulators.

Reading and Writing Signals in DUT Modules

Palladium XP lets you cross-reference nets and registers in DUT modules from your testbench for the purpose of using them in PLI calls, assuming the following conditions are met:

- DUT nets and registers can be read only; you cannot write to these objects.
- DUT nets and registers can be read at run time, only if they are *exported* at compile-time. Exporting a signal from the DUT allows you to read its value during simulation. Signals are automatically exported if:
 - ❑ You reference the DUT net or register in your testbench or other module running in software simulation mode, for example:

```
always @(posedge testbench.chip.sys_clk)
    // read reference to DUT signal
```
 - ❑ For Verilog, you explicitly export the signal using either the `$export_read` or `$export_frcrel` Verilog system tasks from a testbench module. Refer to *Command and System Tasks for SA Mode* chapter of the *UXE Command Reference Manual* for detailed information on these tasks.
 - ❑ For VHDL, you explicitly export the signal using either the `export_read` or `export_frcrel` VHDL procedures from a testbench module. Refer to *Command and System Tasks for SA Mode* chapter of the *UXE Command Reference Manual* for detailed information on these procedures.
- Do not use access routines in your PLI code that attempt to access signals that were not exported.
- If you use **tf** routines, and you attempt to pass a reference to a DUT signal as a parameter to the PLI call, you will get an error during compile-time by default, for example:

```
initial
    $my_pli(testbench.chip.my_reg);
--
```

Error: XMR defined in DUT cannot be assigned in testbench

To address this issue, you must enable read access to the PLI routine. This is done with the `+tfReadOnly+pli_routine` compiler option. To enable read access for several PLI routines, append their names to this option:

```
+tfReadOnly+pli_routine1+pli_routine2+...
```

If you do not specify any routine names, read access will be enabled for all routines in the design.

You can specify the `+tfReadOnly` option with the `ixcom` command.

For a mixed-HDL Design:

```
ixcom -top top_level_design_unit other_options +tfReadOnly+my_pli
```

For a Verilog-only Design:

```
ixcom Verilog_sources other_options +tfReadOnly+my_pli
```

Dealing with Testbench Monitoring Code

The majority of typical testbench code involves the monitoring and manipulation of signals in the Device Under Test (DUT) to direct simulation or to detect error conditions occurring during simulation. This code is usually non-synthesizable and typically represents the largest performance bottleneck in simulation. There are several options to deal with this type of monitor code:

- Exclude the code completely from your simulation. This option provides the fastest performance, but sacrifices some debugging capabilities.
- Use the Testbench Callback technology to automatically execute the non-synthesizable code in the emulator. Refer to the [Executing Non-Synthesizable Code in the Emulator](#) section for more detail.
- Partition the monitor code into a synthesizable error-condition check and a non-synthesizable code segment that executes when the error is detected. This technique is described later in this section.
- Set the monitor code to be active in software simulation mode only. The design is swapped to software simulation time at specific time intervals, or after specific error conditions are detected. This technique is described later in this section.

This section contains the following:

- Partitioning Monitor Code
- Activating Monitor Code Only in Software Simulation
- Monitoring Signals and Displaying Messages

Partitioning Monitor Code

The technique for partitioning monitor code into synthesizable and non-synthesizable segments is based on the premise that much of the monitor code is needed only after one or more error conditions have occurred during simulation.

If an error does occur, the ability to debug it becomes more important than simulating the design very quickly. This technique involves several steps, as illustrated in this example:

```
module testbench ();
..
monitor mon1 (..);
..
endmodule

module monitor (..);
..
reg [0:31] local_mem [0:3];
always @(posedge sys_clk)
begin
    if (packet_status==3'b0 && !packet_received) // error check; synthesizable
begin
    for (i=0;i<3;i=i+1)
        local_mem[i] = testbench.chip.sram.mem[i]; // non-synthesizable
    if (local_mem[0] == 1'b0)
        $display("Invalid header; Packet Number: %d", packet_num);
    end
end
endmodule
```

In this example, the first `if` statement checks for the presence of an error condition, and if encountered, executes the remaining code in the `always` block. Unmodified, this code is unable to be accelerated, and the entire module will run in the software simulator only. To improve the run-time performance of this module, follow these steps:

1. Remove the non-synthesizable code segment that follows the `if` statement and replace it with a special `trigger` signal indicating that the error occurred. Note that the

UXE User Guide

Guidelines for Writing Testbenches

remainder of the `always` block and the `if` statement itself must be written in a *synthesizable* coding style.

In the previous example, you would modify the code, as follows:

```
module monitor (...);
..
reg monitor_dummy_reg;           // declare this trigger register
always @(posedge sys_clk)
begin
    if (packet_status==3'b0 && !packet_received)
        // error check must be synthesizable
        monitor_dummy_reg = 1'b1b1;
        // error encountered; trigger signal set to 1
    else
        monitor_dummy_reg = 1'b0;    // not an error condition: no trigger
    end
endmodule
```

2. Create a *dummy* module and instantiate it in the testbench. In this module, check for the triggering of the dummy signal and add the removed non-synthesizable code, as follows:

```
module testbench ();
..
dummy_mon d_mon ();           // instantiate this dummy module here
monitor mon1 (...);
chip chip1 (...);
..
endmodule

module dummy_mon ();

reg [0:31] local_mem [0:3];

always @(posedge testbench.monitor.monitor_dummy_reg)
    // check trigger signal
begin
    // execute monitor code
    for (i=0;i<3;i=i+1)
        local_mem[i] = testbench.chip.sram.mem[i];
    if (local_mem[0] == 1'b0)
        $display("Invalid header; Packet Number: %d", packet_num);
    end
endmodule
```

3. When compiling a mixed-HDL design, add the modified module **monitor** to the DUT. similarly, when compiling a Verilog-only design, add the modified module **monitor** to the DUT and exclude the new module **dummy_mon** from the DUT:

For a mixed-HDL Design:

```
ixcom -top top_level_design_unit other_options +dut+chip+monitor
```

For a Verilog-only Design:

```
ixcom Verilog_sources other_options +dut+chip+monitor  
+dutexcl+dummy_mon
```

Activating Monitor Code Only in Software Simulation

There are two techniques for making monitor code active in software simulation mode only:

- Marking Monitor Code to Simulate Only in Software Simulation
- Swapping to Software Simulation Mode at the Appropriate Time

Marking Monitor Code to Simulate Only in Software Simulation

At run time, you can *mark* either a whole module in the DUT, or just a portion of it, to be active in software simulation mode only; in Simulation Acceleration mode the code is totally inactive and has no affect. After you swap to software simulation mode, the code becomes active again from the next simulation time-tick. This feature is especially useful for minimizing the impact of monitor code in testbenches. There are two ways to mark modules in this fashion:

- To *mark* an entire module to simulate while in software simulation mode only, specify the `+dutignore+module_name` compiler option. Note that all instances of the module *and its sub-hierarchy* will not simulate at all while running in Simulation Acceleration mode. This functionality differs from that of the `+dutexcl+module_name` compiler option, which causes the module sub-hierarchy to simulate in the software simulator at all times, regardless of the run-time mode.
- To *mark* just a portion of a module to be simulated while in software simulation mode, use the `$axis_rccoff` system task and the `//axis rtl_off` pragma. The `$axis_rccoff` task takes no arguments, returning a boolean `true` if the design is currently running in software simulation mode or `false` if running in Simulation Acceleration mode.

Here is an example:

```
always @(posedge error_flag)  
begin  
    if ($axis_rccoff)
```

UXE User Guide

Guidelines for Writing Testbenches

```
// code executed only if design running in software simulation mode
end
endmodule
```

Note: This system task is non-synthesizable and thus cannot be used in a DUT module.

Swapping to Software Simulation Mode at the Appropriate Time

To simulate the monitor code, you must initiate the swap from Simulation Acceleration mode to software simulation mode, in either of two places:

- **xeDebug CLI (Command-Line Interface):** You can control the swapping feature at the command line, using the commands listed in the *Hot Swap (SW to HW) Options* sub-section of the *SA Run-Time and Debug Commands* chapter of the *UXE Commands Reference Manual*. If you wish to break on a signal in Simulation Acceleration mode, that signal must be exported at compile time, as previously explained.
- **Verilog testbench:** You can use the `$xc_cmd` system task to execute an `xc` command directly from your testbench, for example:

```
module testbench ();
...
always @(posedge sys_clk)
begin
  if (packet_status==3'b0 && !packet_received)
    $xc_cmd("verbose 2"); // enable xc verbose messages
end
endmodule
```

For more information on `$xc_cmd`, refer to the *System Tasks and Procedures* section of the *Commands and System Tasks for SA Mode* chapter in *UXE Command Reference Manual*.

Note: The following commands are not supported through the `$xc_cmd` system task if you are using the xeDebug tool:

on, off, stop, run, tbrun, autorun, nbrun, match, bmatch

Monitoring Signals and Displaying Messages

The `$display` and `$monitor` system tasks, which can be used in the testbench to display message strings and signal values during simulation, are fully supported in the software simulator and produce expected results if used in non-accelerated modules.

UXE User Guide

Guidelines for Writing Testbenches

However, if used in a DUT module, by default, these tasks will not work as expected while running the design in Simulation Acceleration modes, because they are not synthesizable and are thus ignored by the emulator. If you are trying to add a testbench module to the DUT so it can be accelerated, this can be a problem, for example:

```
module dut (...);

always @(posedge sys_clk)
begin
    if (start_mem_cycle && write_en && read_en)
        $display("Invalid memory cycle at time %t", $time);
end
```

In this example, the `$display` system task is ignored if running the design in Simulation Acceleration mode.

Any of the preceding techniques for speeding up monitor code can also be used to display these messages. If using the partitioning technique, you would modify the previous example, as follows:

```
module dut (...);

reg dummy_reg;
always @(posedge sys_clk)
begin
    if (start_mem_cycle && write_en && read_en)
        dummy_reg = 1'b1; //register toggles when error condition detected
    else
        dummy_reg = 1'b0;
end
```

Now add the following code to a testbench module:

```
always @(posedge testbench.dut.dummy_reg)
    $display("Invalid memory cycle at time %t", $time);
```

In the previous example, only the `$display` message itself is simulated in software simulation; actual error checking is accelerated by the emulator.

UXE User Guide

Guidelines for Writing Testbenches

Support for SystemVerilog Constructs

The following sections outline:

- [Support for SVTB Constructs](#)
- [Support for SystemVerilog Design Constructs](#)
- [Limitations when Using SystemVerilog Constructs](#)



The content of this chapter pertains only to the SA mode.

Support for SVTB Constructs

The following table lists the supported SVTB constructs.

Table E-1 SVTB Constructs

IEEE 1800-2005 LRM Section	SVTB Constructs
Data Types	
4.12	Class data type - store object handles in dynamic arrays (see 5.6)
4.12	Class data type - store object handles in queues (see 5.14)
4.12	Class data type - store object handles in associative arrays (see 5.9)
4.12	Class data type - store object handles in mailbox
4.14	Enum static casting
4.15	\$cast dynamic casting (class type)

UXE User Guide

Support for SystemVerilog Constructs

IEEE 1800- 2005 LRM Section

SVTB Constructs

4.16 bit stream casting

Arrays

5.6, 5.9, 5.14, 4.7 Public access to Queues and Dynamic Arrays (QDAs) and strings

5.6, 5.9, 5.14, 4.7 Local and protected access to QDAs and strings

5.6, 5.9, 5.14, 4.7 QDA as local variables in tasks/functions/methods

5.6 Dynamic arrays of strings

5.9, 5.14, 4.7 Queues and associative arrays of strings

5.2 array of mailboxes

5.6 dynamic arrays of classes

5.6 dynamic arrays in classes (public/local)

5.6 dynamic arrays in packages

5.6 dynamic arrays i-- multidimensional

5.6.1 dynamic arrays with copy, resize

5.6.1 dynamic arrays - new[]

5.6.2 dynamic arrays - size()

5.6.3 dynamic arrays - delete()

5.9 associative arrays of classes

5.9 associative arrays in classes (public/local)

5.9.3 class index types

5.9.7 associative arrays - indextype (other user defined type)

5.10 associative array methods

5.12 associative array arguments - pass by reference

5.12 associative array arguments - pass by value

5.14 queues

UXE User Guide

Support for SystemVerilog Constructs

IEEE 1800- 2005 LRM Section

SVTB Constructs

5.14	queues of classes
5.14	queues in classes (public/local)
5.14	queues in packages
5.14	\$ indexing of queues

Data Declarations Procedural Tasks and Functions

6.3	constants (in classes)
6.9	Type compatibility - including passing subclass argument to superclass formal
10.11	Level-sensitive sequence controls
12.1	Pass by value - object handles
12.4.2	Pass mailboxes as arguments to tasks/functions

Classes

7.4	Objects (class instance) - null object handling; can pass as argument, and so on
7.4	Pass classes by reference to tasks/functions
7.4	Pass classes through module ports
7.4	Class instances passed to Out Of Module Reference tasks or functions
7.4	Class instances passed to tasks/functions declared in a package
7.5	Object properties - event vars
7.5	Object properties - unpacked structs
7.5	Object properties - int types, packed structs
7.6	Object methods
7.7	Constructors - must support all argument types as in any function
7.8	Static class properties - of same object type as in SystemVerilog LRM 3.1a, Section 11.5
7.9	Static methods
7.1	this - needs to be polymorphic; must be able to pass arguments

UXE User Guide

Support for SystemVerilog Constructs

IEEE 1800- 2005 LRM Section

SVTB Constructs

7.11	Assignment, renaming, and copying. For example, myClass c = myOtherClass new;
7.12	Instance and subclasses
7.13	Overridden members
7.14	super - need to call with arguments
7.15	\$cast - need for downcasting from base class object handle (see also 3.15)
7.16	Chaining constructors - passing arguments to super.new(...)
7.17	Data hiding and encapsulation
7.17	Data hiding and encapsulation - parsing support
7.18	Constant class properties
7.18	Constant class properties - parsing support
7.19	Abstract classes - can use empty virtual methods
7.19	Virtual methods
7.2	Polymorphism; dynamic method lookup
7.21	Class scope resolution operator ::
7.22	Out of block declarations
7.24	Typedef classes - forward referencing

Randomization and Constraints

13.3	Random Variables - rand (class handles)
13.3	Random Variables - rand (unpacked arrays)
13.3	Random Variables - rand (static arrays)
13.3	Random Variables - rand (dynamic arrays)
13.3	Random Variables - rand (queues)
13.3	Random Variables - rand (enum support)
13.3	Random Variables - rand (multidimensional packed arrays)
13.3	Random Variables - rand (multidimensional arrays)

UXE User Guide

Support for SystemVerilog Constructs

IEEE 1800-2005 LRM Section	SVTB Constructs
13.3	Random Variables - (packed structs)
13.3	Random Variables - (int types)
13.3	Random Variables - (array randomization: using arr.size as rand var)
13.4	Constraint blocks - concatenation within a constraint
13.4	Constraint blocks - var ordering
13.4	Constraint blocks - external
13.4	Constraint blocks - global (contain variables declared in other classes)
13.4	Constraint blocks - iterative
13.4	Constraint blocks - distribution (rand with more than 1 dist constrain)
13.4	Constraint blocks - distribution (combo of weighted and complex constraints)
13.4	Constraint blocks - distribution (range and weight of any integral SV expression)
13.4	Constraint blocks - distribution (dist expression of any integral SV expression)
13.4	Constraint blocks - guards
13.4	Constraint blocks - inheritance (constraint name same in parent and derived)
13.4	Constraint blocks - implication (contain dist constraints)
13.4	Constraint blocks - static
13.4	Constraint blocks - override
13.4	Constraint blocks - named
13.4.3	Constraint blocks - set membership (arrays)
13.4.6	Constraint blocks - if ... else (contain dist constraints)
13.4.7	Constraint blocks - foreach
13.4.9	Constraint blocks - solve before
13.4.11	Constraint blocks - functions in constraints

UXE User Guide

Support for SystemVerilog Constructs

IEEE 1800- 2005 LRM Section

SVTB Constructs

13.5	Randomization methods incl pre/post randomize
13.6	In-line constraints - randomize() with constraints
13.8	constraint_mode
13.10	In-line random variable control
13.11	Randomization of scope Vars - except for the following, which are not supported: ■ class members in constraints or arguments ■ multiple constraint expressions after an if or else
13.11	Randomization of scope Vars - (packed structs defined in classes)
13.11	Randomization of scope Vars - (packed structs defined in a package)
13.11	Randomization of scope Vars - (packed structs in dist expression)
13.11	Randomization of scope Vars - (packed structs in set membership)
13.11	Randomization of scope Vars - (bit or part select of packed structure array member)
13.12.1	\$urandom (in classes)
13.12.2	\$urandom_range (in classes)
13.12.3	\$srandom
13.13	Random stability
13.14	manually seeding randomize
13.15	Randcase
13.16.1	Randsequence - random production weights
13.16.2	Randsequence - if ... else production statements
13.16.3	Randsequence - case production statements
13.16.4	Randsequence - repeat production statements
13.16.6	Randsequence - aborting productions - break and return
Synchronization	
14.2	semaphores

UXE User Guide

Support for SystemVerilog Constructs

IEEE 1800- 2005 LRM Section

SVTB Constructs

14.2	semaphores in packages
14.2	semaphores as protected/public in classes in packages
14.3	mailboxes
14.5.1	Triggering a named event
14.5.2	Non-blocking event triggering
14.5.3	Waiting for a named event
14.5.4	Persistent trigger: Triggered property
14.7	Event variables without assignments

Clocking Blocks

15.3	Input/output skews
15.4	Hierarchical expressions
15.5	Signals in multiple clocking blocks
15.6	Clocking block scope and lifetime
15.7	Multiple clocking blocks
15.8	Clocking blocks inside interfaces
15.9	Clocking block events
15.10	Cycle delay:##
15.11	Default clocking
15.12	Input sampling
15.13	Synchronous events
15.14	Synchronous drives

Hierarchy

19.2	packages - classes in packages
19.2	packages - extern_constraint declaration
19.8	queues, dynamic/associative arrays, classes ports
19.8	strings ports

UXE User Guide

Support for SystemVerilog Constructs

IEEE 1800- 2005 LRM Section

SVTB Constructs

Interfaces

20.4.5 Modports - task/function import

20.8 Virtual Interfaces

Coverage

18.2 Covergroups

18.2 Covergroup in interfaces

18.2 Covergroup in program blocks

18.3 Covergroup in classes

18.4 Defining coverage points

18.4.2 Automatic bin creation for coverpoints

open-ended bins for coverpoints

Options: name, comment, weight, per_instance, at_least, and goal

18.4.3 Wildcard specification of bins

18.4.4 ignore_bins for coverpoints

18.4.5 illegal_bins for coverpoints

18.4.5 Expressions as coverpoints

18.5 Cross products

18.7 Predefined coverage methods (start() and sample())

System Tasks and Functions

22.12 Coverage system functions

All the following rows indicate partially supported constructs.

For details about the limitations of these constructs, refer to the *Limitations* section of *Operators*, *Classes*, and *Randomization and Constraints* categories in the *SystemVerilog Reference Guide* of IUS documentation set.

Operators

8.2 Constraint operators shift, division, modulus, exponent, logical, concat

Classes

UXE User Guide

Support for SystemVerilog Constructs

IEEE 1800-2005 LRM Section

SVTB Constructs

7.4	Pass classes through module ports
7.5	Object properties - dynamic arrays
7.5	Object properties - queues
7.5	Object properties - assoc. arrays
7.5	Object properties - mailboxes (typeclass)
7.5	Object properties - semaphores
7.5	Object properties - strings

Randomization and Constraints

13.4	Constraint blocks - support for operators (/ % << >> <<< >>> ^~ & ^?:)
------	--

Support for SystemVerilog Design Constructs

The following table lists the supported and partially supported SystemVerilog constructs in the design.

Table E-2 Supported SystemVerilog Constructs

IEEE 1800-2005 LRM Section

Supported Constructs

DataTypes

4,10.4.3	enum type method next()
4.10.4.4	enum type method prev()

Data Declaration

6.5	net datatype
-----	--------------

Tasks and Functions

12.4.3	Task/function default argument values
12.4.4	Task/function argument binding by name

UXE User Guide

Support for SystemVerilog Constructs

IEEE 1800-2005 LRM Section

Supported Constructs

Interfaces

20.2.2	Named bundle
20.2.3	Generic bundle
20.3	Ports in interface Note: For net-type ports, allowed data type is limited to Verilog-2000 language subset.
20.4.1	Named port bundle
20.4.2	Connecting port bundle
20.4.3	Connecting port bundle to generic interface
20.6.1	Using tasks in interface
20.7	Parameterized interfaces

Macros and Compiler Directives

23.4	'begin_keywords and 'end_keywords directives Note: The supported version specifiers include 1364-1995, 1364-2001, 1364-2001-noconfig, and 1800-2005.
------	--

All the following rows indicate partially supported SystemVerilog constructs.

Interfaces

20.4	Modports
20.4.5	Clocking blocks and modports Note: Clocking block support restrictions apply.
20.6	Tasks and functions in interfaces
20.6.2	Using tasks in modports Note: REF type ports are not supported in modports.

The following table lists the unsupported SystemVerilog Design Constructs.

Table E-3 Unsupported SystemVerilog Design Constructs

LRM Section	Unsupported Constructs
Literals	
3.4	shortreal literals
3.5	time units in literals (step)
DataTypes	
4.4	shortreal data types
4.7	string operator: {multiplier{s1}}
4.9	User-defined types (interface typedef scoping)
4.10.2	Enumerated data type (OOMR to enum constants)
4.11	Packed structure data type (initializing members)
4.11	Unpacked structure data type (OOMRs to members)
4.11	Packed union data type (tagged)
4.11	Unpacked Union data type
4.11	Unpacked Union data type (tagged)
4.15	\$cast dynamic casting (non-class type)
4.15	\$cast dynamic casting (enums)
4.17	Default attribute type
Arrays	
5.6, 5.9, 5.14	Hierarchical (OOMR) references to QDAs
5.4	unpacked arrays (slices)
5.10	associative array locator methods
5.11	associative array assignment
5.13	associative array literals
5.15	queue locator methods
5.15	array manipulation methods
Data Declaration	
6.8	signal aliasing

UXE User Guide

Support for SystemVerilog Constructs

LRM Section	Unsupported Constructs
6.10	Type operator
Attributes	
	Default attribute type
Operators and Expressions	
8.13	assignment patterns - left hand side assignment
8.13	assignment patterns - associations by type
8.13	assignment patterns - replications factors
8.13	assignment patterns - simple ' type qualification for assignments to OOMRs
8.13	assignment patterns - simple ' type qualification for port connection expressions
8.14	Tagged unions
8.16	Operator Overloading
8.17	Streaming Operators
8.19	Set membership
Procedural Statements	
10.4.1	Selection statements - pattern matching
10.7	iff with level-sensitive controls
10.8	named blocks (statement labels)
Processes	
11.6	join_none (disable)
11.6	join_none (wait on automatic variables with wait or event controls)
11.9	Fine grain process control
Tasks and Functions	
12.1	Function return - events
Classes	
7.23	Parameterized classes

UXE User Guide
Support for SystemVerilog Constructs

LRM Section	Unsupported Constructs
Randomization and Constraints	
13.3	Random Variables - rand (unpacked structures)
13.3	Random Variables - rand (associative arrays)
13.3	Random Variables - rand (dynamic array size)
13.4	Constraint blocks - 1d array for values of constraint inside operator
13.7	rand_mode() - members of unpacked arrays
13.7	rand_mode() - members of unpacked structures
13.10.1	In-line constraint checker
13.11	Randomization of scope Vars - (class members in constraints or arguments)
13.11	Randomization of scope Vars - (multiple constraint expressions after an if or else)
13.12.4	get_randstate()
13.12.5	set_randstate()
13.16.5	Randsequence - interleaving production - rand join
13.16.7	Randsequence - value passing between productions
Synchronization	
14.4	parameterized mailboxes
14.6	Event sequencing
Clocking Blocks	
15.2	Clocking blocks (in generate loops)
Assertions	
17.11	Declaring properties in a clocking block
17.11	Declaring properties: operators AND
17.11	Declaring properties: operators OR
17.11.4	recursive properties
17.12.1	multiple clock support

UXE User Guide

Support for SystemVerilog Constructs

LRM Section	Unsupported Constructs
17.14	clocked sequences
17.14	Default clocking
17.16	expect statement
Hierarchy	
19.2	Packages - covergroup declaration
19.2	Packages - overload declaration
19.2	Packages - anonymous_program
19.6	nested modules
19.7	Extern modules
19.8	event ports
19.8	union ports
19.9	named port declaration
19.11.4	implicit .* port connections (use in generate block)
19.12	ref ports
19.12	port connection rules (see below)
19.13	Extended name spaces
Interfaces	
20.2	Nested interfaces
20.2	Interfaces connected to ports of interface
20.2	Interface arrays
20.2	Interfaces in generates
20.4.4	Modport expressions
20.5	Interfaces and Specify Blocks
20.6	Modports - clocking keyword on Modports
20.6.3	Modports - task/function export
20.6.4	Multiple task exports
20.6	Extern fork/join tasks

UXE User Guide

Support for SystemVerilog Constructs

LRM Section	Unsupported Constructs
20.7	Access without ports (static interface)
20.8	Virtual interfaces
20.9	Access to interface objects
Configuration	
18.4.1	Specifying bins for transitions
18.4.4	Exclusion of coverpoints or transitions
18.4.5	Specifying illegal coverpoints or transitions
18.5.2	Excluding cross products
18.5.3	Specifying illegal cross products
18.6	Procedural setting of options
18.8	coverage system tasks/functions
21.2	config support for interfaces
System Tasks and Functions	
22.2	Elaboration time 'type' keyword
22.4	Range function (\$isunbounded)
22.5	shortreal conversions (\$shortrealtobits, \$bitstoshortreal)
22.9	assertion system functions
22.13	New format specifiers: %u %z [supported with \$fwrite \$fread, unformatted output not shown with \$display]
22.13	\$fread extensions [optional argument (third) of "count" is not supported]
22.14	\$readmemb, \$readmemh [not supported only for associative arrays]
22.15	\$writememb, \$writememh [not supported for associative arrays and dynamic arrays]
22.16	file format considerations for multidimensional unpacked arrays (MDUAs)

UXE User Guide

Support for SystemVerilog Constructs

LRM Section	Unsupported Constructs
22.17	system task arguments for MDUAs
Macros and Compiler Directives	
23.3	'include (angle brackets <i><filename></i>)
APIs	
26.4.6	Export function return types - string
26.4.6	Import/export function/task arguments - packed union of type bit/long
26.4.6	Import/export function/task arguments -enums
26.4.6	Import/export function/task dynamic array arguments (non-strings)
26.4.6	Import/export function/task dynamic array arguments
26.4.6	Import/export function return types - shortreal
26.4	Export function/tasks - time consuming
28	Assertion API
29	Coverage API
30	Data Read API
	VPI Object Model
Annexes	
C	Std Package
D	Link Lists

Limitations when Using SystemVerilog Constructs

- Different behavior for software and hardware runs when using sampled value functions in continuous assignment statements.

When using sampled value functions, such as `$past`, in continuous assignment statements, some mismatches can be observed when the same design is run on the hardware emulator and the software simulator. These mismatches occur because of the way hardware samples values internally. Consider the following examples:

Example 1:

UXE User Guide

Support for SystemVerilog Constructs

```
module dut;
    assign out1 = $past(in1,,,@(posedge CLK));
    always @(posedge CLK)
        $strobe("in1=%d , out1=%d",in1,out1);
endmodule
```

In the above example, `out1` is expected to have the previous or the past value of `in1`. But the current value, and not the earlier sampled value of `in1`, is displayed as the value of `out1`.

Example 2:

```
module dut (clk,in1,a,b,out1);
    input [3:0] in1;
    input wire a,b;
    input clk;
    output [3:0] out1;
    reg [3:0] out1;
    reg mytaskr=1'b0;
    wire mytaskw;
    assign out1= $past(in1, , , @(posedge clk));

    always @(posedge clk)
        mytaskr <= ~mytaskr;

    axis_pulse(mytaskw,mytaskr);
    axis_tbcall(mytaskw,"t1");
    task t1;
        $display("in1=%b,out1=%b",in1,out1);
    endtask
endmodule
```

When the above module is run on the hardware emulator; the task `t1` runs on the software simulator and reads a variable, `out1` that has been assigned a value using the `$past` sampled value function. The testbench call region will not display the earlier sampled value of `in1` as the value of `out1`.

You will see incorrect values because the values for sampled value functions like `$past` are captured during the postponed region of a cycle to be used in the next timeslot. Constructs like `$strobe` and `$monitor` also operate in the postponed region of a timeslot but after the sampling code has executed. The testbench calls occur after the postponed (read-only) regions of the time slot. Hence, the values read by constructs like `$monitor`, `$strobe`, and the testbench calls are the current values and not the previously sampled values.

Handling of Concatenation Operators

A zero or negative multi-concatenation multiplier is not supported in SystemVerilog modules. A replication with a zero replication constant is considered to have a size of zero, and is ignored. Such a replication can appear only within a concatenation in which at least one of the operands of the concatenation has a positive size.

1. For SystemVerilog modules:

`{0{expr}}` within a non-zero size concatenation is ignored with a warning message.

An error is generated in the following cases:

- a.** `{0{expr}}` within a zero size concatenation multiple
- b.** `{0{expr}}` within a zero size concatenation
- c.** `{0{expr}}` not within a concatenation

2. For Verilog 2K modules, each occurrence of `{0{expr}}` is converted to `1'b0` in all the below cases:

- a.** `{0{expr}}` within a non-zero size concatenation
- b.** `{0{expr}}` within a zero size concatenation multiple
- c.** `{0{expr}}` within a zero size concatenation
- d.** `{0{expr}}` not within a concatenation

Using the xeDebug GUI

A design compiled using either xeCompile, IXCOM tools, or *irun* can be set up and run on the Palladium XP emulator using the xeDebug GUI. This chapter describes how to use the xeDebug GUI in ICE and SA modes.

Note: If you will be using the SimVision tool for viewing results, set up the `LD_LIBRARY_PATH` environment variable before invoking the xeDebug GUI. For details, refer to the [Setting Up the Environment for Palladium XP](#) section.

Invoking xeDebug GUI

Execute the following command from the UNIX command line to invoke the xeDebug GUI:

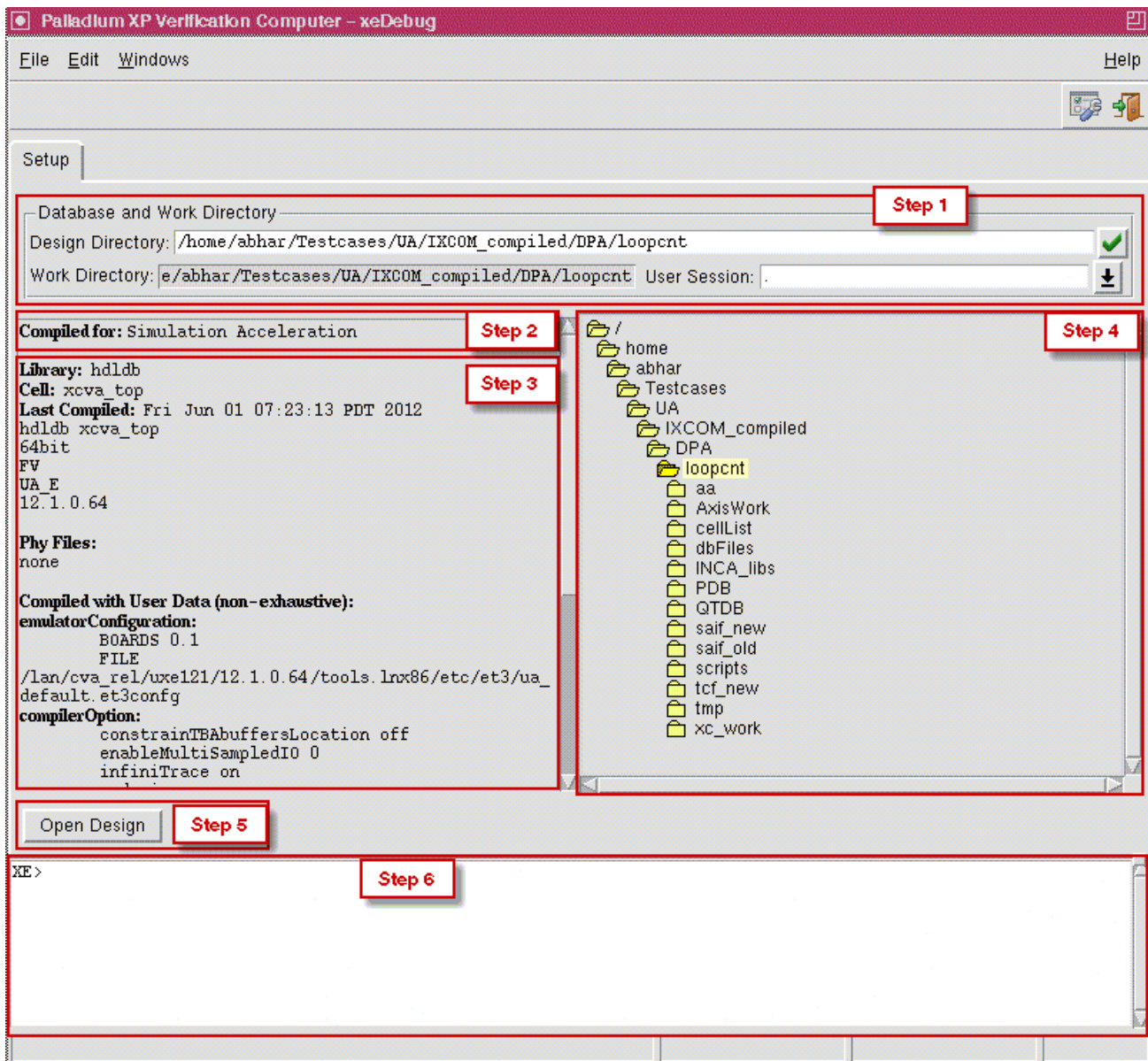
```
xeDebug -gui [-shm|-fsdb|-vcd] \
             [-init <tcl_file>] ... \
             [-log <name>] [-append_log] [-append_key] \
             [--ncsim <option> ... [--]] [script [<arg> ...]]
```

For more information on invoking the debugger tools, refer to the [Invoking the Debugger Tool\(s\)](#) section.

UXE User Guide

Using the xeDebug GUI

Figure F-1 Initial Screen



When the xeDebug GUI is invoked, only the *Setup* tab as illustrated in [Figure F-1](#) is displayed with the following fields:

1. *Database and Work Directory* group box:

- a. *Design Directory* field displays the location of the compiled design database. The path to the compiled design database can be changed manually or by using the directory tree displayed in the area marked as **Step 4** in [Figure F-1](#). However, to bring the change to effect, click the  button that is adjacent to the field.

- b. *Work Directory* field displays the location of directory from which the xeDebug GUI was invoked. This field is linked to the design directory.
- c. *User Session* drop-down list box allows you to enter a new session name, or select an existing session subdirectory.

If you specify a new session name, a directory by that name is created. The initial default user session entry is . (a dot) that means use current directory.

If you select an existing user session directory, the tab shows design properties in case the directory has a valid compiled database; otherwise a message about missing database is displayed.

Note: You can specify multiple user sessions, but can only open one session at a time. To use a session of a different name, you must release the emulator and choose another session to open.

Individual user sessions are saved in a subdirectory specific to that session. This directory also stores files associated with that session, such as the trace waveform database, message files, and so on.

- 2. *Compiled for* field displays the mode for which the design was compiled, that is, *Simulation Acceleration* or *In-circuit Emulation*.
- 3. Design-related information such as, *Library*, *Cell*, *Last Compiled*, *Phy Files*, and *Compiled with User Data* is displayed in non-editable format.
- 4. The *directory tree* is displayed to provide the information about the location of the compiled design database. This area can be used to select the directory that contains the compiled design database and the path displayed in the *Design Directory* field is updated accordingly.
- 5. *Open Design* button when clicked opens the selected design for running and debugging. The layout of the *Setup* tab changes and provides you more fields based on the mode in which you are running the design, that is, SA or ICE mode.

Note: The selected design is opened only if the specified design directory has a valid compiled database; otherwise a message about missing a database is displayed.

For the steps that can be performed on the *Setup* tab after opening the design in SA mode, refer to the [Using Setup Tab in SA Mode](#) section; and for ICE mode, refer to the [Using Setup Tab in ICE Mode](#) section.

- 6. *Command prompt* area that displays the command executed on click of a button from the tab currently selected in the xeDebug GUI. This area also allows you to directly write and execute a supported command. For example, when you click the *Open Design* button on the *Setup* tab, the following command is executed and displayed in the *command prompt* area:

debug .

Note: The *command prompt* area is visible on all tabs that allow debugging functions.

Using xeDebug Menu Bar and Toolbar Options

The xeDebug GUI window consists of a top area, containing drop down menus for *File*, *Edit*, and *Help* on the Menu Bar, and a secondary Toolbar with icons allowing you to access various debugging browsers and viewers. Below these are a series of tabs that enable access to various debugging functions.

Note: The tabs displayed in the xeDebug GUI and the fields that each tab comprises can vary based on the mode for which the design was compiled.

Menu Bar

Before or during a debug session, you can use the available menu items to configure the xeDebug GUI and manage content defined on the tabs.

The menu bar contains the following:

- File Menu
- Edit Menu
- Help Menu

File Menu

The *File* menu items allow you to control various properties of the xeDebug GUI, determine how you want to see or save data, or what data to maintain for the debugging between sessions.

Clicking *File* displays the following menu items:

- *Preferences...* (For details, refer to the Defining the Preferred Settings section.)
- *User GUI Setting...* (For details, refer to the Specifying the User GUI Setting section.)
- *View User Data...* (For details, refer to the Viewing the User Data section.)
- *User Design Information...* (For details, refer to the Viewing the User Design Information section.)

- This menu item is activated after the design has been downloaded to the emulator in the ICE mode or the session has been started in the SA mode.
- *Testbench Probe Selection...*
- *Finish*
- *Exit*

Figure F-2 File Menu Items

File Edit Windows	
P <u>references...</u>	<F4>
U <u>s</u> er GUI Setting...	<F5>
View User <u>D</u> ata...	<F7>
User <u>D</u> esign Information...	
<u>T</u> estbench Probe Selection...	<F6>
F <u>i</u> nish	
E <u>x</u> it	Ctrl-x

Defining the Preferred Settings

Clicking *File – Preferences...* displays the *Preferences* dialog box that has the following tabs:

- Message Tab
- Filter Tab
- Variables Tab
- GUI VariablesTab
- Fonts Tab



Tip

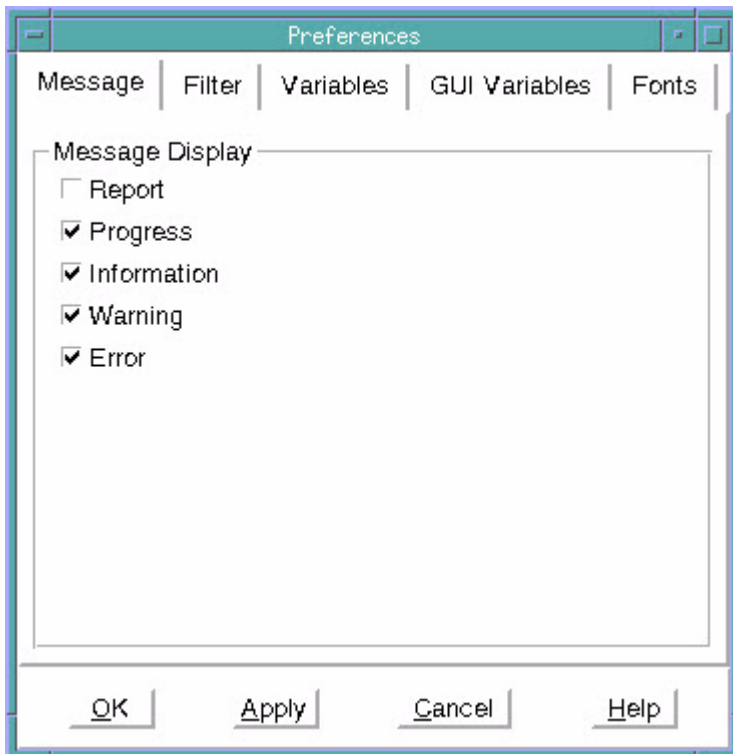
The *Preferences* dialog box can be accessed either from the *File – Preferences...* menu item or the tools icon on the right side of the toolbar.

Message Tab

The *Message* tab (shown in [Figure F-3](#) on page 1172) allows you to select the check box corresponding to the type of messages that must be echoed while running and debugging the design. The supported types of messages are:

- Report: Controls the printing of report messages.
- Progress: Controls the printing of progress messages.
- Information: Controls the printing of information messages.
- Warning: Controls the printing of warning messages.
- Error: Controls the printing of error messages.

Figure F-3 Message Tab

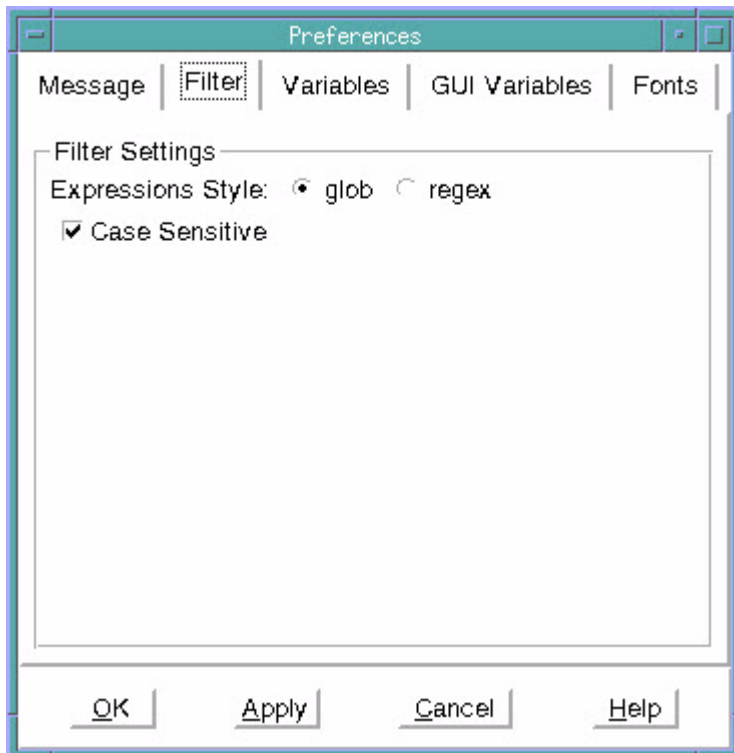


Filter Tab

The *Filter* tab (shown in [Figure F-4](#) on page 1173) allows you to define the filter setting preferences, like whether the global (*glob*) or regular (*regex*) expressions should be used,

and whether case sensitivity should be used or not. By default, the expressions style is set to global and case sensitivity is on.

Figure F-4 Filter Tab



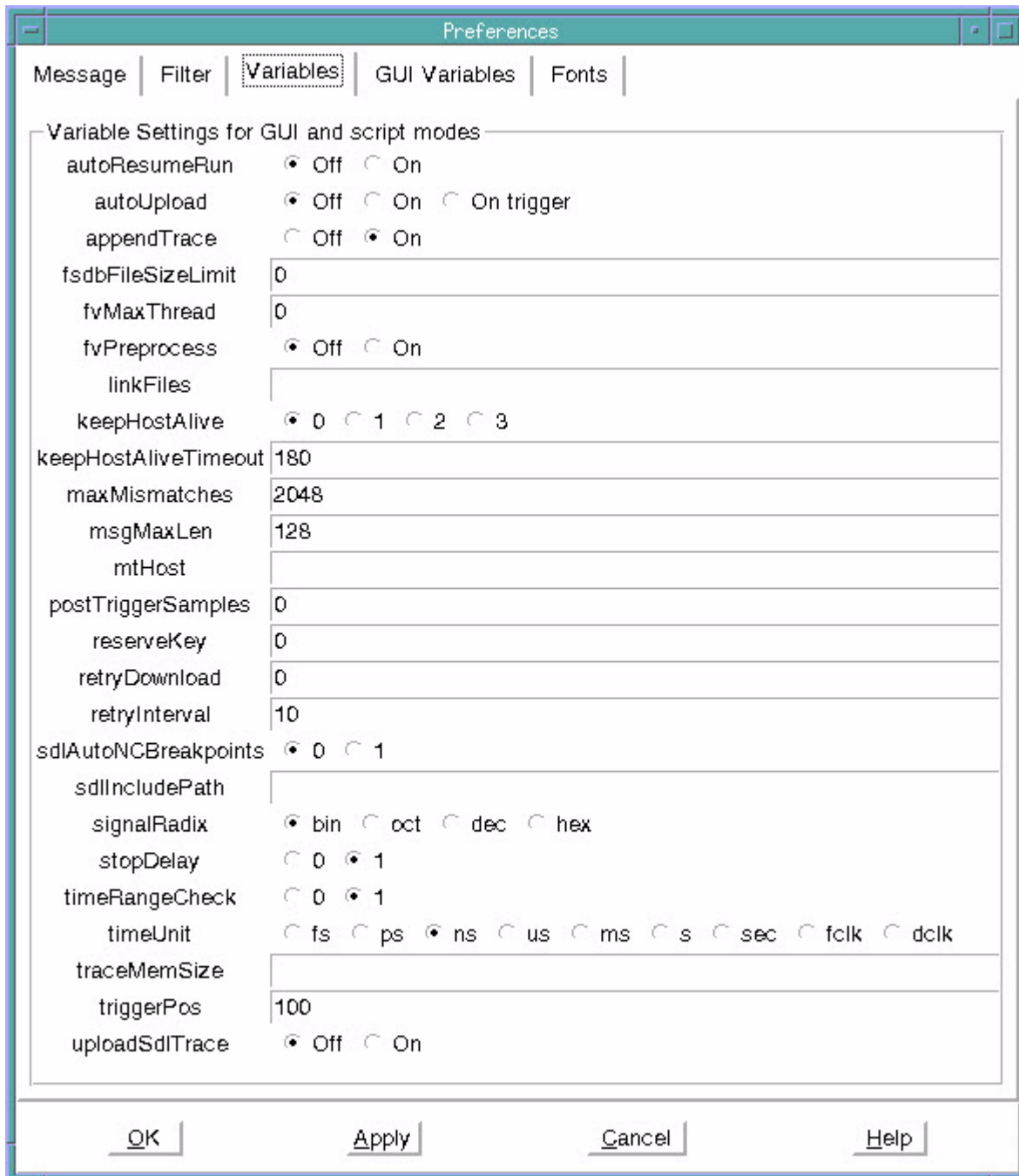
Variables Tab

The *Variables* tab (shown in [Figure F-5](#) on page 1174) allows you to view and configure the run-time parameters that are set using the `xeset` command on the command prompt. Specify the values and click *Apply* for the run-time parameter settings to take effect.

UXE User Guide

Using the xeDebug GUI

Figure F-5 Variables Tab (ICE Mode)



For information about setting the run-time parameters from the command prompt, refer to the `xeset` command description in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

- In the SA mode, keep in mind the following regarding the *Variables* tab:

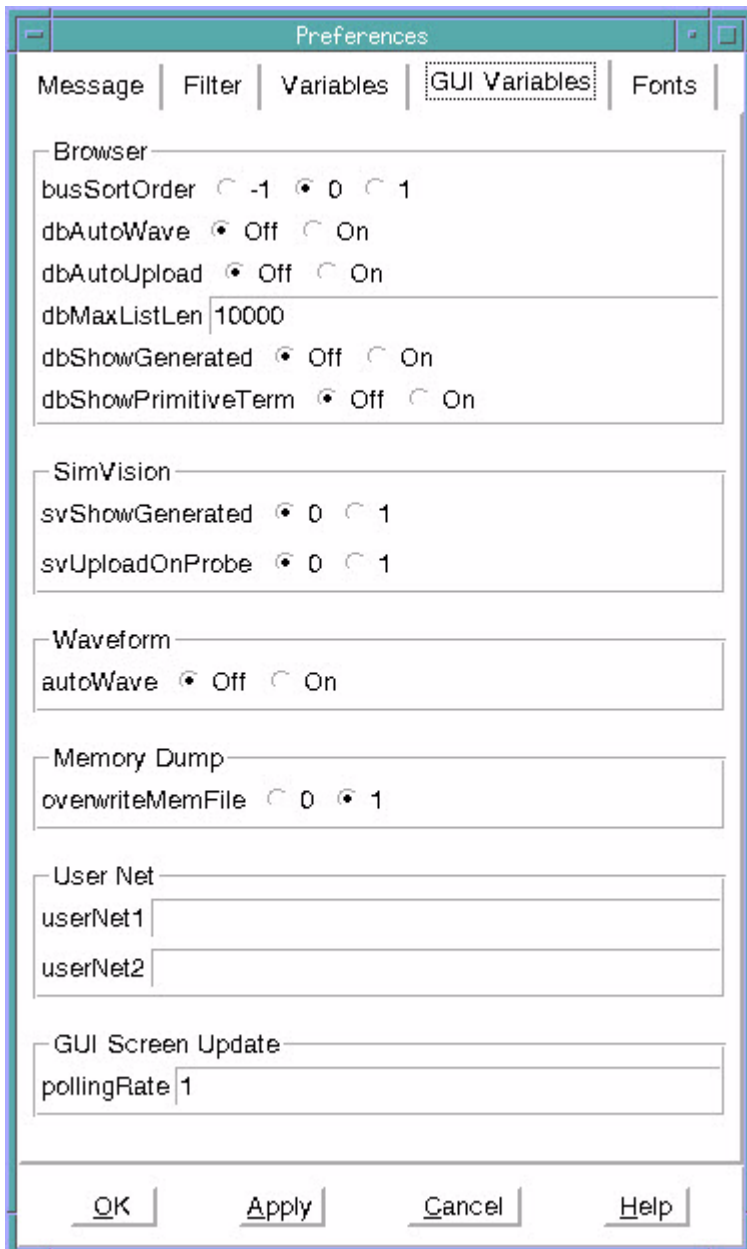
- ❑ The following run-time parameters are not supported, and therefore, not visible on the *Variables* tab:
 - `depositNCsimMode`
 - `probePrimary`
 - `suspendEnable`
- ❑ The *Variables* tab has the `sdlAutoNCBreakpoints` run-time parameter, which is not available in the ICE mode.
- ❑ The `autoResumeRun` run-time parameter can be set to *On* only after swapping the design to run in emulation mode instead of software simulation mode.

GUI VariablesTab

The *GUI Variables* tab (shown in [Figure F-6](#) on page 1176) allows you to view and configure the GUI-related run-time parameters that are set using the `xeset -gui` command on the command prompt. Specify the values and click *Apply* for the GUI-related run-time parameter settings to take effect. After applying the selection, you must close and re-open the Design Browser for the options to take effect.

For information about setting the GUI-related run-time parameters from the command prompt, refer to the `xeset` command description in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

Figure F-6 GUI Variables Tab

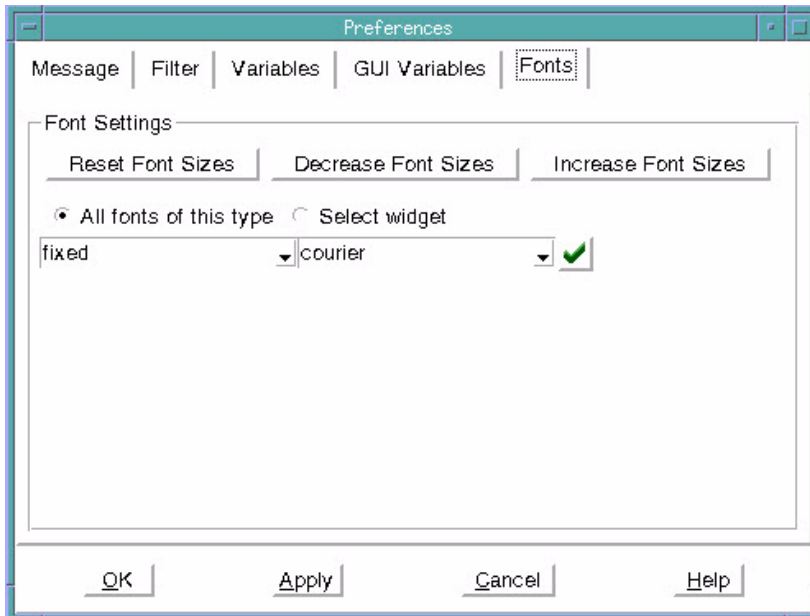


Fonts Tab

The *Fonts* tab (shown in [Figure F-7](#) on page 1177) allows you to change the font of the text displayed in the various dialog boxes of the xeDebug GUI. You can change the font size, font family, or font type of a selected widget.

Note: The font changes are applicable only to the current debug session. When you exit, and re-invoke the xeDebug GUI, the default font settings are displayed.

Figure F-7 Fonts Tab



You can click the *Decrease Font Sizes* or *Increase Font Sizes* button to decrease or increase the font size of the xeDebug GUI. Click the *Reset Font Sizes* button to reset the font size to its original size.

The following table specifies the keys on the keyboard that you can use to change the font size:

Table F-1 Keys to Change the Font Size on xeDebug

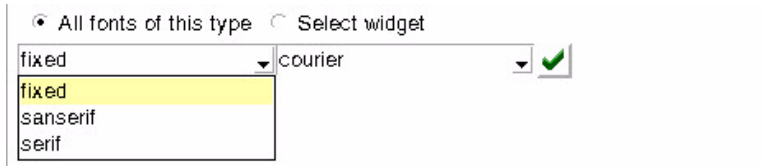
Keys	Description
CTRL+MINUS SIGN (-)	Decreases the font size
CTRL+PLUS SIGN (+)	Increases the font size
CTRL+ZERO (0)	Resets the font size to the original size

You can select the *All fonts of this type* or *Select widget* radio button to apply a new font to the xeDebug GUI. To apply a font, perform the following steps:

UXE User Guide

Using the xeDebug GUI

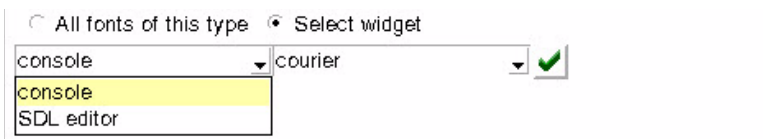
1. Select the *All fonts of this type* radio button to apply a font family to the xeDebug GUI. You can select *fixed*, *san serif*, or *serif*.



The screenshot shows a dialog box with two radio buttons at the top: "All fonts of this type" (selected) and "Select widget". Below the radio buttons are two dropdown menus. The first dropdown menu is open, showing a list of font families: "fixed", "fixed" (highlighted), "san serif", and "serif". The second dropdown menu is closed and shows "courier". To the right of the second dropdown menu is a green checkmark button.

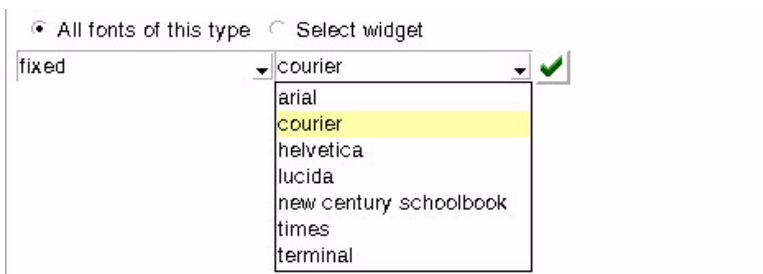
Or,

Select the *Select Widget* radio button to apply the selected font to the selected widget. You can select *console* or *SDL editor*.



The screenshot shows a dialog box with two radio buttons at the top: "All fonts of this type" and "Select widget" (selected). Below the radio buttons are two dropdown menus. The first dropdown menu is open, showing a list of widgets: "console", "console" (highlighted), and "SDL editor". The second dropdown menu is closed and shows "courier". To the right of the second dropdown menu is a green checkmark button.

2. Specify the font type as *arial*, *courier*, *helvetica*, *lucida*, *new century schoolbook*, *times*, or *terminal*.



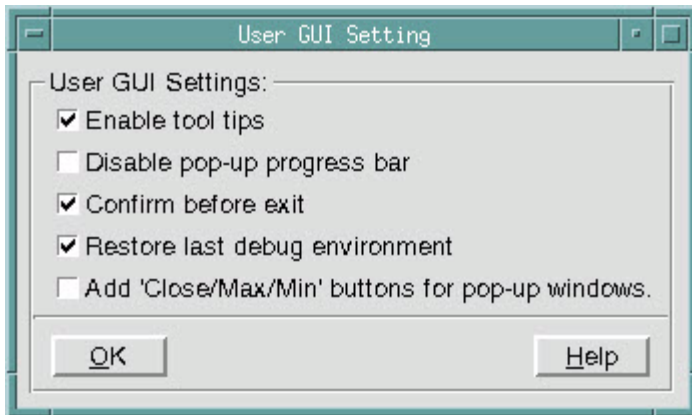
The screenshot shows a dialog box with two radio buttons at the top: "All fonts of this type" (selected) and "Select widget". Below the radio buttons are two dropdown menus. The first dropdown menu is closed and shows "fixed". The second dropdown menu is open, showing a list of font types: "arial", "courier" (highlighted), "helvetica", "lucida", "new century schoolbook", "times", and "terminal". To the right of the second dropdown menu is a green checkmark button.

3. Click the  button to apply the selected font family or type.

Specifying the User GUI Setting

Clicking *File – User GUI Setting...* displays the *User GUI Setting* dialog box (shown in [Figure F-8](#) on page 1179) that allows you to manage the GUI preferences.

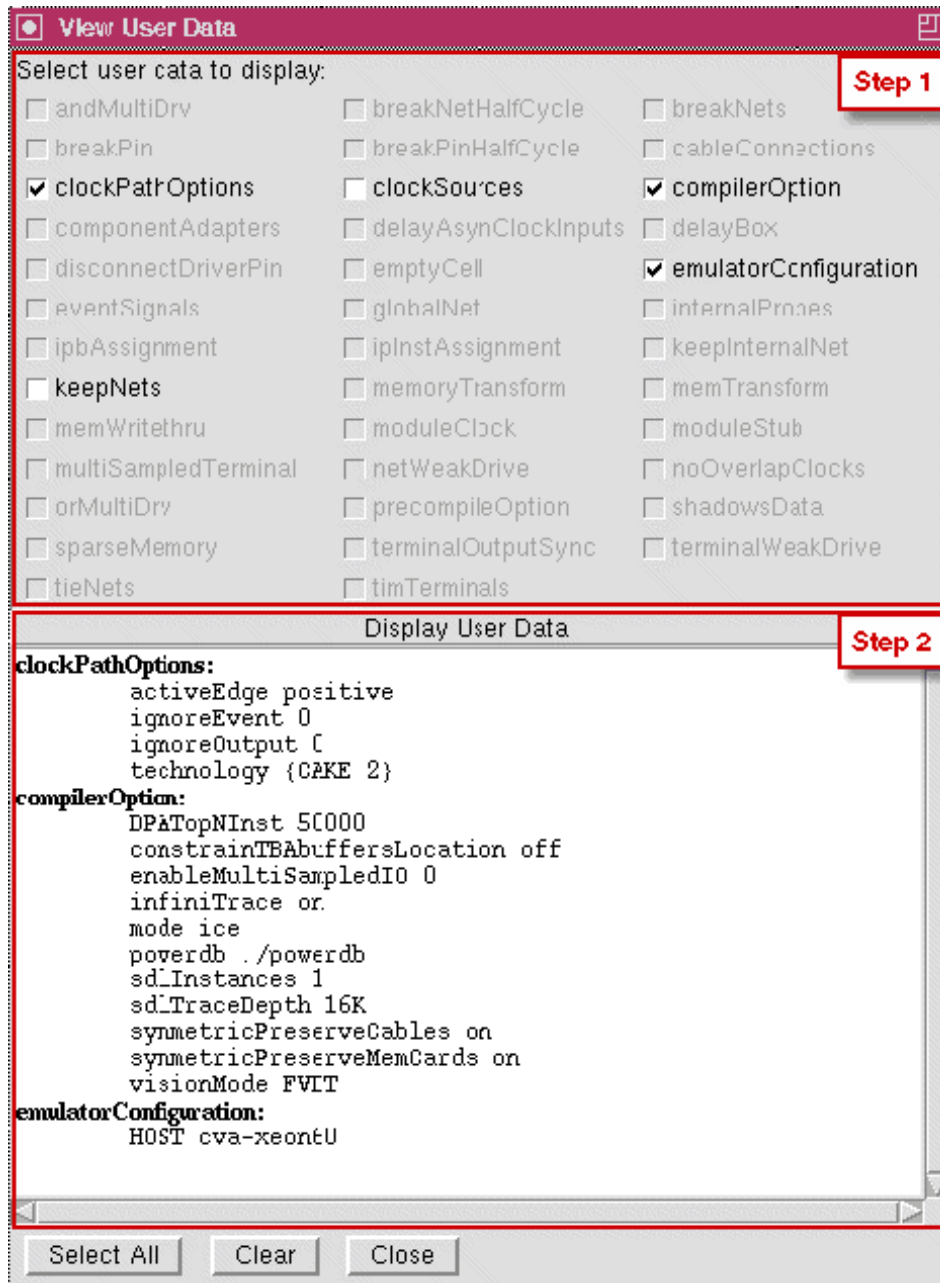
Figure F-8 User GUI Setting Dialog Box



Viewing the User Data

Clicking *File – View User Data* displays the *View User Data* dialog box shown in [Figure F-9](#) on page 1180. You can use this dialog box to view the User Data saved during a previous session.

Figure F-9 View User Data Dialog Box



The *View User Data* dialog box has the following group boxes:

1. *Select user data to display*

This group box lists all the User Data items with a check box adjacent to each. However, only those User Data items are active for which you had defined values at the compilation time.

To view the defined value for a specific User Data, select the check box adjacent to it.

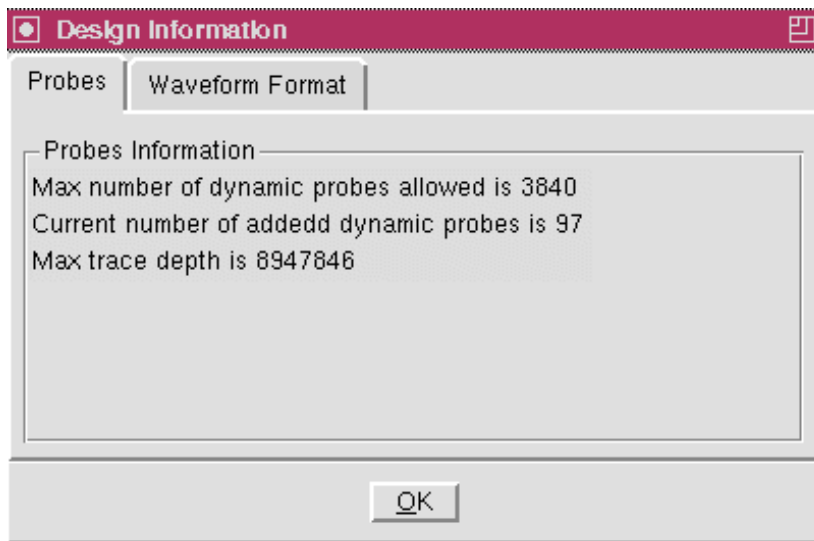
2. Display User Data

The values for the selected User Data items are printed in this group box.

Viewing the User Design Information

Clicking *File – User Design Information...* displays the *Design Information* dialog box shown in [Figure F-10](#) on page 1181. You can use this dialog box to view the design-specific information that is displayed on different tabs, such as *Probes* and *Waveform Format*.

Figure F-10 Design Information Dialog Box

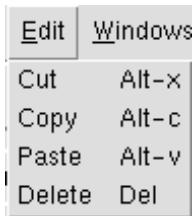


Edit Menu

During a debug session, you can select the *Edit* menu items to perform editing operations such as Cut, Copy, Paste, and Delete, on the selected text.

Clicking *Edit* displays the menu items shown in [Figure F-11](#) on page 1182.

Figure F-11 Edit Menu Options



Help Menu

Clicking *Help* displays the menu items that allow you to access information specific to the currently active tab, the Palladium XP manuals, and Cadence-specific support information.

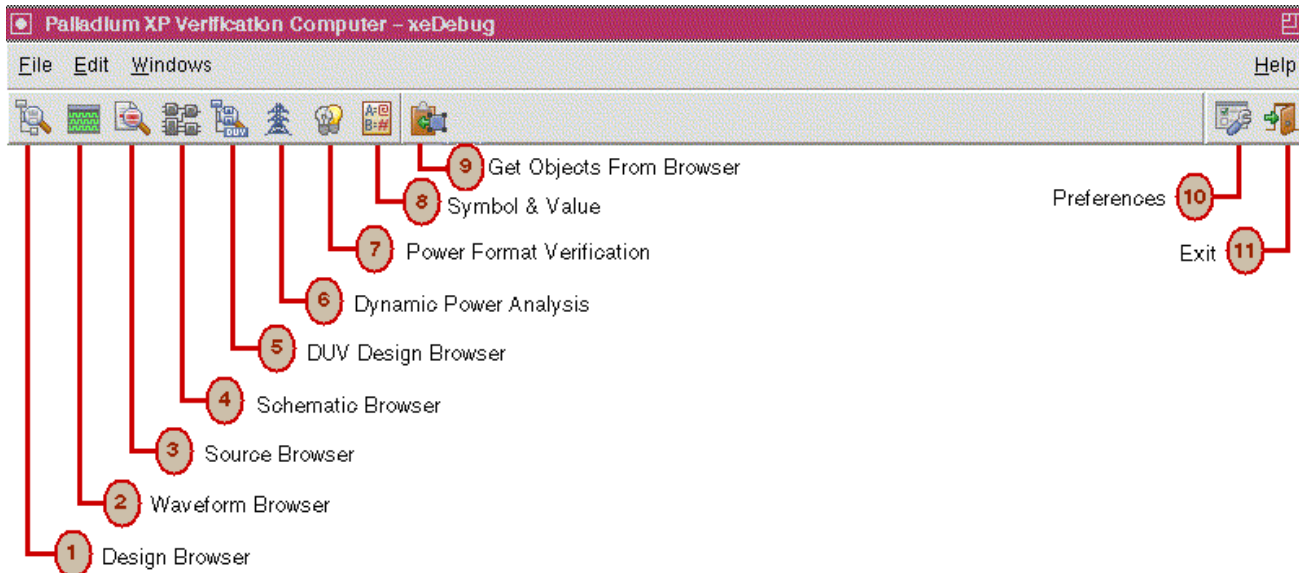
Toolbar

The bar under the menu bar, known as the toolbar, displays the icons used for accessing the debugging browser and viewer tools. Some of the icons represent the following tools:

- Design Browser
- Waveform Viewer
- DUV Design Browser
- Symbols and Values Browser
- Miscellaneous Tools

The top area of the xeDebug GUI window is common to all the tabs. [Figure F-12](#) on page 1183 shows this common area and identifies each icon.

Figure F-12 Common Access Toolbar



Note: The power tower icon on the toolbar requires a special license to enable its functionality. For details, refer to the [DPA Licenses](#) section in [Chapter 15, “Analyzing Power Consumption of the Designs.”](#)

Design Browser

The Design Browser can be accessed from either xeDebug tool or *irun* tool.

Note: The Design Browser is accessible only after you have opened a session in the xeDebug tool.

The Design Browser allows you to examine cells, instances, and nets of a design. It can also be used to examine the design hierarchy, and transfer object names to other windows through drag and drop.

If you specify the `-fsdb` option at the time of invoking xeDebug, the Novas tool design browser is displayed; otherwise, the SimVision Design Browser (as shown in [Figure F-13](#) on page 1185) is displayed.

Choosing Signals

You can choose signals either by entering the information directly into xeDebug GUI tabs and browsers, or by dragging and dropping the desired signal name from the Design Browser into the xeDebug GUI tabs and browsers.

Note: When running a design in SA mode, the SimVision Design Browser provides a *Show/Hide Generated* menu item that enables you to filter signals and instances based on name.

The Design Browser can be used as a source of information for any text field in xeDebug that expects a signal or instance name. Some of the tabs that make use of such information are:

- *SDL* tab
- *Probe* tab
- *Force* tab
- *Deposit* tab (in SA mode)

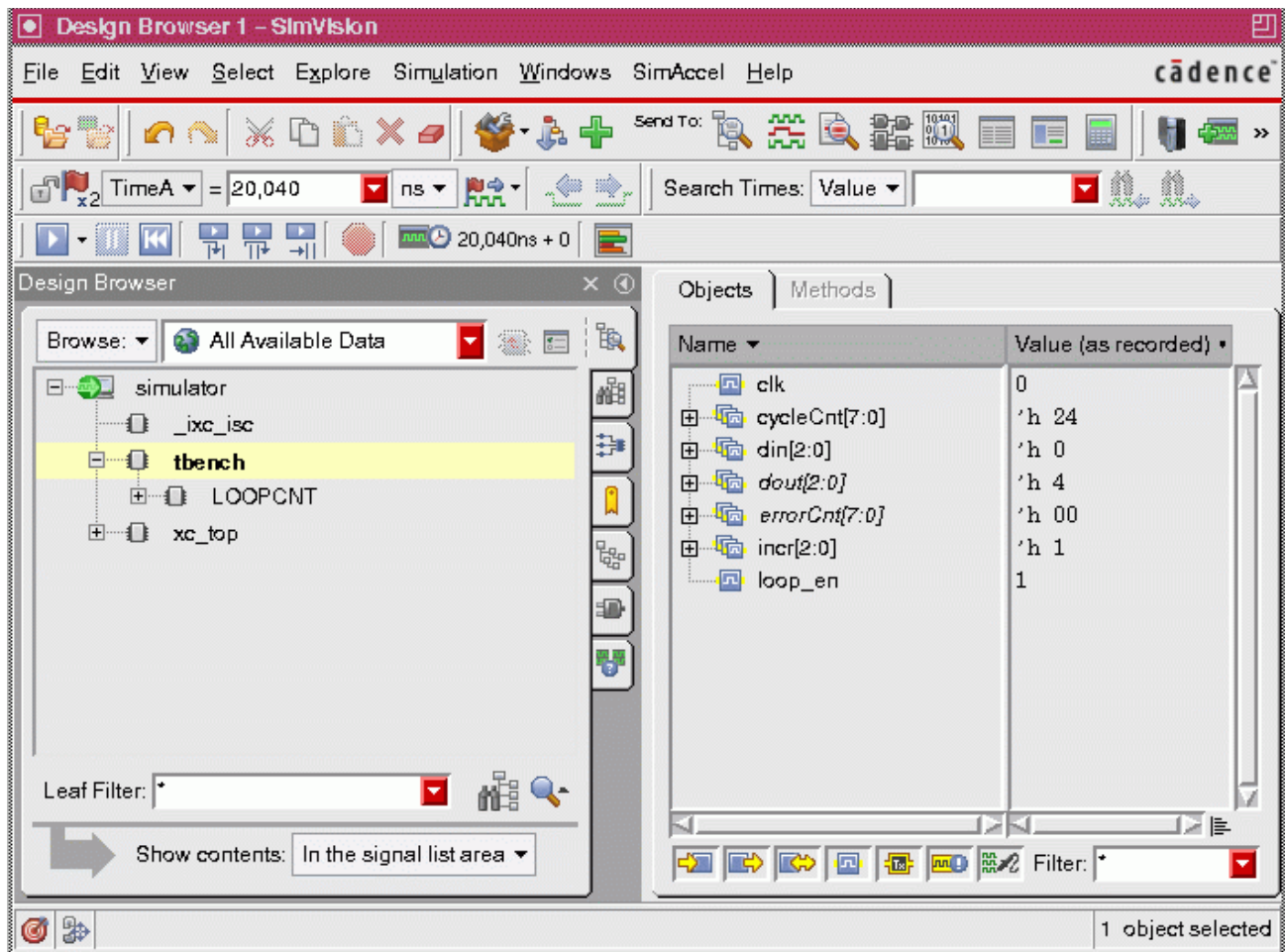
To transfer an object name (instance or signal) into one of these forms:

1. Highlight the object on the Design Browser using the left mouse button.
2. Click with the left mouse button at the target insertion point in the xeDebug GUI.
3. Click the *Get Object from Browsers* icon on the xeDebug toolbar.

UXE User Guide

Using the xeDebug GUI

Figure F-13 Design Browser (SimVision Version)



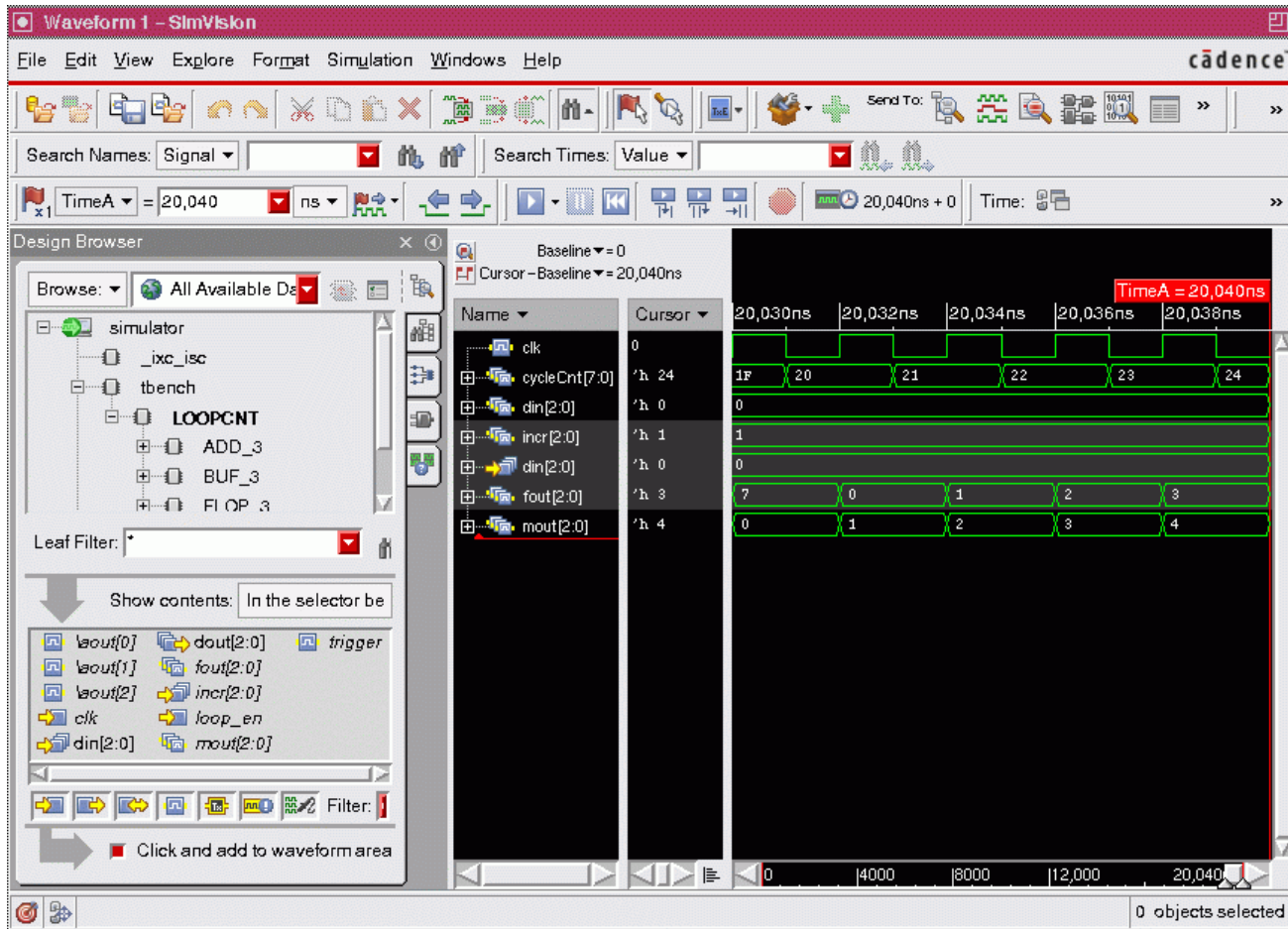
Waveform Viewer

The second icon on the toolbar can be used to display the Waveform Viewer. If the Waveform Viewer is running already, clicking this icon button will update the waveforms. This button is not available (is grayed out) until you have opened a session.

If you specify `-fsdb` option at the time of invoking xeDebug, the Novas tool's Waveform Viewer is displayed, otherwise the SimVision Waveform Viewer is displayed.

Note: If the markers for SDL triggers are not visible in the SimVision Waveform Viewer, access the *File* menu and click *Reload Database* in the SimVision Waveform Viewer window.

Figure F-14 SimVision Waveform Viewer Window

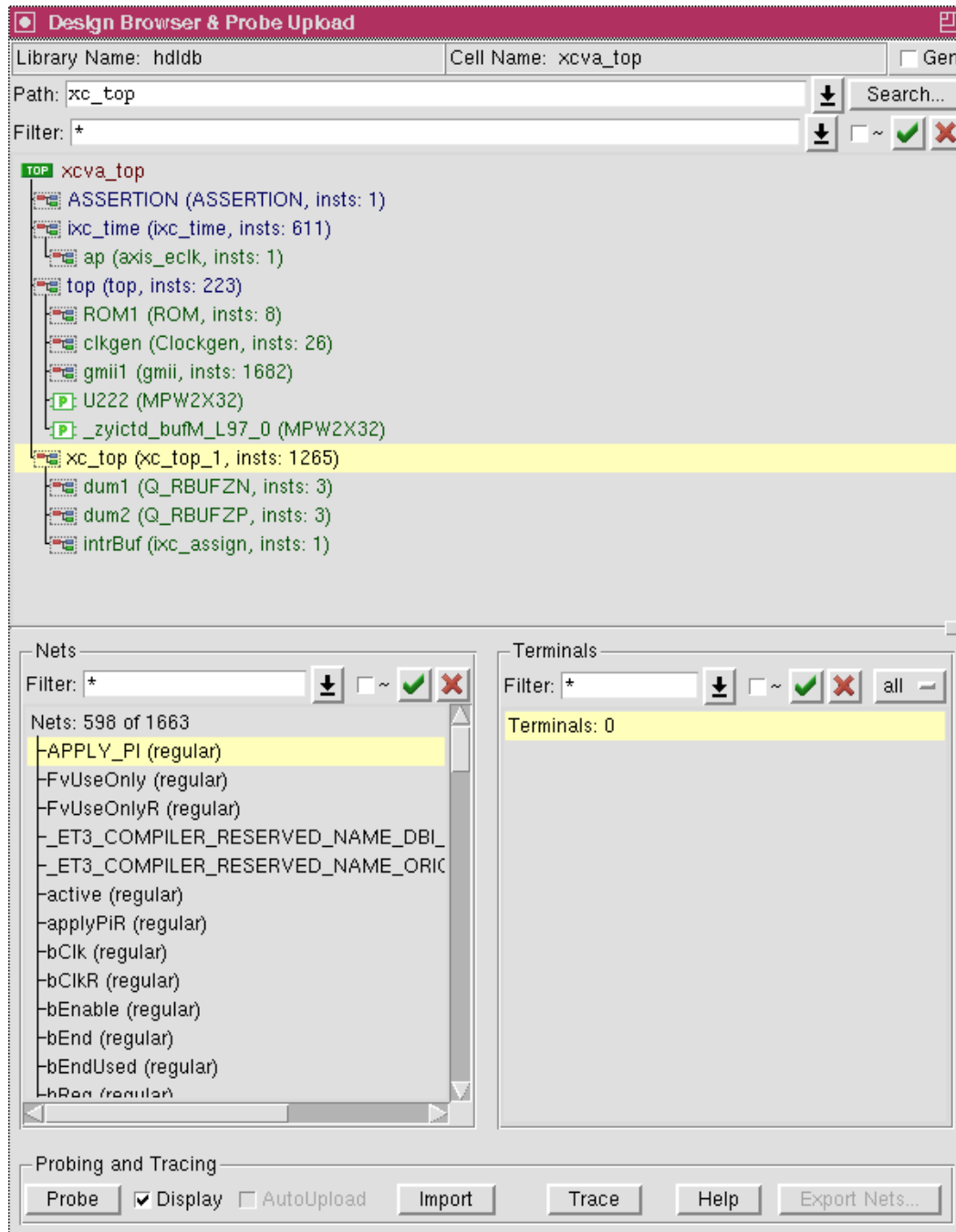


DUV Design Browser

The DUV Design Browser (as shown in [Figure F-15](#) on page 1187) allows you to examine design hierarchy of the entire design in the ICE mode, or only the DUT portion of the design in the SA mode.

This design browser is available only when using the xeDebug tool, and can be used to drag and drop signals to the xeDebug GUI windows.

Figure F-15 DUV Design Browser



To traverse the design hierarchy and view net names in the DUV Design Browser follow the steps mentioned below:

UXE User Guide

Using the xeDebug GUI

1. Expand or contract a design hierarchy by double clicking the instance name at the top of the browser.
2. To view all the nets or I/O terminals (ports) that belong to a specific design instance, select the instance name with the left mouse button, then double click on the line that shows the number of nets (on the left) or the number of terminals (on the right). This will expand the list of nets or terminals.
3. You can show or hide generated objects by selecting the *Gen* check box visible at the top right corner of the *DUV Design Browser* window. To refresh the list you need to close it if it is open and then open it again and do nothing if it was not open.

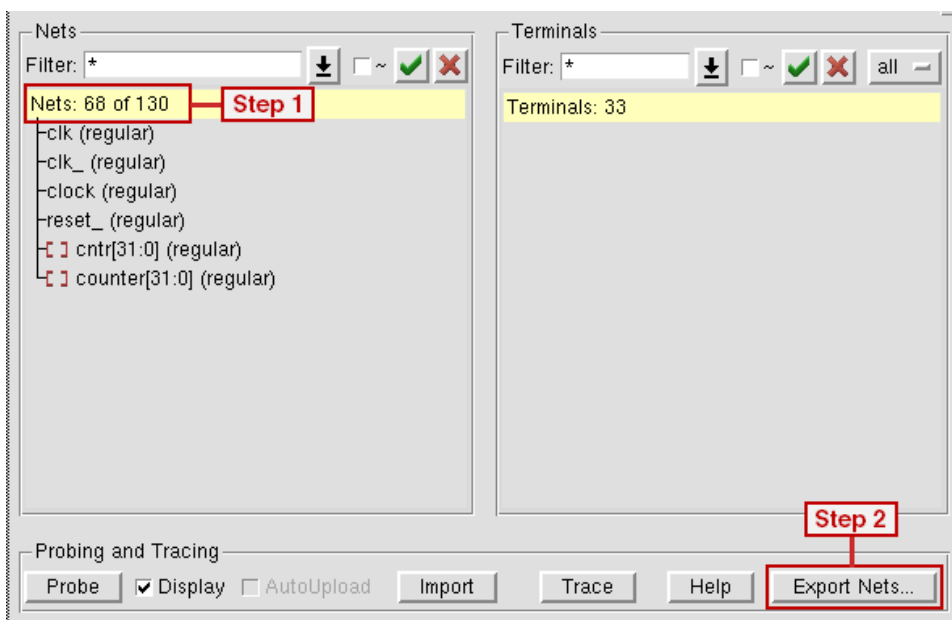
To drag and drop a net or instance name from the browser into one of the xeDebug GUI tabs or browsers:

1. Highlight the object name (instance name or net name) with the left mouse button.
2. Drag and drop the object name from the browser to the xeDebug GUI tab using the right mouse button.

Saving List of Nets to a File

1. In the *Nets* group box of the DUV Design Browser window (refer to [Figure F-16](#)), click the *Nets: <n> of <n>* field to display the hierarchy view of the nets available in the currently accessed design and select the nets you want to export.

Figure F-16 DUV Design Browser Window - Exporting Nets

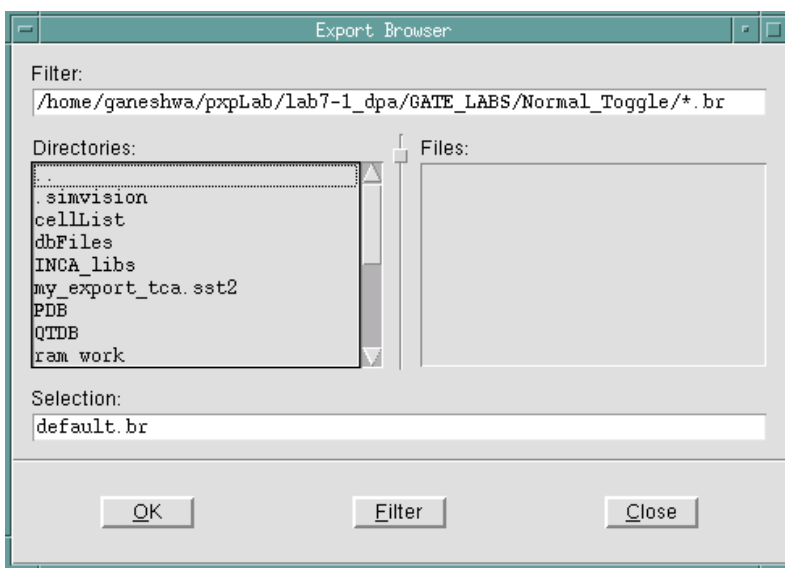


2. Click *Export Nets...* from the *Probing and Tracing* group box. The *Export Browser* dialog box illustrated in [Figure F-17](#) is displayed.

Note: To export all nets from the design, click *Export Nets...* without selecting any specific net from the hierarchy in the *Nets* group box.

3. Specify the file in which the selected nets should be exported using the *Export Browser* dialog box.

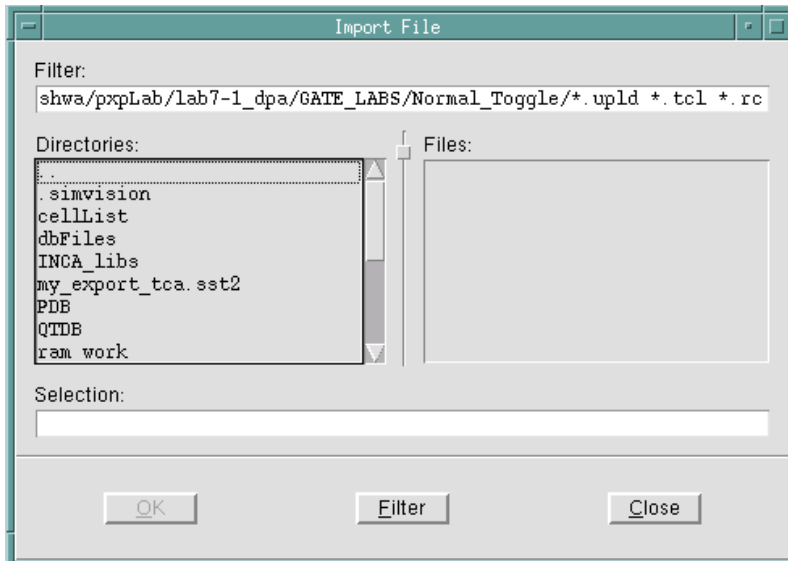
Figure F-17 DUV Design Browser Window - Export Browser Dialog Box



Probing Nets Saved in a File

1. Click *Import* from the *Probing and Tracing* group box of the DUV Design Browser window. The *Import File* dialog box illustrated in [Figure F-18](#) is displayed.

Figure F-18 DUV Design Browser Window - Import File Dialog Box



2. Select the file for import and click *OK*.

Symbols and Values Browser

The Symbols and Values browser window, shown in [Figure F-19](#) on page 1191, is typically used for managing symbols in SDL programs, probe definitions, and force/release/deposit/value operations.

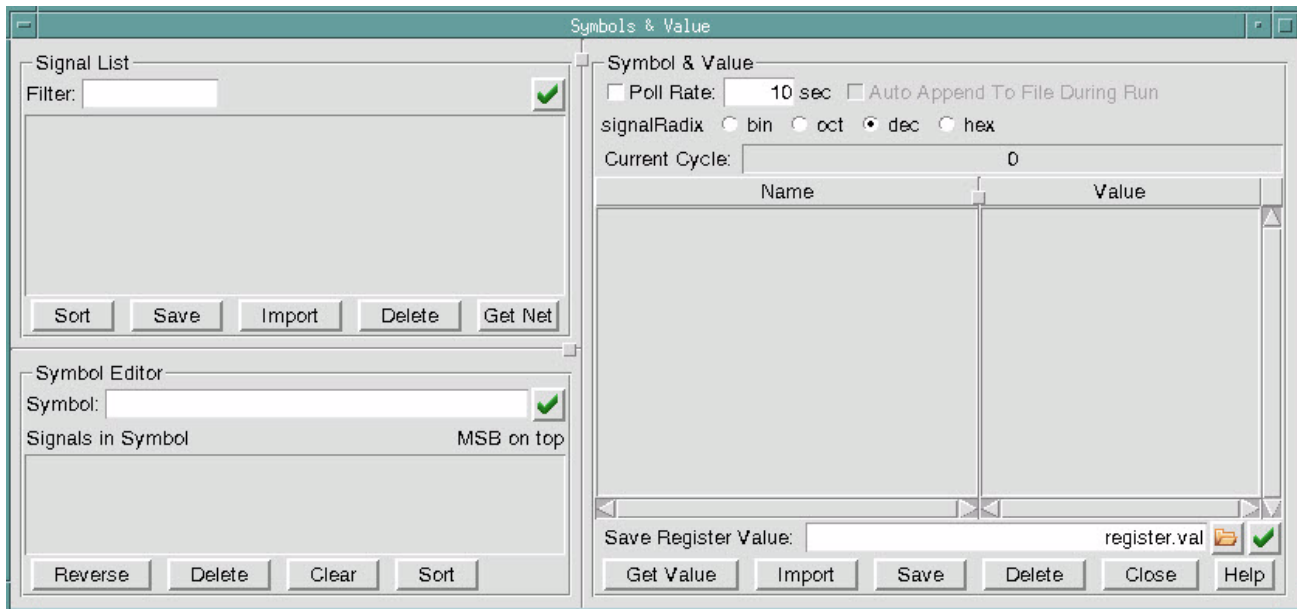
Note: This browser is accessible only after you have opened a session and downloaded the design.

The *Symbols & Values* window is divided into two regions. The left side of the window contains the *Signal List* and *Symbol Editor*. The right side of the window allows you to view emulated signal values during the run.

The *Signal List* group box serves as an intermediate repository for signal names. Use the *Get Net* button in this group box to port a net into this window from the Design Browser. From the Symbols and Values window, you can drag and drop into SDL or other forms by first selecting the net name (or a set of nets) with the left mouse button, and then dragging with the right mouse button.

The *Symbol Editor* group box at the bottom of the window allows you to define symbols that can then be used in other tabs, such as the *SDL*, *Probe*, and *Force* or *Deposit* tabs.

Figure F-19 Symbols and Values



Use the *Symbol Editor* group box to create symbols out of a collection of design signals. These symbols can be used for different purposes such as following:

- Define SDL expressions
- More efficient and faster computation of set/force/get of signal values (through the *Control* tab)

Use the following procedure to create symbols:

1. Enter desired net or signal names into the *Signal List* group box (at the top).
Double-click a single signal or highlight a group of signals, and click the  button to enter the selected signal(s) into the *Symbol Editor* group box.
2. In the *Symbol Editor* group box, assign a name to the *Symbol* and click the  button adjacent to the text field. It will be entered into the table in the *Symbol & Value* group box on the right of the window.

You can also display, but not edit, the symbol value in this window. The values can be displayed in either of the following forms:

- bin - binary
- hex - hexadecimal
- dec - decimal

UXE User Guide

Using the xeDebug GUI

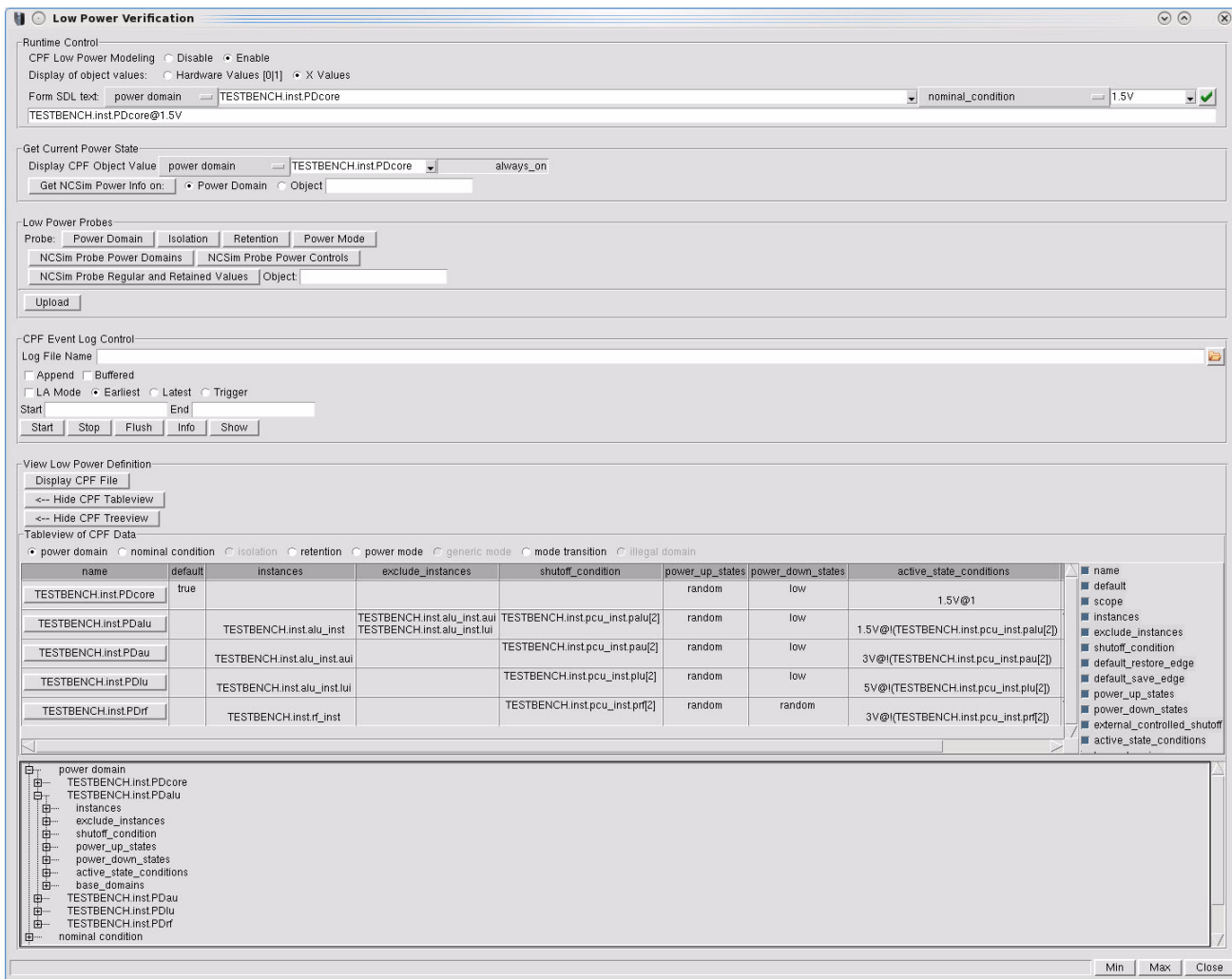
■ oct - octal

Low Power Verification

Clicking the lightbulb icon on the toolbar invokes the *Low Power Verification* window that helps in debugging low-power-aware designs.

For the layout of this window, refer to [Figure F-20](#) on page 1192. For more information about the *Low Power Verification* window, also refer to the [Using the xeDebug GUI for Low Power Verification](#) section of [Chapter 16, “Specifying Low-Power Intent on Palladium XP.”](#)

Figure F-20 Low Power Verification Window



The various group boxes of the *Low Power Verification* window enable you to perform the following actions:

- Run-time control
 - ☐ Enable or disable use of CPF in the design.
 - ☐ Specify that the power-loss values should be displayed as either $\times$ or real emulator values, if the CPF logic is enabled for the design.
 - ☐ Construct SDL statements.
- Get current power state values of the various CPF objects.
Probe the CPF objects.
- Log the CPF events.
- View the CPF data by using:
 - ☐ A text window for displaying the CPF file.
 - ☐ The table view to display the CPF data from the Palladium XP emulator.

The table displays all the CPF objects of the selected CPF object type. The CPF object name is displayed as a button that you can select to probe that object. If a type is grayed out, it means that CPF objects of that type do not exist in the design.
 - ☐ The standard hierarchical browser to display the CPF data in tree view.

Miscellaneous Tools

There are some tools that are available only for specific settings. Following are some of the tools:

- **Dynamic Power Analysis** (the power tower icon on the toolbar)

Invokes the Dynamic Power Analysis (DPA) menus. Requires a special license for use. For more information on DPA, refer to [Chapter 15, “Analyzing Power Consumption of the Designs.”](#)
- **Get Objects from Browser**

This tool is used in conjunction with most of the SimVision and Novas browsers.

When using the SimVision browsers, the *Get Objects from Browser* tool is typically used to get object names from the Schematic Browser, Design Browser, Source Browser, or Waveform Browser. When using the Novas tools, the *Get Objects from Browser* tool is typically used to get objects names from the TurboTracer viewer.

It allows you to highlight a net or instance in the browser and transfer the net name into empty fields on the xeDebug GUI tabs by clicking the button.

■ ***Schematic Browser***

The Schematic Browser displays the schematic view of a design as generated from its HDL source code. Signals can be dragged and dropped into waveform viewer.

■ ***Exit***

Exits the xeDebug tool.

Starting a Debug Session

When the xeDebug GUI is invoked, the initial *Setup* tab (shown in [Figure F-1](#) on page 1168) has the same fields in both SA and ICE modes. For details about the layout of the initial *Setup* tab, refer to the [Invoking xeDebug GUI](#) section.

However, after you open the design, the fields displayed on the *Setup* tab are different in the SA mode (shown in [Figure F-21](#) on page 1195) and ICE mode (shown in [Figure F-26](#) on page 1203). The following sections discuss these differences.

Using Setup Tab in SA Mode

In the SA mode, the *Setup* tab handles the tasks such as specifying the host machine, viewing emulator status, selecting vision and debug modes, and specifying *irun* options.

After the design has been opened, the *Database and Work Directory* group box displays the information in view only mode (as shown in [Figure F-21](#) on page 1195). To start a debug session, perform the following steps:

1. Select whether to use the emulator for debugging in *Online* or *Offline* session.

Note: If you select the *Offline* session, the layout of the *Setup* tab is different and steps discussed below are not required. Instead, refer to the [Setup Tab in SA Mode for Offline Debugging](#) section for the steps that follow the selection of *Offline* session.

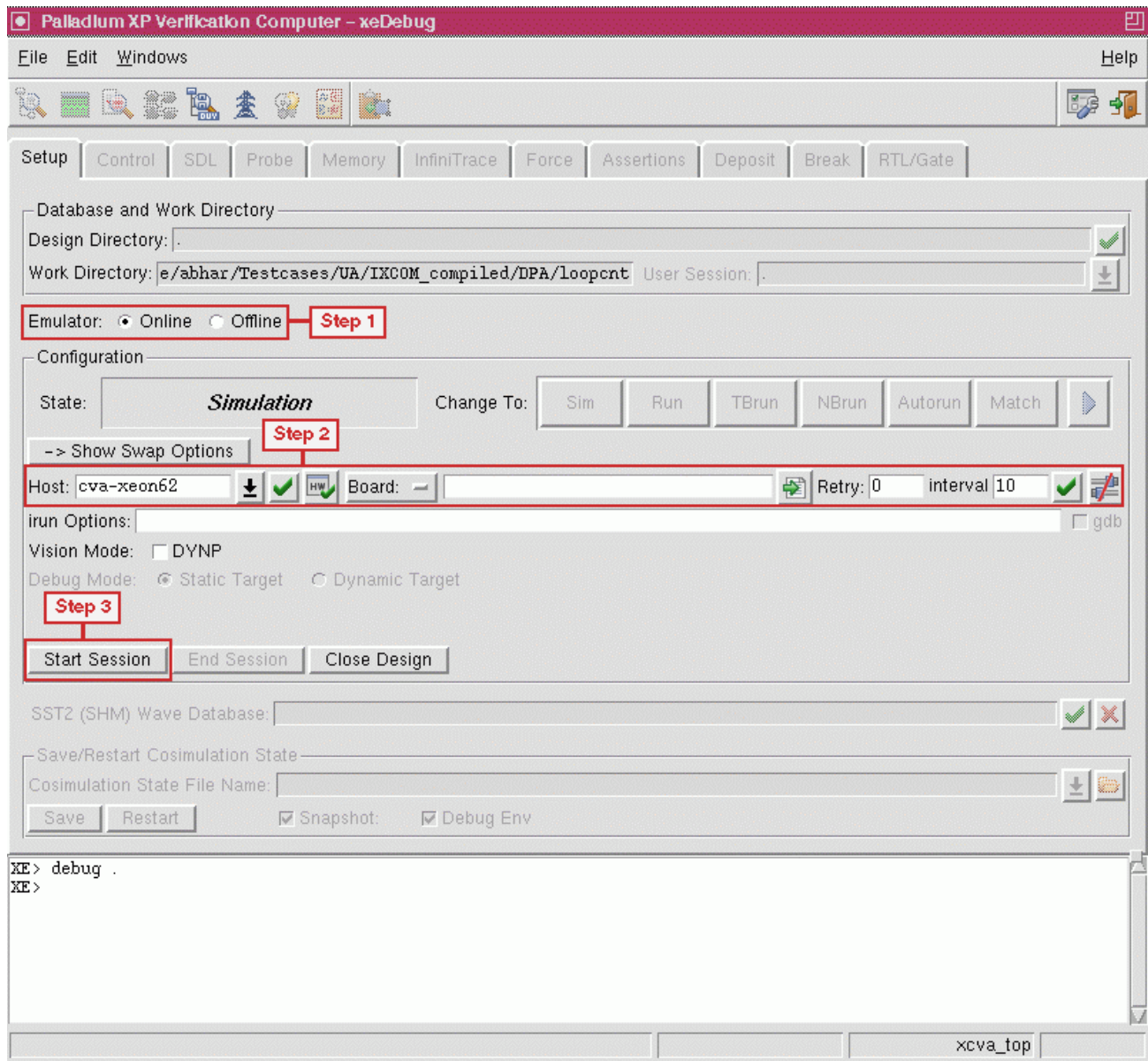
In the *Configuration* group box below the *Emulator* options, the following fields are displayed in disabled format:

- ☐ *State* of the design run as *Simulation*
- ☐ Buttons adjacent to *Change To* field
- ☐ Options accessible on clicking the --> *Show Hot Swap Options* button

UXE User Guide

Using the xeDebug GUI

Figure F-21 Setup Tab after Opening the Design in SA Mode



2. Specify the emulation host in the *Host* field and click the button.

- A record of the emulators selected in the previous debugging sessions is maintained in a drop-down list. You can access this list by clicking the down-arrow button adjacent to the *Host* field. This is a handy shortcut for selecting an emulation host.
- You can use the button to check host and domain availability of the specified emulator. Clicking this button runs the `test_server` program, and displays the available domains and their configuration. This also includes information on the

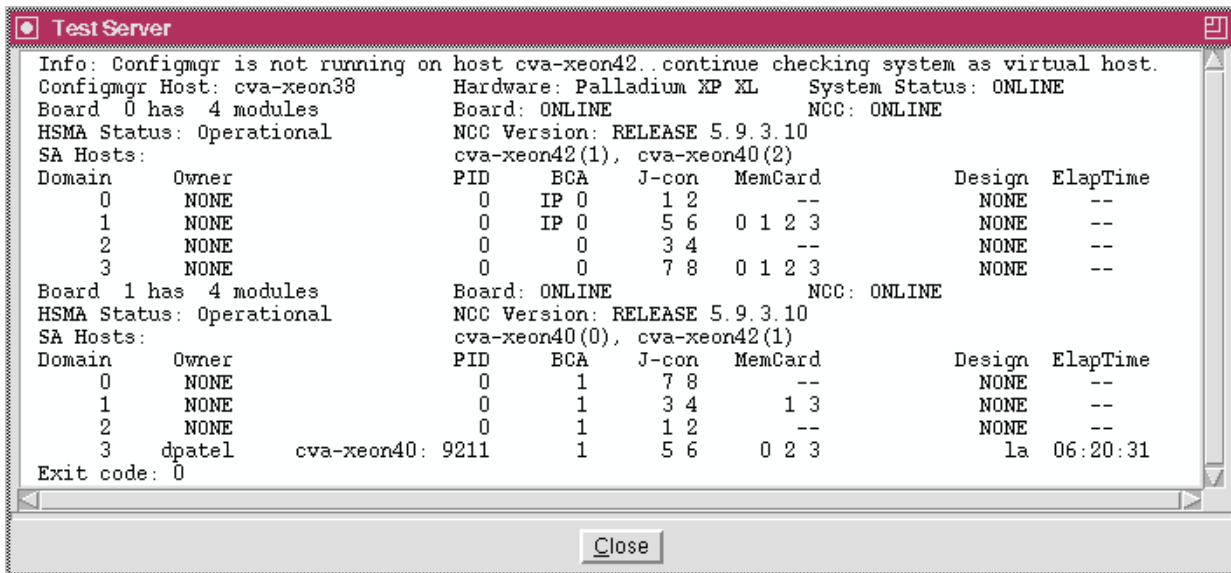
UXE User Guide

Using the xeDebug GUI

domains with SA cards and/or other cable connections. You can fine-tune results of the `test_server` program by using it directly on the command prompt, with its options.

Figure F-22 shows an example of the *Test Server* dialog box displaying the results.

Figure F-22 Sample Test Server Dialog Box



- c. In the *Board* field, you can also specify the name of either the board preference (`.bp`) file or the board restriction (`.br`) file. These files are useful in identifying the specific boards or domains to which you want to download the design.

If you do not have either of these files, you can enter the board numbers or domain numbers to which the design must be downloaded. Board or domain numbers should be separated by spaces.

- d. If the download of the design on the specified emulation host fails, the xeDebug tool retries to download for the number of times specified in the *Retry* field and at the time interval in seconds specified in the *Interval* field. By default, the interval is set to 10 seconds.
- e. There is a button bearing a logo of cables with a slash through them. This button allows you to download the design onto the emulator, even if cables are not present.

If you click this button, a dialog box is displayed. This dialog box allows you to select the target cables to download or skip. Skipping download of cables allows you to download a design to a set of domains that lacks some of the target cables specified in the compile. If you skip downloading a cable, the behavior of inputs and

UXE User Guide

Using the xeDebug GUI

bidirectional I/Os assigned to that cable is unspecified, and the pins of that cable (if it is present) do not drive the target.

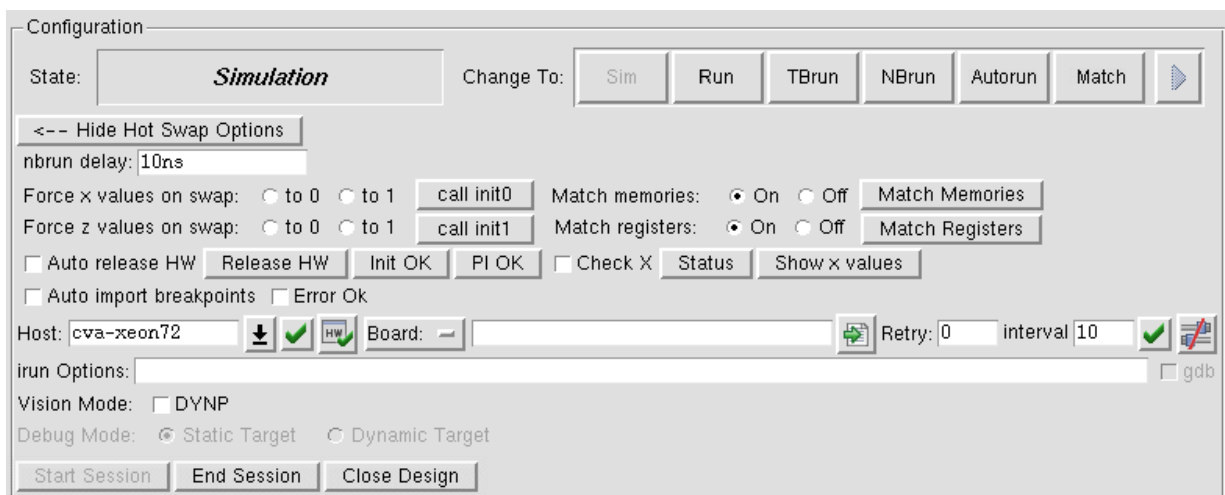
- f. You can also specify the *irun Options*, if required.
 - g. To allow a design compiled in FullVision mode to run in Dynamic Probe (*DYNP*), select the check box adjacent to the *Vision Mode* field. This is particularly useful if increase in upload speed and maximum trace depth is preferred. However, note that switching to Dynamic Probe mode will not affect the emulation speed and step count.
3. Click *Start Session* to begin the debugging session. The layout of the *Setup* tab changes as shown in [Figure F-24](#) on page 1199.

The following buttons corresponding to the *Change To* field get enabled:

- ☐ *Sim* (selected by default for Simulation Acceleration mode)
- ☐ *Run* (for Signal-based Acceleration mode: *Emulation* and Simulation)
- ☐ *TBrun* (for Transaction-based Acceleration mode)
- ☐ *NBrun* (for advancing emulation and simulation time concurrently)
- ☐ *Autorun* (for Emulation mode)
- ☐ *Match* (for DUT in Emulation and Simulation while doing a equivalency match)

Also, the fields displayed on clicking --> *Show Swap Options* get enabled. Refer to [Figure F-23](#) on page 1197.

Figure F-23 Hot Swap Options on Setup Tab



The *Start Session* button is disabled and instead the *End Session* button gets enabled. Also, note that the *Vision Mode* field is disabled indicating that you cannot change to Dynamic Probes mode after the debug session starts.

In addition, the other tabs of the xeDebug tool based on the design intent get enable. For example, [Figure F-24](#) on page 1199 illustrates that the *Control*, *Probe*, *InfiniTrace*, *Force*, *Deposit*, and *Break* tabs have become enabled, while others such as *SDL* and *Memory* are still disabled. For a description of each xeDebug GUI tabs, refer to the [Running the Design Using the xeDebug Tabs](#) section.

4. In the *SST2 (SHM) Wave Database* field (refer to [Figure F-24](#) on page 1199), specify the name of the waveform database to be opened. If you specify a name that does not match an existing waveform database name, a new waveform database is created and the old waveform database is closed. Also, all probes from the old waveform database are copied into the new waveform database.

Note: This field is not visible when the swapping options are displayed using the --> *Show Swap Options* button.

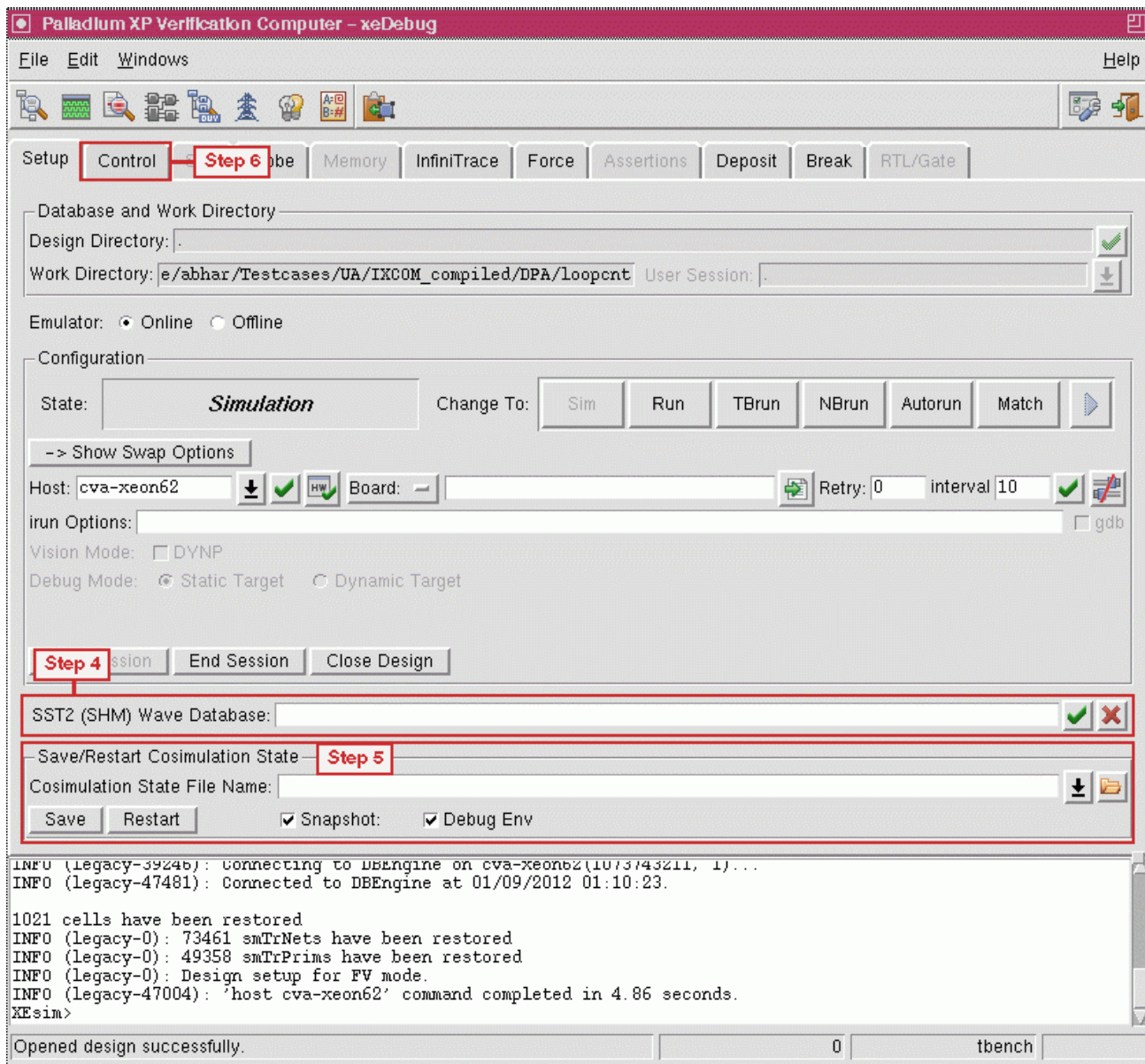
5. The *Save/Restart Cosimulation State* group box allows you to specify an existing *Cosimulation State File Name* to *Restart* the emulator using the settings of a previous design run. Otherwise, you can create a new file to *Save* the cosimulation state for later use.

This group box also has two check boxes, *Snapshot* and *Debug Env*. Selecting these check boxes indicates that the snapshot and/or debug environment needs to be saved/restarted.

UXE User Guide

Using the xeDebug GUI

Figure F-24 Setup Tab after Starting a Debug Session in SA Mode



6. For running the design, access the *Control* tab. For further details, refer to the [Control Tab](#) section.

Setup Tab in SA Mode for Offline Debugging

Selecting the *Offline* radio button on the *Setup* tab allows you to debug a design in the offline session, using only the database files and the waveform viewer; thus, freeing the emulator for other users. Debug utilities available in offline session include adding and displaying

uploaded physical probes, calculating virtual probes, and bringing up waveform and schematic browsers.

Note: In an offline session in SA mode, you do not have the hot swap options to switch from the simulator to emulator or vice versa.

In offline session, the layout of the *Setup* tab is as illustrated in [Figure F-25](#) on page 1201. Follow the steps given below for debugging in an offline session:

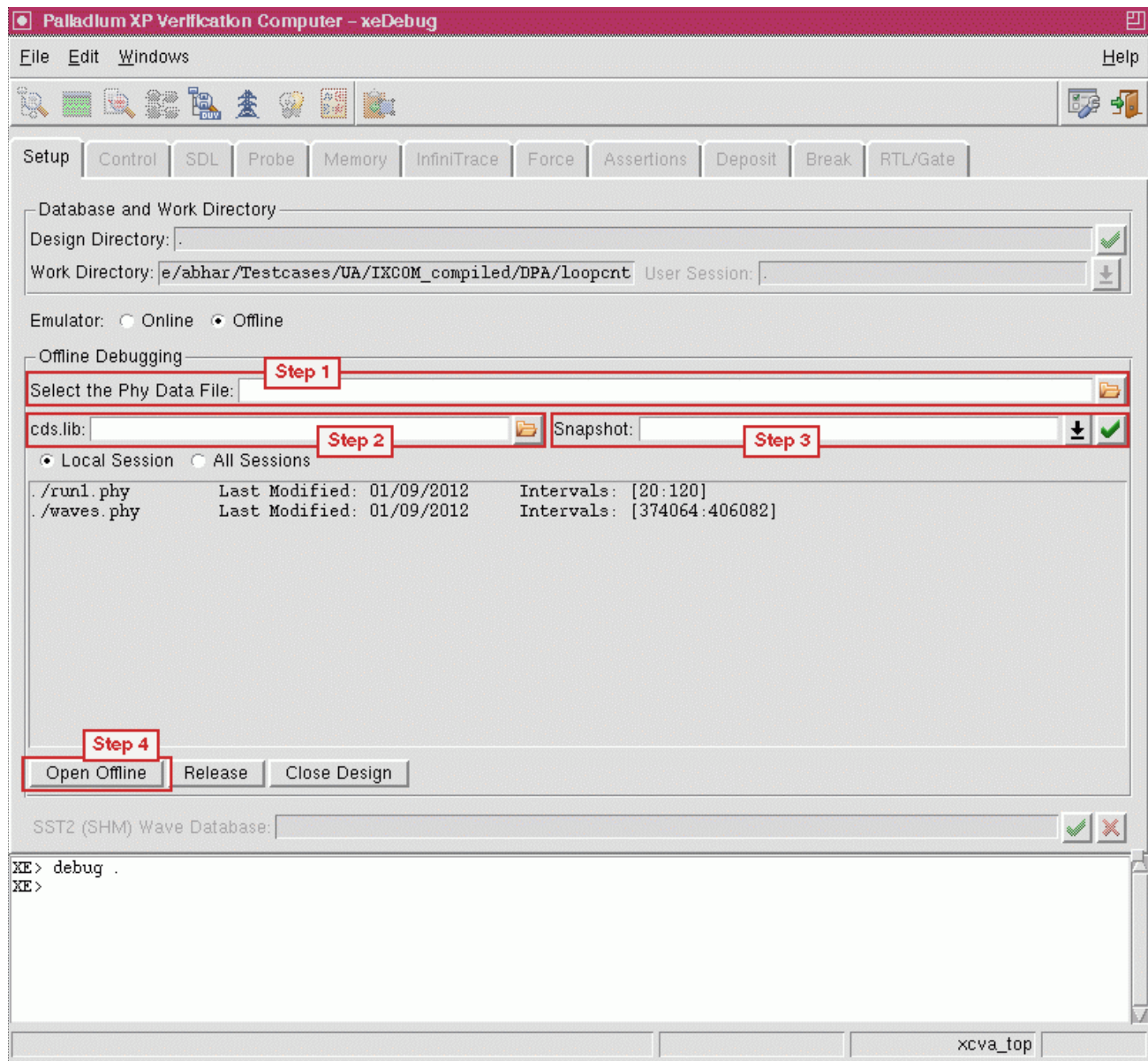
1. In the *Offline Debugging* group box, specify the *Phy File* to use for debugging. You can use the button with directory icon adjacent to the *Phy File* to select the required file with `.phy` extension.
2. Optionally, you can specify the library definition in the *cds.lib* field. By default, the file is located in `INCA_libs/worklib` directory.
3. Specify the design *Snapshot* to provide the connectivity-related information.
4. Click *Open Offline* to start the offline session.
 - a. If you do not want to continue with the offline session at any time, click *Release* to release the offline host.
 - b. Clicking *Close Design* enables you to close the current design and select a new design to debug on the same offline host.

Note: The *SST2 (SHM) Wave Database* field gets enabled after you click *Open Offline*. Specify the name of the waveform database you want to open.

UXE User Guide

Using the xeDebug GUI

Figure F-25 Setup Tab in Offline Debug Mode (SA Mode)



Using Setup Tab in ICE Mode

After opening the design in the ICE mode, the *Setup* tab (as shown in [Figure F-26](#) on page 1203) allows you to:

- Specify whether you will start the debug session in *Online* or *Offline* session
- Define the *Configurations*, such as emulation *Host* and *Vision Mode*.

UXE User Guide

Using the xeDebug GUI

The steps for defining the type of debug session and configuration are somewhat similar to what have been already discussed in the [Using Setup Tab in SA Mode](#) section. Therefore, only the steps that are different in ICE mode are discussed below.

Note: If you select to use the emulator in the *Offline* session, the layout of the *Setup* tab is different and steps discussed below are not required; instead, refer to the [Setup Tab in ICE Mode for Offline Debugging](#) section.

1. Optionally, you can specify the library definition in the *cds.lib* field. By default, the file is located in `INCA_libs/worklib` directory.
2. Select the *Online* radio button to start an online session.

You must initially download the database to an online session and create a trace file, before you can specify an offline session. If you have already saved a trace file from a previous online session, you can specify the trace file and open an Offline session.

Note: If you want to use the *InfiniTrace* option, you must start debugging in an online session.

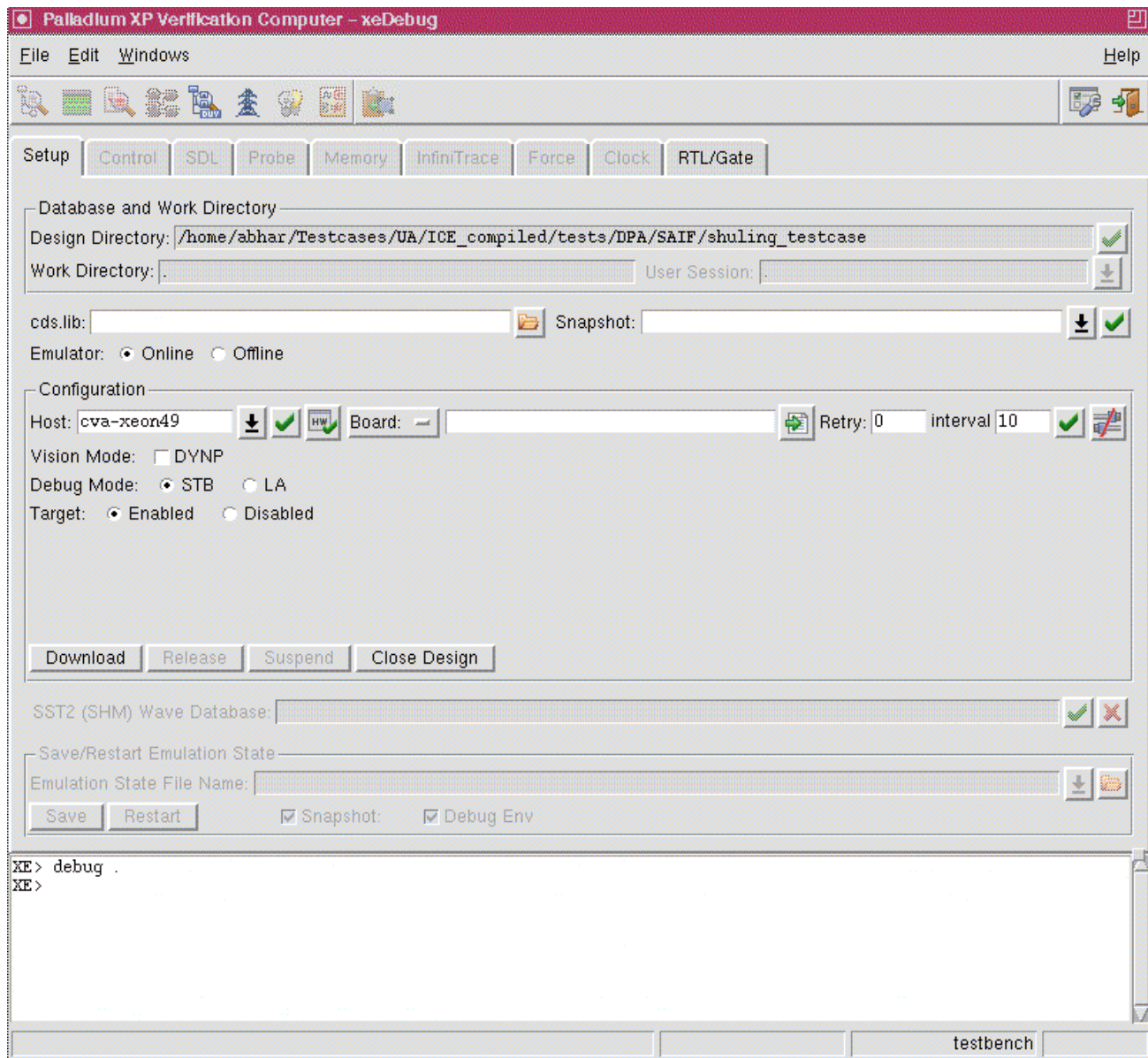
3. Specify the design *Snapshot* to provide the connectivity-related information.
4. In the *Configuration* group box, select the *Debug Mode*, that is, *STB* or *LA*.
5. Select whether the *Target* connection should be *Enabled* or *Disabled*.
6. Click *Download* to start the debug session.

While download is taking place, the status bar at the bottom of the screen shows the text *Busy* with yellow background in the right corner. When the download is complete, this changes to *Done* with green background and the other tabs become accessible based on the design elements.

UXE User Guide

Using the xeDebug GUI

Figure F-26 Setup Tab after Opening the Design in ICE Mode

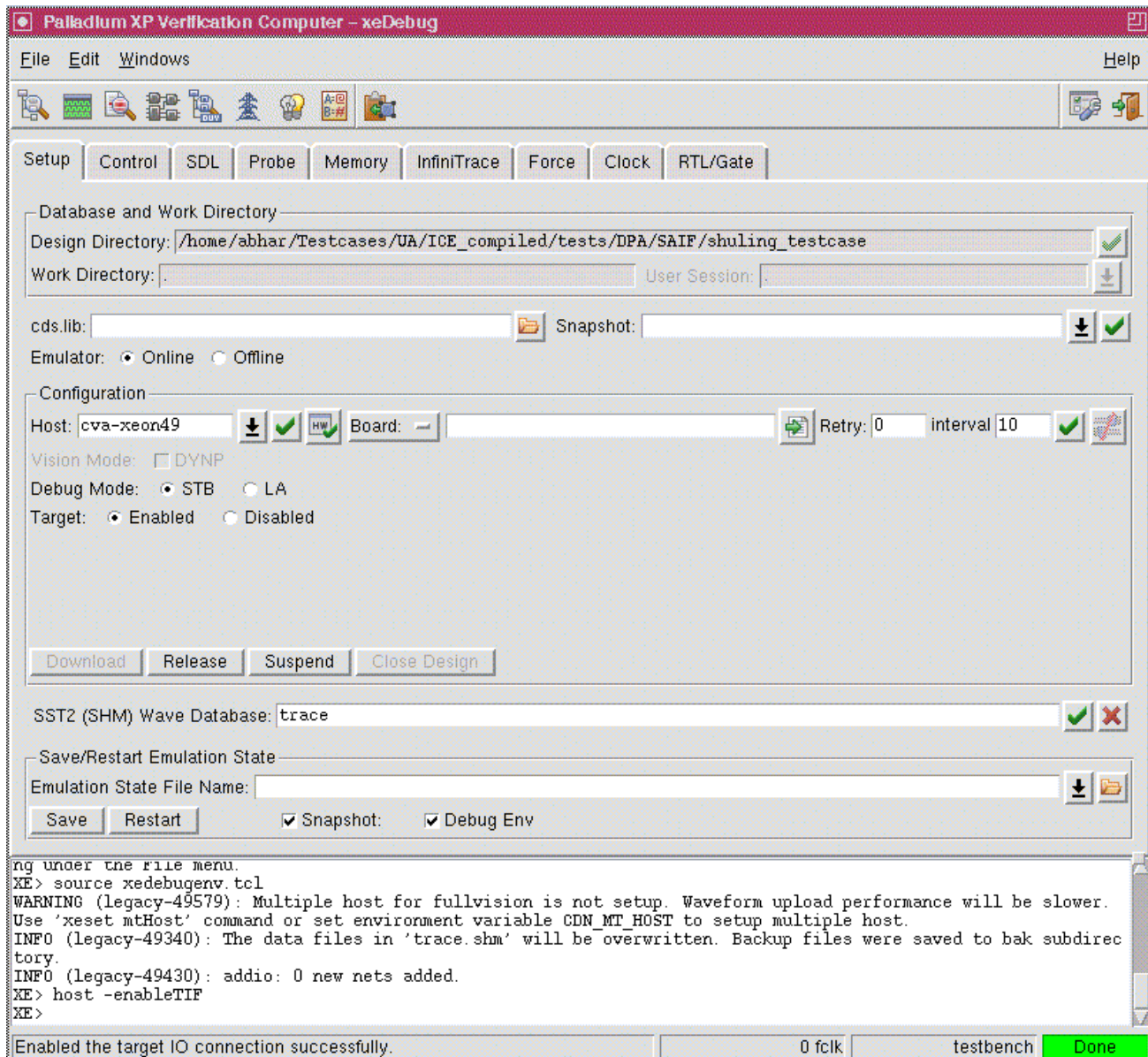


After the design is downloaded to the specified emulator, the changes in the layout of the *Setup* tab are as illustrated in [Figure F-27](#) on page 1204.

UXE User Guide

Using the xeDebug GUI

Figure F-27 Setup Tab after Downloading the Design in ICE Mode

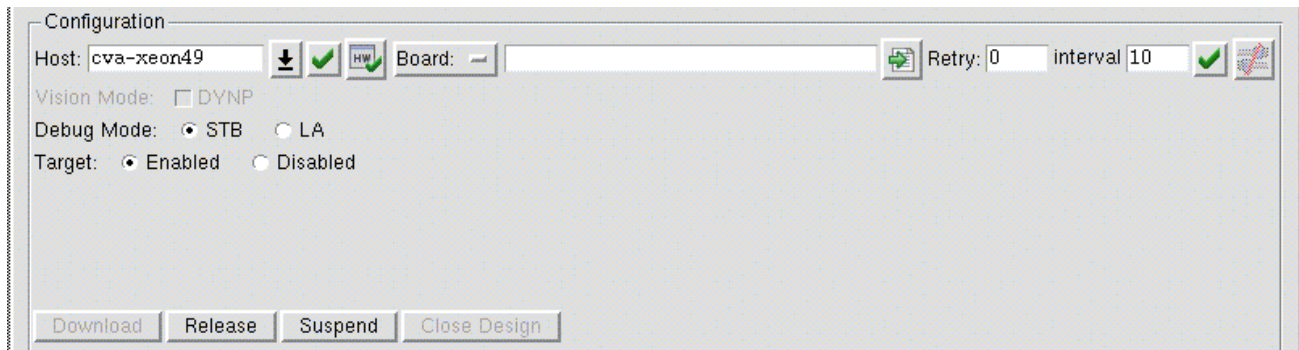


The *Release* and *Suspend* buttons get enabled on the *Setup* tab after design download. Also enabled are the *SST2 (SHM) Wave Database* field and the *Save/Restart Emulation State* group box, which has the function similar to the *Save/Restart Cosimulation State* group box on the *Setup* tab displayed in SA mode. For details, refer to steps 4 and 5 in the [Using Setup Tab in SA Mode](#) section.

Suspending and Resuming the Design Run Using the Setup Tab

The emulation session can be suspended at any time after downloading the design by clicking the *Suspend* button available on the *Setup* tab. Refer to [Figure F-28](#) on page 1205.

Figure F-28 Suspend Button on Setup Tab



In the xeDebug GUI, there is no need to set the `suspendEnable` run-time parameter. The session can be suspended anytime during emulation. You need not be running clocks to suspend the emulation.

For detailed information on the processing of the suspend and resume operations, refer to the [Using Suspend and Resume to Manage Emulator Resources](#) section in [Chapter 2, “Working with the Palladium XP Emulators.”](#)

Setup Tab in ICE Mode for Offline Debugging

If you select the *Offline* radio button on the *Setup* tab, the layout of the tab is as illustrated in [Figure F-29](#) on page 1207. The steps performed while debugging in an offline session in ICE mode are same as in SA mode. Only one step associated to the *Offline Debugging* group box is different.

In the *Offline Debugging* group box, select a radio button adjacent to *Operation Mode* to specify whether you want to use a *Probe Phy File* or *Software InfiniTrace* observe session to generate the waveforms.

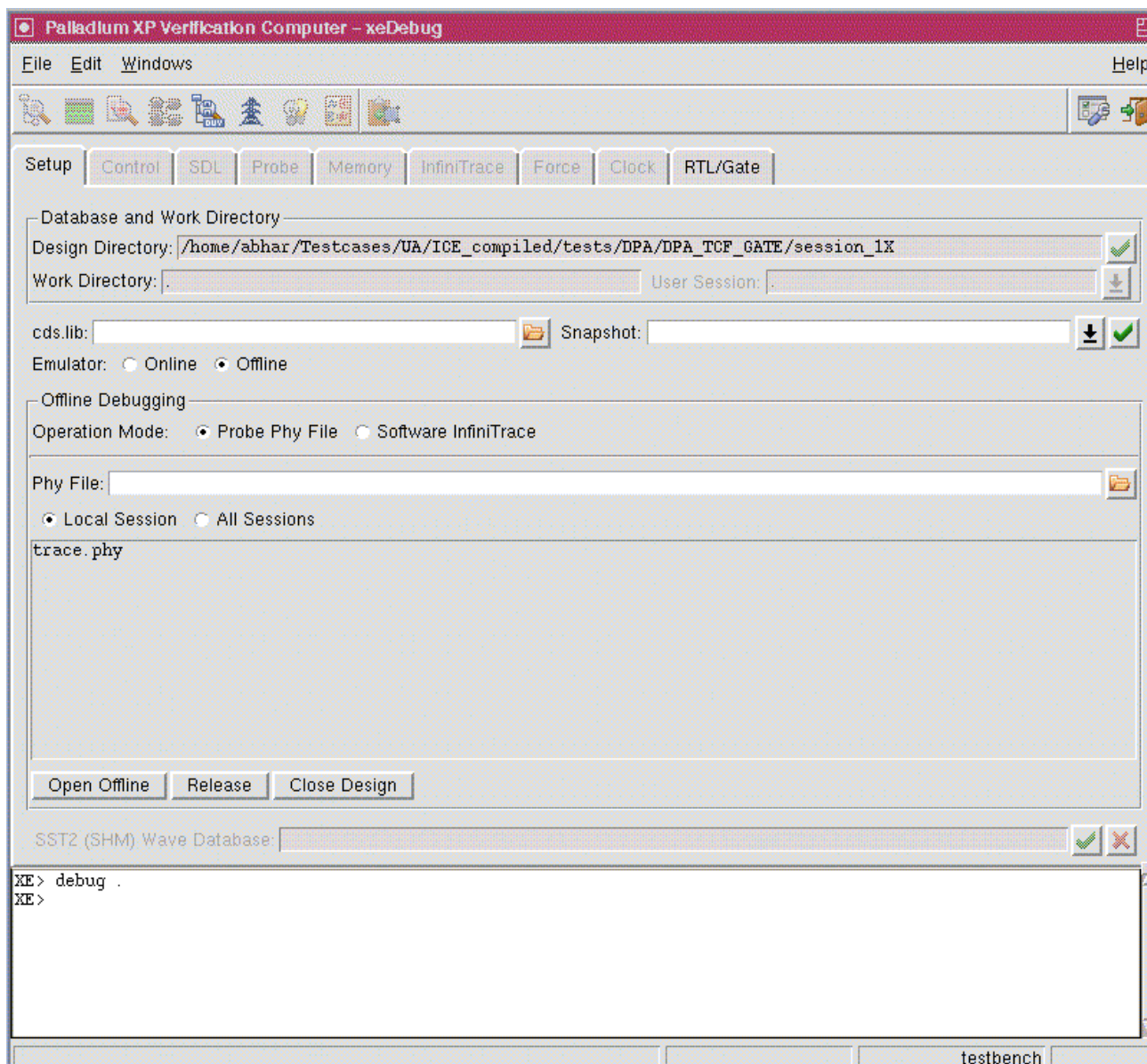
UXE User Guide

Using the xeDebug GUI

- a. If you select the *Probe Phy File* button, you need to specify the *Phy File* to use for debugging. You can use the button with directory icon adjacent to the *Phy File* to select the required file with `.phy` extension.
- b. If you select the *Software InfiniTrace* button, you need to specify the *Software InfiniTrace Observe Session* to use for debugging.

For details about the other fields available on *Setup* tab in the offline session, refer to the Setup Tab in SA Mode for Offline Debugging section.

Figure F-29 Setup Tab in Offline Debug Mode (ICE Mode)



Running the Design Using the xeDebug Tabs

After the design has been downloaded, tabs other than the *Setup* tab are enabled as per their relevance in the design being run. For detailed information on the *Setup* tab, refer to [“Starting a Debug Session”](#) on page 1194.

The following sub-sections discuss the remaining tabs in detail:

■ **Tabs available in both SA and ICE modes**

❑ Control Tab

The *Control* tab controls the running of a design.

In the ICE mode, it contains functions for InfiniTrace capture, forcing nets, triggering, capturing trace data, and monitoring status. Three versions of the *Control* tab in the ICE mode are available: Vector Debug ([Figure F-35](#)), Logic Analyzer, STB FullVision ([Figure F-30](#)).

In the SA mode, it contains only the run controls and ability to specify the waveform database. Refer to [Figure F-31](#) (without uncontrolled clocks) and [Figure F-32](#) (with uncontrolled clocks).

❑ SDL Tab

Allows you to access the SDL event editor and file editor.

○ Using Advanced SDL Controls

❑ Probe Tab

This tab lists all probes added at run time. Use this tab to add probes by net, instance, driver, or add physical probes. You can import probes from a schematic, save probes to a file, import a probe file, or remove probes.

❑ Memory Tab

Controls memory read/write (that is, dump and load) and initialization operations.

❑ InfiniTrace Tab

This tab allows you to save and restore the emulation status for a user-specified period of time. You can observe the emulation status, dump out memory content, get the value of any net in the design, and add probes to upload the waveform of any net.

Note: In the SA mode, this tab is enabled only when the design is swapped to the emulator. Also, InfiniTrace cannot be used in LA mode.

❑ Force Tab

This tab allows you to force, set, and get values for individual nets or symbols. A symbol is one or more probes (signal) represented by a single name.

❑ Assertions

This tab allows you to access the assertion functionality through the menu interface. The tab is divided into three sub-tabs: Config & Settings, Assertion Control, and

Reports. For details, refer to the [Running and Debugging Designs with Assertions](#) section of the [Chapter 10, “Compiling and Running Designs with Assertions.”](#)

- ☐ [RTL/Gate Debug Tab](#)

Controls the RTL files and gate-level netlist debug functions.

■ **Tabs available in only SA mode**

- ☐ [Deposit Tab](#)
- ☐ [Break Tab](#)

■ **Tabs available in only ICE mode**

- ☐ [Clock Tab](#)

If the database is compiled using CAKE or SCM technology, the *Clock* tab lists all the clocks that can be changed at run time. You can use this tab to change the frequency or delay for a specified clock, or to change the oversampling ratio. The clock file can be saved for later import.

Control Tab

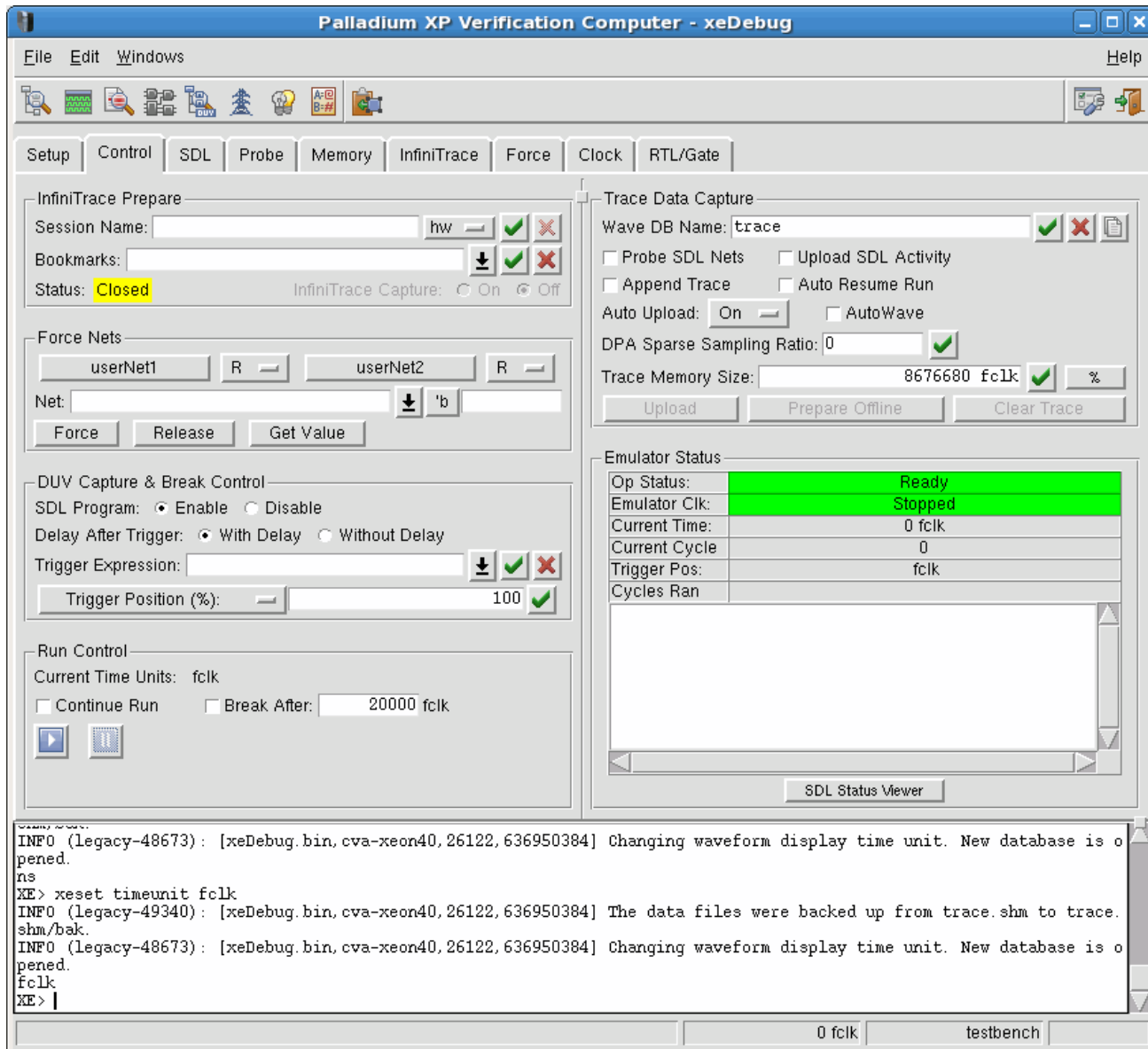
Once the debug session has been opened, the *Control* tab is displayed. The *Control* tab in STB mode is illustrated in [Figure F-30](#). The *Control* tab in SA mode is illustrated in [Figure F-31](#) (without uncontrolled clocks) and [Figure F-32](#) (with uncontrolled clocks). When you have finished configuring the design in the other tabs, click the *Control* tab. You must set up the testing parameters using the other debug forms, before starting the design runs.

Note: In Simulation Acceleration, you can use Ctrl+C to stop an emulation run. This command will not stop a run initiated from the *Control* tab in ICE mode.

UXE User Guide

Using the xeDebug GUI

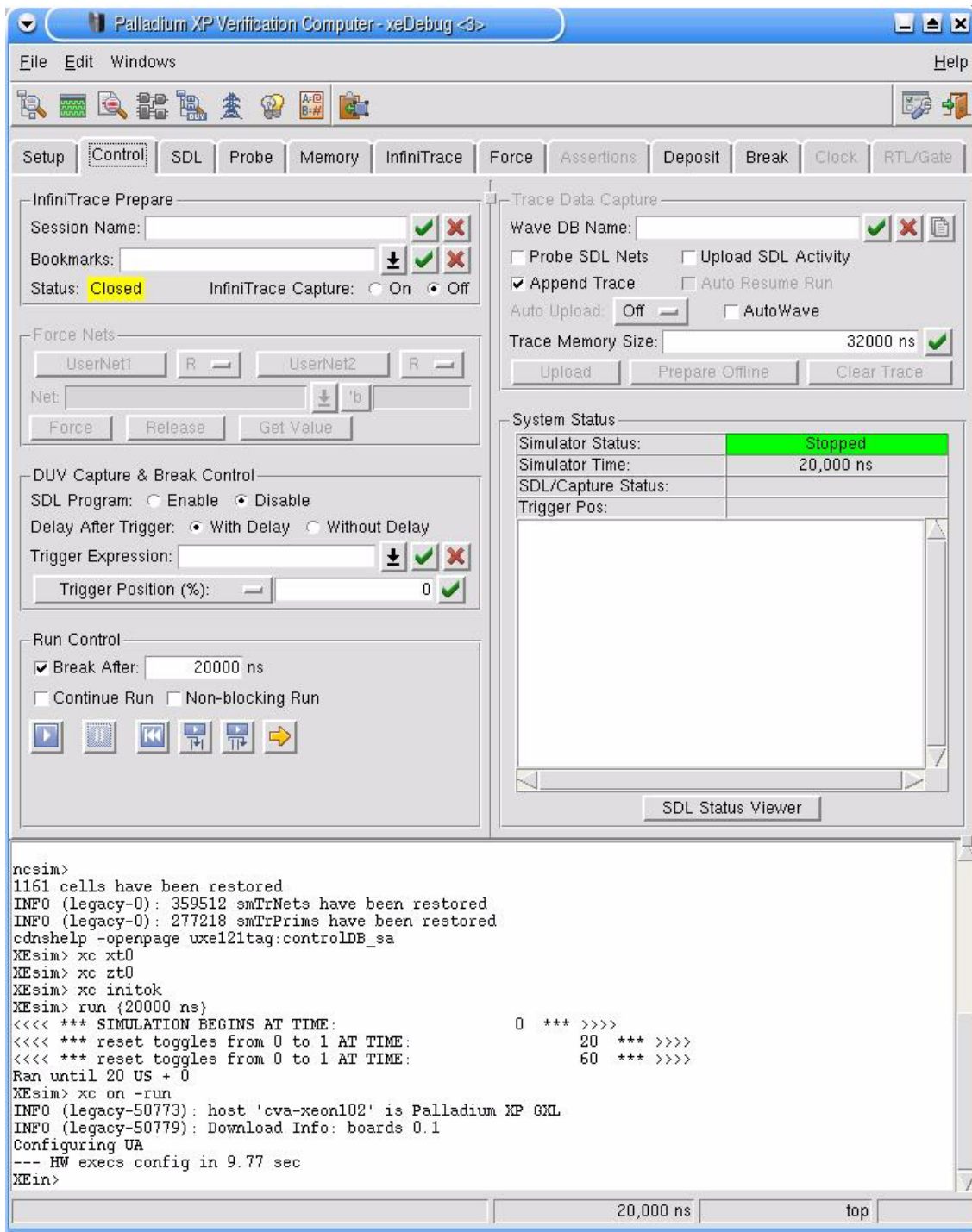
Figure F-30 Control Tab (STB Mode)



UXE User Guide

Using the xeDebug GUI

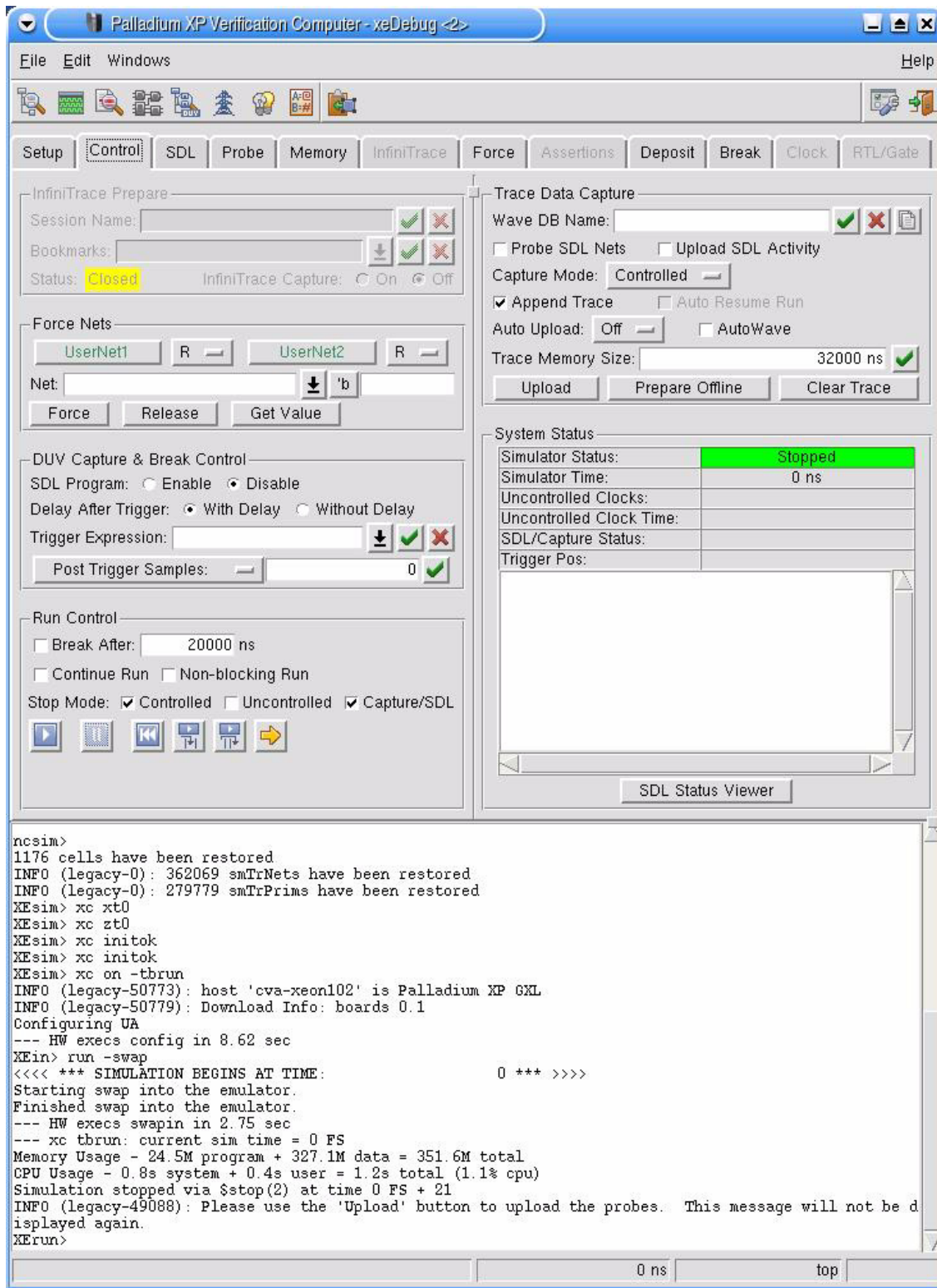
Figure F-31 Control Tab (SA Mode Without Uncontrolled Clocks)



UXE User Guide

Using the xeDebug GUI

Figure F-32 Control Tab (SA Mode With Uncontrolled Clocks)



The *Control* tab is composed of the following functional areas:

1. *InfiniTrace Prepare* group box is used to prepare for an InfiniTrace session. After an InfiniTrace session has been created, the InfiniTrace tab will be enabled.
 - a. Specify the InfiniTrace *Session Name* and click the  button to confirm the selection.
 - b. Specify a *Bookmark* name, if required. For details, refer to the [Adding Bookmarks when Preparing InfiniTrace Session](#) section.
 - c. The *Status* field displays whether the current session is *Opened* or *Closed*.
 - d. Use *InfiniTrace Capture* radio buttons to turn *On* and *Off* InfiniTrace data capture.

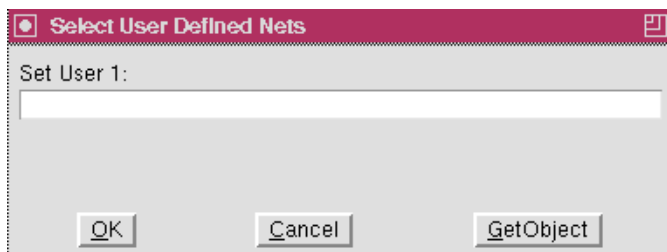
Note: For those debug modes where InfiniTrace Prepare is not available, the corresponding InfiniTrace tab is not functional.

2. *Force Nets* group box allows you to define two user nets.

Note: This group box is enabled only in the ICE mode. In the SA mode, it is disabled even when the design is being run on the hardware-side.

- a. The nets that you have defined can be selected using the two buttons, *UserNet1* and *UserNet2*, which when clicked display the *Select User Defined Nets* dialog box as shown in [Figure F-33](#).

Figure F-33 User-Defined Net Pop-Up



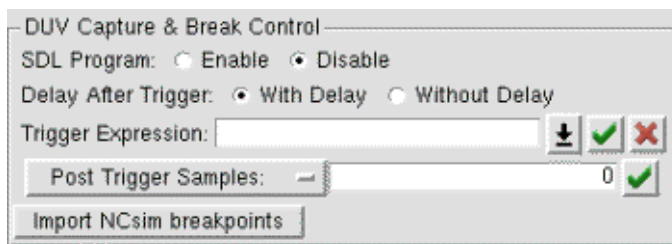
The toggle button adjacent to each user net definition button provides the following options – *R* to release the forced value. *0* to force the signal to a value of 0, and *1* to force the signal to a value of 1.

- b. Specify in the *Net* field the signal and its accompanying value. The adjacent button specifies the format (binary, hex, octal, or decimal).
- c. Clicking the *Force* button sets the state of the selected net to the specified value until explicitly released by clicking *Release*.

- d. Clicking the *Release* button releases a previously-forced signal.
 - e. Clicking the *Get Value* button retrieves the current state of the specified signal.
3. *DUV Capture & Break Control* group box allows you to select trigger modes for SDL instances. If no trigger mode is defined, the design runs without any trigger conditions set.

Note: In the SA mode, this group box is enabled only when the design is being run on the hardware-side. In this case, in addition to the fields explained below, an extra button, *Import NCsim breakpoints*, is displayed (as shown in Figure F-34.) For corresponding details, refer to the [Importing Breakpoints from the NC-Sim Simulator into SDL Using GUI](#) section.

Figure F-34 DUV Capture & Break Control Group Box in SA Mode (Hardware Run)



- a. Select either *Enable* or *Disable* radio button to control the *SDL Programs* that have not been specifically enabled or disabled using the *Advanced SDL Control* window.
- b. Select either *With Delay* or *Without Delay* radio button to define whether delay needs to be added after a trigger occurs.
- c. In the *Trigger Expression* field specify the event or logic expression for triggering by expression. The syntax and semantics of the expressions are the same as SDL expressions.

In STB mode, when this switch is selected, all the components of the SDL program that were defined in the SDL form are disabled without exception, even those that were separately enabled using the following XEL command:

```
sdl -enable <instance_names_list>
```

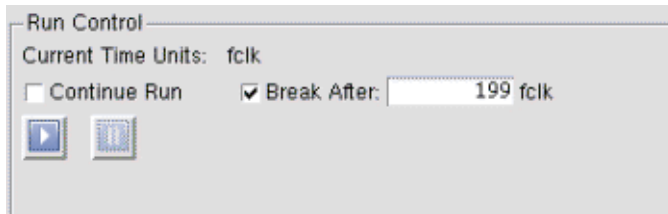
- d. In the *Trigger Position (%)* field, specify a percentage to calculate the number of samples to capture after a trigger occurs. By default, it is set to 100%.

Note: This is a toggle field that allows you to select either of the following options: *Trigger Position (%)*, *Post Trigger Percent (%)*, and *Post Trigger Samples*. Use *Post Trigger Samples* to specify a value in terms of samples. Use either

Trigger Position (%) or *Post Trigger Percent (%)* to specify a percentage of trace memory.

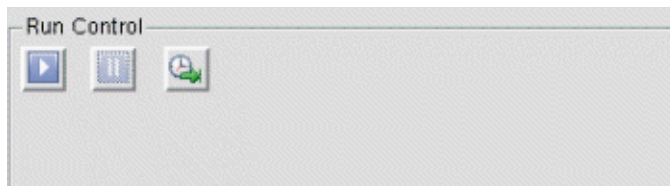
4. *Run Control* group box has different fields based on the mode in which the design is running.

- ☐ STB mode



- a. Select the *Continue Run* check box if you want to run the design continuously without resetting the SDL state from the previous run.
- b. Select the *Break After* check box if you want the run to break after the specified time elapse. The time unit is displayed adjacent to the text field.
- c. There are only two buttons to control the design – one to start the design run and other to stop it.

- ☐ LA mode



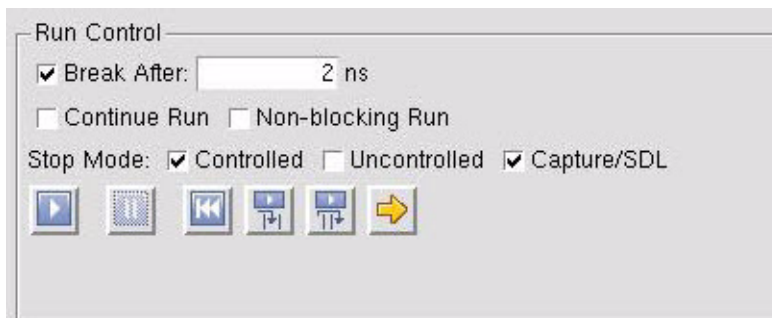
- a. The *Run Control* group box has only three buttons – start design run, stop design run, and start emulator clock.

- ☐ SA mode



Note: The *Stop Mode* field is visible only when a design has both controlled and uncontrolled clocks.

- a. Select the *Continue Run* check box if you want to run the design continuously without resetting the SDL state on execution of the `run` command.
- b. Select the *Non-blocking Run* check box if you want the design run to be non-blocking.
- c. Select the *Break After* check box if you want the run to break after the specified time elapse.
- d. Select the *Controlled*, *Uncontrolled*, or *Capture/SDL* check box adjacent to the *Stop Mode* field.



This is equivalent to issuing the following command:

```
clockConfig -stopMode  
[controlled | *controlled | uncontrolled | both | none | *none]
```

- e. The *Run Control* group box has six buttons – start design run, stop design run, reset simulation, run one statement stepping into sub-program calls, run one statement stepping over any sub-program calls, and show current execution point.
5. *Trace Data Capture* group box provides fields to select how captured trace data is to be dealt with.

- a. Specify the name of the trace file in the *Wave DB Name* field.

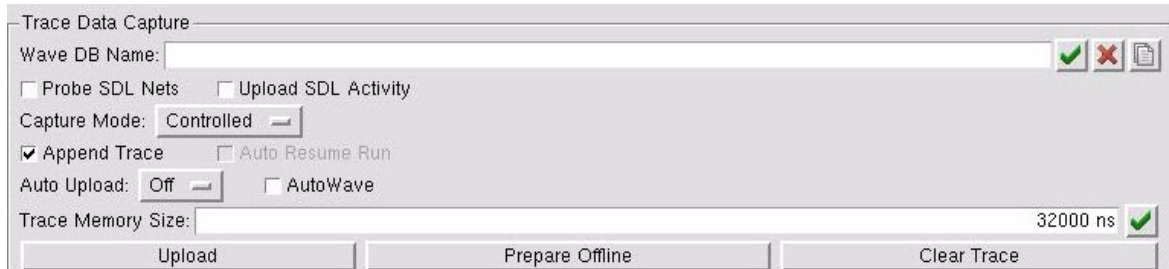
Note: In the SA mode, when the design is running on the software side, this is the only active field in the *Trace Data Capture* group box. The remaining fields discussed below are non-editable.

- b. Select the *Probe SDL Nets* check box if the SDL signals need to be probed.
- c. Select the *Upload SDL Activity* check box if you want to upload the SDL activities with the waveforms.

UXE User Guide

Using the xeDebug GUI

- d. Select *Controlled*, *Uncontrolled*, or *Controlled with Event* from the *Capture Mode* box to control the waveform display for a design with uncontrolled clocks.



This is equivalent to issuing the following command:

```
database -open <name> -captureMode [controlled | uncontrolled] [-event]
```

- e. Select the *Append Trace* check box to show trace data captured during the first run and add to it the data for the current run.
- f. Select the *Auto Resume* check box to restart the run after data capture.
- g. Select the *Auto Upload* condition to automatically upload trace data when the condition occurs.
- h. Select the *Auto Wave* check box to automatically display data in Waveform Viewer.
- i. Specify the *DPA Sparse Sampling Ratio* if you want to use sparse sampling mechanism while analyzing the power consumed by the design and click the  button. Specify the sample ratio before running the emulation so that when the trace memory is uploaded, the data saved in the *.phy file is already sparse.

For detailed information about this mechanism, refer to the [Using Sparse Sampling with DPA](#) section in Chapter 15, “Analyzing Power Consumption of the Designs.”

Note:

- This field is valid only for the ICE mode. Selecting this field is equivalent to issuing the `dpa -sparse_sampling <N>` command.
- Cadence recommends to perform sparse sampling in online session.
- If the specified ratio is not a multiple of 32, it will be automatically rounded-off to the nearest multiple of 32.
- Specifying the ratio as 0 disables sparse sampling.
- The *InfiniTrace* tab also has the *DPA Sparse Sampling Ratio* field. The field on both the tabs shows the same value. Therefore, changing the ratio on any one tab causes the change on the other tab as well.

- j. Specify the *Trace Memory Size*. You can set the value to be interpreted in terms of the specify unit or percentage.
 - k. Click *Upload* to manually upload the trace data to the Waveform Viewer.
 - l. The *Prepare Offline* button is enabled only in FV mode. Clicking this button creates a file for later use in offline session.
 - m. Click *Clear Trace* to delete the trace data from the current trace file.
6. The fields to display current status of the test run are different based on the mode in which the design is running. In ICE mode, the *Emulator Status* group box displays current status of the test run. In SA mode, the *System Status* group box displays current status of the test run.

The fields displayed in the ICE mode are listed in [Table F-2](#). The fields displayed in the SA mode are listed in [Table F-3](#).

Table F-2 Emulator Status in ICE Mode

Emulator Status	Status Indicated
Op Status	The operation status of the emulator, such as Ready, Running, or Done. Note: This status is not visible when the design is running on the software side in the SA mode.
Emulator Clk	The status of the trace clock, such as Running or Stopped. Note: This status is not visible when the design is running on the software side in the SA mode.
Current Time	The current cycle time of design run.
Current Cycle	The current cycle number.
Trigger Pos	The cycle where the trigger occurred.
Cycles Run	Number of cycles since the last stop.

UXE User Guide
Using the xeDebug GUI

Emulator Status	Status Indicated
SDL Status field	The SDL status field at the bottom of the <i>System Status</i> group box provides a limited view of the contents, including whether the instance was triggered, current state, counters, and various information about the trigger, for the instances encountered. Double click on the window or click the <i>SDL Status Viewer</i> button below the field to bring up the <i>SDL Status</i> pop-up window containing the complete status.

Table F-3 System Status in SA Mode

System Status	Status Indicated
Simulator Status	The status of the controlled clocks of the simulator, such as Running or Stopped.
Simulator Time	The cycle time of the controlled clocks of the simulator.
Uncontrolled Clocks	The status of the uncontrolled or free-running clocks, such as Running or Stopped. Note: This status is only visible when the design has uncontrolled clocks.
Uncontrolled Clock Time	The cycle time of the uncontrolled or free-running clocks. Note: This status is only visible when the design has uncontrolled clocks.
SDL/Capture Status	The status of the SDL and capture clocks, such as Running or Stopped.
Trigger Pos	The cycle where the trigger occurred.

The *SDL Status* window contains information on the full list of trigger instances, and can be expanded as necessary. The text displayed below shows a sample of output you can see in this window.

```
# SDL Instances = 3
# Triggered Instances = 2

SDL Instance: I1  <Triggered>
  Current State: A1
  Counter1: 200
```

UXE User Guide

Using the xeDebug GUI

```
Counter2: 300
SDL Filename: trigger.tdf
Trigger Line: 9
Trigger Label:
```

```
SDL Instance: I15 <Triggered>
Current State: NONE
Counter1: 0
Counter2: 0
SDL Filename: trigger.tdf
Trigger Line: 39
Trigger Label: label1
```

```
SDL Instance: I6
Current State: NONE
Counter1: 0
Counter2: 0
```

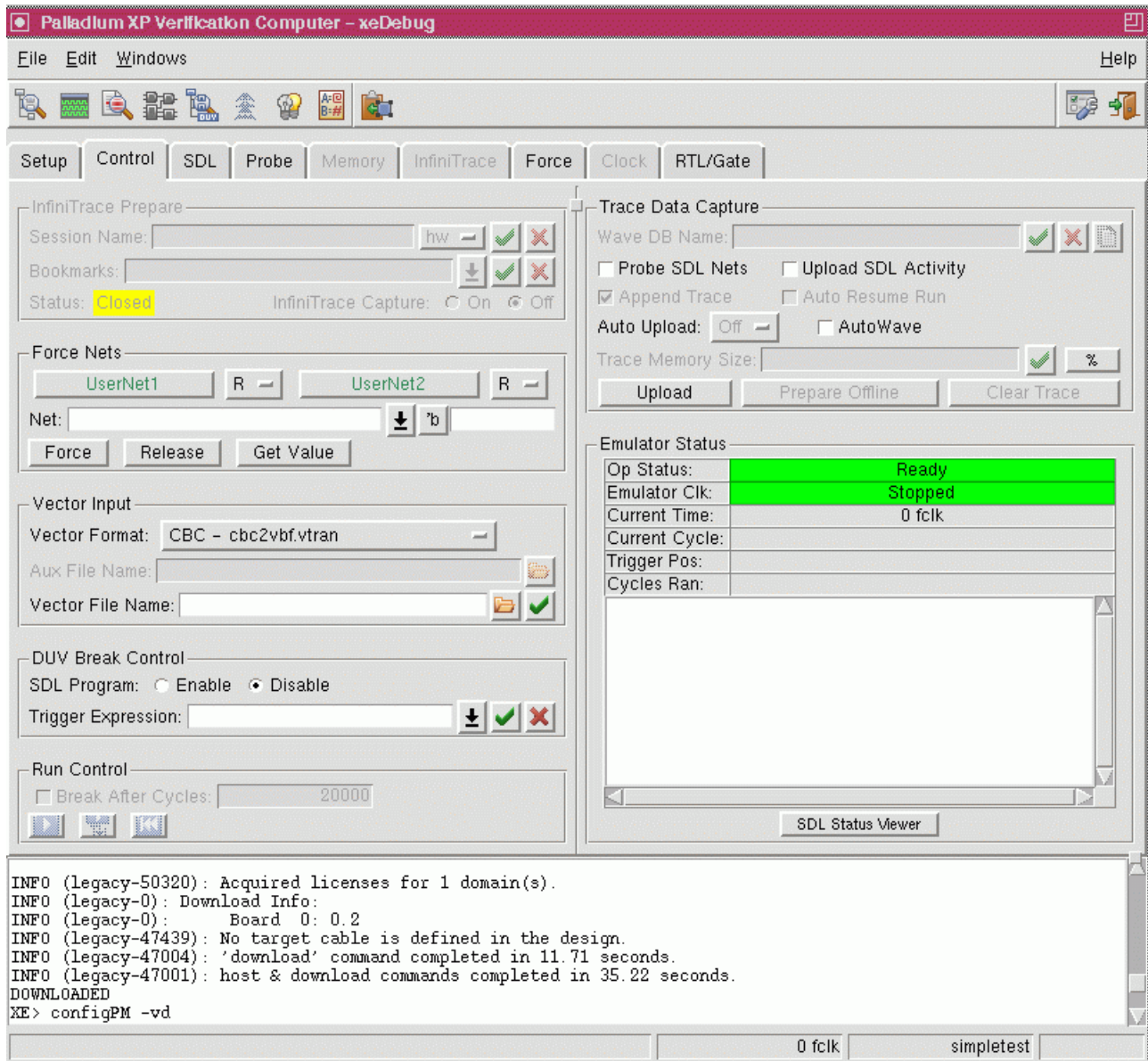
Control Tab in Vector Debug (VD) Mode

Vector Debug (VD) applies test stimulus to the design from a vector file and obtains response vectors. The vector file contains a list of vectors (one per cycle) where each vector defines the stimulus and expected results for that particular cycle. This mode is most suited for regression tests that were created from ATPG (Automatic Test Pattern Generation) software or for comparison with software simulation results. The *Control* tab for VD mode is displayed in [Figure F-35](#) on page 1221.

UXE User Guide

Using the xeDebug GUI

Figure F-35 Control Tab (VD Mode)



You can either call PTRAN as an independent executable to translate from a variety of stimulus file formats into Cadence's standard VBF (Vector Binary Format) file, which Vector Debug mode uses directly, or you can use the GUI (or the `vector` command in command-line mode) to call PTRAN to perform the translation.

Vector Debug mode cannot use a target connection.

In addition to specifying the number of vectors to run, Vector Debug mode allows you to use SDL as a breakpoint mechanism.

SDL Tab

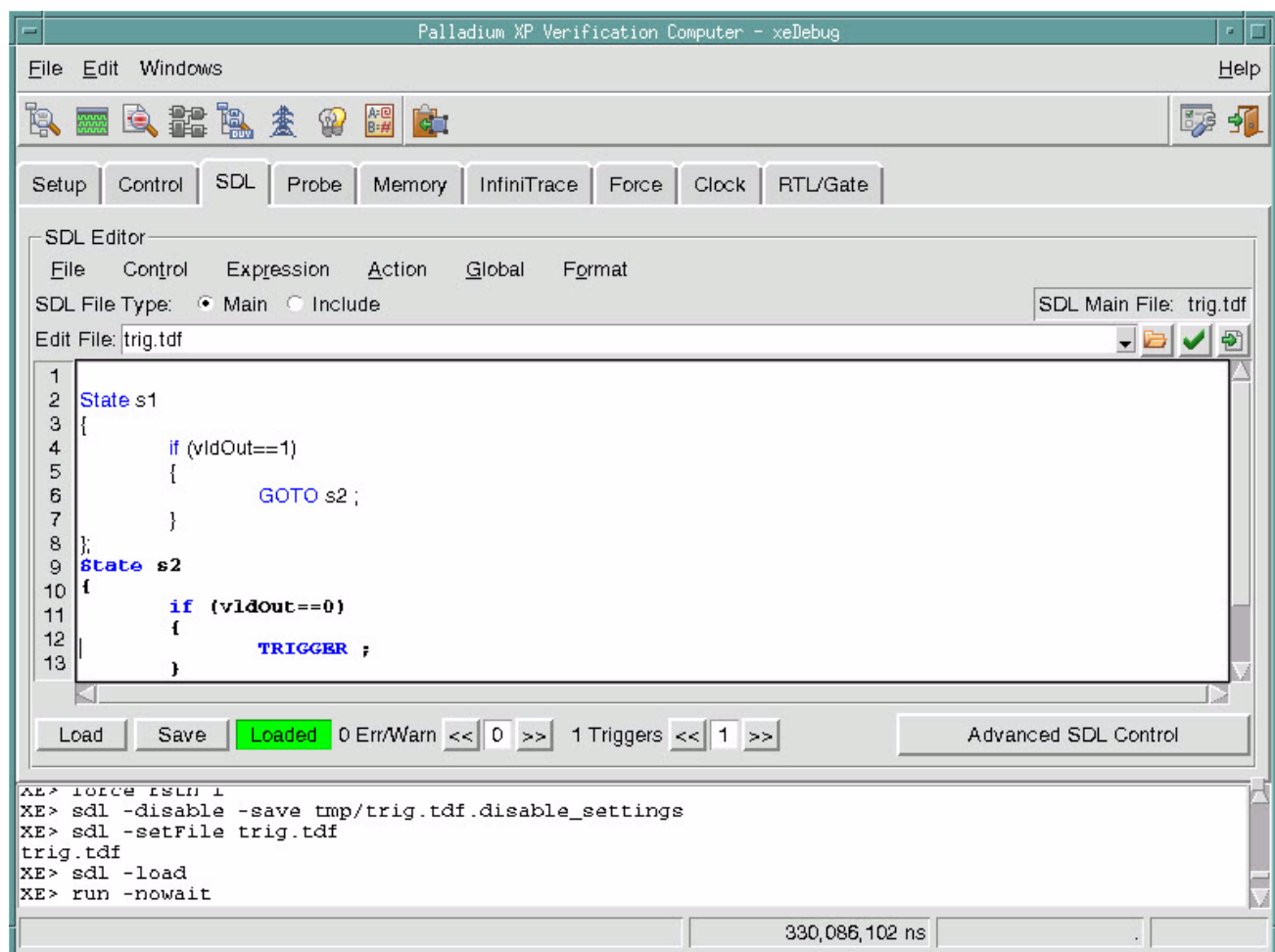
The *SDL* tab, shown in [Figure F-36](#) on page 1222, contains the *SDL Editor* group box that can be used to create and edit SDL (State Description Language) programs. The SDL programs can be saved to and restored from files by using the *Save* and *Load* buttons.

The *SDL Editor* group box provides drop-down menus, such as *File*, *Control*, and *Format*, that help you to build your SDL programs.

Note: You can use the right mouse button to drag and drop symbols/events from various forms, such as the *Symbol Editor*. Also, during the run, the current state of the loaded SDL file is highlighted.

For information on how to create SDL programs, refer to [Chapter 19, “Controlling the Run with SDL.”](#)

Figure F-36 SDL Tab



Using SDL Editor Group Box

The following table lists the drop-down menus available within the *SDL Editor* group box:

Menu Item	Function
File	Provides you the following sub-menu items: ■ <i>New</i>: To create a new SDL program ■ <i>Open</i>: To open an existing SDL program ■ <i>Undo</i>: To revert any edits done to the SDL program. ■ <i>Redo</i>: To re-apply the edits done to the SDL program. ■ <i>Save</i>: To save the new/edited SDL program ■ <i>Save as</i>: To save the SDL program as a file with extension <code>.tdf</code> or <code>.sdl</code>.
Control	Allows you to specify SDL controls by selecting the required control from the following sub-menu items: <i>Instance, Include, State, If, Else If, and Else</i>
Expression	Allows you to select various keywords and operators for creating boolean conditions, such as <code>{}</code> , <code>></code> , <code><</code> , and <code>&</code> , for use inside IF statements.
Action	Allows you to select an action from the following sub-menu items: Goto, Load, Increment, Decrement, Start, Stop, Trigger, No_Acquire, Acquire, DISPLAY, and EXEC
Global	Provides options used in creation of global definitions for TRIGGER, DISPLAY, EXEC, conditional acquisition, and initial load values or count enable expressions for the two 40 bit counters.
Format	Allows you to change the indentation of SDL statements.

Using Advanced SDL Controls

Clicking the *Advanced SDL Control* button on the *SDL* tab displays the Advanced SDL Control window as shown in [Figure F-37](#) on page 1224.

UXE User Guide

Using the xeDebug GUI

Figure F-37 Advanced SDL Control Window (SA Mode)

Advanced SDL Control

SDL Program: ☒ Enable ☐ Disable

Individual SDL Instances of Current SDL Program

SDL Instance	Enabled	Disabled
--------------	---------	----------

Enable Disable Clear Enabled Clear Disabled

Labels

Disable pattern: ☒

Instance	Label	State
----------	-------	-------

Enable Disable Enable All

SDL Probe Control

☐ Probe SDL Nets

Reset to Default SDL Trace/Upload Settings

SDL Activity - Trace Control

☐ Use conditional acquisition filtering

☐ Trace during post trigger

☐ Trace every cycle

Select which actions should cause SDL tracing (subject to the conditional acquisition filter and the posttrigger settings):

☒ Goto

☒ Load

☒ Decrement

☒ Start

☒ Stop

☒ Trigger

☐ No_Acquire/Acquire

☒ Exec

☒ Display

SDL Activity - Upload Control

Clear SDL Trace: ☒ OnRun ☐ OnUpload ☐ Never

Generate SDL Activity Trace

SDL Trace Output File

☐ Text Output ☒ No Lines ☐ Line Info ☐ Filename + Line

☒ All instances ☐ Only one instance

Generate SDL Trace Display SDL Trace

EXEC Control

Overflow: ☐ Ignore ☐ Warning ☒ Error

Display Control

Overflow: ☐ Ignore ☐ Warning ☒ Error

Console: ☒ Yes ☐ No

Output File

Halt On Trigger Control

Halt On Trigger: ☐ Yes ☒ No

This window enables you to control individual SDL instances and SDL activities like probes, trace, and upload through the GUI from the following group boxes:

- *Individual SDL Instances of Current SDL Program* to enable or disable specific SDL instances
 - *SDL Trigger Labels* to enable or disable specific trigger labels
 - *SDL Probe Control* for probing SDL nets
 - *SDL Activity - Trace Control* to identify:
 - ☐ Conditions for trace, such as *Use conditional acquisition filtering*, *Trace during post triggers*, and *Trace every cycle*
 - ☐ Actions that cause SDL tracing, such as *Load*, *Trigger*, *Exec*, *Display*
 - *SDL Activity - Upload Control* to identify when to *Clear SDL Trace*, such as *OnRun* or *OnUpload*
 - *Generate SDL Activity Trace*
 - *NCSim BreakPoints* to import NC-Sim breakpoints to SDL during swap-in
- Note:** This group box is displayed only when you are running the design in SA mode.
- *EXEC Control* to specify control conditions
 - *DISPLAY Control* to specify control conditions
 - *Halt On Trigger Control* to specify control conditions

Importing Breakpoints from the NC-Sim Simulator into SDL Using GUI

In the SA mode, to import the breakpoints from the NC-Sim simulator into SDL, perform the following steps:

1. Access the *Setup* tab after starting a debug session.
2. Click the --> *Show Hot Swap Options* button from the *Configuration* group box to view the available options (refer to [Figure F-23](#) on page 1197). You have the following two check boxes that support the corresponding run-time commands for importing the NC-Sim breakpoints:
 - ☐ *Error Ok* (for `sdl -ncbreakpoints errorok 0|1` command)
 - ☐ *Auto import breakpoints* (for `xeset sdlAutoNCbreakpoints 0|1` command)

Note: These options are also available in the *NCSim BreakPoints* group box of the

Advanced SDL Control window that is accessible from the *SDL* tab (refer to [Figure F-37](#) on page 1224).

3. When running the design on the hardware-side, click the *Import NCsim breakpoints* button from the *DUV Capture & Break Control* group box to execute the `sdl -ncbreakpoints -import` command (refer to [Figure F-34](#) on page 1214).

For detailed information on the importing of NC-Sim breakpoints, refer to the [Importing Breakpoints from the NC-Sim Simulator into SDL](#) section in [Chapter 19, “Controlling the Run with SDL.”](#)

Probe Tab

The *Probe* tab provides the ability to create a probe on either a specific signal (or assertion), a specific scope (that is, all objects under the specified design hierarchy), the input cone of a net, or the nets along the logic path between two given nets, although the last two options are available for objects emulated only on the hardware. The term **probe** refers to the collection of probed objects grouped under a single probe name. The *Upload* and *Display* buttons can be used to upload the data into the waveform database and display the probes in the specified waveform browser. If required, probe lists can be saved and imported using the *Save* and *Import* buttons.

Use the *Edit* button to modify a probe selected in the *Probes Display* group box. The probed objects grouped under the selected probe are displayed in the *Add Probes* group box. After modifying the probe, click the *Apply* button to submit the changes. If a probe name already exists, the old probe is first deleted before the new probe is added.

When you add a probe in the SA mode, and it does not appear in the waveform window, check the XEL console section for `ncsim: *E, DBOBBD: . error` associated with that probe. This error means that SimVision is unable to probe the specified object. If the following error message is generated, the specified probe will not be added and the corresponding signal will not be displayed in the waveform browser:

```
ncsim: *E,DBOBBD: cannot create SHM probe for : <signal_name>
```

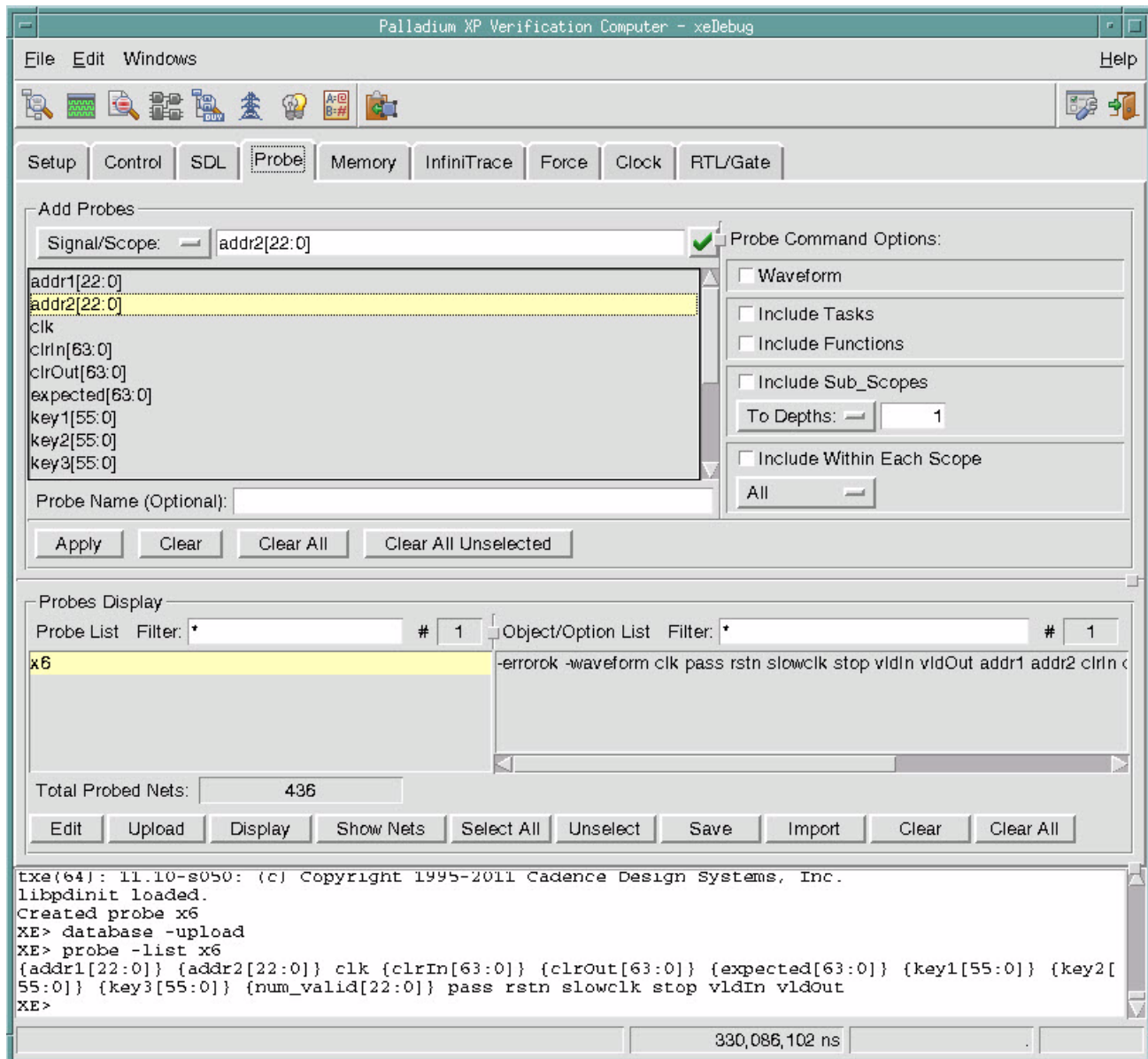
For further help regarding this error, refer to the INCISIV documentation available on the Cadence Online Support website.

If you have probed a scope, you can enable *Include sub-scopes* and choose which sub-scopes you want to probe. As you make your selections, the *Probe* tab changes the scope view to indicate your selection. For example, if you choose `all`, all the hierarchical levels below the selected scope will be probed.

UXE User Guide

Using the xeDebug GUI

Figure F-38 Probe Tab



In the SA mode, you can also add to the probe those objects that are simulated by the software , but are not nets:

Select the *Include Tasks* check box if you want to include tasks in the probe; select the *Include Functions* check box if you want to include functions. You do not need to select these check boxes if the selected scope is a task or function.

Memory Tab

The *Memory* tab, as shown in [Figure F-39](#) on page 1229, controls memory read, write, and initialization operations.

Memory Tab in SA Mode

In SA mode, the *Memory* tab ([Figure F-39](#)) enables the following functions:

1. *Memory Action Edit* group box enables you set the parameters for editing a memory
 - a. In the *Memory Location* field, select whether the edit operation should apply to *all* memories, *tb* (testbench) memories only, or *dut* memories only.
 - b. Specify the *Memory Instance* to be operated upon.
 - c. Specify in the *File* field a memory content file for a specific memory instance.
 - d. Specify a memory *Format*. This field is a listbox. Click the down-arrow on the right of the field to view and select an available format.
 - e. Specify in the *Start Address* and *To* fields the range of memories to be read. Enter hexadecimal values. If these fields are left blank, the entire memory is read. Specify auto, hexadecimal, binary, octal, or decimal forma in the *Format* field.
 - f. Select from the *Memory Action* drop-down the action that you want to be performed on the selected memories. Use the  button to commit the memory contents file to the selected memory instance.

Note: The actions listed in the *Memory Action* drop-down are the same as the radio buttons adjacent to the *Display* field of the *Memory Action Control* group box.

2. *Memory Action Control* group box allows you to set the display and executes the selected action.
 - a. The selected *Display* option determines the content of the display fields in the table: *All*, *Dump* (memories to be read), *Load* (memories to write), *Reset*, *Set*, *SetValue*, and *SetRandomValue*.
 - b. Select the *AutoView* check box to bring up the *Memory Content* window (as shown in [Figure F-40](#) on page 1230) when *Apply* is clicked. This window allows you to directly edit the memory contents, and either write them to memory or save the file for later use.

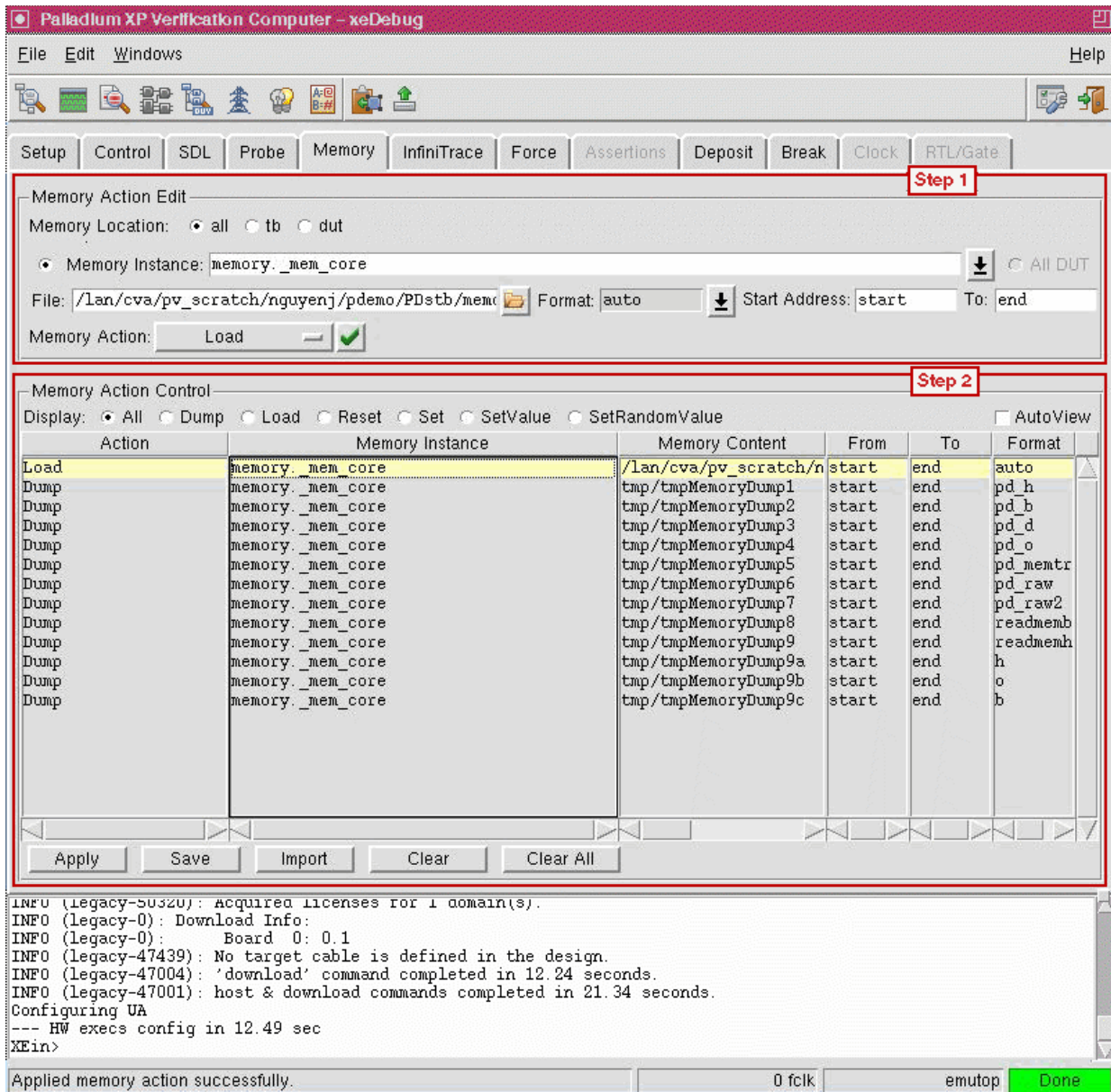
Note: *AutoView* is a FV option for dumping the memory contents list to a file.

UXE User Guide

Using the xeDebug GUI

- c. Click *Apply* to enforce the selected memory control action.

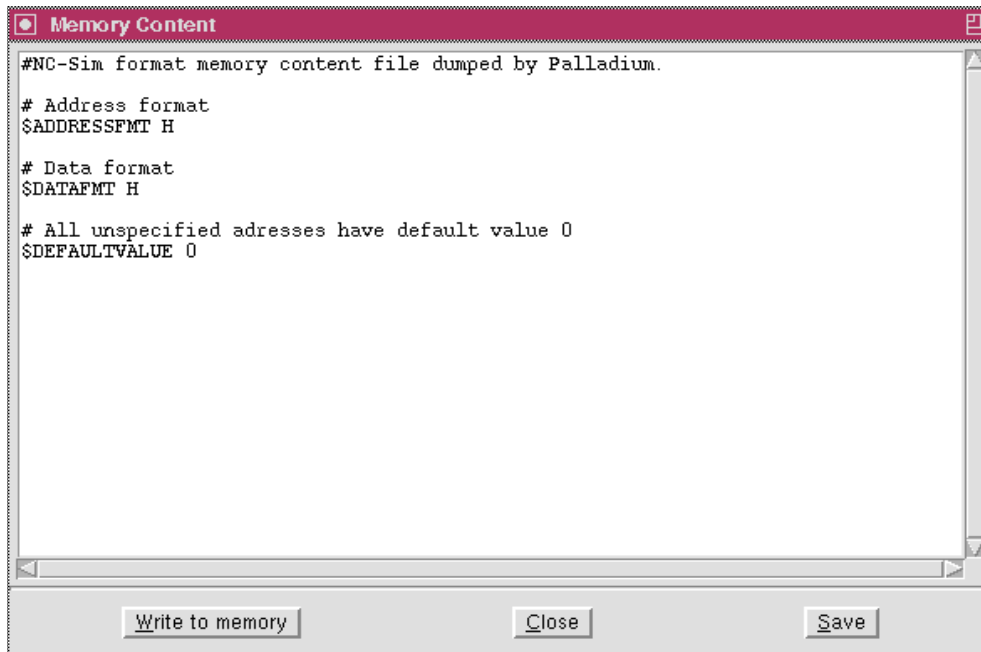
Figure F-39 Memory Tab (SA Mode)



- d. Click *Save* to bring up the dialog box that enables you to save the memory list to a file.
- e. Click *Import* to bring up the dialog box that enables you to load memory data from a previously-saved file.

- f. Click *Clear* or *Clear All* to clear either the select memory or the entire list of memories, respectively.

Figure F-40 Memory Content Window



Memory Tab in Vector Debug Mode

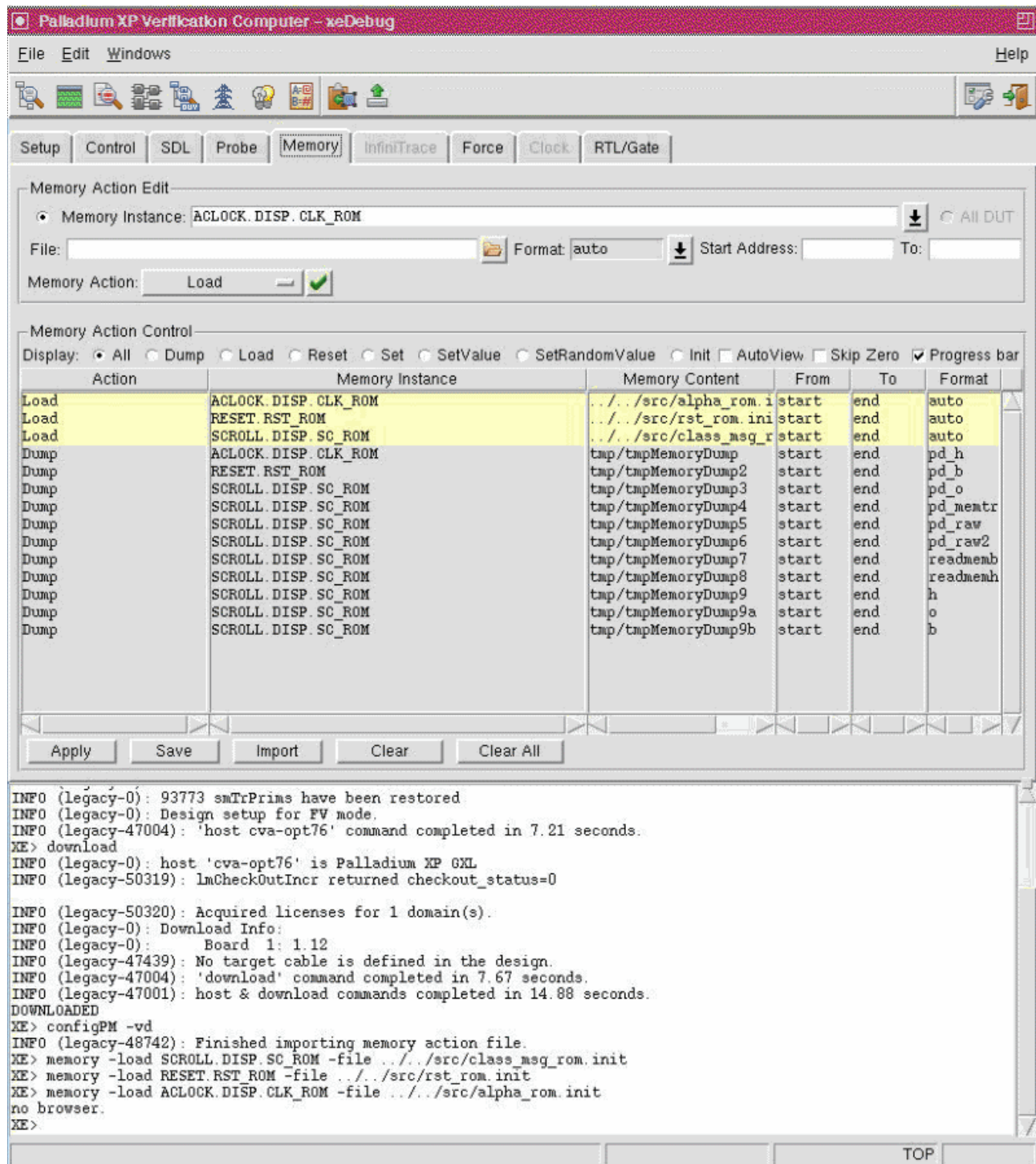
In Vector Debug mode, the *Memory* tab (Figure F-41 on page 1231) is slightly different from the one accessible in SA mode. This section lists only the differences. For details about the common fields, refer to the [Memory Tab in SA Mode](#) section above.

1. The *Memory Action Edit* group box does not have the *Memory Location* field.
2. The *Memory Action Control* group box has the following additional fields:
 - a. The *Display* field provides the following radio buttons for selection: *All*, *Dump* (memories to be read), *Load* (memories to write), *Reset*, *Set*, *SetValue*, *SetRandomValue*, and *Init*.
 - b. Select the *Skip Zero* check box to not include the zero values in the memory dump.
 - c. Select the *Progress bar* check box to disable the progress bar.

UXE User Guide

Using the xeDebug GUI

Figure F-41 Memory Tab (Vector Debug Mode)



UXE User Guide

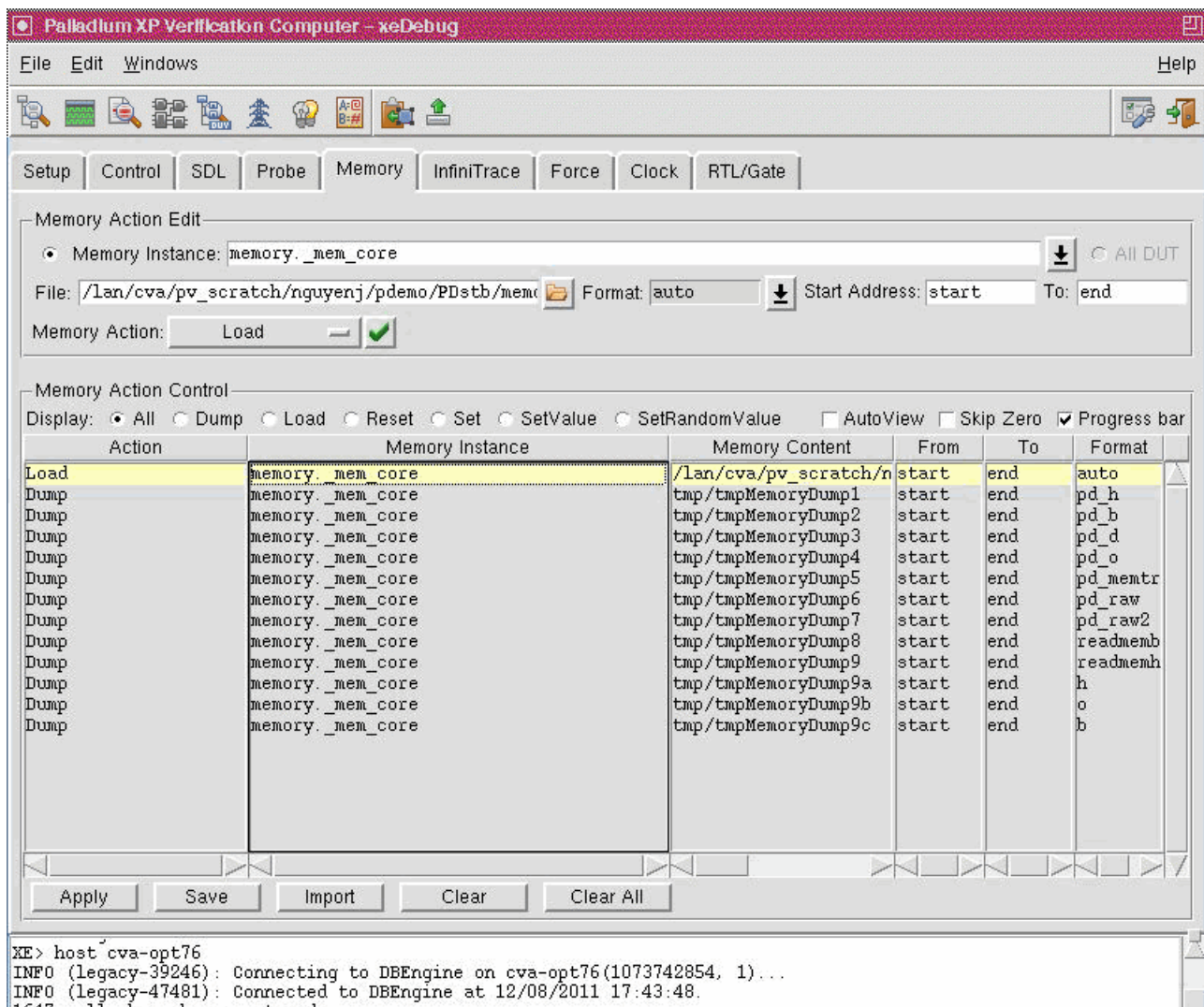
Using the xeDebug GUI

Memory Tab in STB and LA Modes

In STB and LA modes, the layout and fields of the *Memory* tab (Figure F-42 on page 1232) are similar to Vector Debug mode, except for the *Init* radio button adjacent to the *Display* field in the *Memory Action Control* group box. This radio button is not available in the STB and LA modes.

For details about the common fields, refer to the [Memory Tab in Vector Debug Mode](#) section above.

Figure F-42 Memory Tab (STB Mode)



InfiniTrace Tab

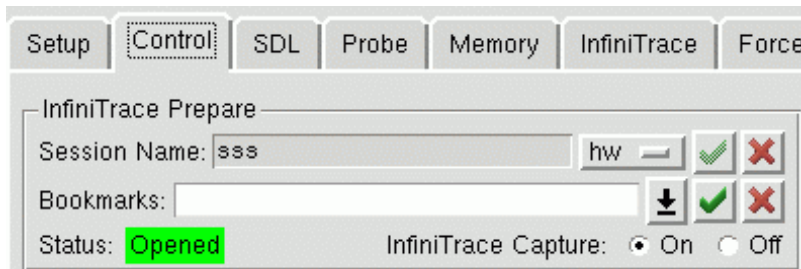
First, you need to name and prepare a session through the *Control* tab. After the session has been prepared, switch to the *InfiniTrace* tab that allows you to save and restore the emulation status for a specified time.

Prepare Session Mode

1. You must be online to prepare a session.
2. Switch to the *Control* tab. In the *InfiniTrace Prepare* group box, in [Figure F-43](#) on page 1233 select whether you need to work in SWIT or HWIT by selecting *hw* or *sw* from the drop-down list.
3. Specify a *Session Name* for your InfiniTrace prepare session. If an existing session is specified, an error is returned. All InfiniTrace data will be stored in this session name. Use the  button to open a session.

Initially, the *Status* for *InfiniTrace Prepare* group box shows as *Closed*. It changes to *Opened* after the session is opened. You cannot delete and open a session. Close the session before deleting it.

Figure F-43 Preparing InfiniTrace Session on Control Tab



4. Select the *InfiniTrace Capture: On* or *Off* option to explicitly enable or disable data capture. After the *Prepare* session is opened, data capture is enabled by default.

Note: The *On* and *Off* radio buttons are enabled only after the InfiniTrace is opened.

5. To observe the InfiniTrace data captured during the *Prepare* session, switch to the *InfiniTrace* tab.
6. Use the InfiniTrace Display functions to upload trace data to the waveform viewer.

Note: If the *InfiniTrace Capture* radio button is set to off, you will not be able to see data for that session.

Running the design is disabled while you are in an InfiniTrace *Observe* session, which is controlled through the options in the InfiniTrace tab.

Adding Bookmarks when Preparing InfiniTrace Session

The following actions can be performed while bookmarking the InfiniTrace session:

- Specify the name of the bookmark in the *Bookmarks* field within the *InfiniTrace Prepare* group box on the *Control* tab and click the  button.
- To remove a bookmark, specify its name in the *Bookmarks* field and click the button with a  cross icon.

Observe Session Mode

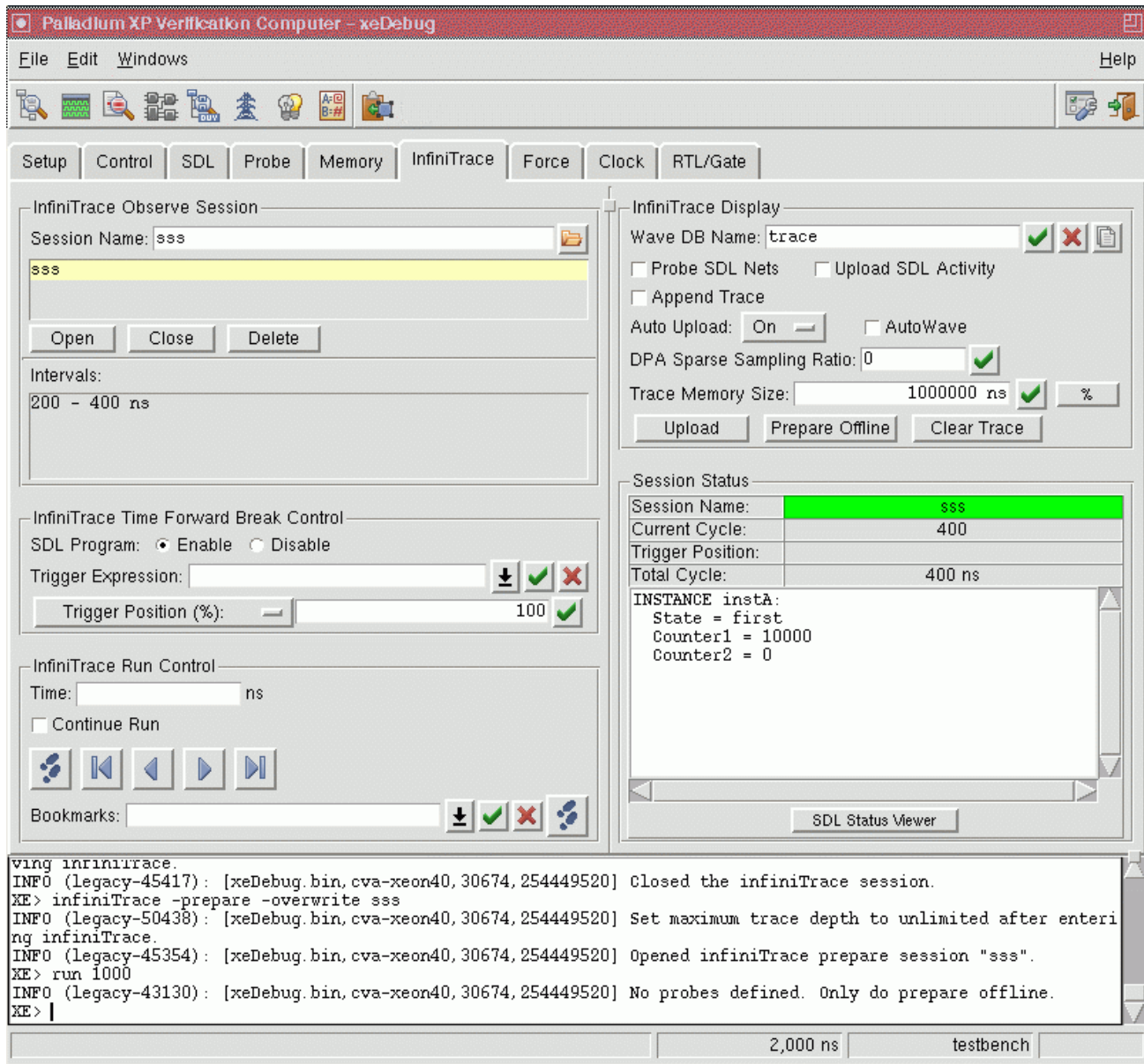
1. Click the *InfiniTrace* tab to access the InfiniTrace *Observe* session, after you have run a *Prepare* session.

Note: The *InfiniTrace* tab is disabled when you attempt to download the design to host after release of the host, or in offline session. After exiting InfiniTrace *Observe* session, you can run the design in the normal mode.

UXE User Guide

Using the xeDebug GUI

Figure F-44 Observing InfiniTrace Session



2. Specify the name of the required *Observe* session and click the *Open* button.

Note: If running in SWIT, you can open an *Observe* session when debugging in an offline session.

3. Go to a specified time in the *Observe* session.
4. Use the *Session Status* group box to observe the time and trigger data.

5. In the *InfiniTrace Run Control* group box, you can go to an existing bookmark by specifying the name in the *Bookmarks* field, or by selecting a bookmark from the drop-down list if you had added bookmarks while preparing for the InfiniTrace session.

Force Tab

The *Force* tab allows you to force, set, and get values for individual signals or nets. These values are used in design runs to control signal values.

Set holds the specified state for the specified symbol or net for one clock cycle only. This value is overwritten during the design run.

Force sets the state of symbol or net to the given binary value until explicitly released by a signal *Release* command.

In the SA mode, the *Force* tab illustrated in [Figure F-45](#) on page 1237 is displayed. Whereas, in the ICE mode, the *Force* tab illustrated in [Figure F-46](#) on page 1238 is displayed.

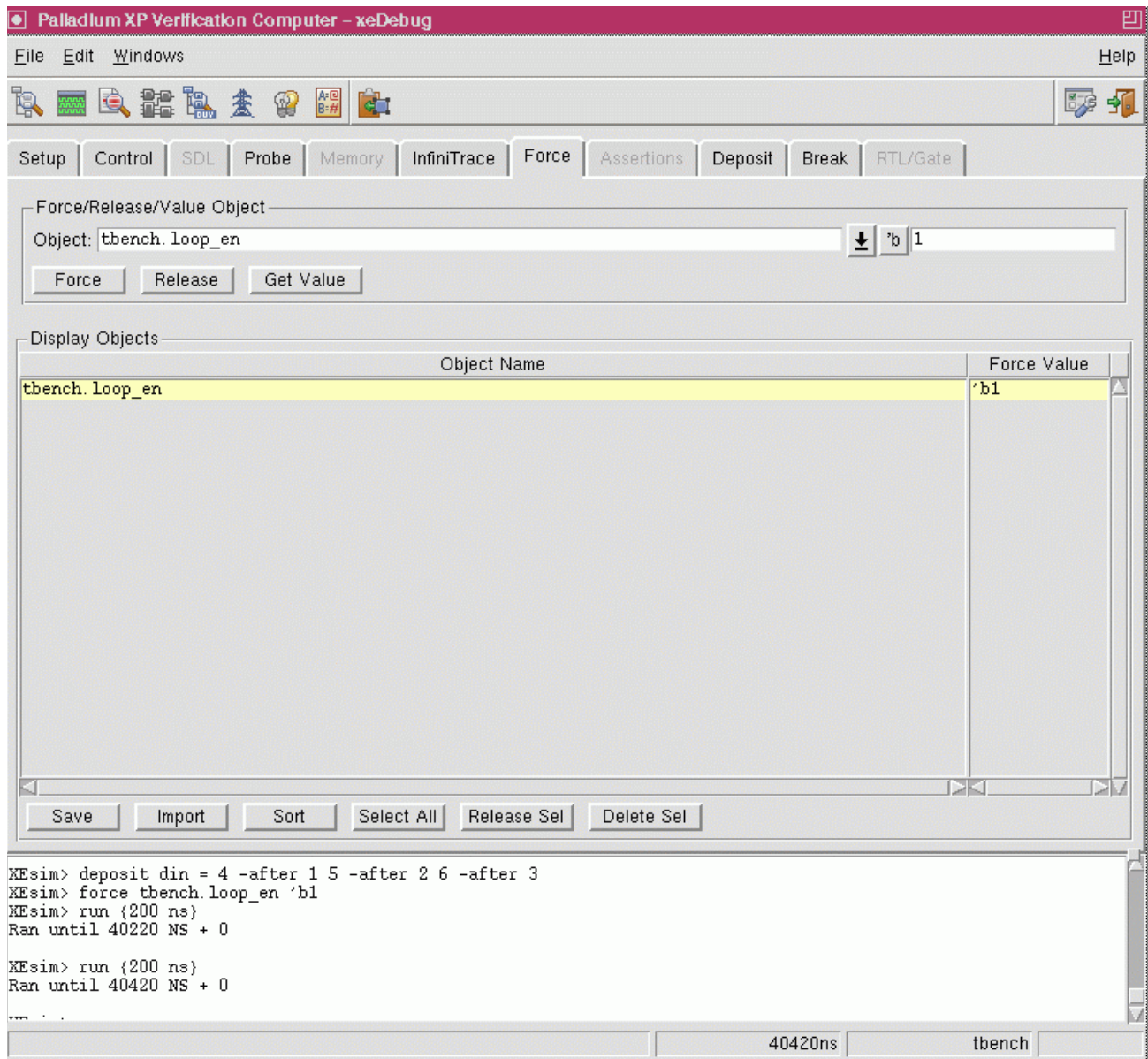
The *Force* tab in both the SA and ICE modes has the following buttons:

- a. Click *Save* to bring up the dialog box that enables you to save the forced values of the listed objects to a file.
- b. Click *Import* to bring up the dialog box that enables you to load forced data from a previously-saved file.
- c. Click *Sort* to arrange the displayed data in a specific order.
- d. Click *Select All* to select all forced values displayed in the *Display Objects* group box.
- e. Click *Release Sel* to release the forced value of the selected object.
- f. Click *Delete Sel* to delete the selected object.

UXE User Guide

Using the xeDebug GUI

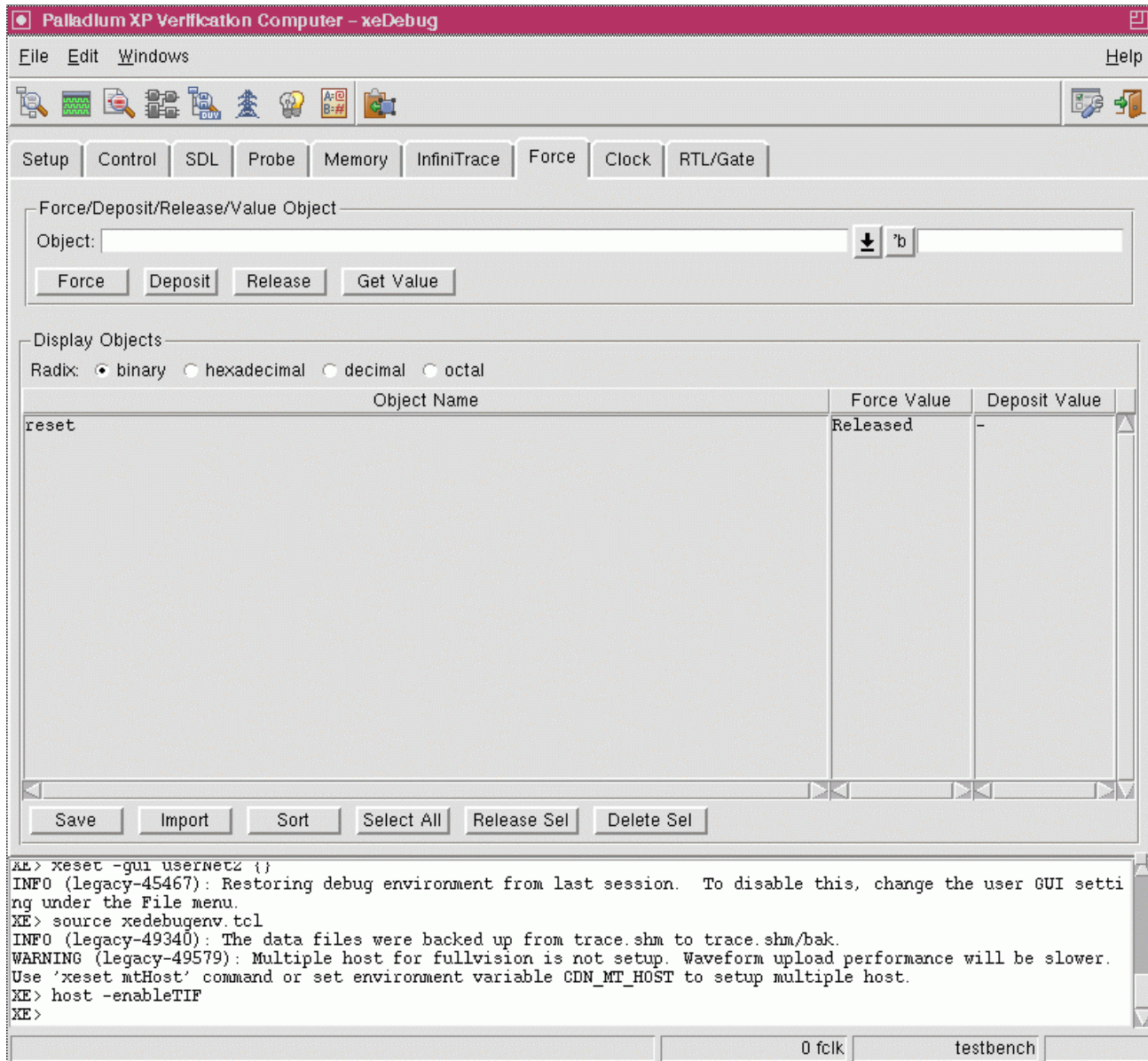
Figure F-45 Force Tab (SA Mode)



UXE User Guide

Using the xeDebug GUI

Figure F-46 Force Tab (ICE Mode)



RTL/Gate Debug Tab

The *RTL/Gate* tab allows you to debug Verilog and VHDL-based RTL files and gate-level netlists. It can be used in conjunction with the RTL Design Viewer for working with RTL files, or the Schematic Viewer for working with gate-level netlists.

This tab controls the import of gate-level netlists, or RTL files. Imported files can then be displayed in the Schematic Viewer or RTL Viewer.

Note: The *RTL/Gate* tab is enabled in the SA mode only when FSDB database is selected, and not with SST2 database.

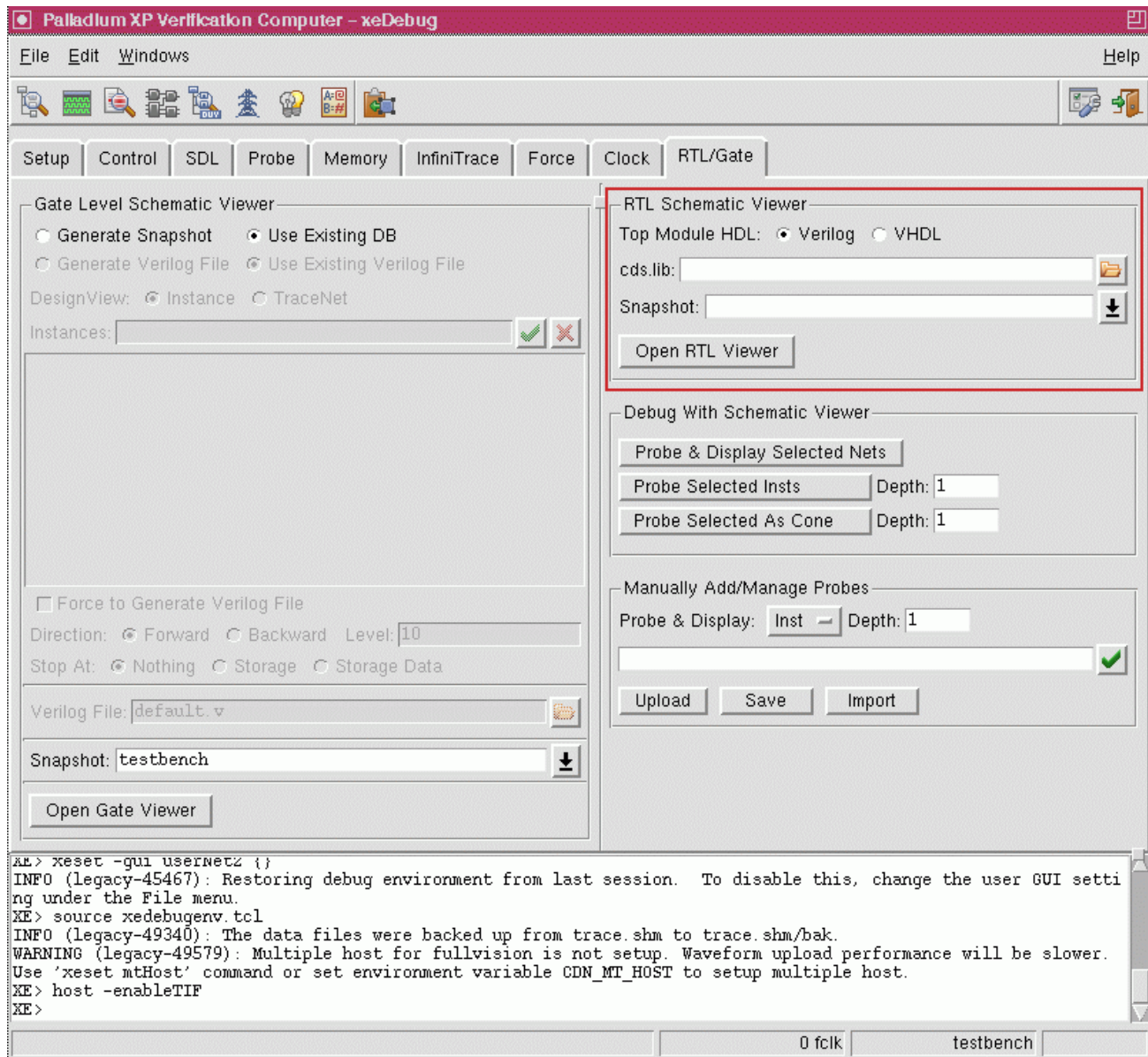
When you select the *Use Existing DB* radio button in the *Gate Level Schematic Viewer* group box, the *RTL Schematic Viewer* group box is activated as shown in [Figure F-47](#) on page 1240. This group box allows you to specify the *cds.lib* file and name of the *Snapshot* in the designated fields. Clicking the *Open RTL Viewer* button enables you to view the RTL source code or schematic display of the specified snapshot in the RTL Viewer.

The *RTL Schematic Viewer* group box is useful for RTL-level debug only. The schematic viewer supports mixed Verilog/VHDL languages. To view a mixed language module, the Verilog and VHDL databases must be separately generated. The design is then viewed by providing the schematic database and top design name.

UXE User Guide

Using the xeDebug GUI

Figure F-47 RTL/Gate Tab with RTL-Level Schematic Group Box Enabled

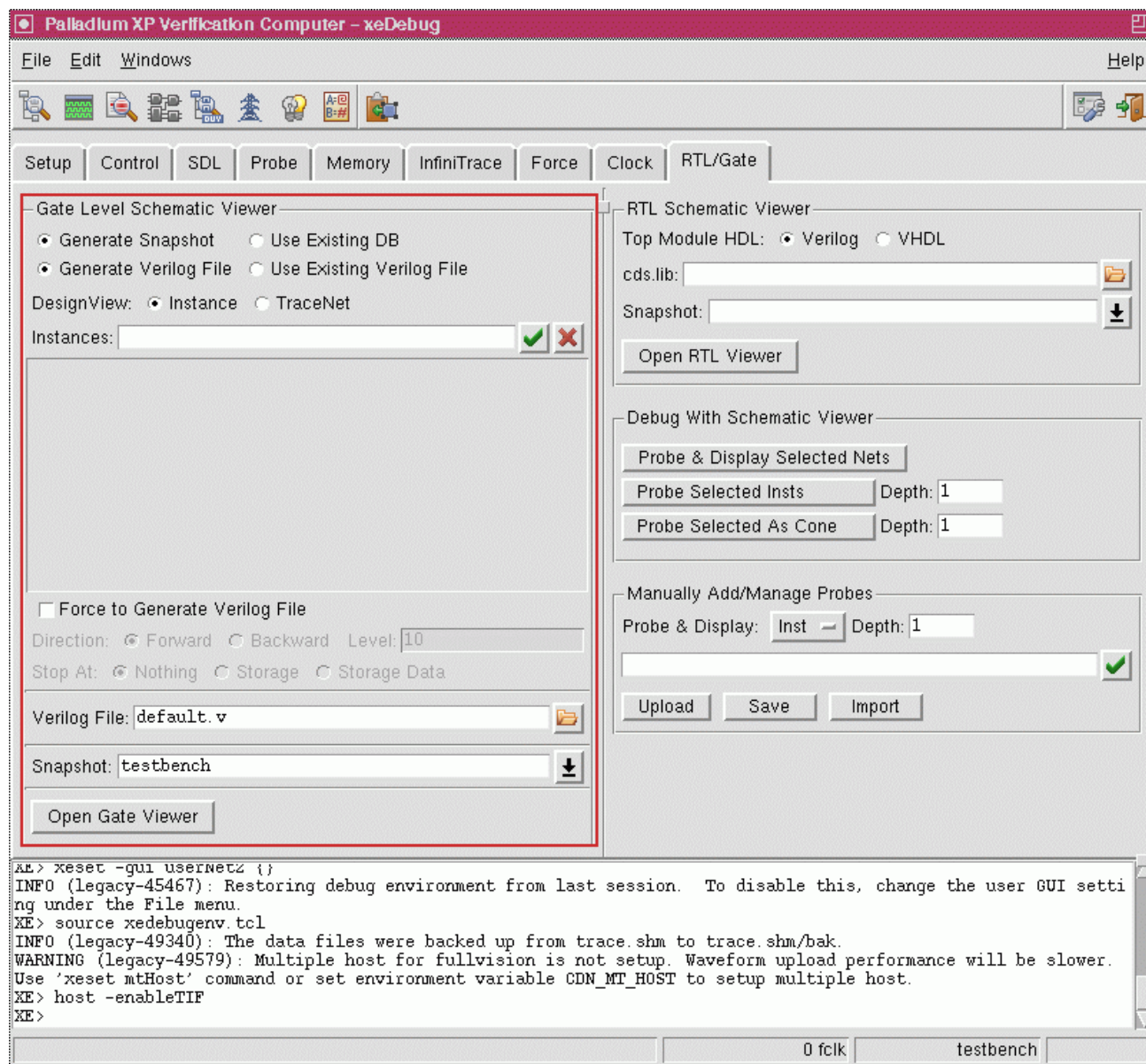


When you select the *Generate Snapshot* radio button in the *Gate Level Schematic Viewer* group box, the remaining fields in the group box get enabled as shown in [Figure F-48](#) on page 1241.

UXE User Guide

Using the xeDebug GUI

Figure F-48 RTL/Gate Tab with Gate-Level Schematic Group Box Enabled



Deposit Tab

The *Deposit* tab is available only in the SA mode. You can use the options available on this tab (Figure F-49 on page 1243) to deposit values for individual signals or nets. This tab also enables you to deposit values for testbench objects. When using the xeDebug GUI in ICE mode, the deposit operation is accessible through the *Force* tab illustrated in Figure F-46 on page 1238.

UXE User Guide

Using the xeDebug GUI

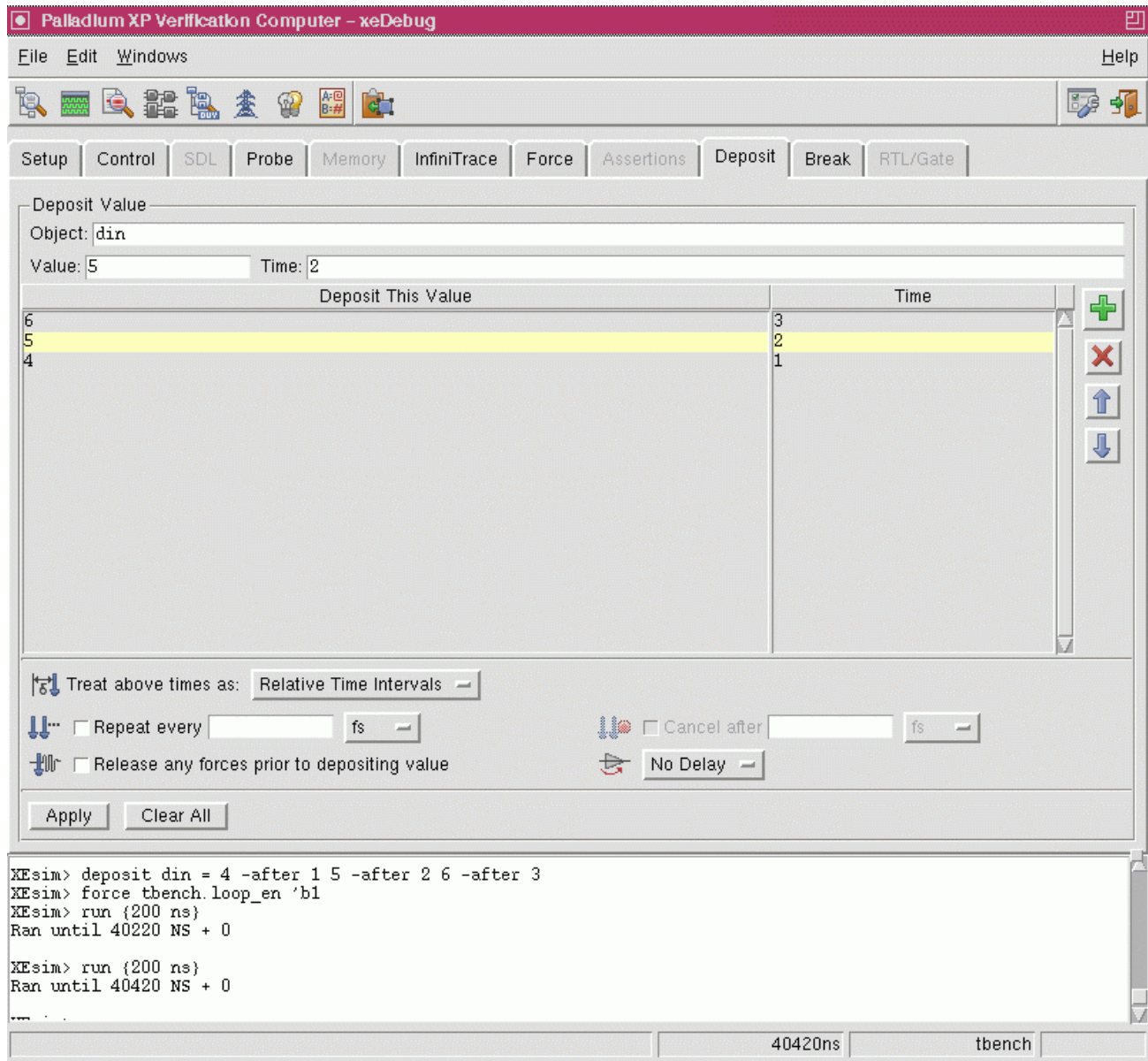
Using the options on the *Deposit* tab, you can perform the following tasks:

- Choose whether the deposit should occur at relative time intervals or at absolute time points.
- If you want to deposit values at regular time intervals, enable the *Repeat every* check box and specify the time interval.
- If you want to stop depositing values after a specified time, enable the *Cancel after* check box and specify the time interval.
- Enable the *Release any forces prior to depositing values* check box if you want to release any forces before depositing the value.
- Choose the type of delay that you want to occur before depositing the value—transport delay, inertial delay, or no delay.

UXE User Guide

Using the xeDebug GUI

Figure F-49 Deposit Tab (SA Mode Only)



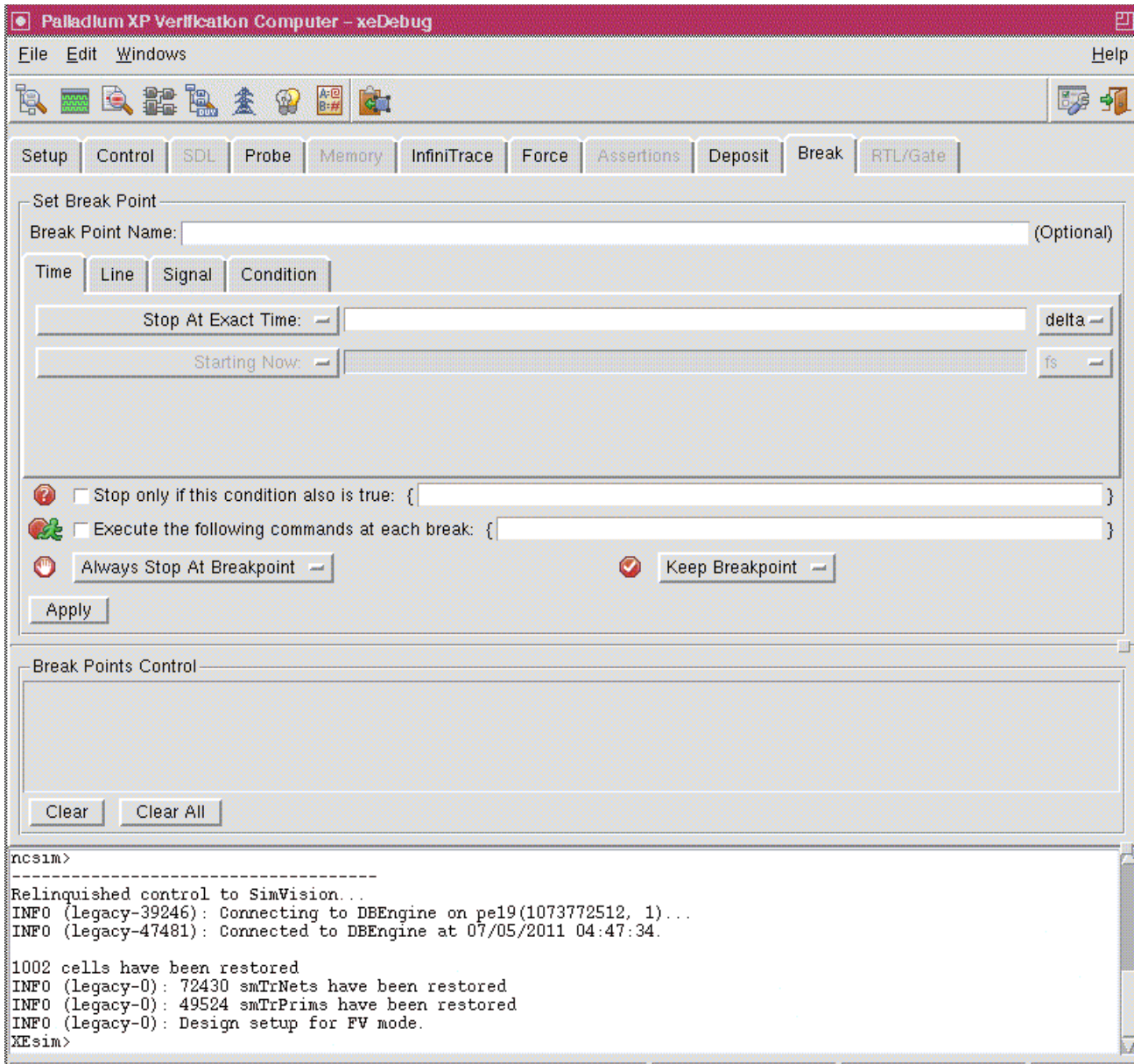
Break Tab

The *Break* tab is available only in the SA mode. You can use the options available on this tab (Figure F-50 on page 1244) to define and control break points.

UXE User Guide

Using the xeDebug GUI

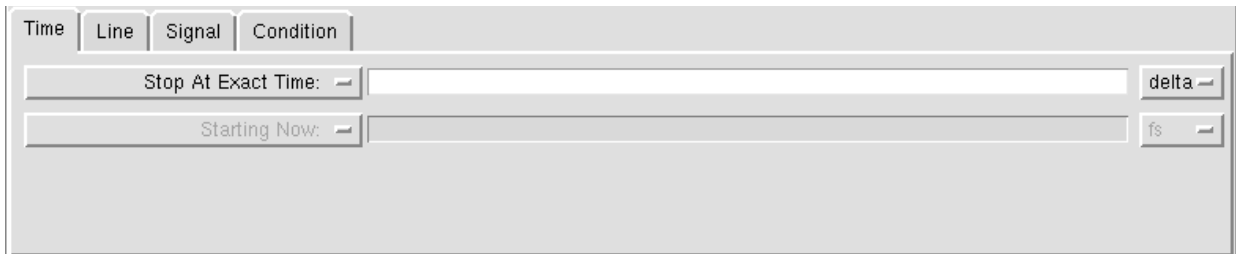
Figure F-50 Break Tab (SA Mode Only)



1. *Set Break Point* group box allows you to define parameters for break points.
 - a. Specify a *Break Point Name*. This is an Optional field.
 - b. There are the following four tabs to define the different parameters for the break point:
 - On the *Time* tab, you can specify a *Stop At Exact Time* or *Stop At Regular Intervals Of* value. You can also specify the unit of time using the drop-down

button adjacent to the first field on the tab. After you have specified this value, the *Starting Now* drop-down list also gets enabled. Clicking this drop-down will display to you two more options – *Starting At Time* and *Starting Now and Including Time*. Specify the required value and select the unit of time using the drop-down list adjacent to the second field.

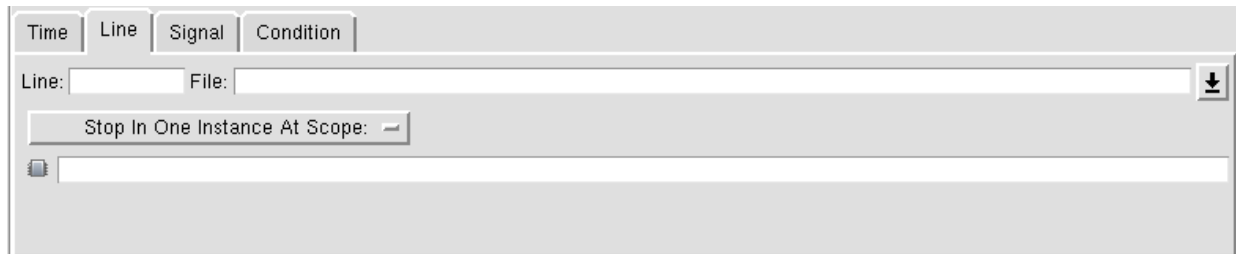
Figure F-51 Break Tab with Time Tab



The screenshot shows the 'Time' tab selected in the Break Tab. The tab bar at the top has 'Time', 'Line', 'Signal', and 'Condition'. The main area contains two rows of input fields. The first row has a label 'Stop At Exact Time:' followed by a text input field and a unit dropdown menu showing 'delta'. The second row has a label 'Starting Now:' followed by a text input field and a unit dropdown menu showing 'fs'.

- On the *Line* tab, you can specify a *Line* number and *File* name. In addition, you can select whether to *Stop In One Instance At Scope* or *Stop In All Instance Of Module/Unit*.

Figure F-52 Break Tab with Line Tab



The screenshot shows the 'Line' tab selected in the Break Tab. The tab bar at the top has 'Time', 'Line', 'Signal', and 'Condition'. The main area contains a 'Line:' label followed by a text input field, a 'File:' label followed by a text input field, and a dropdown arrow button. Below this is a label 'Stop In One Instance At Scope:' followed by a dropdown menu. At the bottom is a large empty text area.

- On the *Signal* tab, you can specify the *Signal* name on which the break point should occur.

Figure F-53 Break Tab with Signal Tab



The screenshot shows the 'Signal' tab selected in the Break Tab. The tab bar at the top has 'Time', 'Line', 'Signal', and 'Condition'. The main area contains a 'Signal:' label followed by a text input field. Below this is a large empty text area.

- On the *Condition* tab, you can specify the *Condition* on which the breakpoint should occur.

Figure F-54 Break Tab with Condition Tab



- c. Select the *Stop only if this condition also is true* check box if you have an additional condition. Use the field adjacent to this check box to specify the condition.
 - d. Select the *Execute the following commands at each break* check box if you have such commands. Use the field adjacent to this check box to specify the commands.
 - e. Select any one of the following conditions using the drop-down list:
 - Always Stop At Breakpoint
 - Skip The First
 - Continue Through Breakpoint
 - f. Select any one of the following conditions using the drop-down list:
 - Keep Breakpoint
 - Delete Breakpoint After
2. *Break Points Control* group box allows you to define parameters for break points.
- a. Click *Clear* to delete the existing breakpoint that has been selected.
 - a. Click *Clear All* to delete all the existing breakpoint that has been selected.

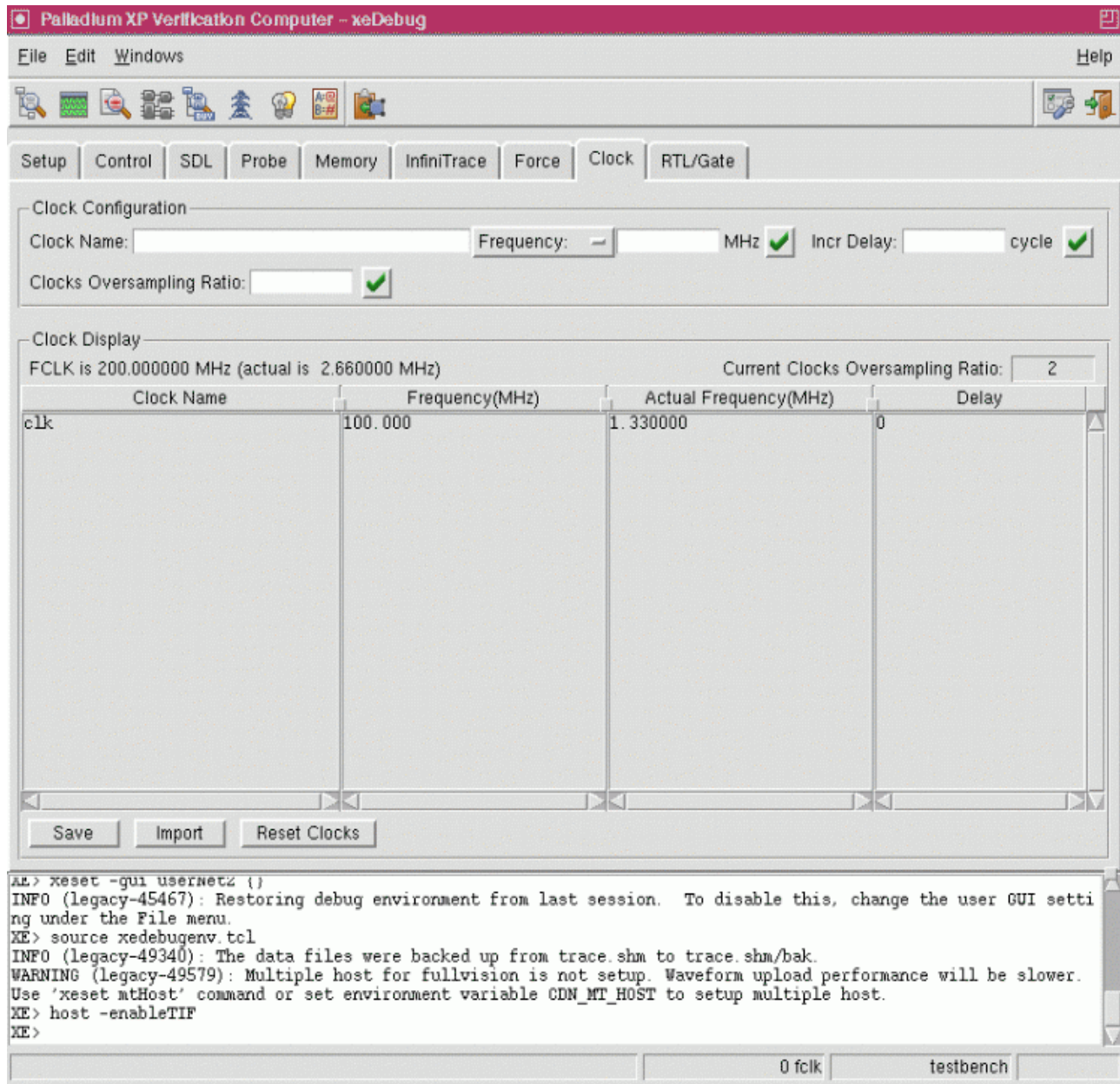
Clock Tab

The *Clock* tab (as shown in [Figure F-55](#) on page 1247) is available only in the ICE mode. It enables you to change the frequency or delay for a specified clock, or to change the oversampling ratio. The initial values result from the values entered into the fields of the *Clock* tab at compile time. By using *Control* tab, you can substitute the new values at run time.

UXE User Guide

Using the xeDebug GUI

Figure F-55 Clock Tab (ICE Mode Only)



1. *Clock Configuration* group box initially takes the values from the clocks specified at compile time. By entering new values, you can change the clock settings at run time, without having to recompile.
 - a. In the Clock Name field specify the name to be assigned to the clock signal.
 - b. Specify the clock *Frequency* in MHz and click the  button to apply entered value.

2. *Clock Display* group box shows the frequency and delay values of the current clocks in the design, as well as the *Current Clocks Oversampling Ratio*.

The *Actual Frequency* column displays the design frequencies based on the maximum emulator operating speed reported in the `xe.msg` log file.

3. Click *Save* to save the values you specified at run time.
4. Click *Import* to bring up the *Import* dialog box that allows you to import clock settings from a previously-saved file.

Saving/Restoring Debug Settings

Most of the tabs in the xeDebug GUI allow you to save and reload their content. This feature enables you to keep multiple versions of the settings that can be reloaded while debugging.

The following tabs/windows allow you to save and reload data:

- Symbols and Values
- Probe
- Clock
- Force
- SDL
- RTL/Gate
- Memory Initialization
- Memory Write
- Memory Read
- Memory Contents

Using Expressions in the Filter Field

The filter field on any tab/window allows use of glob- or regexp-type expressions to show data matching the specified filter string. The selection of glob- and regexp-type expressions is made based on the *Filter* tab of the *Preference* window.

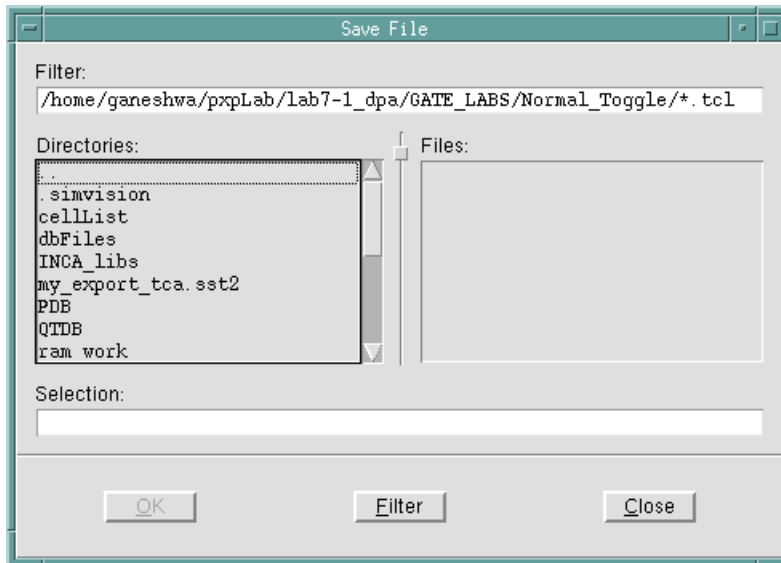
When using glob matching, a wildcard (*) selects all related data.

When using regexp matching, period wildcard (.) selects all related data.

Saving Data Entered on xeDebug GUI Tabs

Clicking the *Save* button on a particular tab displays the *Save File* dialog box illustrated in [Figure F-56](#).

Figure F-56 Save File Dialog Box



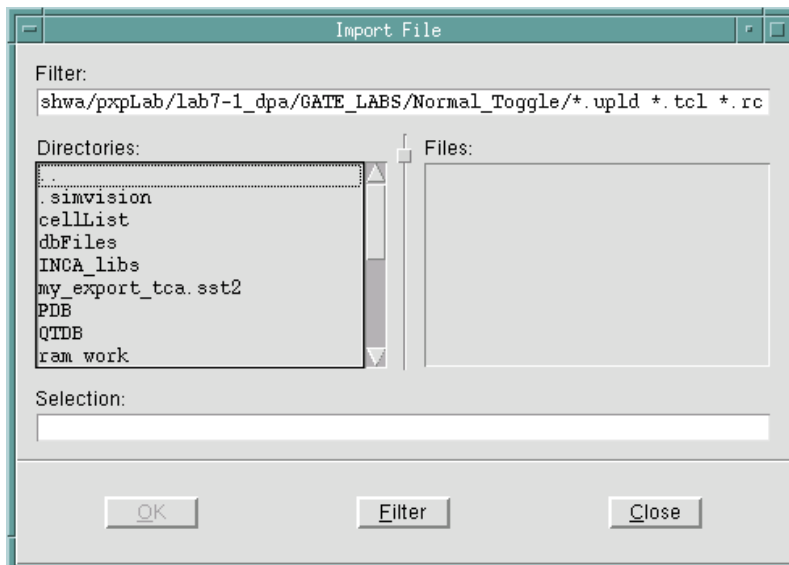
Use the fields and buttons on this dialog box to save the data into a file. When saving, use the recommended suffixes listed in [Table F-4](#) to easily identify the file types.

Table F-4 Recommended Filename Suffixes

Form	File Type	Recommend Filename Suffix
Symbols and Values	XEL Script	.sym
Probe	XEL Script	.upld
Clock	XEL Script	.clk
RTL	XEL Script	.rtlf
Trigger State Editor	SDL program	.tdf
Emulation Control	Vector file	depends on vector format: .in, .cbc
Memory Initialization	Memory list file	.mem
Memory Write	Memory list file	.mem
Memory Read	Memory list file	.mem
Memory Contents	Memory list file	.cont

The saved data can be restored or reloaded later by clicking either the *Import* or *Load* button on the tab. Clicking either of these buttons displays the *Import File* dialog box illustrated in [Figure F-57](#).

Figure F-57 Restoring Form Data Example



1. (Optional) Enter a regular expression in the Filter field.
2. Click *Filter*.

[Table F-4](#) lists the recommended filename suffixes to use in regular expressions. By filtering based on the filename suffix, find the file you want in the *Files* list.

For example, the regular expression in [Figure F-57](#) is `*.mem`. After specifying the relevant expression, perform the following steps:

1. Select file to load.
2. (Optional) Enter a filename in the *Selection* field.
3. Click *OK*.

Customizing the xeDebug GUI

The xeDebug GUI allows you to add customized elements, such as widgets, menu items, and tabs, and personalize the interface to meet your requirements. These customizations are portable across Palladium XP releases; therefore, your customizations remain unaltered even after tool upgrade.

The GUI customization made by you is maintained in predefined spaces because access to the Tcl and Tk environment of the xeDebug tool is restricted. Your code can access only the main Tcl interpreter.

The GUI customization can be made using either of the following methods:

- Using xegui XEL Command
- Loading libtk.so Tk Library (for Advanced Users)

To automatically load the customization when xeDebug GUI is launched at run time, you can create a script file named `.xegui.rc` that can contain the `xegui` commands, or commands to load the `libtk.so` Tk library with native Tk commands. The `.xegui.rc` file can be placed at any of the following locations for automatic upload:

- Palladium installation directory
- Home directory
- Design directory

Using xegui XEL Command

The `xegui` XEL command enables you to manage the following aspects of xeDebug GUI customization:

- Creating the Custom GUI Elements
- Configuring the Custom GUI Elements
- Saving the Custom GUI Elements
- Invoking the Widget Commands
- Destroying the Custom GUI Elements

Creating the Custom GUI Elements

You can create the following types of GUI elements:

- Widgets, such as buttons, check boxes, entry fields, frames (that is, group boxes), images, labels, list boxes, radio buttons, scales, spin boxes, and scroll bars.

For a complete list of the supported types of widgets, refer to the description of the `xegui -create` command in the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

- Menu items (Refer to [Creating Custom Menu](#))
- Tabs (Refer to [Creating Custom Tab](#))

Use the following `xegui` command and its options to create GUI elements customized according to your requirements:

```
xegui -create <name> [-type <type>] [-location <location>] [-pack <options>] \  
      [<Tk widget_options>]
```

For more information on the `xegui` command and its options, refer to the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

The `xegui` command also accepts abbreviated commands, like `-cre` for `-create`, and `-com` for `-command`.

For example:

```
xegui -cre bl -com "run"
```

The default location of each GUI element is at the top and the default type is button.

The Tcl interpreter where the commands associated with the widgets are executed is different from the main Tcl interpreter. The `xegui` command translates the `-variable`, `-textvariable`, and `-command` Tk widget options to hide the difference, but arguments escaped by curly brackets, `{ }`, are substituted in another Tcl interpreter.

If the first character of a command is `@`, the command is executed in the console without echoing.

For example:

```
xegui -create btn -command {@ puts "no echo in console"}
```

Creating Custom Menu

You can create custom menu items of four types – command, separator, check box, and radio button.

UXE User Guide

Using the xeDebug GUI

Creation of custom menu items as illustrated in [Figure F-58](#) on page 1255 require you to specify the name of the menu with the `-create` option of the `xegui` command.

To create a top-level menu that is displayed on the menu bar, you also need to specify the `-type` option as `menu` with the `-create` option. The following example creates a top-level menu, *Custom*:

```
xegui -create Custom -type menu
```

If you want to create a submenu for a top-level menu item, ensure that the name of the menu contains the menu hierarchy labels separated by the greater than (>) sign. The `xegui` command automatically creates all hierarchy levels till the last. For example, use the following command to create a pull-down menu *Custom*, with submenus *Simulation* and *Start*:

```
xegui -create "Custom>Simulation>Start" -command {run}
```

Note: Creating top-level menu items is useful at the time of saving and destroying custom menu. To save or destroy an entire menu hierarchy, you just need to specify the name of the top-level menu item.

The check box and radio button menu items are recognized by the value of the `-type` option, which can be followed by the `-command` option used to associate an action with the menu item. The commands and separators do not require specification of the `-type` option. In other words, this translates the requirements for creating the supported types of menu items as following:

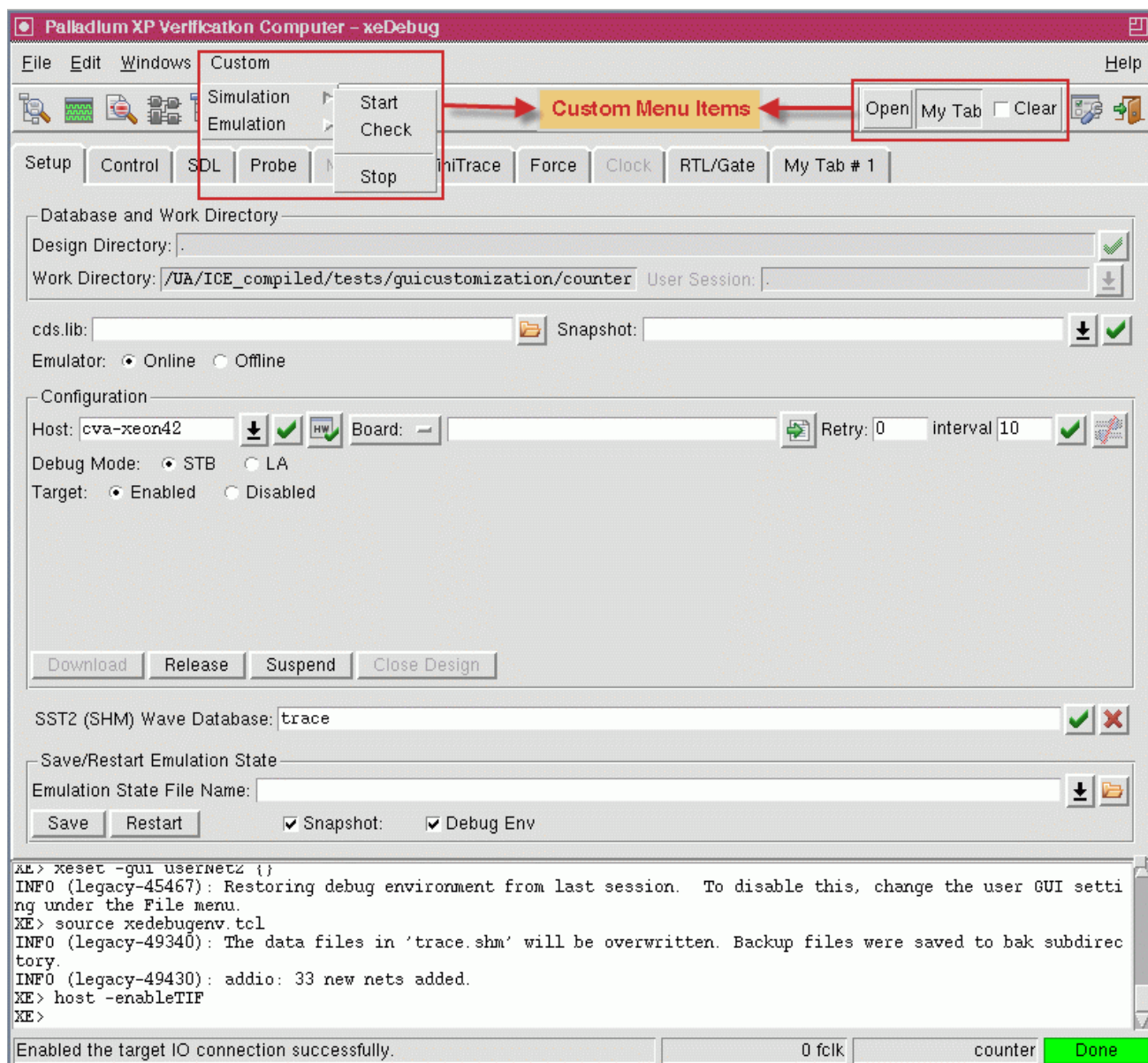
Type of Menu Item	Defined By
Command	Specifying the <code>-command</code> option with the <code>-create</code> option. For example: <pre>xegui -create "Custom>Simulation>Start" \ -command "run" -var mycheck</pre>
Separator	Specifying only the <code>-create</code> option with the hierarchy labels separated by the greater than (>) sign. For example: <pre>xegui -create Custom>Simulation>s1</pre>
Check box	Specifying the <code>-type</code> option as <code>checkboxbutton</code> that can be followed by the <code>-command</code> option. For example: <pre>xegui -create Custom>Simulation>Start -type checkboxbutton -command "deposit"</pre>

UXE User Guide

Using the xeDebug GUI

Type of Menu Item	Defined By
Radio button	Specifying the <code>-type</code> option as <code>radiobutton</code> that can be followed by the <code>-command</code> option. For example: <pre>xegui -create Custom>Simulation>Start -type radiobutton -command "force"</pre>

Figure F-58 Creating Custom Menu



Creating Custom Tab

To add a custom tab, with the `xegui -create <tab_name>` command, specify the value of the `-type` option as `tab`.

Define all nested widgets that need to be placed on the tab using the `-pack` option with the `-in` sub-option. In addition, note that the nested widgets must refer to the tab by its name.

For example, the following set of command creates a tab named *My Tab # 1*:

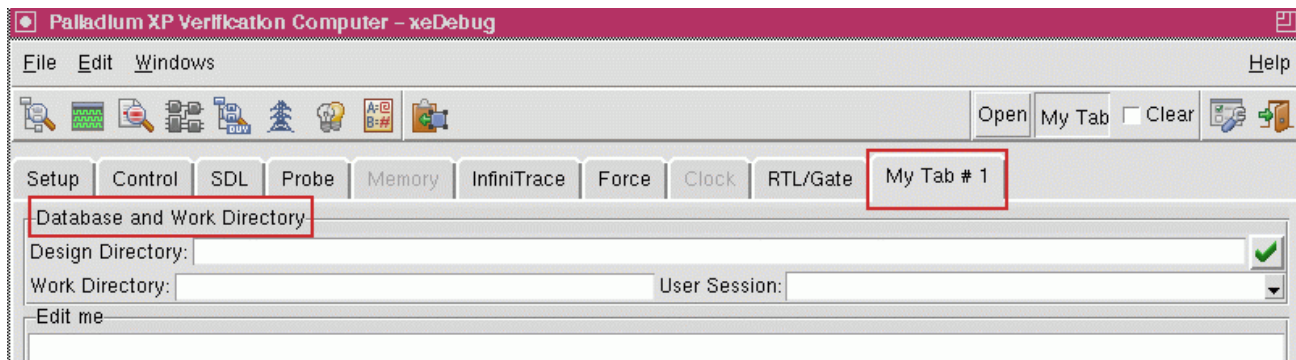
```
xegui -create t1 -type tab -label "My Tab # 1"
```

Then, to add a group box labeled *Database and Work Directory* on *My Tab # 1*, use the following command:

```
set topf [xegui -create t1.top -type ttk::labelframe \  
    -text "Database and Work Directory" -pack "-in t1 -fill x -padx 6"]
```

Refer to [Figure F-59](#) on page 1256 to see the tab and nested widget that gets created from the above commands.

Figure F-59 Creating a Custom Tab



Adding Vertical and Horizontal Scroll Bars

To add a vertical and horizontal scroll bar to the widget (as shown in [Figure F-60](#) on page 1257), with the `xegui -create <name>` command, specify the value of the `-type` option as `scrollbar` and use the `-apply` option as illustrated in the following example:

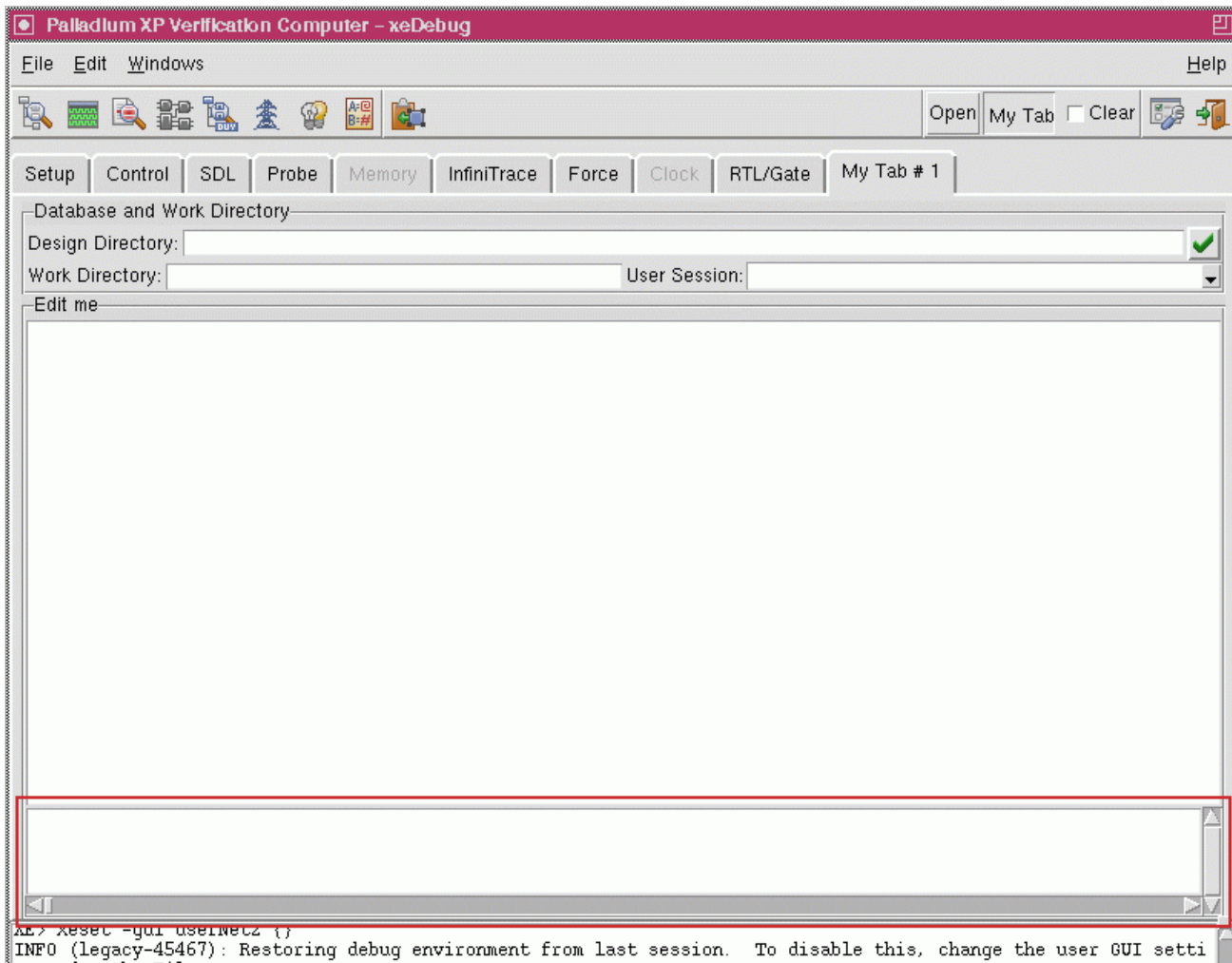
```
xegui -create tabl -type tab -label {My Tab}  
xegui -create frame1 -type ttk::labelframe -text {Edit} -pack {-in tabl \  
    -fill both -padx 6}  
xegui -create scroll1 -type scrollbar -orient vertical -pack {-in frame1 \  
    -side right -fill y} -command {xegui -apply txt1 yview}  
xegui -create scroll2 -type scrollbar -orient horizontal -pack {-in frame1 \  
    -side bottom -fill x}
```

UXE User Guide

Using the xeDebug GUI

```
-side bottom -fill x} -command {xegui -apply txt1 xview}  
xegui -create txt1 -type text -height 30 -pack {-in frame1 -fill both -side top} \  
-yscrollcommand {xegui -apply scroll1 set}
```

Figure F-60 Adding Horizontal and Vertical Scrollbars



Configuring the Custom GUI Elements

Use the following command to modify the options of an existing custom widget:

```
xegui -configure <name> <options>
```

All Tk options of the associated widget are supported.

For example, to configure a submenu named *Run* to run the design for 99 cycles when clicked, use the following commands:

UXE User Guide

Using the xeDebug GUI

```
xegui -config "My Menu>Run" -command "run 99"  
xegui -config "My Menu>Run" -state disabled
```

Saving the Custom GUI Elements

You can also save the current state of a custom GUI element using the following command:

```
xegui -save <filename>
```

Cadence recommends that you use the `xegui` command in interactive mode, then save the result, and later use it with minor modifications as the `.xegui.rc` automatic initialization script.

Note: When you specify the `<filename>` as an empty string, such as `xegui -save ""`, the command returns a Tcl script containing the custom GUI elements created in the current session.

Invoking the Widget Commands

As the xeDebug GUI interface does not allow you to access Tk widgets directly, the `xegui` command provides the following option to invoke any widget's command:

```
xegui -apply <name> <widget_command>
```

The `-apply` option does not check `<widget_command>` specified after `<name>`. It passes the entire content in the following format to the Tk's widget directly:

```
<Tk path> <widget_command>
```

For example:

```
xegui -apply reset1 invoke  
set txt [xegui -apply text1 get 0.0 end] ;# get all content from text widget
```

Destroying the Custom GUI Elements

Use the following command to destroy/delete the specified custom GUI element:

```
xegui -destroy <name>
```

For example:

```
xegui -destroy check1
```

To destroy all custom widgets, use the special name `all`.

Loading libtk.so Tk Library (for Advanced Users)

To build a GUI-rich environment, you can load the `libtk.so` Tk library directly to the XEL's Tcl interpreter either in script mode or interactive mode. Then, you can create your own Tk windows and widgets using the native Tk commands and having direct access to the XEL commands.

Note: All the Tk commands are supported. For more information on Tk widgets and commands, refer to the [Tcl8.5.9 / Tk8.5.9 Documentation](#).

Following is an example of loading the Tk library from the xeDebug command prompt:

```
load libtk.so
wm withdraw .
toplevel .mywin
```

You can also create a slave interpreter and load the Tk library there. However, for this, you will have to create aliases for all XEL commands that are required.

UXE User Guide

Using the xeDebug GUI

Modeling Clocks

This appendix provides the guidelines for modeling clocks in ICE and SA modes:

- ❑ [Modeling Clocks in ICE Mode](#)
- ❑ [Modeling Clocks in SA Mode](#)

Modeling Clocks in ICE Mode

This section provides the guidelines for modeling clocks in ICE mode.

Supported Clock Technologies

In the ICE mode, UXE supports two technologies for generating clocks within the emulator:

- **CAKE (Clock Averaging with Ko-incident Edges)**

For a given set of clock frequencies, the CAKE technology gives the best-possible emulation speed. Therefore, it is, usually, the best technology to use, and preferred by most users.

- **SCM (Simulation Control Module)**

SCM technology, usually, gives slower emulation speed than CAKE because it has more FCLKs per design clock cycle. But, the advantage of using SCM is that it preserves the same edge relationships (that is, coinciding or not coinciding edges), which the clocks of the specified frequencies would have had in simulation, if the duty cycles of all clocks were exactly 50% and the initial rising edges of all clocks coincided.

To specify the technology, use the `clockOption XEL` command:

```
clockOption -add {technology CAKE | technology SCM}
```

For details, refer to the description of the `clockOption` command in the *User Data Commands* chapter of the *UXE Command Reference Manual*.

Specifying Oversampling Ratio for CAKE Technology

When using CAKE technology, you must specify an oversampling ratio. The oversampling ratio is the number of emulator cycles (FCLK cycles) per cycle of the fastest design clock. This ratio can be set to 0.5, 1, or any even number. Oversampling ratio setting of 0.5, 1, 2, and so on is also referred to as CAKE 0.5X mode, CAKE 1X mode, and CAKE 2X mode, respectively.

The smaller the number is, the faster the design runs. However, a small oversampling ratio can have some disadvantages.

The oversampling ratio of 0.5 is good for designs in which the fastest design clock, clocks only a small fraction of the design and certain additional restrictions on clocking are met.

The oversampling ratio of 1 can result in a large capacity cost for some designs. You can try to reduce this capacity cost by providing extra information (shadowsData) to the compiler.

Some designs contain asynchronous loops. In such designs, an oversampling ratio higher than 1, such as 2 or 4 might be needed to make the design function correctly.

For more information about CAKE and SCM technologies, see the description of the `clockOption` command in the *UXE Command Reference Manual*.

Using the 1X Mode

1X mode is a Palladium XP compilation mode that allows the fastest design clock to run at the same frequency as the FCLK of the emulator. Without the use of 1X mode, the fastest design clock would be running at a speed at least two times slower than FCLK.

In most designs, compared to the 2X mode, using the 1X mode doubles the emulation speed and logic analyzer upload speed, with little or no capacity cost. However, in some of these designs, you must provide some extra information about internal clock signals to the compiler in order to avoid the capacity cost for 1X mode.

How the 1X Mode Works

Palladium XP emulators operate by periodically evaluating (or, computing a value for) each net in a design. The value of each net is computed once per cycle of an (imaginary) clock called FCLK. The FCLK frequency depends on the design.

Except for certain target I/O signals, the Palladium XP emulator evaluates each net in its input netlist only once per FCLK cycle. However, in 1X mode, the fastest design clock, and possibly other signals, must change twice per FCLK cycle.

UXE User Guide

Modeling Clocks

To make this possible, in 1X mode, the precompile step of compilation transforms the design as follows, when creating the netlist for the back-end compiler:

For any net in the design whose value can change at the falling (or “inactive”) edge of the fastest design clock, at precompile stage, two nets (called “net” and “shadow net”) are created. These nets represent the value of the net that you have defined in the high and low (or “active” and “passive”) phases of the fastest clock. The “net” and “shadow net” are evaluated in parallel, once per FCLK cycle.

The Palladium XP run-time software (xeDebug, waveform viewer, etc.) hides the existence of shadow nets. Waveforms for all nets in the design are displayed correctly, as though running in 2X mode. The waveform shows two time-points for each FCLK cycle.

The CAKE 2X mode or 2X mode (or oversampling ratio set to 2) means that there are two FCLK cycles per cycle of the fastest design clock.

For each net that has a shadow net, at precompile stage, separate logic must be created to compute the net and the shadow net. If a large fraction of the nets in a design have shadow nets, the capacity cost for 1X mode will be substantial. However, for most designs, only a small fraction of design nets have shadow nets because, typically, the values of only a few design nets change on the falling edge of the fastest design clock. As a result, the capacity cost for 1X mode is usually less than 10%, and often, less than 1%.

There are some situations in which shadow nets are created at precompile stage, even though you have not specified CAKE 1X mode. For example, if you have enabled multisampled I/O and declared at least one multisampled input signal, during precompile stage, shadow nets are created irrespective of whether or not you have specified CAKE 1X mode. Whenever shadow nets are created at precompile stage, the waveform automatically displays two time-points per FCLK; otherwise it displays one time-point per FCLK cycle.

In this appendix, generically, the “1X mode” term is used to describe a compilation in which shadow nets are created, and the “2X mode” term is used for a compilation in which the shadow nets are not created. The term “CAKE 1X mode” or “CAKE 2X mode” is used to mean that you have explicitly declared CAKE 1X or 2X mode.

By default, the sampling of design inputs connected to a target, and updates to the design outputs connected to a target, happen only once per FCLK cycle. Exceptions to this rule are the Multipulse and Pulse outputs to a target that were specified by using the `-edgeInfo` and `-pulse` options of the `terminalTiming` command, respectively. However, if you enable the multisampled I/O feature, inputs from a target can be sampled, and outputs updated, twice per FCLK cycle. When this feature is enabled, the compiler divides the FCLK cycle into two parts. For a multisampled input, the physical input from the target is sampled once during each of the two parts of the FCLK cycle, and the sampled values are transferred to net and

shadow respectively. For a multisampled output, the physical output to the target is updated once during each part of the FCLK cycle, using the values of net and shadow, respectively.

A special CAKE 0.5X compilation mode is available, which allows the fastest design clock to run at double the frequency of FCLK. Usually, for this mode to be useful, the highest-frequency clock of the design must clock only a small fraction of the design, and the design must use only one edge of the highest-frequency clock. CAKE 0.5X mode is described in the [Using the CAKE 0.5X Mode](#) section.

When to Use 1X Mode

1X mode can be used for most designs having the following characteristics:

- The design has multiple clock sources with different, arbitrary, frequencies.
- The design uses complex multiplexing, gating and dividing circuits to generate internal clocks from the clock sources.
- The design uses both edges of its clocks.
- Input and output signals connected to an external target change at every cycle of corresponding design clocks (including, possibly, the fastest design clock), or even twice per cycle.
- Some design clocks (or other design inputs) are driven by sources external to the emulator, asynchronous with FCLK.

When Not to Use 1X Mode

Obstacles to using 1X mode can include the following:

- For a specific design, 1X mode might result in too much capacity cost. You can reduce this cost by using the techniques described in the [1X Mode Capacity Cost](#) section.
- When a design contains combinational loops, precompile breaks the loops by inserting delays. Sometimes, the loopbreaking delay prevents a design from working correctly unless the oversampling ratio (number of FCLK cycles per design clock cycle) is increased. Such a design might not run correctly in 1X mode. The [Avoiding the Need for Oversampling](#) section in the [Appendix A, "Loop-Breaking"](#) describes techniques to avoid problems caused by loopbreaking delay.
- A *1X-type clock source* should be generated using Palladium XP's CAKE clock generation technology. (A *1X-type clock source* refers to a clock source that changes value more than once per FCLK.) If the design and target share common 1X-type clock sources, the emulator, not the target, should drive these clock sources. By using the

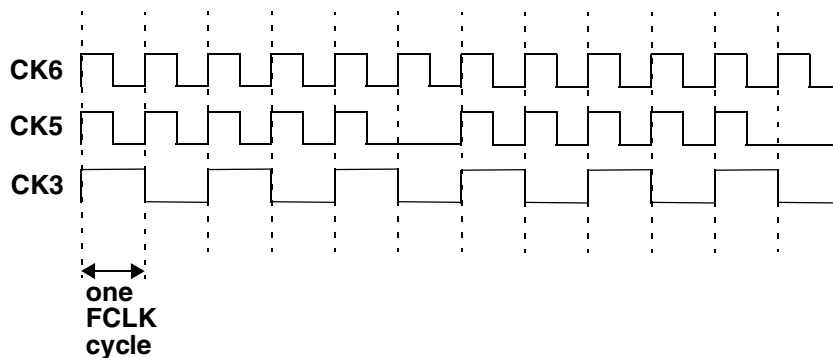
multisampled I/O feature, it is possible to have a 1X-type clock driven by a source external to the emulator, asynchronous with FCLK. However, there will be a significant capacity cost (probably double) for any logic associated with this clock source in the emulated design.

- If a design interfaces with an external target, for 1X mode, you might need to specify I/O timing constraints, which are not required in 2X mode. This is not really an obstacle; it is merely an extra task required for 1X mode. If a 1X-mode design interfaces with an external target, and the clocks associated with the target interface are at least two times slower than the fastest clock, then extra I/O timing constraints for 1X mode are usually not required. Details are provided in the [Specifying a 1X-Mode Target Interface](#) section.

When emulating a design for the first time, Cadence recommends that you use the CAKE 2X (or higher) mode initially, to enable easy bring-up of the design.

Clocks in CAKE 1X Mode

A design compiled in CAKE 1X mode can have multiple clock sources with arbitrary frequencies, driven automatically using the CAKE clock-generation technology. The following diagram shows waveforms of CAKE 1X mode clock sources named CK6, CK5, and CK3, with relative frequencies 6, 5, and 3, respectively.



A clock source like CK5 with frequency less than, but more than half of, the fastest clock frequency, has a waveform that coincides with that of the fastest clock in some FCLK cycles, and is flat low in the remaining FCLK cycles. Since every rising edge of CK5 coincides with a rising edge of CK6, the output of a flip-flop clocked by rising edge of CK5 does not need a shadow net.

A clock source like CK3 with frequency no more than half that of the fastest clock is a "2X-type" clock source. Neither CK3, nor outputs of flip-flops clocked by it, need shadow nets.

Upgrading from 2X to 1X Mode

For most designs, the steps to upgrade from 2X mode to 1X mode are as follows:

1. Change the compile script to specify CAKE 1X mode. (See [Specifying CAKE 1X Mode.](#))
2. If the design connects to an external target, you might need to change the compile script to specify a 1X-mode target interface. (See [Specifying a 1X-Mode Target Interface.](#))
3. Run and debug the design.

If the design works correctly in 2X mode but not in 1X mode, one of the following reasons is likely:

- ❑ The loopbreaking delay requires extra oversampling.
- ❑ The target interface has not been specified correctly in step 2.
- ❑ If a run script contains the command `run <n>`, and the `timeUnit` is the ICE-mode default of `fclk`, then while moving from 2X mode to 1X mode, the value of `<n>` should be decreased by a factor of 2. For example, `run 100` should be changed to `run 50`. If the `timeUnit` is something other than `fclk` (for example, `dcclk` or `ns`), the value of `<n>` should not be changed when moving from 2X mode to 1X mode.

Techniques to overcome problems caused by loopbreaking are described in [Appendix A, “Loop-Breaking.”](#) If these techniques, and review of the target interface specification you created in step 2, do not help in identifying the problem, then you should use FullVision to isolate the problem, just like you would resolve design problems.

4. Improve the compile results (capacity and step count), if necessary. (See [1X Mode Capacity Cost.](#))

Steps 1, 2, and 4 are described in more detail in the following sections. Steps 3 and 4 are usually independent, and can be performed in any order.

Specifying CAKE 1X Mode

The following XEL commands specify CAKE 1X mode for an example design with three clocks named CK6, CK5, and CK3. These commands are given before precompile.

```
clockOption -add {technology CAKE 1}  
clockOption -add {activeEdge positive}  
clockOption -add {ignoreOutput on}  
clockOption -add {ignoreEvent on}  
clockFrequency -add {CK6 6}  
clockFrequency -add {CK5 5}  
clockFrequency -add {CK3 3}
```

The `{technology CAKE 1}` option specifies CAKE 1X mode. In CAKE 1X mode, the fastest design clock has one cycle in each FCLK cycle.

The `{activeEdge positive}` option is appropriate for nearly all designs. The `{activeEdge negative}` option would be appropriate (i.e., would give a smaller 1X mode capacity cost) for an unusual design in which most design signals changed on the falling edge of the fastest clock. If you do not specify the active edge, it is, by default, taken as `positive`.

The `{ignoreOutput on}` option is used in this example, but it is only needed in certain designs. When using CAKE 1X mode, if you enable the multisampled I/O feature (See [Using the Multisampled I/O Feature](#)), then you do not need to specify the `{ignoreOutput on}` option.

If you do not enable the multisampled I/O feature, the following rules apply.

- Outputs to an external target (except multipulse and pulse outputs) can change at most once per FCLK cycle. Inputs from an external target are sampled once per FCLK cycle, and do not have shadow nets.
- If a design output (even one that is not connected to an external target) has a shadow net, precompile will fail with an error unless you specify the `{ignoreOutput on}` option. An output that is not connected to a target still causes an error because precompile does not check whether or not an output is connected to a target. If you specify the `{ignoreOutput on}` option, and a design output with a shadow net is connected to a target, the net (not the shadow net) will be used as the output to the target. The waveform of such an output still shows two samples per FCLK, showing the values of the net and shadow net.

In the example above, there are three CAKE-generated clocks CK6, CK5, and CK3, with relative frequencies 6, 5 and 3 respectively. Any design logic driving these CAKE clock signals is automatically disconnected, and the signals are driven by CAKE clock-generation logic instead.

1X-mode Terminology: Active and Passive Phases

Precompile messages in 1X mode use the terms “primary clock, “active phase” and “passive phase”.

If CAKE 1X mode is specified, the primary clock is the highest-frequency clockSource.

One cycle of the primary clock (which in 1X mode, takes place during one FCLK cycle) consists of, by definition, the active phase followed by the passive phase. With `activeEdge positive`, the active and passive phases are the high and low phases of the fastest clock, respectively. With `activeEdge negative`, the active and passive phases are the low and high

phases. If a user net can have different values in the active and passive phases, it must have a shadow.

The terms “active phase” and “passive phase” always refer to the phases of the primary clock.

Specifying a 1X-Mode Target Interface

When a design compiled in 1X mode has input or output signals (or both) connected to an external target, and some of these signals change more than once per cycle of the primary clock, the target interface is said to run in 1X mode. If the primary clock itself connects to the target, then (since the primary clock changes twice per cycle of the primary clock) the target interface runs in 1X mode. When the target interface runs in 1X mode, additional terminal timing commands are needed, that are usually not needed otherwise.

The following changes might be required:

- ❑ Specifying Clock Outputs
- ❑ Specifying `terminalTiming` constraints on inputs, to avoid output-to-input delays (Constraining Inputs)
- ❑ Specifying multisampled I/O, if needed (Using the Multisampled I/O Feature)

These changes are described in the following sections.

Specifying Clock Outputs

If the primary clock, or any other clock with frequency faster than half that of the primary clock, is an output to the target, user must implement each such clock output using one of the following methods:

- Implement the clock output as a multipulse output. Since a multipulse clock (as seen at the target) has the same waveform in every FCLK, this method can implement the primary clock but cannot implement other clocks which are flat low in some FCLK cycles.

The following command specifies `<signal>` as a multipulse output with rising edge at step 0, falling edge at step 128, and low phase duration at least 25 steps.

```
terminalTiming -add {<signal> -edgeInfo 0 25 0 128}
```

For more information, see the description of the `terminalTiming` command in the *UXE Commands Reference Manual*.

One advantage of using a multipulse clock is that the rising edge can be scheduled at step 0. Scheduling the clock output rising edge as early as possible is desirable, because it allows design inputs that change in response to the clock output to be scheduled

relatively early. This can improve overall step count in a design where a design input is on the critical path.

- Implement the clock output using the multisampled I/O feature, which is described later in this chapter. If the emulator is Palladium XP XL or GXL, and the design has other multisampled I/O signals which change on both edges of the clock output, then user should use the multisampled I/O feature to implement the clock output as well as the other multisampled signals. This allows you to reliably constrain the timing of the other signals relative to the clock output.
- A third method (which is rarely used) is to implement the clock as a (positive or negative) pulse output. A positive pulse output can implement a clock that goes high, then low, in some FCLK cycles, and is flat low in other FCLK cycles. For more information, see the description of the `terminalTiming` command in the *UXE Commands Reference Manual*.

Constraining Non-Clock Outputs

Almost always, non-clock outputs must be constrained to change at a later step within the FCLK cycle than for corresponding clock outputs. This requirement is exactly the same in 1X mode and 2X mode. For more information, see the example "Design with clock outputs" in the description of the `terminalTiming` command in the *UXE Commands Reference Manual*.

Constraining Inputs

For a target interface where the clock is generated by Palladium XP, transitions of inputs to Palladium XP are generally caused by a transition of the clock output. If you do not specify the input signals as multisampled terminals and do not specify timing constraints for them, the inputs will usually be scheduled early in the FCLK cycle. As a result, there will be a one-FCLK delay between the clock output transition and the input transition as seen by the design in Palladium XP.

For a 1X mode design in which the target interface clock frequency is more than half the design's fastest clock frequency, a one-FCLK delay from clock output to input is usually not tolerated by the design. For such a design, you must constrain the inputs and the input components of bidirectional signals to be scheduled long enough after the clock output transition to eliminate the clock-output-to-input delay. For more information, refer to the *Avoiding output-to-input delay* and *Avoiding output-to-input delay for bidirectional signals* sections in the description of `terminalTiming` command, *User Data Commands* chapter, *UXE Commands Reference Manual*.

Using the Multisampled I/O Feature

If any input or output other than the primary clock can change more than once per cycle of the fastest clock, use the multisampled I/O feature. If user enables the multisampled I/O feature, the emulator can sample inputs from an external target twice per FCLK cycle, and can change outputs to an external target twice per FCLK cycle.

In some designs, using the multisampled I/O feature could increase the step count with which the design is compiled, thus reducing the FCLK frequency. Use the feature only when it is needed.

To use the multisampled I/O feature, do the following, before the precompile step:

1. Enable the multisampled I/O feature by giving the XEL command

```
compilerOption -add {enableMultiSampledIO <mode>}
```

where *<mode>* is 1 or 2. Specifying a *<mode>* of 0 disables the feature.

The *<mode>* setting (1 or 2) determines how precompile handles design outputs, including output and enable components of bidirectional signals. If *<mode>* is 1, any output that has a shadow is multisampled. If *<mode>* is 2, only those outputs specified with `multiSampledTerminal` are multisampled; if any other outputs have shadows, user must specify the `ignoreOutput` on command, or precompile will fail.

2. For each input signal which must be multisampled, and each bidirectional signal whose input component must be multisampled, declare the signal as a multisampled terminal using the command

```
multiSampledTerminal -add {<signal>}
```

A tristate output signal is considered to be bidirectional, and the waveform shows its input component. When the `enableMultiSampledIO <mode>` setting is 1, if a bidirectional output or enable component has a shadow (and is therefore multisampled), then the input component is automatically multisampled as well. When the *<mode>* setting is 2, declaring a bidirectional signal as a `multiSampledTerminal` makes the output, enable and input components multisampled.

When the multisampled I/O feature is enabled, the compiler divides the FCLK cycle into two parts. In Palladium XP, the step number where the second part begins is decided by the compiler, so as to minimize total step count. For a multisampled input, the physical input from the target is sampled once during each of the two parts of the FCLK cycle, and the sampled values are transferred to net and shadow respectively. For a multisampled output, the physical output to the target is updated once during each part of the FCLK cycle, using the values of net and shadow, respectively. For either a multisampled input or multisampled output, one can say that the non-shadow signal is scheduled during the first part of the FCLK cycle, and the shadow is scheduled during second part of the FCLK cycle.

Non-multisampled inputs are sampled only once per FCLK cycle, even though the multisampled I/O feature is enabled. Non-multisampled outputs change only once per FCLK cycle.

The `dbFiles/<top_cell>.ctl` file provides the information about which primary I/O signals are multisampled.

If a multisampled signal is scheduled at a certain step N in the first part of the FCLK cycle, its shadow is not necessarily scheduled the same number of steps, N, after the beginning of the second part. Timing constraints apply to multisampled signals in the following way.

- Timing constraints can specify relationships between multisampled and non-multisampled signals, between different multisampled signals, or between multisampled signals and fixed step numbers.
- Timing constraints between different multisampled I/O signals, or between multisampled I/O signals and fixed step numbers, apply to the scheduling of shadows (in the second part of FCLK) as well as to the scheduling of non-shadows (in the first part of FCLK). For example, if A and B are multisampled I/O signals,

- The commands

```
delayBox -add {after_A 10 A}  
terminalTiming -add {B after_A}
```

constrain B to be scheduled at least 10 steps after A, and constrain B's shadow to be scheduled at least 10 steps after A's shadow.

- The commands

```
delayBox -add {after_begin 10}  
terminalTiming -add {B after_begin}
```

constrain B to be scheduled at least 10 steps after step 0, and constrain B's shadow to be scheduled at least 10 steps after the beginning of the second part of the FCLK cycle.

- Timing constraints between multisampled and non-multisampled signals apply to the scheduling of non-shadow signals but have no direct effect on the scheduling of shadow signals. For example, if A is a multisampled signal and C is non-multisampled,

- The following commands constrain C to be scheduled at least 10 steps after the non-shadow A is scheduled, but allow C to be scheduled at any time relative to A's shadow:

```
delayBox -add {after_A 10 A}  
terminalTiming -add {C after_A}
```

- The following commands constrain the non-shadow A to be scheduled at least 10 steps after C is scheduled:

```
delayBox -add {after C 10 A}  
terminalTiming -add {A after_C}
```

These commands have no direct effect on A's shadow (but A's shadow must be scheduled in the second part of the FCLK cycle, which starts at a step later than the latest non-shadow multisampled signal, including A, is scheduled.)

There is a minimum of one TIF (eight steps) interval between the step at which the last I/O signal is scheduled in the first part of the FCLK cycle, and the step at which the first I/O signal is scheduled in the second part. Likewise, there is a minimum of one TIF (eight steps) interval between the step at which the last I/O signal is scheduled in the second part of the FCLK cycle, and the step at which the first I/O signal is scheduled in the next FCLK cycle.

1X Mode Capacity Cost

In some designs, compiling in 1X mode immediately doubles emulation speed with little or no capacity cost compared to 2X mode.

However, in many designs, there is initially a significant capacity cost for 1X mode. The capacity cost can cause a degradation in compilation step count, so that emulation speed in 1X mode is much less than double the speed in 2X mode.

Most designs do not heavily use the falling edges of their clocks. If a design does not heavily use the falling edges of its clocks but still has a significant capacity cost for 1X mode, a likely reason is that most signals do not change value on falling edge of the fastest clock, but precompile is not able to prove it, so precompile must give these signals shadows. In such a design, user might be able to reduce the 1X mode capacity cost by providing precompile extra information about internal clocks, enabling precompile to infer that outputs of storage elements clocked by these clocks do not need shadows.

The process of reducing 1X mode capacity cost is usually an iterative one involving the following steps:

1. Choose, at random within the dumpShadows report, a storage element "E" whose output has a shadow.
2. Provide precompile additional information to eliminate the shadow on E. Usually this information will eliminate shadows on other storage elements as well. Recompile the design, and verify that storage element E no longer appears in the dumpShadows report.
3. If the capacity cost has been reduced to an acceptable level, stop. Otherwise, repeat the process, starting at step 1.

UXE User Guide

Modeling Clocks

Details of this process are given in the following sections. Do not give up if the first few iterations make little progress. Sometimes the first few iterations make little progress, and then suddenly an iteration makes excellent progress.

Analyzing 1X Mode Capacity Cost

In the file `tmp/<LIB>.<TOP>.MultiClock`, precompile reports both the gate count for the design, and the number of shadow nets in a 1X mode compile:

```
Size of design in 2-input gates: 4635760
INFO (MultiClock#38396): [host,22369] Created 184636 shadow nets.
```

Usually, the percent capacity cost for 1X mode is roughly three times the number of shadow nets divided by the number of gates, expressed as a percent.

You can identify the design nets that have shadows, and the reasons for this from the `dumpShadows` report. Run precompile with the `-dumpShadows` option (along with any other options that is normally used). Precompile will create a `dumpShadows` report at `./tmp/precompile.dumpShadows`.

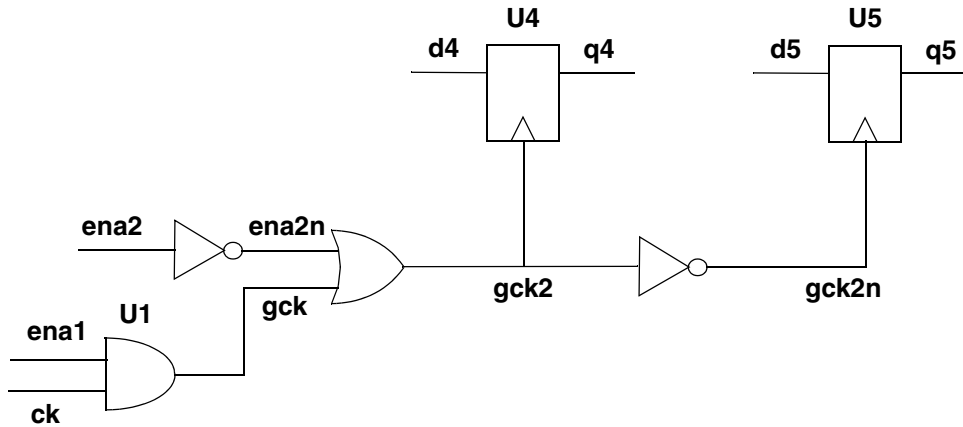
For each net that has a shadow, the `dumpShadows` report shows the gate (or storage element) driving the net, along with the nets connected to the pins of this gate. CAKE clock sources are an exception: no driving gate is shown for a CAKE clock source. Buffers and inverters are not shown in the report; inversion on an input or output pin is shown by a "*" character. Common gate and storage element types include:

FDRSE	D flip-flop
LDRSE	D latch
A2	two-input AND
O2	two-input OR
XO2	two-input XOR
MX2	two-input MUX

UXE User Guide

Modeling Clocks

For example, for the following circuit, where ck is the fastest CAKE clock source:



Here is the dumpShadows report:

```
A2 U1
<- O gck (sh) p=gnd
-> I1 ck (sh) a=pwr p=gnd
-> I0 ena1
---
O2 U2
<- O gck2 (sh) (pos)
->*I1 ena2
-> I0 gck (sh) p=gnd
---
FDRSE U5
<- Q q5 (sh) (both)
-> D d5
->*CK gck2 (sh) (pos)
-> R
-> S
---
```

In the dumpShadows report, any net that has a shadow has the label "(sh)", and also has exactly one of the following five labels:

p=gnd or p=pwr

The net is always low (or always high, respectively) during the passive phase.

A net can also be labeled a=pwr or a=gnd if it has a known value during the active phase. However, active-phase labels are not important in analyzing shadows.

UXE User Guide

Modeling Clocks

(pos) or (neg)	The net is "positive" or "negative". "Positive" or "negative" means this net will never have a rising (or falling, respectively) edge at the transition from active to passive phase. A net with p=gnd (or p=pwr) is always positive (or negative, respectively) but the (pos) or (neg) label is not shown.
(both)	The net is "both". "Both" means during precompile stage, it is not possible to determine if the net is positive or negative.

A power or ground net appearing in the report has label `pwr` or `gnd`, respectively.

The output of a Q_FDP0 flip-flop clocked by rising edge of a p=gnd or (pos) clock can never change at the passive phase, and thus needs no shadow. Likewise, the output of a Q_FDP0 flip-flop clocked by falling edge of a p=pwr or (neg) clock needs no shadow. Thus, from capacity point of view, clocks with labels p=gnd, p=pwr, (pos), or (neg) are desirable clocks.

The output of a flip-flop clocked by any edge of a clock labeled (both) must have a shadow, which will also be labeled (both). Moreover, any nets downstream from this flip-flop will also have shadows and be labeled (both). Thus, a clock with label (both) is always undesirable.

To understand why a given net has a shadow, use the following method. Find, in the `dumpShadows` report, the gate or storage element driving this net. If the driver is a combinational gate, a shadow on the output must be caused by a shadow on one of the inputs. If the driver is a flip-flop, a shadow on the output is usually caused by a (both) label on the clock input, or by a shadow on the set or reset input. (For a flip-flop, there are other possible causes, such as (sh) (pos) on the clock net connected to the flip-flop with "*" indicating inversion.)

The `dumpShadows` report is usually a very large file, but you can use the "vim" text editor to view it. Searching the file in the editor is very slow, so to find the driver of a net NNN it is usually better to use the `grep -n` command in a separate window to get a line number, and then go to that line number in the editor.

```
grep -n '<.* NNN' precompile.dumpShadows
```

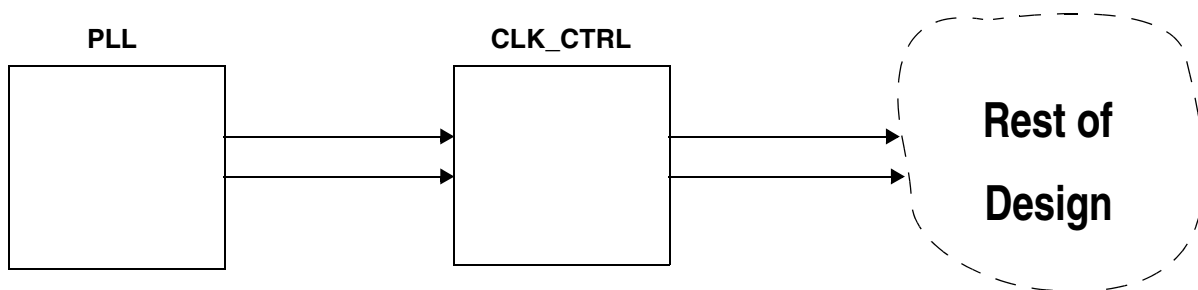
A good alternative to viewing the `dumpShadows` report directly is to use the Schematic Viewer. You can use the script `<install_dir>/share/uxe/gift/misc/shadows.qel` to generate a schematic for the cone of logic generating a given net, with the shadow labels represented as different colors on the schematic. The file `<install_dir>/share/uxe/gift/misc/shadows_howto.txt` gives more information about using `shadows.xel`.

Providing Extra Information to Precompile

This section describes methods of providing extra information to precompile to reduce the number of shadow nets in a design. All the methods except `precompile -moreExclusive` can alter design functionality. Use these methods with caution.

Declaring Extra Clock Sources

In some early releases, declaring internal nets as clock sources was a popular method of reducing 1X capacity cost. For example, one design had clock generation logic as in the diagram below:



Originally, the user declared 10 outputs of the PLL block as clock sources. Compiling in 1X mode required almost double the capacity required for 2X mode. However, when the user additionally declared 40 outputs of `CLK_CTRL` as clock sources, the 1X-mode capacity cost was reduced to about 1%.

Each clock source itself has a capacity cost. If the number of internal clock sources is large, it is better to use the `clockAssign` XEL command to connect internal nets directly to original clock sources.

The `shadowsData` XEL command provides a less drastic and more flexible way of providing clock information, so that you need not declare internal nets as clock sources to improve 1X-mode capacity.

Using shadowsData

You can use the `shadowsData` XEL command to specify behavior of particular clock, reset, or other nets in the design. Specifying this behavior can eliminate shadows on logic downstream from these nets.

To specify `shadowsData` for a net, use the command:

```
XEC> shadowsData -add {<net> <keyword>}
```


UXE User Guide

Modeling Clocks

The following keywords are supported: `p=gnd`, `p=pwr`, `pos`, `neg`, `none`. For simplicity, the description below assumes you have not specified `activeEdge` negative.

<code>p=gnd</code> or <code>p=pwr</code>	Forces <code><net></code> to be low (or high, respectively) during the passive phase, regardless of the value driven by the design at that time. A net declared as <code>p=gnd</code> or <code>p=pwr</code> in <code>shadowsData</code> becomes <code>p=gnd</code> or <code>p=pwr</code> in the <code>dumpShadows</code> report.
<code>pos</code> or <code>neg</code>	In a given FCLK cycle, if <code><net></code> is low in the active phase, the <code>pos</code> keyword in <code>shadowsData</code> forces it to be low in the following passive phase, regardless of the value driven by the design at that time. Similarly, if the <code><net></code> is high in the active phase, the <code>neg</code> keyword forces it to be high in the following passive phase, regardless of the value driven by the design at that time. The <code>pos</code> or <code>neg</code> keywords affect functionality of the <code><net></code> only when the <code><net></code> would have had a rising (or falling, respectively) edge at the passive phase of the primary clock. A net declared as <code>pos</code> or <code>neg</code> in <code>shadowsData</code> becomes (<code>pos</code>) or (<code>neg</code>) in the <code>dumpShadows</code> report because the net cannot have a rising (or falling, respectively) edge at the passive phase.
<code>none</code>	Forces the <code><net></code> to have the same value in the following passive phase as it has in the prior active phase. If <code><net></code> would have changed at the passive phase, the change is delayed to the next active phase, or lost entirely if <code><net></code> would have changed again at that active phase. Declaring a net as <code>none</code> removes the shadow on this net in the <code>dumpShadows</code> report.

Declaring a net as `p=gnd` or `pos` in `shadowsData` has a less drastic effect on the functionality than declaring it a clock source. If the net is a gated version of the fastest clock that is low when disabled, declaring the net as `p=gnd` in `shadowsData` allows the behavior to be preserved, whereas declaring it a clock source requires it to toggle always. Declaring an internal clock net as `pos` is even less drastic because it allows this clock to remain constantly high or constantly low when disabled, and allows it to switch between high and low frequencies.

Another advantage of using `shadowsData` over declaring clock sources is that you need not specify frequencies for the given clocks.

Detecting shadowsData Violations

Specifying `shadowsData` for a design net enforces an assumption about the behavior of the net. For example, `shadowsData none` means that the net never changes at the falling edge of the primary clock. If `shadowsData` incorrectly describes the natural behavior of a net, the `shadowsData` will alter the behavior of the net. Such a situation is called a `shadowsData` violation.

If you specify `shadowsData`, precompile automatically creates the following instrumentation nets that you can use to detect `shadowsData` violations:

- The `CVA_UA_VERIFY_SHADOWS_DATA` net is high only in those FCLK cycles in which a `shadowsData` violation occurs. This net (as well as the other instrumentation nets described below) can be used in SDL.

A typical use is as given below:

```
if (CVA_UA_VERIFY_SHADOWS_DATA == 1) trigger;
```

- For each `shadowsData` net `x.y.z`, precompile creates the following three instrumentation nets at the design top level (names are escaped):
 - ❑ `\x.y.z@shadowsDataOriginal` behaves as the original `x.y.z` would have behaved without `shadowsData`.
 - ❑ `\x.y.z@shadowsDataFail` is the individual violation signal for `x.y.z`. It is high in just those FCLK cycles in which `x.y.z` has a violation.

The global violation signal `CVA_UA_VERIFY_SHADOWS_DATA` is an OR of the individual violation signals.

- ❑ `\x.y.z@shadowsDataEnable` is the enable signal for violation on `x.y.z`, which you have to control. Normally, it is high. You can force it to low to disconnect `\x.y.z@shadowsDataFail` from the OR tree that controls `CVA_UA_VERIFY_SHADOWS_DATA`. In this case, `\x.y.z@shadowsDataFail` is still high when `x.y.z` has a violation, but does not cause `CVA_UA_VERIFY_SHADOWS_DATA` to be high.

You can add all these nets to the waveforms by using the `probe` command. Ensure that the escaped name of the signal, including the backslash at the beginning and the space at the end, are enclosed in curly braces.

For example:

```
probe -add {\x.y.z@shadowsDataOriginal }
```

Tying Mux-Selects or Other Signals to Reduce Capacity Cost of 1X Mode

In one design, by tracing back the reason for shadows on several randomly chosen flip-flop outputs, it was found that most of the 1X capacity cost was caused by a certain clock which was the output of a mux select. The waveform of this clock changed less than once per FCLK, yet the clock had a shadow, which caused other shadows in the design. The clock was the output of a MUX that selected between a 1X-type clock and a 2X-type clock. After investigation, it was found that in the mode that would be tested in emulation, the MUX-select would always be low, selecting the 2X-type clock. When the MUX-select was tied low using `tieNet`, the 1X-mode capacity cost was reduced from 24% to 4%.

Similarly, you can consider tying any important control signal if its value is known not to change. A common example is a `TEST_MODE` signal.

Using precompile -noReset1X Command to Reduce Capacity Cost of 1X Mode

Normally, if a flip-flop set or reset has a shadow, the output of the flip-flop automatically has a shadow. As a result, if a reset signal of the design can change at the falling edge of the fastest clock, it usually causes a large capacity cost for 1X mode. In such a design, using the `-noReset1X` option of the `precompile` command can reduce the 1X mode capacity cost. When this option is used, the shadow on the set or reset of a flip-flop does not propagate to the output of the flip-flop.

If the output of a flip-flop has a shadow because of its clock as well as because of its set or reset, the `-noReset1X` option has no effect. If the output Q of a flip-flop has a shadow due only to a shadow on set or reset, using the `-noReset1X` option removes the shadow on Q, and has the same effect on functionality as declaring the Q signal as `none` in `shadowsData`. The value of Q in the active phase is unaffected; the value of Q in the passive phase is copied from the active phase, even if the set or reset becomes active during the passive phase.

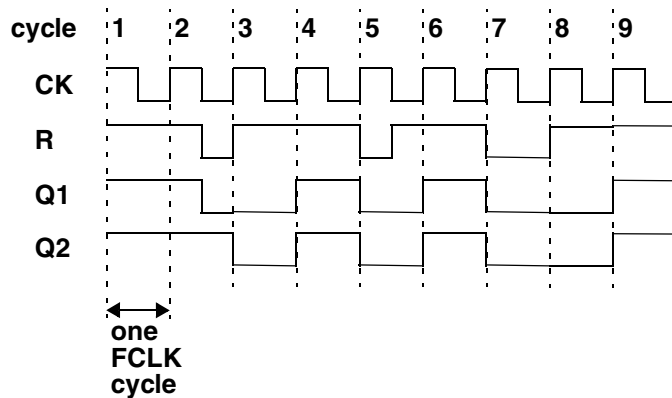
The effect of the `-noReset1X` option on the functionality of a flip-flop is not as severe as the effect of declaring the set or reset signal as `none` in `shadowsData`. If the design drives set or reset active in a short pulse of just a single passive phase, declaring the set or reset signal as `none` eliminates the pulse entirely. Using the `-noReset1X` option does not eliminate the set or reset pulse, but only causes flip-flop outputs to not show the effect until the next active phase.

The following diagram shows the behavior of a `Q_FDP1` flip-flop (rising-edge triggered, with active low reset) in 1X mode without and with the use of the `-noReset1X` option, assuming D is constant high. The diagram assumes that the `-syncReset` and `-asyncReset` options

UXE User Guide

Modeling Clocks

are not used. The waveform Q1 shows the Q output of the flip-flop if the `-noReset1X` option is not used, and Q2 shows Q if the `-noReset1X` option is used.



At cycle 3, there is a “race condition”:

The CK clock has a rising edge at the same time as `reset` R goes inactive. At cycle 3 without the `-noReset1X` option, signal Q1 remains low. (This behavior is required by some designs: if a flip-flop’s clock has an active edge at the same time as reset goes inactive, the flip-flop remains reset for that cycle.) At cycle 3, with the `-noReset1X` option, signal Q2 is low, the same as without the `-noReset1X` option. The only difference is that the signal Q2 goes low one half-cycle later than Q1 does.

By default, the effect of the `-noReset1X` option applies only to flip-flops, not to latches. To obtain the benefit of the `-noReset1X` option on latches, you should use the `-latchResetAsFF` option in addition to the `-noReset1X` option.

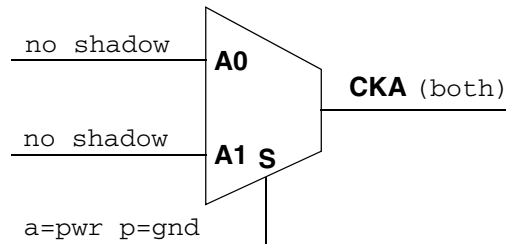
Reducing 1X Capacity Cost

To reduce 1X-mode capacity cost, it is helpful to be able to see a waveform for any signal in the design. A good way to accomplish this is to capture, in a 2X-mode full-vision database, a trace file for a few hundred cycles before and after design reset is de-asserted. Then, you can use offline debug to obtain a waveform for any signal in the design. The waveform will not show all possible waveforms for a given signal, but it shows a possible and likely waveform for each signal. (If emulation waveform waveforms are not available, then simulation waveforms can be used instead.) Sometimes, by consulting someone with knowledge of the design, you can confirm that a signal will *always* behave as it happens to behave in the waveform. Or, a signal may behave a certain way in the operating modes that will usually be tested in emulation.

Here are some situations where 1X-mode capacity cost can be reduced.

1. Shadows are unnecessary for the actual design behavior, but precompile cannot prove it.

For example, in one design, tracing back the reason for the shadow on a randomly chosen storage element revealed an important design clock (CKA) which was labeled (both) in the dumpShadows report, even though its waveform was the same as that of the fastest design clock. The clock was the output of a two-input MUX.



The MUX A0 and A1 inputs were flip-flop outputs and did not have shadows. The S input was the fastest design clock (labeled a=pwr p=gnd). Depending on the values of A0 and A1, it is logically possible that CKA could be either the same, or inverted, compared to the primary clock, so precompile must label it (both). However, the clock module designer said that in actual operation, CKA would never be inverted compared to the primary clock, and would never have a rising edge coinciding with the primary clock falling edge. When clock CKA and four other similar clocks were labeled `pos` in shadowsData, the capacity cost for 1X mode was reduced from near 100%, to less than 1%.

2. Shadows are needed for a certain mode of operation, but that mode will not be used during emulation, or will not usually be used.

In one design, tracing back the reason for the shadow on a randomly chosen storage element revealed an important design clock (BUSCLK) which was labeled (both) in the dumpShadows report, even though its waveform showed a slow clock changing only at the rising edge of the primary clock. The signal was the output of a four-input MUX that selected between four clocks (CK, CK/2, CK/3, CK/4) which were divide-by-one, divide-by-two, divide-by-three and divide-by-four of the primary clock. The (both) attribute on BUSCLK came from the divide-by-three clock. The purpose of these four clocks was to allow different frequency ratios between BUSCLK and another clock (CPUCLK). Since the intention was to use the divide-by-three clock in less than 1% of the testing, tieNet was used to tie the divide-by-three clock `low`. This reduced the 1X-mode capacity cost from 50% to 10%. A separate 2X-mode compilation was used for testing with the divide-by-three clock. Alternatively, if the exact location of edges of the divide-by-three clock relative to the primary clock edges was not important, shadowsData could have been used to enforce a `none` property on this clock.

3. Shadows are caused by an emulation model, not by the design itself.

In one design, tracing back the reason for the shadow on a randomly chosen storage element revealed an important design clock which was labeled (both) in the dumpShadows report. Its waveform showed a slow clock whose transitions always coincided with the falling edge of the primary clock. However, this slow clock was generated by a clock generation model that had been written for emulation and was not part of the real design being verified. When the model was modified so that the slow clock changed at the rising edge of the primary clock, most of the 1X-mode capacity cost was eliminated.

Reducing 1x-mode capacity cost is an iterative process, as described earlier. Each iteration consists of the following steps:

1. Choose, at random within the dumpShadows report, a storage element (flip-flop or latch) whose output has a shadow. You can make a random choice by going to an arbitrary line in the file, such as line 1000000, and then searching forward for the first FDRSE or LDRSE.
2. Observe the labels on the storage element's inputs, and determine which input(s) caused the shadow on the output. Once you know which input (usually, clock or reset) caused the shadow on the output, there are two approaches you can take:
 - ❑ Immediately eliminate the shadow on the storage element output, by applying a shadowsData keyword to the clock or reset, or by one of the other methods described earlier. Usually this will eliminate shadows on some other storage elements as well.
 - ❑ Use the dumpShadows report and the waveform to trace back the reason the clock has label (both), or reset has a shadow. Sometimes many clocks labeled (both) have a common root cause. If you can find a single root cause, it is better solve the problem at its root rather than separately solving many resultant problems. If a clock net labeled (both) is driven by a gate with more than one input labeled (both), it is usually better to trace back an input that is toggling in the waveform, rather than one that is flat. Tracing a toggling input is likely to lead more directly to the single root cause, if there is one.

You can use the following tactics to remove the shadow on a storage element output.

- a. If the shadow on the storage element's output is caused by a shadow on its reset signal, apply keyword `none` to the reset signal in shadowsData, or use `precompile -noReset1X` option.
- b. If the shadow on the storage element output is caused by a (both) label on its clock, but in the waveform, the clock is a low frequency clock which changes only on the active edge of the fastest clock, apply keyword `none` (or `pos`) to the clock in shadowsData. The `none` keyword has a stronger effect in removing shadows, but is more likely to affect design functionality.

- c. If the shadow on a storage element output is caused by a (both) label on the clock, but in the waveform, in every FCLK cycle the clock is either the same as the fastest clock, or flat low, apply keyword `p=gnd` (or `pos`) to the clock in `shadowsData`. The `p=gnd` keyword has a stronger effect in removing shadows, but is more likely to affect design functionality.
- d. If the shadow on the storage element output is caused by a (both) label on the clock, but in the waveform, the clock never has a rising edge that coincides with the falling edge of the fastest clock, apply keyword `pos` to the clock in `shadowsData`. If the clock never has a falling edge that coincides with the falling edge of the fastest clock apply keyword `neg` to the clock in `shadowsData`.
- e. If a clock has both rising and falling edges that coincide with the falling edge of the primary clock, then the (both) label in the `dumpShadows` report accurately describes the clock's behavior, and applying any keyword in `shadowsData` will be inconsistent with the behavior in the waveform. In this case, try to determine the reason for the observed behavior, and determine whether the design can be modified to change the behavior.

Tactics b, c, and d are for cases where a clock is labeled (both) in `dumpShadows`, but the waveform shows a more benign behavior. Tactics a,b,c and d all use `shadowsData` to enforce a desired behavior. If possible, before applying any of tactics a-d for a given signal, ask the designer whether there are modes or conditions where the signal's behavior is expected to be different from the desired behavior.

3. After making the changes, recompile the design and verify that the chosen storage element no longer appears in the `dumpShadows` report.

Additional Information on 1X Mode

This section provides extra information about 1X mode:

- ☐ [Using a Dummy Fastest Clock Source](#)
- ☐ [Implementing Quarter-Cycle Phase-Shifted Clocks](#)
- ☐ [Active- and Passive-Phase Values in SDL](#)

Using a Dummy Fastest Clock Source

Sometimes it is helpful to define a dummy clock source which is faster than the fastest real clock source in the design. For example, in a design having two fastest clocks with relative frequencies 5 and 3, if the clock with frequency 3 generated too many shadows in 1X mode, define a dummy clock source with frequency 6. Then, the clock with frequency 3 becomes a

2X-type clock, and the clock with frequency 5 still runs almost as fast as it would have without the dummy clock.

Note: In 1X mode, the compiler optimizes the capacity because the fastest CAKE clock source is the same as the FCLK of the emulator. Therefore, in some designs, defining a dummy clock source could increase the 1X mode capacity cost.

To specify a net as a dummy clock source, the net must exist in the design. If you do not wish to modify the design netlist, then you can use the `iScopedNets` command to add an unconnected net to a particular module in the design. For example, the following command adds an unconnected net named `my_dummy_clock` to cell `TOP` in library `LIB`:

```
iScopedNets -VERSION3 LIB TOP - my_dummy_clock . h
```

Implementing Quarter-Cycle Phase-Shifted Clocks

Occasionally, a design might require a second clock source which is the same as the fastest clock source, but delayed by one quarter of a cycle. Suppose the names of these first and second clock sources are `CLKA` and `CLKB` respectively. To implement such a design in 2X mode, the period of `CLKA` must be four FCLK cycles, resulting in a lower-than-desired emulation speed. You can implement such a design in 1X mode by defining a dummy fastest CAKE clock source `CLKDUMMY`, and then using `clockAssign` commands to specify the behavior of `CLKA` and `CLKB` relative to that of `CLKDUMMY`:

```
clockOption -add {technology CAKE 1}  
clockFrequency -add {CLKDUMMY 100}  
clockAssign -add {CLKA CLKDUMMY 2}  
clockAssign -add {CLKB ~ CLKDUMMY 2}
```

These `clockAssign` commands specify that signals `CLKA` and `CLKB` each have frequency half that of `CLKDUMMY` and that `CLKA` and `CLKB` change on the rising and falling edges, respectively, of `CLKDUMMY`. Outputs of all flip-flops clocked by `CLKB` will have shadows, so this approach will have a significant capacity penalty unless `CLKB` clocks only a small portion of the design. If `CLKB` clocks more of the design than `CLKA`, swap the names `CLKA` and `CLKB` in the above `clockAssign` commands.

You can add the dummy clock `CLKDUMMY` to the design using `iScopedNets`, as described in the previous section.

Active- and Passive-Phase Values in SDL

SDL samples signals once per FCLK. In 1X mode, if a net can have different values in the active and passive phases of the primary clock, then normally, SDL uses the passive-phase value.

However, if you declare `shadowsData` for a net, precompile provides special visibility; you can use SDL to refer to that net's active- and passive-phase values.

For the net whose hierarchical pathname is `u1.u2.net3`, the special names `\u1.u2.net3@QTactive` and `\u1.u2.net3@QTpassive` refer to the active- and passive-phase values of the net. Each special name includes a space at the end. The regular name `u1.u2.net3` also refers to the passive-phase value. If one or more components of the hierarchical pathname are escaped names, the special name is constructed by changing the space delimiting each escaped component to a `!` character, and then constructing the name as usual.

For example, the following net:

```
\u/1 .\u/2 .\net/3
```

has the following active-phase name:

```
\\u/1!.\u/2!.\net/3!@QTactive
```

You can also use the `value` command with a special name:

```
value {\u1.u2.net3@QTactive }
```

Specifying a `shadowsData` keyword such as `pos` or `none` for a net can affect the net's behavior. To enable special visibility without affecting the net's behavior, use the `both` keyword.

For example:

```
shadowsData -add {u1.u2.net3 both}  
shadowsData -add {\u/1 .\u/2 .\net/3 both}
```

Using the CAKE 0.5X Mode

CAKE 0.5X mode is a compilation mode that allows the fastest design clock to run at double the frequency of FCLK. Usually, for this mode to be useful, the design's highest-frequency clock must clock only a small fraction of the design, and the design must use only one edge of the highest-frequency clock.

To compile in 0.5X mode, give the following command before precompile:

```
XEC> clockOption -add {technology CAKE 0.5}
```

Precompile only allows 0.5X mode if the following restrictions are met:

- All changes in the values of storage elements (latches, flip-flops, and memories) must occur at the active edge (that is, the rising edge, unless `activeEdge negative` is specified) of the fastest design clock.

UXE User Guide

Modeling Clocks

- If a storage element needs to be updated depending on the value of one of its inputs in the passive phase of the fastest clock, and this input can have different values in the active and passive phases, then the value in the passive phase must be a constant known at compile time. This rule precludes certain types of gating for the fastest clock in 0.5X mode. Usually, the fastest clock can be gated by a sequence of AND gates, but if it is gated by an AND followed by an OR gate, then precompile will not allow 0.5X mode.
- If a design output can change at the passive phase of the fastest clock, you must specify
`XEC> clockOption -add {ignoreOutputs on}`

As a result, the output will reflect only the value the signal would have in the active phase.

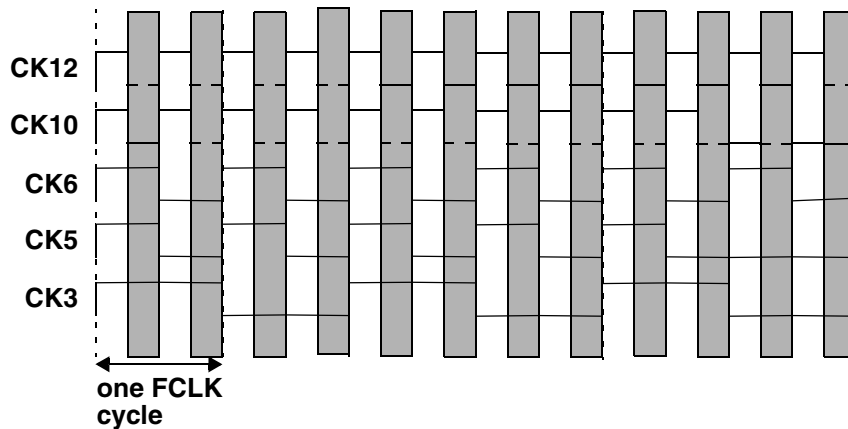
When a design satisfies these restrictions, precompile can implement it in 0.5X mode using no more than net and shadow for each user net (that is, it is not necessary to implement four different nets for each user net). In CAKE 0.5X mode, net and shadow represent a user signal's value in the active phase of even- and odd-numbered cycles of the fastest clock source.

In 0.5X mode, a clock with frequency greater than half that of the fastest clock will run faster than FCLK, and is called a 0.5X-type clock. A flip-flop clocked by a 0.5X-type clock must always have a shadow, even though it is clocked by the rising edge of the clock. So, 0.5X mode will always have a significant capacity cost unless 0.5X-type clocks clock only a small fraction of the design.

At run time, the design is implemented in 1X mode. Waveforms show only two time-points per FCLK cycle; values of design signals during the passive phase of the fastest clock are not shown. The values in the passive phase of the fastest clock are not shown. So, the waveform of the fastest clock always appears flat.

The following diagram shows the behavior of clocks in 0.5X mode for a design with five CAKE clocks having relative frequencies 12, 10, 6, 5, and 3. Logic Analyzer waveforms show only

two time-points per FCLK cycle; the values of signals during the shaded time periods do not appear in the waveform.



Target I/O (except multipulse outputs defined with `terminalTiming -edgeInfo`) can change at most once per cycle of the fastest clock (that is, twice per FCLK). In order to allow target I/O to change twice per FCLK, you must enable the multisampled I/O feature, and specify as a `multiSampledTerminal` any input that should be sampled twice per FCLK.

To connect the fastest CAKE clock to the target, use a `terminalTiming -edgeInfo` command similar to the following, with two rising edges and two falling edges per FCLK cycle:

```
terminalTiming -add {sb_if_clk_out -edgeInfo 0 96 0 96 240 336}
```

After compiling, verify in the `dbFiles/<top>.ttime` file that all non-shadow signals are scheduled earlier than the second rising edge of the multipulse clock. For the above `terminalTiming -edgeInfo` example, the second rising edge is at step 240. The `<top>.ttime` file uses units of TIFs, and one TIF is 8 steps. Thus, for this example, user needs to verify that each non-shadow signal is scheduled earlier than TIF 30. If any signal is TIF 30 or later, adjust the step numbers in the `-edgeInfo` command if necessary, and recompile.

Except for the 0.5X-type clock implementation, the I/O timing constraints for 0.5X mode are normally the same as for 1X mode.

Inserting Delays between Cross-Domain Synchronizers

Cross-domain synchronizers can be verified in designs having Clock Domain Crossing (CDC). During the precompile process, delays are automatically inserted between the cross-domain synchronizers according to the specifications that you provide. These delays can be dynamically switched on and off at run time without interrupting the emulation. To provide the cross-domain synchronization:

UXE User Guide

Modeling Clocks

- Describe the cross-domain synchronizers in a separate CDC file (that is, *.cdc file) and specify this filename using the following `precompileOption` command option:

```
precompileOption -add {cdc <CDC_file_name>}
```

Refer to the [Clock Domain Crossing \(CDC\) Format](#) section of [Appendix K, “Files and File Formats”](#) for details about the format of the CDC file.

- Describe the dynamic control for the delays in a special module called `cdc_control`. Create this module and instantiate only once in the design hierarchy. The location in the design hierarchy does not matter. For an example of the `cdc_control` module, refer to the [Example of Inserting Delays between Cross-Domain Synchronizers](#) section.

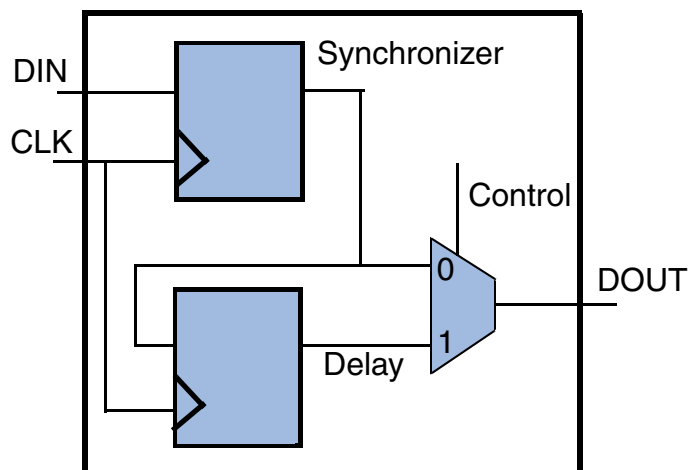
For the controls that you specify in the CDC file, provide the driving logic in the `cdc_control` module by using registers or wires. Note that for a sourceless control, the delay will still be instrumented but it will not be activated unless it is forced to 1 at debug time.

Precompile automatically locates the `cdc_control` module instance in the design hierarchy. If such an instance is missing or there are multiple instances in the hierarchy, an error is generated.

If you set the CDC version to 0 in the CDC file, the instrumentation that is inserted by precompile is as per the schematics given below. The *Control* in this schematics is connected to the corresponding register or wire in the `cdc_control` module, as specified in the CDC file.

Precompile inserts additional internal flip-flop that drives Delay.

Output mux controlled by Control selects one or two clock delays from DIN to DOUT.



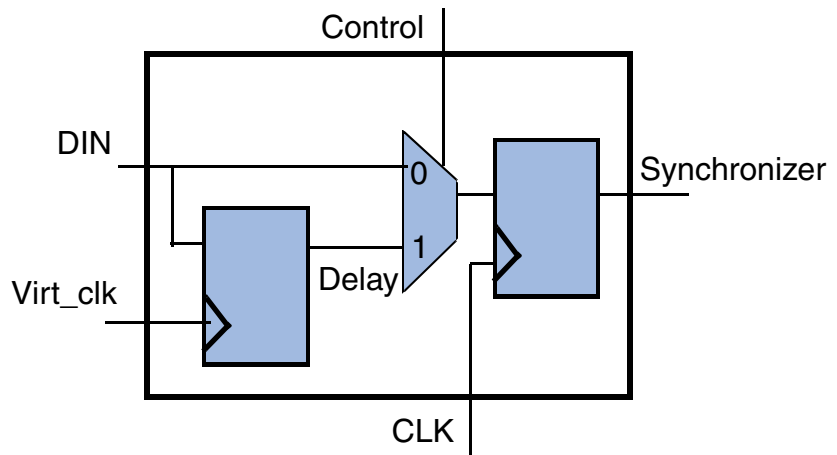
However, if you set the CDC version to 1 in the CDC file, the instrumentation that is inserted by precompile (shown in the schematics below) is different in the following two ways:

- Delay is controlled by the virtual clock `Virt_clk` instead of the design clock `CLK`. (Virtual clock is driven in the `cdc_control` module).

- Delay is placed at *Synchronizer* D input instead of Q output.

Precompile inserts additional internal flip-flop that drives Delay with virtual clock.

Mux controlled by Control selects a delay for DIN going into D input of the Synchronizer.



Example of Inserting Delays between Cross-Domain Synchronizers

This section discusses the following two examples:

- Example of writing and then instantiating the `cdc_control module` in a DUT file.
- Example CDC file named `Example CDC File: 1.cdc` for CDC version 0.

Use the following command to invoke the `1.cdc` file:

```
precompileOption -add {cdc 1.cdc}
```

cdc_control module

```
module cdc_control(clk, cdc);  
input clk, cdc;  
wire sync0_sel, sync1_sel; // controls  
for delaying synchronizers  
reg[2:0] cnt; // just a counter  
always@(negedge clk) cnt = cnt+1;
```

Module with predefined name
`cdc_control` you must provide and
instantiate once in the hierarchy

```
assign sync0_sel = (cnt >= 1 && cnt <= 3) && cdc;  
assign sync1_sel = (cnt >= 3 && cnt <= 6) && cdc;  
endmodule
```

Delays are ON periodically if `cdc` is
asserted

```
module sync(clk, d, q);  
input clk;  
output[7:0] d, q;  
reg[7:0] d, q;  
reg[7:0] sync; // sync is a  
synchronizer between d and q
```

A module with a synchronizer

```
always@(posedge clk) begin  
d = d + 1;  
sync <= d;  
q <= sync;  
end  
endmodule
```

UXE User Guide

Modeling Clocks

```
module sync_and_control(clk, cdc, d,
q);
input clk, cdc;
output[7:0] d, q;
reg[7:0] d, q;
reg[7:0] \!sync ; // \!sync is a
synchronizer between d and q

always@(posedge clk) begin
    d = d + 1;
    \!sync <= d;
    q <= \!sync ;
end

cdc_control CTRL(clk, cdc);
endmodule

module dut(clk, cdc, d0, d1, q0, q1);
input clk, cdc;
output[7:0] d0, d1, q0, q1;

sync SYNC (clk, d0, q0);
// testing escaped instance name
sync_and_control \SYNC&CTRL (clk, cdc,
d1, q1);

endmodule
```

A module with a synchronizer and
cdc_control instance

One and only one instance of module
cdc_control

Top of DUT

Example CDC File: 1.cdc

Controls for synchronizers in module sync:

```
crossing_00 dut.SYNC.sync[0] clk
sync0_sel

crossing_01 dut.SYNC.sync[1] SYNC.clk
sync0_sel

crossing_02 SYNC.sync[2] clk sync0_sel
crossing_03 SYNC.sync[3] dut.SYNC.clk
sync0_sel
```

Path to synchronizer from top, with no
path to clock

Path to synchronizer from top, with path
to clock

Path to synchronizer a level below top

UXE User Guide

Modeling Clocks

#

```
crossing_04 dut.SYNC.sync_reg[4] clk
sync0_sel
crossing_05 dut.SYNC.sync_reg[5]
SYNC.clk_sync0_sel
crossing_06 SYNC.sync_reg[6] clk
sync0_sel
crossing_07 SYNC.sync_reg[7]
dut.SYNC.clk_sync0_sel
```

} Same as above with `_reg` suffix to synchronizer names

Controls for synchronizers in module `sync_and_control`:

```
crossing_10 dut.\SYNC&CTRL .\!sync [0]
clk_sync1_sel
```

} Path to synchronizer from top, with no path to clock

```
crossing_11 dut.\SYNC&CTRL .\!sync [1]
\SYNC&CTRL .clk_sync1_sel
```

} Path to synchronizer from top, with path to clock

```
crossing_12 \SYNC&CTRL .\!sync [2] clk
sync1_sel
crossing_13 \SYNC&CTRL .\!sync [3]
dut.\SYNC&CTRL .clk_sync1_sel
```

} Path to synchronizer a level below top

```
crossing_14 dut.\SYNC&CTRL .\!sync_reg
[4] clk_sync1_sel
crossing_15 dut.\SYNC&CTRL .\!sync_reg
[5] \SYNC&CTRL .clk_sync1_sel
crossing_16 \SYNC&CTRL .\!sync_reg [6]
clk_sync1_sel
crossing_17 \SYNC&CTRL .\!sync_reg [7]
dut.\SYNC&CTRL .clk_sync1_sel
```

} Same as above with `_reg` suffix to synchronizer names

Modeling Clocks in SA Mode

To model clocks in the SA mode, you can opt to use the *ixclkgen* utility, which enables automatic generation of the clocks or manually generate the clocks by using the Verilog delay control statements.

These two methods are described in the following sections:

- [Manually Modeling Clocks in SA Mode](#)
- [Using userData to Create a Clock-Generation Module for SA Mode](#)

Manually Modeling Clocks in SA Mode

This section describes how Verilog delay control statement (*#delay*) can be directly used to model clock generation in the DUT using the IXCOM compiler.

IXCOM supports clock-generation modeling with the *always* procedure as well as *initial* procedure with a *forever* loop. For both modeling styles, the delay control statement (*#delay*) is used to specify the phase delays of the clock.

The following types of delays are supported.

- Non-zero constant delay control
 - Integer constants, such as #1, #20
 - Real constants, such as #1.2, #0.5
 - Time constants, such as #5ns, #10fs

For example:

```
always #1 clk1 = ~clk1;
always begin
    clk2 = 0; #0.5;
    clk2 = 1; #1.5;
end
initial forever #5ns clk3 = ~clk3;
```

- Variable delay control

#Td where *Td* is a net or variable of integer data types

For example:

```
reg [3:0] Td1;
reg [3:0] Td2;
initial begin
    Td1 = 3; Td2 = 3;
```

```
    clk4 = 0;
    forever begin
        #Td1; clk4 = 1;
        #Td2; clk4 = 0;
        // conditionally increment delay to rising edge
        if (Td1 < 10) Td1 = Td1 + 1;
    end
end
```

In addition to delay control statements, the following non-time-consuming Verilog statements are allowed in clock modeling:

- Blocking and non-blocking procedural assignments without intra-assignment delay

- ☐ `clk1 = 0;`
- ☐ `clk2 <= 1;`

- Conditional statements containing no delay control statements

- ☐ `if (condition)...`
- ☐ `if (condition)... else ...`

- Case statements containing no delay control statements

- ☐ `case (expression)...`
- ☐ `casez (expression)...`
- ☐ `casex (expression)...`

Enabling Clock Model Compilation

To compile clock models described using delay control statement (`#delay`) in a module in the DUT, interface compilation for delay needs to be enabled by using either of the following options:

- Specify the module with the `+iscDelay+<mod_name>` option of the `ixcom` command. The `<mod_name>` refers to the name of the DUT module that you have defined.

For example:

```
ixcom -ua clkgen.v +dut+clkgen +iscDelay+clkgen ...
```

In the above command, `clkgen` is the name of the module in the DUT.

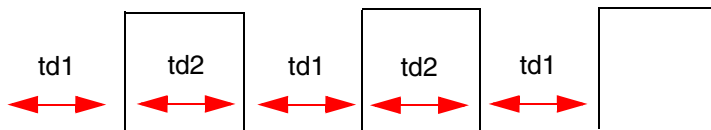
- Add the following `map_delays` directive in an `initial` procedure in the module containing the clock model:

```
$ixc_ctrl("map_delays");
```

Note: IXCOT generates a compilation error if interface compilation for delay is enabled but no valid clock model is found under the specified modules in the DUT.

Modeling Clocks without Initial Delay

Here is a waveform to illustrate clocks without initial delay:



To model a clock without initial delay as shown in the waveform above, the following clock model templates are supported.

■ Clock model with `always` procedure

```
always begin
    [#Td; <non-time-consuming_statements>;]+
end
```

For example:

```
reg clk = 0;
reg [3:0] td1 = 5;
reg [3:0] td2 = 10;
always begin
    #td1; clk = 1;
    #td2; clk = 0;
end
```

■ Clock model with a `forever` loop in `initial` procedure

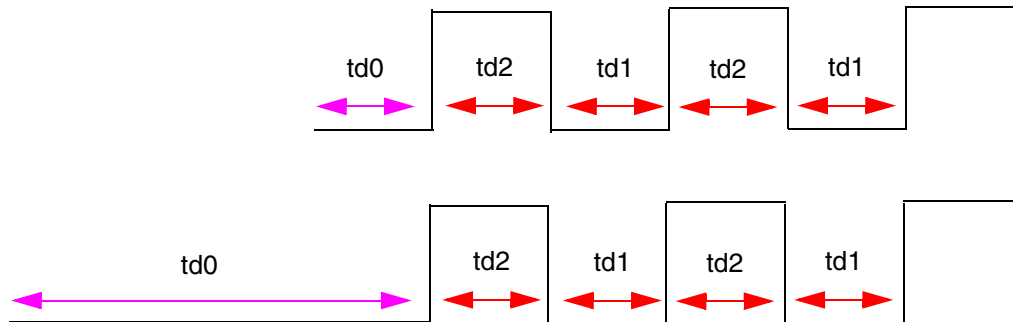
```
initial begin
    [<non-time-consuming_statements>;]*
    forever begin
        [#Td; <non-time-consuming_statements>;]+
    end
end
```

For example:

```
initial begin
    // initialize clk and phase delays td1 and td2
    clk = 0;
    td1 = 5; td2 = 10;
    forever begin
        #td1; clk = 1;
        #td2; clk = 0;
    end
end
```

Modeling Clocks with Initial Delay

The following waveforms illustrate clocks with initial delay:



To model a clock with an initial phase delay as shown in the waveforms above, the following clock model templates are supported.

■ Clock model with always procedure

```
initial td = <initial_delay_to_first_clock_edge>;
always begin
    #td; <first_clock_edge>;
    #<regular_delay_to_second_clock_edge>; <second_clock_edge>;
    td = <regular_delay_to_frist_clock_edge>;
end
```

For example:

```
reg clk = 0;
reg [3:0] td0 = 20;
reg [3:0] td1 = 5;
reg [3:0] td2 = 10;
reg [3:0] td;
initial td = td0;
always begin
    #td; clk = 1;
    #td2; clk = 0;
    td = td1;
end
```

■ Clock model with a forever loop in initial procedure

```
initial begin
    [non-time-consuming statements;]*
    #<initial_delay_to_first_edge>;
    forever begin
        <first_clock_edge>;
        #<regular_delay_to_second_clock_edge>;
        <second_clock_edge>;
        #<regular_delay_to_frist_clock_edge>;
    end
end
```

For example:

```
initial begin
    // initialize clk and phase delays td0, td1 and td2
    clk = 0;
    td0 = 20; td1 = 5; td2 = 10;
    #td0;
    forever begin
        clk = 1;
        #td2;
        clk = 0;
        #td1;
    end
end
```

Modeling Clocks with Variable Phase Delay

To model a clock with variable phase delay, use a variable to capture the phase delay and change it conditionally or unconditionally inside the clock model. All the clock model templates presented in the [Manually Modeling Clocks in SA Mode](#) section support variable phase delay.

■ Example of clock model using `always` procedure with conditional statement

```
module clkgen(clk, change_delay, new_delay);
    output clk;
    input change_delay;
    input [3:0] new_delay;
    reg clk = 0;
    reg [3:0] td = 10; // initialize td to first delay;
    always begin
        #td; clk = ~clk;
        if (change_delay) begin
            td = new_delay;
        end
    end
end
endmodule
```

■ Example of clock model using a `forever` loop in `initial` procedure with `case` statement

```
module clkgen(clk, rising_delay, falling_delay, d1, d2, d3);
    output clk;
    input [1:0] rising_delay, falling_delay;
    input [3:0] d1, d2, d3;
    reg clk;
    reg [3:0] td1, td2;
    initial begin
        // initialize clk and rising/falling delays td1/td2
        clk = 0;
        td1 = 5; td2 = 10;
        forever begin
            #td1; clk = 1;
            case (falling_delay)
                0: td2 = d1;
                1: td2 = d2;
                2: td2 = d3;
            endcase
        end
    end
end
```

```
        //3: td2 stays the same
    endcase
    #td2; clk = 0;
    case (rising_delay)
        0: td1 = d1;
        1: td1 = d2;
        2: td1 = d3;
        //3: td1 stays the same
    endcase
end
end
endmodule
```

IXCOM Messages Related to Clock Model Compilation

Following is a list of messages related to clock-model compilation, generated during IXCOM interface compilation for delay:

■ ISCDELAY_IGNORE_INITIAL_CLOCK

- ❑ **Warning message:** “Blocking delay in initial block does not match supported clock models”
- ❑ **Impact:** Delay control not matching supported clock models would stay in the `initial` block, possibly preventing swapping into emulator until the simulator completes execution of the specified delay.

■ ISCDELAY_INITIAL_CLOCK_FOUND

- ❑ **Info message:** “Found clock specification in initial block”
- ❑ **Impact:** Clock model in indicated `initial` block is successfully identified and extracted to an `always` block for execution in the emulator.

■ ISCDELAY_CLOCK_FOUND

- ❑ **Info message:** “Found clock specification in always block”
- ❑ **Impact:** Clock model in indicated `always` block is successfully identified and converted to clock-generation logic for the emulator.

This includes `always` block extracted from initial block
[ISCDELAY_INITIAL_CLOCK_FOUND].

■ ISCDELAY_INITIAL_CLOCK_MISSING

- ❑ **Warning message:** “Clock specification extracted from initial block is missing during transformation”
- ❑ **Impact:** Clock model extracted from `initial` block is missing. No clock-generation logic is created.

When a clock model specified in an `always` procedure has been successfully transformed to clock-generation logic, a corresponding `ISCDELAY_CLOCK_FOUND` message is displayed.

When a clock model specified with a `forever` loop in an `initial` procedure has been successfully transformed to clock-generation logic, both `ISCDELAY_INITIAL_CLOCK_FOUND` and `ISCDELAY_CLOCK_FOUND` messages are displayed.

The first message indicates that the clock specification has been successfully identified in an `initial` procedure and has been extracted as an `always` clock specification for hardware execution. The second message indicates that the extracted `always` clock specification has been successfully transformed to clock-generation logic.

Dynamic Clock Modeling

Dynamic clock frequency and shape modeling can be carried out by first specifying the variable delays in the code, and then using the `force` command on the `xe>` prompt to change the delay value before the simulation starts. The delay value can also be changed in the middle of the simulation, before and after swapping the design between hardware and software.

For example:

```
reg clk = 0;
reg [7:0] hT = 5;
reg [7:0] lT = 3;
always begin
    #hT clk = ~clk;
    #lT clk = ~clk;
end
```

You can change the frequency and the shape of this clock at the `xe>` command prompt:

```
xe> force hT 7
```

Using Clock Generators with VHDL

To program one or more clocks in a VHDL design, write a Verilog module for clock generation, and then instantiate that module in the VHDL DUT. The Verilog module for clock generation is given below:

```
`timescale 1ns/1ns
module clk_gen(vlg_clk);
output reg vlg_clk = 0; //clock is output to VHDL
always #5 vlg_clk = ~vlg_clk;
endmodule
```

Then, instantiate the Verilog module in the DUT as follows:

UXE User Guide

Modeling Clocks

```
library IEEE;
use IEEE.std_logic_1164.all;
entity DUT is -- note: DUT does not have ports for emulation
end DUT;

architecture RTL of DUT is
  component clk_gen
  port (vlg_clk : out std_logic);
  end component;
  signal sys_clk : std_logic;
begin

  clk_gen1 : clk_gen
  port map (vlg_clk => sys_clk);

  . . .
end RTL;
```

Analyze and compile the design as follows:

```
vlan -work work clk_gen.v
vhan -work work clk_gen.vhd
vhan -work work DUT.vhd [other_options]

ixcom -top DUT -ua +dut+DUT +iscDelay+clk_gen
```

Handling Single-Edged Clocks

If the IXCOM compiler detects a primary input (clock) that is active on a single edge (posedge-only or negedge-only), the compiler skips the hardware evaluation of the unused edge. This can result in a performance improvement of nearly 2X for some designs.

Keep in mind the following implications of the above performance enhancement measure:

- The clock waveform from hardware probes only shows a zero-delay pulse at the rising (or falling) edge of the original clock signal. The waveform does not show changes for all the DUT signals at the falling (or rising) edge of the clock signal. The `-event` option of the `probe` command needs to be used.
- In `xeDebug`, the `value` command for the clock signal returns 0 (or 1) all the time.



Tip

For the list of single-edged clock inputs optimized by the compiler, in the `tmp/precompile.msg` file, search for `PECLK` or `NECLK`.

Performance Considerations for Modeling Clocks in SA Mode

All the clocks should be either in the DUT or testbench partition, but not in both partitions. When clocks exist in both hardware and software partitions, clock synchronization overheads adversely impact the performance.

If the clocks need to exist in the hardware as well as software partition, choose any of the following options for performance reasons:

- Generate all clocks in the testbench (or software side)

Use `+dutexcl+` to force the clock generation modules into software. You must use `xc on -run` instead of `xc on -tbrun` in this scenario.

- Generate all clocks in the DUT (or hardware side)

Use the hardware-modeling methods as defined above to model clock generators in DUT. You can send the DUT-generated clock to testbench through output ports or cross-module reference. You also need to use the `$export_event` directive as shown below:

```
initial $export_event(dut.to_TB_clk):
```

The `$export_event` directive exports a DUT clock to testbench so that IXCOM compiler can send out the clock event accurately from the DUT to the testbench to preserve the functionality.

In this scenario, you can use `xc tbrun` or `xc run` modes without incurring any expensive clock synchronization overheads because the timed event is in hardware partition.

Synchronizing the Fastest Design Clock with Emulator FCLK

On the emulator side, 1X mode is a compilation mode that allows the fastest design clock to run at the same frequency as the emulator's FCLK. Without the use of 1X mode, the fastest design clock would run at a speed at least two times slower than FCLK.

In the SA mode, using the 1X mode improves the performance of Palladium XP, but it might increase capacity requirements. To ensure the usage of 1X mode, use the `+1xua` option with the `ixcom` command. Note that IXCOM passes this option automatically to the back-end compiler during the pre-compilation state. You must not pass this option through the `compilerOptions.qel` file.

The 1X mode can be invoked in both single-step and multi-step compilation.

Multi-Step Compilation with +1xua Option

For a mixed-HDL or VHDL design, the multi-step compilation process needs to be used, involving a combination of the `vlan`, `vhan`, and `ixcom` commands.

The `+1xua` option must be used with the `ixcom` command.

1. Analyze the VHDL and Verilog parts of the design separately, using the `vlan` and `vhan` commands.

```
vlan ${UXE_HOME}/etc/ixcom/IXCclkgen.sv
```

2. Compile the entire design to generate the simulation snapshot using the `ixcom` command.

```
ixcom -ua +1xua -top <top_module>
```

Single-Step Compilation with +1xua Option

For a Verilog-only Design, the single-step compilation process is used to generate the simulation snapshot, involving only the `ixcom` command. Separate `vlan` and `vhan` steps are not required.

```
ixcom -ua +1xua <all_source_files> -v ${UXE_HOME}/etc/ixcom/IXCclkgen.sv
```

Using the 1X Mode Clock Generator Library: IXCclkgen.sv

The `${UXE_HOME}/etc/ixcom/IXCclkgen.sv` library file is sourced for the RTL modules required for generating the clocks for 1X mode.

Using the 1X Mode Base Clock Generator IXCclkgen for Controlled Clock Domains

Use the `IXCclkgen` module for modeling controlled simulation clock generator:

```
IXCclkgen #(phase_length) (phi1, phi2);
```

Following is the simulation model for `IXCclkgen`:

```
reg phi1 = 0;
always #phase_delay phi1 = ~phi1;
assign phi2 = ~phi1;
```

Only one instance of this base clock-generator module should be instantiated in the entire design, and this must be the fastest clock in the entire design. This controlled clock module has both simulation and emulation models and supports hot swap.

This clock generator can be used in both 1X and 2X modes.

- ❑ In the 1X mode, both the edges of this clock are executed in a single FCLK. So, each FCLK advances simulation time by $2 * \text{phase_delay}$.
- ❑ In the 2X mode, only one edge of this clock is executed in a single FCLK. So, each FCLK advances simulation time by phase_delay .

Using the Base Clock Generator `ixc_hwclksrc` for Uncontrolled Clocks

The `ixc_hwclksrc` module is used for modeling the free-running uncontrolled clock generator.

```
ixc_hwclksrc(output uclk, input enable);
```

For software simulation, the output `uclk` is a dangling wire that never toggles, and hot swap is not supported with the use of uncontrolled clock. Refer to [Dynamic Swap between Hardware and Software in SA Mode](#) section for details about the hot swap feature.

The uncontrolled clock has no effect on simulation time of the controlled clock domain.

The output `uclk` toggles every FCLK after swapping into hardware as long as the `enable` is asserted. In 2X mode, the `uclk` runs at half the speed of FCLK. In 1X mode, the `uclk` runs at the same speed as FCLK. A design must have at least one IXCOM-controlled clock generator (IXCclkgen) to have an uncontrolled clock generator. Having only an uncontrolled clock generator is not allowed.

You can set the `enable` input to `ixc_hwclksrc` to 0 in order to disable (or turn off) the uncontrolled clock from the hardware logic. Typically, this would be a constant 1 in your logic. The uncontrolled clock can then be stopped at run time by using SDL triggers and XEL commands.

Specifying Time Unit of the Controlled Clock Generators

In the `IXCclkgen`-clock-instantiating module, use the same time unit and time precision for timescale specification and define ``IXCclkgenTs` to the same timescale specification. The SystemVerilog time literal can also be used to specify the floating-point `phase_delay` of `IXCclkgen`.

For example:

```
`timescale 100ps/100ps
`define IXCclkgenTs 100ps/100ps
module clkgen;
  wire clk1;
  IXCclkgen1x #(2.5ns) c0 (clk1);
endmodule
```

Specifying CAKE Clocks by Frequency Ratio

It is necessary to specify the fastest base clock by `IXCclkgen` by using phase delay as parameter. However, you can use the `IXCcake2x` and `IXCcake1x` modules defined in the `IXCclkgen1x.sv` file to align all other CAKE clocks to the base clock by frequency ratio.

The `IXCcake1x` module matches the CAKE 1X mode alignment algorithm. The `IXCcake2x` matches the CAKE 2X mode alignment algorithm.

```
IXCcake2x #(FREQ, BFREQ) (clkOut, baseClock);
IXCcake1x #(FREQ, BFREQ) (clkOut, baseClock);
```

For example, if the base clock is running for 1GHz and two CAKE1X clocks are running at 433 MHz and 677MHz, respectively, specify these as follows:

```
`timescale 100ps/100ps
`define IXCclkgenTs 100ps/100ps

module clkgen;
  wire mclk, clk1, clk2, clk3;
  IXCclkgen #(0.5ns) cm (mclk);
  IXCcake1x #(433, 1000) f1 (clk1, mclk);
  IXCcake1x #(677, 1000) f2 (clk2, mclk);
endmodule
```

The frequency must be specified by integer. If there is another clock at 67.555MHz, scale the master clock to KHz:

```
IXCcake1x #(67555, 1000000) f3 (clk3, mclk);
```

Notice that the frequency scaling to integer is local to each CAKE clock-generator instance. The frequency ratios are used to generate and align CAKE clocks, and the absolute unit - KHz or MHz does not matter. For example, `IXCcake1x #(500, 1000)` is the same as `IXCcake1x #(1, 2)`. This means that the frequency of the output CAKE clock is 50% of the input base clock. However, only 32-bit unsigned integers are allowed for frequency-ratio specification. No frequency parameter should exceed the maximum allowed integer.

You can also use `IXCcake1x` and `IXCcake2x` to generate the CAKE clocks aligned to the uncontrolled clock.

For example:

```
ixc_hwclksrc uclkgen(uclk, 1'b1);

IXCcake2x #(1500, 2000) d1(clk7, uclk); //clk7 frequency is 3/4 of uclk frequency
ixc_clkbind f1( top.clk7 , clk7);

IXCcake1x #(30, 2000) d2(clk8, hclk); //clk8 frequency is 3/200 of uclk frequency
ixc_clkbind f2( top.clk8 , clk8);
```

Connecting CAKE Clock Signals to DUT and Automatically Disconnecting Other DUT Clock Drivers

Use the `ixc_clkbind` module defined in `IXCclkgen.sv` to connect a CAKE clock signal to DUT by cross-module reference.

For example:

```
ixc_clkbind b0( top.clkm, mclk);  
ixc_clkbind b1( top.clk1, clk1);
```

The `ixc_clkbind` provides the force semantics (disconnecting the existing DUT clock drivers) of the CAKE clocks. The `ixc_clkbind` has two ports. Connect the first port to the DUT clock signal (force destination). Connect the second port to the clock source signal generated from the CAKE modules (force source).

Changing CAKE Clock Parameters at Run Time without Needing Recompilation

The base clock parameters (`IXCclkgen`) cannot be changed at run time. However, the parameters of all other CAKE modules (`IXCcake1x` and `IXCcake2x`) can be changed. For all these CAKE modules, the static-module parameters (`FREQ`) are fixed at the compile time. The run-time internal variables (`freq`) are initiated to the static parameter (`FREQ`), and are programmable at run time.

The `${UXE_HOME}/etc/ixcom/IXCclkgen.sv` library file contains all the source code for the CAKE modules.

For example, the `xeDebug deposit` command can be used to program a CAKE clock at run time:

```
xeDebug> deposit clkgen.fl.freq = 500
```

Note that all the run-time variables are 32-bit integers.

Using userData to Create a Clock-Generation Module for SA Mode

This section describes how ICE clocks can be automatically generated while using the `IXCOM` compiler in SA mode by using the `ixclkgen` utility.

The automatic generation of the ICE clocks is done by a standalone utility called `ixclkgen`, which is provided in the UXE installation directory to generate clocks from User Data. This utility accept the XEL User Data as input, and generates a Verilog module containing the modeled clocks.

UXE User Guide

Modeling Clocks

In the ICE mode, the design clocks are specified using the XEL User Data commands instead of through the HDL description. Then, the ICE compiler automatically generates the clocks for the design based on the User Data specification, such as the following:

- ❑ Clock-generation technology (edge alignment) and sampling
- ❑ Clock specification using relative frequency
- ❑ Automatic driver-logic insertion to disconnect the original driver for the clock source in the design

In the SA mode, you need to specify the design clocks by using the RTL model. You can either describe the clocks using always blocks or use the *clock-generation module(s)* provided as a part of the UXE software in the installation directory (<UXE_Install_Dir>/etc/ixcom/IXCclkgen.sv) to model ICE-clock-generation technology. The modeled clock(s) are compiled with the original RTL as a part of a side file (in Verilog or System Verilog).

The *ixclkgen* utility supports the ICE-clock generation and enables you to:

- Specify the clocking information using ICE User Data (or command) syntax.
- Generate uncontrolled (or free running) clocks as in ICE mode.
- Automatically generate the Verilog module to model clocks for the information that you specified by using the ICE User Data as described in the section [Clock Module Generation by ixclkgen Utility](#) on page 1310.

To use the *ixclkgen* utility, use the following command:

```
ixclkgen -input <file> \  
-output <file>\   
[ -controlled <clock_source>[,<clock_source> ...] ] \  
[ -default <controlled | uncontrolled> ] \  
[ -external_netlist <string> ] \  
[ -help ] \  
[ -hierarchy <string> ] \  
[ -module <name> ] \  
[ -rtc ] \  
[ -spd ] \  
[ -uncontrolled <clock_source>[,<clock_source> ...] ]
```

Refer to the description of the *ixclkgen* command in the Commands and System Tasks for the SA Mode chapter of the UXE Command Reference manual for details of the options of

UXE User Guide

Modeling Clocks

the `ixclkgen` command. Some options with their descriptions in brief are listed in the following table.

<code>-input <file></code>	Specifies the file that contains the clocking User Data in XEL format. See the Specifying Clocking Information section for a list of the User Data that can be specified to <code>ixclkgen</code> .
<code>-output <file></code>	Specifies the output filename for the generated clock module.
<code>-controlled <clock_sources></code>	Specifies the controlled clock sources as comma-separated names.
<code>-default <controlled uncontrolled></code>	Specifies the default clock-generation behaviour for the clock sources not listed explicitly using the <code>-controlled</code> or <code>-uncontrolled</code> option. The default behavior for the clock source is controlled.
<code>-external_netlist <string></code>	Specifies the hierarchical path to the external netlist module instance and module name. The syntax for argument value is: [<code><instance_hierarchy></code>]= <code><module></code> . This option generates the <code><module>-shell.inc</code> file with all the required clock-bind statements that can be included inside the shell module for the netlist module.
<code>-hierarchy <string></code>	Specifies the hierarchical path for the clock signals referred in the User Data.
<code>-module <name></code>	Specifies the name of the Verilog module for the clock-generation module. If you do not specify this option, the <code>ixclkgen</code> utility uses the default module name, which is <code>_ixc_clkgen</code> .
<code>-rtc</code>	Generates the internal master clock to enable the User-clock configuration at run time using <code>clockConfig</code> command.
<code>-uncontrolled <clock_sources></code>	Specifies the uncontrolled or free-running clock sources as comma-separated names. The default behavior for the clock source is controlled.

The generated Verilog module uses the clock-generation modules defined either in the following directory:

```
<UXE_installation_directory>/etc/ixcom/IXCclkgen.sv
```

Or in the following file, if you specified the `-spd` option with the `ixclkgen` command:

```
<UXE_installation_directory>/etc/ixcom/SPDclkgen1x.v
```

UXE User Guide

Modeling Clocks

You need to compile the generated Verilog module file with the original RTL source files by using the IXCOM tools (vlan, vhan, ixcom) either in single-step or multi-step compilations as shown below:

- To generate and compile the clock module as the top-level DUT design unit in a single-step IXCOM compile, use the following command:

```
ixclkgen -input ud.qel -output clk.sv -module Clk
ixcom <source_files_and_options> -ua +dut+Clk clk.sv \
    -v <UXE_install_directory>/etc/ixcom/IXCclkgen.sv \
    -timescale <unit_or_precision>
```

- To generate and compile the clock module as the top-level DUT design unit in multi-step IXCOM compile, use the following command:

```
ixclkgen -input ud.qel -output clk.sv
vlan <source_files_and_options>
vlan [options] clk.sv -v <UXE_install_directory>/etc/ixcom/IXCclkgen.sv
ixcom [options] -ua +dut+_ixc_clkgen -top _ixc_clkgen -top <du> \
    -timescale <unit_or_precision>
```

Use `ixclkgen -help` for more examples with additional options to the utility.

The *ixclkgen* utility inserts the timescale directive into the clock-generation module. The IXCOM compiler follows the language standard to process the `timescale` directive. Both single-step and multi-step compilation require explicit specification of the `timescale` for the design. For single-step compilation, specify the generated clock-generation module as the last source file. To use CAKE 1 clocking technology, specify `+lxua` option with the `ixcom` command.

Specifying Clocking Information

The syntax and format for specifying the XEL User Data (or command) is the same as in the ICE mode. The following subset of XEL User Data (commands) are supported for specifying clocking information when using the *ixclkgen* utility:

- `clockOption`: Specifies the clock-generation technology (CAKE or SCM) and oversampling.

```
clockOption -add { technology CAKE N } // CAKE oversampling value N
                                         // specified must be 1 or higher.

clockOption -add { activeEdge both | positive | negative } // Supported

clockOption -add { ignoreOutput on | off } // Supported, but ignored by
the                                         // ixclkgen utility with an
                                         // information message

clockOption -add { ignoreEvent on | off } // Supported, but ignored by the
```


UXE User Guide

Modeling Clocks

```
// ixclkgen utility with an
// information message
```

- **clockSource:** Selects the net (or signal) in the design as clock source. This User Data also disconnects the original driver logic on the net (or signal) when driving with the CAKE-or SCM-generated clock.

```
clockSource -add <clock_source> // Supported
clockSource -rm <clock_source> // Supported
```

Note: This User Data has no effect on the generated code.

```
primaryClockSource -add <clock_source> // Supported
primaryClocksource -rm <clock_source> // Supported
```

Note: This User Data has no effect on the generated code.

- **clockFrequency:** Specifies frequency for the given clock source

```
clockFrequency -add { <clock_source> <frequency> } // Supported
clockFrequency -rm <clock_source> //Supported
```

- **clockAssign:** Specifies the derivation of the behavior of one signal from the behavior of another signal

```
clockAssign -add { <clock_signal_1> [ ~ ] // Supported
                  <clock_signal_2> [ N ] } // Supported
```

- **clockDelay:** Adds, removes, or modifies the clock delay value, which can be a whole number, greater or equal to zero. *<nDly>* is interpreted as the delay in FCLK cycles). You cannot use this command if the design is compiled for 1X mode.

```
clockDelay -add { <clock_source> <nDly> //Supported
clockDelay -rm { <clock_source> } //Supported
```

The *ixclkgen* utility supports the following ICE-mode features:

- ❑ Hierarchical name is allowed for clock signals or nets referred in *clockSource*, *primaryClockSource*, *clockFrequency*, *clockAssign*, and *clockDelay* User Data.
- ❑ XEL-style comment lines can be specified using the # symbol.
- ❑ XEL-style double-quote (") can be specified as delimiters in place of open-braces and close-braces.

The following limitations apply to the content in the input XEL file:

- ❑ IXCOM does not support CAKE clocking with 0.5x oversampling or SCM technology. The User Data specification for this clocking technology is reported as unsupported.
- ❑ For all User Data, which is valid, but is unsupported and, therefore, ignored by the *ixclkgen* utility, the utility writes out the User Data into a file named *compilerOptions_ixclkgen.qel*. You can include or source this file from

the `compilerOptions.gel` file, which is currently passed with the `ixcom` command during the IXCOM compilation process.

Clock Module Generation by `ixclkgen` Utility

The *ixclkgen* utility processes the User Data commands and generates a Verilog module with the modeled clocks to be compiled with the IXCOM compiler. These steps include:

- ❑ Determining the clocking technology and oversampling from the input User Data (`clockOption`).
- ❑ Converting the clock-frequency value to phase-length values and scale to module timescale.
- ❑ Collecting the clocking delay values specified.
- ❑ Mapping the information specified above to generate clocks IXCOM primitives.
- ❑ Generating the HDL to disconnect the driver logic for the clock source and drive the original clock source with the generated clock.
- ❑ Generating the HDL for clock assign (if the `clockAssign` User Data is specified).
- ❑ Generating the information about clocks for use at run time.
- ❑ Generating the clock-bind statements to connect the clock signals in the external netlist module hierarchy (if the `-external_netlist` option is specified with the `ixclkgen` command)

The following sections describe the clock-module generation by the `ixclkgen` utility for different User Data command sets.

For all the modules and primitives mentioned in the following examples, refer to the following file:

```
<UXE_installation_directory>/etc/ixcom/IXCclkgen.sv
```

Note that in the releases prior to UXE 11.1 QSR1, you need to refer to `SPDclkgen1x.v` instead of `IXCclkgen.sv`.

Using **CAKE 1X Technology**

The *ixclkgen* utility generates the CAKE 1X clocks if the input file contains the following set of User Data:

```
clockOption -add { technology CAKE 1 }  
clockFrequency -add { <clock_source> <frequency> }
```

UXE User Guide

Modeling Clocks

Following is an illustration of the generation of the CAKE 1X clock module using a sample set of User Data:

Command:

```
ixclkgen -input clock-ud.qel -output clk.sv
```

Input File:

```
# File: clock-ud.qel

clockOption -add { technology CAKE 1 }

clockFrequency -add { dut.clkA 500 MHz }
clockFrequency -add { dut.clkB 300 MHz }
```

Output File:

```
// This is an auto-generated file using the ixclkgen utility.
//   Input   File: clock-ud.qel
//   Output  File: clk.sv
// NOTE: Please refer to UXE user guide for clock modeling in IXCOM. Try:
//   ixclkgen -help
//   for usage and quick guide to compiling generated clock module.
////////////////////////////////////

`timescale 1 ns / 1 ns
`define IXCclkgenTs 1 ns / 1 ns
module _ixc_clkgen;
  wire clkA_0;
  wire clkB_1;

  initial $ixc_ctrl("map_delays");

  // Generate logic for clock sources
  IXCclkgen  #(1) ixcg_0 (clkA_0, );
  IXCcakelx  #(300, 500, 0) ixcg_1 (clkB_1, clkA_0);

  // Bind clock sources to generated clock signal
  ixc_clkbind ixcb_0 (dut.clkA, clkA_0);
  ixc_clkbind ixcb_1 (dut.clkB, clkB_1);
  // ...
endmodule
```

Important

If the clocking technology is not specified, by default the technology is set to CAKE 2X.

Using CAKE 2X Technology

The *ixclkgen* utility generates CAKE 2X clocks if the input file contains the following set of User Data:

```
clockOption -add { technology CAKE N } # N = 2
clockFrequency -add { <clock_source> <frequency> }
```

UXE User Guide

Modeling Clocks

Following is an illustration of the generation of the CAKE 2X clock module using a sample set of User Data:

Command:

```
ixclkgen -input clock-ud.qel -output clk.sv
```

Input File:

```
# File: clock-ud.qel

clockOption -add { technology CAKE 2 }

clockFrequency -add { dut.clkA 500 MHz }
clockFrequency -add { dut.clkB 300 MHz }
```

Output File:

```
// This is an auto-generated file using the ixclkgen utility.
// Input File: clock-ud.qel
// Output File: clk.sv
// NOTE: Please refer to UXE user guide for clock modeling in IXCOM. Try:
// ixclkgen -help
// for usage and quick guide to compiling generated clock module.
////////////////////////////////////

`timescale 1 ns / 1 ns
`define IXCclkgenTs 1 ns / 1 ns
module _ixc_clkgen;
  wire clkA_0;
  wire clkB_1;

  initial $ixc_ctrl("map_delays");

  // Generate logic for clock sources
  IXCclkgen #(1) ixcg_0 (clkA_0, );
  IXCcake2x #(300, 500, 0) ixcg_1 (clkB_1, clkA_0);

  // Bind clock sources to generated clock signal
  ixc_clkbind ixcb_0 (dut.clkA, clkA_0);
  ixc_clkbind ixcb_1 (dut.clkB, clkB_1);
  // ...
endmodule
```

Note: In the SA mode, the IXCOM compiler maps the higher oversampling value (CAKE N > 2) to CAKE 2X.

Using SCM Technology

In the ICE mode, the following User Data commands are used to generate SCM clocks. However, because SCM is not supported by the *ixclkgen* utility, it reports an error for the following User Data.

```
clockOption -add { technology SCM }
```

Specifying the Active Clock Edge

The *ixclkgen* utility supports active-edge specification using the following User Data:

UXE User Guide

Modeling Clocks

```
clockOption -add { activeEdge both | positive | negative }
```

This User Data has an effect only with CAKE 1X technology.

- ❑ If the above User Data is not specified, by default, the active edge is set to positive.
- ❑ Just like in ICE mode, specifying the active edge as `both` has the same effect as specifying positive active edge.
- ❑ If active edge is specified as negative, the *ixclkgen* utility uses the `IXCcake1xN` primitive as shown in the example below.

Command:

```
ixclkgen -input clock-ud.qel -output clk.sv
```

Input File:

```
# File: clock-ud.qel

clockOption -add { technology CAKE 1 }

clockOption -add { activeEdge negative }

clockFrequency -add { dut.clkA 500 MHz }
clockFrequency -add { dut.clkB 300 MHz }
```

Output File:

```
// This is an auto-generated file using the ixclkgen utility.
// Input File: clock-ud.qel
// Output File: clk.sv
// NOTE: Please refer to UXE user guide for clock modeling in IXC. Try:
// ixclkgen -help
// for usage and quick guide to compiling generated clock module.
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
`timescale 1 ns / 1 ns
`define IXCcclkgenTs 1 ns / 1 ns
module _ixc_clkgen;
wire clkA_0;
wire clkB_1;

    initial $ixc_ctrl("map_delays");

    // Generate logic for clock sources
    IXCcclkgen #(1) ixcg_0 (clkA_0, );
    IXCcake1xN #(300, 500, 0) ixcg_1 (clkB_1, clkA_0);

    // ...
    // See Using CAKE 1X Technology section for the rest of the module content.
endmodule
```

Specifying Clock Delays

Clock-delay specification is supported using the following User Data syntax:

```
clockDelay -add { <clock_source> <delay> }
clockDelay -rm { <clock_source> }
```

UXE User Guide

Modeling Clocks

In the generated clock module, the delay value is applied to the clock signal as a parameter in the `IXCcake1X`, `IXCcake1N`, and `IXCcake2x` primitive instances for the corresponding clock signal.

The following example shows the generated clock module with delay specification:

Command:

```
ixclkgen -input clock-ud.qel -output clk.sv
```

Input File:

```
# File: clock-ud.qel

clockOption -add { technology CAKE 2 }

clockFrequency -add { dut.clkA 500 MHz }
clockFrequency -add { dut.clkB 300 MHz }

clockDelay -add { dut.clkB 25 }
```

Output File:

```
// This is an auto-generated file using the ixclkgen utility.
// Input File: clock-ud.qel
// Output File: clk.sv
// NOTE: Please refer to UXE user guide for clock modeling in IXC. Try:
// ixclkgen -help
// for usage and quick guide to compiling generated clock module.
////////////////////////////////////

`timescale 1 ns / 1 ns
`define IXCclkgenTs 1 ns / 1 ns
module _ixc_clkgen;
  wire clkA_0;
  wire clkB_1;

  initial $ixc_ctrl("map_delays");

  // Generate logic for clock sources
  IXCclkgen #(1) ixcg_0 (clkA_0, );
  IXCcake2x #(300, 500, 25) ixcg_1 (clkB_1, clkA_0);

  // ...
endmodule
```

For CAKE 1X technology, the clock delay value is applied to the `IXCcake1x` and `IXCcake1xN` primitive instances similar to the example shown above.

Specifying Clock Assignment

Clock assignment specification is supported using the following User Data syntax:

```
clockAssign -add { <clock_signal-1> [ ~ ] <clock_signal-2> [ N ] }
```

Following is the generated clock module for a sample set of User Data with different clock assignments:

Command:

UXE User Guide

Modeling Clocks

```
ixclkgen -input clock-ud.qel -output clk.sv
```

Input File:

```
# File: clock-ud.qel

clockOption -add { technology CAKE 1 }

clockFrequency -add { dut.clkA 0.5 Ghz }

clockAssign -add { dut.clkB      dut.clkA      }
clockAssign -add { dut.clkC    ~ dut.clkA      }
clockAssign -add { dut.clkD      dut.clkA 2     }
clockAssign -add { dut.clkE      dut.clkA 4     }
clockAssign -add { dut.clkF    ~ dut.clkA 8     }
```

Output File:

```
// This is an auto-generated file using the ixclkgen utility.
//   Input File: clock-ud.qel
//   Output File: clk.sv
// NOTE: Please refer to UXE user guide for clock modeling in IXCOC. Try:
//   ixclkgen -help
//   for usage and quick guide to compiling generated clock module.
////////////////////////////////////

`timescale 1 ns / 1 ns
`define IXCclkgenTs 1 ns / 1 ns
module _ixc_clkgen;
  wire clkA_0;

  initial $ixc_ctrl("map_delays");

  // Generate logic for clock sources
  IXCclkgen #(1) ixcg_0 (clkA_0, );

  // Bind clock sources to generated clock signal
  ixc_clkbind ixcb_0 (dut.clkA, clkA_0);

  // Generate logic for clock assigns
  ixc_lxbuflp ixca_0 (dut.clkB, clkA_0);

  ixc_lxbufln ixca_1 (dut.clkC, ~ clkA_0);

  wire clkD_3;
  ixc_posedge_clk_divider #(2, 1) ixcd_2 (clkD_3, clkA_0);
  ixc_lxbuflp ixca_2 (dut.clkD, clkD_3);

  wire clkE_4;
  ixc_posedge_clk_divider #(4, 2) ixcd_3 (clkE_4, clkA_0);
  ixc_lxbuflp ixca_3 (dut.clkE, clkE_4);

  wire clkF_5;
  ixc_negedge_clk_divider #(8, 3) ixcd_4 (clkF_5, clkA_0);
  ixc_lxbufln ixca_4 (dut.clkF, clkF_5);

  // ...
endmodule

module ixc_posedge_clk_divider(output clkOut, input clkIn);
  parameter Divisor = 2;
  parameter Bits = 1;
```

UXE User Guide

Modeling Clocks

```
reg [Bits-1:0] counter = Divisor / 2;
reg clkOut = 0;

always @(posedge clkIn)
begin
    if (counter == 1)
    begin
        clkOut = ~clkOut;
        counter = Divisor / 2;
    end
    else
        counter = counter - 1;
    end
end

endmodule // _ixc_posedge_clkgen_divider

module ixc_negedge_clk_divider(output clkOut, input clkIn);
    parameter Divisor = 2;
    parameter Bits = 1;

    reg [Bits-1:0] counter = Divisor / 2;
    reg clkOut = 1;

    always @(negedge clkIn)
    begin
        if (counter == 1)
        begin
            clkOut = ~clkOut;
            counter = Divisor / 2;
        end
        else
            counter = counter - 1;
        end
    end

endmodule // _ixc_negedge_clkgen_divider
```

Note: In the above example, the CAKE 1X clock is generated to show the variants of generated logic. The clock signal referenced in the `clockAssign` User Data can be any of the design signals; it does not have to be driven by a clock source.

Using Uncontrolled Hardware Clocks

The default behavior for CAKE clocks is controlled if you are using the STB mode or with a static or no target in ICE mode. On the other hand, the clocks are uncontrolled (or free running) in the ICE mode if you use a dynamic target (`configPM -la` command).

In the SA mode, all clocks are generated as controlled clocks. To generate ICE-mode-like uncontrolled clocks, you must specify a list of uncontrolled clocks by using the `-uncontrolled` option of the `ixclkgen` command as shown in the example below:

Command:

```
ixclkgen -input clock-ud.gel -output clk.sv \
-uncontrolled dut.clkB -uncontrolled dut.clkC,dut.clkD
```

Input File:

UXE User Guide

Modeling Clocks

```
# File: clock-ud.qel

clockOption -add { technology CAKE 2 }

clockFrequency -add { dut.clkA 677 MHz }
clockFrequency -add { dut.clkB 433 MHz }
clockFrequency -add { dut.clkC 344 MHz }
clockFrequency -add { dut.clkD 500 MHz }
clockFrequency -add { dut.clkE 545 MHz }
```

Output File:

```
// This is an auto-generated file using the ixclkgen utility.
//   Input File: clock-ud.qel
//   Output File: clk.sv
// NOTE: Please refer to UXE user guide for clock modeling in IXCOM. Try:
//   ixclkgen -help
//   for usage and quick guide to compiling generated clock module.
///////////////////////////////////////////////////////////////////
`timescale 1 ps / 1 ps
`define IXCclkgenTs 1 ps / 1 ps
module _ixc_clkgen;
  wire clkA_0;
  wire clkB_1;
  wire clkC_2;
  wire clkD_3;
  wire clkE_4;

  initial $ixc_ctrl("map_delays");

  // Generate logic for clock sources
  IXCclkgen #(738) ixcg_0 (clkA_0, );
  IXCcake2x #(433, 500, 0) ixcg_1 (clkB_1, clkD_3);
  IXCcake2x #(344, 500, 0) ixcg_2 (clkC_2, clkD_3);
  ixc_hwclksrc ixcg_3 (clkD_3, 1'b1);
  IXCcake2x #(545, 677, 0) ixcg_4 (clkE_4, clkA_0);

  // Bind clock sources to generated clock signal
  ixc_clkbind ixcb_0 (dut.clkA, clkA_0);
  ixc_clkbind ixcb_1 (dut.clkB, clkB_1);
  ixc_clkbind ixcb_2 (dut.clkC, clkC_2);
  ixc_clkbind ixcb_3 (dut.clkD, clkD_3);
  ixc_clkbind ixcb_4 (dut.clkE, clkE_4);

  //...
endmodule
```

Note: The default clock generation behavior for the clock sources can be specified separately using the `-default` option. Use `ixclkgen -help` for more details.

Specifying Hierarchy

In the ICE mode, a reference clock signal (or source) in the User Data does not include the top-level module information. Also, in the SA mode, you may want to instantiate generated clock module in testbench hierarchy relative to the design referred by the User Data.

UXE User Guide

Modeling Clocks

The following tool invocation shows the use of the `-hierarchy` option to insert the hierarchy that you have specified for the clock signals referred in the User Data file:

Command:

```
ixclkgen -input clock-ud.qel -output clk.sv -hierarchy "Tb.dut."
```

Input File:

```
# File: clock-ud.qel

clockOption -add { technology CAKE 2 }

clockOption -add { technology CAKE 2 }
clockFrequency -add { clkA 100 MHz }
clockFrequency -add { sub.clkB 125 Mhz }
```

Output File:

```
// This is an auto-generated file using the ixclkgen utility.
// Input File: clock-ud.qel
// Output File: clk.sv
// NOTE: Please refer to UXE user guide for clock modeling in IXCOM. Try:
// ixclkgen -help
// for usage and quick guide to compiling generated clock module.
////////////////////////////////////
`timescale 1 ns / 1 ns
`define IXCclkgenTs 1 ns / 1 ns
module _ixc_clkgen;
  wire _clkA_0;
  wire clkB_1;

  initial $ixc_ctrl("map_delays");

  // Generate logic for clock sources
  IXCcake2x #(100, 125, 0) ixcg_0 (clkA_0, clkB_1);
  IXCclkgen #(4) ixcg_1 (clkB_1, );

  // Bind clock sources to generated clock signal
  ixc_clkbind ixcb_0 (Tb.dut.clkA, clkA_0);
  ixc_clkbind ixcb_1 (Tb.dut.sub.clkB, _clkB_1);
  // ...
endmodule
```

Importing External Netlists

The IXCOM compiler allows an external netlist to be imported by bypassing the conventional compile steps. In this use model, the IXCOM compiler does not have the visibility into objects inside the external-netlist-module hierarchy. For more details on this use model, refer to [Importing External Netlists into IXCOM Compiler](#).

For this use model, the *ixclkgen* utility allows you to specify the information about the hierarchical path to the external netlist module instance and the module name by using the `-external_netlist` option. This option also generates the `<module_name>-shell.inc` file, which you can include in the shell-file for the external netlist module.

UXE User Guide

Modeling Clocks

Following is an illustration of the generation of the clock module and include file with clock-signal references to the external netlist module hierarchy:

Command:

```
ixclkgen -input clock-ud.qel -output clk.sv -external_netlist "Tb.top.dut=Dut"
```

Input File:

```
# File: clock-ud.qel
# Design hierarchy information
# Tb - testbench
# Top - top-level h/w module, instantiated under Tb
# Sub - non-top h/w module, instantiated under Top
# Dut - non-top external netlist h/w module, instantiated under Top
# Chk - internal hierarchy within netlist h/w module
# Clk - top-level h/w module (generated clock module)
clockOption -add { technology CAKE 2 }
# Clock source in h/w module and reference to it in h/w module
clockFrequency -add { Tb.top.clk 500Mhz }
clockAssign -add { Tb.top.sub.clk Tb.top.clk }
# Clock source in external netlist h/w module and reference to it in h/w module
clockFrequency -add { Tb.top.dut.clk 300Mhz }
clockAssign -add { Tb.top.sub.clkn ~ Tb.top.dut.clk }
# Clock source and reference within external netlist h/w module
clockAssign -add { Tb.top.dut.chk.clkn ~ Tb.top.dut.clk }
```

Output File:

```
// This is an auto-generated file using the ixclkgen utility
// Input File: ./clock-ud.qel
// Output File: clk.sv
// NOTE: Please refer to UXE user guide for clock modeling in IXC. Try:
// ixclkgen -help
// for usage and quick guide to compiling generated clock module.
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
`timescale 1 ns / 1 ns
`define IXCclkgenTs 1 ns / 1 ns
module _ixc_clkgen;
    wire _clk_0;
    wire clk_1;
    initial $ixc_ctrl("map_delays");
    // Generate logic for clock sources
    IXCclkgen #(1) ixcg_0 (clk_0, );
    IXCcake2x #(300, 500, 0) ixcg_1 (clk_1, clk_0);
    // Bind clock sources to generated clock signal
    ixc_clkbind ixcb_0 (Tb.top.clk, clk_0);
    ixc_clkbind ixcb_1 (Tb.top.dut.clk_1, clk_1);
    // Generate logic for clock assigns
    ixc_clkbind ixca_0 (Tb.top.sub.clk, clk_0);
    ixc_clkbind ixca_1 (Tb.top.sub.clkn, ~ Tb.top.dut.clkn_3_r);
    ixc_clkbind ixca_2 (Tb.top.dut.clkn_4_w, ~ Tb.top.dut.clkn_4_r);
    // ...
endmodule
// This is an auto-generated file using the ixclkgen utility.
// NOTE: This file contains logic for binding shadow signals in the shell module
// to actual signals in the external netlist module (imported as blackbox).
Use:
// `include "Dut-shell.inc"
// to insert this logic into shell module declaration.
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
wire clk_1;
```

UXE User Guide

Modeling Clocks

```
wire clk_n_3_r, clk_n_3_w;
wire clk_n_4_r, clk_n_4_w;
`ifndef AXIS
initial begin
    // Bind shadow clock signals to actual clock signals in the imported netlist
    $ixc_ctrl("hdlICE", "wire clk_1; ixc_clkbind ixcb_0 (i0.clk, clk_1);");
    $ixc_ctrl("hdlICE", "wire clk_n_3_r; ixc_bind ixsb_1 (i0.clk, clk_n_3_r);");
    $ixc_ctrl("hdlICE", "wire clk_n_4_r; ixc_bind ixsb_2 (i0.clk, clk_n_4_r);");
    $ixc_ctrl("hdlICE", "wire clk_n_4_w; ixc_clkbind ixcb_2 (i0.chk.clk_n,
                                                              clk_n_4_w);");
end
`endif
```

Note: If the instance hierarchy is not specified with the `-external_netlist` option, all references to the clock signals in the User Data are interpreted as netlist objects. See `ixclkgen -help` for usage details. You can specify the `-external_netlist` option multiple times to specify multiple netlist module instances, but you can specify only one top-level netlist module.

Hierarchical reference to a netlist clock signal referred in an input file must be an internal signal. Specifying netlist interface (IO) signals may result in a multi-driven clock error when generated module is compiled.

Specifying the Clock Configuration at Run Time with `ixclkgen` Utility

By default, the `ixclkgen` utility maps the design clocks to the minimum required clock-generation primitives. The generated clocks cannot be configured at run time. To enable run-time configuration of the design clocks using the `clockConfig` command, you must specify the `-rtc` option of the `ixclkgen` command.

To support run-time configurable user clocks, the `ixclkgen` utility generates an internal clock, `_ixci_clk` for controlled clocks and `_ixci_hclk` for uncontrolled clocks, with the fastest user-clock frequency and all the design clocks are generated relative to the generated internal clock.

Command:

```
ixclkgen -input clock-ud.qel -output clk.sv -rtc
```

Input File:

```
# File: clock-ud.qel

clockOption -add { technology CAKE 2 }

clockFrequency -add { dut.clkA 500 MHz }
clockFrequency -add { dut.clkB 300 MHz }
```

Output File:

```
// This is an auto-generated file using the ixclkgen utility.
//   Input File: clock-ud.qel
//   Output File: clk.sv
// NOTE: Please refer to UXE user guide for clock modeling in IXCOM. Try:
//   ixclkgen -help
//   for usage and quick guide to compiling generated clock module.
```

UXE User Guide

Modeling Clocks

```
////////////////////////////////////  
'timescale 1 ns / 1 ns  
'define IXCclkgenTs 1 ns / 1 ns  
module _ixc clkgen;  
  wire clkA_0;  
  wire clkB_1;  
  wire _ixci_clk;  
  
  initial $ixc_ctrl("map_delays");  
  
  // Generate logic for clock sources  
  IXCcake2x #(500, 500, 0) ixcg_0 (clkA_0, _ixci_clk);  
  IXCcake2x #(300, 500, 0) ixcg_1 (clkB_1, _ixci_clk);  
  IXCclkgen #(1) ixcg_2 (_ixci_clk, );  
  
  // Bind clock sources to generated clock signal  
  ixc_clkbind ixcb_0 (dut.clkA, clkA_0);  
  ixc_clkbind ixcb_1 (dut.clkB, clkB_1);  
  
  `ifdef AXIS  
    initial begin  
      $ua_cmd("cakeClk", "dut.clkA", "ixcg_0", "500MHz", "C ", "0");  
      $ua_cmd("cakeClk", "dut.clkB", "ixcg_1", "300MHz", "C ", "0");  
      $ua_cmd("cakeClk", "_ixci_clk", "ixcg_2", "500MHz", "B2", "0");  
    end  
  `endif  
endmodule
```

The following syntax is supported for the `clockConfig` command when the *ixclkgen* utility is being used:

```
clockConfig  
  [-sample <value>]  
  [-dly <clockName> <value>]  
  [-clk <clockName> <value>]  
  [-list]  
  [-restore] [<filename>]  
  [-save] [<filename>]  
  [-actual]
```

For details about the run-time clock configuration using the `clockConfig` command, refer to *UXE Command Reference Manual*.

Specifying the Clock Configuration

To support clock configuration at run-time using the `clockConfig` user command, the *ixclkgen* utility passes the information about the clocks by using the `$ua_cmd` statements in the generated code with the following syntax:

```
$ua_cmd("cakeClk", "<clock_path>", "<instance_name>", "<frequency>", "type",  
        "delay");
```

Here,

`cakeClk`: Specifies the fixed tag for the `ixc_ctrl` statement.

UXE User Guide

Modeling Clocks

<clock_path>: Specifies the hierarchical path to clock source from the User Data

<instance_name>: Specifies the IXC CAKE clock primitive instance name

<frequency>: Specifies the clock frequency value from User Data

type: Specifies one of the type values to indicate the cake clock properties or attributes

“B1”: Is used for controlled base clock 1X mode;

“B2”: Is used for controlled base clock 2X mode;

“C”: Is used for controlled CAKE clock;

“b”: Is used for uncontrolled ixc_hwclksrc

“c”: Is used for uncontrolled CAKE clock.

“delay”: Specifies the clock delay value (if any) from User Data

For the controlled and uncontrolled clocks referenced in the [Using Uncontrolled Hardware Clocks](#) section, additional HDL is generated in the Verilog module as shown below:

Command:

```
ixclkgen -input clock-ud.qel -output clk.sv
```

Output File:

```
// This is an auto-generated file using the ixclkgen utility.
// Input File: clock-ud.qel
// Output File: clk.sv
// NOTE: Please refer to UXE user guide for clock modeling in IXC. Try:
//   ixclkgen -help
//   for usage and quick guide to compiling generated clock module.
/////////////////////////////////////////////////////////////////
`timescale 1 ps / 1 ps
`define IXCclkgenTs 1 ps / 1 ps
module _ixc_clkgen;
  // See section Using Uncontrolled Hardware Clocks section for the rest of the
  // generated module content
  `ifdef AXIS
    initial begin
      $ua_cmd("cakeClk", "dut.clkA", "ixcg_0", "677MHz", "B2", "0");
      $ua_cmd("cakeClk", "dut.clkB", "ixcg_1", "433MHz", "c ", "0");
      $ua_cmd("cakeClk", "dut.clkC", "ixcg_2", "344MHz", "c ", "0");
      $ua_cmd("cakeClk", "dut.clkD", "ixcg_3", "500MHz", "b ", "0");
      $ua_cmd("cakeClk", "dut.clkE", "ixcg_4", "545MHz", "C ", "0");
    end
  `endif
endmodule
```

irun - IXCOM Option Mapping

When *irun* is used for simulation acceleration, it internally invokes INCISIV and IXCOM tools to compile the source files, generate the snapshot for simulation acceleration, and invoke the simulator. *irun* dispatches the specified command-line options to INCISIV and IXCOM tools. If you specify *irun* command options that have functionally equivalent behavior and either identical or non-identical syntax as IXCOM options, these *irun*-command options are mapped to IXCOM.

This appendix lists the INCISIV-tool options that are mapped to the IXCOM options when using *irun* for simulation acceleration in the following tables:

- ❑ Table H-1, “INCISIV tool and IXCOM Options with Identical Syntax and Functional Behavior,” on page 1324
- ❑ Table H-2, “irun and IXCOM Options with Different Syntax but Equivalent Functional Behavior,” on page 1326

Mapping irun Options to IXCOM Options with the Same Syntax and Behavior

The following table lists the INCISIV tool options are mapped to IXCOM options with identical syntax and functionally equivalent behavior.

Table H-1 INCISIV tool and IXCOM Options with Identical Syntax and Functional Behavior

INCISIV Tool Option	IXCOM Option
-append_log	-append_log
-assert	-assert
-assert_vlog	-assert_vlog
-assert_vhdl	-assert_vhdl
-clean	-clean
-coverage <string>	-coverage <string>
-default_ext <arg>	-default_ext <arg>
+define+<macro>	+define+<macro>
-dpiheader	-dpiheader
-dpi_void_task	-dpi_void_task
-f <file>	-f <file>
-F <file>	-F <file>
-help	-help
-incdir <path>	-incdir <path>
+incdir+<path>	+incdir+<path>
-l <file>	-l <file>
-lexpragma	+rtlCommentPragma Supports: ■ cadence/synopsys/pragma translate_off/on ■ cadence/synopsys/pragma synthesis_off/on Does not support -rtl_synthesis off/on

UXE User Guide

irun - IXCOT Option Mapping

INCISIV Tool Option	IXCOM Option
+libext+<extension>	+libext+<extension>
+liborder	+liborder
+librescan	+librescan
+libverbose	+libverbose
+loadplil+<library>	+loadplil+<library>
+loadvpi+<library>	+loadvpi+<library>
-lps_cpf <file>	-lps_cpf <file>
-makelib <arg>	-work <lib>
-neverwarn	-neverwarn
-noassert	-noassert
-override_precision	-override_precision
-override_timescale	-override_timescale
-q	-q
-reflib <arg>	-L <lib>
-smartorder	-sort
-sysv_ext [+]<extns>	-sysv_ext [+]<extns>
-timescale <arg>	-timescale <arg>
-top <du>	-top <du>
-v <file>	-v <file>
-v93	-93
-verbose	-verbose
-version	-version
-vhdl_time_precision <arg>	-vhdl_time_precision <arg>
-vlog95_ext [+]<extns>	-vlog95_ext [+]<extns>
-vlog_ext [+]<extns>	-vlog [+]<extns>

INCISIV Tool Option	IXCOM Option
-vtimescale <arg>	-vtimescale <arg>
-work <library>	-work <library>
-y <directory>	-y <directory>

Mapping irun Options with IXCOR Options with Different Syntax but the Same Behavior

The following table provides mapping details for INCISIV-tool options that have different syntax but functionally equivalent behavior as an IXCOR option.

Table H-2 irun and IXCOR Options with Different Syntax but Equivalent Functional Behavior

irun Option	IXCOR Option
-32bit	-32
-64bit	-64
-autoprotect	+autoprotect
-define <macro>	+define+<macro>
-loadplil <library>	+loadplil+<library>
-loadvpi <library>	+loadvpi+<library>
-propfile_vlog <file>	-propfile <file>
-propfile_vhdl <file>	-propfile <file>
-sv	+sv
-v1995	+v1995
-v200x	+v2000

See *Using irun for Simulation Acceleration* guide for more information on the `irun` command options.

Compiling and Running Designs with Memories

A memory is defined as a one-dimensional unpacked array of a variable with a one-dimensional packed datatype. In netlists, memory instances typically refer to elements that are not completely defined in either the netlist or the ASIC Library. This lack of definition leads to a blackbox description of these elements. Generally, a memory compiler generates ASIC memories, rather than providing a library definition for every possible memory configuration. Typically, memories are mapped into the xeCompile database during netlist import. While compiling designs with memories, ROM and RAM cell definitions created by the Netlist Importer can be generated either using the automatic memory process or manually.

Note: This chapter pertains to compiling and running designs with memories in the ICE mode. For the handling of memories in the SA mode using IXCOM, refer to the [Direct Access of DUT Memory Array and Register](#) section in [Chapter 4, “Compiling Designs for Simulation Acceleration Using IXCOM.”](#)

ASIC vendors generally use a memory compiler to generate ASIC memories, rather than providing a library definition for every possible memory configuration. The same method is used in adding memory cells to the Palladium XP database.

Memories can be described in the following ways:

- Memory RTL files

Memory RTL files are usually the best method for describing memories. Create a Memory RTL file written in Verilog or VHDL, and synthesize it using xeCompile. xeCompile automatically maps the memories to the corresponding Memory Primitives. xeCompile memory synthesis process is described in the [Synthesizing Designs with Memories](#) of [Chapter 3, “Compiling Designs for In-Circuit Emulation with xeCompile.”](#)

You can use xeCompile to synthesize Memory RTL files even if you do not use it to synthesize the design itself.

- Memory Primitives

The memory primitives are the lowest-level way to describe memories. These are also known as MPR (Memory Port Read) and MPW (Memory Port Write), and represent memory read and write ports, respectively. This approach works well for legacy designs that already describe memories using MPR and MPW primitives.

Note: Any memory which is not modeled as gates is actually modeled with MPR/MPW memory primitives. These primitives can be combined to create a memory with any number of read and write ports. However, these primitives can only be accelerated, not simulated, so a congruent simulation is not possible. Consult with the assigned Cadence AE before using these primitives.

In the ICE mode, a maximum of 63 address bits are supported at compile time and at run time the limit of memory support is defined by the available emulator resources. In the SA mode, the limit is 29 address bits.

Supported Memory File Formats

In emulation, the supported file formats are as follows:

- For ASCII:
 - ☐ PD ASCII (bin, oct, dec, hex)
 - ☐ readmemb and readmemh
 - ☐ NC_SIM
 - ☐ Denali memory format
- For binary:
 - ☐ memtran
 - ☐ raw
 - ☐ raw2

Transforming Memories

`et3compile` can transform some design memories. The goals of memory transformations are:

- Fit all design memories in emulator memory for the given hardware configuration.
- Increase the probability of successful compilation.

- Decrease the emulation step count.

The following types of memory transformations can be performed:

- Transform shallow memory instances into gate level models.
- Merge memory instances.
- Split memory instances with multiple read ports.
- Squeeze memory instances with constant address pins.
- Perform memory compaction.
- Perform sparse memory modeling for memory instances that you have requested.

You can force transformation of some memory instances and/or prevent transformation of other memory instances by use of the following `memoryTransform` command:

```
memoryTransform -add {<memory> <parameter_name> <parameter_value>}
```

To remove the memory instance name from User Data, use the following command:

```
memoryTransform -rm { <memory> [<parameter_name>] }
```

A list of transformed memories can be found in the `dbFiles/`
`<topcell>.memTransformList` file.

Some memory transformations are compatible (that is, can be performed on the same memory instance), and some are not. There are the following rules:

- A shallow memory instance represented as a gate array cannot (naturally) participate in other memory transformations.
- A memory instance that you have defined as sparse cannot participate in other memory transformations.
- Memory splitting, memory squeezing and memory compaction are fully compatible transformations. For example, memory M can first be split into 3 chains ($M \Rightarrow \{M1, M2, M3\}$), then each chain can be squeezed ($M1 \Rightarrow M1.1$, $M2 \Rightarrow M2.1$, $M3 \Rightarrow M3.1$), and finally chains M1.1 and M3.1 can be compacted.

Transforming Shallow Memory Instances into Gate Arrays

Some designs can contain big numbers of “shallow” memory instances. Transformation of shallow memories into gate-level models can be beneficial due to the following two reasons:

- It can allow to fit a design in the given hardware configuration if the bottleneck was a lack of memory capacity rather than gate capacity.

- It can decrease the step count.

The decision on which memory instances should be transformed is based on capacity considerations and on pursuit of the minimal estimated emulation memory access.

Memory instances that you have explicitly defined and instrumentation memory instances are not transformed.

To prohibit transformation of a shallow memory into gate array, use the following `memoryTransform` command:

```
memoryTransform -add {<memory> SHALLOW off}
```

To restore the default behavior, use the following command:

```
memoryTransform -add {<memory> SHALLOW on}
```

To remove the shallow memory transformation of the specified memory instance, use the following command:

```
memoryTransform -rm {<memory> SHALLOW}
```

To prohibit transformation of **all** shallow memories into gate arrays, use the following command:

```
memoryTransform -add {* SHALLOW off}
```

To restore default behavior for **all** shallow memories, use the following command:

```
memoryTransform -add {* SHALLOW on}
```

The number of shallow memory instances transformed into gates depends on the *shallow memory utilization factor* discussed below.

Setting the Shallow Memory Utilization Factor

By default, transformation of shallow memory into gates starts from the smallest memory instances and continues while gate utilization (in percents) is below the threshold value (80% by default). This value can be changed using the following command:

```
memoryTransform -add {* shallowMemoryUtilization <number>}
```

where *<number>* is a non-negative integer between 0 and 100.

Merging Memory Instances

If two memory ports have the same type (read or write), the same depth, the same address bus and they are independent of each other, they are called mergeable ports.

If several memory instances M1, M2, ..., Mk have the same number of ports and their corresponding ports are mergeable, they can be merged into memory instance M by placing their data signals side-by-side.

To prohibit merging of memory instances, use the following `memoryTransform` command:

```
memoryTransform -add {<memory> MERGE off}
```

To restore the default behavior, use the following command:

```
memoryTransform -add {<memory> MERGE on}
```

To remove the merge memory transformation of the specified memory instance, use the following command:

```
memoryTransform -rm {<memory> MERGE}
```

Splitting Memory Read Ports

Each memory instance is represented in the data base as a chain of Read (MPR) and Write (MPW) ports. Multi-port memory instances can be responsible for high step count. If a memory instance has more than one MPR, it is possible to decrease its chain length by using N identical memory storages (instead of the original one) and splitting the given chain into N chains as follows. Each chain contains the same MPW as in the original chain. In other words, each original Write port is “represented” in each chain. Each Read port (MPR) is represented in only one chain. The port order in each chain will be the same as in the original chain. Pins A, DI, BS, WE, DO of each “new” port will be connected to the same nets as the corresponding pins of the corresponding original port.

Such a splitting can decrease the step count for a design. The trade-off is an increase in the required hardware memory size.

Currently splitting can apply only to memory instances “destined” to be mapped to XDRAM and only if such a splitting would decrease the estimated maximum (among the physical XDRAM) number of accesses to a physical XDRAM.

To prohibit split memory transformation, use the following `memoryTransform` command:

```
memoryTransform -add {<memory> SPLIT off}
```

To restore the default behavior, use the following command:

```
memoryTransform -add {<memory> SPLIT on}
```

To remove the split memory transformation of the specified memory instance, use the following command:

```
memoryTransform -rm {<memory> SPLIT}
```

Squeezing Memory Instances with Constant Address Pins

If a memory instance has an address input pin tied (no matter whether the pin is tied up or down, but it is tied the same way for each memory port), such a pin can be removed. Removal of one address pin reduces the memory size two times, removal of two address pins reduces it four times, and so on (in general, removal of N address pins reduces the memory size 2^N times.) Such a “squeezing” can allow for some designs to save considerable resources of physical memory.

To prohibit memory squeezing, use the following `memoryTransform` command:

```
memoryTransform -add {<memory> SQUEEZE off}
```

To restore the default behavior, use the following command:

```
memoryTransform -add {<memory> SQUEEZE on}
```

To remove the memory squeeze transformation of the specified memory instance, use the following command:

```
memoryTransform -rm {<memory> SQUEEZE}
```

Compacting Memories

If a design memory instance with data width not equal to a multiple of 32 is mapped on emulator memory, a part of emulator memory space would be lost. Memory compaction transforms the design memory so that the emulator memory space can be used effectively.

Following types of memories cannot be compacted in this way:

- Memory instances with address bus width equal to 6 or less (that is, with depth equal to 64 or less)
- Memory instances with data bus width equal to a multiple of 64 (64, 128, 192, and so on)

The default behavior for memory compaction is as follows. The process browses the design database and collects memory instances. It then calculates the required DRAM size (D) and compares it with the available DRAM size (A). If $D \leq A$, no compaction is performed. Otherwise, it finds a minimum subset of memory instances, the compaction of which would reduce D to D_c where $D_c \leq A$. (If this is unachievable, ALL “compactible” memory instances will be transformed).

To prohibit memory compaction, use the following `memoryTransform` command:

```
memoryTransform -add {<memory> COMPACT off}
```

To force memory compaction, use the following command:

```
memoryTransform -add {<memory> COMPACT on}
```


UXE User Guide

Compiling and Running Designs with Memories

To remove the memory compaction of the specified memory instance, use the following command:

```
memoryTransform -rm {<memory> COMPACT}
```

The number of memory instances that can be compacted depends on the *memory utilization factor* discussed below.

Setting the Utilization Factor

To change the number of compacted memory instances, use the following command:

```
memoryTransform -add {* memoryUtilization <number>}
```

where *<number>* is a non-negative integer representing the utilization factor (U), given in percent for available emulator memory size.

Setting $U < 100$ will force more memory instances to be compacted; if $U = 0$, all compactible memory instances will be compacted because $A_m = 0$.

To remove the utilization factor from User Data, use the following command:

```
memoryTransform -rm {* memoryUtilization}
```

Handling Sparse Memories

Memory instances that have the following characteristics are referred to as sparsely used memories or sparse memories:

- ❑ A large/deep address range in their declaration
- ❑ Only a small percentage of the address range accessed during simulation or emulation

In conventional memory mapping, if very deep sparse memories are present, a design could require a very large amount of hardware resources. When the design with sparse memories, is accelerated in the emulator, for efficient use of emulator resources, instead of implementing the entire/declared address range for the sparse memory, only the percentage that is accessed during simulation should be implemented.

To handle sparse memories, the UXE software includes a strategy known as a *page model*. The page model requires much less memory space, compared to a conventional memory model. Refer to [Handling Sparse Memories through the Page Model](#) for details.

Handling Sparse Memories through the Page Model

The UXE software uses the Page Model to implement sparsely-used memories. This model is used to dynamically map only the accessed pages of a sparsely-used memory onto the emulation (hardware) memory. The fraction of the accessed pages should be small.

To understand the page model, consider that a sparsely-used memory M has a_w address bits, that is, its depth is equal to 2^{a_w} words. Consider the address space of M as a sequence of $K=2^k$ pages (blocks) of size 2^p each. Then, $a_w=k+p$. These 2^k pages are called *virtual pages*.

Note: ‘Words’ mean addressable words of the corresponding memory instance.

The page model consists of a pool memory which contains $N=2^n$ physical pages and a catalogue that maintains the mapping from virtual pages to physical pages. If a virtual page is not mapped to any physical page, then that virtual page is considered to have the default value.

Conditions When Sparse Memory Implementation Does Not Improve Capacity

It is possible that declaring a design memory as sparse can increase the amount of emulator memory required. In such a situation, the compiler will issue an error or warning:

- If the sparse memory implementation with the parameters provided consumes the same or more memory resources than the 'normal' implementation, an ERROR message is issued.

If the sparse memory implementation consumes less memory than 'normal' implementation but more than in the case of memory compaction, a WARNING message is issued.

Modeling Sparsely-Used Memories at Compile Time in ICE Mode

During design emulation, some addresses of each memory instance are accessed while others are not. If the percentage of accessed addresses for a memory instance is very low, the memory is said to be sparsely used.

Some designs contain very deep sparsely-used memories. The conventional way of mapping such memories onto emulator's memory can require huge hardware resources and become the bottleneck for emulation.

In the ICE mode, if you know that certain memories in your design are sparsely used, you can use the `memoryTransform` command at compile time to declare these memories as "*sparse*". For each sparsely-used memory, you specify a number of "pages" to be implemented physically in the emulator. If at run time, a test writes to more than this number of pages, the emulation stops with an error. (In LA mode, the emulation issues an error message but does not stop.) You can use the `memory -reset` command at run time to reset a sparsely-used memory, setting the memory contents to zero, and freeing all of its pages.

For detailed information on access to sparsely-used memories at run time, refer to the [Accessing Sparsely-Used Memories at Run Time in ICE Mode](#) section.

Declaring Sparsely-Used Memories in ICE Mode

Sparsely-used memories must be declared as sparse using the following XEL command:

```
memoryTransform -add {<name> SPARSE <n> <p>}
```

where,

<name> is a hierarchical name of the memory

<n> is used to calculate the number of physical pages, that is, Number of physical pages = 2^n

<p> is used to calculate the size of each page, that is, Number of words per page = 2^p

UXE User Guide

Compiling and Running Designs with Memories

This command specifies that a memory instance with name *<name>* should be treated as a sparsely-used memory which can use at most $N=2^n$ physical pages, and each page should contain 2^p words.

For example, the following command specifies that memory instance `top.M123` should be modeled as a sparsely-used memory with $2^M (2^{21})$ physical pages, and each page should be $64 (2^6)$ addresses deep (that is, contain 64 words):

```
memoryTransform -add {top.M123 SPARSE 21 6}
```

The `memoryTransform` command should precede the `compile` command in the compiler script.

<n> and *<p>* should be positive numbers such that $n+p < aw$, where *aw* is the number of address bits of the sparsely-used memory. If these rules are violated, a corresponding error message is issued. There are no default values for *<n>* and *<p>*. You must choose these values based on the knowledge of the design and understanding that the sparse model consists of two memory instances of the following sizes instead of the original memory size, $2^{aw} \times dw$:

- $2^{aw-p} \times cw$ (catalogue)

Note: The catalogue width *cw* is either 32 or 64, depending on the implementation.

- $2^{n+p} \times dw$ (pool)

For example, if a sparsely-used memory instance has size $2^{32} \times 64$ and you have defined *p*=9 (page size is 512 words) and *n*=16 (has 64K physical pages), then without 'sparse implementation', this instance will consume 4G of 64-bit wide words of the emulator memory. The sparse model will require 32M of 64-bit wide words for pool memory, and 8M of 64 or 32-bit wide words for catalogue memory.

The emulator's memories are at least 32-bits wide. When necessary, the compiler compacts the narrow memories, replacing them with shallower memories of width 32 or 64. However, the compiler cannot compact the catalogue or pool of a sparsely-used memory. Keep this fact in mind when choosing the values for *<n>* and *<p>*.

After compiling, the `dbFiles/<top>_mpart` file shows the depth and width of the memories as actually implemented.

Removing Sparsely-Used Memories in ICE Mode

You can use the following XEL command to remove a specific sparsely-used memory:

```
memoryTransform -rm {<name> SPARSE}
```

Where, *<name>* is a hierarchical name of the memory.

Accessing Sparsely-Used Memories at Run Time in ICE Mode

Once designs containing sparsely-used memories have been compiled, those memories can be accessed at run time by using either of the following methods:

- Using the *Memory* tab in the xeDebug GUI
- Executing the `memory -load` and `memory -dump` XEL commands on the command line

For detailed information about handling sparsely-used memories in a design at compile time, refer to the [Modeling Sparsely-Used Memories at Compile Time in ICE Mode](#) section.

While accessing sparsely-used memories at run time, you must keep the following points in mind:

- For `memory -load` command, remember that the memory was defined as sparse, and therefore, only some part(s) of its address space (not exceeding the POOL size) can be loaded.
- If a sparse memory was dumped with `memory -dump` command, and then loaded with `memory -load` command, the memory content will be the same as before dumping.
- `memory -reset` command sets all virtual pages to have the default value 'all zeroes'. In the other words, it 'frees up' the memory.
- `memory -set` command performs a similar operation as `memory -reset` command. However, it sets all virtual pages to have the default value of 'all ones' instead of 'all zeroes'.
- If during the emulation, one of the sparse memories overflows, the emulation stops and an error message (containing the name of the overflowed memory instance) is issued. In this case, if required, you can create a script to catch this error, reset the overflowed sparse memory, and continue emulation.

You can use the following XEL commands to obtain information about the sparsely-used memories:

- `memory -sparse_mem_info [-v] <mem_name>`
Provides information such as, page size, number of virtual pages, number of physical pages, and number of used physical pages for the specified sparse memory. Using the `-v` option displays the information in a more user-friendly format.
- `memory -sparse_mem_info -usedpages [-v] <mem_name>`

Provides a list of page numbers for all used virtual pages, that is, virtual pages that do not have the default value for the specified sparse memory. Note that the list can be very long. Using the `-v` option displays the information in a more user-friendly format.

- `memory -sparse_mem_info -statistic [-v] [-byusage]`

Provides statistic information for all sparse memories. The output of this command lists the percentage of used physical pages for each memory name. The list is sorted by memory name. If you specify the `-byusage` option, the list is sorted by usage.

- `memory -sparse_mem_info -overflow`

Provides a list of overflowed sparse memory instances.

Important

In the SA mode, the above-listed `memory` command options for accessing sparsely-used memories at run time are not supported.

For more information on the `memory` XEL command, refer to the *Run-Time Commands* chapter of the *UXE Command Reference Manual*.

Also, note that the SA-specific `readmem` memory-access command is not supported for sparsely-used memories.

Examples

Listing Sparse Memories

```
XE> memory -sparse_mem_info -statistic
18% top.ABin0.Alwrite.Alnfbuf.Alnfbuf_ram.mem
```

Getting Information about a Sparse Memory

```
XE> memory -sparse_mem_info
top.ABin0.Alwrite.Alnfbuf.Alnfbuf_ram.mem
32 3216 7
XE> memory -sparse_mem_info -v top.ABin0.Alwrite.Alnfbuf.Alnfbuf_ram.mem
page size 32, 32 virtual pages, 16 physical pages, 7 used physical pages
XE> memory -sparse_mem_info -usedpages -v
top.ABin0.Alwrite.Alnfbuf.Alnfbuf_ram.mem
7 virtual pages are used:
0 1 2 3 4 5 6
```

Clearing a Sparse Memory (Reset)

```
XE> memory -sparse_mem_info -statistic
43% top.ABin0.Alwrite.Alnfbuf.Alnfbuf_ram.mem
XE> memory -reset top.ABin0.Alwrite.Alnfbuf.Alnfbuf_ram.mem
XE> memory -sparse_mem_info -statistic
0% top.ABin0.Alwrite.Alnfbuf.Alnfbuf_ram.mem
```

Support for Sparse Memories in SA Mode

In the ICE mode, a memory can be identified and implemented as a sparse memory by using the `memoryTransform` command. However, the functionality provided by this command is not sufficient for the SA mode. The `memoryTransform` command implements the specified memory as sparse only in the hardware emulator; it does not have the ability to do the same in the software simulator. In the SA mode, both the software simulator and the hardware emulator need to have the same sparse memory implementation model in order to implement key features like hot-swap.

Therefore, a separate implementation for sparse memories is required in the SA mode.

Note: *irun* supports the `/* sparse */` pragma, which can be used to declare a sparse memory for simulation purposes. The IXCOM compiler recognizes this pragma in your HDL, and passes it on to *irun* by dumping it in the HDL that it generates for *irun* consumption. The IXCOM support for the *irun* `/* sparse */` pragma is orthogonal (that is, it has no co-relation) to the sparse memory support in IXCOM mode.

This section describes how you can specify a memory to be implemented as a sparse memory in the SA mode, compile these sparse memories, and access these memories at run time. This section also describes the limitations of this sparse memory implementation. This section also provides a comparison between the sparse memory implementations in ICE and SA modes.

Specifying the Memory as Sparse Memory in SA Mode

You can identify a memory to be implemented as a sparse memory in the SA mode by using the following `$ixc_ctrl` system task:

```
$ixc_ctrl("sparse_mem", <memory_name>, <num_pages>, <page_size>);
```

Here:

- ❑ `<memory_name>` is the reference to memory to be implemented as sparse.
- ❑ `<num_pages>` is the number of pages of memory accessed by the testcase.

UXE User Guide

Compiling and Running Designs with Memories

- `<page_size>` is the size of each memory page (unit: memory word / address).

Note: `<num_pages>` and `<page_size>` are rounded up the nearest power of 2.

Only a subset of the actual memory declaration, which is controlled by the `<page_size>` and `<num_pages>` parameters is implemented in both the software simulator and the hardware emulator. To be specific, only the `<num_pages>` pages of the memory are implemented with each page containing `<page_size>` addresses. This sparse memory implementation model involves implementing only a subset of the actual memory declaration. This implementation does NOT have a cache-memory concept, where the memory is implemented as declared in the software simulator, and only the needed subset is “*cached*” into the hardware emulator.

The page boundaries are calculated solely on the basis of the `<page_size>` parameter, and are independent of the memory declaration. So if the `<page_size>` parameter is 1024, the page boundaries are 0-1023, 1024-2047, 2048-3071, and so on.

The `<num_pages>` pages of memory accessed by the testcase/simulation do not have to be contiguous; these can be any located anywhere within the memory declaration.

The first *write* access to any memory address within a page will mark that page as *used* or *allocated*, and count it against the `<num_pages>` total. *Read* access to a memory address does not mark the page containing it as *used* or *allocated*.

Read access to a memory address, which has not yet been written to, results in a default value being returned ('x' for four-state, 0 for two-state).

An attempt to access more memory pages than were allocated during compilation (via the `<num_pages>` parameter results in a run-time error.

The sparse memory implementation checks for the following in the `$ixc_ctrl` statement used to specify sparse memory.

```
$ixc_ctrl("sparse_mem", <memory_name>, <num_pages>, <page_size>);
```

The second argument (`<memory_name>`) must be a reference to a valid memory instance (one-dimensional unpacked array of a one-dimensional packed array).

The third and fourth arguments (`<num_pages>` and `<page_size>`) must be positive integers.

The ratio of the size of memory in the sparse memory implementation to the size of the memory in the original declaration must be less than 0.5 (that is, 50%). This ratio is calculated using the formula:

UXE User Guide

Compiling and Running Designs with Memories

$$\frac{\langle num_pages \rangle * \langle page_size \rangle}{\langle num_addresses_in_the_original_declaration \rangle}$$

The values used for `<num_pages>` and `<page_size>` in this calculation are the ones after these have been rounded up to the nearest power of 2.

Violation of any of these conditions result in a compile-time error.

- ❑ For every reference to a sparse memory that occurs in an unsupported context as outlined above, the `UNSUP_SPMEM_REF` (Unsupported Sparse Memory Reference) error is issued.
- ❑ For every reference (in an LHS context) to a sparse memory that occurs in an unsupported LHS context as outlined above, the `UNSUP_SPMEM_WRITE_REF` (Unsupported Sparse Memory Write Reference) error is issued.
- ❑ For every XMR reference to a sparse memory that occurs in an unsupported context as outlined above, the `UNSUP_SPMEM_XMR_REF` (Unsupported Sparse Memory Out Of Module Reference) error is issued.

Example of Sparse Memory Implementation in SA Mode

Assume that a design includes a 1G address memory, and the testcase or simulation only accesses 16M addresses of that memory during simulation:

```
reg [63:0] mem[(2**30 -1):0];    // 1G memory decl
```

One way to specify sparse memory implementation for this memory is as follows:

```
initial $ixc_ctrl("sparse_mem", mem, 16, 2**20);
```

The above syntax implements `mem` as a sparse memory of 16 pages, with each page containing `2**20` addresses.

The page boundaries will be:

0M to (1M -1), 1M to (2M -1), ...and so on.

The testcase or simulation can do a *write* access to addresses within any 16 such pages within the memory declaration, and a *read* access to any address within the entire memory declaration.

The 16 pages with *write* accesses do not have to be contiguous. These can be any 16 of the 1024 pages (1G divided by 1M) located within the memory declaration.

Attempts to do a *write* access to a 17th page will result in a run-time error.

Note that page boundaries are dictated solely by the `<page_size>` parameter, and the address range specified in the memory declaration has no effect on the page boundaries. This will not have any impact in most cases because memory declarations usually have 0 as the lower address bound.

Assume a sparse memory declaration is as follows:

```
reg [63:0] mem [258:321];  
initial $ixc_ctrl("sparse_mem", mem, 2, 16);
```

For this memory, the page boundaries that straddle the memory declaration are:

256 - 271

272 - 287

288 - 303

304 - 319

320 - 335

So *write* access to address 271 and 272 will count as accessing two separate pages, even though they are within the first 16 memory addresses as per the memory declaration (258 - 273). On the flip side address 320 and 321 will be considered to be part of the same page as (258 - 271).

Accessing Sparse Memories at Run Time in SA Mode

The sparse memory implementation instruments all (legal) access to a sparse memory in the HDL. However at run time, care must be taken to ensure that attempts to access a sparse memory go through the sparse memory implementation. All references to the original memory in the HDL are instrumented at compile time, and the original memory declaration cannot be accessed at run time via the GUI.

The various ways to access a memory at run time are discussed below:

■ xeDebug commands:

Only the `memory` command can be used to access DUT sparse memories via xeDebug. All other xeDebug commands that access memory (`value`, `deposit`, `$readmem`, `$writemem`, and so on) access the original memory declaration and not the corresponding sparse memory declaration. The `memory` command (for a DUT memory) calls the equivalent `xc memory` command. The `xc memory` command is executed by IXCOM runtime and uses the sparse memory declaration instead of the original memory declaration. All other xeDebug commands that enable memory access are not routed

through IXCOM runtime, and hence cannot be updated to use the sparse memory implementation instead of the original declaration.

■ **irun commands:**

Just like xeDebug commands, access to sparse memory is supported only via the `xc memory tcl` command. Using any other `irun` command to access the sparse memory accesses the original declaration and not the sparse memory implementation.

■ **DPI, VPI, and other interfaces:**

Access to sparse memories is only allowed via the `marg` programming interface. Access through DPI, VPI, or other means accesses the original memory declaration.

Limitations of Sparse Memory Handling in SA Mode

In sparse memory implementation all *read* accesses of the sparse memory are replaced with a function call and all *write* accesses of the sparse memory are replaced with a task call.

Therefore, sparse memory references are supported only in the constructs listed below:

- ❑ `initial` and `always` processes
- ❑ Primitives and continuous assignments
- ❑ Tasks and functions
- ❑ Property and Sequence declarations
- ❑ Assertions

Note: Only covergroup-related constructs are included.

- *Read* access to a sparse memory is allowed in all instances in which a function call expression is legal (within the supported constructs listed above).
- Access to a sparse memory via `$readmem*` and `$writemem*` system tasks is supported.
- Access to a sparse memory via the “`marg`” interface (both the C and SystemVerilog incarnations) is supported.
- OOMRs to sparse memory as an argument to the `$readmem*/$writemem*/marg` system tasks are supported.
- OOMRs to DUT sparse memories are supported in the testbench partition. OOMR access to DUT memories in the testbench partition gets converted to corresponding

“marg” interface calls. Access to sparse memory through the “marg” interface is supported. Therefore, OOMRs to DUT sparse memories in the testbench partition are also supported.

Errors in Sparse Memory Implementation in SA Mode

Compile-time errors in are generated in the following cases:

- ❑ Reference to a sparse memory in a construct other than the ones listed above.
- ❑ Any forms of *write* access to a sparse memory except those in the LHS of a procedural assignment.
- ❑ An expression that accesses more than one memory word (like slice select) or an expression that accesses parts of a memory word (like bit/part select on a memory word). Both *read* and *write* access to a sparse memory should be on a memory word basis.
- ❑ Access to a sparse memory via any other `$sysf` except `$readmem*` and `$writemem*` system tasks.
- ❑ OOMRs (Out Of Module References) to DUT sparse memories in the hardware partition.

At run time, the sparse memory runtime implementation issues the following error message when the testcase or simulation tries to access more pages within the sparse memory than were allocated for its implementation at compile time:

```
** IXCOR Runtime ERROR **  
For sparse memory (mem) in module (dut)  
Attempt to write to address 37, results in  
number of blocks accessed during simulation exceeding the  
number of blocks allocated at compile time (2)
```

This error will also result in the simulation being stopped, that is, a `$stop` will be executed immediately after this statement is issued.

Comparison of Sparse Memory Support between SA and ICE Modes

Following are the differences in the support for sparse memories in SA and ICE modes.

Syntactical Differences

- ❑ SA: (HDL in the module containing the sparse memory)

UXE User Guide

Compiling and Running Designs with Memories

```
$ixc_ctrl("sparse_mem", <mem_name>, <num_pages>, <page_size>)
```

❑ ICE: (XEL command to xeCompile)

```
memoryTransform -add {<mem_name>, SPARSE, <log2_num_pages>, \  
                    <log2_page_size>}
```

The difference is in `<num_pages>` and `<page_size>` parameter specification. Assume the implementation is for 16 pages of 1024 addresses each. The parameter values would be as follows:

	Required	IXCOM	ICE
num_pages	16	16	4
page_size	1024	1024	10

If an XEL command is used to specify a sparse memory in the SA mode, an error message is issued.

In the SA mode, access to more pages than are allocated at compilation results in the simulation being stopped with the invocation of `$stop`, and an error message is issued.

Run-Time Differences for Sparse Memories support in SA and ICE Modes

In the IXCOM mode, the following run-time commands are not supported for sparse memories:

❑ `memory -set | reset`

Use the above command to reset the sparse memory implementation, that is, to free or unallocate all the used or allocated pages.

❑ `memory -sparse_mem_info`

Use the above command to view a report containing information related to sparse memory usage.

If you attempt to access the original memory declaration, it can lead to errors. All (legal) accesses to sparse memories (in the HDL) are instrumented by the IXCOM compiler to use the sparse memory implementation instead of the original memory declaration. All illegal accesses to sparse memories (in the HDL) result in a compile-time error.

Similarly at run time, (legal) accesses to sparse memory through either the `marg_interface` or the `xc memory` command are executed by to access the spare

memory implementation instead of the original memory declaration. However at run time, the original memory declaration can be accessed through DPI, VPI, and GUI and so on.

Using Memory Primitives

A memory is created with memory primitives by creating an MPR for each read port and an MPW for each write port. The address and data ports are connected as appropriate. The active-high write enable should be connected so that the data and address for the write port is valid during the entire period that the write enable is active. If the address or data changes at the time that the write enable becomes active, the write operation uses the new value for the address and data lines. Since there is no hold requirement, the address and data lines can change at the same time the write enable becomes inactive.

The SYNC_IN and SYNC_OUT pins are used to tie the memory ports together into a single memory. The sync pins also control the order of evaluation for the memory ports. The first memory port needs to have its SYNC_IN pin tied to ground. The remaining ports need to be connected SYNC_OUT to SYNC_IN with the last port's SYNC_OUT pin left unconnected. The write ports should generally be placed before the read ports. This will cause a read from the same location as is being written to return the value currently being written. This ordering mechanism can also control which write port will have the precedence when there are multiple write ports. The instance name of the first port is the name that is used as the instance name of the memory for reading and writing.

Probing Restrictions

SYNC_IN and SYNC_OUT nets are not real nets. They are sequencing signals introduced to model port ordering in a multi-port memory. The nets will not exist in the design being emulated. As such, it is invalid to specify them as probe or keepnet.

Building Memory Primitives

The memory primitive module names are Memory Port Read (MPR) and Memory Port Write (MPW). Multiple MPRs and MPWs tie together to form a single memory with multiple ports that access that memory.

The designImport recognizes instances of modules with names of the form [MPR | MPW MPW_BS] <depth>X<width> and creates the module definitions as needed. There is no need to predefine MPRs, MPWs, and MPW_BSs.

For depths of 1K to 512K, the depth is referred to with K notation. For depths of 1M to 512M, the depth is referred to with M notation. For depths of 1G to 512G, the depth is referred to with

UXE User Guide

Compiling and Running Designs with Memories

G notation. For depths of 1T to 512T, the depth is referred to with T notation. For depths of 1P to 512P, the depth is referred to with P notation. For depths of 1E to 8E, the depth is referred with E notation. 8E (63 address bits) is the maximum supported depth.

The pins used in the modules must be in a particular order, but you can choose the pin names. The pin names are then referenced by the modules that instantiate these MPR and MPWs. The following is an example of an 8-deep 2-bit wide MPR and MPW:

```
module MPW8X2 (WR_ADDR2, WR_ADDR1, WR_ADDR0, DATAIN1, DATAIN0, WR_ENBL, SYNC_IN,
  SYNC_OUT);
input WR_ADDR2, WR_ADDR1, WR_ADDR0;
input WR_ENBL;
input DATAIN1, DATAIN0;
input SYNC_IN;
output SYNC_OUT;
endmodule

module MPR8X2 (RD_ADDR2, RD_ADDR1, RD_ADDR0, SYNC_IN, DATAOUT1, DATAOUT0, SYNC_OUT);
input RD_ADDR2, RD_ADDR1, RD_ADDR0;
input SYNC_IN;
output DATAOUT1, DATAOUT0;
output SYNC_OUT;
endmodule
```

In the example, the address and data pins are ordered from most significant bit to least significant bit. The SYNC_IN and SYNC_OUT signals are the special signals described in the [Using Memory Primitives](#) section. The write enable (WR_ENBL) signal is an active high signal. There are no read enable signals because the read port is always active.

Enforcing Design Rules for Memory Primitives

Memory primitive cell names are:

```
- MPR<size>           // read port
- MPW<size>           // write port
- MPW_BS<size>        // write port with Byte-select signals
```

```
<size>    ::= <depth>X<width> | <depth>x<width>
```

```
<depth>   ::= 1<suffix> | <number>[<suffix>]
```

```
<number>  ::= 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 | 512
```

```
<suffix>  ::= K | M | G | T | P | E
```

```
EXAMPLES. MPW8X17; MPR128GX36; MPW_BS1EX1
```

NOTE: K stands for kilo (2 power 10); M stands for mega (2 power 20);

UXE User Guide

Compiling and Running Designs with Memories

G stands for giga (2 power 30); T stands for tera (2 power 40);
P stands for peta (2 power 50); E stands for exa (2 power 60).
(See http://en.wikipedia.org/wiki/Binary_prefix)

NOTE. The minimum depth is 2 (aw = 1). The maximum depth is 8E (aw = 63).

A memory primitive cell name implicitly defines the number of terminals.

- For MPR:

```
#inputs = 1 + #addr_inputs
#outputs = 1 + <width>
```

- For MPW:

```
#inputs = 2 + #addr_inputs + <width>
#outputs = 1
```

- For MPW_BS:

```
#inputs = 2 + #addr_inputs + <width> + <width>/8
#outputs = 1
```

Note: The *<width>* of MPW_BS must be divisible by 8 (such as 8, 16, 24, ...).

addr_inputs can be calculated as follows:

<code><depth1></code>	2	4	8	16	32	64	128	256	512	
<code><suffix></code>										

-	1	2	3	4	5	6	7	8	9	
<code>#addr_inputs</code>	1	2	3	4	5	6	7	8	9	
<code><depth2>K</code>	1	2	4	8	16	32	64	128	256	512

<code>#addr_inputs</code>	10	11	12	13	14	15	16	17	18	19
<code><depth3>M</code>	1	2	4	8	16	32	64	128	256	512

<code>#addr_inputs</code>	20	21	22	23	24	25	26	27	28	29
<code><depth4>G</code>	1	2	4	8	16	32	64	128	256	512

<code>#addr_inputs</code>	30	31	32	33	34	35	36	37	38	39

UXE User Guide

Compiling and Running Designs with Memories

<depth5>T	1	2	4	8	16	32	64	128	256	512

#addr_inputs	40	41	42	43	44	45	46	47	48	49
<depth6>P	1	2	4	8	16	32	64	128	256	512

#addr_inputs	50	51	52	53	54	55	56	57	58	59
<depth7>E	1	2	4	8	16	32	64	128	256	512

#addr_inputs	60	61	62	63						

Note: K stands for kilo (2 power 10); M stands for mega (2 power 20); G stands for Giga (2 power 30); T stands for Tera (2 power 40); P stands for Peta (2 power 50); E stands for Exa (2 power 60);

The minimum depth is 2 (aw = 1). The maximum depth is 8E (aw = 63).

Examples:

MPR16Kx4 has 15 (1+14) inputs and 5 (1+4) outputs.
 MPW8X32 has 37 (2+3+32) inputs and 1 output.
 MPR512Mx8 has 30 (1+29) inputs and 9 (1+8) outputs.

Running Designs with Memories

This section describes the detailed information on loading and dumping memories.

Both memory loads and memory dumps can be run in background. Use the memory command option `-bg` to run the memories in background. The operation is added to a task queue and the command is returned immediately without waiting for the memory operation to finish. It can also be added to a group of memory load and dump commands, which can improve performance.

If a memory dump or load is run without using the background option, it is a foreground operation. Use the `memory -fg` command to run in foreground, and wait for all background tasks to finish. If an error is found in the background tasks, an error message is generated.

Global Memory Options

User-defined options applied to all subsequent memory operations are known as global options. Global options append to other options. Global options that conflict with separate memory commands can be overridden by specifying the desired memory option as a separate memory command, on its own line.

The following options are global options:

- `-append`
- `-clearAddrReg`
- `-full`
- `-skipzero`
- `-z`
- `-dir`
- `-background` (or `-bg`)

The following command options can be used to manipulate the global options:

- `memory -opt <option specifications>`
- `memory -optrm <option specifications>`
- `memory -optclear`

The `memory -opt` command with no specifications returns a string showing the current settings of the global options. With option specifications, it adds the specified options to the existing set of options. Use the `memory -optrm <option specifications>` command to remove specified options, or `memory -optclear` to clear all options.

Global stack options allow you to maintain several sets of global options settings. Each set of global option settings is one level on the global options stack. The level of the top of the stack is the current global options. The following commands manipulate the memory option stack:

■ `memory -optstack`

This command shows the values of all levels of global options. Each level is marked with a number, and the top level is marked as number 1.

■ `memory -optpush`

Create a new set of global options. This command option pushes a new level of the stack top, a duplicate of the current stack top.

■ `memory -optpop`

Pop a level of global options. This command option removes a set of global options. Removes the top level and the second level becomes the new top level.

■ `memory -optuse <n>`

Bring level `<n>` to the top of the memory stack. The order of all other levels will be maintained. Thus, the command `memory -optuse 3` will bring level 3 to the top of the stack, but levels 1 and 2 will retain their order.

The Memory Address Register

The memory address register is the register that drives the write address of a memory instance. Various options can be used to specify the range of memory addresses to be dumped from the address register. Some memory instances are used to capture data starting from address 0.

Loading Memories

The system tasks `$readmemb/$readmemh` are used to read and load data from text files into memory. As the file is read into memory, each number is assigned a specific word element in the memory. Addressing is controlled by specifying start and/or finish address in the system task invocation as well as by specifying addresses in the data file.

Addresses in the data file appear as an @ character, followed by a hexadecimal number. Both uppercase and lowercase is allowed. When the system encounters an address specification, it loads the subsequent data, starting at that memory address. If no addressing information is specified, and no address specifications are found in the data file, the default start address becomes the left-hand address in the declaration of the memory. Consecutive words are read until either the memory is full or the data file is completely read. If the start address is specified in the task without a finish address, loading will start at the specified start address and continue toward the right-hand address given in the declaration of the memory.

If both start and finish addresses are specified as task parameters, loading begins at the start address and continues to the finish address, regardless of how addresses are specified in the memory declarations

When addressing information is specified in both the system task and the data file, the addresses in the data file must be within the address range specified by the system task parameters. Otherwise, an error message is generated and the load operation is terminated. A warning message is issued if the number of data words in the file differs from the number of files in the range implied by the start through finish addresses.

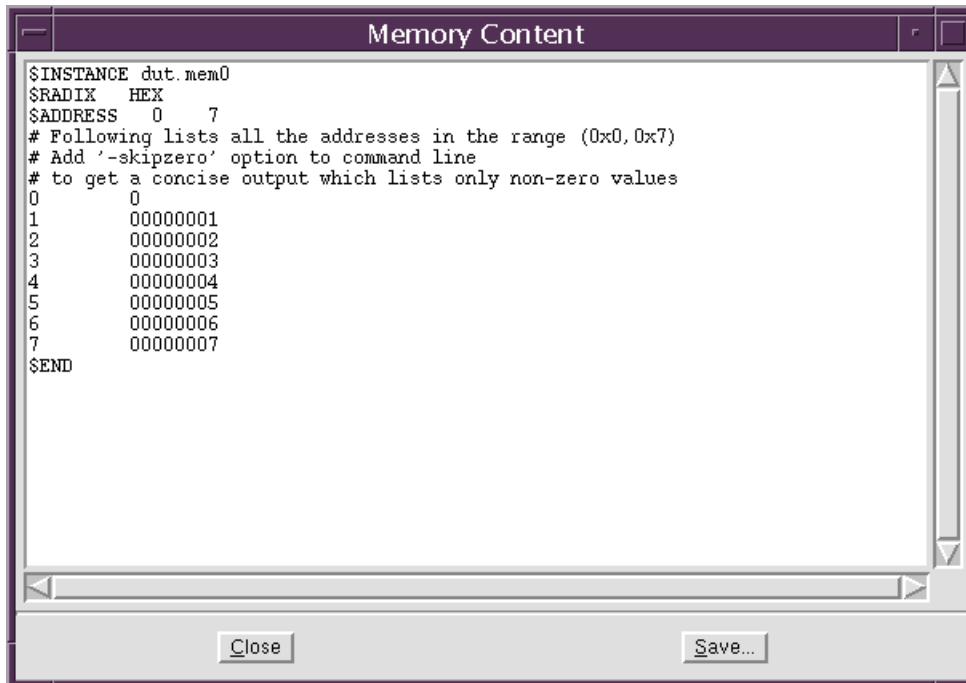
For more information, refer to IEEE Standard 1364-2001.

Loading/Dumping Memory

Select the AutoView option in the *Memory Action Control* group box to launch the *Memory Content* window that lists all the memories in the design. The contents of this window can be edited and then written back to memory by using the *Save* button in the *Memory Content* window.

Figure I-1 displays the *Memory Content* window.

Figure I-1 Memory Content Window



memTran Utility

By using the memTran utility, you can translate between different file formats.

The command syntax is:

```
memTran -translate [in_format] <in_file> [out_format] <out_file> \
    [<memory dimension options>] [<format specific options>]
```

The format string is identical to the format string accepted by the `memory` command. As some formats (such as ASCII, memtran, raw, and NC-Sim) can identify themselves by file format, the input format is optional. Input format must be different from the output format.

Memory Dimension Options

Memory dimension options specify the width and depth of the memory instance and address ranges whose data are specified in the memory content file. This information is included in some file formats.

If the output format requires information that the input format does not contain, either default values are used, or you must specify the appropriate command-line option. If no option is specified, and a default value is not available, an error message is generated. The following

UXE User Guide

Compiling and Running Designs with Memories

table lists the options and available file formats for conversion, as well as whether the file of that format contains the information, and if a default value is available. If “No” is indicated, the file format does not contain the information, and cannot provide a default value.

	memtran	raw	raw2	PD ASCII	readmemb/ readmemh	NC-Sim
-width	Yes	Yes	No	No	No	No
-depth	Yes	Yes	end-start+1	end-start+1	end-start+1	end-start+1
-start	Yes	Yes	0	Yes	0	0
-end	Yes	Yes	Last addr in file	Yes	Last addr in file	Last addr in file

Format-Specific Options

The following format-specific options are available:

Option	Format	Use
-p <passwd>	%pd_memtran	Specifies password for file encryption. If the input file is encrypted, a correct password must be specified to convert the input file to another format. If the output file is in %pd_memtran format and a password is specified, the output file is encrypted. To specify a password for both the input and output file, use two -p <passwd> options, one for the input file and one for the output file.
-skipzero	PD ASCII	Filters out words containing all-zero data. Used with output files in PD ASCII (%pd_b, %pd_o, %pd_d, %pd_h).

Comparing File Formats

To compare file formats, use the command-line syntax:

```
memTran -compare [format1] <file1> [format2] <file2> [-summary] [-width <n>]  
[<format_specific_options>]
```

UXE User Guide

Compiling and Running Designs with Memories

If the file can identify its format type, format specification is optional. If neither file being compared contains width information, you just use the `-width` parameter to specify memory width.

If the `-summary` option is specified, the utility will only print a one line summary message, for example:

```
file1 and file2 same
```

or

```
file1 and file2 differ
```

Otherwise, a full report of the differences is printed, as shown in the example below:

```
00000000: 05 06          06 00
00000007: 15 16          16 00
0000000f: 25 26          26 00
```

Each line describes a single word. It shows the address, content of the memory word in the first file, in hexadecimal, and content of the memory word in the second file in hexadecimal. Whether or not the `-summary` option is specified, you can check the return code of the `memTran` program. In the C shell, it is `$?`. A 0 means there is no difference in the files.

Dumping Memories

The `memory -dump` command reads data from the memory instance and dumps it to the specified file. In XEL, it uses the following syntax:

```
memory -dump {<file_format> <memory_name> -file <file_name>} [-append] \
    [-start <start_number>] [-end <end_number>] [-nooverwrite] [-skipzero] \
    [-addrReg <register_name>] [-clearAddrReg] [-full <number>] \
    [-addrRadix <radix>] [-dataRadix <radix>] [-z] [-gz] \
    [-dir <directory_name>] [-background | -bg]
```

Optional starting and ending addresses can be specified for memory data to be dumped. The `-skipzero` option will ignore all lines whose data consists of zeros. The `memory -dump` command must specify the file format. Some memory options can only be used with specific formats. For more information, refer to the *UXE Reference Manual*.

As not all memory data is useful, it might be desirable to use options to specify the range of memory desired, using the additional `memory -dump` command options `-addrReg <register_name>`, `-clearAddrReg`, and `-full <number>`. The `-addrReg` option defines the address range to dump, from address 0 to the current value of the specified register. It cannot be used in connection with the `-start` and `-end` options. The `-full`

UXE User Guide

Compiling and Running Designs with Memories

option can be combined with `-addrReg` to indicate memories that should be dumped if they are more than `<number>` percent full. This can be calculated using the current value of the address register and the memory depth. Use the `-clearAddrReg` option to clear the register after dumping the memory.

The following example shows the use of these options:

```
memory -dump %pd_raw Top.mem1 -file mem1.dump -addrReg Top.mem1_wa[19:0]  
-clearAddrReg -full 90
```

This command dumps Top.mem1 in raw format. The memory address range to be dumped is from 0 to register Top.mem1sa[19:0]. This memory is not to be dumped unless the address register has a value greater than 9437184, which is 90% of the memory depth. The memory is to be cleared after dumping.

Specify the `-z` option if you want to save the data file in compressed format, as generated by the UNIX compress command. It applies only to the raw or raw2 formats. Contents are written to a file with the `.Z` postfix. The file can be uncompressed through UNIX commands `uncompress` or `zcat`. Use of `-z` cannot be combined with the `-append` and/or `-fileoffset` raw2 format option.

The output of the memory command can be difficult to read if the design has a large number of memories. To make reading memory names easier, execute the following Tcl commands:

```
set memlist [memory]  
foreach item $memlist {  
    puts $item}
```

Using Memory Streaming

Memory streaming simplifies the use of large data stream files in the in-circuit emulation. A data stream file can be used either as an input or output to the emulation.

If the data stream file is used as an input to the emulation, data is transferred from the file to the specified design memory in "chunks".

Typically, one chunk is enough to fill the memory. When the design has finished processing that chunk, another chunk is transferred, filling the memory again. The chunks are taken in sequence from a file that contains enough data to fill the memory many times.

If the data stream file is an output from the emulation, data is transferred from a design memory to the file in chunks. Each chunk is appended to the file, so that eventually the file contains a stream of data much larger than the size of the memory itself.

The XEL commands to use memory streaming are as detailed in the following sections:

Using XEL for Writing Memories

The following command shows the syntax for writing memories.

```
memory -load %pd_raw2 <inst> -file <file> -fileOffset <offset> [-nofill]
        [-start <start>] [-end <end>]
```

To use the `memory -fileOffset <offset>` or `-getFileOffset` options, or the `-nofill` option the memory contents file must be in `raw2` format (`%pd_raw2`). Otherwise, `-fileOffset` and `-getFileOffset` options will generate errors.

Normally, you create the file using `memTran`, as described later in this section.

`<offset>` can be either one of the following:

- `<number>` - Takes data from the file starting from byte `<number>`, where a byte is 8 bits. When `memTran` creates a file in `raw2` format, if the width specified in the `memTran` command line is not a multiple of 8 bits, `memTran` automatically pads the data so that each word takes a whole number of bytes in the file.
- `continue` - Takes data from the file, starting at the byte immediately after the last byte in the file that was taken in a previous operation.

If `-nofill` is specified, the address range from end of the `%pd_raw2` file to end of memory is not changed. Otherwise this range will be filled with 0s.

If the `-start` and `-end` parameters are specified, the command only takes enough bytes from the file to fill the section of the memory specified by `-start` or `-end`. Unlike `<offset>`, the start /end is measured in words, where the word width is determined by the memory width.

The `-getFileOffset` parameter returns the `<offset>` in the file from which the next data will be taken. If no data remains in the file, -1 is returned. Usually, a script uses `-getFileOffset` only to test whether any data remains in the file.

If the file does not contain enough data to fill the memory, the command fills part of the memory with data from the file, then fills the rest of the memory with zeros. To fill only part of the memory, use `-start` and `-end`.

The software remembers one file offset for each file being written, rather than one for each (instance,file) pair. Therefore, to use `-fileOffset continue` to load two memories independently from the same file, you must make two copies of the file (although one can be a soft link).

UXE User Guide

Compiling and Running Designs with Memories

Using XEL for Memory Read

The following command shows a sample of the syntax for reading memories.

```
memory -dump %pd_raw2 <inst> -file <file> [-append]
        [-start <start>] [-end <end>]
```

The `-append` parameter gives an error if the format is not raw2. Using `-append`, the file is appended. Without `-append`, the file is overwritten.

The following shows a section of a run time script that repeatedly loads data from file `in_file.raw2` to memory `in_mem`, runs, and dumps data from memory `out_mem` to file `out_file.raw2`. The process repeats until there is no more data in `in_file.raw2`.

```
# This part of the script assumes the logic analyzer has been
# set up to trigger each time the design finishes processing
# the data from in_mem. For example, we could trigger on in_mem
# read address having a particular value and clock having a rising edge.

rm -f out_file.raw2
while {[memory -getFileOffset in_raw2]!=-1} {
    memory -load %pd_raw2 in_mem -file in.raw2 -fileOffset continue
    run
    waitWhileBusy
    memory -dump %pd_raw2 out_mem -file out_file.raw2 -append
}
```

Using memTran to Translate to and from raw2 File Formats

You can use the `memTran` program to convert a file to raw2 file formats. The command-line syntax is:

```
memTran -translate <input_format>] <in_file> <%pd_raw2> <out_file>
        [<memory_dimension_options>] [<format-specific_options> ...]
```

A file in `readmemh` format uses one line of hex data for each memory word. A file with 12-bit-wide words would contain lines like the following:

```
000
123
456
789
abc
def
```

UXE User Guide

Compiling and Running Designs with Memories

The width of the words in the file normally should match the width of the memory to be written, but the depth of the file need not match the memory depth.

Suppose the readmemh file is named `in_file.readmemh`, and the words are 12 bits wide. To create the raw2 file named `in_file.raw2`, use the following command:

```
memTran -translate %readmemh in_file.readmemh %pd_raw2 in_file.raw2 -width 12
```

To convert a raw2 file to readmemh format, use a command such as the following:

```
memTran -translate %pd_raw2 out_file.raw2 %readmemh out_file.readmemh -width 12
```

Files in raw2 format do not contain memory width information (nor any other header information). You must specify the correct `-width` information to the `memTran` command, and must also, when using `memory -load`, make sure the file has the correct width for the memory being written.

To convert from a Palladium XP text file to raw2 format, use the following command line:

```
memTran -translate ascii_file %pd_raw2 bin_file -width 12
```

to compare raw2 format files:

```
memTran -compare %pd_raw2 bin_file1 %pd_raw2 bin_file2 -width12
```

Note: The `.bin` extension is no longer supported.

UXE User Guide

Compiling and Running Designs with Memories

Support for axis_smem Memory Model

IXCOM provides a special primitive `axis_smem` to model multi-port synchronous/asynchronous memories. The `axis_smem` primitive can be mapped to the hardware while compiling designs with memories.

Important

The support for `axis_smem` is provided for legacy designs from Xtreme customers. Cadence recommends you to model the memory using the synthesizable constructs of Verilog.

This feature is available only for Verilog; do not use it for VHDL. In Verilog syntax, `axis_smem` is a special module name. User provides an instance name, which is the name of the memory. There are parameters for configuring the characteristics of the memory, such as width and depth. There are ports to connect with signals, such as addresses and data.

Following is the syntax of the `axis_smem`

```
axis_smem #(addr_width, data_width, num_ports, init_val, [rd_wr_ctrl],
            ["place_ctrl"], ["tag_inst_name"]) mem_inst_name
    ( {output_port, ...}, {input_port, ...},
      {address_port, ...}, {write_enable_port, ...}, {chip_enable_port, ...},
      {clock_port, ...}, {write_mask_port, ...});
```

The following parameters of the `axis_smem` syntax are supported:

- `addr_width`
Specifies the number of address bits in memory. All ports have the same number of address bits.
- `data_width`
Specifies the number of data bits. All ports have the same number of input and/or output bits.
- `num_ports`
Specifies the number of ports.

UXE User Guide

Support for axis_smem Memory Model

■ rd_wr_ctrl

Enables to separately control read and write operations to memory using either the *chip_enable_portn* or *write_enable_portn* pin. *rd_wr_ctrl* can be used to disable *axis_smem* write-through behavior (that is, when an address location is written to using input data signals, the same address is immediately read thereafter on the output data signals). Here is how to use *rd_wr_ctrl*:

If *rd_wr_ctrl* is set to 1'b0 (default), the *write_enable_portn* pin controls both read and write operations as follows. The *chip_enable_portn* port disables or enables the memory.

<i>write_enable</i> <i>portn</i>	<i>chip_enable</i> <i>portn</i>	operation
0	0	No memory operation
0	1	Read address to <i>output_portn</i>
1	0	No memory operation
1	1	Write-through: write <i>input_portn</i> to address and read same address to <i>output_portn</i>

If *rd_wr_ctrl* is set to 1'b1, read operations are controlled by the *chip_enable_portn* port and write operations by the *write_enable_portn* pin. In this mode, there is no *chip_enable* capability.

<i>write_enable</i> <i>portn</i>	<i>chip_enable</i> <i>portn</i>	operation
0	0	No memory operation
0	1	Read address to <i>output_portn</i>
1	0	Write <i>input_portn</i> to address
1	1	Write-through: write <i>input_portn</i> to address and read same address to <i>output_portn</i>

The following parameters are accepted, but ignored:

■ init_val

Sets the initial value in all memory locations, which can be either 1'b1 or 1'b0.

The following parameter is rejected with an error message:

■ place_ctrl

Controls the placement of the memory, in a specific type of memory resource available in the emulator system, as follows:

UXE User Guide

Support for axis_smem Memory Model

- ❑ **A** = automatically chosen by hardware (default)
- ❑ **B** = onboard SRAM
- ❑ **C** = internal (FPGA) memory
- ❑ **D** = extended memory (SDRAM)

Note: The letter must be enclosed within double quotes (" "); for example, "D".

■ tag_inst_name

Specifies the instance name of a *tag memory*, specified by an `axis_smem`, when setting up a sparse memory model. The name must be enclosed within double quotes (" "); for example, "*mytag*".

The description of the other parameters in the `axis_smem` syntax is given below:

<code>mem_inst_name</code>	Specifies the instance name of the memory that is being modeled with <code>axis_smem</code> .
<code>output_portn</code>	Connect this terminal to the output data signals for port <i>n</i> if specifying a read or read-write port. For a write-only port, use all <code>Z</code> values or <code>{dataWidth{1'bZ}}</code> , instead. The number of bits in the vector connected to this terminal must be equal to <i>data_width</i> .
<code>input_portn</code>	Connect this terminal to the input data signals for port <i>n</i> if specifying a write or read-write port. For a read-only port, use all <code>Z</code> values or <code>{dataWidth{1'bZ}}</code> , instead. The number of bits of input data signals must be equal to <i>data_width</i> .
<code>address_portn</code>	Connect this terminal to the address lines for port <i>n</i> . The number of bits in the address lines must be equal to <i>addr_width</i> . Note: Constants cannot be used on this terminal.
<code>write_enable_portn</code>	Connect this terminal to the write-enable signal for port <i>n</i> if specifying a write or read-write port. If the signal has a value of 1, data on <i>input_portn</i> is written to the memory. A value of 0 causes the memory to be read to <i>output_portn</i> . If the port is a read-only port, use <code>1'b0</code> instead of a signal connection.

UXE User Guide

Support for axis_smem Memory Model

<i>chip_enable_portn</i>	Connect this terminal to the chip-enable signal for port <i>n</i> (active-high enabled).
<i>clock_portn</i>	If port <i>n</i> is synchronous, connect this terminal to its clock signal; otherwise, use 1'bz to specify an asynchronous port.
<i>write_mask_portn</i>	Connect this terminal to the write-mask signals for port <i>n</i> if specifying a write or read-write port. A value of 1 in a mask signal means the corresponding signal in <i>input_portn</i> is written to memory; a 0 means that bit is not written. For a read-only port, or if not using the write-mask capability, use {dataWidth{1'bz}} or all Z values instead of a mask signal.

An *axis_smem* instantiation looks as illustrated below:

```
axis_smem #('INDEX+1, 38, 2, 1) tlb_data
( {axis_dout, {38{1'bz}}}, //Output Port (1,2)
  {{38{1'bz}}, axis_din}, //Input Port (1,2)
  {axis_raddr, axis_waddr}, // Address Port (1,2)
  {1'b0 ,axis_wen}, //Write Enable
  {axis_ren, 1'b1 }, //Chip Enable (1,2)
  {1'bz , 1'bz }, //Port Clocks (1,2)
  {{38{1'bz}}, {38{1'bz}} }); //Write Mask
```

Concatenation statements must be used to specify the connections to the memory port. One concatenation statement is used for each type of terminal on all the ports; so there is one statement for the output data terminals, another for the input data terminals, and so on. Each port can be synchronous or asynchronous, can be read-only, write-only or read-write, can have its own address lines, write-mask bits, and so on.

Although it is not allowed in standard Verilog syntax, the instance name for an *axis_smem* is used like a memory in the rest of the design. For example, it can be used in an expression, to set a value with an assignment statement, or it can be initialized by a \$readmemh system task.

The memory model by *axis_smem* is always mapped to hardmacro memories modeled by Palladium XP memory primitives MPW and MPR. The -memory<#> option and the no_hard_macro directive do not have any effect on such memories.

Memory C API Functions

To access memories modeled using the `axis_smem` primitive through C API, use the Marg library. Refer to the [Examples: Passing Memory Data from Testbench to DUT](#) section in Chapter 4, “[Compiling Designs for Simulation Acceleration Using IXCOM.](#)”

UXE User Guide

Support for axis_smem Memory Model

Files and File Formats

This section describes files that the UXE Compiler either generates or reads.

Topics discussed in this chapter include:

- Configuration Files
- Design Files
- Memory File Format
- Binary Memory File Formats
 - Raw Format
 - memTran Format
- Cycle-By-Cycle (CBC) Format
- Clock Domain Crossing (CDC) Format

Configuration Files

The configuration file lists the available resources installed on a Palladium XP system or the resources allocated for a design. Configuration file extensions must be `et3config`. This file uses the following conventions for defining resources:

- Any line starting with a plus (+) symbol lists an available resource.
- Any line starting with a minus (-) symbol lists a defective or excluded resource.
- Any line starting with an asterisk (*) is a comment line.

All resources are identified by name and resource specific configuration information.

The initial configuration file is generated when the emulator is first started, through the automatic configuration utility when the system is first set up. (The automatic configuration script is run at the same time as the initial setup diagnostics.)

Configuration File Information

The configuration file contains:

- The target configuration table
- The maximum step count
- Optional constraints for signal assignments, such as I/O signals, trigger signals, and probe signals
- Clock phasing information

Basic Language Elements

The table below defines the variables used in configuration files.

Table K-1 Configuration File Variables

Variable	Example
<code><alu_number></code>	0..1
<code><board_name></code>	48_CHIP_BOARD or 192_CHIP_BOARD
<code><board_number></code>	0..3 0..3 0..31
<code><board_Jconn></code>	0..4
<code><board_type></code>	ET5L, ET5D, or ET5E
<code><cable_type></code>	CABLE_<length> For example: CABLE_106, CABLE_154, CABLE_195, CABLE_230
<code><connector_location></code>	<code><module_location></code> <code><connector_position></code>
<code><connector_number></code>	<code><board_number></code> <code><module_number></code> <code><board_Jconn></code>
<code><connector_position></code>	T B L R or J2 J4 J1 J3 (T op B ot L eft R ight)
<code><domain_on_board></code>	0..3 0..3 0..15
<code><module_location></code>	<code><board_number></code> <code><module_number></code>
<code><module_number></code>	0..3 0..3 0..15

External Timing Files

The Compiler creates the `<design>.xtime` external timing file.

The `<design>.xtime` file shows the timing for all primary output signals (one line per primary output signal).

Syntax

```
<primary_output_signal_name> <emulation_step_number> <pulse>
```

where:

`<primary_output_signal_name>` is a primary output signal name.

`<emulation_step_number>` is the emulation step number when the primary output signal changes.

`<pulse>` can be one of the following designations:

- +
- -
- +number
- -number

The + symbol specifies an initial value of zero for the primary output signal.

The - symbol specifies an initial value of one for the primary output signal.

Initial value means the value that must be set on that signal when the design is downloaded into the emulator (similar to time zero initialization in simulation). The net returns this value until it is changed by a stimulus applied to the design. The initial values are useful for protecting target systems that have tristate interfaces to the emulator.

+number specifies a pulse width of the `number` of emulation steps for the primary output signal. The primary output signal is observable at the external pin from the time specified by the `<timing_constraint>` for at least the `number` of emulation steps. All other times, the external pin is forced to zero.

-number specifies a pulse width of `number` emulation steps for the primary output signal. The primary output signal is observable at the external pin from the time specified by the `<timing_constraint>` for at least the `number` of emulation steps. All other times the external pin is forced to one.

Example

```
PWATL_IN 27 -  
WFB_EXT_BCLK 111 +10  
WFB_MP_CLK 6 -128
```

Design Files

When you successfully import an RTL Verilog or VHDL design, the xeCompile creates gate-level design files. The format of these files depends on the format specified with the `hdlOutputFile` XEL command. The file format options are Verilog and VHDL.

A separate file is created in each specified format, for each module of the design. This includes a separate file for the top-level module, each hierarchical module, and each of the lowest-level instances.

The subset of features supported in these files conform to standard Verilog HDL (IEEE 1364) and VHDL IEEE STD1076-1993.

Memory File Format

The file format for memories is as follows.

Format:

```
$INSTANCE <memory_instance_name>  
$RADIX BIN|HEX|OCT|DEC  
$ADDRESS <start_address> <end_address>  
    <address_x> <data>  
    <address_x+1> <data>  
    .  
    .  
    .  
$END
```

where

<memory_instance_name> specifies the hierarchical name of the memory instance.

<start_address> and *<end_address>* are the hexadecimal starting and ending addresses for initialization.

<address_x> and *<data>* are the address-data pairs. Addresses are hexadecimal and do not have to be contiguous. Palladium fills unspecified addresses with 0s.

The radix options BIN, HEX, OCT, and DEC apply to the data fields only. The address range and the address fields are hexadecimal values.

For example:

```
$INSTANCE U1__RAMBLK0
$RADIX BIN
$ADDRESS 0 5
  0      11011011
  1      00001111
  2      11011011
  3      00100010
  4      11001110
  5      00101010
```

Note: Lines in the memory content file starting with a semicolon (;), dash (-), or hatchmark (#) are treated as comments. All characters following these characters on the line will be ignored.

Note: Memory content files in real binary (raw) format must have a `.bin` extension. Do not use this extension when specifying the memory content filename for a memory read/write operation. Palladium will append the `.bin` extension to the specified filename before executing the read/write operation.

Binary Memory File Formats

A binary memory file format can speed up the `memory -load` and `memory -dump` commands significantly as compared to text-based formats.

Two types of binary memory file formats are available: "raw" format, and "memTran" format. Both can significantly improve performance.

Raw Format

A raw format file can be generated through memory dump, using the following command:

```
memory -dump [%pd_raw | %pd_raw2] <memory_name> -file <file_name>
```

This raw format file cannot be viewed. It can only be used during next `memory -dump`.

The `memTran` utility can translate between raw format and ASCII format by adding the `-raw` option to the `memTran` command line.

Table K-2 shows the raw format of the binary memory file.

Table K-2 Binary Memory File Format

Offset	Description	Value
0-5	signature	QTMEM\0
6	major version	1
7	minor version	0
8-11	reserved	0X12345678
12-47	reserved	0
48-51	depth of memory instance	
52-55	width of memory in number of bits	
56-59	number of bytes for each memory word. Should be calculated as (memory_width+7)/8	Depends on the specific memory instance.
60-63	start address	
64-67	end address	
68-71	instance name length	
72-xx	instance name	
yy-zz	memory data (left justified)	

The memory data are aligned to the byte boundary for each line of data. The bits of each line are left-justified in the bytes, that is, the most significant bit of the memory is the most significant bit of the first byte.

For example, if memory width is 50 bits, then each line will occupy 7 bytes. Bit 49 of the memory is bit 7 (that is, 0x80) of the first byte. Bit 48 of the memory is bit 6 (that is, 0x40) of the first byte. Bit 0 of the memory is bit 6 (i.e. 0x40) of the 7th byte.

memTran Format

By using the memTran utility, you can translate between ASCII format files and binary format files, as well as other file types. The generated binary format file can then be used during the `memory -dump` to improve performance. The memTran format has extra overhead, which could result in larger file sizes than using raw format.

The memTran format is better than the raw format. Use memTran format in general cases.

Use of raw format is most desirable for backward compatibility and publishing the raw format. You can write a C program to generate/analyze raw format.

memTran format can also be used to encrypt memory files. The encrypted file can then be used to perform a `memory -dump`, but it cannot be dumped unless you use a correct password.

The UNIX `memTran` command translates to/from memTran format.

The following examples show use of the `memTran` command.

To convert to a binary format, use the command:

```
memTran <ascii_file> <bin_file> bits [-depth n] [-p password]
```

where:

`bits` specifies the width of memory.

`-depth n` specifies the depth of memory. By default, the depth is derived from the `$ADDRESS` line of the ASCII file.

To dump the contents of the binary file, use the command:

```
memTran -d <bin_file> [-p <password>]
```

You can also compare two binary format files in the memTran tool using the following command:

```
memTran -compare <binfile1> <binfile2> [-p <passwd>] [-summary]
```

where:

- `-p <passwd>` specifies a password (if the binary file is encrypted and a password is required) User can only specify one password. If either of the two files use the specified password, only one of them needs the password.
- `-summary` notifies the program to print out only a one-line message saying whether the files are same. If `-summary` is not specified, verbose messages will be printed out for all the differences.

Cycle-By-Cycle (CBC) Format

CBC format is used in PTRAN vector support.

Cycle-By-Cycle (CBC) vectors contain a time stamp and signal values at fixed time intervals, as opposed to Print-on-Change (POC) vectors that change at irregular increments. To support CBC-format vectors in PTRAN, use the SIMULATOR quest statement.

Use CBC vectors to validate the emulation model. When you run Vector Debug on a design, the CBC vectors stimulate an emulation model of that design. Vector Debug uses the emulator's instrumentation to provide the inputs and clocks to the model.

For design input terminals, the emulator's instrumentation drives the emulated design. The possible input signal values are:

- 0 - LOW
- 1 - HIGH
- X - don't care

For 0 and 1, the emulator's instrumentation monitors the design input terminals for a mismatch that can occur in case of a contention (although a contention does not guarantee a mismatch). For X, the emulator's instrumentation can drive either (0 or 1) value without checking for mismatches.

For design output terminals, the emulator's instrumentation listens and checks for mismatches. The possible output signal values are:

- L - LOW
- H - HIGH
- Z - don't care

L and H are expected values. For Z, the emulator's instrumentation is tristated and does not check the value.

The following is an example of a CBC file:

```
pieVector .0;
vectorFormat CBC;
vectorType stimulus;
numSignal 10;
numVector 8;

PINS CLOCK RESET DATA[7] DATA[6] DATA[5] DATA[4] DATA[3]
DATA[2] DATA[1] DATA[0] ;

// CRDDDDDDDDDD
// LEAAAAAAAAAA
// OSTTTTTTTT
// CEAAAAAAAAAA
// KT76543210
> 19 - XXXXXXXXXXX;
> 199 - 10LLLLLLHH;
```

```
> 239 - 10LLLLLHLL;  
> 319 - 10LLLLLHLH;  
> 419 - 10LLLLLHHL;  
> 539 - 10LLLLLHHH;  
> 679 - 10LLLLHLLL;  
> 839 - 10LLLLHLLH;  
<
```

The header information shows the version of the CBC format, the vector format (CBC), the vector type, the number of signals, and the number of vectors.

The signal names (PINS) are associated with the data value for each cycle. The comment field (//) is optional. In this example, the comment field contains the same signal names (PINS), read vertically.

The CBC values show the time stamp and vector data values, indicated by the greater than (>) symbol at the left side of the line. Each data section in the file ends with a semicolon (;).

This example has two inputs and eight outputs.

The two last lines at the end of the file, each containing a single less than (<) symbol, mark the end of the data file. The emulator ignores the time stamp (second token on a data line), which indicates the real time in nanoseconds.

Clock Domain Crossing (CDC) Format

Each line of the CDC file describes one synchronizer. Buses must be split into individual bits. Lines that start with a hash (#) are comments, which are ignored by the software along with empty lines.

Optionally, the CDC file can start with a line of the following format that specifies the CDC version number:

```
set cdc_version <number>
```

Valid version numbers are 0 (default) or 1. Differences between the two versions are discussed below in the corresponding paragraphs.

Each line that is not a comment must contain the following four tokens:

```
<synchronizer_name> <hierarchical_path_to_synchronizing_reg> <clock>  
<control>
```

<synchronizer_name>

User-defined name without spaces, given just for readability. It is ignored by the software.

<hierarchical_path_to_synchronizing_reg>

Path defined according to Verilog standard. It can contain spaces for escaped names. It can start from the design top module (Verilog style), or start one level down (UXE style). The name of the synchronizer can be appended with a `_reg` suffix. However, this is not a must because the software accepts the names with and without this suffix.

If the CDC version number is 0, the synchronizing register must be a flip-flop.

However, for version 1, the synchronizing register must be either flip-flop or latch.

<clock>

Name of the clock for the synchronizer. If this name contains a dot, it is treated as hierarchical path to clock. If there's no dot, the name is treated as a local name in the same hierarchical node as the synchronizer.

When CDC version number is 0 and the clock name contains a dot, it is treated as hierarchical path to the clock. On the contrary, if there is no dot, the name is treated as local name in the same hierarchical node as the synchronizer.

When CDC version number is 1, the clock must be a register or wire that is local to the `cdc_control` module (a so called virtual clock). If this register or wire is not found, the synchronizer is ignored (that is, delay is not inserted). Multiple delays can be controlled by the same clock.

<control>

Name of the delay control. This must be the name of the register or wire local to the `cdc_control` module. If this register or wire is not found, the synchronizer is ignored (that is, the delay is not inserted). Many delays can be controlled by the same control.

For an example of a CDC file illustrating the use of the four tokens, refer to [Example CDC File: 1.cdc](#).

Using Utilities

These utilities have been developed for mixed-HDL simulation:

- **cds2libmap**
- **vhdel**
- **vhls**
- **vhmake**
- **vhshell**

cds2libmap

Purpose

The `cds2libmap` utility converts a `cds.lib` file to an equivalent `libmaprc` file. This utility can also be used to convert IXE specific `va.lib` files to equivalent `libmaprc` files. See [Converting cds.lib Files to libmaprc Format](#) on page 213 for more information.

Syntax

```
cds2libmap [-h] <cdslib_file> [-o <output_file>]
```

Parameters

<code>-h</code>	Prints help for this utility.
<code>cdslib_file</code>	Specifies the <code>cds.lib</code> file to be converted to <code>libmaprc</code> file

UXE User Guide

Using Utilities

<code>-o</code>	(Optional) Name of the output (libmaprc) file. If no output file is specified, the <code>cds.lib</code> file is converted to a libmaprc file with the default name <code><cdslib_file>_libmaprc</code>
-----------------	--

vhdel

Purpose

The `vhdel` utility lets you delete one or more VHDL or Verilog design units from an analyzed library.

Syntax

```
vhdel -work library -e entity -a architecture | -p package | -b package_body | -c  
      configuration -v Verilog_module
```

Parameters

<code>-work <i>library</i></code>	Specifies the analyzed library (default WORK) from which to delete design units.
<code>-e <i>entity</i></code> <code>-a <i>architecture</i></code>	Specifies the entity and/or architecture to delete. If the architecture is not specified, the entire entity is deleted; otherwise, only the specified architecture for that entity is deleted.
<code>-p <i>package</i></code>	Deletes the specified package and its package body.
<code>-b <i>package_body</i></code>	Deletes only the specified package body for the specified package.
<code>-c <i>configuration</i></code>	Deletes the specified configuration.
<code>-v <i>Verilog_module</i></code>	Deletes the specified Verilog module from the library.

UXE User Guide

Using Utilities

<code>-libmaprc file</code>	Specifies a file containing logical-library-to-physical-directory associations, as required by mixed-HDL tools. See Associating Logical Libraries to Physical Directories Using libmaprc File on page 210 for more information.
-----------------------------	---

vhls

Purpose

The `vhls` utility lists the design units (Verilog or VHDL) compiled into one or more libraries and provides information on these units.

Syntax

```
vhls [-h] [-work library] design_unit [-e|-a|-p|-b|-c] [-l] [-s]
```

Parameters

<code>-h</code>	Prints help for this utility.
-----------------	-------------------------------

<code>-work library</code>	(Optional) Limits the scope of the utility to the logical library specified. Without this option, the utility operates on all libraries in the <code>libmaprc</code> file.
----------------------------	--

<code>design_unit</code>	(Optional) Lists information for the specified <i>design_unit</i> only in <i>library</i> . If <i>design_unit</i> is not specified, then all <i>design_unit(s)</i> in the <i>library(ies)</i> are listed.
--------------------------	--

<code>-e -a -p -b -c</code>	Lists all design units of the specified type in the compiled library(ies): ■ <code>-e</code> specifies all entities ■ <code>-a</code> specifies all architectures ■ <code>-p</code> specifies all packages ■ <code>-b</code> specifies all package bodies ■ <code>-c</code> specifies all configurations
-----------------------------	---

UXE User Guide

Using Utilities

-l	Provides additional information on listed design units, such as logical library, source-file path, line number, compilation status (up-to-date or not), and last-compiled date.
-s	By default, the <code>vhls</code> utility does not provide information for the STD or IEEE system libraries. Use this option to list the packages and package bodies in these libraries. The physical directory paths for these libraries are listed in the <code>libmaprc</code> file. See Associating Logical Libraries to Physical Directories Using libmaprc File on page 210 for more information on the <code>libmaprc</code> file.
-libmaprc <i>file</i>	Specifies a file containing logical-library-to-physical-directory associations, as required by mixed-HDL tools. See Associating Logical Libraries to Physical Directories Using libmaprc File on page 210 for more information on the <code>libmaprc</code> file.

vhmake

Purpose

To manage the compilation of VHDL, Verilog, or mixed-HDL designs, `vhmake` or generates a UNIX `Makefile` with:

- The paths to the original VHDL and Verilog sources and the proper order in which to compile them.
- An incremental compile feature—the `Makefile` recompiles only sources that have changed and other sources that have dependent design units.

Syntax

```
vhmake [-h] [-work library] design_unit [-d design_unit] > Makefile
```


UXE User Guide

Using Utilities

Parameters

<code>-h</code>	Prints help for this utility.
<code>-work library</code>	The logical library of a top-level design unit if one is specified; otherwise, it is the library for which you wish to generate a <code>Makefile</code> .
<code>design_unit</code>	The name of the design unit (VHDL entity or configuration or Verilog module) for which you wish to generate a <code>Makefile</code> . The <code>Makefile</code> will be generated for all of the hierarchy below the specified design unit.
<code>-d</code>	The dependency listing feature. You must specify <code>design_unit</code> with this option. For each design unit, the following information is given: dependent design units and their source files and line numbers.

Description

`vhmake` has a dependency listing feature (`vhmake -d`). Starting with the highest level entity or configuration, the design units of a particular design are listed in order of dependency:

- For each package body, the dependent package header is listed.
- For each entity, the dependent packages are listed.
- For each architecture, the dependent entities are listed.
- For each configuration, the dependent entities and architectures are listed.
- For each design unit, the corresponding source file and line number where the unit is defined are listed.

Note

- To execute the generated `Makefile`, run the UNIX utility `make`.

vhshell

Purpose

The `vhshell` utility is used to generate a VHDL **shell** file and component declaration for a Verilog module.



The `vhshell` utility is not required with this release. The DUT elaborates without the shell. The description of this utility has been provided for backward compatibility.

Syntax

```
vhshell [module | -all -o output_file| -multi_file][-comp] [-arch architecture]  
        [-work library][-f options_file] [-u] [-g] [-ncvhd1]
```

Parameters

<code>-comp</code>	Causes only the component declaration to be generated to either <code>module.inst</code> or as specified with the <code>-o</code> option.
<code>-arch <i>architecture</i></code>	Lets you change the default architecture name of the specified <i>architecture</i> in the generated shell file. This is useful if the shell file is used to replace a VHDL entity with a Verilog module, and you would like to use the original configuration for the VHDL entity. For example, when replacing a VHDL behavioral memory with one mapped to the Verilog primitive axis_smem , you might wish to use the original configuration to bind the replacement memory. This switch cannot be used with the <code>-all</code> or <code>-f</code> options

UXE User Guide

Using Utilities

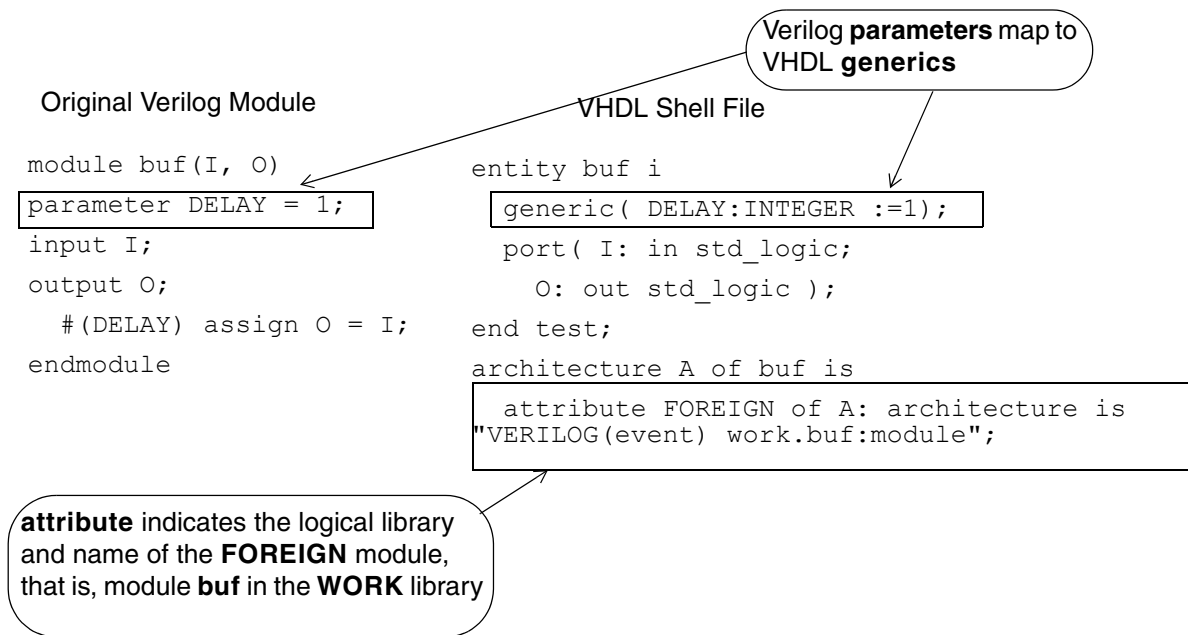
<code>module -all -o output_file - multi_file</code>	Lets you specify the Verilog module name(s) from which to extract shell files: To extract a single shell, specify just the Verilog <i>module</i>. Use the <code>-f options_file</code> option to specify multiple module names to extract. Two files are generated for each module: <i>module.vhd</i>, which is the shell file, itself, and <i>module.inst</i>, which contains a VHDL component declaration for the Verilog module. To extract all modules from a library, use <code>-all</code> with either the <code>-o output_file</code> or <code>-multi_file</code> option. If <code>-o output_file</code> is specified, all shells go into a single file, <i>output_file.vhd</i>, and all component declarations go into the file, <i>output_file.inst</i>. If <code>-multi_file</code> is specified, <i>vhshell</i> generates a file, <i>module.vhd</i> (the shell file itself), and a file, <i>module.inst</i> (containing a VHDL component declaration), for each Verilog module.
<code>-work library</code>	Specifies the compiled Verilog library from which to extract shell file(s) (<code>WORK</code> is the default).
<code>-f options_file</code>	Specifies a file with additional options and module names to extract.
<code>-u</code>	Lets you specify <code>STD_ULONGIC</code> as the port type, instead of the default of port type <code>STD_LOGIC</code> .
<code>-g</code>	Suppresses generation of VHDL generics for Verilog parameters, instead of the default in which they are generated.
<code>-ncvhdl</code>	Generates Cadence NC-VHDL-compatible shell files, which is required if you wish to use the shell files with the Xplore graphical user interface.

Notes

- In most cases, the **shell** is not required when instantiating Verilog in VHDL, except if you have a VHDL configuration that refers to a Verilog component.

- When analyzing VHDL shell files, do not pass the `-87` option to `vhan`. Shells require VHDL 1993 compatibility. But if you are using the `-mti` option, be sure to pass the `-93` option.

Example L-1 VHDL shell File Generated for a Verilog Module



Example L-2 Translating Verilog Parameters to VHDL Generics

Generics in the shell file are created based on the *initial* values of the Verilog parameters in the module:

- String initial values: Generates generics of type `string`. For example, for the following Verilog initial value:

```
parameter input_file="sram_input.dat";
```

The shell generic is:

```
generic input_file: string := "sram_input.dat";
```

- Integer and real initial values: Generates generics of type `integer`, `real`. For example, for the following Verilog initial value:

```
parameter RC_Const=1.1;
```

The shell generic is:

```
generic RC_Const: real := 1.1;
```

- Binary-, hex- or octal- formatted initial values: Generates generics of type `std_logic_vector` with the appropriate size. For example, for the following Verilog initial value:

```
parameter init=8'hFF;
```

The shell generic is:

```
generic init: std_logic_vector(7 downto 0) := "11111111";
```

- For all other initial values, the default is a shell generic of type `integer`.

Example L-3 Using Vector Ports Constrained by Parameters

The `vhshell` utility generates an unconstrained VHDL port in the shell file if the associated Verilog port has its size constrained by parameters. The VHDL port will be constrained appropriately when the component is instantiated in the VHDL design, for example:

```
parameter addr_width=8;  
input [addr_width-1:0] ADDR;
```

The VHDL shell file will contain the following port:

```
port(ADDR: in std_logic_vector);
```

UXE User Guide

Using Utilities

Using \$ixc_ctrl Pragma

The `$ixc_ctrl` pragma should be put into an `initial` block. These are pragmas for corresponding code transformations in IXC mode. In general, an `$ixc_ctrl` pragma falls into one of the following categories:

- IXC_CTRL Pragmas for Elaboration and Mapping
- IXC_CTRL Pragmas for Interface Compilation
- IXC_CTRL Pragmas for Behavioral Compilation
- IXC_CTRL Pragmas for Signal-Driven HMPI Mode

IXC_CTRL Pragmas for Elaboration and Mapping

The following pragma makes the instances of the module unique during elaboration:

```
$ixc_ctrl("unique_instance");
```

The following pragma marks the module as a module-inlining candidate:

```
$ixc_ctrl("xmrModInline");
```

The following pragma, for 'qtrefcell' module, dumps `var_decl` in the module, which has been compiled by HDL Compiler, and contains this directive:

```
$ixc_ctrl("hdlce", var_decl);
```

The following pragma forces the specified two-dimensional array variables to be bit-blasted into a register vector:

```
$ixc_ctrl("map_to_reg", local_var[, ...]);
```

Note: The `map_to_reg` pragma is primarily intended for internal use. Instead of `map_to_reg`, use the `no_hardmacro` memory-synthesis directive as follows:

```
// quickturn no_hardmacro <local_var>
```

Or

```
-- quickturn no_hardmacro <local_var>
```

The following pragma forces the specified two-dimensional array variables to be mapped as compact RTL memory:

```
$ixc_ctrl("map_to_mem", local_var[, ...]);
```

Note: The `map_to_mem` pragma is primarily intended for internal use. Instead of `map_to_mem`, use the `use_hardmacro` memory-synthesis directive as follows:

```
// quickturn use_hardmacro <local_var>
```

Or,

```
-- quickturn use_hardmacro <local_var>
```

IXC_CTRL Pragmas for Interface Compilation

- The following pragma turns on delay transformation for the module:

```
$ixc_ctrl("map_delays");
```

The above pragma is equivalent to specifying the `+iscDelay+<mod_name>` option with the `ixcom` command.

- The following pragma declares an import task or function to run in the blocking mode. By default, it uses primary I/O resource to transfer input and output. With `tb_mem`, it uses memory to do I/O transfer.

```
$ixc_ctrl("tb_import", "<task_name>"[, "tb_mem"])
```

- The following pragma is equivalent to `$ixc_ctrl("tb_import", "<task_name>", "tb_mem")`.

```
$ixc_ctrl("tb_task", "<task_name>")
```

- The following pragma is equivalent to `$ixc_ctrl("tb_import", "<task_name>")`.

```
$ixc_ctrl("tb_task_pio", "<task_name>")
```

- The following pragma declares an import task or function to run in the non-blocking mode, and uses memory to transfer input and output. X defines the size of a task group, which is the number of task calls that are accumulated in the buffer before an automatic release, while Y defines the number of task groups accommodated by the memory buffer.

```
$ixc_ctrl("tb_import", "<tf_name>", "nb=X,Y");
```

- The following pragma declares an export task or function to run in the blocking mode. By default, it uses primary I/O resource to transfer input and output. With `tb_mem`, it uses memory to do I/O transfer.

```
$ixc_ctrl("tb_export", <task_name>[, "tb_mem"])
```

- The following pragma declares an export task or function to run in the non-blocking mode, and use memory to transfer input and output. X and Y are the same as those for import task or function in non-blocking mode.


```
$ixc_ctrl("tb_export", "<tf_name>", "nb=X,Y");
```

- The following pragma declares an export task or function to run in dynamic concurrent mode, and use memory to transfer input and output. max_threads specifies concurrency level, and is 64 by default

```
$ixc_ctrl("tb_export", "<tf_name>", "dynamic_threads[<=max_threads]");
```

- The following pragma declares an import or export task or function (depending on whether the DPI-C task or function is declared as import or export) to run in blocking HMPI mode. By default, it uses primary I/O resource to transfer input and output. With tb_mem, it uses memory to do I/O transfer.

```
$ixc_ctrl("hmpi", "<dpi_c_tf_name>"[, "tb_mem"]);
```

IXC_CTRL Pragmas for Behavioral Compilation

- The following pragma turns on behavioral transformation for the module, and is equivalent to +bc+<mod_name>.

```
$ixc_ctrl("map_bc");
```

- The following pragma unrolls the loops with static constant loop bound in the module when behavioral compilation is enabled.

```
$ixc_ctrl("bc_unroll_loop");
```

- The following pragma marks the specified local vars to be behaviorally driven vars, to be used to overwrite default behavior of behavioral compiler.

```
$ixc_ctrl("beh_var", <local_var_reference>[, ...]);
```

- The following pragma marks the specified local variables to be pure RTL-driven variables, to be used to overwrite default behavior of the behavioral compiler.

```
$ixc_ctrl("rtl_var", <local_var_reference>[, ...]);
```

- The following pragma marks the specified local variables to be RTL clock; this enables behavioral compiler to apply certain performance optimization.

```
$ixc_ctrl("rtl_clk", <local_var_reference>[, ...]);
```

IXC_CTRL Pragmas for Signal-Driven HMPI Mode

- The following pragma specifies that the ports of the enclosing module are to be sent and received data automatically from CVP. This module must also be one of the +dut top modules.

```
$ixc_ctrl("hmpi_top");
```

- The following pragma specifies that additional internal DUT signals should be sent to CVP automatically.

UXE User Guide

Using \$ixc_ctrl Pragma

```
$ixc_ctrl("hmpi_read", <sig_var_reference>[, ...]);
```

- The following pragma specifies the DUT signal(s) whose value changes are to be used to synchronize CVP events.

```
$ixc_ctrl("hmpi_event", <sig_var_reference>[, ...]);
```

Methodology Guidelines for Compiling Assertions with the IXCOT Compiler

This appendix includes the following topics:

- [Inspecting IXCOT Log File for Assertion-Related Information](#)
- [Number of Assertions Synthesized by the IXCOT Compiler](#)
- [Advanced Support for SVA and PSL Assertions with Behavioral Compilation](#)
- [Differences in Handling of Assertions](#)
- [Best Practices for Assertions](#)

Inspecting IXCOT Log File for Assertion-Related Information

After compiling the design with the IXCOT compiler, you can refer to the compilation log file for assertion-related information, and get answers to questions, such as the following:

- How many assertions were present in the design?
- Did the IXCOT compiler find a syntax or semantic violation (error) in the definition of the assertion (property or methods)? See [Inspecting IXCOT Log File for Assertion-Related Information](#).
- Did the IXCOT compiler ignore an assertion because:
 - ☐ It is not supported in the current release?
 - ☐ Another compiler switch was required?

For example, you need to use the `+assertMaxIter+<N>` option to increase the limit of repetitions.

- Did the IXCOM compiler optimize away an assertion in the `always` or `never` `succeed` category?

Number of Assertions Synthesized by the IXCOM Compiler

During compilation, the IXCOM compiler gives an error if the design is too large to compile for the specified number of domains. In other cases, the IXCOM compiler might be able to compile the design, but report in the log file that the design requires a large number of steps to run. Occasionally, even if the design compiles successfully, it is possible that the design requires a large number of behavioral cycles while running on the emulator. If you observe such behavior, you can check the IXCOM log file to see if the number of assertions is unusually large.

After elaborating the HDL files, the IXCOM compiler lists the following in the log file, **before** generating the instrumentation for the assertions:

- ❑ Number of unique assertions found in the design and testbench
- ❑ Number of total instances of the assertions

An assertion might have more than one instance when it is in a `generate` block or if the module containing the assertion is instantiated multiple times.

- ❑ Number of PSL assertions, number of concurrent SystemVerilog assertions, and the number of concurrent SystemVerilog assertions present in a procedural context

After generating the instrumentation for the assertions, the IXCOM log file includes the following list of:

- Supported and unsupported assertions

Note: The instrumentation is generated only for the supported assertions.

- Number of assertions in the DUT
- Number of assertions in the testbench

Example (Count of assertions displayed in the `ixcom.log` file):

```
Top level design units:
    tb
Packages:
Design hierarchy summary:
```

UXE User Guide

Methodology Guidelines for Compiling Assertions with the IXCOM Compiler

	Instances	Unique	Splits
Modules:	6	7	6

```
Warning! in file logic.v at line 15 [ASSERTION_UNSUPPORTED]
    Unsupported concurrent assertion LOGICINVALIDCMD: sequence match items and
    multilocked assertions
```

```
Warning! in file logic.v at line 15 [ASSERTION_UNSUPPORTED]
    Unsupported concurrent assertion LOGICINVALIDCMD: sequence match items and
    multilocked assertions
```

Assertions Statistics:

Assertion summary by types:

	Instances	Unique
Concurrent Assertions	: 8	8
PSL	: 4	4
SVA	: 3	3
Procedural SVA	: 1	1

Post processing summary of concurrent assertion instances:

Supported (Evaluated at runtime)	: 7
Assertions in DUT (Mapped to HW partition)	: 5
Assertions in TB (Mapped to SW partition)	: 2
Un-supported	: 1

Errors Generated during Compilation of a Design with Assertions

The IXCOM compiler might generate errors or warnings during compilation of the design during the analyzer or elaborator stages, before it has synthesized the assertions. These messages can be found in the log files.

The following sections may help understand the different types of errors and warnings.

Finding Syntax and Semantic Errors in a Log File

If an assertion is written in illegal HDL syntax, the analyzer gives an error and aborts. If an assertion is written in violation of the LRM, the elaborator gives an error message, prefixed with `ASSERTION_ILLEGAL`.

UXE User Guide

Methodology Guidelines for Compiling Assertions with the IXCOM Compiler

Example (Compilation Errors in ixcom.log File):

```
Compiling source file "controller.v"
Highest level modules:
    top
Design hierarchy summary:
           Instances      Unique      Splits
Modules:           3          3          3
Assertions:         2          2
Error! in file controller.v at line 31          [ASSERTION_ILLEGAL_RANGE]
    Range must be ascending in a repeat sequence or a range sequence
    Errors : 1 Warnings : 0 Info : 0
// Log end : Wed Jul  6 19:31:27 2011
```

Finding Unsupported Constructs in a Log File

The IXCOM compiler supports a large variety of SVA and PSL constructs, but some constructs are only partially supported because these cannot be mapped to the emulator. Such cases are reported in the IXCOM log file as warnings, prefixed with `ASSERTION_UNSUPPORTED`.

The IXCOM compiler ignores such assertions and allows the compilation to proceed. One such example is shown below.

Note that the IXCOM compiler reports fewer synthesized assertions (0 assertions in this case).

Example (Warnings for Unsupported Assertions in ixcom.log File):

```
Compiling source file "controller.v"
Highest level modules:
    top
Design hierarchy summary:
           Instances      Unique      Splits
Modules:           3          3          3
Assertions:         0          0
Warning! in file controller.v at line 31          [ASSERTION_UNSUPPORTED]
    Unsupported concurrent assertion DUT1: real operand type in a boolean
expression
    Errors : 0 Warnings : 1 Info : 0
// Log end : Wed Jul  6 19:19:34 2011
```

Optimized Assertions

If the sequence expression of an assertion is a constant expression, the IXCOM compiler computes it during compilation. The IXCOM compiler generates a warning about such an assertion, noting in the log that it either `always succeeds` or `never succeeds`, and ignores it while mapping the DUT to the emulator. This way, the IXCOM compiler can save hardware capacity while compiling a design. To understand how to rewrite such a concurrent assertion so that it is not optimized away, refer to the [Using Immediate Assertions when the Sequence Expression is a Constant Expression](#) section.

Assertions Using X or Z Values

The `x` or `z` values are not supported on the emulator. These are converted to 0 or 1 when the design is moved to hardware.

Those assertions that check the value of a variable to be `x` or `z`, and meet the criteria listed below are optimized from the hardware partition and run only on the software simulator:

- The property must not be temporal and are only consequent. This means that the properties using immediate implication are not optimized.
- The sequence expressions must result in a constant on the hardware because of checks for `x` and `z` values. For example, the expression `(var === 1'bX)` will always be false on the hardware and, will, therefore, reduce all expressions AND'ing with it to a constant.

Example (IXCOM Compiler Optimizes an Assertion That Checks for x or z):

```
property p1;
  ((sel[1] === 1'bZ || sel[2] === 1'bZ) &&
   (A[width-1:0] !== 1'bZ && B[width-1:0] !== 1'bZ) ||
   !assert_driven_sel)
endproperty
```

In the above example the assertion runs only on the software simulator, and a warning is issued during compilation:

```
Warning in file test.v at line 45      [ASSERTION_OPTIMIZED_ON_HW]
Assertion checking 'X' or 'Z' values A1:  it will execute only when design is run
on simulator
```

If an assertion that checks for `x` or `z` values does not meet the above criteria, the IXCOM compiler cannot automatically direct such assertions to be run only on the software simulator. As a result, the value conversion (from `x` or `z` to 0 or 1) when a design swaps to hardware renders the `x` or `z` checking assertion as useless for verification, and unnecessarily

consumes emulator capacity. As an alternative, you can use ``ifdef` constructs to conditionally enable such assertions in a *simulation-only* environment.

Example (Writing Assertions within ``ifdef` to Conditionally Enable or Disable These):

```
`ifdef SIMONLY
ISXORZ : assert property (@(posedge clk) (a == 1`bx));
`endif
ixcom +sv +define+SIMONLY test.v //to enable the assertion in a simulation-only
                                environment
ixcom +sv test.v //to disable or ignore the assertion in an emulation environment
```

Advanced Support for SVA and PSL Assertions with Behavioral Compilation

The IXCOM compiler can map most of the assertion constructs in the DUT to the emulator so that the module or entity that contains these assertions is not treated as an embedded module. However, some constructs cannot be easily mapped to hardware and would normally be part of the software partition. These include assertions that contain multiple clocks or local variables. The IXCOM compiler can map these assertion constructs to the emulator by compiling the modules and entities containing these assertions as behavioral modules. Thus, more assertions can be supported on the emulator, and need not be restricted to the software partition.

Mapping the assertions in this manner increases step count and behavioral evaluations, thereby, slowing down the emulator and/or using extra hardware capacity, depending on the complexity of the assertions, and the number of times these are invoked. This would also reduce the resources available for other parts of the design.

When a module is compiled as a behavioral module, the following information message, implying that one or more assertions in a module requires behavioral compilation, is generated in the `xc_work/bc.log` file. Note that you need not give any additional compiler options. The IXCOM compiler switches to behavioral compilation automatically to handle these types of assertions.

```
Module memory at line 45 in mclktest.v
  id:2, svaBehMod
-----
```

```
Latch mode not used: sva module
```

```
Automatic Setting of rtl_clk:
```


UXE User Guide

Methodology Guidelines for Compiling Assertions with the IXCOM Compiler

```
vars marked as rtl_clk:
    clk
```

Beh Procs found at lines 55, 55

Multiple Drivers:

```
rtl_clk used as XpClk:
    clk
```

Detected dynamic loop related to line 55 in module memory

Check the log file for messages similar to those shown above to determine if the step count is unexpectedly high or the operating speed is low.



Tip

To determine the reason for an unusual step count and operating speed, temporarily ignore the assertions (by using the `+no_sva+<mod>` and `+no_psl+<mod>` options of the `ixcom` command) in the design or specifically in the module, which was compiled as a behavioral module, and recompile the design. Then, examine the new statistics.

```
INFO (db2util-1067): This design is scheduled in 110 steps.
```

```
INFO (qt2dadab-1086): Maximum emulator operating speed is 4310 kHz.
```

To see the run-time impact of behavioral cycles, check for messages in the run-time log files like `xeDebug.log`.

```
XERun> exit
```

```
--- HW execs 9 evals 4 bevals 8 tbsyncs in 3.05 sec (338445.33 us/eval;
```

Assertions with Multi-Clocked Consequent

Multi-clocked assertions of the form shown below are supported on the emulator by using behavioral compilation.

Cadence recommends that you write single-clocked assertions when possible. Single-clocked assertions use fewer hardware resources on the emulator, and facilitate easy debug if the assertion fails.

Example (Single-Clocked and Multi-Clocked Assertions in the Design):

```
module memory(input wire CLK1,CLK2,a,b,c,d);
```

```
//multi-clocked assertion MA1
MA1: assert property (@(posedge CLK1) a ##1 b | => c ##1 @(posedge CLK2) d);
//singly-clocked assertion SA1
SA1: assert property (@(posedge CLK1) a ##1 b | => c ##1 d);
endmodule
```

Local Variables and Sequence Match Items

Assertions containing local variables (or sequence match items) of the form shown below are supported on the emulator with behavioral compilation.

Cadence recommends that you write assertions without local variables whenever possible. Performance is generally better if the local variable stays within a sequence, and does not cross over a property operator.

Example (Assertions Containing Local Variables):

```
module memory(input wire CLK,a,b,c,d);

    property LP1;
        reg lv1;
        (a,lv1=1'b0) ##1 (b,lv1=1'b1) | => (lv1) ##1 c;
    endproperty
    //assertion with local variables
    LA1: assert property (@(posedge CLK) LP1());

    reg seqMatch2;

    property LP2;
        (a,seqMatch2=1'b0) ##1 (b,seqMatch2=1'b1) | => (seqMatch2) ##1 c;
    endproperty
    //assertion with sequence match items
    SM2: assert property (@(posedge CLK) LP2());
endmodule
```

Handling of Large Delays

The IXCOM compiler uses behavioral compilation to handle large delays in some cases.

- Single large delay in property declaration:

If a single large delay is found in the form seen in property P2 below, and the assertion C2, which asserts this property, the IXCOM compiler internally uses behavioral compilation to compile this assertion for the same reason as used for C1 above.

```
property P2;
    (@(posedge clk) a ##[1:250] b | => c[*10:$] ##1 d);
endproperty
C2 : assert property (P3());
```

■ **Single large delay in assertion statement:**

If a single large delay is used in an assertion that directly asserts an expression, the IXCOM compiler generates a warning about an unsupported assertion, and suggests that the `+assertMaxIter+<N>` option should be used for compilation. When you include this option, behavioral compilation is not required for this assertion.

```
C3 : assert property (@(posedge clk) a ##[1:250] b | => c[*10:$] ##1 d);
```

Handling expect Statements

The IXCOM compiler supports `expect` statements in the DUT and testbench. However, if the `expect` statement exists in a DUT module, you need to explicitly specify behavioral compilation for that module by using one of the following methods:

- Use the `+bc+<module_name>` option of the `ixcom` command
- Add an `ixc_ctrl` directive in the module

Example (using the +bc option):

```
ixcom +sv +dut+dut +bc+dut dut.v tb.v
```

Example (using the \$ixc_ctrl directive):

```
module dut(...);
    initial $ixc_ctrl("map_bc");
    ...
    ...
endmodule
```

Differences in Handling of Assertions

The following sections describe the differences between assertion handling by

- Between IES Simulator and Emulator (Compiled Using the IXCOM Compiler)

- ❑ Between the IXCOM Compiler and LRM

Between IES Simulator and Emulator (Compiled Using the IXCOM Compiler)

Clearly, an assertion can fail when the design and assertions are run on the emulator. If the same assertion does not fail, or fails at a different simulation time, or fails a different number of times on a simulator such as *irun*, the discrepancy needs to be investigated.

Trace-Based Counting

One source for such discrepancies is that IXCOM always does trace-based counting whereas another simulator (like *irun*) may perform attempt-based counting by default. In attempt-based counting, an assertion fails or finishes only once. Multiple finishes or failures from a single attempt are not counted or reported. In trace-based counting, each trace of the assertion attempt may fail or finish individually. Refer to the *Assertion Checking in Simulation* guide for more explanation about the assertion counting modes. To run *irun* with trace-based counting, use the `-assert_count_traces` option of the *irun* command.

It is also possible that a failing assertion in the IES simulator might not fail when the design is run on the emulator. This is possible when the IXCOM compiler does not support a particular assertion and ignores it during compilation. If IES supports this assertion, the results between IXCOM and IES are different when the assertion passes or fails.

In-Flight Assertions at End of Simulation

Another difference between IXCOM and IES is the treatment of the assertion-in-flight at the end of simulation. The IXCOM compiler reports these as failures whereas IES, by default, does not report these as failures.

Between the IXCOM Compiler and LRM

The following sections outline the differences between the handling of assertions as done by the IXCOM compiler and that prescribed by the LRM.

Vacuous Passes

The IXCOM compiler does not support vacuous passes of assertions if the antecedent itself failed while evaluating the property. Such cases do not trigger the execution of the pass action

block. Simulators, such as IES, typically, trigger the execution of pass action block even when the assertion passes vacuously.

Naming generate Blocks

The IXCOM compiler and IES have a different naming strategy when inlining generate blocks. In the following example, the IXCOM compiler will generate inlined blocks with names like gblk_0_, gblk_1_ etc. IES, on the other hand, will generate inlined blocks with names like gblk[0], gblk[1] etc.

This will cause differences anywhere assertions are referred to using hierarchies like:

- ❑ Assertion summary results will show assertion as “top.dt.gblk_0_.C1” in the IXCOM compiler and “top.dt.gblk[0].C1” in IES.
- ❑ The TCL commands used for turning off assertions in the IXCOM compiler and IES will be different because of the full hierarchical name
- ❑ The full path leading to “an assertion in a generate block” will need to be different in ACTs like \$assertoff, \$asserton or \$assertkill

Example:

```
genvar i;
generate for(i=0;i<5;i++) begin : gblk
    C1 : assert #0 (b & c | d);
    assert #0 (b & c | d);
end
endgenerate
```

Naming vunit Blocks

When a vunit is bound to a module by name, the IXCOM compiler maps it to a named inlined block in the module. IES instead will inline it in the module. This causes assertions to have a different hierarchical name in the IXCOM compiler as compared to that in IES. For example, consider the 2 assertion failure messages below for the same assertion. The first one is when IRUN compiles the design. The second one is when design is compiled by the IXCOM compiler. The name of the vunit in this example is “myvunit1”.

```
ncsim: *E,ASRTST (./a.prop,6): (time 5 NS) Assertion top.dtl.AV1 has failed
ncsim: *E,ASRTST (a.prop,6): (time 5 NS) Assertion top.dtl.myvunit1.AV1 has failed
```

Best Practices for Assertions

The following sections suggest some best practices for compiling the designs with assertions with the IXCOM compiler and running these designs on the Palladium XP emulator.

- ❑ [Using ifdef, Bind, or Vunits to Write Assertions](#)
- ❑ [Using Immediate Assertions when the Sequence Expression is a Constant Expression](#)
- ❑ [Using Variables Instead of Constants in Sequence Expressions](#)
- ❑ [Labeling Assertions](#)

Using ifdef, Bind, or Vunits to Write Assertions

It is possible to add an assertion into the design after the design has been coded. This is useful in the case of system level integration or when using a third-party IP.

You can use the *bind* approach to writing SystemVerilog Assertions and *attach* or *bind* separate files including assertions with a module in the DUT. Similarly, you can use PSL vunits to attach PSL assertions to modules within the DUT. Both approaches facilitate writing and adding of assertions in the verification environment much later in the developmental cycle, without having to modify the DUT. Such assertions can also be intentionally and temporarily ignored easily at compile time by not including the *bind* or *vunit* file, containing the assertions content, in the compile script.

For more information and examples on *bind* and *vunit*, refer to the SystemVerilog and PSL LRM documents, respectively.

Another way of conditionally including SystemVerilog Assertions or PSL in Verilog assertions is by writing them in ``ifdef <define_name>` blocks in the HDL. This allows you to enable and disable assertions during compilation by defining the appropriate `<define_name>` during compilation.

Using Immediate Assertions when the Sequence Expression is a Constant Expression

If the sequence expression of an assertion is a constant expression, the IXCOM compiler computes it, and reports it in the IXCOM log file that an assertion was found to be of the `always succeeding` or `never succeeding` type. Such an assertion is ignored at run time, and does not `finish` or `fail` because the IXCOM compiler optimizes it away after issuing a warning message about this condition.

You might want to use assertions to safeguard against an accidental change in the design from setting an inappropriate value to one or more parameters, which would otherwise lead to an incorrect value of a constant sequence expression. With such usage of assertions, a compile-time warning about `always succeed` or `never succeed` might not be enough. If you want such an assertion to fail at run time, Cadence recommends that you rewrite this concurrent assertion as an immediate assertion in an `initial` block. Immediate assertions have the least cost in terms of capacity and run-time performance. The IXCOM compiler does not ignore or optimize away immediate assertions during compilation.

Example (Rewriting a Concurrent Assertion as an Immediate Assertion):

This example illustrates a concurrent assertion that checks the value of a parameter in the procedural code. The IXCOM compiler regards an assertion that includes a constant (parameter) and no other variables, as either `always succeed` or `never succeed` depending on the value of the parameter; it can be over-ridden.

```
module memory(input wire CLK,a,b);
  parameter DEPTH=32'b0100;
  always @(posedge CLK) begin
    A1: assert property (DEPTH < 32'b0100);
    <other_code...>
  end
endmodule
```

The IXCOM compiler ignores such concurrent assertions, and generates the following message in the IXCOM log file:

```
Info in file proc.v at line 4           [ASSERTION_IGNORED]
      Ignore concurrent assertion A1: always succeed
```

Rewrite such assertions as `immediate` assertions in the following manner if you want a run-time failure when the value of constant expression is invalid. Immediate assertions are easier to synthesize and ideal for such cases.

```
module memory(input wire CLK,a,b);
  parameter DEPTH=32'b0100;
  initial begin
    A1: assert (DEPTH < 32'b0100);
  end
  always @(posedge CLK) begin
    <other_code>
  end
endmodule
```

Using Variables Instead of Constants in Sequence Expressions

In the previous section, a concurrent assertion was converted to an immediate assertion to check the value of a parameter at run time. However, at times, you might want an assertion to be evaluated continuously instead of only once in an `initial` block. In the following example, the assertion has been written to check for a sequence that never happens. As a result, a constant consequent 0 has been written so that it always fails once the antecedent has been found successfully.

Example (Rewriting an Assertion by Replacing Constants with Variables to Prevent it from Getting Optimized):

This example illustrates a concurrent assertion that is an equivalent of `never` because it has a constant expression 0 as consequent. The IXCOM compiler optimizes away such assertions in the `never` `succeed` category. This helps in making the hardware capacity available on the emulator.

```
module memory(input wire CLK,a,b);
  always @(posedge CLK) begin
    A1: assert property (a ##1 b | => 1'b0);
    <other_code>
  end
endmodule
```

If this assertion checks that the `a ##1 b` sequence `never` happens, and you do not want this assertion to be optimized away, rewrite the assertion as follows:

```
module memory(input wire CLK,a,b);
  wire zero = 1'b0;
  always @(posedge CLK) begin
    A1: assert property (a ##1 b | => zero);
    <other_code>
  end
endmodule
```

Labeling Assertions

For the most efficient use of assertion-related features when using the IXCOM compiler, it is important to not leave any assertions in the design unlabeled. The IXCOM compiler generates the instrumentation for unlabeled assertions, but assertions with auto-generated labels (in PSL) or no labels (in SVA) are not very informative, and you would need to rely on the line or file information to match the real assertion in the design with the IXCOM-generated label of the assertion. Debugging and gathering coverage information of such assertions,

UXE User Guide

Methodology Guidelines for Compiling Assertions with the IXCOM Compiler

therefore, is not a very straightforward process. In the absence of a naming ideology in the System Verilog LRM, all verification tools have their own labeling mechanism for such assertions, thereby, further increasing the inconsistency. The difference in label generation for unlabeled assertions across different tools makes it difficult to track reports generated from different tools. So, label the assertions to ease debugging and collecting of coverage data from different tools.

UXE User Guide
Methodology Guidelines for Compiling Assertions with the IXCOM Compiler

Invoking Palladium XP Compilers and Debugging Tools

This appendix describes how the various compiler and debugging tools for Palladium XP emulator are invoked.

Invoking xeCompile

xeCompile is invoked by executing the `xeCompile` command on the UNIX command prompt.



xeCompile must be invoked from the design directory.

The `xeCompile` command can be executed with or without parameters.

When specified without a parameter, the `xeCompile` command invokes xeCompile in an interactive command-line mode.

Note: After the xeCompile tool initialization is complete, the command prompt changes to `XEC>`.

To compile a design in batch mode using a Tcl script, execute the following `xeCompile` command on the UNIX command prompt:

```
xeCompile <script_name>
```

XEL Compile Commands

UXE supports its own set of XEL commands, variables, and parameters. XEL is an interpreted language that allows you to use the UXE verification functions either via a script or directly on the command line. XEL is based on the Tool Command Language (Tcl).

The XEL command prompt is available both in the xeCompile and xeDebug windows, as well as through a dedicated XEL shell that can be invoked from a UNIX window.

The XEL commands are documented in the *UXE Command Reference Manual*, and are also available as XEL-level `help` commands. Type `help <XEL_command_name>` to view the documentation for a specific XEL command.

Note: You should not run the debug XEL commands within `xeCompile` and the compile XEL commands within the UXE Debugger tool called `xeDebug`. Doing so will generate an error because the XEL shell for the compiler and debugger tools call different sets of XEL commands based on which tool you invoke.

Invoking xeDebug Tool

`xeDebug` is invoked by executing the `xeDebug` command, with or without parameters, on the UNIX or Linux command prompt.

Following are the `xeDebug` command options that allow you to invoke the `xeDebug` tool in six alternative ways:

- ❑ `xeDebug [-gui|-noecho] [-shm|-fsdb|-vcd] \`
 `[-init <tcl_file>] ... \`
 `[-log <name>] [-append_log] [-append_key] \`
 `[--ncsim <option> ... [--]] [script [<arg> ...]]`
- ❑ `xeDebug [-shm|-fsdb] -wave <tracefile> \`
 `[script|probelist|rcfile]`
- ❑ `xeDebug [-shm|-fsdb] -offline <session> <tracefile> \`
 `[script|probelist|rcfile]`
- ❑ `xeDebug -clean | -cleanall`
- ❑ `xeDebug -version`
- ❑ `xeDebug -help`

Note: These are alternative methods, and options specific to one method cannot be mixed with another. When specified without a parameter, the `xeDebug` command invokes the `xeDebug` tool in an interactive command-line mode. After the initialization of the `xeDebug` tool is complete, the command prompt changes to `XE>`.

For more information on the usage and the syntax, refer to the *Programs and Utilities* chapter of the *UXE Command Reference Manual*.

The `xeDebug` tool can be launched in the following modes:

■ GUI mode

UXE User Guide

Invoking Palladium XP Compilers and Debugging Tools

```
xeDebug -gui [-shm|-fsdb] [-init <tcl_file>] \  
[--ncsim <ncsim_options> ... [--]] \  
[<script_name> [<script_options> ...]]
```

The usage of the xeDebug GUI is described in [Appendix F, “Using the xeDebug GUI.”](#)

■ Script mode

```
xeDebug [-noecho] [-shm|-fsdb] [-init <tcl_file>] \  
[--ncsim <ncsim_options> ... [--]] \  
[<script_name> [<script_options> ...]]
```

Note: The [--ncsim <ncsim_options> ... [--]] is applicable only to SA mode.

■ Offline mode

- ❑ For viewing FullVision probes:

```
xeDebug [-shm | -fsdb | -vcd] [-offline <session_name> <phyfile>] \  
[<pscript> | <plist> | <rcfile>]
```

- ❑ For uploading probes and viewing the waveform with only trace files specified:

```
xeDebug [-shm | -fsdb | -vcd] [-wave <phyfile> ] \  
[<pscript> | <plist> | <rcfile>]
```

For details, refer to the [Debugging in Offline Session](#) section of [Chapter 18, “Debugging Designs on Palladium XP.”](#)

Invoking *irun* Tool (for SA Mode)

To invoke *irun* in script mode, use the following command:

```
irun -R -tcl
```

To invoke *irun* in GUI mode, use the following command:

```
irun -R -gui
```

irun provides the native debug environment of IES simulator, but you do not have full debugging access to the DUT in the emulator. The xeDebug tool provides full debugging access to both the testbench in the simulator and the DUT in the emulator, though xeDebug debugging environment is different from *irun*. If you invoke *irun* in its default mode, the design is run in pure software-simulation mode.

If you want to use *irun* for Simulation Acceleration mode, and run the designs on the Palladium XP emulator with *irun* as the Debugger, you can internally invoke the xeDebug tool from *irun*. For this, first use the -hw and -ua options of the *irun* command to compile the design for hardware and then use the -xeDebug option with the *irun* command. Refer to [Chapter 5, “Compiling and Running Simulation Acceleration Designs with *irun*.”](#)

The IXCOM compiler sets the default access to read and write (`-access +rw`) for the design objects. For performance reasons some designs do not require this access level. You can use the `-access <file>` option to modify the default value as follows:

1. Create an access file with the desired visibility access for the design objects

For example:

```
// File: ./access.file
DEFAULT +r-wc
```

For details on writing and using an access file, see *Using an Access File* in the IES documentation.

2. Specify an access specification in the `./uxc_hdl.var` file

For example:

```
DEFINE NCELABOPTS -afile ./access.file
```

3. Compile and simulate the design

Note: *irun* can be launched from both UNIX and Linux shells. However, the SimVision snapshot is platform dependent, that is, if the snapshot was generated on a Linux platform, the xeDebug and *irun* tools need to be run on Linux platform too.

Using the XEL Shell

The XEL shell is a text-only XEL interpreter. Based on whether you are compiling the design or debugging it, the XEL shell is populated with the corresponding set of commands.

Initializing the XEL Shell for Design Compilation

Execute the `xeCompile` command on the UNIX prompt to start the XEL shell for compiling a design. This command initializes the XEL shell, prints out messages, and returns the `XEC>` prompt for executing the XEL compile commands. For details about the `xeCompile` command, refer to *Programs and Utilities* chapter of the *UXE Command Reference Manual*.

Initializing the XEL Shell for Design Debug

Execute the `xeDebug` command on the UNIX prompt to start the XEL shell for debugging a design. This command initializes the XEL shell, prints out messages, and returns the `XE>` prompt for executing the XEL debug commands. For details about the usage of the `xeDebug`

command, refer to the *Programs and Utilities* chapter of the *UXE Command Reference Manual*.

Quitting the XEL Shell

Typing either `quit` or `exit` on the `XEC>` or `XE>` command prompt enables you to logout of the XEL shell. Consequently, all active subsystems are closed and the UNIX prompt returns.

Important

Cadence recommends that you do not use the `exit` command because the `exit` command immediately stops any post-simulation-IES-processing and exits the shell. When the `exit` command is used, the information (related to covergroups) collected while swapped into hardware will not be processed. But this processing is essential in certain cases, such as handling covergroups while running `xeDebug` in SA mode. Therefore, it is best to use the `quit` command to quit the XEL shell.

Using Help for XEL Command Syntax

For help on a specific XEL command, from within the XEL shell, type `help`, followed by the name of the XEL command. The help page contains information on syntax of the specified XEL command.

For example, to view help about creating a User Data file, enter the following command on the `xeCompile` command prompt:

```
XEC> help userData
```

For a list of topics, enter the following command on the `xeCompile` or `xeDebug` command prompt:

```
help *
```

For details on conventions used in describing XEL command syntax, enter the following command:

```
help Syntax
```

Using Special Characters

Use of wildcard characters in XEL varies by command, with some allowing use of wildcard, and some not allowing. For specific usage, refer to the *UXE Command Reference Manual*.

Table O-1 Wildcard Characters

Character	Description
*	Matches any sequence of characters.
?	Matches any single character.
[char]	Matches any single character from the character list. If the character list contains a sequence of the form a-b, any character between a and b, inclusive, will match.
\x	Matches the single character represented by x. This provides a way to avoid special interpretation for any of the characters *?[\] in the pattern.

Using XEL Scripts

When you are familiar with XEL, you can create XEL scripts to accomplish emulation tasks. Scripts can automate long or repetitive tasks. Sample XEL scripts are provided both in the *UXE Command Reference Manual* and in the `<install_dir>/share/uxe/gift` directory.

XEL scripting supports the entire command set of Tcl version 8.3.

Palladium XP records all emulation commands as XEL commands within a log file named `xeCompile.log`. After completing a series of emulation tasks, this file can be used as an example of how to construct an XEL script. You can also use this log as an input for creating an XEL script file.

Some XEL commands or arguments for obtaining status information are purposely not logged to limit the size of the log file.

Importing Netlists as Blackboxes in SA Mode

In the conventional Simulation Acceleration flow, the IXCOM compiler performs design analysis, elaboration, synthesis checks to map portions of the design that can be accelerated in emulator hardware. For synthesizable netlist files, these checks and processes are unnecessary, and the netlist can be directly presented to the hardware-mapping process of the compiler.

 Important

The use model described in this appendix is offered only for backward compatability. It is not recommended, and is likely to be obsolete in a future release. It is highly recommended that you use the process described in [“Importing External Netlists into IXCOM Compiler”](#) on page 341.

This section describes the process for importing netlists as blackboxes in the SA mode using the IXCOM compiler. This use model allows you to bypass the conventional Simulation Acceleration compilation flow.

This use model has the following limitations apply:

- Design must be swapped to the hardware mode at time 0 (at the beginning of the run), and run only in the hardware mode. Swapping back to the software mode is not possible because hot-swap capabilities are not available with this feature.
- Limited testbench access (read, write, force and so on) into the design objects in the imported netlist.

Note: The User Data and commands for the ICE flow can be used. However, the scripts should be modified to reflect the hierarchical path created by the IXCOM compiler.

- Testbench access to memories in the imported netlist is currently not supported.

Use Model for Importing a Netlist as Blackbox Using IXCOM

This section provides details for importing blackbox netlist in IXCOM flow. The following design modules are used to describe the functional use model for this feature:

```
// File: tb.v
module Tb;
    reg clk;
    reg [1:0] din;
    wire [1:0] dout;
    initial
    begin
        // see sub-sections below for read, deposit and force access from testbench
    end
    // ...
    Dut d(clk, din, dout);
endmodule // Tb

// File: dut-netlist.v
module Dut ( clk, din, dout);
    input clk;
    input [1:0] din;
    output [1:0] dout;
    wire en; // internal net
    wire [1:0] fout; // internal net
    // ...
endmodule
```

To import the blackbox netlist into the emulator hardware by using the IXCOM compiler (with no full visibility to design objects in the imported netlist), the use model requires certain steps different from the conventional IXCOM compilation flow.

Note: The RTL modules instantiated within a netlist hierarchy should be specified as exported modules by using `NL_EXPORT`. When the external netlist configuration file does not contain `NL_EXPORT` for a module, the following error message is issued:

```
ncelab: *F, CUITOP: A defined top level module Chk has also been instantiated
```

Following are the steps to import netlists as blackboxes using the IXCOM compiler:

1. Create a shell module for every top-level design unit in the netlist, and enclose the shell modules with the ``qtrefcel` and ``endqtrefcell` compiler directives.

```
// File::dut-shell.sv
`qtrefcell
```

UXE User Guide

Importing Netlists as Blackboxes in SA Mode

```
module Dut(clk, din, dout);
    input clk;
    input [1:0] din;
    output [1:0] dout;
endmodule // Dut
`endgtrefcell
```

Note: This shell module allows the IXCOM compiler to compile netlist as blackbox with the interface provided. By default, I/O interface provided in the shell file is the only visibility into netlist module at IXCOM compile time. You must explicitly specify additional visibility into netlist object as explained in [Testbench Access to Netlist Objects](#) on page 1416. The IXCOM compiler ignores all statements in this module except for the I/O interface and \$ixc_ctrl statements.

2. Provide information about the netlist file to the Palladium XP back-end compiler (xeCompile) in the externalNetlists.qel file in the working directory.

```
// File: externalNetlists.qel
netlistFile {verilog dut-netlist.sv}
```

Note: The file specified using netlistFile must be Verilog. You can use a file with other formats by converting the file to a Verilog netlist file through HDLICE.

3. Compile source files along with the shell module files.

```
ixcom tb.sv dut-shell.sv -ua +dut+Dut
```

For the above compile, IXCOM generates the following kind of design hierarchy:

```
// Tb module (from tb.sv)
module Tb ;
    // ...
    Dut_xcm2 dut (,,);
endmodule

// Dut module from shell file (dut-shell.sv)
module Dut_xcm2 ( clk, din, dout);
    // pragma CVASTRPROP MODULE HDLICE HDL_MODULE_ATTRIBUTE "0 vlog"
    input clk;
    input [1:0] din;
    output [1:0] dout;
    // ...
    // Dut instantiation of the original module from netlist
    Dut i0(clk, {din[1], din[0]}, {dout[1], dout[0]});
endmodule
```

UXE User Guide

Importing Netlists as Blackboxes in SA Mode

```
// Dut from original netlist file (dut-netlist.sv)
module Dut ( clk, din, dout);
    input clk;
    input [1:0] din;
    output [1:0] dout;
    // ...
endmodule
```

For the above design, the hierarchical reference to the netlist object must include an additional `i0` hierarchy introduced by the IXCOM compiler. For example, `Tb.dut.i0.clk` refers to the `clk` signal in the `Dut` netlist module.

Compile a netlist module imported as a top-level design unit as follows:

```
ixcom dut-shell.sv -ua +dut+Dut
```

The shell module name is modified by the IXCOM compiler to differentiate the shell module from the actual netlist module. For the above compile, the IXCOM compiler creates the following design hierarchy:

```
module Dut_ ( clk, din, dout);
    // pragma CVASTRPROP MODULE HDLICE HDL_MODULE_ATTRIBUTE "0 vlog"
    input clk;
    input [1:0] din;
    output [1:0] dout;
    // ...
    Dut i0 ( clk, { din[1], din[0]}, { dout[1],dout[0]});
endmodule
```

For the above design, the hierarchical reference to the netlist object needs to include both the modified module name in addition to the `i0` hierarchy introduced by the compiler. For example, `Dut_.i0.clk` refers to the `clk` signal in the `Dut` netlist module.

Testbench Access to Netlist Objects

For the top-level port interface objects, IXCOM automatically infers the primary input, output, or inout (bi-directional) property, based purely on the direction of the ports of this shell module. A port with input direction is treated as primary input; a port with output direction is treated as primary output; the port with inout direction is treated as a bidirectional signal, which has drivers from both DUT and testbench side. The marking of inout (when it is really not a bidirectional signal) signals do not affect the functionality, not marking a bidirectional signal as inout would produce incorrect results. However, it might have a performance impact as bidirectional property adds two extra pins at the interface.

Internal objects refer to nets (or signals) and registers in the imported netlist are the ones not specified in the port interface. If there are any references to internal nets that have been optimized away or cannot be found, an error message is generated.

To access (read and write) internal netlist objects from testbench, you need to specify access information in the shell file as follows:

1. Create a shadow object in the shell module defined within the ``qtfrcell` and ``endqtfrcell` compiler directives.
2. Use the `$ixc_ctrl(...)` statement to bind (or connect) the shadow object to the netlist object.
3. Use the `$export_read` and `$export_deposit` directives in an `initial` statement of the testbench to explicitly give access information.
4. Access the netlist object with hierarchical reference in the testbench (using HDL statement or primitives).

The `ixc` control statement uses the following syntax:

```
initial
$ixc_ctrl("<tag>", "[<declaration>;] <primitive instantiation>;");
```

Where:

- ❑ `<tag>` is always set to `hdlICE` for later use by back-end compiler.
- ❑ `<declaration>` is SystemVerilog declaration for the shadow object.

Note: This part can be optional in some control statements.

- ❑ `<primitive instantiation>` is used to connect shadow objects to the objects in the netlist. This primitive can be any one of the allowed `ixc` primitive.

Reading Netlist Objects from Testbench

To read an internal netlist object, use the following steps:

1. Create a shadow object in the shell module for the read access to the netlist object.
2. Use the `$ixc_ctrl(...)` statement to bind shadow object to the netlist object.
3. Use `$export_read(<signal path>)` in an `initial` statement of the testbench to explicitly give access information. Where, `<signal_path>` is hierarchical reference to the shadow object.
4. Access (read) the netlist object using hierarchical reference to the shadow object.

UXE User Guide

Importing Netlists as Blackboxes in SA Mode

Following is an example illustrating the read access to an internal netlist object using the above steps:

```
// File: tb.v
module Tb;
  // ...
  initial
  begin
    $export_read(d.en_QTRC);
  end
  initial
  begin
    if (d.en_QTRC) // access to Dut.en through shadow object
    // ...
  end
  // ...
  Dut d(clk, din, dout);
endmodule // Tb

// File: dut-shell.v
`qtfcell
module Dut(clk, din, dout);
  input clk;
  input [1:0] din;
  output [1:0] dout;
  reg en_QTRC; // shadow object
  initial
  // bind shadow object to object in the net-list
  $ixc_ctrl("hdlICE",
    "wire en_QTRC; ixc_bind #(1) (en_QTRC, i0.en);");
endmodule // Dut (shell)
`endqtfcell
```

Write (or Deposit) to Netlist Object from Testbench

To deposit to an internal netlist object:

1. Create a shadow object in the shell module for netlist object that needs to be deposited.
2. Use `$ixc_ctrl(...)` statement to bind shadow object to netlist object.

UXE User Guide

Importing Netlists as Blackboxes in SA Mode

3. Use `$export_deposit(<signal path>)` in an initial statement of the testbench to explicitly give access information. Where, `<signal_path>` is hierarchical reference to the shadow object.
4. Deposit to netlist object through reference to shadow object.

Following is an example illustrating the write (or deposit) access to an internal netlist object using the above steps:

```
// File: tb.v
module Tb;
    // ...
    initial
    begin
        $export_deposit(d.fout_QTRC);
    end
    initial
    begin
        // ...
        d.fout_QTRC = 2'b11; // deposit to internal netlist object
    end
    // ...
    Dut d(clk, din, dout);
endmodule // Tb

// File: dut-shell.v
`qtrefcell
module Dut(clk, din, dout);
    input clk;
    input [1:0] din;
    output [1:0] dout;
    reg [1:0] fout_QTRC; // shadow object
    initial
    // bind shadow object to netlist object
    $ixc_ctrl("hdlICE",
        "wire [1:0] fout_QTRC; ixc_bind #(2) (fout_QTRC[1:0], i0.fout[1:0]);");
endmodule // Dut (shell)
`endqtrefcell
```

Forcing Netlist Objects from Testbench

To force an internal netlist object:

UXE User Guide

Importing Netlists as Blackboxes in SA Mode

1. Create a shadow object in the shell module for netlist object that needs to be forced.
2. Use the `$ixc_ctrl(...)` statement to bind shadow object to netlist object.
3. Use `$export_deposit(<signal path>)` in an `initial` statement of the testbench to explicitly give access information. Where, `<signal_path>` is hierarchical reference to the shadow object.
4. Force the netlist object through reference to the shadow object.

Following is an example illustrating the access to force an internal netlist object using the above steps:

```
// File: tb.v
module Tb;
    // ...
    initial
    begin
        $export_deposit(d.fout_QTRC);
    end
    initial
    begin
        // ...
        #1 force d.fout_QTRC = 2'b11; // force to internal netlist object
    end
    // ...
    Dut d(clk, din, dout);
endmodule // Tb

// File: dut-shell.v
`qtfcell
module Dut(clk, din, dout);
    input clk;
    input [1:0] din;
    output [1:0] dout;
    reg [1:0] fout_QTRC; // shadow object
    initial
    // bind shadow object to object in the net-list
    $ixc_ctrl("hdlICE",
        "wire [1:0] fout_QTRC; ixc_bind #(2) (fout_QTRC, i0.fout[1:0]);");
endmodule // Dut (shell)
`endqtfcell
```


Driving the Clock Source in Imported Netlists

When using imported netlists, you need to use the `$ixc_ctrl` control statement to drive the clock source in the imported netlist.

The example below shows driving connecting clock (generated in the testbench) to clock source in the imported netlist:

```
// File: tb.sv
//
module Tb;
    reg clk;
    reg [3:0] din;
    wire [3:0] dout;
    // ...
    // generate cake clock
    SPDclkgen1x #(4) cakeGen (clk_CAKE, );
    Dut d(clk_CAKE, din, dout);
endmodule // Tb

// File: dut-shell.v
`qtrefcell
module Dut(clk_CAKE, din, dout);
    input clk_CAKE;
    input [3:0] din;
    output [3:0] dout;
    // drive clock signal in the netlist with clock from testbench
    initial
        $ixc_ctrl("hdlICE","ixc_clkbind icb (i0.clk, clk_CAKE);")
endmodule // Dut (shell)
`endqtrefcell
```

In the above example, `clk` in netlist file gets driven by `clk_CAKE` generated using `SPDclkgen*` (CAKE- or SCM-modeled clock) by using the `ixc_clkbind` primitive. The `ixc_clkbind` primitive disconnects driver logic and drives signal with the specified input signal. This behavior remains the same and these details should be under the `ixc_clkbind` primitive description. Refer to [Modeling Clocks in SA Mode](#) on page 1293 for more information.

UXE User Guide

Importing Netlists as Blackboxes in SA Mode

Third Party License Terms

BLT License Copyright 1991-1998 Lucent Technologies, Inc.

Lucent Technologies disclaims all warranties with regard to this software, including all implied warranties of merchantability and fitness. In no event shall Lucent Technologies be liable for any special, indirect or consequential damages or any damages whatsoever resulting from loss of use, data or profits, whether in an action of contract, negligence or other tortuous action, arising out of or in connection with the use or performance of this software.

Bootp v. 2.4 Copyright 1988, 1991 by Carnegie Mellon University. All rights reserved.

BSD License - Tix Copyright (c) 1993-95 loi Kim Lam. Copyright (c) 1996 Expert Interface Technologies.

In no event shall the authors of this software be liable to any party for direct, indirect, special, incidental, or consequential damages arising out of the use of this software and its documentation, even if the authors of this software have been advised of the possibility of such damage. The authors of this software specifically disclaim any warranties of merchantability and fitness for a particular purpose. The software provided hereunder is on an "as is" basis and the authors of this software have no obligation to provide maintenance, support, updates, and enhancements or modifications.

GNU General Public License, Version 2, June 1991 Copyright 1989, 1991 Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA, All rights reserved.

GNU General Public License, Version 2, is provided AS IS and comes with absolutely NO WARRANTY of any kind, whether express or implied, and is subject to the GNU General Public License terms contained in the file called gpl.txt. This is free software and you are welcome to redistribute it under the terms of such license. Modified GNU software carries notices stating that the files were changed by Cadence and the date of any change. The fibre channel driver is licensed under the GNU General Public License, Version 2, June 1991. If you desire a copy of the source, contact Cadence CVA.

Sample Object-Oriented Programming: Copyright 1995 by Prentice Hall. All Rights Reserved.

UXE User Guide

Third Party License Terms

Tcl/Tk -version 8.x.x Copyright by the Regents of the University of California, Sun Microsystems, Inc., Scriptics Corporation, and other parties.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only “Restricted Rights” in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52, 227, 19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as “Commercial Computer Software” and the Government shall have only “Restricted Rights” as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.